WEBSITE BOOK EDITION

Pixels → Perception

從像素到理解：視覺、語言與學習

基礎・影像・語言・學習・理論

第 3 冊（共 4 冊）

機器學習與電腦視覺深度學習

繁體中文版・第 3 冊

2026-10-17

https://blog.shyandram.dev/

本書說明

本冊是 Pixels → Perception 網站完整課程分成的第 3 冊（共 4 冊），主題為「機器學習與電腦視覺深度學習」，收錄 22 篇已發布教學與 0 篇部落格文章，依網站的學習單元順序編排。這是網站內容的書籍排版版本，並非另經同儕審查的出版品。

正文、程式碼、公式與參考文獻沿用網站內容。練習提示在本書中直接展開；靜態影像保留，互動示範則提供網站連結。原本的影像比較器改列為輸入與輸出影像。篇末參考文獻的編號僅適用於該篇教學。

本站內容仍在審閱中。製作過程使用 AI 協助，內容可能仍有錯誤或疏漏；請搭配原始文獻閱讀，並以網站的後續修訂為準。

兩種語言、各冊分別提供 PDF。連到同冊教學的連結會在本 PDF 內跳轉；連到其他冊的教學、特定小節、互動操作、筆記本與外部資料仍連至網站。內容快照: 2026-10-17

來源版本: `6b02ce5`

https://blog.shyandram.dev/

目錄

I 機器學習 1

ML 01・機器學習入門 2

ML 01A・學習所需的線性代數 28

ML 01B・機率與資訊理論 48

ML 01C・數值計算與梯度檢查 70

ML 02・類神經網路：從感知器到深度網路 93

ML 03・卷積、影像濾波與 CNN 117

ML 03A ·循環與遞迴神經網路 139

ML 04・注意力與視覺 Transformer 161

ML 05・表示學習與自監督 180

ML 05A・線性因子模型與自編碼器 201

ML 06・生成模型：自編碼器、VAE、GAN 與正規化流 223

ML 07・擴散模型與流匹配 245

ML 08・視覺自迴歸：像素、Tokens 與尺度 275

ML 09・基礎模型與視覺語言學習 295

ML 10・評估、效率與部署 315

II 電腦視覺深度學習 340

DLCV 01・深度學習的影像處理與電腦視覺 341

DLCV 02・深度影像分類 364

DLCV 03・物件偵測：從 R-CNN 到 DETR 401

DLCV 04・深度影像復原：低階視覺 428

DLCV 05・低光影像增強 471

DLCV 06・物件追蹤：從單一物件到多物件 505

DLCV 07・世界模型：影片、潛在動態與規劃 532

篇 I

機器學習

ML 01・機器學習入門

電腦如何從資料中學習：學習範式、線性迴歸與邏輯迴歸、成本函數、梯度下降、過擬合與模型評估，全部用 NumPy 從頭實作。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-01-introduction/

你將學到

- 「從資料中學習」是什麼意思，以及它和手寫規則有什麼不同；全篇以手寫數字的小影像當作貫穿的例子。
- 主要的學習範式：監督式學習（迴歸與分類）、非監督式學習（分群、降維、異常偵測）、自監督式學習，以及強化學習，包括「基於人類回饋的強化學習」（RLHF）如何調校聊天模型。
- 一個學習問題的基本詞彙：模型 $f_{w,b}(x)$、參數、特徵、目標值、預測值、訓練／驗證／測試集，以及成本函數。
- 以均方誤差為成本的線性迴歸，以及用來最小化成本的梯度下降：更新規則、學習率、收斂，以及批次、小批次與隨機梯度下降的差別。
- 為什麼要做特徵縮放；邏輯迴歸如何用 sigmoid 函數與交叉熵損失把一條直線變成分類器；如何用驗證資料發現過擬合，並用 L2 正則化修正。
- 如何用準確率、精確率、召回率與 MSE 誠實地回報結果。

先看全貌

大多數程式都像食譜：有人把問題想清楚，再把每一個步驟寫下來。機器學習把這件事反過來。我們寫一個**有彈性**、裡面有很多可調整數字的程式，給它看大量「正確完成任務」的例子，再讓一個演算法去調整這些數字，直到程式把任務做好。範例取代了食譜。

> **用最簡單的話說**
>
> 想想你小時候怎麼學會分辨貓和狗。沒有人給你一本規則手冊（「尖耳朵加鬍鬚、不會汪汪叫，就是貓」）。大人只是一次又一次指著說「貓」、「狗」、「貓」，你猜錯了就被糾正，直到你「就是知道」。機器學習也是這樣：我們給電腦看很多附上正確答案的例子，它先猜，我們告訴它錯了多少，它每次把猜測稍微修正一點，直到很少出錯。

為什麼重要：幾乎所有現代電腦視覺系統，從手機的臉部解鎖到醫學影像分析，都是學習出來的模型，而不是手寫的程式。本篇的幾個概念：帶有參數的模型、衡量誤差的成本、讓成本下降的梯度下降，以及用另外的資料檢查結果，正是今天訓練最大型神經網路所用的同一套想法，只是模型的規模不同。把這幾個概念弄清楚之後，本系列接下來的內容（ML 02 與各篇深度學習教學）主要就是在談「用**哪一種**模型」與「用**哪一種**成本」。

> **關於本篇**

本篇是由我先前的筆記〈Introduction to Machine Learning〉[1] 擴寫而成的正式教學（原文為英文）。那份筆記是跟著吳恩達（Andrew Ng）的 Machine Learning Specialization [4] 與《Deep Learning》一書 [3] 整理的。

什麼是機器學習？

白話版。機器學習系統藉由觀察資料來把任務做得更好，而不是由程式設計師寫下每一個決定。

用學習取代寫程式

這個想法由來已久。1959 年，Arthur Samuel 描述了一個學會下西洋跳棋（checkers）、而且下得比他本人還好的程式：它下了很多盤棋，並據此調整自己評估棋局的方式 [2]。他的重點是：一台能從經驗中學習的電腦，可以省去這項任務原本需要的大量細部手寫程式。那句著名的一行定義，把機器學習說成「讓電腦不需明確寫程式就能學習的研究領域」，其實是後人對這個想法的轉述，通常被歸於 Samuel，但並不是他論文裡的原句。

教科書中常見一個更具操作性的定義，包含三個部分：一個**任務**（例如辨識數字）、一個**效能指標**（辨識正確的比例），以及**經驗**（一批有標註的數字影像）。如果程式在這個任務上的效能隨著經驗

增加而提升，我們就說它在**學習** [3]、[7]。

影像的例子：規則與學習

我們貫穿全篇的例子是 scikit-learn 的 digits 資料集 [5]、[6]：1,797 張手寫數字的灰階影像，每張只有 8×8 個像素，數值從 0 到 16。就像〈Images as Arrays〉介紹的，每張影像只是一個小小的數字陣列，攤平之後就是 64 個數字組成的向量。

假設我們只需要分辨 **0** 和 **1**。程式設計師可能會寫一條規則：「1 很細、0 很寬，所以數一數有幾欄有墨跡；不超過五欄就是 1。」這條規則容易理解，對大多數直立的 1 也是對的。但人們寫 1 的方式各不相同，有的傾斜、有的帶襯線或底座，而傾斜的 1 佔的欄數和 0 一樣多（圖 1）。這條規則大約只對了 79%。想修好它就得加更多規則（「除非它是斜的 ……」），而每條新規則又會帶來新的例外。

機器學習的做法則**不寫任何**針對數字的規則。它挑一個有彈性的函數家族（這裡是邏輯迴歸，本篇後面會從頭實作），再讓演算法從範例中選出其中的 65 個數字。在沒看過的影像上，它把 0 和 1 完全分對；而且一模一樣的程式碼，不加任何新規則就能處理全部十個數字：

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

digits = load_digits()                      # 1,797 images, 8x8 pixels, values 0..16
X, y = digits.data, digits.target           # X: (1797, 64) flattened pixels; y: 0..9
```

```
# Hand-written rule for "0 versus 1": a 1 is thin, a 0 is wide.
keep = (y == 0) | (y == 1)
imgs, labels = digits.images[keep], y[keep]
width = (imgs.max(axis=1) > 4).sum(axis=1)   # how many columns contain ink
rule = np.where(width <= 5, 1, 0)
print("hand rule, 0 vs 1:", round((rule == labels).mean(), 3))          # 0.789

# Learned rule for the same task, judged on images it never saw.
X01 = imgs.reshape(len(imgs), -1)
Xa, Xb, ya, yb = train_test_split(X01, labels, test_size=0.3, random_state=0)
print("learned, 0 vs 1:", LogisticRegression(max_iter=1000).fit(Xa, ya).score(Xb, yb))   #
 ↪  1.0

# The same recipe scales to all ten digits with no new rules.
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
print("learned, 10 digits:",
      round(LogisticRegression(max_iter=5000).fit(X_tr, y_tr).score(X_te, y_te), 3))  # 0.953
```

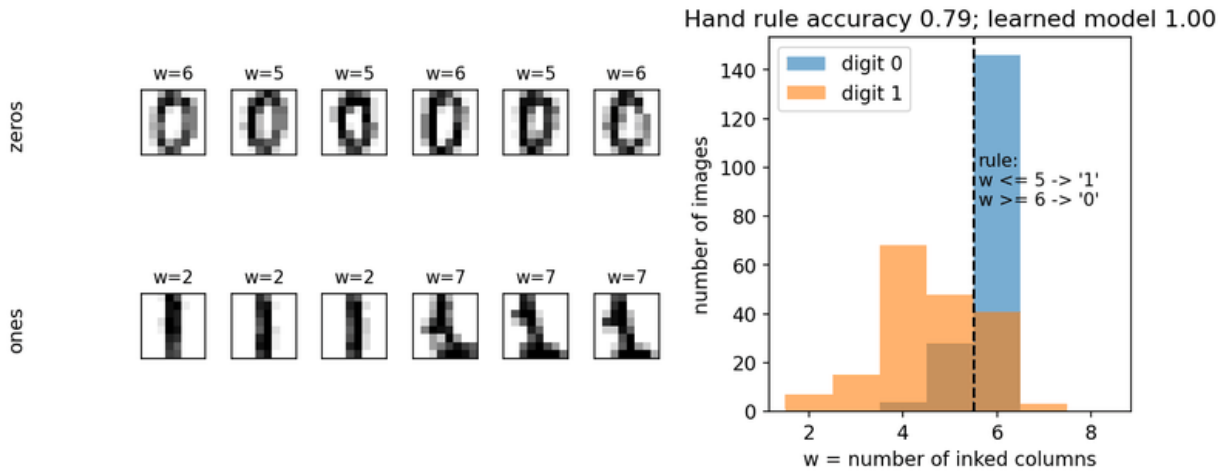


圖 *1* — 為什麼手寫規則很脆弱。左：*digits* 資料集中的 *0* 與 *1*；傾斜的 *1* 和 *0* 一樣寬。右：「數有墨跡的欄數」這個特徵在兩類之間大量重疊，所以任何門檻都會犯很多錯。學習出來的模型則同時使用全部 *64* 個像素。

這個小實驗包含了支持機器學習的全部理由：當規則難以用文字說清楚、範例卻很容易取得時，就從範例中學出規則。它也顯示了代價：學到的模型是一串 65 個數字，比「數欄數」難讀得多。

學習範式

白話版。各種學習問題的主要差別在於**學習者得到什麼樣的回饋**：每個範例都有正確答案、完全沒有答案、可以從資料本身製造答案，或是只偶爾得到一個「做得好不好」的分數。

範式	資料長什麼樣子	學到什麼	典型例子
監督式	成對的 (x, y)：輸入與「正確答案」	從 x 到 y 的函數	數字影像 → 數字標籤
非監督式	只有輸入 x	結構：群組、方向、離群值	把相似的數字分在一起
自監督式	只有輸入 x，但把 x 的一部分藏起來當作目標	通用的表示（representation）	預測被遮住的字詞或影像區塊
強化學習	代理人的動作，以及動作換來的獎勵	策略：在每種情況下該採取哪個動作	玩遊戲、調校聊天模型

我原本的筆記列了第一、第二和第四種；自監督式學習後來已成為預訓練大型模型的主要方式，所以這裡讓它單獨佔一列 [9]、[10]。

監督式學習

在監督式學習（supervised learning）中，學習者會拿到正確答案。資料是一組範例 $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$，其中 $x^{(i)}$ 是輸入，$y^{(i)}$ 是期望的輸出（稱為**標籤**或**目標值**），m 是範例數。目標是找到一個函數 f，使 $f(x)$ 接近 y，**而且對資料中沒有的新輸入也一樣**。最後這一點稱為**泛化**（generalization），它才是真正的目標；把範例背起來很容易。

非監督式學習

在非監督式學習（unsupervised learning）中沒有答案，只有輸入 $\{x^{(i)}\}$。目標是從資料中找出有意思的東西：相似範例組成的群組、精簡的描述，或是格格不入的範例。由於沒有唯一「正確」的輸出，這類方法很適合用來探索資料，但也比較難評估。

自監督式學習

自監督式學習（self-supervised learning）使用沒有標註的資料，但把一部分藏起來、要模型預測被藏的部分，藉此把問題變成監督式問題。在文字上，遮住一個字詞，從句子的其餘部分預測它。在影像上，遮住大部分的區塊再把它們重建出來：遮罩自編碼器（masked autoencoder）會遮住一張影像 75% 的區塊，訓練網路把它們補回來 [12]。LeCun 與 Misra 認為，自監督式學習讓模型能從原始觀察中累積廣泛的背景知識，因為標籤是免費得到的 [9]。

重點不在「填空」本身，而在於：一個很會填空的模型，必定已經學到很多關於資料結構的知識。這個學到的內部**表示**可以再用很少的標籤，調整去做許多其他任務。這正是**基礎模型**（foundation model）背後的做法：在大規模、廣泛的資料上訓練（通常是自監督式），再調整去做各式各樣的下游任務 [10]。GPT-3 只是被訓練來在大量文字上預測下一個字，卻能憑提示中給的幾個例子完成許多語言任務，不需要再訓練 [11]。

強化學習

在強化學習（reinforcement learning, RL）中，一個**代理人**（agent）與**環境**互動。每一步它觀察一個**狀態**、選一個**動作**，然後得到一個**獎勵**（一個數字）和新的狀態。沒有人告訴它正確的動作；

它必須自己發現哪些動作能在長期累積較高的總獎勵，而獎勵常常在造成它的動作之後很久才出現。學到的是一個**策略**（policy），也就是在每個狀態下選擇動作的規則 [8]。Samuel 的跳棋程式就是這種情境的早期例子 [2]。

今天，強化學習最為人所知的用途是調校大型語言模型。在 Ouyang 等人描述的**基於人類回饋的**

強化學習（reinforcement learning from human feedback, RLHF）[13] 中，先用人寫的示範答案對預訓練語言模型做微調（監督式學習）。接著由人對模型針對同一個提示所產生的幾個答案排序，再訓練一個獨立的**獎勵模型**來預測這些排序。最後用強化學習最佳化語言模型，讓它產生獎勵模型評分高的答案。作者回報，標註人員偏好他們 13 億參數的調校模型，勝過 1,750 億參數的 GPT-3 [13]。更近期，DeepSeek-R1 用大規模強化學習，搭配可自動檢查的獎勵（例如數學答案是否正確），來提升逐步推理的能力 [14]。

注意 RLHF 串起了三種範式：自監督式預訓練、監督式微調，以及強化學習。真實的系統會自由混用各種範式；上面那張表是一張地圖，而不是一道道牆。

監督式學習：迴歸與分類

白話版。監督式學習主要有兩種。**迴歸**預測一個數字；**分類**預測一個類別。

迴歸

在迴歸（regression）中，目標值 y 是一個實數，所以可能的輸出有無限多個。例如：預測股價、用之前的考試成績預測學生的期末成績，或從臉部影像預測一個人的年齡。最後一個例子說明：輸入可以是一張影像，輸出仍然只是一個數字。

分類

在分類（classification）中，目標值是一小組固定類別中的某一個。例如：垃圾郵件或非垃圾郵件（兩類，二**元分類**）、臉型的類別，或是腫瘤類型，如良性、惡性第 1 型、惡性第 2 型（**多類別分類**）。類別是名稱，不是數量：「第 2 類」並不是「第 1 類」的兩倍。分類器通常會對每個類別輸出一個機率，再挑最可能的那一個。

```
from sklearn.datasets import make_regression, make_classification
from sklearn.linear_model import LinearRegression, LogisticRegression

# Regression: the target is a real number.
X_r, y_r = make_regression(n_samples=100, n_features=1, noise=10.0, random_state=0)
reg = LinearRegression().fit(X_r, y_r)
print(reg.predict(X_r[:3]).round(1), y_r[:3].round(1))   # numbers vs numbers

# Classification: the target is one of a few labels.
X_c, y_c = make_classification(n_samples=100, n_features=2, n_redundant=0,
                               n_clusters_per_class=1, random_state=0)
clf = LogisticRegression().fit(X_c, y_c)
```

```
print(clf.predict(X_c[:5]), y_c[:5])                    # labels vs labels
print(clf.predict_proba(X_c[:2]).round(3))              # class probabilities
```

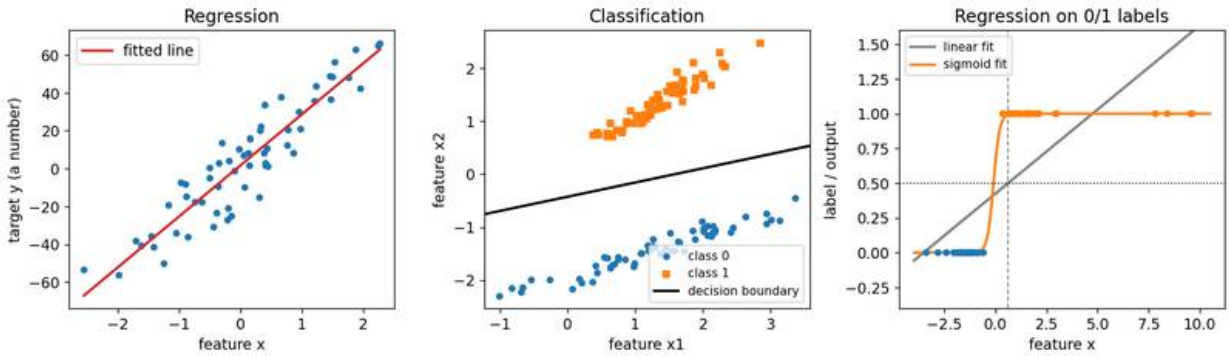


圖 2 — 左：迴歸讓曲線穿過數值目標。中：分類用決策邊界分開有標籤的點。右：對 *0/1* 標籤擬合一條迴歸直線（灰色），輸出會超出 $[0, 1]$，而且遠處的點會拉動它跨過 *0.5* 的位置；*sigmoid*（邏輯迴歸，橘色）才是合適的工具。

為什麼不用迴歸來做分類？

把類別編成 0 和 1、擬合一條迴歸直線、大於 0.5 就判為 1，這個做法很誘人。在簡單的資料上它也許可行，但通常表現很差，原因有兩個，都能在圖 2（右）看到。第一，直線會輸出 −0.4 或 1.7 這類數值，它們不是機率。第二，平方誤差會懲罰那些**已經分對**、只是離邊界很遠的點，所以少數極端但容易的範例會把直線拉過去，讓它跨過 0.5 的位置移到錯誤的地方。本篇後面的邏輯迴歸同時解決了這兩個問題。

非監督式學習

白話版。即使沒有答案，我們仍然可以問：哪些範例屬於同一群？怎樣用更少的數字描述每個範例？哪些範例很奇怪？

分群

分群（clustering）把相似的範例歸在一起。經典的演算法是 **k-means** [15]。先決定群數 K，從 K 個中心點出發，然後重複兩個步驟直到不再變化：把每個範例指派給最近的中心，再把每個中心移到被指派給它的範例的平均位置。每一步都只會讓範例到其中心的總平方距離

$$\sum_{i=1}^{m} \lVert x^{(i)} - \mu_{c^{(i)}} \rVert^2$$

變小或不變，其中 μ_k 是第 k 群的中心，$c^{(i)}$ 是範例 i 被指派到的群。所以演算法一定會停下來，但可能停在不好的局部解，這也是為什麼要從好幾個隨機起點各跑一次。Jain 回顧五十年分群研

究的文章 [15] 強調，一般而言沒有「最好」的分群演算法：「正確」的群組取決於「相似」對這份資料代表什麼意思。

一個常見的混淆：**k 近鄰法**（k-nearest neighbors, KNN）**不是**分群方法。KNN 對一個新範例分類的方式，是讓 *k* 個最相似的**有標籤**訓練範例投票，所以它是監督式方法 [5]。k-means 的 k 是群數；KNN 的 k 是鄰居數。

降維

降維（dimensionality reduction）用更少的數字描述每個範例，同時盡量保留有用的資訊。**主成分分析**（principal component analysis, PCA）找出資料變化最大的幾個方向，再把資料投影到其中前幾個方向上；新的座標彼此不相關，並依序盡可能捕捉最多的變異 [16]。兩個 PCA 座標就能把 64 個像素的數字影像變成平面上的點，讓我們看得到（圖 3 中）。

異常偵測

異常偵測（anomaly detection）找出與其他資料模式不符的範例：詐騙交易、瑕疵零件、損壞的影像。Chandola、Banerjee 與 Kumar 的綜述 [17] 依照各技術對「正常長什麼樣子」所做的假設，把眾多技術加以分類。一個簡單的版本使用 PCA：在資料上擬合幾個主成分，用它們重建每個範例，再標出重建誤差大的範例，因為主成分只描述了常見的部分。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)             # y is used only to check, never to fit

# Clustering: ten groups, found without looking at y.
km = KMeans(n_clusters=10, n_init=10, random_state=0).fit(X)
purity = sum(np.bincount(y[km.labels_ == k]).max() for k in range(10)) / len(y)
print("cluster purity:", round(purity, 3))      # 0.792: most clusters are one digit

# Dimensionality reduction: 64 numbers per image -> 2 numbers.
Z = PCA(n_components=2).fit_transform(X)        # shape (1797, 2)

# Anomaly detection: images that a 20-component PCA rebuilds badly are unusual.
pca20 = PCA(n_components=20).fit(X)
err = ((X - pca20.inverse_transform(pca20.transform(X))) ** 2).sum(axis=1)
print("most unusual images:", np.argsort(err)[-5:])
```

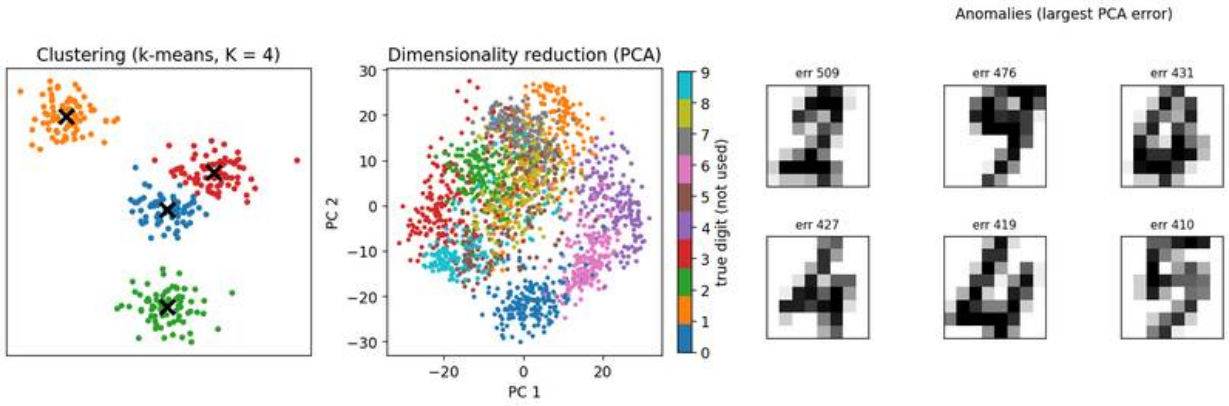


圖 3 — 非監督式學習。左：*k-means* 把點聚集在 K 個中心周圍。中：*PCA* 把每個 *64* 像素的數字壓縮成兩個數字（顏色是 *PCA* 從沒看過的真實數字）。右：*PCA* 重建誤差最大的數字，正是看起來最奇怪的那些。

程式中的純度（purity）檢查只在分群**之後**才用到標籤，用來看找出的群組是否和數字吻合。大約 79% 的影像落在以它自己的數字為主的群裡，即使 k-means 從沒看過任何標籤。

學習問題的詞彙

白話版。在談數學之前，我們需要替各部分取名字：什麼進去、什麼出來、正確答案是什麼、模型是什麼，以及怎麼衡量它的錯誤。

輸入、輸出與目標值

- **特徵**（features，輸入）x：描述一個範例的數字。對數字影像來說是 64 個像素值；對一棟房子來說是坪數和房間數。有 n 個特徵時寫成 $x = (x_1, \dots, x_n)$，x_j 是第 j 個特徵。
- **目標值**（target）y：正確的輸出，也叫標籤或**真值**（ground truth）。只有監督式學習有目標值。
- **預測值**（prediction）$\hat{y}$（讀作 y-hat）：模型對某個輸入的輸出。帽子代表「估計的」。
- **訓練範例** i：成對的 $(x^{(i)}, y^{(i)})$。括號中的上標是索引，不是次方。$x_j^{(i)}$ 是範例 i 的第 j 個特徵。

模型與參數

模型是把特徵對應到預測值的函數。我們寫成 $f_{w,b}$，表示它取決於稱為**參數**的可調整數字，這裡是 w（**權重**，weights）與 b（**偏差**，bias）：

$$\hat{y} = f_{w,b}(x).$$

選擇 f 的**形式**（直線、曲線、神經網路）是設計上的決定；選擇 w 和 b 的**數值**則是學習要做的事。由我們手動設定的數字，例如學習率或多項式的次數，稱為**超參數**（hyperparameters），以和學習出來的參數區分。

成本函數

損失（loss）$L(\hat{y}, y)$ 衡量一個預測錯了多少。**成本函數**（cost function）$J(w, b)$ 把損失對整個訓練集取平均，因此它是一個數字，說明在某組參數下整個模型錯了多少。模型是 f；成本是 J。把兩者分清楚可以避免很多混淆：$f_{w,b}(x)$ 吃一個輸入、吐出預測值；$J(w, b)$ 吃一組參數、吐出誤差。

於是訓練的**目標**就很明確：找出讓 J 盡可能小的參數，使所有訓練範例的 $\hat{y}^{(i)}$ 都接近 $y^{(i)}$。

訓練集、驗證集與測試集

資料會切成三份，各有不同的工作：

- **訓練集**用來擬合參數；
- **驗證集**在開發過程中用來比較模型、選擇超參數；
- **測試集**只在最後使用一次，用來估計最終模型在新資料上的表現。

測試集絕不能影響任何決定，否則估計值會過於樂觀。我們會在過擬合那一節回來說明**為什麼**要這樣切。

線性迴歸

白話版。在資料中畫一條直線。這條線有兩個旋鈕：斜率和高度。學習就是轉動旋鈕，直到直線盡可能貼近所有的點。

模型

只有一個特徵時，線性迴歸（linear regression）模型是

$$f_{w,b}(x) = wx + b,$$

其中 x 是特徵，w 是斜率（x 增加 1 時預測值改變多少），b 是截距（$x = 0$ 時的預測值）。改變 w 會讓直線傾斜；改變 b 會讓直線上下移動（圖 4 左）。

有 n 個特徵時，同一個模型是加權總和：

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = \sum_{j=1}^{n} w_j x_j + b,$$

其中 $\mathbf{w} = (w_1, \ldots, w_n)$ 為每個特徵各存一個權重。常見的技巧是加上一個常數特徵 $x_0 = 1$，偏差就變成另一個權重 w_0，模型縮寫成 $\mathbf{w} \cdot \mathbf{x}$。無論哪種寫法，「線性」指的都是對**參數**是線性的；之後我們會餵給它平方或三次方的特徵，它仍然是線性迴歸。

成本函數：均方誤差

對每個範例，**誤差**（殘差）是 $f_{w,b}(x^{(i)}) - y^{(i)}$。取平方讓誤差變成正數，並讓大誤差比小誤差受到更重的懲罰。對 m 個訓練範例取平均，就得到**均方誤差**（mean squared error, MSE）成本：

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2,$$

其中 m 是訓練範例數，$(x^{(i)}, y^{(i)})$ 是第 i 個範例。多出來的 $\frac{1}{2}$ 只是為了方便：微分平方項時出現的 2 會和它抵消，而且它不會改變最小值的位置。

把成本想像成參數上的一片地形。如果固定 b、只改變 w，J 是一條拋物線：太平或太陡的直線誤差都很大，介於中間的某個斜率最好（圖 4 中）。同時考慮 w 和 b 時，J 是一個碗，我們把它畫成由橢圓組成的等高線圖（圖 4 右）。最好的直線就在碗底。

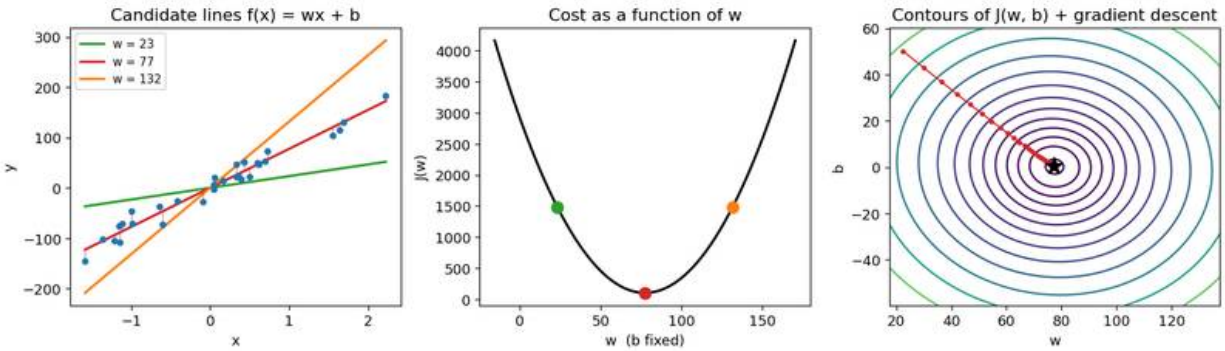


圖 4 — 從直線到成本。左：三條候選直線；垂直線段是最佳直線的殘差。中：固定 b 時，$J(w)$ 是拋物線，每條直線都是上面的一個點。右：$J(w,b)$ 是一個碗（等高線），梯度下降沿著下坡走到碗底。

為什麼這個碗只有一個底

對線性迴歸來說，MSE 成本是 (w,b) 的**凸函數**（convex）：曲面上任兩點之間的直線段都位於曲面上方或曲面上。凸函數沒有其他的局部最小值；每個最小值都是全域最小值。這是一個特別友善的情況。對許多其他模型，包括ML 02 的神經網路，成本地形有許多山谷，最佳化方法可能停在不是最低的那一個。原本的筆記正是在講這一點：一般而言，我們能期待的最好結果是找到一個夠好、接近全域最小值的局部最小值。

對線性迴歸，我們甚至可以用公式直接解出碗底（正規方程式，`np.polyfit` 就是這麼做的）。我們仍然使用梯度下降，因為在沒有公式可用時，它依然有效。

梯度下降

白話版。你在濃霧中站在成本地形的某處。感覺一下地面往哪邊傾斜，往下坡走一小步，再重複。等地面平了，你就到底了。

更新規則

導數 $\partial J/\partial w$ 告訴我們 w 稍微增加時 J 會怎麼變：正值表示「往右是上坡」，負值表示「往左是上坡」。它的正負號給出方向，大小說明坡有多陡。梯度下降（gradient descent）讓每個參數**逆著**它的導數走一小步：

$$w \leftarrow w - \alpha \frac{\partial J(w,b)}{\partial w},$$
$$b \leftarrow b - \alpha \frac{\partial J(w,b)}{\partial b},$$

其中 $\alpha > 0$ 是**學習率**（步長），$\leftarrow$ 表示「以右邊取代」。兩個導數都要在**目前**的 (w,b) 計算完之後

才更新任何一個參數，這叫做**同步更新**。所有偏導數組成的向量就是**梯度** ∇J，它指向上升最快的方向，所以這個更新是朝下降最快的方向移動 [3]。

對線性迴歸的 MSE 成本，連鎖律給出

$$\frac{\partial J}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)x^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right).$$

用文字讀：w 的梯度是「平均誤差乘以特徵」；b 的梯度是「平均誤差」。如果預測值平均而言太高，b 就往下調。如果預測值主要在 x 很大的地方太高，w 就往下調。有 n 個特徵時，$\partial J/\partial w_j$ 用 $x_j^{(i)}$ 取代 $x^{(i)}$。

從頭實作

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=100, n_features=1, noise=10.0, random_state=0)
x = X[:, 0]                                    # one feature, shape (m,)
m = len(x)

def cost(w, b):
    return ((w * x + b - y) ** 2).sum() / (2 * m)

w, b, alpha = 0.0, 0.0, 0.1
for step in range(200):
    err = w * x + b - y                        # f_wb(x_i) - y_i for every i
    dj_dw = (err * x).sum() / m
    dj_db = err.sum() / m
    w, b = w - alpha * dj_dw, b - alpha * dj_db   # simultaneous update
    if step % 50 == 0:
        print(f"step {step:3d}  J = {cost(w, b):8.2f}")

print("gradient descent:", round(w, 3), round(b, 3))   # 42.619 -0.814
print("closed form     :", np.polyfit(x, y, 1).round(3))  # [42.619 -0.814]
```

成本在 50 步內從大約 800 降到 57，之後就不再變化。梯度下降和精確公式的結果在小數點後三位都一致。剩下的成本不是零，因為資料中含有任何直線都無法解釋的雜訊。

收斂：導數為零的地方

「重複直到收斂」需要一個定義。在最小值處地形是平的，所以那裡的**導數為零**，更新 $w - \alpha \cdot 0$ 不會改變 w。要注意變成零的是**導數**，不是成本：最小值處的成本只是能達到的最小數值，在上面的例子中是 57。實務上，當兩次迭代之間成本下降得比一個小容許值還少、梯度的大小低於某個門檻，或是用完固定的步數時，就停止。

一個有用的推論：越接近最小值，導數越小，所以即使 α **固定**，步伐 $\alpha\,\partial J/\partial w$ 也會自動變小。梯度下降在接近谷底時會自然放慢。

選擇學習率

學習率是梯度下降最重要的超參數（圖 5）：

- **太小：**每一步都很小。演算法確實在下坡，但可能需要非常多步。
- **剛好：**成本快速下降並穩定下來。
- **太大：**一步跨過谷底跳到山谷另一側，甚至可能比起點還高。參數來回彈跳，可能永遠停不下來（**越過頭**，overshoot），甚至一路飛向無限大（**發散**，divergence）。

標準的診斷工具是**學習曲線**：把 J 對迭代次數作圖。學習率合適時，J 每一步都會下降。如果 J 曾經**上升**，就是 α 太大（或梯度的程式有錯）。實用的做法是以大約 3 倍的間隔嘗試數值，例如 0.001、0.003、0.01、0.03、0.1、0.3，然後挑曲線仍平穩下降的最大那一個。

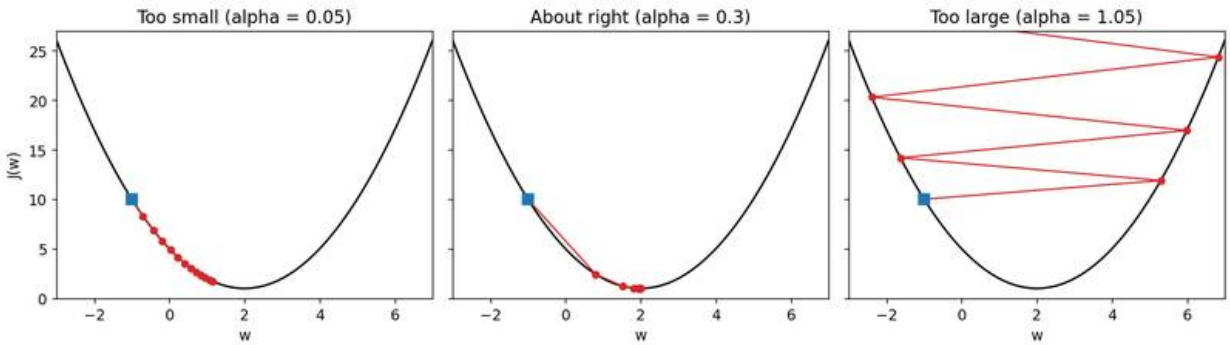


圖 5 — 同一個成本 $J(w)$、同一個起點、三種學習率。太小：進展緩慢。剛好：快速收斂。太大：每一步越過得更遠，迭代發散。

許多最佳化方法會在訓練過程中調整步長：用一個**排程**（schedule）隨時間降低 α，或像 Adam 那樣，用每個參數過去梯度的移動統計量來縮放它的步伐 [19]。這讓「接近最小值時步伐變小」成為明確的設計，而不只是副作用。

批次、小批次與隨機梯度下降

梯度公式要對全部 m 個範例加總。100 個範例沒問題，一億個就是麻煩了。關鍵的洞見是：梯度是一個**平均值**，也就是對資料的期望值，而平均值可以從樣本估計。[3] 對此有清楚的說明，Bottou、Curtis 與 Nocedal [18] 則做了深入的分析。

- **批次梯度下降**（batch gradient descent）每次更新都使用**整個**訓練集。每一步都精確，但也昂貴。
- **小批次梯度下降**（mini-batch gradient descent）先打亂資料，每次更新只用一個隨機的小子集（**小批次**），大小為 B 個範例，通常是 32 到幾百。每一步是對完整梯度帶雜訊但不偏的估計，成本只有 B/m。
- **隨機梯度下降**（stochastic gradient descent, SGD）在嚴格意義上每次更新只用一個範例（$B = 1$）。實務上「SGD」也常指小批次的版本。

完整走過訓練資料一遍稱為一個 **epoch**。注意程式中的命名：深度學習函式庫中的 `batch_size` 參數指的是**小批次**的大小，而不是批次梯度下降的「整個資料集」。

小批次梯度的雜訊不只可以容忍，往往還有幫助：在相同的 epoch 數內，小批次下降比批次下降多做了非常多次更新。Bottou 等人認為，這正是隨機方法主導大規模學習的原因：它們用遠少於精確方法的計算量，就能達到有用的準確度 [18]。

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=10_000, n_features=5, noise=10.0, random_state=0)
m, n = X.shape
rng = np.random.default_rng(0)

def minibatch_gd(batch_size, alpha=0.05, epochs=5):
    w, b = np.zeros(n), 0.0
    for epoch in range(epochs):
        order = rng.permutation(m)                    # shuffle once per epoch
        for start in range(0, m, batch_size):
            idx = order[start:start + batch_size]
            err = X[idx] @ w + b - y[idx]
            w -= alpha * X[idx].T @ err / len(idx)    # gradient estimated on the batch
            b -= alpha * err.mean()
    return ((X @ w + b - y) ** 2).mean() / 2

for bs in [m, 64, 1]:                                 # batch, mini-batch, stochastic
    print(f"batch size {bs:5d}: J = {minibatch_gd(bs, alpha=0.05 if bs > 1 else 0.005):.2f}")
# batch size 10000: J = 6127.00   (only 5 updates in 5 epochs)
# batch size    64: J = 49.93     (785 updates)
# batch size     1: J = 50.64     (50,000 noisy updates)
```

同樣走過資料五遍，批次下降只走了五步，離谷底還很遠；小批次下降則基本上已經收斂（這份

資料的雜訊程度讓可達到的最佳 J 大約是 50）。單一範例的 SGD 也能到達，但因為梯度雜訊較大，需要較小的學習率，而且無法有效利用向量化運算。

特徵縮放

白話版。如果一個特徵以千為單位、另一個以小數為單位，成本地形會變成又長又窄的山谷，梯度下降會左右來回鋸齒狀前進。把所有特徵放到相近的尺度，山谷就會變成圓圓的碗，很容易走到底。

為什麼尺度很重要

假設特徵 x_1（例如以平方英尺計的房屋面積）的範圍是好幾千，特徵 x_2（房間數）的範圍是 1 到 5。w_1 稍微改變一點，預測值就變很多；w_2 做同樣的改變卻幾乎沒影響。所以 J 沿著 w_1 方向非常陡，沿著 w_2 方向非常平：等高線是細長的橢圓。學習率若小到不會在陡的方向發散，在平的方向上就幾乎沒有進展（圖 6 左）。

標準化與最小–最大縮放

最常見的解法是**標準化**（standardization，z 分數正規化）：

$$x'_j = \frac{x_j - \mu_j}{\sigma_j},$$

其中 μ_j 與 σ_j 是特徵 j **在訓練集上計算**的平均值與標準差。縮放後每個特徵的平均為 0、標準差為 1。另一個做法是**最小–最大縮放**（min–max scaling），$x'_j = (x_j - \min_j)/(\max_j - \min_j)$，把訓練集的範圍對應到 $[0, 1]$，這也是像素強度的常用選擇（8 位元影像除以 255）。兩者在 scikit-learn 中分別是 `StandardScaler` 與 `MinMaxScaler` [5]。

兩條規則可以避免隱微的錯誤。只在訓練集上計算 μ_j, σ_j，再把**同一組**數字套用到驗證、測試和未來的資料。並且記得縮放會改變學到的權重的意義：權重現在代表特徵每增加一個標準差時預測值改變多少。

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=200, n_features=2, noise=5.0, random_state=1)
X = X * np.array([1000.0, 1.0])               # feature 1 now has a ~1000x wider range

def gd(X, y, alpha, steps=500):
    m, n = X.shape
    w, b = np.zeros(n), 0.0
    for _ in range(steps):
        err = X @ w + b - y
        w -= alpha * X.T @ err / m
        b -= alpha * err.mean()
    return ((X @ w + b - y) ** 2).mean() / 2
```

```
print("raw,    alpha=1e-7:", round(gd(X, y, 1e-7), 2))    # 158.01; alpha=3e-6 diverges
mu, sigma = X.mean(axis=0), X.std(axis=0)
Xs = (X - mu) / sigma                          # z-score standardization
print("scaled, alpha=0.1 :", round(gd(Xs, y, 0.1), 2))    # 13.47
```

在原始特徵上，任何大於約 2×10^{-6} 的學習率都會發散；用安全的學習率跑 500 步，成本仍卡在 158 左右（跑 5,000 步也還是 158 左右），因為第二個權重幾乎不動。標準化之後，$\alpha = 0.1$ 輕鬆到達最小值（大約 13，也就是雜訊的程度）。

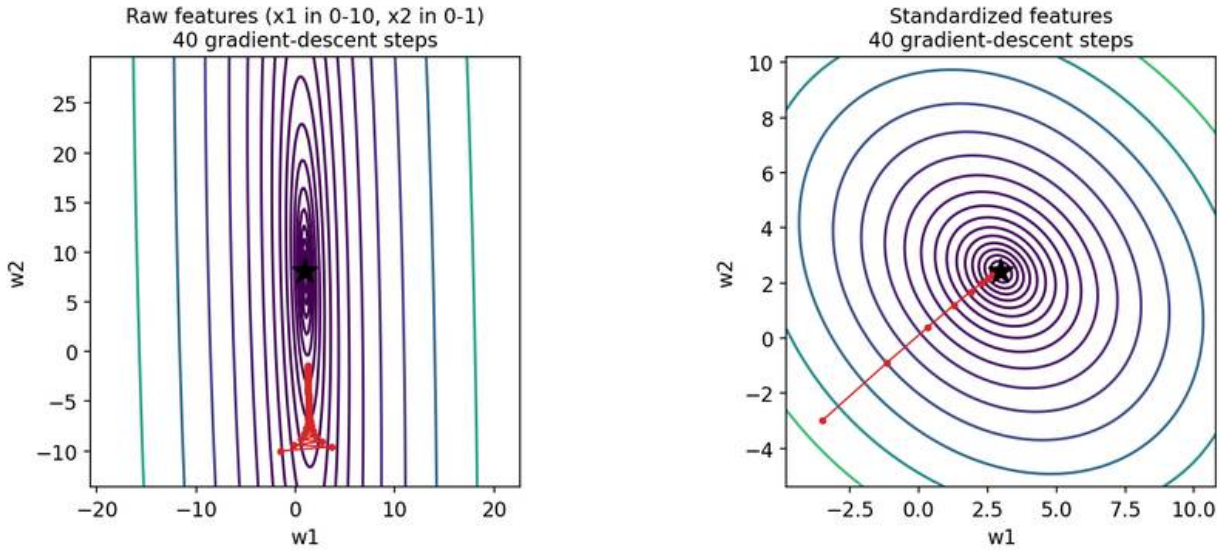


圖 *6* — 特徵縮放重塑了成本地形。左：尺度差異很大的特徵形成狹窄的山谷，梯度下降呈鋸齒狀前進。右：標準化之後，等高線接近圓形，同樣的步數就能到達最小值。

用於分類的邏輯迴歸

白話版。保留線性迴歸的加權總和，但把輸出壓進 0 到 1 之間，讓它可以被解讀成機率。然後選擇權重，讓真實類別的機率越高越好。

Sigmoid 函數

Sigmoid（logistic）函數是

$$g(z) = \frac{1}{1 + e^{-z}},$$

其中 z 是任意實數。它把 $-\infty$ 對應到 0、$+\infty$ 對應到 1、0 恰好對應到 0.5，中間是一條 S 形曲線（圖 7 左）。它還有對稱性 $g(-z) = 1 - g(z)$，以及方便的導數 $g'(z) = g(z)\,(1 - g(z))$。

模型與決策邊界

邏輯迴歸（logistic regression）把線性模型送進 sigmoid：

$$f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w}\cdot\mathbf{x}+b) = \frac{1}{1+e^{-(\mathbf{w}\cdot\mathbf{x}+b)}},$$

並把輸出解讀為標籤為 1 的估計機率，$f_{\mathbf{w},b}(\mathbf{x}) = P(y=1 \mid \mathbf{x})$。名字雖然叫「迴歸」，它其實是分類方法。做決定時，只要這個機率至少 0.5 就預測為 1。由於 $g(z) \geq 0.5$ 恰好在 $z \geq 0$ 時成立，規則就是

$$\hat{y} = 1 \quad \text{if} \quad \mathbf{w}\cdot\mathbf{x}+b \geq 0, \qquad \hat{y} = 0 \text{ otherwise.}$$

滿足 $\mathbf{w}\cdot\mathbf{x}+b=0$ 的集合就是**決策邊界**（decision boundary）。有兩個特徵時它是一條直線；一般而言它是一個平的超平面，而 $\mathbf{w}$ 與它垂直。離邊界很遠的點，機率接近 0 或 1；靠近邊界的點，機率接近 0.5。若想得到彎曲的邊界，就餵給模型非線性的特徵，例如 x_1^2 或 x_1x_2；模型對參數仍然是線性的。

成本：交叉熵（對數損失）

為什麼不沿用平方誤差？和 sigmoid 結合之後，平方誤差成本對 $(\mathbf{w},b)$ **不是**凸的：它會出現平坦的高原和多個山谷。所以邏輯迴歸改用**對數損失**（log loss），也稱為**二元交叉熵**（binary cross-entropy）：

$$L(f,y) = -y\log f \;-\; (1-y)\log(1-f),$$

其中 $f = f_{\mathbf{w},b}(\mathbf{x})$ 是預測機率，$y \in \{0,1\}$ 是真實標籤。每個範例只會用到兩項中的一項。若 $y=1$，損失是 $-\log f$：$f=1$ 時為零，$f \to 0$ 時無限增大。若 $y=0$，損失是 $-\log(1-f)$，正好是鏡像（圖 7 中）。信心滿滿卻答錯會受到很重的懲罰；信心滿滿且答對則幾乎沒有成本。

成本是整個訓練集上的平均：

$$J(\mathbf{w},b) = \frac{1}{m}\sum_{i=1}^{m}\Big[-y^{(i)}\log f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - (1-y^{(i)})\log\big(1-f_{\mathbf{w},b}(\mathbf{x}^{(i)})\big)\Big].$$

這個選擇並不隨意。如果把每個標籤看成一次擲硬幣，出現 1 的機率是 $f_{\mathbf{w},b}(\mathbf{x}^{(i)})$，那麼 J 正好是訓練標籤的負對數概似（negative log-likelihood）除以 m。最小化它就是**最大概似估計**（maximum likelihood estimation）[3]。而且搭配 sigmoid 時，這個成本是凸的，所以梯度下降又能找到唯一的全域最小值。

再一次梯度下降

利用 $g' = g(1-g)$，對數損失的導數會化簡得非常漂亮：

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\big(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}\big)\,x_j^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\big(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}\big).$$

它和線性迴歸的梯度**形式完全相同**：平均誤差乘以特徵。唯一的差別在 f 的內部。這不是巧合；只要輸出函數和它相配的概似函數成對使用，整個模型家族都有這個性質，這也是梯度下降程式碼這麼容易重複使用的原因之一。

```
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=200, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, class_sep=1.5, random_state=3)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def log_loss(p, y, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)              # avoid log(0)
    return -(y * np.log(p) + (1 - y) * np.log(1 - p)).mean()

w, b, alpha = np.zeros(2), 0.0, 0.5
for step in range(1000):
    p = sigmoid(X @ w + b)                    # predicted P(y = 1 | x)
    w -= alpha * X.T @ (p - y) / len(y)       # same form as linear regression
    b -= alpha * (p - y).mean()

p = sigmoid(X @ w + b)
print("log loss:", round(log_loss(p, y), 3))  # 0.06
print("accuracy:", ((p >= 0.5) == y).mean())  # 0.99
print("boundary: %.2f*x1 + %.2f*x2 + %.2f = 0" % (w[0], w[1], b))
```

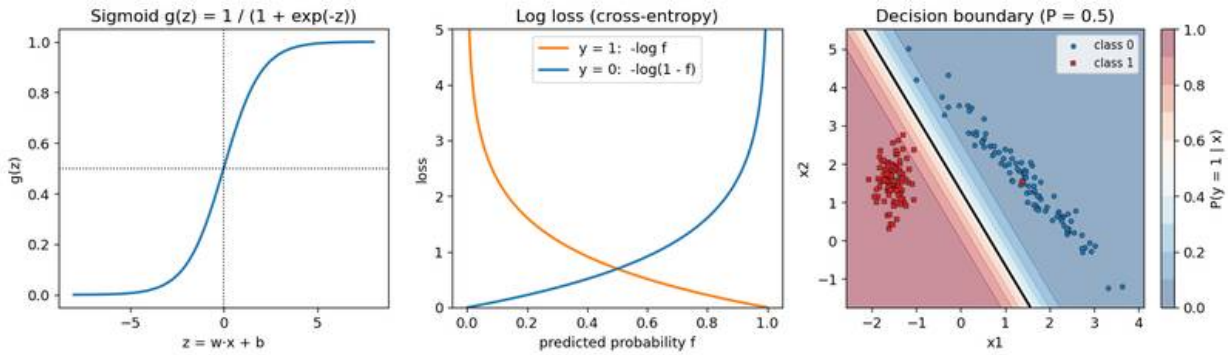


圖 7 — 邏輯迴歸。左：*sigmoid* 把任何分數壓進 $(0, 1)$。中：$y = 1$ 與 $y = 0$ 的對數損失，對信心滿滿的錯誤懲罰很重。右：從頭訓練的模型；底色是預測機率，黑線是決策邊界 $w{\cdot}x + b = 0$。

超過兩個類別

當類別數 $K > 2$ 時，標準的推廣方式是讓每個類別有自己的權重向量和分數 $z_k = \mathbf{w}_k \cdot \mathbf{x} + b_k$，再用 **softmax** 函數把分數轉成總和為 1 的機率：$P(y = k \mid \mathbf{x}) = e^{z_k} / \sum_{l=1}^{K} e^{z_l}$。成本同樣是真實類別的負對數機率的平均（類別交叉熵）[3]。本篇開頭 scikit-learn 的 `LogisticRegression` 處理十個數字時做的就是這件事，而它也正是ML 02 中大多數分類網路的輸出層。

過擬合、欠擬合與正則化

白話版。太簡單的模型抓不到模式（**欠擬合**，underfitting）。太有彈性的模型會把訓練範例連同雜訊一起背下來，遇到新範例就失敗（**過擬合**，overfitting）。我們用模型沒訓練過的資料來分辨兩者，並用懲罰大權重的方式馴服過於有彈性的模型。

親眼看它發生

取 25 個帶雜訊的平滑曲線樣本，擬合不同次數的多項式，也就是在特徵 $x, x^2, \dots, x^d$ 上做線性迴歸（圖 8）：

- **1 次（欠擬合，高偏差）**。直線跟不上曲線。不論在訓練集**還是**新資料上，誤差都很大。
- **3 次（擬合良好）**。曲線跟著趨勢走，忽略雜訊。兩邊的誤差都很小。
- **15 次（過擬合，高變異）**。曲線幾乎精確地穿過每個訓練點，所以訓練誤差接近零。但在點與點之間和點的範圍之外，它劇烈擺盪，新資料上的誤差暴增。

光看訓練誤差無法分辨好模型和過擬合的模型；它甚至偏好過擬合的那個。只有模型沒看過的資料才能揭露差別。這就是我們保留驗證集的原因。

使用驗證集與測試集

流程是：

1. 在**訓練集**上擬合每個候選模型（每種次數、每種正則化強度）。
2. 在**驗證集**上比較候選模型，保留最好的那個。
3. 在**測試集**上回報所選模型的誤差，只做一次。

為什麼要三份而不是兩份？因為在驗證集上從眾多候選中挑出最好的，本身就是一種擬合：勝出者有一部分只是剛好在那些特定範例上運氣好。沒動過的測試集才能給出不偏的最終估計。資料很少時，可以用**交叉驗證**（cross-validation）取代單一的驗證集：把訓練資料切成 k 份，訓練 k 次，每次用不同的一份做驗證，再取平均。

要留意的模式：如果訓練誤差和驗證誤差都很高，模型欠擬合（換更有彈性的模型或更好的特徵）。如果訓練誤差很低、驗證誤差卻高得多，模型過擬合（取得更多資料、簡化模型，或加上正則化）。

L2 正則化

正則化（regularization）保留有彈性的模型，但不鼓勵極端的參數值。15 次多項式那條狂野的曲線，需要非常大、又彼此精細抵消的權重。**L2 正則化**在成本上加入權重平方大小的懲罰：

$$J_{\text{reg}}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2,$$

其中 J 是原本的成本（MSE 或對數損失），$\lambda \geq 0$ 是**正則化強度**（一個超參數），加總的範圍是所有權重，但慣例上不包括偏差 b。$\lambda = 0$ 時回到原本的模型；λ 越大，權重越被拉向零，擬合出的函數越平滑；λ 太大則會欠擬合。對線性迴歸來說，這就是**脊迴歸**（ridge regression），由 Hoerl 與 Kennard 提出，用來在特徵高度相關時穩定最小平方估計 [20]。

從梯度下降的更新可以看出懲罰項做了什麼：

$$w_j \leftarrow w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \frac{\partial J}{\partial w_j}.$$

在每次一般的更新之前，每個權重都先乘上一個略小於 1 的數。這就是為什麼 L2 正則化在神經網路訓練中也被稱為**權重衰減**（weight decay）[3]。

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(0)
x = rng.uniform(0, 1, 25)
y = np.sin(2 * np.pi * x) + rng.normal(0, 0.25, 25)      # true curve + noise
X = x[:, None]

# 60% train, 20% validation, 20% test
X_tmp, X_te, y_tmp, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
X_tr, X_va, y_tr, y_va = train_test_split(X_tmp, y_tmp, test_size=0.25, random_state=0)

def fit(degree, lam=0.0):
    reg = Ridge(alpha=lam) if lam > 0 else LinearRegression()   # sklearn calls lambda
    ↪  "alpha"
    model = make_pipeline(PolynomialFeatures(degree), StandardScaler(), reg)
    return model.fit(X_tr, y_tr)

for degree, lam in [(1, 0), (3, 0), (15, 0), (15, 1e-2)]:
    model = fit(degree, lam)
    tr = mean_squared_error(y_tr, model.predict(X_tr))
    va = mean_squared_error(y_va, model.predict(X_va))
    print(f"degree {degree:2d}, lambda {lam:g}: train MSE {tr:.3f}, val MSE {va:.3g}")

best = fit(15, 1e-2)                                       # chosen on validation only
print("test MSE:", round(mean_squared_error(y_te, best.predict(X_te)), 3))
```

輸出說明了一切：1 次的訓練／驗證 MSE 是 0.144 ／ 0.292（欠擬合），3 次是 0.031 ／ 0.052（良好），沒有正則化的 15 次訓練 MSE 只有 0.003，驗證 MSE 卻高達數百萬，因為有些驗證點落在訓練點之間很寬的空隙中（例如 $x \approx 0.27$ 到 $x \approx 0.61$ 之間），多項式在那裡劇烈擺盪。對同一個 15 次模型加上溫和的 L2 懲罰（$\lambda = 0.01$），得到 0.041 ／ 0.035，測試 MSE 為 0.075。

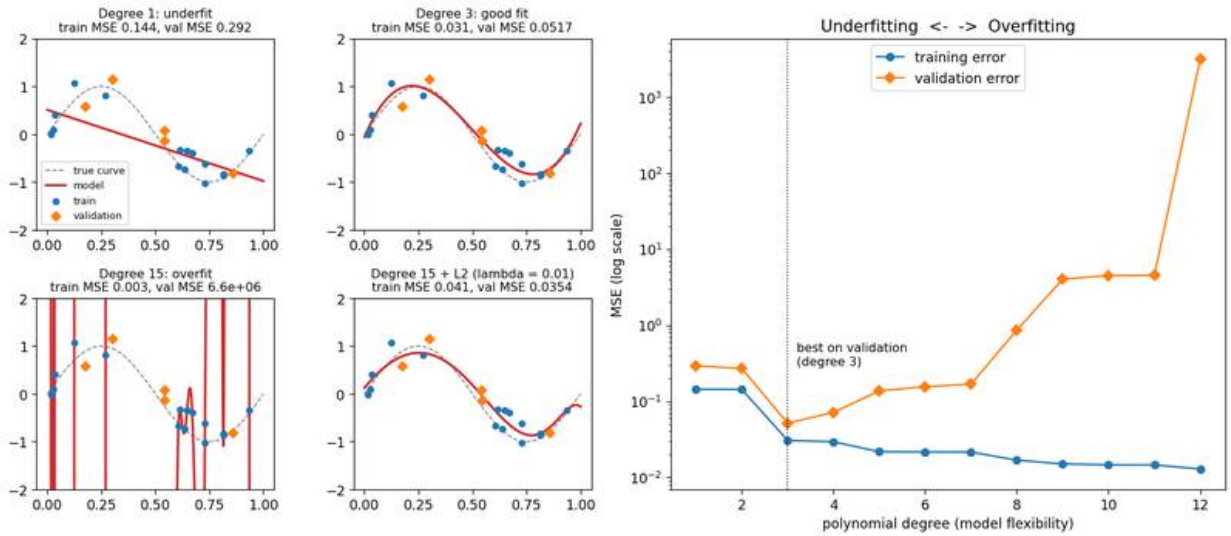


圖 *8* — 欠擬合、良好擬合、過擬合，以及被 *L2* 正則化馴服的過擬合（藍色為訓練點，橘色為驗證點）。右：次數增加時，訓練誤差持續下降，驗證誤差則先降後升；驗證曲線的最低點就是該選的模型。

評估模型

白話版。單一個數字永遠說不完整個故事。選一個符合「犯錯代價」的指標，而且一定要和簡單的基準比較。

迴歸指標

迴歸最常用的指標是 MSE，$\frac{1}{m}\sum_i (\hat{y}^{(i)} - y^{(i)})^2$，這裡不再有只是為了訓練方便的 $\frac{1}{2}$。它的平方根 **RMSE** 和 y 的單位相同，比較好解讀（「平均大約差 3 分」）。**平均絕對誤差**（mean absolute error），$\frac{1}{m}\sum_i |\hat{y}^{(i)} - y^{(i)}|$，對少數大誤差比較不敏感 [5]。

分類指標

分類中，**準確率**（accuracy）是預測正確的比例。它容易理解，也容易騙人。在 digits 資料中，大約 10% 的影像是 9。一個永遠回答「不是 9」的「模型」有 90% 的準確率，卻完全沒用。

混淆矩陣（confusion matrix）計算二元問題的四種結果，「陽性」代表我們在意的那一類：

	預測為陽性	預測為陰性
實際為陽性	真陽性（TP）	偽陰性（FN）
實際為陰性	偽陽性（FP）	真陰性（TN）

由此得到

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

精確率（precision）回答「模型說是陽性時，有多常是對的？」**召回率**（recall）回答「所有真正的陽性中，模型找到了多少？」兩者互相拉扯：把邏輯迴歸的 0.5 決策門檻調低，會找到更多陽性（召回率較高），但也會發出更多假警報（精確率較低）。哪個比較重要取決於每種錯誤的代價：癌症篩檢不該漏掉腫瘤（高召回率），垃圾郵件過濾器則不該把真正的郵件藏起來（高精確率）。**F1 分數**是精確率與召回率的調和平均，把兩者合成一個數字 [5]。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

X, y = load_digits(return_X_y=True)
y = (y == 9).astype(int)                  # "is it a 9?": only about 10% positives
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

lazy = np.zeros_like(y_te)                # always answers "not a 9"
model = LogisticRegression(max_iter=5000).fit(X_tr, y_tr)
pred = model.predict(X_te)

print("lazy accuracy :", round(accuracy_score(y_te, lazy), 3), " recall:",
 ↪  recall_score(y_te, lazy))
print("model accuracy:", round(accuracy_score(y_te, pred), 3),
      " precision:", round(precision_score(y_te, pred), 3),
      " recall:", round(recall_score(y_te, pred), 3))
print(confusion_matrix(y_te, pred))       # rows: true 0/1, columns: predicted 0/1
# lazy accuracy : 0.9   recall: 0.0
# model accuracy: 0.974  precision: 0.87  recall: 0.87
```

偷懶的基準有 90% 的準確率，召回率卻是零。真正的模型準確率 97.4%，聽起來只好一點點，但 0.87 的召回率顯示它確實找到了大部分的 9。一定要回報基準；類別不平衡時，一定要看準確率以外的指標。同樣的指標會在 DIP 的圖樣分類一章再次出現，並更仔細地處理門檻與多類別的問題。

現代觀點

本篇的機制（帶參數的模型、成本、以梯度為基礎的最佳化，以及保留資料的評估）幾十年來沒有改變，下面談到的每個系統也都還在使用它們。改變的是**特徵從哪裡來**、**標籤從哪裡來**，以及**模型有多大**。以下幾篇回顧文章把這些轉變描繪得很好。

機器學習這個領域。 Jordan 與 Mitchell 發表在《Science》的回顧 [7] 以一個問題來界定機器學習：如何打造能透過經驗自動進步的電腦；並把它定位在電腦科學與統計學的交會處、人工智慧與資料科學的核心。他們把機器學習在科學、醫療、製造、教育與金融等領域的快速擴散，歸因

於三件事同時到來：新的學習演算法與理論、可取得資料量的爆炸性成長，以及便宜的運算。這個框架是對抗炒作的好解藥：方法的好壞，取決於它得到的資料與經驗。

從特徵工程到表示學習。很長一段時間，傳統模型的成敗主要取決於人工設計的特徵，也就是由專家來做圖 1 中「數有墨跡的欄數」這一步：邊緣直方圖、紋理描述子、關鍵點描述子（見 IP 12）。Bengio、Courville 與 Vincent [21] 認為這種依賴是一個弱點，因為正確的表示決定了任務的難易，並主張直接從資料學習表示本身。他們的回顧涵蓋了做到這件事的工具（機率模型、自編碼器、流形學習與深度網路），以及讓表示變好的通用先驗，例如把資料背後各自獨立的解釋因子分離開來。用本篇的語言來說，深度網路仍然是「特徵加上邏輯迴歸」，只是特徵由前面的層產生，而這些層的權重也是用同樣的梯度下降學出來的。這就是ML 02 的主題。

自監督式學習。學習表示需要資料，而標籤是昂貴的部分。Jing 與 Tian 關於自監督式視覺特徵學習的綜述 [22] 把不需要標籤的「前置任務」（pretext task）分成四類：以生成為基礎（例如重建影像或為影像上色）、以上下文為基礎（利用空間或時間結構，例如影像區塊的排列）、以免費語意標籤為基礎（使用自動得到的標籤，例如來自繪圖引擎或傳統方法），以及跨模態（判斷兩個訊號，例如影片與聲音，是否屬於同一事件）。它也確立了至今仍是標準的評估流程：用表示遷移到分類、偵測、分割等下游任務的表現來評判它。Gui 等人較新的綜述 [23] 則把這個領域重新整理為：以上下文為基礎的方法、對比學習（利用負例、自蒸餾或特徵去相關）、以遮罩影像建模為主的生成式方法，以及結合對比與生成目標的混合方法，反映出遮罩自編碼器 [12] 之後轉向遮罩預測的趨勢。對實務工作者，Balestriero 等人的《A Cookbook of Self-Supervised Learning》[24] 把現代方法分成深度度量學習家族、自蒸餾家族、典型相關分析家族與遮罩影像建模，並坦白指出結果有多依賴資料擴增與訓練時使用的小型「投影器」（projector）網路等細節。

基礎模型。 Bommasani 等人的報告 [10] 為由此產生的範式命名：在大規模、廣泛的資料上訓練（通常是自監督式），再調整去做許多下游任務的模型。報告強調了兩個後果。**湧現**（emergence）：像是憑提示中的幾個例子就能遵循指示 [11] 這類能力，是隨著規模出現的，而不是被設計出來的。**同質化**（homogenization）：許多應用如今都由同樣少數幾個基礎模型調整而來，這帶來巨大的槓桿效果，但也代表基礎模型的缺陷會被建立在它之上的一切所繼承。報告也強調，我們對這些模型如何運作、何時會失敗的理解仍非常有限。

基於人類回饋的強化學習。強化學習已經從遊戲走到語言模型訓練的最後階段 [13]、[14]。Casper 等人 [25] 整理了可能出錯的地方。他們依照 RLHF 的三個步驟（收集人類回饋、擬合獎勵模型、最佳化策略），逐一列出每一步的問題：人的回饋帶有雜訊、前後不一致，而且可能被操弄；獎勵模型只是「人們真正想要的東西」的不完美替代；而針對這個替代品用力最佳化的策略，可能會鑽它的漏洞。他們區分了在 RLHF 框架內用更好的方法就能解決的問題，以及需要換一種方法才能克服的根本限制，並主張提高透明度、把 RLHF 與其他安全措施結合，而不是只依賴它。

大規模最佳化。 Bottou、Curtis 與 Nocedal [18] 解釋了為什麼成為機器學習主力的，是樸素的隨機梯度法，而不是傳統數值分析中更精巧的最佳化方法：資料充足時，便宜而帶雜訊的一步勝過昂貴而精確的一步。Adam [19] 等自適應方法則在其上加入每個參數各自的步長。本節提到的每個模型，無論多大，都是用本篇小批次迴圈的後代訓練出來的。

未解問題。儘管規模驚人，這篇入門教學中的幾個問題仍然是最困難的。**泛化**：非常大的模型幾乎完美地擬合訓練資料，卻仍能泛化，這和圖 8 中簡單的過擬合圖像不符，其原因尚未完全被理解。**評估**：當模型用了相當大比例的公開網路資料來訓練，就很難保證測試集沒有出現在訓練資

料中，因此誠實評估的基礎，乾淨的訓練／測試分離，變得難以驗證。**目標**：成本函數是模型唯一會最佳化的東西，而 RLHF 顯示出要寫出一個真正反映人們需求的成本有多難。還有**資料**：模型能學到什麼，受限於它看到的資料與標籤，以及其中所有的缺口與偏差。

針對電腦視覺，傳統路線（人工特徵加上簡單分類器）在 DIP 的圖樣分類一章中有完整發展；學習的路線則延續到ML 02 與〈Deep Learning for Image Processing〉。

重點整理

- 機器學習用一個參數由範例擬合而來的彈性模型，取代手寫規則；當規則難以說清楚、範例卻容易取得時，它最能發揮。
- 各範式的差別在於回饋：監督式（有正確答案；數字用迴歸、類別用分類）、非監督式（沒有答案；分群、降維、異常偵測）、自監督式（答案由資料本身製造），以及強化學習（動作換來獎勵）。KNN 是監督式；k-means 是分群。
- 把模型 $f_{w,b}(x)$ 和成本 $J(w,b)$ 分開：模型把輸入對應到預測值；成本把參數對應到誤差。訓練就是最小化 J。
- 梯度下降重複 $w \leftarrow w - \alpha\, \partial J/\partial w$。在最小值處，**導數**為零。學習率必須小到不會越過頭，又大到能有進展；小批次讓每一步都很便宜。
- 做梯度下降前先縮放特徵，而且只用訓練集的統計量。
- 邏輯迴歸對線性分數套上 sigmoid，並最小化交叉熵；它的決策邊界是 $\mathbf{w}\cdot\mathbf{x}+b=0$，梯度形式和線性迴歸相同。
- 用模型沒訓練過的資料評判它：用驗證集做選擇、在測試集上只回報一次、模型過擬合時加上（L2）正則化，類別不平衡時要看準確率以外的指標。

練習

1. **手算導數**。對單一特徵的 MSE 成本 $J(w,b)=\frac{1}{2m}\sum_i (wx^{(i)}+b-y^{(i)})^2$，推導 $\partial J/\partial w$ 與 $\partial J/\partial b$。接著對三個點 $(1,2),(2,4),(3,6)$，從 $w=0, b=0$ 出發，以 $\alpha=0.1$ 計算一步梯度下降。

   提示

   對每個平方項用連鎖律得到 $2(wx^{(i)}+b-y^{(i)})\cdot x^{(i)}$，2 和 $\frac{1}{2}$ 抵消，就得到梯度下降一節中的公式。在 $w=b=0$ 時誤差是 $-2,-4,-6$。所以 $\partial J/\partial w = (-2\cdot 1-4\cdot 2-6\cdot 3)/3 = -28/3$，$\partial J/\partial b = -12/3 = -4$。一步之後 $w = 0.1\cdot 28/3 \approx 0.933$，$b = 0.4$。

2. **找出發散的門檻**。對 $J(w)=\frac{1}{2m}\sum_i (wx^{(i)}-y^{(i)})^2$（沒有偏差），證明一步梯度下降會把到最佳值 w^* 的距離乘上 $1-\alpha s$，其中 $s=\frac{1}{m}\sum_i (x^{(i)})^2$。$\alpha$ 在什麼範圍內梯度下降會收斂？用特徵縮放一節中的 `gd` 函數檢查你的答案。

   提示

   因為 J 是曲率為 s、最小值在 w^* 的拋物線，$\partial J/\partial w = s\,(w-w^*)$。所以 $w_{\text{new}} - w^* = (1-\alpha s)(w-w^*)$。當 $|1-\alpha s|<1$，也就是 $0<\alpha<2/s$ 時收斂；$\alpha = 1/s$ 時一步就落在最小值上。有多個特徵時，s 換成 $\frac{1}{m}X^\top X$ 的最大特徵值，這就是為什麼一個尺度巨大的特徵會逼得 α 必須非常小。

3. **用迴歸做分類**。用 `make_classification(n_features=1, n_informative=1, n_redundant=0, n_clusters_per_class=1)` 產生一維資料，再加上五個 x 非常大的第 1 類範例。分別擬合 `LinearRegression`（以 0.5 為門檻）與 `LogisticRegression`。比較加入額外點前後兩者的準確率，並解釋差異。

   提示

   額外的點很容易分類，但平方誤差仍然希望直線經過它們附近。直線的斜率下降，跨過 0.5 的位置往第 1 類移動，一些靠近邊界的普通第 1 類點因此被分錯。分類正確且離得很遠的點，對數損失已經接近 0，所以邏輯迴歸幾乎不受影響。

4. **縮放 digits**。用本篇從頭實作的邏輯迴歸，在 digits 資料上訓練「是 9 或不是 9」，一次用原始像素（0 到 16），一次用除以 16 的像素。使用相同的步數，找出各自可用的最大學習率。縮放改變的是最終準確率、速度，還是兩者都有？

   提示

   這裡所有像素的單位相同，所以效果比圖 6 溫和，但原始輸入大了 16 倍，梯度的大小和地形的曲率也隨之放大。原始像素的安全學習率大約小 $16^2 = 256$ 倍。學習率都調好時，兩者能達到相近的準確率；縮放過的版本只是比較容易調。

5. **選擇 lambda**。用多項式的例子，固定 15 次，嘗試 $\lambda \in \{10^{-6}, 10^{-4}, 10^{-2}, 1, 100\}$。以對數橫軸畫出訓練與驗證 MSE 對 λ 的變化。哪一端欠擬合、哪一端過擬合？你會選哪個 λ？為什麼不能用測試集來做這個選擇？

   提示

   很小的 λ 等於沒有正則化（過擬合：訓練誤差低、驗證誤差高）。很大的 λ 把曲線壓成接近常數（欠擬合：兩種誤差都高）。選驗證誤差最低的 λ。如果測試集參與了選擇，它就不再是新資料上效能的不偏估計。

6. **哪一種範式？**把每個問題歸類為監督式（迴歸或分類）、非監督式、自監督式或強化學習，並說出目標值或獎勵：(a) 用附有實際結果的使用紀錄，估計手機剩餘的電池時間；(b) 把一百萬張沒有標籤的商品照片分組以便瀏覽；(c) 藉由預測每張照片缺少的四分之一來預訓練影像模型；(d) 教機械手臂堆積木，只有在積木堆穩時才給分數；(e) 在沒有任何故障標註的情況下，標出工廠感測器資料流中的異常讀數。

   提示

   (a) 監督式迴歸；目標值是剩餘時數。(b) 非監督式分群。(c) 自監督式；目標值是被藏起來的像素，由資料本身產生。(d) 強化學習；獎勵是積木是否堆穩，要到最後才給。(e) 非監督式異常偵測。

接下來的機器學習／深度學習路線

若數學符號還不熟悉，先讀線性代數、機率與資訊理論 和數值計算，再進入類神經網路。

後續從影像濾波與 CNN 連接循環／遞迴網路、Transformer 和表示學習。接著由因子模型與自編碼器進入 VAE、GAN、正規化流、擴散／ flow matching、自迴歸與可重用的基礎模型，最後用評估與部署 完成一個可靠實驗。統計決策與推論的觀點見 圖形識別，具體視覺任務則見 視覺深度學習。

參考文獻

1. Shyandram, “Introduction to Machine Learning,” Medium post, 2024; updated 2026. link
2. A. L. Samuel, “Some studies in machine learning using the game of checkers,” *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, 1959. doi
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. book site
4. A. Ng, E. Shyu, A. Bagul and G. Ladwig, “Machine Learning Specialization,” DeepLearning.AI and Stanford Online, Coursera. course page
5. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. link
6. E. Alpaydin and C. Kaynak, “Optical Recognition of Handwritten Digits,” UCI Machine Learning Repository, 1998. doi
7. M. I. Jordan and T. M. Mitchell, “Machine learning: Trends, perspectives, and prospects,” *Science*, vol. 349, no. 6245, pp. 255–260, 2015. doi
8. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. book page
9. Y. LeCun and I. Misra, “Self-supervised learning: The dark matter of intelligence,” Meta AI blog, 2021. link
10. R. Bommasani, D. A. Hudson, E. Adeli et al., “On the opportunities and risks of foundation models,” arXiv:2108.07258, 2021. arXiv
11. T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 33, 2020. link
12. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), pp. 15979–15988, 2022. arXiv
13. L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright et al., “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 35, 2022. link
14. DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *Nature*, vol. 645, pp. 633–638, 2025. arXiv
15. A. K. Jain, “Data clustering: 50 years beyond K-means,” *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010. doi
16. I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, 20150202, 2016. doi

17. V. Chandola, A. Banerjee and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, 2009. doi
18. L. Bottou, F. E. Curtis and J. Nocedal, “Optimization methods for large-scale machine learning,” *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. doi
19. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015. arXiv
20. A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970. doi
21. Y. Bengio, A. Courville and P. Vincent, “Representation learning: A review and new perspectives,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013. doi
22. L. Jing and Y. Tian, “Self-supervised visual feature learning with deep neural networks: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 11, pp. 4037–4058, 2021. doi
23. J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo and D. Tao, “A survey on self-supervised learning: Algorithms, applications, and future trends,” arXiv:2301.05712, 2023. arXiv
24. R. Balestriero, M. Ibrahim, V. Sobal, A. Morcos, S. Shekhar et al., “A cookbook of self-supervised learning,” arXiv:2304.12210, 2023. arXiv
25. S. Casper, X. Davies, C. Shi et al., “Open problems and fundamental limitations of reinforcement learning from human feedback,” *Transactions on Machine Learning Research*, 2023. TMLR ·arXiv

ML 01A・學習所需的線性代數

向量、矩陣與張量、範數、特徵分解、奇異值分解、偽反矩陣、跡與行列式，並一步步推導主成分分析，附經過測試的 NumPy 程式碼。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-01a-linear-algebra/

你將學到

- 純量、向量、矩陣與張量如何表示影像、批次與學習到的轉換，以及如何把它們的形狀弄清楚。
- 矩陣乘法的四種讀法，以及單位矩陣、反矩陣、生成空間、秩與零空間對線性方程組的意義。
- 範數如何量測大小、特殊矩陣（對角、對稱、正交）如何簡化計算，以及內積如何定義相似度。
- 特徵分解與奇異值分解（SVD）揭露了矩陣的哪些性質，以及為什麼 SVD 給出最佳的低秩近似。
- Moore–Penrose 偽反矩陣如何解最小平方與欠定問題，以及條件數和嶺正則化為什麼重要。
- 跡與行列式的用法，以及如何只靠這些工具，從頭推導主成分分析（PCA）。

先看全貌

神經網路裡幾乎每個運算，都是「線性映射」接上一個簡單的非線性函數。全連接層是乘上權重矩陣；卷積是有結構的矩陣乘法；注意力機制先計算向量之間的內積，再用一個權重矩陣把向量混合起來。線性代數就是描述這一切的語言。它告訴我們一個映射保留了什麼、丟掉了什麼、拉長了什麼，以及怎樣計算才不會損失精確度。

> **用最簡單的話說**
>
> 向量是一串測量值，例如一個人的身高、體重和年齡。矩陣像一台機器：把一串數字放進去，它會混合出一串新的數字。有的機器只是把這串數字「轉個方向」（旋轉），有的把某些部分放大、某些部分縮小（拉伸），有的則把某些部分壓扁，資訊從此就找不回來。線性代數就是這些機器的說明書：它告訴你每台機器朝哪個方向拉、拉多少，以及能不能把動作復原。

本篇對應 Goodfellow、Bengio 與 Courville 所著《Deep Learning》的「應用數學」部分 [1]，以及《Mathematics for Machine Learning》的線性代數章節 [2]。想多做練習，Strang 的教科書 [3] 是最平易近人的完整課程；Golub 與 Van Loan [4]、Trefethen 與 Bau [5] 則是「如何在電腦上可靠地完成這些計算」的標準參考書。如果你還不熟悉矩陣符號，建議在ML 02 之前先讀本篇。姊妹篇是ML 01B・機率與資訊理論 與ML 01C・數值計算。

本篇有一條主線。我們從物件與形狀開始，接著談乘法與解方程式，再用範數量測大小。特徵分解與 SVD 說明一個矩陣的「個性」；偽反矩陣、跡與行列式則是建立在它們之上的工具。最後，我們用上述每一項工具推導 PCA——這個經典的資料壓縮方法——推導方式與 [1] 相同。

純量、向量、矩陣與張量

白話版。機器學習裡的每個數字都住在一個有命名軸的陣列裡。做任何乘法之前，先說清楚每個軸代表什麼。

純量（scalar）是單一數字，寫成 $s \in \mathbb{R}$。**向量**（vector）是 n 個數字的有序清單：$x \in \mathbb{R}^n$，分量為 $x_1, \dots, x_n$；依慣例，向量是一個行向量（直的）。**矩陣**（matrix）是二維的數字格子：$A \in \mathbb{R}^{m \times n}$ 有 m 列（row）與 n 行（column），A_{ij} 是第 i 列、第 j 行的元素。**張量**（tensor）則是任意多個軸的陣列。

這些物件可以直接對應到資料：

- 高 H、寬 W 的灰階影像是 $\mathbb{R}^{H \times W}$ 的矩陣；攤平後就是向量 $x \in \mathbb{R}^{HW}$（見影像即陣列）。
- n 筆樣本、每筆 d 個特徵，每列放一筆樣本，就得到**設計矩陣**（design matrix）$X \in \mathbb{R}^{n \times d}$。
- 權重 $W \in \mathbb{R}^{d \times m}$、偏差 $b \in \mathbb{R}^m$ 的全連接層一次處理整個批次：$Y = XW + \mathbf{1}b^\top \in \mathbb{R}^{n \times m}$，其中 $\mathbf{1} \in \mathbb{R}^n$ 是全為 1 的向量。
- PyTorch 的影像批次通常是形狀 $B \times C \times H \times W$（批次、通道、高、寬）的四維張量；Transformer 的 token 嵌入批次通常是 $B \times N \times D$（批次、序列長度、嵌入維度）。

轉置（transpose）$A^\top$ 把列與行對調：$(A^\top)_{ij} = A_{ji}$。它滿足 $(A^\top)^\top = A$ 與 $(AB)^\top = B^\top A^\top$——注意順序反過來了。

廣播

寫 $\mathbf{1}b^\top$ 在數學上很誠實，但很繁瑣，所以程式庫允許你直接把向量加到矩陣的每一列。這叫**廣播**（broadcasting）：兩個陣列形狀不同時，長度為 1 的軸（或缺少的前導軸）會被延展成相同大小。廣播很方便，也很危險，因為形狀錯了程式往往照樣執行。最常見的錯誤是把形狀 $(n, 1)$ 的行向量和形狀 $(n,)$ 的一維陣列相減，結果悄悄變成 $n \times n$ 的矩陣，而不是 n 個殘差。

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(32, 64))      # batch of 32 examples, 64 features
W = rng.normal(size=(64, 10))      # linear map 64 -> 10
b = rng.normal(size=10)            # one bias per output
Y = X @ W + b                      # b broadcasts over the 32 rows
assert Y.shape == (32, 10)

# A classic broadcasting bug: (32,) - (32, 1) silently becomes (32, 32)
y_true = rng.normal(size=32)
y_pred = rng.normal(size=(32, 1))
print("buggy residual shape:", (y_true - y_pred).shape)
print("fixed residual shape:", (y_true - y_pred[:, 0]).shape)

# Four equivalent views of the same matrix product
A, B = rng.normal(size=(3, 4)), rng.normal(size=(4, 2))
entry = np.array([[A[i] @ B[:, j] for j in range(2)] for i in range(3)])
cols = np.stack([A @ B[:, j] for j in range(2)], axis=1)
```

```
outer = sum(np.outer(A[:, k], B[k]) for k in range(4))
assert np.allclose(A @ B, entry) and np.allclose(A @ B, cols) and np.allclose(A @ B, outer)
print("all views agree")
```

錯誤的那一行印出 `(32, 32)`，修正後是 `(32,)`。對 32×32 陣列取平均仍然只得到一個數字，所以用這種方式算出的損失函數可以順利訓練，卻在最佳化錯誤的目標。凡是重要的程式碼，都替形狀加上 `assert`。

矩陣與向量的乘法

白話版。矩陣乘法把兩台線性機器串成一台。輸出的每個數字，是第一個矩陣的某一列與第二個矩陣的某一行的內積。

對 $A \in \mathbb{R}^{m\times n}$ 與 $B \in \mathbb{R}^{n\times p}$，**矩陣乘積** $C = AB \in \mathbb{R}^{m\times p}$ 為

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

其中 i 是 A（也是 C）的列索引，j 是 B（也是 C）的行索引，k 走遍共同的內部維度 n。內部維度必須相同——遇到形狀錯誤時，這是第一個要檢查的地方。

同一個乘積有四種讀法，各有用處：

1. **元素觀點**。C_{ij} 是 A 的第 i 列與 B 的第 j 行的內積。注意力分數 $QK^\top$ 就是這樣讀：每個 query 和每個 key 比較。
2. **行觀點**。C 的第 j 行等於 A 乘上 B 的第 j 行；C 的每一行都是 A 各行的線性組合。
3. **列觀點**。C 的第 i 列等於 A 的第 i 列乘上 B。在 $Y = XW$ 中，每筆輸出樣本就是它的輸入列通過同一組權重。
4. **外積觀點**。$C = \sum_k a_{:k} b_{k:}$，是 n 個秩一矩陣的總和，每個由 A 的第 k 行與 B 的第 k 列構成。這個觀點會在本篇後面解釋低秩近似與 LoRA。

上面的程式碼驗證了四種讀法結果相同。

矩陣乘法滿足**結合律** $(AB)C = A(BC)$ 與**分配律** $A(B + C) = AB + AC$，但**不滿足交換律**：一般而言 $AB \neq BA$，而且其中一個乘積常常根本沒有定義。**逐元素**（Hadamard）乘積 $A \odot B$ 則是把對應元素相乘，需要形狀相同。在 NumPy 與 PyTorch 中，`A @ B` 是矩陣乘法，`A * B` 是逐元素乘法。兩個向量 $x, y \in \mathbb{R}^n$ 的**內積**是特例 $x^\top y = \sum_i x_i y_i$，而且對稱：$x^\top y = y^\top x$。

單位矩陣、反矩陣與線性方程組

m 條方程式、n 個未知數的線性方程組可以簡寫成

$$Ax = b, \qquad A \in \mathbb{R}^{m\times n},\ x \in \mathbb{R}^n,\ b \in \mathbb{R}^m.$$

A 的每一列是一條方程式的係數，x 是未知數，b 是等號右邊的值。

單位矩陣 I_n 對角線為 1、其餘為 0，不改變任何向量：$I_n x = x$。若方陣 A 存在矩陣 A^{-1} 使 $A^{-1}A = AA^{-1} = I$，就稱 A **可逆**，此時方程組恰有一解 $x = A^{-1}b$。

這個公式很適合推導，卻不適合計算。直接算出 A^{-1}，比用 LU、Cholesky 或 QR 分解直接求解更費時、也更不精確 [4], [5]。在 NumPy 中請寫 `np.linalg.solve(A, b)`，而不是 `np.linalg.inv(A) @ b`。本篇中的 A^{-1} 是思考工具。

線性相依、生成空間與秩

白話版。秩計算一個矩陣真正能產生多少個不同方向。重複或多餘的行不會增加新東西。

向量 $v^{(1)}, \dots, v^{(k)}$ 的**線性組合**是 $\sum_i c_i v^{(i)}$，c_i 為純量。一組向量的**生成空間**（span）是它們所有線性組合構成的集合。由於 $Ax = \sum_j x_j\, a_{:j}$，向量 Ax 是 A 各行 $a_{:j}$ 的線性組合。因此 $Ax = b$ 有解，若且唯若 b 落在各行的生成空間裡，這個空間稱為 A 的**行空間**（column space，或值域）。

若一組向量中沒有任何一個是其他向量的線性組合——等價地說，$\sum_i c_i v^{(i)} = 0$ 只在所有 $c_i = 0$ 時成立——就稱它們**線性獨立**。A 的**秩**（rank）是線性獨立的行數，而它永遠等於線性獨立的列數。對 $A \in \mathbb{R}^{m\times n}$，$\operatorname{rank}(A) \le \min(m, n)$；等號成立時稱 A **滿秩**。

A 的**零空間**（null space）是所有滿足 $Az = 0$ 的向量 z，也就是被 A 摧毀的輸入方向。**秩–零度定理**說

$$\operatorname{rank}(A) + \dim \operatorname{null}(A) = n,$$

其中 n 是行數，dim 是零空間中獨立方向的個數。這一條等式就解釋了 $Ax = b$ 的三種情況：

形狀與秩	會發生什麼	典型的機器學習情境
方陣、滿秩	恰有一解	良好設定的座標轉換
高瘦（$m > n$）、行滿秩	通常沒有精確解；改求最小平方解	樣本數多於特徵數的迴歸
矮胖（$m < n$）或秩不足	若有解則有無限多解	參數多於限制條件，例如深度網路

對方陣來說，「可逆」「滿秩」「各行線性獨立」「零空間只有零向量」是同一件事；不滿足的方陣稱為**奇異**（singular）矩陣。

範數：量測大小

白話版。範數把一個向量變成一個非負數，表示它有多大。不同的範數對「大」有不同看法。

範數是滿足以下條件的函數 f：$f(x) = 0 \Rightarrow x = 0$；$f(x + y) \le f(x) + f(y)$（三角不等式）；對任何純量 α，$f(\alpha x) = |\alpha| f(x)$。最常見的是 L^p 範數：

$$\|x\|_p = \Big(\sum_i |x_i|^p \Big)^{1/p}, \qquad p \ge 1.$$

- $\|x\|_2$ 是**歐幾里得**範數，也就是一般的長度，常簡寫為 $\|x\|$。它的平方 $\|x\|_2^2 = x^\top x$ 更好微分，出現在均方誤差損失與權重衰減中。

- $\|x\|_1 = \sum_i |x_i|$ 在每個地方——包括靠近零的地方——都以相同速率成長。當「恰好為零」和「稍微不為零」的差別很重要時它特別有用，這也是 L^1 懲罰會鼓勵稀疏解的原因。
- $\|x\|_\infty = \max_i |x_i|$ 是**最大範數**，即最大元素的大小。對抗強健性的研究常用它限制擾動。
- 非零元素的個數有時被稱為「L^0 範數」，但它不是範數：把 x 乘以 2 並不會改變它。

內積透過夾角與歐幾里得範數相連：$x^\top y = \|x\|_2 \|y\|_2 \cos\theta$，$\theta$ 是 x 與 y 的夾角。**餘弦相似度**

$$\cos(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}$$

只對非零向量有定義，並且忽略尺度。這對某些嵌入很有用（對比學習正因如此會把特徵正規化），但當大小本身有意義時——例如影像亮度——就不適用。

矩陣範數

矩陣最常用兩種範數。**Frobenius 範數**把矩陣當成一條長向量：

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}.$$

譜範數（spectral norm，即算子 2-範數）量測 A 能把單位向量拉長的最大倍數：

$$\|A\|_2 = \max_{\|x\|_2 = 1} \|Ax\|_2 = \sigma_{\max}(A),$$

其中 $\sigma_{\max}(A)$ 是最大奇異值，定義見下方 SVD 一節。若以全部奇異值 $\sigma_1 \geq \sigma_2 \geq \dots$ 表示，則 $\|A\|_F^2 = \sum_i \sigma_i^2$，而 $\|A\|_2 = \sigma_1$。所以許多小奇異值可以讓 Frobenius 範數變大，卻不改變最大放大倍數。譜範數限制了一層網路最多能把輸入擾動放大多少，因此出現在強健性與 GAN 穩定性的研究中 [17]。

```
import numpy as np

x = np.array([3.0, -4.0, 0.0, 1.0])
for p in [1, 2, np.inf]:
    print(f"L{p} norm:", np.linalg.norm(x, p))
print("L0 'norm' (count of nonzeros):", np.count_nonzero(x))

A = np.array([[3.0, 0.0], [4.0, 5.0]])
s = np.linalg.svd(A, compute_uv=False)
print("singular values:", s.round(4))
print("Frobenius:", np.linalg.norm(A, "fro"), "=", np.sqrt((s**2).sum()))
print("spectral :", np.linalg.norm(A, 2), "=", s[0])
assert np.isclose(np.linalg.norm(A, "fro")**2, np.trace(A.T @ A))
```

對 $x = (3, -4, 0, 1)$，程式印出 $L^1 = 8$、$L^2 = \sqrt{26} \approx 5.10$、$L^\infty = 4$，以及三個非零元素。這個 2×2 矩陣的奇異值是 $3\sqrt{5} \approx 6.71$ 與 $\sqrt{5} \approx 2.24$，因此 $\|A\|_F = \sqrt{50} \approx 7.07$、$\|A\|_2 \approx 6.71$。

特殊的矩陣與向量

白話版。有些矩陣比其他矩陣好處理得多。學會辨認它們，可以省下計算量，也能理解許多設計選擇。

- **對角矩陣** $D = \operatorname{diag}(v)$ 只有主對角線上有非零元素。乘上它等於把每個座標分別縮放：$\operatorname{diag}(v)x = v \odot x$。當每個 $v_i \neq 0$ 時，它的反矩陣是 $\operatorname{diag}(1/v_1, \dots, 1/v_n)$。正規化層中逐通道的縮放，其實就是對角矩陣。長方形的對角矩陣也存在，會出現在 SVD 中。
- **對稱矩陣**滿足 $A = A^\top$。共變異數矩陣、Gram 矩陣 $X^\top X$、平滑函數的 Hessian 與距離矩陣都是對稱的。
- **單位向量**滿足 $\|x\|_2 = 1$。若 $x^\top y = 0$，兩向量**正交**；若它們同時也是單位向量，就是**單範正交**（orthonormal）。
- **正交矩陣** Q 是各列、各行皆單範正交的方陣，因此 $Q^\top Q = QQ^\top = I$ 且 $Q^{-1} = Q^\top$——反矩陣不花成本。它保持長度與角度不變，$\|Qx\|_2 = \|x\|_2$，幾何上是旋轉、鏡射或兩者的組合。

若 $Q \in \mathbb{R}^{n \times k}$ 有 $k < n$ 個單範正交的行，則 $Q^\top Q = I_k$，但 $QQ^\top \neq I_n$。矩陣 $P = QQ^\top$ 是投影到 Q 各行生成空間的**正交投影**。投影 Px 是該子空間中離 x 最近的點，殘差 $x - Px$ 與 Q 的每一行都垂直。投影兩次不會再改變任何東西：$P^2 = P$。下面的 PCA 正是在尋找最好的這種 Q。

特徵分解

白話版。大多數向量被矩陣作用後會改變方向。特徵向量是那些只被拉長或縮短的特殊方向，特徵值則告訴你拉長或縮短了多少。

方陣 $A \in \mathbb{R}^{n \times n}$ 的**特徵向量**是滿足

$$Av = \lambda v$$

的非零向量 v，純量 λ 是對應的**特徵值**。若 v 是特徵向量，任何 $c \neq 0$ 的 cv 也是，所以通常把特徵向量正規化成單位長度。

假設 A 有 n 個線性獨立的特徵向量 $v^{(1)}, \dots, v^{(n)}$，特徵值為 $\lambda_1, \dots, \lambda_n$。把特徵向量當成 V 的各行，把特徵值放進 $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$，就有 $AV = V\Lambda$，因此

$$A = V \Lambda V^{-1}.$$

這就是**特徵分解**。它的意思是：先換到特徵向量座標（V^{-1}），每個座標各自縮放（Λ），再換回來（V）。不是每個矩陣都有特徵分解。剪切矩陣 $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ 只有一個獨立的特徵向量；旋轉 90° 的矩陣特徵值是 $\pm i$，是複數，因為沒有任何實數方向能保持不轉。

對稱的情形

對稱矩陣是好相處的情形，也是機器學習最常遇到的。**譜定理**指出，每個實對稱矩陣都可以寫成

$$A = Q \Lambda Q^\top,$$

其中 Q 是正交矩陣（各行是單範正交的特徵向量），Λ 是實數對角矩陣 [3], [4]。因為 $Q^{-1} = Q^{\top}$，完全不需要求反矩陣。幾何上，A 沿著方向 $q^{(i)}$（Q 的第 i 行）把空間拉伸 λ_i 倍。除了特徵值的排列順序，以及重複特徵值內部基底的選擇之外，這個分解是唯一的；慣例上把特徵值由大到小排列。

對對稱的 A，把**二次型** $f(x) = x^{\top}Ax$ 限制在單位向量上，其值被最大與最小特徵值夾住：

$$\lambda_{\min} \le x^{\top}Ax \le \lambda_{\max} \qquad \text{當} \|x\|_2 = 1,$$

最大值出現在最大特徵值的特徵向量，最小值出現在最小特徵值的特徵向量。比值 $x^{\top}Ax / x^{\top}x$ 稱為 **Rayleigh 商**。這個事實是後面 PCA 推導的引擎。

正定矩陣

特徵值的正負號可以為對稱矩陣分類：

名稱	特徵值	等價條件
正定（PD）	全部 $\lambda_i > 0$	對所有 $x \neq 0$，$x^{\top}Ax > 0$
半正定（PSD）	全部 $\lambda_i \ge 0$	對所有 x，$x^{\top}Ax \ge 0$
負定（半負定）	全部 $\lambda_i < 0$（或 ≤ 0）	不等式反向
不定	正負都有	$x^{\top}Ax$ 正負皆可能

每個共變異數矩陣與每個 Gram 矩陣 $X^{\top}X$ 都是半正定的，因為 $v^{\top}X^{\top}Xv = \|Xv\|_2^2 \ge 0$。高斯密度需要正定的共變異數矩陣才能求反。損失函數的 Hessian 在嚴格局部最小值處是正定的，在鞍點處則是不定的；ML 01C 會用這一點解釋最佳化為什麼可能很慢。正定矩陣有 **Cholesky 分解** $A = LL^{\top}$（L 為下三角矩陣），這是用它解方程組、以及從高斯分佈抽樣最快又穩定的方式 [4]。

```
import numpy as np

rng = np.random.default_rng(1)
M = rng.normal(size=(4, 4))
S = M @ M.T + 0.1 * np.eye(4)        # symmetric positive definite
lam, Q = np.linalg.eigh(S)           # ascending eigenvalues, orthonormal Q
assert np.allclose(Q @ np.diag(lam) @ Q.T, S)
assert np.allclose(Q.T @ Q, np.eye(4))
print("eigenvalues:", lam.round(3), "all positive:", bool((lam > 0).all()))

# The quadratic form x^T S x is bounded by the extreme eigenvalues
x = rng.normal(size=4); x /= np.linalg.norm(x)
print("Rayleigh quotient in range:", lam[0] <= x @ S @ x <= lam[-1])

# A non-symmetric matrix can have complex eigenvalues
R = np.array([[0.0, -1.0], [1.0, 0.0]])  # 90-degree rotation
print("rotation eigenvalues:", np.linalg.eigvals(R))
```

對稱矩陣請用 `np.linalg.eigh`：它比較快，回傳由小到大排列的實數特徵值與單範正交的特徵向量。`np.linalg.eig` 處理一般情形，可能回傳複數，如旋轉矩陣所示。

奇異值分解

白話版。任何形狀的矩陣，都依序做三件事：旋轉、沿著互相垂直的軸拉伸、再旋轉一次。SVD 告訴你這兩次旋轉與各個拉伸倍數。

特徵分解需要方陣，而且不一定存在。**奇異值分解**（singular value decomposition, SVD）則對每個實矩陣 $A \in \mathbb{R}^{m\times n}$ 都存在：

$$A = U\Sigma V^{\top}.$$

- $U \in \mathbb{R}^{m\times m}$ 是正交矩陣，各行是**左奇異向量**，屬於輸出空間的方向。
- $\Sigma \in \mathbb{R}^{m\times n}$ 是長方形對角矩陣，對角元素 $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ 是**奇異值**。
- $V \in \mathbb{R}^{n\times n}$ 是正交矩陣，各行是**右奇異向量**，屬於輸入空間的方向。

由右往左讀，$Ax = U\Sigma V^{\top}x$ 先把 x 表示成 V 座標（旋轉或鏡射），再把第 i 個座標乘上 σ_i，最後沿著 U 的方向擺放。圖 1 以 2×2 矩陣示範：單位圓變成一個橢圓，半軸長分別是 σ_1 與 σ_2。

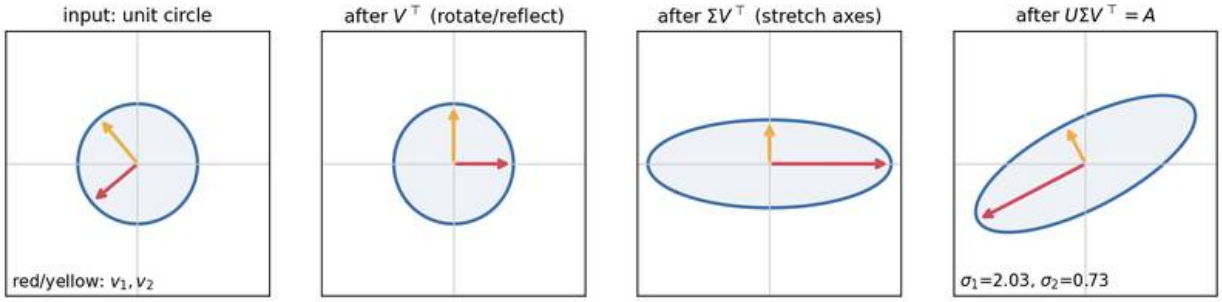


圖 *1 — SVD* 就是「旋轉、拉伸、旋轉」。右奇異向量 *v1*、*v2*（紅、黃）先被 V 的轉置轉到座標軸上，再依奇異值拉伸，最後由 U 旋轉。最終的橢圓就是單位圓經過 A 映射後的像。

SVD 與特徵分解關係密切。由於 $A^{\top}A = V\Sigma^{\top}\Sigma V^{\top}$ 且 $AA^{\top} = U\Sigma\Sigma^{\top}U^{\top}$，右奇異向量是 $A^{\top}A$ 的特徵向量，左奇異向量是 $AA^{\top}$ 的特徵向量，而 $\sigma_i = \sqrt{\lambda_i(A^{\top}A)}$。對於對稱半正定矩陣，SVD 與特徵分解一致。

非零奇異值的個數等於秩 r。只保留它們，就得到**精簡**（thin）SVD：$A = U_r\Sigma_r V_r^{\top} = \sum_{i=1}^{r}\sigma_i u^{(i)}v^{(i)\top}$，也就是 r 個秩一矩陣的總和——又是矩陣乘法的外積觀點。

最佳低秩近似

把這個總和截斷在第 k 項，得到 $A_k = \sum_{i=1}^{k}\sigma_i u^{(i)}v^{(i)\top}$。**Eckart–Young 定理** [6] 指出，A_k 是 A 在 Frobenius 範數下的最佳秩 k 近似：

$$\min_{\operatorname{rank}(B)\leq k}\|A - B\|_F^2 = \|A - A_k\|_F^2 = \sum_{i>k}\sigma_i^2.$$

同一個 A_k 在譜範數下也是最佳的，誤差為 $\|A - A_k\|_2 = \sigma_{k+1}$ [4]。換句話說：丟掉最小的那些奇異值，損失的能量剛好就是丟掉的部分。儲存 A_k 只需要 $k(m+n+1)$ 個數字，而不是 mn 個。

圖 2 把這套方法用在一張 256 × 256 的灰階照片上。自然影像的奇異值衰減得很快，所以秩 50 的近似已經保留了主要結構。低秩時的條紋狀瑕疵，來自每個秩一項都是一個行樣式與一個列樣式的外積。

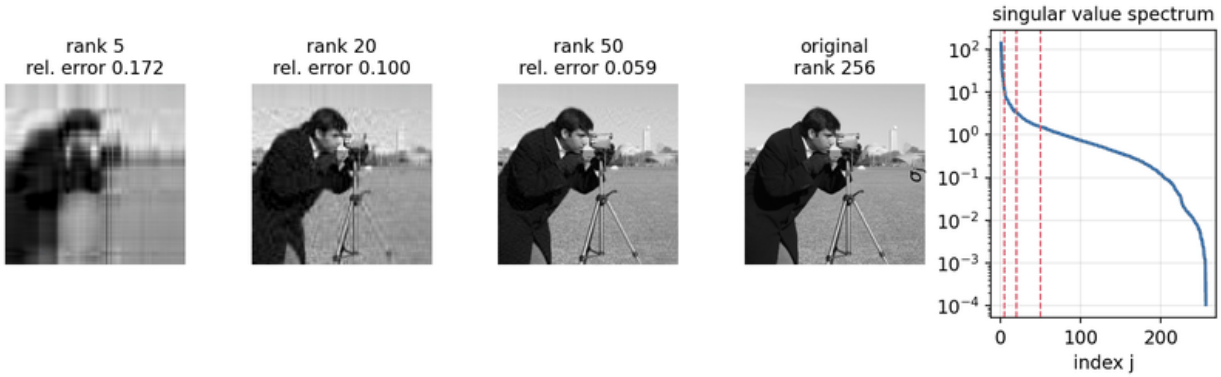


圖 *2 — scikit-image* 中 *256×256*「*camera*」影像的秩 *k* 近似，標題標示相對 *Frobenius* 誤差。右圖：對數尺度下的奇異值，虛線標出 $k = 5$、20、50。每個近似的誤差由該虛線右邊的奇異值決定。

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.normal(size=(12, 8))
U, s, Vt = np.linalg.svd(A, full_matrices=False)
assert np.allclose(U @ np.diag(s) @ Vt, A)
k = 3
Ak = (U[:, :k] * s[:k]) @ Vt[:k]
err_F = np.linalg.norm(A - Ak, "fro")**2
err_2 = np.linalg.norm(A - Ak, 2)
assert np.isclose(err_F, (s[k:]**2).sum()) and np.isclose(err_2, s[k])
print("rank-k squared Frobenius error:", round(err_F, 3))
print("rank-k spectral error = sigma_{k+1}:", round(err_2, 3), round(s[k], 3))
# Eigenvalues of A^T A are squared singular values of A
assert np.allclose(np.sort(np.linalg.eigvalsh(A.T @ A))[::-1], s**2)
print("storage: full", A.size, "numbers; rank-3 factors", k * (12 + 8 + 1))
```

`full_matrices=False` 回傳精簡 SVD，這幾乎總是你要的。`(U[:, :k] * s[:k])` 用廣播縮放 U 的各行，不必建立對角矩陣。兩條 Eckart–Young 等式都在機器精確度內成立。

一個提醒：奇異值最大的方向承載最多能量，卻不一定承載最有用的資訊。對分類而言，變異數小的方向可能正是把類別分開的那個方向。PR 05 正是為此比較了 PCA 與判別式方法。

Moore–Penrose 偽反矩陣

白話版。矩陣沒有反矩陣時，偽反矩陣給出次佳的答案：盡可能貼近資料的解，而在同樣好的解之中，選最小的那一個。

只有滿秩方陣才有反矩陣。對任意 $A \in \mathbb{R}^{m\times n}$，**Moore–Penrose 偽反矩陣**（pseudo-inverse）$A^{+} \in \mathbb{R}^{n\times m}$ 是唯一滿足以下四個 Penrose 條件的矩陣 [7]：

$$AA^{+}A = A, \quad A^{+}AA^{+} = A^{+}, \quad (AA^{+})^{\top} = AA^{+}, \quad (A^{+}A)^{\top} = A^{+}A.$$

用 SVD 計算最容易。若 $A = U\Sigma V^{\top}$，則

$$A^{+} = V\Sigma^{+}U^{\top},$$

其中 $\Sigma^{+} \in \mathbb{R}^{n\times m}$ 是把 Σ 轉置、並把每個非零奇異值 σ_i 換成 $1/\sigma_i$（零維持為零）。Goodfellow 等人給出等價的極限形式 $A^{+} = \lim_{\alpha\searrow 0}(A^{\top}A + \alpha I)^{-1}A^{\top}$ [1]，這已經暗示了嶺迴歸。

在秩的表格的每一種情況下，偽反矩陣都給出正確答案：

- **高瘦、行滿秩**（$m > n$）。$Ax = b$ 通常沒有精確解。此時 $x = A^{+}b = (A^{\top}A)^{-1}A^{\top}b$ 是**最小平方**解：它使 $\|Ax - b\|_2^2$ 最小。
- **矮胖、列滿秩**（$m < n$）。解有無限多個。此時 $x = A^{+}b = A^{\top}(AA^{\top})^{-1}b$ 是 $\|x\|_2$ 最小的解，稱為**最小範數**解。其他任何解都多加了一個零空間向量，只會讓範數變大。
- **秩不足**。$A^{+}b$ 同時兼具兩者：在所有最小平方解之中，範數最小的那一個。

最小範數性質是通往一個現代問題的小窗口。對欠定的最小平方問題從零開始做梯度下降，會收斂到最小範數解，因為每個梯度 $A^{\top}(Ax - b)$ 都落在 A 的列空間中。過參數化的網路不是線性的，但思考「在眾多完美擬合中，訓練演算法會選哪一個」時，這個簡單情形通常是起點。

最小平方、正規方程式與條件數

對迴歸設計矩陣 $X \in \mathbb{R}^{n\times d}$ 與目標 $y \in \mathbb{R}^{n}$，最小平方法最小化 $\|Xw - y\|_2^2$。令梯度為零，得到**正規方程式**

$$X^{\top}Xw = X^{\top}y.$$

它們是對的，但在數值上有風險。線性問題的敏感度用**條件數** $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$ 量測。粗略地說，資料中的相對誤差在解中最多可能被放大 κ 倍 [5]。形成 $X^{\top}X$ 會把條件數平方：$\kappa(X^{\top}X) = \kappa(X)^2$，於是輕微病態的問題變成嚴重病態。程式庫的最小平方求解器（`np.linalg.lstsq`、`torch.linalg.lstsq`）直接對 X 做 QR 或 SVD 分解，避免了這個平方 [4]。

當兩個特徵幾乎相同時，$\sigma_{\min}$ 非常小，即使預測沒問題，個別係數也會極度敏感。**嶺迴歸**（ridge regression）[8] 加上懲罰項 $\lambda\|w\|_2^2$，$\lambda > 0$：

$$w_{\text{ridge}} = (X^{\top}X + \lambda I)^{-1}X^{\top}y = \sum_{i} \frac{\sigma_i}{\sigma_i^2 + \lambda}\,(u^{(i)\top}y)\,v^{(i)}.$$

右邊的 SVD 形式說明了嶺迴歸在做什麼。一般最小平方使用係數 $1/\sigma_i$，在 σ_i 很小時爆炸；嶺迴歸使用 $\sigma_i/(\sigma_i^2 + \lambda)$，上限是 $1/(2\sqrt{\lambda})$。大 σ 的方向幾乎不受影響，小 σ 的方向被壓抑，代價是引入偏差。在神經網路中，同一個懲罰項稱為**權重衰減**（weight decay）。

```
import numpy as np

rng = np.random.default_rng(0)
```

```
# Over-determined: more equations than unknowns -> least squares
x = np.linspace(-1, 1, 40)
X = np.c_[np.ones_like(x), x]
y = 2 + 3*x + rng.normal(0, .05, len(x))
w, residuals, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print("intercept and slope:", w.round(3), "rank:", rank)
assert np.allclose(w, np.linalg.pinv(X) @ y)

# Under-determined: fewer equations than unknowns -> minimum-norm solution
A = rng.normal(size=(2, 5)); b = rng.normal(size=2)
w_min = np.linalg.pinv(A) @ b
null = np.linalg.svd(A)[2][-1]             # a direction with A @ null = 0
w_other = w_min + 0.7 * null
print("both solve Aw=b:", np.allclose(A @ w_min, b), np.allclose(A @ w_other, b))
print("norms:", np.linalg.norm(w_min).round(3), "<", np.linalg.norm(w_other).round(3))

# Near-collinear features: condition number and ridge
z = rng.normal(size=200)
F = np.c_[z, z + 1e-4 * rng.normal(size=200)]
t = z + 0.01 * rng.normal(size=200)
print("cond(F) = %.1e, cond(F^T F) = %.1e" % (np.linalg.cond(F), np.linalg.cond(F.T @ F)))
w_ls = np.linalg.lstsq(F, t, rcond=None)[0]
lam = 1e-2
w_ridge = np.linalg.solve(F.T @ F + lam * np.eye(2), F.T @ t)
print("least squares:", w_ls.round(2), " ridge:", w_ridge.round(3))
```

直線擬合得到截距 $\approx$ 2.00、斜率 $\approx$ 3.01，而 `pinv` 與 `lstsq` 結果一致。欠定系統有兩個精確解，偽反矩陣給的那一個範數較小（0.885 對 1.128）。在共線的例子中，$\kappa(F) \approx 2 \times 10^4$，而 $\kappa(F^\top F) \approx 4 \times 10^8$，正好是它的平方。最小平方回傳誇張的係數 $(-2.63, 3.63)$，但兩者相加仍約為 1，所以預測沒有問題；嶺迴歸則回傳穩定的 $(0.5, 0.5)$。第一個例子中的截距行讓直線可以有非零偏移。在實際流程中，置中與縮放只能在訓練集上擬合，再原封不動套用到驗證與測試資料。

跡運算子

白話版。跡就是把對角線加起來。它看起來簡單到不值一提，卻能把麻煩的總和改寫成矩陣運算式，並讓你在式子裡搬動矩陣。

方陣的**跡**（trace）是

$$\operatorname{Tr}(A) = \sum_i A_{ii}.$$

它有這些好用的性質：

- **Frobenius 範數**。$\|A\|_F = \sqrt{\operatorname{Tr}(AA^\top)} = \sqrt{\operatorname{Tr}(A^\top A)}$。
- **轉置**。$\operatorname{Tr}(A) = \operatorname{Tr}(A^\top)$。
- **循環性質**。只要乘積有定義，$\operatorname{Tr}(ABC) = \operatorname{Tr}(CAB) = \operatorname{Tr}(BCA)$，即使矩陣不是方陣、中間乘積形狀不同也成立。特別地，對 $A \in \mathbb{R}^{m\times n}$、$B \in \mathbb{R}^{n\times m}$，$\operatorname{Tr}(AB) = \operatorname{Tr}(BA)$。可以循環，但不能任意重排：一般而言 $\operatorname{Tr}(ABC) \neq \operatorname{Tr}(ACB)$。

- **純量**。純量等於自己的跡，$a = \mathrm{Tr}(a)$。這能把二次型變成跡：$x^\top Ax = \mathrm{Tr}(x^\top Ax) = \mathrm{Tr}(Axx^\top)$。
- **特徵值**。$\mathrm{Tr}(A) = \sum_i \lambda_i$。對共變異數矩陣而言，這就是總變異數，也就是沿每個主方向的變異數總和。

許多期望值就是用這個純量技巧算出來的。若隨機向量 x 的平均為零、共變異數為 Σ，則 $\mathbb{E}[x^\top Ax] = \mathbb{E}[\mathrm{Tr}(Axx^\top)] = \mathrm{Tr}(A\Sigma)$，因為跡與期望值都是線性的。

行列式

白話版。方陣的行列式說明它把面積（二維）或體積（更高維）放大了多少倍。若行列式為零，矩陣就把空間壓扁了，無法復原。

方陣的**行列式** $\det(A)$ 是一個數字，具有以下性質：

- $\det(A) = \prod_i \lambda_i$，即特徵值的乘積。
- $|\det(A)|$ 是 A 縮放體積的倍數：單位立方體被映射成體積為 $|\det(A)|$ 的平行多面體。
- 正負號記錄方向（定向）。行列式為負表示 A 包含鏡射。
- $\det(A) = 0$ 恰好表示 A 是奇異的：某個方向被壓成零，資訊遺失。
- $\det(AB) = \det(A)\det(B)$、$\det(A^\top) = \det(A)$、$\det(A^{-1}) = 1/\det(A)$。

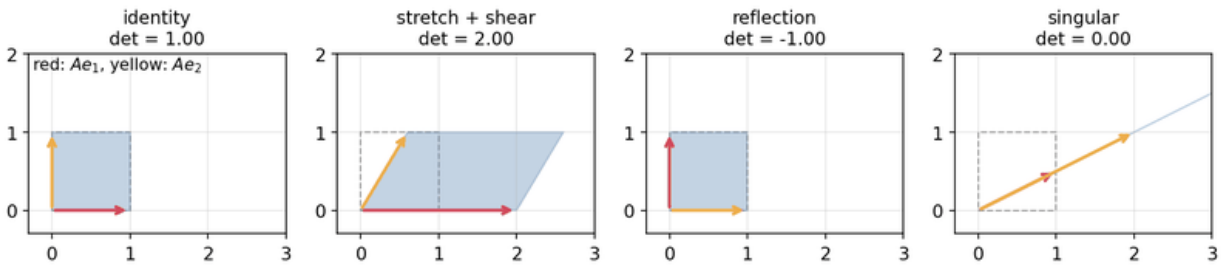


圖 3 — 行列式就是帶正負號的面積。虛線：單位正方形；陰影：它的像，由 *Ae1*（紅）與 *Ae2*（黃）張成。剪切不改變面積；鏡射讓正負號翻轉；奇異矩陣把正方形壓成一條線（$det = 0$）。

行列式在機器學習中主要出現在兩個地方。多變量高斯密度中有 $\det(\Sigma)^{-1/2}$，是共變異數的體積正規化；ML 01B 會用到它。正規化流（normalizing flow，見ML 06）需要 Jacobian 矩陣 J 的 $\log|\det J|$，來記錄可逆網路如何拉伸機率質量。這兩種情況都需要對數，絕不要直接算原始值：一個對角元素皆為 10^{-3} 的 500×500 對角矩陣，行列式是 10^{-1500}，在 float64 中會下溢成零。

```
import numpy as np

rng = np.random.default_rng(2)
A, B, C = rng.normal(size=(3, 4)), rng.normal(size=(4, 5)), rng.normal(size=(5, 3))
print("cyclic trace:", np.isclose(np.trace(A @ B @ C), np.trace(C @ A @ B)),
      np.isclose(np.trace(A @ B @ C), np.trace(B @ C @ A)))

S = rng.normal(size=(4, 4))
lam = np.linalg.eigvals(S)
print("trace = sum of eigenvalues:", np.isclose(np.trace(S), lam.sum().real))
```

```
print("det = product of eigenvalues:", np.isclose(np.linalg.det(S), np.prod(lam).real))

# log-determinant of a covariance without overflow/underflow
Sigma = np.diag(np.full(500, 1e-3))
print("det underflows:", np.linalg.det(Sigma))
sign, logdet = np.linalg.slogdet(Sigma)
print("slogdet:", sign, round(logdet, 2))
L = np.linalg.cholesky(Sigma)
print("via Cholesky:", round(2 * np.log(np.diag(L)).sum(), 2))
```

直接計算行列式印出 `0.0`，而 `slogdet` 與 Cholesky 兩種做法都得到 $\log\det = 500\log 10^{-3} \approx -3453.88$。對正定矩陣，若 $A = LL^\top$，則 $\det(A) = \prod_i L_{ii}^2$，所以 $\log\det A = 2\sum_i \log L_{ii}$。

實例：推導主成分分析

白話版。PCA 要找出資料點雲最貼近的那個平坦子空間。把資料投影上去，每個點就能壓縮成少數幾個數字，同時損失最少。

PCA 可追溯到 Pearson 的「最貼近點集的直線與平面」[9] 與 Hotelling 對「主成分」的分析 [10]；Jolliffe 與 Cadima 提供了現代的綜述 [11]。我們依照 [1] 的方式，只用上面介紹的工具來推導。

設定。有 n 個點 $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^d$，想讓每個點只用 $l < d$ 個數字儲存。先從每個點減去平均 $\mu = \frac{1}{n}\sum_i x^{(i)}$，因此可以假設資料已置中。把置中後的點當作 $X \in \mathbb{R}^{n\times d}$ 的各列。

步驟 1：選定編碼的形式。使用線性**解碼器** $g(c) = Dc$，$D \in \mathbb{R}^{d\times l}$。為了讓問題良好設定，要求 D 的各行單範正交：$D^\top D = I_l$。（否則可以無止境地縮小 c、放大 D。）

步驟 2：固定解碼器，找最佳編碼。對給定的 x，選擇 c 使重建誤差最小：

$$c^\ast = \arg\min_c \|x - Dc\|_2^2 = \arg\min_c \left(-2x^\top Dc + c^\top D^\top Dc \right) = \arg\min_c \left(-2x^\top Dc + c^\top c \right).$$

我們展開平方、丟掉與 c 無關的 $x^\top x$，並使用 $D^\top D = I$。對 c 的梯度是 $-2D^\top x + 2c$，令其為零就得到**編碼器**

$$f(x) = D^\top x.$$

所以編碼就是一次矩陣–向量乘法，而重建 $r(x) = g(f(x)) = DD^\top x$ 正是 x 到 D 生成空間的正交投影。

步驟 3：找最佳解碼器。現在選擇 D，使所有點的總重建誤差最小：

$$D^\ast = \arg\min_D \|X - XDD^\top\|_F^2 \quad \text{subject to } D^\top D = I_l.$$

先看 $l = 1$，此時 D 是單一單位向量 d。把 Frobenius 範數寫成跡並展開：

$$\begin{aligned}\|X - Xdd^\top\|_F^2 &= \operatorname{Tr}\left((X - Xdd^\top)^\top (X - Xdd^\top)\right) \\ &= \operatorname{Tr}(X^\top X) - 2\operatorname{Tr}(X^\top Xdd^\top) + \operatorname{Tr}(dd^\top X^\top Xdd^\top) \\ &= \operatorname{Tr}(X^\top X) - 2\,d^\top X^\top Xd + d^\top X^\top Xd\,(d^\top d) \\ &= \operatorname{Tr}(X^\top X) - d^\top X^\top Xd.\end{aligned}$$

第二項與第三項用循環性質把 $d^\top$ 移到最前面，最後一行用 $d^\top d = 1$ 化簡。第一項與 d 無關，所以最小化誤差等同於

$$d^* = \arg\max_{d}\; d^\top X^\top Xd \quad \text{subject to } d^\top d = 1.$$

這是對稱半正定矩陣 $X^\top X$ 的 Rayleigh 商。由特徵分解一節可知，它在 $X^\top X$ **最大特徵值對應的特徵向量**處達到最大。

步驟 4：多個主成分。對一般的 l 重複這個論證（例如用歸納法，要求每個新方向與先前的方向正交），答案是：D 的各行是 $X^\top X$ 前 l 大特徵值對應的特徵向量 [1]。最小誤差等於被捨棄的特徵值總和。

步驟 5：變異數觀點與 SVD 的連結。把 $X^\top X$ 除以 n，得到樣本共變異數 $S = \frac{1}{n}X^\top X$（有些教科書用 $n-1$）。$d^\top Sd$ 是投影後資料 Xd 的變異數。所以「最小化重建誤差」與「最大化保留的變異數」選出的是同樣的方向——Pearson 與 Hotelling 的觀點一致。最後，若置中資料矩陣的精簡 SVD 為 $X = U\Sigma V^\top$，則 $X^\top X = V\Sigma^2 V^\top$：主方向就是右奇異向量 V，特徵值是 σ_i^2，編碼是 $XV_l = U_l\Sigma_l$。實務上 PCA 是用 X 的 SVD 計算，而不是先形成共變異數矩陣，理由與最小平方法中的條件數相同。

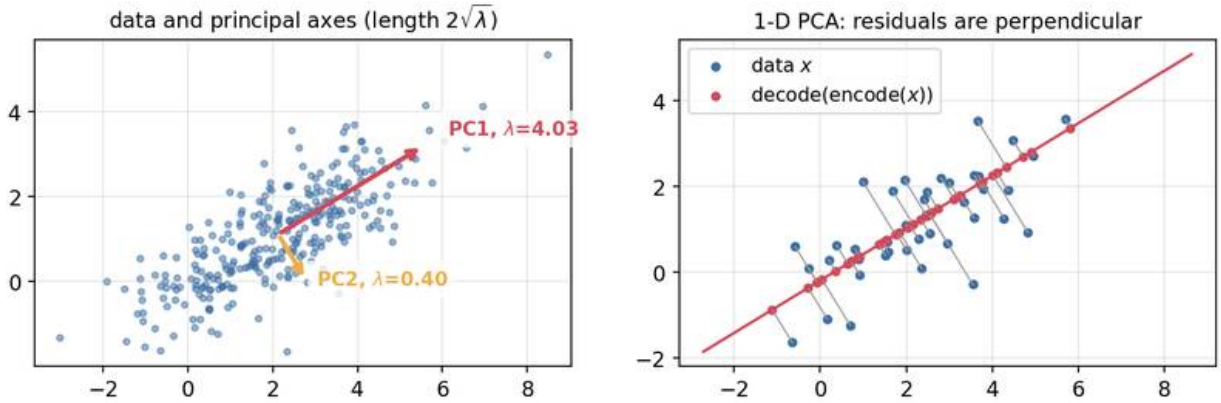


圖 4 — 對 300 個相關二維資料點做 PCA。左：由樣本共變異數特徵分解得到的主軸，長度畫為 $2\sqrt{\lambda}$。右：一維 PCA 的重建 $decode(encode(x))$；每條灰色殘差都與紅線垂直，因為重建就是正交投影。

```
import numpy as np

rng = np.random.default_rng(0)
n, d, l = 500, 5, 2
Z = rng.normal(size=(n, l))
X = Z @ rng.normal(size=(l, d)) + 0.1 * rng.normal(size=(n, d)) + 3.0
```

```
mu = X.mean(axis=0)
Xc = X - mu                                  # centre first
S = Xc.T @ Xc / n                            # sample covariance (d x d)
lam, Q = np.linalg.eigh(S)
lam, Q = lam[::-1], Q[:, ::-1]               # descending
D = Q[:, :l]                                 # decoder matrix, orthonormal columns

codes = Xc @ D                               # encode: f(x) = D^T (x - mu)
X_hat = codes @ D.T + mu                     # decode: g(c) = D c + mu
mse = np.mean(np.sum((X - X_hat)**2, axis=1))
print("explained variance ratio:", (lam[:l].sum() / lam.sum()).round(4))
print("reconstruction MSE:", mse.round(5), " sum of discarded eigenvalues:",
 ↪  lam[l:].sum().round(5))
assert np.isclose(mse, lam[l:].sum())

# Same subspace from the SVD of the centred data matrix
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
assert np.allclose(s[:l]**2 / n, lam[:l])
assert np.allclose(np.abs(Vt[:l] @ D), np.eye(l), atol=1e-8)
print("SVD and eigendecomposition agree")
```

這份合成資料是五維的點，實際上落在一個二維平面附近，再加上小雜訊。兩個主成分解釋了 99.36% 的變異數，而平均平方重建誤差剛好等於被捨棄的三個特徵值總和，與步驟 4 的預測一致。SVD 的做法找到同樣的方向（差一個正負號，特徵向量的正負號本來就是任意的）。

三個實務提醒。一定要先置中，否則第一個「主成分」只會指向平均值。若特徵的單位無法比較，要先標準化，因為 PCA 不具尺度不變性。平均與方向都只能在訓練資料上擬合。PCA 是自編碼器的線性祖先；ML 05A 會發展它的機率版本與非線性後繼者。

你會遇到的矩陣微積分

白話版。訓練就是沿著梯度走，而機器學習中的大多數梯度，都是少數幾條矩陣公式反覆使用。

對向量 $x \in \mathbb{R}^n$ 的純量函數 $f(x)$，**梯度** $\nabla_x f \in \mathbb{R}^n$ 收集所有偏導數 $\partial f / \partial x_i$。對向量函數 $f : \mathbb{R}^n \to \mathbb{R}^m$，**Jacobian 矩陣** $J \in \mathbb{R}^{m \times n}$ 的元素為 $J_{ij} = \partial f_i / \partial x_j$。最常用的規則如下：

函數	梯度	備註
$a^\top x$	a	線性函數
$x^\top A x$	$(A + A^\top)x$	A 對稱時為 $2Ax$
$\tfrac{1}{2}\lVert Xw - y \rVert_2^2$	$X^\top(Xw - y)$	最小平方損失
$\lVert x \rVert_2^2$	$2x$	權重衰減
$\operatorname{Tr}(AB)$ 對 A	$B^\top$	搭配上面的跡技巧

每一條都可以把運算式寫成對索引的總和、逐項微分來驗證 [2]。向量函數的**連鎖律**是把 Jacobian 相乘：若 $y = g(x)$，而 $L = f(y)$ 是純量，則 $\nabla_x L = J_g^\top \nabla_y L$。反向傳播逐層套用這條規則，把轉置的局部 Jacobian 乘上傳進來的梯度，而不必真的建出完整的 Jacobian（那會大得驚人）。ML 02 會詳細建構這個過程，ML 01C 則加入 Hessian，也就是二階導數矩陣。

令最小平方梯度為零，就回到前一節的正規方程式。這個損失的 Hessian 是 $X^\top X$，為半正定，所以損失是凸函數，每個駐點都是全域最小值。用有限差分快速檢查公式：

```
import numpy as np

rng = np.random.default_rng(3)
X, y = rng.normal(size=(30, 4)), rng.normal(size=30)
w = rng.normal(size=4)
L = lambda w: 0.5 * np.sum((X @ w - y)**2)
grad = X.T @ (X @ w - y)                    # analytic gradient
eps = 1e-6
num = np.array([(L(w + eps*e) - L(w - eps*e)) / (2*eps) for e in np.eye(4)])
print("max gradient mismatch:", np.abs(grad - num).max())
# Setting the gradient to zero gives the normal equations
w_star = np.linalg.lstsq(X, y, rcond=None)[0]
print("gradient at solution ~ 0:", np.abs(X.T @ (X @ w_star - y)).max() < 1e-10)
```

解析梯度與數值梯度相差約 10^{-8}，正是有限差分捨入誤差預期的大小。

現代觀點

本篇的定義已有百年歷史，但它們在機器學習中的用法仍不斷改變。以下四條研究方向值得認識。

隨機化數值線性代數。大型矩陣的精確 SVD 很昂貴，大約需要 $O(mn\,\min(m,n))$ 次運算。Halko、Martinsson 與 Tropp 在 SIAM Review 的綜述 [12] 讓一個簡單的想法成為標準做法：用一個細長的隨機矩陣 Ω 乘上 A，「草描」出它的值域；把結果正交化得到基底 Q；再對小矩陣 $Q^\top A$ 做精確 SVD。加上少量過取樣與幾次冪迭代，對奇異值會衰減的矩陣就能得到準確的結果。Martinsson 與 Tropp 在 *Acta Numerica* 的綜述 [13] 涵蓋理論與大量演算法：範數估計、隨機嵌入、迴歸、低秩近似、半正定矩陣的 Nyström 近似、串流演算法，以及機器學習中的核矩陣。Murray 等人 [14]（LAPACK Working Note 299，2023）從軟體角度切入：他們指出 RandNLA 方法已進入 MATLAB、NAG Library、NVIDIA 的 cuSOLVER 與 scikit-learn，並主張建立標準的「RandBLAS」與「RandLAPACK」程式庫。scikit-learn 的隨機化 PCA 求解器就是直接的後代。核心演算法只要幾行：

```
import numpy as np

def randomized_svd(A, k, oversample=10, power_iters=2, seed=0):
    rng = np.random.default_rng(seed)
    Omega = rng.normal(size=(A.shape[1], k + oversample))  # random test matrix
    Y = A @ Omega                                          # sample the range of A
    for _ in range(power_iters):                           # sharpen a slow spectrum
        Y = A @ (A.T @ Y)
    Q, _ = np.linalg.qr(Y)                                 # orthonormal basis
    B = Q.T @ A                                            # small (k+p) x n matrix
    Ub, s, Vt = np.linalg.svd(B, full_matrices=False)
```

```
    return (Q @ Ub)[:, :k], s[:k], Vt[:k]

rng = np.random.default_rng(1)
m, n, r = 2000, 500, 20
A = rng.normal(size=(m, r)) @ rng.normal(size=(r, n)) + 0.01 * rng.normal(size=(m, n))
U, s, Vt = randomized_svd(A, k=r)
s_exact = np.linalg.svd(A, compute_uv=False)[:r]
print("max relative error in top singular values:", np.max(np.abs(s - s_exact) / s_exact))
rel = np.linalg.norm(A - (U * s) @ Vt) / np.linalg.norm(A)
print("relative reconstruction error:", round(rel, 5))
```

對這個近似秩 20 的矩陣，前 20 個奇異值與精確值的相對誤差約 10^{-15}，而秩 20 重建的誤差（約 0.2%）由加入的雜訊決定。冪迭代是奇異值衰減緩慢時仍能成功的關鍵；若矩陣的奇異值完全平坦，任何秩 k 的方法——隨機或非隨機——都做不好。

大型模型中的低秩結構。矩陣乘法的外積觀點，已經成為調整大型網路的實用工具。LoRA [15] 凍結預訓練權重 $W_0 \in \mathbb{R}^{d\times k}$，只學習更新量 $\Delta W = BA$，其中 $B \in \mathbb{R}^{d\times r}$、$A \in \mathbb{R}^{r\times k}$，且 $r \ll \min(d,k)$。可訓練的數字只有 $r(d+k)$ 個，而不是 dk 個；以 $d = k = 4096$、$r = 8$ 為例，每個矩陣是 65,536 個，而不是約 1,680 萬個。B 初始化為零，所以訓練恰好從預訓練模型出發。GaLore [16]（ICML 2024）則讓所有權重都可訓練，但把梯度投影到低秩子空間，以縮小最佳化器的記憶體用量；作者認為像 LoRA 那樣把權重更新本身限制在低秩子空間，可能不如全秩訓練。兩者都依賴 Eckart–Young 的直覺：矩陣中有用的內容大多集中在少數幾個奇異方向。這在特定任務上是否成立，是實證問題，不是定理。

控制譜範數。網路對輸入擾動的敏感度，上限是各層譜範數的乘積（在 ReLU 這類 1-Lipschitz 激勵函數下）。譜正規化（spectral normalization）[17] 把每個權重矩陣除以最大奇異值的估計值，而這個估計在每個訓練步驟只需一次冪迭代，成本很低。它原本是為了穩定 GAN 的判別器而提出（見ML 06），凡是需要 Lipschitz 上界的地方都能看到同樣的想法。

超越矩陣。卷積核與影片特徵天生就是三軸以上的張量。Kolda 與 Bader 在 SIAM Review 的綜述 [18] 是張量分解的標準入門，涵蓋 CP 分解（秩一張量的總和，推廣了外積觀點）與 Tucker 分解（一個小的核心張量沿每個模態乘上一個矩陣，推廣了 SVD）。與矩陣不同，張量的秩一般而言很難計算，最佳低秩近似也不一定存在，Eckart–Young 那幅清楚的圖像無法直接沿用。這些分解被用來壓縮網路層，以及分析多向資料。

開放問題。為什麼訓練後的權重矩陣常有快速衰減的奇異值？這和泛化有什麼關係？調整用的秩可以多低才不損失品質？這取決於任務還是模型？隨機化演算法要如何與低精度硬體結合——那裡的捨入誤差遠大於 float64？這些問題都還沒有定論，而本篇的工具正是閱讀相關論文所需要的。若想找把線性代數與深度學習連起來的完整教科書，可參考 [1]、[2] 與 [19]。

重點整理

- 為每個軸命名。大多數無聲的錯誤都是形狀或廣播錯誤，`assert` 能抓到它們。
- 矩陣乘法有四種讀法（元素、行、列、外積總和）；選擇讓問題最清楚的那一種。
- 秩與零空間決定 $Ax = b$ 有零個、一個或無限多個解。程式中不要形成 A^{-1}，改用求解器。
- 對稱矩陣有實數特徵值與單範正交的特徵向量；特徵值的正負號決定二次型是碗、鞍還是倒扣的碗。

- 每個矩陣都有 SVD：旋轉、拉伸、旋轉。截斷 SVD 給出最佳低秩近似，誤差等於被丟掉的奇異值。
- 偽反矩陣給出最小平方解與最小範數解。小奇異值會放大雜訊；條件數量測放大多少，嶺正則化則能抑制它。
- 跡與行列式分別是特徵值的和與積。推導時用跡的循環性質，程式中用對數行列式。
- PCA 是投影到共變異數前幾大特徵向量上的投影，等價於置中資料的前幾個右奇異向量。最小化重建誤差與最大化變異數得到相同答案。

練習

1. **形狀與偏差**。一個批次形狀為 32×64，權重矩陣形狀為 64×10。偏差應該是什麼形狀？若改用形狀 $(32, 1)$ 的偏差，會出什麼問題？

   提示

   形狀 $(10,)$ 的偏差會沿 32 列廣播：每個輸出單元一個偏移量，由所有樣本共用。$(32, 1)$ 的偏差則沿 10 行廣播，變成每個樣本一個偏移量。程式照樣能跑，但「偏差」現在取決於樣本在批次中的位置，這沒有意義，而且會隨批次大小改變。

2. **投影**。設 $Q \in \mathbb{R}^{n \times k}$ 的各行單範正交，$P = QQ^\top$。證明 $P^2 = P$、$P^\top = P$，且 $x - Px$ 與 Q 的每一行正交。P 的特徵值是什麼？

   提示

   $P^2 = Q(Q^\top Q)Q^\top = QIQ^\top = P$。正交性：計算 $Q^\top(x - Px) = Q^\top x - Q^\top QQ^\top x = 0$。由 $P^2 = P$，任何特徵值滿足 $\lambda^2 = \lambda$，所以 $\lambda \in \{0, 1\}$：特徵值 1 的重數為 k（子空間本身），0 的重數為 $n - k$（它的正交補空間）。因此 $\operatorname{Tr}(P) = k$。

3. **手算 SVD**。透過計算 $A^\top A$ 的特徵值，求 $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$ 的奇異值。驗證 $\|A\|_F^2 = \sigma_1^2 + \sigma_2^2$ 以及 $|\det A| = \sigma_1 \sigma_2$。

   提示

   $A^\top A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$ 的特徵值是 45 與 5，所以 $\sigma_1 = 3\sqrt{5}$、$\sigma_2 = \sqrt{5}$。於是 $\sigma_1^2 + \sigma_2^2 = 50 = 9 + 16 + 25$，且 $\sigma_1 \sigma_2 = 15 = |\det A| = |3 \cdot 5 - 0 \cdot 4|$。一般而言，方陣的 $|\det A| = \prod_i \sigma_i$，因為 $|\det U| = |\det V| = 1$。

4. **條件數與嶺迴歸**。從最小平方的程式碼出發，把第二個特徵換成第一個特徵的完全複本。`np.linalg.lstsq` 會回傳什麼？為什麼它是最小範數解？接著推導嶺迴歸解的 SVD 形式，並證明每個係數 $\sigma/(\sigma^2 + \lambda)$ 最多是 $1/(2\sqrt{\lambda})$。

   提示

   兩行完全相同時秩為 1，解集合是一條直線 $w_1 + w_2 = c$。`lstsq` 使用 SVD，回傳偽反矩陣解，也就是平分權重 $w_1 = w_2 = c/2$，因為直線上離原點最近的就是這一點。至於上界，最大化 $g(\sigma) = \sigma/(\sigma^2 + \lambda)$：$g'(\sigma) = 0$ 發生在 $\sigma = \sqrt{\lambda}$，此時 $g = 1/(2\sqrt{\lambda})$。

5. **PCA 的誤差**。利用本篇的推導，證明 l 個主成分的 PCA 總重建誤差等於 $\sum_{i>l} \lambda_i(X^\top X)$，而保留的變異數比例是 $\sum_{i\le l}\lambda_i / \sum_i \lambda_i$。接著對 `sklearn.datasets.load_digits()` 做 PCA，找出保留 90% 變異數所需的最小 l。

   提示

   寫成 $\|X - XDD^\top\|_F^2 = \mathrm{Tr}(X^\top X) - \mathrm{Tr}(D^\top X^\top X D)$（使用 $D^\top D = I$ 與循環性質）。當 D 是前 l 個特徵向量時，第二個跡等於 $\sum_{i\le l}\lambda_i$，而 $\mathrm{Tr}(X^\top X) = \sum_i \lambda_i$。手寫數字資料：把 1797×64 的矩陣置中，計算奇異值，再對 σ_i^2 取累積和。請回報你自己算出的數字，不要相信別人引用的數字。

6. **行列式與體積**。某個 2×2 矩陣把單位正方形映射成頂點為 0、$(2,1)$、$(1,3)$、$(3,4)$ 的平行四邊形。寫出這樣的矩陣，計算它的行列式，並說明為什麼正規化流必須記錄每一層的 $\log|\det J|$。

   提示

   把兩條邊向量放進各行：$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$，$\det A = 5$，所以面積放大 5 倍。若一個映射把體積放大 5 倍，機率密度就必須縮小同樣倍數，總機率才能維持為 1。疊加多層時這些倍數相乘，取對數後變成相加，這就是正規化流逐層加總對數行列式的原因。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 2 “Linear Algebra”. Book
2. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book
3. G. Strang, *Introduction to Linear Algebra*, 6th ed., Wellesley-Cambridge Press, 2023. Book site
4. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. DOI
5. L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*, SIAM, 1997. DOI
6. C. Eckart and G. Young, “The Approximation of One Matrix by Another of Lower Rank,” *Psychometrika*, vol. 1, pp. 211–218, 1936. DOI
7. R. Penrose, “A Generalized Inverse for Matrices,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 51, pp. 406–413, 1955. DOI
8. A. E. Hoerl and R. W. Kennard, “Ridge Regression: Biased Estimation for Nonorthogonal Problems,” *Technometrics*, vol. 12, pp. 55–67, 1970. DOI
9. K. Pearson, “On Lines and Planes of Closest Fit to Systems of Points in Space,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, pp. 559–572, 1901. DOI
10. H. Hotelling, “Analysis of a Complex of Statistical Variables into Principal Components,” *Journal of Educational Psychology*, vol. 24, pp. 417–441, 1933. DOI
11. I. T. Jolliffe and J. Cadima, “Principal Component Analysis: A Review and Recent Developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, 20150202, 2016. DOI

12. N. Halko, P.-G. Martinsson and J. A. Tropp, “Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011. DOI ·arXiv
13. P.-G. Martinsson and J. A. Tropp, “Randomized Numerical Linear Algebra: Foundations and Algorithms,” *Acta Numerica*, vol. 29, pp. 403–572, 2020. DOI ·arXiv
14. R. Murray, J. Demmel, M. W. Mahoney, N. B. Erichson, M. Melnichenko, O. A. Malik, L. Grigori, P. Luszczek, M. Dereziński, M. E. Lopes, T. Liang, H. Luo and J. Dongarra, “Randomized Numerical Linear Algebra: A Perspective on the Field With an Eye to Software,” LAPACK Working Note 299, 2023. arXiv
15. E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2022. arXiv
16. J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar and Y. Tian, “GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection,” *ICML*, 2024. arXiv
17. T. Miyato, T. Kataoka, M. Koyama and Y. Yoshida, “Spectral Normalization for Generative Adversarial Networks,” *ICLR*, 2018. arXiv
18. T. G. Kolda and B. W. Bader, “Tensor Decompositions and Applications,” *SIAM Review*, vol. 51, no. 3, pp. 455–500, 2009. DOI
19. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book

ML 01B・機率與資訊理論

隨機變數、常見分布、貝氏定理、期望值與共變異數、概似、熵、交叉熵、KL 散度與結構化模型，並連結到神經網路的訓練與評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-01b-probability-information/

你將學到

- 機器學習為什麼需要機率，以及解讀機率的兩種常見方式（頻率與信念程度）。
- 隨機變數、機率質量函數與機率密度函數如何運作，以及邊際、條件與聯合機率如何透過加法規則、乘法規則與連鎖規則彼此連結。
- 期望值、變異數、共變異數與相關係數如何描述一個分布，以及為什麼共變異數為零不代表獨立。
- 深度學習裡反覆出現的分布——Bernoulli、類別分布、高斯、指數、Laplace、Dirac、經驗分布與混合分布——以及用來參數化它們的 sigmoid 與 softplus 函數。
- 貝氏定理、變數變換公式、最大概似估計與 MAP 估計如何運作，並附上實際測試過的 NumPy 程式碼。
- 熵、交叉熵、KL 散度與互資訊各自衡量什麼，為什麼分類損失就是交叉熵，以及結構化（圖）模型如何把龐大的聯合分布拆成小因子。

先看全貌

神經網路永遠看不到整個世界。它只看到有限的訓練集——由不完美的感測器量測、由忙碌的人標註——接著卻要回答從沒看過的影像。這個過程的每一步都牽涉**不確定性**。機率論就是描述不確定性的數學：它告訴我們如何表達自己不知道的事、如何結合證據，以及如何做出承認懷疑的預測。與它關係密切的資訊理論，則衡量不確定性有多少，以及使用錯誤分布要付出多少代價。兩者合起來，幾乎可以解釋本站 ML 與 DL 系列中的每一個損失函數。

> **用最簡單的話說**
>
> 想像一個看不到內部的彈珠袋。**機率**是你對下一顆彈珠的最佳猜測：「大概是紅色，也許是藍色，幾乎不可能是綠色。」每摸出一顆彈珠，你就更了解這個袋子，並更新你的猜測——這個更新就是**貝氏定理**。**熵**是你預期會有多驚訝：全是紅色彈珠的袋子永遠不會讓你驚訝；十種顏色各占一樣多的袋子則常常讓你驚訝。機器學習模型是一個很厲害的猜測者，而訓練就是讓它對真正摸出的彈珠越來越不驚訝。

本篇依循 Goodfellow、Bengio 與 Courville《Deep Learning》中的「應用數學」部分 [1]，並補充 Bishop 與 Bishop [2]、Murphy [3] 以及《Mathematics for Machine Learning》[4] 的內容。它位於ML 01A・線性代數與ML 01C・數值計算之間。如果你接下來要讀圖形識別系列，PR 01・貝氏決策會直接延伸這裡的貝氏定理一節。

為什麼需要機率？

白話版。電腦科學通常處理確定的事實：一個位元不是 0 就是 1，程式不是停止就是不停止。機器學習沒辦法這樣運作，因為資料、世界和模型都不完美。

Goodfellow 等人 [1] 列出三種不確定性的來源：

1. **系統本身的隨機性**。有些系統本來就是隨機的——光子抵達相機感測器的時間（散粒雜訊）、骰子的點數，或行人踏下人行道的確切時刻。
2. **觀測不完整**。即使系統是決定性的，只要我們看不到它所有的變數，它看起來就是隨機的。一張汽車照片無法告訴你煞車燈是不是快要亮起。
3. **建模不完整**。模型必須捨棄資訊才有用處。只看 32×32 縮圖的分類器無法找回降取樣時遺失的細節，所以它對細粒度類別的預測必然帶有不確定性。

使用帶有不確定性的簡單規則（「大多數鳥會飛」），往往比使用永遠正確但極其複雜的規則（「鳥會飛，但企鵝、鴕鳥、受傷的鳥……除外」）更實際。

解讀機率的兩種方式

機率可以有兩種解讀，而兩者遵守相同的數學規則。

- **頻率學派機率**描述可重複事件的長期頻率：「這枚硬幣擲很多次，大約 70% 會出現正面」。
- **貝氏機率**描述對某件不一定能重複的事的信念程度：「看了這張 X 光片，我有 40% 的把握病人罹病」。

只要要求信念程度滿足幾項符合常識的一致性性質，就能證明它們必須遵守與頻率完全相同的規則，因此同一套計算可以同時服務兩種解讀 [1], [3]。實務上，深度學習混用兩者：資料被視為從某個分布抽出的樣本（頻率觀點），而模型的輸出常被解讀成對標籤的信念（貝氏觀點）。

隨機變數與機率分布

白話版。隨機變數是事先不知道值的量；機率分布說明每個值出現的可能性有多大。

[1] 用正體字母（例如 x）表示隨機變數；本篇用 X 表示變數、x 表示它的一個取值。隨機變數可以是**離散的**（有限或可數個值，例如 $\{1, \dots, K\}$ 中的類別標籤），也可以是**連續的**（實數值，例如被視為實數的像素強度）。

機率質量函數

離散變數有**機率質量函數**（probability mass function, PMF）$P(X = x)$，常簡寫為 $P(x)$。它必須滿足

$$0 \le P(x) \le 1 \text{ for every } x, \qquad \sum_x P(x) = 1.$$

這裡的總和涵蓋 X 所有可能的值 x。第一個條件說沒有事件比「確定」更確定；第二個條件說總有某件事會發生。聯合 PMF $P(X = x, Y = y)$ 則是兩個事件同時發生的機率。

機率密度函數

連續變數有**機率密度函數**（probability density function, PDF）$p(x)$，必須滿足

$$p(x) \ge 0, \qquad \int p(x)\,dx = 1.$$

密度不是機率。X 落在區間 $[a, b]$ 的機率是密度曲線下的面積：

$$P(a \le X \le b) = \int_a^b p(x)\,dx.$$

任何單一精確值的機率都是零。因為只有面積必須不超過一，密度本身可以大於一：標準差為 0.25 的窄高斯，峰值約為 1.6（圖 1 右上）。把密度值當成「機率 1.6」回報，是常見錯誤。

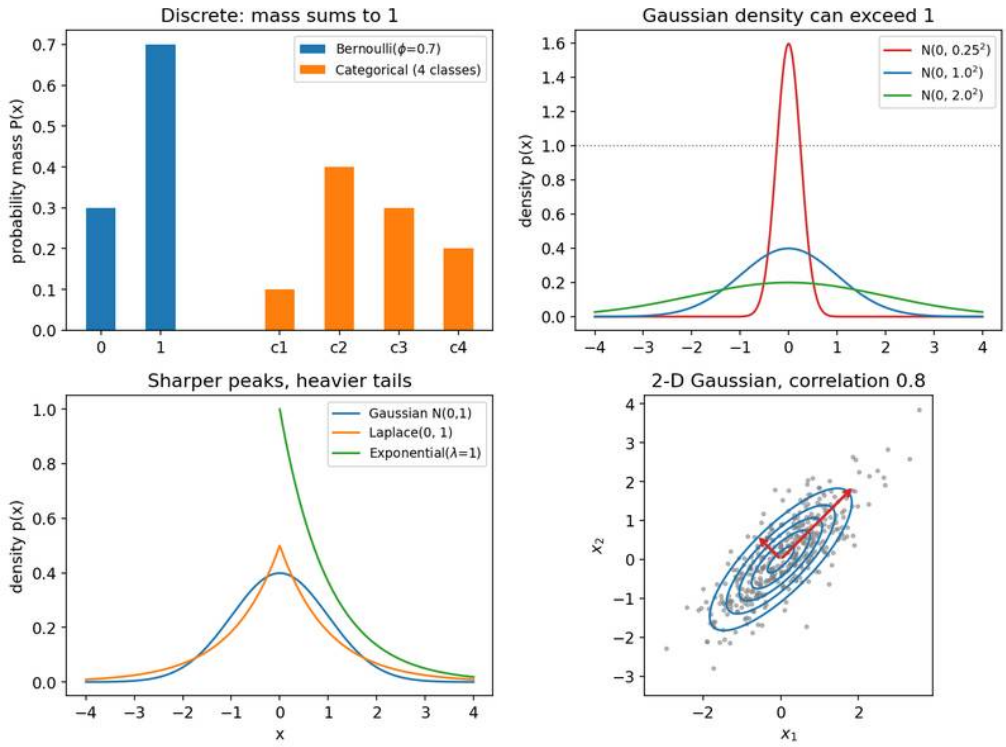


圖 *1* — 常見分布。左上：離散機率質量總和為一。右上：密度可以超過一，因為只有面積必須等於一。左下：*Laplace* 密度峰值比高斯尖、尾巴比高斯厚；指數分布只定義在 $x \ge 0$。右下：相關係數 *0.8* 的二維高斯；紅色箭頭是共變異數矩陣的特徵向量，長度為兩倍標準差。

邊際機率

白話版。如果知道兩件事如何一起變化，就可以把其中一件「加總掉」，得到另一件事單獨的變化方式。

給定 X 與 Y 的聯合分布，X 的**邊際**分布為

$$P(x) = \sum_{y} P(x, y) \quad \text{(discrete)}, \qquad p(x) = \int p(x, y)\, dy \quad \text{(continuous)}.$$

這就是**加法規則**（sum rule）。如果把聯合分布寫成表格，列代表 x、欄代表 y，那麼 x 的邊際分布就是每一列的總和——過去習慣寫在紙張的邊緣（margin），因此得名。

條件機率

白話版。條件機率回答的是：「在已經知道那件事的情況下，這件事有多可能？」

$$P(y \mid x) = \frac{P(x, y)}{P(x)}, \qquad P(x) > 0.$$

$P(y \mid x)$ 是觀察到 $X = x$ 之後 $Y = y$ 的機率，只有在條件真的可能發生時才有定義。移項後得到**乘法規則**（product rule）$P(x, y) = P(y \mid x) P(x)$。

不要把條件化和介入混為一談。「有人帶傘的情況下街道是濕的」機率很高，但強迫大家帶傘並不會讓街道變濕。關於介入的問題屬於因果推論，光有聯合分布是不夠的。

機率的連鎖規則

反覆套用乘法規則，可以把任何 n 個變數的聯合分布分解成

$$P(x^{(1)}, \dots, x^{(n)}) = P(x^{(1)}) \prod_{i=2}^{n} P(x^{(i)} \mid x^{(1)}, \dots, x^{(i-1)}).$$

這裡 $x^{(i)}$ 是第 i 個變數的值。這個恆等式是精確的——不需要任何假設。它是自迴歸模型的骨幹：語言模型或自迴歸影像模型，會根據前面所有的 token 或像素，一次預測一個（ML 08、NLP 01）。建模的選擇在於如何參數化每一個條件分布，而不在分解本身。

獨立與條件獨立

如果

$$p(x, y) = p(x)\, p(y) \quad \text{for all } x, y,$$

就稱兩個變數**獨立**，記為 $X \perp Y$。如果

$$p(x, y \mid z) = p(x \mid z)\, p(y \mid z) \quad \text{for all } x, y, z,$$

就稱它們在給定 Z 下**條件獨立**，記為 $X \perp Y \mid Z$。

冰淇淋銷量和曬傷人數彼此相關，但在給定天氣之後，兩者大致獨立。條件獨立是讓高維模型變得可計算的主要工具——單純貝氏（naive Bayes）假設特徵在給定類別下彼此獨立；隱藏式馬可夫模型假設給定目前狀態後，未來與過去獨立。這些都是假設 [3]：應該由問題本身來支持，而不是因為能讓代數變簡單就採用。

程式碼：聯合機率表、邊際與條件機率

下面的例子為天氣與帶傘建立一張小型聯合機率表，計算邊際與條件機率，並檢查是否獨立。它也展示了兩個變數明顯相依、共變異數卻（幾乎）為零的情況。

```
import numpy as np

# Joint table P(weather, umbrella): rows = sunny/rainy, columns = no/yes
P = np.array([[0.56, 0.14],
              [0.06, 0.24]])
assert np.isclose(P.sum(), 1)

p_weather = P.sum(axis=1)            # marginal over umbrella
p_umbrella = P.sum(axis=0)           # marginal over weather
p_umb_given_w = P / p_weather[:, None]   # each row sums to 1
print("P(weather):", p_weather)
print("P(umbrella):", p_umbrella)
print("P(umbrella | weather):\n", p_umb_given_w)
print("independent?", np.allclose(P, np.outer(p_weather, p_umbrella)))

# Zero covariance does not imply independence
rng = np.random.default_rng(0)
x = rng.uniform(-1, 1, 200_000)
y = x**2                              # y is a deterministic function of x
print("cov(x, y) ~", np.cov(x, y)[0, 1].round(4))
```

輸出：邊際分布為 $[0.7, 0.3]$ 與 $[0.62, 0.38]$；條件機率表為 $[[0.8, 0.2], [0.2, 0.8]]$；兩個變數不獨立；而 x 與 x^2 的樣本共變異數約為 -0.0007，幾乎是零——即使 y 完全由 x 決定。

期望值、變異數與共變異數

白話版。期望值是長期平均；變異數是數值通常偏離平均多遠；共變異數則說明兩個量是否傾向一起上升、一起下降。

函數 f 在某個分布下的**期望值**為

$$\mathbb{E}_{x\sim P}[f(x)] = \sum_{x} P(x) f(x), \qquad \mathbb{E}_{x\sim p}[f(x)] = \int p(x) f(x)\, dx.$$

下標 $x \sim P$ 指出對哪個分布取平均；意思明確時可以省略。期望值是**線性的**：對常數 α, β，$\mathbb{E}[\alpha f(x) + \beta g(x)] = \alpha \mathbb{E}[f(x)] + \beta \mathbb{E}[g(x)]$。幾乎每一個訓練損失都是對資料分布的期望值，並用小批次（mini-batch）的平均來估計。

變異數衡量數值在平均 $\mu = \mathbb{E}[f(x)]$ 周圍的分散程度：

$$\operatorname{Var}(f(x)) = \mathbb{E}\big[(f(x) - \mu)^2\big] = \mathbb{E}[f(x)^2] - \mu^2.$$

它的平方根是**標準差**，單位與 $f(x)$ 相同。

兩個量的**共變異數**衡量它們如何一起變化：

$$\operatorname{Cov}(f(x), g(y)) = \mathbb{E}\big[(f(x) - \mathbb{E}[f(x)])\,(g(y) - \mathbb{E}[g(y)])\big].$$

共變異數為正，代表兩者傾向同時高於（或低於）各自的平均；為負，代表一個高時另一個傾向低。它的大小取決於單位，所以我們常把它正規化成**相關係數**

$$\rho = \frac{\operatorname{Cov}(X, Y)}{\sqrt{\operatorname{Var}(X)\operatorname{Var}(Y)}} \in [-1, 1].$$

獨立意味著共變異數為零，但**共變異數為零不代表獨立**。共變異數只衡量線性相依。在上面的程式碼中，x 在 $[-1, 1]$ 上均勻分布、$y = x^2$：知道 x 就能確切知道 y，但因為關係對稱且非線性，共變異數為零。稍後介紹的互資訊能捕捉任何形式的相依。

共變異數矩陣

對平均為 $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$ 的隨機向量 $\mathbf{x} \in \mathbb{R}^n$，**共變異數矩陣**為

$$\Sigma = \operatorname{Cov}(\mathbf{x}) = \mathbb{E}\big[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top\big], \qquad \Sigma_{ij} = \operatorname{Cov}(x_i, x_j).$$

對角線是各分量的變異數 $\Sigma_{ii} = \operatorname{Var}(x_i)$。$\Sigma$ 是對稱半正定矩陣，因此（見ML 01A）具有非負實特徵值與正交特徵向量。特徵向量是資料雲的主軸，特徵值是沿主軸的變異數——也就是圖 1 中的紅色箭頭——這正是 PCA 所計算的東西。對仿射映射 $\mathbf{y} = A\mathbf{x} + \mathbf{b}$，有 $\mathbb{E}[\mathbf{y}] = A\boldsymbol{\mu} + \mathbf{b}$ 與 $\operatorname{Cov}(\mathbf{y}) = A\Sigma A^\top$。

常見機率分布

白話版。有一小群分布會一再出現。選擇其中一個，就是對資料或目標做出一個假設。

Bernoulli 分布

單一二元變數 $X \in \{0, 1\}$，參數 $\phi \in [0, 1]$：

$$P(X = 1) = \phi, \quad P(X = 0) = 1 - \phi, \quad P(X = x) = \phi^x (1 - \phi)^{1 - x}.$$

平均為 ϕ，變異數為 $\phi(1 - \phi)$。輸出層使用 sigmoid 的二元分類器，就是在預測標籤之 Bernoulli 分布的參數 ϕ。

類別分布（multinoulli）

具有 K 個狀態的單一離散變數，以機率向量 $\mathbf{p} \in [0,1]^K$（$\sum_k p_k = 1$）為參數：

$$P(X = k) = p_k.$$

若標籤以 one-hot 編碼為 $\mathbf{y} \in \{0,1\}^K$，則 $P(\mathbf{y}) = \prod_k p_k^{y_k}$。softmax 分類器輸出的正是這樣的向量。（多項分布計算多次抽取的結果次數；只抽一次時就退化成類別分布，Goodfellow 等人稱之為「multinoulli」[1]。）

高斯（常態）分布

最常用的連續分布：

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\!\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big).$$

μ 是平均（也是峰值位置），σ^2 是變異數。有時改用**精確度**（precision）$\beta = 1/\sigma^2$，在變異數經常改變時計算起來比較省。

高斯之所以成為預設選擇有兩個原因。**中央極限定理**說，許多變異數有限的獨立變數相加，會趨近高斯分布，所以由許多微小因素組成的量測雜訊往往接近高斯。而在實數線上、平均與變異數固定的所有分布中，高斯的**熵最大**：它加入的額外結構最少。

在 n 維中，

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \exp\!\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x}-\boldsymbol{\mu})\Big),$$

其中 $\boldsymbol{\mu} \in \mathbb{R}^n$ 是平均向量，$\Sigma \in \mathbb{R}^{n \times n}$ 是對稱正定的共變異數矩陣。**精確度矩陣** $\Lambda = \Sigma^{-1}$ 有個漂亮的意義：$\Lambda_{ij} = 0$ 恰好代表在給定其他所有座標下，x_i 與 x_j 條件獨立。**等向**（isotropic）高斯的 $\Sigma = \sigma^2 I$——這正是擴散模型使用的雜訊分布（ML 07）。

它和你會用到的損失函數有一個根本的連結：如果迴歸網路預測一個固定變異數之高斯的平均，那麼最大化概似就等於最小化**均方誤差**。

指數分布與 Laplace 分布

有時候我們想在特定位置放一個尖峰。

$$p(x; \lambda) = \lambda\, \mathbf{1}_{x \ge 0}\, e^{-\lambda x}, \qquad \text{Laplace}(x; \mu, \gamma) = \frac{1}{2\gamma} \exp\!\Big(-\frac{|x-\mu|}{\gamma}\Big).$$

$\lambda > 0$ 是速率，$\mathbf{1}_{x \ge 0}$ 是指示函數（$x \ge 0$ 時為 1，否則為 0），μ 是位置，$\gamma > 0$ 是尺度。和高斯相比，Laplace 密度的峰值更尖、尾巴更厚（圖 1 左下）。用最大概似預測 Laplace 分布的位置，等同於最小化**絕對誤差**（L_1 損失），這就是 L_1 比 L_2 更能容忍離群值的原因；而對權重使用 Laplace 先驗，就得到 L_1 正則化（稀疏性）。

Dirac 分布與經驗分布

Dirac delta $\delta(x-\mu)$ 把所有質量放在點 μ：它在其他地方都是零，積分卻等於一。它是一種廣義函數，由它在積分中的作用來定義：$\int \delta(x-\mu) f(x)\,dx = f(\mu)$。

訓練集 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ 定義了**經驗分布**

$$\hat{p}(\mathbf{x}) = \frac{1}{m}\sum_{i=1}^{m} \delta(\mathbf{x} - \mathbf{x}^{(i)}),$$

在 m 個樣本上各放 $1/m$ 的質量。把損失對整個資料集取平均來訓練模型，就等於在 $\hat{p}$ 下取期望值。經驗分布也是讓訓練資料概似最大的分布——這正是只最小化訓練損失會過擬合（overfitting）的原因。

混合分布與潛在變數

白話版。混合分布先隨機選一個「類型」，再從那個類型的分布中抽樣。

$$P(\mathbf{x}) = \sum_{k=1}^{K} P(z=k)\, P(\mathbf{x} \mid z=k).$$

z 是一個類別型的**潛在變數**（latent variable）——我們從未直接觀察到的變數——以機率 $\pi_k = P(z=k)$ 選出，$P(\mathbf{x} \mid z=k)$ 是第 k 個成分。**高斯混合模型**（Gaussian mixture model, GMM）使用高斯成分 $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \Sigma_k)$；只要成分夠多，它可以把任何平滑密度近似到任意指定的非零誤差 [1]。給定一個觀測 $\mathbf{x}$，貝氏定理給出**後驗責任值** $P(z=k \mid \mathbf{x})$：該點由第 k 個成分產生的可能性。用期望最大化（EM）擬合 GMM 是 PR 07 的主題。

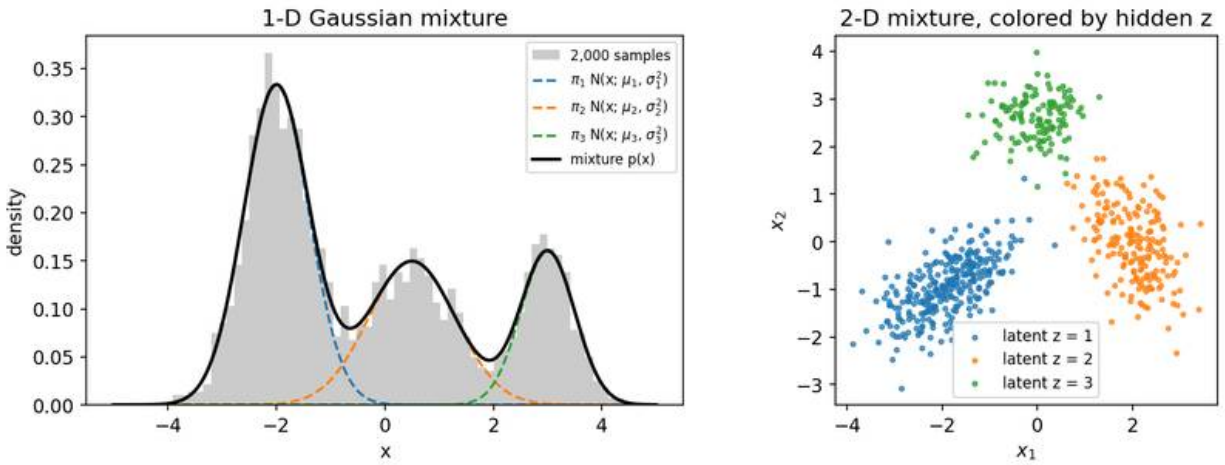


圖 2 — 高斯混合。左：黑色密度是三個成分的加權總和；灰色直方圖是先選成分、再從該成分抽樣得到的 2,000 個樣本。右：在二維中，顏色代表隱藏成分 z——真實資料集並不會告訴你這個資訊。

潛在變數是現代生成模型的核心。VAE（ML 06）粗略地說，就是潛在變數 $\mathbf{z}$ 為連續、且 $p(\mathbf{x} \mid \mathbf{z})$ 由神經網路表示的混合模型 [8]；此時邊際分布 $p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$ 無法直接計算，只能近

似。

```
import numpy as np

rng = np.random.default_rng(0)
pi = np.array([0.5, 0.3, 0.2])      # mixing weights P(z = k)
mu = np.array([-2.0, 0.5, 3.0])     # component means
sd = np.array([0.6, 0.8, 0.5])      # component standard deviations

# Ancestral sampling: first the hidden component z, then x given z
z = rng.choice(3, size=100_000, p=pi)
x = rng.normal(mu[z], sd[z])

def log_normal(x, m, s):
    return -0.5*((x - m)/s)**2 - np.log(s) - 0.5*np.log(2*np.pi)

def mixture_logpdf(x):
    # log sum_k pi_k N(x; mu_k, sd_k^2), computed with log-sum-exp
    a = np.log(pi) + log_normal(x[:, None], mu, sd)
    m = a.max(axis=1, keepdims=True)
    return (m + np.log(np.exp(a - m).sum(axis=1, keepdims=True))).ravel()

grid = np.linspace(-6, 6, 4001)
print("density integrates to",
 ↪  (np.exp(mixture_logpdf(grid)).sum()*(grid[1]-grid[0])).round(4))
print("sample mean", x.mean().round(3), "vs exact", (pi*mu).sum().round(3))
# Posterior responsibility P(z | x) for one observation
a = np.log(pi) + log_normal(np.array([0.0])[:, None], mu, sd)
resp = np.exp(a - a.max()); resp /= resp.sum()
print("P(z | x=0):", resp.round(3))
```

密度積分為 1.0，樣本平均（−0.255）與精確平均 $\sum_k \pi_k \mu_k = -0.25$ 相符，而點 $x = 0$ 以 0.99 的機率被歸給中間的成分。這裡用到的 log-sum-exp 技巧在ML 01C 中說明。

實用函數：sigmoid 與 softplus

白話版。 sigmoid 把任何實數壓進 0 到 1 之間，變成一個機率；softplus 是 $\max(0, x)$ 的平滑版本，輸出永遠為正。

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \qquad \zeta(x) = \log\left(1 + e^{x}\right).$$

logistic sigmoid σ 產生 Bernoulli 分布的參數 ϕ；當 $|x|$ 很大時它會**飽和**（變平），使那裡的梯度很小。**softplus** ζ 產生正數，例如標準差 σ 或精確度 β；它的名字來自它是「正部」$x^{+} = \max(0, x)$ 的柔化版本。

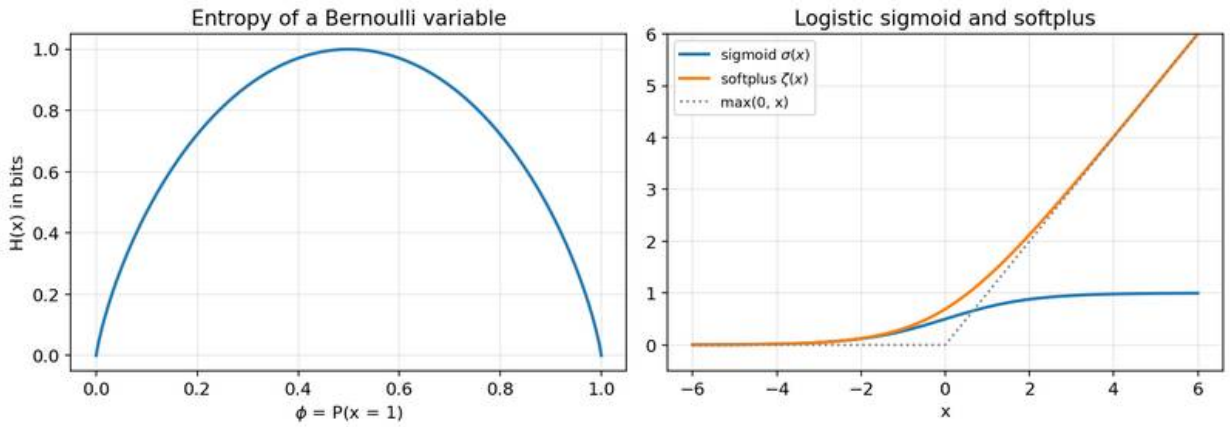


圖 3 — 左：*Bernoulli* 變數的熵，在 $\varphi = 0.5$ 時最大（1 位元），結果確定時為零。右：*logistic sigmoid* $\sigma(x)$ 與 *softplus* $\zeta(x)$，後者是 $max(0, x)$ 的平滑版本。

下列恆等式值得記住，因為每次化簡損失函數時都會用到 [1]：

$$1 - \sigma(x) = \sigma(-x), \qquad \frac{d}{dx}\sigma(x) = \sigma(x)(1 - \sigma(x)),$$

$$\log \sigma(x) = -\zeta(-x), \qquad \frac{d}{dx}\zeta(x) = \sigma(x),$$

$$\sigma^{-1}(p) = \log \frac{p}{1-p}, \qquad \zeta(x) - \zeta(-x) = x.$$

σ^{-1} 是 **logit**（對數勝算）函數。恆等式 $\log \sigma(x) = -\zeta(-x)$ 說明了為什麼二元交叉熵應該從 logit 直接計算：$-\log \sigma(x) = \zeta(-x)$，而 softplus 可以在不溢位的情況下求值。

```
import numpy as np

def sigmoid(x):
    return 1/(1 + np.exp(-x))

def softplus(x):
    return np.logaddexp(0, x)          # log(1 + exp(x)) without overflow

x = np.linspace(-30, 30, 13)
assert np.allclose(1 - sigmoid(x), sigmoid(-x))
assert np.allclose(np.log(sigmoid(x)), -softplus(-x))
assert np.allclose(softplus(x) - softplus(-x), x)
h = 1e-6
assert np.allclose((softplus(x+h) - softplus(x-h))/(2*h), sigmoid(x), atol=1e-6)
p = sigmoid(x[4:9])
assert np.allclose(np.log(p/(1-p)), x[4:9])   # logit inverts sigmoid
print("naive log(1+exp(1000)):", np.log1p(np.exp(np.float64(1000))))
print("stable softplus(1000):", softplus(1000.0))
```

所有恆等式都通過檢查。直接計算的寫法傳回 `inf`（並出現溢位警告），而 `np.logaddexp` 傳回 1000.0。

貝氏定理

白話版。貝氏定理把「假設成立時，看到這個證據的可能性有多大？」轉換成「看到證據之後，這個假設有多可能？」——並且堅持你必須考慮這個假設原本有多常見。

把乘法規則從兩個方向寫出來，$P(a, b) = P(a \mid b)P(b) = P(b \mid a)P(a)$，於是

$$P(a \mid b) = \frac{P(b \mid a)\,P(a)}{P(b)}, \qquad P(b) = \sum_{a'} P(b \mid a')\,P(a').$$

- $P(a)$ 是**先驗**（prior）：看到證據之前 a 的可信度。
- $P(b \mid a)$ 是在假設 a 下證據 b 的**概似**（likelihood）。
- $P(a \mid b)$ 是**後驗**（posterior）：更新後的可信度。
- $P(b)$ 是**證據**（evidence，或稱邊際概似），對所有假設 a' 加總得到的正規化常數。

實例：基本比率效應

考慮一個假想的瑕疵偵測器，**靈敏度** $P(+ \mid \text{defect}) = 0.9$、**偽陽性率** $P(+ \mid \text{no defect}) = 0.05$，而**盛行率**（基本比率）$P(\text{defect}) = 0.01$。被判為陽性的物品真的有瑕疵的機率是

$$P(\text{defect} \mid +) = \frac{0.9 \cdot 0.01}{0.9 \cdot 0.01 + 0.05 \cdot 0.99} \approx 0.154.$$

即使偵測器能抓到 90% 的瑕疵，警報中只有約 15% 是真的。用數量來想就很清楚：在 10,000 件物品中，有 100 件有瑕疵，其中 90 件觸發警報；但另外 9,900 件正常品中，也有 495 件觸發警報（圖 4 右）。同一個偵測器在 10% 盛行率下則得到 $0.09/(0.09 + 0.045) = 2/3$。這是數學示範，不是

任何真實系統的效能宣稱。它的教訓適用於每一個部署中的分類器：**光有靈敏度與偽陽性率，無法決定陽性預測有多可信；部署環境的盛行率同樣關鍵**。 ML 10 在討論分布偏移下的精確率時會再回到這一點。

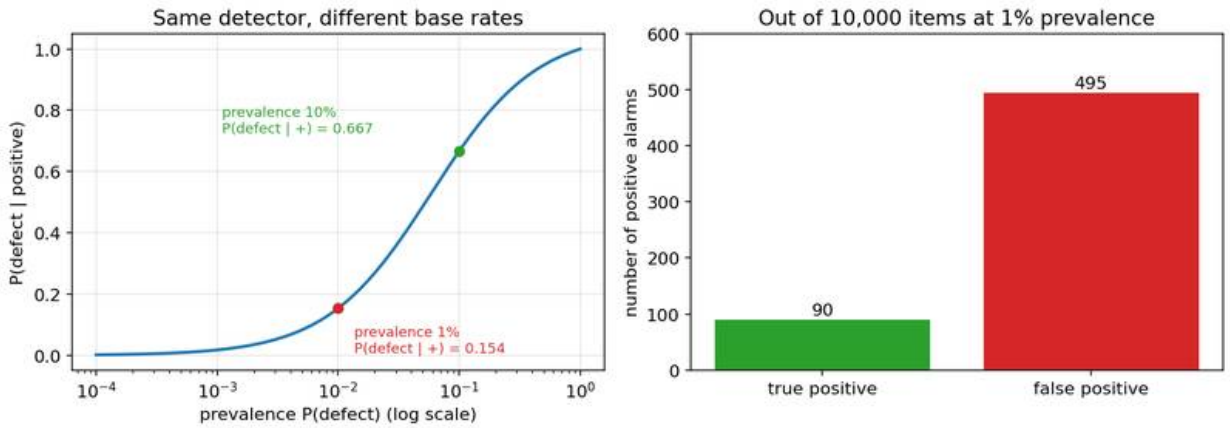


圖 4 — 假想偵測器（靈敏度 *0.9*、偽陽性率 *0.05*）的基本比率效應。左：後驗 $P(\textit{defect} \mid +)$ 強烈依賴盛行率。右：在 *1%* 盛行率下，假警報比真警報多出五倍以上。

```
import numpy as np

def posterior_positive(prevalence, sensitivity=0.9, false_pos=0.05):
    evidence = sensitivity*prevalence + false_pos*(1 - prevalence)   # P(+)
    return sensitivity*prevalence / evidence                          # P(defect | +)

for prev in [0.001, 0.01, 0.1, 0.5]:
    print(f"prevalence {prev:>5}: P(defect | +) = {posterior_positive(prev):.3f}")

# Simulation check with 1,000,000 synthetic items
rng = np.random.default_rng(0)
defect = rng.random(1_000_000) < 0.01
alarm = np.where(defect, rng.random(defect.size) < 0.9, rng.random(defect.size) < 0.05)
print("simulated:", defect[alarm].mean().round(3))
```

公式對四種盛行率分別給出 0.018、0.154、0.667 與 0.947，而一百萬件物品的模擬重現了 0.154。

貝氏定理也是**貝氏分類器**的基礎：預測後驗 $P(y = k \mid \mathbf{x}) \propto p(\mathbf{x} \mid y = k) P(y = k)$ 最大的類別 k，可以讓期望錯誤率最小。PR 01 會深入發展這套決策理論。

密度的變數變換

白話版。拉伸一張橡皮布，上面的墨水會變淡。隨機變數經過變換後，它的密度也必須依照變換把空間拉伸了多少來重新縮放。

一個誘人的錯誤：若 $y = g(x)$、x 的密度為 p_x，就以為 $p_y(y) = p_x(g^{-1}(y))$。這是錯的，因為它的積分不等於一。對可逆且可微的 g，一維的正確規則是

$$p_y(y) = p_x(g^{-1}(y)) \left| \frac{d\, g^{-1}(y)}{dy} \right|.$$

小區間內的機率必須守恆：$|p_y(y)\, dy| = |p_x(x)\, dx|$。在 n 維中，若 $\mathbf{y} = g(\mathbf{x})$，則

$$p_x(\mathbf{x}) = p_y(g(\mathbf{x})) \left| \det \frac{\partial g(\mathbf{x})}{\partial \mathbf{x}} \right|,$$

其中 $\partial g / \partial \mathbf{x}$ 是 g 的 $n \times n$ **Jacobian 矩陣**（第 (i, j) 個元素為 $\partial y_i / \partial x_j$），$|\det(\cdot)|$ 是其行列式的絕對值——也就是 g 放大小體積的倍率（行列式見ML 01A）。這個公式是**正規化流**（normalizing flows）的全部基礎：它堆疊 Jacobian 行列式容易計算的可逆網路，得到精確的概似 [9]。

```
import numpy as np

rng = np.random.default_rng(0)
x = rng.normal(0, 1, 500_000)        # x ~ N(0, 1)
y = np.exp(x)                        # y = g(x), invertible on (0, inf)

def p_x(x):
```

```
    return np.exp(-0.5*x**2)/np.sqrt(2*np.pi)

def p_y(y):
    # p_y(y) = p_x(g^{-1}(y)) * |d g^{-1}(y) / dy| = p_x(log y) / y
    return p_x(np.log(y))/y

counts, edges = np.histogram(y, bins=np.linspace(0.05, 4, 80), density=False)
centers = 0.5*(edges[:-1] + edges[1:])
empirical = counts/(y.size*np.diff(edges))
print("max |histogram - formula|:", np.abs(empirical - p_y(centers)).max().round(3))
print("wrong formula (no Jacobian) error:", np.abs(empirical -
 ↪  p_x(np.log(centers))).max().round(3))
```

加上 Jacobian 因子 $1/y$ 後，直方圖與公式的差距在 0.011 以內（抽樣雜訊）；少了它，誤差達到 0.46。

概似、最大概似與 MAP

白話版。固定已觀察到的資料，問哪些參數值最能讓這些資料顯得合理。如果你對參數另有先驗知識，也把它一併考慮進去。

假設模型 $p_{\boldsymbol{\theta}}(\mathbf{x})$ 的參數為 $\boldsymbol{\theta}$，並觀察到由獨立同分布（i.i.d.）樣本組成的資料集 $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$。**概似**為

$$L(\boldsymbol{\theta}) = p_{\boldsymbol{\theta}}(\mathcal{D}) = \prod_{i=1}^{m} p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}).$$

作為 $\boldsymbol{\theta}$ 的函數，它不是參數上的機率分布：它對 $\boldsymbol{\theta}$ 的積分不必等於一。**最大概似估計**（maximum likelihood estimate, MLE）為

$$\boldsymbol{\theta}_{\mathrm{ML}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = \arg\min_{\boldsymbol{\theta}} \; \mathbb{E}_{\mathbf{x}\sim\hat{p}}[-\log p_{\boldsymbol{\theta}}(\mathbf{x})].$$

我們取對數，因為許多小於一的數相乘會下溢，而對數相加在數值上很安全，且最大值位置不變。右邊的形式說明 MLE 是在經驗分布 $\hat{p}$ 下**最小化平均負對數概似**（negative log-likelihood, NLL）。對分類器 $p_{\boldsymbol{\theta}}(y \mid \mathbf{x})$，同樣的想法給出條件 NLL $-\log p_{\boldsymbol{\theta}}(y \mid \mathbf{x})$，也就是下面會看到的交叉熵損失。

兩個經典結果：n 次擲硬幣得到 k 次正面時，$\phi_{\mathrm{ML}} = k/n$；對高斯而言，μ_{ML} 是樣本平均，$\sigma^2_{\mathrm{ML}} = \frac{1}{m}\sum_i (x^{(i)} - \mu_{\mathrm{ML}})^2$——這是略帶偏差的估計（改除以 $m-1$ 可以消除偏差）。

MAP 估計

貝氏推論把 $\boldsymbol{\theta}$ 本身視為隨機變數，給它一個**先驗** $p(\boldsymbol{\theta})$，再計算**後驗**

$$p(\boldsymbol{\theta} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})}{p(\mathcal{D})} \;\propto\; p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

完整的後驗往往難以計算，常見的捷徑是取它的眾數，即**最大後驗**（maximum a posteriori, MAP）估計：

$$\boldsymbol{\theta}_{\text{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[\log p(\mathcal{D} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) \right].$$

若使用高斯先驗 $p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, \frac{1}{\lambda} I)$，則 $-\log p(\boldsymbol{\theta})$ 等於 $\frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$ 加上常數：**權重衰減就是使用高斯先驗的 MAP**。Laplace 先驗則給出 L_1 正則化。這把機率假設和ML 02 中的正則化項連結起來。但 MAP 點估計仍然只是一個參數向量，無法告訴你參數有多不確定。近似整個後驗是 PR 08 與變分推論 [10] 的主題。

```
import numpy as np

rng = np.random.default_rng(0)
flips = rng.random(10) < 0.7          # 10 coin flips, true P(heads) = 0.7
k, n = flips.sum(), flips.size

phi = np.linspace(1e-3, 1 - 1e-3, 999)
log_lik = k*np.log(phi) + (n - k)*np.log(1 - phi)
a, b = 5.0, 5.0                        # Beta(5, 5) prior, centered on 0.5
log_prior = (a - 1)*np.log(phi) + (b - 1)*np.log(1 - phi)

print(f"{k} heads in {n} flips")
print("MLE  grid:", phi[log_lik.argmax()].round(3), " closed form:", k/n)
print("MAP  grid:", phi[(log_lik + log_prior).argmax()].round(3),
      " closed form:", round((k + a - 1)/(n + a + b - 2), 3))

# Gaussian MLE: sample mean and the 1/m (biased) variance
x = rng.normal(2.0, 3.0, 50)
print("Gaussian MLE mean, var:", x.mean().round(3), x.var().round(3),
      " unbiased var:", x.var(ddof=1).round(3))
```

在這個隨機種子下，10 次擲出 6 次正面。MLE 為 0.6；Beta(5, 5) 先驗把 MAP 估計拉向 0.5，得到 0.556，與閉合解 $(k + a - 1)/(n + a + b - 2)$ 一致。資料越多，概似就越占主導，兩種估計會逐漸接近。高斯的 MLE 變異數（7.71）小於不偏估計（7.87），符合預期。

資訊理論

白話版。資訊理論衡量驚訝程度。罕見的事件一旦發生，帶來大量資訊；確定的事件則毫無資訊。把驚訝程度取平均就是熵；使用錯誤分布所多出的驚訝，就是 KL 散度。

Claude Shannon 為了研究透過有雜訊的通道傳送訊息需要多少位元而創立了這個領域 [5]；標準教科書是 Cover 與 Thomas [7]。對機器學習而言，最有用的概念是量化兩個分布有多不同。

自資訊與熵

結果 x 的**自資訊**（self-information）為

$$I(x) = -\log P(x).$$

使用自然對數時單位為 **nat**；使用 $\log_2$ 時單位為**位元**（bit）（1 nat $= 1/\ln 2 \approx 1.443$ 位元）。確定事件（$P = 1$）的資訊量為零；不太可能的事件資訊量很大；而獨立事件的資訊可以相加，因為對數把乘積變成總和。

Shannon 熵是自資訊的期望值：

$$H(P) = \mathbb{E}_{x\sim P}[I(x)] = -\sum_x P(x)\log P(x).$$

我們採用 $0\log 0 = 0$ 的約定，理由是 $p \to 0$ 時 $p\log p \to 0$。熵是編碼從 P 抽出之結果所需的最少平均位元數（或 nat 數）。決定性結果的熵為零；K 個結果的均勻分布熵最大，為 $\log K$。參數為 ϕ 的 Bernoulli 變數，$H = -\phi\log\phi - (1-\phi)\log(1-\phi)$，在 $\phi = 0.5$ 時達到 1 位元的峰值（圖 3 左）。

對連續變數，$h(p) = -\int p(x)\log p(x)\,dx$ 稱為**微分熵**。使用時要小心：它可以是負的（$[0, \frac{1}{2}]$ 上的均勻分布 $h = -\log 2$），而且座標尺度一改它就跟著改。它並不是離散位元數的極限版本。

KL 散度

Kullback–Leibler 散度 [6] 衡量分布 Q 與參考分布 P 有多不同：

$$D_{\mathrm{KL}}(P\,\|\,Q) = \mathbb{E}_{x\sim P}\Big[\log\frac{P(x)}{Q(x)}\Big] = \sum_x P(x)\log\frac{P(x)}{Q(x)}.$$

用編碼的語言說，它是：當編碼是為 Q 設計、卻用來編碼 P 的樣本時，額外需要的 nat 數。它的主要性質：

- **非負**：$D_{\mathrm{KL}}(P\|Q) \geq 0$，等號成立若且唯若 $P = Q$（離散情況；連續情況為「幾乎處處」相等）。這來自 Jensen 不等式，因為 $-\log$ 是凸函數。
- **在無支撐處為無限大**：若某個 x 使 $P(x) > 0$ 但 $Q(x) = 0$，散度為無限大。
- **不對稱**：一般而言 $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$，而且不滿足三角不等式。因此 KL 不是距離度量，儘管它常被描述成「分布之間的距離」。

正向與反向 KL

當近似用的分布族太簡單時，這種不對稱會產生看得見的後果。圖 5 用單一高斯 q 去擬合有兩個分得很開之峰的目標 p。

- 最小化**正向 KL** $D_{\mathrm{KL}}(p\|q)$ 時，若 q 在 p 有質量的地方很小，就會受到重罰。因此最佳高斯會**涵蓋兩個峰**，把平均放在兩峰之間（$\mu = 0$、$\sigma \approx 2.62$），並把大量質量放在空蕩的谷底。對高斯 q 而言，正向 KL 的最佳解就是匹配 p 的平均與變異數。
- 最小化**反向 KL** $D_{\mathrm{KL}}(q\|p)$ 時，若 q 在 p 很小的地方很大，就會受到重罰。因此最佳高斯會**鎖定其中一個峰**（$\mu \approx 2.49$、$\sigma \approx 0.81$），忽略另一個。

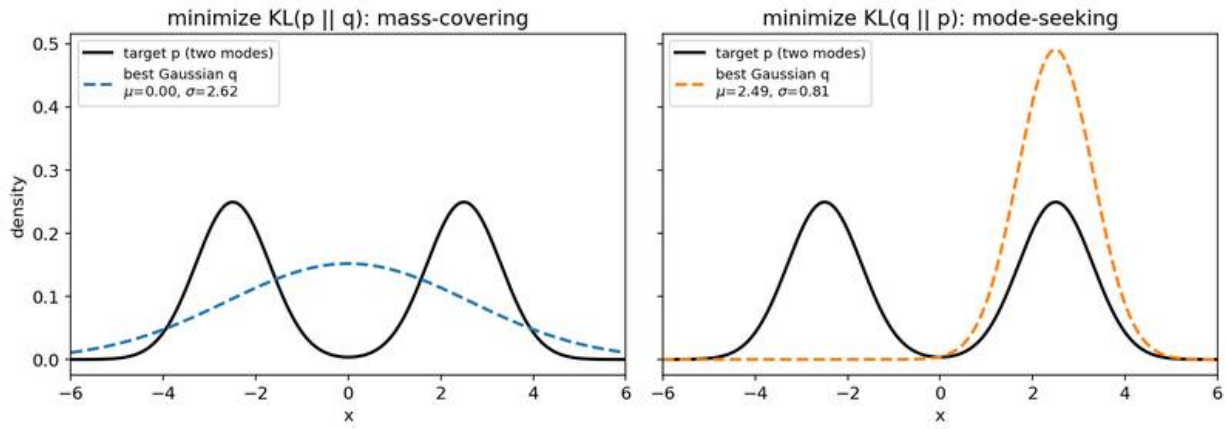


圖 5 — *KL* 不對稱。用單一高斯 q 擬合雙峰目標 p：正向 $KL(p\|q)$ 得到以兩峰中間為中心、涵蓋質量的擬合；反向 $KL(q\|p)$ 得到只落在一個峰上、尋找眾數的擬合（落在另一個峰上是同樣好的鏡像解）。

要最小化哪一個方向，是設計上的選擇。**最大概似最小化的是從資料分布到模型的正向 KL**，因此以概似訓練的模型（自迴歸模型、正規化流、VAE）傾向把機率分散到所有資料上，有時會產生模糊的樣本。**變分推論通常最小化從近似分布到真實後驗的反向 KL** [10]，因此傾向低估後驗的不確定性。

交叉熵

Q 相對於 P 的**交叉熵**為

$$H(P,Q) = -\mathbb{E}_{x\sim P}[\log Q(x)] = H(P) + D_{\mathrm{KL}}(P\,\|\,Q).$$

因為 $H(P)$ 與 Q 無關，對 Q 最小化交叉熵就等於最小化正向 KL。在分類中，P 是經驗標籤分布——每個樣本是一個 one-hot 向量——而 Q 是模型的 softmax 輸出 $\hat{\mathbf{y}}$。對真實類別為 y 的單一樣本，

$$H(\mathbf{e}_y, \hat{\mathbf{y}}) = -\sum_k [\mathbf{e}_y]_k \log \hat{y}_k = -\log \hat{y}_y,$$

其中 $\mathbf{e}_y$ 是類別 y 的 one-hot 向量。對整個資料集取平均，這正是條件負對數概似。所以「交叉熵損失」、「對數損失」與「類別模型的負對數概似」是同一個目標的三個名字。任何由負對數概似構成的損失，都是經驗分布與模型之間的交叉熵——這也是為什麼均方誤差同樣是高斯模型的「交叉熵」[1]。

互資訊

互資訊衡量知道一個變數能減少多少對另一個變數的不確定性：

$$I(X;Y) = D_{\mathrm{KL}}(p(x,y)\,\|\,p(x)p(y)) = H(X) - H(X\mid Y) = H(X) + H(Y) - H(X,Y).$$

其中 $H(X \mid Y) = -\mathbb{E}_{x,y}[\log p(x \mid y)]$ 是條件熵，$H(X,Y)$ 是聯合熵。$I(X;Y) \geq 0$，等號成立若且唯若 X 與 Y 獨立——和共變異數不同，它能偵測任何形式的相依。它對 X 與 Y 是對稱的。

程式碼：資訊量

```
import numpy as np

p = np.array([.7, .2, .1])
q = np.array([.5, .3, .2])
entropy = -(p*np.log(p)).sum()
cross_entropy = -(p*np.log(q)).sum()
kl = (p*np.log(p/q)).sum()
reverse_kl = (q*np.log(q/p)).sum()
assert np.allclose(cross_entropy - entropy, kl) and kl >= 0
print("H(p), H(p,q), KL(p||q), KL(q||p) in nats:",
      np.round([entropy, cross_entropy, kl, reverse_kl], 4))
print("H(p) in bits:", (entropy/np.log(2)).round(4))

# Mutual information from a joint table (same weather/umbrella table)
P = np.array([[0.56, 0.14],
              [0.06, 0.24]])
px, py = P.sum(1), P.sum(0)
mi = (P*np.log(P/np.outer(px, py))).sum()
H = lambda t: -(t[t > 0]*np.log(t[t > 0])).sum()
assert np.isclose(mi, H(px) + H(py) - H(P.ravel()))
print("I(weather; umbrella) =", mi.round(4), "nats")

# Cross-entropy loss on one-hot labels equals mean -log q(y)
probs = np.array([[0.7, 0.2, 0.1], [0.1, 0.8, 0.1], [0.3, 0.3, 0.4]])
labels = np.array([0, 1, 2])
print("mean cross-entropy:", (-np.log(probs[np.arange(3), labels])).mean().round(4))
```

輸出：$H(p) = 0.8018$、$H(p,q) = 0.8869$、$D_{\mathrm{KL}}(p\|q) = 0.0851$、$D_{\mathrm{KL}}(q\|p) = 0.0920$ nats——兩個方向的值不同。$H(p)$ 為 1.1568 位元。天氣與帶傘的互資訊為 0.1637 nats，兩種公式算出的結果一致。三筆預測的平均交叉熵為 0.4987 nats。在真正的訓練程式中，請用融合函數直接從 logit 計算交叉熵，而不是先算機率（見ML 01C）。

結構化機率模型

白話版。多個變數的聯合分布大到無法寫成表格。結構化模型畫一張圖，標明哪些變數直接影響哪些變數，聯合分布就能拆成許多小塊。

100 個二元變數的聯合機率表需要 $2^{100} - 1$ 個數字。**結構化機率模型**，又稱**圖模型**（graphical models），利用條件獨立把聯合分布分解成小因子 [1], [3]。主要有兩類。

有向模型（貝氏網路）使用有向無環圖 $\mathcal{G}$。每個變數只依賴它的父節點 $Pa_{\mathcal{G}}(x_i)$：

$$p(\mathbf{x}) = \prod_i p(x_i \mid Pa_{\mathcal{G}}(x_i)).$$

例如在 $a \to b \to c$ 且 $a \to c$ 時，$p(a,b,c) = p(a)\,p(b \mid a)\,p(c \mid a,b)$；刪掉邊 $a \to c$ 就是宣稱 $c \perp a \mid b$，最後一個因子縮短為 $p(c \mid b)$。有向圖很適合描述生成故事（「先選類別，再產生像素」），而混合模型 $z \to \mathbf{x}$ 是最簡單的例子。

無向模型（馬可夫隨機場）使用無向圖。每個**團**（clique）$C^{(i)}$（彼此全部相連的節點集合）有一個非負的**因子** $\phi^{(i)}$，且

$$p(\mathbf{x}) = \frac{1}{Z}\prod_i \phi^{(i)}(C^{(i)}), \qquad Z = \sum_{\mathbf{x}} \prod_i \phi^{(i)}(C^{(i)}).$$

配分函數 Z 用來正規化乘積；計算它往往是最困難的部分。無向模型適合對稱的交互作用，例如影像先驗中相鄰像素的關係，或用於分割的條件隨機場。

圖是對分布的一種描述，不是另一種分布：任何分布都可以用兩種方式寫出，只是其中一種可能精簡得多。深度學習常隱含地使用這些概念——自迴歸模型是全連接的有向模型，VAE 則是條件分布由神經網路表示的雙節點有向模型 $\mathbf{z} \to \mathbf{x}$ [8]。

現代觀點

本篇的機率與資訊理論是古典內容。深度學習時代改變的是它如何被使用，以及實務上會在哪裡出錯。本節綜覽相關綜述文獻與近年方向；只整理它們的框架，不轉述基準數字。

校準：預測機率誠實嗎？

如果分類器在所有預測信心為 0.8 的輸入中，大約有 80% 是正確的，就稱它是**校準良好**的（calibrated）。Guo 等人 [11]（ICML 2017）發現現代深度網路常常**過度自信**，並指出深度、寬度、權重衰減與批次正規化都會影響校準。他們主要推薦的**溫度縮放**（temperature scaling）——把所有 logit 除以一個在驗證集上擬合的純量 T——是 Platt scaling 的單參數版本；因為它只改變信心、不改變預測類別，至今仍是強力的基準方法。Wang 的綜述 [12] 把校準方法分為四大類：事後校準、訓練期間的正則化、不確定性估計，以及這些方法的組合；其修訂版也涵蓋大型語言模型的校準。從機率的角度看，教訓是：softmax 輸出是 $P(y \mid \mathbf{x})$ 的一個模型，和任何模型一樣，它的不確定性也可能是錯的，而不只是 argmax 會錯。

不確定性量化

兩篇主要綜述描繪了深度學習中不確定性研究的全貌。Abdar 等人 [13]（*Information Fusion*，2021）回顧了貝氏近似、集成方法以及它們在許多領域的應用。Gawlikowski 等人 [14]（*Artificial Intelligence Review*，2023）區分了**可降低的模型（認知，epistemic）不確定性**與**不可降低的資料（偶然，aleatoric）不確定性**——分別對應本篇開頭所說的第三與第一種不確定性來源——並比較了確定性單一網路、貝氏方法、集成與測試時資料擴增，以及校準的量測方式。

有兩篇原始研究特別有影響力。**深度集成**（deep ensembles）[15] 從不同的隨機初始化訓練多個網路，再平均它們的預測分布；這個簡單、可平行化的方法與近似貝氏神經網路表現相當，並且在分布外（out-of-distribution）輸入上表現出較高的不確定性。Ovadia 等人 [16]（NeurIPS 2019）在**資料集偏移**下評測不確定性方法，發現以分布內驗證資料做的事後校準難以延續，而對多個模型做邊際化的方法（如集成）表現較穩定。

另一種假設較少的互補方法是**共形預測**（conformal prediction）。Angelopoulos 與 Bates [17] 提供了一份淺顯的入門：利用一組保留的校準資料，共形方法能把任何預訓練模型的分數轉成預測集合，並以使用者指定的機率包含真實標籤，前提是校準資料與測試資料可交換（exchangeable）。它不要求模型機率本身已經校準，因此對被當成黑盒子使用的視覺與語言系統很有吸引力。

表示學習中的互資訊

互資訊曾是自監督學習（ML 05）中熱門的目標。對比預測編碼（Contrastive Predictive Coding）[18] 提出了 **InfoNCE** 損失：它在多個負樣本中辨認出正樣本對，可以證明是在最大化互資訊的一個下界。Poole 等人 [19]（ICML 2019）統一了許多變分互資訊界，並指出當真實互資訊很大時這些界會退化，在高偏差與高變異數之間取捨。Tschannen 等人 [20]（ICLR 2020）主張這些方法的成功「不能只歸因於互資訊本身的性質」：編碼器架構與估計器的參數化至少同樣重要。現在的共識是把 InfoNCE 類的目標視為有資訊理論動機、行為良好的對比損失，而不是精確的互資訊估計器。

生成模型中的散度與潛在變數

生成建模就是應用機率。VAE [8] 最大化證據下界（evidence lower bound），它結合了重建概似與一個 KL 項；Blei 等人 [10] 回顧了變分推論，即在一個容易處理的分布族中最小化（反向）KL 來近似難解的後驗。正規化流 [9] 完全建立在變數變換公式上。擴散模型與流匹配模型（ML 07）結合了高斯雜訊、條件期望值與概似界。選擇哪一種散度（正向 KL、反向 KL 或其他）決定了模型是涵蓋全部資料，還是集中在少數眾數上——與圖 5 是同一幅圖像，只是規模大得多。

尚待解決的問題

- **分布偏移下的校準**。在驗證集上成立的校準，換到新的相機、醫院或提示詞時可能失效 [16]；如何偵測這種情況仍是開放問題。
- **區分不確定性類型**。把預測不確定性拆成認知與偶然兩部分，在概念上很清楚，但對大型網路很難可靠地做到 [14]。
- **高維資訊量的估計**。變分互資訊界有根本的偏差–變異數限制 [19]，因此對深度表示做資訊理論宣稱時需要格外謹慎。
- **基礎模型的不確定性**。把校準與共形方法應用到輸出開放式內容的大型視覺–語言模型與語言模型，是活躍的研究方向 [12], [17]。

重點整理

- 機率描述來自隨機性、部分觀測與簡化模型的不確定性；頻率學派與貝氏學派的解讀遵守同一套規則。
- PMF 給出機率；PDF 給出密度，而密度可以大於一。連續事件的機率是面積。
- 加法規則（邊際化）、乘法規則（條件化）與連鎖規則都是精確的；獨立與條件獨立則是建模假設。
- 共變異數只衡量線性共變——共變異數為零不代表獨立；互資訊能偵測任何相依。
- 選擇分布就是選擇損失：Bernoulli／類別分布給出交叉熵，高斯給出平方誤差，Laplace 給出絕對誤差；高斯與 Laplace 先驗分別給出 L_2 與 L_1 正則化。
- 貝氏定理把證據與基本比率結合；同一個偵測器在某個盛行率下可信，在另一個盛行率下卻可能誤導。
- 變換隨機變數時，密度要乘上 Jacobian 行列式的絕對值來重新縮放。
- 交叉熵等於熵加 KL；最小化它就是最大概似。KL 不對稱：正向 KL 涵蓋各峰，反向 KL 只追一個峰。

練習

1. **單位與確定性**。 (a) 把 1 nat 的損失換算成位元。(b) 分別計算公平硬幣、確定結果與公平八面骰的熵（以位元為單位）。

   提示

   (a) 除以 $\ln 2$：約 1.443 位元。(b) 1 位元、0 位元與 $\log_2 8 = 3$ 位元。K 個結果的均勻分布熵為 $\log K$，是 K 個結果所能達到的最大值。

2. **盛行率偏移**。使用貝氏定理一節中的偵測器，找出讓陽性結果恰好像擲硬幣、即 $P(\text{defect} \mid +) = 0.5$ 的盛行率。

   提示

   令盛行率為 π，解 $0.9\pi = 0.05(1-\pi)$，得 $\pi = 0.05/0.95 \approx 0.053$，約 5.3%。低於這個值時，大多數警報都是假的。

3. **相依但不相關**。令 X 以相同機率取 $-1, 0, 1$，且 $Y = X^2$。證明 $\operatorname{Cov}(X, Y) = 0$，並以位元計算 $I(X; Y)$。

   提示

   $\mathbb{E}[X] = 0$ 且 $\mathbb{E}[XY] = \mathbb{E}[X^3] = 0$，所以共變異數為零。因為 Y 是 X 的函數，$H(Y \mid X) = 0$，於是 $I(X;Y) = H(Y)$。$Y = 1$ 的機率為 2/3、$Y = 0$ 的機率為 1/3，所以 $I \approx 0.918$ 位元。

4. **高斯 MLE 等於平方誤差**。證明在 σ 固定時，對 $\boldsymbol{\theta}$ 最大化 $\prod_i \mathcal{N}(y^{(i)}; f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), \sigma^2)$ 等同於最小化均方誤差。如果 σ 也由網路預測，會有什麼改變？

提示

負對數概似為 $\sum_i \frac{(y^{(i)}-f_\theta(\mathbf{x}^{(i)}))^2}{2\sigma^2} + m\log\sigma + \text{const}$。$\sigma$ 固定時，只有平方誤差與 $\boldsymbol{\theta}$ 有關。如果網路也預測 $\sigma(\mathbf{x})$（例如透過 softplus），每個樣本的誤差會被 $1/\sigma^2$ 加權，而 $\log\sigma$ 懲罰項會防止變異數無限制地變大——這就是異質變異（heteroscedastic）迴歸模型。

5. **變數變換**。若 $X \sim \mathcal{U}(0,1)$（均勻分布）且 $Y = -\frac{1}{\lambda}\log X$，$\lambda > 0$，$Y$ 的密度是什麼？用 NumPy 抽樣驗證你的答案。

提示

$g^{-1}(y) = e^{-\lambda y}$ 且 $|dg^{-1}/dy| = \lambda e^{-\lambda y}$，而在 $(0,1)$ 上 $p_X = 1$。所以當 $y \geq 0$ 時 $p_Y(y) = \lambda e^{-\lambda y}$：指數分布。這就是反轉換抽樣（inverse-transform sampling）的技巧。

6. **KL 的方向**。對 $p = (0.5, 0.5, 0)$ 與 $q = (0.4, 0.4, 0.2)$，計算 $D_{\mathrm{KL}}(p\|q)$ 與 $D_{\mathrm{KL}}(q\|p)$。哪一個是無限大？這對以最大概似訓練模型有什麼啟示？

提示

$D_{\mathrm{KL}}(p\|q) = \log(0.5/0.4) \approx 0.223$ nats（第三項是 $0\log 0 = 0$）。$D_{\mathrm{KL}}(q\|p) = \infty$，因為 q 在 p 沒有質量的地方放了質量。最大概似最小化的是 $D_{\mathrm{KL}}(p_{\text{data}}\|q_{\text{model}})$；只要模型對任何一筆觀察到的樣本給出零機率，它就會變成無限大——所以以概似訓練的模型必須在資料可能出現的每個地方都分配一些機率。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 3 “Probability and Information Theory”. Book
2. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
3. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book
4. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book
5. C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948. DOI
6. S. Kullback and R. A. Leibler, “On Information and Sufficiency,” *The Annals of Mathematical Statistics*, vol. 22, pp. 79–86, 1951. DOI
7. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006. DOI
8. D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *Proc. International Conference on Learning Representations (ICLR)*, 2014. arXiv
9. G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed and B. Lakshminarayanan, “Normalizing Flows for Probabilistic Modeling and Inference,” *Journal of Machine Learning Research*, vol. 22, no. 57, pp. 1–64, 2021. arXiv
10. D. M. Blei, A. Kucukelbir and J. D. McAuliffe, “Variational Inference: A Review for Statisticians,” *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, 2017. DOI

11. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” *ICML*, 2017. arXiv
12. C. Wang, “Calibration in Deep Learning: A Survey of the State-of-the-Art,” arXiv:2308.01222, 2023 (revised 2025). arXiv
13. M. Abdar et al., “A Review of Uncertainty Quantification in Deep Learning: Techniques, Applications and Challenges,” *Information Fusion*, vol. 76, pp. 243–297, 2021. DOI
14. J. Gawlikowski et al., “A Survey of Uncertainty in Deep Neural Networks,” *Artificial Intelligence Review*, vol. 56, pp. 1513–1589, 2023. DOI
15. B. Lakshminarayanan, A. Pritzel and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles,” *NeurIPS (NIPS)*, 2017. arXiv
16. Y. Ovadia et al., “Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift,” *NeurIPS*, 2019. arXiv
17. A. N. Angelopoulos and S. Bates, “A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification,” arXiv:2107.07511, 2021. arXiv
18. A. van den Oord, Y. Li and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” arXiv:1807.03748, 2018. arXiv
19. B. Poole, S. Ozair, A. van den Oord, A. A. Alemi and G. Tucker, “On Variational Bounds of Mutual Information,” *ICML*, 2019. arXiv
20. M. Tschannen, J. Djolonga, P. K. Rubenstein, S. Gelly and M. Lucic, “On Mutual Information Maximization for Representation Learning,” *ICLR*, 2020. arXiv

ML 01C・數值計算與梯度檢查

浮點數、溢位與下溢、穩定的 softmax、條件數、梯度、Jacobian 與 Hessian、牛頓類方法、KKT 條件與混合精度：讓數學模型在真實計算中站得住腳。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-01c-numerical-computation/

你將學到

- 電腦如何儲存實數、機器 epsilon 的意義，以及 float16、bfloat16、float32、float64 為何表現不同。
- 正確的公式為何會溢位、下溢或遺失有效位數，以及如何穩定地計算 softmax、log-sum-exp 與 logistic 損失。
- 條件數衡量什麼，以及「問題條件不良」與「演算法不穩定」的差別。
- 梯度、Jacobian 與 Hessian 如何描述損失曲面，梯度下降為何會鋸齒前進，牛頓法與擬牛頓法又改變了什麼。
- 如何用投影、廣義拉格朗日函數與 KKT 條件處理限制條件。
- 如何檢查導數、除錯非有限值，並安全地使用混合精度。

先看全貌

機器學習用實數的語言寫成，但電腦沒有真正的實數。它只有有限個浮點數，每一次算術運算都會把結果捨入到其中一個。大多數時候這種捨入看不見；有時卻是災難：指數溢位成無限大、極小的機率變成剛好 0 而它的對數變成負無限大，或是兩個幾乎相等的數相減，絕大部分有效位數就此消失。

故事的另一半是最佳化。訓練模型就是最小化一個有數百萬甚至數十億個參數的函數，而我們只能用數值方法做到：依照導數的指引走很多小步。這些步伐前進得多快、多可靠，取決於損失曲面的形狀（斜率與曲率），也取決於我們怎麼走。本篇以 Goodfellow、Bengio 與 Courville 的「Numerical Computation」一章 [1] 為骨架，補上現代低精度訓練所依賴的浮點細節，並用小型 NumPy 實驗串起每個概念。

> **用最簡單的話說**
>
> 想像一台只能顯示七位數的計算機。你問它一百萬加 0.1，它顯示一百萬：0.1 從螢幕上掉下去了。接著再減掉一百萬，你得到 0 而不是 0.1。好的數值程式會安排計算步驟，讓重要的位數永遠不會掉出螢幕。最佳化則是另一種遊戲：你蒙著眼站在起伏的田野上，想走到最低點。你能用腳感覺坡度（梯度），多花點力氣還能感覺地面怎麼彎（Hessian）。數值計算要做的，就是在計算機不騙你的前提下，安全地走下山。

如果 softmax 與交叉熵對你還很陌生，請先讀ML 01B；特徵值與奇異值分解則可隨時回頭參考ML 01A。之後的ML 02 會用到本篇的所有內容來訓練神經網路。

浮點數

白話版。浮點數儲存一個正負號、幾位有效數字和一個 2 的次方。相鄰浮點數之間的相對間距大致固定，所以大數的絕對捨入誤差大，小數的絕對捨入誤差小。

浮點數如何儲存

IEEE 754 標準 [3] 定義了幾乎所有 CPU 與 GPU 使用的二進位浮點格式。一個正規（非特殊值）數字儲存為

$$x = (-1)^s \times \left(1 + \sum_{i=1}^{t} b_i\, 2^{-i}\right) \times 2^{e},$$

其中 $s \in \{0,1\}$ 是正負號位元，$b_1, \dots, b_t \in \{0,1\}$ 是有效數（significand）中儲存的 t 個小數位元（開頭的 1 是隱含的），e 是限制在固定範圍內的整數指數。最重要的兩個數：

- **機器 epsilon** $\varepsilon_{\text{mach}} = 2^{-t}$：1 與下一個可表示的較大數之間的間距。
- **單位捨入誤差**（unit roundoff）$u = \varepsilon_{\text{mach}}/2$：「捨入到最近值」的相對誤差上界。對正規範圍內的任意實數 x，儲存值為 $\mathrm{fl}(x) = x(1+\delta)$，且 $|\delta| \le u$。

標準也要求每個基本運算 $(+, -, \times, \div, \sqrt{\cdot})$ 都像先精確計算再捨入一次那樣：$\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta)$，$|\delta| \le u$。這條規則就是捨入誤差分析的基礎 [2]、[4]。

比最小正規數更小的範圍，格式改用**次正規數**（subnormal），犧牲前導精度、讓數值逐步降到 0。特殊值則補足整體：結果太大時為 $\pm\infty$；無效運算如 $0/0$、$\infty - \infty$、$\infty \times 0$ 則得到 **NaN**（not a number）。NaN 一旦出現，幾乎所有碰到它的運算都會回傳 NaN，這就是為什麼單一個壞值就能毀掉整次訓練。

你會遇到的四種格式

下表以 `numpy.finfo` 計算（bfloat16 因 NumPy 不支援，改用 `torch.finfo`）。「位元」欄列出正負號／指數／小數位元數。

格式	位元	$\varepsilon_{\text{mach}}$	最大有限值	最小正規數	最小次正規數
float16（IEEE 半精度）	1/5/10	9.77×10^{-4}	65504	6.10×10^{-5}	5.96×10^{-8}
bfloat16	1/8/7	7.81×10^{-3}	3.39×10^{38}	1.18×10^{-38}	9.18×10^{-41}
float32（單精度）	1/8/23	1.19×10^{-7}	3.40×10^{38}	1.18×10^{-38}	1.40×10^{-45}
float64（雙精度）	1/11/52	2.22×10^{-16}	1.80×10^{308}	2.23×10^{-308}	4.94×10^{-324}

這張表要從兩個方向讀。**指數位元**決定範圍：float16 超過 65504 就溢位，而 bfloat16 保留了 float32 的 8 個指數位元，所以範圍與 float32 相同。**小數位元**決定精度：float32 約有 7 位有效十進位數字，float64 約 16 位，float16 約 3 位，bfloat16 只有約 2 位。bfloat16「範圍寬、精度粗」

與 float16「範圍窄、精度較細」的取捨，正是兩種半精度格式需要不同訓練做法的原因（見後文混合精度一節與 [5]、[12]）。

```
import numpy as np

for dtype in (np.float16, np.float32, np.float64):
    info = np.finfo(dtype)
    print(f"{info.dtype}: eps={float(info.eps):.3e}  max={float(info.max):.3e}  "
          f"tiny={float(info.tiny):.3e}  subnormal={float(info.smallest_subnormal):.3e}")

# Rounding: 1 + eps/2 rounds back to 1 in float32
e = np.finfo(np.float32).eps
print(np.float32(1) + np.float32(e/2) == np.float32(1))   # True
print(np.float32(1) + np.float32(e) == np.float32(1))     # False

# Addition is not associative
a, b, c = np.float32(1e8), np.float32(-1e8), np.float32(1.0)
print((a + b) + c, a + (b + c))                           # 1.0 0.0
```

最後一行值得停下來想。精確算術下兩種順序都得到 1；但在 float32 中，10^8 附近相鄰浮點數的間距是 8，所以 $-10^8 + 1$ 會捨入回 -10^8，那個 1 就不見了。實務上的後果是：用不同順序加總同一組數字（例如換了 GPU 數量，或換了歸約〔reduction〕核心），最後幾位數就可能不同。這是每次執行結果不完全一致的主要來源之一，而且不是你程式的錯 [13]。

溢位、下溢與消去

白話版。溢位是結果大到存不下；下溢是結果小到變成 0（或以次正規數形式失去精度）。消去（cancellation）是兩個幾乎相等的數相減：捨入誤差留了下來，真正的訊號卻被丟掉。

溢位與下溢

在機器學習中，最常見的元兇是指數函數。只要 $z > \ln(3.40 \times 10^{38}) \approx 88.7$，$\exp(z)$ 就超過 float32 的最大值，因此 logit 為 100 時在 float32 中會溢位（在 float16 中，logit 約 11.1 就會溢位，因為 $\ln 65504 \approx 11.09$）。另一方向，$\exp(-110) \approx 1.7 \times 10^{-48}$ 比 float32 最小的次正規數還小，結果剛好是 0。這兩種情況預設都不會報錯，只會產生 `inf` 與 `0`，而下一個運算就把它們變成 `nan` 或 $-\infty$。

```
import numpy as np

with np.errstate(over="ignore", under="ignore", invalid="ignore"):
    print(np.exp(np.float32(100.0)))        # inf: overflow
    print(np.exp(np.float32(-110.0)))       # 0.0: underflow
    big = np.exp(np.float32(100.0))
    print(big / big)                        # nan: inf / inf

# Catastrophic cancellation and its cure
```

```
x = 1e-10
print(np.log(1 + x), np.log1p(x))          # roundoff vs accurate
print(np.exp(x) - 1, np.expm1(x))

# Variance: the one-pass formula cancels badly when the mean is large
rng = np.random.default_rng(0)
data = (1e4 + rng.normal(0, 1, 100_000)).astype(np.float32)
one_pass = np.mean(data**2) - np.mean(data)**2
two_pass = np.mean((data - data.mean())**2)
print("one-pass:", one_pass, " two-pass:", two_pass)
```

在這台機器上，單趟（one-pass）變異數得到 −8.0——負的變異數——而雙趟（two-pass）公式給出正確的約 1.0。資料平均值為 10^4、變異數為 1，所以 $\mathbb{E}[X^2] \approx 10^8$、$(\mathbb{E}X)^2 \approx 10^8$；在這個量級上 float32 只能分辨約 8 的差異，真正的差值 1 完全淹沒在雜訊裡。

為什麼減法是危險的運算

假設 $\hat{a} = a(1+\delta_a)$、$\hat{b} = b(1+\delta_b)$ 的儲存相對誤差滿足 $|\delta_a|, |\delta_b| \le u$。即使減法本身是精確的，

$$\frac{|(\hat{a}-\hat{b}) - (a-b)|}{|a-b|} = \frac{|a\delta_a - b\delta_b|}{|a-b|} \le u\,\frac{|a|+|b|}{|a-b|}.$$

其中 a, b 是真實值，u 是單位捨入誤差。當 $a \approx b$ 時，因子 $(|a|+|b|)/|a-b|$ 非常大：減法並沒有製造誤差，而是把兩數共有的位數消掉，暴露出輸入原本就帶有的誤差。解法是代數上的：改寫公式，讓這個減法根本不發生。你會用到的例子：

- `log1p(x)` 計算 $\log(1+x)$、`expm1(x)` 計算 $e^x - 1$，在 x 很小時都很精確，天真寫法則幾乎失去所有位數（見上方輸出）。
- 變異數：先減去平均值（雙趟），或使用 Welford 的串流更新。
- Softmax 與 log-sum-exp：先減去最大值，下一節說明。
- 距離：$\|x-y\|^2 = \|x\|^2 + \|y\|^2 - 2x^\top y$ 很快，但在 $x \approx y$ 時嚴重消去；需要精確時，把結果截在 0 以上，或直接計算 $x - y$。

穩定的 softmax 與 log-sum-exp

白話版。取指數前先減去一個共同常數；對數機率要直接計算，而不是先產生極小的機率再取對數。

Softmax 把 logit 向量 $z \in \mathbb{R}^K$ 轉成機率：

$$\operatorname{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad i = 1, \dots, K.$$

把同一個常數 c 加到每個 logit 上，softmax 不變，因為因子 $\exp(c)$ 在分子與分母間抵銷。取 $c = -m$，其中 $m = \max_i z_i$：

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}.$$

現在每個指數都 ≤ 0，不會溢位；最大的一項剛好是 $\exp(0) = 1$，所以分母至少為 1，也不會下溢成 0。同樣的技巧也能穩定 **log-sum-exp** 函數：

$$\mathrm{LSE}(z) = \log \sum_j \exp(z_j) = m + \log \sum_j \exp(z_j - m), \qquad \log p_i = z_i - \mathrm{LSE}(z).$$

LSE 是最大值的平滑版本：$\max_i z_i \le \mathrm{LSE}(z) \le \max_i z_i + \log K$；它的梯度恰好就是 softmax，$\partial\,\mathrm{LSE}/\partial z_i = p_i$。

對目標類別 y，交叉熵損失為 $-\log p_y = \mathrm{LSE}(z) - z_y$，它對 logit 的梯度是那個著名的簡單形式 $p - e_y$，其中 e_y 是目標的 one-hot 向量。先算 `softmax` 再取 `log` 仍可能失敗：若 p_y 下溢成 0，它的對數就是 $-\infty$，即使 $\mathrm{LSE}(z) - z_y$ 是個完全正常的有限數。這就是框架提供接受 logit 的融合函數 `log_softmax` 與 `cross_entropy` 的原因。

程式：會讓天真指數運算溢位的 logit

這個獨立的 NumPy 範例檢查穩定的機率、對數機率，以及平移 logit 的不變性。

```
import numpy as np

def logsumexp(z):
    m = np.max(z)
    return m + np.log(np.exp(z-m).sum())
def logsoftmax(z):
    return z-logsumexp(z)

z = np.array([1000., 1001., 999.], dtype=np.float64)
logp = logsoftmax(z)
p = np.exp(logp)
assert np.allclose(p.sum(), 1)
assert np.allclose(logsoftmax(z+12345), logp)
print("probabilities:", p)
print("cross-entropy for class 1:", -logp[1])
with np.errstate(over="ignore", invalid="ignore"):
    naive = np.exp(z) / np.exp(z).sum()
print("naive softmax:", naive)
```

穩定版輸出機率 $[0.245, 0.665, 0.090]$，類別 1 的交叉熵為 0.408 nats；天真版輸出 `[nan nan nan]`，因為 $\exp(1000)$ 即使在 float64 也會溢位（float64 的上限是 $\exp(709.78)$）。

穩定的 logistic（二元交叉熵）損失

二元分類以另一種形式遇到同樣的問題。設 logit 為 z、標籤 $y \in \{0, 1\}$、sigmoid 為 $\sigma(z) = 1/(1 + e^{-z})$，損失 $-y \log \sigma(z) - (1 - y) \log(1 - \sigma(z))$ 可化簡為

$$\ell(z, y) = \mathrm{softplus}(z) - yz, \qquad \mathrm{softplus}(z) = \log(1 + e^{z}) = \max(z, 0) + \log\!\left(1 + e^{-|z|}\right).$$

右邊的寫法從不對正數取指數，`log1p` 則讓小修正項保持精確。請優先使用 `binary_cross_entropy_with_logits`，不要自己先算 sigmoid 再取對數。

```
import numpy as np

def bce_naive(z, y):
    s = 1 / (1 + np.exp(-z))
    return -(y*np.log(s) + (1-y)*np.log(1-s))

def bce_stable(z, y):
    # softplus(z) - y*z, with softplus(z) = max(z,0) + log1p(exp(-|z|))
    return np.maximum(z, 0) + np.log1p(np.exp(-np.abs(z))) - y*z

z = np.array([-800., -30., 0., 30., 800.])
y = np.array([1., 1., 0., 0., 0.])
with np.errstate(over="ignore", divide="ignore", invalid="ignore"):
    print("naive :", bce_naive(z, y))
print("stable:", bce_stable(z, y))
```

天真版對兩個「信心十足的錯誤」（$z = \pm 800$）回傳 `inf`，在 $z = 30$ 時回傳 30.001 而非 30.000，因為 $1 - \sigma(30)$ 是靠消去算出來的。穩定版回傳 800、30、$\log 2$、30、800，全部正確。

條件不良

白話版。有些問題天生敏感：輸入的微小改變會造成答案的巨大改變，不論你算得多小心。**條件數**（condition number）就是衡量這種敏感度的量。

線性系統的條件數

考慮對可逆方陣 A 解 $Ax = b$。若資料變成 $b + \delta b$，解變成 $x + \delta x$，則

$$\frac{\|\delta x\|}{\|x\|} \le \kappa(A)\,\frac{\|\delta b\|}{\|b\|}, \qquad \kappa(A) = \|A\|\,\|A^{-1}\| = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \ \text{(2-norm)}.$$

其中 $\|\cdot\|$ 是歐氏向量範數及其誘導矩陣範數，$\sigma_{\max}, \sigma_{\min}$ 是 A 的最大與最小奇異值。對稱正定矩陣的奇異值等於特徵值，因此 $\kappa = \lambda_{\max}/\lambda_{\min}$。Goodfellow 等人用特徵分解對函數 $f(x) = A^{-1}x$ 陳述同一件事：它的條件數是 $\max_{i,j} |\lambda_i/\lambda_j|$ [1]。實用的經驗法則：$\kappa \approx 10^{k}$ 時，相對於輸入的精度，大約會損失 k 位十進位精度 [4]。

```
import numpy as np

A = np.array([[1.0, 1.0],
              [1.0, 1.0001]])
print("condition number:", np.linalg.cond(A))
```

```
b = np.array([2.0, 2.0001])
x = np.linalg.solve(A, b)
db = np.array([0.0, 0.0001])                # tiny change in the data
x2 = np.linalg.solve(A, b + db)
rel_in = np.linalg.norm(db) / np.linalg.norm(b)
rel_out = np.linalg.norm(x2 - x) / np.linalg.norm(x)
print("x:", x, " x after perturbation:", x2)
print(f"relative change in b: {rel_in:.1e}, in x: {rel_out:.1e}, "
      f"amplification: {rel_out/rel_in:.0f}")
```

A 的兩欄幾乎相同，$\kappa(A) \approx 4\times10^4$；$b$ 只有 3.5×10^{-5} 的相對改變，解卻從 $(1,1)$ 移到 $(0,2)$——百分之百的改變，放大約 28,000 倍，仍在 κ 的上界之內。沒有任何計算出錯，是這個問題本身條件不良。

條件不良的問題 vs. 不穩定的演算法

請把兩個概念分開：

- **條件不良的問題**會放大輸入的小變化。任何演算法都無法修正；你需要更好的資料、正則化（它改變了問題）或重新表述問題。
- **不穩定的演算法**會替一個可能條件良好的問題加入可避免的誤差。天真的 softmax 與單趟變異數就是不穩定的演算法，換演算法就能解決。

最小平方法同時展示兩者。如果特徵幾乎共線，係數本身就是條件不良的（問題）。解正規方程 $X^\top X w = X^\top y$ 會讓情況比必要的更糟，因為 $\kappa(X^\top X) = \kappa(X)^2$（演算法）；基於 QR 或 SVD 的求解器則直接處理 X。Ridge 迴歸加入 $\lambda\|w\|^2$，把問題改成 $(X^\top X + \lambda I)w = X^\top y$，其條件數 $(\sigma_{\max}^2 + \lambda)/(\sigma_{\min}^2 + \lambda)$ 較小——ML 01A 會進一步發展這個主題。

以梯度為基礎的最佳化

白話版。要最小化一個函數，就看它的斜率、往下坡走。梯度指向上升最陡的方向，所以往反方向走；步伐要夠小，讓你剛量到的斜率大致仍然有效。

導數、梯度與方向導數

我們要最小化**目標函數**（也稱成本、損失或誤差函數）$f:\mathbb{R}^n \to \mathbb{R}$。單變數時，導數 $f'(x)$ 給出一階近似 $f(x+\epsilon) \approx f(x) + \epsilon f'(x)$，所以往 $f'(x)$ 的反號方向走一小步就能讓 f 下降。多變數時，**梯度** $\nabla_x f(x) \in \mathbb{R}^n$ 收集所有偏導數 $\partial f/\partial x_i$。單位方向 u 上的**方向導數**是沿 u 的斜率：

$$\frac{\partial}{\partial \alpha} f(x+\alpha u)\Big|_{\alpha=0} = u^\top \nabla_x f(x) = \|\nabla_x f(x)\|_2 \cos\theta,$$

其中 θ 是 u 與梯度的夾角。當 $\theta = \pi$，也就是 u 指向 $-\nabla_x f(x)$ 時，它最負。這就是**最陡下降法**，也就是**梯度下降**：

$$x \leftarrow x - \eta \nabla_x f(x),$$

其中 $\eta > 0$ 是**學習率**（步長）。η 可以取一個小常數、可以試幾個值並保留目標函數最低者（**線搜尋**，line search），也可以依照排程變化。當梯度的每個分量都（接近）0 時，梯度下降就收斂了。

臨界點：極小值、極大值與鞍點

滿足 $\nabla_x f(x) = 0$ 的點稱為**臨界點**或**駐點**。**局部極小值**比所有鄰近點都低，**局部極大值**比所有鄰近點都高，**鞍點**兩者皆非：它沿某些方向是極小、沿另一些方向是極大。**全域極小值**是整個定義域上的最低值；一個函數可能有好幾個全域極小值，也可能有許多不是全域極小的局部極小值。

對深度網路，我們很少找到真正的全域極小值，也不需要；只要參數讓損失夠低而且泛化良好就行。在高維空間中，對許多「類隨機」函數而言，鞍點比差勁的局部極小值更常見，而且因為梯度在鞍點附近變小，它們會拖慢以梯度為基礎的最佳化器 [8]。圖 1 顯示三種臨界點。

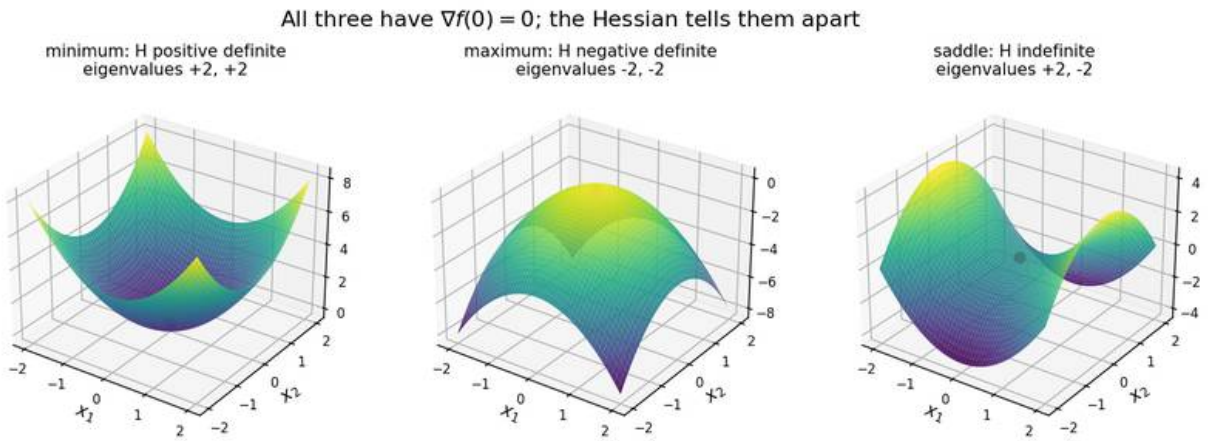


圖 *1* — 三個在原點梯度為零的函數。*Hessian* 的特徵值可以區分它們：全正（極小值）、全負（極大值）、正負皆有（鞍點）。

Jacobian 與 Hessian 矩陣

白話版。 Jacobian 收集向量值函數的所有一階導數。Hessian 收集純量函數的所有二階導數：它告訴你斜率本身如何變化，也就是曲率。

定義

對函數 $f : \mathbb{R}^m \to \mathbb{R}^n$，**Jacobian 矩陣** $J \in \mathbb{R}^{n \times m}$ 的元素為

$$J_{ij} = \frac{\partial f_i(x)}{\partial x_j}, \qquad i = 1, \dots, n,\ j = 1, \dots, m.$$

第 i 列是輸出 f_i 的梯度。反向傳播就是不斷乘上 Jacobian；反向模式自動微分計算向量–Jacobian 乘積 $v^\top J$，卻從不顯式建出 J，這就是為什麼一次梯度的成本只是一次函數求值的小常數倍 [11]。

對純量函數 $f : \mathbb{R}^n \to \mathbb{R}$，**Hessian 矩陣** $H(f)(x) \in \mathbb{R}^{n \times n}$ 是梯度的 Jacobian：

$$H(f)(x)_{ij} = \frac{\partial^2 f(x)}{\partial x_i \, \partial x_j}.$$

只要二階偏導數連續，微分順序就不影響結果，所以 $H_{ij} = H_{ji}$：Hessian 是對稱矩陣。實對稱矩陣有實數特徵值 $\lambda_1, \dots, \lambda_n$ 與一組正交歸一的特徵向量 $v_1, \dots, v_n$。沿單位方向 d 的二**階方向導數**是 $d^\top H d$。若 d 是特徵向量 v_i，它等於 λ_i；其他方向則是特徵值的加權平均 $\sum_i \lambda_i (d^\top v_i)^2$，權重介於 0 與 1 之間且總和為 1。所以 $\lambda_{\max}$ 是任一方向上的最大曲率，$\lambda_{\min}$ 是最小曲率。

一步梯度到底做了什麼：二階泰勒展開

令 $g = \nabla_x f(x^{(0)})$、$H = H(f)(x^{(0)})$。在 $x^{(0)}$ 附近的二階泰勒近似為

$$f(x) \approx f(x^{(0)}) + (x - x^{(0)})^\top g + \tfrac{1}{2}(x - x^{(0)})^\top H (x - x^{(0)}).$$

代入一步梯度 $x = x^{(0)} - \eta g$ 得到

$$f(x^{(0)} - \eta g) \approx f(x^{(0)}) - \eta\, g^\top g + \tfrac{1}{2}\eta^2\, g^\top H g.$$

三項分別是目前的值、斜率預測的改善量，以及曲率的修正。若 $g^\top H g \le 0$，η 越大，近似值就越小（直到近似本身失效為止）。若 $g^\top H g > 0$，曲率項最終會勝出，近似值在下式取得最小：

$$\eta^* = \frac{g^\top g}{g^\top H g}.$$

最壞情況下 g 與 $\lambda_{\max}$ 的特徵向量同向，此時 $\eta^* = 1/\lambda_{\max}$。曲率大，就只能走小步。

二階導數檢定

在臨界點（$g = 0$），泰勒展開只剩二次項，所以由 Hessian 決定這是哪一種點：

- H **正定**（所有 $\lambda_i > 0$）：局部極小值。
- H **負定**（所有 $\lambda_i < 0$）：局部極大值。
- H 至少有一個正特徵值與一個負特徵值：**鞍點**。
- 其他情況（有些 $\lambda_i = 0$，其餘同號）：檢定無法判斷。

一維中不可能有鞍點；但在 n 維中，只要有一個特徵值的符號「不對」就是鞍點，所以隨著 n 變大，鞍點變得非常普遍 [8]。

Hessian 的條件數：梯度下降為何鋸齒前進

在二次函數 $f(w) = \frac{1}{2} w^\top H w$（$H$ 對稱正定）上，梯度下降是線性的：$w \leftarrow (I - \eta H) w$。把 w 用 H 的特徵向量展開，沿 v_i 的分量每一步都乘上 $(1 - \eta\lambda_i)$。因此：

- 只有當每個 i 都滿足 $|1 - \eta\lambda_i| < 1$，也就是 $0 < \eta < 2/\lambda_{\max}$ 時，迭代才**穩定**。
- **最慢**的是平坦方向，每步只縮小 $|1 - \eta\lambda_{\min}|$ 倍。
- 最佳固定步長 $\eta = 2/(\lambda_{\max} + \lambda_{\min})$ 讓誤差每步縮小 $(\kappa - 1)/(\kappa + 1)$ 倍，其中 $\kappa = \lambda_{\max}/\lambda_{\min}$ 是 Hessian 的條件數。

所以條件數大，表示某個方向要求極小的步伐（否則發散），另一個方向卻需要很多步才有進展。梯度下降把時間浪費在狹窄山谷的陡峭兩壁間來回彈跳——圖 2 中的鋸齒——同時沿著谷底緩慢前進。輸入特徵尺度不一致正好造成這種情況，這也是標準化與正規化層能加快訓練的原因之一。

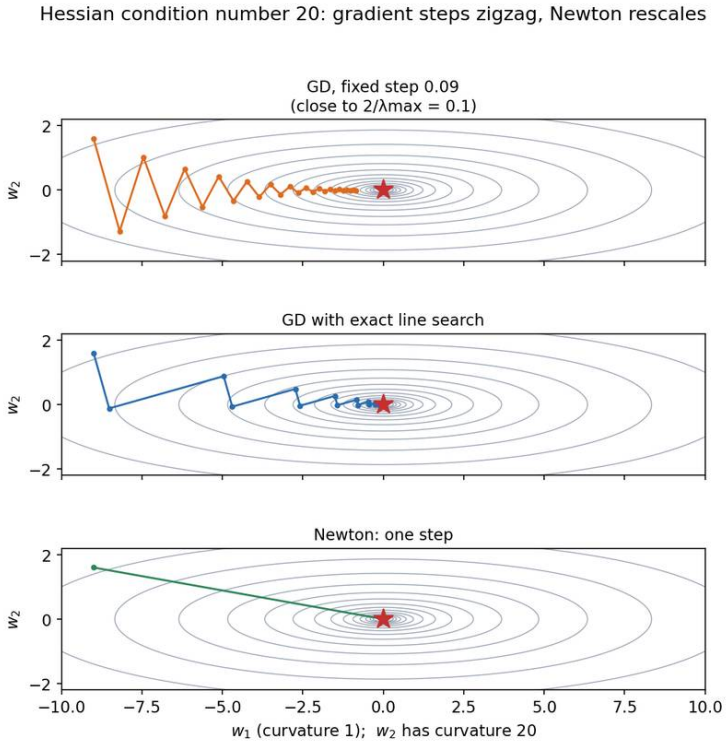


圖 2 — 最小化 *Hessian* 條件數為 *20* 的二次函數。步長接近穩定極限的梯度下降在陡峭方向來回振盪；精確線搜尋使相鄰兩步互相垂直；牛頓法以 *Hessian* 的反矩陣重新縮放，對二次函數一步就到達極小值。

圖 3 以數值呈現穩定極限。低於 $2/\lambda_{\max}$ 時損失下降；稍高於極限時，陡峭分量每步放大 $|1 - \eta\lambda_{\max}| > 1$ 倍，損失隨之爆炸。

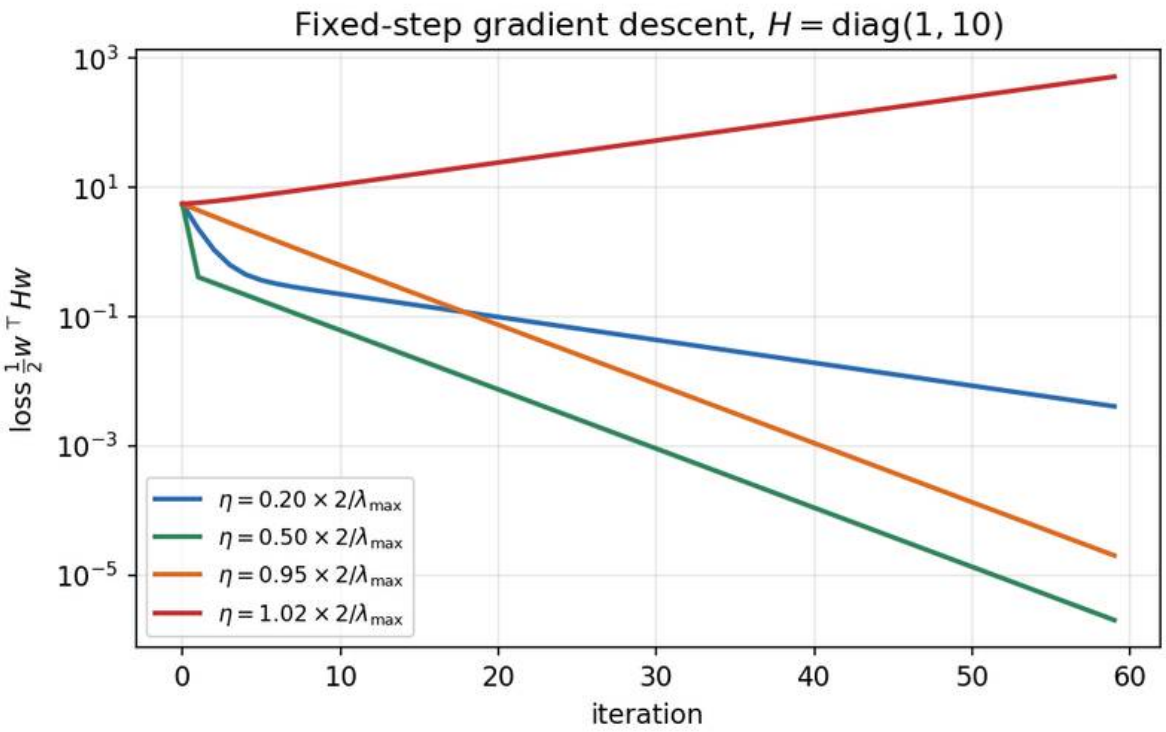


圖 *3* — 在特徵值為 *1* 與 *10* 的二次函數上做固定步長梯度下降。小於 $2/\lambda max$ 的步長都會收斂（在 $2/(\lambda max+\lambda min)$ 附近最快）；只比極限大 *2%* 的步長就會發散。

```
import numpy as np

H = np.diag([1.0, 25.0])                    # Hessian: condition number 25
f = lambda w: 0.5 * w @ H @ w
grad = lambda w: H @ w

def run_gd(eta, steps=50):
    w = np.array([10.0, 1.0])
    for _ in range(steps):
        w = w - eta * grad(w)
    return f(w)

lam_max = np.linalg.eigvalsh(H).max()
for eta in (0.01, 0.07, 0.079, 0.081):       # stability limit 2/25 = 0.08
    print(f"eta={eta:<6} loss after 50 steps: {run_gd(eta):.3e}")

# Exact line search along -g for a quadratic: eps* = g'g / g'Hg
w = np.array([10.0, 1.0])
g = grad(w)
eps_star = (g @ g) / (g @ H @ g)
print("optimal step along the gradient:", round(eps_star, 4))

# One Newton step lands on the minimizer of a quadratic
w_newton = w - np.linalg.solve(H, g)
```

```
print("after one Newton step:", w_newton)
```

輸出說明了一切：$\eta = 0.01$ 穩定但慢（50 步後損失 18.3），$\eta = 0.07$ 降到 0.035，$\eta = 0.079$ 勉強穩定但陡峭分量仍在振盪（損失約 1.0），$\eta = 0.081$ 發散（損失 148）。沿第一個梯度的最佳步長是 0.046，而一步牛頓法正好落在 $(0, 0)$。

二階方法

白話版。不只跟著斜率走，也利用曲率：依照函數在每個方向彎得多急來縮放步伐，陡的方向走小步、平的方向走大步。

牛頓法

令二階泰勒近似的梯度為零，得到**牛頓步**：

$$x^* = x^{(0)} - H(f)(x^{(0)})^{-1}\,\nabla_x f(x^{(0)}).$$

在正定二次函數上，這一步直接跳到極小值（圖 2 下）。對局部接近正定二次函數的一般函數，反覆套用這個更新比梯度下降收斂快得多——在標準假設下，於解附近呈二次收斂 [6]。代價是 Hessian：儲存它需要 $O(n^2)$ 記憶體，用它求解需要 $O(n^3)$ 時間，對一個有 10^8 個參數的網路根本不可能。

對非凸目標還有更根本的問題。牛頓法找的是梯度為零的點，它對待鞍點和極小值一視同仁：在鞍點附近，牛頓步會指向鞍點。在充滿鞍點的地形上，單純的牛頓法正好被我們想逃離的點吸引。Dauphin 等人提出「saddle-free Newton」，改用 Hessian 特徵值的絕對值來縮放，以避免這個問題 [8]。相對地，梯度下降不會以同樣方式被鞍點吸引，雖然它在鞍點附近可能走得很慢。

擬牛頓法與 L-BFGS

擬牛頓法（quasi-Newton）只用梯度建立 $B_k \approx H$（或其反矩陣）的近似。走了一步 $s_k = x_{k+1} - x_k$、梯度變化為 $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ 之後，它要求**割線條件** $B_{k+1} s_k = y_k$，意思是模型沿這一步的曲率要與觀察到的斜率變化一致。BFGS 是這類更新中最常用的；**L-BFGS**（有限記憶 BFGS）只保存最近 m 組 (s_k, y_k)，以 $O(mn)$ 時間隱式套用反 Hessian 的近似 [6]、[10]。L-BFGS 很適合平滑、確定性、中等規模的問題（logistic 迴歸、單張影像的風格轉換、擬合幾千個參數）；但當梯度是有雜訊的小批次估計時，它的曲率估計會受影響。

Hessian–向量乘積

許多利用曲率的方法其實不需要 H 本身，只需要乘積 Hv。這可以用大約幾次梯度計算的成本精確算出，因為 $Hv = \nabla_x(\nabla_x f(x)^\top v)$ 是某個純量的梯度；Pearlmutter 說明了如何用自動微分做到 [9]。有限差分版本 $Hv \approx [\nabla f(x + rv) - \nabla f(x - rv)]/(2r)$ 是很實用的檢查。以 Hessian–向量乘積做冪迭代（power iteration）可以估計 $\lambda_{\max}$，研究者正是用這個方法追蹤訓練中損失的尖銳度（sharpness）。

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(50, 4))
y = (rng.random(50) < 0.5).astype(float)

def grad(w):                                    # logistic regression, mean loss
    p = 1 / (1 + np.exp(-(X @ w)))
    return X.T @ (p - y) / len(y)

def hessian(w):
    p = 1 / (1 + np.exp(-(X @ w)))
    return (X.T * (p * (1 - p))) @ X / len(y)

w = rng.normal(size=4)
v = rng.normal(size=4)
r = 1e-5
hvp_fd = (grad(w + r*v) - grad(w - r*v)) / (2*r)   # never forms H
print("Hv (finite diff):", hvp_fd.round(6))
print("Hv (explicit)   :", (hessian(w) @ v).round(6))
print("eigenvalues of H:", np.linalg.eigvalsh(hessian(w)).round(4))
```

兩種乘積在小數點後六位都相同，而且 Hessian 的特徵值全為正：logistic 迴歸是凸的，它的 Hessian 處處半正定。

Lipschitz 連續與凸性

深度學習的目標函數幾乎沒有什麼保證，但有兩個性質能解釋大部分理論。若對所有 x, y 都有 $|f(x) - f(y)| \le L\|x - y\|_2$，則函數是常數為 L 的 **Lipschitz 連續**：它的變化率有上界。若梯度是 L-Lipschitz（函數「L-平滑」），曲率就以 L 為上界，而步長 $\eta \le 1/L$ 的梯度步保證不會讓函數變大：

$$f(x - \eta \nabla f(x)) \le f(x) - \frac{\eta}{2}\|\nabla f(x)\|_2^2 \qquad (0 < \eta \le 1/L).$$

這是前面「$\eta < 2/\lambda_{\max}$」規則的一般形式，也是大多數收斂證明的起點 [7]、[14]。

若函數（二次可微時）的 Hessian 處處半正定，它就是**凸函數**。凸函數沒有鞍點，也沒有虛假的局部極小值：每個局部極小值都是全域極小值。最小平方、logistic 迴歸與支援向量機都是凸的；深度網路不是。凸最佳化是許多工具的來源，例如對偶性與下面的 KKT 條件，深度學習也借用了它們 [7]。

有限制條件的最佳化

白話版。有時答案必須落在允許的區域內——範數有上限的權重、總和為一的機率、必須為正的物理量。要嘛走一步之後投影回區域內，要嘛為每個限制條件加上一個懲罰項，並讓它的強度自動決定。

投影梯度下降

我們要在可行集合 $\mathbb{S} \subseteq \mathbb{R}^n$ 上最小化 $f(x)$。如果投影到 $\mathbb{S}$ 很容易，就先走一般的梯度步，再回到最近的可行點：

$$x \leftarrow \operatorname{Proj}_{\mathbb{S}}\left(x - \eta \nabla f(x)\right), \qquad \operatorname{Proj}_{\mathbb{S}}(v) = \arg\min_{x \in \mathbb{S}} \|x - v\|_2 .$$

對歐氏球 $\{x : \|x\|_2 \le r\}$，投影為 $v \cdot \min(1, r/\|v\|_2)$：太長就縮短。權重的最大範數限制、把參數截在一個方盒內、投影到機率單形（simplex）都屬於這個模式。另一種做法是重新參數化，讓限制條件自動成立，例如用 $x = r\,(\cos\theta, \sin\theta)$ 表示圓上的向量，或用 softmax 產生機率。

廣義拉格朗日函數

更一般的做法是用 m 個等式限制 $g^{(i)}(x) = 0$ 與 k 個不等式限制 $h^{(j)}(x) \le 0$ 描述可行集合：

$$\mathbb{S} = \{x \mid g^{(i)}(x) = 0 \ \forall i,\ h^{(j)}(x) \le 0 \ \forall j\}.$$

為每個限制條件引入一個 **KKT 乘數**，等式用 λ_i、不等式用 α_j，並定義**廣義拉格朗日函數**

$$L(x, \lambda, \alpha) = f(x) + \sum_{i=1}^{m} \lambda_i\, g^{(i)}(x) + \sum_{j=1}^{k} \alpha_j\, h^{(j)}(x).$$

原本有限制的問題等價於沒有限制的 min–max 問題

$$\min_{x} \max_{\lambda} \max_{\alpha,\ \alpha \ge 0} L(x, \lambda, \alpha).$$

為什麼可行？若 x 違反某個限制，內層最大化可以選一個很大的乘數，把 L 推到 $+\infty$，所以不可行點永遠不會被選上。若 x 可行，內層最大化最多只能讓這些項為零（等式本來就是 0；對 $h^{(j)}(x) \le 0$，最好的非負 α_j 會使 $\alpha_j h^{(j)}(x) = 0$），因此值恰好是 $f(x)$ [1]。

KKT 條件

在解 x^* 處（在稱為「限制條件規範性」〔constraint qualification〕的溫和正則條件下），存在乘數滿足 **Karush–Kuhn–Tucker（KKT）條件** [1]、[7]：

1. **穩定性（stationarity）：** $\nabla_x L(x^*, \lambda^*, \alpha^*) = 0$。
2. **原始可行性：** x^* 滿足所有限制條件。
3. **對偶可行性：** $\alpha^* \ge 0$。
4. **互補鬆弛性：**對每個 j，$\alpha_j^*\, h^{(j)}(x^*) = 0$。

互補鬆弛性是說，每個不等式要嘛**有效**（active，$h^{(j)}(x^*) = 0$：解落在邊界上，限制條件以 $\alpha_j \ge 0$ 的力量把它推回來），要嘛**無效**（inactive，$h^{(j)}(x^*) < 0$：限制條件在局部不起作用，$\alpha_j = 0$）。從幾何上看，穩定性條件是說：在有效限制處，下坡方向 $-\nabla f$ 正好垂直指出可行集合之外，與 ∇h 平行；任何可行的移動都會往上坡走（圖 4）。對凸問題，KKT 條件也是最佳性的充分條件 [7]。

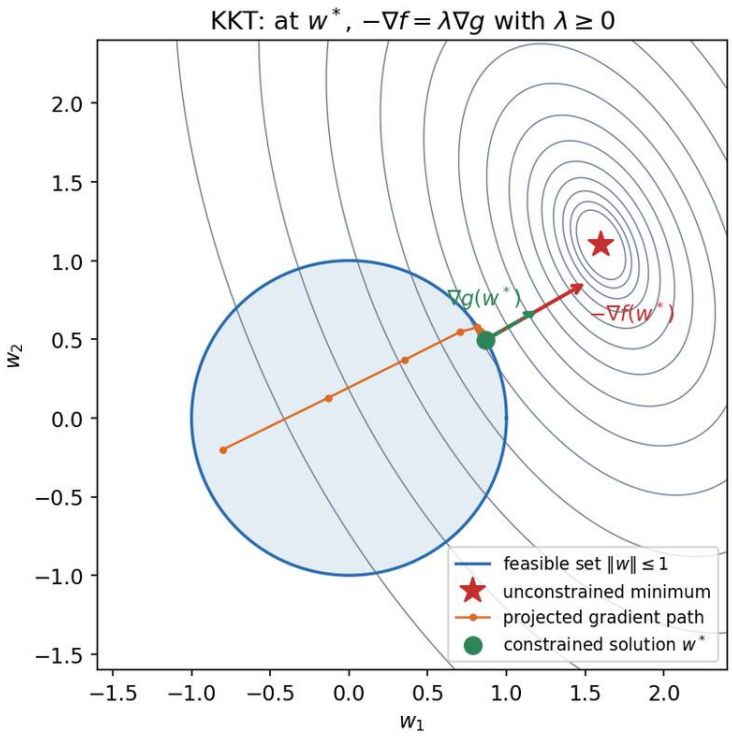


圖 4 — 在單位圓盤上最小化二次函數。無限制的極小值（星號）不可行。投影梯度下降停在邊界上的 w^*，此處 $-\nabla f$ 與限制條件的外法向量 ∇g 平行：這正是乘數為正時的 *KKT* 穩定性條件。

範例：帶範數限制的最小平方

依照 [1] 的例子，在限制 $h(w) = w^\top w - r^2 \leq 0$ 下最小化平方誤差 $f(w) = \|Aw - b\|_2^2$，其中 $A \in \mathbb{R}^{N \times n}$、$b \in \mathbb{R}^N$。拉格朗日函數為

$$L(w, \alpha) = \|Aw - b\|_2^2 + \alpha\,(w^\top w - r^2).$$

穩定性條件給出 $2A^\top(Aw - b) + 2\alpha w = 0$，所以

$$w(\alpha) = (A^\top A + \alpha I)^{-1} A^\top b.$$

這就是懲罰係數為 α 的 ridge 迴歸。若一般最小平方解 $w(0)$ 已經滿足 $\|w\| \leq r$，限制條件無效，$\alpha^* = 0$，問題解決。否則限制條件有效，就增大 α 直到 $\|w(\alpha)\| = r$；因為 $\|w(\alpha)\|$ 隨 α 單調遞減，用二分法就能找到。這也正是對 α 做內層最大化的行為：限制被違反時乘數變大，剛好滿足時停下。下面的程式用兩種方法求解，並檢查全部四個 KKT 條件。

```
import numpy as np

# minimize ||Aw - b||^2  subject to  ||w||^2 <= r^2
rng = np.random.default_rng(1)
A = rng.normal(size=(30, 5))
```

```
b = rng.normal(size=30)
r = 0.2

def w_of(lam):                                  # stationarity of the Lagrangian
    return np.linalg.solve(A.T @ A + lam * np.eye(5), A.T @ b)

w_ls = w_of(0.0)
print("unconstrained norm:", np.linalg.norm(w_ls).round(3))

lo, hi = 0.0, 1e6                               # find lambda with ||w|| = r
for _ in range(200):
    mid = 0.5 * (lo + hi)
    if np.linalg.norm(w_of(mid)) > r:
        lo = mid
    else:
        hi = mid
lam = hi
w = w_of(lam)
print("lambda:", round(lam, 3), " ||w||:", np.linalg.norm(w).round(6))

# KKT checks
stationarity = 2 * A.T @ (A @ w - b) + 2 * lam * w
print("stationarity residual:", np.abs(stationarity).max())
print("primal feasible:", w @ w <= r**2 + 1e-9, " dual feasible:", lam >= 0)
print("complementary slackness:", lam * (w @ w - r**2))

# Projected gradient descent reaches the same point
L = 2 * np.linalg.eigvalsh(A.T @ A).max()     # Lipschitz constant of the gradient
v = np.zeros(5)
for _ in range(5000):
    v = v - (1 / L) * 2 * A.T @ (A @ v - b)
    n = np.linalg.norm(v)
    if n > r:
        v = v * (r / n)                         # projection onto the ball
print("projected GD matches KKT solution:", np.allclose(v, w, atol=1e-6))
```

無限制解的範數為 0.219，大於 $r = 0.2$，所以限制條件有效。二分法找到 $\alpha^* \approx 1.81$，穩定性殘差約 10^{-15}，互補鬆弛性成立，而步長 $1/L$ 的投影梯度下降收斂到同一個向量。請注意，梯度的 Lipschitz 常數 L 決定了安全的步長，正如上一節所說。

檢查導數

白話版。檢查程式算出的梯度，是否與「輕推參數時損失實際的變化」一致。

中央有限差分估計偏導數：

$$\frac{\partial f}{\partial x_j} \approx \frac{f(x + \epsilon e_j) - f(x - \epsilon e_j)}{2\epsilon},$$

其中 e_j 是第 j 個標準基底向量，$\epsilon > 0$ 是小步長。它的**截斷誤差**（忽略高階泰勒項造成）是 $O(\epsilon^2)$，優於前向差分 $[f(x + \epsilon e_j) - f(x)]/\epsilon$ 的 $O(\epsilon)$。但分子是兩個幾乎相等的數相減，所以**捨入誤差**

以 $u|f|/\epsilon$ 的速度增長。總誤差在中等步長時最小——當 f 與其導數為單位尺度時，中央差分約在 $\epsilon \approx u^{1/3}$、前向差分約在 $u^{1/2}$。圖 5 顯示這種 V 形曲線，也說明為什麼 float32 的梯度檢查遠不如 float64 精確。

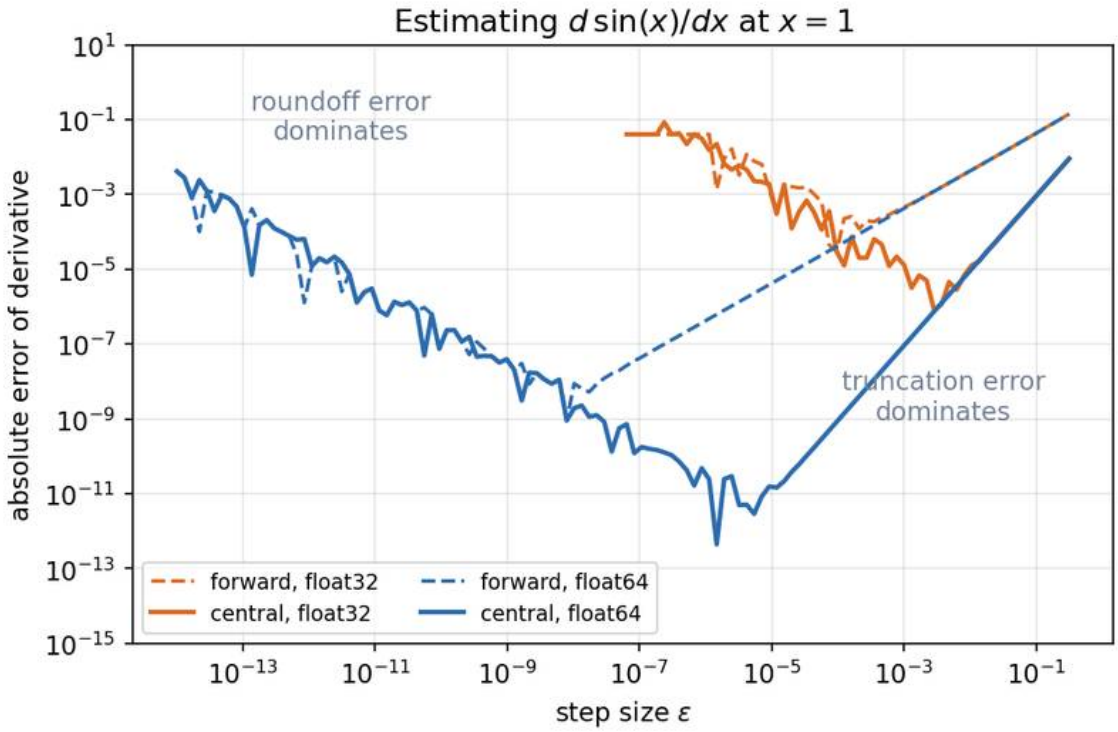


圖 *5* — 在 *x=1* 處以有限差分估計 $sin(x)$ 導數的誤差。最低點右側由截斷誤差主導（前向差分斜率 *1*、中央差分斜率 *2*）；左側由捨入誤差主導。中央差分在 *float32* 最低約 *1e−6*，在 *float64* 可低到約 *1e−11* 或更小。

實用原則：用 float64 做檢查；嘗試多個步長；用相對誤差比較，例如 $|a-n|/\max(|a|,|n|,\text{tiny})$；並在平滑的點上測試。ReLU 在 0 處不可微，那裡沒有單一導數可以比對。參數很多時，**方向檢查**更省：選一個隨機單位向量 v，比較 $\nabla f(w)^\top v$ 與沿 v 的有限差分——只需要一對函數求值，而不是 $2n$ 次。

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(20, 5))
y = rng.normal(size=20)
w = rng.normal(size=5)
v = rng.normal(size=5)
v /= np.linalg.norm(v)
def loss(w):
    return .5*np.mean((X@w-y)**2)
grad = X.T@(X@w-y)/len(y)
eps = 1e-5
numeric = (loss(w+eps*v)-loss(w-eps*v))/(2*eps)
```

```
analytic = grad@v
assert np.allclose(numeric, analytic, rtol=1e-6, atol=1e-8)
print("directional derivative:", numeric, analytic)
```

兩個數字大約有九位有效數字相同。但要記得「檢查通過」代表什麼：自動微分計算的是被記錄下來的程式的導數，而不是你心中想寫的程式 [11]。被 detach 的張量、錯誤的歸約方式、意外的廣播（broadcasting）或目標洩漏（target leakage），都會產生一個完全可微卻錯誤的目標函數，而梯度檢查會很樂意地確認它的梯度。請在已知答案的情況下，直接檢查損失值本身。

混合精度

白話版。大部分運算用 16 位元數字，可以讓記憶體傳輸量減半並使用高速硬體單元；但 16 位元不夠應付所有事情，少數敏感的量要保留在 32 位元。

Micikevicius 等人提出的混合精度訓練 [12] 結合三個想法：

1. 保留一份 **float32 主權重**（master weights）。當更新量小於所加權重的約 2^{-11} 倍時，在 float16 中加上去會被完全捨去（與上文 $1 + \varepsilon_{\text{mach}}/2 = 1$ 是同一個現象），而很多更新就是這麼小。
2. float16 要用**損失縮放**（loss scaling）：反向傳播前把損失乘上因子 S，讓小梯度不會下溢到 float16 最小次正規數（6×10^{-8}）以下，更新前再把梯度除以 S。動態損失縮放在出現溢位（inf 或 NaN）時降低 S，並跳過該步。
3. 即使輸入是 float16，內積與歸約也以 float32 **累加**。

bfloat16 的指數與 float32 一樣有 8 位元，因此很少需要損失縮放；Kalamkar 等人發現，bfloat16 訓練在多種模型上都能達到 float32 的結果，而且不需要調整超參數 [5]。代價是精度：$\varepsilon_{\text{mach}} \approx 0.0078$ 的 bfloat16 無法區分 $1 + 0.003$ 與 1，所以長串加總與權重更新仍需要 float32 累加。

損失縮放保護的是 float16 的小梯度；它無法修正錯誤的目標函數，也無法修正網路其他地方的溢位（例如注意力 logit 或正規化統計量）。改變精度也可能改變最佳化軌跡與輸出，特別是歸約運算與長時間訓練。請在目標硬體上驗證；較小的資料型別不保證每個運算都會變快 [13]。

非有限值的除錯順序

損失變成 `nan` 或 `inf` 時，別急著把所有東西都截斷。先找出第一個產生非有限值的運算，再判斷它屬於本篇哪一種失效模式。實用的順序：

1. **輸入：**檢查形狀、範圍、資料型別與缺值。一個 NaN 像素或一張沒有正規化的 0–255 影像，就會改變下游的一切。
2. **前向傳播：**檢查激活值與損失是否有限。留意 `exp`、`log`、除法、`sqrt`，以及變異數接近 0 的正規化層。
3. **損失：**使用以 logit 為輸入的損失（`cross_entropy`、`binary_cross_entropy_with_logits`），而不是先算機率再取 `log`。
4. **梯度：**在小模型上做梯度檢查；查看各層的梯度範數。

5. **更新：**比較更新量與權重的大小；學習率超過曲率極限 $2/\lambda_{\max}$ 時，即使算術完全精確，也會振盪甚至發散。

6. **精度：**先在 float32（或 float64）中重現，再加入混合精度或編譯。

先讓模型過擬合一個極小的批次，確認整條流程能學習。梯度範數截斷（gradient-norm clipping）限制的是傳給最佳化器的梯度，並不直接限制自適應最佳化器的更新量，也無法讓 NaN 梯度變得有意義。ML 02 會更詳細地介紹最佳化器與正規化。

現代觀點

本篇的核心概念已有數十年歷史，但因為模型更大、訓練更久、硬體不斷把精度往下壓，它們仍站在當前研究的中心。

大規模學習的最佳化理論。 Bottou、Curtis 與 Nocedal 發表在 SIAM Review 的綜述 [14] 是這個領域的標準地圖。它解釋為什麼隨機梯度法主導大規模機器學習（資料有冗餘時，便宜而有雜訊的步伐勝過昂貴而精確的步伐），在類似上文 Lipschitz 條件的平滑性假設下分析其收斂，並把改進方法分成兩大類：降低雜訊（更大或逐漸增大的批次、梯度聚合／變異數縮減、迭代平均）與二階方法（Hessian-free 牛頓法、Gauss–Newton、擬牛頓法、自然梯度與對角縮放）。Nocedal 與 Wright [6]、Boyd 與 Vandenberghe [7] 仍是確定性與凸理論的標準教科書；Baydin 等人 [11] 則綜述了讓梯度變得便宜的自動微分。

損失曲面實際長什麼樣子。 Dauphin 等人 [8] 主張，高維非凸最佳化的主要障礙是鞍點，而不是差勁的局部極小值。後來 Cohen 等人 [15] 觀察到，神經網路上的全批次梯度下降通常運作在「穩定性邊緣」（edge of stability）：Hessian 的最大特徵值會上升到剛好略高於 $2/\eta$——正是上文推導的古典穩定極限——而損失在長時間尺度上仍持續下降，只是不再單調。古典的二次分析預測此時會發散，所以如何解釋這個狀態仍是開放的理論問題。

考慮曲率的最佳化器。完整的牛頓法在大規模下不可行，但結構化的近似很實用。K-FAC [16] 以每層小矩陣的 Kronecker 乘積近似 Fisher 資訊矩陣（一種半正定的曲率矩陣，與 Hessian 的 Gauss–Newton 近似密切相關），讓求反矩陣變得便宜。Shampoo [17] 為張量的每個維度各維護一個預條件（preconditioning）矩陣。更近期的 Muon 對每個權重矩陣的更新量做近似正交化；Liu 等人 [18] 報告，在加入權重衰減並仔細調整每個參數的更新尺度後，它在其實驗中以約兩倍於所比較之自適應最佳化器基準的計算效率訓練大型語言模型。這些方法都在重新回答本篇用 Hessian 提出的問題：一步更新應該如何依方向重新縮放？

更低的精度。格式的演進從搭配損失縮放的 float16 [12]、bfloat16 [5]，一路走到 8 位元浮點數。Micikevicius 等人 [19] 提出兩種 FP8 編碼：E4M3（精度較高）與 E5M2（範圍較大），並在影像與語言模型上展示了與 16 位元基準相當的訓練結果。上方表格中的範圍–精度取捨，正是這些設計要平衡的東西，而它們放寬的是 IEEE 754 的規則 [3]。逐張量或逐區塊的縮放因子，扮演著 float16 時代損失縮放的角色。

大規模下的穩定 softmax。減去最大值的技巧有串流版本。Milakov 與 Gimelshein [20] 證明，只要維護一個執行中的最大值，並在最大值改變時重新縮放累加和，就能一趟算出 softmax 的正規化因子。FlashAttention [21] 以此為基礎，在高速的晶片上記憶體中逐區塊計算精確的注意

力，從不實際建出完整的注意力矩陣；FlashAttention-3 [22] 進一步延伸到 FP8，並以區塊量化與「incoherent processing」降低量化誤差。這個線上技巧的核心只要幾行：

```
import numpy as np

def online_logsumexp(chunks):
    m, s = -np.inf, 0.0                 # running max and rescaled running sum
    for c in chunks:
        m_new = max(m, c.max())
        s = s * np.exp(m - m_new) + np.exp(c - m_new).sum()
        m = m_new
    return m + np.log(s)

rng = np.random.default_rng(0)
z = rng.normal(0, 50, size=10_000)
chunks = np.array_split(z, 37)          # seen one block at a time
m = z.max()
reference = m + np.log(np.exp(z - m).sum())
print(online_logsumexp(chunks), reference)
assert np.isclose(online_logsumexp(chunks), reference)
```

兩個值都印出 174.0918752579 ...，在顯示的位數內完全相同，雖然線上版本從未同時看到所有 logit。ML 04 介紹注意力本身，ML 10 介紹部署時的量化。

開放問題。梯度方法為何能在穩定性邊緣成功；如何讓低精度（8 位元及以下）訓練像 float32 一樣穩健，而不必手動調整縮放；如何讓考慮曲率的最佳化器夠便宜、夠穩健，足以取代一階方法成為預設；以及在平行歸約改變加總順序時，如何得到可重現的結果。

重點整理

- 浮點數有固定的相對精度（$\varepsilon_{\text{mach}}$）與固定的範圍；指數位元決定範圍，小數位元決定精度。
- 溢位、下溢與消去都是可預期的。用代數方式修正：`exp` 之前先減最大值、使用 `log1p` ／ `expm1`、使用以 logit 為輸入的損失、平方之前先減平均值。
- 條件數是問題的性質；穩定性是演算法的性質。條件數 10^k 大約會讓你損失 k 位數字。
- Hessian 的特徵值給出曲率。梯度下降只有在 $\eta < 2/\lambda_{\max}$ 時穩定，並會依條件數 $\lambda_{\max}/\lambda_{\min}$ 變慢。
- 牛頓法依曲率重新縮放，在極小值附近收斂很快，但昂貴且會被鞍點吸引；擬牛頓法與 Hessian–向量乘積是實用的折衷。
- 限制條件可以用投影或廣義拉格朗日函數處理；KKT 條件刻畫了解的性質。
- 梯度檢查需要 float64、合理的步長與平滑的測試點，而且它只檢查你實際寫出的程式。
- 混合精度需要 float32 主權重與累加，float16 還需要損失縮放；要驗證，不要假設行為完全相同。

練習

1. **Softmax 平移**。證明 softmax 在 $z_i \mapsto z_i + c$ 下不變，並說明取 $c = -\max_i z_i$ 時分母落在 $[1, K]$ 之間。

   提示

   共同因子 $\exp(c)$ 在分子與分母間抵銷。平移後每一項 $\exp(z_j - m)$ 都在 $(0, 1]$ 之間，最大的一項等於 1，所以 K 項的總和介於 1 與 K 之間，既不會溢位，也不會下溢成 0。

2. **有限差分掃描**。對 $f(x) = e^x$ 在 $x = 0$ 處，以 $\epsilon = 10^{-1}, \dots, 10^{-12}$ 分別在 float32 與 float64 中計算前向與中央差分。各自在哪個 ϵ 誤差最小？與 $u^{1/2}$、$u^{1/3}$ 比較。

   提示

   float64 的 $u \approx 1.1 \times 10^{-16}$，所以 $u^{1/2} \approx 10^{-8}$（前向）、$u^{1/3} \approx 5 \times 10^{-6}$（中央）。float32 的 $u \approx 6 \times 10^{-8}$，對應約 2×10^{-4} 與 4×10^{-3}。觀察到的最佳步長應與這些值相差不到一個數量級，最低誤差則大約分別是 $u^{1/2}$ 與 $u^{2/3}$。

3. **穩定極限**。對 $f(w) = \tfrac{1}{2} w^\top H w$，$H = \operatorname{diag}(2, 50)$，求最大的穩定固定學習率，以及使最壞情況收縮因子最小的步長。用這個最佳步長，需要幾次迭代才能把誤差縮小 10^6 倍？

   提示

   穩定需要 $\eta < 2/50 = 0.04$。最佳步長為 $2/(\lambda_{\max} + \lambda_{\min}) = 2/52 \approx 0.0385$，收縮因子為 $(\kappa - 1)/(\kappa + 1) = 24/26 \approx 0.923$。解 $0.923^k \le 10^{-6}$：$k \ge \ln 10^6 / (-\ln 0.923) \approx 173$ 次。牛頓法只要一步。

4. **判斷臨界點**。找出 $f(x, y) = x^3 - 3x + y^2$ 的臨界點並加以分類。

   提示

   $\nabla f = (3x^2 - 3,\ 2y) = 0$ 給出 $(1, 0)$ 與 $(-1, 0)$。Hessian 為 $\operatorname{diag}(6x,\ 2)$：在 $(1, 0)$ 是 $\operatorname{diag}(6, 2)$，正定，為局部極小值；在 $(-1, 0)$ 是 $\operatorname{diag}(-6, 2)$，不定，為鞍點。兩者都不是全域極小值，因為 $x \to -\infty$ 時 $f \to -\infty$。從 $(-1, 0)$ 附近開始的牛頓法會收斂到鞍點。

5. **單形上的 KKT**。在 $\sum_i x_i = 1$ 且 $x_i \ge 0$ 的限制下，對 $c = (0.8, 0.6, -0.4)$ 最小化 $f(x) = \tfrac{1}{2}\|x - c\|_2^2$。寫出拉格朗日函數與 KKT 條件，並求出解。

   提示

   $L = \tfrac{1}{2}\|x - c\|^2 + \lambda(\sum_i x_i - 1) - \sum_i \alpha_i x_i$，$\alpha_i \ge 0$。穩定性給出 $x_i = c_i - \lambda + \alpha_i$；互補鬆弛性給出 $x_i = \max(c_i - \lambda, 0)$。猜測只有前兩個為正：$(0.8 - \lambda) + (0.6 - \lambda) = 1$，所以 $\lambda = 0.2$、$x = (0.6, 0.4, 0)$。驗證：$c_3 - \lambda = -0.6 < 0$，所以 $x_3 = 0$，且 $\alpha_3 = 0.6 \ge 0$。這就是到機率單形的歐氏投影。

6. **錯誤損失的正確梯度**。在梯度檢查的範例中，把 mean 歸約改成 sum。檢查還會通過嗎？自動微分錯了嗎？訓練上還有什麼會改變？

提示

如果同時一致地修改損失與解析梯度，檢查會通過：對新的目標函數而言，導數是正確的。自動微分沒有錯。但梯度變成原來的 N 倍，所以最大的穩定學習率縮小為 $1/N$；在多項損失中，各項之間的平衡也會改變。梯度檢查無法告訴你目標函數選錯了。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 4 (Numerical Computation) and ch. 8. Book
2. D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991. DOI
3. IEEE, "IEEE Standard for Floating-Point Arithmetic," IEEE Std 754-2019, 2019. DOI
4. N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002. DOI
5. D. Kalamkar et al., "A Study of BFLOAT16 for Deep Learning Training," arXiv:1905.12322, 2019. arXiv
6. J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006. DOI
7. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. Book
8. Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli and Y. Bengio, "Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization," *NeurIPS*, 2014. Paper
9. B. A. Pearlmutter, "Fast Exact Multiplication by the Hessian," *Neural Computation*, vol. 6, no. 1, pp. 147–160, 1994. DOI
10. D. C. Liu and J. Nocedal, "On the Limited Memory BFGS Method for Large Scale Optimization," *Mathematical Programming*, vol. 45, pp. 503–528, 1989. DOI
11. A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind, "Automatic Differentiation in Machine Learning: a Survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018. arXiv
12. P. Micikevicius et al., "Mixed Precision Training," *ICLR*, 2018. arXiv
13. PyTorch documentation, "Numerical Accuracy." Documentation
14. L. Bottou, F. E. Curtis and J. Nocedal, "Optimization Methods for Large-Scale Machine Learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. DOI
15. J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter and A. Talwalkar, "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability," *ICLR*, 2021. arXiv
16. J. Martens and R. Grosse, "Optimizing Neural Networks with Kronecker-factored Approximate Curvature," *ICML*, 2015. Paper
17. V. Gupta, T. Koren and Y. Singer, "Shampoo: Preconditioned Stochastic Tensor Optimization," *ICML*, 2018. Paper
18. J. Liu, J. Su, X. Yao et al., "Muon is Scalable for LLM Training," arXiv:2502.16982, 2025. arXiv
19. P. Micikevicius et al., "FP8 Formats for Deep Learning," arXiv:2209.05433, 2022. arXiv

20. M. Milakov and N. Gimelshein, “Online Normalizer Calculation for Softmax,” arXiv:1805.02867, 2018. arXiv
21. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022. Paper
22. J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani and T. Dao, “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision,” *NeurIPS*, 2024. Paper

ML 02・類神經網路：從感知器到深度網路

從單一感知器到深度網路：激活函數、正規化層、輸出層與損失函數、反向傳播、最佳化器，以及讓網路訓練得起來又能泛化的實務習慣。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-02-neural-networks/

你將學到

- 感知器（perceptron）是什麼、它的學習規則如何運作，以及它和邏輯迴歸（logistic regression）到底差在哪裡。
- 把層疊起來如何得到多層感知器（multilayer perceptron, MLP）、通用近似定理（universal approximation theorem）保證了什麼、沒保證什麼，以及「深」究竟是什麼意思。
- 目前常用的激活函數（sigmoid、tanh、softmax、ReLU、Leaky ReLU、GELU、SiLU/Swish、SwiGLU）與正規化層（BatchNorm、LayerNorm、RMSNorm），連同公式與導數。
- 如何讓輸出層搭配正確的損失函數，以及反向傳播（backpropagation）如何算出每一個梯度：我們會為兩層網路親手推導，並用 NumPy 做數值檢查。
- 實務上怎麼訓練：初始化、SGD、momentum、Adam 與 AdamW、學習率排程、batch size、梯度裁剪（gradient clipping），以及怎麼讀損失曲線。
- 如何診斷欠擬合（underfitting）與過擬合（overfitting）、對付過擬合的正則化工具，以及殘差連接（residual connection）如何一路通往 Transformer。

先看全貌

類神經網路是一個帶有許多可調旋鈕（也就是**參數**）的函數。它接收輸入，例如影像的像素或平面上一個點的兩個座標，然後產生輸出，例如類別標籤。所謂訓練，就是每次把旋鈕轉動一點點，讓模型在已知範例上的輸出越來越接近正確答案。本篇的所有內容都在回答三個問題之一：這個函數長什麼樣子（架構）、怎麼衡量「更接近」（損失函數），以及怎麼把旋鈕轉好（最佳化）。

> **用最簡單的話說**
>
> 想像一排人在傳紙條。第一個人看著一張圖片，寫下幾個簡單的觀察（「這裡有一條弧線」、「這一塊很亮」）。下一個人讀了這些紙條，寫出稍微聰明一點的結論（「這看起來像一隻眼睛」）。最後一個人說「貓」或「狗」。答錯的時候，老師從隊伍尾巴往回走，告訴每個人他的紙條要負多少責任，每個人就把自己的習慣改一點點。往回走的那一趟就是**反向傳播**，重複幾千幾萬次就是**訓練**。

為什麼重要：今天在影像、語音與語言領域的幾乎所有系統，都是由本篇介紹的零件組成的。卷積網路、Transformer 和擴散模型（diffusion model）用的是同一套材料：線性層、非線性激活函數、正規化、殘差連接、損失函數、反向傳播，以及自適應最佳化器。把這些弄清楚之後，深度學習系列談的就只是如何為影像排列這些零件。

關於本篇

本篇由作者先前的學習筆記〈機器學習及類神經網路筆記〉[1] 擴充改寫而成。閱讀前建議先熟悉ML 01 的觀念：資料、特徵、損失函數與梯度下降。

感知器

白話版。感知器是最簡單的人工神經元。它把每個輸入乘上一個權重，加總後再加上偏差（bias），總和為正就回答「是」，否則回答「否」。它只在答錯時才微調權重，藉此學習。

模型

Rosenblatt 在 1958 年提出感知器，作為大腦如何儲存與辨識圖樣的模型 [4]。對輸入向量 $\mathbf{x} \in \mathbb{R}^d$，它計算

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \mathbf{1}[z > 0],$$

其中 $\mathbf{w} \in \mathbb{R}^d$ 是權重向量，$b \in \mathbb{R}$ 是偏差，z 是**激活前的值**（pre-activation，即加權和），$\mathbf{1}[\cdot]$ 是指示函數（條件成立時為 1，否則為 0），$\hat{y} \in \{0, 1\}$ 是預測類別。這個指示函數就是**階梯函數**（step function）。從幾何上看，$\mathbf{w}^\top \mathbf{x} + b = 0$ 是一個超平面（在二維中是一條直線），感知器依照點落在超平面的哪一側來分類。

感知器學習規則

把訓練範例 $(\mathbf{x}_i, y_i)$ 一個一個拿來看。每看一個，就算出 $\hat{y}_i$ 並更新

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_i - \hat{y}_i)\,\mathbf{x}_i, \qquad b \leftarrow b + \eta\,(y_i - \hat{y}_i),$$

其中 $\eta > 0$ 是**學習率**（步長），$y_i \in \{0, 1\}$ 是真實標籤。預測正確時 $y_i - \hat{y}_i = 0$，什麼都不變。如果感知器答 0 但正確答案是 1，權重會往 $\mathbf{x}_i$ 的方向移動，下次遇到這個輸入時 z 就會變大；反方向的錯誤則讓權重遠離它。當兩個類別可以被一個超平面分開（資料**線性可分**）時，這個規則在有限次更新後就不會再犯錯；如果不可分，它永遠不會停下來。

```
import numpy as np

def perceptron_train(X, y, epochs=20, eta=1.0):
    """X: (N, d)，y 為 0 或 1。回傳 w, b。"""
    w, b = np.zeros(X.shape[1]), 0.0
    for _ in range(epochs):
        mistakes = 0
        for x_i, y_i in zip(X, y):
            y_hat = float(w @ x_i + b > 0)       # 階梯函數
            if y_hat != y_i:                     # 只在答錯時更新
                w += eta * (y_i - y_hat) * x_i
                b += eta * (y_i - y_hat)
```

```
                mistakes += 1
        if mistakes == 0:
            break
    return w, b

# AND 線性可分；XOR 不是
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
for name, y in [("AND", [0, 0, 0, 1]), ("XOR", [0, 1, 1, 0])]:
    w, b = perceptron_train(X, np.array(y, float))
    print(name, (X @ w + b > 0).astype(int))
# AND [0 0 0 1]
# XOR [1 1 0 0]   <- 錯了：沒有任何一條直線能做到
```

感知器與邏輯迴歸的差別

這兩個模型很容易混淆，因為第一步完全相同，都是 $z = \mathbf{w}^\top \mathbf{x} + b$。但它們是不同的方法：

	感知器	邏輯迴歸
輸出函數	階梯：$\hat{y} = \mathbf{1}[z > 0]$	sigmoid：$\hat{p} = \sigma(z) = \dfrac{1}{1+e^{-z}}$
輸出意義	0/1 的硬決策	機率 $P(y = 1 \mid \mathbf{x})$
訓練方式	感知器規則，只在答錯時更新	以梯度下降最小化交叉熵（最大概似）
資料不可分時	永不收斂	收斂到定義明確的最佳解

邏輯迴歸對單一範例的損失梯度是 $(\hat{p}_i - y_i)\,\mathbf{x}_i$，所以梯度下降的一步是 $\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_i - \hat{p}_i)\,\mathbf{x}_i$。這看起來幾乎和感知器規則一模一樣，但 $\hat{p}_i$ 是介於 0 與 1 之間的機率，因此**每一個**範例都有貢獻，而且非常有把握又答對的範例貢獻很小。正是這種平滑性，讓邏輯迴歸可以用梯度下降訓練，也讓它能被疊成更大的可微分網路。相對地，階梯函數幾乎處處導數為零，梯度根本無法通過它。

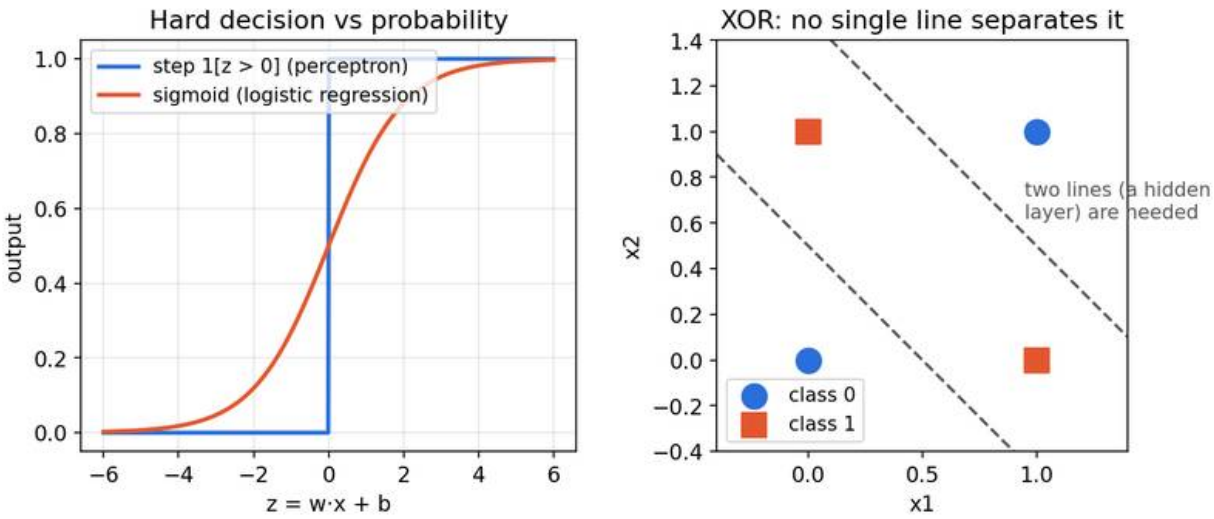


圖 1 — 左：感知器的階梯輸出與邏輯迴歸的 *sigmoid* 機率。右：*XOR*，任何單一線性邊界都無法解決的經典問題；需要兩條邊界（也就是一個隱藏層）。

多層感知器（MLP）

白話版。一個神經元只能畫一條直線。把很多神經元並排就有很多條直線，再把它們的輸出送進下一層組合起來，組合的結果可以彎曲，網路就能切出彎曲的區域。

把層疊起來

只有一個隱藏層的 MLP 計算

$$\mathbf{h} = f(W_1\mathbf{x} + \mathbf{b}_1),$$
$$\hat{\mathbf{y}} = g(W_2\mathbf{h} + \mathbf{b}_2),$$

其中 $\mathbf{x} \in \mathbb{R}^d$ 是輸入，$W_1 \in \mathbb{R}^{m\times d}$ 與 $\mathbf{b}_1 \in \mathbb{R}^m$ 是第一層的權重與偏差，m 是**寬度**（隱藏單元數），f 是逐元素作用的非線性**激活函數**（activation function），$\mathbf{h} \in \mathbb{R}^m$ 是隱藏表示，$W_2 \in \mathbb{R}^{K\times m}$ 與 $\mathbf{b}_2 \in \mathbb{R}^K$ 把它映射到 K 個輸出，g 則是依任務選擇的輸出函數（見後文）。每個隱藏單元都是一個「類似邏輯迴歸」的神經元；「多層感知器」這個名字是歷史沿用，現代 MLP 用的是平滑的激活函數，不是階梯函數。

非線性 f 是關鍵。少了它，$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$ 又只是一個線性映射：再多層的線性層都會塌縮成一層，網路依然只能畫一個超平面。以 XOR 為例，兩個隱藏單元可以各畫出圖 1 中的一條虛線，輸出單元再回答「是否落在兩條線之間」。

通用近似

Cybenko 在 1989 年證明：只要隱藏層夠寬，一**個**由 sigmoid 型單元組成的隱藏層，就能在有界的方塊（單位超立方體）上以任意精度逼近任何連續函數 [5]。後續研究把結果推廣到其他激活函數，包括 ReLU。這個定理令人安心，但要仔細讀：

- 它講的是**存在性**。它只說合適的權重存在，完全沒說梯度下降找不找得到。
- 它沒說「夠寬」到底要**多寬**。對某些函數，所需的寬度會隨輸入維度極快地成長。
- 它沒談**泛化**：把訓練點擬合得很好，不代表能準確預測新的點。

所以這個定理說明了 MLP 在原理上夠有彈性，而本篇其餘部分談的，正是它留下來的那些實務問題。

深度神經網路：深度與寬度

白話版。「深」只是代表依序疊了很多層。並沒有一個官方的層數門檻，超過就算深。

深度神經網路（deep neural network, DNN）是具有多個隱藏層的 MLP 或其他層狀網路：$\mathbf{h}_\ell = f(W_\ell\mathbf{h}_{\ell-1} + \mathbf{b}_\ell)$，$\ell = 1, \dots, L$，且 $\mathbf{h}_0 = \mathbf{x}$。「深」沒有公認的門檻。常見的非正式說法是隱藏層超過一層就算深，但各教材計算總層數或隱藏層數的方式不同，所以「三層」在不同書裡指的東西

也不一樣。重要的是概念而不是數字：深度網路是把很多簡單步驟**組合**（composition）起來得到答案。

增加模型能力有兩種方式：**寬度**（每層更多單元）與**深度**（更多層）。兩者並不等價。加寬是多放幾塊只組合一次的零件；加深則讓網路重複利用前面各層的特徵，建構出更抽象的特徵。LeCun、Bengio 與 Hinton 的回顧論文 [2] 正是以此解釋深度學習的成功：每一層都把前一層的表示轉換成稍微抽象一點的表示，疊很多這樣的層，網路就能直接從原始資料學到非常複雜的函數，而且特徵是學出來的，不是人工設計的。

從歷史來看，1990 年代以**淺層**方法為主流。支持向量機（SVM）是當時的主要工具，它是一種核方法（kernel method），並不是很寬的類神經網路；那時深度網路很難訓練。從 2000 年代中期開始，更多的資料、GPU 與更好的訓練技巧讓深度變得可行，深度網路開始在一個又一個基準測試中勝出 [2]。不過深度並不是全部：單純疊更多普通的層，最後反而會讓訓練**變差**，這個問題要靠殘差連接才得以解決（見最後一個主要章節）。

圖 2 以 scikit-learn 的雙月形（two moons）資料集展示寬度與深度的作用。一層只有 2 或 8 個隱藏單元時，邊界只由少數幾段直線組成，無法沿著月牙走。更寬或更深的網路能把邊界彎曲到貼合資料；最寬又最深的那個甚至繞著個別的雜訊點打轉，這是過擬合的第一個徵兆。

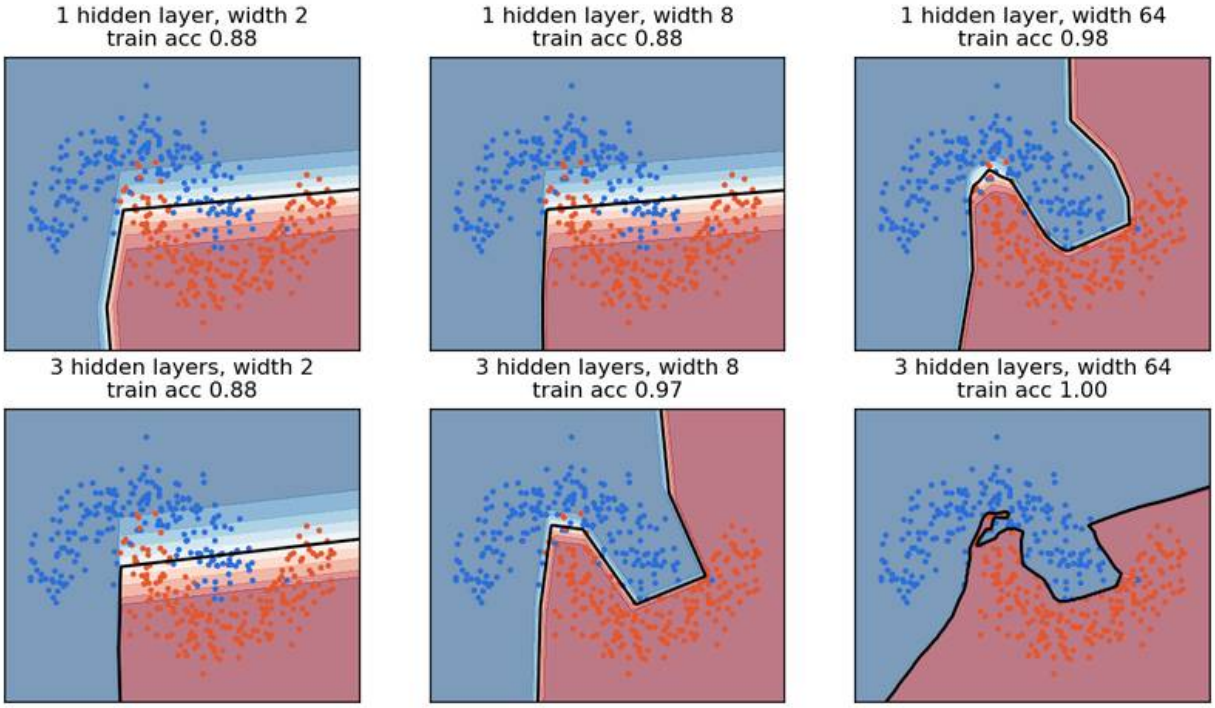


圖 *2 — ReLU MLP 在 make_moons（400 點、雜訊 0.2）上的決策邊界，欄為寬度、列為深度，每個模型都以 Adam 做 1,500 步全批次訓練。黑色曲線是機率 0.5 的等高線。*

激活函數

白話版。激活函數是每次加權和之後加上的「彎曲」。沒有它，整個網路就只是直線模型。不同的彎曲方式在平滑度、速度，以及梯度能否順利通過很多層之間取捨。

圖 3 畫出常見的選擇與它們的導數。導數和函數本身一樣重要，因為反向傳播會一層一層地把導數乘起來（見訓練章節）。

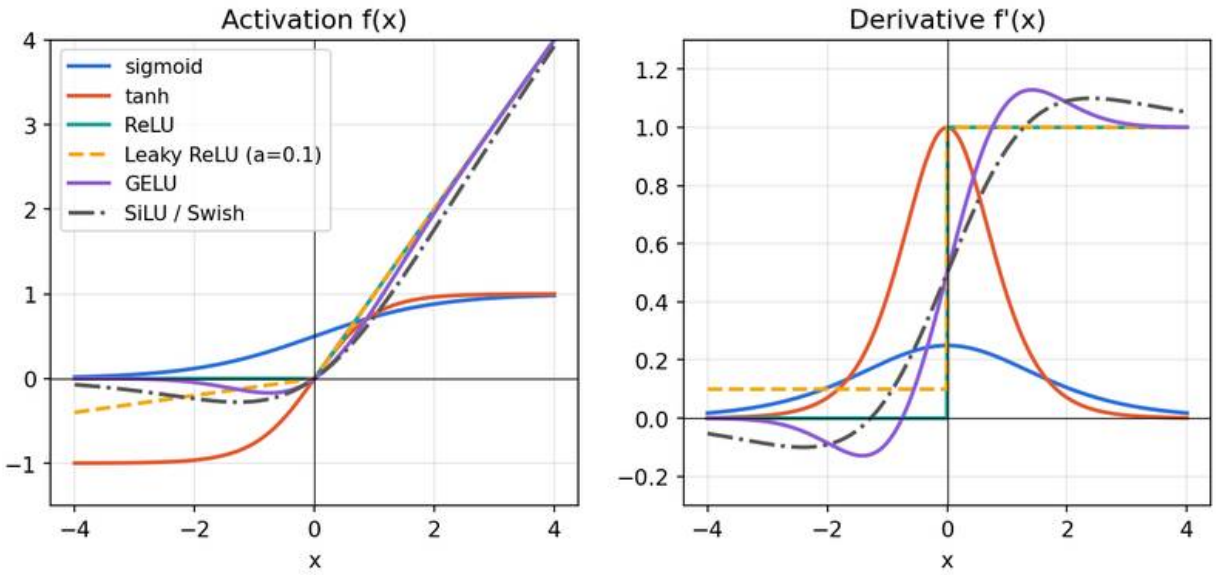


圖 3 — 常見激活函數（左）與其導數（右）。注意 *sigmoid* 的導數處處都很小（最多 *0.25*），而 *sigmoid* 和 *tanh* 在輸入很大時都變得非常平坦。

Sigmoid 與 tanh

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1-\sigma(x)); \qquad \tanh(x) = 2\sigma(2x) - 1, \quad \tanh'(x) = 1 - \tanh^2(x).$$

Sigmoid 把任何實數映射到 $(0,1)$；tanh 映射到 $(-1,1)$，並以零為中心。兩者都會**飽和**(saturate)：$|x|$ 很大時曲線很平，導數幾乎為零，梯度幾乎過不去。Sigmoid 的導數最大只有 $1/4$（在 $x=0$）。這兩件事讓深層 sigmoid 網路很難訓練，Glorot 與 Bengio 對此有詳細分析 [17]。今天 sigmoid 主要用在二元分類器的**輸出**與門控機制（例如下面的 SwiGLU），tanh 則出現在某些遞迴網路中。

Softmax

Softmax 把 K 個實數分數組成的向量 $\mathbf{z}$（稱為 **logits**）轉成機率分布：

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j),$$

其中 $k = j$ 時 δ_{kj} 為 1，否則為 0。每個 p_k 都是正數，而且總和為 1。和本節其他函數不同，softmax 作用在**整個向量**上，而不是逐元素作用，所以幾乎只用在多類別分類器的輸出層（以及 attention 內部），不當作隱藏層的激活函數。寫程式時，先減去 $\max_j z_j$ 再取指數；結果不變，但可以避免溢位。

ReLU 與 Leaky ReLU

$$\mathrm{ReLU}(x) = \max(0, x), \quad \mathrm{ReLU}'(x) = \mathbf{1}[x > 0]; \qquad \mathrm{LeakyReLU}_a(x) = \max(ax, x), \quad 0 < a < 1.$$

整流線性單元（rectified linear unit, ReLU）讓正的輸入原封不動通過，其餘歸零。它對所有正輸入的導數恰好是 1，所以梯度通過活躍單元時不會縮小，而且計算成本極低。ReLU 是讓深層監督式網路不必靠非監督式預訓練也能訓練起來的改變之一 [2]。它的弱點是**死亡 ReLU**（dying ReLU）：如果某個單元對所有範例的輸入都是負的，它的梯度就是零，從此不再學習。Leaky ReLU 在負側保留一個小斜率 a（例如 0.01），讓一些梯度永遠能流過。Dubey、Singh 與 Chaudhuri 的激活函數綜述 [7] 把這些以及後來的許多變體歸為 ReLU 家族。

GELU 與 SiLU/Swish

這些是 ReLU 的平滑版本，會讓小的負值通過：

$$\mathrm{GELU}(x) = x\,\Phi(x), \qquad \mathrm{GELU}'(x) = \Phi(x) + x\,\varphi(x),$$

其中 Φ 是標準常態分布的累積分布函數（CDF），φ 是它的密度函數 [8]。GELU 不像 ReLU 依正負號硬切，而是依輸入的大小來加權：$\Phi(x)$ 是標準常態隨機變數小於 x 的機率。常用的快速近似是 $0.5x(1 + \tanh[\sqrt{2/\pi}\,(x + 0.044715x^3)])$ [8]。GELU 是許多 Transformer 模型的預設激活函數。

$$\mathrm{Swish}_\beta(x) = x\,\sigma(\beta x), \qquad \mathrm{SiLU}(x) = x\,\sigma(x), \qquad \mathrm{SiLU}'(x) = \sigma(x)(1 + x(1 - \sigma(x))).$$

sigmoid 加權線性單元（SiLU）由 Elfwing、Uchibe 與 Doya 為強化學習提出 [9]。Ramachandran、Zoph 與 Le 以自動搜尋找到同樣的形狀，β 可固定或可學習，並稱之為 Swish [10]；$\beta = 1$ 時 Swish 就是 SiLU。GELU 和 SiLU 長得非常像（圖 3）。兩者都不是單調函數：對中等大小的負輸入，它們會稍微低於零。

SwiGLU：帶門控的前饋區塊

門控線性單元（gated linear unit, GLU）把兩個線性投影逐元素相乘，其中一個先通過一個像閘門一樣的非線性函數。Shazeer 在 Transformer 的前饋（feed-forward）區塊中測試了幾種變體 [11]。SwiGLU 版本以論文的列向量寫法、且省略偏差，寫成

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(\mathbf{x}) = \left(\mathrm{Swish}_1(\mathbf{x}W) \odot \mathbf{x}V\right) W_2,$$

其中 $\mathbf{x}$ 是大小為 d_{model} 的列向量，W 與 V 是 $d_{\text{model}} \times d_{\text{ff}}$ 矩陣，W_2 是 $d_{\text{ff}} \times d_{\text{model}}$ 矩陣，$\odot$ 是逐元素相乘。在 Shazeer 的實驗中，包括 SwiGLU 與 GEGLU 在內的 GLU 變體，表現都優於該區塊的 ReLU 與 GELU 版本。由於 SwiGLU 有三個權重矩陣而不是兩個，d_{ff} 要縮小為原本的 2/3，

參數量才會相同 [11]。例如 LLaMA 就以 SwiGLU 取代 ReLU，隱藏維度用 $\frac{2}{3} \cdot 4d$ [12]；SwiGLU 已成為大型語言模型的常見預設。

激活函數	值域	平滑	目前的典型用途
sigmoid	$(0, 1)$	是	二元／多標籤輸出、門控
tanh	$(-1, 1)$	是	某些遞迴網路
softmax	機率向量	是	多類別輸出、attention 權重
ReLU / Leaky ReLU	$[0, \infty)$ / $\mathbb{R}$	否（0 處有折角）	CNN、MLP，安全的預設
GELU、SiLU	約 $[-0.17, \infty)$、$[-0.28, \infty)$	是	Transformer、現代 CNN
SwiGLU	$\mathbb{R}$	是	Transformer 的前饋區塊

正規化層

白話版。訓練進行中，網路內部流動的數值可能飄到很大或很小的尺度，讓學習變得不穩定。正規化層（normalization）在每一步把它們重新縮放到標準大小，再讓網路自己選擇偏好的最終尺度。

三種常見的正規化層共用同一個模板。對一組數值 $x_1, \ldots, x_n$，

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2, \qquad y_i = \gamma\,\frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta,$$

其中 μ 與 σ^2 是平均與變異數，ϵ 是為了數值安全的小常數，γ, β 是可學習的縮放與平移（如果有幫助，網路可以藉此還原正規化）。三者的差別在於**哪些數值**被放在同一組（圖 4）。

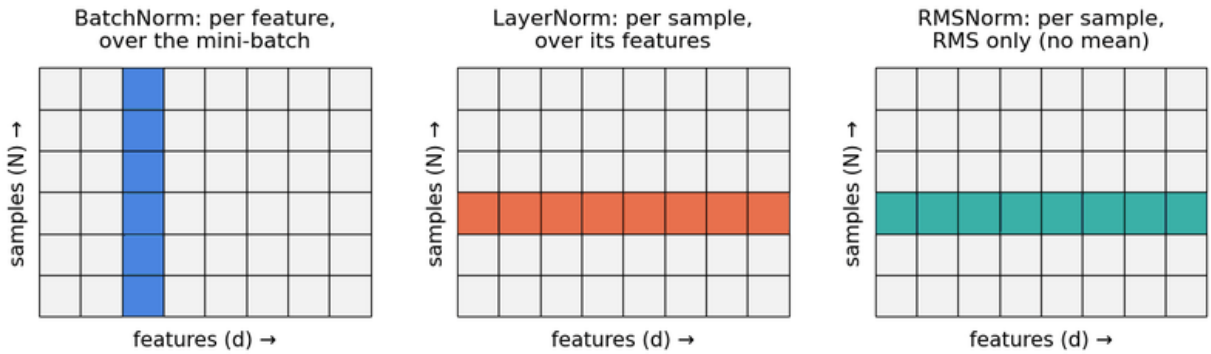


圖 4 — 哪些元素共用統計量。*BatchNorm* 對每個特徵在 *mini-batch* 上正規化；*LayerNorm* 與 *RMSNorm* 對每個樣本在它自己的特徵上正規化。

- **BatchNorm**（Ioffe 與 Szegedy [13]）對每個特徵（在 CNN 中是每個通道）**跨 mini-batch** 計算 μ, σ^2。它能加快訓練，成為 CNN 的標準配置。由於統計量取決於 batch，它在訓練與推論時的行為不同：推論時改用訓練期間累積的移動平均。batch 很小時統計量變得很吵，效果也會變差。
- **LayerNorm**（Ba、Kiros 與 Hinton [14]）**對單一樣本的所有特徵**計算 μ, σ^2。它與 batch 無關，訓練和測試時行為一致，也很自然地適用於序列。它是 Transformer 的標準正規化方式。

- **RMSNorm**（Zhang 與 Sennrich [15]）省略減去平均的步驟，只除以均方根：$y_i = \gamma_i x_i / \sqrt{\frac{1}{n}\sum_j x_j^2 + \epsilon}$。作者假設 LayerNorm 的重新置中（re-centering）是可有可無的，並報告品質相當而執行時間更短 [15]。LLaMA 採用 RMSNorm，並把它放在每個 Transformer 子層的**輸入端**（pre-normalization），而不是輸出端 [12]。

```
import torch

x = torch.randn(4, 8) * 5 + 3                          # 4 個樣本、8 個特徵
ln = torch.nn.LayerNorm(8, elementwise_affine=False)
bn = torch.nn.BatchNorm1d(8, affine=False)
rms = x / torch.sqrt(x.pow(2).mean(dim=1, keepdim=True) + 1e-6)

print(ln(x).mean(dim=1))     # 每個樣本（列）約為 0
print(bn(x).mean(dim=0))     # 每個特徵（欄）約為 0（訓練模式）
print(rms.pow(2).mean(dim=1))  # 每個樣本約為 1：RMS 為 1，但平均沒有被移除
```

Huang 等人 [16] 的綜述整理了這個家族，並把任何正規化方法拆成三個選擇：哪些數值放在一起（正規化區域）、做什麼運算（標準化、置中、縮放或白化），以及之後如何還原表示（可學習的仿射步驟）。BatchNorm、LayerNorm 與 RMSNorm 主要差在前兩項。

設計輸出層與選擇損失函數

白話版。最後一層必須說答案的語言：數量就輸出任意實數，是非題就輸出機率，「K 選一」就輸出機率分布。每種輸出都有一個天生搭配的損失函數，下表就是該用的組合。

任務	輸出層	輸出範圍	損失函數
迴歸	線性（不加激活函數）	$\mathbb{R}$	均方誤差（MSE，L2）或平均絕對誤差（MAE，L1）
二元分類	1 個單元 + sigmoid	$(0,1)$	二元交叉熵（binary cross-entropy）
多標籤（多個是非題）	K 個單元，各接 sigmoid	$(0,1)^K$	每個標籤各算二元交叉熵
多類別（K 選一）	K 個單元 + softmax	機率向量	交叉熵（cross-entropy）

常見的錯誤是在迴歸輸出接 sigmoid。sigmoid 的值域是開區間 $(0,1)$，它的工作是為二元或多標籤分類輸出機率。迴歸的輸出層通常是**線性**的，才能輸出任何實數。只有當目標值本身已知落在 0 到 1 之間（例如正規化後的像素強度或比例）時，迴歸輸出接 sigmoid 才合理。

對單一範例，損失函數為

$$\begin{aligned} \text{MSE：} \quad & \ell = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|^2, \\ \text{二元交叉熵：} \quad & \ell = -[\, y \log \hat{p} + (1-y)\log(1-\hat{p}) \,], \\ \text{交叉熵：} \quad & \ell = -\sum_{k=1}^{K} y_k \log p_k = -\log p_c, \end{aligned}$$

其中 $\mathbf{y}$ 是目標，$\hat{p} = \sigma(z)$，$\mathbf{p} = \mathrm{softmax}(\mathbf{z})$，$y_k$ 是 one-hot 標籤，c 是正確類別。（MSE 的 $\frac{1}{2}$ 只是為了方便；PyTorch 的 `MSELoss` 取平均且不含這個因子。）MAE 即 $\|\hat{\mathbf{y}} - \mathbf{y}\|_1$，比 MSE 更不受離群值影響。每一組搭配都是最大概似估計：MSE 對應目標上的高斯雜訊，二元交叉熵對應 Bernoulli 標籤，交叉熵對應類別（categorical）標籤 [3]。

這些搭配還有一個漂亮的實務理由。三種情況下，損失對輸出層**激活前的值**的梯度都只是

$$\frac{\partial \ell}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y} \quad \text{（預測減目標）},$$

這裡的 $\hat{\mathbf{y}}$ 分別指線性輸出、$\sigma(z)$ 或 $\mathrm{softmax}(\mathbf{z})$。會飽和的 sigmoid 或 softmax 導數被抵消掉了，所以即使模型非常有把握地答錯，梯度仍然很大。（練習 3 請你為 softmax 驗證這件事。）這也是為什麼框架會把最後的激活函數併進損失函數：PyTorch 的 `nn.CrossEntropyLoss` 與 `nn.BCEWithLogitsLoss` 接收的是 **logits**，所以模型本身在訓練時不能以 softmax 或 sigmoid 結尾。

訓練（一）：前向傳遞與反向傳播

白話版。前向傳遞（forward pass）從輸入到輸出逐層算出預測與損失。反向傳播再從損失往回走到輸入，用連鎖律（chain rule）算出每個權重對誤差的貢獻。它會重複利用中間結果，所以算出所有梯度的成本大約只等於多做一次前向傳遞。

Rumelhart、Hinton 與 Williams 讓反向傳播成為訓練多層網路的普及方法 [6]。它其實就是微積分的連鎖律，只是安排得讓任何計算都不重複。圖 5 畫出我們要微分的網路的計算圖。

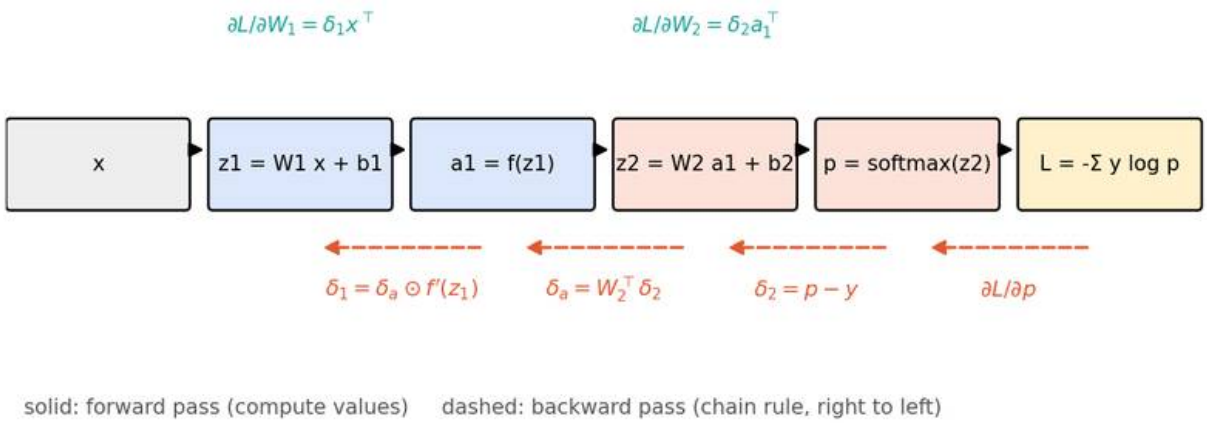


圖 5 — 搭配 *softmax* 交叉熵的兩層 *MLP* 的計算圖。實線：前向傳遞。虛線：反向傳遞，把誤差訊號 δ 從損失往輸入方向傳回去。

兩層 MLP 的推導

取一個範例 $(\mathbf{x}, \mathbf{y})$，$\mathbf{x} \in \mathbb{R}^d$，$\mathbf{y} \in \{0,1\}^K$ 是 one-hot 標籤。前向傳遞為

$$\mathbf{z}_1 = W_1\mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^m, \quad \mathbf{a}_1 = f(\mathbf{z}_1) \in \mathbb{R}^m,$$
$$\mathbf{z}_2 = W_2\mathbf{a}_1 + \mathbf{b}_2 \in \mathbb{R}^K, \quad \mathbf{p} = \mathrm{softmax}(\mathbf{z}_2), \qquad L = -\sum_k y_k \log p_k,$$

其中 $W_1 \in \mathbb{R}^{m\times d}$、$W_2 \in \mathbb{R}^{K\times m}$，$f$ 是逐元素的激活函數。定義某一層的**誤差訊號**為損失對該層激活前的值的梯度：$\delta_\ell = \partial L/\partial \mathbf{z}_\ell$。

步驟 1：輸出層。由上一節可知，softmax 搭配交叉熵給出

$$\delta_2 = \frac{\partial L}{\partial \mathbf{z}_2} = \mathbf{p} - \mathbf{y} \in \mathbb{R}^K.$$

步驟 2：輸出層權重。因為 $z_{2,k} = \sum_j W_{2,kj}\, a_{1,j} + b_{2,k}$，所以 $\partial z_{2,k}/\partial W_{2,kj} = a_{1,j}$，於是

$$\frac{\partial L}{\partial W_2} = \delta_2\, \mathbf{a}_1^\top \in \mathbb{R}^{K\times m}, \qquad \frac{\partial L}{\partial \mathbf{b}_2} = \delta_2.$$

一個權重的梯度等於（它輸出端的誤差）乘以（它輸入端的激活值）。形狀和 W_2 相同，這是很好用的檢查。

步驟 3：往回穿過線性層。每個隱藏激活值 $a_{1,j}$ 都影響所有輸出，所以它的梯度要對所有輸出加總：$\partial L/\partial a_{1,j} = \sum_k \delta_{2,k} W_{2,kj}$，寫成矩陣形式為

$$\frac{\partial L}{\partial \mathbf{a}_1} = W_2^\top \delta_2 \in \mathbb{R}^m.$$

步驟 4：往回穿過激活函數。因為 f 逐元素作用，它的 Jacobian 是對角矩陣，連鎖律就變成逐元素相乘：

$$\delta_1 = \frac{\partial L}{\partial \mathbf{z}_1} = (W_2^\top \delta_2) \odot f'(\mathbf{z}_1).$$

步驟 5：第一層權重。和步驟 2 完全一樣，

$$\frac{\partial L}{\partial W_1} = \delta_1\, \mathbf{x}^\top \in \mathbb{R}^{m\times d}, \qquad \frac{\partial L}{\partial \mathbf{b}_1} = \delta_1.$$

更深的網路只是重複步驟 3 到 5：$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$，以及 $\partial L/\partial W_\ell = \delta_\ell\, \mathbf{a}_{\ell-1}^\top$。對 N 個範例組成的 mini-batch，損失取平均，所以每個梯度也是各範例梯度的平均。

用 NumPy 從零實作，並做梯度檢查

下面的程式把範例存成**列**（`X` 的形狀是 $N \times d$），這是程式中常見的慣例。於是 `X @ W1`（`W1` 形狀為 $d\times m$）扮演數學式中 $W_1\mathbf{x}$ 的角色，上面每個公式在程式裡都以轉置的形式出現：例如 `dW2 = A1.T @ dZ2` 就是把 $\delta_2\mathbf{a}_1^\top$ 沿 batch 疊起來。**梯度檢查**把一個解析導數和中央差分 $(L(w+\epsilon)-L(w-\epsilon))/2\epsilon$ 比較；如果兩者在很多位數上都一致，反向傳遞就是對的。只要親手寫梯度，就一定要做這個檢查。

```
import numpy as np
from sklearn.datasets import make_moons
```

```
rng = np.random.default_rng(0)
X, y = make_moons(n_samples=200, noise=0.2, random_state=0)
N, d, h, K = X.shape[0], 2, 16, 2
Y = np.eye(K)[y]                                    # one-hot 目標，(N, K)

# He 式初始化（標準差 = sqrt(2 / fan_in)）
W1 = rng.normal(0, np.sqrt(2 / d), (d, h)); b1 = np.zeros(h)
W2 = rng.normal(0, np.sqrt(2 / h), (h, K)); b2 = np.zeros(K)

def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1                                # (N, h)
    A1 = np.maximum(Z1, 0)                          # ReLU
    Z2 = A1 @ W2 + b2                               # (N, K) logits
    Z2 = Z2 - Z2.max(axis=1, keepdims=True)         # 數值穩定
    P = np.exp(Z2) / np.exp(Z2).sum(axis=1, keepdims=True)
    loss = -np.mean(np.sum(Y * np.log(P + 1e-12), axis=1))
    return Z1, A1, P, loss

def backward(X, Z1, A1, P, W2):
    dZ2 = (P - Y) / N                               # 輸出層的 delta
    dW2 = A1.T @ dZ2;  db2 = dZ2.sum(axis=0)
    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * (Z1 > 0)                            # ReLU'(z) = 1[z > 0]
    dW1 = X.T @ dZ1;   db1 = dZ1.sum(axis=0)
    return dW1, db1, dW2, db2

# 對 W1 的一個元素做梯度檢查（中央差分）
Z1, A1, P, loss = forward(X, W1, b1, W2, b2)
dW1, db1, dW2, db2 = backward(X, Z1, A1, P, W2)
eps = 1e-6; E = np.zeros_like(W1); E[0, 3] = eps
num = (forward(X, W1 + E, b1, W2, b2)[3] - forward(X, W1 - E, b1, W2, b2)[3]) / (2 * eps)
print(f"analytic {dW1[0, 3]:.8f}  numeric {num:.8f}")

# 單純的梯度下降
lr = 0.5
for step in range(2000):
    Z1, A1, P, loss = forward(X, W1, b1, W2, b2)
    dW1, db1, dW2, db2 = backward(X, Z1, A1, P, W2)
    W1 -= lr * dW1; b1 -= lr * db1; W2 -= lr * dW2; b2 -= lr * db2
acc = (P.argmax(axis=1) == y).mean()
print(f"final loss {loss:.3f}, train accuracy {acc:.3f}")
# analytic -0.01941581  numeric -0.01941581
# final loss 0.063, train accuracy 0.970
```

實務上你永遠不必自己寫反向傳遞：PyTorch 等框架會記錄前向計算，在你呼叫 `loss.backward()` 時自動執行完全相同的程序。但親手寫過一次，仍是理解這個呼叫在做什麼、以及為什麼計算圖中某處導數為零時它會失靈的最好方法。

訓練（二）：讓最佳化真正可行

白話版。只知道梯度還不夠。你還需要一個合理的起點（初始化）、把梯度變成步伐的規則（最佳化器）、隨時間改變的步長（排程）、每一步看多少範例的選擇（batch size），以及防止偶發超大步伐的保護（裁剪）。

梯度消失與梯度爆炸

把遞迴式 $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$ 沿 L 層展開，第一層的梯度會包含 L 個權重矩陣與 L 個激活函數導數的**連乘**。如果典型的因子小於 1，連乘會隨深度指數縮小，前面的層幾乎學不到東西：這

就是**梯度消失**（vanishing gradient）。用 sigmoid 時每個 f' 最多 0.25，十層就可能把訊號縮小到 $0.25^{10} \approx 10^{-6}$。如果典型因子大於 1，例如初始權重太大，連乘會指數成長，更新變得巨大，損

失變成 `inf` 或 `NaN`：這就是**梯度爆炸**（exploding gradient）。

下面介紹的補救方法，從不同方向處理這個連乘：

- 在大範圍內導數為 1 的激活函數（ReLU 與它的平滑親戚）；
- 讓訊號大小逐層保持不變的初始化；
- 重新標準化激活值的正規化層；
- **殘差連接**，給梯度一條因子為 1 的通道（最後一個主要章節）；
- **梯度裁剪**，對付爆炸的標準緊急煞車。

有時也會列出其他做法，例如微調（fine-tuning）預訓練模型或改用較小的網路；它們讓整體訓練變得比較容易，但並不直接解決梯度問題。

權重初始化

如果所有權重一開始都相同，每個隱藏單元算的東西都一樣、拿到的梯度也一樣，它們就永遠一模一樣。所以權重要隨機初始化。尺度很重要：我們希望激活值（前向）與梯度（反向）的變異數在層與層之間大致維持不變。對有 n_{in} 個輸入、n_{out} 個輸出的線性層：

- **Xavier（Glorot）初始化** [17] 為 tanh 等對稱激活函數平衡前向與反向條件：$\mathrm{Var}(W_{ij}) = \dfrac{2}{n_{\text{in}} + n_{\text{out}}}$，例如在 $[-\sqrt{6/(n_{\text{in}}+n_{\text{out}})}, \sqrt{6/(n_{\text{in}}+n_{\text{out}})}]$ 上均勻分布。
- **He（Kaiming）初始化** [18] 考慮到 ReLU 會把約一半的輸入歸零，使每層的變異數減半，因此把尺度加倍：$\mathrm{Var}(W_{ij}) = \dfrac{2}{n_{\text{in}}}$，例如 $W_{ij} \sim \mathcal{N}(0, 2/n_{\text{in}})$。

偏差通常從零開始。He 等人證明這種初始化能讓非常深的 ReLU 網路從零開始訓練 [18]。上面的 NumPy 範例就是用它。

最佳化器

下面每個最佳化器都是**隨機梯度下降**（stochastic gradient descent, SGD）的變體：在一個小的隨機 mini-batch 上估計梯度，往反方向走一步，然後重複。記 $\mathbf{g}_t = \nabla_\theta L_{\text{batch}}(\theta_t)$ 為第 t 步的 mini-batch 梯度，θ 為所有參數，η 為學習率。

SGD。$\theta_{t+1} = \theta_t - \eta\,\mathbf{g}_t$。簡單、不需額外記憶體，但在狹長的山谷中會來回鋸齒狀前進，在平坦方向上又走得很慢。

SGD with momentum。維持一個移動的速度：$\mathbf{v}_{t+1} = \mu\,\mathbf{v}_t + \mathbf{g}_t$，$\theta_{t+1} = \theta_t - \eta\,\mathbf{v}_{t+1}$，$\mu$ 約為 0.9。就像一顆沉重的球滾下山坡，它把鋸齒平均掉，並在方向一致時累積速度。

Adam（Kingma 與 Ba [19]）為每個參數維持兩個移動平均，分別是梯度與梯度平方：

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2,$$
$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon},$$

其中平方、開根號與除法都是逐元素運算，$\beta_1 \approx 0.9$、$\beta_2 \approx 0.999$ 是衰減率，ϵ 是小常數。第一個平均就是 momentum。除以第二個平均的平方根，讓每個參數有自己的步長，這是 Adam 所借鏡的 RMSprop 的概念 [19]：梯度一直很大的參數走小步，很少被更新的參數走大步。**偏差修正**（bias correction）$1/(1-\beta^t)$ 是因為兩個平均都從零開始，若不修正，在最初幾步會太小。

AdamW（Loshchilov 與 Hutter [20]）。權重衰減（weight decay）在每一步把權重往零縮。對單純的 SGD，這等同於在損失中加入 L2 懲罰 $\frac{\lambda}{2}\|\theta\|^2$。對 Adam 則不然：L2 項會進入 $\mathbf{g}_t$，再被 $\sqrt{\hat{\mathbf{v}}_t}$ 除掉，於是梯度大的權重反而被正則化得**比較少**。AdamW 把衰減和自適應步伐**解耦**：

$$\theta_t = \theta_{t-1} - \eta\left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda\,\theta_{t-1}\right).$$

作者證明這能改善 Adam 的泛化能力 [20]，AdamW 現在是訓練 Transformer 與大型語言模型最常用的最佳化器。在 PyTorch 中，`torch.optim.Adam(..., weight_decay=λ)` 是耦合的 L2 版本，`torch.optim.AdamW` 才是解耦版本。

較新的最佳化器（簡述）。有兩個近年的選項值得認識。**Lion** 是透過自動化的程式搜尋找到的 [21]：它只保留一個 momentum 緩衝區（所以比 Adam 省記憶體），並以內插後 momentum 的**正負號**（sign）更新每個參數，因此所有更新的大小都相同；它需要比 AdamW 更小的學習率。**Muon** [22] 作用在隱藏層的二維權重矩陣上：它取 momentum 更新，用幾次 Newton–Schulz 迭代把它近似正交化；嵌入層、輸出層、偏差以及其他純量或向量參數仍以 AdamW 訓練。Liu 等人報告，在加入權重衰減與逐參數更新尺度調整之後，Muon 在他們的運算最適（compute-optimal）大型語言模型訓練實驗中，達到約 AdamW 兩倍的運算效率 [23]。這些都還是活躍的研究主題；AdamW 仍是安全的預設。

學習率排程

學習率通常是最重要的超參數。訓練初期合適的學習率，到後期常常太大，因為模型那時需要小而謹慎的步伐，所以學習率會隨時間改變（圖 6）。

- **階梯衰減**（step decay）：在固定的 epoch 把學習率乘上 0.1 之類的因子。以 SGD 訓練 CNN 的傳統做法。
- **餘弦退火**（cosine annealing）：$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t / T))$，在 T 步內從 $\eta_{\max}$ 平滑降到 $\eta_{\min}$；SGDR [24] 把它和週期性的熱重啟（warm restart）結合。
- **暖身**（warmup）：在最初幾百或幾千步內，把學習率從接近零線性升高。訓練初期 Adam 的二階動差估計很吵，權重也離任何好區域都很遠，直接用全尺寸的步伐可能造成不穩定。

以一個已發表的大型模型為具體例子：LLaMA 使用 AdamW（$\beta_1 = 0.9$、$\beta_2 = 0.95$）、權重衰減 0.1、梯度裁剪 1.0、2,000 步暖身，以及結束於峰值 10% 的餘弦排程 [12]。

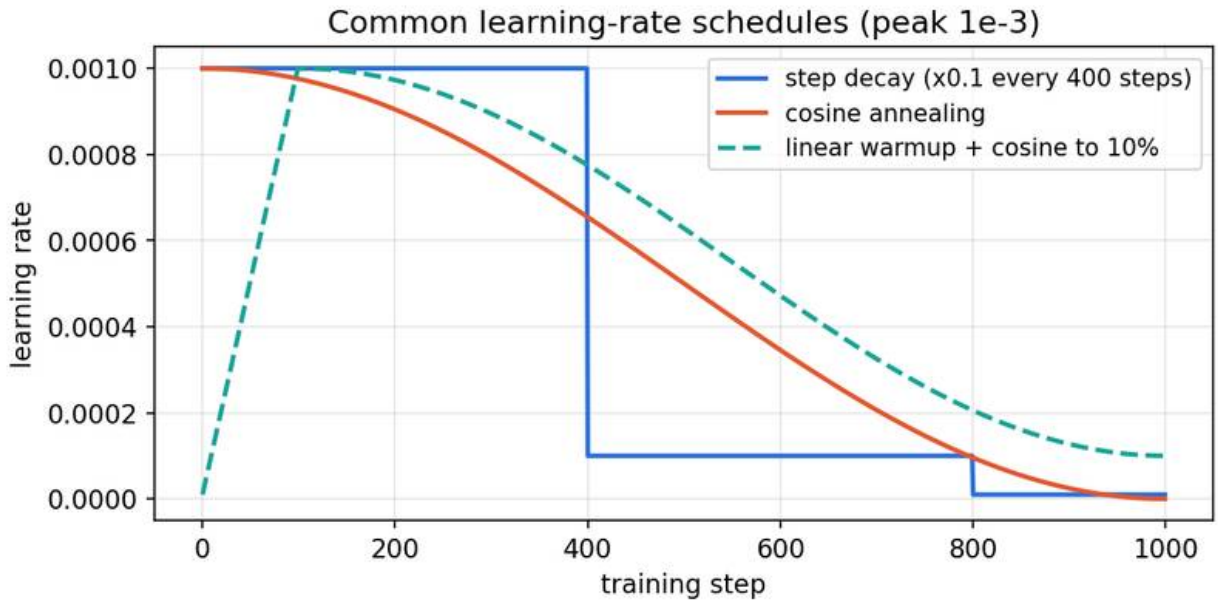


圖 *6* — 三種常見的學習率排程，共 *1,000* 步，峰值 *1e-3*。

Mini-batch、iteration 與 epoch

Mini-batch（常簡稱 batch）是用來做一次梯度估計的 B 個隨機範例。一次 **iteration**（迭代）是一次參數更新，使用一個 mini-batch。一個 **epoch** 是把整個訓練集完整走過一遍，對 N 個範例需要 $\lceil N/B \rceil$ 次 iteration。例如 $N = 50{,}000$、$B = 128$ 時，每個 epoch 有 391 次 iteration。

Batch size B 同時影響速度與結果，而且**並非越大越好**：

- **小 batch** 的梯度估計很吵（mini-batch 平均的變異數大約以 $1/B$ 下降），所以損失曲線跳動，每個 epoch 要走很多步，硬體利用率也差。

- **大 batch** 的梯度穩定，在 GPU 上平行效率高，但每個 epoch 的更新次數較少，而且在其他設定不變時往往泛化較差。Keskar 等人發現，大 batch 訓練傾向收斂到**尖銳極小值**（sharp minima），也就是解附近損失急遽上升的地方；小 batch 訓練則因為梯度雜訊，傾向找到泛化較好的**平坦極小值**（flat minima）[25]。

實務上，選一個能舒服放進記憶體、而且仍然訓練得好的最大 batch；改變 B 時要重新調整學習率，兩者交互作用很強。

梯度裁剪

即使網路設計得很好，偶爾某個 batch 也會產生巨大的梯度。**梯度範數裁剪**（gradient-norm clipping）在梯度範數超過門檻 c 時把整個梯度縮小：

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right),$$

其中 $\|\mathbf{g}\|$ 是把所有參數梯度串接起來後的範數。它保留方向、限制步伐大小，是對付梯度爆炸的標準防線，在遞迴網路與 Transformer 中尤其常用 [3]。門檻 1.0 是常見的選擇。

在 PyTorch 中組合起來

下面的程式以 mini-batch、AdamW、餘弦排程與梯度裁剪，在 `make_moons` 上訓練一個小型 MLP。在 CPU 上幾秒鐘就跑完。

```
import torch, torch.nn as nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
X, y = make_moons(n_samples=1000, noise=0.25, random_state=0)
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, random_state=0)
Xtr, Xva = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xva, dtype=torch.float32)
ytr, yva = torch.tensor(ytr), torch.tensor(yva)

model = nn.Sequential(
    nn.Linear(2, 64), nn.ReLU(),
    nn.Linear(64, 64), nn.ReLU(),
    nn.Linear(64, 2),               # 輸出 logits：這裡不加 softmax
)
loss_fn = nn.CrossEntropyLoss()     # 內部會做 log-softmax
opt = torch.optim.AdamW(model.parameters(), lr=1e-2, weight_decay=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=200)

for epoch in range(200):
    model.train()
    perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 64):                # 每個 mini-batch 64 筆
```

```
        idx = perm[i:i + 64]
        loss = loss_fn(model(Xtr[idx]), ytr[idx])
        opt.zero_grad()
        loss.backward()                                  # 反向傳播
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        opt.step()
    sched.step()

model.eval()
with torch.no_grad():
    acc = (model(Xva).argmax(dim=1) == yva).float().mean().item()
print(f"validation accuracy: {acc:.3f}")   # 約 0.94
```

注意 `model.train()` 與 `model.eval()`：它們讓 dropout、BatchNorm 等層在訓練行為與推論行為之間切換。測試時忘了 `eval()` 是經典的 bug。

泛化：欠擬合、過擬合與正則化

白話版。目標不是在訓練資料上表現好，而是在模型從沒看過的資料上表現好。模型有兩種相反的失敗方式：太弱，連訓練資料都學不會（欠擬合）；或者把訓練資料連同雜訊一起背下來，遇到新資料就失敗（過擬合）。

模型能力與兩種失敗模式

模型的**能力**（capacity）粗略地說就是它能擬合多複雜的函數；它隨參數量、深度、寬度與訓練時間增加。能力提高時，訓練誤差持續下降，驗證誤差則先降後升，因為模型開始擬合雜訊。兩者之間的差距稱為**泛化差距**（generalization gap）[3]。

- **欠擬合**：**訓練**誤差本身就很高。模型或訓練程序抓不到資料的規律。
- **過擬合**：訓練誤差很低，但驗證或測試誤差高得多。

處理欠擬合，依序是：

1. **先檢查程式與資料**。 bug、輸入與標籤沒對齊、前處理錯誤，或損失套在錯誤的張量上，看起來都像欠擬合。
2. **提高模型能力**：更寬或更深的模型，或更好的架構。
3. **改善最佳化**：訓練更久、調整學習率與排程、換最佳化器、檢查初始化與正規化。

把模型**變小**永遠不能解決欠擬合，那只會讓能力更低。在準確率已經足夠之後，為了速度或記憶體選擇較小的模型是合理的，但那是效率上的考量，不是解方。

處理過擬合：用更多或更多樣的資料、加入正則化（見下文）、降低能力，以及提早停止。也值得確認訓練集和驗證集真的來自同一個分布；兩者不一致時看起來也像過擬合，但需要的是修正資料，而不是正則化。

讀懂損失曲線

把訓練損失與驗證損失都對 iteration 或 epoch 作圖。下面四種情況涵蓋大部分狀況：

- **A. 兩條曲線一開始就是平的**。如果損失很高，模型根本沒在學：懷疑 bug、學習率太小或太大，或模型與資料不匹配。如果損失已經很低，可能是任務非常簡單，或每個記錄點之間隔了太多 iteration（一次記錄涵蓋的資料太多），把曲線的形狀藏起來了；請更頻繁地記錄。
- **B. 訓練損失持續下降，驗證損失轉而上升**。典型的過擬合。保留驗證損失最低時的參數（提早停止），並加入正則化。
- **C. 兩條曲線都下降，並在彼此接近處持平**。可能模型已接近最佳，這在訓練與驗證資料非常相似時很常見；或兩者都還很高，那就是欠擬合：訓練更久或提高能力。
- **D. 驗證損失高於訓練損失，但兩者都穩定下降**。正常、健康的情況。繼續訓練，並保留驗證損失最低的檢查點。

正則化工具

圖 7 刻意製造過擬合，並展示兩種解法。一個很寬的 MLP（兩個各 256 單元的隱藏層）只用 100 個有雜訊的雙月形資料點訓練。沒有正則化時，訓練損失降向零，驗證損失則很早就到達最低點，接著急遽上升。權重衰減與 dropout 都大幅減緩了這個上升。

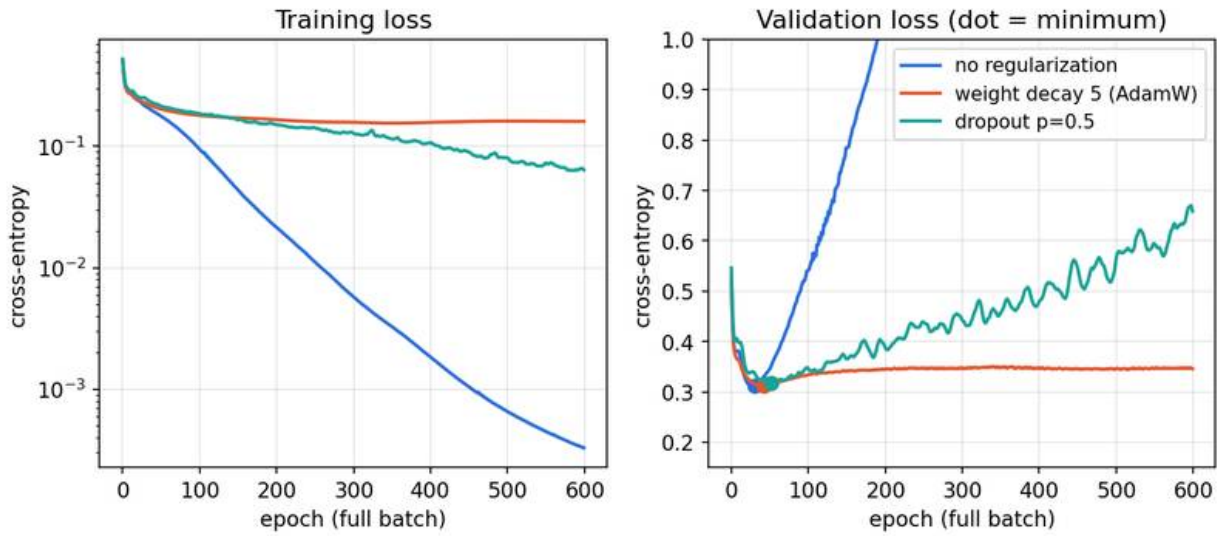


圖 7 — 以 *make_moons*（雜訊 *0.35*）的 *100* 個訓練點製造過擬合，並以 *2,000* 個驗證點評估。全批次 *AdamW*，學習率 *3e-3*。圓點標示每次執行的最低驗證損失，也就是提早停止會保留模型的位置。

權重衰減（L2 正則化）。在損失中加入 $\frac{\lambda}{2}\|\theta\|^2$ 來懲罰大權重；使用 Adam 時則改用解耦的衰減（AdamW）。大權重讓網路能圍著個別資料點畫出尖銳、彎曲的邊界；讓權重保持小，就偏好較平滑的函數。在圖 7 中，$\lambda = 5$ 的解耦衰減讓驗證損失在最低點之後幾乎持平。

Dropout（Srivastava 等人 [27]）。訓練時，每個隱藏單元以機率 p 被設為零，每個範例、每一步都獨立抽樣，存活的單元再乘上 $1/(1-p)$，讓激活值的期望不變（「inverted dropout」）。測試時

使用所有單元，不做縮放。於是每一步訓練的都是一個不同的「變瘦」子網路；單元無法依賴特定的夥伴，必須學出本身就有用的特徵，而測試時的完整網路就像許多瘦網路的平均 [27]。這也是 `model.eval()` 很重要的原因。

資料擴增（data augmentation）。用不改變標籤的轉換產生新的訓練範例：對影像做翻轉、裁切、小角度旋轉、色彩抖動與加雜訊。它常常是影像領域最有效的單一正則化方法，因為它直接教模型該有的不變性。但它必須符合任務：水平翻轉對貓沒問題，對辨識文字就不行。

提早停止（early stopping）。訓練時監看驗證損失，保留它最低那個 epoch 的參數；如果連續若干個 epoch（「patience」）都沒有改善就停止。它不花成本、不必修改模型，並藉由限制參數能離初始值多遠而發揮正則化作用 [3]。

更多資料與更好的策略。原始筆記列出的其他手段值得再強調：更多訓練資料是對付過擬合最可靠的方法；更強的架構（更深，或結構更好以利收斂）在模型欠擬合時有幫助；更好的訓練策略（最佳化器、學習率排程、batch size）則兩者都有幫助。

殘差連接與通往 Transformer 之路

白話版。疊很多層時，每一層不但要改進訊號，還得把它完整傳下去，這其實很難。殘差連接給每個區塊一條快速道路：區塊只需要學一個加在輸入上的修正量。

He 等人觀察到，在已經很深的網路上再加普通的層，連**訓練**誤差都會變差；這是最佳化的問題，不是過擬合 [28]。他們的**殘差區塊**計算

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}),$$

其中 $\mathbf{x}$ 是區塊的輸入，F 是一小疊層（在 ResNet 中是卷積加上 BatchNorm 與 ReLU）。如果一個區塊最好的做法是什麼都不做，它只需要把 F 推向零，這很容易。反向傳遞說明了為什麼它好訓練：

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = I + \frac{\partial F}{\partial \mathbf{x}},$$

其中 I 是單位矩陣。單位項讓梯度直接穿過每一個區塊，所以許多區塊的連乘不再必然縮到零。靠著殘差連接，He 等人在 ImageNet 上訓練了多達 152 層的網路，並贏得 ILSVRC 2015 的多個項目 [28]。

Transformer（Vaswani 等人 [29]）結合了本篇的每一個概念。每個區塊有兩個子層：多頭自注意力（multi-head self-attention，用 softmax 在不同位置之間混合資訊）與逐位置的 MLP，每個子層都包在殘差連接與 LayerNorm 裡。原始論文是在相加之後才正規化；許多近期模型（包括 LLaMA）改為正規化子層的**輸入**（pre-normalization），並使用 RMSNorm 與 SwiGLU [12]：

$$\mathbf{h} = \mathbf{x} + \mathrm{Attention}\,(\,\mathrm{Norm}(\mathbf{x})), \qquad \mathbf{y} = \mathbf{h} + \mathrm{FFN}\,(\,\mathrm{Norm}(\mathbf{h})).$$

自 2017 年起，這個設計從機器翻譯擴展到語言模型、影像、語音等領域，也是今日大型語言模型的基礎。它的訓練方法同樣來自本篇：AdamW、暖身加餘弦衰減、梯度裁剪與權重衰減 [12]。深度學習系列會從這裡接著談用於影像的卷積網路與 Transformer，第一個任務是影像分類。

現代觀點

上述的基本工具已經穩定好幾年了；研究持續探討它們為什麼有效，以及每個零件更好的版本。以下幾篇回顧從不同角度提供了很好的導覽。

全貌：LeCun、Bengio 與 Hinton [2]（Nature，2015）。這篇短回顧至今仍是「深度學習是什麼」最清楚的說明：以多層進行表示學習（representation learning），每一層把前一層的表示轉成更抽象的表示，特徵以反向傳播從資料中學得，而不是人工設計。它介紹了以 SGD 進行的監督式學習、ReLU 的角色、處理影像的卷積網路、處理序列的遞迴網路，以及分散式表示（distributed representation），最後預測非監督式學習，以及表示學習與推理的結合，在未來會更加重要。若要以教科書的方式學習本篇的每個主題（包括損失函數的最大概似觀點、正則化與最佳化），Goodfellow、Bengio 與 Courville 的書 [3] 仍是標準參考。

最佳化：Sun [26]。Sun 的綜述問的是「類神經網路何時、為什麼能被成功訓練？」，並把答案分成三部分：梯度的問題（爆炸、消失）以及透過謹慎的初始化與正規化來解決；實用的演算法（SGD、Adam 等自適應方法，以及分散式訓練）與它們的收斂理論；以及損失地形（loss landscape）的整體圖像，包括不好的局部極小值、模式連通性（mode connectivity）、樂透彩券假說（lottery ticket hypothesis）與無限寬度極限。它是從上述實務走向理論的好橋樑。

正規化：Huang 等人 [16]（IEEE TPAMI，2023）。這篇綜述從最佳化的角度給出正規化方法的統一觀點，並提出一套分類法，把任何方法拆成正規化區域、正規化運算與表示還原三部分。它涵蓋從 BatchNorm 開始的發展史、正規化為何能加速訓練並幫助泛化的分析，以及在不同領域的應用。讀它可以把 BatchNorm、LayerNorm、分組式與以權重為主的正規化，看成同一個設計空間中的不同點。

激活函數：Dubey、Singh 與 Chaudhuri [7]（Neurocomputing，2022）。這篇綜述把激活函數分為 logistic sigmoid ／ tanh 類、ReLU 類、ELU 類與可學習類，依值域、單調性與平滑性等性質加以描述，並在不同網路架構與資料類型上評測其中 18 種。它的實務訊息是：沒有哪一種激活函數處處最好，選擇會和架構與資料互相影響。

現況（2023–2026）。從已發表的模型訓練配方可以看到幾個趨同的方向：

- **架構**。採用 pre-normalization 與殘差連接的 Transformer 區塊，是語言領域的主流架構，在影像領域也越來越普遍；RMSNorm 與 SwiGLU（如 LLaMA [12]）是大型語言模型的常見預設。
- **最佳化**。AdamW [20] 搭配線性暖身、餘弦（或類似的）衰減與梯度裁剪是標準配方。不只做逐座標縮放的最佳化器是活躍的研究方向：Lion [21] 使用正負號更新、較省記憶體；Muon [22] 把矩陣更新正交化，並有在大規模下提升運算效率的報告 [23]。這些增益在不同任務與規模下有多穩健，仍在研究中，因此 AdamW 依然是安全的選擇。
- **泛化**。尖銳與平坦極小值的觀點 [25] 是有用的直覺，而不是完整的理論；高度過度參數化（over-parameterized）的網路為什麼還能泛化得這麼好，至今仍是開放問題，Sun 的綜述 [26] 整理了主要的研究方向。

開放問題。深度網路可靠的泛化理論；同時選擇學習率、batch size 與排程的原則性規則，特別是在放大模型規模時；正規化、殘差連接與自適應最佳化器之間究竟如何交互作用；以及不靠目前配方中那些大量小技巧，也能在極大規模下穩定訓練。

重點整理

- 感知器是線性分數接上階梯函數，以「答錯才更新」的規則訓練；邏輯迴歸則用 sigmoid 機率與交叉熵。結構相似但方法不同，而且只有平滑的那一個能被疊起來並以梯度訓練。
- 具有非線性激活函數的隱藏層，才讓網路不只是線性模型。一個夠寬的隱藏層在原理上能逼近任何連續函數，但那只是存在性結果；深度讓有用的函數更容易被表示與學習。「深」沒有固定的層數定義。
- ReLU 家族（ReLU、Leaky ReLU、GELU、SiLU）讓梯度保持暢通；sigmoid 與 softmax 主要放在輸出層；SwiGLU 是 Transformer 前饋層的現代選擇。BatchNorm 沿 batch 正規化，LayerNorm 與 RMSNorm 沿單一樣本的特徵正規化。
- 輸出層要配合任務：迴歸用線性加 MSE，二元或多標籤用 sigmoid 加二元交叉熵，多類別用 softmax 加交叉熵。每種情況的輸出梯度都是預測減目標。
- 反向傳播是從損失往回套用連鎖律：$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$，$\partial L/\partial W_\ell = \delta_\ell \mathbf{a}_{\ell-1}^\top$。親手寫的梯度要做數值檢查。
- 實務訓練：He 或 Xavier 初始化、AdamW 搭配暖身與衰減排程、梯度裁剪，以及和學習率一起選擇的 batch size，而不是「越大越好」。
- 欠擬合代表訓練誤差高，需要更多模型能力或更好的最佳化，絕不是更小的模型；過擬合則需要更多資料、資料擴增、權重衰減、dropout 與提早停止。
- 殘差連接讓非常深的網路可以訓練，而 Transformer 正是這些概念的集大成。

練習

1. **手算感知器**。對 AND 資料執行感知器規則，$\eta = 1$、$\mathbf{w} = \mathbf{0}$、$b = 0$，依 $(0,0), (0,1), (1,0), (1,1)$ 的順序拜訪各點。寫下每次更新後的 $\mathbf{w}$ 與 b，直到某個 epoch 完全沒有錯誤。接著用一句話解釋，為什麼同樣的程序用在 XOR 上永遠不會停。

   提示

   在嚴格判斷 $z > 0$ 下，第一個點 $(0,0)$ 的 $z = 0$，預測為 0，是正確的。第一個錯誤發生在 $(1,1)$，得到 $\mathbf{w} = (1,1)$、$b = 1$。繼續做下去：在標籤為 0 的點上犯錯就減去 $\mathbf{x}$ 和 1，在標籤為 1 的點上犯錯就加上它們。以這個順序，五個 epoch 內共更新十次，第六個 epoch 完全正確，此時 $\mathbf{w} = (2,1)$、$b = -2$，也就是直線 $2x_1 + x_2 = 2$。換一個拜訪順序會得到另一條同樣有效的直線。對 XOR 而言，沒有任何 $(\mathbf{w}, b)$ 能把四個點全部分對，所以永遠有某個點是錯的，也就永遠會更新。

2. **sigmoid 的梯度消失得多快？** 證明 $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \le 1/4$。接著考慮一條由 20 個 sigmoid 層組成、所有權重都等於 1 的鏈。梯度在往回傳的路上，最多會被乘上多大的因子？對活躍單元的 ReLU 重做一次推理。

   提示

   $s(1-s)$ 在 $s \in (0,1)$ 上是開口向下的拋物線，在 $s = 1/2$ 有最大值 $1/4$。二十個最多 $1/4$ 的因子相乘，最多是 $4^{-20} \approx 9 \times 10^{-13}$。對活躍的 ReLU 單元，$f' = 1$，所以激活函數完全不會造成縮小；只有權重會，這就是初始化重要的原因。

3. 「**預測減目標**」**梯度**。利用 $\partial p_k/\partial z_j = p_k(\delta_{kj} - p_j)$，證明當 $\mathbf{y}$ 為 one-hot 時，$L = -\sum_k y_k \log p_k$ 滿足 $\partial L/\partial z_j = p_j - y_j$。再對 sigmoid 搭配二元交叉熵證明同樣的結果。

提示

$\partial L/\partial z_j = -\sum_k (y_k/p_k)\, p_k(\delta_{kj} - p_j) = -\sum_k y_k\delta_{kj} + p_j\sum_k y_k = -y_j + p_j$，因為 $\sum_k y_k = 1$。對 sigmoid，$\partial \ell/\partial \hat{p} = -y/\hat{p} + (1-y)/(1-\hat{p})$，且 $\partial \hat{p}/\partial z = \hat{p}(1-\hat{p})$；相乘並化簡即得 $\hat{p} - y$。

4. **擴充梯度檢查**。在 NumPy 範例中，把 `W2`、`b1`、`b2` 的每個元素（而不只是 `W1` 的一個元素）都和有限差分比較，並回報最大相對誤差 $|a-n|/\max(|a|,|n|,10^{-12})$。然後故意放一個 bug，例如拿掉 `* (Z1 > 0)` 這個因子，看看誤差會變多大。

提示

用 `np.ndindex(param.shape)` 跑過每個元素，把它擾動 $\pm 10^{-6}$ 並重新計算損失。正確的實作相對誤差約在 10^{-7} 或更小（少數緊鄰 ReLU 折角的元素可能較大）。拿掉 ReLU 導數後，`W1` 與 `b1` 會出現量級為 1 的誤差，而 `W2` 與 `b2` 仍然正確，因為 bug 只在隱藏層以下的反向路徑上。

5. **Iteration、epoch 與 batch size**。某資料集有 60,000 張訓練影像。(a) $B = 32$ 與 $B = 1{,}024$ 時，一個 epoch 各有多少次 iteration？(b) 兩者都訓練 20 個 epoch，各做了多少次參數更新？(c) 根據 batch size 的討論，舉出兩個理由說明在相同學習率下，$B = 1{,}024$ 的執行可能得到較差的驗證準確率，並提出一個可能有幫助的調整。

提示

(a) $\lceil 60000/32 \rceil = 1875$ 與 $\lceil 60000/1024 \rceil = 59$。(b) 37,500 次對 1,180 次更新。(c) 同樣的 epoch 數下更新次數少很多；梯度雜訊也較少，而尖銳極小值的研究把這和較差的泛化連在一起。常見的調整是針對新的 batch size 重新調學習率（通常調大，並加上暖身），以及訓練更多 epoch。

6. **診斷曲線**。同事的網路訓練損失一開始下降很快，接著停在一個很高的值不動；驗證損失和它幾乎貼在一起。同事提議把隱藏單元數減半，「讓它收斂得快一點」。發生了什麼事？你會先檢查什麼？你會改提出什麼建議？

提示

兩條損失都很高而且彼此接近：這是欠擬合（數值偏高的情況 C），不是過擬合。先檢查 bug：標籤是否與輸入對齊、損失是否作用在 logits 上、資料是否已正規化、學習率是否合理。接著提高模型能力或改善最佳化（更寬或更深的模型、學習率排程、訓練更久）。把模型縮小會降低能力，只會讓欠擬合更嚴重。

參考文獻

1. Shyandram, “機器學習及類神經網路筆記,” Medium post, 2024; updated 2026. link
2. Y. LeCun, Y. Bengio and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. doi
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. online book
4. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, pp. 386–408, 1958. doi
5. G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, pp. 303–314, 1989. doi
6. D. E. Rumelhart, G. E. Hinton and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986. doi
7. S. R. Dubey, S. K. Singh and B. B. Chaudhuri, “Activation functions in deep learning: A comprehensive survey and benchmark,” *Neurocomputing*, vol. 503, pp. 92–108, 2022. arXiv
8. D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” arXiv:1606.08415, 2016. arXiv
9. S. Elfwing, E. Uchibe and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” *Neural Networks*, vol. 107, pp. 3–11, 2018. arXiv
10. P. Ramachandran, B. Zoph and Q. V. Le, “Searching for activation functions,” arXiv:1710.05941, 2017. arXiv
11. N. Shazeer, “GLU variants improve Transformer,” arXiv:2002.05202, 2020. arXiv
12. H. Touvron, T. Lavril, G. Izacard et al., “LLaMA: Open and efficient foundation language models,” arXiv:2302.13971, 2023. arXiv
13. S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. International Conference on Machine Learning* (*ICML*), PMLR 37, pp. 448–456, 2015. PMLR
14. J. L. Ba, J. R. Kiros and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016. arXiv
15. B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2019. arXiv
16. L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu and L. Shao, “Normalization techniques in training DNNs: Methodology, analysis and application,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 10173–10196, 2023. arXiv
17. X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. International Conference on Artificial Intelligence and Statistics* (*AISTATS*), PMLR 9, pp. 249–256, 2010. PMLR
18. K. He, X. Zhang, S. Ren and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification,” in *Proc. IEEE International Conference on Computer Vision* (*ICCV*), pp. 1026–1034, 2015. arXiv
19. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
20. I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2019. arXiv
21. X. Chen, C. Liang, D. Huang, E. Real, K. Wang, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu and Q. V. Le, “Symbolic discovery of optimization algorithms,” in *Advances in Neural*

Information Processing Systems (*NeurIPS*), 2023. arXiv
22. K. Jordan, "Muon: An optimizer for hidden layers in neural networks," Medium post, December 2024. link
23. J. Liu, J. Su, X. Yao et al., "Muon is scalable for LLM training," arXiv:2502.16982, 2025. arXiv
24. I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
25. N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
26. R. Sun, "Optimization for deep learning: Theory and algorithms," arXiv:1912.08957, 2019. arXiv
27. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. JMLR
28. K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition* (*CVPR*), pp. 770–778, 2016. arXiv
29. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2017. arXiv

ML 03・卷積、影像濾波與 CNN

從模糊與 Sobel 卷積核到學習式濾波器：卷積與互相關的差別、邊界處理、填補、步幅、膨脹、池化、感受野、卷積的反向傳播，以及經典 CNN 架構。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-03-convolutional-networks/

筆記本: https://github.com/Shyandram/cv-blog/blob/6b02ce5/notebooks/image-processing/convolution.ipynb

你將學到

- 什麼是卷積核（kernel）、它如何在影像上滑動，以及為什麼同一種運算可以模糊、銳化或找出邊緣。
- **互相關**（correlation）與**卷積**（convolution）的確切差別，以及 NumPy、SciPy、OpenCV、PyTorch 實際計算的是哪一種。
- 如何用 NumPy 從零寫出 2-D 濾波，在 OpenCV、SciPy 與 PyTorch 中重現同樣結果，並有意識地處理影像邊界。
- 卷積層如何推廣固定濾波器：通道、權重共享、填補、步幅、膨脹、1×1 卷積與池化。
- 感受野如何隨深度成長，以及梯度如何反向穿過一個卷積。
- 經典架構（LeNet、AlexNet、VGG、GoogLeNet、ResNet、U-Net、MobileNet、EfficientNet、ConvNeXt）各自的貢獻，以及 CNN 在 2026 年的定位。

先看全貌

影像是一個數字網格（見 影像就是陣列）。我們想對影像做的許多事都是**局部的**：一個像素是在邊緣上、在平滑區域內，還是在雜訊斑塊裡，大多取決於它的鄰居。**卷積**正是把「看一小塊鄰域、用固定權重組合、然後在每個位置重複」這件事形式化的運算。它是古典影像處理的主力工具，也是**卷積神經網路**（convolutional neural network, CNN）的基本積木；差別在於 CNN 從資料中學習權重，而不是由人手動設計。

> **用最簡單的話說**
>
> 想像你有一張開了九個小洞的硬紙板，每個洞後面寫著一個數字。把紙板放在圖畫上，把每個洞看到的顏色乘上洞後面的數字，全部加起來，把答案寫在一張新紙上。接著把紙板滑一格，再做一次，直到蓋過整張圖。如果九個數字都一樣，新的圖就是模糊的平均。如果左邊的數字是負的、右邊是正的，只有在圖從暗變亮的地方才會得到大答案：這就是邊緣偵測器。CNN 就是一台同時滑動成千上萬片這種紙板，並且**自己學會**洞後面該寫什麼數字的機器。

本篇分成兩半。前半（濾波）是古典工具箱，只需要 NumPy。後半（CNN）建立在ML 02 之上，你在那裡已經認識了網路層、損失函數與反向傳播。若之後想更深入固定濾波器，IP 03 介紹空間濾波，IP 04 則透過卷積定理說明頻率域觀點。深度學習方面，DLCV 02 詳細探討現代分類骨幹

網路，ML 03A 轉向序列資料，ML 04 則介紹卷積的主要替代方案：注意力機制。標準教科書可參考 Goodfellow 等人 [1] 的第 9 章與 Prince [10] 的第 10 章。

第一部分・用固定卷積核濾波

滑動的加權和

白話版。每個輸出像素都是輸入影像在相同位置附近像素的加權和。那一小格權重稱為**卷積核**（kernel，也稱為濾波器 filter 或遮罩 mask）。

精確版。令 I 為灰階影像，$I(x, y)$ 是第 x 欄、第 y 列的強度。令 K 為大小 $(2a+1) \times (2b+1)$ 的卷積核，垂直索引從 $-a$ 到 a、水平索引從 $-b$ 到 b，因此 $K(0,0)$ 是它的中心。濾波後的影像 G 為

$$G(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j)\, I(x + j,\; y + i).$$

其中 i 索引卷積核的列、j 索引卷積核的行，$G(x, y)$ 是相同位置的輸出，雙重總和恰好走訪卷積核的每個係數一次。以 3×3 卷積核為例，$a = b = 1$，每個輸出像素需要九次乘法與八次加法。

圖 1 手算了一個輸出值。橘色視窗蓋住填補後輸入左上角的 3×3 鄰域。與卷積核逐元素相乘再相加：$1 \cdot 1 + 0 \cdot 2 + (-1) \cdot 0 + 2 \cdot 0 + 0 \cdot 1 + (-2) \cdot 3 + 1 \cdot 2 + 0 \cdot 0 + (-1) \cdot 1 = -4$。這個數字就成為 G 在視窗中心的值。把視窗往右滑一格再重複，就填出下一個輸出。

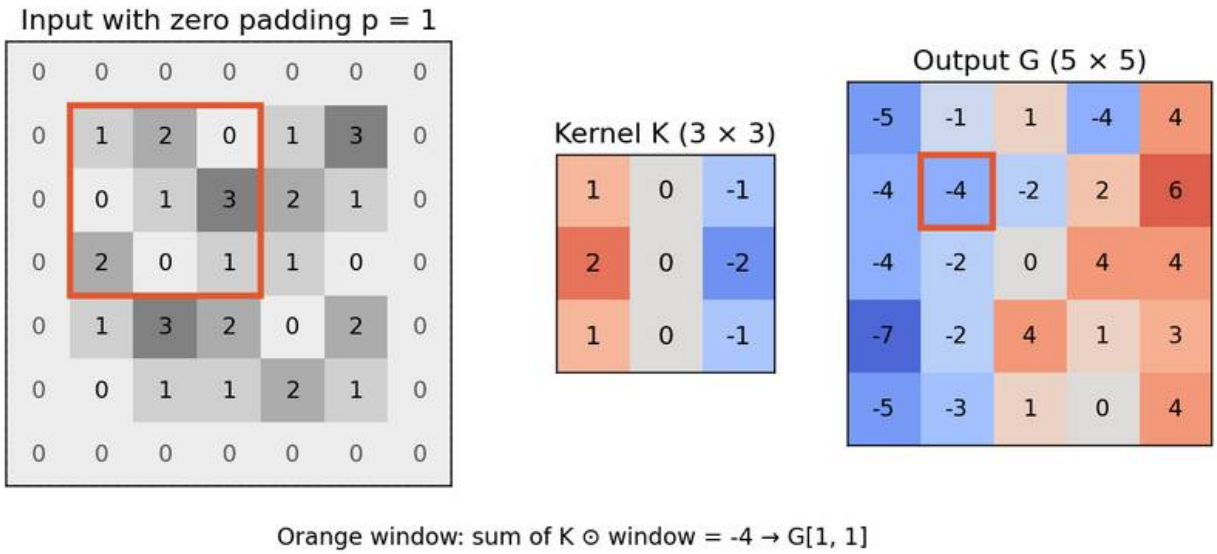


圖 *1* — 滑動加權和的一步。橘色 3×3 視窗與 K 逐元素相乘再加總，在對應的輸出位置得到 -4。零填補（灰色）讓輸出維持與輸入相同的大小。

有兩個觀察對後面所有內容都很重要。第一，同樣的九個權重用在每一個位置：這個運算具有**平移不變性**（shift-invariant，也稱為空間不變）。第二，這個運算是**線性的**：對 $\alpha I_1 + \beta I_2$ 濾波會得

到 $\alpha G_1 + \beta G_2$。線性平移不變（linear shift-invariant, LSI）運算恰好就是卷積能表達的那一類，這也是卷積在訊號處理中如此核心的原因。

互相關與卷積

白話版。上面的公式在技術上是**互相關**（cross-correlation）。真正的**卷積**做同樣的事，只是卷積核先旋轉 180 度。對稱的卷積核兩者結果相同；不對稱的則不同。

精確版。數學上的卷積寫成 $K * I$：

$$(K * I)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j)\, I(x - j,\ y - i) = \sum_{i,j} K(-i, -j)\, I(x + j,\ y + i).$$

第二種寫法顯示，卷積等於用翻轉後的卷積核 $K(-i, -j)$ 做互相關。看出差別最乾淨的方法，是對單一亮點（**脈衝**，impulse）濾波。互相關會留下一個旋轉 180 度的卷積核副本；卷積則留下卷積核的原樣副本。這個副本就是**脈衝響應**（impulse response），也是訊號處理偏好卷積的原因：任何影像的輸出，都是一堆經過平移、縮放的脈衝響應之和。

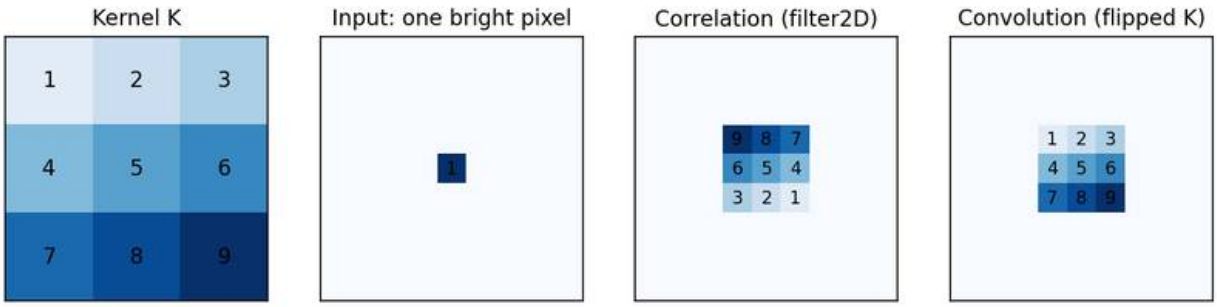


圖 2 — 對脈衝濾波。互相關（*cv2.filter2D* 的計算方式）印出旋轉 *180°* 的卷積核；卷積印出卷積核本身。

卷積具有互相關所沒有的代數性質。它滿足**交換律**（$K * I = I * K$）與**結合律**（$K_2 * (K_1 * I) = (K_2 * K_1) * I$），因此連續套用兩個濾波器，等於套用一個較大的濾波器。例如連續做兩次 3×3 方框模糊，就等於一個 5×5 卷積核。結合律也解釋了**可分離**（separable）卷積核：$k \times k$ 的高斯核等於一個 $k \times 1$ 的行濾波器再接一個 $1 \times k$ 的列濾波器，每個像素的運算量從 k^2 降到 $2k$。**卷積定理**指出，空間中的卷積等於傅立葉域中的乘法；IP 04 會發展這個觀點。

各函式庫計算的是哪一種？

函式庫呼叫	計算內容	預設邊界
`cv2.filter2D`	互相關 [11]	reflect-101（`BORDER_DEFAULT`）
`scipy.ndimage.correlate` / `convolve`	互相關 / 卷積	`reflect`（重複邊緣樣本）
`scipy.signal.convolve2d`	卷積	補零
`torch.nn.functional.conv2d`、`nn.Conv2d`	互相關 [31]	零填補

對於手工設計的不對稱卷積核（Sobel、浮雕、運動模糊），翻轉與否會改變結果的正負號或方向，所以需要真正的卷積時，要刻意翻轉，例如用 `cv2.flip(K, -1)`。對於學習出來的卷積核，這個差別就消失了：一個「本來應該」使用翻轉卷積核的網路，會直接把翻轉後的版本學起來。這就是為什麼深度學習框架把互相關稱為「卷積」也無傷大雅。

模糊、邊緣與銳化

白話版。總和為一的正權重會平均鄰居而產生模糊。總和為零的權重會抵消平坦區域、只留下變化，因此能找出邊緣。把一點點「影像減去其模糊版本」加回影像，就能銳化。

方框模糊（box blur）讓每個鄰居權重相等；3×3 版本有九個 $1/9$。**高斯模糊**取樣

$$K(i,j) = \frac{1}{Z} \exp\!\left(-\frac{i^2 + j^2}{2\sigma^2}\right),$$

其中 σ（以像素為單位）決定模糊寬度，Z 將權重正規化使總和為一。近的像素比遠的像素重要，因此能避免方框模糊的塊狀瑕疵。實務上常用約 3σ 的卷積核半徑。

權重總和告訴你在每個像素都等於 c 的平坦區域會發生什麼：輸出為 $c \sum K$。總和為一的模糊保持亮度；總和為二的卷積核會讓亮度加倍（並使 8 位元影像飽和）；總和為零的卷積核則輸出零。

Sobel 卷積核估計水平與垂直方向的導數，同時在垂直方向上做平滑：

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}, \qquad S_y = S_x^{\top}.$$

S_x 對沿 x 方向的強度變化有反應，所以它凸顯**垂直**邊界；S_y 凸顯**水平**邊界。輸出 G_x、G_y 帶有正負號：由暗到亮與由亮到暗的邊緣正負相反。**梯度強度**（gradient magnitude）$\sqrt{G_x^2 + G_y^2}$ 是非負的邊緣強度，$\operatorname{atan2}(G_y, G_x)$ 則是邊緣方向。上面的分解顯示 Sobel 是可分離的：一個 $[1,2,1]^{\top}$ 平滑行向量乘上一個 $[-1,0,1]$ 中央差分。

拉普拉斯（Laplacian）卷積核 $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ 近似 $\partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$；它對細線以及邊緣的兩側都有反應。**反銳化遮罩**（unsharp masking）以下式銳化：

$$I_{\text{sharp}} = I + \lambda\,(I - G_\sigma * I),$$

其中 $G_\sigma * I$ 是高斯模糊後的副本，$I - G_\sigma * I$ 是高頻細節，$\lambda > 0$ 控制強度。λ 太大會放大雜訊，並在邊緣兩側產生亮暗光暈。

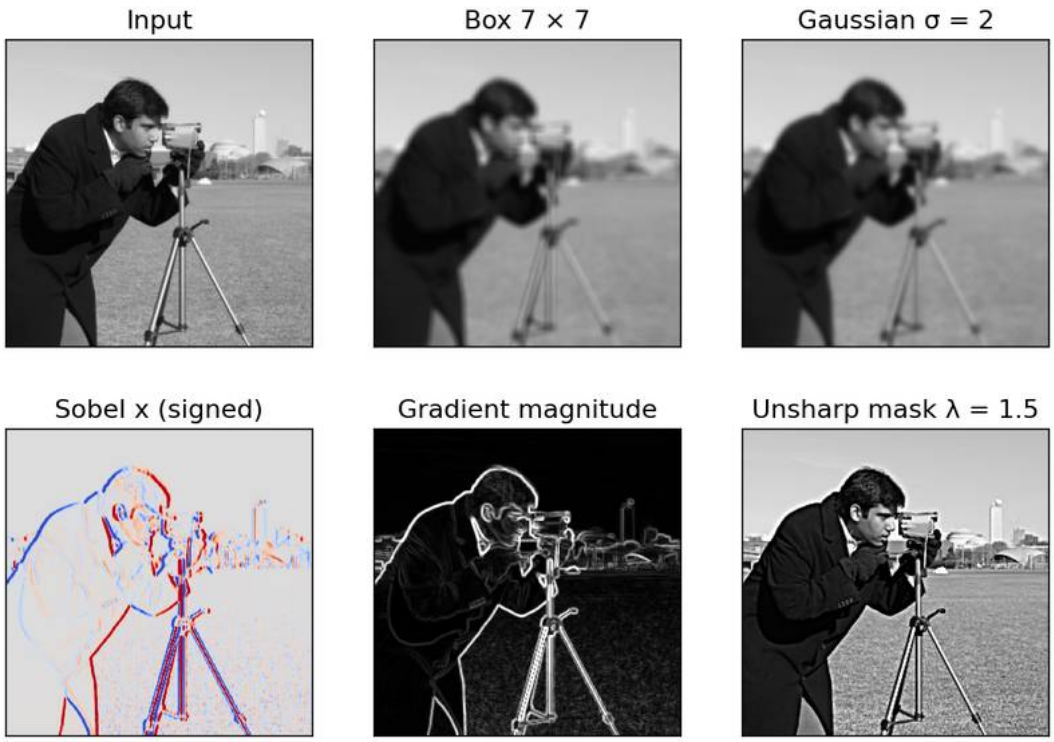


圖 3 — 同一張影像經過六種卷積核。方框與高斯模糊壓抑細節；*Sobel x* 帶有正負號（紅 = 由暗到亮，藍 = 由亮到暗）；梯度強度勾勒出物體輪廓；反銳化遮罩把放大的細節加回去。

動手試試：卷積核實驗室

編輯下方的卷積核格子，觀察輸出即時更新。先從 Sobel x 開始，然後把它轉置，改為偵測水平邊緣而非垂直邊緣。把模糊預設的每個權重都調大，你會看到平坦區域變亮，正如權重總和規則的預測。試試拉普拉斯與浮雕預設，並在看結果之前先預測它們的卷積核總和。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

下方的滑桿比較一張合成測試影像（圓盤、正方形、棋盤格與條紋）與它經過 5×5 高斯模糊（$\sigma = 1.4$）的結果。細條紋與棋盤格最先消失，因為模糊是一種**低通濾波器**：它移除快速的強度變化，保留緩慢的變化。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

程式碼：從零開始，再到 OpenCV、SciPy 與 PyTorch

下面的迴圈就是整個演算法。它不逐一走訪每個像素（在 Python 裡很慢），而是走訪卷積核的每個係數，每個係數加上一份平移過的填補影像，讓 NumPy 做向量化計算。接著程式檢查：只要

對齊定義（互相關或卷積）與邊界規則，四個函式庫的結果就會一致。需要 numpy、scipy、ope
ncv-python 與 torch。

```
import numpy as np
import cv2
import torch
import torch.nn.functional as F
from scipy import ndimage, signal

def correlate2d(img, K, mode="edge"):
    """Cross-correlation of a 2-D float image with an odd-sized kernel.
    mode is any np.pad mode: "constant" (zeros), "edge", "reflect", "wrap"."""
    kh, kw = K.shape
    ph, pw = kh // 2, kw // 2
    padded = np.pad(img, ((ph, ph), (pw, pw)), mode=mode)
    H, W = img.shape
    out = np.zeros((H, W), dtype=np.float64)
    for i in range(kh):              # loop over kernel taps, not pixels:
        for j in range(kw):          # each step adds one shifted copy of the image
            out += K[i, j] * padded[i:i + H, j:j + W]
    return out

def convolve2d(img, K, mode="edge"):
    return correlate2d(img, K[::-1, ::-1], mode)     # convolution = correlation with
     ↪  flipped K

rng = np.random.default_rng(0)
img = rng.random((32, 40))
K = rng.standard_normal((3, 5))                      # deliberately asymmetric

ours_corr = correlate2d(img, K, "reflect")
ours_conv = convolve2d(img, K, "reflect")
# OpenCV: filter2D computes correlation; BORDER_REFLECT_101 is np.pad's "reflect"
cv_corr = cv2.filter2D(img, cv2.CV_64F, K, borderType=cv2.BORDER_REFLECT_101)
# SciPy: ndimage.correlate / ndimage.convolve with mode="mirror" (= reflect-101)
sp_corr = ndimage.correlate(img, K, mode="mirror")
sp_conv = ndimage.convolve(img, K, mode="mirror")
# PyTorch: conv2d is correlation; pad first with the same rule
t = torch.tensor(img)[None, None]
t = F.pad(t, (2, 2, 1, 1), mode="reflect")
pt_corr = F.conv2d(t, torch.tensor(K)[None, None])[0, 0].numpy()

for name, a in [("OpenCV", cv_corr), ("SciPy", sp_corr), ("PyTorch", pt_corr)]:
    print(f"{name:8s} correlation matches:", np.allclose(ours_corr, a))
print("SciPy    convolution matches:", np.allclose(ours_conv, sp_conv))
print("conv == corr for this kernel?", np.allclose(ours_conv, ours_corr))
# zero padding: our convolution equals signal.convolve2d with mode="same"
same = signal.convolve2d(img, K, mode="same", boundary="fill")
print("zero-padded 'same' convolution matches:", np.allclose(convolve2d(img, K, "constant"),
 ↪  same))
```

除了 conv == corr 因為隨機卷積核不對稱而印出 False 之外，所有檢查都印出 True。注意命

名陷阱：NumPy 的 `"reflect"` 填補（`dcb|abcd|cba`，不重複邊緣樣本）對應 OpenCV 的 `BORDER_REFLECT_101` 與 SciPy 的 `"mirror"`；而 SciPy 的 `"reflect"` 會重複邊緣樣本（`dcba|abcd|dcba`）。

第二段程式碼展示了初學者處理真實影像時最常遇到的兩個錯誤：整數飽和與卷積核總和規則。

```
import numpy as np
import cv2
from skimage import data

img_u8 = data.camera()                                  # 512 x 512 uint8 test image
img = img_u8.astype(np.float32) / 255.0
sobel_x = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], np.float32)

gx = cv2.filter2D(img, -1, sobel_x)                     # float in, float out: negatives kept
gy = cv2.filter2D(img, -1, sobel_x.T)
mag = np.hypot(gx, gy)
print("float Sobel x range:", gx.min().round(3), gx.max().round(3))

gx_u8 = cv2.filter2D(img_u8, -1, sobel_x)               # uint8 in, uint8 out: saturated
print("uint8 Sobel x range:", gx_u8.min(), gx_u8.max(),
      "| pixels clipped to 0:", f"{(gx_u8 == 0).mean():.0%}")

blur = cv2.GaussianBlur(img, (5, 5), 1.4)               # 5 x 5 Gaussian, sigma 1.4
print("mean brightness kept by blur:", np.isclose(img.mean(), blur.mean(), atol=1e-3))
K2 = 2 * np.ones((3, 3), np.float32) / 9                # weights sum to 2
print("mean after sum-2 kernel:", round(float(cv2.filter2D(img, -1, K2).mean() /
 ↪  img.mean()), 3))
```

在浮點數下，Sobel 響應大約介於 −3.4 到 3.3。若輸入與輸出都是 `uint8`，所有負的響應都會被截成零（這裡超過一半的像素），因此所有由亮到暗的邊緣都消失了。請一律用 `float32` 濾波，只在顯示時才轉回整數。

邊界處理

白話版。在影像邊緣附近，卷積核有一部分會懸在影像外面。你必須決定要假裝那裡有什麼值。

常見選擇有：**零填補**（常數填補），假裝外面的世界是黑色；**複製**（replicate），重複邊緣像素；**反射**（reflect），在邊界處鏡射影像；**環繞**（wrap），把影像視為週期性的，離散傅立葉轉換隱含的就是這種假設。OpenCV 的預設 `BORDER_REFLECT_101` 會鏡射但不重複邊緣像素 [11]。PyTorch 的 `nn.Conv2d` 預設使用零填補，除非你把 `padding_mode` 設為 `'reflect'`、`'replicate'` 或 `'circular'` [31]。

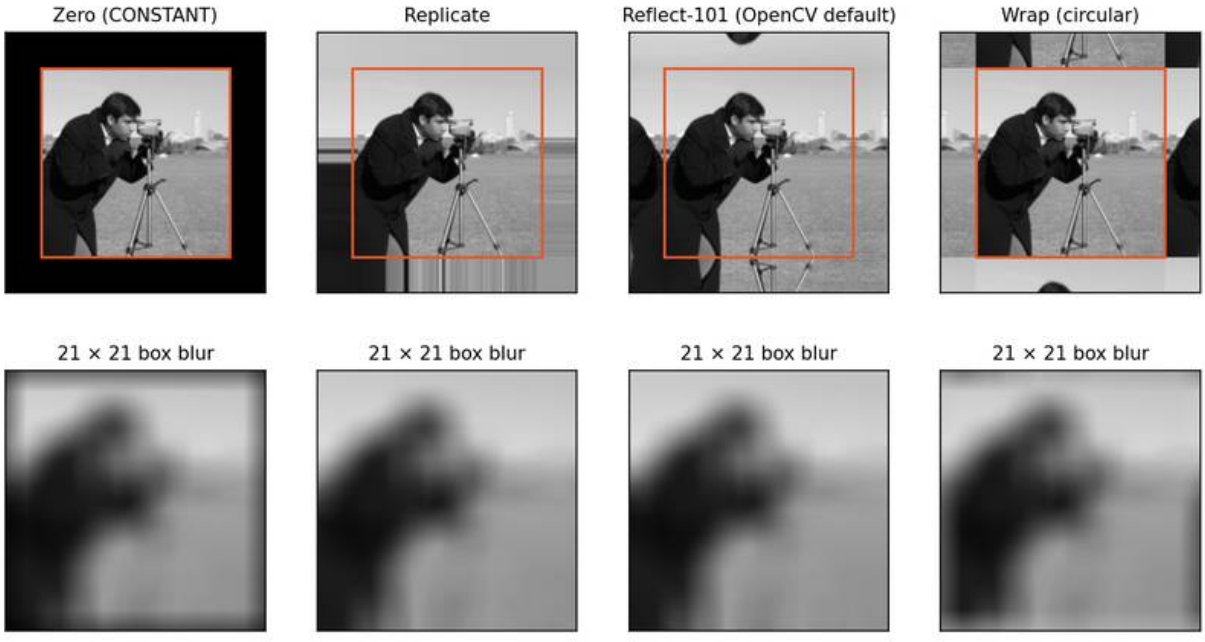


圖 *4* — 四種邊界規則及其對大型 *21 × 21* 方框模糊的影響。零填補讓每個邊框變暗；環繞混合了相對的邊；複製與反射讓平坦的邊框維持平坦。

零填補會讓模糊影像的邊框變暗，因為黑色像素被平均進去了。環繞則把影像頂部混進底部。對照片而言，複製與反射通常最不顯眼。卷積核小的時候，差別只影響一圈細框；但在深層 CNN 中，每一層都會加上自己的一圈：經過許多零填補層之後，特徵圖中很大一部分都「看過」人工邊界，這讓網路能從填補中推斷出絕對位置。

第二部分・卷積神經網路

從固定濾波器到學習式網路層

白話版。卷積層是一組濾波器，其權重以梯度下降學習。每個濾波器同時看所有輸入通道，產生一個輸出通道，稱為**特徵圖**（feature map）。

精確版。令輸入為 $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$（通道、高、寬），權重為 $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$，偏差為 $b \in \mathbb{R}^{C_{\text{out}}}$。步幅為一、零填補為 p 的網路層計算

$$Y_{o,u,v} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} K_{o,c,i,j}\, X_{c,\,u+i-p,\,v+j-p}.$$

o 索引輸出通道（濾波器），c 索引輸入通道，(u, v) 索引輸出位置，(i, j) 索引卷積核係數；影像外的位置讀取填補值。接著逐元素套用 ReLU 等非線性函數。一個 RGB 濾波器有三片 $k \times k$，每

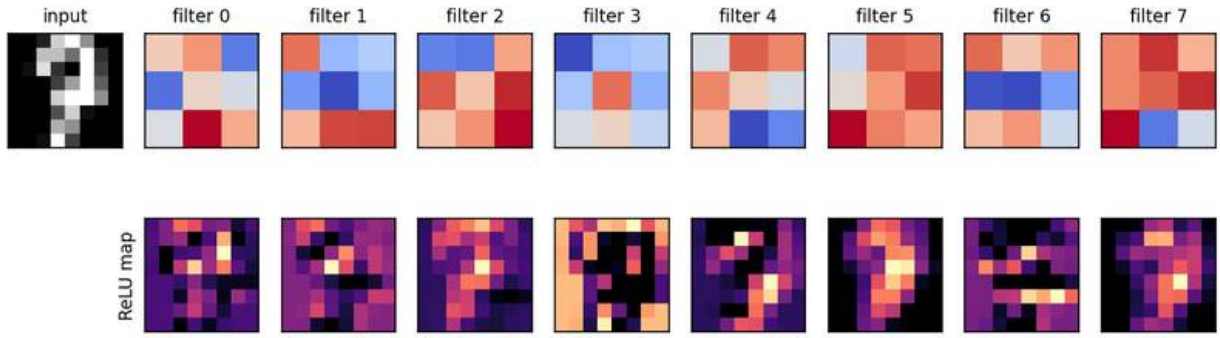
個顏色通道一片，它們的響應會**加總**成每個位置的一個數字：一個濾波器不會產生三張分開的影像。

這個公式內建了兩個結構假設：

- **局部連接**（local connectivity）：每個輸出只取決於一個 $k \times k$ 鄰域，而不是整張影像。
- **權重共享**（weight sharing，又稱參數共享）：同一個 K 用在每個位置 (u, v)。

兩者合起來就是 CNN 的**歸納偏差**（inductive bias），也就是對哪些函數比較可能的假設。它的回報是效率。一個 $C_{\text{in}} = 3$、$C_{\text{out}} = 32$、$k = 3$ 的網路層有 $32 \times (3 \cdot 9 + 1) = 896$ 個參數，與影像大小無關。若改用全連接層，把攤平的 $224 \times 224 \times 3$ 影像映射到區區 32 個數字，就需要 $150{,}528 \times 32 \approx 4.8$ 百萬個權重，而且還得在每個位置分別學會同一個邊緣偵測器。

Goodfellow 等人 [1] 把動機總結為稀疏交互作用、參數共享與等變性。圖 5 展示一個小型 CNN 在後面分類器程式碼所用的 8×8 數字資料集上訓練後，第一層學到了什麼。學到的 3×3 濾波器看起來像有方向的導數與斑點偵測器：網路自己重新發現了類似 Sobel 的卷積核，因為它們有用，而不是因為有人告訴它。Zeiler 與 Fergus [30] 透過視覺化大型 ImageNet 網路的濾波器與特徵圖，在大尺度上展示了同樣的現象。



Top: the 8 learned 3 × 3 first-layer filters (red +, blue −). Bottom: each filter's ReLU response to the input digit.

圖 *5* — 在 *8 × 8* 數字上學到的第一層濾波器（上）以及它們對一個測試數字的 *ReLU* 響應（下）。許多看起來像有方向的邊緣或筆畫偵測器。

填補、步幅與膨脹

白話版。填補（padding）控制邊界處的行為以及輸出是否維持大小。步幅（stride）跳過一些位置以縮小輸出。膨脹（dilation）把卷積核係數分散開來，不增加權重就能看得更遠。

對輸入大小 n（高或寬）、卷積核大小 k、每邊填補 p、步幅 s 與膨脹 d，輸出大小為 [12]

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

$d(k-1)+1$ 是**有效卷積核大小**：膨脹 d 會在係數之間插入 $d-1$ 個空隙。三種填補慣例有名稱：**valid**（$p = 0$，輸出縮小 $k-1$）、**same**（奇數 k 且 $s = 1$ 時 $p = (k-1)/2$，輸出維持大小）、**full**

($p = k - 1$，計入所有部分重疊，輸出增加 $k - 1$）。步幅二大致把每個空間維度減半，是池化之外常見的下採樣方式。

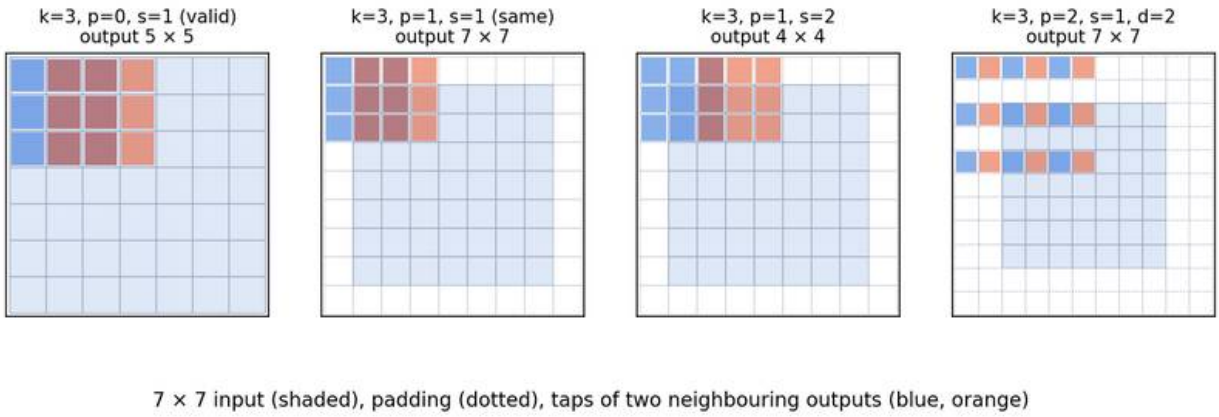


圖 *6* — 四種設定下兩個相鄰輸出（藍與橘；重疊處呈褐色）的係數位置。步幅讓視窗移動 s；膨脹讓係數相隔 d。

膨脹（atrous）卷積由 Yu 與 Koltun [18] 推廣到密集預測：堆疊膨脹率為 1、2、4、… 的網路層，可以在維持完整解析度的同時，讓視野呈指數成長。單一膨脹層只看到稀疏的輸入格點，因此混合膨脹率時要小心，以免產生「網格」（gridding）瑕疵。**轉置卷積**（transposed convolution）把帶步幅卷積的前向映射反過來執行以上採樣；它會在下面的反向傳播小節再次出現。Dumoulin 與 Visin 的卷積算術指南 [12] 為這些設定的每種組合都畫了圖解。

下面的程式碼用 PyTorch 檢查公式、計算參數量、說明 1×1 卷積就是逐像素的線性層，並量測平移等變性：

```
import torch
from torch import nn

def out_size(n, k, p=0, s=1, d=1):
    return (n + 2 * p - d * (k - 1) - 1) // s + 1

x = torch.randn(1, 3, 64, 64)                          # one RGB 64 x 64 image
for k, p, s, d in [(3, 1, 1, 1), (5, 2, 2, 1), (3, 2, 1, 2), (1, 0, 1, 1)]:
    conv = nn.Conv2d(3, 16, k, stride=s, padding=p, dilation=d)
    y = conv(x)
    n_params = sum(q.numel() for q in conv.parameters())
    assert y.shape[-1] == out_size(64, k, p, s, d)
    assert n_params == 16 * (3 * k * k + 1)
    print(f"k={k} p={p} s={s} d={d}: output {tuple(y.shape[1:])}, parameters {n_params}")

# A 1x1 convolution is the same linear layer applied at every pixel
conv1 = nn.Conv2d(16, 4, kernel_size=1)
lin = nn.Linear(16, 4)
lin.weight.data = conv1.weight.data[:, :, 0, 0].clone()
lin.bias.data = conv1.bias.data.clone()
z = torch.randn(2, 16, 8, 8)
same = torch.allclose(conv1(z), lin(z.permute(0, 2, 3, 1)).permute(0, 3, 1, 2), atol=1e-6)
```

```
print("1x1 conv == per-pixel Linear:", same)

# Translation equivariance away from borders (stride 1, no padding effects in the centre)
conv = nn.Conv2d(1, 4, 3, padding=1, bias=False)
img = torch.zeros(1, 1, 32, 32); img[..., 10:14, 8:12] = 1
shift = lambda t: torch.roll(t, shifts=(3, 5), dims=(-2, -1))
with torch.no_grad():
    a, b = conv(shift(img)), shift(conv(img))
print("shift then conv == conv then shift:", torch.allclose(a, b, atol=1e-6))
conv2 = nn.Conv2d(1, 4, 3, padding=1, stride=2, bias=False)
with torch.no_grad():
    y0, y1 = conv2(img), conv2(torch.roll(img, 1, dims=-1))   # shift by one pixel
print("stride 2, 1-pixel shift: max output change =", round((y0 - y1).abs().max().item(), 3))
```

它印出的輸出形狀為 $16 \times 64 \times 64$、$16 \times 32 \times 32$、$16 \times 64 \times 64$ 與 $16 \times 64 \times 64$，參數量分別為 448、1216、448 與 64；1×1 與等變性檢查都通過；而平移一個像素會讓步幅二的輸出產生明顯非零的變化（確切數值取決於隨機初始化的權重）。

通道與 1×1 卷積

白話版。通道是特徵圖的「是什麼」維度；高與寬是「在哪裡」維度。1×1 卷積在每個像素混合通道，不看鄰居。

具有 C_{in} 個輸入與 C_{out} 個輸出的 1×1 卷積，會把每個像素的通道向量乘上同一個 $C_{\text{out}} \times C_{\text{in}}$ 矩陣，正如上面程式碼所驗證。Lin 等人的 Network in Network [17] 把這個想法當作迷你的逐像素 MLP，並提出**全域平均池化**（global average pooling）作為分類器頭部。GoogLeNet [16] 在昂貴的 3×3 與 5×5 分支之前用 1×1 網路層減少通道數，ResNet 的瓶頸區塊 [5] 則把 3×3 層夾在 1×1 縮減層與 1×1 擴張層之間。動機都相同：空間濾波的成本是通道混合的 k^2 倍，所以便宜的部分交給 1×1 網路層，昂貴的部分保持窄小。

池化與下採樣

白話版。池化（pooling）用一個數字（通常是最大值或平均值）概括特徵圖的每個小區塊。它縮小特徵圖，並讓表示對小幅平移較不敏感。

視窗 2×2、步幅二的**最大池化**保留每個區塊中的最大值：「這附近有沒有出現這個特徵？」**平均池化**保留平均值。**全域平均池化**把每個通道在所有位置上平均，不論輸入大小，每個通道都產生一個數字，這就是現代分類器能接受不同影像大小的原因。池化沒有可學習的參數。它的梯度很簡單：最大池化把整個上游梯度交給每個視窗中勝出的輸入；平均池化則把梯度平均分給視窗內所有輸入。

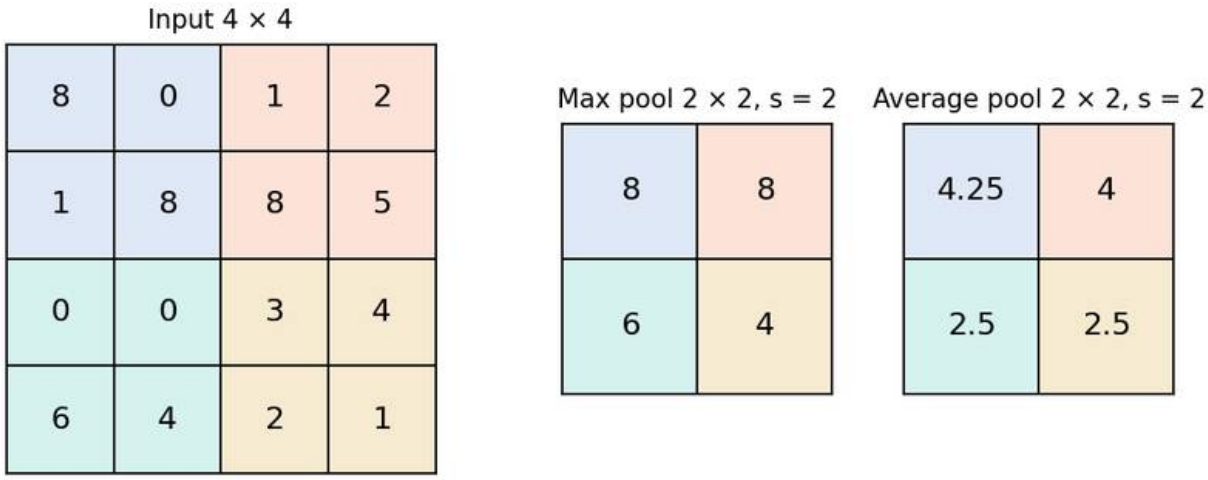


圖 7 — 視窗 2 × 2、步幅 2 的最大池化與平均池化。每個彩色區塊變成一個數字。

無論用池化或帶步幅的卷積，下採樣都能節省運算並擴大後續的感受野，但它也是一種沒有適當低通濾波的**次取樣**（subsampling）。高頻會產生混疊（aliasing），輸入只移動一個像素，輸出就可能明顯改變，正如上面程式碼所量測。Zhang [20] 證明在次取樣前插入一個小模糊（「抗混疊」或「blur pooling」），能讓標準 CNN 的平移不變性顯著提升：這是古典訊號處理規則修好深度網路的漂亮例子。

等變性不等於不變性

白話版。平移輸入，卷積特徵圖也會跟著平移。但這不等於最終預測保持不變。

令 T_Δ 把影像平移 Δ 個像素。在遠離邊界處，步幅為一的卷積具有**平移等變性**（translation-equivariant）：$f(T_\Delta X) = T_\Delta f(X)$。分類器要的是**不變性**（invariance）：$g(T_\Delta X) = g(X)$。CNN 藉由堆疊等變的網路層再加上池化（特別是會捨棄位置資訊的全域平均池化），得到近似的不變性。填補、步幅、裁切與有限的影像大小都會破壞精確的等變性。旋轉與尺度等變性**並不是**內建的；必須靠資料擴增或特別設計的群等變（group-equivariant）網路層來提供。

這個區別在實務上很重要。偵測器必須是等變的：人移動了，預測框也要跟著移動。分類器應該是不變的：人移動了，標籤不該改變。同一個骨幹網路可以服務兩者，只是頭部不同。

感受野

白話版。前面的網路層看到影像的小碎片；較深的網路層把這些碎片組合起來，看到較大的區域。

一個單元的**感受野**（receptive field）是能影響它的輸入像素集合。令 r_ℓ 為第 ℓ 層之後的感受野寬度，j_ℓ 為**跳距**（jump），即第 ℓ 層相鄰單元之間相隔的輸入像素數。從 $r_0 = j_0 = 1$ 開始，卷積核大小 k_ℓ、膨脹 d_ℓ、步幅 s_ℓ 的網路層給出

$$r_\ell = r_{\ell-1} + (d_\ell(k_\ell - 1))\, j_{\ell-1}, \qquad j_\ell = s_\ell\, j_{\ell-1}.$$

三個堆疊的步幅一 3×3 網路層看到 7×7 個像素，面積與單一 7×7 網路層相同，但權重是 $3 \cdot 9C^2 = 27C^2$ 而不是 $49C^2$，並且有三個非線性而非一個；這正是 VGG [15] 的核心設計論點。步幅二的網路層會讓跳距加倍，因此之後每一層的感受野都以兩倍速度成長。

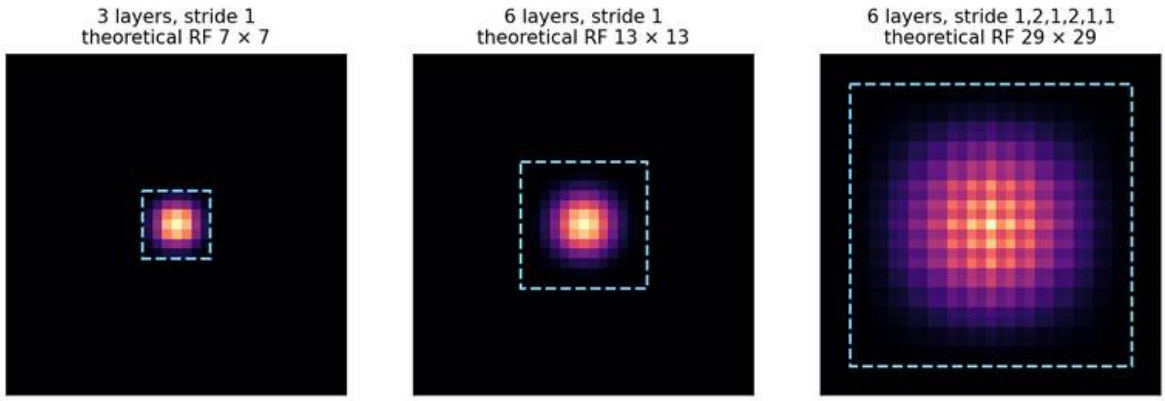


圖 *8* — 理論感受野（虛線）與實際影響力的比較，實際影響力以一個中央輸出穿過 *3 × 3* 平均網路層堆疊後的輸入梯度量測。影響力呈類高斯分布並集中在中央；步幅則加上棋盤狀圖樣。

理論感受野說明哪些像素**可能**有影響。Luo 等人 [3] 透過輸入梯度量測**有效感受野**（effective receptive field），如圖 8 所示，發現它大致呈高斯形狀，只占理論感受野的一小部分，而且對均勻的網路層堆疊而言，大約隨深度的平方根成長。由此得到兩個實務教訓：網路可能需要比理論感受野所暗示的更多上下文；而下採樣、膨脹與大型卷積核，都是不必無限加深就能擴大有效感受野的方法。

卷積的反向傳播

白話版。卷積是線性的，所以它的反向傳播也是卷積：對卷積核的梯度是把輸入與上游梯度做互相關，對輸入的梯度則是把上游梯度與卷積核做卷積。

精確版。取單一通道、無填補、步幅一：$Y_{u,v} = \sum_{i,j} K_{i,j} X_{u+i,v+j}$。令 L 為損失，$G_{u,v} = \partial L / \partial Y_{u,v}$

為反向傳播傳來的**上游梯度**，如ML 02 所述。每個權重 $K_{i,j}$ 用在每個輸出位置，所以依連鎖律，它的梯度是對這些位置的加總：

$$\frac{\partial L}{\partial K_{i,j}} = \sum_{u,v} G_{u,v}\, X_{u+i,v+j}.$$

這是輸入 X 與 G 的 valid 互相關，也說明了為什麼權重共享不會讓訓練變複雜：共享的權重只是把每個位置的梯度累加起來。每個輸入像素 $X_{m,n}$ 會貢獻給所有視窗涵蓋它的輸出，對應係數為 $(i,j) = (m-u, n-v)$：

$$\frac{\partial L}{\partial X_{m,n}} = \sum_{i,j} G_{m-i,n-j}\, K_{i,j},$$

其中 G 在其範圍外視為零。這是 G 與 K 的 **full 卷積**，等價於把零填補後的 G 與翻轉的卷積核做互相關。這正是**轉置卷積**所計算的東西，轉置卷積也因此得名。偏差的梯度就是每個輸出通道的 $\sum_{u,v} G_{u,v}$。若有多個通道，輸入梯度還要對輸出通道加總，卷積核梯度還要對批次加總。

```
import torch
import torch.nn.functional as F

torch.manual_seed(0)
X = torch.randn(1, 1, 6, 7, requires_grad=True)     # input
K = torch.randn(1, 1, 3, 3, requires_grad=True)     # kernel
Y = F.conv2d(X, K)                                   # 'valid' correlation: 4 x 5
G = torch.randn_like(Y)                              # pretend upstream gradient dL/dY
(Y * G).sum().backward()                             # so that dL/dY = G exactly

# 1) dL/dK: correlate the input with the upstream gradient
dK = F.conv2d(X.detach(), G)
# 2) dL/dX: 'full' convolution of G with K = correlation of zero-padded G with flipped K
dX = F.conv2d(F.pad(G, (2, 2, 2, 2)), K.detach().flip(-1, -2))
# 3) the same thing is what a transposed convolution computes
dX_t = F.conv_transpose2d(G, K.detach())

print("dL/dK matches autograd:", torch.allclose(dK, K.grad, atol=1e-5))
print("dL/dX matches autograd:", torch.allclose(dX, X.grad, atol=1e-5))
print("conv_transpose2d gives the same:", torch.allclose(dX_t, X.grad, atol=1e-5))
```

三行都印出 `True`。`(Y * G).sum().backward()` 這個技巧讓 autograd 使用我們指定的上游梯度 G，是測試任何手推反向傳播的好方法。

經典架構速覽

白話版。每個知名的 CNN 都是卷積、非線性與下採樣的堆疊。這些年改變的是深度、如何讓深層堆疊可以訓練，以及如何聰明地分配運算量。

模型（年份）	關鍵想法	帶來的啟示
LeNet-5（1998）[13]	卷積 → 次取樣 → 卷積 → 次取樣 → 全連接，端到端訓練	學習出來的卷積特徵在數字辨識上勝過手工特徵
AlexNet（2012）[14]	更深更寬、ReLU、dropout、資料擴增、GPU 訓練	大量標註資料加上 GPU 讓 CNN 贏下 ImageNet
VGG（2014）[15]	只用 3×3 卷積，16–19 層	以小卷積核加深既簡單又有效
GoogLeNet / Inception（2014）[16]	平行的 1×1、3×3、5×5 分支，搭配 1×1 縮減	以受控成本取得多尺度特徵
ResNet（2015）[5]	殘差連接 $y = x + F(x)$、BatchNorm	非常深的網路（100 層以上）變得可以訓練
U-Net（2015）[4]	編碼器–解碼器，在每個解析度加上跳接	用少量影像得到密集、像素對齊的輸出
MobileNetV2（2018）[7]	深度可分離卷積、反轉殘差、線性瓶頸	適合手機的高準確率模型
EfficientNet（2019）[8]	深度、寬度、解析度的複合縮放	各維度要一起縮放，而不是一次一個

模型（年份）	關鍵想法	帶來的啟示
ConvNeXt（2022）[9]	7×7 深度卷積核、LayerNorm、反轉瓶頸、Transformer 時代的訓練方法	現代化的純 CNN 可與視覺 Transformer 競爭

從 LeNet 到 ResNet 的模式是深度穩定增加。AlexNet [14] 證明，用 GPU、ReLU 與 dropout 在 ImageNet 上訓練的深層 CNN 能大幅勝過傳統流程。VGG [15] 主張使用一致的 3×3 網路層堆疊，如感受野小節所計算。GoogLeNet [16] 用 1×1 縮減讓寬度變便宜。接著深度撞上了最佳化之牆：層數更多的普通網路反而有**更高的訓練誤差**，這不是過擬合（overfitting），而是最佳化失敗。ResNet [5] 用恆等捷徑解決了這個問題。DLCV 02 會沿著這條路線繼續探討訓練方法、遷移學習與評估。

殘差區塊與正規化

白話版。不要求一個區塊計算整個輸出，而是要它計算對輸入的修正量。

殘差區塊計算 $y = x + F_\theta(x)$，其中 F_θ 通常是兩到三個卷積，搭配正規化與 ReLU。它的雅可比矩陣為 $\partial y/\partial x = I + \partial F_\theta/\partial x$：即使 F_θ 貢獻很小，恆等路徑仍能把訊號往前送、把梯度往回傳。若 $F_\theta \approx 0$ 就是一個區塊所能做到最好的，它也能輕易學會，所以增加區塊不再傷害最佳化。當形狀改變（更多通道、步幅二）時，捷徑變成投影 $P(x)$，通常是帶步幅的 1×1 卷積。

批次正規化（batch normalization）[19] 在訓練時用批次平均與變異數標準化每個通道，再套用學習出來的縮放與平移；評估時則使用移動平均。它讓訓練更快、對初始化較不敏感，但在批次非常小或相關性高時表現不佳，此時**群組正規化**（GroupNorm）[6]（在每個樣本內對通道群組計算統計量）較可靠。驗證之前一定要呼叫 `model.eval()`，讓 BatchNorm 使用移動統計量。

```
import torch
from torch import nn

count = lambda m: sum(p.numel() for p in m.parameters())

C = 64
dense = nn.Conv2d(C, C, 3, padding=1, bias=False)
separable = nn.Sequential(
    nn.Conv2d(C, C, 3, padding=1, groups=C, bias=False),   # depthwise: one 3x3 per channel
    nn.Conv2d(C, C, 1, bias=False))                        # pointwise: mix channels
print("dense:", count(dense), "| depthwise+pointwise:", count(separable),
      "| ratio:", round(count(separable) / count(dense), 4), "vs 1/C + 1/9 =",
      round(1 / C + 1 / 9, 4))

class BasicBlock(nn.Module):
    """ResNet-style block: y = relu(x + F(x)), with a 1x1 projection when shapes change."""
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False), nn.BatchNorm2d(c_out),
            ↪  nn.ReLU(),
            nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False), nn.BatchNorm2d(c_out))
        self.proj = (nn.Identity() if stride == 1 and c_in == c_out else
```

```
                    nn.Sequential(nn.Conv2d(c_in, c_out, 1, stride, bias=False),
                                  nn.BatchNorm2d(c_out)))
    def forward(self, x):
        return torch.relu(self.proj(x) + self.f(x))

x = torch.randn(2, 64, 32, 32)
print("same shape :", tuple(BasicBlock(64, 64)(x).shape))
print("downsample :", tuple(BasicBlock(64, 128, stride=2)(x).shape))
```

輸出顯示密集網路層有 36,864 個權重，可分離組合則有 4,672 個，比例 0.1267 與下面的公式一致；區塊輸出形狀分別為 $64 \times 32 \times 32$ 與 $128 \times 16 \times 16$。

高效率 CNN：把空間濾波與通道混合分開

白話版。「看周圍」（空間濾波）與「組合特徵」（通道混合）不必在同一個昂貴的運算中完成。

密集的 $k \times k$ 卷積有 $k^2 C_{\text{in}} C_{\text{out}}$ 個權重。**深度**（depthwise）卷積分別濾波每個通道（$k^2 C_{\text{in}}$ 個權重，PyTorch 中為 `groups=C`）；接著的**逐點**（pointwise）1×1 卷積混合通道（$C_{\text{in}} C_{\text{out}}$ 個權重）。當 $C_{\text{in}} = C_{\text{out}} = C$ 時，

$$\frac{\text{depthwise} + \text{pointwise}}{\text{dense}} = \frac{k^2 C + C^2}{k^2 C^2} = \frac{1}{C} + \frac{1}{k^2}.$$

當 $k = 3$ 且 C 很大時，這個比例趨近 1/9。

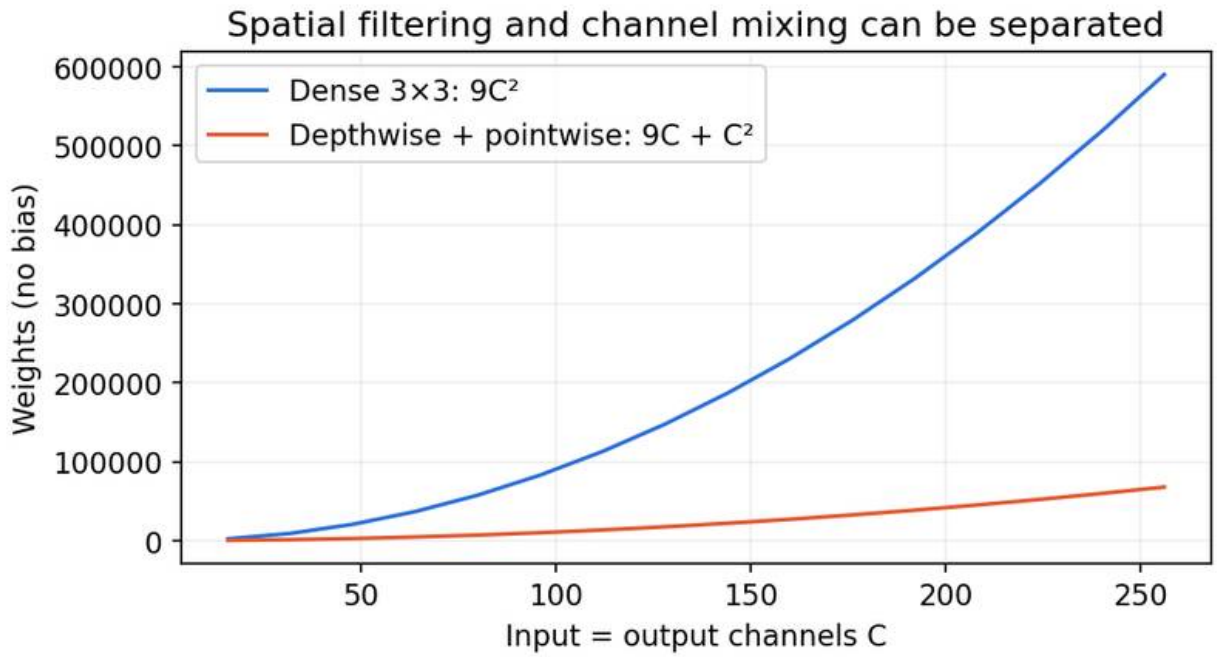


圖 *9* — 輸入與輸出通道數相同時的權重數解析計算。權重較少不代表延遲一定較低：深度卷積的運算強度低，在某些硬體上可能跑得很慢。

MobileNetV2 [7] 的區塊先用 1×1 網路層擴張通道，以深度 3×3 濾波，再以線性（無 ReLU）1×1 瓶頸投影回去，並在兩個窄端之間加上殘差連接。EfficientNet [8] 用單一複合係數同時縮放深度、寬度與輸入解析度。ConvNeXt [9] 從 ResNet 出發，一次一項地套用與視覺 Transformer 相關的設計選擇：patchify 起始層、7×7 深度卷積核、反轉瓶頸、較少的激勵函數、LayerNorm，以及搭配大量資料擴增的較長訓練。結果與相近大小的 Swin Transformer 相當，顯示 Transformer 的優勢有很大一部分來自訓練方法與巨觀設計，而不只是注意力本身。

程式碼：完整的小影像分類器

這個範例需要 `torch`、`numpy` 與 `scikit-learn`。它使用內建的 8×8 數字資料集，因此不需要下載或 GPU。分層切分得到彼此獨立的訓練、驗證與測試集：驗證集用來挑選最佳的訓練週期，測試集只在最後評估一次。這是教學用的基準，而不是 ImageNet 的訓練方法。

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
d = load_digits()
X = torch.tensor(d.images[:, None] / 16.0, dtype=torch.float32)
y = torch.tensor(d.target, dtype=torch.long)
train, test = train_test_split(np.arange(len(y)), test_size=.2,
                               stratify=d.target, random_state=0)
train, val = train_test_split(train, test_size=.2,
                              stratify=d.target[train], random_state=1)
model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),                     # B × 32 × 4 × 4
    nn.Flatten(), nn.Linear(32 * 4 * 4, 10),
)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
best_loss, best_state = float("inf"), None
for epoch in range(12):
    model.train()
    order = torch.tensor(train)[torch.randperm(len(train))]
    for idx in order.split(64):
        loss = nn.functional.cross_entropy(model(X[idx]), y[idx])
        opt.zero_grad()
        loss.backward()
        opt.step()
    model.eval()
    with torch.no_grad():
```

```
        val_loss = nn.functional.cross_entropy(model(X[val]), y[val]).item()
    if val_loss < best_loss:
        best_loss = val_loss
        best_state = copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)
model.eval()
with torch.no_grad():
    accuracy = (model(X[test]).argmax(1) == y[test]).float().mean().item()
print("parameters:", sum(p.numel() for p in model.parameters()))
print("best validation loss:", round(best_loss, 4))
print("held-out accuracy:", round(accuracy, 4))
```

在我們的 CPU 上執行，它回報 9,930 個參數，保留測試集準確率約 96%（確切數字會因 PyTorch 版本略有差異）。交叉熵接收的是 **logits**，前面不接 softmax。頭部把 $32 \times 4 \times 4$ 特徵圖攤平，而不是做全域平均池化，因為在很小的數字影像上，筆畫的位置本身就帶有資訊；對於大小不一的影像，自適應或全域池化能提供固定大小的頭部。

要檢查錯誤，而不只看分數。哪些數字配對容易混淆？把每個測試數字平移一個像素會不會變差？在把所有進步都歸功於卷積之前，先與參數量相當的 MLP 以及古典 SVM 比較，如 IP 13 所示。

現代觀點

綜述文獻。有幾篇綜述描繪了 CNN 的全貌，各有不同重點。Gu 等人 [23]（*Pattern Recognition*，2018）依元件整理 CNN 的進展：卷積變體、池化、激勵函數、損失函數、正規化與最佳化，接著是應用。Khan 等人 [22]（*Artificial Intelligence Review*，2020）依主要創新提出架構分類法：空間利用（卷積核大小）、深度、多路徑設計（捷徑、Inception 分支）、寬度、特徵圖利用（通道注意力）以及基於注意力的設計。Li 等人 [21]（*IEEE TNNLS*，2022）涵蓋從 LeNet 開始的歷史，比較一維、二維與多維卷積，並綜覽輕量化網路與應用，給出選擇超參數的經驗法則。Alzubaidi 等人 [2]（*Journal of Big Data*，2021）以 CNN 架構為中心，對深度學習做了廣泛、應用導向的回顧，並討論資料稀少、遷移學習與硬體。這些綜述都早於 ConvNeXt，因此請把它們當作 2012–2020 年代的地圖來讀。

Transformer 之後的 CNN（2022–2026）。視覺 Transformer（ML 04）促使學界重新檢視卷積，主要發現是兩者的差距比早期比較所顯示的要小：

- **現代化的 CNN**。 ConvNeXt [9] 證明經過細心現代化的 ResNet 能與 Swin 相當。ConvNeXt V2 [24]（CVPR 2023）加入以稀疏卷積實作的遮罩自編碼器預訓練，以及全域響應正規化層，把自監督預訓練（見ML 05）帶進純 CNN。
- **大型卷積核**。 RepLKNet [26]（CVPR 2022）使用大到 31×31 的深度卷積核，並以重新參數化的小卷積核降低最佳化難度，藉此擴大有效感受野。UniRepLKNet [27]（CVPR 2024）把大卷積核設計延伸到音訊、影片、點雲與時間序列。
- **動態與可變形卷積**。 InternImage [25]（CVPR 2023）把取樣位置由輸入預測的可變形卷積，擴展成用於偵測與分割的大型基礎骨幹網路。
- **規模**。 Smith 等人 [29]（2023）在大型 JFT-4B 影像資料集上預訓練 NFNet CNN，發現在可比較的預算下，其效能遵循與視覺 Transformer 相似的運算量縮放趨勢，主張在大規模下主導結

果的是運算量與資料，而不是架構。
- **邊緣裝置**。 MobileNetV4 [28]（ECCV 2024）設計了在 CPU、GPU、DSP 與加速器上都有良好延遲的區塊與神經架構搜尋，承認光看參數量無法預測速度。

CNN 在 2026 年的定位。卷積仍是下列情境的預設選擇：受延遲限制與裝置端的推論、高解析度的密集預測（復原、分割、醫學影像，見DLCV 04）、局部性偏差有幫助的小資料集，以及 CNN–Transformer 混合模型的起始層或前段。大型視覺基礎模型多半以 Transformer 為主，但它們的 patch 嵌入、上採樣解碼器與擴散 U-Net 中經常包含卷積。

開放問題。下採樣下的精確平移等變性（混疊 [20]）；以低成本內建旋轉與尺度對稱性；理解一個任務真正需要多大的有效感受野 [3]；以及架構之間的公平比較，這需要先對齊資料、資料擴增、解析度、訓練排程與運算量，才能把差異歸因於卷積或注意力。

重點整理

- 卷積核在影像上滑動，並在每個位置計算加權和。函式庫通常計算的是**互相關**；真正的卷積會翻轉卷積核，這只對不對稱的手工卷積核有影響。
- 卷積核權重的總和決定它對平坦區域的響應：總和為 1 時模糊但保持亮度；總和為 0 時偵測變化。請用浮點數濾波，並刻意選擇邊界規則。
- 卷積層是一組具有**局部連接**與**權重共享**的學習式濾波器。它的參數量 $C_{\text{out}}(C_{\text{in}}k^2 + 1)$ 與影像大小無關。
- 填補、步幅與膨脹決定輸出大小 $\lfloor (n + 2p - d(k-1) - 1)/s \rfloor + 1$。$1 \times 1$ 卷積混合通道；池化與步幅能下採樣，但可能產生混疊。
- 卷積具有平移**等變性**；不變性來自池化。感受野隨深度與步幅成長，但有效感受野遠小於理論感受野。
- 卷積的反向傳播本身也是卷積：卷積核梯度是輸入與上游梯度的互相關，輸入梯度則是轉置卷積。
- 架構史是一段關於深度（LeNet → VGG → ResNet）、有效率地使用通道（1×1、深度可分離）與更好訓練方法的故事。在預算相當時，現代化的 CNN 仍能與 Transformer 競爭。

練習

1. **設計並驗證**。寫出一個只偵測水平邊緣的 3×3 卷積核，以及另一個只偵測從左上到右下對角邊緣的卷積核。在卷積核實驗室中，或用 `cv2.filter2D` 在一張含有水平線、垂直線與對角線的合成影像上驗證兩者。

   提示

   水平邊緣是沿 y 方向的變化，所以使用 $S_y = S_x^\top$（各列為 $-1,-2,-1$ / $0,0,0$ / $1,2,1$）。對角邊緣則把導數放在跨越對角線的方向，例如 $\begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$。它的權重總和為零，所以平坦區域輸出為零。

2. **卷積核總和**。為什麼模糊卷積核的權重總和應該為一？先預測再檢查：一張中灰色的 `uint8` 影像經過權重都等於 2/9 的 3×3 卷積核濾波後會怎樣？若改用 `float32` 濾波又有什麼不同？

   提示

   數值為 c 的平坦區域會映射到 $c\sum K$。總和為二時，灰階 128 會變成 256，在 `uint8` 中飽和為 255；所有比中灰亮的區域都變成白色。在 `float32` 中數值加倍但不會被截斷，之後可以再重新縮放。

3. **組合**。證明連續做兩次 3×3 方框模糊等於一個 5×5 卷積核，並寫出這個卷積核。在遠離邊界處以數值驗證，並解釋為什麼在邊界處等式可能不成立。

   提示

   依結合律，組合後的卷積核是方框核與自己的卷積：$[1,2,3,2,1]$ 與自己的外積再除以 81。在邊界處，第一次濾波已經編造了填補值，第二次再對那個結果填補；而單次 5×5 濾波填補的是原始影像，所以兩者在寬度為二的外框中不同。

4. **網路層算術**。對 64×64 的 RGB 輸入，以及一個有 16 個濾波器、$k=5$、$p=2$、$s=2$ 的網路層：寫出輸出形狀與參數數量。接著計算三個步幅分別為 1、2、1 的 3×3 網路層之後的 (r,j)，並與所有步幅都為一的情況比較。

   提示

   $\lfloor (64+4-4-1)/2\rfloor +1=32$，所以輸出為 $16\times 32\times 32$，參數為 $16(3\cdot 25+1)=1216$。感受野：$(3,1)\to(5,2)\to(9,2)$。步幅全為一則得到 $(7,1)$。

5. **手算梯度**。對 1×4 的輸入 $X=[x_1,x_2,x_3,x_4]$ 與卷積核 $K=[k_1,k_2]$ 做 valid 互相關，寫出 Y，再以 $G=\partial L/\partial Y$ 推導 $\partial L/\partial K$ 與 $\partial L/\partial X$。用反向傳播程式碼中的 autograd 技巧檢查你的公式。

   提示

   $Y=[k_1x_1+k_2x_2,\ k_1x_2+k_2x_3,\ k_1x_3+k_2x_4]$。於是 $\partial L/\partial k_1=G_1x_1+G_2x_2+G_3x_3$、$\partial L/\partial k_2=G_1x_2+G_2x_3+G_3x_4$（$X$ 與 G 的互相關）。$\partial L/\partial X=[G_1k_1,\ G_1k_2+G_2k_1,\ G_2k_2+G_3k_1,\ G_3k_2]$，即 G 與 K 的 full 卷積。

6. **數字分類的消融實驗**。在分類器中，把 `Flatten + Linear(512, 10)` 換成全域平均池化 + `Linear(32, 10)`。比較參數量、驗證損失與測試準確率，再用平移一個像素的測試數字評估兩個模型。解釋你觀察到的取捨。

   提示

   使用 `nn.AdaptiveAvgPool2d(1)` 再接 `nn.Flatten()`。池化移除了大部分頭部參數與位置資訊，可能有助於對平移的穩健性，但在筆畫位置很重要的 8×8 數字上可能變差。其他設定保持不變，只用驗證集做選擇，並同時回報準確率與參數量。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 9. Book
2. L. Alzubaidi et al., “Review of deep learning: concepts, CNN architectures, challenges, applications, future directions,” *Journal of Big Data*, 2021. Review
3. W. Luo, Y. Li, R. Urtasun and R. Zemel, “Understanding the Effective Receptive Field in Deep Convolutional Neural Networks,” NeurIPS, 2016. Paper
4. O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” MICCAI, 2015. Paper
5. K. He, X. Zhang, S. Ren and J. Sun, “Deep Residual Learning for Image Recognition,” CVPR, 2016. Paper
6. Y. Wu and K. He, “Group Normalization,” ECCV, 2018. Paper
7. M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018. Paper
8. M. Tan and Q. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” ICML, 2019. Paper
9. Z. Liu et al., “A ConvNet for the 2020s,” CVPR, 2022. Paper
10. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023, ch. 10. Book and notebooks
11. OpenCV, “Image Filtering” (`filter2D`, border types), documentation, version 4.x. Documentation
12. V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning,” arXiv:1603.07285, 2016. arXiv
13. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, 1998. DOI
14. A. Krizhevsky, I. Sutskever and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” NeurIPS, 2012. Paper
15. K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” ICLR, 2015. arXiv
16. C. Szegedy et al., “Going Deeper with Convolutions,” CVPR, 2015. arXiv
17. M. Lin, Q. Chen and S. Yan, “Network In Network,” ICLR, 2014. arXiv
18. F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” ICLR, 2016. arXiv
19. S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015. arXiv
20. R. Zhang, “Making Convolutional Networks Shift-Invariant Again,” ICML, 2019. arXiv
21. Z. Li, F. Liu, W. Yang, S. Peng and J. Zhou, “A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects,” *IEEE Transactions on Neural Networks and Learning Systems*, 2022. DOI
22. A. Khan, A. Sohail, U. Zahoora and A. S. Qureshi, “A Survey of the Recent Architectures of Deep Convolutional Neural Networks,” *Artificial Intelligence Review*, 2020. arXiv
23. J. Gu et al., “Recent Advances in Convolutional Neural Networks,” *Pattern Recognition*, 2018. arXiv
24. S. Woo et al., “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” CVPR, 2023. arXiv

25. W. Wang et al., “InternImage: Exploring Large-Scale Vision Foundation Models with Deformable Convolutions,” CVPR, 2023. arXiv
26. X. Ding et al., “Scaling Up Your Kernels to 31x31: Revisiting Large Kernel Design in CNNs” (RepLKNet), CVPR, 2022. arXiv
27. X. Ding et al., “UniRepLKNet: A Universal Perception Large-Kernel ConvNet for Audio, Video, Point Cloud, Time-Series and Image Recognition,” CVPR, 2024. arXiv
28. D. Qin et al., “MobileNetV4: Universal Models for the Mobile Ecosystem,” ECCV, 2024. arXiv
29. S. L. Smith, A. Brock, L. Berrada and S. De, “ConvNets Match Vision Transformers at Scale,” arXiv:2310.16764, 2023. arXiv
30. M. D. Zeiler and R. Fergus, “Visualizing and Understanding Convolutional Networks,” ECCV, 2014. arXiv
31. PyTorch, `torch.nn.Conv2d` documentation. Documentation

ML 03A ・循環與遞迴神經網路

從序列與樹結構學習：共享的循環狀態、時間反向傳播、梯度消失與爆炸、LSTM 與 GRU 閘門、雙向與編碼器–解碼器模型、作為通往 Transformer 橋梁的注意力機制、遞迴網路，以及它們在電腦視覺中的應用。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-03a-recurrent-recursive-networks/

你將學到

- 循環網路如何在每個時間步重複使用同一組權重，因而能讀取任意長度的序列。
- 如何用時間反向傳播（backpropagation through time, BPTT）訓練它，以及為什麼跨越長時間的梯度會消失或爆炸。
- LSTM 與 GRU 的閘門如何建立一條受保護的記憶路徑；我們逐條寫出方程式，並與 PyTorch 的實作比對。
- 雙向、堆疊與編碼器–解碼器（序列到序列）模型如何組裝，以及注意力機制如何移除編碼器的瓶頸，進而催生 Transformer。
- 遞迴網路如何沿著樹狀結構、而不是鏈狀結構組合表示。
- 循環結構在電腦視覺中的用途（影像描述、影片、時空預測），以及 Mamba 與 Mamba-2 如何使用選擇性狀態空間、平行掃描與固定大小的串流記憶。

先看全貌

ML 03 對影像只用了一個假設：相鄰像素彼此相關，而且同一個圖樣可能出現在任何位置，所以同一個濾波器可以在**空間**上重複使用。序列也適用同樣的技巧，只是換成在**時間**上重複使用。影片是一連串影格，圖說是一連串字詞，筆跡是一連串座標。**循環神經網路**（recurrent neural network, RNN）一次讀入序列中的一個元素，把一份稱為**隱藏狀態**（hidden state）的摘要從這一步帶到下一步，並在每一步使用相同的權重 [1]。**遞迴神經網路**（recursive neural network）則把這個想法從鏈推廣到樹：它反覆套用同一條共享的組合規則，把局部拼成整體。

中文文獻常把這兩個英文名詞混用；本篇固定以「循環神經網路」指 recurrent neural network、以「遞迴神經網路」指 recursive neural network，以免混淆。

> **用最簡單的話說**
>
> 聽故事時，你不會記住每一個字，而是在腦中保留一幅「到目前為止發生了什麼」的小圖畫，每聽到新的一句就更新這幅圖。循環網路就是這樣做的：它有一本小筆記本（隱藏狀態），一次讀一個字或一張影格，每讀完一個就用同一套規則改寫筆記。LSTM 是一本附了橡皮擦和筆的筆記本，網路會學著小心使用它們，讓開頭的重要筆記到了最後也不會被塗糊。遞迴網路則像玩 LEGO：把兩塊小積木扣成一塊大的，再把大的扣成更大的，每次都用同一種扣法。

既然 Transformer 已經主宰大規模的語言與視覺任務，為什麼還要學 RNN？有三個理由。第一，現代序列建模的每一個概念——教師強制、編碼器–解碼器、注意力、遮罩、因果式生成——都是在 RNN 上發展出來的，而把 Transformer 理解成「對 RNN 局限的回答」是最容易的理解方式。第二，循環結構很適合**串流**處理：每一步只需要固定大小的狀態，適合機器人、感測器和裝置端的影片。第三，循環結構回來了：Mamba 等現代狀態空間模型本質上就是參數化方式很巧妙的循環式，而ML 04 會把它們應用到影像上。

循環狀態

白話版。在每一步，把目前的輸入和前一步的狀態結合成新的狀態，每次都使用相同的權重。需要輸出時，就從狀態讀出來。

精確版。設輸入序列為 $x_1, \dots, x_T$，其中 $x_t \in \mathbb{R}^d$；隱藏狀態為 $h_t \in \mathbb{R}^m$，起始狀態 h_0 固定（通常是零）。簡單（Elman [9]）RNN 為

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \qquad \hat{y}_t = g\big(W_y h_t + c\big).$$

$W_x \in \mathbb{R}^{m\times d}$ 把輸入映射到狀態空間，$W_h \in \mathbb{R}^{m\times m}$ 是把前一步資訊帶過來的**循環**權重矩陣，b 與 c 是偏差，W_y 把狀態映射成輸出，g 是輸出函數，例如 softmax。關鍵在於這些參數都不依賴 t。這就是**跨時間的參數共享**，相當於 CNN 權重共享在時間上的版本：不論序列有 5 步還是 5,000 步，模型的參數數量都相同，而在第 3 步學到的圖樣也能在第 300 步被辨認出來。

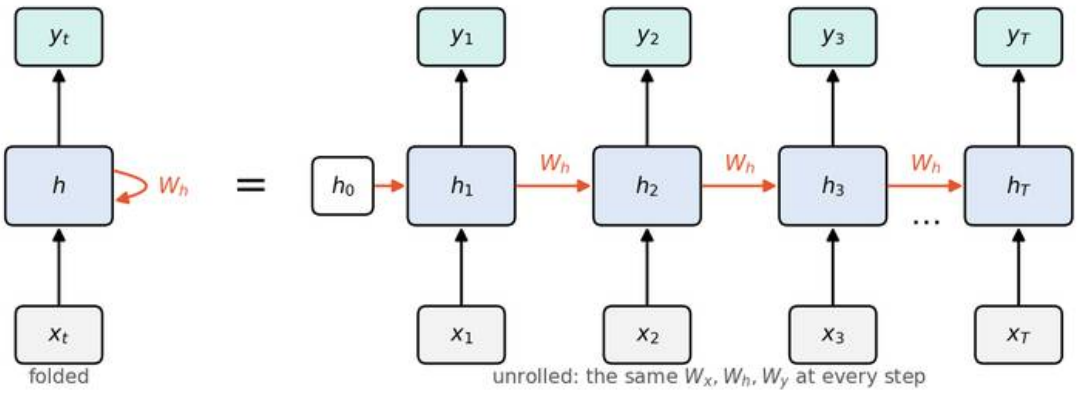


圖 *1 — RNN* 的折疊與展開兩種畫法。展開後，迴圈變成一個很深的前饋網路，而且每一層共用同一組權重。

展開（unrolling，圖 1）是最關鍵的思考步驟。把迴圈寫開 T 步，就得到一個 T 層深的普通前饋網路，而每一層的權重都相同。你在ML 02 學到的反向傳播，全部都能套用在這張展開後的計算圖上。

同一個循環式可以支援好幾種輸入–輸出型態：

型態	例子	從哪裡讀取輸出
多對一	分類一段影片；判斷一句話的情感	最後的狀態 h_T（或池化後的狀態）

型態	例子	從哪裡讀取輸出
多對多、對齊	為每張影格貼標籤；為每個字詞標註詞性	每一個 h_t
一對多	從一個影像向量生成圖說	一個把自己的輸出再餵回去的解碼器
多對多、不對齊	翻譯一句話；描述一段影片	**編碼器–解碼器**（稍後介紹）

由 $x_{\le t}$ 預測 x_{t+1} 的 RNN 是一個**自迴歸**（autoregressive）模型，從中取樣就能生成新的序列；但當作編碼器使用的 RNN 並不會自動變成生成器。決定任務的是輸出頭與損失函數。

程式碼：用 NumPy 寫出 RNN 與它的反向傳播

下面的程式碼手動實作前向傳遞與時間反向傳播，損失為 $L = v^\top h_T$，再用有限差分檢查其中一個梯度元素。請注意 `dWh += …`這一行：因為 W_h 在每一步都被使用，它的梯度是所有步貢獻的**總和**。

```
import numpy as np

rng = np.random.default_rng(0)
d, m, T = 3, 5, 7                                  # input size, state size, length
Wx = rng.normal(0, 0.5, (m, d))
Wh = rng.normal(0, 0.5, (m, m))
b = np.zeros(m)

def rnn_forward(xs, h0):
    hs = [h0]
    for x in xs:                                    # the SAME Wx, Wh, b at every step
        hs.append(np.tanh(Wx @ x + Wh @ hs[-1] + b))
    return hs

def rnn_backward(xs, hs, dhT):
    """BPTT for the loss L = dhT . h_T; returns dL/dWh and dL/dx_t for every t."""
    dWh, dxs, dh = np.zeros_like(Wh), [None] * len(xs), dhT
    for t in reversed(range(len(xs))):
        da = dh * (1 - hs[t + 1] ** 2)              # through tanh
        dWh += np.outer(da, hs[t])                  # shared weight: gradients ADD up
        dxs[t] = Wx.T @ da
        dh = Wh.T @ da                              # pass the gradient one step back
    return dWh, dxs

xs = rng.normal(size=(T, d))
hs = rnn_forward(xs, np.zeros(m))
dhT = rng.normal(size=m)
dWh, dxs = rnn_backward(xs, hs, dhT)

# finite-difference check of one entry of dL/dWh
eps, (i, j) = 1e-6, (2, 3)
Wh[i, j] += eps; up = dhT @ rnn_forward(xs, np.zeros(m))[-1]
Wh[i, j] -= 2 * eps; down = dhT @ rnn_forward(xs, np.zeros(m))[-1]
Wh[i, j] += eps
```

```
print("BPTT vs finite difference:", np.isclose(dWh[i, j], (up - down) / (2 * eps)))
print("|dL/dx_t| from first to last step:", np.round([np.linalg.norm(g) for g in dxs], 3))
```

它會印出 `True`，而輸入梯度的範數為 $[0.009, 0.015, 0.006, 0.029, 0.383, 0.212, 0.433]$：即使序列只有七步，早期輸入收到的梯度也比後期小了大約十到五十倍。這就是縮小版的梯度消失問題。

時間反向傳播

白話版。把網路展開，在展開後的計算圖上執行普通的反向傳播，再把每個共享權重的梯度加總起來。

精確版。把激活前的值寫成 $a_t = W_x x_t + W_h h_{t-1} + b$，因此 $h_t = \tanh(a_t)$。假設損失是各步的總和 $L = \sum_t L_t(h_t)$。從 $t = T$ 往回推，抵達 h_t 的總梯度為

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \left(\frac{\partial h_{t+1}}{\partial h_t}\right)^{\top} \frac{\partial L}{\partial h_{t+1}}, \qquad \frac{\partial h_{t+1}}{\partial h_t} = \operatorname{diag}(1 - h_{t+1}^2)\, W_h.$$

第一項是第 t 步的局部損失；第二項是 h_t 影響到的所有後續部分，往回傳了一步。令 $\delta_t = (1 - h_t^2) \odot \partial L / \partial h_t$（激活前的梯度），參數梯度就是對時間的總和：

$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^{T} \delta_t\, h_{t-1}^{\top}, \qquad \frac{\partial L}{\partial W_x} = \sum_{t=1}^{T} \delta_t\, x_t^{\top}, \qquad \frac{\partial L}{\partial b} = \sum_{t=1}^{T} \delta_t.$$

這個演算法稱為**時間反向傳播（BPTT）**，Werbos [8] 對它有清楚的闡述。它的計算量與 T 成線性，但記憶體也一樣：每個 h_t 都必須保存下來，供反向傳遞使用。

截斷式 BPTT

對非常長的資料流（一小時的影片、連續的感測紀錄）而言，保存所有狀態是不可能的。**截斷式 BPTT**（truncated BPTT）把資料流切成每段 k 步的視窗來處理。它會把隱藏狀態從一個視窗帶到下一個視窗，所以模型仍然「看得到」完整的過去；但它在每個視窗邊界把狀態從計算圖上**切斷**（detach），所以梯度最多只往回傳 k 步。因此，模型只有在視窗內的梯度訊號促使它學會保存長距離資訊時，才能利用這些資訊。

```
import torch
from torch import nn

torch.manual_seed(0)
rnn, head = nn.GRU(1, 16, batch_first=True), nn.Linear(16, 1)
params = list(rnn.parameters()) + list(head.parameters())
opt = torch.optim.Adam(params, lr=1e-2)
t = torch.arange(0, 2000) * 0.05
stream = torch.sin(t) + 0.1 * torch.randn_like(t)   # one long noisy signal
x, y = stream[:-1, None][None], stream[1:, None][None]   # predict the next sample

h, window = None, 50
```

```
for start in range(0, x.shape[1] - window, window):
    out, h = rnn(x[:, start:start + window], h)
    loss = nn.functional.mse_loss(head(out), y[:, start:start + window])
    opt.zero_grad()
    loss.backward()                       # gradients flow back at most `window` steps
    nn.utils.clip_grad_norm_(params, 1.0)
    opt.step()
    h = h.detach()                        # keep the state, cut the graph
print("final window MSE:", round(loss.item(), 4), "(noise variance is 0.01)")
```

把資料流走過一遍之後，最後一個視窗的均方誤差約為 0.014，接近加入雜訊所設定的 0.01 下限。如果忘了 `h.detach()`，第二個視窗會試圖反向傳播進一張已經被釋放的計算圖，PyTorch 會丟出錯誤；反過來，若在每個視窗都把 `h = None` 重設，就等於把上下文丟掉了。

梯度消失與梯度爆炸

白話版。從第 t 步傳回第 s 步的梯度，在每一步都會乘上大致相同的矩陣。反覆相乘不是把它縮到接近零，就是把它放大到爆掉，所以單純的 RNN 很難學到超過幾十步的依賴關係。

精確版。把每一步的 Jacobian 串起來，得到

$$\frac{\partial h_t}{\partial h_s} = \prod_{k=s+1}^{t} \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=s+1}^{t} \operatorname{diag}\!\left(1 - h_k^2\right) W_h,$$

這是 $t-s$ 個矩陣依序相乘的乘積（矩陣乘法不可交換，所以順序很重要）。因為 $|\tanh'| \le 1$，

$$\left\| \frac{\partial h_t}{\partial h_s} \right\| \le \left(\sigma_{\max}(W_h) \right)^{t-s},$$

其中 $\sigma_{\max}$ 是最大奇異值。若 $\sigma_{\max}(W_h) < 1$，來自遠處步驟的梯度會以指數速度**消失**。若 W_h 有很大的奇異值、而且神經元沒有飽和，梯度就可能**爆炸**。Bengio、Simard 與 Frasconi [6] 在 1994 年證明，這不是某種特定訓練方法的缺陷：一個能在雜訊下穩健保存資訊的系統，必然處於長距離依賴梯度會縮小的狀態。Pascanu、Mikolov 與 Bengio [7] 用動態系統的工具重新分析這個問題，並提出處理梯度爆炸的實用方法。

用純量線性 RNN $h_t = \lambda h_{t-1} + x_t$ 最容易看出直覺：此時 $\partial h_t / \partial h_s = \lambda^{t-s}$；當 $\lambda = 0.9$ 時是 $0.9^{80} \approx 2 \times 10^{-4}$，當 $\lambda = 1.1$ 時是 $1.1^{80} \approx 2 \times 10^{3}$。圖 2 顯示在真實網路中量測到的相同行為。

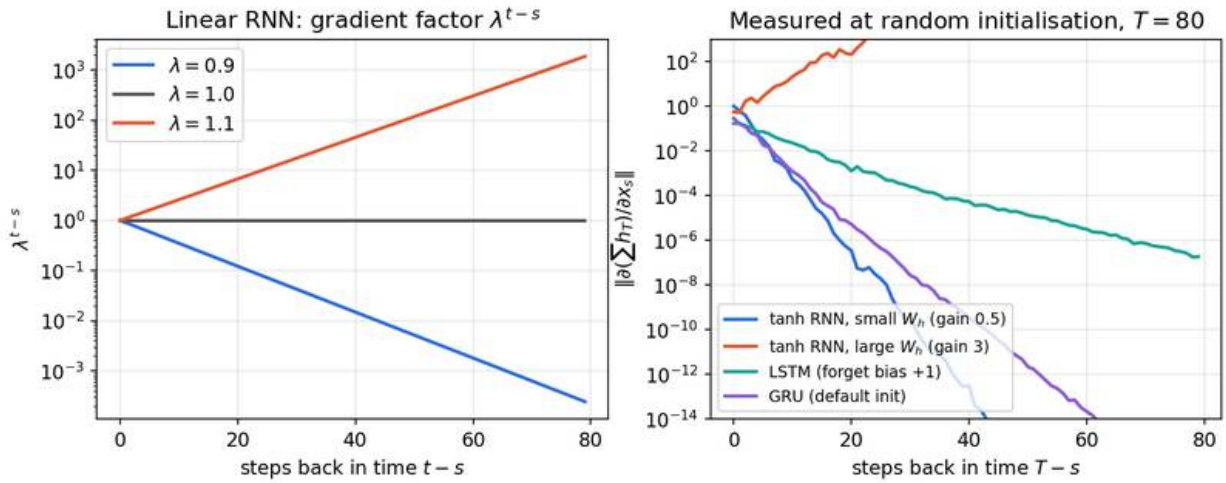


圖 2 — 左：純量因子 $\lambda^{(t-s)}$。右：在隨機初始化下量測最後狀態對每個輸入的梯度（$T = 80$）。單純的 *tanh RNN* 不是消失就是爆炸；*LSTM* 的加法記憶路徑衰減得慢得多。

對策。梯度爆炸用**梯度裁剪**（gradient clipping）[7] 處理：若梯度範數 $\|g\|$ 超過門檻 τ，就把它縮放成 $g \leftarrow \tau\, g/\|g\|$。這樣能保留更新方向、同時限制步長。PyTorch 的 `clip_grad_norm_` 做的正是這件事。梯度消失就比較難處理：需要換一種架構，讓狀態更新接近恆等映射，梯度才能穿過許多步。這種架構就是 LSTM。

長短期記憶（LSTM）

白話版。給網路一條獨立的記憶通道，也就是**細胞狀態**（cell state），它透過加法而不是反覆的矩陣乘法來更新，並讓學習得到的**閘門**（gate）決定要擦掉什麼、寫入什麼、顯露什麼。

精確版。以 $[x_t, h_{t-1}]$ 表示串接、σ 表示 logistic sigmoid，標準的 LSTM 一步計算為

$$
\begin{aligned}
f_t &= \sigma\big(W_f [x_t, h_{t-1}] + b_f\big) && \text{forget gate} \\
i_t &= \sigma(W_i [x_t, h_{t-1}] + b_i) && \text{input gate} \\
o_t &= \sigma(W_o [x_t, h_{t-1}] + b_o) && \text{output gate} \\
\tilde{c}_t &= \tanh(W_c [x_t, h_{t-1}] + b_c) && \text{candidate memory} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{cell update} \\
h_t &= o_t \odot \tanh(c_t) && \text{exposed state}
\end{aligned}
$$

每個閘門都是一個介於 0 到 1 之間的數值向量，而 $\odot$ 是逐元素相乘，所以每個閘門就像一排調光開關，每個記憶單元各一個。遺忘閘（forget gate）f_t 決定保留多少舊記憶，輸入閘（input gate）i_t 決定寫入多少候選記憶 $\tilde{c}_t$，輸出閘（output gate）o_t 決定把多少記憶顯露成 h_t。

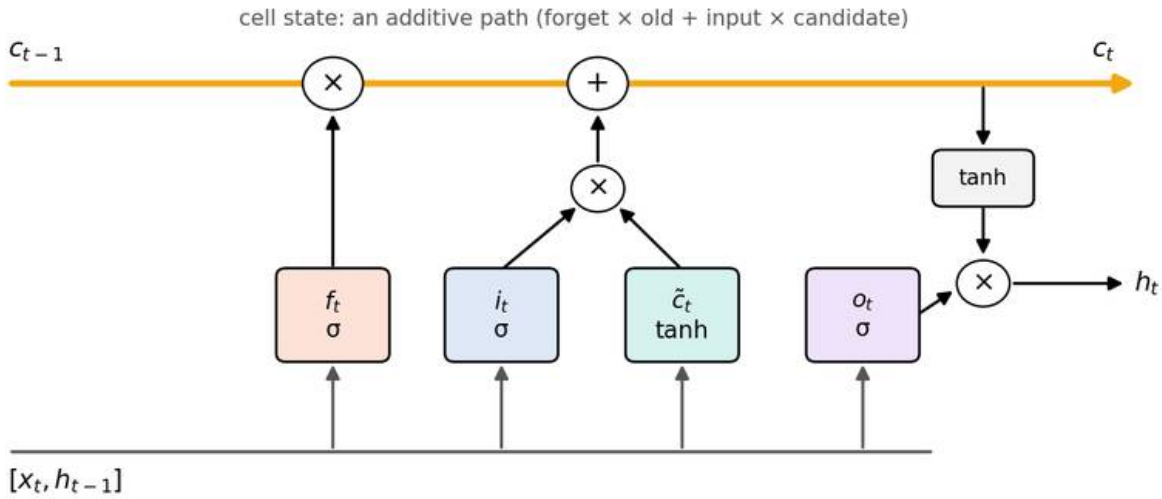


圖 *3* — *LSTM* 細胞的內部。細胞狀態（金色）只經過逐元素縮放與加法，這給了梯度一條穿越時間的受保護路徑。

為什麼有效。沿著細胞狀態路徑，$\partial c_t / \partial c_{t-1} = \mathrm{diag}(f_t)$（忽略經過閘門的較小間接路徑）。這條路徑上沒有反覆出現的權重矩陣，也沒有壓縮型的非線性函數。只要遺忘閘維持在接近 1，梯度就能幾乎不變地穿過許多步。這就是圖 2 中 LSTM 曲線衰減得這麼慢的原因，也是常見的初始化會把遺忘閘偏差設成正值（例如 +1）的原因：讓網路一開始就傾向於記住。但這並不保證無限長的記憶：遺忘閘能夠、也確實會學著關閉，而長距離的功勞分配（credit assignment）仍然會隨距離變得更困難。

歷史與出處。 Hochreiter 與 Schmidhuber 在 1997 年提出 LSTM [2]，當時用的是「恆定誤差旋轉木馬」(constant-error carousel)：細胞狀態原封不動地往下傳，**沒有遺忘閘**。Gers、Schmidhuber 與 Cummins 在 2000 年加入遺忘閘 [5]，讓細胞能學會重設記憶，例如在連續資料流中相互獨立的片段之間。上面這組含遺忘閘的方程式，才是實務上與所有主流框架使用的版本。Greff 等人 [19] 在語音、手寫與音樂任務上比較了八種 LSTM 變體，發現沒有一種顯著勝過標準架構，而遺忘閘與輸出激活函數是其中最關鍵的元件。

參數量。四個閘門區塊各有一個 $m \times (d + m)$ 的權重矩陣和一個偏差，所以一層 LSTM 大約有 $4m(d + m + 1)$ 個參數，是相同狀態大小的簡單 RNN 的四倍。（PyTorch 把輸入偏差和循環偏差分開存，所以是 $4m(d + m + 2)$。）

程式碼：從零寫出一步 LSTM，並與 PyTorch 比對

PyTorch 依照輸入閘、遺忘閘、候選細胞（g）、輸出閘的順序把四個閘門區塊堆疊在一起。親手實作一步，再和 `nn.LSTMCell` 比較，是確認自己沒讀錯方程式的最好方法：

```
import torch
from torch import nn

torch.manual_seed(0)
```

```
d, m = 4, 6
cell = nn.LSTMCell(d, m)                # PyTorch stacks gate weights in order i, f, g, o

def lstm_step(x, h, c):
    z = x @ cell.weight_ih.T + cell.bias_ih + h @ cell.weight_hh.T + cell.bias_hh
    i, f, g, o = z.chunk(4, dim=-1)
    i, f, o = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
    g = torch.tanh(g)                   # candidate memory, often written c~_t
    c = f * c + i * g                   # additive cell update
    h = o * torch.tanh(c)
    return h, c

x = torch.randn(2, d)
h0, c0 = torch.randn(2, m), torch.randn(2, m)
h_ref, c_ref = cell(x, (h0, c0))
h_ours, c_ours = lstm_step(x, h0, c0)
print("matches nn.LSTMCell:", torch.allclose(h_ours, h_ref, atol=1e-6),
      torch.allclose(c_ours, c_ref, atol=1e-6))
print("parameters:", sum(p.numel() for p in cell.parameters()), "= 4 * m * (d + m + 2) =",
      4 * m * (d + m + 2))
```

它會印出 `True True` 以及 `288 = … = 288`。

閘控循環單元（GRU）

白話版。 GRU 是精簡版的 LSTM：它把細胞狀態和隱藏狀態合而為一，並用兩個閘門取代三個。

精確版。依照 Cho 等人 [3] 以及 PyTorch 的 `nn.GRU`：

$$
\begin{aligned}
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{reset gate} \\
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{update gate} \\
\tilde{h}_t &= \tanh(W x_t + r_t \odot (U h_{t-1}) + b) && \text{candidate} \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t && \text{interpolation}
\end{aligned}
$$

重設閘（reset gate）r_t 決定在形成候選狀態時使用多少舊狀態；更新閘（update gate）z_t 在保留舊狀態（$z_t \approx 1$）與替換成新狀態（$z_t \approx 0$）之間內插。這個內插扮演了 LSTM 加法細胞路徑的角色。有兩點值得注意。有些論文與教科書會把 z_t 與 $1 - z_t$ 的角色對調；兩種模型等價，但閘門的意義正好相反。此外，原始論文把重設閘放在循環矩陣之前（$U(r_t \odot h_{t-1})$），而 PyTorch 與 cuDNN 則像上式一樣放在之後。在替閘門賦予意義之前，務必先查閱框架的文件。

GRU 有三個閘門區塊而不是四個，所以在相同寬度下參數比 LSTM 少約 25%。兩者都沒有在所有任務上穩定勝出；當資料或運算資源有限時，GRU 是合理的預設選擇。

程式碼：能推廣到新序列的循環分類器

這個完整的 CPU 範例學習判斷序列總和的正負號，必須把全部十二步的資訊累積起來才辦得到。一組全新的測試資料用來確認模型不是只把訓練序列背下來。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
rnn = nn.GRU(input_size=1, hidden_size=24, batch_first=True)
head = nn.Linear(24, 2)
opt = torch.optim.Adam(list(rnn.parameters())+list(head.parameters()), lr=.003)
def batch(n):
    x = torch.randn(n, 12, 1)
    y = (x.sum(dim=(1, 2)) > 0).long()
    return x, y
for step in range(400):
    x, y = batch(64)
    _, h = rnn(x)
    loss = F.cross_entropy(head(h[-1]), y)
    opt.zero_grad(); loss.backward()
    nn.utils.clip_grad_norm_(list(rnn.parameters())+list(head.parameters()), 1.)
    opt.step()
rnn.eval(); head.eval()
with torch.no_grad():
    x, y = batch(512)
    _, h = rnn(x)
    acc = (head(h[-1]).argmax(1) == y).float().mean()
    print("fresh-sequence accuracy:", acc.item())
```

它在長度 12 的新序列上達到約 99% 的準確率。即使設定了 `batch_first=True`，回傳的隱藏狀態張量形狀仍是 `layers × batch × hidden`；這個旗標只影響輸入與輸出序列的排列方式。用更長的序列測試是在問另一個問題——長度推廣（length generalization）——模型可能在長度 12 成功、卻在長度 100 失敗（見練習 2）。

深層、雙向與串流 RNN

堆疊。深層 RNN 把某一層循環網路輸出的隱藏狀態序列，餵給另一層循環網路（PyTorch 中的 `num_layers`）。較低的層可以捕捉短距離的圖樣，較高的層則捕捉較長距離的圖樣。常見的是二到四層；更深的堆疊通常需要在層與層之間加入殘差連接。

雙向。 Schuster 與 Paliwal [10] 提出讓一個 RNN 沿時間往前跑、另一個往後跑，再把兩者的狀態串接起來：$h_t = [\vec{h}_t; \overleftarrow{h}_t]$。如此每個位置都同時總結了它的過去與未來。這非常適合整段輸入都已取得的**離線**任務（標註錄好的影片、編碼一個句子），但它**不具因果性**：不能用在必須在未來影格抵達之前就做出預測的即時任務。把離線雙向模型的準確率拿來報告線上應用，會高估部署後系統實際能掌握的資訊。

變動長度。一個批次裡的序列長度不一，會被填補（padding）到相同長度。若不小心處理，填補值會滲入狀態與損失。請使用每條序列的實際長度（PyTorch 的 `pack_padded_sequence`）或

遮罩，讓填補的步驟既不更新狀態、也不計入損失。

串流。對即時資料流，要在多次呼叫之間保留狀態，只在彼此獨立的資料流之間重設。不小心把某位使用者、某份文件或某段影片的狀態帶到下一個，會洩漏資訊並造成無法重現的結果。

編碼器–解碼器模型與教師強制

白話版。要把一個序列轉換成另一個長度不同的序列（把句子翻譯成另一種語言、把影片轉成描述），就用一個 RNN 把輸入讀成摘要，再用第二個 RNN 根據這份摘要，一次一個符記地寫出輸出。

精確版。在 Sutskever、Vinyals 與 Le [11] 的**序列到序列**（sequence-to-sequence, seq2seq）模型，以及 Cho 等人 [3] 密切相關的編碼器–解碼器中，**編碼器** RNN 讀入 $x_1, \dots, x_T$，其最終狀態成為上下文向量 $c = h_T$。以 c 初始化的**解碼器** RNN 建模

$$p(y_1, \dots, y_U \mid x) = \prod_{u=1}^{U} p(y_u \mid y_{<u}, c), \qquad s_u = \mathrm{RNN}(s_{u-1}, y_{u-1}),$$

其中 s_u 是解碼器狀態，$p(y_u \mid \cdot)$ 是在詞彙表上的 softmax。當解碼器輸出序列結束符記時生成就停止，所以輸入與輸出的長度互不相關。

教師強制（teacher forcing）。訓練時，解碼器在第 u 步的輸入是**真實答案**中的前一個符記 y_{u-1}，而不是模型自己的預測。這讓所有步驟都能針對已知目標平行訓練，大幅穩定學習。然而在測試時，模型必須吃進自己的預測，而這些預測可能含有訓練時從未見過的錯誤。這種訓練與測試之間的落差稱為**曝光偏差**（exposure bias）。排程取樣（scheduled sampling）[25] 在訓練過程中逐步用模型的取樣取代真實輸入，以減輕這個問題。解碼時通常使用貪婪搜尋或集束搜尋（beam search），而不是隨機取樣。

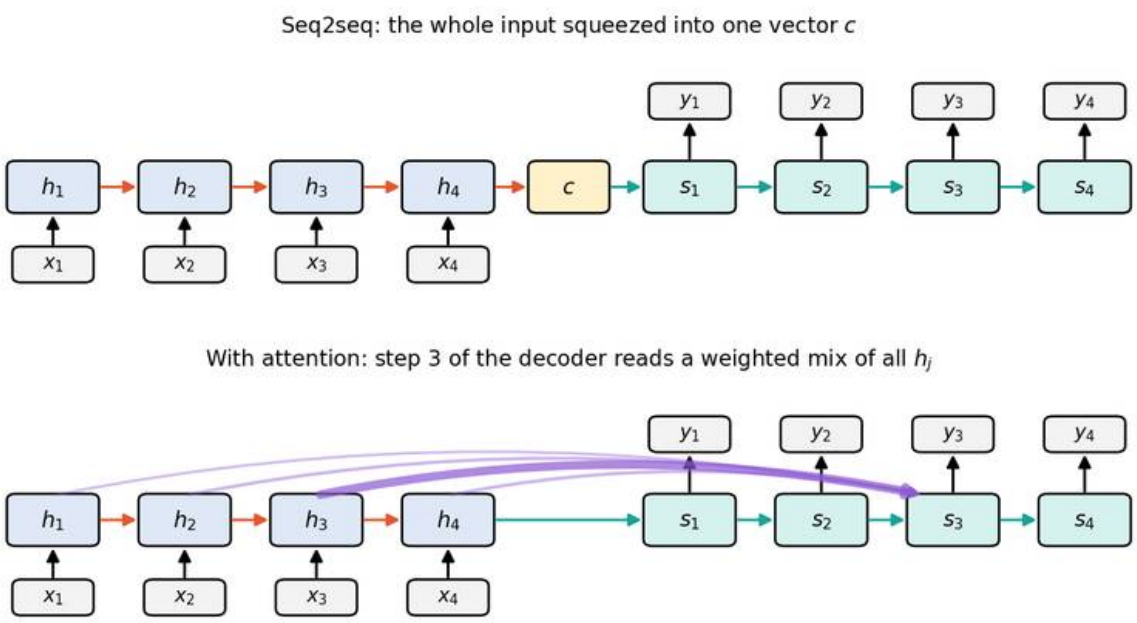


圖 4 — 上：單純的 *seq2seq* 把整段輸入擠進一個向量 c。下：加上注意力後，解碼器的每一步都讀取所有編碼器狀態經學習得到的加權組合。

圖 4（上）清楚顯示了弱點：解碼器對輸入所知道的一切，都必須通過一個固定大小的向量 c，不論輸入有多長。單純 seq2seq 模型的翻譯品質在長句子上會下降。Sutskever 等人發現，單純把**來源句子反轉**就有幫助，因為這縮短了對應字詞之間的距離——這暗示問題出在瓶頸與過長的路徑 [11]。

注意力：通往 Transformer 的橋梁

白話版。與其強迫解碼器只依賴一個摘要向量，不如讓它在每一步都回頭看**所有**編碼器狀態，並自行決定此刻哪些最相關。

精確版。 Bahdanau、Cho 與 Bengio [12] 保留了每一個編碼器狀態 $h_1, \dots, h_T$（來自雙向編碼器），並在解碼器的每一步 u 計算

$$e_{u,j} = v^\top \tanh\!\big(W s_{u-1} + U h_j\big), \qquad \alpha_{u,j} = \frac{\exp(e_{u,j})}{\sum_{k=1}^{T} \exp(e_{u,k})}, \qquad c_u = \sum_{j=1}^{T} \alpha_{u,j}\, h_j .$$

$e_{u,j}$ 是解碼器當下需求（s_{u-1}）與編碼器位置 j 之間經學習得到的**對齊分數**（alignment score）；W、U 與 v 是可學習參數。softmax 把分數轉成皆為正、總和為一的權重 $\alpha_{u,j}$，而**上下文向量** c_u 就是用這些權重對編碼器狀態做的加權平均。解碼器接著把 c_u 與自己的狀態一起使用。這種形式稱為**加法注意力**（additive attention）。因為 c_u 在每一步都會改變，而且能直接存取每一個輸入位置，任何輸入到任何輸出之間的路徑長度都變成一步，與序列長度無關。

```
import torch
from torch import nn
```

```
class AdditiveAttention(nn.Module):
    """Bahdanau-style scores e_j = v^T tanh(W s + U h_j), then softmax over j."""
    def __init__(self, d_dec, d_enc, d_att):
        super().__init__()
        self.W = nn.Linear(d_dec, d_att, bias=False)
        self.U = nn.Linear(d_enc, d_att, bias=False)
        self.v = nn.Linear(d_att, 1, bias=False)

    def forward(self, s, H, mask=None):
        # s: B x d_dec (decoder state), H: B x T x d_enc (all encoder states)
        e = self.v(torch.tanh(self.W(s)[:, None] + self.U(H))).squeeze(-1)   # B x T
        if mask is not None:                        # True where the input is padding
            e = e.masked_fill(mask, float("-inf"))
        alpha = e.softmax(dim=-1)                   # attention weights, rows sum to 1
        context = (alpha[:, :, None] * H).sum(1)    # B x d_enc weighted average
        return context, alpha

torch.manual_seed(0)
att = AdditiveAttention(d_dec=16, d_enc=32, d_att=8)
H = torch.randn(2, 5, 32)
s = torch.randn(2, 16)
pad = torch.tensor([[False] * 5, [False, False, False, True, True]])   # 2nd sequence has
 ↪  length 3
ctx, alpha = att(s, H, pad)
print("context:", tuple(ctx.shape), "| weights sum to 1:",
      torch.allclose(alpha.sum(-1), torch.ones(2)))
print("weights on padding:", alpha[1, 3:].tolist())
```

它會印出形狀為 `(2, 32)` 的上下文，確認權重總和為一，並顯示填補位置的權重恰好為零：在 softmax 之前用 $-\infty$ 遮罩，是讓注意力忽略填補（以及在ML 04 中忽略未來）的標準做法。

注意力權重也可以解讀成一種軟性的**對齊**（alignment）。圖 5 顯示一個帶有這種注意力的小型 GRU 編碼器–解碼器，在 CPU 上訓練不到一分鐘，學習把隨機數字序列反轉（在教師強制下，對新序列的逐符記準確率約 96%）。沒有人告訴它，它卻學會了在寫第 u 個輸出時去看第 $T+1-u$ 個輸入位置：也就是反對角線。

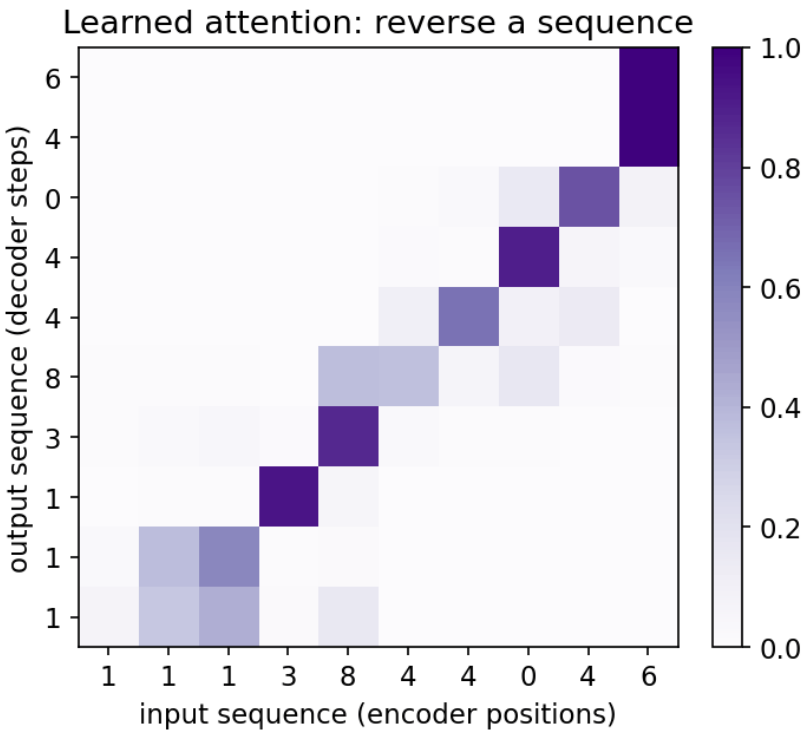


圖 5 — 訓練來反轉序列的玩具編碼器–解碼器的注意力權重（由圖片腳本產生）。每個輸出步驟都集中在鏡射位置的輸入上。

從這裡走向 Transformer。注意力解決了瓶頸，但編碼器與解碼器仍然是循環式的，所以處理長度 T 的序列需要 T 個無法在訓練時平行化的循序步驟。Vaswani 等人 [28] 問：如果完全拿掉循環，**在編碼器與解碼器內部也使用注意力**（自注意力，self-attention），再用位置編碼恢復順序，會怎麼樣？答案就是 Transformer：它能在所有位置上平行訓練，成為語言領域的主流架構，並且隨著 ViT 成為視覺領域的主流架構。ML 04 會從縮放點積注意力（scaled dot-product attention）接續這個故事，它以 $q^\top k/\sqrt{d_k}$ 取代加法分數。

循環網路在電腦視覺中的應用

影像描述（image captioning）。 Show and Tell [13]（Vinyals 等人）是一個直接的影像到序列模型：一個 CNN（ML 03）把影像編碼成一個向量，用它初始化一個 LSTM 語言模型，再由這個語言模型逐字生成圖說，並以教師強制在影像–圖說配對上訓練。Show, Attend and Tell [14]（Xu 等人）保留 CNN 的空間特徵網格而不是單一向量，讓 LSTM 在生成每個字時對影像區域做注意力，所產生的注意力圖常常會凸顯正在被說出名稱的物體。這是最早廣為人知的視覺注意力展示之一。

影片理解。長期循環卷積網路（Long-term Recurrent Convolutional Networks, LRCN）[15]（Donahue 等人）對每張影格套用 CNN、跨影格套用 LSTM，並端到端訓練，用於動作辨識、影像描述與影片描述。CNN 處理空間，RNN 處理時間：這是處理任何逐影格特徵序列的常見模式，例

如姿態軌跡，或物件追蹤中的逐影格偵測結果。

時空預測。當狀態本身就應該是一張影像時，全連接的閘門會浪費空間結構。Shi 等人的 **ConvLSTM** [16] 把 LSTM 閘門中的矩陣乘法換成卷積，$i_t = \sigma(W_{xi} * X_t + W_{hi} * H_{t-1} + \dots)$，所以輸入、狀態與閘門都是三維張量（通道 × 高 × 寬）。它最初是為了從雷達圖做降雨即時預報而提出，如今廣泛用於影片預測與其他網格上的預測任務。

發生了什麼改變。在大規模的影像描述與影片分類上，Transformer 與視覺–語言模型（見ML 09）已大致取代 LSTM，因為它們能平行訓練，而且隨著資料增加擴展得更好。循環與卷積–循環模型仍常見於影格逐張抵達、記憶體必須維持固定的場合：線上影片處理、追蹤、機器人與低功耗裝置。

遞迴網路：沿著樹組合

白話版。遞迴網路沿著一棵樹，套用同一條共享的組合規則，把兩個子節點的表示合併成父節點，直到抵達根節點。

精確版。對一棵葉節點帶有向量（詞嵌入、區域特徵）的二元樹，每個內部節點計算

$$h_{\text{parent}} = \tanh\!\big(W\,[h_{\text{left}}; h_{\text{right}}] + b\big),$$

每個節點都使用**同一個** $W \in \mathbb{R}^{D \times 2D}$ 與 b。根節點向量代表整個結構，可以拿來分類；每個節點也可以有自己的標籤。訓練使用**結構反向傳播**（backpropagation through structure）：和 BPTT 是同一個想法，只是沿著樹的邊進行。長度為 n 的鏈是完全不平衡的樹這個特例；平衡樹把任一葉節點到根節點的路徑從 $n-1$ 步縮短到大約 $\log_2 n$ 步（圖 6），縮短了梯度要走的路。

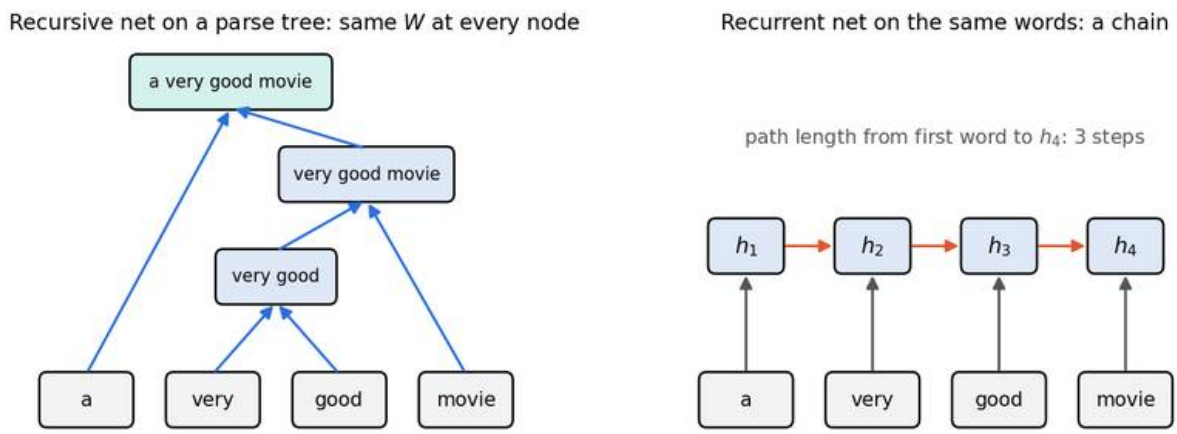


圖 6 — 遞迴網路沿著樹前進（左）；循環網路沿著鏈前進（右）。兩者都共享權重，但樹反映了輸入本身的結構。

Socher 等人 [18] 最早是在視覺情境中使用遞迴網路：他們把影像切成超像素，接著同時學習把相鄰區域合併成場景剖析樹、以及計算區域表示，並用同一套方法剖析句子。他們後來的遞迴神經張量網路（Recursive Neural Tensor Network）與 Stanford Sentiment Treebank [17] 顯示，沿著剖析樹組合能捕捉否定（“not very good”）之類的現象。Tai、Socher 與 Manning 的 **Tree-LSTM**

[4] 為樹狀組合加入 LSTM 式的閘門，每個子節點各有一個遺忘閘，讓父節點能有選擇地保留來自每棵子樹的資訊。

```
import torch
from torch import nn

torch.manual_seed(0)
D = 8
vocab = {"a": 0, "very": 1, "good": 2, "movie": 3}
emb = nn.Embedding(len(vocab), D)
compose = nn.Linear(2 * D, D)            # ONE weight matrix shared by every internal node
classify = nn.Linear(D, 2)               # e.g. negative / positive sentiment

def encode(tree):
    """tree is a word (leaf) or a pair (left, right)."""
    if isinstance(tree, str):
        return emb(torch.tensor(vocab[tree]))
    left, right = tree
    return torch.tanh(compose(torch.cat([encode(left), encode(right)])))

tree = ("a", (("very", "good"), "movie"))          # (a ((very good) movie))
root = encode(tree)
loss = nn.functional.cross_entropy(classify(root)[None], torch.tensor([1]))
loss.backward()                                     # backpropagation through structure
print("root vector:", tuple(root.shape), "| loss:", round(loss.item(), 4))
print("gradient reached the shared W:", compose.weight.grad.abs().sum().item() > 0)
swapped = encode((("movie", ("good", "very")), "a"))
print("swapping children changes the root:", not torch.allclose(root, swapped))
```

遞迴就是一般的 Python 遞迴呼叫，而 autograd 會即時建立樹狀的計算圖。交換子節點會改變結果，因為串接讓左右兩個位置扮演不同的角色。

實務上的主要限制在於樹本身：得有人提供它（剖析器、分割階層），而且要有效率地把形狀各異的樹組成批次並不容易。同時學習結構與表示是可行的，但更困難。在目前的實務中，自注意力能在沒有明確樹結構的情況下，學到柔性、隨輸入而變的分組，這也是遞迴網路如今少見的原因之一；不過階層式組合的想法仍延續在階層式視覺骨幹網路與場景圖（scene graph）中。

Mamba：選擇性狀態空間與掃描

有用的記憶應該能在干擾之間保留重要觀察，並在情境改變時覆寫舊資訊。Mamba [21] 讓記憶更新取決於目前輸入，同時維持更新對前一個狀態的仿射形式。這個差別讓模型保有循環的解讀方式，又能用平行前綴掃描（parallel prefix scan）計算整段序列。

從連續動態到選擇性更新

先從小型的連續時間記憶開始。穩定的轉移使儲存的資訊逐漸衰減；輸入投影寫入新資訊；輸出投影讀出記憶：

$$\frac{dh(\tau)}{d\tau} = Ah(\tau) + Bu(\tau), \qquad y(\tau) = Ch(\tau).$$

其中 τ 是連續時間，$u(\tau) \in \mathbb{R}$ 是單一輸入通道，$h(\tau) \in \mathbb{R}^N$ 是該通道的 N 維狀態，$A \in \mathbb{R}^{N\times N}$ 是轉移矩陣，$B \in \mathbb{R}^{N\times 1}$ 負責寫入輸入，$C \in \mathbb{R}^{1\times N}$ 負責讀取狀態，$y(\tau) \in \mathbb{R}$ 是輸出。完整的神經網路區塊會以可學習投影混合多個通道，並加入局部卷積與閘控 [21]。這個方程式只抽出記憶機制。

在每個區間內保持輸入與寫入矩陣不變。精確的零階保持（zero-order hold）離散化得到

$$\bar{A}_t = \exp(\Delta_t A), \qquad \bar{B}_t = \left(\int_0^{\Delta_t} \exp(sA)\, ds\right) B_t,$$
$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t u_t, \qquad y_t = C_t h_t.$$

其中 t 是離散 token 的索引，$\Delta_t > 0$ 是區間長度，s 是積分變數，u_t 是固定的輸入，B_t, C_t 是目前的寫入與讀取矩陣，上橫線表示離散係數。即使 A 是奇異矩陣，積分形式仍然成立；使用反矩陣的公式則需要額外假設。若 A 是對角矩陣，這些運算可以逐元素執行。

選擇性表示從目前 token 預測 Δ_t, B_t, C_t。例如，可學習的仿射投影接上 $\mathrm{softplus}(z) = \log(1 + e^z)$，就能產生正的區間長度。底層可學習的 A 可以保持固定，但 $\bar{A}_t$ 仍會透過 Δ_t 改變。當對角元素為負時，較長的區間會降低舊狀態的保留比例。選擇性的寫入與讀取也決定哪些內容進入或離開記憶。因此，即使每次狀態更新都是仿射的，整體映射對輸入仍是非線性的。圖 7 使用合成係數單獨呈現保留效果，並未量測訓練好的模型。

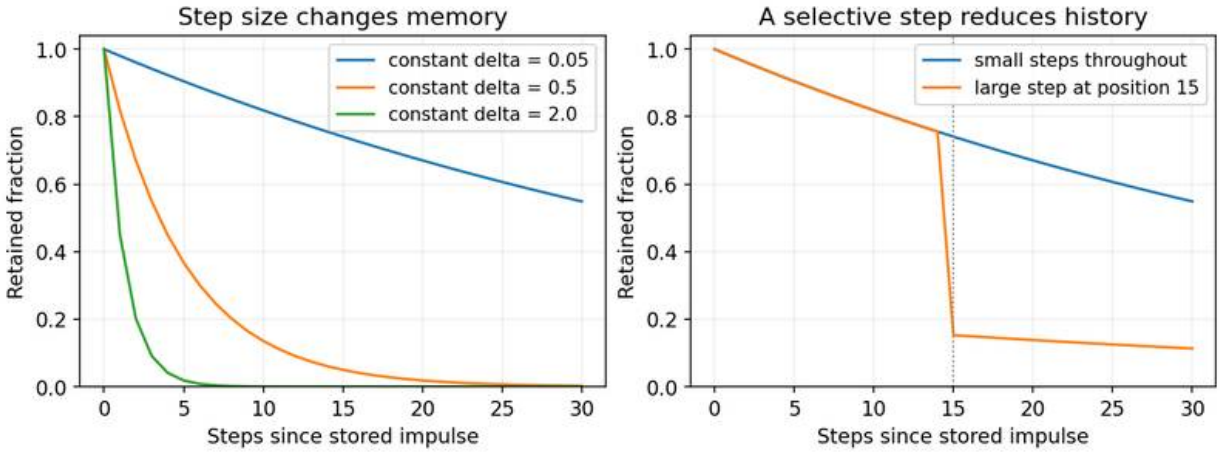


圖 7 — 純量轉移 $A = -0.4$ 時，歷史保留比例是 $exp(-0.4\ delta)$ 的連乘。選定的大區間會明顯降低舊記憶，小區間則能保留得更久。

為什麼這種循環可以掃描

輸入相依的係數算好後，組合兩次仿射更新仍然只需要一次仿射更新：

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 a_1,\ a_2 b_1 + b_2).$$

其中配對 (a_i, b_i) 表示映射 $h \mapsto a_i h + b_i$，$i = 1, 2$ 標示連續的兩次更新，。表示先套用第一次，再套用第二次。組合具有結合律；對角轉移的乘法是逐元素運算。因此，前綴掃描可以把運算重新分組成樹狀結構，卻保留時間順序造成的效果。一般 LSTM ／ GRU 的閘門取決於前一個隱藏狀態，通常無法在狀態尚未算出之前先準備這些係數。

以下可獨立執行的 CPU 範例使用精確的對角離散化，以及手動指定的選擇性係數，檢查因果性與分塊之間的狀態連續性。這是教學用的循環，並非預訓練 Mamba 實作或速度基準。

```
import numpy as np

rng = np.random.default_rng(44)
u = rng.normal(size=24)
A = -np.array([0.1, 0.8])  # two stable diagonal entries

def selective_scan(values, state=None):
    h = np.zeros(2) if state is None else state.copy()
    outputs = []
    for x in values:
        delta = np.logaddexp(0.0, 0.5 * x)
        B = np.array([1.0, np.tanh(x)])
        C = np.array([1.0, 0.5 + 0.1 * np.tanh(x)])
        a = np.exp(delta * A)
        b = np.expm1(delta * A) / A * B * x
        h = a * h + b
        outputs.append(C @ h)
    return np.array(outputs), h

y, final_state = selective_scan(u)
changed = u.copy()
changed[12:] += 10.0
changed_y, _ = selective_scan(changed)
assert np.allclose(y[:12], changed_y[:12])
left, boundary = selective_scan(u[:12])
right, resumed_state = selective_scan(u[12:], boundary)
assert np.allclose(y, np.r_[left, right])
assert np.allclose(final_state, resumed_state)
reset_right, _ = selective_scan(u[12:])
assert not np.allclose(right, reset_right)
print('causal prefix and carried-state chunk checks passed')
```

輸出確認兩項檢查都通過。在批次邊界把狀態歸零會改變計算；反過來，在獨立範例之間共用狀態則會洩漏資訊。串流實作必須追蹤每個狀態屬於哪段序列，並在真正的序列邊界重設。

Mamba-2 與注意力的連結

Mamba-2 [29] 同時改變內部序列混合器與外圍區塊。其結構化狀態空間對偶性（structured state-space duality）在每個頭內使用「純量乘上單位矩陣」的轉移，並把計算切成適合加速器硬體執行密集矩陣乘法的區塊。它也重新安排參數投影與正規化。這比在原始 Mamba 區塊中加入注意力更為具體。

初始狀態為零時，純量轉移的循環可以展開為

$$y_t = \sum_{j=1}^{t} \left[C_t \left(\prod_{k=j+1}^{t} a_k \right) B_j \right] u_j.$$

其中 j 索引先前輸入，k 索引中間的轉移，a_k 是乘在單位矩陣上的離散純量轉移，B_j 是離散寫入列向量，C_t 是讀取行向量，空的連乘等於一。括號內是具有結構的因果混合係數。這提供類似注意力的矩陣觀點，卻不需要一般 softmax 注意力，也不必儲存完整矩陣。Hu 等人 [30]（ICML 2026）把對偶性延伸到對角轉移，並釐清為何無法直接延伸到標準 softmax 注意力。

在固定寬度與狀態維度下，循環推論只攜帶固定大小的狀態；一般具有快取的因果注意力則保留 key 與 value，儲存量隨上下文長度增加。壓縮能省下記憶體，但精確回想許多互不相關細節的能力也受到考驗。序列成本呈線性並不代表訓練記憶體固定：活化值、梯度與重新計算策略都會影響結果。應一起評估檢索品質、上下文長度與裝置實測延遲。影像特有的排序問題接著在ML 04 討論。

現代觀點

綜述文獻。 Goodfellow 等人書中的第 10 章 [1] 至今仍是循環與遞迴網路、回聲狀態（echo-state）想法與長期依賴問題最清楚的統一論述。Lipton、Berkowitz 與 Elkan [20] 在 2015 年寫了一篇 RNN 序列學習的批判性回顧：以時間順序、自成一體地交代三十年來通往 LSTM 與雙向 RNN 的研究，統一了互相衝突的記號與名稱，並涵蓋影像描述、翻譯與手寫辨識等應用。Greff 等人 [19] 與其說是綜述，不如說是大型實證研究，但它是「LSTM 哪些元件重要」這個問題的參考依據。Tiezzi 等人近年的兩篇回顧，則為 Transformer 時代重新定位循環結構：一篇詳盡的綜述 [26] 把 Transformer–RNN 混合模型、具循環形式的注意力變體，以及深度狀態空間模型（一類特殊的線性 RNN）統一放在「循環」這個大傘之下，並強調從可能無限長的資料流中線上學習、而非處理固定長度序列所帶來的研究機會；另一篇刊登於 *Nature Machine Intelligence* 的較短觀點文章 [27]，則重新檢視位於 Transformer 與狀態空間模型交會處的循環處理。

循環的回歸（2023–2026）。 Transformer 處理所有位置兩兩之間的關係，記憶體與運算量隨長度呈平方成長。一系列模型重新採用循環結構，以固定大小的狀態取得線性時間的推論：

- **線性循環單元**。 Orvieto 等人 [24]（ICML 2023）證明，一個經過精心參數化的**線性**循環式——對角的複數狀態矩陣、穩定的初始化與正規化——在長距離基準測試上可媲美深度狀態空間模型，而且非線性可以放在循環式之外。
- **選擇性狀態空間**。 Mamba [21] 讓狀態空間的參數成為輸入的函數，所以模型能像 LSTM 的閘門那樣選擇要儲存或忽略什麼，同時以考慮硬體特性的平行掃描（parallel scan）維持訓練效率。視覺上的變體，以及它們對影像是否真有必要的爭論，會在ML 04 討論。
- **復興的 LSTM 與 GRU**。 xLSTM [22]（NeurIPS 2024）為 LSTM 加入指數型閘門與矩陣形式的記憶，並把它擴展到語言模型的規模。Feng 等人 [23]（2024）移除 LSTM 與 GRU 閘門對隱藏狀態的依賴（“minLSTM”、“minGRU”），使訓練時能使用平行掃描，並報告了與近期序列模型相當的結果。

共同的教訓是：為 LSTM 發明的**閘控、加法式狀態更新**是對的；限制經典 RNN 的主要是循序訓練。讓循環式對狀態呈線性（因此能用平行掃描計算），就能恢復平行訓練，同時保留便宜的串流

推論。

開放問題。超出訓練長度範圍的長度推廣；在非常長的資料流上、不用完整 BPTT 的功勞分配；固定大小的狀態相較於注意力不斷成長的快取究竟能記住多少——對需要精確回想遠處細節的任務而言，這是實際存在的限制；以及為二維影像與三維影片選擇掃描順序，因為它們沒有唯一自然的序列順序。

重點整理

- RNN 在每一步套用同一個共享更新 $h_t = f(x_t, h_{t-1})$，所以能用固定數量的參數處理任意長度的序列。展開後，它就是一個權重綁定的深層網路。
- BPTT 就是在展開後的計算圖上做普通的反向傳播；共享權重的梯度對時間加總。截斷式 BPTT 會把狀態往前帶，但限制梯度往回傳的距離。
- 長距離梯度是 Jacobian 的連乘，傾向於消失或爆炸。裁剪能處理爆炸；具有近似恆等、加法式狀態路徑的架構能處理消失。
- LSTM 的閘門（遺忘、輸入、輸出）與 GRU 的閘門（重設、更新）控制一條受保護的記憶路徑。遺忘閘是 2000 年才加入 1997 年 LSTM 的。務必確認框架的閘門慣例。
- 雙向編碼器使用未來的上下文，不具因果性；串流模型必須小心管理狀態。
- Seq2seq 以教師強制把序列對應到序列；注意力移除了它單一向量的瓶頸，而不含循環的自注意力就成了 Transformer。
- 遞迴網路以一條共享規則沿著樹組合。在電腦視覺中，循環結構推動了影像描述、影片模型與 ConvLSTM 預測，而閘控線性循環以狀態空間模型的形式重新回歸。
- Mamba 從輸入預測寫入、讀取與區間係數，同時讓更新對狀態維持仿射形式；Mamba-2 透過結構化對偶性高效率地分塊計算。壓縮省下快取記憶體，也限制精確回想能力。

練習

1. **因果性**。你的團隊報告一個使用雙向 LSTM 的動作辨識模型，逐影格準確率達 92%，並提議把它部署成即時警報系統。這個計畫有什麼問題？你要如何公平地評估可部署的替代方案？執行選擇性掃描範例，再加上反向掃描：哪個因果性檢查應該失敗？

   提示

   第 t 張影格上的反向 LSTM 已經讀過 t 之後的影格，而即時系統還沒看到這些影格。請重新訓練一個單向（因果）模型，或允許固定的前瞻延遲，並以完全相同的延遲做評估。把準確率表示成延遲的函數來報告。反向掃描讀過被改動的未來，因此即使正向前綴不變，合併後的前綴通常仍會改變。

2. **長度推廣**。用長度 12 訓練 GRU 總和分類器，接著在不重新訓練的情況下，用長度 6、24、100 與 400 的新序列評估。畫出準確率對長度的圖。再用 LSTM、以及用長度 6–24 混合訓練，重複一次。

提示

每種長度都要產生新的資料。總和的尺度隨 $\sqrt{T}$ 成長，所以即使邏輯正確，狀態在長輸入上也可能飽和。混合長度訓練通常比更換細胞種類更有幫助。在長度 12 上成功，本身並不能說明長度 400 的表現。

3. **梯度上界**。對純量 RNN $h_t = \tanh(w h_{t-1} + x_t)$，證明 $|\partial h_t / \partial h_s| \le |w|^{t-s}$。梯度在 w 取哪些值時可能爆炸？為什麼 tanh 的飽和會抑制爆炸？

提示

每個因子都是 $w(1 - h_k^2)$，且 $0 \le 1 - h_k^2 \le 1$。爆炸需要 $|w| > 1$，但很大的 $|w|$ 也會把 h_k 推向 ± 1，使 $1 - h_k^2$ 變小。梯度只會在停留於非飽和區域的軌跡上爆炸，這就是實務上爆炸往往是突發性的原因。

4. **LSTM 的記憶**。在 LSTM 方程式中，假設從 s 到 t 的每一步都有 $f_t = 1$ 且 $i_t = 0$。c_t 用 c_s 表示是多少？沿著細胞路徑的 $\partial c_t / \partial c_s$ 是多少？如果像 1997 年原始設計那樣，把遺忘閘永遠固定為 1，在一個永不結束的輸入流上會發生什麼事？

提示

$c_t = c_s$ 且 $\partial c_t / \partial c_s = I$：完美的記憶與完美的梯度流動。若 $f_t \equiv 1$ 而寫入不為零，細胞狀態會在無止盡的資料流上無限累積，這正是 Gers 等人讓網路學會遺忘所解決的問題。

5. **注意力的形狀與遮罩**。在加法注意力的程式碼中，一個批次的編碼器狀態形狀為 $8 \times 40 \times 256$（其中有些序列經過填補），解碼器狀態形狀為 8×512。請給出 e、α 與上下文的形狀，並說明若用 0 而不是 $-\infty$ 遮罩填補位置，會出什麼問題。

提示

e 與 α 是 8×40；上下文是 8×256。分數 0 只是一個普通的分數：$e^0 = 1$ 仍會給填補位置正的權重，上下文就會混入沒有意義的填補狀態。

6. **遞迴與循環**。對一棵有 $n = 1024$ 個葉節點的平衡二元樹，葉節點與根節點之間有幾個組合步驟？與一個由左到右讀取同樣 1024 個輸入的 RNN 比較，並把答案與梯度消失的上界連結起來。

提示

$\log_2 1024 = 10$ 步，鏈則最多 1023 步。梯度上界的衰減方式是（因子）$^{\text{path length}}$，所以從每個輸入到輸出的路徑越短，越有利於最佳化。注意力把這一點推到極致：路徑長度為一。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 10. Book
2. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997. DOI
3. K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014. arXiv
4. K. S. Tai, R. Socher and C. D. Manning, "Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks," ACL, 2015. arXiv
5. F. A. Gers, J. Schmidhuber and F. Cummins, "Learning to Forget: Continual Prediction with LSTM," *Neural Computation*, 2000. DOI
6. Y. Bengio, P. Simard and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, 1994. DOI
7. R. Pascanu, T. Mikolov and Y. Bengio, "On the difficulty of training Recurrent Neural Networks," ICML, 2013. arXiv
8. P. J. Werbos, "Backpropagation through time: what it does and how to do it," *Proceedings of the IEEE*, 1990. DOI
9. J. L. Elman, "Finding Structure in Time," *Cognitive Science*, 1990. DOI
10. M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, 1997. DOI
11. I. Sutskever, O. Vinyals and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS, 2014. arXiv
12. D. Bahdanau, K. Cho and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR, 2015. arXiv
13. O. Vinyals, A. Toshev, S. Bengio and D. Erhan, "Show and Tell: A Neural Image Caption Generator," CVPR, 2015. arXiv
14. K. Xu et al., "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention," ICML, 2015. arXiv
15. J. Donahue et al., "Long-term Recurrent Convolutional Networks for Visual Recognition and Description," CVPR, 2015. arXiv
16. X. Shi et al., "Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting," NeurIPS, 2015. arXiv
17. R. Socher et al., "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank," EMNLP, 2013. Paper
18. R. Socher, C. C.-Y. Lin, A. Y. Ng and C. D. Manning, "Parsing Natural Scenes and Natural Language with Recursive Neural Networks," ICML, 2011. Paper
19. K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink and J. Schmidhuber, "LSTM: A Search Space Odyssey," *IEEE Transactions on Neural Networks and Learning Systems*, 2017. arXiv
20. Z. C. Lipton, J. Berkowitz and C. Elkan, "A Critical Review of Recurrent Neural Networks for Sequence Learning," arXiv:1506.00019, 2015. arXiv
21. A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," COLM, 2024. arXiv
22. M. Beck et al., "xLSTM: Extended Long Short-Term Memory," NeurIPS, 2024. Paper
23. L. Feng et al., "Were RNNs All We Needed?," arXiv:2410.01201, 2024. arXiv
24. A. Orvieto et al., "Resurrecting Recurrent Neural Networks for Long Sequences," ICML, 2023.

arXiv
25. S. Bengio, O. Vinyals, N. Jaitly and N. Shazeer, “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks,” NeurIPS, 2015. arXiv
26. M. Tiezzi, M. Casoni, A. Betti, T. Guidi, M. Gori and S. Melacci, “On the Resurgence of Recurrent Models for Long Sequences: Survey and Research Opportunities in the Transformer Era,” arXiv:2402.08132, 2024. arXiv
27. M. Tiezzi, M. Casoni, A. Betti, T. Guidi, M. Gori and S. Melacci, “Back to recurrent processing at the crossroad of transformers and state-space models,” *Nature Machine Intelligence*, 2025. DOI
28. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. arXiv
29. T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” ICML, PMLR 235, 2024. Paper
30. J. Y.-C. Hu, X. Zhang, A. Elsheikh, W. Wu and H. Liu, “On Structured State-Space Duality,” ICML, PMLR 306, 2026. Paper

ML 04・注意力與視覺 Transformer

從 query、key、value 建立自注意力；加入多頭、位置編碼與 Transformer 區塊；用 ViT 把影像切成 patch token；再認識 DeiT、Swin、CNN 與 Transformer 混合模型、高效率注意力與狀態空間模型。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-04-vision-transformers/

你將學到

- 自注意力（self-attention）如何算出一個隨輸入而變的加權平均；query、key、value 各自扮演什麼角色；為什麼分數要除以 $\sqrt{d_k}$。
- 多頭注意力（multi-head attention）、因果遮罩（causal mask）與交叉注意力（cross-attention）怎麼建構，並用程式碼與 PyTorch 逐一比對。
- 為什麼注意力需要位置資訊，以及正弦、可學習、相對與旋轉（rotary）位置編碼如何提供它。
- Transformer 區塊如何組合注意力、MLP、LayerNorm 與殘差連接。
- Vision Transformer（ViT）如何把影像變成 patch token、為什麼它需要那麼多資料，以及 DeiT、Swin 與 CNN–Transformer 混合設計如何回應這個問題。
- 注意力為什麼昂貴；FlashAttention、視窗與 token 合併各自改變了什麼；Mamba 與 Mamba-2 和注意力的差異，以及影像掃描順序如何影響全域上下文與方向偏差。

先看全貌

CNN（ML 03）讓資訊透過固定的局部鄰域傳遞：一個像素要經過很多層，才能「聽到」遠處像素的消息。**注意力**（attention）則讓每個位置根據自己目前的內容，自行決定要讀取哪些其他位置，而且一步就到。要辨認一隻鳥，含有鳥頭的 patch 可以直接參考遠處含有翅膀的 patch。

> **用最簡單的話說**
>
> 想像一間教室，每個小朋友手上拿著一片拼圖。每個人都對自己那一片提出一個問題（「我這片有藍色的邊，誰有天空？」），其他人則各自舉起一張卡片，描述自己手上那一片。每個小朋友會最專心聽那些卡片最符合自己問題的同學，然後更新自己對這片拼圖的想法。接著大家用更新後的想法再來一輪。注意力就是這個「聽誰說話」的規則；Transformer 則是重複很多輪聆聽，中間再穿插一點自己思考的時間。

注意力的想法來自用循環網路做的機器翻譯：ML 03A 說明了它如何移除編碼器–解碼器 RNN 的單一向量瓶頸。Vaswani 等人的 Transformer [1] 走得更遠，完全拿掉循環結構，編碼器與解碼器都只由注意力和逐 token 的小型網路組成。Dosovitskiy 等人的 Vision Transformer（ViT）[2] 則幾乎原封不動地把 Transformer 編碼器套用到影像 patch 序列上。Bishop 與 Bishop [10] 提供了現代的教科書式推導，而現代觀點一節討論的綜述 [3]、[11]、[12] 則整理了眾多視覺變體。讀

完本篇之後，DLCV 02 會在實務上比較 CNN 與 Transformer 骨幹在分類上的表現，ML 05 則說明這些骨幹如何在沒有標註的情況下預訓練。

自注意力：query、key 與 value

白話版。每個 token 產生三個向量：**query**（「我在找什麼？」）、**key**（「我含有什麼？」）以及 **value**（「如果你聽我的，我要傳出什麼？」）。一個 token 的輸出，就是所有 value 的平均，權重取決於它的 query 與每個 key 的吻合程度。

精確版。把 N 個 token 疊成矩陣 $X \in \mathbb{R}^{N\times D}$ 的各列，其中 D 是模型寬度。三個可學習的矩陣將它們投影：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

其中 $W_Q, W_K \in \mathbb{R}^{D\times d_k}$、$W_V \in \mathbb{R}^{D\times d_v}$。**縮放點積注意力**（scaled dot-product attention）[1] 為

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}_{\mathrm{row}}\!\left(\frac{QK^\top}{\sqrt{d_k}}\right)V = AV.$$

$QK^\top$ 是一個 $N \times N$ 的**分數**矩陣：第 (i,j) 項是 query i 與 key j 的點積，衡量 token j 對 token i 有多相關。逐列的 softmax 把每一列轉成非負且總和為一的權重，得到**注意力矩陣** A。輸出 $Y = AV \in \mathbb{R}^{N\times d_v}$ 每個 token 一列：第 i 列是 $\sum_j A_{ij} v_j$，也就是 value 向量的加權平均。

為什麼要三個不同的投影？如果 query 和 key 是同一個向量，每個 token 都會和自己最吻合，而且相關性被迫是對稱的。分開的 W_Q 與 W_K 讓「我要找的東西」可以不同於「我能提供的東西」（眼睛的 patch 可以去找鼻子，而不必自己就是鼻子）；分開的 W_V 則讓「我能提供的東西」不同於「我實際傳出的內容」。

為什麼要除以 $\sqrt{d_k}$？若 query 與 key 的各分量互相獨立、平均為零、變異數為一，則點積 $q\cdot k = \sum_{l=1}^{d_k} q_l k_l$ 的平均為零、變異數為 d_k。不縮放的話，頭越寬分數就越大，softmax 會飽和（幾乎所有權重集中在一個 token 上），梯度也會變得極小。除以 $\sqrt{d_k}$ 讓變異數回到一。這是一種尺度慣例，並不是把每個 query 正規化成單位長度。

決定誰跟誰交談的是內容，而不是位置。卷積的權重綁定在相對位置上；注意力的權重則是在執行時由資料算出來的。由此衍生一個性質：自注意力是**排列等變**（permutation-equivariant）的。把輸入 token 打亂，輸出也會以相同方式被打亂，因為公式裡沒有任何東西涉及 token 的順序。下面的程式碼同時檢查形狀與這個對稱性。

```
import torch

torch.manual_seed(0)
X = torch.randn(5, 8)                    # N=5, D=8
Wq, Wk, Wv = [torch.randn(8, 4) for _ in range(3)]
def attention(x):
    q, k, v = x @ Wq, x @ Wk, x @ Wv
    a = (q @ k.T / q.shape[-1]**.5).softmax(dim=-1)
    return a @ v, a
```

```
y, a = attention(X)
perm = torch.tensor([3, 0, 4, 1, 2])
yp, _ = attention(X[perm])
assert torch.allclose(a.sum(-1), torch.ones(5), atol=1e-6)
assert torch.allclose(yp, y[perm], atol=1e-5)
print("output shape:", tuple(y.shape))
```

這個對稱性很適合處理集合，但影像不是一堆 patch 的集合：眼睛長在嘴巴下面的臉，就不是一張臉。下面介紹的位置資訊會打破這個對稱性。

多頭注意力

白話版。平行執行好幾個注意力運算，每個都有自己較小的投影，讓不同的頭可以追蹤不同的關係（鄰近紋理、同一個物體、對稱的部位）。最後把它們的輸出串接起來再混合。

精確版。有 h 個頭時，第 r 個頭有自己的 $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{D\times d_k}$，通常取 $d_k = d_v = D/h$：

$$\text{head}_r = \text{Attention}\big(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}\big), \qquad \text{MHA}(X) = [\,\text{head}_1, \dots, \text{head}_h\,]\,W_O,$$

其中方括號把 h 個寬度為 D/h 的輸出串接回寬度 D，$W_O \in \mathbb{R}^{D\times D}$ 再把它們混合。由於每個頭寬度只有 D/h，多頭注意力的成本與一個全寬度的頭差不多，所有投影合計 $4D^2$ 個權重（另加偏差）。各個頭不保證會學到人類看得懂的角色；有些頭是多餘的，可以被剪除。

下面的實作把投影切成多個頭、逐頭計算分數、串接後再投影，結果與 `nn.MultiheadAttention` 完全一致。它也示範了**因果遮罩**：在 softmax 之前把禁止的分數設為 $-\infty$，這些位置的權重就會恰好為零。

```
import torch
from torch import nn

torch.manual_seed(0)
B, N, D, h = 2, 6, 16, 4
mha = nn.MultiheadAttention(D, num_heads=h, batch_first=True, bias=False)
X = torch.randn(B, N, D)

def multi_head(x):
    Wq, Wk, Wv = mha.in_proj_weight.chunk(3)          # each D x D (rows = outputs)
    q, k, v = x @ Wq.T, x @ Wk.T, x @ Wv.T              # B x N x D
    split = lambda t: t.view(B, N, h, D // h).transpose(1, 2)   # B x h x N x d_k
    q, k, v = split(q), split(k), split(v)
    scores = q @ k.transpose(-2, -1) / (D // h) ** 0.5  # B x h x N x N
    A = scores.softmax(dim=-1)
    heads = A @ v                                       # B x h x N x d_k
    concat = heads.transpose(1, 2).reshape(B, N, D)     # put the heads side by side
    return concat @ mha.out_proj.weight.T, A

ours, A = multi_head(X)
ref, _ = mha(X, X, X)
print("matches nn.MultiheadAttention:", torch.allclose(ours, ref, atol=1e-5))
```

```
print("attention tensor:", tuple(A.shape), "| parameters:",
      sum(p.numel() for p in mha.parameters()), "= 4 * D^2 =", 4 * D * D)

# Causal mask: query i may only read keys j <= i
causal = torch.triu(torch.ones(N, N, dtype=torch.bool), diagonal=1)   # True = forbidden
out_c, w_c = mha(X, X, X, attn_mask=causal)
print("no weight on the future:", torch.all(w_c[:, causal] == 0).item())
```

輸出為 `True`、形狀 `(2, 4, 6, 6)` 的注意力張量（批次、頭、query、key）、`1024 = 4 * D^2 = 1024`，以及因果檢查的 `True`。正式的程式碼請呼叫 `torch.nn.functional.scaled_dot_product_attention`，它計算的是同一件事，並會在裝置、資料型別與形狀允許時，自動改用融合（fused）、省記憶體的核心 [7]。

連接 token 的三種方式

白話版。一個 query 可以讀取哪些 token，是模型設計的一部分，以分數矩陣上的遮罩來表達。

機制	query 可讀取	典型用途
雙向自注意力	同一序列中的所有 token	ViT 及其他影像編碼器
因果（遮罩）自注意力	只有自己與更早的位置	下一個 token 的生成、自迴歸影像模型
交叉注意力	來自另一個序列的 token	解碼器讀取編碼器；影像 token 讀取文字

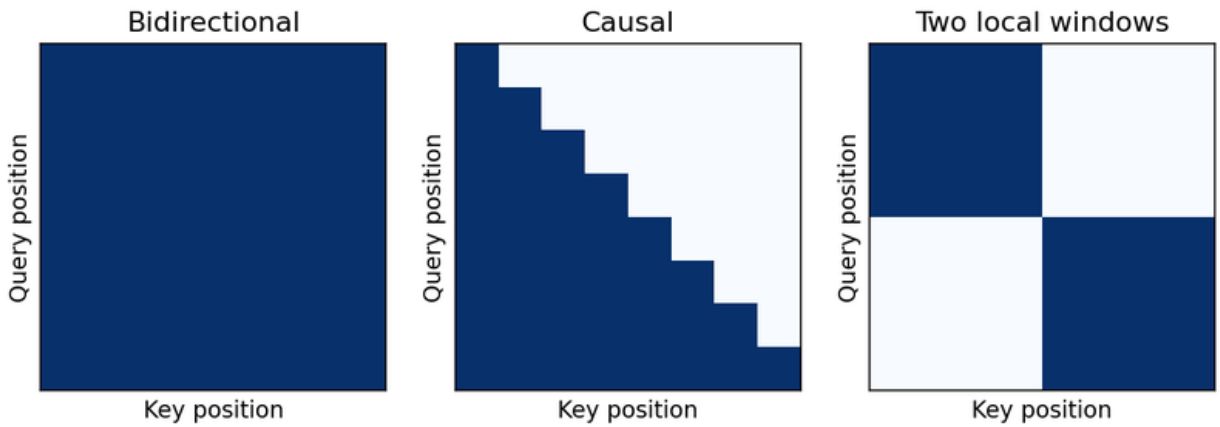


圖 *1* — 注意力遮罩：藍色表示允許，白色表示禁止。因果生成還會把輸入平移一個位置，讓目標無法被直接複製。

在**交叉注意力**中，query 來自一個序列 X，key 與 value 則來自另一個序列 Z：$Q = XW_Q$、$K = ZW_K$、$V = ZW_V$。分數矩陣的形狀是 $N_X \times N_Z$，輸出每個 query 一列。原始 Transformer 的解碼器就是這樣讀取編碼器；影像描述模型這樣讀取影像特徵；文字生成影像的擴散模型也這樣讀取文字提示（ML 07），而不需要在影像 patch 上做自迴歸。

注意力權重是計算的一部分，並不自動等於一種解釋。很大的權重可能乘上一個毫無資訊的 value，而殘差與 MLP 路徑也會影響輸出。

位置資訊

白話版。注意力不理會順序，所以我們必須告訴模型每個 token 在哪裡。最簡單的方法是在第一層之前，為每個 token 加上一個與位置有關的向量；更精緻的方法則把相對位置直接注入注意力分數。

正弦位置編碼。原始 Transformer [1] 在位置 pos 的 token 上加一個固定向量 $p_{pos} \in \mathbb{R}^D$：

$$p_{pos,2i} = \sin\!\left(\frac{pos}{10000^{2i/D}}\right), \qquad p_{pos,2i+1} = \cos\!\left(\frac{pos}{10000^{2i/D}}\right), \qquad i = 0, \dots, \tfrac{D}{2} - 1.$$

每一對維度 $(2i, 2i+1)$ 都是一個以角頻率 $\omega_i = 10000^{-2i/D}$ 繞圓旋轉的點，頻率從很快（每個位置一弧度）到非常慢都有。由三角函數的和角公式可得兩個性質。第一，對任一固定位移 k，都有 $p_{pos+k} = p_{pos} R_k$，其中 R_k 是一個與 pos 無關的區塊對角旋轉矩陣；因此一個線性層就能學會「往後看 k 個位置」。第二，$p_m \cdot p_n = \sum_i \cos\left(\omega_i (m-n)\right)$ 只與位移 $m-n$ 有關（圖 2 中）。

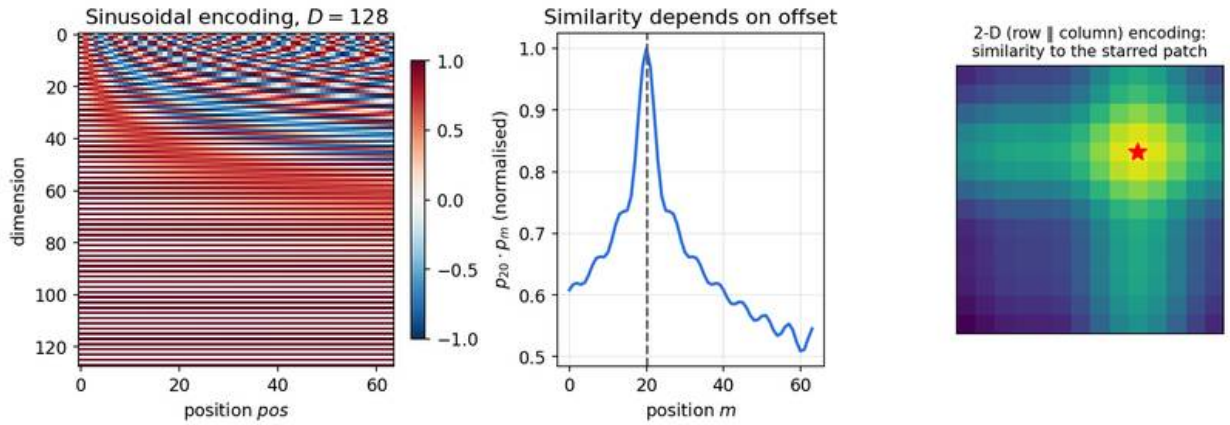


圖 2 — 正弦位置編碼。左：每一對維度以各自的頻率振盪。中：編碼之間的相似度只取決於位移。右：把列與行的編碼串接起來，就得到 14×14 *patch* 網格的二維編碼。

```
import numpy as np

def sinusoidal(n_pos, D):
    pos = np.arange(n_pos)[:, None]
    omega = 1.0 / 10000 ** (np.arange(0, D, 2) / D)    # one frequency per sin/cos pair
    pe = np.zeros((n_pos, D))
    pe[:, 0::2] = np.sin(pos * omega)
    pe[:, 1::2] = np.cos(pos * omega)
    return pe, omega

pe, omega = sinusoidal(100, 16)
k = 7                                                  # a fixed offset
```

```
# block-diagonal rotation that maps p_pos to p_{pos+k}, the same for every pos
R = np.zeros((16, 16))
for i, w in enumerate(omega):
    c, s = np.cos(w * k), np.sin(w * k)
    R[2 * i:2 * i + 2, 2 * i:2 * i + 2] = [[c, -s], [s, c]]
print("p[pos+k] == p[pos] @ R for all pos:", np.allclose(pe[:-k] @ R, pe[k:]))
print("dot products depend only on offset:",
      np.isclose(pe[10] @ pe[17], pe[50] @ pe[57]))
```

兩項檢查都印出 `True`。

其他做法。

- **可學習的絕對位置嵌入**：每個位置一個可訓練的向量，ViT [2] 就是如此。ViT 依光柵（raster）順序編號的一維可學習嵌入，效果與明確的二維嵌入差不多；訓練後，同一列或同一行的 patch 嵌入會變得相似，代表模型自己學會了網格結構。換到新的解析度時，先用二維內插調整嵌入網格的大小，再進行微調。
- **相對位置表示**（Shaw 等人 [17]）在 token i 與 j 的分數上，加一個取決於位移 $j - i$ 的可學習項。Swin [5] 則在每個視窗的分數上加一張可學習的**相對位置偏差**（relative position bias）表。
- **旋轉位置嵌入**（rotary position embedding, RoPE；Su 等人 [16]）把每個 query 與 key 依其位置旋轉一個成比例的角度，使用的正是上面程式碼中的那種旋轉。於是分數 $q_m^\top k_n$ 只透過 $m - n$ 依賴位置。RoPE 已是大型語言模型的標準配備，在視覺中也越來越常見，做法是對影像的兩個軸分別旋轉。
- **不加明確編碼**：網路內部的卷積（如 CvT [19]），或邊界的零填補，本身就可能洩漏足夠的位置資訊。

Transformer 區塊

白話版。一個區塊交替做兩件事：注意力讓 token 互相交換資訊；接著一個小型 MLP 分別處理每個 token。殘差連接與正規化讓整個堆疊保持穩定。

精確版。 ViT 與大多數現代模型使用的**前置正規化**（pre-norm）區塊為

$$U = X + \mathrm{MHA}(\mathrm{LN}(X)), \qquad Y = U + \mathrm{MLP}(\mathrm{LN}(U)), \qquad \mathrm{MLP}(u) = W_2\,\mathrm{GELU}(W_1 u + b_1) + b_2.$$

LN 是**層正規化**（layer normalization）[14]：它把每個 token 的 D 個特徵標準化為平均零、變異數一，再套用可學習的縮放與平移；與 BatchNorm 不同，它不依賴同一批次中的其他樣本。MLP 把每個 token 從 D 擴展到通常 $4D$ 個隱藏單元，套用 GELU 非線性 [26]（$\mathrm{GELU}(z) = z\,\Phi(z)$，$\Phi$ 為標準常態分布的累積分布函數），再投影回去。殘差相加代表每個子層只需學習一個修正量，與 ResNet 完全相同。

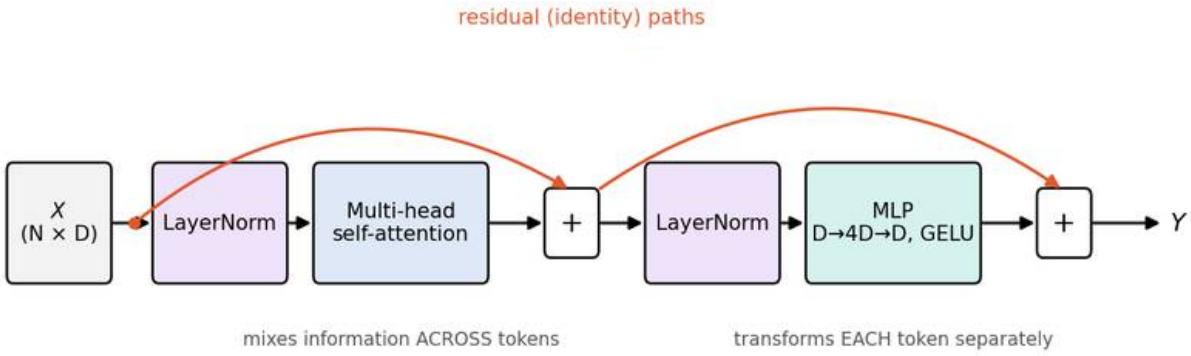


圖 3 — 前置正規化的 *Transformer* 區塊。注意力在 *token* 之間混合資訊；*MLP* 分別轉換每個 *token*；殘差路徑（橘色）把輸入繞過每個分支。

原始 Transformer 把 LayerNorm 放在每次殘差相加**之後**（後置正規化，post-norm）。Xiong 等人 [15] 證明，使用後置正規化時，初始化階段靠近輸出的梯度很大，這正是它需要學習率暖身（warm-up）的原因；前置正規化則讓梯度保持穩定，可以不用暖身就訓練。所以只說「一個 Transformer」並不完整，還得說明正規化放在哪裡。每個區塊的權重數：注意力 $4D^2$、MLP $8D^2$，合計約 $12D^2$，其中三分之二在 MLP。

Vision Transformer（ViT）

白話版。把影像切成方形 patch，把每個 patch 變成向量，加上位置資訊，然後送進標準的 Transformer 編碼器。答案從一個額外的特殊 token 讀出。

切成 patch。對一張 $H \times W$ 像素、C 個通道的影像，若 patch 大小 P 能整除 H 與 W，就有 $N = HW/P^2$ 個 patch。把每個 patch 攤平成 $x_i^{\text{patch}} \in \mathbb{R}^{P^2C}$，再做線性投影：

$$z_0 = [\, x_{\text{cls}};\ x_1^{\text{patch}}E;\ \ldots;\ x_N^{\text{patch}}E \,] + E_{\text{pos}},$$

其中 $E \in \mathbb{R}^{P^2C \times D}$ 是可學習的 **patch 嵌入**，$x_{\text{cls}} \in \mathbb{R}^D$ 是加在序列最前面的可學習**類別 token**（class token），$E_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ 存放可學習的位置嵌入。經過 L 個區塊後，對類別 token 的輸出套用一個 LayerNorm 與線性分類頭，就得到 logits。把所有 patch 的輸出取平均（mean pooling）也是同樣常見的替代做法。

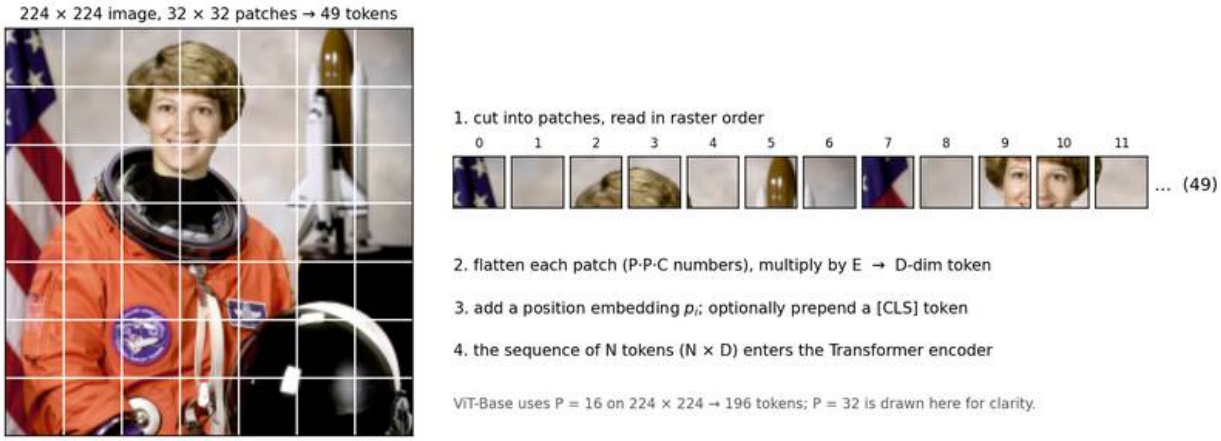


圖 *4* — 切成 *patch*。為了清楚起見這裡畫的是 *32* 像素的格子；*ViT-Base* 使用 *16 × 16* 的 *patch*，把 *224 × 224* 的影像變成 *196* 個 *token*。

對不重疊的 $P \times P$ patch 套用線性映射，恰好等於核大小與步幅都是 P 的卷積，實作上也正是這樣寫：

```
import torch
from torch import nn

torch.manual_seed(0)
B, C, H, W, P, D = 2, 3, 32, 32, 8, 48
x = torch.randn(B, C, H, W)

# 1) explicit: cut into P x P patches, flatten each, multiply by E
patches = x.unfold(2, P, P).unfold(3, P, P)                 # B, C, H/P, W/P, P, P
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, -1, C * P * P)   # B, N, P*P*C
E = torch.randn(C * P * P, D) * 0.02
tokens_explicit = patches @ E                                # B, N, D

# 2) the same linear map written as a convolution with kernel = stride = P
conv = nn.Conv2d(C, D, kernel_size=P, stride=P, bias=False)
conv.weight.data = E.T.reshape(D, C, P, P)
tokens_conv = conv(x).flatten(2).transpose(1, 2)             # B, N, D

print("tokens:", tuple(tokens_conv.shape), "| identical:",
      torch.allclose(tokens_explicit, tokens_conv, atol=1e-5))
```

輸出為 16 個寬度 48 的 token，以及 `True`。

模型大小。標準設定有 ViT-Base（12 個區塊、$D = 768$、12 個頭、約 8,600 萬個參數）、ViT-Large（24 個區塊、$D = 1024$、16 個頭）與 ViT-Huge（32 個區塊、$D = 1280$、16 個頭）[2]。「ViT-B/16」代表使用 16×16 patch 的 Base 模型。**patch 大小是最主要的運算量旋鈕**：P 減半，N 變成四倍，注意力分數的數量變成十六倍。小 patch 保留細節；大 patch 便宜，但比 patch 還細的資訊都會遺失，而且後面任何一層都救不回來。

資料的教訓。 Dosovitskiy 等人 [2] 發現，只在 ImageNet-1k（約 130 萬張影像）上訓練的 ViT，表現略遜於規模相當的 ResNet；但若先在 ImageNet-21k（1,400 萬張）或 JFT-300M（3 億張）上預訓練再微調，它就能追平甚至超越 ResNet，而且預訓練成本更低。他們的解釋是**歸納偏置**（inductive bias）：CNN 在每一層都假設局部性與平移等變性，而 ViT 只在 patch 嵌入內是局部的，空間關係必須從資料中**學出來**。Raghu 等人 [18] 比較了 ViT 與 ResNet 的內部表徵，發現 ViT 各層的表徵更加一致、較低層同時兼顧局部與全域的注意力，而且要在早期層學到局部注意力，需要大規模的預訓練。

程式碼：在手寫數字上訓練一個小型 ViT

這個完整範例在與ML 03 CNN 相同的 8×8 手寫數字資料上訓練一個小型 ViT，使用 2×2 的 patch（16 個 token）、一個類別 token、可學習的位置嵌入，以及四個前置正規化區塊。在 CPU 上約半分鐘即可跑完。

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)

class TinyViT(nn.Module):
    def __init__(self, img=8, patch=2, chans=1, D=64, depth=4, heads=4, classes=10):
        super().__init__()
        N = (img // patch) ** 2
        self.patch = nn.Conv2d(chans, D, kernel_size=patch, stride=patch)   # patchify + E
        self.cls = nn.Parameter(torch.zeros(1, 1, D))                       # [CLS] token
        self.pos = nn.Parameter(torch.randn(1, N + 1, D) * .02)             # learned
        ↪  positions
        layer = nn.TransformerEncoderLayer(D, heads, dim_feedforward=4 * D, dropout=0.1,
                                           activation="gelu", batch_first=True,
                                           norm_first=True)                 # pre-norm block
        self.blocks = nn.TransformerEncoder(layer, depth, enable_nested_tensor=False)
        self.norm, self.head = nn.LayerNorm(D), nn.Linear(D, classes)

    def forward(self, x):
        t = self.patch(x).flatten(2).transpose(1, 2)                         # B, N, D
        t = torch.cat([self.cls.expand(len(t), -1, -1), t], dim=1) + self.pos
        return self.head(self.norm(self.blocks(t))[:, 0])                   # read [CLS]

d = load_digits()
X = torch.tensor(d.images[:, None] / 16.0, dtype=torch.float32)
y = torch.tensor(d.target)
train, test = train_test_split(np.arange(len(y)), test_size=.2, stratify=d.target,
                               random_state=0)
train, val = train_test_split(train, test_size=.2, stratify=d.target[train], random_state=1)
```

```
model = TinyViT()
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.05)
best = (float("inf"), None)
for epoch in range(30):
    model.train()
    for idx in torch.tensor(train)[torch.randperm(len(train))].split(64):
        loss = nn.functional.cross_entropy(model(X[idx]), y[idx])
        opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        v = nn.functional.cross_entropy(model(X[val]), y[val]).item()
    if v < best[0]:
        best = (v, copy.deepcopy(model.state_dict()))
model.load_state_dict(best[1]); model.eval()
with torch.no_grad():
    acc = (model(X[test]).argmax(1) == y[test]).float().mean().item()
print("parameters:", sum(p.numel() for p in model.parameters()))
print("best validation loss:", round(best[0], 4), "| held-out accuracy:", round(acc, 4))
```

在我們的執行中，它約有 202,000 個參數，保留測試集準確率約 95%；而 ML 03 的 CNN 在相同的資料切分下，只用不到 10,000 個參數就達到約 96%。在小型資料集上，CNN 內建的局部性大約值二十倍的參數量。這是「資料的教訓」的縮影，而不是對大規模效能的定論。

訓練配方：DeiT 及其後續

白話版。早期 ViT 與 CNN 在 ImageNet 規模資料上的差距，很大一部分其實來自訓練方式，而不是架構。

DeiT（Touvron 等人 [4]）**只用 ImageNet-1k** 就把 ViT 訓練到具競爭力，靠的是強力的資料擴增（RandAugment、Mixup、CutMix、隨機抹除）、大量正則化（stochastic depth、label smoothing）以及較長的訓練排程。它還引入了**蒸餾 token**（distillation token）：一個類似類別 token 的額外可學習 token，其輸出被訓練去符合一個預訓練 CNN 教師模型的預測。CNN 的歸納偏置因此透過它的預測轉移過來。後來的 DeiT III [27] 證明，一個更簡單但仔細調校的監督式配方（輕量的「3-Augment」擴增、簡單的裁切，以及二元交叉熵損失函數）能讓純 ViT 更強。Steiner 等人 [25] 對資料、擴增與正則化做了大規模研究（「AugReg」），發現擴增、正則化與運算量的適當組合，可以追上用大約十倍資料訓練的模型。

一般性的教訓與ML 03 討論 ConvNeXt 時相同：只有在資料、擴增、解析度、訓練排程與運算量都對齊的情況下，比較架構才有意義。

Swin 與階層式 Transformer

白話版。對數千個 token 做全域注意力很昂貴，而且純 ViT 從頭到尾維持同一個粗解析度，不利於偵測與分割。Swin 把注意力限制在局部視窗內，在層與層之間平移視窗讓資訊仍能擴散，並合併 patch 來建立類似 CNN 的金字塔。

視窗。 Swin [5] 把 token 網格切成互不重疊、每個 $M \times M$ 個 token 的視窗（通常 $M = 7$），只在每個視窗內部計算自注意力。對 $h \times w$ 的 token 網格、C 個通道，論文給出的成本為

$$\Omega(\text{global MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{window MSA}) = 4hwC^2 + 2M^2hwC,$$

其中 $4hwC^2$ 是四個投影的成本，第二項則是計算分數與加權的乘法。全域成本對 token 數 hw 是二次方；固定 M 時，視窗成本是線性的。

平移視窗。若視窗固定不動，不同視窗的 token 永遠不會互動。Swin 讓一般視窗與在兩個方向各平移 $\lfloor M/2 \rfloor$ 個 token 的視窗交替出現。在第 ℓ 層位於視窗邊界附近的 token，到第 $\ell + 1$ 層會與相鄰視窗的 token 同在一個視窗，資訊因此得以跨越邊界（圖 5）。高效率的實作會把特徵圖做循環平移，並用遮罩避免從對側邊緣繞過來的 token 互相注意。

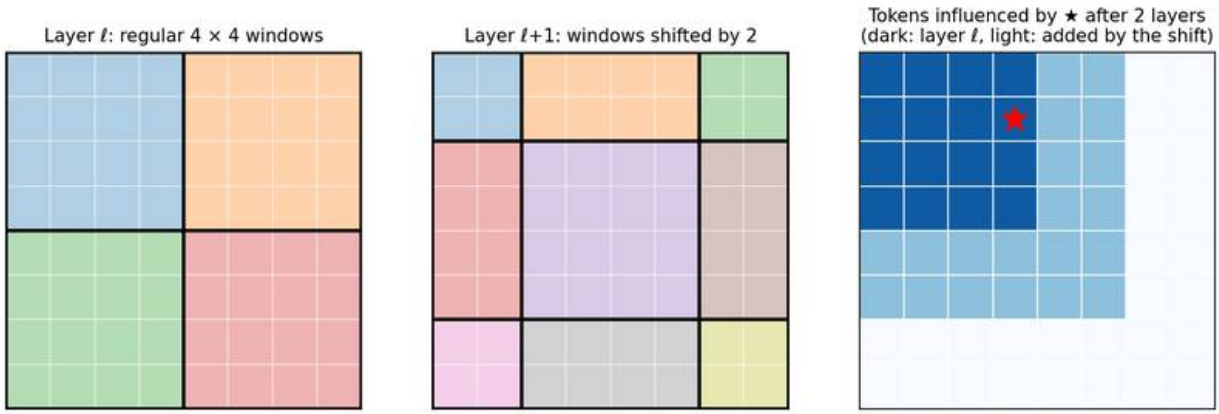


圖 *5 — Swin 在 8 × 8 token* 網格上的一般視窗與平移視窗。兩層之後，星號 *token* 的資訊會到達深色視窗（第 ℓ 層），再加上平移視窗帶來的淺色帶狀區域（第 ℓ*+1* 層）。

階層結構。在各階段之間，**patch 合併**（patch merging）把每 2×2 組相鄰 token 串接後做線性投影，使解析度減半、通道數加倍，就像步幅卷積一樣。四個階段分別產生輸入解析度 1/4、1/8、1/16、1/32 的特徵圖，與 ResNet 的金字塔相同，因此 Swin 可以在偵測與分割框架中直接取代 CNN 骨幹。每個視窗的分數也會加上可學習的**相對位置偏差**。Swin V2 [32] 用殘差後置正規化、縮放餘弦注意力（scaled cosine attention），以及連續、對數間隔的位置偏差來擴大規模，後者在不同視窗大小與解析度之間的遷移效果更好。

CNN–Transformer 混合模型

白話版。卷積擅長在資料少時處理局部細節；注意力擅長長距離互動。混合模型讓兩者各自在適合的地方發揮。

- **卷積起始層（stem）**。原始 ViT 論文就已測試過一種「混合」版本，其 patch 取自 ResNet 的特徵圖而不是原始像素 [2]；在較小的運算預算下有幫助。
- **在注意力裡放卷積**。 CvT [19] 在每個階段使用卷積式 token 嵌入，並用深度可分離卷積產生 query、key、value 的投影，藉此內建局部性，也因此可以省去明確的位置嵌入。

- **依階段堆疊**。 CoAtNet [20] 在前期放深度卷積階段（此時解析度高、局部模式為主），後期放相對注意力階段；它在各種資料規模下都報告了強勁的結果，因為卷積階段在資料有限時提供了歸納偏置。
- **行動裝置混合模型**。 MobileViT [28] 在 MobileNet 風格的網路中插入小型 Transformer 區塊，以行動裝置等級的成本加入全域脈絡。
- **反方向**。 ConvNeXt（見ML 03）把 Transformer 時代的設計選擇引進純 CNN，並追平了 Swin。

成本與高效率注意力

白話版。全域注意力要把每個 token 和其他每個 token 比較，所以影像邊長加倍，工作量就變成十六倍。高效率方法要嘛讓精確注意力跑得更快，要嘛限制哪些 token 互動，要嘛減少 token 數量。

對 N 個寬度為 D 的 token，每層分數矩陣約需 N^2D 次乘加，投影與 MLP 約需 $12ND^2$ 次。單純的實作會為每個頭儲存一個 $N \times N$ 矩陣。固定 patch 大小時，影像高與寬各加倍，N 變成四倍，成對分數變成十六倍。

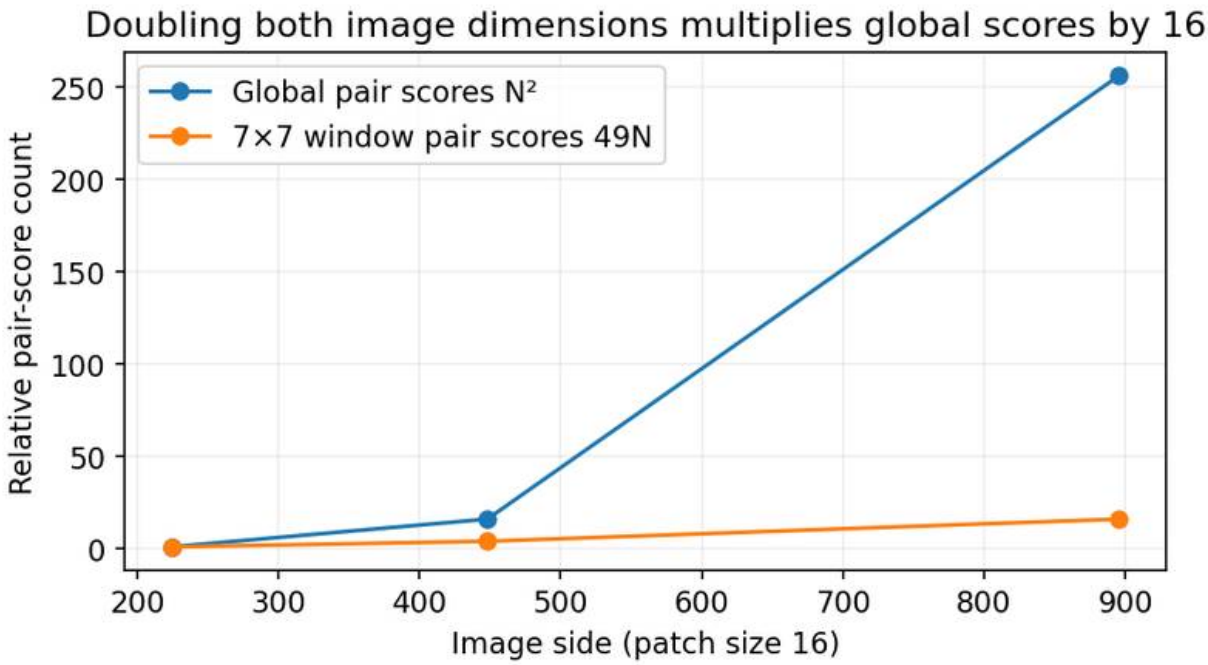


圖 *6* — 成對分數的成長趨勢，並非實測的執行時間。投影、*MLP*、記憶體存取與核心實作品質也會影響延遲。

主要的方法家族：

- **精確但更快**。 FlashAttention [6] 把計算切成區塊（tiling），讓 query、key、value 的區塊留在快速的晶片內記憶體中，並且從不把完整的 $N \times N$ 矩陣寫到較慢的 GPU 記憶體，藉此計算**精確**的注意力；FlashAttention-2 [29] 進一步改善了工作切分方式。算術量仍是二次方，但記

憶體用量變成線性，實際執行時間也大幅下降。PyTorch 的 `scaled_dot_product_attention` 在可行時會自動選用這類核心 [7]。

- **限制連接方式**。視窗（Swin）或其他稀疏模式減少了成對的數量，代價是需要好幾層才能達成全域溝通。
- **減少 token**。 Token Merging（ToMe）[21] 在預訓練好的 ViT 內，以快速的配對步驟逐步合併相似的 token，不需重新訓練就能提高吞吐量。較大的 patch、token 剪枝與池化階段則以更粗略的方式達到相同效果。
- **更小的模型**。 Papa 等人的綜述 [13] 把高效率 ViT 分為精簡架構、剪枝、知識蒸餾與量化四類，並提出「Efficient Error Rate」指標，用來在參數量、位元數、FLOPs 與模型大小之間比較模型。

狀態空間模型與混合模型

白話版。不必比較每一對 token，而是像循環網路（ML 03A）一樣，讓一個精簡的狀態沿著序列傳遞。成本隨 token 數線性成長。

對 token 序列，一個簡化的狀態空間遞迴為

$$h_t = \bar{A}_t\, h_{t-1} + \bar{B}_t\, x_t, \qquad y_t = C_t\, h_t,$$

其中 h_t 是固定大小的隱藏狀態，$\bar{A}_t$、$\bar{B}_t$、C_t 是（離散化後的）狀態空間矩陣。在 Mamba 這類**選擇性**（selective）模型中，寫入／讀取係數與步長取決於輸入 x_t；底層連續轉移可以保持固定，離散化後的係數卻會改變。輸入選擇與仿射狀態更新使平行掃描（parallel scan）成為可能 [33]。影像沒有天然的一維順序，因此視覺版本必須選擇**掃描順序**：Vision Mamba（Vim）[30] 加上位置嵌入，對 patch 序列做正向與反向掃描；VMamba [31] 則沿著二維網格的四個方向掃描。Visual Mamba 綜述 [8] 回顧了 200 多篇論文，並指出掃描方式是視覺應用中最關鍵的設計決策。

MambaOut [9]（CVPR 2025）是一個有用的反例。作者主張 Mamba 適合長序列與因果（自迴歸）結構的任務，並在保留閘控卷積區塊的同時拿掉狀態空間元件，建立了 MambaOut 模型。他們的模型在 ImageNet 分類上勝過視覺 Mamba 模型，顯示 Mamba 在這裡並非必要；而對於涉及長 token 序列的偵測與分割，他們認為狀態空間模型仍值得探索。更廣泛的重點在於方法論：要把某個結果歸功於某種機制（注意力、掃描、卷積）之前，必須先把它與閘控、正規化、訓練配方和參數預算分開來看。

影像掃描是一種歸納偏置

Mamba [33] 提供高效率的序列運算，但不會替我們決定影像的幾何結構。Vim [30] 使用 patch 序列、位置嵌入與正向／反向處理；VMamba [31] 則把四條路徑的選擇性掃描放進階層式視覺骨幹中。單靠模型名稱，無法判斷感受野的影響強度、解析度行為或執行時間。狀態維度、掃描路徑、局部運算與降取樣階段都會影響結果。

在固定掃描順序下，影像中相鄰的兩個像素，可能在序列中相隔許多步。反過來，一列最後的像素與下一列第一個像素，雖然在空間上相距甚遠，卻在序列中連續。局部卷積可以先提供鄰近證據，再由掃描把上下文傳得更遠。多條路徑提供額外的傳遞方式，但不會自動恢復所有幾何對稱性。位置嵌入也能編碼有用的位置，卻讓移轉到不同網格成為另一個需要處理的設計問題。

可以用數學明確呈現這個選擇：

$$Y^{(r)} = P_r^{-1}\mathcal{S}_r(P_r X), \qquad Y = \frac{1}{m}\sum_{r=1}^{m} Y^{(r)}.$$

其中 X 包含展平後的影像 token，r 索引 m 條路徑中的一條，P_r 把 token 排列成該路徑的順序，$\mathcal{S}_r$ 執行該路徑的序列循環，P_r^{-1} 還原影像座標，Y 對產生的特徵取平均。這是概念性的路徑融合模型。可學習的視覺區塊可以使用不同投影或融合規則；這裡選擇平均，是為了讓實驗容易解讀。

CPU 實驗：量測方向偏差

下一個範例使用單一純量狀態，每條路徑的衰減係數都相同。先比較正向／反向的逐列掃描，以及加入逐欄掃描的四條路徑；接著把輸入旋轉 90 度、處理後再把輸出轉回來。全部使用合成的 6×6 脈衝，不下載權重或影像。這個實驗單獨檢查排序，並不重現訓練好的 Vim 或 VMamba。

```
import numpy as np

def scan_image(image, four=False):
    ids = np.arange(image.size).reshape(image.shape)
    row, col = ids.ravel(), ids.T.ravel()
    orders = [row, row[::-1]]
    if four:
        orders += [col, col[::-1]]
    outputs = []
    for order in orders:
        h = 0.0
        out = np.zeros(image.size)
        for i in order:
            h = 0.8 * h + 0.2 * image.ravel()[i]
            out[i] = h
        outputs.append(out.reshape(image.shape))
    return np.mean(outputs, axis=0)

image = np.zeros((6, 6))
image[1, 4] = 1.0
errors = []
for four in [False, True]:
    original = scan_image(image, four=four)
    rotated_back = np.rot90(scan_image(np.rot90(image), four=four), -1)
    error = np.max(np.abs(original - rotated_back))
    errors.append(error)
    print('four routes:', four, 'rotation discrepancy:', round(error, 6))
assert 0.0 < errors[1] < errors[0]
```

兩條路徑的最大差異為 `0.063622`，四條路徑則為 `0.026738`。在這個範例中，增加路徑減少方向偏差，但剩餘差異並非零。旋轉逐列路徑時，其中一個遍歷軸會反向；變換後的路徑未必屬於原本四條路徑的集合。因此，即使係數共用且以平均融合，也不保證旋轉等變性。可學習的方向投

影、位置資訊與局部運算又引入更多選擇。應直接測試需要的性質，而不是從「二維掃描」推定它存在。

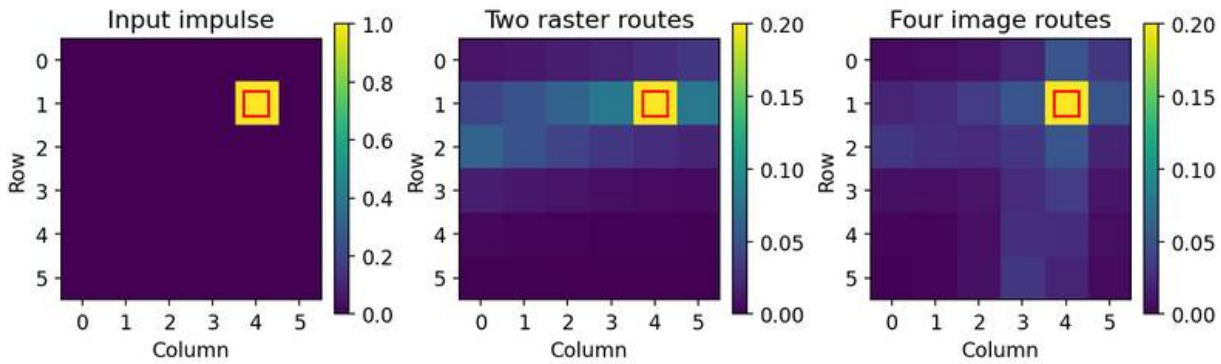


圖 7 — 紅色外框標示輸入脈衝。兩條逐列路徑不均勻地傳遞上下文；加入逐欄路徑後，影響圖會改變。這些合成響應使用範例中的相同循環，並非學得的注意力圖或訓練好的 *VMamba* 活化值。

資訊能沿著完整掃描傳遞，因此循環可以具有全域感受野。這表示依賴關係可能存在，並不代表它在所有距離都保持強大影響。純量衰減為 0.8 時，攜帶的貢獻每經過一步就再乘上 0.8。ML 03A 推導的選擇性轉移可以改變這種保留方式，但訓練仍須學會哪些資訊值得保存。即使可以讀取遠處上下文，狀態壓縮或降取樣仍可能遺失細緻邊界與小物件。

Mamba-2 改變了什麼，又該如何評估

Mamba-2 [34] 以結構化狀態空間對偶性與高效率的分塊計算，重新安排選擇性序列混合器。它改變轉移結構與區塊參數投影，並沒有讓任意影像 patch 序列都等同於 Transformer。具有結構的因果係數矩陣可以用類似注意力的方式解讀，但標準 softmax 注意力仍保有不同的表達與計算性質。應用到影像時，仍須指定方向與融合方式。因此，對偶性回答的是計算問題，並未消除幾何問題。

公平的視覺比較應區分三種實驗：在共用區塊中只替換混合器；在對齊的訓練預算下比較完整骨幹；以及在目標硬體上量測部署模型。固定資料、資料增強、token 解析度與下游預測頭。除了參數量，也記錄狀態大小，並在不同影像解析度與批次大小下量測延遲及記憶體峰值。token 數的線性項可以同時伴隨昂貴的投影、路徑排列與小型運算核心的額外成本。上面的 Python 迴圈無法支持 GPU 吞吐量的宣稱。

對密集任務，檢查邊界附近、小型結構與不同長寬比下的錯誤；對影片，區分離線雙向編碼器與因果即時處理。除了分布內分類，也應納入旋轉／裁切敏感度與更長網格的評估。MambaOut [9] 提供重要的移除混合器消融實驗。結果支持依任務比較，並不是證明所有影像問題都不需要掃描的定理。Xu 等人 [8] 依骨幹、掃描機制，以及影像／影片／點雲／多模態應用整理視覺 Mamba。在已核實的紀錄中，這篇綜述仍是 2024 年預印本。其分類架構有助於選擇控制變因的實驗；提出的變體數量並不能確立普遍的贏家。

現代觀點

綜述文獻。 Khan 等人 [3]（*ACM Computing Surveys*，2022）從基礎（自注意力、大規模預訓練、雙向編碼）談起，接著綜覽 Transformer 在辨識、生成模型、視覺問答等多模態任務、影片、低階視覺與三維分析上的應用，最後列出開放問題。Han 等人 [11]（*IEEE TPAMI*，2023）依任務整理視覺 Transformer（骨幹、偵測與分割等中高階視覺、低階視覺、影片），也涵蓋高效率 Transformer 方法。Liu 等人 [12]（*IEEE TNNLS*，2024）回顧了一百多個用於影像、點雲與視覺–語言資料之分類、偵測與分割的視覺 Transformer，並提出三個未來方向。在效率方面，Papa 等人 [13]（*IEEE TPAMI*，2024）最有系統。狀態空間視覺模型請參考 Xu 等人 [8]。教科書式的推導則見 Bishop 與 Bishop [10]。

2023–2026 年的最新發展。純 ViT 已成為預設的**基礎模型**（foundation model）骨幹。ViT-22B [24]（ICML 2023）把 ViT 擴大到 220 億個參數，並需要一些架構修改（例如對 query 與 key 做正規化）才能在這個規模下穩定訓練。DINOv2 [23]（TMLR 2024）以自監督方式在大型精選影像集合上訓練 ViT，得到的通用特徵在凍結狀態下就能用於分類、分割與深度估計（ML 05）。Darcet 等人 [22]（ICLR 2024）發現大型 ViT 會把資訊量低的背景 patch 挪作內部暫存空間，在注意力圖

中產生高範數的假影 token；加入少數額外的**暫存器 token**（register tokens）就能消除這些假影，並改善密集預測。視覺–語言基礎模型（ML 09）與擴散 Transformer（ML 07）都建立在同樣的區塊上。與此同時，階層式與混合設計在高解析度的密集任務上仍很常見；在預算對齊時，現代化的 CNN 依然具有競爭力（ML 03）；而狀態空間與線性注意力模型仍持續被探索，用於影片這類非常長的 token 序列。

開放問題。高解析度與長影片下的二次方成本；純 ViT 對資料與運算量的龐大需求；能在不同解析度與長寬比之間泛化的位置編碼；如何解讀注意力而不過度詮釋；以及在注意力、卷積與狀態空間機制之間，做出運算量對齊的公平比較。

重點整理

- 自注意力是一個可學習、隨輸入而變的加權平均：$\mathrm{softmax}(QK^\top/\sqrt{d_k})V$。除以 $\sqrt{d_k}$ 讓分數維持單位變異數，softmax 才不會飽和。
- 多頭注意力以約等於一個頭的成本，平行執行 h 個較窄的頭；遮罩（因果、視窗）與交叉注意力決定誰讀取誰。
- 注意力是排列等變的，因此必須注入位置資訊：正弦、可學習絕對、相對偏差或旋轉編碼。
- 前置正規化的 Transformer 區塊先做 $X + \mathrm{MHA}(\mathrm{LN}(X))$，再做 $U + \mathrm{MLP}(\mathrm{LN}(U))$；注意力混合 token，MLP 轉換每個 token。
- ViT = 切成 patch（步幅為 P 的卷積）+ 位置嵌入 + 類別 token + Transformer 編碼器。它的歸納偏置弱，需要大規模預訓練或強力的訓練配方（DeiT、AugReg）。
- Swin 透過平移視窗與 patch 合併找回局部性與階層結構；CvT、CoAtNet、MobileViT 等混合模型結合了卷積與注意力。
- FlashAttention 讓精確注意力更快，但不改變其二次方的算術量；視窗、token 合併與狀態空間掃描則改變了實際計算的內容。比較機制時一定要對齊預算。

- Vision Mamba 與 VMamba 把選擇性狀態空間應用到影像序列。多條路徑會改變影響傳遞方式，但不自動保證旋轉等變性，也不保證保留所有遠處細節。

練習

1. **Token 預算**。對一張 512×512 的影像，分別計算 patch 大小為 16 與 8 時的 token 數，以及每個頭的注意力分數個數。patch 大小減半時，分數個數增加幾倍？

   提示

   $N = (512/16)^2 = 1024$ 與 $(512/8)^2 = 4096$ 個 token；$N^2 = 1{,}048{,}576$ 與 $16{,}777{,}216$ 個分數。patch 大小減半，N 乘以 4，分數乘以 16。

2. **縮放檢查**。對 $d_k = 4, 64, 1024$，隨機產生各分量為獨立標準常態的 query 與 key。分別量測有無除以 $\sqrt{d_k}$ 時 $q \cdot k$ 的變異數，以及在 100 個 key 上 softmax 的最大權重。不縮放時，softmax 會發生什麼事？

   提示

   原始點積的變異數接近 d_k；縮放後接近 1。不縮放時，在 $d_k = 1024$ 下 softmax 幾乎把所有權重都放在單一個 key 上（近乎 one-hot），流經其他 key 的梯度因此消失。

3. **移除位置資訊**。在 TinyViT 程式碼中，把位置嵌入設為零、拿掉類別 token 改用平均池化，然後在送進區塊之前打亂 patch token 的順序，並在評估模式下比較 logits。解釋結果，並說明這對一個沒有位置資訊的影像分類器意味著什麼。

   提示

   logits 在浮點誤差範圍內相同，因為每個區塊都是排列等變的，而平均池化是排列不變的。記得關掉 dropout（`eval()`），並打亂 token 而不是 patch 內的像素。這樣的模型無法分辨一張臉和用相同 patch 拼湊出來的亂序臉。

4. **交叉注意力的形狀**。在一個文字生成影像的模型中，256 個寬度 320 的影像 token 讀取 77 個寬度 768 的文字 token，使用 8 個寬度 40 的頭。寫出 W_Q、W_K、W_V、單一個頭的分數矩陣以及輸出的形狀。

   提示

   $W_Q \in \mathbb{R}^{320 \times 320}$（8 個頭 × 40），$W_K, W_V \in \mathbb{R}^{768 \times 320}$。每個頭的分數為 256×77。輸出跟隨 query：經過輸出投影後為 256×320。key 與 value 的長度必須彼此相同，而不必與 query 長度相同。

5. **Swin 的算術**。利用成本公式，比較 56×56 的 token 網格、$C = 96$、$M = 7$ 時的全域注意力與視窗注意力。兩種情況各由哪一項主導？

提示

$hw = 3136$。全域：$4 \cdot 3136 \cdot 96^2 \approx 1.16 \times 10^8$ 加上 $2 \cdot 3136^2 \cdot 96 \approx 1.89 \times 10^9$，由二次方項主導。視窗：同樣的 1.16×10^8 加上 $2 \cdot 49 \cdot 3136 \cdot 96 \approx 2.95 \times 10^7$，由投影主導。此例中視窗版本約便宜 14 倍。

6. **公平比較**。你有 2,000 張標註影像，以及預訓練好的 CNN、ViT 與視覺 Mamba 骨幹。設計一個實驗來決定要部署哪一個骨幹，並列出必須固定的條件。納入移除混合器的消融實驗，以及解析度／旋轉測試。

提示

先在凍結的特徵上比較線性探測（linear probe），再在驗證集上以對齊的預算、為各模型分別調校學習率來做微調。記錄預訓練資料、輸入解析度、參數量、FLOPs，以及在目標裝置上實測的延遲。拿預訓練的 ViT 去比從頭訓練的 CNN，測到的是預訓練，而不是架構。對掃描骨幹，記錄狀態大小，並測試保留閘控與局部卷積、移除混合器後，優勢是否仍存在。解析度與旋轉測試要使用一致的前處理。

參考文獻

1. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. arXiv
2. A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” ICLR, 2021. arXiv
3. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan and M. Shah, “Transformers in Vision: A Survey,” *ACM Computing Surveys*, 2022. arXiv
4. H. Touvron et al., “Training data-efficient image transformers & distillation through attention,” ICML, 2021. arXiv
5. Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” ICCV, 2021. arXiv
6. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS, 2022. arXiv
7. PyTorch, `scaled_dot_product_attention` and `TransformerEncoderLayer` documentation. Attention, Encoder layer
8. R. Xu et al., “Visual Mamba: A Survey and New Outlooks,” arXiv:2404.18861, 2024. arXiv
9. W. Yu and X. Wang, “MambaOut: Do We Really Need Mamba for Vision?,” CVPR, 2025. Paper
10. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024, ch. 12. Book
11. K. Han et al., “A Survey on Vision Transformer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. DOI
12. Y. Liu et al., “A Survey of Visual Transformers,” *IEEE Transactions on Neural Networks and Learning Systems*, 2024. DOI
13. L. Papa, P. Russo, I. Amerini and L. Zhou, “A Survey on Efficient Vision Transformers: Algorithms, Techniques, and Performance Benchmarking,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. DOI
14. J. L. Ba, J. R. Kiros and G. E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. arXiv

15. R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” ICML, 2020. arXiv
16. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, 2024. DOI
17. P. Shaw, J. Uszkoreit and A. Vaswani, “Self-Attention with Relative Position Representations,” NAACL, 2018. arXiv
18. M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang and A. Dosovitskiy, “Do Vision Transformers See Like Convolutional Neural Networks?,” NeurIPS, 2021. arXiv
19. H. Wu et al., “CvT: Introducing Convolutions to Vision Transformers,” ICCV, 2021. arXiv
20. Z. Dai, H. Liu, Q. V. Le and M. Tan, “CoAtNet: Marrying Convolution and Attention for All Data Sizes,” NeurIPS, 2021. arXiv
21. D. Bolya et al., “Token Merging: Your ViT But Faster,” ICLR, 2023. arXiv
22. T. Darcet, M. Oquab, J. Mairal and P. Bojanowski, “Vision Transformers Need Registers,” ICLR, 2024. arXiv
23. M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. arXiv
24. M. Dehghani et al., “Scaling Vision Transformers to 22 Billion Parameters,” ICML, 2023. arXiv
25. A. Steiner et al., “How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers,” *Transactions on Machine Learning Research*, 2022. arXiv
26. D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” arXiv:1606.08415, 2016. arXiv
27. H. Touvron, M. Cord and H. Jégou, “DeiT III: Revenge of the ViT,” ECCV, 2022. arXiv
28. S. Mehta and M. Rastegari, “MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer,” ICLR, 2022. arXiv
29. T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” ICLR, 2024. arXiv
30. L. Zhu et al., “Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model,” ICML, 2024. arXiv
31. Y. Liu et al., “VMamba: Visual State Space Model,” NeurIPS, 2024. arXiv
32. Z. Liu et al., “Swin Transformer V2: Scaling Up Capacity and Resolution,” CVPR, 2022. arXiv
33. A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” COLM, 2024. Author version
34. T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” ICML, PMLR 235, 2024. Paper

ML 05・表示學習與自監督

什麼是表示、為什麼分散式特徵能遷移，以及 SimCLR、MoCo、BYOL、SimSiam、DINO、MAE、JEPA 與 CLIP 如何在沒有標記的情況下學到特徵，並用線性探測與微調誠實評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-05-representation-learning/

你將學到

- 什麼是**表示**（representation）、為什麼**分散式**（distributed）表示比 one-hot 編碼更能泛化，以及好的表示應該（和不應該）具備哪些不變性。
- **遷移學習**（transfer learning）如何運作：預訓練一次編碼器，再透過 k-NN、線性探測（linear probe）或微調（fine-tuning）重複使用。
- 主要的**自監督學習**（self-supervised learning）家族如何從無標記資料中造出訓練訊號：對比式（SimCLR、MoCo，以及 InfoNCE 損失函數）、沒有負例的教師–學生法（BYOL、SimSiam、DINO）、冗餘抑制（Barlow Twins、VICReg）、遮罩建模（BEiT、MAE）與特徵預測（I-JEPA、V-JEPA 2）。
- 為什麼「讓兩個視圖一致」可能會**塌縮**（collapse），以及對齊性與均勻性如何把這件事說清楚。
- **CLIP** 如何把圖說當作監督訊號，並做到零樣本（zero-shot）分類。
- 如何誠實地評估一個表示，包括為什麼微調可能輸給線性探測，以及 LP-FT 如何改善。

先看全貌

假設你有一百萬張沒有標記的影像，但真正關心的任務只有 500 個標記樣本。只用這 500 個樣本

從頭訓練一個大型網路，等於把幾乎所有可用的資料都丟掉了。**表示學習**把工作拆成兩步。第一步，從一個不需要你的標記的廣泛訊號學出編碼器 $h = f_\theta(x)$。第二步，用手上少量的標記，在 h 之上訓練一個小型的任務頭（head）。

> **用最簡單的話說**
>
> 小孩在學會每種鳥的名字之前，就已經會注意翅膀、鳥嘴、顏色和動作。後來爸媽指著三隻知更鳥和兩隻烏鴉，這樣就夠了：從此小孩就能分辨知更鳥和烏鴉。「注意」這件事是免費學來的，只是靠著看這個世界。表示就是這種可以重複使用的觀察方式，而自監督學習就是「只是看」的那個部分。

為什麼重要：幾乎所有現代視覺系統都從一個預訓練的編碼器出發。影像分類器、物件偵測器、分割模型、影像–文字模型，以及生成模型裡的影像編碼器，都建立在用這種方式學到的表示之上。起點的好壞，往往比任務頭的細節更重要。

本章先談什麼是表示、為什麼遷移有效，接著逐一介紹實務上主流的自監督目標，最後討論如何衡量一個表示到底好不好。本章假設你已經熟悉神經網路（ML 02）、CNN（ML 03）與 Vision Transformer（ML 04）。線性因子模型與自編碼器是學習表示的古典路線，另有專章：ML 05A。

什麼是表示？

白話版。表示就是模型用來描述一個輸入的那串數字。好的表示讓你關心的東西容易讀出來，並把你不關心的東西藏起來。

精確版。編碼器是一個函數 $f_\theta : \mathcal{X} \to \mathbb{R}^D$，把輸入 x（這裡是影像）對應到特徵向量 $h = f_\theta(x)$，其中 θ 是可學習的參數，D 是特徵維度。任務頭 g 把 h 對應到預測，所以完整模型是 $g \circ f_\theta$。如果一個簡單的任務頭（例如線性分類器）只用少量標記樣本就能從 h 解決任務，我們就說 h 是這個任務的好表示。Bengio、Courville 與 Vincent 的綜述 [1] 是這個觀點的經典陳述；它列出讓表示有用的「先驗」（priors），例如平滑性、多個解釋因子、由淺到深逐漸抽象的特徵階層，以及跨任務共享的因子。

局部表示與分散式表示

類別最簡單的表示是 **one-hot**：每個概念一個單元，且恰好只有一個單元是開啟的。它容易讀，卻無法泛化。知道「紅色汽車」完全不會告訴你任何關於「紅色卡車」的事，因為它們沒有共同的開啟單元。**分散式**表示則用許多特徵來描述每個輸入，而每個特徵又被許多輸入共享：「紅」、「藍」、「汽車」、「卡車」。於是「紅色卡車」就能重複使用從紅色汽車學來的「紅」這個單元。

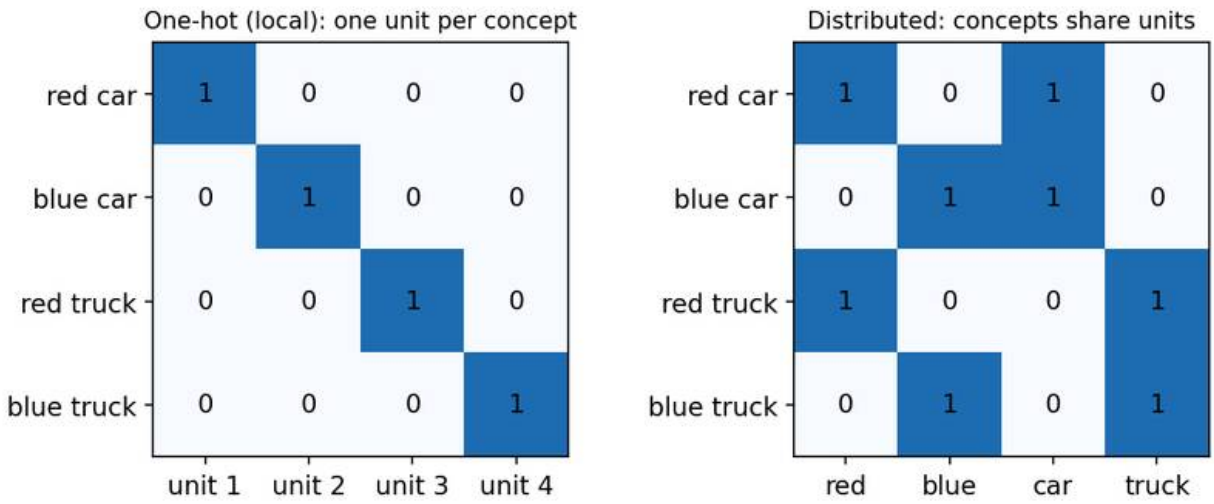


圖 1 — 四個概念的局部（*one-hot*）編碼與分散式編碼。在分散式編碼下，像紅色卡車這樣的新組合可以重複使用從其他樣本學到的特徵。

計數論證很驚人。用 D 個二元特徵，one-hot 編碼只能分辨 D 個概念，而分散式編碼原則上能分辨 2^D 種組態 [1]。深度網路會自動學出分散式編碼：每個隱藏單元對許多輸入有反應，每個輸入

也會啟動許多單元。表示學習大部分的實際價值都來自這種共享。

不變性與等變性

有用的表示會保留與任務相關的差異，並壓抑會造成干擾的差異。

- 若 $f(T(x)) = f(x)$，特徵就對轉換 T 具有**不變性**（invariance）：把一隻貓平移兩個像素，不應該改變「這是一隻貓」。
- 若 $f(T(x)) = T'(f(x))$，特徵就具有**等變性**（equivariance），也就是以可預測的方式跟著改變：平移那隻貓，牠的分割遮罩也應該平移同樣的距離。

「有用」永遠取決於下游任務。顏色不變性有助於物體辨識，卻會傷害水果成熟度的估計。旋轉不變性對紋理有幫助，卻會搞混 6 和 9。密集預測（分割、深度）需要空間上等變的區塊（patch）特徵，而不是每張影像一個不變的向量。後面會看到，自監督目標主要是透過所使用的**資料增強**（data augmentation）來選擇不變性，所以這個選擇是建模決策，而不是細節。

學習距離：度量學習

PCA 在ML 01 介紹過，並在ML 05A 深入討論，它保留變異最大的方向。但變異大的方向可能來

自光照，而不是物體身分。監督學習依照標記來塑造特徵；**度量學習**（metric learning）則依照應該配對的樣本來塑造距離。給定錨點（anchor）a、正例 p（同一身分）與負例 n（不同身分），三元組損失（triplet loss）是

$$\mathcal{L}_{\text{triplet}} = \max\left(0,\ \|h_a - h_p\|^2 - \|h_a - h_n\|^2 + m\right),$$

其中 h_a, h_p, h_n 是三個特徵向量，$m > 0$ 是邊界（margin）。當正例比負例近至少 m 時，損失為零。太容易的負例損失為零，什麼也學不到，所以挑選有資訊量的（「困難的」）負例很重要。下面的對比式自監督學習，就是一種用資料增強而不是標記來產生正例的度量學習。

遷移學習：預訓練一次，重複使用很多次

白話版。在一個大任務上訓練網路，保留它的特徵擷取器，再拿去用在小任務上。你可以凍結特徵、只訓練新的任務頭，也可以讓整個網路繼續學習。

精確版。預訓練在來源資料集或目標上找出編碼器參數 $\theta^\star$。遷移到一個有標記集合 $\{(x_i, y_i)\}_{i=1}^{n}$ 的目標任務時，有三種常見做法：

1. **k 最近鄰（k-NN）：**不訓練。用測試特徵 $f_{\theta^\star}(x)$ 最相似的 k 個訓練特徵的標記來分類。
2. **線性探測：**凍結 $\theta^\star$，在 $h = f_{\theta^\star}(x)$ 上擬合線性分類器 $\hat{y} = \text{softmax}(Wh + b)$，其中 W 與 b 是唯一被訓練的參數。
3. **微調：**從 $\theta^\star$ 出發，連同任務頭一起更新編碼器，通常使用較小的學習率。

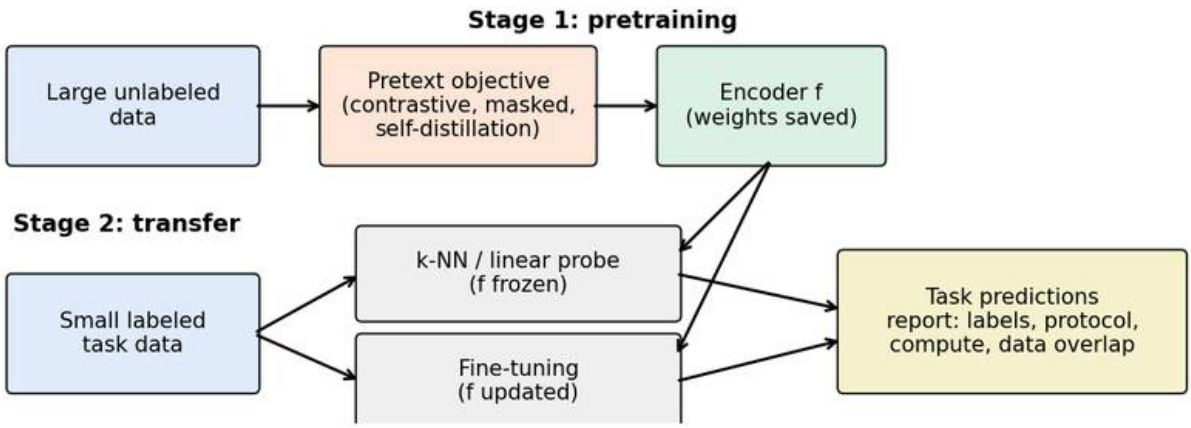


圖 2 — 表示學習的兩階段模式。預訓練昂貴但只做一次；遷移便宜且每個任務各做一次。

遷移為什麼會有效？Yosinski 等人 [2] 在 ImageNet 訓練的 CNN 上逐層量測。第一層特徵（很像 Gabor 濾波器的邊緣與顏色偵測器）是通用的：在任務之間遷移得很好。越高層的特徵越專屬於來源任務，而且來源任務與目標任務差異越大，可遷移性就越低。他們也發現，從遷移來的特徵初始化再微調，即使在目標任務上，也比從隨機初始化訓練有更好的泛化。

多年來，預訓練的預設來源是有監督的 ImageNet 分類。可是標記很昂貴，而且像「狗」這樣的標記，完全沒有告訴網路那隻狗的姿勢、草地或光線。自監督學習讓資料自己監督自己，移除了標記的瓶頸。

自監督學習：讓資料自己產生標記

白話版。發明一個答案可以從資料本身算出來的任務：「這兩個裁切來自同一張照片嗎？」、「這個遮罩下面原本是什麼？」、「哪一句圖說配這張影像？」。解這種**代理任務**（pretext task），會迫使網路學到有用的特徵。

精確版。自監督目標是只從無標記輸入（或像影像–圖說這種天然配對的資料）計算出來的損失 $\mathcal{L}(\theta; x)$。Liu 等人 [3] 與 Gui 等人 [4] 的綜述，以及 Balestriero 等人實用的「食譜」[5]，把這個領域整理成下表的幾個家族：

家族	訓練訊號	代表方法	主要風險
對比式	拉近正配對，推開其他樣本	SimCLR、MoCo	需要大量負例；假負例
自蒸餾（無負例）	學生在另一個視圖上模仿教師	BYOL、SimSiam、DINO	缺少不對稱或中心化時會塌縮
冗餘抑制	一致性加上去相關、非常數的特徵維度	Barlow Twins、VICReg	正則項帶來額外超參數
遮罩建模	預測被遮住的像素或符元	BEiT、MAE	特徵可能偏重低階細節
特徵預測	預測被遮住區域的特徵	I-JEPA、V-JEPA 2	需要像蒸餾一樣的防塌縮設計
語言監督	讓影像與其圖說配對	CLIP、SigLIP	繼承網路資料的偏誤與雜訊

這些家族是學習目標，而不是架構：每一種都可以用來訓練 CNN 或 Transformer。現代系統常

常結合好幾種。以下逐一介紹。

對比式學習與 InfoNCE 損失

白話版。讓同一張影像的兩個增強版本彼此一致，並且和批次中其他所有影像不一致。這是一道選擇題：「在這 N 個候選中，哪一個是我的搭檔？」

InfoNCE

van den Oord、Li 與 Vinyals 在對比預測編碼（Contrastive Predictive Coding, CPC）[6] 中提出 **InfoNCE** 損失。取一個查詢 q、一個「屬於」它的正例鍵 k^+，以及 $N-1$ 個負例鍵 $k_1^-, \dots, k_{N-1}^-$。給定相似度分數 $s(\cdot,\cdot)$ 與溫度 $\tau > 0$，

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp\left(s(q,k^+)/\tau\right)}{\exp\left(s(q,k^+)/\tau\right) + \sum_{j=1}^{N-1} \exp\left(s(q,k_j^-)/\tau\right)}.$$

這正是一個 N 類分類問題的 softmax 交叉熵，正確類別就是正例。由此得到兩個結果。

- **互資訊的下界**。CPC 證明 $I(q;k^+) \geq \log N - \mathcal{L}_{\text{InfoNCE}}$，其中 I 是互資訊（mutual information）。最小化損失會提高這個下界，而負例越多（N 越大）下界可以越高。實務上這個界很鬆，特徵品質也高度依賴編碼器與資料增強，所以請把它當成動機，而不是特徵為什麼好的解釋。
- **溫度控制難度**。除以較小的 τ 會讓 softmax 更尖銳，使損失集中在與查詢最相似的負例（困難負例）上。

SimCLR

SimCLR [7] 是最簡單形式的 InfoNCE。取一批 B 張影像，各產生兩個增強視圖（隨機裁切並縮放、顏色扭曲、模糊），共 $2B$ 個視圖。骨幹網路 f 把每個視圖對應到 h，一個小型 MLP **投影頭**（projection head）g 再把 h 對應到 $z = g(h)$，相似度用餘弦相似度 $s(z_i, z_j) = z_i^\top z_j / (\|z_i\|\|z_j\|)$。對於搭檔為 j 的視圖 i，

$$\ell_{i,j} = -\log \frac{\exp(z_i^\top z_j/\tau)}{\sum_{k \neq i} \exp(z_i^\top z_k/\tau)},$$

其中 z 已經做過 ℓ_2 正規化，總和遍及其他所有 $2B-1$ 個視圖。批次裡其他每張影像都是負例，因此不需要記憶庫，但這個方法受益於大批次。總損失對每個正配對的兩種順序取 $\ell_{i,j}$ 的平均；論文稱之為 NT-Xent 損失（normalized temperature-scaled cross-entropy）。

SimCLR 論文有三個值得記住的發現 [7]。資料增強的組合非常重要，其中隨機裁切加上顏色扭曲特別關鍵。投影頭有幫助：投影頭之前的特徵（h）比之後的（z）更能遷移，所以預訓練後會丟掉投影頭。而且對比式學習比監督學習更受益於大批次與更長的訓練。

程式：檢查損失與塌縮時的值

這段獨立的 PyTorch 範例用合成的配對向量驗證損失、自身遮罩與梯度。它不訓練編碼器。

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
B, D = 8, 16
base = torch.randn(B, D)
views = torch.cat([base + .1*torch.randn_like(base),
                   base + .1*torch.randn_like(base)]).requires_grad_()
z = F.normalize(views, dim=-1)
logits = z @ z.T / .2
self_mask = torch.eye(2*B, dtype=torch.bool)
logits = logits.masked_fill(self_mask, float("-inf"))
positive = (torch.arange(2*B) + B) % (2*B)
loss = F.cross_entropy(logits, positive)
loss.backward()
assert torch.isfinite(views.grad).all()
print("paired loss:", round(loss.item(), 4))

collapsed = F.normalize(torch.ones(2*B, D), dim=-1)
scores = (collapsed @ collapsed.T / .2).masked_fill(self_mask, float("-inf"))
collapsed_loss = F.cross_entropy(scores, positive)
print("collapsed loss:", round(collapsed_loss.item(), 4))
print("expected:", round(torch.log(torch.tensor(2*B-1.)).item(), 4))
```

它印出配對損失 0.1423 與塌縮損失 2.7081，正好等於 $\log 15$。如果所有表示都相同，正例得到的機率是 $1/(2B-1)$，損失就是 $\log(2B-1)$，也就是均勻亂猜的最差值。因此對比式的鑑別讓完全塌縮成為一個很差的解，雖然最佳化與負例選擇仍然可能很困難。

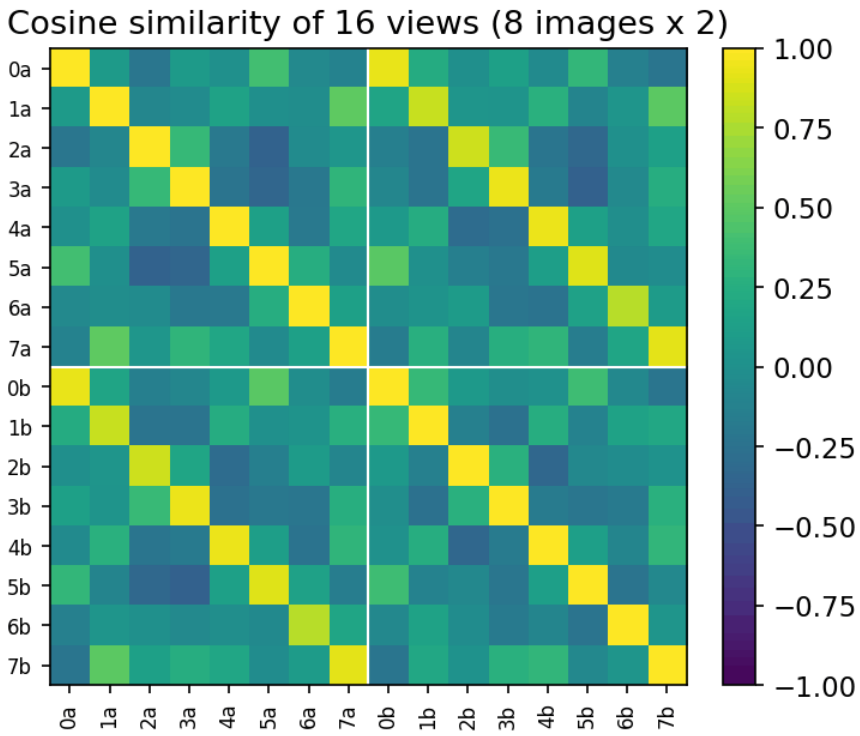


圖 *3* — 用本章稍後的小型 *SimCLR* 編碼器訓練後，*16* 個視圖（*8* 張數字影像，各兩個增強）之間的餘弦相似度。明亮的副對角線就是正配對；*InfoNCE* 把它們往上推，把其他所有元素往下壓。

MoCo：負例佇列與動量編碼器

InfoNCE 偏好大量負例，但 SimCLR 只能從當前批次取得負例。**動量對比**（Momentum Contrast, MoCo）[8] 用兩個想法把負例數量和批次大小脫鉤。

1. **佇列**。最近幾個批次編碼出來的鍵被存進長度為 K 的先進先出佇列，重複當作負例使用。無論批次多小，查詢都能看到 K 個負例。

2. **動量編碼器**。佇列裡的鍵是由稍早版本的網路產生的。如果鍵編碼器變化太快，新舊鍵就會不一致。因此 MoCo 用另一個編碼器計算鍵，它的參數 θ_k 緩慢地跟隨查詢編碼器的參數 θ_q：

$$\theta_k \leftarrow m\,\theta_k + (1 - m)\,\theta_q,$$

其中 $m \in [0, 1)$ 是動量，接近 1（論文預設 0.999）。只有 θ_q 接收梯度。這種**指數移動平均**（exponential moving average, EMA）教師的想法，在 BYOL、DINO 與 JEPA 中都會再次出現。

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
```

```
B, D, K = 8, 16, 32          # batch, embedding size, queue length

# --- MoCo-style InfoNCE: one positive key per query, negatives from a queue
q = F.normalize(torch.randn(B, D), dim=1)                 # queries (online encoder)
k = F.normalize(q + 0.1 * torch.randn(B, D), dim=1)       # keys (momentum encoder)
queue = F.normalize(torch.randn(K, D), dim=1)             # keys from past batches
l_pos = (q * k).sum(1, keepdim=True)                      # B x 1
l_neg = q @ queue.T                                       # B x K
logits = torch.cat([l_pos, l_neg], dim=1) / 0.07
loss_moco = F.cross_entropy(logits, torch.zeros(B, dtype=torch.long))
queue = torch.cat([k, queue])[:K]                         # enqueue new, drop oldest
print("MoCo logits:", tuple(logits.shape), "loss:", round(loss_moco.item(), 3))

# momentum (EMA) update of the key encoder's parameters
theta_q, theta_k, m = torch.randn(5), torch.zeros(5), 0.999
theta_k = m * theta_k + (1 - m) * theta_q
print("key params moved by:", round((theta_k.norm() / theta_q.norm()).item(), 4))
```

每個查詢面對一個 $1 + K$ 類的分類問題，正確答案永遠在第 0 欄，所以目標全為零。輸出顯示 logits 的形狀是 `(8, 33)`，而 EMA 一步只讓鍵編碼器往查詢編碼器移動了 0.1%（從零開始、走一步）。實際實作中，q 與 k 來自兩個編碼器分別作用在同一批影像的兩個增強版本上。

對齊性與均勻性

Wang 與 Isola [9] 從幾何角度清楚說明對比式損失在單位球面上做了什麼。在正規化特徵下，損失在漸近意義上最佳化兩個性質：

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{(x,x^+)} \|f(x) - f(x^+)\|_2^2, \qquad \mathcal{L}_{\text{unif}} = \log \mathbb{E}_{x,y}\, e^{-t\|f(x)-f(y)\|_2^2},$$

其中 (x, x^+) 是正配對，x, y 是獨立樣本，$t > 0$（常用 $t = 2$）。正配對被映射到相近的點時，**對齊性**（alignment）就低。特徵在球面上分散開來時，**均勻性**（uniformity）就低，也就是盡可能保留輸入的資訊。塌縮的編碼器有完美的對齊性（0）和最差的均勻性（0）；有用的編碼器兩者都要小。

```
import torch
from torch.nn import functional as F

def alignment(z1, z2):            # mean squared distance between positive pairs
    return (z1 - z2).square().sum(1).mean()

def uniformity(z, t=2.0):         # log of mean Gaussian potential over all pairs
    return torch.pdist(z).square().mul(-t).exp().mean().log()

torch.manual_seed(0)
n, D = 512, 16
collapsed = F.normalize(torch.ones(n, D), dim=1)
spread = F.normalize(torch.randn(n, D), dim=1)
noisy_view = F.normalize(spread + 0.1 * torch.randn(n, D), dim=1)
print("collapsed: align", alignment(collapsed, collapsed).item(),
```

```
      "uniform", round(uniformity(collapsed).item(), 3))
print("spread:    align", round(alignment(spread, noisy_view).item(), 3),
      "uniform", round(uniformity(spread).item(), 3))
```

塌縮的集合得到對齊性 0.0、均勻性 0.0；分散的集合大約是 0.138 與 −3.51。圖 4 追蹤下面訓練數字編碼器時的這兩個量。在第 0 步，隨機網路把每個視圖都映射到幾乎同一點，所以對齊性接近零只是因為一切都塌縮了。訓練先把特徵分散開來（均勻性急遽下降），再逐步把正例拉回彼此身邊。

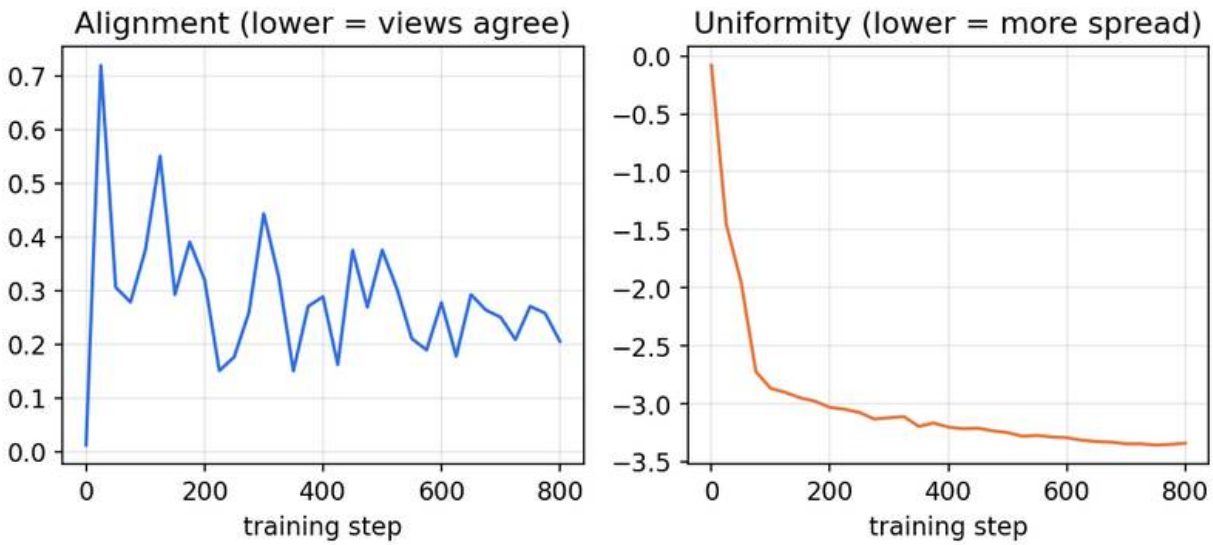


圖 *4* — 在 *8×8* 數字上以 *SimCLR* 訓練時，投影特徵的對齊性與均勻性。單獨的低對齊性沒有意義（常數編碼器也有）；有用的狀態是低對齊性與低均勻性同時成立。

實作範例：在 8×8 數字上跑 SimCLR

白話版。完全不使用標記，在 scikit-learn 的數字資料集上訓練一個迷你 SimCLR 編碼器，再檢查它的特徵能不能讓最近鄰分類器更穩健。

這裡的資料增強是最多一個像素的隨機平移、隨機亮度變化與像素雜訊。因此我們預期學到的特徵對小平移大致不變，這是原始像素做不到的。評估時，每一種特徵都使用同一組有標記的訓練集，並分別在乾淨的數字與最多平移一個像素的數字上測試。

```
import copy
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier

torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(1)
d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16., d.target, test_size=.3,
                                      random_state=0, stratify=d.target)
Xtr_t = torch.tensor(Xtr, dtype=torch.float32)

def augment(x):                       # the augmentations define the invariances
    img = F.pad(x.view(-1, 1, 8, 8), (1, 1, 1, 1))
    i, j = np.random.randint(0, 3, size=2)               # shift by up to 1 px
    img = img[:, :, i:i + 8, j:j + 8]
    img = img * (0.7 + 0.6 * torch.rand(len(img), 1, 1, 1))  # brightness
    return (img + 0.1 * torch.randn_like(img)).reshape(len(x), 64)  # noise

backbone = nn.Sequential(nn.Unflatten(1, (1, 8, 8)),
                         nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
                         nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
                         nn.AdaptiveAvgPool2d(2), nn.Flatten())  # h: 256-d
random_backbone = copy.deepcopy(backbone)                    # baseline
head = nn.Sequential(nn.ReLU(), nn.Linear(256, 64))          # projection g
opt = torch.optim.Adam([*backbone.parameters(), *head.parameters()], lr=1e-3)

def nt_xent(z1, z2, tau=0.2):
    z = F.normalize(torch.cat([z1, z2]), dim=-1)
    n = len(z1)
    logits = (z @ z.T / tau).fill_diagonal_(float("-inf"))
    return F.cross_entropy(logits, (torch.arange(2 * n) + n) % (2 * n))

for step in range(800):                                      # about a minute on CPU
    x = Xtr_t[torch.randint(len(Xtr_t), (256,))]
    loss = nt_xent(head(backbone(augment(x))), head(backbone(augment(x))))
    opt.zero_grad(); loss.backward(); opt.step()

def shift(x, dy, dx):                    # move one 8x8 image, fill with zeros
    img = np.zeros((10, 10)); img[1:9, 1:9] = x.reshape(8, 8)
    return img[1 - dy:9 - dy, 1 - dx:9 - dx].ravel()

moves = np.random.default_rng(0).integers(-1, 2, size=(len(Xte), 2))
Xte_shift = np.stack([shift(x, *m) for x, m in zip(Xte, moves)])
enc = lambda net: (lambda A: net(torch.tensor(A, dtype=torch.float32)).detach().numpy())
pca = PCA(32, random_state=0).fit(Xtr)
for name, f in [("raw pixels", lambda A: A), ("PCA-32", pca.transform),
                ("random CNN", enc(random_backbone)), ("SimCLR CNN", enc(backbone))]:
    knn = KNeighborsClassifier(5, metric="cosine").fit(f(Xtr), ytr)
    print(f"{name:11s} clean {knn.score(f(Xte), yte):.3f}  "
          f"shifted {knn.score(f(Xte_shift), yte):.3f}")
```

在我們的機器上，輸出為：

```
raw pixels  clean 0.978  shifted 0.524
PCA-32      clean 0.978  shifted 0.507
```

```
random CNN   clean 0.974  shifted 0.524
SimCLR CNN   clean 0.987  shifted 0.919
```

確切數字會因 PyTorch 版本與硬體略有不同，重點是模式。在乾淨的數字上，原始像素已經非常好：這個資料集很簡單，花俏的編碼器幫助不大。在平移過的數字上，原始像素、PCA 以及同架構但未訓練的 CNN 都掉到 50% 左右，而自監督特徵保住了大部分的準確率。網路學到了我們透過資料增強要求的不變性，而且僅止於此：如果改用旋轉過的數字測試（資料增強從未出現過旋轉），就不應期待同樣的穩健性。

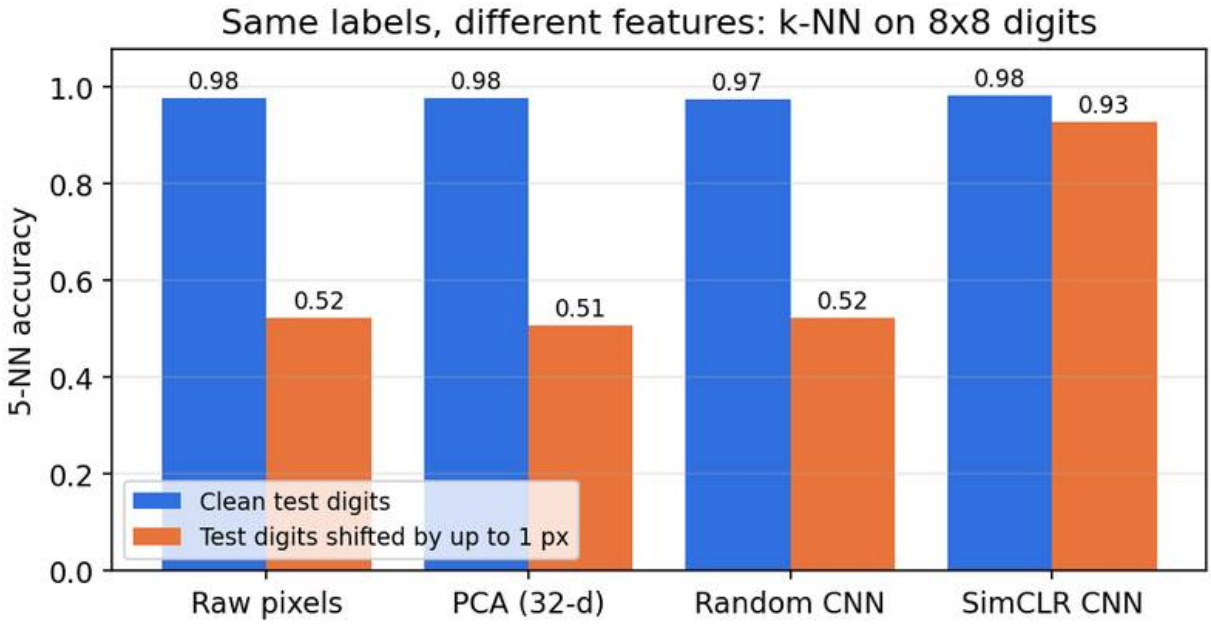


圖 *5* — 相同的標記、相同的分類器、不同的特徵。繪圖腳本使用另一串亂數，所以數字與上面印出的結果最多相差 *0.01*。隨機 *CNN* 排除了「光靠卷積就能帶來穩健性」這個解釋。

沒有負例：為什麼只求一致會塌縮

白話版。如果唯一的指令是「讓兩個輸出一致」，那麼對每張影像都輸出同一個常數就完美達成了。沒有負例的方法必須靠別的機制阻止這件事。

對兩個視圖 v_1, v_2 最小化 $\|f_\theta(v_1) - f_\theta(v_2)\|^2$，有一個平凡解 $f_\theta(x) = c$，其中 c 是常數。光是一致，並不要求特徵攜帶任何資訊。**完全塌縮**把所有東西映射到同一點；**維度塌縮**（dimensional collapse）比較隱微：特徵會變化，但只沿著少數幾個方向，浪費了 D 維中的大部分。

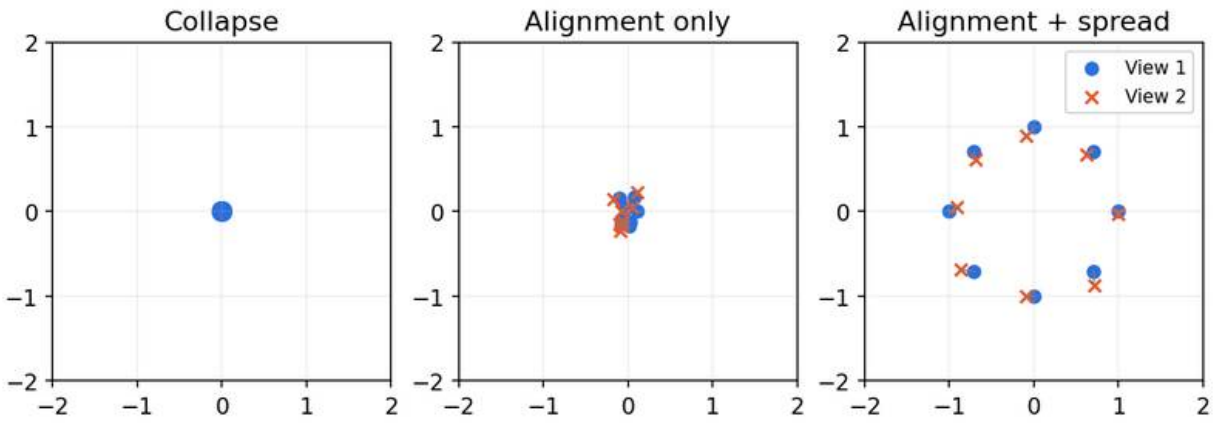


圖 6 — 塌縮、只有對齊沒有分散，以及對齊且分散的示意圖。這些點是刻意構造的說明，不是已發表模型學到的嵌入。

BYOL：線上網路追逐緩慢的目標

BYOL [10] 使用兩個網路。**線上**（online）網路（編碼器、投影器，再加一個**預測器** q_ϕ）被訓練去預測**目標**（target）網路對同一張影像另一個視圖的投影。目標網路不靠梯度訓練，而是線上權重的 EMA，和 MoCo 一樣：

$$\theta_{\text{target}} \leftarrow m\,\theta_{\text{target}} + (1-m)\,\theta_{\text{online}}.$$

損失是正規化後的預測與正規化後的目標投影之間的平方距離，並對兩個視圖做對稱化。目標輸出被當作常數（**停止梯度**，stop-gradient）。沒有負例，BYOL 實務上卻不會塌縮。不對稱性（一邊有預測器，另一邊有停止梯度與 EMA）以及完整的訓練配方都很重要；光有 EMA 並不能證明不可能塌縮。

SimSiam：停止梯度是關鍵成分

Chen 與 He 的 SimSiam [11] 把 BYOL 再進一步精簡：兩個視圖共用一個編碼器，一個分支上有預測器 h，**沒有**動量編碼器、**沒有**負例，也不需要大批次。令 $p_1 = h(f(v_1))$、$z_2 = f(v_2)$，

$$\mathcal{L} = \tfrac{1}{2}\,\mathcal{D}(p_1, \text{sg}(z_2)) + \tfrac{1}{2}\,\mathcal{D}(p_2, \text{sg}(z_1)), \qquad \mathcal{D}(p,z) = -\frac{p^\top z}{\|p\|\,\|z\|},$$

其中 sg 是停止梯度，$\mathcal{D}$ 是負的餘弦相似度。論文實驗顯示，拿掉停止梯度會讓模型很快塌縮，而預測器也是必要的。作者把這個方法詮釋為一種交替式、類似期望最大化（EM）的最佳化。實務上的教訓是：非常小的架構不對稱，就決定了這些方法是在學習還是在塌縮。

DINO：帶中心化與銳化的自蒸餾

DINO [12] 把一致性問題變成帶軟標記的分類。教師與學生都透過帶溫度的 softmax，在 K 個「原型」維度上輸出一個機率向量。看到視圖 x' 的學生，被訓練去符合教師在另一個視圖 x 上的分布：

$$\mathcal{L} = -\sum_{k=1}^{K} P_t(x)_k \, \log P_s(x')_k, \qquad P_t(x) = \mathrm{softmax}\Big(\frac{g_t(x) - c}{\tau_t}\Big),$$

其中 g_t 是教師網路（學生的 EMA），c 是教師輸出的移動平均（**中心化**，centering），τ_t 是較低的教師溫度（**銳化**，sharpening）。中心化防止某一維獨大；銳化防止輸出變成均勻分布。兩者各自單獨使用都會以不同方式塌縮；合在一起才會平衡。DINO 也使用**多重裁切**（multi-crop）：教師只看大的全域裁切，學生還會看到小的局部裁切，因此學生必須從局部證據推斷全域內容。搭配 Vision Transformer 時，DINO 的注意力圖在完全沒有分割標記的情況下就能凸顯物體邊界，這點很受矚目。

Barlow Twins 與 VICReg：直接規範統計量

另一條路是直接在損失中禁止塌縮。**Barlow Twins** [13] 計算兩個視圖（在批次上標準化後）嵌入之間的 $D \times D$ 互相關矩陣 C，並把它推向單位矩陣：

$$\mathcal{L}_{\mathrm{BT}} = \sum_i (1 - C_{ii})^2 + \lambda \sum_i \sum_{j \neq i} C_{ij}^2,$$

第一項讓每一維在兩個視圖間一致，第二項（權重 $\lambda > 0$）讓不同維度去相關，避免攜帶重複的資訊。**VICReg** [14] 把同樣的想法拆成三個有名字的項：**不變性**（兩個嵌入間的均方距離）、**變異數**（用 hinge 損失讓每一維的標準差保持在門檻之上，直接禁止常數輸出）與**共變異數**（懲罰非對角的共變異數元素）。「沒有負例」不代表「沒有防塌縮設計」；這些方法只是把設計放進損失，而不是放進架構。

遮罩建模：從填空中學習

白話版。把輸入的一部分藏起來，預測缺了什麼。BERT 就是這樣學會語言的；只要選對遮住的比例，這招對影像也有效。

BEiT [15] 把 BERT 式的預訓練帶進 Vision Transformer。影像被切成區塊，部分區塊被遮住，網路為被遮住的位置預測離散的**視覺符元**（visual token），這些符元來自另外訓練好的影像符元化器（tokenizer）。

MAE [16] 則直接預測原始像素。它遮住很大比例的區塊，只把可見的區塊送進編碼器，再用一個輕量的解碼器重建缺失的像素。令被遮住的區塊索引集合為 $\mathcal{M}$，

$$\mathcal{L}_{\mathrm{mask}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{x}_i - x_i\|^2,$$

其中 x_i 是區塊 i 的真實像素向量，$\hat{x}_i$ 是預測值。MAE 的設計是刻意的。高遮罩比例（預設 75%）讓任務夠難，網路無法只靠鄰居內插。編碼器只處理可見區塊，降低預訓練成本。用每個目標區塊自身的平均與變異數做正規化，可以改善特徵。預訓練完成後，解碼器就被丟掉。

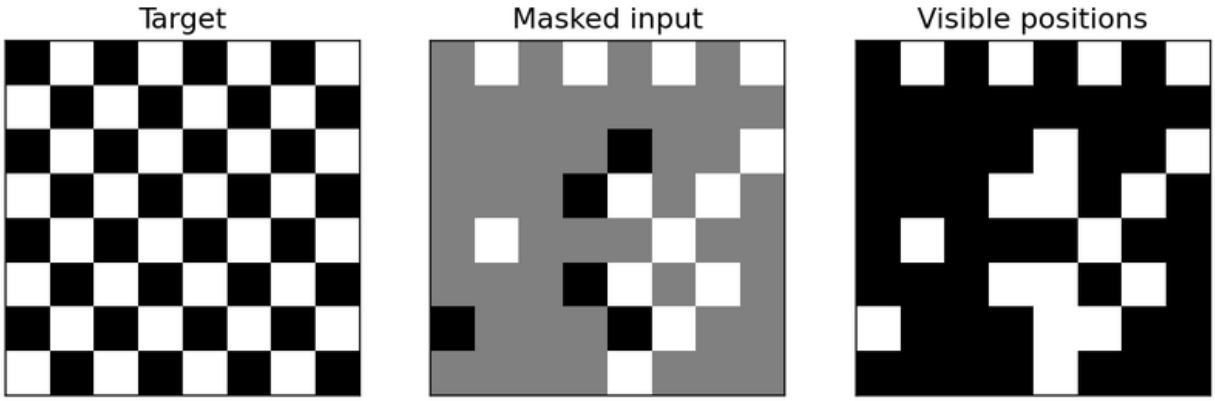


圖 7 — 遮罩預測從輸入本身造出目標。棋盤格只是說明遮罩操作，並不代表語意理解。

重建不等於生成。從可見像素預測被遮住像素的模型，並沒有被訓練成從零開始取樣出完整影像。低重建損失也不保證得到最好的語意特徵：背景紋理可能主導像素誤差。遮罩建模的特徵在線性探測下通常不如對比式或蒸餾式特徵，但微調之後很強，這也是為什麼評估方式必須講清楚。

預測特徵而不是像素：JEPA

白話版。與其把每個不確定的像素都畫出來，不如預測被遮住區域的描述。

I-JEPA [17]（影像聯合嵌入預測架構，Image Joint-Embedding Predictive Architecture）編碼一個可見的**脈絡**（context）區塊，並從中預測數個**目標**（target）區塊的編碼器特徵。目標編碼器是脈絡編碼器的 EMA，預測器則接收脈絡特徵以及目標的位置。簡化後的目標函數是

$$\mathcal{L}_{\text{pred}} = \left\| g_\phi\big(f_\theta(x_{\text{context}}), \text{positions}\big) - \text{sg}\left(f_{\bar{\theta}}(x_{\text{target}})\right) \right\|^2,$$

其中 f_θ 是脈絡編碼器，$f_{\bar{\theta}}$ 是 EMA 目標編碼器，g_ϕ 是預測器，sg 是停止梯度。這個式子只是概括想法；遮罩策略（大的、語意尺度的目標區塊）與架構都是方法的一部分，不是可有可無的細節。

V-JEPA 2 [18] 把特徵預測大規模延伸到影片。它的報告把無動作的影片預訓練，與之後用少量機器人資料訓練、用於規劃的動作條件模型區分開來。預測未來的潛在狀態、辨識動作、生成逼真的未來畫面，是不同的能力；一個「世界模型」的主張應該說清楚它指的是哪一種。

以語言作為監督：CLIP

白話版。網路上到處都是附有圖說的影像。訓練一個影像編碼器和一個文字編碼器，讓每張影像落在自己的圖說旁邊。之後你只要把類別名稱寫成句子，就能分類影像。

CLIP [19] 用從網路收集的影像–文字配對來訓練。在一批 N 個配對中，影像編碼器產生正規化嵌入 $u_1, \dots, u_N$，文字編碼器產生 $v_1, \dots, v_N$。$N \times N$ 的相似度矩陣 $S_{ij} = u_i^\top v_j / \tau$（$\tau$ 是可學習的溫度）的對角線就是正確配對。損失是兩個 InfoNCE 損失的平均，一個沿著列（哪句圖說配這張影像？），一個沿著行（哪張影像配這句圖說？）：

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\Big[\frac{1}{N}\sum_i -\log \frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}} + \frac{1}{N}\sum_j -\log \frac{e^{S_{jj}}}{\sum_i e^{S_{ij}}}\Big].$$

零樣本分類隨之而來。為每個類別嵌入一句提示，例如「a photo of a dog」，再把影像分到提示嵌入最相似的類別。不需要任何任務專屬的標記，不過提示的措辭會影響準確率。

SigLIP [20] 把 softmax 換成對每一個影像–文字配對各自獨立的 **sigmoid** 損失：矩陣中的每個元素都變成一個是非題：「這兩個配對嗎？」這樣就不需要對整個批次做正規化，簡化了大規模分散式訓練。

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
D = 16

# --- CLIP-style symmetric loss on N matched image-text pairs
N = 6
img = F.normalize(torch.randn(N, D), dim=1)
txt = F.normalize(img + 0.3 * torch.randn(N, D), dim=1)   # caption i matches image i
logit_scale = torch.tensor(1 / 0.07)
sim = logit_scale * img @ txt.T                           # N x N, diagonal = matches
labels = torch.arange(N)
loss_clip = (F.cross_entropy(sim, labels) + F.cross_entropy(sim.T, labels)) / 2
print("CLIP loss:", round(loss_clip.item(), 3))

# zero-shot classification: embed one prompt per class, pick the most similar
class_prompts = txt[:3]                                   # stand-ins for "a photo of a
 ↪  {class}"
pred = (img @ class_prompts.T).argmax(1)
print("zero-shot predictions for images 0-2:", pred[:3].tolist())

# SigLIP-style alternative: an independent yes/no decision for every pair
t, b = 10.0, -10.0                                        # learnable in practice
signs = 2 * torch.eye(N) - 1                              # +1 on matches, -1 elsewhere
loss_sig = -F.logsigmoid(signs * (t * img @ txt.T + b)).sum() / N
print("sigmoid loss:", round(loss_sig.item(), 3))
```

合成的圖說是影像嵌入加上雜訊的副本，所以 CLIP 損失很小（0.054），零樣本步驟也找回了正確的「類別」`[0, 1, 2]`。在真正的 CLIP 中，兩個編碼器分別是 Vision Transformer 或 CNN 與文字 Transformer，而困難的部分在資料：圖說描述了什麼、遺漏了什麼，決定了表示知道什麼。sigmoid 損失的值（3.473）不能和 softmax 損失直接比較；它的單位不同，還多了偏置項 b。

DINOv2 與 DINOv3：把特徵當成產品

白話版。只要有足夠的精選資料與算力，預訓練就能產出一個可以凍結、並重複用在許多不同任務上的編碼器，從分類到深度估計都行。

DINOv2 [21] 結合影像層級的自蒸餾（如 DINO）與區塊層級的遮罩目標，並且非常重視**資料策展**（data curation）：從未經整理的影像中，透過與精選影像比對檢索，建出一個大型、多樣且去除重複的預訓練集。成果是一系列凍結後就能搭配簡單任務頭、全域與區塊特徵都好用的編碼器。2025 年發表的 DINOv3 [22] 進一步擴大規模，並加入 **Gram anchoring**，以對抗長時間訓練中密集（區塊）特徵品質下降的問題。它不只是讓每個特徵各自匹配，而是約束區塊特徵之間的相似度矩陣，使其接近較早期模型的矩陣。報告也把一個非常大的教師模型蒸餾成較小的模型以便部署。

實務上的問題是：凍結的全域或密集特徵能不能解決**你的**任務。一個擅長語意檢索的模型，可能會漏掉製造零件上的一道小刮痕。提高輸入解析度並使用區塊特徵可能有幫助，但這會改變運算預算與比較的方式。

誠實地評估遷移

白話版。只有在沒被用來建立或挑選它的資料與任務上也有幫助，一個表示才稱得上好。你怎麼測，會改變答案。

評估方式	訓練什麼	衡量什麼
k-NN	不訓練；與有標記參考特徵的相似度	表示的局部結構
線性探測	線性任務頭；編碼器凍結	簡單讀出就能取得的資訊
微調	任務頭加上部分或全部編碼器層	在給定標記與運算預算下的可調適性
密集遷移	在區塊特徵圖上的空間任務頭	是否適合像素或區域層級的輸出

線性探測與微調

微調的自由度較高，所以當目標資料充足、且與預訓練資料相似時，它通常會贏。但它不一定比較好。Kumar 等人 [23] 從理論與實驗上證明，完整微調可能會**扭曲**好的預訓練特徵：當隨機初始化的任務頭還是錯的時候，它的大梯度流進編碼器，把特徵改往適合訓練分布、卻傷害**分布外**（out-of-distribution, OOD）輸入的方向。線性探測不會扭曲特徵。他們的對策 **LP-FT** 分兩步：先凍結編碼器訓練線性任務頭，再從這個任務頭出發微調全部參數。

我們在數字範例上重現了這個定性效應。編碼器是上面的 SimCLR CNN；有標記的訓練數字是乾淨的；測試數字則是平移過的，屬於輕微的分布偏移。每種標記預算都對三次隨機抽取的標記樣本取平均。

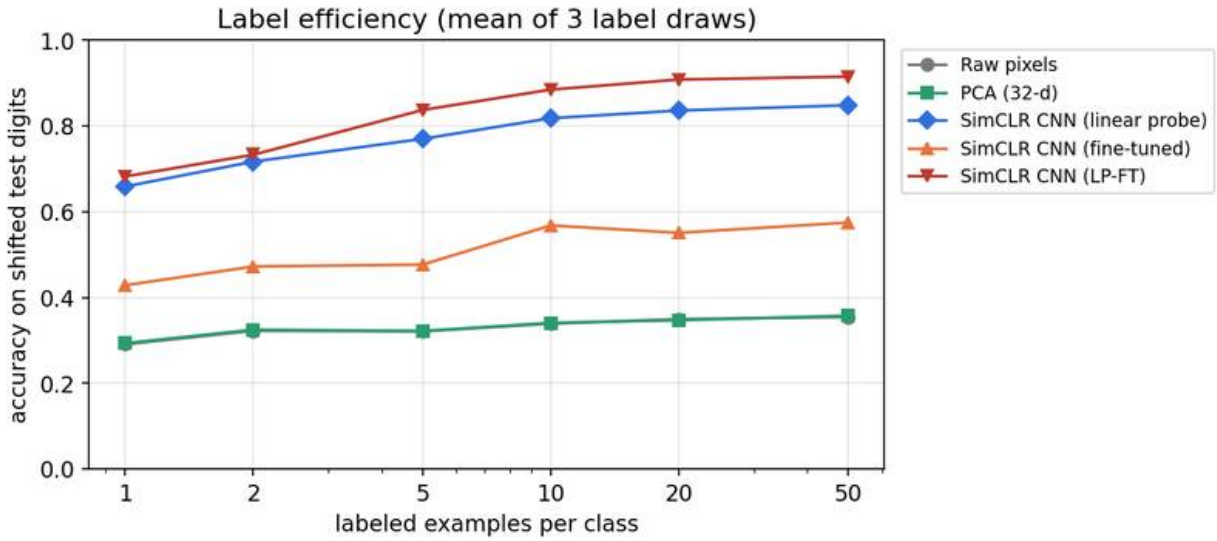


圖 *8* — 平移測試數字上的標記效率。自監督特徵比像素或 *PCA* 更省標記。在固定的一套訓練設定下，從隨機任務頭開始的一般微調降低了平移測試的準確率，而 *LP-FT* 則比探測更好。這是玩具實驗的結果：調好學習率或有更多標記時，一般微調可能表現更好。

請看圖 8 的形狀，而不是確切數值。凍結的自監督特徵搭配線性任務頭，每類只要少數幾個標記，就能勝過每類 50 個標記的原始像素。在這裡，從隨機初始化任務頭開始的一般微調輸給了凍結的探測，而 LP-FT 兩者都贏，這和 Kumar 等人在分布偏移下回報的順序相同。微調需要自己的超參數搜尋，任務頭的初始化也很重要。

防止資料洩漏的規則

- 做相似度比較時，特徵要一致地正規化（餘弦 *k*-NN 需要正規化向量）。
- 標準化、PCA 與分類器的超參數，只能在訓練與驗證資料上擬合。
- 凍結的編碼器仍然有訓練歷史：預訓練資料與評估資料的重疊可能灌水結果，網路規模的模型尤其如此。
- 回報標記數量、骨幹網路、檢查點、輸入解析度、資料增強、任務頭、調參預算與評估方式。在測試集上從許多編碼器中挑最好的，就是測試資料洩漏，即使沒有任何一個被微調。

現代觀點

綜述與回顧論文。四篇綜述從互補的角度描繪這個領域。

- **Bengio、Courville 與 Vincent（TPAMI 2013）[1]** 為整件事定下框架。他們問一個表示憑什麼比另一個好，並以一組通用先驗回答：分散式編碼、深度與抽象、解釋因子的解糾纏（disentangling）、流形與自然分群。他們回顧了機率模型、自編碼器與流形學習。這篇早於現代自監督學習，但仍是「我們為什麼要學表示」最好的陳述。

- **Liu 等人（TKDE 2023）[3]** 把橫跨視覺、語言與圖的自監督方法分成**生成式**（自編碼與自迴歸重建）、**對比式**（脈絡–實例與實例–實例）與**生成–對比式**（對抗式）三組，並討論當時對比式目標為何在視覺基準上勝出。
- **Gui 等人（TPAMI 2024）[4]** 對演算法做了較新的整理，涵蓋對比式、自蒸餾、冗餘抑制與遮罩影像建模方法，並討論應用與開放問題，例如理論、代理任務的選擇與多模態自監督學習。
- **Balestriero 等人（2023）[5]** 是由領域內許多作者合寫的實務指南。它把方法分成深度度量學習家族（SimCLR 及其相關方法）、自蒸餾家族（BYOL、SimSiam、DINO）、典型相關分析家族（VICReg、Barlow Twins）以及遮罩影像建模，並收集了論文常常埋在細節裡的「訣竅」（資料增強、投影器、EMA 排程、批次大小的影響）。Prince 的教科書 [24] 則以平易、圖文並茂的方式並附上 notebook 介紹同樣的概念。

實務現況（2023–2026）。從上面引用的論文中可以看到以下趨勢：

- **結合多種目標**。領先的影像編碼器把影像層級的蒸餾與區塊層級的遮罩預測結合起來（DINOv2 [21]、DINOv3 [22]），而不是只依賴一種代理任務。
- **資料是一級設計選擇**。 DINOv2 的策展流程，以及 CLIP [19] 與 SigLIP [20] 背後網路規模的配對收集，都顯示預訓練吃進什麼資料和損失函數一樣重要。
- **密集特徵**。分割、深度與對應關係需要好的區塊特徵，這促成了像 Gram anchoring [22] 這樣的目標。
- **在潛在空間中預測，並延伸到影片**。 I-JEPA [17] 與 V-JEPA 2 [18] 預測特徵而不是像素，並延伸到影片與動作條件預測。

- **與語言對齊的編碼器成為多模態模型的預設骨幹**（見ML 09）。

開放問題。我們仍然缺少一套能預測「哪種代理任務適合哪種下游任務」的理論；防止塌縮靠的多半是配方而不是保證；評估結果高度依賴評估方式，以及預訓練與測試資料之間的重疊；網路規模的預訓練也繼承了資料中的偏誤與缺口。我們對 [3]、[4]、[5]、[21]、[22] 的解讀是：資料與目標的設計值得和架構一樣多的關注。這是對文獻的詮釋，不是「某個目標到處都贏」的定理。

接下來的章節換一個問題：與其描述一張影像，模型要如何描述並**取樣**整個分布？自編碼器見ML 05A，生成模型見ML 06。

重點整理

- 如果簡單的任務頭只用少量標記就能從表示中解決任務，這個表示就是有用的。分散式編碼能泛化，因為特徵在樣本之間共享。
- 你想要的不變性取決於任務。在自監督學習中，是資料增強在選擇不變性，平移數字的例子就說明了這點。
- 對比式方法（SimCLR、MoCo）就是 InfoNCE 分類：在負例中找出你的正例。塌縮會讓它們付出 $\log(2B-1)$ 的代價。
- 沒有負例的方法（BYOL、SimSiam、DINO）靠不對稱避免塌縮：預測器、停止梯度、EMA 教師、中心化與銳化。Barlow Twins 與 VICReg 則把防塌縮項放進損失。
- 遮罩建模（BEiT、MAE）重建被遮住的內容；JEPA 預測被遮住的特徵。CLIP 讓影像與圖說對齊，做到零樣本分類。

- *k*-NN、線性探測與微調回答的是不同的問題。在分布偏移下，微調可能扭曲特徵；LP-FT 是簡單的解法。

練習

1. **溫度與塌縮**。在第一段程式中，把溫度從 0.2 改成 0.05 和 1.0。塌縮損失會變嗎？配對損失呢？

   提示

   塌縮損失仍是 $\log 15$：所有非自身的 logits 都相等，所以除以任何 τ 之後 softmax 仍是均勻的。配對損失會改變，因為溫度縮放的是分數之間的差距；有真正的差距時，τ 越小 softmax 越尖銳。

2. **資料增強就是假設**。你想要的特徵用於 (a) 顯微影像中的細胞計數、(b) 鳥種辨識、(c) 讀取路標文字。各舉一個你會拿掉的標準 SimCLR 資料增強，並說明原因。

   提示

   (a) 隨機裁切可能改變視野中的細胞數量，所以「相同」的標記其實不再相同。(b) 強烈的顏色抖動可能抹去羽色，而羽色是重要的鳥種線索。(c) 水平翻轉會把文字變成鏡像文字。先決定什麼必須保留，再選擇不變性。

3. **MAE 的運算量**。一張影像有 196 個區塊，MAE 遮住 75%。有多少個區塊進入編碼器？編碼器中兩兩注意力分數的數量縮小為幾分之幾？

   提示

   49 個可見區塊進入編碼器。注意力分數隨符元數量的平方成長：$(49/196)^2 = 1/16$。解碼器仍然要處理全部 196 個位置，但它很小。

4. **破壞數字實驗**。修改 SimCLR 數字程式，讓 `augment` 不做平移。重新訓練，並比較平移測試的 *k*-NN 準確率與原本的結果。這告訴你什麼？

   提示

   把隨機的 `i, j` 換成 `i, j = 1, 1`。在我們的機器上，平移測試的準確率從 0.919 掉到 0.743，而乾淨測試的準確率維持在 0.981。少了平移增強，目標中就沒有任何東西要求平移不變性；剩下的穩健性來自訓練過的卷積加池化編碼器本來就會學到的東西（主範例中未訓練的那個完全沒有）。決定學到什麼的主要是資料增強，而不是「自監督」這個名詞。

5. **紙上的 SimSiam 消融實驗**。在 SimSiam 中，假設你拿掉停止梯度，讓兩個分支都接收梯度。用兩句話說明為什麼常數輸出會變成很容易達到的最小值，以及為什麼光有預測器不足以阻止它。

提示

兩邊都有梯度時，只要 p_1 和 z_2 指向同一方向，損失 $-\cos(p_1, z_2)$ 就達到最小，而一個常數編碼器加上一個輸出同樣常數方向的預測器，對每個輸入都做得到。停止梯度改變了最佳化動態，讓目標在每一步中保持固定；SimSiam 論文回報，沒有它就會迅速塌縮。

6. **公平的探測**。設計一個實驗，在每類 10、50、100 個標記下，比較凍結的預訓練編碼器與 PCA 加邏輯斯迴歸。列出三件必須固定的事，以及兩件必須回報的事。

提示

必須固定：有標記的子集（兩者使用相同樣本）、保留的測試切分，以及選擇正則化強度的程序（只用驗證資料）。必須回報：預訓練編碼器的訓練資料（以及與你測試集的任何重疊），以及隨機抽取標記的次數與其變異範圍。

參考文獻

1. Y. Bengio, A. Courville and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013. Paper ·arXiv
2. J. Yosinski, J. Clune, Y. Bengio and H. Lipson, "How transferable are features in deep neural networks?" NeurIPS, 2014. Paper
3. X. Liu et al., "Self-supervised Learning: Generative or Contrastive," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, 2023 (online 2021). Paper ·arXiv
4. J. Gui et al., "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. Paper ·arXiv
5. R. Balestriero et al., "A Cookbook of Self-Supervised Learning," arXiv:2304.12210, 2023. Paper
6. A. van den Oord, Y. Li and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," arXiv:1807.03748, 2018. Paper
7. T. Chen, S. Kornblith, M. Norouzi and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," ICML, 2020. Paper
8. K. He, H. Fan, Y. Wu, S. Xie and R. Girshick, "Momentum Contrast for Unsupervised Visual Representation Learning," CVPR, 2020. Paper
9. T. Wang and P. Isola, "Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere," ICML, 2020. Paper
10. J.-B. Grill et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," NeurIPS, 2020. Paper
11. X. Chen and K. He, "Exploring Simple Siamese Representation Learning," CVPR, 2021. Paper
12. M. Caron et al., "Emerging Properties in Self-Supervised Vision Transformers," ICCV, 2021. Paper
13. J. Zbontar, L. Jing, I. Misra, Y. LeCun and S. Deny, "Barlow Twins: Self-Supervised Learning via Redundancy Reduction," ICML, 2021. Paper
14. A. Bardes, J. Ponce and Y. LeCun, "VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning," ICLR, 2022. Paper

15. H. Bao, L. Dong, S. Piao and F. Wei, “BEiT: BERT Pre-Training of Image Transformers,” ICLR, 2022. Paper
16. K. He et al., “Masked Autoencoders Are Scalable Vision Learners,” CVPR, 2022. Paper
17. M. Assran et al., “Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture,” CVPR, 2023. Paper
18. M. Assran et al., “V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning,” technical report, 2025. Report
19. A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021. Paper
20. X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer, “Sigmoid Loss for Language Image Pre-Training,” ICCV, 2023. Paper
21. M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. Paper
22. O. Siméoni et al., “DINOv3,” technical report, 2025. Report
23. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, “Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution,” ICLR, 2022. Paper
24. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks

ML 05A・線性因子模型與自編碼器

PCA、機率 PCA、因子分析、ICA、稀疏編碼、NMF 與自編碼器（欠完備、稀疏、去噪、收縮）：壓縮、雜訊假設與流形觀點如何一路通往 VAE。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-05a-factor-models-autoencoders/

你將學到

- 少數幾個隱藏的**因子**（factor）如何解釋大量彼此相關的量測值，以及為什麼這個想法同時支撐了古典統計與現代深度生成模型。
- PCA、機率 PCA 與因子分析的差別，以及它們各自對雜訊做了什麼假設。
- 獨立成分分析（ICA）、稀疏編碼（sparse coding）與非負矩陣分解（NMF）如何藉由不同的額外假設，挑出不同的因子。
- 線性與非線性自編碼器（autoencoder）和這些模型的關係，以及稀疏、去噪、收縮懲罰如何改變自編碼器學到的東西。
- **流形觀點**（manifold view）對高維資料說了什麼，以及它為何自然地把我們從自編碼器帶到變分自編碼器（VAE）。

先看全貌

一張 8×8 的數字影像有 64 個像素值，一張手機照片則有數百萬個。然而，一批這類影像中真正有意義的變化，只取決於少得多的潛在量：是哪個數字、筆畫粗細、傾斜角度、位置、光照。**因子模型**試圖用少數幾個隱藏（潛在）座標來解釋每一筆觀測。**自編碼器**則是一種神經網路，它學著把輸入擠進這樣的座標，再展開還原。

> **用最簡單的話說**
>
> 想像你在電話裡向別人描述朋友的長相。你不會逐一唸出每顆雀斑，而是說「圓臉、短髮、笑容很大、光從左邊打過來」。這幾個詞就是因子：聽的人靠它們就能在腦中拼出一張不錯的畫面。因子模型要找的，是能同時解釋最多張臉的那幾個詞。自編碼器則是一對小幫手：一個把圖片變成幾個詞，另一個把詞變回圖片。兩人一起練習，直到還原的圖片看起來夠像為止。

為什麼要用一整章談這個？因為現代表示學習與生成模型裡幾乎每個想法，都能在這裡找到祖先。用於視覺化的降維、去噪、壓縮、異常偵測、把影像轉成緊湊編碼供擴散模型使用的「tokenizer」，以及 VAE 的機率架構，全都從同一個問題出發：哪一小組原因可能產生了這些觀測？從線性因子模型到自編碼器的脈絡，依循 Goodfellow、Bengio 與 Courville [1, 第 13–14 章] 以及 Murphy [2]；Bengio、Courville 與 Vincent 的回顧論文 [3] 則說明了學習到的表示為何重要。

一個範本，多種模型

本章每一個方法都套用同一個範本。**編碼器**（encoder）把觀測 $x \in \mathbb{R}^d$ 映射成編碼 $z \in \mathbb{R}^k$，**解碼器**（decoder）再把編碼映回重建結果 $\hat{x}$。各模型的差別只在三個選擇：

選擇	本章出現的選項
編碼器／解碼器的形式	線性（PCA、因子分析、ICA、NMF、稀疏編碼）或非線性（深度自編碼器）
編碼的大小	**欠完備**（undercomplete，$k < d$：瓶頸迫使模型壓縮）或**過完備**（overcomplete，$k \geq d$：必須靠其他機制阻止模型直接複製）
額外假設或懲罰	高斯雜訊（PPCA、FA）、獨立且非高斯（ICA）、稀疏（稀疏編碼、稀疏自編碼器）、非負（NMF）、對破壞具穩健性（去噪）、對微小輸入變化不敏感（收縮）

請記住這張表。兩個方法結論不同時，幾乎總是因為它們在某一列做了不同的選擇，而不是其中一個「錯了」。

PCA：最佳的線性重建子空間

白話版。找幾個彼此正交的方向，使得每筆資料只保留在這些方向上的座標時，損失的資訊越少越好。

令 $X \in \mathbb{R}^{n \times d}$ 的每一列是一筆已中心化（減去各欄平均）的資料，共 n 筆。PCA 選擇一個欄向量互相正交且長度為 1 的矩陣 $W \in \mathbb{R}^{d \times k}$，使平方重建誤差最小：

$$\min_{W} \ \|X - XWW^\top\|_F^2 \qquad \text{subject to} \qquad W^\top W = I_k.$$

其中 $\|\cdot\|_F$ 是 Frobenius 範數（所有元素平方和的平方根），$Z = XW \in \mathbb{R}^{n \times k}$ 是**編碼**（也稱分數），$\hat{X} = ZW^\top$ 是重建結果。編碼器是 $x \mapsto W^\top x$，解碼器是 $z \mapsto Wz$：PCA 本身就是一個自編碼器，只是線性且權重共用。

解從哪裡來

兩種等價的觀點會得到同一個答案。

最大變異數。取一個單位向量 w。投影後資料 Xw 的變異數是 $w^\top S w$，其中 $S = \frac{1}{n-1} X^\top X$ 是樣本共變異數矩陣。用拉格朗日乘數 λ，在 $w^\top w = 1$ 的限制下最大化 $w^\top S w$：

$$\nabla_w (w^\top S w - \lambda(w^\top w - 1)) = 0 \quad \Longrightarrow \quad Sw = \lambda w.$$

所以 w 必須是 S 的特徵向量，而它捕捉到的變異數就是對應的特徵值 λ。最好的方向是特徵值最大的特徵向量；在與第一個方向正交的限制下，次好的方向是第二個特徵向量，依此類推。

最小重建誤差。由於 W 的欄向量正交歸一，誤差可以拆成「總變異數減去保留的變異數」。因此最小化誤差等同於最大化保留的變異數，而殘差恰好是被捨棄的特徵值總和（乘上 $n-1$）。Eckart–Young 定理用矩陣語言說了同一件事：把奇異值分解 $X = U\Sigma V^{\top}$ 截斷到最大的 k 個奇異值，就得到 Frobenius 範數下最佳的秩 k 近似。PCA 方向就是前 k 個右奇異向量（V 的欄），而共變異數的特徵值為 $\sigma_j^2/(n-1)$。

程式：用 SVD 實作 PCA

下面的程式在 scikit-learn 內建 digits 資料集的訓練部分上擬合 PCA，在保留的測試影像上評估重建，並與 scikit-learn 的實作互相核對。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)

mean = train.mean(axis=0)
U, s, Vt = np.linalg.svd(train - mean, full_matrices=False)
var = s**2 / (len(train) - 1)              # eigenvalues of the covariance
ratio = np.cumsum(var) / var.sum()
for k in (2, 10, 20):
    W = Vt[:k].T                            # d x k, orthonormal columns
    rec = (test - mean) @ W @ W.T + mean
    print(f"k={k:2d}  explained={ratio[k-1]:.3f}  "
          f"test MSE={np.mean((test - rec)**2):.4f}")

sk = PCA(n_components=10).fit(train)
print("matches sklearn:", np.allclose(np.abs(sk.components_), np.abs(Vt[:10])))
```

輸出：

```
k= 2  explained=0.291  test MSE=0.0546
k=10  explained=0.740  test MSE=0.0202
k=20  explained=0.896  test MSE=0.0084
matches sklearn: True
```

最後一行的 `np.abs` 並不是投機取巧：每個主方向本來就只確定到正負號，因為 w 與 $-w$ 張成同一條直線。也請注意，平均值只在訓練資料上估計，再套用到測試資料；若用測試資料估計平均值，就會洩漏資訊。

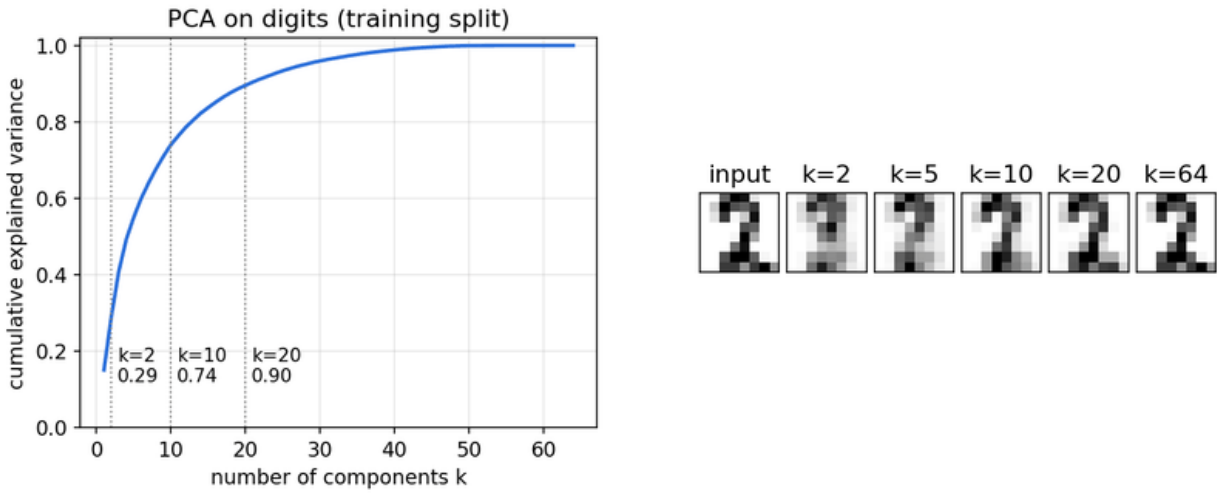


圖 *1* — 在內建 *digits* 資料集上的 *PCA*。左：累積解釋變異數（在訓練資料上擬合）。右：一張保留的測試數字用 k 個成分重建；$k = 64$ 時可完整還原輸入。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

PCA 沒有承諾的事

從目標函數可以直接得出三個提醒。

- **中心化與尺度會改變答案**。沒有中心化時，第一個「成分」大多指向平均值。若把每個特徵標準化為單位變異數，每個像素的權重就相同，因此原始像素 PCA 與標準化 PCA 解的是不同問題。
- **變異數不等於相關性**。變異數最大的方向可能是光照或背景，而不是你關心的類別。PCA 是非監督式的；它最大化保留的變異數，而不是類別分離度（那是線性判別分析等方法的工作）。
- **PCA 是線性的**。若資料落在彎曲的曲面上，平坦的子空間可能需要很多維度，才能描述其實是低維的形狀。我們會在流形一節回到這一點。

機率 PCA 與因子分析

白話版。與其只找一個子空間，不如寫下一個資料如何產生的故事：先抽出幾個隱藏因子，把它們線性混合，再加上雜訊。接著用最大概似法擬合這個故事。

兩個模型共用同一條生成方程式：

$$x = \mu + Wz + \epsilon, \qquad z \sim \mathcal{N}(0, I_k),$$

其中 $\mu \in \mathbb{R}^d$ 是平均值，$W \in \mathbb{R}^{d \times k}$ 是**負荷矩陣**（loading matrix，描述每個因子如何出現在每個量測座標上），z 是潛在因子向量，ϵ 是與 z 獨立的雜訊。兩者只差在雜訊。

機率 PCA（PPCA）

在 PPCA [4] 中，雜訊是等向的，$\epsilon \sim \mathcal{N}(0, \sigma^2 I_d)$：每個座標的雜訊變異數都相同。把 z 積分掉，得到高斯邊際分布

$$x \sim \mathcal{N}(\mu,\, C), \qquad C = WW^{\top} + \sigma^2 I_d.$$

這個共變異數有清楚的解讀：低秩部分 $WW^{\top}$ 由因子解釋，再加上一個因子解釋不了的固定「底噪」σ^2。Tipping 與 Bishop 證明最大概似解有封閉形式 [4]：

$$W_{\mathrm{ML}} = U_k (\Lambda_k - \sigma^2 I)^{1/2} R, \qquad \sigma^2_{\mathrm{ML}} = \frac{1}{d-k} \sum_{j=k+1}^{d} \lambda_j,$$

其中 U_k 是樣本共變異數前 k 個特徵向量，Λ_k 是對應的特徵值 $\lambda_1, \dots, \lambda_k$，$R$ 是任意 $k \times k$ 正交矩陣。換句話說：負荷矩陣恰好張成 PCA 子空間，而雜訊變異數就是 PCA 丟掉的變異數的平均。因此 PPCA 就是 PCA 再加上一個概似函數。這個概似函數很有用：你可以在保留資料上比較不同 k 的模型、用期望最大化（EM）演算法處理缺值，也能建立 PPCA 的混合模型。

給定一筆觀測，其因子的後驗分布是高斯分布，平均為

$$\mathbb{E}[z \mid x] = M^{-1} W^{\top} (x - \mu), \qquad M = W^{\top} W + \sigma^2 I_k.$$

這就是機率式的編碼器。當 $\sigma^2 \to 0$ 時，它趨近一般的 PCA 投影；當 $\sigma^2 > 0$ 時，它會把編碼往零收縮，因為看到的東西有一部分被歸因於雜訊。

因子分析（FA）

因子分析（factor analysis）保留相同結構，但允許對角的雜訊共變異數，$\epsilon \sim \mathcal{N}(0, \Psi)$，$\Psi = \mathrm{diag}(\psi_1, \dots, \psi_d)$。每個觀測座標各有自己的殘差雜訊水準 ψ_i，而潛在因子負責解釋座標之間**共享**的共變異數。邊際分布為 $x \sim \mathcal{N}(\mu, WW^{\top} + \Psi)$，因子的後驗平均為 $\mathbb{E}[z \mid x] = W^{\top}(WW^{\top} + \Psi)^{-1}(x - \mu)$。

FA 沒有封閉解，因此用 EM 或直接最佳化概似函數來擬合。當量測來自雜訊程度不一的感測器，或某些像素幾乎恆定（例如 digits 影像邊框上永遠空白的像素）時，這份額外的彈性就很重要。PCA 會毫不猶豫地把成分花在任何高變異數的座標上；FA 則可以把個別雜訊很大的座標解釋成「雜訊」，把因子留給座標之間的相關性。

旋轉問題

兩個模型都有同一種不確定性。對任意正交矩陣 R，把 W 換成 WR、z 換成 $R^{\top} z$，$WW^{\top}$ 不變，因此 x 的分布也不變，因為 $R^{\top} z$ 仍是標準常態。子空間可以被確定，個別的座標軸卻不行。這就是因子分析文獻使用 varimax 等旋轉來挑選較好解讀的座標軸的原因，也說明了因子不會自動等同於物理上的成因。若要確定個別因子，我們必須在高斯以外再假設些什麼，而 ICA 正是這麼做的。

獨立成分分析（ICA）

白話版。一個房間裡好幾個人同時講話，幾支麥克風各自錄到不同比例的混音。ICA 試著找出統計上盡可能獨立、而且盡可能不像高斯分布的組合，藉此還原每個人的聲音。

無雜訊的 ICA 模型為

$$x = As, \qquad s = (s_1, \dots, s_k) \text{ with independent components,}$$

其中 $x \in \mathbb{R}^d$ 是觀測到的混合訊號，$A \in \mathbb{R}^{d \times k}$ 是未知的**混合矩陣**，來源 s_j 彼此獨立且**非高斯**。目標是找一個解混矩陣 $B \approx A^{-1}$（或其偽逆），使 $\hat{s} = Bx$ 還原出來源 [5]。

為什麼關鍵在於非高斯

上一節的旋轉問題說明了 ICA 為何需要非高斯來源。先把資料**白化**（whitening）：轉換 x 使其共變異數為單位矩陣（先做 PCA，再把每個成分縮放到單位變異數即可）。白化之後，剩下的未知量只是一個旋轉。如果來源是高斯，獨立單位高斯的任何旋轉仍然是獨立單位高斯，於是沒有任何旋轉比較好。ICA 的可識別性定理說的是反方向：只要**至多一個**來源是高斯，來源就能被還原，只剩三種無傷大雅的不確定性 [5]：

- **順序**（我們無法得知哪個來源是「第一個」），
- **正負號**，以及
- **尺度**（大聲的來源配上較小的混合係數，看起來一模一樣）。

中央極限定理提供了找到正確旋轉的直覺：獨立變數的總和往往比各部分更像高斯。所以 x 的投影中最不像高斯的那些，最可能是單一來源而不是混合。因此 ICA 演算法會最大化某種非高斯程度的量度。一個有理論依據的量度是**負熵**（negentropy）：

$$J(y) = H(y_{\text{gauss}}) - H(y) \;\geq 0,$$

其中 H 是微分熵，y_{gauss} 是與 y 變異數相同的高斯變數。在變異數固定的所有分布中，高斯分布的熵最大，所以 $J(y) = 0$ 恰好發生在 y 為高斯時。由於熵很難估計，**FastICA** 演算法用 $G(u) = \log\cosh u$ 等對比函數近似負熵，並以快速的不動點迭代找出每個解混方向 [5]。最大化非高斯程度，與最小化各估計成分之間的互資訊、以及 ICA 模型的最大概似估計密切相關。

程式：把三個訊號解混

這個玩具問題用了一個正弦波、一個方波與一個鋸齒波，全都是非高斯，並以固定矩陣混合。

```
import numpy as np
from sklearn.decomposition import FastICA, PCA

rng = np.random.default_rng(0)
t = np.linspace(0, 8, 2000)
S = np.c_[np.sin(2*t),                  # smooth sine
```

```
          np.sign(np.sin(3*t)),        # square wave
          2*(t % 1) - 1]               # sawtooth
S += 0.05 * rng.normal(size=S.shape)
A = np.array([[1.0, 1.0, 1.0],
              [0.5, 2.0, 1.0],
              [1.5, 1.0, 2.0]])        # unknown mixing matrix
X = S @ A.T                            # three "microphones"

def best_match(est):
    C = np.abs(np.corrcoef(S.T, est.T)[:3, 3:])
    return np.round(C.max(axis=1), 3)

ica = FastICA(n_components=3, whiten="unit-variance", random_state=0)
print("ICA |corr| with each source:", best_match(ica.fit_transform(X)))
print("PCA |corr| with each source:", best_match(PCA(3).fit_transform(X)))
```

輸出：

```
ICA |corr| with each source: [0.997 0.999 0.997]
PCA |corr| with each source: [0.689 0.811 0.735]
```

我們用相關係數的絕對值並取最佳配對來比較，因為 ICA 回傳的來源順序、正負號與尺度都是任意的。PCA 只做去相關：它找的是高變異數的正交方向，在這裡仍然是混合訊號。

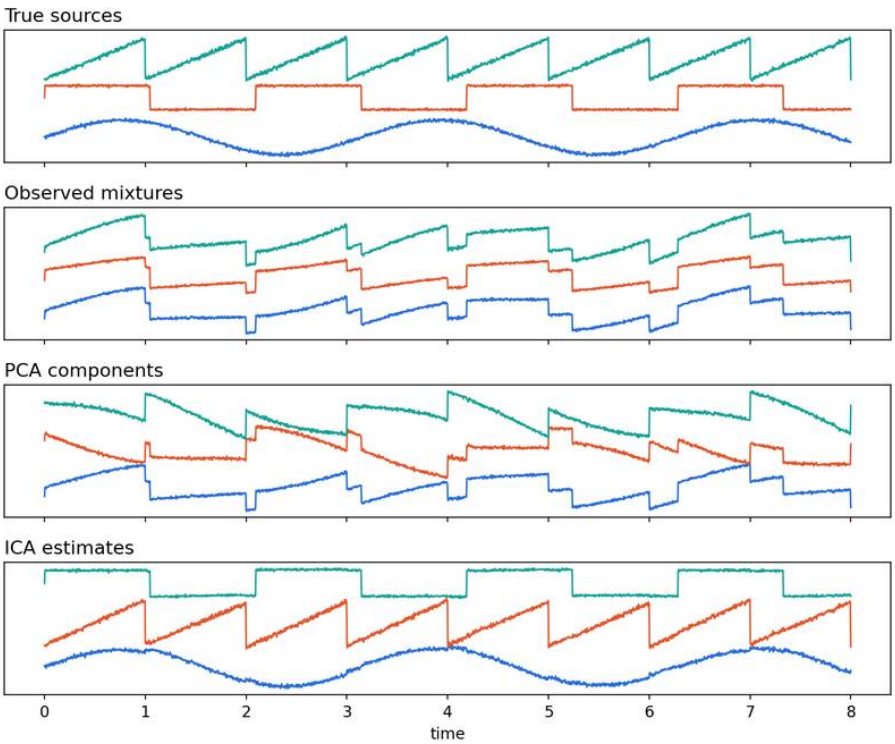


圖 2 — 盲訊號分離。*PCA* 成分仍是混合訊號；*FastICA* 在順序、正負號與尺度的不確定性之內，還原出正弦波、方波與鋸齒波。

ICA 廣泛用於分離腦電圖（EEG）等感測紀錄中的雜訊與偽影。它的假設也劃出了它的極限：它需要真正獨立、非高斯的來源，以及接近線性且瞬時的混合過程。在影像區塊（patch）上，ICA 往往學到局部化、有方向性的邊緣狀濾波器，這也引出一個密切相關的想法。

稀疏編碼

白話版。準備一本很大的小圖樣字典（「原子」，atom），每次描述一個輸入時只用其中少數幾個。

稀疏編碼學習一個字典 $D = [d_1, \dots, d_m] \in \mathbb{R}^{d \times m}$，並為每筆資料 x_i 學一個稀疏編碼 $\alpha_i \in \mathbb{R}^m$，求解

$$\min_{D,\,\{\alpha_i\}} \sum_{i=1}^{n} \left(\tfrac{1}{2} \|x_i - D\alpha_i\|_2^2 + \lambda \|\alpha_i\|_1 \right) \qquad \text{subject to} \qquad \|d_j\|_2 \le 1.$$

第一項要求準確重建；ℓ_1 範數 $\|\alpha\|_1 = \sum_j |\alpha_j|$ 把大多數編碼元素推成恰好為零；$\lambda > 0$ 決定兩者的取捨。原子的範數限制防止模型作弊，也就是把原子放得很大、編碼縮得很小。字典常是**過完備**的（$m > d$）：原子比像素維度多，但對任何一個輸入只有少數幾個會啟動。

Olshausen 與 Field [6] 在自然影像的區塊上訓練這類模型，發現學到的原子是局部化、有方向性、帶通的，非常像初級視覺皮質中簡單細胞的感受野。這個結果影響深遠，因為它顯示：只靠稀疏性、不需要任何標籤，就能解釋生物視覺的一項已知特性。

有兩個實務重點讓稀疏編碼不同於本章稍後的自編碼器：

- **推論本身是一個最佳化問題**。給定 D，要為新輸入找出 α，就得解一個 lasso 問題。這裡沒有固定的編碼器網路；「編碼器」是一個演算法。訓練時交替更新編碼（固定 D）與字典（固定編碼）。
- **稀疏性是一種先驗**。ℓ_1 懲罰對應於編碼上的 Laplace 先驗，所以稀疏編碼可以看成：在一個具有稀疏、非高斯先驗的線性因子模型中做最大後驗（MAP）推論，而這正是 ICA 所屬的同一個家族。

程式：稀疏程度與重建精確度

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import DictionaryLearning
from sklearn.model_selection import train_test_split

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)
for alpha in (0.1, 1.0):
    dl = DictionaryLearning(n_components=32, alpha=alpha, max_iter=30,
                            transform_algorithm="lasso_lars",
                            transform_alpha=alpha, random_state=0)
    dl.fit(train[:600])
```

```
    codes = dl.transform(test)
    rec = codes @ dl.components_
    active = np.mean(np.abs(codes) > 1e-8, axis=1).mean()
    print(f"alpha={alpha}: active atoms per image={active*32:.1f}/32, "
          f"test MSE={np.mean((test - rec)**2):.4f}")
```

輸出（在筆電 CPU 上約需半分鐘）：

```
alpha=0.1: active atoms per image=17.9/32, test MSE=0.0075
alpha=1.0: active atoms per image=3.0/32, test MSE=0.0401
```

提高 λ（這裡的 `alpha`）可以換到稀疏得多的描述，每張影像約啟動 3 個原子而不是 18 個，代價是重建誤差變高。兩種設定都沒有「對錯」：壓縮編解碼器、去噪器與可解釋性工具各自會做不同的選擇。請注意，這個字典沒有減去平均值，所以它的誤差不能直接與上面的 PCA 數字比較。

非負矩陣分解（NMF）

白話版。把每筆資料描述成非負部件的加法組合：你可以加上筆畫，但永遠不能減掉筆畫。

對非負資料 $X \geq 0$（像素強度、詞頻、光譜），NMF 尋找 $Z \geq 0$ 與 $H \geq 0$，使下列重建準則最小：

$$\min_{Z \geq 0,\, H \geq 0} \|X - ZH\|_F^2,$$

其中 $H \in \mathbb{R}^{k \times d}$ 的每一列是一個非負部件，$Z \in \mathbb{R}^{n \times k}$ 的每一列是非負的混合權重。因為沒有東西可以互相抵消，這些部件往往看起來像物體的局部，而不是 PCA 那種有正有負的「幽靈」圖樣。Lee 與 Seung [7] 在人臉與文字上展示了這種以部件為基礎的行為，並推廣了簡單的乘法更新規則來擬合模型。

兩點提醒。第一，以部件為基礎是一種傾向，而不是保證：在某些資料集上，因子並不是乾淨的部件。第二，目標函數對 Z 與 H 聯合而言是非凸的，所以初始化很重要，不同次執行可能得到不同的因子。scikit-learn 的 `init="nndsvda"` 用以 SVD 為基礎的起點讓結果更穩定。

程式：比較各因子模型的重建

這個 CPU 範例只在訓練用的數字上擬合各模型，並在保留的影像上評估重建。它比較的是不同的模型假設，並不是宣稱某個方法在所有任務上都勝出。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA, FactorAnalysis, NMF

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)
```

```
models = {
    "PCA": PCA(n_components=12),
    "factor analysis": FactorAnalysis(n_components=12, random_state=0),
    "NMF": NMF(n_components=12, init="nndsvda", max_iter=1000,
               random_state=0),
}
for name, model in models.items():
    model.fit(train)
    z = model.transform(test)
    if name == "factor analysis":
        reconstructed = z @ model.components_ + model.mean_
    else:
        reconstructed = model.inverse_transform(z)
    mse = np.mean((test-reconstructed)**2)
    print(name, "held-out reconstruction MSE:", round(mse, 5))
```

輸出：

```
PCA held-out reconstruction MSE: 0.01652
factor analysis held-out reconstruction MSE: 0.02501
NMF held-out reconstruction MSE: 0.02207
```

PCA 在平方重建誤差上勝出，這是必然的：在所有秩 12 的線性重建中，對這個準則而言 PCA 子空間就是最佳解（只差在訓練／測試切分）。但這不代表它是最好的模型。上面因子分析的重建是後驗平均重建，它刻意把每個像素的一部分歸因於雜訊；而 FA 真正的產出——一個帶有逐像素雜訊的完整機率模型——完全無法用 MSE 概括。NMF 則用一些誤差換來非負、可加的部件。

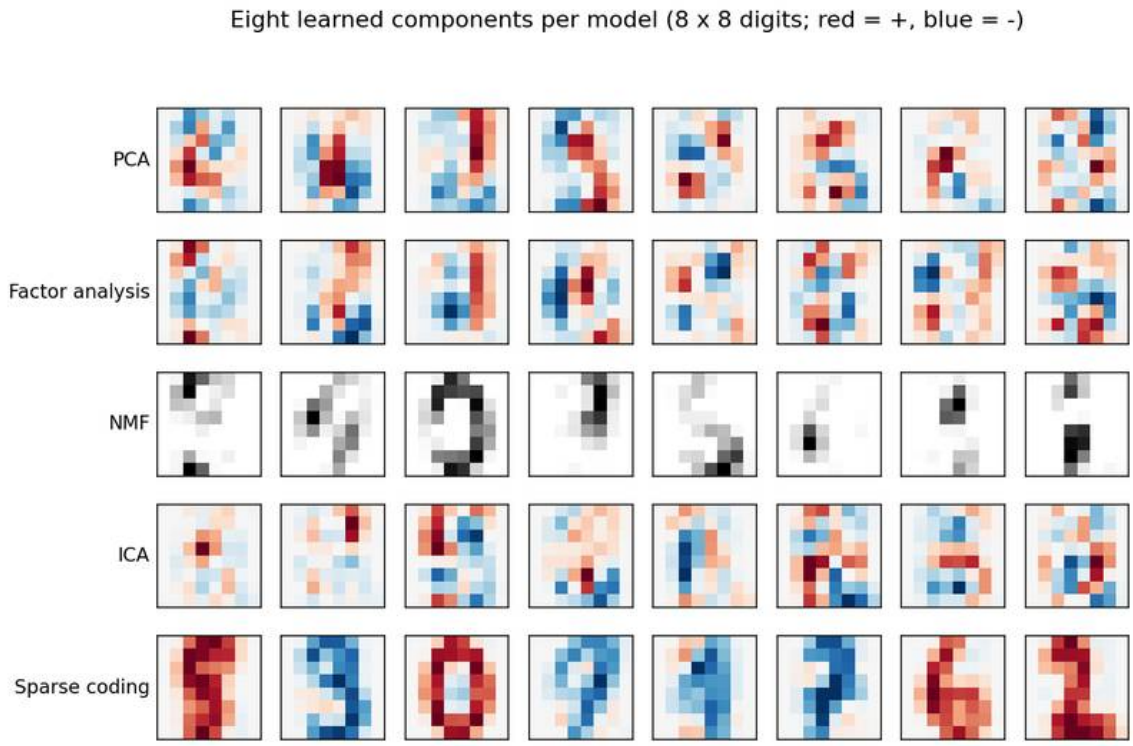


圖 3 — 各線性因子模型在同一批數字上學到的八個成分。*PCA* 與 *FA* 給出有正負號的全域圖樣；*NMF* 給出非負的筆畫；*ICA* 給出有正負號但較局部的圖樣；稀疏編碼的原子像是原型，每次只組合少數幾個。

線性自編碼器與 PCA

白話版。如果你訓練一個完全沒有非線性的神經網路去壓縮並重建輸入，它最後做的就是 PCA，只是座標可能被旋轉或拉伸。

對中心化的輸入，線性自編碼器計算

$$z = W_e x, \qquad \hat{x} = W_d z, \qquad \mathcal{L}(W_e, W_d) = \frac{1}{n}\sum_{i} \|x_i - W_d W_e x_i\|^2,$$

其中編碼器 $W_e \in \mathbb{R}^{k \times d}$、解碼器 $W_d \in \mathbb{R}^{d \times k}$，且 $k < d$。乘積 $W_d W_e$ 的秩至多為 k，所以由 Eckart–Young 定理，最好的乘積就是投影到前 k 個主成分子空間。Baldi 與 Hornik [8] 證明了更多：對這種平方誤差的線性網路，每個局部極小值都是全域極小值，而在全域極小值處，網路投影到主成分子空間。然而個別的權重矩陣並不唯一。對任意可逆的 $k \times k$ 矩陣 G，把 W_e 換成 $G W_e$、W_d 換成 $W_d G^{-1}$，乘積不變，所以編碼座標可以是 PCA 分數的任何可逆線性轉換。

程式：線性自編碼器重新找到 PCA 子空間

```
import numpy as np, torch
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from scipy.linalg import subspace_angles

torch.manual_seed(0)
X = load_digits().data / 16.
X = X - X.mean(axis=0)
Xt = torch.tensor(X, dtype=torch.float32)
k = 5
enc = torch.nn.Linear(64, k, bias=False)
dec = torch.nn.Linear(k, 64, bias=False)
opt = torch.optim.Adam([*enc.parameters(), *dec.parameters()], lr=1e-2)
for step in range(3000):
    loss = (dec(enc(Xt)) - Xt).square().mean()
    opt.zero_grad(); loss.backward(); opt.step()

Wd = dec.weight.detach().numpy()            # 64 x k decoder columns
P = PCA(n_components=k).fit(X).components_.T
angles = np.degrees(subspace_angles(Wd, P))
print("largest angle to PCA subspace (deg):", round(angles.max(), 3))
print("decoder columns orthonormal?",
      np.allclose(Wd.T @ Wd, np.eye(k), atol=1e-2))
```

輸出：

```
largest angle to PCA subspace (deg): 0.003
decoder columns orthonormal? False
```

子空間夾角（兩個 k 維子空間之間的最大夾角）基本上是零：網路找到了 PCA 子空間。但它的解碼器欄向量並不正交歸一，所以它們不是主成分本身。這就是座標不確定性的實際表現，也是本章反覆出現的主題的第一個線索：光靠重建，只能確定保留了什麼，無法確定如何標記它。

非線性自編碼器

白話版。把線性映射換成神經網路，編碼就能描述任何平坦子空間都無法捕捉的彎曲結構。

一般的自編碼器有編碼器 f_ϕ 與解碼器 g_θ，兩者都是神經網路：

$$z = f_\phi(x), \qquad \hat{x} = g_\theta(z), \qquad \min_{\phi,\theta} \frac{1}{n}\sum_i \ell(x_i,\, g_\theta(f_\phi(x_i))),$$

其中 ℓ 是重建損失函數，例如實數值用平方誤差，$[0,1]$ 之間的值用二元交叉熵。ℓ 的選擇本身就是一個假設：平方誤差對應以 $\hat{x}$ 為中心的高斯概似，交叉熵對應 Bernoulli 概似。

Hinton 與 Salakhutdinov [9] 在 2006 年展示：先以逐層貪婪預訓練初始化、再端到端微調的深度自編碼器，在相同維度下找到的低維編碼，重建影像的效果比 PCA 好得多。今天，更好的初始化、正規化層與最佳化器已讓預訓練步驟變得不必要，但核心教訓依然成立：當資料是彎曲的，非線性編碼器可以比線性編碼器壓縮得更徹底。

欠完備還不夠，過完備需要規則

欠完備自編碼器（$k < d$）被迫壓縮。這很有用，但並不保證能學到好特徵。夠強大的編碼器與解碼器在原理上可以把每個訓練樣本映到它自己的任意編碼（例如它的索引），再把解碼記下來，在沒學到任何可重用結構的情況下達到很低的訓練誤差 [1, 第 14 章]。**過完備**自編碼器（$k \geq d$）則可以直接學成恆等函數。兩種情況下，讓表示有用的都不是瓶頸本身，而是歸納偏誤（inductive bias）：有限的容量、架構（例如卷積），以及明確的懲罰項。下一節介紹三種經典懲罰，合稱**正則化自編碼器**（regularized autoencoders）。

正則化自編碼器：稀疏、去噪與收縮

每一種懲罰都用不同方式回答「編碼應該忽略什麼？」這個問題。

稀疏自編碼器

白話版。允許有很多編碼單元，但要求對任何一個輸入只有少數幾個啟動。

損失函數在編碼 $h = f_\phi(x)$ 上加一個稀疏懲罰：

$$\mathcal{L} = \ell(x,\, g_\theta(h)) + \lambda\, \Omega(h), \qquad \Omega(h) = \sum_j |h_j|.$$

其他做法包括：懲罰每個單元平均啟動率偏離一個小目標值的程度，或只保留最大的 k 個啟動值（「top-k」或 k-稀疏自編碼器）。稀疏自編碼器是稀疏編碼的「攤銷式」（amortized）表親：編碼器網路一次前向傳遞就預測出稀疏編碼，而不必為每個輸入解一次 lasso。Goodfellow 等人把這個懲罰解讀為編碼上的近似先驗，而不是一般的權重正則化 [1, 第 14 章]。正如現代觀點一節會說明的，稀疏自編碼器近年已成為解讀大型神經網路內部的關鍵工具。

去噪自編碼器

白話版。給網路看一個受損的輸入，要它重建出乾淨的版本。要做到這點，它必須學會乾淨的資料長什麼樣子。

去噪自編碼器（denoising autoencoder，DAE）先抽出一個受破壞的副本 $\tilde{x} \sim C(\tilde{x} \mid x)$，再最小化

$$\mathcal{L} = \mathbb{E}_x\, \mathbb{E}_{\tilde{x} \sim C(\cdot \mid x)} \big[\, \ell(x,\, g_\theta(f_\phi(\tilde{x}))) \,\big],$$

其中 C 是事先選定的破壞過程：加性高斯雜訊、遮罩雜訊（把隨機一部分輸入設為零），或椒鹽雜訊 [10], [11]。Vincent 等人在 2008 年提出這個想法 [10]，並在 2010 年證明把 DAE 層堆疊起來，能為分類器提供有用的特徵 [11]。由於輸入與目標不同，即使編碼是過完備的，直接複製也不再是解。

破壞過程決定了模型學會對什麼具有穩健性。用高斯雜訊訓練的 DAE 沒有理由去補齊缺失的區塊，而用遮罩雜訊訓練的 DAE 則會學到補全。現代的遮罩影像建模，例如ML 05 中的遮罩自編碼器（MAE），就是遮罩雜訊 DAE 的直系後代。

這裡還有更深一層的幾何解讀。對微小的高斯破壞，向量 $g(f(\tilde{x})) - \tilde{x}$ 會從受損的點指回資料密度較高的區域；它近似估計了**分數**（score）$\nabla_x \log p(x)$，也就是對數密度的梯度 [1, 第 14 章]。這個關聯是分數式與擴散生成模型的根源之一，詳見ML 07。

程式：在數字上訓練去噪自編碼器

這個獨立的 PyTorch 範例在訓練用的數字上訓練一個小型非線性 DAE，並評估從加了雜訊的保留輸入重建乾淨影像的效果。

```
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(1)
train, test = train_test_split(load_digits().data/16., test_size=.2, random_state=0)
X = torch.tensor(train, dtype=torch.float32)
Xt = torch.tensor(test, dtype=torch.float32)
encoder = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 12))
decoder = nn.Sequential(nn.Linear(12, 32), nn.ReLU(), nn.Linear(32, 64), nn.Sigmoid())
model = nn.Sequential(encoder, decoder)
opt = torch.optim.Adam(model.parameters(), lr=.003)
for step in range(2000):
    clean = X[torch.randint(len(X), (64,))]
    noisy = (clean + .2*torch.randn_like(clean)).clamp(0, 1)
    loss = (model(noisy)-clean).square().mean()
    opt.zero_grad(); loss.backward(); opt.step()
model.eval()
with torch.no_grad():
    noisy = (Xt + .2*torch.randn_like(Xt)).clamp(0, 1)
    print("noisy-input MSE:", (noisy-Xt).square().mean().item())
    print("reconstructed MSE:", (model(noisy)-Xt).square().mean().item())
```

輸出（含雜訊輸入約 0.0248，經過 DAE 後約 0.0197；最後幾位數依 PyTorch 版本而異）：

```
noisy-input MSE: 0.024781296029686928
reconstructed MSE: 0.01968955807387829
```

改善是真實的，但幅度不大，圖 4 說明了原因：在 12 維的編碼下，DAE 去除了斑點雜訊，但也把一些筆畫細節抹平了。去噪與忠實壓縮往不同方向拉扯；更寬的編碼或卷積架構會改變兩者的平衡。

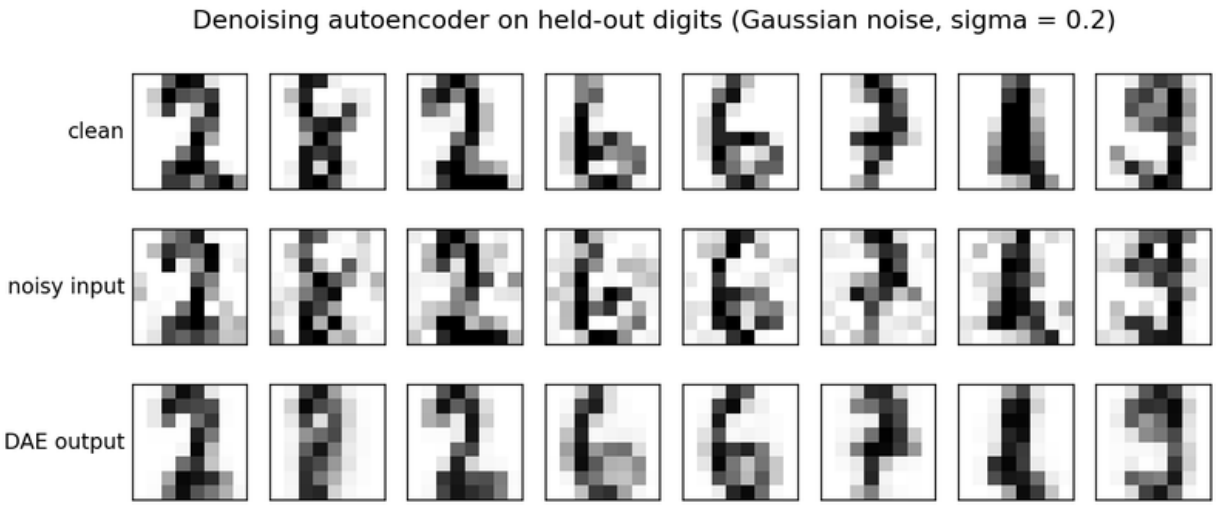


圖 *4* — 去噪自編碼器在保留數字上的輸出。背景斑點被去除；部分細微筆畫細節因 *12* 個單元的瓶頸而遺失。

收縮自編碼器

白話版。只要輸入的微小變化造成編碼的大幅變動，就懲罰編碼器。編碼應該只對重要的變化有反應。

Rifai 等人 [12] 加上編碼器 Jacobian 矩陣的 Frobenius 範數平方：

$$\mathcal{L} = \ell(x,\, g_\theta(f_\phi(x))) + \lambda \left\| \frac{\partial f_\phi(x)}{\partial x} \right\|_F^2, \qquad \left\| \frac{\partial f_\phi(x)}{\partial x} \right\|_F^2 = \sum_{j,i} \left(\frac{\partial h_j}{\partial x_i} \right)^2 .$$

其中 $h = f_\phi(x)$，Jacobian 元素 $\partial h_j / \partial x_i$ 衡量輸入座標 i 變動時，編碼單元 j 移動多少。對單層 sigmoid 編碼器 $h = \sigma(Wx + b)$，該元素為 $h_j(1 - h_j)W_{ji}$，所以懲罰項很容易計算：

$$\left\| \frac{\partial h}{\partial x} \right\|_F^2 = \sum_j \left(h_j(1-h_j) \right)^2 \sum_i W_{ji}^2 .$$

重建項希望編碼隨輸入改變（否則無法重建不同的輸入），懲罰項則希望編碼盡量少變。折衷的結果是：編碼只沿著訓練資料實際變化的少數方向保持敏感。DAE 以隱含方式達到相關的效果：對微小高斯雜訊，去噪訓練懲罰的是整個重建函數的敏感度 [1, 第 14 章]；而 CAE 則明確地只懲罰編碼器。

程式：收縮懲罰的實際效果

```
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

train, test = train_test_split(load_digits().data/16., test_size=.2, random_state=0)
X = torch.tensor(train, dtype=torch.float32)
Xt = torch.tensor(test, dtype=torch.float32)

def fit(lam):
    torch.manual_seed(0)
    W = nn.Linear(64, 16)                    # encoder h = sigmoid(Wx + b)
    dec = nn.Linear(16, 64)
    opt = torch.optim.Adam([*W.parameters(), *dec.parameters()], lr=3e-3)
    for step in range(1500):
        x = X[torch.randint(len(X), (64,))]
        h = torch.sigmoid(W(x))
        rec = (dec(h) - x).square().sum(1).mean()
        # ||J||_F^2 = sum_j (h_j(1-h_j))^2 * ||W_j||^2 for a sigmoid layer
        jac = ((h*(1-h))**2 * W.weight.square().sum(1)).sum(1).mean()
        loss = rec + lam*jac
        opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        h = torch.sigmoid(W(Xt))
        mse = (dec(h) - Xt).square().mean().item()
        jac = ((h*(1-h))**2 * W.weight.square().sum(1)).sum(1).mean().item()
    return mse, jac

for lam in (0.0, 0.5):
    mse, jac = fit(lam)
    print(f"lambda={lam}: test MSE={mse:.4f}, mean ||J||_F^2={jac:.2f}")
```

輸出：

```
lambda=0.0: test MSE=0.0141, mean ||J||_F^2=4.79
lambda=0.5: test MSE=0.0194, mean ||J||_F^2=0.70
```

懲罰項把編碼器的平均敏感度降低了將近七倍，代價是重建誤差略為上升。這筆交易值不值得，取決於下游用途：較平滑的編碼通常有助於最近鄰檢索或少標籤分類，而壓縮編解碼器則會偏好未加懲罰的模型。

流形觀點

白話版。真實資料並不會填滿它們所在的高維空間。它們聚集在一個又薄又彎曲的曲面附近，就像一張捲起來放在房間裡的紙。好的表示描述的是你在這張紙上的位置。

流形假說（manifold hypothesis）認為，影像等自然資料集中在嵌入高維輸入空間中的低維流形附近 [3]。一個隨機的 64 像素陣列幾乎不可能像一個數字；合理數字的集合是一個極小而有結構的區域。在任何資料點附近，流形局部看起來像一個平面，即**切空間**（tangent space），由少數幾個能讓你保持「真實」的移動方向張成（筆畫變粗、稍微傾斜、小幅平移）。與切空間正交的方向則會離開流形、進入雜訊。

這個觀點為本章所有內容提供了單一的視角：

- **PCA** 擬合一張全域的平面紙。流形接近平坦時有效，彎曲時就失效。
- **PPCA 或 FA 的混合模型**擬合許多張局部平面紙。

- **非線性自編碼器**學到流形的彎曲座標圖：編碼器給出紙上的座標，解碼器再把座標映回房間裡。
- **去噪**教模型把掉出流形的點移回流形上。
- **收縮**懲罰讓編碼器對垂直於流形的方向不敏感，而重建項讓它沿著切方向保持敏感 [1, 第 14 章], [12]。

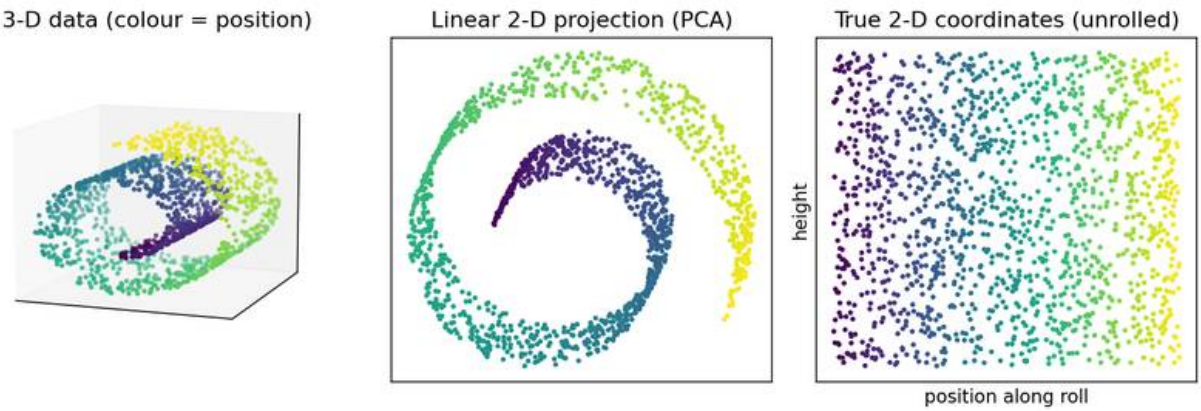


圖 *5* — 瑞士捲是一張捲在三維空間裡的二維紙。線性投影（*PCA*）把它壓成螺旋，無法保留沿紙面的位置；真實的內在座標則是一個平坦的矩形。

圖 5 讓線性方法的限制變得具體。瑞士捲有兩個內在維度——沿捲方向的位置與高度——但沒有任何二維線性投影能把它攤開：PCA 的視圖保留了螺旋，紙面上相距很遠的點並沒有依序排開。非線性編碼器原則上可以學到攤開後的座標。實務上能否做到，取決於容量、正則化與資料量；高維空間中的流形學習至今仍然困難。

從自編碼器到 VAE

白話版。一般的自編碼器學會壓縮與解壓縮，卻沒有提供憑空創造新編碼的規則。VAE 加上了這條規則。

假設訓練好的自編碼器把每個訓練用的數字映射到 $\mathbb{R}^{12}$ 中的一個編碼。為了生成新數字，我們想隨機挑一個編碼再解碼。但要挑哪個隨機編碼？訓練編碼可能占據一個奇怪、成團的區域，而它們之間的隨機點解碼後可能是一團亂。重建損失函數從來沒有要求編碼空間平滑、填滿，或符合

任何已知分布。所以極佳的重建並不代表良好的生成。

變分自編碼器（variational autoencoder）[13] 的解法是回到 PPCA 與 FA 的機率因子模型故事，只是改用神經網路解碼器：

$$z \sim p(z) = \mathcal{N}(0, I), \qquad x \sim p_\theta(x \mid z),$$

其中 $p_\theta(x \mid z)$ 是一個分布，其參數由解碼器網路產生。由於真正的後驗 $p_\theta(z \mid x)$ 此時已無法計算，就由第二個網路——編碼器——輸出近似後驗 $q_\phi(z \mid x)$，通常是對角高斯分布。訓練時最大化 $\log p_\theta(x)$ 的**證據下界**（evidence lower bound，ELBO）：

$$\log p_\theta(x) \;\geq\; \underbrace{\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]}_{\text{reconstruction}} \;-\; \underbrace{D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{keep codes near the prior}}.$$

對照前面所有內容來讀這兩項：

- **重建**項（reconstruction）正是自編碼器的損失，只是現在在編碼中注入了雜訊。高斯解碼器下它變成平方誤差，Bernoulli 解碼器下則是二元交叉熵。
- **KL** 項（讓編碼靠近先驗）是一個正則化項，就像稀疏與收縮懲罰，但它的設計讓所有訓練樣本的編碼合起來填滿先驗分布。這正是從 $z \sim \mathcal{N}(0, I)$ 取樣之所以有意義的原因。
- 若使用**線性**高斯解碼器，這個模型就是 PPCA。它的精確後驗是高斯分布，可以被線性高斯編碼器完全匹配，所以下界變成等號，最佳解與 PPCA 的最大概似解一致。VAE 是我們一開始那些因子模型的非線性推廣。

Kingma 與 Welling 的導論 [14] 詳細發展了這個關聯。把 KL 項乘上 $\beta > 1$ 就得到 **β-VAE** [15]，它的提出是為了以重建品質為代價、鼓勵學到解耦（disentangled）的因子；請注意當 $\beta \neq 1$ 時，目標函數就不再是 ELBO。ELBO 的完整推導、重參數化技巧，以及後驗崩塌（posterior collapse）等失效模式，請見ML 06。

重建誤差能告訴你什麼、不能告訴你什麼

離開自編碼器之前，先給一個實務上的警告。重建誤差常被當作異常分數：用正常資料訓練的自編碼器，應該能把正常輸入重建得很好、把異常輸入重建得很差。這有時行得通，但當自編碼器泛化得太好、連異常輸入也重建得不錯，或異常只存在於瓶頸本來就會丟掉的細節時，就會失敗。同樣地，重建得好的編碼可能不利於分類，而分類效果好的編碼可能丟掉了影像復原任務所需的像素細節。壓縮、去噪、下游效用與生成，請分開評估。

現代觀點

白話版。本章的古典模型並沒有消失，而是變成了零件：生成模型的壓縮階段、視覺的預訓練目標，以及觀察大型網路內部的顯微鏡。

值得一讀的綜述

- **Bengio、Courville 與 Vincent（2013）** [3] 是表示學習的標準回顧。它用一串通用先驗來界定好的表示（平滑性、多重解釋因子、概念的階層、跨任務共享的因子、流形假說、稀疏性等），並在這個框架下串起自編碼器、機率模型與流形學習。想知道本章那些懲罰項為什麼存在，它是很好的地圖。
- **Bank、Koenigstein 與 Giryes（2020）** [16] 是一篇精簡的自編碼器綜述章節。它依序介紹正則化變體（稀疏、去噪、收縮）、VAE 與解耦變體、各種應用（生成建模、分類、分群、異常偵測、推薦系統與降維），以及結合對抗式訓練的混合方法，是本篇很好的補充讀物。
- **Kingma 與 Welling（2019）** [14] 是作者親自撰寫的 VAE 延伸導論，涵蓋 ELBO、重參數化技巧、更豐富的後驗分布與階層模型，並把 VAE 放進其他深度潛在變數模型的脈絡中。
- **Goodfellow、Bengio 與 Courville** [1, 第 13–14 章] 與 **Murphy** [2] 仍是線性因子模型、DAE 與分數的關聯，以及流形詮釋最清楚的教科書說明。

解耦：一個冷靜的結果

一個反覆出現的期望是：非監督式因子模型能找回那組真正的潛在因子，每個編碼單元對應一個（「解耦」，disentanglement）。Locatello 等人 [17] 從理論上證明，若對模型與資料都沒有歸納偏誤，非監督式地學習解耦表示在根本上是不可能的，這在更大的尺度上呼應了因子分析的旋轉不確定性。他們在一個大規模、可重現的研究中也發現：沒有監督就無法辨認出解耦良好的模型，而且更高的解耦程度並不會可靠地降低下游任務的樣本複雜度。實務教訓與 ICA 相同：可識別性永遠來自額外的假設，所以請把假設講清楚。

自編碼器作為 tokenizer

自編碼器在現代最重要的角色，是大型生成模型的第一階段。**VQ-VAE** [18] 把連續編碼換成一本學習出來的離散向量碼本，使影像變成離散 token 的網格，供自迴歸模型預測（見ML 08）。**潛在擴散**（latent diffusion）[19] 先訓練一個做感知壓縮的自編碼器，再在它的潛在空間、而不是像素空間中執行擴散模型，大幅降低運算量。兩者都受到自編碼器重建品質的上限限制：生成器無法還原編碼器已經丟掉的細節。如何設計這些 tokenizer——在壓縮、重建，以及潛在空間是否容易被生成器建模之間取得平衡——在 2026 年仍是活躍的研究領域。

用於可解釋性的稀疏自編碼器（2023–2026）

稀疏自編碼器找到了一個嶄新且非常活躍的應用：大型語言與視覺模型的**機制可解釋性**（mechanistic interpretability）。出發點是一個觀察：單一神經元往往是多義的（polysemantic），會對好幾個不相關的概念產生反應。Cunningham 等人 [20] 訓練過完備的稀疏自編碼器來重建語言模型的內部啟動值，找到一些稀疏啟動的特徵，自動化評估認為它們比神經元或其他分解方式更具可解釋性、更單義。Gao 等人 [21] 使用 k-稀疏（top-k）自編碼器直接控制稀疏程度，報告了自編碼器大小與稀疏程度的乾淨縮放定律，提出評估特徵品質的新指標，並在 GPT-4 的啟動值上訓練了一個有 1,600 萬個潛在單元的自編碼器。概念上，這就是把稀疏編碼（Olshausen 與 Field 的想法）套用到網路的啟動值，而不是影像區塊。

開放問題

- **評估**。重建誤差、稀疏程度與下游準確率彼此不一致，而且對於學到的特徵是否「正確」——無論是解耦還是可解釋性——都沒有公認的衡量方式。
- **可識別性**。什麼時候可以相信學到的因子對應真實的成因？非線性 ICA 與因果表示學習研究的是哪些額外假設（時間結構、輔助變數、介入）能讓這件事成立。
- **Tokenizer 設計**。要壓縮多少、編碼該是離散還是連續、如何讓潛在表示既利於重建又容易建模，仍是開放的工程與理論問題。

重點整理

- 本章所有模型共用一個範本：編碼器映到編碼、解碼器映回來，再加上一個決定偏好哪種編碼的假設。
- PCA 最小化線性重建誤差；其方向是共變異數的特徵向量，可由中心化資料的 SVD 求得。
- PPCA 加入等向高斯雜訊，並以最大概似找回 PCA 子空間；因子分析允許每個座標有各自的雜訊水準。兩者都能確定子空間，但無法確定座標軸。
- ICA 假設來源獨立且非高斯，藉此確定座標軸（只剩順序、正負號與尺度的不確定性）；稀疏編碼與 NMF 則以稀疏性與非負性挑選因子。
- 線性自編碼器找回的是 PCA 子空間，而不是唯一的 PCA 權重。非線性自編碼器需要瓶頸及／或懲罰（稀疏、去噪、收縮）才能學到有用的結構。
- 流形觀點把這一切統一起來：好的編碼沿著資料的低維曲面移動，並忽略離開曲面的方向。
- 重建本身不提供取樣器。VAE 加入先驗與近似後驗；若解碼器是線性的，它就退化為 PPCA。

練習

1. **雜訊假設**。你記錄了 20 個感測器，其中 5 個的雜訊遠大於其他感測器，但它們全都反映同樣幾個潛在成因。你會從 PPCA 還是因子分析開始？PCA 會怎麼處理那些雜訊大的感測器？

   提示

   選因子分析：它的對角 Ψ 可以把較大的殘差變異數分配給那 5 個雜訊大的感測器，而不必為它們花費因子。PCA（以及只有單一共享 σ^2 的 PPCA）可能會把成分花在高變異數的雜訊感測器上，因為 PCA 眼中只有變異數。

2. **潛在不確定性**。對可逆的 $k \times k$ 矩陣 G，證明把 W_e 換成 GW_e、W_d 換成 $W_d G^{-1}$，線性自編碼器的重建結果不變。哪些矩陣 G 也能保持 PPCA 的概似函數不變？

   提示

   乘積為 $W_d G^{-1} G W_e = W_d W_e$。對 PPCA，概似函數取決於 $WW^\top$；把 W 換成 WG 得到 $WGG^\top W^\top$，當 $GG^\top = I$，也就是 G 為正交矩陣時不變。

3. **為什麼 ICA 對高斯來源失效**。產生兩個獨立的標準高斯來源，用任意可逆的 2×2 矩陣混合，再用不同隨機種子執行 FastICA 數次。你觀察到什麼？為什麼？

提示

白化之後，兩個獨立單位高斯的任何旋轉仍是兩個獨立單位高斯，所以沒有「正確」的旋轉可找。不同種子會給出不同但同樣合理的解混方向，與真實來源的相關係數每次都不一樣。

4. **稀疏編碼與 PCA 編碼**。使用稀疏編碼的程式，比較每張測試影像的非零編碼元素數量，與相同維度（32）的 PCA 編碼。接著把 `alpha` 設為 0.05、0.1、0.3、1.0，畫出稀疏編碼的測試 MSE。

提示

PCA 編碼是稠密的：32 個元素基本上全都非零。隨著 `alpha` 增大，啟動的原子數下降、誤差上升，畫出一條稀疏程度與精確度的取捨曲線。記得字典沒有平均值項，所以請比較曲線，而不是拿單一數字與 PCA 比。

5. **去噪的不匹配**。用高斯雜訊訓練本篇的 DAE，再拿隨機一個 3×3 區塊被設為零的輸入來測試。接著用這種遮罩破壞訓練第二個 DAE，並把兩個模型在兩種破壞下都測一遍。

提示

這是兩種不同的破壞過程。用高斯雜訊訓練的模型從沒被要求補齊缺失的結構，所以通常會留下空洞；用遮罩訓練的模型則學會補全。請列出一張 2×2 的表（訓練破壞 × 測試破壞），而不是從其中一種的成功推論出對另一種的穩健性。

6. **手算收縮懲罰**。對單一 sigmoid 單元 $h = \sigma(w^\top x + b)$，推導 $\|\partial h / \partial x\|^2$。它何時最大？這說明收縮自編碼器允許編碼在什麼地方保持敏感？

提示

$\partial h / \partial x = h(1-h)w$，所以平方範數為 $h^2(1-h)^2\|w\|^2$。它在 $h = 1/2$（sigmoid 中段最陡處）最大，單元飽和時則幾乎為零。這個懲罰把單元推向對大多數輸入都飽和的狀態，只在資料確實需要變化的地方留在敏感區間。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 13–14. Book
2. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book
3. Y. Bengio, A. Courville and P. Vincent, “Representation Learning: A Review and New Perspectives,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013. Paper
4. M. E. Tipping and C. M. Bishop, “Probabilistic Principal Component Analysis,” *Journal of the Royal Statistical Society: Series B*, vol. 61, no. 3, 1999. Paper
5. A. Hyvärinen and E. Oja, “Independent component analysis: algorithms and applications,” *Neural Networks*, vol. 13, no. 4–5, 2000. Paper

6. B. A. Olshausen and D. J. Field, “Emergence of simple-cell receptive field properties by learning a sparse code for natural images,” *Nature*, vol. 381, 1996. Paper
7. D. D. Lee and H. S. Seung, “Learning the parts of objects by non-negative matrix factorization,” *Nature*, vol. 401, 1999. Paper
8. P. Baldi and K. Hornik, “Neural networks and principal component analysis: Learning from examples without local minima,” *Neural Networks*, vol. 2, no. 1, 1989. Paper
9. G. E. Hinton and R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks,” *Science*, vol. 313, 2006. Paper
10. P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” ICML, 2008. Paper
11. P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio and P.-A. Manzagol, “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” *JMLR*, vol. 11, 2010. Paper
12. S. Rifai, P. Vincent, X. Muller, X. Glorot and Y. Bengio, “Contractive Auto-Encoders: Explicit Invariance During Feature Extraction,” ICML, 2011. Paper
13. D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014. Paper
14. D. P. Kingma and M. Welling, “An Introduction to Variational Autoencoders,” *Foundations and Trends in Machine Learning*, vol. 12, no. 4, 2019. Paper
15. I. Higgins et al., “β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework,” ICLR, 2017. Paper
16. D. Bank, N. Koenigstein and R. Giryes, “Autoencoders,” survey chapter, arXiv:2003.05991, 2020. Survey
17. F. Locatello et al., “Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations,” ICML, 2019. Paper
18. A. van den Oord, O. Vinyals and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017. Paper
19. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” CVPR, 2022. Paper
20. H. Cunningham, A. Ewart, L. Riggs, R. Huben and L. Sharkey, “Sparse Autoencoders Find Highly Interpretable Features in Language Models,” arXiv:2309.08600, 2023. Paper
21. L. Gao et al., “Scaling and evaluating sparse autoencoders,” arXiv:2406.04093, 2024. Paper

ML 06・生成模型：自編碼器、VAE、GAN 與正規化流

從學習標籤走向學習分布：概似式與隱式模型、ELBO 與重參數化、對抗訓練與 Wasserstein GAN、正規化流，以及生成模型該如何評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-06-generative-models/

你將學到

- 建模一個分布 $p(x)$ 和預測標籤有什麼不同，以及為什麼最大概似等同於最小化 KL 散度。
- 兩大家族：寫得出（或能界定）$\log p(x)$ 的**概似式模型（likelihood-based）**，以及只提供取樣器的**隱式模型（implicit）**。
- VAE 如何把無法計算的積分變成可訓練的下界（ELBO），以及重參數化技巧為什麼讓它可以微分。
- GAN 如何透過判別器學習、原始目標函數其實在最小化什麼、模式崩塌（mode collapse）為何發生，以及 Wasserstein GAN 改變了什麼。
- 正規化流如何用變數變換公式算出精確概似，以及能量模型、自迴歸模型與擴散模型在地圖上的位置。
- 如何用概似、Inception Score、FID 與精確率／召回率評估生成器，以及每個指標**無法**告訴你什麼。

先看全貌

分類器問：「這張影像裡有什麼？」**生成模型（generative model）**問：「可能出現哪些影像？」如果它學到有用的分布，我們就能取樣出新的例子、補上缺失區域、壓縮資料、偵測異常輸入，或依文字、類別標籤、另一張影像來條件式地生成。

> **用最簡單的話說**
>
> 會認房子的小朋友，看到圖可以說「房子」。懂得房子怎麼畫的小朋友，則能自己畫出一棟有門、有窗、有屋頂的新房子。生成模型想學的是例子背後的「食譜」，好煮出新的菜——而且好廚師每次端出來的菜都不一樣，但都好吃，而不是同一道菜一直重複。

學分布比學標籤難得多。標籤只是一個數字；一張影像有成千上萬個像素，彼此以複雜的方式相互依賴。本章的每個家族，都是對同一個工程問題的不同回答：要怎麼表示一個極高維資料上的分布，才能既從例子**訓練**它，之後又能從它**取樣**？

本章採用 Bishop 與 Bishop [1] 以及 Prince [2] 發展的機率觀點。Bond-Taylor 等人的比較綜述 [3] 依照我們將遇到的取捨來整理這些家族——訓練速度、樣本品質、多樣性，以及是否能得到概似；Papamakarios 等人的正規化流綜述 [4] 則特別有助於區分「學到的分布」與「方便的取樣程序」。擴散模型與自迴歸模型各有專章（ML 07 與ML 08），因為它們的訓練與取樣值得仔細討論；這裡先把它們放上地圖。

用分布描述資料

白話版。找一個機率模型，讓觀察到的例子在它之下看起來「很可能」，再用這個模型產生看起來像出自同一來源的新例子。

最大概似

假設資料 $x_1, \dots, x_n$ 是從未知分布 p_{data} 獨立抽出的樣本，p_θ 是參數為 θ 的模型。最大概似估計最小化平均**負對數概似（negative log-likelihood，NLL）**：

$$\mathcal{L}_{\text{NLL}}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta(x_i).$$

其中 n 是訓練樣本數，$\log p_\theta(x_i)$ 是模型給第 i 個樣本的對數密度。

為什麼這是合理的目標？當 n 變大，平均值會趨近於對資料分布的期望值，而且

$$\mathbb{E}_{x\sim p_{\text{data}}}[-\log p_\theta(x)] = D_{\text{KL}}(p_{\text{data}} \,\|\, p_\theta) + H(p_{\text{data}}).$$

$D_{\text{KL}}(p\|q) = \mathbb{E}_p[\log p - \log q]$ 是 Kullback–Leibler 散度，只有兩個分布相同時才為零；$H(p_{\text{data}})$ 是資料的熵，與 θ 無關。因此**最大概似就等於最小化從資料到模型的前向 KL 散度**。

KL 的方向很重要。前向 KL 會重罰「在真實資料所在之處給出接近零的密度」的模型，所以概似訓練是「涵蓋質量（mass-covering）」的：它寧可把機率攤到每一個模態（mode）上，即使因此在模態之間也放了一些機率。請記住這點；稍後會看到 GAN 的行為不同。

密度、離散像素與每維位元數

對連續值而言，$p_\theta(x)$ 是**密度**，不是「恰好觀察到 x」的機率；密度可以大於 1。然而真實影像存的是 0–255 這類整數。直接用連續密度去擬合整數是病態問題：模型可以在整數格點上放無限高的尖峰，讓概似趨於無窮大。標準解法是**去量化（dequantization）**——在擬合連續密度之前，對每個整數值加上 $[0, 1)$ 的均勻雜訊。Theis 等人 [5] 證明，連續模型在去量化資料上的對數概似，會是對應離散模型對數概似的下界，這讓不同論文的數字可以相互比較。另一種作法是直接使用離散概似（對 256 個值做 softmax，或像下方程式碼一樣對二值像素用 Bernoulli 分布）。

影像的概似通常以**每維位元數（bits per dimension）**回報：

$$\text{bpd} = \frac{\mathcal{L}_{\text{NLL}}}{D \ln 2},$$

其中 $\mathcal{L}_{\text{NLL}}$ 以每張影像的 nat 為單位，D 是像素值的個數（高 × 寬 × 通道數）。每維位元數就是「以此模型為基礎的無損壓縮器」平均每個子像素需要的位元數，因此概似與壓縮是一體兩面。

條件式與無條件生成

判別式分類器建模 $p(y \mid x)$。條件式生成器建模 $p(x \mid c)$，其中 c 可以是文字、類別、低解析度影像或已觀察到的像素。貝氏定理把相關的量連在一起，但分類器本身並不提供影像取樣器。

條件化也解釋了一個常見的失敗。最小化像素 MSE 會預測**條件平均** $\mathbb{E}[x \mid c]$。如果一個模糊的輸入有好幾個合理輸出——例如一張灰階照片裡的襯衫可能是紅色也可能是藍色——平均值會把它們混成褐色。生成模型則嘗試表示整個分布 $p(x \mid c)$，一次取樣出一個合理的答案。

兩大家族：概似式與隱式

以下每個模型都需要兩種能力：從資料**訓練**的方法，以及**取樣**的方法。它們的差別在於能不能計算 $p_\theta(x)$。

- **概似式（顯式）模型**寫下密度或其界限，再加以最大化。自迴歸模型與正規化流給出精確概似；VAE（以及從變分角度看的擴散模型）最佳化一個下界；能量模型只定義到一個未知正規化常數為止的密度。
- **隱式模型**只定義一個取樣器——「把雜訊推過一個網路」——而沒有 $p_\theta(x)$ 的公式。GAN 是主要例子。因為沒有概似可以最大化，它們必須用別的方式比較樣本與資料來訓練。

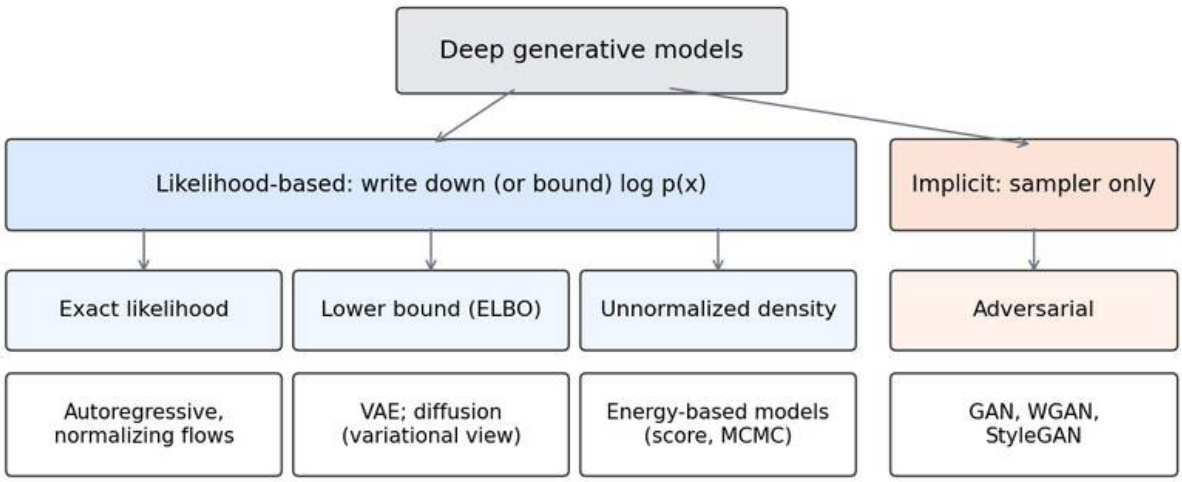


圖 *1* — 生成模型的教學用分類，採用比較綜述 [3] 的顯式／隱式劃分。實際系統常結合多個分支。

一般自編碼器：有壓縮、沒有先驗

白話版。把一個例子編碼成精簡的潛在碼，再解碼回來。這對壓縮與表示學習非常好，但它本身並不告訴你怎麼「發明」新的潛在碼。

編碼器 $z = f_\phi(x)$ 與解碼器 $\hat{x} = g_\theta(z)$ 最小化 $\|x - \hat{x}\|^2$ 之類的重建損失。自編碼器、它們的線性親戚（PCA、因子分析）及各種正則化變體，在ML 05A 中有完整介紹。本章要強調的只有一點：一般自編碼器並不要求潛在碼**落在哪裡**。訓練資料的潛在碼可能只佔據潛在空間中一塊又薄又怪的區域。若你接著取樣 $z \sim \mathcal{N}(0, I)$ 再解碼，可能落在解碼器從沒見過的空白區域，得到一團垃圾。因此重建再好，也不保證無條件生成的品質。

VAE 透過「給潛在碼一個先驗分布」，把自編碼器變成機率模型來解決這個問題。

VAE：潛在變數機率模型

白話版。想像每張影像的產生過程是：先從一個簡單分布隨機挑幾個隱藏的「設定」（姿態、筆畫粗細、風格），再根據這些設定加上一點雜訊畫出影像。VAE 同時學習這個「繪製器」（解碼器），以及一個快速的「猜測器」（編碼器）：讀一張影像，提出它大概是由哪些設定產生的。

模型

選一個潛在向量 $z \in \mathbb{R}^k$ 的先驗 $p(z)$，通常是標準高斯 $\mathcal{N}(0, I)$，以及解碼器 $p_\theta(x \mid z)$——一個輸出影像分布參數的神經網路（二值像素用 Bernoulli 機率，或高斯平均值）。模型對影像的密度是邊際分布

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz.$$

取樣很容易：先抽 $z \sim p(z)$，再抽 $x \sim p_\theta(x \mid z)$。計算 $p_\theta(x)$ 則很難：積分要跑過所有可能的潛在碼，而解碼器是非線性網路，沒有封閉解。真正的後驗 $p_\theta(z \mid x) = p_\theta(x \mid z)p(z)/p_\theta(x)$ 也因此無法計算。

推導證據下界

Kingma 與 Welling [6] 引入一個近似後驗的**編碼器**（或稱推論網路）$q_\phi(z \mid x)$。以下逐步推導。因為 $\log p_\theta(x)$ 與 z 無關，我們可以對任何 z 的分布取期望而不改變它的值：

$$\begin{aligned}
\log p_\theta(x) &= \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x)\big] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x, z)}{p_\theta(z \mid x)}\right] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x, z)}{q_\phi(z \mid x)}\right] + \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\right] \\
&= \mathrm{ELBO}(x) + D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)).
\end{aligned}$$

第二行用了 $p_\theta(x) = p_\theta(x, z)/p_\theta(z \mid x)$；第三行在對數裡同乘、同除 $q_\phi(z \mid x)$。因為 KL 散度永遠不為負，第一項是 $\log p_\theta(x)$ 的下界：**證據下界（evidence lower bound，ELBO）**。下界與真實對數概似之間的**差距**，恰好是近似後驗與真實後驗之間的 KL 散度。編碼器越好，下界越緊；完美的編碼器會讓差距歸零。

把 $p_\theta(x,z) = p_\theta(x \mid z)p(z)$ 展開，得到程式中使用的形式：

$$\mathrm{ELBO}(x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z))}_{\text{stay close to the prior}}.$$

第一項（重建）獎勵能讓解碼器重現 x 的潛在碼；第二項（貼近先驗）懲罰分布偏離先驗太遠的潛在碼。用 Jensen 不等式（$\log \mathbb{E} \geq \mathbb{E} \log$）一行就能得到同樣的下界，但上面的分解更有用，因為它明確指出了差距是什麼。訓練時對 θ 與 ϕ 同時最大化資料上的平均 ELBO，也就是最小化**負 ELBO**。

當編碼器是對角高斯 $q_\phi = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$、先驗是標準高斯時，KL 項有封閉解：

$$D_{\mathrm{KL}}(q_\phi \| p) = \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right),$$

其中 μ_j 與 σ_j^2 是編碼器在第 j 個潛在座標上的平均值與變異數。每一項只有在 $\mu_j = 0$ 且 $\sigma_j^2 = 1$ 時才為零。

重參數化

重建項是對 $z \sim q_\phi(z \mid x)$ 的期望值，而我們需要它對 ϕ 的梯度。照字面寫的取樣無法微分：隨機抽樣是個黑盒子。**重參數化技巧（reparameterization trick）** [6] 把隨機性移到一個固定的雜訊變數上：

$$\epsilon \sim \mathcal{N}(0, I), \qquad z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon.$$

$\odot$ 是逐元素相乘。現在 z 是 ϕ 與獨立雜訊 ϵ 的確定性、可微函數，一般的反向傳播就能透過 μ 與 σ 給出低變異的梯度估計。程式中編碼器預測 `logvar` $= \log \sigma^2$，再用 `exp(.5 * logvar)` 當標準差，不需任何限制就能保證為正。

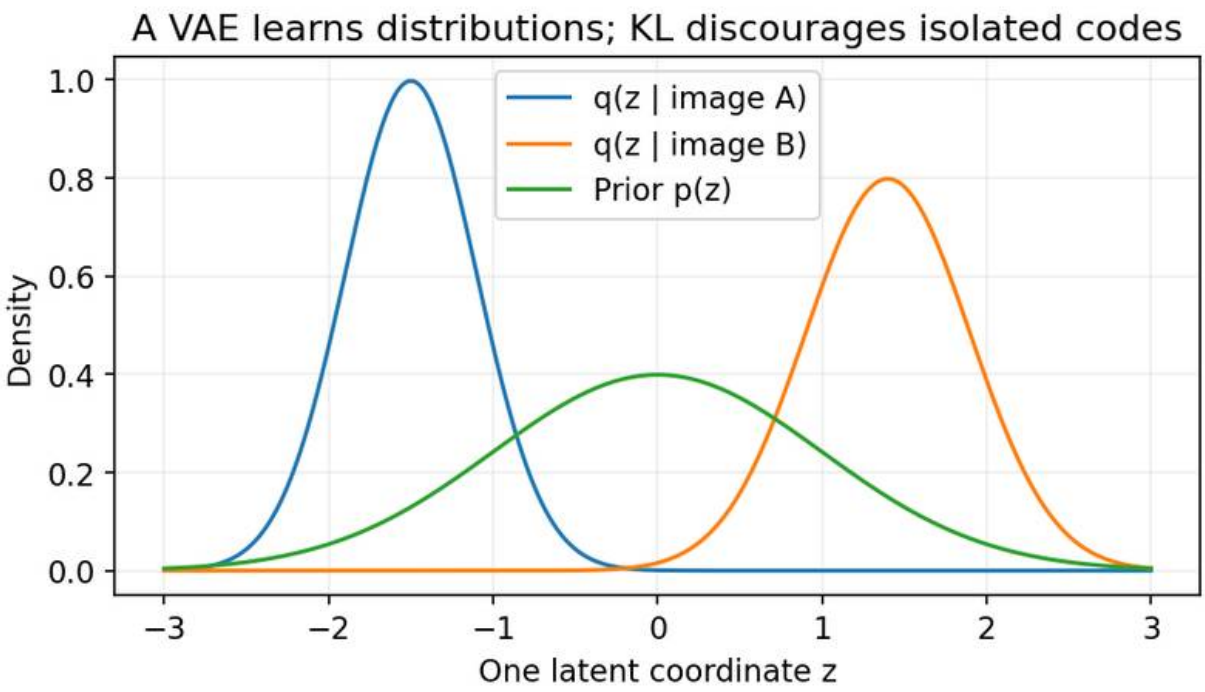


圖 2 — *VAE* 以分布表示潛在碼的不確定性，並鼓勵它與共用的先驗相容。曲線為示意。

程式：二值手寫數字的小型 VAE

這個完整的 CPU 範例使用 scikit-learn 內建的 8 × 8 手寫數字，並先二值化，讓 Bernoulli 解碼器成為明確的概似模型。重建項對像素加總、KL 對潛在維度加總，然後兩者都對樣本取平均；若只偷偷對其中一項取平均，就會改變兩者的比重。

```
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
d = load_digits()
train, test = train_test_split(d.data, test_size=.2, random_state=0,
                               stratify=d.target)
X = torch.tensor((train > 8).astype("float32"))
Xt = torch.tensor((test > 8).astype("float32"))
enc = nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 16))
dec = nn.Sequential(nn.Linear(8, 64), nn.ReLU(), nn.Linear(64, 64))
opt = torch.optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=.002)

def terms(x):
```

```
    mu, logvar = enc(x).chunk(2, dim=-1)
    z = mu + (.5*logvar).exp() * torch.randn_like(mu)
    recon = F.binary_cross_entropy_with_logits(dec(z), x, reduction="none").sum(1)
    kl = .5*(mu.square() + logvar.exp() - logvar - 1).sum(1)
    return recon, kl

for step in range(400):
    x = X[torch.randint(len(X), (64,))]
    recon, kl = terms(x)
    loss = (recon + kl).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
enc.eval(); dec.eval()
with torch.no_grad():
    recon, kl = terms(Xt)
    samples = dec(torch.randn(4, 8)).sigmoid().reshape(4, 8, 8)
    print("test negative ELBO estimate:", round((recon + kl).mean().item(), 3))
    print("reconstruction / KL:", round(recon.mean().item(), 3),
          round(kl.mean().item(), 3))
    print("sample probability maps:", tuple(samples.shape))
```

在我們的機器上，它印出測試集負 ELBO 約為每張影像 20.5 nats，其中重建約 17.0 nats、KL 約 3.5 nats。因為 ELBO 是對數概似的下界，20.5 nats 就是測試 NLL 的**上界**。輸出是 Bernoulli 機率圖；若要得到二值樣本，就依這些機率對每個像素取樣。測試估計帶有隨機性，多取幾次潛在樣本再平均會更穩定。這裡沒有用測試集挑選檢查點。

圖 3 顯示同樣資料上、潛在空間只有**兩維**的 VAE（由圖檔腳本訓練更久）學到了什麼。即使從沒用到標籤，相同類別的數字仍落在彼此附近；對一個規則網格上的 z 解碼，數字形狀會平滑地漸變。這正是一般自編碼器缺少的性質：先驗主體附近的每一點，解碼出來都像個數字。

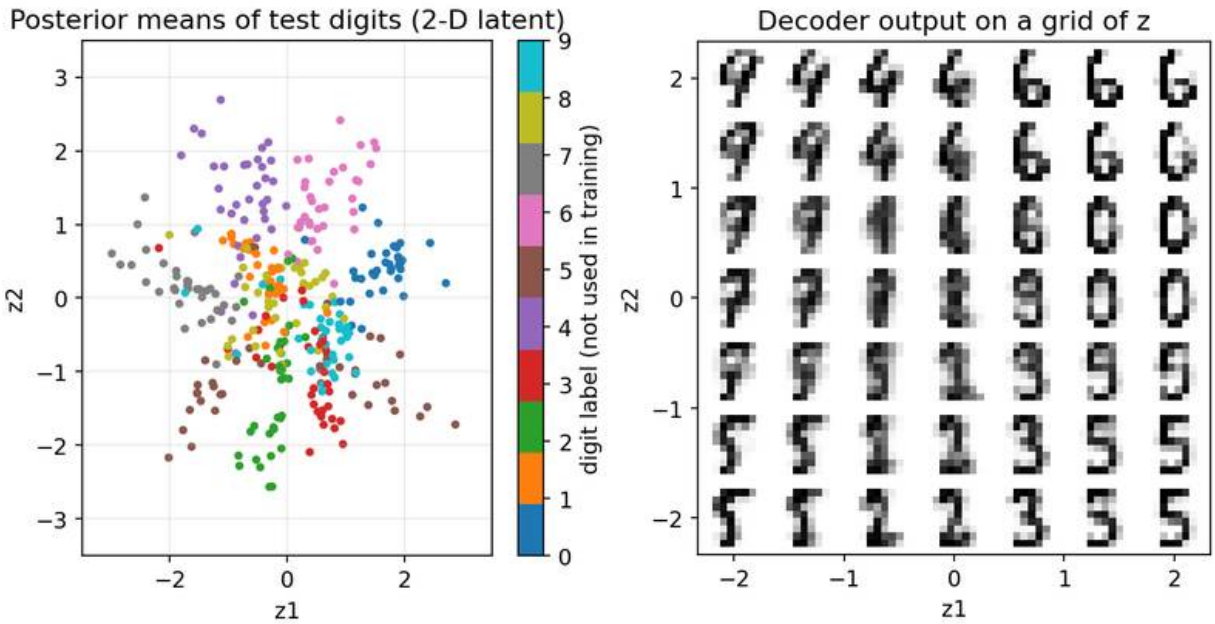


圖 3 — 在二值化 8×8 數字上訓練、潛在空間為 2 維的 *VAE*。左：測試影像的編碼器平均值（標籤只用於著色）。右：在潛在值網格上的解碼器輸出。

後驗崩塌、β-VAE 與更深的 VAE

如果解碼器非常強大（例如本身就能建模像素的自迴歸解碼器），提高 ELBO 最便宜的方式可能是完全忽略 z：令 $q_\phi(z \mid x) = p(z)$ 使 KL 項為零，讓解碼器包辦一切。這就是**後驗崩塌（posterior collapse）**。Bowman 等人 [7] 在文字 VAE 上觀察到這個現象，並提出 **KL 退火（KL annealing）**：一開始給 KL 項很小的權重，訓練中逐步升到 1，讓模型在懲罰生效前先學會使用潛在碼。監看每個潛在維度的 KL 是最簡單的診斷——KL 接近零的維度不攜帶任何資訊。

Higgins 等人 [8] 則反其道而行：**β-VAE** 把 KL 項乘上 $\beta > 1$，以犧牲重建為代價，迫使模型學出更分解、更「解纏（disentangled）」的潛在碼。要精確理解這件事：當 $\beta \neq 1$，目標函數就不再是 ELBO，它的值也不再是 $\log p_\theta(x)$ 的下界；Kingma 與 Welling 的教學綜述 [9] 討論了這點以及許多其他變體。最後，單層、因子化高斯的 VAE 容易產生模糊影像，部分原因是高斯解碼器會對不確定性取平均。**階層式 VAE** 疊了好幾層潛在變數；NVAE [10] 顯示，經過精心設計、搭配穩定化正則化的深層階層，可以在高達 256×256 像素的自然影像上產生清晰樣本。

GAN：與對手一起學習

白話版。兩個網路在玩遊戲。**生成器（generator）**把隨機雜訊變成假的例子；**判別器（discriminator）**看一個例子並猜它是真是假。生成器的勝利條件是騙過判別器，這逼得它的輸出越來越像真實資料。

極小極大遊戲

令 G 把雜訊 $z \sim p(z)$ 映到樣本 $G(z)$，其分布記為 p_g。令 $D(x) \in (0,1)$ 是判別器估計 x 為真的機率。Goodfellow 等人 [11] 的原始 GAN 目標是

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))].$$

判別器最大化 V：這就是一般的二元交叉熵，真資料標 1、假資料標 0。生成器最小化 V：它希望 $D(G(z))$ 越大越好。

這個遊戲在最小化什麼

固定生成器，把 V 寫成對 x 的積分：$V = \int [p_{\text{data}}(x)\log D(x) + p_g(x)\log(1 - D(x))]dx$。對每個 x 分別來看，當 $a, b > 0$，函數 $a \log D + b \log(1 - D)$ 在 $D = a/(a+b)$ 有最大值（令導數 $a/D - b/(1-D)$ 為零即可）。所以**最佳判別器**是

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

它就是「若兩者事前機率相同，一個點來自資料而非生成器」的機率。把 D^* 代回並加減 $\log 4$，得到

$$V(D^*, G) = -\log 4 + 2\,\mathrm{JSD}(p_{\text{data}} \,\|\, p_g),$$

其中 $\mathrm{JSD}(p\|q) = \frac{1}{2}D_{\mathrm{KL}}(p\|m) + \frac{1}{2}D_{\mathrm{KL}}(q\|m)$、$m = \frac{1}{2}(p+q)$，稱為 **Jensen–Shannon 散度**。因此在完美判別器之下，生成器是在最小化資料與模型間的一個對稱散度，全域最佳解是 $p_g = p_{\text{data}}$，此時處處 $D^* = 1/2$ [11]。注意整個過程從沒計算過概似：判別器扮演一個學出來、會自我調整的「這看起來像資料嗎？」檢定。

非飽和損失與實作方式

訓練初期生成器很差，$D(G(z)) \approx 0$，$\log(1 - D(G(z)))$ 幾乎是平的，生成器的梯度消失。實務上生成器改為最小化 $-\mathbb{E}\log D(G(z))$，也就是同一篇論文 [11] 提出的**非飽和（non-saturating）**損失：正好在生成器落後時給出強梯度。它的固定點相同，但動態不同。

兩者交替更新。更新 D 時要把生成樣本 **detach**，讓梯度不會流到 G；更新 G 時讓梯度穿過 D 流進 G，但只執行 G 的最佳化器。用 logits 搭配 `binary_cross_entropy_with_logits`，在數值上比先套 sigmoid 再取 log 更安全。

程式：在八個高斯上訓練 GAN

測試模態涵蓋的經典場景，是二維平面上排成一圈的八個緊密高斯群。它小到能在 CPU 上一分鐘內訓練完，而且你可以**親眼看到**每個模態有沒有被涵蓋。

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
angles = torch.arange(8) * (2 * math.pi / 8)
centers = 2.0 * torch.stack([angles.cos(), angles.sin()], dim=1)

def sample_data(n):
    idx = torch.randint(8, (n,))
    return centers[idx] + 0.05 * torch.randn(n, 2)

def mlp(i, o):
    return nn.Sequential(nn.Linear(i, 128), nn.ReLU(),
                         nn.Linear(128, 128), nn.ReLU(), nn.Linear(128, o))

G, D = mlp(8, 2), mlp(2, 1)
```

```
opt_g = torch.optim.Adam(G.parameters(), lr=1e-3, betas=(.5, .999))
opt_d = torch.optim.Adam(D.parameters(), lr=1e-3, betas=(.5, .999))
for step in range(3000):
    real = sample_data(256)
    fake = G(torch.randn(256, 8))
    # Discriminator: logits, real -> 1, fake -> 0; detach so G is untouched.
    d_loss = (F.binary_cross_entropy_with_logits(D(real), torch.ones(256, 1))
              + F.binary_cross_entropy_with_logits(D(fake.detach()),
                                                   torch.zeros(256, 1)))
    opt_d.zero_grad(); d_loss.backward(); opt_d.step()
    # Generator: non-saturating loss -log D(G(z)).
    g_loss = F.binary_cross_entropy_with_logits(D(fake), torch.ones(256, 1))
    opt_g.zero_grad(); g_loss.backward(); opt_g.step()

with torch.no_grad():
    x = G(torch.randn(5000, 8))
    dist = torch.cdist(x, centers)
    nearest = dist.min(1)
    good = nearest.values < 0.15          # within 3 std of some mode
    counts = torch.bincount(nearest.indices[good], minlength=8)
print("fraction near a mode:", round(good.float().mean().item(), 3))
print("modes with >= 1% of samples:", int((counts >= 50).sum()))
print("samples per mode:", counts.tolist())
```

輸出為：

```
fraction near a mode: 0.896
modes with >= 1% of samples: 7
samples per mode: [590, 14, 710, 276, 651, 731, 863, 646]
```

約 90% 的樣本落在某個模態上，但 45° 方向的模態在 5,000 個樣本中只分到 14 個，另一個模態也明顯偏少。這就是當場抓到的**部分模式崩塌**。圖 4 呈現生成器的演變：早期是一團沒有結構的點雲，接著是一個涵蓋所有模態、但也涵蓋模態之間空隙的圓環，最後是更銳利的群聚，卻幾乎放棄了一個模態。精確數字會隨亂數種子與套件版本改變，但這個定性現象很典型。

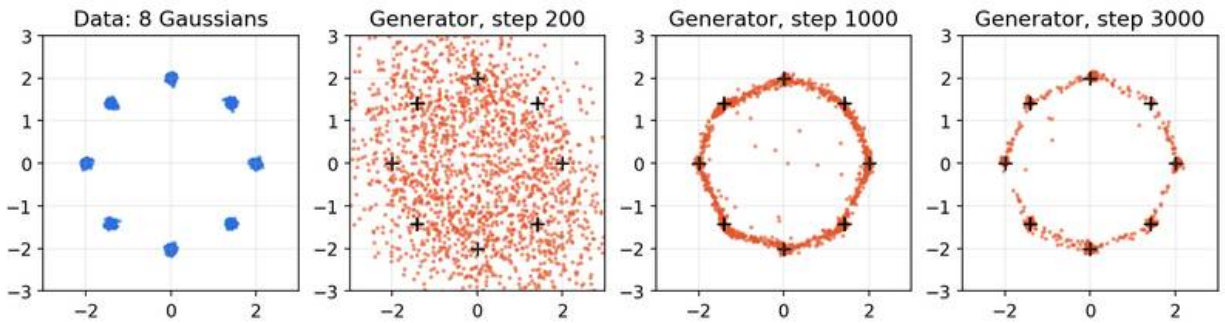


圖 *4* — 小型 *GAN* 學習八個高斯組成的圓環（十字標記真實模態）。到第 *3000* 步時樣本很清晰，但有一個模態幾乎被丟掉：清晰度與涵蓋度是兩回事。

模式崩塌及其成因

當生成器產生的輸出種類太少，就發生了**模式崩塌**。生成器得到的獎勵來自「目前的判別器接受的樣本」，而不是多樣性。如果它找到一塊判別器喜歡的區域，就可以把樣本都堆在那裡；判別器接著學會拒絕那塊區域，生成器再跳到另一塊，雙方可能互相追逐而不收斂。與最大概似不同，生成器的逐樣本損失中，沒有任何一項會直接懲罰**漏掉**某個模態。Salimans 等人 [12] 提出了特徵匹配（feature matching）與小批次判別（minibatch discrimination）等早期對策，讓判別器看整批樣本的統計量，於是多樣性不足就能被察覺。

所以一組清晰、逼真的影像，仍可能只涵蓋資料分布的一小部分。品質與涵蓋度一定要分開量測——我們會在評估一節回到這點。

Wasserstein GAN

當 p_{data} 與 p_g 分布在又薄、又互不重疊的集合上（影像在像素空間中正是如此），JSD 會卡在最大值 $\log 2$，無法提供「兩者**相距多遠**」的有用梯度。Arjovsky 等人 [13] 改用 **Wasserstein-1（推土機）距離**：把一個分布的機率質量搬到另一個分布的最小成本。根據 Kantorovich–Rubinstein 定理，它的對偶形式是

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \le 1} \; \mathbb{E}_{x\sim p_{\text{data}}}[f(x)] - \mathbb{E}_{x\sim p_g}[f(x)],$$

上確界取遍所有 1-Lipschitz 函數 f（輸入從 x' 變到 x 時，輸出變化不超過 $\|x - x'\|$）。網路 f 這時改稱為**評論器（critic）**：它輸出無上下界的分數，而不是機率。即使兩個支撐集不重疊，生成器樣本越靠近資料，Wasserstein 距離也會平滑地縮小。

難處在於如何落實 Lipschitz 限制。原始 WGAN 把評論器的每個權重裁剪到一個小區間，作法粗糙，可能讓評論器過於簡單。**WGAN-GP** [14] 改為加上梯度懲罰

$$\lambda\, \mathbb{E}_{\hat{x}}\big[(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2\big],$$

其中 $\hat{x}$ 是在真實樣本與生成樣本配對之間的直線上均勻抽取，λ 是權重（論文使用 10）。另外兩個廣泛使用的穩定技巧直接作用在判別器上：**頻譜正規化（spectral normalization）** [15] 把每個權重矩陣除以它最大的奇異值，逐層限制 Lipschitz 常數；Mescheder 等人 [16] 分析的 **R1 懲罰**則在真實資料上懲罰 $\frac{\gamma}{2}\|\nabla_x D(x)\|^2$，他們證明這能帶來局部收斂，而未正則化的 GAN 訓練可能會震盪。

架構，以及 GAN 現在用在哪裡

StyleGAN [17] 說明架構與損失同樣重要：它把雜訊映到一個中介潛在空間，再把這個「風格」注入每一個解析度，得到可控制、高品質的人臉合成。GAN 只需一次生成器前向傳遞就能取樣，推論很快。對抗損失也常作為其他系統的**元件**出現——例如在影像 tokenizer 中用來重建清晰細節。但這不代表每個使用這種 tokenizer 的模型都是 GAN 生成器；分布可能完全由別的東西建模。

正規化流：可逆轉換

白話版。從簡單的雜訊（例如標準高斯）開始，把它推過一連串可逆轉換。因為每一步都能還原，任何影像都可以倒推回「會產生它的那個雜訊」——同時精確記錄每一步把空間拉伸或擠壓了多少，於是得到精確的概似。

變數變換

令 $z \sim p_0(z)$ 是簡單的基底分布，$x = f_\theta(z)$ 是同維度、可逆且可微的映射。則

$$\log p_\theta(x) = \log p_0\big(f_\theta^{-1}(x)\big) - \log \left| \det \frac{\partial f_\theta}{\partial z} \right|_{z = f_\theta^{-1}(x)}.$$

$\partial f_\theta / \partial z$ 是映射的 Jacobian 矩陣，它的行列式衡量映射把 z 附近的小體積放大了多少。機率質量守恆，所以若一塊區域被拉伸成兩倍體積，它的密度就必須減半。在一維中，$x = 2z$ 讓每個區間加倍，對數密度加上 $-\log 2$。組合映射 $f = f_K \circ \cdots \circ f_1$ 只要把各步的對數行列式加起來。

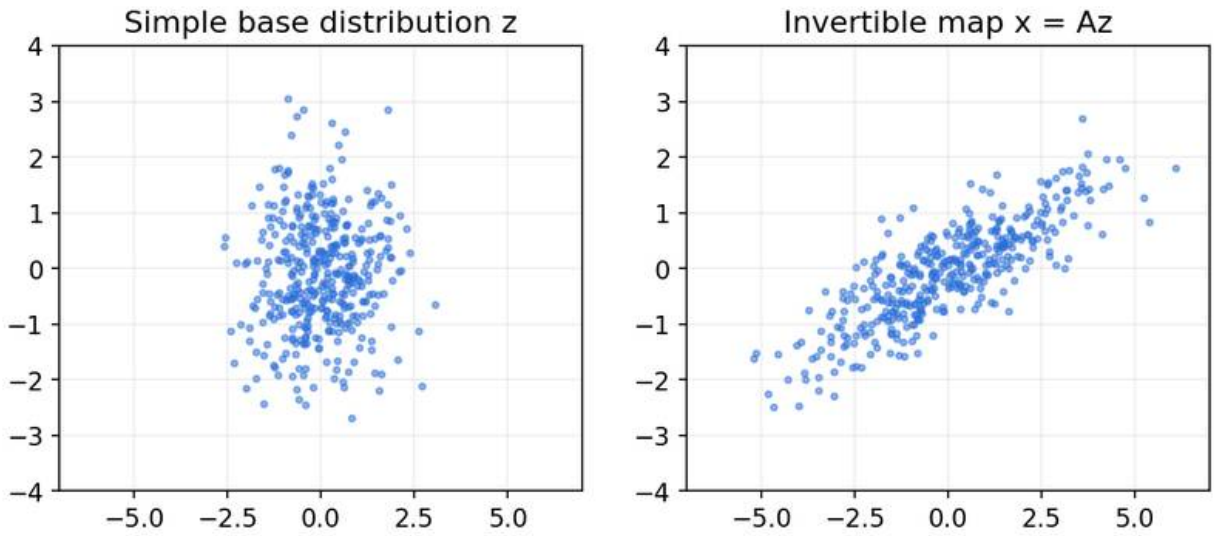


圖 5 — 變數變換保持總機率不變，同時改變密度。這個仿射映射的行列式可以解析計算。

耦合層

公式是精確的，但一般 $D \times D$ 行列式的成本是 $O(D^3)$——對影像來說毫無希望。流的層因此被設計成 Jacobian 為**三角矩陣**，行列式就只是對角線的乘積。RealNVP [18] 的仿射耦合層把 x 分成兩半 (x_1, x_2)，計算

$$y_1 = x_1, \qquad y_2 = x_2 \odot \exp\big(s(x_1)\big) + t(x_1),$$

其中 s 與 t 是任意神經網路。Jacobian 是區塊三角矩陣：第一塊是單位矩陣，第二塊是 $\operatorname{diag}(\exp s(x_1))$，所以 $\log|\det J| = \sum_j s_j(x_1)$。反函數一樣便宜：$x_2 = (y_2 - t(y_1)) \odot \exp(-s(y_1))$。

關鍵在於 s 與 t 永遠不需要求逆，所以可以任意複雜。交替切分方式堆疊多層，就能讓每個座標都被轉換。Glow [19] 則在耦合層之間加入學習得到的可逆 1×1 卷積，作為廣義的通道排列。

程式：驗證耦合層的對數行列式

下面的程式建立一個仿射耦合層，並把它便宜算出的對數行列式，與 autograd 算出的完整 Jacobian 做比對。

```
import torch
from torch import nn

torch.manual_seed(0)

class AffineCoupling(nn.Module):
    """RealNVP-style layer: keep x1, scale-and-shift x2 using x1."""
    def __init__(self, dim, hidden=32):
        super().__init__()
        self.d = dim // 2
        self.net = nn.Sequential(nn.Linear(self.d, hidden), nn.Tanh(),
                                 nn.Linear(hidden, 2 * (dim - self.d)))

    def forward(self, x):
        x1, x2 = x[..., :self.d], x[..., self.d:]
        s, t = self.net(x1).chunk(2, dim=-1)
        s = torch.tanh(s)                       # keep scales moderate
        y2 = x2 * s.exp() + t
        return torch.cat([x1, y2], -1), s.sum(-1)   # output, log|det J|

    def inverse(self, y):
        y1, y2 = y[..., :self.d], y[..., self.d:]
        s, t = self.net(y1).chunk(2, dim=-1)
        s = torch.tanh(s)
        return torch.cat([y1, (y2 - t) * (-s).exp()], -1)

layer = AffineCoupling(4)
x = torch.randn(4)
y, logdet = layer(x)
J = torch.autograd.functional.jacobian(lambda v: layer(v)[0], x)
print("log|det J| from layer:   ", round(logdet.item(), 5))
print("log|det J| from autograd:", round(torch.linalg.slogdet(J)[1].item(), 5))
print("upper-right block is zero:", bool(torch.allclose(J[:2, 2:], torch.zeros(2, 2))))
print("inverse error:", (layer.inverse(y) - x).abs().max().item() < 1e-6)

# Log density of y under a standard-normal base after one layer:
base = torch.distributions.Normal(0., 1.)
z = layer.inverse(y)
log_px = base.log_prob(z).sum() - layer(z)[1]
print("log p(y):", round(log_px.item(), 4))
```

兩個對數行列式都印出 0.14867，Jacobian 右上區塊恰好為零（三角結構），反函數也還原了輸入。

最後一行就是變數變換公式的實際運作。訓練一個流模型，就是用一疊這樣的層，在資料上最大化這個 `log p`。

優點、限制與連續流

流模型提供精確概似、精確反演（適合影像編輯，也能把資料映到性質良好的潛在空間），以及一次前向傳遞的取樣。代價在於架構：每一層都必須可逆且行列式便宜，而且潛在空間必須與資料**同維度**，這會把容量浪費在表示所有像素方向上的雜訊。Rezende 與 Mohamed [20] 最初提出流，是為了讓 VAE 的近似後驗更有彈性，這說明同一個工具既能用於推論，也能用於生成。

連續正規化流把離散的層疊換成常微分方程 $\mathrm{d}x/\mathrm{d}t = v_\theta(x, t)$。Neural ODE [21] 指出，這時對數密度是透過 v_θ 的**散度**（Jacobian 的跡）而非行列式來演化，於是解除了架構限制，但計算概似需要解一次 ODE。ML 07 介紹的**流匹配（flow matching）** [22]，則用簡單的迴歸而非最大概似來訓練這種向量場。所以「流」這個詞指的是相關的轉換，但訓練目標不一定是概似。

能量模型

白話版。與其描述怎麼產生影像，不如給每一張可能的影像打一個分數——「能量」——能量越低代表越「像真的」。能量低的地方，機率就高。

能量模型（energy-based model，EBM）定義

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}, \qquad Z_\theta = \int \exp(-E_\theta(x))\, dx,$$

其中 E_θ 是任何輸出純量的網路，Z_θ 是讓密度積分為一的**配分函數（partition function）**。EBM 極有彈性——任何架構都行——但在高維中 Z_θ 無法計算，所以概似與精確樣本都無法直接取得。Goodfellow、Bengio 與 Courville [23] 涵蓋了經典工具（配分函數與近似推論等章節），Song 與 Kingma [24] 則整理了現代的訓練途徑：搭配 MCMC 樣本（例如 Langevin 動力學）的最大概似；**分數匹配（score matching）**，它擬合 $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$，因而完全不需要 Z_θ；以及雜訊對比類方法。分數匹配正是通往擴散模型的橋樑。

自迴歸模型（簡介）

白話版。一次產生影像的一小塊，每一塊都以目前已產生的全部內容為條件，就像一個字一個字地寫句子。

對 x 的 D 個元素採用任何順序，機率的連鎖律都是精確的：

$$p_\theta(x) = \prod_{i=1}^{D} p_\theta(x_i \mid x_1, \dots, x_{i-1}).$$

每個因子都是一般的分類或迴歸問題，所以訓練就是簡單、穩定、具精確概似的最大概似。PixelRNN 與 PixelCNN [25] 就是這樣對原始像素建模，每個子像素用一個 256 類的 softmax。缺

點是取樣必須循序進行：D 個元素要 D 次網路計算。現代系統改為預測壓縮後的 token、一組組的 token 或整個尺度。ML 08 會詳細介紹這個家族。

擴散模型（簡介）

白話版。教一個網路從帶雜訊的影像中去掉一點點雜訊。然後從純雜訊出發，反覆套用這個去雜訊器，慢慢雕出一張影像。

擴散模型定義一個固定的前向過程，逐步對資料加入高斯雜訊，直到只剩雜訊；再學習一個逐步還原的反向過程。Ho 等人 [26] 證明，訓練（在權重之外）可以化約成預測被加入的雜訊——一個簡單的迴歸——而且這個目標是重新加權的變分下界，這也是圖 1 把擴散模型放在 VAE 旁邊的原因。與分數匹配的連結（見能量模型一節）提供了第二個等價觀點。Yang 等人的綜述 [27] 整理了眾多變體。擴散模型與流匹配如今主導高品質影像生成，是ML 07 的主題。

家族地圖

家族	基本訓練訊號	取樣機制	概似	主要限制
VAE	變分下界（ELBO）	取樣潛在變數再解碼（一次傳遞）	下界	下界取決於近似推論；樣本可能模糊
GAN	對抗判別	生成器前向傳遞（一次）	無	遊戲不穩定；模式崩塌
正規化流	變數變換概似	可逆轉換（一次）	精確	可逆性與同維度限制
自迴歸	條件概似	循序條件取樣	精確	生成時的相依性可能代價高
擴散	去雜訊／分數目標	反向雜訊過程（多步）	下界或基於 ODE	品質取決於模型與取樣器
流匹配	速度場迴歸	對學到的向量場積分	需額外計算	積分成本
能量模型	能量學習（MCMC、分數匹配）	常需迭代取樣	未正規化	正規化與混合可能困難

Bond-Taylor 等人 [3] 把這些取捨總結成一個很難同時滿足的三角：快速取樣、高樣本品質、良好的模態涵蓋。GAN 快又清晰，但可能漏掉模態；概似式模型涵蓋模態，但過去較慢或較模糊；擴散模型以多步取樣為代價，達到高品質與高涵蓋。這張表是教學用的整理，不是互斥的分類。實際模型常結合多個家族：VAE 壓縮影像、Transformer 建模它的潛在分布，再用擴散目標訓練那個 Transformer。

評估生成模型

白話版。生成器可以在不同面向上「好」：給真實資料高機率、產生個別看來逼真的樣本、涵蓋資料中的所有多樣性，或遵循它的條件。沒有任何單一數字能涵蓋全部，所以要用多個指標，並清楚每個指標漏掉了什麼。

Theis 等人 [5] 把這點講得很透徹：在高維中，平均對數概似、樣本品質，以及 Parzen 窗估計之類的舊代理指標，彼此可以**大致獨立**。模型可能概似很好但樣本很差，反之亦然。請依你實際的用途來評估。

概似

對概似式模型而言，保留資料上的 NLL（或每維位元數）有理論根據，在去量化與前處理一致時也可以相互比較。GAN 則無法計算它。而且高概似不代表能可靠地偵測分布外資料：Nalisnick 等人 [28] 發現，在 CIFAR-10 上訓練的流模型、VAE 與 PixelCNN，可能給不相關的 SVHN 門牌數字影像**更高**的概似，因為概似主要受平滑度這類低階統計量主導，而非語意內容。

Inception Score

Salimans 等人 [12] 提出 **Inception Score（IS）**。把每張生成影像 x 送進預訓練的 ImageNet 分類器（Inception-v3），得到標籤分布 $p(y \mid x)$，並令 $p(y) = \mathbb{E}_x[p(y \mid x)]$ 為對樣本的平均。則

$$\mathrm{IS} = \exp\Big(\mathbb{E}_{x \sim p_g}\, D_{\mathrm{KL}}(p(y \mid x) \,\|\, p(y))\Big).$$

當每張影像都被有信心地分類（$p(y \mid x)$ 尖銳）、整組樣本又涵蓋許多類別（$p(y)$ 分散）時，IS 就高。它從不看真實資料，所以無法判斷樣本是否像**你的**資料集，而且只對類似 ImageNet 的內容有意義。

Fréchet Inception Distance

Heusel 等人 [29] 提出的 **Fréchet Inception Distance（FID）**在特徵空間中比較真實與生成影像。對兩組影像擷取 Inception-v3 特徵，各自擬合一個高斯——平均值 μ_r, μ_g 與共變異數 Σ_r, Σ_g——再計算兩個高斯之間的 Fréchet 距離：

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right).$$

越低越好。FID 使用真實資料，對品質與多樣性都敏感，因此成為標準指標。但它不是逐張影像的真實度量：它取決於特徵擷取器、影像縮放方式，特別是**樣本數**（有限樣本下它有偏差，所以只能比較樣本數相同的 FID），而且它把品質與涵蓋度混成一個數字。

精確率與召回率

為了把兩者分開，Sajjadi 等人 [30] 定義了**分布的精確率與召回率**：精確率問的是生成樣本中有多少比例落在真實資料所在之處（品質），召回率問的是真實分布有多少比例被生成器涵蓋（多樣性）。Kynkäänniemi 等人 [31] 提出一個簡單又穩健的估計方法，以下就來實作。用一組球的聯集近似每組樣本的流形，每顆球的半徑等於該點到同組中第 k 個最近鄰的距離。於是

- **精確率** = 落在某個真實樣本的球內的生成樣本比例；
- **召回率** = 落在某個生成樣本的球內的真實樣本比例。

在影像上，這些點是深度特徵而非像素。在我們的二維圓環上，可以直接用座標：

```
import numpy as np

rng = np.random.default_rng(0)
angles = np.arange(8) * 2 * np.pi / 8
centers = 2 * np.stack([np.cos(angles), np.sin(angles)], 1)

def ring(n, modes):
    idx = rng.choice(modes, n)
    return centers[idx] + .05 * rng.standard_normal((n, 2))

def knn_radius(a, k):
    d = np.linalg.norm(a[:, None] - a[None], axis=-1)
    return np.sort(d, axis=1)[:, k]          # column 0 is the point itself

def coverage(query, ref, k=3):
    """Fraction of query points inside some reference point's k-NN ball."""
    r = knn_radius(ref, k)
    d = np.linalg.norm(query[:, None] - ref[None], axis=-1)
    return np.mean((d <= r[None]).any(axis=1))

real = ring(1000, range(8))
fakes = {
    "all 8 modes": ring(1000, range(8)),
    "only 2 modes": ring(1000, [0, 4]),
    "8 modes, blurry": ring(1000, range(8)) + .4 * rng.standard_normal((1000, 2)),
}
for name, fake in fakes.items():
    precision = coverage(fake, real)   # are fake samples on the real manifold?
    recall = coverage(real, fake)      # is the real manifold covered by fakes?
    print(f"{name:16s} precision={precision:.2f} recall={recall:.2f}")
```

```
all 8 modes      precision=0.97 recall=0.96
only 2 modes     precision=0.97 recall=0.26
8 modes, blurry  precision=0.07 recall=0.99
```

模式崩塌的生成器維持很高的精確率——它的每個樣本都很完美——但召回率掉到約 2/8。模糊的生成器恰好相反：它涵蓋每個模態，但大多數樣本偏離資料。像 FID 這樣的單一純量會把兩者都判為「比較差」，卻說不出**為什麼**。圖 6 呈現這三種情況。

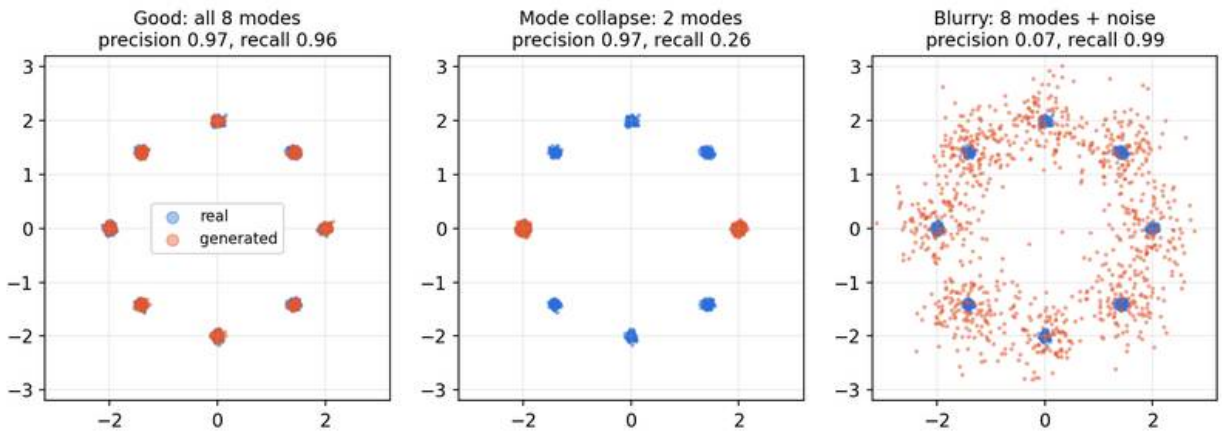


圖 *6* — 玩具圓環上的 k 近鄰精確率與召回率：模式崩塌降低召回率，模糊降低精確率。

現行指標的已知限制

評估本身就是活躍的研究領域。Stein 等人 [32] 進行了大規模人類實驗，發現沒有任何現有自動指標與人類的真實感判斷有強相關；特別是以 Inception 特徵計算的 FID，低估了擴散模型的感知真實度。他們建議 FID 與精確率／召回率之類的指標改用更豐富的自監督特徵空間（DINOv2 ViT-L/14）。Jayasumana 等人 [33] 主張 FID 的常態假設與 Inception 特徵不適合現代文字生成影像模型，並提出 **CMMD**：在 CLIP 嵌入上計算的最大平均差異（maximum mean discrepancy），不需要高斯假設。實務建議：回報多個互補指標並附上確切協定（特徵擷取器、樣本數、解析度、參考集）、展示未經挑選的樣本，而當主張涉及「看起來的品質」時，進行人類評估。

條件式任務還要加上**對齊度**的量測（影像是否符合文字或低解析度輸入？）。在影像復原中，一個漂亮的生成紋理可能沒有觀測資料支持；「看起來合理的樣本」與「量測到的證據」之間的區別，是DLCV 04 的核心議題。

現代觀點

值得一讀的綜述。四篇綜述從不同角度涵蓋本章內容。

- **Bond-Taylor 等人 [3]**（IEEE TPAMI 2022）是 VAE、GAN、正規化流、能量模型與自迴歸模型最好的單篇比較綜述。它依核心技術與已知病徵整理每個家族，並從訓練速度、取樣速度、樣本品質、多樣性與概似加以比較，結論是每個家族都在某處用弱點換來優點，而混合方法是自然的前進方向。
- **Papamakarios 等人 [4]**（JMLR 2021）是正規化流的統一論述：表達能力、耦合式／自迴歸式／殘差式／連續式流的設計空間，以及在密度估計、推論與模擬上的用途。
- **Kingma 與 Welling [9]**（Foundations and Trends in ML 2019）是 VAE 發明者撰寫的權威教學綜述：推導、重參數化估計量、更緊的下界、更有彈性的後驗（包括流）、階層式模型，以及加權 β 的變體。

- **Yang 等人 [27]**（ACM Computing Surveys 2023）綜述擴散模型——取樣加速、概似改進、結構化資料，以及與 VAE、GAN、流和 EBM 的關聯——是通往下一章的橋樑。

Bishop 與 Bishop [1]、Prince [2] 的教科書對所有家族都有一致的推導，Prince 的 notebook 更提供可執行的配套程式。

領域現況（2023–2026）。擴散與流匹配模型——通常在學到的潛在空間中運作——如今產出了大

部分最先進的影像與影片生成結果。這讓**自編碼器**再次成為承重元件：VQ-VAE [34] 展示了如何學習離散潛在碼，讓另一個先驗模型去建模；潛在擴散 [35] 則證明，把生成模型放在壓縮後的自編碼器潛在空間中執行，能在保留細節的同時大幅降低成本。tokenizer 必須在壓縮率、重建品質，以及潛在空間是否容易建模之間取得平衡；請先評估它的重建，因為下游先驗無法還原編碼器已經丟掉的細節。

GAN 並沒有消失。GigaGAN [36] 把 GAN 擴展到文字生成影像，並凸顯 GAN 仍然最擅長的事：快到足以互動使用的單次傳遞取樣、高解析度上取樣，以及平滑、可編輯的潛在空間。Huang 等人 [37] 重新檢視訓練目標本身，推導出具局部收斂保證的正則化相對式（relativistic）GAN 損失，並顯示一個不依賴傳統技巧大全的極簡現代基準（R3GAN）在標準基準上能超越 StyleGAN2。對抗損失也繼續存在於其他系統中，例如用來讓 tokenizer 的重建更銳利。

在評估方面，2023–2024 年對 FID 的批評 [32]、[33] 推動領域轉向更豐富的特徵擷取器、分開的精確率／召回率式指標與人類評估，但至今仍沒有公認的單一指標。

開放問題。（1）與人類一致、且能區分品質、多樣性與記憶的可靠評估。（2）偵測模型何時在複製訓練影像而非泛化。（3）真正能用於分布外偵測的校準概似 [28]。（4）速度、品質與涵蓋度的三角取捨：在不失去模態的前提下，以 GAN 的速度達到擴散模型的品質。（5）學出同時具備壓縮性、忠實性又容易建模的潛在空間。

重點整理

- 生成需要一個分布或一個明確定義的取樣程序。最大概似等於最小化 $D_{\mathrm{KL}}(p_{\text{data}} \| p_\theta)$，這會鼓勵涵蓋每一個模態。
- 概似式模型（自迴歸、流、VAE、EBM、擴散的下界）寫下或界定 $\log p(x)$；隱式模型（GAN）只提供取樣器。
- 一般自編碼器提供重建，卻沒有潛在碼的先驗，所以解碼隨機潛在碼不一定得到逼真的樣本。
- ELBO 等於 $\log p_\theta(x)$ 減去 $D_{\mathrm{KL}}(q_\phi(z \mid x) \| p_\theta(z \mid x))$；重參數化讓它能用反向傳播訓練。
- 在最佳判別器下，原始 GAN 最小化 $2\,\mathrm{JSD} - \log 4$；非飽和損失、Wasserstein 評論器、梯度懲罰、頻譜正規化與 R1 讓這個遊戲可以訓練，但模式崩塌仍必須另外量測。
- 流模型透過變數變換公式計算精確概似，耦合層的三角 Jacobian 讓它變得便宜。
- 概似、IS、FID 與精確率／召回率量測的是不同的東西；請回報多個指標，並附上確切協定。

練習

1. **KL 健全性檢查**。把 $\mu = 0$、$\sigma^2 = 1$ 代入高斯 KL。接著計算一維中 $\mu = 1, \sigma^2 = 1$ 的 KL。

提示

$\mu = 0, \sigma^2 = 1$ 時每一項都是零，因為後驗與先驗是同一個分布。$\mu = 1, \sigma^2 = 1$ 時得到 $\frac{1}{2}(1+1-0-1) = 0.5$ nats：把平均值移動一個標準差，代價是半個 nat。

2. **損失縮放**。把 VAE 的重建項改成對像素取平均而不是加總。若 KL 保持不變，會發生什麼事？

提示

相對於 KL，重建項變小了 64 倍。這等同於對加總損失使用 $\beta = 64$ 的 β-VAE：它實際上改變了正則化強度（往往導向後驗崩塌），而不只是改變回報的單位。

3. **最佳判別器**。在 $p_{\text{data}}(x) = 0.3$、$p_g(x) = 0.1$ 的點 x 上，$D^*(x)$ 是多少？當 $p_g = p_{\text{data}}$ 時，D^* 處處是多少？此時遊戲的值是多少？

提示

$D^*(x) = 0.3/(0.3 + 0.1) = 0.75$。兩個分布相同時，處處 $D^* = 1/2$，$V = \log \frac{1}{2} + \log \frac{1}{2} = -\log 4$，與 JSD $= 0$ 一致。

4. **仿射流**。對 $x = Az + b$、$z \sim \mathcal{N}(0, I)$，推導密度修正項，並說明為何奇異的 A 會使它失效。再用已知的高斯 $\mathcal{N}(b, AA^\top)$ 檢查你的答案。

提示

減去 $\log|\det A|$。因為 $\det(AA^\top)^{1/2} = |\det A|$，這與 $\mathcal{N}(b, AA^\top)$ 的正規化常數一致。奇異映射不可逆，會把體積壓扁，所以同維度的變數變換公式不適用。

5. **辨認崩塌**。分別為 VAE 後驗崩塌與 GAN 模式崩塌提出不同的檢查方法。接著把 GAN 程式改成 WGAN-GP（評論器不加 sigmoid、梯度懲罰 $\lambda = 10$、每次生成器更新前做多次評論器更新），並比較各模態的樣本數。

提示

VAE：檢查每個潛在維度的 KL，以及改變 z 時解碼輸出是否跟著變。GAN：統計每個模態的樣本數（如我們的程式）、計算召回率，並在固定條件下找重複的樣本。這是兩種不同的失敗。WGAN-GP：取樣 $\hat{x} = \alpha x_{\text{real}} + (1-\alpha)x_{\text{fake}}$、$\alpha \sim U(0,1)$，呼叫 `torch.autograd.grad` 並設 `create_graph=True`；若訓練震盪就降低學習率。下結論前請比較多個亂數種子。

6. **指標的盲點**。利用精確率／召回率程式，建構一組生成樣本，使它的精確率與召回率和好的生成器差不多，但一半的樣本只集中在四個模態。這告訴你這個指標有什麼限制？

提示

模態權重不均仍可能得到高召回率，因為召回率只問每個真實點附近是否有**某個**生成點，而不問模態的頻率是否相符。精確率／召回率量測的是支撐集的重疊，而不是密度；請搭配 FID 這類分布層級的指標或逐模態直方圖。

參考文獻

1. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
2. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks
3. S. Bond-Taylor, A. Leach, Y. Long and C. G. Willcocks, "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022. Paper
4. G. Papamakarios et al., "Normalizing Flows for Probabilistic Modeling and Inference," *JMLR*, 2021. Review
5. L. Theis, A. van den Oord and M. Bethge, "A note on the evaluation of generative models," ICLR, 2016. Paper
6. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," ICLR, 2014. Paper
7. S. R. Bowman et al., "Generating Sentences from a Continuous Space," CoNLL, 2016. Paper
8. I. Higgins et al., "β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," ICLR, 2017. Paper
9. D. P. Kingma and M. Welling, "An Introduction to Variational Autoencoders," *Foundations and Trends in Machine Learning*, 2019. Review
10. A. Vahdat and J. Kautz, "NVAE: A Deep Hierarchical Variational Autoencoder," NeurIPS, 2020. Paper
11. I. Goodfellow et al., "Generative Adversarial Nets," NeurIPS, 2014. Paper
12. T. Salimans et al., "Improved Techniques for Training GANs," NeurIPS, 2016. Paper
13. M. Arjovsky, S. Chintala and L. Bottou, "Wasserstein GAN," ICML, 2017. Paper
14. I. Gulrajani et al., "Improved Training of Wasserstein GANs," NeurIPS, 2017. Paper
15. T. Miyato et al., "Spectral Normalization for Generative Adversarial Networks," ICLR, 2018. Paper
16. L. Mescheder, A. Geiger and S. Nowozin, "Which Training Methods for GANs do actually Converge?" ICML, 2018. Paper
17. T. Karras, S. Laine and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," CVPR, 2019. Paper
18. L. Dinh, J. Sohl-Dickstein and S. Bengio, "Density estimation using Real NVP," ICLR, 2017. Paper
19. D. P. Kingma and P. Dhariwal, "Glow: Generative Flow with Invertible 1x1 Convolutions," NeurIPS, 2018. Paper
20. D. J. Rezende and S. Mohamed, "Variational Inference with Normalizing Flows," ICML, 2015. Paper
21. R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, "Neural Ordinary Differential Equations," NeurIPS, 2018. Paper
22. Y. Lipman et al., "Flow Matching for Generative Modeling," ICLR, 2023. Paper
23. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 18–20. Book
24. Y. Song and D. P. Kingma, "How to Train Your Energy-Based Models," arXiv, 2021. Paper
25. A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," ICML, 2016. Paper
26. J. Ho, A. Jain and P. Abbeel, "Denoising Diffusion Probabilistic Models," NeurIPS, 2020. Paper
27. L. Yang et al., "Diffusion Models: A Comprehensive Survey of Methods and Applications,"

ACM Computing Surveys, 2023. Survey
28. E. Nalisnick et al., “Do Deep Generative Models Know What They Don’t Know?” ICLR, 2019. Paper
29. M. Heusel et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” NeurIPS, 2017. Paper
30. M. S. M. Sajjadi, O. Bachem, M. Lucic, O. Bousquet and S. Gelly, “Assessing Generative Models via Precision and Recall,” NeurIPS, 2018. Paper
31. T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen and T. Aila, “Improved Precision and Recall Metric for Assessing Generative Models,” NeurIPS, 2019. Paper
32. G. Stein et al., “Exposing flaws of generative model evaluation metrics and their unfair treatment of diffusion models,” NeurIPS, 2023. Paper
33. S. Jayasumana et al., “Rethinking FID: Towards a Better Evaluation Metric for Image Generation,” CVPR, 2024. Paper
34. A. van den Oord, O. Vinyals and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017. Paper
35. R. Rombach et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” CVPR, 2022. Paper
36. M. Kang et al., “Scaling up GANs for Text-to-Image Synthesis,” CVPR, 2023. Paper
37. Y. Huang, A. Gokaslan, V. Kuleshov and J. Tompkin, “The GAN is dead; long live the GAN! A Modern GAN Baseline,” NeurIPS, 2024. Paper

ML 07・擴散模型與流匹配

推導雜訊、乾淨影像與速度預測，以 JiT 和表徵自編碼器比較像素與潛在生成，並理解預訓練影像／影片的擴散與流生成流程。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-07-diffusion-flow-matching/

你將學到

- 一個「已知的加噪過程」如何把「生成一張影像」變成普通的監督式迴歸問題。
- DDPM 的簡化雜訊預測損失如何一步一步從變分下界推導出來。
- 雜訊、乾淨影像與速度預測如何在代數上換算，為什麼損失與有限容量網路不同，以及分數估計與 SDE 觀點如何連結它們。
- DDPM、DDIM 與快速 ODE 求解器如何取樣，以及為什麼步數是一個設計選擇。
- 分類器引導（classifier guidance）與無分類器引導（classifier-free guidance）如何操控取樣器，代價又是什麼。
- 像素與潛在擴散、DiT、JiT、表徵自編碼器、流匹配與少步生成如何連結，以及預訓練影像／影片流程如何評估。

先看全貌

擴散模型學的是「把受控的破壞復原」。訓練時，我們拿一個真實樣本，加上已知強度的 Gaussian 雜訊，然後請神經網路告訴我們加了什麼雜訊。因為雜訊是我們自己加的，正確答案永遠拿得到：不需要標籤，也不需要對手，就是迴歸而已。生成時，我們從純雜訊出發，反覆套用學到的「去掉一點雜訊」步驟，直到一個樣本浮現出來。

> **用最簡單的話說**
>
> 想像你慢慢用鉛筆在一張畫上亂塗。你把畫在各種塗鴉程度下都留一份：幾條線、很多線、多到原本的畫完全看不見。模型只練習一件小事：看一張被塗過的紙，猜出哪些線是後來加上去的。練習夠多之後，你給它一張「只有」塗鴉的紙。它擦掉一點、再看一次、再擦掉一點，一張全新的畫就慢慢出現了——而且這張畫從來不在那疊紙裡。

為什麼重要？從 2021 年起，擴散類模型在影像、影片與音訊合成上拿下了許多最先進的結果，同一套機制也用於影像修補、超解析度、影像編輯、3D 生成與科學資料。擴散模型也解決了ML 06中較早模型家族的兩個痛點：GAN 需要脆弱的雙人賽局，VAE 的樣本則容易模糊。擴散模型的訓練是穩定的迴歸，而因為生成被拆成許多小而容易的步驟，樣本也很清晰。

代價是取樣成本：經典取樣器每產生一張影像要呼叫網路數百次。2021 年以來的大量研究——DDIM、快速 ODE 求解器、蒸餾、一致性模型、流匹配、整流流——都是在維持品質的同時減少網路呼叫次數。

本章依照幾篇主要綜述的架構安排。Yang 等人 [1] 把領域分成高效取樣、改進概似（likelihood）與特殊結構資料三個方向；Croitoru 等人 [27] 依公式化方式（DDPM、雜訊條件分數網路、SDE）分類；Lipman 等人的流匹配指南 [2] 與 Lai 等人的專書 [29] 則把擴散與流視為同一族「學習出來的傳輸」。Prince 的教科書 [14] 是推導時很好的入門參考。全文**每一節只使用一種明確的時間方向慣例**，換慣例時會特別說明——混用慣例是這個領域最常見的正負號錯誤來源。

前向擴散有封閉解

白話版。每一步保留影像的大部分，再加上一點獨立的 Gaussian 雜訊。經過足夠多步之後，影像什麼都不剩——只剩分布已知的雜訊。

這個想法可追溯到 Sohl-Dickstein 等人 [15]：用緩慢的前向擴散破壞結構，再學習把它反轉；DDPM [3] 讓它變得實用。令 $x_0 \in \mathbb{R}^d$ 為一筆資料（把影像攤平成 d 個數字，大致縮放到 $[-1, 1]$）。選一組**雜訊排程（noise schedule）** $\beta_1, \dots, \beta_T$，其中 $0 < \beta_t < 1$，並記 $\alpha_t = 1 - \beta_t$。前向（加噪）過程是一條馬可夫鏈

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\ \beta_t I\big), \qquad t = 1, \dots, T,$$

其中 $\mathcal{N}(\mu, \Sigma)$ 表示平均為 μ、共變異數為 Σ 的 Gaussian，I 是 $d \times d$ 單位矩陣。係數 $\sqrt{\alpha_t}$ 把訊號稍微縮小，讓總變異數維持有界而不會一路變大；這也是這一族被稱為**變異數保持（variance preserving, VP）**的原因。

推導一次到位的公式

把一步寫成 $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_t$，其中 $\epsilon_t \sim \mathcal{N}(0, I)$。代入前一步，

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\,\epsilon_{t-1}\right) + \sqrt{1-\alpha_t}\,\epsilon_t \\ &= \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\,\epsilon_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_t}_{\text{獨立 Gaussian 之和}}. \end{aligned}$$

兩個雜訊項是獨立、平均為零的 Gaussian，所以相加後仍是 Gaussian，變異數為 $\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}$。把同樣的論證一路推回 x_0，令 $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$，得到

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I\big) \quad \Longleftrightarrow \quad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

這裡 $\bar{\alpha}_t$ 是到第 t 步仍保留的訊號變異數比例，ϵ 是一個標準 Gaussian 向量。這條公式讓訓練很便宜：我們可以直接跳到任何雜訊程度，不必模擬前面的每一步。

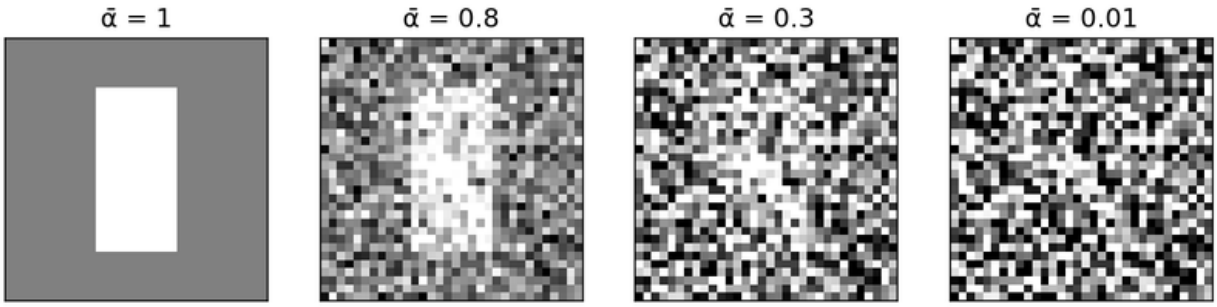


圖 *1* — 封閉形式前向過程在幾個訊號程度下的樣子。這裡共用同一組雜訊以便看出效果；實際訓練時每筆資料、每一步都抽新的雜訊。

雜訊排程與訊雜比

第 t 步的**訊雜比（signal-to-noise ratio, SNR）**為

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t},$$

即訊號變異數與雜訊變異數之比（假設資料變異數為 1）。它從 $t=1$ 時非常大，單調下降到 $t=T$ 時接近零。排程有兩個要求：$\bar{\alpha}_T$ 必須接近 0，讓 x_T 實際上就是標準 Gaussian（取樣器從 $\mathcal{N}(0, I)$ 出發）；中間各步則應把訓練資源花在網路真的有東西可學的雜訊程度上。

DDPM [3] 使用 $T = 1000$ 與**線性排程**，β_t 從 10^{-4} 線性增加到 0.02。Nichol 與 Dhariwal [19] 觀察到這樣會太快摧毀資訊：鏈的相當一大段裡，影像幾乎已經是純雜訊。他們的**餘弦排程**定義 $\bar{\alpha}_t = f(t)/f(0)$，其中 $f(t) = \cos^2\!\left(\frac{t/T+s}{1+s}\cdot\frac{\pi}{2}\right)$，小偏移量 $s = 0.008$，並把每個 β_t 截在 0.999。圖 2 比較兩者。Karras 等人 [20] 後來主張：排程、網路輸入輸出的縮放，以及訓練時雜訊程度的分布，是應該分開調整的獨立設計選擇。

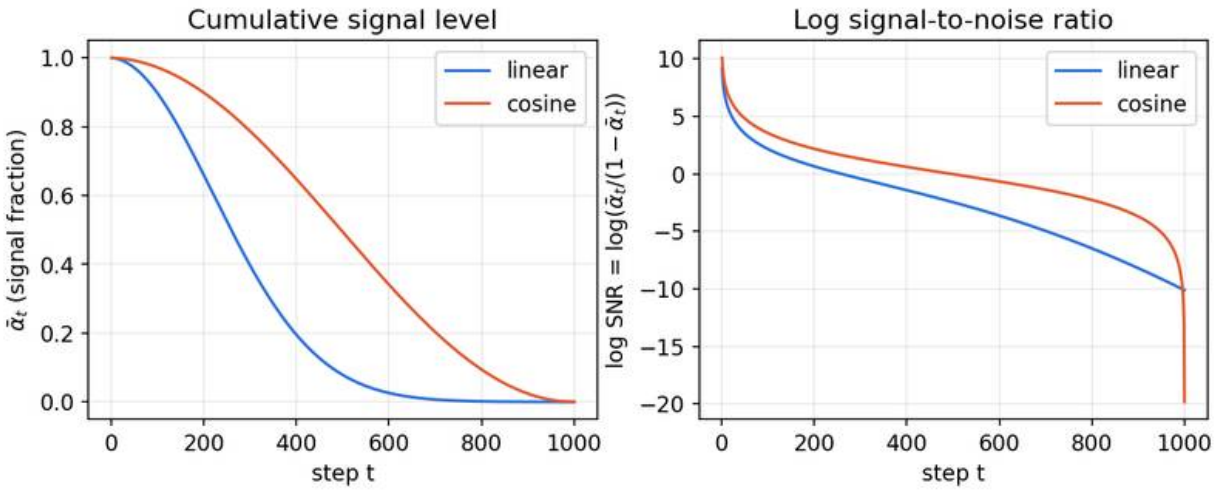


圖 *2* — *$T = 1000$* 時的線性排程與餘弦排程。餘弦排程讓訊號保留得更久，對數訊雜比在各步之間分布得更平均。

反向過程與變分下界

白話版。要生成，就需要反向的一步：「給一個有雜訊的樣本，稍微乾淨一點的樣本長什麼樣？」對真實資料我們無法直接算出來，但如果同時知道乾淨樣本，就算得出來——而這正好給出一個簡單的訓練損失。

模型與下界

生成模型是一條反向馬可夫鏈，從 $p(x_T) = \mathcal{N}(0, I)$ 出發，使用學習出來的 Gaussian 轉移

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\, \sigma_t^2 I\big),$$

其中 μ_θ 是參數為 θ 的神經網路，σ_t^2 是我們稍後要選的變異數。這是一個以 $x_{1:T}$ 為潛在變數的模型，就像一個編碼器 q 固定而非學習的深層 VAE（見ML 06）。同樣的 Jensen 不等式論證給出證據下界（ELBO）。依照 [3]、[15]，負的下界可以整理成一串 KL 散度的和：

$$-\log p_\theta(x_0) \;\le\; \mathbb{E}_q\Big[\underbrace{D_{\mathrm{KL}}(q(x_T \mid x_0)\,\|\,p(x_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0)\,\|\,p_\theta(x_{t-1} \mid x_t))}_{L_{t-1}} \underbrace{-\log p_\theta(x_0 \mid x_1)}_{L_0}\Big].$$

D_{KL} 是 Kullback–Leibler 散度。L_T 沒有可學參數（當 $\bar\alpha_T \approx 0$ 時很小）；L_0 是最後一步的解碼項；每個 L_{t-1} 則把模型的反向一步與**前向後驗分布** $q(x_{t-1} \mid x_t, x_0)$ 相比——也就是「如果知道乾淨樣本 x_0，反向一步會是什麼」的精確答案。

前向後驗是 Gaussian

由貝氏定理，$q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)$。兩個因子對 x_{t-1} 都是 Gaussian，所以乘積也是 Gaussian。把 x_{t-1} 的二次項與一次項收集起來（配方），得到

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(\tilde\mu_t(x_t, x_0),\, \tilde\beta_t I\big),$$

$$\tilde\mu_t = \frac{\sqrt{\bar\alpha_{t-1}}\,\beta_t}{1-\bar\alpha_t}\, x_0 + \frac{\sqrt{\alpha_t}\,(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}\, x_t, \qquad \tilde\beta_t = \frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}\,\beta_t.$$

後驗平均是乾淨樣本與目前雜訊樣本的加權平均；後驗變異數 $\tilde\beta_t$ 比 β_t 略小。你不必相信代數：下面的程式抽樣前向鏈，並以 x_t 對 x_{t-1} 做迴歸。

```
import numpy as np

rng = np.random.default_rng(0)
beta = np.linspace(1e-4, 0.02, 1000)
alpha = 1 - beta
abar = np.cumprod(alpha)
t, x0, n = 300, 1.3, 400_000           # check q(x_{t-1} | x_t, x_0) at step t

# Sample the forward chain: x_{t-1} from q(x_{t-1}|x_0), then one more step.
x_prev = np.sqrt(abar[t-1])*x0 + np.sqrt(1-abar[t-1])*rng.normal(size=n)
```

```
x_t = np.sqrt(alpha[t])*x_prev + np.sqrt(beta[t])*rng.normal(size=n)

# Empirical regression of x_{t-1} on x_t (the posterior mean is linear in x_t).
slope, intercept = np.polyfit(x_t, x_prev, 1)
resid_var = np.var(x_prev - (slope*x_t + intercept))

# Closed-form posterior from the derivation.
c_x0 = np.sqrt(abar[t-1])*beta[t] / (1-abar[t])
c_xt = np.sqrt(alpha[t])*(1-abar[t-1]) / (1-abar[t])
var = beta[t]*(1-abar[t-1]) / (1-abar[t])
print(f"slope     {slope:.4f}  vs  {c_xt:.4f}")
print(f"intercept {intercept:.4f}  vs  {c_x0*x0:.4f}")
print(f"variance  {resid_var:.6f}  vs  {var:.6f}")
```

輸出在小數點後三到四位吻合（斜率 0.9934 對 0.9930，變異數 0.006050 對 0.006052）；微小差距是蒙地卡羅誤差。

從下界到簡化損失

現在把前向公式反過來用：$x_0 = (x_t - \sqrt{1-\bar\alpha_t}\,\epsilon)/\sqrt{\bar\alpha_t}$。代入 $\tilde\mu_t$，再用 $\bar\alpha_t = \alpha_t\bar\alpha_{t-1}$ 化簡，

$$\tilde\mu_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\epsilon\right).$$

取樣時網路看得到 x_t，卻看不到 ϵ。所以 DDPM 讓模型平均採用完全相同的形式，只是把未知的雜訊換成預測值 $\epsilon_\theta(x_t, t)$：

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\epsilon_\theta(x_t, t)\right).$$

兩個共變異數同為固定 $\sigma_t^2 I$ 的 Gaussian，其 KL 散度就是兩個平均之間平方距離的縮放，因此

$$L_{t-1} = \frac{1}{2\sigma_t^2}\|\tilde\mu_t - \mu_\theta\|^2 + C = \frac{\beta_t^2}{2\sigma_t^2\,\alpha_t\,(1-\bar\alpha_t)}\,\|\epsilon - \epsilon_\theta(x_t, t)\|^2 + C,$$

其中 C 與 θ 無關。下界的每一項因此都是**加權的雜訊預測誤差**。Ho 等人 [3] 發現，拿掉權重並均勻抽樣 t，訓練效果反而更好：

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\,t\sim\mathcal{U}\{1,\dots,T\},\,\epsilon\sim\mathcal{N}(0,I)}\left\|\epsilon - \epsilon_\theta\big(\sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon,\ t\big)\right\|^2.$$

把它當成演算法來讀：挑一筆訓練資料、挑一個隨機步數、挑隨機雜訊，用一行算出 x_t，然後迴歸這個雜訊。時間 t 會餵給網路（通常透過正弦嵌入），因為同樣的像素值在不同雜訊程度下代表不同意義。

有兩點容易弄錯。第一，$\mathcal{L}_{\text{simple}}$ 不是概似下界：拿掉權重相對降低了低雜訊步的比重，這有助於樣本品質，卻會讓報告的對數概似變差。Improved DDPM [19] 用混合損失學習變異數 σ_t^2（在 β_t 與 $\tilde\beta_t$ 之間內插），找回了好的概似。第二，同一個網路也給出**乾淨估計**

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}},$$

每個現代取樣器內部都會用到它。當 $\bar{\alpha}_t$ 非常小時，這個除法會放大預測誤差，所以高雜訊時的 $\hat{x}_0$ 是一個模糊的平均，而不是一個樣本。

雜訊、乾淨資料、速度與分數

白話版。預測雜訊、預測乾淨影像、預測「往哪個方向走會讓這個樣本更典型」，是表達同一份知識的三種方式。差別在縮放，而縮放會改變訓練損失對不同雜訊程度的加權。

三種參數化

因為 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ 是固定的線性關係，網路可以輸出下列任何一個：

- 雜訊 ϵ（DDPM [3]），
- 乾淨樣本 x_0，
- **速度** $v = \sqrt{\bar{\alpha}_t}\,\epsilon - \sqrt{1-\bar{\alpha}_t}\,x_0$，由 Salimans 與 Ho [22] 為了蒸餾而提出，在排程兩端都維持良好的尺度。

給定 x_t 與 t，三者可以彼此精確換算。但是對 ϵ、x_0 或 v 做不加權的 MSE，對各雜訊程度的加權並不相同，所以這個選擇會影響訓練。Karras 等人 [20] 用輸入輸出的「預條件（preconditioning）」係數把這件事講清楚：讓網路在每個雜訊程度下的輸入與目標都維持單位變異數。注意，這裡擴散模型的「速度」是訊號與雜訊的一種重新參數化；它與本章後面流匹配的傳輸速度相關，但不相同。

精確轉換不代表有限容量網路等價

對保留變異數的路徑，令 $a = \sqrt{\bar{\alpha}_t}$ 與 $b = \sqrt{1-\bar{\alpha}_t}$，因此 $a^2 + b^2 = 1$，而 $x_t = ax_0 + b\epsilon$。擴散速度 $v_{\mathrm{VP}} = a\epsilon - bx_0$ 與前向狀態共同構成一個旋轉：

$$x_0 = ax_t - bv_{\mathrm{VP}}, \qquad \epsilon = bx_t + av_{\mathrm{VP}},$$
$$\hat{x}_0^{(\epsilon)} = (x_t - b\hat{\epsilon})/a, \quad \hat{\epsilon}^{(x)} = (x_t - a\hat{x}_0)/b.$$

這裡 a, b 是已知的排程係數；v_{VP} 是 [22] 使用的目標；帽號表示估計值；上標指出網路直接輸出的量。最後兩種轉換的除數必須非零。即使內部時間的代數精確，端點附近仍有數值與統計上的不穩定問題。

雜訊誤差 δ_ϵ 會變成乾淨影像誤差 $-(b/a)\delta_\epsilon$。因此，固定雜訊程度下互相對應的預測滿足：

$$\|\hat{x}_0 - x_0\|^2 = \frac{b^2}{a^2}\|\hat{\epsilon} - \epsilon\|^2 = \frac{1}{\mathrm{SNR}(t)}\|\hat{\epsilon} - \epsilon\|^2, \qquad \|\hat{x}_0^{(v)} - x_0\|^2 = b^2\|\hat{v}_{\mathrm{VP}} - v_{\mathrm{VP}}\|^2.$$

$\mathrm{SNR}(t) = a^2/b^2$ 是訊雜比；第二個恆等式由前述旋轉得到。相同形式的無加權損失因此會強調不同時間。改變目標卻未記錄時間抽樣與損失權重，就混入了其他實驗變因 [20]、[22]。

另外還有容量問題。若網路直接輸出乾淨影像，最後的表徵可以捨棄干擾性的雜訊。直接輸出雜訊的網路可能必須保留輸入中更多座標，或透過跳接取得那些座標。事後轉換某個網路的輸出，不會讓它的架構等同於直接學習另一種目標的網路。對大像素區塊與高維潛在表徵，這個差異尤其值得檢查；後面的 JiT 討論提供近期證據 [42]。這不代表雜訊預測普遍失敗：前面的 DDPM 範例與密集影像去雜訊網路就是反例。

```
import numpy as np

rng = np.random.default_rng(7)
x0 = rng.normal(size=32)
eps = rng.normal(size=32)
a, b = np.sqrt(.2), np.sqrt(.8)
xt = a*x0 + b*eps
v_vp = a*eps - b*x0
assert np.allclose(a*xt - b*v_vp, x0)
assert np.allclose(b*xt + a*v_vp, eps)

# A deliberately imperfect noise estimate: compare equivalent losses.
eps_hat = eps + .03*rng.normal(size=32)
x_hat = (xt - b*eps_hat)/a
noise_mse = np.mean((eps_hat - eps)**2)
clean_mse = np.mean((x_hat - x0)**2)
assert np.allclose(clean_mse/noise_mse, b*b/(a*a))
print("Clean/noise MSE ratio:", round(clean_mse/noise_mse, 3))

# A straight flow uses a different path and a different velocity.
t = .35
zt = (1-t)*eps + t*x0
u = x0 - eps
x_from_flow = zt + (1-t)*u
u_from_clean = (x0 - zt)/(1-t)
assert np.allclose(x_from_flow, x0)
assert np.allclose(u_from_clean, u)
print("VP velocity equals straight-flow target:", np.allclose(v_vp, u))
```

範例印出損失比例 4.0，並以 `False` 指出兩種速度目標不同。文獻有時都用字母 v；檢查點的排程器必須指出真正的路徑、正規化與時間方向。

> **互動示範（網站版）**：請開啟本篇教學的網站版本，操作此示範。開啟示範

什麼是分數？為什麼去雜訊能估計它？

密度 p 的**分數（score）**是 $\nabla_x \log p(x)$：一個指向更高機率方向的向量場。如果知道分數，就能用 **Langevin 動力學**取樣：沿著分數走一小步、加一點雜訊，反覆進行。Song 與 Ermon [17] 在多

個雜訊程度上訓練「雜訊條件分數網路（NCSN）」，並以退火 Langevin 動力學取樣——這是一條與擴散平行的路線，後來證明是同一個想法。

連接兩者的橋梁是**去雜訊分數匹配（denoising score matching）**（Vincent [16]）。對 Gaussian 破壞而言，（在均方誤差意義下）最好的雜訊預測器是一個條件期望值 $\epsilon^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t]$，而 **Tweedie 公式** [40] 把它連到雜訊邊際分布 p_t 的分數：

$$\nabla_{x_t} \log p_t(x_t) = -\frac{\mathbb{E}[\epsilon \mid x_t]}{\sqrt{1-\bar{\alpha}_t}} \quad \Longrightarrow \quad s_\theta(x_t, t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}},$$

其中 s_θ 是學到的分數。同一個公式用「乾淨」單位寫成 $\mathbb{E}[x_0 \mid x_t] = (x_t + (1-\bar{\alpha}_t)\nabla \log p_t(x_t))/\sqrt{\bar{\alpha}_t}$。兩個提醒：這是第 t 層加噪後的邊際分布的分數，不是任何單張訓練影像的梯度；而且它對最佳預測器成立——訓練出來的網路只是近似。

你可以在一切都已知的情況下驗證 Tweedie 公式：一維 Gaussian 資料，此時最佳雜訊預測器對 x_t 是線性的。

```
import numpy as np

rng = np.random.default_rng(0)
s, abar, n = 0.7, 0.4, 500_000     # data x0 ~ N(0, s^2); one noise level
x0 = s*rng.normal(size=n)
eps = rng.normal(size=n)
xt = np.sqrt(abar)*x0 + np.sqrt(1-abar)*eps

# "Train" the best linear noise predictor eps_hat = w * x_t by least squares.
w = (xt @ eps) / (xt @ xt)
score_from_eps = -w / np.sqrt(1-abar)       # s(x_t) = -eps_hat / sqrt(1-abar)

# Exact: x_t ~ N(0, abar s^2 + 1 - abar), so grad log p_t(x) = -x / var.
exact = -1 / (abar*s**2 + 1 - abar)
print(f"score slope from noise predictor: {score_from_eps:.4f}")
print(f"exact score slope:                {exact:.4f}")
```

它印出 -1.2560 對 -1.2563：對雜訊做迴歸就能找回分數。

SDE 觀點

Song 等人 [4] 讓步數趨於無窮。用連續時間 $t \in [0, 1]$ 與速率 $\beta(t)$，VP 前向過程變成**隨機微分方程（SDE）**

$$dx = -\tfrac{1}{2}\beta(t)\, x\, dt + \sqrt{\beta(t)}\, dw,$$

其中 w 是標準 Wiener 過程（布朗運動）。一個經典結果指出：這個過程倒著時間跑，仍然是一條 SDE，而其中唯一未知的量就是分數：

$$dx = \big[-\tfrac{1}{2}\beta(t)\, x - \beta(t)\, \nabla_x \log p_t(x) \big] dt + \sqrt{\beta(t)}\, d\bar{w},$$

此處 dt 為負，$\bar{w}$ 是反向時間的 Wiener 過程。此外，還存在一條確定性的**機率流 ODE**（**probability-flow ODE**），其解在每個時間點的邊際分布 p_t 都相同：

$$\frac{dx}{dt} = -\tfrac{1}{2}\beta(t)\big[x + \nabla_x \log p_t(x)\big].$$

這把整個領域統一起來。DDPM 的祖先取樣（ancestral sampling）是反向 SDE 的離散化；NCSN [17] 對應到「變異數爆炸（variance exploding）」SDE；而這條 ODE 把擴散模型變成連續正規化流，因此可以用瞬時變數變換公式算出精確的對數概似 [4]。這條 ODE 也解釋了 DDIM 這類確定性取樣器為什麼有效，以及為什麼可以套用一般的 ODE 求解器。

取樣是一個反向過程

白話版。在單一隨機雜訊程度上算出的損失並不是生成器。要生成，必須從純雜訊出發，以一致的排程串起許多反向步驟——而你可以選擇要走幾步、每步注入多少新的隨機性。

DDPM 祖先取樣

DDPM 沿著學到的反向鏈取樣：

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I), \qquad \sigma_t^2 \in \{\beta_t,\ \tilde{\beta}_t\},$$

最後一步不再加入雜訊。Ho 等人 [3] 報告 $\sigma_t^2 = \beta_t$ 與 $\sigma_t^2 = \tilde{\beta}_t$ 的結果相近；學習出的變異數 [19] 在步數較少時幫助最大。

程式：二維分布上的 DDPM

這個完整的 CPU 範例（約 10 秒）使用兩個中心在 $x = \pm 1.5$、標準差 0.25 的 Gaussian 群集。它包含影像模型的每個運算——排程、隨機時間訓練、雜訊預測、反向平均——卻不需要影像等級的計算量。在 100 步與這個排程下，到 $t = T$ 時幾乎沒有訊號殘留。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
T = 100
beta = torch.linspace(.001, .15, T)
alpha = 1-beta
abar = alpha.cumprod(0)
previous = torch.cat([torch.ones(1), abar[:-1]])
post_var = beta * (1-previous) / (1-abar)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
```

```
def data(n):
    side = torch.randint(2, (n, 1))*2-1
    return torch.cat([side*1.5, torch.zeros(n, 1)], 1) + .25*torch.randn(n, 2)
for step in range(3000):
    x0 = data(128)
    t = torch.randint(T, (len(x0),))
    eps = torch.randn_like(x0)
    a = abar[t, None]
    xt = a.sqrt()*x0 + (1-a).sqrt()*eps
    predicted = net(torch.cat([xt, t[:, None].float()/(T-1)], 1))
    loss = F.mse_loss(predicted, eps)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()
with torch.no_grad():
    x = torch.randn(512, 2)
    for t in reversed(range(T)):
        time = torch.full((len(x), 1), t/(T-1))
        eps = net(torch.cat([x, time], 1))
        mean = (x-beta[t]/(1-abar[t]).sqrt()*eps)/alpha[t].sqrt()
        x = mean + post_var[t].sqrt()*torch.randn_like(x) if t else mean
    assert torch.isfinite(x).all()
    print("terminal signal:", round(abar[-1].item(), 6))
    left = x[x[:, 0] < 0]
    print("fraction in left cluster:", round((x[:, 0] < 0).float().mean().item(), 3))
    print("left cluster: mean x", round(left[:, 0].mean().item(), 2),
          "std", round(left[:, 0].std().item(), 2), "(data: -1.5 and 0.25)")
```

在我們的機器上，它印出終端訊號 0.000349、左群集比例 0.441，左群集平均與標準差為 -1.47 與 0.25。群集的形狀是對的；44/56 的分配則顯示這個小網路沒有精確學到各模態的權重。這就是為什麼單一摘要數字永遠不夠：一定要把樣本畫出來，並對照新抽的資料檢查模態涵蓋、模態內離散程度與離群值。

DDIM：同樣的訓練，不同的取樣器

Song、Meng 與 Ermon [5] 注意到 $\mathcal{L}_{\text{simple}}$ 只依賴邊際分布 $q(x_t \mid x_0)$，而不依賴產生它們的馬可夫鏈。許多非馬可夫的前向過程擁有相同的邊際分布，每一個都給出不同的反向取樣器——而且全部都能用同一個訓練好的 ϵ_θ。對任何遞減的步數子序列 $\tau_1 > \tau_2 > \cdots$，**DDIM** 的更新是

$$x_{\tau_{i+1}} = \sqrt{\bar\alpha_{\tau_{i+1}}}\,\hat{x}_0 + \sqrt{1-\bar\alpha_{\tau_{i+1}}-\sigma^2}\,\epsilon_\theta(x_{\tau_i},\tau_i) + \sigma z, \qquad \sigma = \eta\sqrt{\frac{1-\bar\alpha_{\tau_{i+1}}}{1-\bar\alpha_{\tau_i}}\Big(1-\frac{\bar\alpha_{\tau_i}}{\bar\alpha_{\tau_{i+1}}}\Big)},$$

其中 $\hat{x}_0$ 是在 x_{τ_i} 的乾淨估計，$\eta \ge 0$ 控制隨機性。讀法是：「先估計乾淨樣本，再沿著預測的雜訊方向，把它重新加噪到下一個較低的雜訊程度。」當 $\eta = 1$ 且步數連續時，它還原 DDPM 的後驗變異數；當 $\eta = 0$ 時，取樣器是**確定性**的，也是機率流 ODE 的一種離散化。確定性有實際影響：同樣的起始雜訊永遠得到同樣的輸出，所以可以在雜訊空間內插，或把真實影像反推回它的雜訊（DDIM inversion），這是許多影像編輯方法的基礎。

以下獨立範例重複相同的小型訓練設定，讓程式能單獨執行。在同一工作階段實驗時，可直接重用已訓練的 `net`、`abar` 與 `T`：

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
T = 100
beta = torch.linspace(.001, .15, T)
alpha = 1-beta
abar = alpha.cumprod(0)
previous = torch.cat([torch.ones(1), abar[:-1]])
post_var = beta * (1-previous) / (1-abar)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
def data(n):
    side = torch.randint(2, (n, 1))*2-1
    return torch.cat([side*1.5, torch.zeros(n, 1)], 1) + .25*torch.randn(n, 2)
for step in range(3000):
    x0 = data(128)
    t = torch.randint(T, (len(x0),))
    eps = torch.randn_like(x0)
    a = abar[t, None]
    xt = a.sqrt()*x0 + (1-a).sqrt()*eps
    predicted = net(torch.cat([xt, t[:, None].float()/(T-1)], 1))
    loss = F.mse_loss(predicted, eps)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()

def ddim_sample(n, steps, eta=0.0, seed=1):
    g = torch.Generator().manual_seed(seed)
    taus = torch.linspace(T-1, 0, steps).round().long()
    x = torch.randn(n, 2, generator=g)
    with torch.no_grad():
        for i, t in enumerate(taus):
            time = torch.full((n, 1), t.item()/(T-1))
            eps = net(torch.cat([x, time], 1))
            a_t = abar[t]
            a_prev = abar[taus[i+1]] if i+1 < steps else torch.tensor(1.)
            x0_hat = (x - (1-a_t).sqrt()*eps) / a_t.sqrt()
            sigma = eta*((1-a_prev)/(1-a_t)*(1-a_t/a_prev)).sqrt()
            x = (a_prev.sqrt()*x0_hat + (1-a_prev-sigma**2).sqrt()*eps
                 + sigma*torch.randn(n, 2, generator=g))
    return x

for steps in [5, 20, 100]:
    s = ddim_sample(2000, steps)
    left = s[s[:, 0] < 0]
    print(f"{steps:3d} steps: left fraction {(s[:, 0] < 0).float().mean():.3f}, "
          f"left-cluster mean x {left[:, 0].mean():.2f}, std {left[:, 0].std():.2f}")
```

輸出呈現典型的規律。用 5 步時，左群集中心在 −1.11、標準差 0.39——樣本被拉向中間並被抹

開，因為大步長是用直線去走一條彎曲的軌跡。用 20 步時是 −1.37 與 0.30；100 步時是 −1.42 與 0.24，接近資料。改變步數完全不需要重新訓練。

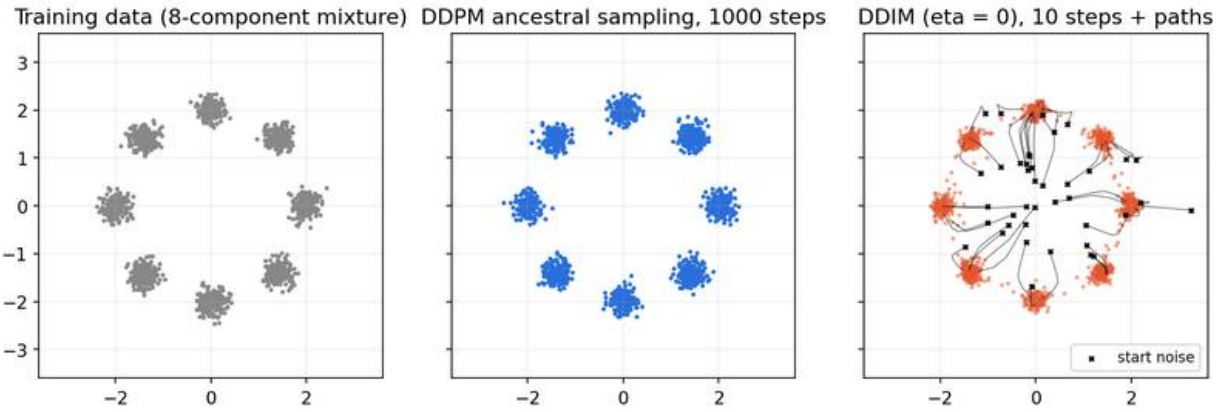


圖 *3* — 取樣器都由八成分混合的精確去雜訊器驅動，所以只有取樣規則不同。*1000* 步 *DDPM* 重現了資料；*10* 步確定性 *DDIM* 落在模態附近，但群集略寬、被抹開。黑線是從起始雜訊（×）出發的 *DDIM* 軌跡。

更快的求解器

一旦取樣等同於「解一條 ODE」，數值分析就派得上用場。Karras 等人（EDM）[20] 使用二階 Heun 求解器搭配仔細選擇的雜訊程度間距，並證明取樣器、排程、預條件與訓練雜訊分布可以分開調整。DPM-Solver [21] 利用擴散 ODE 的半線性結構——線性部分可以精確積分——在大約 10 次網路評估內達到良好品質。隨機取樣器在每一步加一點新雜訊；這需要更多步，但可以修正累積的誤差。無論用哪種求解器，都要報告**函數評估次數（number of function evaluations, NFE）**：網路被呼叫的次數才是真正的取樣成本，而不是「步數」。

操控生成：條件與引導

白話版。想生成「一隻貓」而不是「任何東西」，就把條件交給網路。引導則像一個旋鈕：更嚴格地遵循條件，代價是多樣性。

條件化

條件模型學習 $\epsilon_\theta(x_t, t, c)$，其中 c 是條件。類別標籤透過嵌入加到時間嵌入上；文字透過**交叉注意力（cross-attention）**，讓影像特徵去關注文字編碼器的 token [7]；影像（用於超解析度、修補、編輯）則串接到輸入通道，或經由額外的編碼器送入。

分類器引導

對加噪後的邊際分布套用貝氏定理，得到 $\nabla \log p_t(x_t \mid y) = \nabla \log p_t(x_t) + \nabla \log p_t(y \mid x_t)$。Dhariwal 與 Nichol [18] 在加噪影像上另外訓練一個分類器 $p_\phi(y \mid x_t)$，把它的梯度乘上 w 加到無條件模型上。以雜訊單位表示，

$$\hat{\epsilon} = \epsilon_\theta(x_t, t) - w\sqrt{1-\bar{\alpha}_t}\,\nabla_{x_t} \log p_\phi(y \mid x_t).$$

尺度 $w > 1$ 大致是從 $p_t(x_t)\,p_t(y \mid x_t)^w$ 取樣——一個被銳化的分布，用多樣性換取擬真度。加上架構改進，這篇論文展示了擴散模型在 ImageNet [35] 的樣本品質上勝過 GAN [18]。缺點是需要額外的分類器，而且必須在每個雜訊程度的雜訊輸入上訓練。

無分類器引導（CFG）

Ho 與 Salimans [6] 拿掉了分類器。訓練時，隨機一部分樣本的條件被換成「空」標記，於是同一個網路同時學到 $\epsilon_\theta(x_t, t, c)$ 與 $\epsilon_\theta(x_t, t, \varnothing)$。貝氏定理告訴我們，分類器梯度等於條件分數與無條件分數之差，所以上式的「隱式分類器」版本是

$$\epsilon_{\text{guided}} = \epsilon_\theta(x_t, t, \varnothing) + w\big(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\big).$$

在這個慣例下，$w = 0$ 是無條件，$w = 1$ 是普通的條件模型，$w > 1$ 則外插超過它。（有些論文把同一件事寫成 $(1 + w')\epsilon_c - w'\epsilon_u$，因此他們的「尺度」是我們記號中的 $w - 1$；比較數字之前一定要先確認慣例。）

下面的程式使用四團混合分布的精確雜訊預測器，因此任何奇怪的行為都來自引導本身，而不是網路沒訓練好。類別 1 是 $x > 0$ 的那兩團。

```
import numpy as np

# Data: four Gaussian blobs at the corners of a square; class c is the sign of x.
MU = np.array([[-1.5, -1.5], [-1.5, 1.5], [1.5, -1.5], [1.5, 1.5]])
SIG = 0.2
CLASS = np.array([0, 0, 1, 1])

def eps_exact(x, abar, comps):
    """Exact E[eps | x_t] for a Gaussian mixture restricted to `comps`."""
    m = np.sqrt(abar)*MU[comps]
    var = abar*SIG**2 + 1 - abar
    logw = -((x[:, None] - m[None])**2).sum(-1) / (2*var)
    w = np.exp(logw - logw.max(1, keepdims=True))
    w /= w.sum(1, keepdims=True)
    score = -(w[..., None]*(x[:, None] - m[None])).sum(1) / var
    return -np.sqrt(1-abar)*score

abar = np.cumprod(1 - np.linspace(1e-4, 0.02, 1000))
taus = np.linspace(999, 0, 50).round().astype(int)
everything, class1 = np.arange(4), np.where(CLASS == 1)[0]
print("data: mean distance to its mode", round(SIG*np.sqrt(np.pi/2), 3))
for w in [0, 1, 3, 8]:
    x = np.random.default_rng(0).normal(size=(4000, 2))
```

```
    for i, t in enumerate(taus):                    # deterministic DDIM
        e_u, e_c = eps_exact(x, abar[t], everything), eps_exact(x, abar[t], class1)
        e = e_u + w*(e_c - e_u)                     # classifier-free guidance
        x0 = (x - np.sqrt(1-abar[t])*e) / np.sqrt(abar[t])
        a_prev = abar[taus[i+1]] if i+1 < len(taus) else 1.0
        x = np.sqrt(a_prev)*x0 + np.sqrt(1-a_prev)*e
    dist = np.linalg.norm(x[:, None] - MU[None], axis=-1).min(1)
    print(f"w={w}: class-1 fraction {(x[:, 0] > 0).mean():.3f}, "
          f"mean distance to nearest mode {dist.mean():.3f}")
```

資料到其模態的平均距離是 0.251。$w = 0$ 時約一半樣本（0.490）屬於類別 1；$w = 1$ 時全部屬於類別 1，距離為 0.225（50 步 DDIM 略微低估離散程度）。$w = 3$ 時距離為 0.259，$w = 8$ 時增加到 0.446：強引導不只是「讓它更像類別 1」——它把樣本推超過資料，這是影像模型在非常高引導下出現過飽和色彩與變形的玩具版本。圖 4 在八模態混合上呈現同樣的效應。

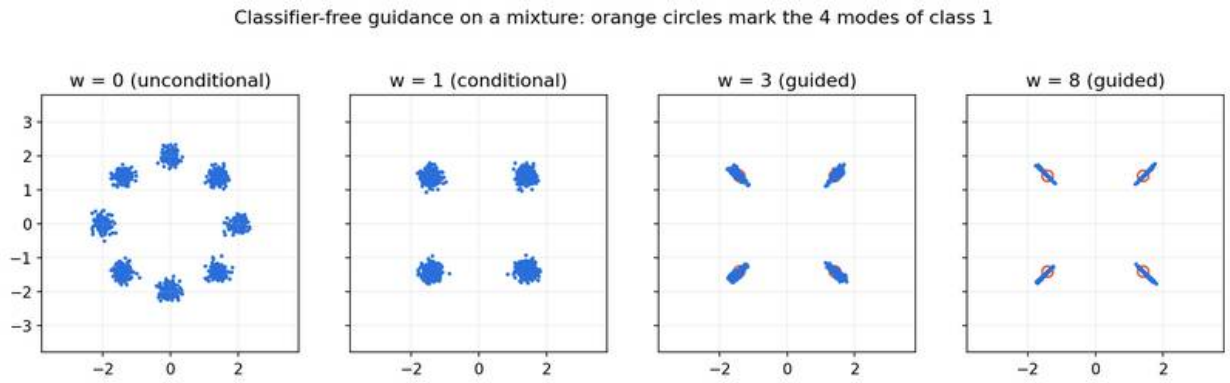


圖 *4* — 在八模態混合上的精確無分類器引導，其中類別 *1* 有 *4* 個模態（橘色圓圈）。$w = 1$ 已經把樣本限制在類別 *1*；更大的 w 讓群集在某些方向收縮、在其他方向拉長——樣本不再服從任何資料分布。

引導也有計算成本：每一步都需要一次條件預測和一次無條件預測，所以 30 步的引導取樣器需要 60 次網路評估，除非兩者被合併計算。而且 w 是取樣的旋鈕，不是影像與提示詞相符程度的校準指標。

潛在擴散與擴散 Transformer

白話版。像素很昂貴。先用自編碼器把影像壓縮成較小的特徵網格，在這個壓縮空間裡做擴散，最後再解碼。另外，去雜訊網路本身也可以是 Transformer，而不是卷積式 U-Net [37]。

潛在擴散

Rombach 等人 [7] 先訓練一個自編碼器：編碼器 $z = \mathcal{E}(x)$ 把 $H \times W \times 3$ 影像映射成 $h \times w \times c$ 的潛在表示，空間降取樣倍率為 $f = H/h$；解碼器 $\mathcal{D}$ 再映射回來。自編碼器以重建、感知與對抗損失，加上輕量的正則化（朝向 Gaussian 的小 KL 項，或向量量化）訓練，讓潛在空間保持良好性

質。接著在 z 上照原樣訓練擴散模型，生成最後以 $\hat{x} = \mathcal{D}(z_0)$ 結束。當 $f = 8$ 時，512×512 影像變成 64×64 的潛在表示：每次網路呼叫的空間位置少了 64 倍。

取捨是重建瓶頸：編碼器丟棄的資訊無法由該代碼忠實找回，雖然解碼器可能合成合理細節。潛在表示的尺度也必須與擴散模型訓練時一致；實作上會把潛在表示重新縮放到大約單位變異數。[7] 的文字條件化在 U-Net 中使用交叉注意力層，這也是 Stable Diffusion 系列背後的設計。

擴散 Transformer

DiT（Peebles 與 Xie [8]）把 U-Net 換成在潛在 patch 上運作的 Vision Transformer [41]（見ML 04）。大小為 $h \times w$ 的潛在表示被切成 $p \times p$ 的 patch，得到 hw/p^2 個 token；p 越小，token 越多、計算量越大、品質越好。時間步與類別標籤透過**自適應層正規化（adaptive layer norm）**為每個區塊提供條件：它們預測每個 LayerNorm 的縮放與平移；在「adaLN-Zero」變體中還預測一個初始化為零的閘控係數，讓每個殘差區塊一開始等於恆等映射。這篇論文的主要發現是一個縮放趨勢：在不同模型大小與 patch 大小之間，Transformer 計算量（GFLOPs）越多，FID 就穩定地越低 [8]。

把三個選擇分開來看很有幫助。**架構**（U-Net 或 Transformer）是函數近似器。**目標**（雜訊、x_0、v 或流速度）是它要迴歸的對象。**取樣器**（DDPM、DDIM、ODE 求解器）則決定預測如何變成影像。Stable Diffusion 3 [9] 是一種新組合的例子：一個潛在 Transformer，影像與文字 token 串流使用各自的權重但共同做注意力（「MM-DiT」），以整流流目標訓練，並讓訓練時間步的分布偏向中間雜訊程度。

對影片而言，潛在表示與注意力必須同時涵蓋時間與空間，因此壓縮與注意力模式成為主要的工程問題。Melnik 等人的綜述 [31] 回顧了影片擴散的架構、時間一致性機制與評估。在影片上，逐格品質不夠：物體恆存、動作合理性與長時間一致性都需要各自的評估。

像素空間、潛在空間與表徵自編碼器

白話版。生成器需要一個反覆更新的空間。它可以是影像本身、緊湊的重建代碼，也可以是視覺表徵模型學到的特徵。加入潛在階段，是選擇特定的計算與統計取捨，而不是生成模型必備的條件。

為什麼像素要變成潛在表徵，再變回像素？

訓練影像以像素形式進來，輸出也必須是可觀看的像素。在兩端之間，編碼器可以去除冗餘，產生適合反覆更新、成本較低的網格 [7]。先用重建訓練編碼器與解碼器，再對編碼後的訓練影像學習生成分布。兩項工作不同：重建保留某個指定樣本；生成學習哪些編碼樣本合理。

訓練與取樣路徑如下：

$$\text{training: } x \xrightarrow{\mathcal{E}} z_{\text{data}} \xrightarrow{\text{corrupt}} z_t \xrightarrow{\text{prediction loss}} \theta,$$
$$\text{generation: } z_{\text{noise}} \xrightarrow{\text{learned sampler}} \hat{z} \xrightarrow{\mathcal{D}} \hat{x},$$
$$\text{reconstruction check: } x \xrightarrow{\mathcal{E}} \mathcal{E}(x) \xrightarrow{\mathcal{D}} \tilde{x}.$$

這裡 x 是訓練影像；$\mathcal{E}$ 與 $\mathcal{D}$ 是編碼器與解碼器；z_{data} 是編碼後的影像；z_t 是其受污染狀態；θ 是生成預測器的參數；$\hat{z}$ 是生成的代碼；$\hat{x}$ 是生成影像；$\tilde{x}$ 是重建影像。純文字到影像取樣**不需要**先編碼一張輸入影像。編輯與影像到影像生成則可以編碼實際輸入，從部分污染的代碼開始。

主要節省來自去雜訊器或流預測器會反覆執行，而編碼訓練資料與解碼樣本的次數相對少。空間縮減因子 f 使網格位置減少 f^2 倍，但潛在通道數同樣重要。三通道影像與 c 通道潛在表徵的純量數量縮減比例是 $3f^2/c$。不能把網格縮減比例直接說成所有浮點數量或總執行時間的縮減比例。

對 Transformer，還必須區分詞元數與詞元寬度。完整注意力儲存的矩陣元素數與 N^2 成正比，其中 N 是詞元數；特徵投影也受詞元寬度影響 [41]。像素與潛在空間都能以區塊大小降低 N。這些數量本身不能預測實際速度，因為運算核心、記憶體傳輸、解碼器與條件化也有成本。

```
# Synthetic shape bookkeeping, not a benchmark or a specific checkpoint.
H = W = 512
channels, spatial_factor, patch = 4, 8, 2
pixel_values = H*W*3
h = H//spatial_factor
w = W//spatial_factor
latent_values = h*w*channels
pixel_tokens = (H//patch)*(W//patch)
latent_tokens = (h//patch)*(w//patch)
print("Spatial positions reduction:", (H*W)//(h*w))
print("Scalar values reduction:", pixel_values//latent_values)
print("Dense attention entries reduction:", pixel_tokens**2//latent_tokens**2)
assert pixel_values//latent_values == 48
assert pixel_tokens//latent_tokens == 64
assert pixel_tokens**2//latent_tokens**2 == 4096
```

此人工例子得到空間位置少 64 倍、純量少 48 倍、注意力矩陣元素少 4096 倍，條件是**兩個座標空間都採用數值相同的區塊大小**。改變區塊大小，比較結果就會改變。像素 Transformer 可以使用大得多的區塊，得到與潛在 Transformer 相同的詞元數。

代價是資訊流失。若兩張影像具有相同代碼，確定性的解碼器無法還原各自不同的精確細節。解碼器可以補出合理紋理，但那沒有找回原始影像的資訊。因此，生成訓練之前應獨立評估重建。檢查細線、文字與跨影格變化，而不只看平均影像誤差。

下例的簡單編碼器對區塊取平均，解碼器重複這些平均值。它示範瓶頸，不宣稱重現學習式 VAE 或 RAE。

```
import numpy as np

y, x = np.mgrid[:64, :64]
checker = ((x+y) % 2).astype(float)
smooth = x/63
```

```
image = .6*smooth + .4*checker
f = 4
latent = image.reshape(64//f, f, 64//f, f).mean(axis=(1, 3))
reconstruction = np.repeat(np.repeat(latent, f, axis=0), f, axis=1)
print("Synthetic reconstruction MSE:", round(np.mean((image-reconstruction)**2), 6))
assert np.mean((image-reconstruction)**2) > .04
# Recovering the exact latent does not recover the missing checkerboard.
assert np.allclose(reconstruction[0, :f], reconstruction[0, 0])
```

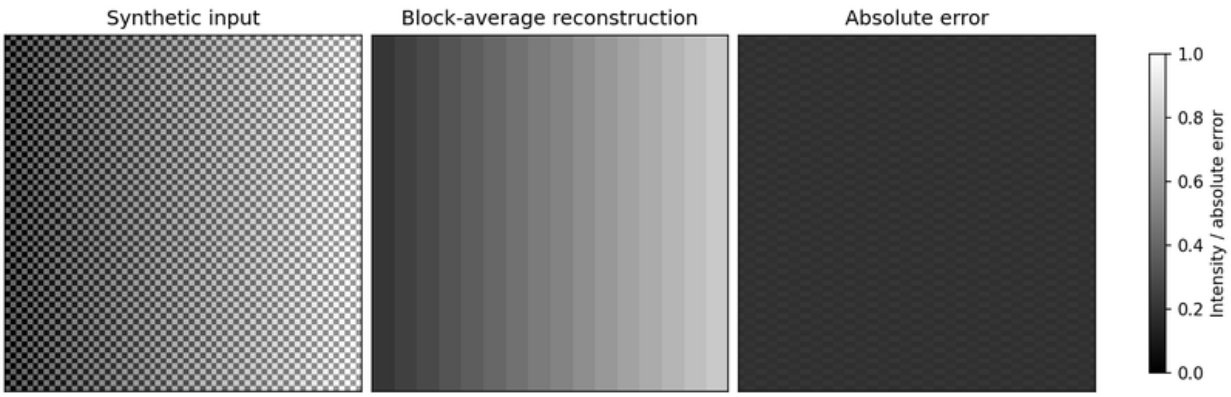


圖 5 — 潛在瓶頸可能在生成模型訓練前就移除細節。區塊平均範例用來解釋資訊流失，並非學習式自編碼器的評估。

在編輯時，反覆編碼與解碼結果可能累積失真。若流程允許，可將中間編輯保留在相容的潛在空間，並檢查最後的像素輸出。優良的無條件生成器仍可能不擅長重建。自編碼器的上限指的是對編碼資訊的忠實還原，並不表示生成解碼器永遠無法合成新的細緻紋理。

重建代碼與語意表徵代碼

VAE 編碼器以重建目標與分布正則化訓練，其代碼設計是為了支援解碼與取樣 [7]。預訓練辨識編碼器則學習有用的視覺特徵。兩者目標可以一致，但辨識所需的不變性也可能捨棄忠實重建需要的細節。解碼器與保留下來的特徵結構，決定表徵是否適合成為生成空間。

Zheng、Ma、Tong 與 Xie 的**表徵自編碼器（Representation Autoencoders，RAEs）**[43]，把凍結的預訓練表徵編碼器搭配可訓練解碼器。其 ICLR 2026 研究以語意視覺特徵作為生成模型的空間，檢查高維詞元特徵、依維度調整的雜訊排程、解碼器穩健性，以及淺而寬的擴散輸出頭。這改變了模型生成的表徵，而不只是對傳統潛在預測器加入特徵對齊損失。

不要把「潛在」等同於「每一維都更小」。空間稀疏的特徵網格仍可能有很多通道。RAE 也不會自動成為變分模型：凍結特徵擷取器加上解碼器，本身並未定義 Gaussian 後驗、KL 正則化或概似下界。應分別測量重建、表徵品質與生成品質。

Tong 等人 [44] 在 2026 年將此方法延伸到自由形式的文字到影像生成。研究在更大規模比較 RAE 與 VAE 空間，並檢查解碼器資料組成。結果顯示，依維度調整的雜訊排程仍重要，而某些小規

模架構與解碼器調整在其較大規模設定下幫助有限。這提醒我們在規模與資料變化時重新測試配方，不能把 ImageNet 結果當成普遍設計規則。

對新應用，檢查所選編碼器是否保留重要內容：文件的細小文字、檢測影像的小缺陷，或影片的影格間差異。用保留的真實特徵與生成特徵分別評估解碼器。只用精確訓練代碼學習的解碼器，可能對取樣代碼的小幅分布外誤差敏感。依訓練配方正規化特徵，並將編碼器版本與生成檢查點一起記錄。

JiT：在像素上直接預測乾淨影像

Li 與 He 的 **Just image Transformers（JiT）**[42] 在 2025 年發布預印本，並於 CVPR 2026 發表，重新檢查直接操作像素的大區塊 Transformer。網路預測乾淨影像，再由該預測推得取樣用的流速度。其控制實驗支持以流形解釋的觀點：瓶頸能保留有結構的乾淨影像資訊，卻可能讓直接預測高維雜訊或速度變困難。該實驗設定不使用預訓練潛在詞元化器或額外表徵損失。這是特定架構與訓練條件的證據，不是像素模型總能勝過潛在生成器的證明。

關鍵是區分**網路直接輸出**與**損失或求解器使用的量**。對後面定義的直線路徑，使用 $y_t = (1-t)\epsilon + tx$，$t = 0$ 是雜訊，$t = 1$ 是乾淨資料。直接預測 $\hat{x}_\theta(y_t, t)$ 的網路可以提供：

$$\hat{u}_\theta(y_t, t) = \frac{\hat{x}_\theta(y_t, t) - y_t}{1-t}, \qquad \|\hat{u}_\theta - (x-\epsilon)\|^2 = \frac{\|\hat{x}_\theta - x\|^2}{(1-t)^2}.$$

這裡 x 是乾淨影像；ϵ 是抽出的雜訊；y_t 是兩者內插；$\hat{x}_\theta$ 是直接預測；$\hat{u}_\theta$ 是對應的傳輸速度。恆等式適用於 $t < 1$ 且尚未進行數值截限的情況。因此，輸出 x 的網路可以用速度空間的損失訓練。稱為「乾淨預測」不代表損失是無加權的乾淨影像 MSE。JiT 的釋出實作清楚呈現此差異 [42]。

$t = 1$ 附近，除以 $1 - t$ 必須有端點慣例與數值保護。就內部時間的代數而言，固定乾淨影像誤差越靠近端點，放大程度越高。取樣時應使用檢查點的時間分布、雜訊縮放與求解器配方。只改直接輸出的標籤，無法重現方法。

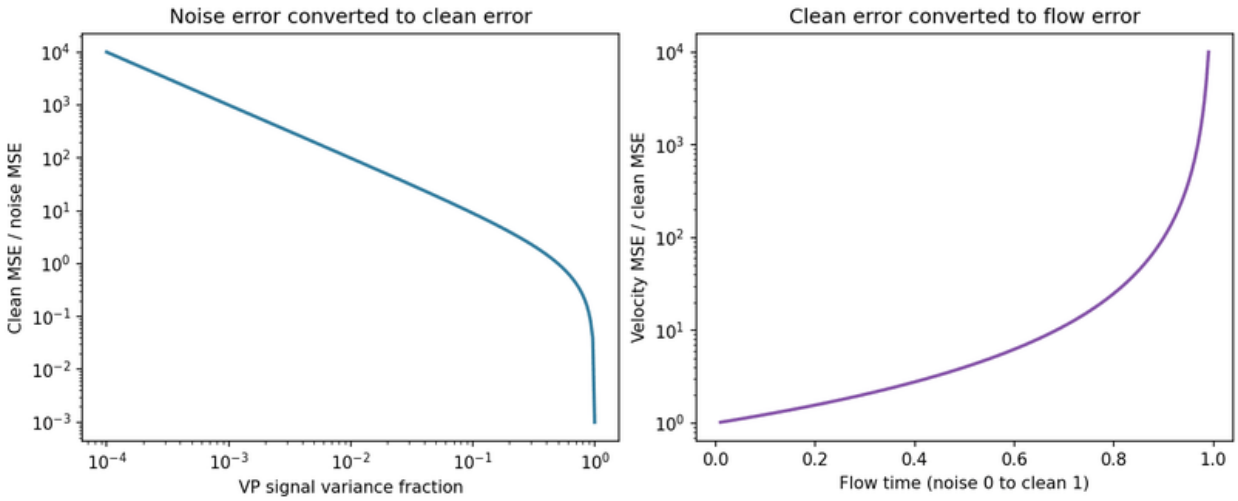


圖 6 — 等價預測使用不同單位。左圖是低訊號下的雜訊誤差放大；右圖是接近乾淨端點時的速度誤差放大。曲線是代數恆等式，而非模型效能測量。

有意義的比較應固定訓練資料、計算量、解析度、詞元數與取樣器，再逐一改變表徵或目標。依重建需求、生成品質與總成本，比較像素 Transformer、緊湊 VAE 潛在 Transformer 與表徵潛在 Transformer。記錄編碼器／解碼器成本與表徵預訓練。即使生成階段從零開始，使用預訓練視覺編碼器的模型仍承接了先前的訓練。

流匹配：學習一個傳輸速度

白話版。先把雜訊排程放一邊。抽一個雜訊點和一個資料點，用直線連起來，訓練網路預測沿著這類直線上每一點的行進方向。生成時，丟下一個新的雜訊點，跟著預測的方向走。

時間慣例改變。本節的時間從 $t = 0$ **的雜訊走到** $t = 1$ **的資料**，與前面擴散各節相反（前面 t 越大代表雜訊越多）。

條件流匹配目標

連續正規化流用一條 ODE $dx_t/dt = v(x_t, t)$ 移動點。如果速度場 v 正確，它會把雜訊分布 $p_0 = \mathcal{N}(0, I)$ 傳輸到資料分布 p_1。Lipman 等人 [10]（以及同時期的 Liu 等人 [11]、Albergo 與 Vanden-Eijnden 的隨機內插 [34]）展示了如何在完全不模擬 ODE 的情況下訓練 v。

把雜訊 $z \sim \mathcal{N}(0, I)$ 與資料 $x \sim p_1$ 配對，定義直線的**條件路徑**

$$x_t = (1 - t)\, z + t\, x, \qquad \frac{dx_t}{dt} = u_t(x_t \mid z, x) = x - z.$$

每一對都以等速沿直線移動。我們真正想要的邊際速度，是通過該點的所有配對之條件速度的平均：

$$u_t(y) = \mathbb{E}[\, x - z \mid x_t = y \,].$$

我們算不出這個平均，但其實也不需要。比較「理想」損失與我們算得出來的損失：

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x_t} \| v_\theta(x_t, t) - u_t(x_t) \|^2, \qquad \mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, z, x} \| v_\theta(x_t, t) - (x - z) \|^2,$$

其中 $t \sim \mathcal{U}[0, 1]$。把兩個平方展開。$\|v_\theta\|^2$ 項完全相同。交叉項也相等，因為由重複期望值定律，$\mathbb{E}[v_\theta(x_t, t)^\top (x - z)] = \mathbb{E}[v_\theta(x_t, t)^\top \, \mathbb{E}[x - z \mid x_t]] = \mathbb{E}[v_\theta^\top u_t(x_t)]$。剩下的項都不含 θ。因此

$$\nabla_\theta \, \mathcal{L}_{\text{CFM}} = \nabla_\theta \, \mathcal{L}_{\text{FM}},$$

也就是說，迴歸每一對的速度 $x - z$ 就能訓練出邊際速度場。這和擴散模型是同一個技巧：一個對單一樣本而言充滿雜訊的目標，其條件平均卻是對的。生成時用任何 ODE 求解器，從 $t = 0$ 到 $t = 1$ 解 $dx/dt = v_\theta(x, t)$。

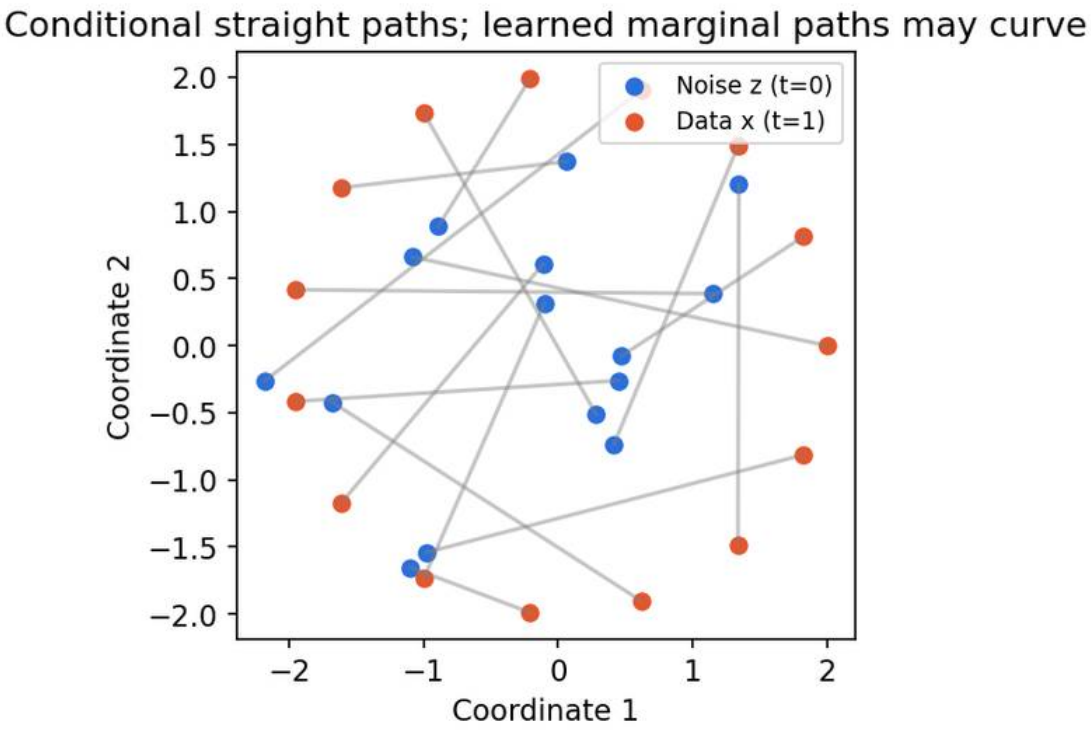


圖 7 — 條件訓練路徑是直線。不同配對的路徑會交叉，所以交叉點的邊際速度是平均值，學到的軌跡可能彎曲。

流匹配與擴散的關係

線性路徑是一條 Gaussian 路徑：給定 x，$x_t \sim \mathcal{N}(t\,x,\ (1-t)^2 I)$。記 $\hat{z} = \mathbb{E}[z \mid x_t]$（一個雜訊預測），並利用 $x = (x_t - (1-t)z)/t$，

$$u_t(x_t) = \frac{x_t - \hat{z}}{t}, \qquad \nabla \log p_t(x_t) = -\frac{\hat{z}}{1-t}, \qquad \text{因此} \qquad u_t(x_t) = \frac{x_t + (1-t)\,\nabla \log p_t(x_t)}{t}.$$

所以對這條路徑而言，流匹配速度、雜訊預測器與分數仍然可以互相換算；不同的是路徑（直線或 VP 曲線）、損失對雜訊程度的加權，以及時間慣例。擴散模型的機率流 ODE 本身就是沿著某條特定 Gaussian 路徑的流。Lipman 等人的指南 [2] 與隨機內插框架 [34] 都把兩者視為同一個建構的特例。實務上的建議與參數化時相同：這些目標是靠推導彼此關聯，而不是改個名字就能互換。

程式：兩個群集上的流匹配

這個獨立範例用線性條件路徑學習與 DDPM 程式相同的兩群集分布，再用步長 $1/N$ 的 Euler 法取樣。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
for step in range(3000):
    z = torch.randn(128, 2)
    side = torch.randint(2, (128, 1))*2-1
    x = torch.cat([1.5*side, torch.zeros(128, 1)], 1) + .25*torch.randn(128, 2)
    t = torch.rand(128, 1)
    xt = (1-t)*z+t*x
    loss = F.mse_loss(net(torch.cat([xt, t], 1)), x-z)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()
def euler_sample(n, steps, seed=1):
    x = torch.randn(n, 2, generator=torch.Generator().manual_seed(seed))
    with torch.no_grad():
        for k in range(steps):
            t = torch.full((n, 1), k/steps)
            x = x + net(torch.cat([x, t], 1))/steps
    return x

for steps in [1, 4, 50]:
    s = euler_sample(2000, steps)
    assert torch.isfinite(s).all()
    left = s[s[:, 0] < 0]
    print(f"{steps:2d} Euler steps: left fraction {(s[:, 0] < 0).float().mean():.3f}, "
          f"left-cluster mean x {left[:, 0].mean():.2f}, std {left[:, 0].std():.2f}")
```

對 1、4、50 步，它印出左群集平均 -0.18、-1.20、-1.45，標準差 0.12、0.42、0.29。一步 Euler 幾乎落在整體資料平均：在 $t = 0$ 時邊際速度是 $\mathbb{E}[x] - z$，所以一整步會把每一個雜訊點都送到 $\mathbb{E}[x]$。直線的條件路徑並不會讓學到的邊際流變直。圖 8 用八模態混合的精確速度呈現同樣的效應。

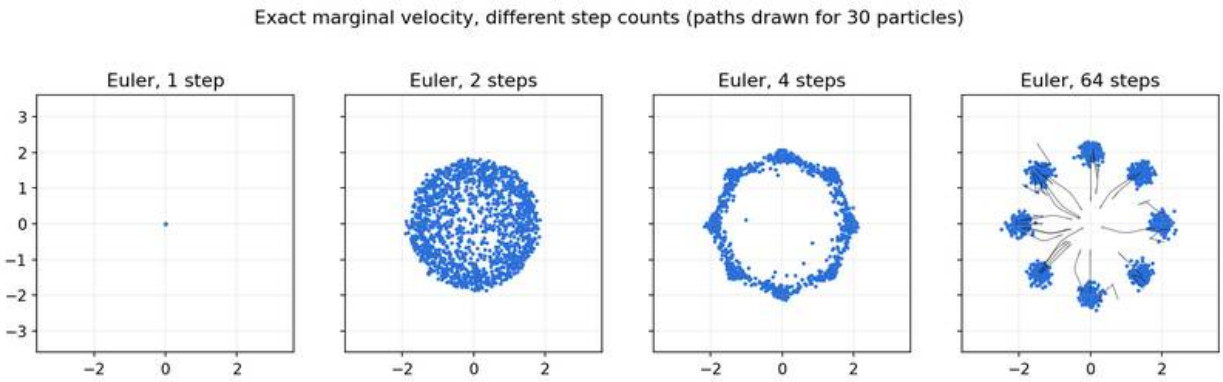


圖 *8* — 即使使用精確的邊際速度，*Euler* 取樣仍需要許多步：一步會把所有樣本塌縮到資料平均，*64* 步的軌跡（黑線）在路徑後段才轉向各模態。

整流流與 reflow

Liu、Gong 與 Liu [11] 使用同樣的直線內插（「整流流」），並加上 **reflow**：訓練後，從每個 z 解學到的 ODE 產生配對 $(z, \hat{x})$，再用這些已耦合的配對（而非獨立配對）重新訓練。因為新配對本來就由流相連，它們的直線較少交叉，學到的軌跡變得更直，需要的 Euler 步數也更少；夠直的流還可以再蒸餾成一步模型。Stable Diffusion 3 [9] 證明了整流流訓練的大型潛在 Transformer 在文字生成影像的規模上可行。

預訓練影像與影片生成器是完整流程

白話版。預訓練生成器是一組相容元件。只載入去雜訊器權重，還不能確定表徵、文字條件、預測目標與取樣排程正確。從影像跨到影片，也會改變重建必須保留的內容。

執行檢查點前，先閱讀元件

實務流程結合文字編碼器、可選的影像／影片編碼器、時間條件預測器、取樣器，以及可選的解碼器。初次生成之後，可能還有獨立的細化模型。每個元件都預期特定張量形狀與縮放。排程器宣告 `v_prediction` 時，可能指 VP 速度；流排程器則可能預期直線內插的導數。應檢查訓練路徑，而不是從設定字串猜測。

以下已核實的例子呈現架構差異。它們是固定的研究基準，不是最新產品排行榜。參考文獻包含論文或技術報告，以及官方釋出文件。

基準	生成表徵與預測器	預訓練流程需要檢查的項目
SDXL [45]	重建潛在表徵與卷積去雜訊器	文字條件、潛在縮放、可選的基礎／細化模型組合與相容取樣設定
Stable Diffusion 3 [9]	潛在整流流與聯合文字／影像 Transformer	流的時間慣例、文字串流、時間抽樣與自編碼器相容性
FLUX.1 [46]	潛在整流流 Transformer	確切釋出版本、引導行為、文字條件、自編碼器與釋出條款
JiT [42]	直接在像素上做類別條件乾淨影像預測	影像正規化、區塊／解析度設定、推得的速度與取樣配方；不是自由文字到影像檢查點
HunyuanVideo [47]	壓縮影片潛在表徵與流匹配 Transformer	影片自編碼器、影像／影片聯合訓練、時間維度與文字條件元件
Wan2.1 [48]	影片擴散 Transformer 與時空自編碼器	文字到影片或影像到影片版本、因果影片壓縮、時間長度與解碼快取

SDXL [45] 說明較大的潛在影像生成架構與可選細化階段。Stable Diffusion 3 [9] 示範相同的大方向如何改用整流流與聯合文字／影像注意力。FLUX.1 的官方模型卡 [46] 描述釋出的整流流 Transformer，`.1-dev` 版本採用引導蒸餾。這些特性不代表檢查點可以互換。一般雙次推論的 CFG 與蒸餾後的引導輸入，可能有不同含義與成本。

影像到影像生成會編碼輸入，並加入部分雜訊，在保留原始內容與改變能力間取捨。實際雜訊程度由排程決定，不只是介面上「strength」這類標籤。比較編輯時，保留相同輸入，記錄初始時間、種子、提示、引導與重建基準。否則可能把重建偽影誤當成文字要求的編輯。

影片加入時間壓縮與一致性問題

令影片有 F 個影格、高度 H 與寬度 W。假想編碼器以 f_t 壓縮時間，以 f_s 壓縮兩個空間軸，約得到 $(F/f_t) \times (H/f_s) \times (W/f_s)$ 個潛在位置。確切形狀取決於填補與首影格的特殊處理。切分區塊後，跨所有位置的密集注意力成本隨剩餘詞元數平方成長；分別壓縮時間與空間，因此可能大幅改變成本。

HunyuanVideo 技術報告 [47] 結合 3D 影片自編碼器、流匹配 Transformer 與分階段影像／影片訓練。Wan 報告 [48] 說明因果影片 VAE，包括首影格特殊處理、空間與時間壓縮，以及分塊編碼／解碼的特徵快取。自編碼器具因果性，不代表整個影片生成器就以自回歸方式取樣；仍須另外檢查生成注意力與取樣程序。

單一影格可能很漂亮，但整段影片可能改變物體身分、突然反轉運動，或忘記被遮蔽的物體。生成前先測試保留影片的重建，再評估外觀、運動、提示對應與跨影格一致性。Melnik 等人 [31] 的綜述提供這些影片特有問題的閱讀地圖。單張影像指標無法診斷時間行為。

預訓練影片生成器也不會自動成為可靠的世界模型。預測合理影片與預測指定動作的後果不同。世界模型章節以狀態、動作、展開預測與評估進一步說明此差異。使用實際的條件化介面，並評估系統宣稱要解決的任務。

可重現的第一個實驗

先記錄流程清單：檢查點版本、編碼器／解碼器版本、文字編碼器、表徵形狀與縮放、預測目標、時間方向、求解器、評估次數、引導、資料型別與種子。若有自編碼器，接著做重建檢查，再對固定的小組提示與種子生成結果。一次比較一項變動。報告潛在方法成本時，納入解碼延遲與記憶體峰值。

本章 CPU 範例不下載權重，用來檢驗基本恆等式與失敗情況。完整解析度的預訓練生成，應依官方釋出說明 [42]、[45]–[48]，在另外配置資源的實驗中執行。實際測量、論文已發表結果與簡化計算，是三種不同證據。

少步與一步生成

白話版。如果取樣器需要 50 次網路呼叫，就訓練第二個模型，用幾次呼叫沿著軌跡跳躍——或者從頭訓練一個本來就設計成會跳的模型。

主要家族有：

- **漸進式蒸餾** [22]：學生模型學著用一步匹配老師的兩個確定性步驟；重複這個程序，每一輪把步數減半。
- **一致性模型** [12]：學習一個函數 $f_\theta(x_t, t)$，把機率流 ODE 軌跡上的任何一點映射到該軌跡的起點（乾淨點）。它必須滿足邊界條件：在最小時間 ϵ 時 $f_\theta(x, \epsilon) = x$，作法是寫成 $f_\theta(x,$

$t) = c_{\text{skip}}(t)\,x + c_{\text{out}}(t)\,F_\theta(x,t)$，並令 $c_{\text{skip}}(\epsilon)=1$、$c_{\text{out}}(\epsilon)=0$。訓練要求同一條軌跡上相鄰兩點的輸出一致。在一致性蒸餾中，相鄰點來自預訓練擴散模型的一步 ODE 求解；在一致性訓練中，則來自同一筆資料在兩個雜訊程度下的加噪版本，不需要老師。取樣只需一步，或幾步「跳到乾淨、重新加噪、再跳」。改進的訓練方法 [32] 簡化了連續時間一致性模型，並在 ImageNet 512 × 512 上把它擴展到 15 億參數、以兩步取樣。

- **對抗式蒸餾**：ADD [33] 結合來自大型預訓練擴散老師的分數蒸餾與對抗損失，讓學生模型在一到四步內產生清晰影像。

- **平均速度模型**：MeanFlow [13] 學習一段區間上的平均速度 $u(x_t, r, t) = \frac{1}{t-r}\int_r^t v(x_s, s)\,ds$，而不是瞬時速度 v。把 $(t-r)\,u = \int_r^t v\,ds$ 對 t 微分，得到恆等式 $u = v - (t-r)\frac{d}{dt}u$，它不需要老師就能提供訓練目標。之後只要對整個區間呼叫一次網路就能生成一個樣本；論文報告在 ImageNet 256 × 256 上從頭訓練、一次評估的 FID 為 3.43 [13]。

少步數是學出來的能力，而不是換個漂亮方程式就免費得到的結果。比較時要計算 NFE（包含引導評估）、潛在模型中解碼器的成本、解析度、品質以及多樣性——也要記得，一步網路每次呼叫可能比多步網路更大、更慢。

評估生成模型

白話版。「看起來好不好？」至少藏了三個問題：單一樣本是否逼真（擬真度）、樣本集合是否涵蓋資料的多樣性（多樣性）、樣本是否符合條件（對齊）。沒有任何單一數字能同時回答三者。

Fréchet Inception Distance

FID（Heusel 等人 [23]）用 Inception-v3 網路 [36] 嵌入真實與生成影像，對兩組特徵各擬合一個 Gaussian，再計算兩個 Gaussian 之間的 Fréchet（2-Wasserstein）距離：

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \operatorname{tr}\!\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right),$$

其中 μ_r, Σ_r 與 μ_g, Σ_g 是真實集合與生成集合的特徵平均與共變異數，tr 是矩陣的跡。越低越好。下面的程式把公式套用在二維「特徵」上。

```
import numpy as np
from scipy import linalg

def frechet_distance(a, b):
    """Frechet distance between Gaussians fitted to feature sets a and b."""
    mu1, mu2 = a.mean(0), b.mean(0)
    c1, c2 = np.cov(a, rowvar=False), np.cov(b, rowvar=False)
    covmean = linalg.sqrtm(c1 @ c2).real
    return float(((mu1 - mu2)**2).sum() + np.trace(c1 + c2 - 2*covmean))

rng = np.random.default_rng(0)
centers = np.array([[-1.5, 0.], [1.5, 0.]])
real = centers[rng.integers(2, size=5000)] + .25*rng.normal(size=(5000, 2))
good = centers[rng.integers(2, size=5000)] + .25*rng.normal(size=(5000, 2))
```

```
one_mode = centers[[0]] + .25*rng.normal(size=(5000, 2))      # mode collapse
blurry = .6*rng.normal(size=(5000, 2))                         # mass in the gap
print("good     :", round(frechet_distance(real, good), 3))
print("one mode :", round(frechet_distance(real, one_mode), 3))
print("blurry   :", round(frechet_distance(real, blurry), 3))
small = [frechet_distance(real[:50], good[i*50:(i+1)*50]) for i in range(20)]
print("good, 50 samples: mean", round(np.mean(small), 3), "std", round(np.std(small), 3))
```

正確的取樣器得到約 0.001，只會產生左邊模態的取樣器得到 3.9，把機率質量放在兩個模態之間的「模糊」取樣器得到 0.976。最後一行顯示一個實務陷阱：只用 50 個樣本時，即使完美的取樣器也得到 0.077 ± 0.073，因為樣本數少時 FID 有向上的偏差。只有在樣本數、參考集合、特徵擷取器與前處理都相同時，才能比較 FID。

FID 之外

- **精確率與召回率**（Kynkäänniemi 等人 [24]）把擬真度與多樣性分開：在特徵空間用 k 近鄰球估計真實與生成資料的「流形」；精確率是落在真實流形內的生成樣本比例，召回率則是落在生成流形內的真實樣本比例。我們的「單一模態」取樣器會有高精確率、低召回率。

- **特徵擷取器很重要**。Stein 等人 [25] 做了大規模的人類評估，發現沒有任何常用指標與人類判斷強烈相關；尤其擴散模型的感知真實度沒有反映在 FID 上。他們建議以 DINOv2-ViT-L/14 [39] 的特徵取代 Inception，並指出現有指標無法把記憶（memorization）與欠擬合或模態收縮區分開來。
- **CMMD**（Jayasumana 等人 [26]）把 Inception 特徵換成 CLIP [38] 嵌入，並把 Gaussian 假設換成使用 Gaussian RBF 核的最大平均差異（MMD），得到一個不偏且更省樣本的估計量。
- **概似**。透過機率流 ODE，擴散模型可以報告精確的對數概似（以每維位元數表示）[4]、[19]。概似衡量的東西與視覺品質不同，兩者可能不一致。
- **條件模型**需要對齊檢查（影像是否呈現提示詞所說的內容？），通常靠人工評分或視覺語言模型；也需要對訓練集做記憶檢查。
- **成本**。報告 NFE、引導評估次數、解析度，以及在註明硬體上的實際時間。

評估章節ML 10 會再回到樣本數效應與誠實比較。

現代觀點

綜述與參考書。文獻很多，以下幾篇回顧是很好的地圖。

- Yang 等人 [1]（*ACM Computing Surveys*，2023）把擴散研究組織成三個改進方向——高效取樣、改進概似估計、特殊結構資料（流形、離散資料、不變性）——並涵蓋與 VAE、GAN、流的結合，以及跨領域應用。
- Croitoru 等人 [27]（*IEEE TPAMI*，2023）聚焦於視覺。他們指出三種通用公式化——DDPM、雜訊條件分數網路與 SDE——並將視覺應用分類，例如生成、超解析度、修補、編輯、轉換與分割。

- Cao 等人 [28]（*IEEE TKDE*，2024）從方法面切入：基礎公式化、演算法改進（取樣加速、目標設計、條件化），以及從影像到生物與醫療的應用。
- Lipman 等人 [2]（2024）是一份附程式碼的流匹配指南，涵蓋 Gaussian 與一般機率路徑、與擴散的關係，以及延伸到離散與黎曼資料。
- Lai、Song、Kim、Mitsufuji 與 Ermon [29]（2025）是一本專書長度的著作，把變分、分數與流三種觀點呈現為通往同一個時間相依速度場的三條路，接著討論引導、快速求解器與流映射（少步）模型。
- Shen 等人 [30]（*TMLR*，2025）從演算法層面（訓練、取樣、壓縮）、系統層面（硬體與服務部署）與框架層面綜述效率問題。
- Melnik 等人 [31]（*TMLR*，2024）綜述影片擴散：架構、時間一致性機制、資料集與評估。

2023–2026 的現況。綜合這些回顧與本章的原始論文，可以看到幾個趨勢。(1) 潛在空間中的 *Transformer*：DiT 類主幹 [8] 在許多大型系統中取代了 U-Net，而以整流流目標訓練的文字–影像聯合 Transformer [9] 成為文字生成影像的標準配方之一。(2) 流匹配成為預設目標：直線路徑的流匹配 [10]、[11] 現在與雜訊預測並用或取而代之，擴散則被理解為機率路徑的一種選擇 [2]、[29]。(3) 更少的評估次數：蒸餾 [22]、[33]、一致性模型 [12]、[32] 與平均速度模型 [13] 把取樣降到一到四次網路呼叫，品質逼近多步取樣器。(4) 更好的評估：社群正從以 Inception 為基礎的 FID 轉向更豐富的特徵空間、人類研究與對齊指標 [25]、[26]。(5) 影像之外：影片、3D、音訊與科學資料都重用同一套機制，其中影片的時間一致性與成本是主要的開放挑戰 [31]。

表徵與預測，2025–2026。 JiT [42] 測試像素上的直接乾淨影像預測；RAE [43] 測試凍結語意特徵空間中的生成；後續文字到影像研究 [44] 檢查更大規模的行為。這些研究區分模型操作的空間與網路直接預測的目標。前述預訓練例子 [45]–[48] 說明實際影像／影片流程為何需要元件相容性與時間重建。比較時應匹配計算量與資料，並納入表徵預訓練。

開放問題。高解析度與長影片的取樣成本；不犧牲多樣性、也不造成過飽和的有原則的引導；與人類判斷一致且能偵測記憶的評估指標；理解泛化（為什麼模型會產生新樣本而不是複製訓練資料）；有保證的可控性與編輯；以及資料來源、授權同意與濫用——這些是社會問題，而不只是技術問題。下一章ML 08 介紹主要的另一條路線：在像素與 token 上的自迴歸生成，以及它與擴散的混合方法。

重點整理

- 前向過程有封閉解 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$，所以訓練就是「挑一個隨機雜訊程度，迴歸雜訊」。
- 變分下界的每一項都是加權的雜訊預測誤差；$\mathcal{L}_{\text{simple}}$ 拿掉權重，樣本變好，但它不是概似。
- 在指定路徑下，雜訊、乾淨影像、VP 速度、分數與流速度可以換算；時間方向、損失加權、端點穩定性與網路容量都重要。JiT 直接輸出乾淨像素，再推得訓練與取樣用的速度。
- 取樣就是解反向 SDE 或 ODE：DDPM、DDIM 與高階求解器用同一個訓練好的網路，在步數與精確度之間取捨；要計算 NFE，而不是「步數」。
- 引導用多樣性換取遵循度，CFG 讓每步成本加倍；非常大的權重會把樣本推離資料分布。
- 像素、重建潛在表徵與 RAE 特徵是不同生成空間。潛在壓縮節省反覆更新的成本，但可能流失資訊；預訓練影像／影片流程需要相容的編碼器、預測器、引導與取樣器。
- 條件流匹配透過迴歸每一對的直線速度來訓練邊際速度，但直線的條件路徑不會讓學到的流變直——少步生成需要 reflow、蒸餾、一致性或平均速度訓練。

- 沒有單一指標能衡量品質：FID 依賴樣本數與特徵擷取器；要搭配精確率／召回率、人類或對齊研究，以及成本。

練習

1. **找回乾淨估計**。從 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ 出發，由雜訊預測推導 $\hat{x}_0$。如果 ϵ_θ 有誤差 δ，在 $\bar{\alpha}_t = 0.25$ 與 $\bar{\alpha}_t = 10^{-4}$ 時，$\hat{x}_0$ 的誤差有多大？接著由乾淨預測推導直線流速度，說明 JiT 為何能輸出 x 卻使用速度損失。

   提示

   $\hat{x}_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta)/\sqrt{\bar{\alpha}_t}$，所以誤差是 $\sqrt{1-\bar{\alpha}_t}\,\delta/\sqrt{\bar{\alpha}_t}$：在 $\bar{\alpha}_t = 0.25$ 時約 $1.7\,\delta$，在 10^{-4} 時約 $100\,\delta$。高雜訊時，$\hat{x}_0$ 是許多可能影像的平均，而不是一張影像。

2. **後驗平均的兩種形式**。把 $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon)/\sqrt{\bar{\alpha}_t}$ 代入 $\tilde{\mu}_t$，證明它等於 $\frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon)$。接著把後驗程式中的 `t` 改成 50 與 900，確認公式仍然吻合。

   提示

   使用 $\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$ 與 $\beta_t = 1-\alpha_t$。x_t 的係數變成 $\frac{\beta_t + \alpha_t(1-\bar{\alpha}_{t-1})}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)} = \frac{1}{\sqrt{\alpha_t}}$，因為分子等於 $1-\bar{\alpha}_t$。

3. **計算成本**。一個潛在文字生成影像模型用 30 步 DDIM 取樣，並以分開的條件與無條件前向計算做無分類器引導，最後跑一次解碼器。總共需要幾次去雜訊器評估？若換成一個已把引導「內建」的 4 步蒸餾模型，次數如何變化？對人工形狀範例，把潛在通道改成 16、像素區塊大小改成 16，重新計算純量與注意力矩陣元素數。哪些節省仍成立，哪些取決於區塊切分？

   提示

   60 次去雜訊器評估加一次解碼器；把兩個 CFG 分支批次處理可以減少實際時間，但不會減少計算量。內建引導的 4 步蒸餾模型需要 4 次評估加解碼器。要比較品質與多樣性，而不只是次數。

4. **時間慣例**。擴散各節使用「t 大＝雜訊多」；流匹配一節使用「$t=0$ 是雜訊、$t=1$ 是資料」。用 $s = 1-t$ 重新參數化這個流。dx/ds 是什麼？如果只改時間輸入的標籤，會出什麼錯？

   提示

   $dx/ds = -v(x, 1-s)$。如果把 s 餵給一個以 t 訓練的網路，卻沒有翻轉正負號與時間輸入，就會朝錯誤方向積分，把樣本推離資料。

5. **引導掃描**。在 CFG 程式中加入 $w \in \{0.5, 1.5, 5\}$，並記錄類別 1 樣本中落得比較靠近錯誤類別模態的比例。畫出到最近模態的平均距離對 w 的關係。引導從哪裡開始不再有幫助？

提示

在 $w = 1$ 時，精確的條件模型就已把每個樣本放進類別 1，所以更大的 w 在這裡無法提升類別正確率——只會讓群集變形。對一個學習出來、並不完美的條件模型而言，適度的 $w > 1$ 可以提升遵循度，這也是實務上使用它的原因。

6. **取樣器消融實驗**。固定訓練好的流匹配網路。把 1、2、5、20、100 步的 Euler 與二階 Heun 步驟（先用 Euler 預測，再平均兩端的速度）比較。使用相同的起始雜訊，測量群集平均與離散程度。在相同網路呼叫次數下，2 步 Heun 取樣器是否勝過 4 步 Euler？

提示

Heun 每步需要兩次網路呼叫。要畫出樣本，而不只看數字。對一個以瞬時速度訓練的模型做一步 Euler，並不能檢驗另外訓練的一步模型（例如一致性模型或 MeanFlow）。

參考文獻

1. L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui and M.-H. Yang, “Diffusion Models: A Comprehensive Survey of Methods and Applications,” *ACM Computing Surveys*, 2023. arXiv
2. Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu and I. Gat, “Flow Matching Guide and Code,” arXiv:2412.06264, 2024. arXiv
3. J. Ho, A. Jain and P. Abbeel, “Denoising Diffusion Probabilistic Models,” *NeurIPS*, 2020. arXiv
4. Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,” *ICLR*, 2021. arXiv
5. J. Song, C. Meng and S. Ermon, “Denoising Diffusion Implicit Models,” *ICLR*, 2021. arXiv
6. J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications; arXiv:2207.12598, 2022. arXiv
7. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” *CVPR*, 2022. arXiv
8. W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” *ICCV*, 2023. arXiv
9. P. Esser, S. Kulal, A. Blattmann et al., “Scaling Rectified Flow Transformers for High-Resolution Image Synthesis,” *ICML*, PMLR 235, 2024. PMLR
10. Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel and M. Le, “Flow Matching for Generative Modeling,” *ICLR*, 2023. arXiv
11. X. Liu, C. Gong and Q. Liu, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow,” *ICLR*, 2023. arXiv
12. Y. Song, P. Dhariwal, M. Chen and I. Sutskever, “Consistency Models,” *ICML*, 2023. arXiv
13. Z. Geng, M. Deng, X. Bai, J. Z. Kolter and K. He, “Mean Flows for One-step Generative Modeling,” *NeurIPS*, 2025. arXiv
14. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. book site
15. J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan and S. Ganguli, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics,” *ICML*, PMLR 37, 2015. PMLR

16. P. Vincent, “A Connection Between Score Matching and Denoising Autoencoders,” *Neural Computation*, vol. 23, no. 7, pp. 1661–1674, 2011. doi
17. Y. Song and S. Ermon, “Generative Modeling by Estimating Gradients of the Data Distribution,” *NeurIPS*, 2019. arXiv
18. P. Dhariwal and A. Nichol, “Diffusion Models Beat GANs on Image Synthesis,” *NeurIPS*, 2021. NeurIPS
19. A. Q. Nichol and P. Dhariwal, “Improved Denoising Diffusion Probabilistic Models,” *ICML*, PMLR 139, 2021. PMLR
20. T. Karras, M. Aittala, T. Aila and S. Laine, “Elucidating the Design Space of Diffusion-Based Generative Models,” *NeurIPS*, 2022. arXiv
21. C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li and J. Zhu, “DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps,” *NeurIPS*, 2022. arXiv
22. T. Salimans and J. Ho, “Progressive Distillation for Fast Sampling of Diffusion Models,” *ICLR*, 2022. arXiv
23. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” *NeurIPS* (*NIPS*), 2017. arXiv
24. T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen and T. Aila, “Improved Precision and Recall Metric for Assessing Generative Models,” *NeurIPS*, 2019. arXiv
25. G. Stein, J. C. Cresswell, R. Hosseinzadeh et al., “Exposing Flaws of Generative Model Evaluation Metrics and Their Unfair Treatment of Diffusion Models,” *NeurIPS*, 2023. arXiv
26. S. Jayasumana, S. Ramalingam, A. Veit, D. Glasner, A. Chakrabarti and S. Kumar, “Rethinking FID: Towards a Better Evaluation Metric for Image Generation,” *CVPR*, 2024. CVF
27. F.-A. Croitoru, V. Hondru, R. T. Ionescu and M. Shah, “Diffusion Models in Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. doi
28. H. Cao, C. Tan, Z. Gao, Y. Xu, G. Chen, P.-A. Heng and S. Z. Li, “A Survey on Generative Diffusion Models,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 2814–2830, 2024. doi
29. C.-H. Lai, Y. Song, D. Kim, Y. Mitsufuji and S. Ermon, “The Principles of Diffusion Models,” monograph, arXiv:2510.21890, 2025. arXiv
30. H. Shen, J. Zhang, B. Xiong et al., “Efficient Diffusion Models: A Survey,” *Transactions on Machine Learning Research*, 2025. arXiv
31. A. Melnik, M. Ljubljanac, C. Lu, Q. Yan, W. Ren and H. Ritter, “Video Diffusion Models: A Survey,” *Transactions on Machine Learning Research*, 2024. arXiv
32. C. Lu and Y. Song, “Simplifying, Stabilizing and Scaling Continuous-Time Consistency Models,” *ICLR*, 2025. arXiv
33. A. Sauer, D. Lorenz, A. Blattmann and R. Rombach, “Adversarial Diffusion Distillation,” *ECCV*, 2024. arXiv
34. M. S. Albergo and E. Vanden-Eijnden, “Building Normalizing Flows with Stochastic Interpolants,” *ICLR*, 2023. arXiv
35. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *CVPR*, 2009, pp. 248–255. doi
36. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, “Rethinking the Inception Architecture for Computer Vision,” *CVPR*, 2016, pp. 2818–2826. arXiv
37. O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional Networks for Biomedical Image

Segmentation," *MICCAI*, LNCS 9351, pp. 234–241, 2015. arXiv
38. A. Radford, J. W. Kim, C. Hallacy et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021. arXiv
39. M. Oquab, T. Darcet, T. Moutakanni et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
40. B. Efron, "Tweedie's Formula and Selection Bias," *Journal of the American Statistical Association*, vol. 106, no. 496, pp. 1602–1614, 2011. doi
41. A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
42. T. Li and K. He, "Back to Basics: Let Denoising Generative Models Denoise," *CVPR*, 2026 (preprint 2025). paper, publication record, official JiT code.
43. B. Zheng, N. Ma, S. Tong and S. Xie, "Diffusion Transformers with Representation Autoencoders," *ICLR*, 2026 (preprint 2025). proceedings, manuscript.
44. S. Tong, B. Zheng, Z. Wang, B. Tang, N. Ma, E. Brown, J. Yang, R. Fergus, Y. LeCun and S. Xie, "Scaling Text-to-Image Diffusion Transformers with Representation Autoencoders," *CVPR Findings Track*, 2026. manuscript, author publication record.
45. D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna and R. Rombach, "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis," *ICLR*, 2024. proceedings, official model card.
46. Black Forest Labs, "FLUX.1 [dev]," official pretrained-model card, 2024. model card.
47. W. Kong, Q. Tian, Z. Zhang et al., "HunyuanVideo: A Systematic Framework For Large Video Generative Models," technical report, arXiv:2412.03603, 2024. report, official release.
48. Team Wan, A. Wang, B. Ai, B. Wen et al., "Wan: Open and Advanced Large-Scale Video Generative Models," technical report, arXiv:2503.20314, 2025. report, official Wan2.1 release.

ML 08・視覺自迴歸：像素、Tokens 與尺度

把影像分布拆成一連串條件機率：PixelCNN、VQ-VAE 與 VQGAN tokenizer、GPT 式 token transformer、next-scale 預測、遮罩生成模型、連續 token，以及它們與擴散模型的比較。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-08-autoregressive-vision/

你將學到

- 機率的鏈鎖法則（chain rule）如何把「建模整張影像」變成一連串「預測下一塊」的問題，以及如何用每維位元數（bits per dimension）衡量結果。
- 為什麼取樣必須逐步進行，訓練卻可以平行；因果遮罩（causal mask）與遮罩卷積（masked convolution）如何讓預測器看不到自己的答案。
- PixelRNN 與 PixelCNN 如何直接對像素建模，以及過長的序列為什麼讓領域轉向學習出來的 token。
- VQ-VAE 與 VQGAN tokenizer 如何把影像轉成一小格整數，包括 codebook 損失、commitment 損失與直通（straight-through）梯度。
- GPT 式 token transformer（Image GPT、Parti、LlamaGen）、next-scale 預測（VAR）、遮罩生成模型（MaskGIT、Muse）與連續 token 模型（MAR）如何改變「預測單位」。
- 取樣控制（temperature、top-k、top-p、classifier-free guidance、KV cache）如何運作，以及自迴歸生成器和擴散模型的比較。

先看全貌

自迴歸（autoregressive, AR）生成器一塊一塊地建立樣本。每一步，它看著目前為止產生的所有內容，預測下一塊的機率分布，從中抽樣，再把結果接到序列後面。這裡的「一塊」可以是一個像素值、學習出來的 codebook 中的一個整數 token、一個連續特徵向量，甚至是下一個解析度的整張圖。

> **用最簡單的話說**
>
> 想像一格一格地畫漫畫。每一格新的畫面都要和前面已經畫好的格子接得上。練習的時候，你臨摹真正的漫畫，所以永遠知道前面幾格長什麼樣子。自己創作新漫畫時，就只能接受自己先前畫的格子，即使有點歪也一樣。自迴歸影像模型就是這樣：用真實影像練習，再一塊一塊地創造新影像。

這為什麼重要？自迴歸正是大型語言模型背後的配方：用 transformer 做下一個 token 預測。把影像帶進這套配方，就能用同一種架構讀寫文字與圖片、沿用語言模型的部署工程，並用精確的似然（likelihood）衡量進展。代價是影像本來就不是序列，必須有人決定**什麼是 token**，以及**token 用什麼順序產生**。本章大部分內容都在討論這兩個決定。

Xiong 等人的綜述 [1] 依照預測單位，把視覺 AR 模型分成三大類：**以像素為基礎**、**以 token 為基礎**、**以尺度為基礎**。我們沿用這張地圖，再加上遮罩式（非由左至右）與連續 token 模型，它們模糊了 AR 與擴散之間的界線。前面關於生成模型與擴散與 flow matching的章節提供本章用到的 VAE 與擴散背景；Prince 的教科書 [9] 則以教材層級涵蓋一般的機率基礎。

鏈鎖法則不需要獨立假設

白話版：任何聯合分布都可以寫成一連串「給定前面元素，下一個元素的分布」的乘積。這是機率論的事實，而不是建模假設。

對於有順序的序列 $x = (x_1, \dots, x_N)$，

$$p_\theta(x) = \prod_{i=1}^{N} p_\theta(x_i \mid x_{<i}), \qquad \mathcal{L}_{\text{NLL}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}).$$

其中 x_i 是所選順序中的第 i 個元素，$x_{<i} = (x_1, \dots, x_{i-1})$ 是它之前的所有元素，θ 是網路參數，$\mathcal{L}_{\text{NLL}}$ 是單一訓練樣本的負對數似然（實務上會對整個資料集取平均）。

鏈鎖法則對**任何**順序都精確成立。近似只來自模型本身：有限的網路、有限的上下文視窗，或對每個條件分布形式的限制。離散值的條件分布通常是 softmax；二值像素可用 Bernoulli 分布；連續特徵則需要密度函數或另一個條件生成器（MAR 一節會看到一個例子）。

順序是計算上的選擇，不是對物理世界的主張。**光柵順序（raster order）**（由左到右、由上到下）很方便，但對二維影像來說是任意的：你下方的像素和右邊的像素一樣相關。不同順序會產生不同的條件分布，而有限的模型可能覺得某些順序比較容易學。圖 1 展示本章用到的三種預測單位。

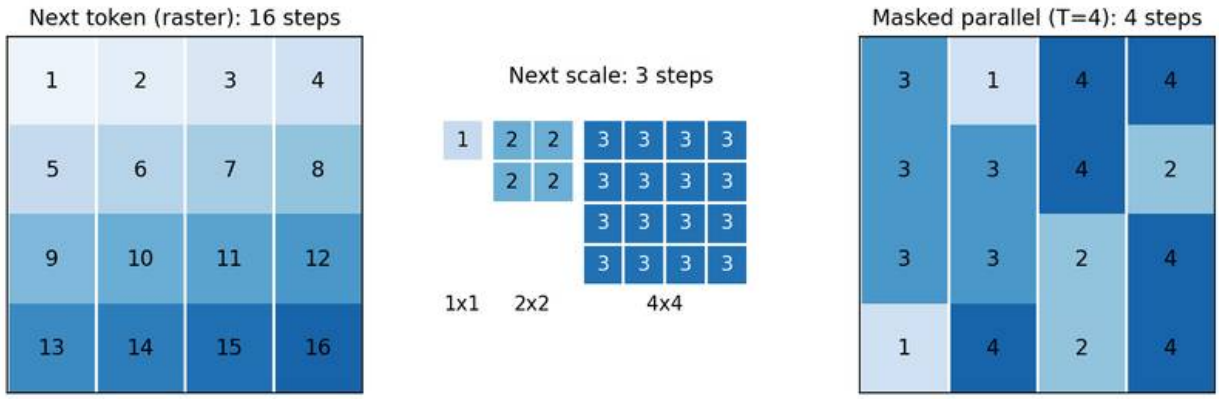


圖 *1* — 以三種生成方式產生同樣 *16* 個最終 *token*。數字代表產生該 *token* 的循序步驟：光柵自迴歸需要 *16* 步，*next-scale* 預測需要 *3* 步（每個尺度一步），$T = 4$ 的遮罩平行解碼需要 *4* 步。僅為示意圖，沒有訓練任何模型。

衡量 AR 模型：每維位元數

因為鏈鎖法則給出精確的 $\log p_\theta(x)$，AR 模型通常用**每維位元數（bits per dimension, bpd）**表示的負對數似然來比較：

$$\text{bpd}(x) = \frac{-\log_2 p_\theta(x)}{D},$$

其中 D 是影像中離散值的個數（例如 32×32 的 RGB 影像 $D = 3 \cdot 32 \cdot 32$），$\log_2$ 把 nat 換成位元。如果模型對每個 8 位元值都給均勻機率，分數恰好是 8 bpd；任何更低的值都代表模型學到了結構。bpd 在保留（held-out）影像上量測，因此同時能偵測過擬合（overfitting）與欠擬合。

有兩點要注意。第一，bpd 和視覺品質可能不一致：模型可能把大部分似然花在肉眼看不到的雜訊上，卻在整體構圖上犯奇怪的錯。第二，對 **token** 模型而言，似然是定義在 codebook 索引上，而非像素上。有損的 tokenizer 會丟掉資訊，所以 token 層級的似然不是精確的像素空間似然，不能直接和像素模型的 bpd 比較。

Teacher forcing、因果遮罩與曝光偏差

白話版：訓練時，模型永遠看到真實的歷史；取樣時，它看到的是自己產生的歷史。遮罩確保訓練過程中沒有任何位置能偷看自己的答案。

使用 **teacher forcing** 時，訓練損失中的每個條件分布都以真實的前文計算。由於整張訓練影像都在手上，只要禁止位置 i 讀取位置 $\geq i$，所有 N 個條件分布就能在一次前向傳遞中算完。因果 transformer 用注意力遮罩做到這一點；PixelCNN 則用遮罩過的卷積核。

常見的輸入慣例有兩種，遮罩必須與慣例相符：

- 在**位移輸入（shifted-input）**的解碼器中，輸入 `[BOS, x1, x2, ...]`，預測 `[x1, x2, x3, ...]`。這時包含對角線的下三角遮罩是正確的，因為位置 i 的輸入是前一個元素。
- 在接收**未位移**值的模型中，依賴關係必須**嚴格**在前。若包含對角線，網路就能直接複製它該預測的值。

推論時，先前抽樣的錯誤會進入後續步驟的上下文。這種訓練與推論的落差常稱為**曝光偏差（exposure bias）**。它不會讓最大似然變成錯的（分解依然精確），但對不完美的模型和長序列確實有實務影響：影像版面一開始的小錯，可能把後續步驟推進訓練中從未出現過的情境。

像素模型：PixelRNN 與 PixelCNN

白話版：把每個像素的強度當成 256 類的分類問題，輸入是它上方和左方的所有像素。

PixelRNN 與 PixelCNN 由 van den Oord、Kalchbrenner 與 Kavukcuoglu 在同一篇論文中提出 [2]。兩者都用 **256 類 softmax** 而不是高斯分布來建模每個 8 位元值，因此分布可以是多峰的（像素可以「要嘛黑、要嘛白，但不是灰」）。色彩則在每個像素內部採固定順序：先紅、再綠、再藍，讓綠色可以依賴紅色，藍色可以依賴兩者。PixelRNN 使用二維 LSTM 層（Row LSTM 與 Diagonal BiLSTM），依賴範圍可能沒有上限；PixelCNN 則以堆疊的遮罩卷積取代，感受野有限，但因為所有位置可以平行計算，訓練快得多。

遮罩卷積

遮罩卷積把卷積核乘上一個二值遮罩，將「未來」位置的權重設為零。在 [2] 中，**遮罩 A** 只用在第一層：它排除中心像素，所以第一層永遠看不到它要預測的值（彩色時，也只看得到已預測的通道）。**遮罩 B** 用在之後的所有層：它允許中心位置，因為第一層之後，中心的特徵已經只匯總了較早的像素。正式地說，對中心在 (c, c)、$c = \lfloor k/2 \rfloor$ 的 $k \times k$ 卷積核，

$$M_{uv} = \begin{cases} 1 & u < c, \\ 1 & u = c \text{ and } v < c \text{ (A) or } v \le c \text{ (B)}, \\ 0 & \text{otherwise}, \end{cases}$$

其中 u 是卷積核的列索引，v 是行索引。下面的程式建立兩種遮罩、堆疊五層，並用梯度找出 9×9 影像中，哪些輸入像素能影響中心位置的輸出。

```
import torch
from torch import nn
from torch.nn import functional as F

class MaskedConv2d(nn.Conv2d):
    """PixelCNN-style masked convolution for one channel (raster order)."""
    def __init__(self, cin, cout, k, mask_type):
        super().__init__(cin, cout, k, padding=k//2, bias=False)
        mask = torch.ones(k, k)
        c = k//2
        mask[c, c + (mask_type == "B"):] = 0   # centre (type A) and pixels to its right
        mask[c+1:] = 0                         # all rows below
        self.register_buffer("mask", mask)
    def forward(self, x):
        return F.conv2d(x, self.weight*self.mask, padding=self.padding)

torch.manual_seed(0)
layers = [MaskedConv2d(1, 8, 3, "A"), nn.Tanh()]
for _ in range(4):                             # 5 layers: reach of 5 pixels
    layers += [MaskedConv2d(8, 8, 3, "B"), nn.Tanh()]
net = nn.Sequential(*layers, MaskedConv2d(8, 1, 1, "B"))
print(net[0].mask.int())
x = torch.rand(1, 1, 9, 9, requires_grad=True)
net(x)[0, 0, 4, 4].backward()                  # logit for pixel (row 4, col 4)
dep = (x.grad[0, 0].abs() > 0).int()
print(dep)
later = torch.zeros(9, 9, dtype=torch.bool)
later[4, 4:] = True; later[5:] = True
assert dep[later].sum() == 0                   # never reads itself or later pixels
print("earlier pixels never seen (blind spot):", int(((dep == 0) & ~later).sum()))
```

assert 通過：目標位置本身及之後的像素都不會影響它的 logit，所以模型是合法的自迴歸分解。但輸出還揭露了另一件事：有六個在光柵順序中較早的像素，在目標右上方排成階梯狀，梯度為零。這就是堆疊遮罩卷積的**盲點（blind spot）**（圖 2）：每一層的感受野往左上方擴張，右上方的區域卻永遠到不了。模型依然正確，只是得到的資訊比它能得到的少。

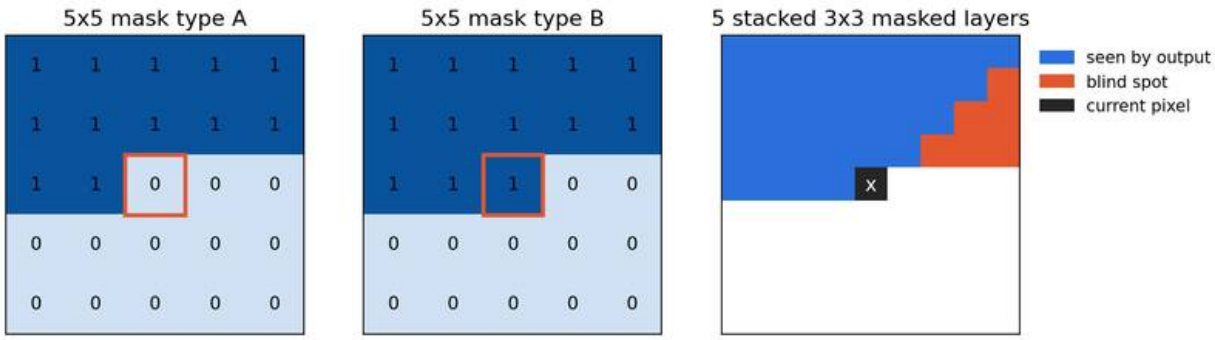


圖 2 — 左、中：遮罩 *A* 排除中心權重，遮罩 *B* 保留中心權重。右：疊了五層 3×3 遮罩卷積後，用梯度量出哪些輸入像素會影響深色格子的預測。橘色格子在光柵順序中較早，卻永遠看不到，也就是盲點。

van den Oord 等人的 Gated PixelCNN [3] 把網路拆成兩個堆疊來消除盲點：**垂直堆疊**不受限制地看到目前這列以上的所有列，**水平堆疊**看到目前這列中直到目前像素為止的部分，兩者在每層之後合併。它也把 ReLU 換成閘控單元 $y = \tanh(W_f * x) \odot \sigma(W_g * x)$，其中 $*$ 是卷積，σ 是 logistic sigmoid，$\odot$ 是逐元素乘法；並示範如何以類別標籤或嵌入向量為條件。加上這些改變後，PixelCNN 在 ImageNet [23] 上達到與 PixelRNN 相當的似然，計算成本卻低得多 [3]。

程式：最小的自迴歸模型

遮罩卷積只是強制因果性的一種方式，也不一定需要 transformer。以下完整的 CPU 實驗產生 4×4 二值影像，每一列都重複同樣四個隨機位元。一個帶遮罩的線性層，用嚴格下三角遮罩排除目前像素與之後所有像素，直接學習條件分布。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
class BinaryAR(nn.Module):
    def __init__(self, n=16):
        super().__init__()
        self.weight = nn.Parameter(torch.zeros(n, n))
        self.bias = nn.Parameter(torch.zeros(n))
        self.register_buffer("mask", torch.tril(torch.ones(n, n), diagonal=-1))
    def forward(self, x):
        return F.linear(x, self.weight*self.mask, self.bias)

model = BinaryAR()
opt = torch.optim.Adam(model.parameters(), lr=.04)
for step in range(600):
    first_row = torch.randint(2, (128, 4)).float()
    x = first_row.repeat(1, 4)
    loss = F.binary_cross_entropy_with_logits(model(x), x)
    opt.zero_grad(); loss.backward(); opt.step()
```

```
with torch.no_grad():
    a = torch.rand(2, 16)
    b = a.clone(); b[:, 8:] = torch.rand(2, 8)
    assert torch.allclose(model(a)[:, :9], model(b)[:, :9])
    samples = torch.zeros(8, 16)
    for i in range(16):
        p = model(samples)[:, i].sigmoid()
        samples[:, i] = torch.bernoulli(p)
    rows = samples.reshape(-1, 4, 4)
    print("one generated image:\n", rows[0].int())
    print("repeat agreement:", (rows[:, 1:] == rows[:, :1]).float().mean().item())
```

assert 檢查因果性：改變第 8 個位置以後的輸入，不會改變位置 0 到 8 的 logit，因為這些輸出只讀較早的輸入。訓練之後，後面各列與第一列的一致率達到 1.0，而第一列仍保持隨機（模型不應該背下某一列）。

真實分布有 16 張機率相同的影像，所以熵是每張 $\log_2 16 = 4$ 位元，也就是每像素 0.25 位元。後面幾列不含任何新的不確定性。若模型把 16 個像素都當成獨立的公平硬幣，每張影像需要 16 位元。16 與 4 位元之間的差距，正是自迴歸模型能捕捉、而獨立像素模型捕捉不到的結構。

為什麼像素模型擴展不下去

一張 256×256 的 RGB 影像有 $3 \cdot 256 \cdot 256 = 196{,}608$ 個值，所以像素層級的取樣需要大約二十萬次循序的網路計算。大部分似然也被花在細微、局部容易預測的細節上，而人們真正注意的整體構圖只占位元的一小部分。Image GPT [11] 證明，在低解析度影像上以自迴歸方式預測像素的純序列 transformer，學到的特徵在線性探測（linear probing）下遷移效果很好；但在較高解析度下，像素路線依然太昂貴。下一步就是學習更短、更有意義的序列。

離散視覺 token：VQ-VAE 與 VQGAN

白話版：另外訓練一個網路，把影像壓縮成一小格來自學習字典的「單字」，然後對單字做自迴歸，而不是對像素。

VQ-VAE

VQ-VAE [4] 有一個編碼器，在降取樣網格的每個位置輸出特徵向量 $z_e(x) \in \mathbb{R}^d$；一個由 K 個學習向量組成的 **codebook** $\{e_1, \dots, e_K\} \subset \mathbb{R}^d$；以及一個解碼器。每個特徵都被替換成最近的 code：

$$k^\star = \arg\min_k \|z_e(x) - e_k\|_2^2, \qquad z_q(x) = e_{k^\star}.$$

影像於是由 $k^\star$ 值組成的整數網格表示。訓練目標是

$$\mathcal{L}_{\text{VQ}} = \underbrace{\|x - D(z_q)\|^2}_{\text{reconstruction}} + \underbrace{\|\text{sg}[z_e] - e_{k^\star}\|^2}_{\text{codebook}} + \beta \underbrace{\|z_e - \text{sg}[e_{k^\star}]\|^2}_{\text{commitment}},$$

其中 D 是解碼器，sg[·] 是**停止梯度（stop-gradient）**運算子（前向傳遞時是恆等函數，反向傳遞時梯度為零），β 是 commitment 項的小權重（下面的程式用 0.25）。這裡把重建項寫成平方誤差；也可以換成任何重建似然。

三項各司其職。**codebook** 項把選中的 code 拉向編碼器輸出，就像 k-means 的更新。**commitment** 項把編碼器拉向它的 code，讓編碼器輸出不會離 codebook 太遠。`argmin` 沒有有用的梯度，所以重建損失的梯度要透過**直通估計器（straight-through estimator）**傳到編碼器：前向傳遞使用 z_q，反向傳遞則把 z_q 的梯度原封不動複製給 z_e，彷彿量化是恆等函數。在程式中只要一行：`z_q = z_e + (e - z_e).detach()`。

下面的玩具例子在四個群集上訓練一個二維「編碼器」、codebook 與解碼器，共有八個 code 可用：

```
import torch
from torch import nn
from torch.nn import functional as F

def quantize(z_e, codebook, beta=0.25):
    """z_e: (n, d) encoder outputs; codebook: (K, d). Returns z_q with straight-through
     ↪  gradient."""
    dist = torch.cdist(z_e, codebook)            # (n, K) Euclidean distances
    idx = dist.argmin(1)                         # nearest code index per vector
    e = codebook[idx]
    codebook_loss = F.mse_loss(e, z_e.detach())  # moves codes toward encoder outputs
    commit_loss = F.mse_loss(z_e, e.detach())    # keeps the encoder near its code
    z_q = z_e + (e - z_e).detach()               # forward: e; backward: identity
    return z_q, idx, codebook_loss + beta*commit_loss

torch.manual_seed(0)
torch.set_num_threads(1)
centres = torch.tensor([[2., 0.], [-2., 0.], [0., 2.], [0., -2.]])
encoder = nn.Linear(2, 2)                        # toy "encoder" and "decoder"
decoder = nn.Linear(2, 2)
codebook = nn.Parameter(torch.randn(8, 2)*.1)    # K = 8 codes, 4 are enough
opt = torch.optim.Adam([*encoder.parameters(), *decoder.parameters(), codebook], lr=.03)
for step in range(800):
    x = centres[torch.randint(4, (256,))] + .1*torch.randn(256, 2)
    z_q, idx, vq_loss = quantize(encoder(x), codebook)
    loss = F.mse_loss(decoder(z_q), x) + vq_loss
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    x = centres.repeat_interleave(100, 0) + .1*torch.randn(400, 2)
    z_q, idx, _ = quantize(encoder(x), codebook)
    print("reconstruction MSE:", round(F.mse_loss(decoder(z_q), x).item(), 4))
    print("codes used:", idx.unique().numel(), "of", len(codebook))
    print("codes per cluster:", [idx[i*100:(i+1)*100].unique().tolist() for i in range(4)])
z = torch.randn(3, 2, requires_grad=True)
z_q, _, _ = quantize(z, codebook.detach())
z_q.sum().backward()
```

```
print("straight-through gradient:", z.grad[0].tolist())
```

輸出顯示重建 MSE 約 0.0096，只用了 **8 個 code 中的 4 個**（每個群集一個），直通梯度是 `[1.0, 1.0]`。這些數字藏著兩個教訓。重建誤差幾乎正好等於群集內的雜訊變異數 $0.1^2 = 0.01$：量化保留了「屬於哪一群」，丟掉了「在群內哪個位置」。另外，一半的 codebook 是**死碼（dead code）**：從未被選到的 code 收不到 codebook 梯度，永遠不會移動（圖 3）。在真正的 tokenizer 中，codebook 使用率低會浪費容量，是需要監控的典型失敗。

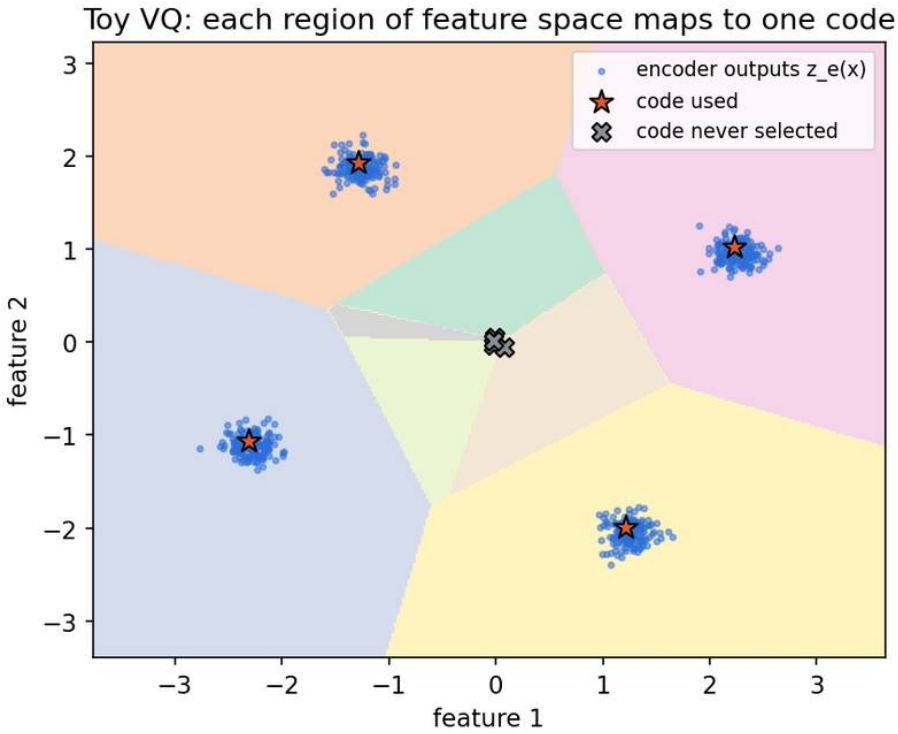


圖 *3* — 玩具向量量化。每個色塊是會被映射到同一個 *code* 的特徵集合。四個 *code*（星形）被使用，每群一個；另外四個（叉號）從未被選中，停留在初始化位置附近。

兩個階段：先 tokenizer，再先驗

VQ-VAE 本身只是自編碼器，並不會告訴你哪些 token 網格比較可能出現。因此生成分成**兩個階段**（圖 4）。第一階段訓練 tokenizer。第二階段把它凍結，把每張訓練影像轉成索引網格，再對 M 個索引訓練自迴歸模型 $p_\psi(k_1, \dots, k_M)$。取樣時，先從先驗抽出索引網格，再解碼。[4] 中的先驗是建立在 code 上的 PixelCNN；VQ-VAE-2 [10] 使用兩層 code 圖（較粗的上層與較細的下層）搭配 PixelCNN 式先驗，產生了多樣且高保真度的 ImageNet 樣本。

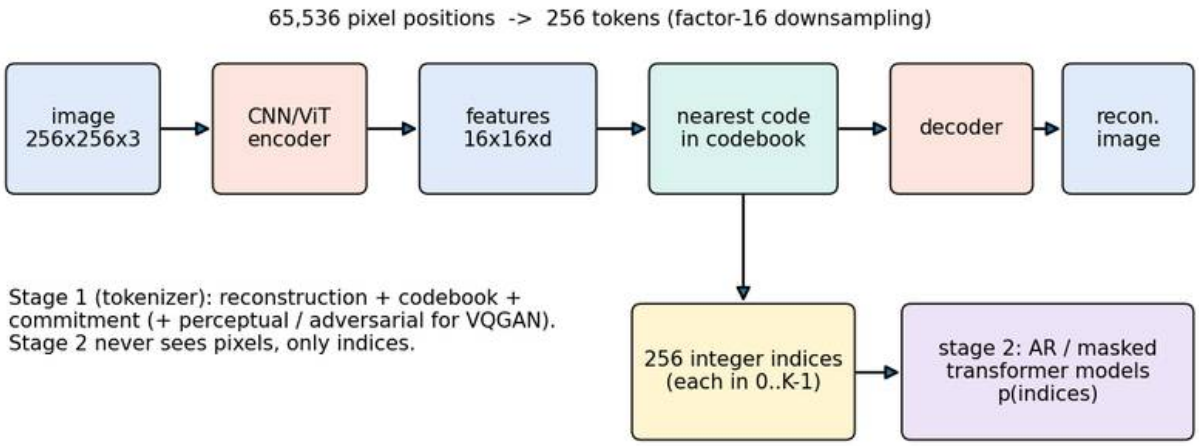


圖 *4* — 兩階段生成。*tokenizer*（上排）為重建而訓練；先驗（右下）只看得到整數索引。以 *16* 倍降取樣，一張 *256×256* 影像變成 *256* 個 *token*，而不是 *65,536* 個像素位置。

簡單的算術說明了它的吸引力。若 tokenizer 把 256×256 影像在每個方向降取樣 16 倍，先驗看到的是 $16 \times 16 = 256$ 個 token，而不是 65,536 個像素位置（或 196,608 個色彩值）。自注意力的成本隨序列長度平方成長，這就是可行與不可行之間的差別。

VQGAN：感知導向的 token

Esser、Rombach 與 Ommer 的 VQGAN [5] 保留兩階段設計，但訓練 tokenizer 時除了重建損失，還加上**感知損失（perceptual loss）**與**以區塊為單位的對抗損失（patch-based adversarial loss）**，並用 transformer 對 token 網格建模。對抗與感知項讓解碼器能從很少的 token 產生清晰、逼真的紋理，因為 token 不再需要精確編碼每個像素，只需要編碼足夠的資訊，讓解碼器產生一個合理的版本。這就是 VQGAN 式 tokenizer 成為 token 式生成標準前端的原因。注意分工：**tokenizer** 使用對抗訓練，**先驗**則是用最大似然訓練的普通自迴歸模型。

Codebook 的問題與替代方案

真實的 tokenizer 反覆遇到三個問題：死碼（如玩具例子）、tokenizer 能重建的內容與先驗能建模的內容之間的落差，以及文字、細線等小細節的遺失。幾種設計分別回應這些問題：

- **ViT-VQGAN** [12] 使用 Vision Transformer 的編碼器與解碼器，在低維的**分解（factorized）**空間中查找 code，並對 code 與特徵做 ℓ_2 **正規化**，改善了 codebook 使用率與重建品質。
- **RQ-VAE** [15] 使用**殘差量化（residual quantization）**：每個位置由一小疊 code 表示，每個 code 量化前面幾個 code 沒表示到的部分。這讓空間網格可以更粗（序列更短），保真度卻相近。
- **FSQ** [16]（有限純量量化，finite scalar quantization）把特徵投影到少數幾維，每一維四捨五入到少數固定的值。隱含的 codebook 大小是各維層級數的乘積；沒有會崩潰的學習 codebook，也不需要 commitment 損失。

壓縮一定會造成瓶頸。請**分別**量測 codebook 使用率、重建品質與生成器品質：生成器無法產生 tokenizer 表示不了的細節。

建立在 token 上的自迴歸 transformer

白話版：一旦影像變成整數序列，生成它就和生成文字是同一個問題：用僅解碼器（decoder-only）的 transformer 預測下一個 token。

從像素到 token

Image GPT [11] 把 GPT 式 transformer 直接用在低解析度像素序列上。它的主要發現是關於**表**

徵學習：只訓練來預測下一個像素的模型，其特徵在上面訓練線性分類器時遷移得很好。改用 token 而不是像素，序列短到足以在更高解析度下生成。VQGAN transformer [5] 是經典例子：一個在 VQGAN 索引上以光柵順序運作的類 GPT 模型，讓 transformer 能合成若以像素序列表示會長得無法處理的高解析度影像。

把文字生成影像當成翻譯：Parti

Parti [13] 把文字生成影像視為機器翻譯。transformer **編碼器**讀取文字提示；transformer **解碼器**以它為條件，自迴歸地生成 ViT-VQGAN 影像 token [12]。報告把這個序列到序列（sequence-to-sequence）模型擴展到 200 億參數，並提出 PartiPrompts：一個涵蓋多種內容類別、超過 1,600 個英文提示的基準。

原封不動的語言模型架構：LlamaGen

LlamaGen [14] 問的是：沒有任何影像專屬歸納偏置的標準語言模型架構，擴大規模後能不能把影像生成好？它在 16 倍降取樣、$K = 16{,}384$ 個 8 維 code 的 VQ tokenizer 上，使用 Llama 式 [24] 解碼器（RMSNorm 前置正規化、SwiGLU 前饋層、二維旋轉位置編碼）。類別標籤或投影後的文字嵌入作為**前綴（prefix）**token 輸入，之後以光柵順序生成。論文釋出 1.11 億到 31 億參數的類別條件模型，並指出標準的 LLM 推論服務軟體（vLLM）能加速推論。它更廣的訊息是架構重用：除了 tokenizer，影像生成器幾乎可以和文字模型共用一切。

自迴歸模型的 classifier-free guidance

Classifier-free guidance（CFG）[18] 原本是為擴散模型提出的，但可以直接搬到 AR 的 logit 上。訓練時，條件 c 會隨機被替換成學習出來的**空（null）**嵌入，讓同一個網路同時學會條件與無條件預測。取樣的每一步計算兩組 logit 再組合：

$$\ell_{\mathrm{g}} = \ell_{\mathrm{u}} + s\,(\ell_{\mathrm{c}} - \ell_{\mathrm{u}}),$$

其中 ℓ_c 是有條件時的下一個 token logit，ℓ_u 是空條件時的 logit，s 是 guidance 強度；$s = 1$ 就回到普通的條件模型。LlamaGen 使用的正是這個形式 [14]。較大的 s 通常提升對提示或類別的貼合度與樣本銳利度，代價是多樣性。除非把兩次計算合併成一個批次，否則它也會讓每步計算量加倍，所以只看步數會低估成本。

Next-scale 預測（VAR）

白話版：不再依固定閱讀順序預測下一個 token，而是在所有較粗版本的條件下，預測整張影像下一個更清晰的版本。

光柵順序會造成奇怪的依賴：影像左上角在完全不知道右下角的情況下被生成。VAR（Visual AutoRegressive modeling）[6] 主張這種單向順序違背影像 token 之間的雙向相關與空間局部性，因此改用解析度遞增的 token 圖 $r_1, \dots, r_K$ 上的**由粗到細**順序：

$$p(r_1, \dots, r_K) = \prod_{k=1}^{K} p(r_k \mid r_1, \dots, r_{k-1}),$$

其中 r_k 是 $h_k \times w_k$ 的 token 索引網格，r_1 是 1×1，r_K 是完整的潛在解析度。在較粗的圖給定之下，r_k 的所有 token **平行**預測，所以循序步數等於尺度數，而不是 token 數。

tokenizer 是方法的一部分。VAR 的多尺度 VQ 編碼器以**殘差**方式運作：從編碼器特徵圖 f 開始，對 $k = 1, \dots, K$，把 f 降取樣到 $h_k \times w_k$，量化得到 r_k（所有尺度共用一個 codebook），把查到的 code 升取樣回完整解析度，經過一個小卷積 ϕ_k，再從 f 減掉，然後進入下一個尺度 [6]。因此每個尺度編碼的是較粗尺度還沒解釋的部分。單純把普通 token 圖縮放並不能重現這個方法。

訓練時，**區塊式因果（block-wise causal）**注意力遮罩讓 r_k 的每個 token 可以注意 $r_{\le k}$ 的所有 token，但看不到更細的尺度；推論時以 KV cache 儲存較粗的尺度。VAR 也使用 CFG。論文報告在類別條件的 ImageNet 256×256 上，VAR 把其 AR 基準的 FID 從 18.65 改善到 1.73，推論也更快，而且模型呈現類似語言模型的冪律擴展趨勢 [6]。VAR 獲得 NeurIPS 2024 最佳論文獎。和往常一樣，這些數字只適用於論文的資料、解析度與訓練設定。

尺度內的平行預測本身就是一個假設：r_k 中的 token 在給定 $r_{<k}$ 時被獨立抽樣。遮罩模型一節會在玩具問題上展示這個假設的代價。

遮罩生成模型（MaskGIT 與 Muse）

白話版：從一格全是空白的網格開始。每一輪同時猜所有空格，只保留最有把握的猜測，其餘的再塗回空白。

MaskGIT [8] 使用與 AR 模型相同的 VQ token，但改用能看到所有位置的**雙向** transformer。訓練目標是遮罩 token 預測：選一個遮罩比例，把這個比例的 token 換成特殊的 `[MASK]` token，然後在被遮住的位置上最小化交叉熵，

$$\mathcal{L}_{\text{mask}} = -\mathbb{E}\Big[\sum_{i \in M} \log p_\theta(y_i \mid y_{\bar{M}}) \Big],$$

其中 y 是 token 網格，$\mathcal{M}$ 是被遮住的位置集合，$y_{\bar{\mathcal{M}}}$ 是沒被遮住的 token。遮罩比例取為 $\gamma(r)$，其中 $r \sim \mathcal{U}(0,1)$，**遮罩排程（mask schedule）** γ 從 1 遞減到 0。

生成進行 T 次迭代。第 t 次迭代時，模型平行預測並抽樣所有被遮住的 token；每個樣本的機率就是它的**信心（confidence）**。這次迭代後仍保持遮住的 token 數為

$$n_t = \lceil \gamma(t/T)\, N \rceil,$$

N 是 token 總數；信心最低的 n_t 個新 token 會被重新遮住，其餘的則固定下來。MaskGIT 比較了數種排程，選擇**餘弦**排程 $\gamma(r) = \cos(\pi r/2)$：早期（模型所知甚少時）只固定少數 token，後期才固定較多；它以 16×16 的 token 網格和 1,024 個條目的 codebook，在 8 次迭代內生成 256×256 影像 [8]。Muse [17] 把同樣的想法擴展到文字生成影像，以預訓練大型語言模型的文字嵌入作為遮罩 token transformer 的條件。

MaskGIT **不是**一般由左到右的鏈鎖法則自迴歸：順序是依信心即時決定的，而且同一步會抽樣好幾個 token。這些同時產生的 token 在目前上下文給定下是**獨立**抽樣的。下面的玩具例子顯示這何時會造成影響。它在前面那個重複列的 4×4 影像上訓練一個小型雙向網路，並用餘弦排程解碼。

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
N, MASK = 16, 2                                  # 16 binary tokens; state 2 = [MASK]
gamma = lambda r: math.cos(math.pi*r/2)          # cosine schedule: fraction still masked
net = nn.Sequential(nn.Flatten(), nn.Linear(3*N, 128), nn.ReLU(), nn.Linear(128, N))

def data(n):                                     # 4x4 images whose rows repeat
    return torch.randint(2, (n, 4)).repeat(1, 4)

def logits(tokens):                              # sees every position: bidirectional
    return net(F.one_hot(tokens, 3).float())

opt = torch.optim.Adam(net.parameters(), lr=3e-3)
for step in range(1500):
    x = data(256)
    ratio = torch.tensor([gamma(r) for r in torch.rand(256).tolist()])
    n_mask = (ratio*N).ceil().clamp(min=1)
    masked = torch.rand(256, N).argsort(1) < n_mask[:, None]   # random positions
    inp = torch.where(masked, MASK, x)
    loss = F.binary_cross_entropy_with_logits(logits(inp)[masked], x[masked].float())
    opt.zero_grad(); loss.backward(); opt.step()

@torch.no_grad()
def generate(n, T=4):
    tokens = torch.full((n, N), MASK)
    for t in range(1, T+1):
        p = logits(tokens).sigmoid()
        sample = torch.bernoulli(p).long()
```

```
        conf = torch.where(sample == 1, p, 1-p)              # prob. of the drawn value
        still = tokens == MASK
        conf = torch.where(still, conf, torch.inf)           # never re-mask fixed tokens
        proposal = torch.where(still, sample, tokens)
        n_keep_masked = math.floor(gamma(t/T)*N)             # 0 after the last step
        order = conf.argsort(1)                              # least confident first
        remask = torch.zeros_like(still)
        remask.scatter_(1, order[:, :n_keep_masked], True)
        tokens = torch.where(remask, MASK, proposal)
    return tokens

for T in (1, 2, 4, 8, 16):
    rows = generate(1000, T).reshape(-1, 4, 4)
    agree = (rows[:, 1:] == rows[:, :1]).float().mean().item()
    valid = (rows == rows[:, :1]).all(2).all(1).float().mean().item()
    print(f"T={T:2d}  repeat agreement={agree:.3f}  valid images={valid:.3f}")
```

（程式用 `floor` 而不是論文中的上取整，好讓這個小例子在 $t = T$ 時剩餘數量剛好是零。）輸出為：

```
T= 1  repeat agreement=0.511  valid images=0.000
T= 2  repeat agreement=0.812  valid images=0.100
T= 4  repeat agreement=0.908  valid images=0.360
T= 8  repeat agreement=0.998  valid images=0.982
T=16  repeat agreement=1.000  valid images=0.999
```

$T = 1$ 時，每個 token 都從全遮罩的上下文獨立抽樣，所以各列幾乎從不一致。迭代次數增加後，前幾個 token 早早被固定，後面的 token 可以以它們為條件；到 $T = 8$ 時幾乎每個樣本都合法。在自然影像上，同一輪解碼的 token 之間依賴較弱，所以少數幾次迭代就夠用；但**較少步數**與**較多獨立假設**之間的取捨是真實存在的（圖 5）。

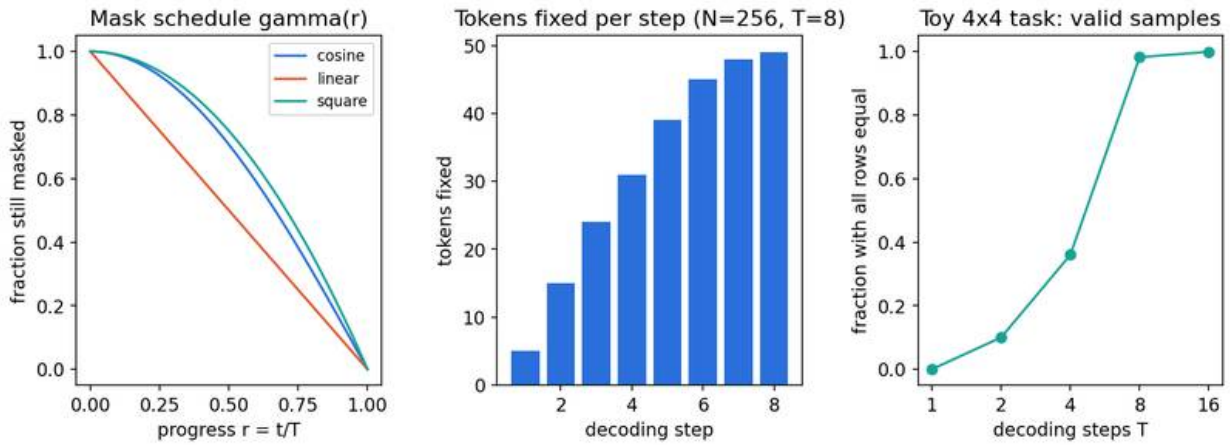


圖 *5* — 左：候選的遮罩排程 $\gamma(r)$。中：在餘弦排程、*256* 個 *token*、$T = 8$ 時，早期只固定少數 *token*，後期固定較多。右：在重複列的玩具任務中，合法樣本比例隨解碼迭代次數增加，因為同一步中被獨立抽樣的 *token* 變少了。

連續 token 與擴散條件分布（MAR）

白話版：自迴歸決定接下來是什麼；一個小型擴散模型可以決定怎麼抽樣它，因此 token 不必再是整數。

向量量化有損，而且不好訓練。Li 等人 [7] 指出，自迴歸需要的是一種建模 $p(x_i \mid x_{<i})$ 的方法，不一定是類別分布。他們讓 AR（或遮罩 AR）transformer 為每個連續值 token x_i 輸出條件向量 z_i，並以**擴散損失（diffusion loss）**訓練一個小型 MLP 去噪器 ϵ_θ：

$$\mathcal{L}(z_i, x_i) = \mathbb{E}_{\epsilon,t}\|\epsilon - \epsilon_\theta(x_{i,t} \mid t, z_i)\|^2, \qquad x_{i,t} = \sqrt{\bar{\alpha}_t}\, x_i + \sqrt{1-\bar{\alpha}_t}\, \epsilon,$$

其中 $\epsilon \sim \mathcal{N}(0, I)$，$t$ 是擴散時間步，$\bar{\alpha}_t$ 是ML 07 中的雜訊排程。梯度也會經由 z_i 流進 transformer。要抽樣一個 token，就以 z_i 為條件執行小型去噪器的反向擴散過程；temperature 用來縮放注入的雜訊。論文的主要變體 **MAR**（masked autoregressive）採隨機順序、每步預測一組 token，把 MaskGIT 的外層結構和內層的連續取樣器結合在一起 [7]。

因此，「AR 對擴散」並不是二選一。一個系統可以用自迴歸決定 token 的順序與分組，再用擴散抽樣每一組。同樣地，語言解碼器可以自迴歸地生成描述影像的文字，而完全不生成像素；統一這些方法的是分解的概念，而不是某種特定的輸出類型。

取樣、temperature 與快取

白話版：訓練好的模型在每一步給出一個分布；取樣器決定抽樣時要多大膽，快取則避免重算過去。

對 K 個可能值的 logit ℓ_k，**temperature** T 會重新縮放分布：

$$p_T(k) = \frac{\exp(\ell_k / T)}{\sum_{j=1}^{K} \exp(\ell_j / T)}.$$

較低的 T 讓機率集中在最高的幾個值；較高的 T 則讓機率分散（圖 6）。**Top-k** 取樣只保留最高的 k 個 logit；**top-p**（nucleus）取樣保留機率總和達到 p 的最小集合。兩者都會把剩下的部分重新正規化。

```
import torch

def filtered_probs(logits, temperature=1.0, top_k=None, top_p=None):
    """Return the (renormalized) distribution actually sampled from."""
    logits = logits / temperature
    if top_k is not None:                                  # keep the k largest logits
        kth = logits.topk(top_k).values[..., -1, None]
        logits = logits.masked_fill(logits < kth, -torch.inf)
    if top_p is not None:                                  # smallest set with mass >= p
        sorted_logits, order = logits.sort(descending=True)
        cum = sorted_logits.softmax(-1).cumsum(-1)
        drop = cum - sorted_logits.softmax(-1) >= top_p  # mass before this token already >=
         ↪  p
        sorted_logits = sorted_logits.masked_fill(drop, -torch.inf)
```

```
        logits = torch.empty_like(logits).scatter(-1, order, sorted_logits)
    return logits.softmax(-1)

logits = torch.tensor([2.0, 1.0, 0.0, -1.0])
for name, kw in [("T=1", {}), ("T=0.5", {"temperature": .5}), ("T=2", {"temperature": 2.}),
                 ("top-k=2", {"top_k": 2}), ("top-p=0.8", {"top_p": .8})]:
    p = filtered_probs(logits, **kw)
    entropy = -(p[p > 0]*p[p > 0].log2()).sum().item()
    print(f"{name:10s}", [round(v, 3) for v in p.tolist()], f"entropy={entropy:.2f} bits")
```

模型本身的分布（$T = 1$）熵為 1.37 位元；$T = 0.5$ 把它降到 0.66 位元，$T = 2$ 提高到 1.80 位元。在這個例子中，top-$k = 2$ 與 top-$p = 0.8$ 都保留最可能的兩個值（第一個值的機率 0.644 小於 0.8，所以還需要第二個）。這些設定全都改變了實際抽樣的分布，因此不要把取樣器的視覺品質和模型的似然混為一談。

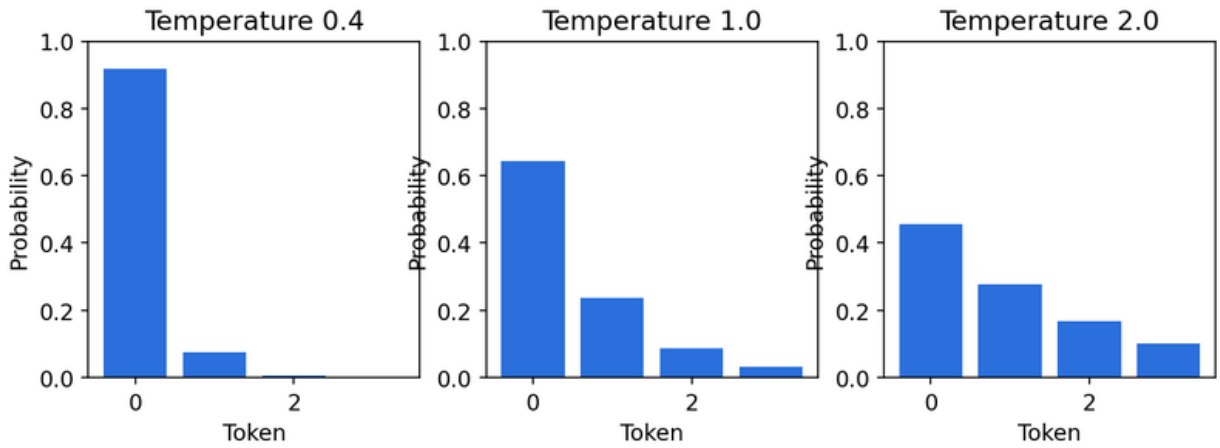


圖 6 — *temperature* 改變每個條件分布的多樣性，但無法找回 *tokenizer* 已經丟掉的資訊。

KV cache

在因果 transformer 中，新增一個 token 時，較早 token 的 key 與 value 不會改變。**KV cache** 把它們存起來，因此每一步只需計算最新 token 的 query、key 與 value，再對快取做注意力：

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
d, n = 8, 6
Wq, Wk, Wv = (torch.randn(d, d) for _ in range(3))
x = torch.randn(n, d)                                   # embeddings of n tokens

# Full causal attention over the whole sequence (what training computes).
q, k, v = x@Wq, x@Wk, x@Wv
mask = torch.ones(n, n).tril().bool()
```

```
scores = (q@k.T/d**.5).masked_fill(~mask, -torch.inf)
full = scores.softmax(-1)@v

# Incremental decoding: each step computes q, k, v for the newest token only.
K_cache, V_cache, outputs = [], [], []
for i in range(n):
    K_cache.append(x[i]@Wk); V_cache.append(x[i]@Wv)
    K, V = torch.stack(K_cache), torch.stack(V_cache)
    att = F.softmax((x[i]@Wq)@K.T/d**.5, -1)        # attends to tokens 0..i
    outputs.append(att@V)
assert torch.allclose(full, torch.stack(outputs), atol=1e-5)
print("cached decoding matches full causal attention")
print("dot products per step:", [i+1 for i in range(n)], "total:", n*(n+1)//2)
```

assert 確認快取解碼的結果與訓練時的因果輸出完全相同（這裡只有一層；真實模型會快取每一層、每個 head）。快取省去重算，卻沒有消除循序依賴，而且會隨序列長度線性成長。第 i 步仍要對 i 個快取的 key 做注意力，所以長度 n 的序列總注意力工作量是 $n(n+1)/2$，對 n 呈平方成長。這就是更短的序列（更好的 tokenizer）與更少的循序步驟（next-scale 或遮罩解碼）如此重要的原因。

對影片而言，token 可以依空間與時間排序，或分組成區塊。這時快取、壓縮與時間順序主導了成本，而物件一致性與長時間漂移需要另外評估。

自迴歸與擴散的比較

白話版：兩個家族都把一個困難的生成問題拆成許多比較簡單的問題。AR 把影像切成小塊；擴散則把雜訊程度切成多個步驟。

面向	自迴歸（token／尺度）	遮罩生成	擴散／ flow（ML 07）
每步生成什麼	一個 token（或一個尺度）	依信心選出的一組 token	對整個樣本的一次修正
循序步數	token 數或尺度數	固定且通常很少（T）	求解器步數（NFE）
常見表示	離散 code（或搭配擴散頭的連續 token [7]）	離散 code	連續的像素或潛在表示
訓練目標	精確 NLL（交叉熵）	遮罩交叉熵	去噪／速度回歸（與 ELBO 相關）
似然	在 token 上精確	不是單一精確的鏈鎖法則似然	下界或基於 ODE 的估計
天然優勢	與 LLM 共用架構與部署；容易與文字混合	步數少；修補與編輯容易	影像品質強；guidance 彈性大
常見弱點	循序解碼長；tokenizer 瓶頸；錯誤累積	同一步內的獨立假設	除非蒸餾，否則網路計算次數多

有三點讓這張表保持公允。第一，家族之間會混合：MAR [7] 在 AR 迴圈中放了一個擴散模型，而潛在擴散也依賴一個類似 tokenizer 的學習自編碼器。第二，某個家族「勝過」另一個的說法（例如 LlamaGen 的論文標題 [14]，或 VAR 與 DiT 的比較 [6]），只在特定基準、tokenizer、模型大小與 guidance 設定下成立，並非普遍成立。第三，成本應該以網路計算次數、guidance 計算次數、序列長度與實際延遲比較，不能只看「步數」。

現代觀點

這個領域已經從原始像素走向學習出來的表示，從固定的光柵順序走向彈性的預測單位，從獨立的影像生成器走向能同時讀寫文字與影像的模型。以下幾篇綜述描繪了這片版圖：

- **Xiong 等人 [1]**（TMLR）是最直接的視覺自迴歸綜述。它依表示方式分類（以像素、token、尺度為基礎的模型），也就是本章採用的分類，並涵蓋影像、影片、3D、多模態生成，以及具身智慧與醫學等應用。
- **Bond-Taylor 等人 [19]**（TPAMI 2022）並列比較 VAE、GAN、正規化流（normalizing flow）、能量模型與自迴歸模型，包括執行時間、樣本多樣性與架構限制上的取捨。它把 AR 模型放進更大的生成模型版圖中：精確的似然與穩定的訓練，代價是緩慢的循序取樣。
- **Li 等人 [20]**（TPAMI 2026）綜述多模態 LLM 的離散 tokenization，整理向量量化的各種變體，以及它們在傳統系統、單一模態 LLM 與多模態 LLM 中的應用，並討論 codebook 崩潰與梯度估計不穩定等問題。
- **Jia 等人 [22]** 綜述離散 tokenizer 在生成、理解、推薦與檢索中的應用，先拆解 tokenizer 的子模組與運作機制，再依應用整理方法，並列出現有限制與未來方向。
- **Zhao 等人 [21]** 綜述統一的多模態理解與生成模型，把它們分成以擴散為基礎、以自迴歸為基礎，以及 AR 與擴散混合三類，這正是 2025–2026 年大部分研究活動所在。

最新進展（2023–2026）。在經過查證的文獻中，有三條主線特別突出。(i) 在更好的 *tokenizer* 上擴展單純的 *AR transformer*：LlamaGen [14] 顯示，幾乎沒有影像專屬設計的語言模型架構，只要 tokenizer 與訓練夠好，就能在類別條件 ImageNet 上具競爭力。(ii) 改變預測單位：VAR 的 next-scale 預測 [6] 與 MaskGIT、Muse 的遮罩解碼 [8], [17] 減少了循序步數；MAR [7] 用每個 token 一個擴散頭，去掉了向量量化。(iii) 統一的多模態模型：同一套下一個 token 預測機制在單一模型中生成文字與影像 token，tokenizer 也被設計成同時服務理解與生成 [20], [21]。我們的解讀是：選擇**表示方式與預測單位**，如今至少和選擇骨幹網路一樣重要。

開放問題。這些綜述反覆指出以下幾點：

- *Tokenizer* 品質。重建能力決定了生成品質的上限；小字、人臉與精細幾何通常是最先受損的部分。如何在不丟細節的前提下提升 codebook 使用率與語意結構，仍是活躍的研究主題 [20], [22]。
- 離散與連續 *token*。離散 token 適合 LLM 的機制；連續 token 重建得更好，但需要連續的條件模型，例如擴散頭 [7]。
- 速度。循序解碼受延遲限制。next-scale、遮罩與平行解碼以獨立假設換取更少步數。
- 評估。token 似然、FID 類指標、提示貼合度與人類偏好衡量的是不同的東西；有損 tokenizer 的 token 似然並不是像素似然。
- 影片與長上下文。token 數量隨時間暴增，時間一致性、物件恆存性與漂移都需要專門的評估 [1]。

重點整理

- 自迴歸是一種分解方式，不是「transformer」的同義詞：鏈鎖法則對任何順序都精確成立，模型的容量與上下文決定每個條件分布的好壞。

- 在 teacher forcing 下訓練是平行的，取樣則是循序的。位移輸入與因果遮罩（注意力遮罩或 PixelCNN 的遮罩 A ／ B）必須一起設計。
- 像素模型精確但序列很長；學習出來的 tokenizer（VQ-VAE、VQGAN、RQ-VAE、FSQ）縮短序列，代價是生成器無法超越的重建瓶頸。
- Token transformer（Image GPT、VQGAN transformer、Parti、LlamaGen）重用語言模型的機制，包括 logit 上的 classifier-free guidance 與 KV cache。
- Next-scale（VAR）、遮罩（MaskGIT、Muse）與連續 token（MAR）模型改變了預測的**單位**；循序步數越少，通常代表條件獨立假設越多。
- Temperature、top-k 與 top-p 重塑抽樣分布，在多樣性與保真度之間取捨，但無法補回 tokenizer 丟掉的資訊。
- AR 與擴散是互補的分解方式，而且越來越常結合；比較它們時，要在相同的資料、tokenizer、guidance 與計算量下進行。

練習

1. **洩漏答案**。把 `BinaryAR` 的遮罩改成包含對角線（`diagonal=0`）。說明為什麼訓練損失可以降到接近零，取樣卻會失敗。

   提示

   每個輸出現在都能讀到自己的目標並直接複製，所以訓練損失崩落。取樣時，計算目前位置的 logit 時它的值還是 0（尚未抽樣），學到的「複製」規則因此產生無意義的結果。程式中的因果性 assert 也會失敗。

2. **每維位元數**。對重複列的 4×4 二值資料，計算以下情況的每像素位元數：(a) 真實分布；(b) 把每個像素當成獨立公平硬幣的模型；(c) 完美學會第一列、但把後面各列當成獨立公平硬幣的模型。

   提示

   (a) 4 位元 / 16 像素 = 0.25。(b) 16 / 16 = 1.0。(c) 每張 4 + 12 = 16 位元，同樣是每像素 1.0 位元：學會第一列的邊際分布沒有幫助，因為它的位元本來就是均勻的。所有的改善都來自建模後面各列對第一列的依賴。

3. **Tokenizer 預算**。一張 512×512 影像以 8 倍或 16 倍降取樣做 tokenization。分別計算序列長度，以及完整自注意力的成本比。用 16 倍時可能失去什麼？

   提示

   $64 \times 64 = 4096$ 對 $32 \times 32 = 1024$ 個 token。注意力成本隨長度平方成長，所以比值是 16。用 16 倍時，每個 token 要描述一個 16×16 區塊，除非 codebook 或解碼器能補償，否則小字和細線更容易遺失。

4. **盲點**。把遮罩卷積的程式改成 7 層、11×11 輸入、目標在中心。有多少較早的像素永遠看不到？接著畫出垂直堆疊加水平堆疊如何消除它們。

提示

盲點是目標右上方、往左上擴張的感受野永遠到不了的階梯區域；計算光柵順序中較早、梯度為零的格子，答案是 10。垂直堆疊看到上方完整的列（沒有左右限制），並把結果送進只處理目前這列的水平堆疊，於是每個較早的像素都變得可達。

5. **遮罩解碼步數**。用 $n_t = \lceil \gamma(t/T)N \rceil$、餘弦排程、$N = 256$、$T = 8$，計算每一步固定多少 token。為什麼排程一開始只固定少數 token？

提示

剩餘遮罩數為 252、237、213、182、143、98、50、0，所以每步固定 4、15、24、31、39、45、48、50 個 token（圖中使用 floor，數字最多差一）。在精確算術中 $\gamma(1) = 0$；但浮點數的 $\cos(\pi/2)$ 是極小的正數，直接取上整數會留下一個被遮住的 token，這也是程式改用 floor 的另一個原因。一開始模型的上下文很少，平行猜測最不可靠、也最可能互相矛盾；只固定少數 token，能讓後面的步驟以它們為條件。

6. **為方法分類**。對 PixelCNN、VQGAN transformer、Parti、LlamaGen、VAR、MaskGIT 與 MAR，寫出預測單位、表示方式（像素、離散 token、連續 token、多尺度圖）、條件分布類型，以及 16×16 token 網格所需的循序步數。

提示

PixelCNN：光柵順序的像素，256 類 softmax，每個值一步。VQGAN transformer、Parti、LlamaGen：光柵順序的離散 token，codebook 上的 softmax，256 步。VAR：多尺度離散圖，每個 token 一個 softmax、尺度內平行，每個尺度一步。MaskGIT：離散 token，雙向遮罩預測，T 步（論文中為 8）。MAR：隨機順序分組的連續 token，每個 token 一個擴散頭，步數等於組數（每組另有內層擴散步驟）。

參考文獻

1. J. Xiong, G. Liu, L. Huang et al., "Autoregressive Models in Vision: A Survey," *Transactions on Machine Learning Research* (*TMLR*), 2025 (arXiv:2411.05902, 2024). arXiv
2. A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," in *Proc. ICML*, PMLR 48, 2016. PMLR
3. A. van den Oord, N. Kalchbrenner, O. Vinyals, L. Espeholt, A. Graves and K. Kavukcuoglu, "Conditional Image Generation with PixelCNN Decoders," in *NeurIPS*, 2016. NeurIPS
4. A. van den Oord, O. Vinyals and K. Kavukcuoglu, "Neural Discrete Representation Learning," in *NeurIPS*, 2017. NeurIPS
5. P. Esser, R. Rombach and B. Ommer, "Taming Transformers for High-Resolution Image Synthesis," in *Proc. IEEE/CVF CVPR*, 2021. CVF
6. K. Tian, Y. Jiang, Z. Yuan, B. Peng and L. Wang, "Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction," in *NeurIPS*, 2024 (Best Paper Award). arXiv
7. T. Li, Y. Tian, H. Li, M. Deng and K. He, "Autoregressive Image Generation without Vector Quantization," in *NeurIPS*, 2024. arXiv

8. H. Chang, H. Zhang, L. Jiang, C. Liu and W. T. Freeman, “MaskGIT: Masked Generative Image Transformer,” in *Proc. IEEE/CVF CVPR*, 2022. CVF
9. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book
10. A. Razavi, A. van den Oord and O. Vinyals, “Generating Diverse High-Fidelity Images with VQ-VAE-2,” in *NeurIPS*, 2019. NeurIPS
11. M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan and I. Sutskever, “Generative Pretraining From Pixels,” in *Proc. ICML*, PMLR 119, 2020. PMLR
12. J. Yu, X. Li, J. Y. Koh et al., “Vector-quantized Image Modeling with Improved VQGAN,” in *ICLR*, 2022. arXiv
13. J. Yu, Y. Xu, J. Y. Koh et al., “Scaling Autoregressive Models for Content-Rich Text-to-Image Generation,” *Transactions on Machine Learning Research*, 2022. arXiv
14. P. Sun, Y. Jiang, S. Chen, S. Zhang, B. Peng, P. Luo and Z. Yuan, “Autoregressive Model Beats Diffusion: Llama for Scalable Image Generation,” arXiv:2406.06525, 2024. arXiv
15. D. Lee, C. Kim, S. Kim, M. Cho and W.-S. Han, “Autoregressive Image Generation using Residual Quantization,” in *Proc. IEEE/CVF CVPR*, 2022. CVF
16. F. Mentzer, D. Minnen, E. Agustsson and M. Tschannen, “Finite Scalar Quantization: VQ-VAE Made Simple,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2024. arXiv
17. H. Chang, H. Zhang, J. Barber et al., “Muse: Text-To-Image Generation via Masked Generative Transformers,” in *Proc. ICML*, PMLR 202, 2023. PMLR
18. J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” arXiv:2207.12598, 2022 (short version at the NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications). arXiv
19. S. Bond-Taylor, A. Leach, Y. Long and C. G. Willcocks, “Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models,” *IEEE TPAMI*, vol. 44, no. 11, 2022. doi
20. J. Li, Y. Fu, J. Liu, L. Cao, W. Ji, M. Yang, I. King and M.-H. Yang, “Discrete Tokenization for Multimodal LLMs: A Comprehensive Survey,” *IEEE TPAMI*, 2026 (arXiv:2507.22920). doi
21. S. Zhao, X. Zhang, J. Guo et al., “Unified Multimodal Understanding and Generation Models: Advances, Challenges, and Opportunities,” arXiv:2505.02567, 2025; revised 2026. arXiv
22. J. Jia, J. Gao, B. Xue et al., “From Principles to Applications: A Comprehensive Survey of Discrete Tokenizers in Generation, Comprehension, Recommendation, and Information Retrieval,” arXiv:2502.12448, 2025. arXiv
23. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *CVPR*, 2009, pp. 248–255. doi
24. H. Touvron, T. Lavril, G. Izacard et al., “LLaMA: Open and Efficient Foundation Language Models,” arXiv:2302.13971, 2023. arXiv

ML 09・基礎模型與視覺語言學習

什麼樣的模型稱為基礎模型：縮放定律、預訓練目標、CLIP 與 SigLIP、DINOv2 特徵、提示式分割、多模態語言模型、以 linear probe 與 LoRA 適配，以及如何評估風險。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-09-foundation-models/

你將學到

- 什麼樣的模型稱為基礎模型（foundation model）；為什麼這個詞描述的是訓練與重用方式，而不是某種架構，也不一定要能生成文字。
- 縮放定律（scaling laws）說了什麼（帶有下限的冪律、計算最佳配置），以及沒有說什麼。
- 主要的預訓練目標——監督式、影像文字對比、遮罩影像建模、自蒸餾與生成式——以及各自傾向產生什麼樣的特徵。
- CLIP、SigLIP、DINOv2/DINOv3 與 Segment Anything 系列如何作為可重用的視覺元件。
- 多模態大型語言模型如何把視覺編碼器接到語言模型，以及 LLaVA 式指令微調如何運作。
- 如何適配預訓練模型——linear probe、微調、LP-FT、adapter、prompt tuning、LoRA、提示——以及如何誠實地評估結果，包括偏誤、幻覺與測試集污染。

先看全貌

在電腦視覺的大部分歷史裡，每個任務各有一個模型：一個網路分類花朵、另一個偵測汽車、再一個分割腫瘤，每個都從頭訓練，或從 ImageNet [43] 分類器開始。**基礎模型**則在廣泛的資料上、以大規模訓練一次，接著**適配**到許多下游任務 [1]。這個詞描述的是模型如何被訓練與重用，而不是一種新的網路層。DINOv2 [15] 或 DINOv3 [16] 影像編碼器、CLIP 式影像文字模型 [3]、SAM [5] 這類提示式分割器、多模態語言模型，以及廣泛預訓練的影像生成器，都可以扮演這個角色。

> **用最簡單的話說**
>
> 與其每換一份工作就從零教一個新學生，不如找一位已經看過幾百萬張圖、讀過幾百萬句圖說的人。他已經知道邊緣、輪子、臉和狗長什麼樣子。遇到新工作時，你只需要給他幾個例子、一句簡短的說明，或一件小小的新工具——不必重新上一整套學校。

為什麼重要？有三個理由。第一，**標註很昂貴**，而預訓練把大部分學習移到不需要人工標籤的資料上（圖說、影像本身）。第二，**一個骨幹網路服務許多任務**，讓工程投入更集中。第三，它改變了**介面**：很多時候不必重新訓練，只要用一段文字提示、一次點擊或幾個範例就能指定任務。Bommasani 等人把這種方法的集中稱為同質化（homogenization），並提醒它同時也集中了失敗：共用骨幹的一個缺陷，會被建立在它之上的每個系統繼承 [1]。

本章延續ML 05・表示學習（什麼樣的特徵可以重用）、ML 04・Vision Transformer（常用骨幹），以及生成模型章節ML 07 與ML 08。Bishop 與 Bishop 的教科書 [14] 對遷移學習與表示有系統性

的說明，可作為參考。本章沿用 Awais 等人的視覺基礎模型綜述 [39] 與 Caffagni 等人的多模態綜述 [2] 所畫出的架構地圖，並引用為 2026 年 10 月版本查證過的代表性報告。這些例子用來說明模型家族，不是在替商業產品排名，也不宣稱某個 checkpoint 在所有情況都最好。

什麼樣的模型稱為「基礎」模型

白話版。基礎模型是在廣泛資料上、以通用目標預訓練的大型模型，它的內部表示或使用介面可以重用在從未專門訓練過的任務上。

Bommasani 等人 [1] 在 2021 年提出這個名稱，並強調兩個性質：

- **湧現（emergence）**。能力來自規模與訓練目標，而不是被明確設計出來。例如 CLIP 只被訓練去配對影像與圖說，卻能把影像分類到用文字命名的類別 [3]；DINO 完全沒有使用標籤，它的注意力圖卻能勾勒出物體輪廓 [24]。
- **同質化（homogenization）**。許多應用收斂到少數幾個預訓練骨幹與訓練配方。

使用「湧現」時要小心。本章中它只表示一種不是訓練目標的能力，並不代表無法預測或神秘的跳躍；某個能力看起來是否「突然」出現，常常取決於用來衡量它的指標。

形式上，令 f_θ 為在上游資料集 $\mathcal{D}_{\text{up}}$ 上以目標 $\mathcal{L}_{\text{pre}}$ 預訓練的模型：

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{x \sim \mathcal{D}_{\text{up}}}[\mathcal{L}_{\text{pre}}(f_\theta; x)].$$

下游任務 k 提供一個（通常很小的）資料集 $\mathcal{D}_k$ 與任務損失 ℓ_k。適配就是選一小組新的或被修改的參數 ϕ_k，並求解

$$\phi_k^\star = \arg\min_{\phi_k} \sum_{(x,y)\in\mathcal{D}_k} \ell_k(h_{\phi_k}(f_{\theta^\star}, x), y),$$

其中 h_{ϕ_k} 是我們放在預訓練模型上方或內部的任何東西：線性分類頭、adapter、LoRA 更新、提示，或什麼都不放（純提示，$\phi_k = \emptyset$）。整章都圍繞上面兩行：$\mathcal{L}_{\text{pre}}$ 與 $\mathcal{D}_{\text{up}}$ 應該是什麼，以及 ϕ_k 可以多小。

基礎模型**不一定**要產生文字。一個凍結的影像編碼器，若它的特徵供一百個不同分類器使用，就是基礎模型。反過來，為單一狹窄任務訓練的大模型，不論多大都不算。

縮放定律

白話版。用更多計算量訓練更大的模型、餵更多資料，測試損失通常會平滑且可預測地下降——在對數–對數圖上大致是一條直線——直到接近某個下限。正是這種可預測性，讓研究團隊能用小規模實驗規劃大型訓練。

Kaplan 等人 [17] 訓練了多組 Transformer 語言模型，報告損失與模型大小、資料量及計算量之間呈冪律關係，部分趨勢跨越超過七個數量級；在測試範圍內，寬度與深度的影響很小。對模型大小 N 的冪律寫成

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N},$$

其中 L 是保留資料上的交叉熵損失，N 是不含嵌入層的參數數量，N_c 是與 N 同單位的擬合常數，$\alpha_N > 0$ 是擬合指數。取對數得 $\log L = \alpha_N \log N_c - \alpha_N \log N$：斜率為 $-\alpha_N$ 的直線。

真實曲線會彎曲，因為有一部分損失永遠消不掉（資料本身就有雜訊或歧義）。因此 Hoffmann 等人 [18] 擬合了帶有不可約項的形式：

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

其中 D 是訓練 token 數，E 是不可約損失，A, B, α, β 是擬合常數。對稠密 Transformer，訓練計算量常用的近似是 $C \approx 6ND$ 次浮點運算 [17]（每個參數、每個 token，前向約 2 次、反向約 4 次）。固定 C，在 N 上最小化 L（令 $D = C/6N$），就得到**計算最佳**（compute-optimal）的模型大小。Hoffmann 等人訓練了超過 400 個語言模型，結論是：在計算最佳的訓練下，模型大小與訓練 token 數應等比例擴大——模型大小每加倍，token 數也應加倍 [18]。這和 Kaplan 等人較早「用相對適量的資料訓練非常大的模型」的建議不同 [17]；這個分歧提醒我們，擬合出的指數取決於實驗流程（學習率排程、納入哪些實驗、計算量怎麼算）。

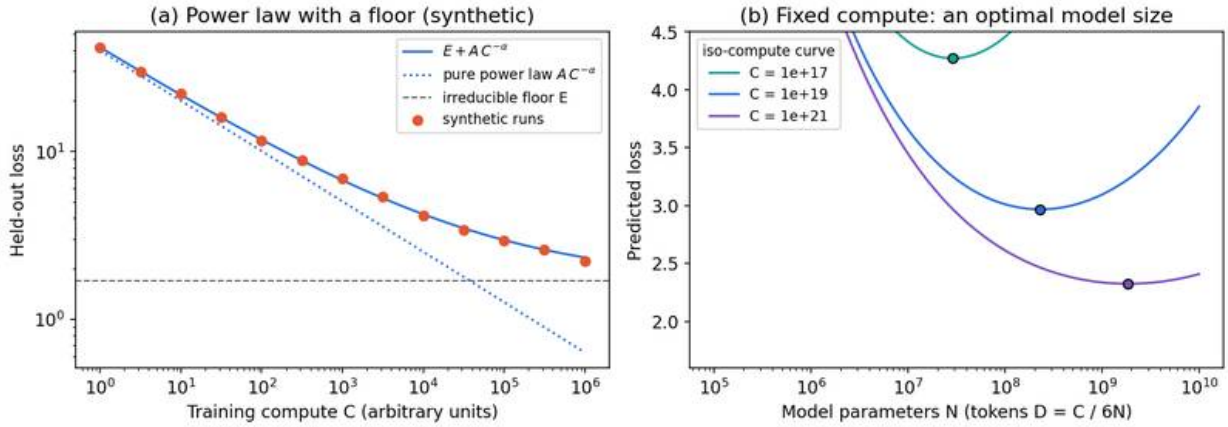


圖 *1* — (*a*) 帶有不可約下限 E 的冪律：純冪律（點線）會一直下降，在大規模時過度樂觀。(*b*) 固定計算量時，模型太小會欠擬合，太大則看到的 *token* 太少，因此每個預算都有最佳大小。所有常數皆為合成。

視覺領域也有自己的縮放研究。Zhai 等人 [19] 針對在大型標註資料上訓練的 Vision Transformer [21]，刻畫錯誤率、資料量與計算量的關係，並擴大到二十億參數的模型。Cherti 等人 [20] 在公開的 LAION 資料集 [34] 上訓練 CLIP 式模型，發現零樣本分類、檢索、linear probing 與微調都呈冪律縮放——但也發現 OpenAI 的 CLIP 與 OpenCLIP 在架構相同、配方相近的情況下，縮放行為仍不同，因為兩者的**訓練資料分布**不同。資料不是一個純量。

程式：擬合縮放定律並外插

下面的程式在 13 個合成訓練實驗中，只用較便宜的那幾個擬合 $L(C) = E + A\,C^{-\alpha}$，再預測昂貴的那幾個。所有資料都是合成的；重點在流程。

```
import numpy as np
from scipy.optimize import curve_fit

rng = np.random.default_rng(0)
# Synthetic "runs": loss = E + A * C^(-alpha), with 2% multiplicative noise.
E_true, A_true, alpha_true = 1.7, 40.0, 0.3
compute = np.logspace(0, 6, 13)            # arbitrary compute units
loss = (E_true + A_true * compute**-alpha_true) * np.exp(.02 * rng.normal(size=13))

def law(c, E, A, alpha):
    return E + A * c**-alpha

# Fit only the cheaper runs, then extrapolate to the larger ones.
small = compute <= 1e4
(E, A, alpha), _ = curve_fit(law, compute[small], loss[small], p0=(1, 10, .5))
print(f"fit: E={E:.2f}  A={A:.1f}  alpha={alpha:.3f}")
for c, l in zip(compute[~small], loss[~small]):
    print(f"C={c:9.0f}  observed {l:.3f}  predicted {law(c, E, A, alpha):.3f}")

# Naive straight line in log-log space ignores the floor E.
slope = np.polyfit(np.log(compute[small]), np.log(loss[small]), 1)[0]
print(f"pure power-law slope without floor: {slope:.3f}")
```

輸出：擬合得到 $E \approx 1.73$、$A \approx 40.0$、$\alpha \approx 0.300$；外插的損失與含雜訊的觀測值相差約 7% 以內；若在對數–對數空間畫一條不含下限的直線，斜率約為 -0.25，指數是錯的。有兩個教訓可以帶到實務：一定要納入下限；跨越多個數量級的外插應視為帶誤差範圍的預測。

縮放定律沒告訴你的事

- 它預測的是預訓練分布上的**平均損失**，不是你的任務準確率。下游指標可能平滑改善、飽和，或呈階梯式變化。
- 它只在**固定配方與資料分布**下成立 [20]。更好的資料過濾或不同的目標都會改變常數。
- 它完全不談**推論成本**。用更多資料訓練的較小模型，在相同損失下部署可能更便宜，這也是許多上線模型會訓練超過計算最佳點的原因。
- 資料是有限的。高品質資料用完後，不論再加多少計算量，B/D^{β} 這一項都不會再縮小。

預訓練目標：目標值從哪裡來？

白話版。每種預訓練方法都會發明一個預測任務。差別在於「答案」從哪裡來：人工標籤、圖說、同一張影像被遮住的部分、另一個網路，或下一個 token。

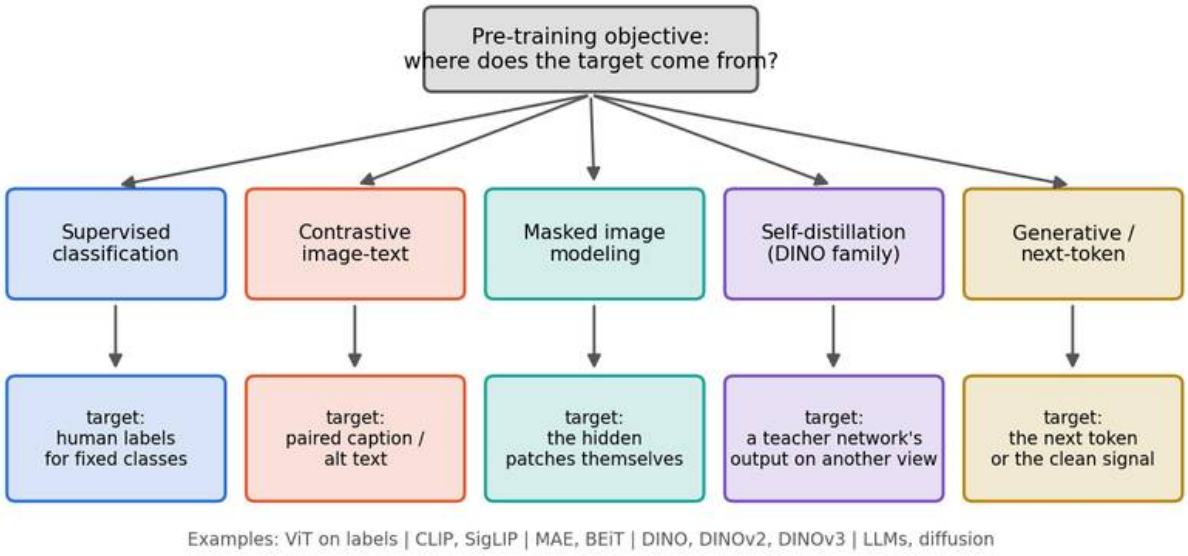


圖 2 — 依訓練目標值的來源，把預訓練目標分成五個家族。

目標	目標值來源	代表模型	典型強項
監督式分類	人工標籤	在大型標註資料上訓練的 ViT [19], [21]	在標籤空間內有強的全域語意
影像文字對比	成對的圖說／ alt text	CLIP [3]、ALIGN [25]、SigLIP [4]	以文字定義的辨識、檢索
遮罩影像建模	同一張影像被遮住的 patch	BEiT [23]、MAE [22]	微調後很強；保留局部細節
自蒸餾	教師網路在另一個視角上的輸出	DINO [24]、DINOv2 [15]、DINOv3 [16]	凍結特徵、密集對應
生成式／下一個 token	下一個 token 或乾淨訊號	LLM、擴散（ML 07）、自迴歸（ML 08）	生成；語言介面

監督式預訓練

在大型標註資料集上用交叉熵訓練分類器，然後拿掉最後一層。這是十年來的 ImageNet 配方，在大規模下仍然有效 [19], [21]。它的弱點是特徵被固定的標籤集合塑形：標籤不需要的區分可能會被丟掉。

影像文字對比學習

網路影像本來就附有圖說，因此 CLIP [3] 與 ALIGN [25] 同時訓練影像編碼器與文字編碼器，讓批次中配對正確的組合分數高於配錯的組合。ALIGN 顯示，只做最少過濾、雜訊很多的大型 alt-text 語料，仍能得到強的表示 [25]。損失函數的細節在下一節。

遮罩影像建模

把影像大部分遮住，再預測缺少的部分。BEiT [23] 遮住 patch 後預測它們的離散視覺 token，借用 BERT [44] 的遮罩語言建模概念。MAE [22] 直接預測原始像素，並觀察到遮住高比例（例如 75%）的 patch 會形成一個不平凡的任務；它的編碼器只看可見的 patch，再由輕量解碼器重建其餘部分，使訓練很有效率。令 $\mathcal{M}$ 為被遮住的 patch 索引集合，x_p 為 patch p 的（正規化）像素，$\hat{x}_p$ 為重建結果，

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{p \in \mathcal{M}} \|\hat{x}_p - x_p\|_2^2.$$

損失只在被遮住的 patch 上計算：預測可見的 patch 只是複製，沒有意義。遮罩建模的特徵在微調後常常非常好，但作為 linear probe 的凍結特徵時，通常比對比式或自蒸餾的特徵弱。

自蒸餾：DINO 家族

DINO [24]——作者形容為「沒有標籤的自蒸餾」——訓練一個**學生**網路 g_s，讓它在同一張影像的不同增強視角（裁切）上去配合**教師**網路 g_t。兩者都輸出 K 維分數，再用溫度 τ_s 與 τ_t 的 softmax 轉成機率：

$$P_s(x') = \text{softmax}\left(\frac{g_s(x')}{\tau_s}\right), \qquad P_t(x) = \text{softmax}\left(\frac{g_t(x) - c}{\tau_t}\right), \qquad \mathcal{L} = -\sum_{k=1}^{K} P_t(x)_k \log P_s(x')_k.$$

其中 x 與 x' 是兩個視角，c 是教師輸出的移動平均（**置中**，centering），較小的 τ_t 讓教師分布變得尖銳（**銳化**，sharpening）。置中避免某一維獨大；銳化避免輸出變成均勻分布；兩者合起來防止**崩塌**（collapse）——也就是每張影像都對應到相同輸出的平凡解。教師不接收梯度，它的權重是學生權重的指數移動平均（動量編碼器）：

$$\theta_t \leftarrow \lambda\, \theta_t + (1 - \lambda)\, \theta_s, \qquad 0 < \lambda < 1.$$

DINO 也使用多重裁切（multi-crop）訓練：學生看幾個小的局部裁切，教師看全域裁切，迫使模型學會局部與全域的對應。作者報告，所得到的 ViT 特徵含有明確的物體分割資訊，這在監督式 ViT 或卷積網路中沒有那麼明顯；而且這些特徵很適合作為 k 最近鄰分類器 [24]。

DINOv2 [15] 把這個配方擴大，關鍵是整理資料：用自動化流程建立專門、多樣且經過整理的影像資料集，而不是使用未整理的網路影像；訓練一個 10 億參數的 ViT，再蒸餾成較小的模型。作者報告這些凍結特徵在他們大多數影像層級與像素層級的基準上超越 OpenCLIP [15]。DINOv3 [16] 進一步擴大資料與模型，並提出 **Gram anchoring**，針對一個已知問題：長時間訓練時，密集（逐 patch）特徵圖的品質會退化。它也加入事後（post-hoc）策略，提升對解析度、模型大小與文字對齊的彈性。

為什麼密集特徵重要。分類器每張影像只需要一個向量；分割、深度估計、對應與追蹤則需要**每個 patch** 都有好的向量。自蒸餾編碼器之所以成為這些密集任務常用的凍結骨幹，正是因為它們的 patch 特徵不需要微調就保有語意。

生成式預訓練

預測下一個 token（語言模型、ML 08 中的自迴歸影像模型）或去雜訊（ML 07 的擴散模型）也是預訓練目標。對語言來說，它產生最有彈性的介面——用自然語言下指令——這也是多模態模型把視覺接到預訓練語言模型上，而不是反過來的原因。

影像與文字的共享空間：CLIP 與 SigLIP

白話版。訓練一個影像編碼器與一個文字編碼器，讓對應的圖片與圖說指向相近的方向。之後任何一句話都可以變成一個「類別」。

CLIP [3] 把影像嵌入 v_i 與文字嵌入 t_j 正規化成單位長度，並以可學習的溫度 τ 形成分數 $s_{ij} = v_i^\top t_j / \tau$。對一個含 B 組配對樣本的批次，

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left[\frac{1}{B}\sum_i -\log \frac{\exp(s_{ii})}{\sum_j \exp(s_{ij})} + \frac{1}{B}\sum_j -\log \frac{\exp(s_{jj})}{\sum_i \exp(s_{ij})}\right].$$

每張影像應在 B 個圖說中選出配對的那一個（第一項，逐列 softmax），每個圖說也應選出配對的影像（第二項，逐欄）。批次中其他組合都當作負樣本。訓練使用成對的文字監督，通常來自既有的圖說或 alt text；下游的「零樣本」使用並不代表模型訓練時沒有監督。

推論時，把「a photo of a dog」這類候選描述編碼，再與影像比較。文字嵌入取代了學習式分類器的類別權重：

$$\hat{y} = \arg\max_k v(x)^\top t(\text{prompt}_k).$$

候選集合與措辭都會影響結果；把多個模板（「a photo of a k」「a close-up of a k」）的嵌入平均，是降低敏感度的常見作法。零樣本的意思是在所述流程下沒有針對任務訓練參數；它不保證這些類別、甚至測試影像本身，不曾出現在預訓練資料中（見〈風險與評估〉）。

程式：配對損失與檢索機制

以下是**合成嵌入**，用來讓計算一目了然。它們不是預訓練模型的輸出，配對結果也不能證明真實世界的零樣本準確率。

```
import torch
from torch.nn import functional as F

images = F.normalize(torch.tensor([[1., .1, 0.], [.1, 1., 0.],
                                   [0., .1, 1.]]), dim=-1)
texts = F.normalize(torch.eye(3), dim=-1)
scores = images @ texts.T / .1
labels = torch.arange(3)
loss = (F.cross_entropy(scores, labels) +
        F.cross_entropy(scores.T, labels))/2
print("matching text:", scores.argmax(1).tolist())
```

```
print("paired loss:", round(loss.item(), 4))
ambiguous = F.normalize(torch.tensor([[.5, .5, .2]]), dim=-1)
print("cosines:", (ambiguous @ texts.T).tolist())
```

它會印出 `matching text: [0, 1, 2]`、約 `0.0002` 的配對損失，以及模糊影像約 `0.68, 0.68, 0.27` 的餘弦值：它與兩個描述一樣接近，任何 softmax 都無法告訴你哪一個才對。

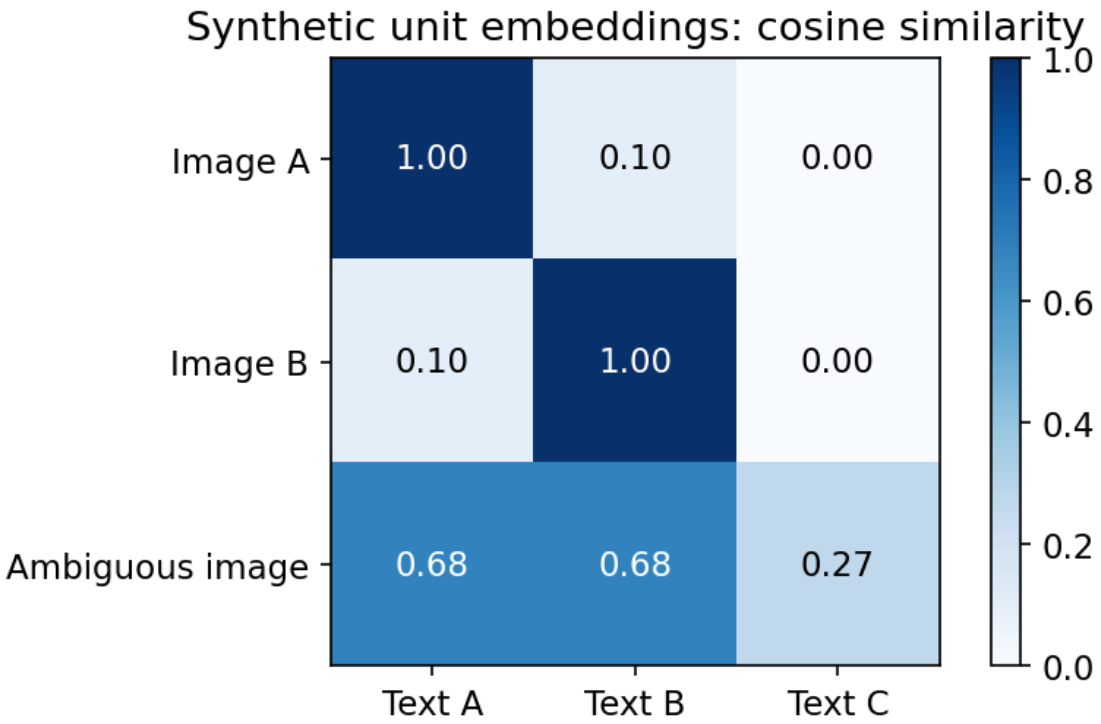


圖 3 — 餘弦相似度可以為候選排序，但它不是概念存在與否的校準機率。

SigLIP：用 sigmoid 取代 softmax

SigLIP [4] 把批次正規化的 softmax 換成對每一組影像文字配對各自做二元判斷。令配對正確時 $y_{ij} = +1$、否則 -1，並有可學習的尺度 a 與偏差 b，

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{j=1}^{B}\log \sigma(y_{ij}(a\, v_i^{\top} t_j + b)),$$

其中 $\sigma(u) = 1/(1+e^{-u})$，因此每一項等於 $\log(1+\exp(-y_{ij}(a\, v_i^{\top} t_j + b)))$。因為沒有任何一項需要對整個批次求和，這個損失不需要跨裝置的全域正規化，讓大批次訓練更簡單。由於正樣本只有 B 個、負樣本有 $B^2 - B$ 個，偏差 b 的初始值會讓初始預測強烈偏向負類。

共享嵌入支援檢索與開放詞彙辨識，但細粒度計數、否定（「一隻**沒有**項圈的狗」）、空間關係與罕見的領域詞彙都是已知弱點。這些需求要直接評估，不要從亮眼的整體分類分數推論。

提示式分割：Segment Anything 系列

白話版。提示告訴模型你想要哪個區域或概念，輸出則是遮罩。

SAM [5] 結合一個較重的影像編碼器（每張影像跑一次）、輕量的提示編碼器與輕量的遮罩解碼器，因此同一張影像上的許多提示都能很快回應。空間提示——點、框或粗略遮罩——不需要固定的類別詞彙就能選取物體。因為單次點擊有歧義（點在襯衫上可能指襯衫，也可能指整個人），SAM 會預測幾個候選遮罩並附上信心分數。它在 SA-1B 上訓練：超過 10 億個遮罩、1100 萬張具授權且尊重隱私的影像，透過「模型在迴圈中」的資料引擎蒐集 [5]。

SAM 2 [6] 把提示式分割延伸到影片，加入串流記憶：模型儲存過去影格與被提示物體的特徵，並沿時間傳遞遮罩；在之後影格上的修正點擊會更新記憶。

SAM 3 [7] 於 2025 年發布，報告在 2026 年 3 月修訂，提出**提示式概念分割**（promptable concept segmentation）：給定一個短名詞片語（例如「yellow school bus」）、影像範例或兩者，它會回傳**所有**符合實例的遮罩與身分。它的影像層級偵測器與基於記憶的影片追蹤器共用一個骨幹，並以 **presence head** 把「這個概念是否出現」的辨識與「實例在哪裡」的定位分開 [7]。

這些介面提供的資訊不同：空間點說明在哪裡，名詞片語說明是什麼，範例說明長什麼樣子。請報告建立提示的成本，以及提示來自人、真值標註或自動偵測器。用真值框當提示得到的「零樣本分割」分數，其實隱藏了大量的標註資訊。

多模態大型語言模型

白話版。把視覺特徵轉成語言模型讀得懂的「字」，再教它遵循視覺指令。

常見的設計是

$$\text{image/video} \xrightarrow{\text{vision encoder}} h \xrightarrow{\text{connector}} z \xrightarrow{\text{language decoder with text}} \text{answer.}$$

這裡 h 是視覺特徵網格（對 ViT 而言，每個 patch 一個向量），z 是位於語言模型 token 嵌入空間中的向量序列，解碼器把 z 當成一般文字的嵌入來處理。

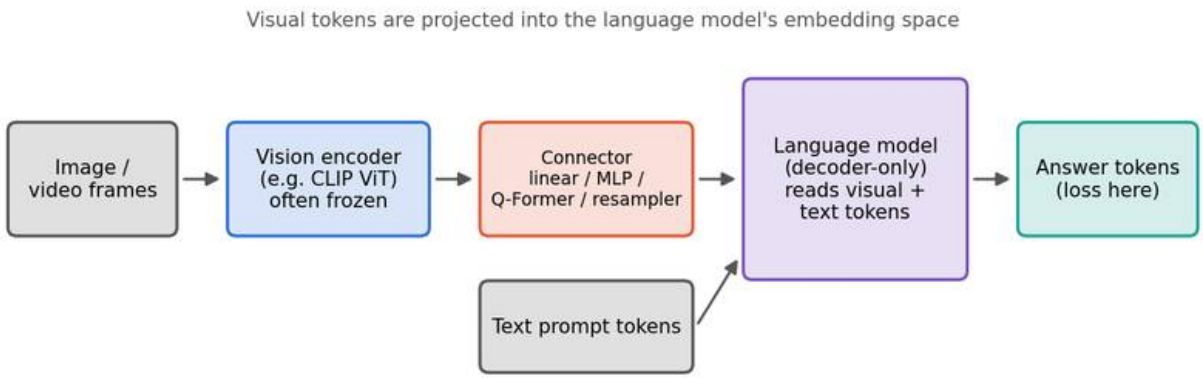


圖 *4* — 典型的多模態 *LLM* 流程。視覺特徵被轉換成語言模型嵌入空間中的 *token*。

連接器

- **線性或 MLP 投影。** LLaVA [8] 用單一投影把 CLIP 特徵映射到語言模型的嵌入空間；LLaVA-1.5 [28] 報告簡單的 MLP 連接器（搭配 336 像素解析度的 CLIP ViT-L 與學術 VQA 資料）出奇地強大且節省資料。每個 patch 變成一個視覺 token。

- **可學習查詢。** BLIP-2 [26] 在凍結的影像編碼器與凍結的語言模型之間插入輕量的 Querying Transformer（Q-Former）。一組固定的可學習查詢向量對影像特徵做交叉注意力，因此語言模型永遠只看到少量、固定數目的視覺 token。BLIP-2 分兩階段訓練 Q-Former：先對凍結的影像編碼器做表示學習，再對凍結的語言模型做生成學習。
- **重取樣與交叉注意力。** Flamingo [27] 橋接凍結的視覺與語言模型，並能處理任意交錯的影像文字序列，使模型能從直接寫在提示中的範例做少樣本學習。

取捨在於 token 數與細節：每個 patch 一個 token 保留細節，但序列很長（多張影像或影片時很昂貴）；少數可學習查詢很便宜，但可能遺失小字。

視覺指令微調

LLaVA [8] 用只處理語言的 GPT-4 [45]，根據影像的圖說與框生成多模態指令遵循資料，再分兩階段訓練：先對齊連接器（特徵對齊），再在指令資料上微調連接器與語言模型。損失是一般的下一個 token 交叉熵，只算在**答案** token 上：

$$\mathcal{L} = -\sum_{i \in \mathcal{A}} \log p_\theta(w_i \mid z,\, w_{<i}),$$

其中 w_i 是文字 token，$\mathcal{A}$ 是屬於助理回答的位置集合，z 是視覺 token。指令與提示 token 作為上下文可見，但不計入損失。

Qwen3-VL 的 2025 年報告 [9] 呈現目前的方向：一系列稠密與混合專家（mixture-of-experts）模型，原生支援最長 256K token 的影像、影片與文字交錯上下文。它是代表性報告，不代表每個任務都會因更長的上下文或更多生成的推理而受益。

示意圖沒畫出的事

閱讀文件可能需要以切塊（tiling）或動態解析度保留小字；影片問題需要時間戳與時間順序。這些細節會改變 token 數與前處理；通用的「視覺編碼器 + LLM」示意圖只是起點。流暢的回答也

可能提到影像中根本不存在的物體（**物體幻覺**），或數錯數量。請區分物體定位、OCR、空間推理、事實問答與風格上的指令遵循——它們需要不同的證據與指標（見〈風險與評估〉）。整個設計空間的綜述可參考 Yin 等人 [40] 與 Caffagni 等人 [2]。

適配基礎模型

白話版。用能解決目標任務的最便宜適配方式，並在看起來像實際部署的資料上檢查。

策略	可訓練元件	適合的起點
提示／上下文範例	無	模型介面本來就符合任務
凍結特徵 + 線性分類頭（linear probe）	小型監督式讀出層	標籤少、特徵遷移良好
Prompt tuning ／ visual prompt tuning	少數可學習的輸入 token	許多任務共用同一個凍結骨幹
Adapter ／ LoRA	小型插入模組或低秩模組	記憶體有限下的領域適配
部分微調	選定的區塊加分類頭	想做一些適配，又不想負擔完整最佳化器成本
完整微調（可搭配 LP-FT）	所有模型參數	資料與計算足夠；單靠遷移不夠

Linear probe

凍結編碼器 f，先算好特徵，再擬合線性分類器 $\hat{y} = \text{softmax}(Wf(x) + b)$。它便宜、不容易過擬合，也是衡量表示品質的標準**量尺**：如果線性讀出就能成功，代表資訊在特徵中是線性可取得的。

程式：為什麼預訓練特徵在標籤少時最有幫助

這個玩具實驗模擬整個故事。觀測值是 300 維，混合了 8 個**語意**因子（決定類別）、8 個很大的**干擾**因子（想像成光線與姿態）以及雜訊。「預訓練」看到 50 個上游類別，標籤充足，並學到一個 8 維投影（線性判別分析，代替深度編碼器）。下游有四個**新**類別，每類只有少數標籤。

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(0)
D, k_sem, k_nui = 300, 8, 8
sem_map = rng.normal(size=(k_sem, D))   # semantic factors -> pixels
nui_map = rng.normal(size=(k_nui, D))   # nuisance factors (light, pose...)

def sample(centers, n):
    y = rng.integers(len(centers), size=n)
    z = centers[y] + 0.7 * rng.normal(size=(n, k_sem))
    u = 4.0 * rng.normal(size=(n, k_nui))          # large nuisance variation
    x = z @ sem_map + u @ nui_map + rng.normal(size=(n, D))
    return x, y

# 1) "Pre-training": many upstream classes with plenty of labels.
up_centers = rng.normal(size=(50, k_sem))
x_up, y_up = sample(up_centers, 10_000)
encoder = LinearDiscriminantAnalysis(n_components=k_sem).fit(x_up, y_up)

# 2) Downstream: four NEW classes, few labels, linear probe only.
new_centers = rng.normal(size=(4, k_sem))
x_te, y_te = sample(new_centers, 2_000)
f_te = encoder.transform(x_te)          # frozen features, computed once
```

```
for shots in [1, 5, 20, 100]:
    raw, feat = [], []
    for repeat in range(10):
        x_tr, y_tr = sample(new_centers, 4 * shots)
        raw.append(LogisticRegression(max_iter=3000)
                   .fit(x_tr, y_tr).score(x_te, y_te))
        feat.append(LogisticRegression(max_iter=3000)
                    .fit(encoder.transform(x_tr), y_tr).score(f_te, y_te))
    print(f"{shots:>3} labels/class | raw pixels {np.mean(raw):.3f}"
          f" | frozen features {np.mean(feat):.3f}")
```

在我們的機器上，它印出的準確率大約是：每類 1 個標籤時 0.29 對 0.71、5 個時 0.62 對 0.95、20 個時 0.96 對 0.99、100 個時 0.98 對 0.99（原始輸入對凍結特徵；隨機猜測為 0.25）。預訓練投影已經丟掉干擾方向，所以少數標籤就夠了；標籤夠多時，原始輸入上的 probe 也會自己學會忽略干擾，差距隨之消失。上游類別與下游類別不同——特徵之所以能遷移，是因為兩者共享相同的語意因子。如果下游任務依賴的正是預訓練丟掉的因子（這裡是干擾因子），那麼不論加多少標籤，凍結特徵都會失敗。這就是實務上要測試的失敗模式。

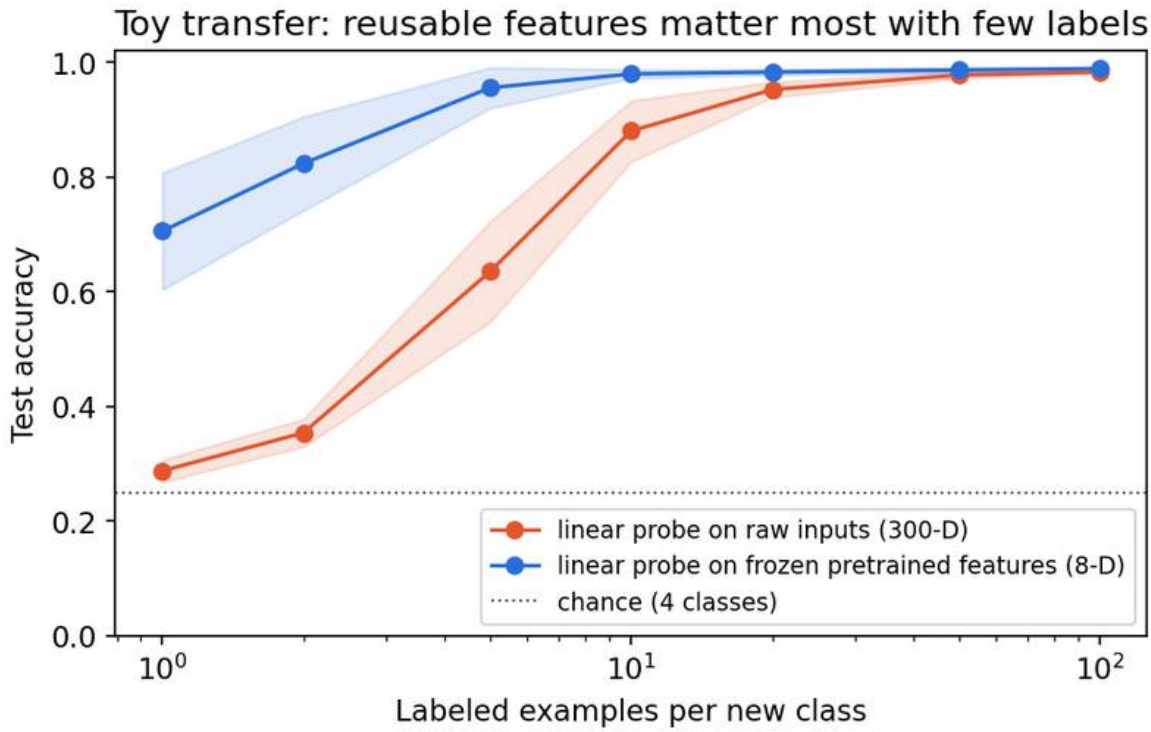


圖 *5* — 在更多標籤預算下的玩具遷移實驗（*10* 次抽樣的平均 ± 一個標準差）。標籤稀少時，可重用特徵最重要。

微調、LP-FT 與穩健微調

完整微調會更新所有權重，資料足夠時通常有最好的分布內準確率。但 Kumar 等人 [32] 指出它可能扭曲好的預訓練特徵：在他們的基準上，微調平均而言分布內準確率較高，分布外準確率卻比 linear probing 低。他們的解釋是，隨機初始化的分類頭在訓練初期把又大又雜的梯度送進骨幹。**LP-FT**——先擬合 linear probe，再從這個分類頭開始微調全部參數——減少了這種扭曲，並在他們的實驗中勝過兩種單獨作法 [32]。**WiSE-FT** [33] 走另一條路：在權重空間線性內插零樣本模型與微調模型，$\theta_\gamma = (1-\gamma)\theta_{\text{zero-shot}} + \gamma\,\theta_{\text{ft}}$，對 CLIP 類模型在分布偏移下提升了穩健性，同時保持強的目標任務準確率。兩者都很便宜，在斷言「微調傷害了穩健性」之前值得先試。

參數高效微調（PEFT）

Adapter [29] 在每個 Transformer 區塊內插入小型瓶頸 MLP，只訓練它們，每個任務只增加少量參數，原網路保持不變。**Prompt tuning** 學習幾個額外的輸入 token；**Visual Prompt Tuning（VPT）** [30] 把這個作法用在 ViT 上，在凍結骨幹的 patch 序列前面加上可學習的 token。Xu 等人 [42] 綜覽 PEFT，提出加法式（additive）、部分（partial）、重參數化（reparameterized）、混合（hybrid）與統一（unified）微調的分類，並實驗比較參數與記憶體效率。

LoRA [10] 把凍結權重 $W_0 \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ 的更新重參數化為

$$W = W_0 + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r\times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}}\times r},$$

其中秩 $r \ll \min(d_{\text{in}}, d_{\text{out}})$，$\alpha$ 是縮放常數。凍結 W_0，只學 A, B。可訓練權重從 $d_{\text{out}}d_{\text{in}}$ 降到 $r(d_{\text{in}} + d_{\text{out}})$。把 B 初始化為 0，使初始更新為零；A 隨機初始化，學習才能開始。訓練後 BA 可以合併回 W_0，因此推論不增加成本。QLoRA [31] 以 4 位元精度儲存凍結的基礎權重，並穿過它們反向傳播到 LoRA adapter，大幅減少記憶體，讓非常大的模型也能在單張 GPU 上微調。

程式：凍結基礎權重上的低秩更新

這個獨立範例在合成輸入上訓練一步 adapter。它展示參數化方式與初始行為，並沒有重現完整的預訓練模型微調流程。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
class LoRALinear(nn.Module):
    def __init__(self, din, dout, rank=4, alpha=4):
        super().__init__()
        self.base = nn.Linear(din, dout, bias=False)
        self.base.weight.requires_grad_(False)
        self.A = nn.Parameter(torch.randn(rank, din) * .02)
        self.B = nn.Parameter(torch.zeros(dout, rank))
        self.scale = alpha/rank
    def forward(self, x):
```

```
        return self.base(x) + self.scale * F.linear(F.linear(x, self.A), self.B)

layer = LoRALinear(32, 16)
x = torch.randn(8, 32)
assert torch.allclose(layer(x), layer.base(x))
opt = torch.optim.Adam([layer.A, layer.B], lr=.01)
loss = F.mse_loss(layer(x), torch.randn(8, 16))
opt.zero_grad(); loss.backward(); opt.step()
assert layer.base.weight.grad is None
print("trainable:", sum(p.numel() for p in layer.parameters() if p.requires_grad))
```

它印出 `trainable: 192`，也就是 $r(d_{\text{in}} + d_{\text{out}}) = 4 \times (32 + 16)$，完整矩陣則是 512。第一個 assertion 檢查初始化為零的 B 在一開始不改變模型；第二個檢查凍結的基礎權重沒有收到梯度。

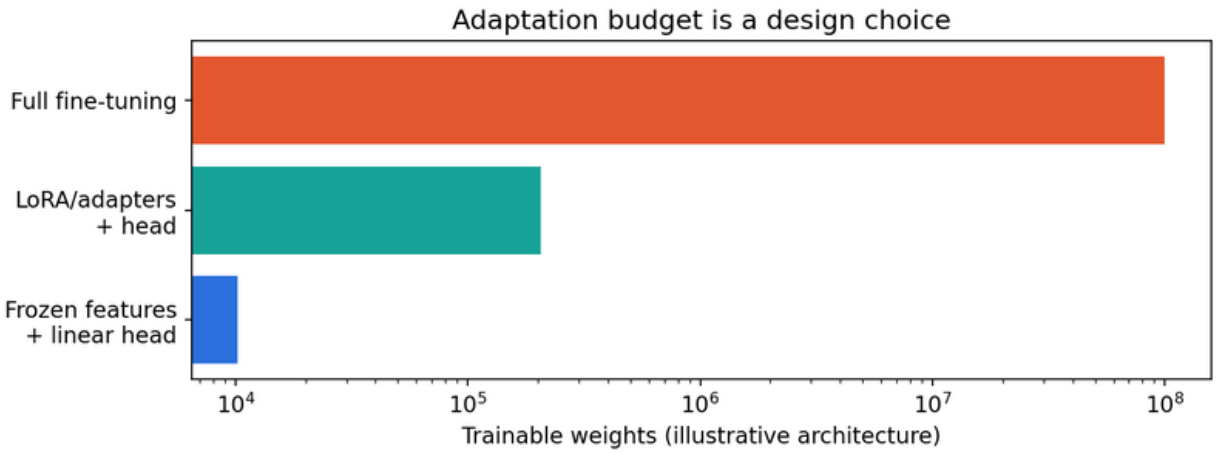


圖 *6* — 示意架構（特徵維度 *1024*、*10* 類、*12* 個矩陣上 *rank 8*、總共 *1* 億參數）的可訓練權重預算解析計算。活化記憶體、凍結權重儲存與最佳化器狀態必須另外計算。

LoRA 是一種參數化方式，不保證不會過擬合或遺忘。Adapter 仍需要領域驗證。凍結的 BatchNorm 統計量、影像正規化、輸出標籤對應，以及各 checkpoint 特有的前處理，也都需要刻意處理。

提示與上下文學習

最便宜的適配不改任何權重：為 CLIP 寫更好的文字提示、給 SAM 一個框，或像 Flamingo 那樣把幾個已解好的範例放進多模態模型的上下文 [27]。Zhang 等人的視覺語言綜述 [41] 比較了提示、特徵適配與微調在各種視覺任務上的表現。提示迭代很快，但很脆弱：措辭稍微改變就可能影響結果，因此要固定並報告提示流程。

影片、幾何與世界模型

白話版。除了辨認單一影格，模型還可以預測隨時間的變化、推論三維結構，或支援動作規劃。

V-JEPA 2 [11] 研究在特徵空間（而非像素空間）做預測的自監督影片模型，並以條件於動作的延伸版本進行規劃。VGGT [12] 以一次前饋，從一個或多個視角預測相機參數、深度、點雲圖（point maps）與軌跡。它們代表超越分類的兩條路：時間狀態建模與幾何推論。

看起來逼真的生成影片不一定遵守物理限制；場景重建即使不產生寫實像素也可能很有用。請依所宣稱的能力，量測多視角一致性、幾何、條件於動作的結果與時間行為。DLCV 07・世界模型會更詳細地討論部分可觀測性、學習動態與規劃。

風險與評估

白話版。用網路上一大片資料訓練出的模型，會繼承那片資料的偏誤與缺口，可能自信地描述不存在的東西，也可能早就看過你的「測試」集。評估必須針對這些事實來設計。

資料來源、偏誤與隱私

網路規模的影像文字資料集大到無法人工檢查。LAION-5B [34] 公開了這樣的資料集（58.5 億組經 CLIP 過濾的配對），因而促成 OpenCLIP 這類開放重現 [20]；Birhane 等人 [35] 對其前身 LAION-400M 的稽核，則記錄了影像文字配對中露骨與刻板印象的內容。下游系統會繼承這些關聯。授權與同意也很重要：訓練資料、模型權重與生成輸出可能各有不同條款；個人資料（人臉、姓名、醫療影像）帶來的隱私義務，不會因為資料是公開的就消失。SAM 的 SA-1B 是一個在說明中強調具授權且尊重隱私影像的資料集例子 [5]；部署每個模型時都應記錄這類性質。

幻覺

多模態 LLM 可能提到影像中不存在的物體。POPE [37] 以輪詢方式衡量物體幻覺：針對特定物體是否存在提出是非題（包括常見物體與經常共同出現的干擾物體），使量測比評分自由生成的描述更不受模型生成風格影響。POPE 錯誤率低並不涵蓋計數、屬性或關係；這些要另外測試。

污染與「零樣本」

如果基準影像（或近似重複的影像）在預訓練資料中，「零樣本」分數有一部分其實是在測記憶。Mayilvahanan 等人 [36] 探討 CLIP 的泛化是否主要來自訓練與測試的高相似度：他們把 LAION 中與常見分布外基準太相似的影像剪除後重新訓練，部分基準的表現下降，但整體仍然很強。教訓是雙向的：相似度解釋了一部分報告的表現，但不是全部——所以要實際量測，不要預設任何一方。實務檢查包括：把測試集與可取得的訓練資料做去重複、在模型資料截止日期之後蒐集的資料上報告結果，並完整說明零樣本流程（類別名稱、模板、候選集合）。

基準

MMMU [38] 這類綜合性多模態基準——從考試、測驗與教科書蒐集 11.5K 個大學程度問題，橫跨六大學科，包含圖表、示意圖、地圖、表格等多種影像——測試知識與推理，但單一數字混合了感知、OCR、領域知識與答案格式。部署時，請建立一個小型、針對任務的評估集並納入已知

的失敗案例，報告不同隨機種子或提示措辭下的變異，並與簡單基準比較（凍結特徵上的 linear probe 常常強得令人意外）。ML 10・評估與部署 會深入討論校準、資料切分與部署。

現代觀點

值得閱讀的綜述。

- **Bommasani 等人 [1]**——為基礎模型命名的報告。內容涵蓋能力（語言、視覺、機器人、推理）、應用（法律、醫療、教育）、技術（建模、訓練、適配、評估、資料）與社會影響。它的核心觀點——湧現、同質化與被繼承的失敗——至今仍是這個領域的框架。
- **Awais 等人 [39]**——電腦視覺綜述，依架構設計、訓練目標（對比式與生成式）、資料集、微調機制與提示方式整理基礎模型，並列出評估、實際應用、對抗穩健性與可解釋性等開放挑戰。
- **Zhang 等人 [41]**——TPAMI 上關於視覺任務的視覺語言模型綜述：網路架構、預訓練目標、下游任務、資料集、遷移學習與知識蒸餾，並對所回顧的方法做基準比較。
- **Yin 等人 [40]** 與 **Caffagni 等人 [2]**——多模態大型語言模型綜述：架構（編碼器、連接器、LLM）、訓練策略與資料（預訓練、指令微調、對齊）、評估與幻覺。
- **Han 等人 [13]**——把地圖擴展到多模態生成模型的綜述，依生成類別與技術組織。
- **Xu 等人 [42]**——參數高效微調的批判性回顧，把方法分成加法式、部分、重參數化、混合與統一五個家族，並實驗比較代表性方法。

2023–2026 年的現況。有四個趨勢特別明顯。(1) **更好的凍結特徵**：DINOv2 [15] 與 DINOv3 [16] 讓自監督編碼器強到許多密集任務直接從凍結的 patch 特徵開始。(2) **概念層級的提示**：分割從「點一個物體」（SAM [5]）走到影片記憶（SAM 2 [6]），再到「分割這個片語的每一個實例」（SAM 3 [7]）。(3) **長且交錯的多模態上下文**：Qwen3-VL [9] 等模型可同時讀取多張影像、影片與文字。(4) **超越辨識**：用於規劃的影片預測（V-JEPA 2 [11]）與前饋式幾何（VGGT [12]）把基礎模型當成世界與場景的估計器。

開放問題。

- 有依據的可靠性：減少幻覺，並讓模型在適當時候說出「看不到」[37]。
- 在污染與基準飽和下做誠實的評估 [36]。
- 資料：把整理資料當成一個縮放軸 [15], [20]，以及來源、授權與同意 [34], [35]。
- 有效率的適配與推論：高解析度影像與影片的 token 預算；能保留穩健性的 PEFT [32], [33]。
- 讓密集、三維與時間理解達到影像層級模型的語意水準。

面對任何新的「基礎」模型，請記錄它的輸入／輸出介面、訓練目標、資料來源、支援的適配方式、checkpoint 授權、前處理與任務層級的評估。這能避免把模型大小或開放式聊天介面當成具備通用能力的證據。

重點整理

- 「基礎」描述的是廣泛預訓練加上重用，而不是特定架構；這類模型不一定要生成文字。
- 縮放定律描述帶有下限、近似冪律的平滑損失改善；計算最佳的訓練會平衡模型大小與資料量；常數取決於資料與配方。

- 預訓練目標決定特徵擅長什麼：對比式適合以文字定義的辨識，遮罩建模適合微調，自蒸餾適合凍結的密集特徵，生成式提供有彈性的介面。
- CLIP 式對齊支援檢索與以文字定義的分類；餘弦相似度不是校準過的信心。
- 空間、概念與語言提示提供不同種類的資訊；請報告提示由誰產生。
- 多模態 LLM = 視覺編碼器 + 連接器 + 語言模型，以只算答案的下一個 token 損失訓練；連接器決定細節與 token 數的取捨。
- 適配從便宜的開始（提示、linear probe），再到 PEFT（LoRA、adapter），最後才是完整微調；LP-FT 與權重內插有助於保留穩健性。
- 明確評估偏誤、幻覺與污染；單一綜合基準分數不足以證明通用能力。

練習

1. **候選集合的影響**。在合成 CLIP 範例中加入第四個描述。影像不變時，softmax 的「信心」會不會改變？

提示

會：softmax 的分母改變了。影像與某個固定描述的相似度不變，但正規化後的機率取決於候選集合。試著加入一個與模糊影像嵌入很接近的文字向量。

2. **計算最佳配置**。利用參數化定律 $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ 與 $C = 6ND$，證明最佳模型大小的成長為 $N^{\star} \propto C^{\beta/(\alpha+\beta)}$。$\alpha = \beta$ 代表什麼？

提示

代入 $D = C/(6N)$，對 N 微分並令其為零：$\alpha A N^{-\alpha-1} = \beta B (6N/C)^{\beta}/N$。解得 $N^{\alpha+\beta} \propto C^{\beta}$。當 $\alpha = \beta$ 時，$N^{\star} \propto C^{1/2}$，因此 $D^{\star} \propto C^{1/2}$：參數與 token 等比例擴大，與 Hoffmann 等人的結論一致。

3. **讓遷移失效**。在 linear probe 實驗中，讓下游類別取決於干擾因子而非語意因子。隨著標籤增加，凍結特徵上的 probe 會怎樣？這對選擇基礎模型有什麼啟示？

提示

改為對 u 抽類別中心，而不是 z。凍結的 8 維特徵被訓練成丟掉干擾方向，所以不論加多少標籤都接近隨機猜測，而原始輸入上的 probe 會進步。骨幹只有在預訓練保留了你的任務所需資訊時才有用——投入適配之前，先用 linear probe 檢查。

4. **LoRA 預算**。一個 ViT-B 區塊有四個 768×768 的注意力投影，以及兩個大小分別為 3072×768 與 768×3072 的 MLP 矩陣。計算這六個矩陣完整微調與全部使用 rank-8 LoRA 時的可訓練權重數。

提示

完整：$4 \cdot 768^2 + 2 \cdot 3072 \cdot 768 = 2{,}359{,}296 + 4{,}718{,}592 = 7{,}077{,}888$。LoRA：$8 \cdot [4 \cdot (768 + 768) + 2 \cdot (3072 + 768)] = 8 \cdot (6144 + 7680) = 110{,}592$，約為完整數量的 1.6%。

5. **提示的公平性**。設計一個能把遮罩品質與提示品質分開的 SAM 評估。

   提示

   固定提示流程並計算人工互動次數。把自動提議與使用理想真值提示得到的遮罩分開評估，兩者都要報告。對 SAM 3，在同一批影像上比較名詞片語提示與範例提示。

6. **污染稽核**。有一個新的影像文字模型宣稱在某個公開基準上有很高的零樣本準確率。在相信這個數字之前，列出你會做的三項檢查。

   提示

   (a) 以嵌入相似度等方式，在可取得的訓練資料中搜尋測試影像的近似重複；(b) 在資料截止日期之後新蒐集的資料上評估；(c) 檢查對提示模板與候選類別清單的敏感度；也可以與 linear probe 及簡單基準比較。

參考文獻

1. R. Bommasani et al., "On the Opportunities and Risks of Foundation Models," Stanford CRFM report, arXiv:2108.07258, 2021; revised 2022. arXiv
2. D. Caffagni et al., "The Revolution of Multimodal Large Language Models: A Survey," *Findings of ACL*, 2024. arXiv
3. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021. arXiv
4. X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer, "Sigmoid Loss for Language Image Pre-Training," *ICCV*, 2023. arXiv
5. A. Kirillov et al., "Segment Anything," *ICCV*, 2023. arXiv
6. N. Ravi et al., "SAM 2: Segment Anything in Images and Videos," *ICLR*, 2025. arXiv
7. N. Carion et al., "SAM 3: Segment Anything with Concepts," arXiv:2511.16719, 2025; revised March 2026. arXiv
8. H. Liu, C. Li, Q. Wu and Y. J. Lee, "Visual Instruction Tuning," *NeurIPS*, 2023. arXiv
9. S. Bai et al., "Qwen3-VL Technical Report," arXiv:2511.21631, 2025. arXiv
10. E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022. arXiv
11. M. Assran et al., "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning," arXiv:2506.09985, 2025. arXiv
12. J. Wang et al., "VGGT: Visual Geometry Grounded Transformer," *CVPR*, 2025. arXiv
13. L. Han et al., "A Survey of Generative Categories and Techniques in Multimodal Generative Models," arXiv:2506.10016, 2025. arXiv
14. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
15. M. Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
16. O. Siméoni et al., "DINOv3," arXiv:2508.10104, 2025. arXiv
17. J. Kaplan et al., "Scaling Laws for Neural Language Models," arXiv:2001.08361, 2020. arXiv

18. J. Hoffmann et al., “Training Compute-Optimal Large Language Models,” arXiv:2203.15556, 2022; published as “An empirical analysis of compute-optimal large language model training,” *NeurIPS*, 2022. arXiv
19. X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer, “Scaling Vision Transformers,” *CVPR*, 2022. arXiv
20. M. Cherti et al., “Reproducible Scaling Laws for Contrastive Language-Image Learning,” *CVPR*, 2023. arXiv
21. A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. arXiv
22. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, “Masked Autoencoders Are Scalable Vision Learners,” *CVPR*, 2022. arXiv
23. H. Bao, L. Dong, S. Piao and F. Wei, “BEiT: BERT Pre-Training of Image Transformers,” *ICLR*, 2022. arXiv
24. M. Caron et al., “Emerging Properties in Self-Supervised Vision Transformers,” *ICCV*, 2021. arXiv
25. C. Jia et al., “Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision,” *ICML*, 2021. arXiv
26. J. Li, D. Li, S. Savarese and S. Hoi, “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” *ICML*, 2023. PMLR
27. J.-B. Alayrac et al., “Flamingo: a Visual Language Model for Few-Shot Learning,” *NeurIPS*, 2022. arXiv
28. H. Liu, C. Li, Y. Li and Y. J. Lee, “Improved Baselines with Visual Instruction Tuning,” *CVPR*, 2024. arXiv
29. N. Houlsby et al., “Parameter-Efficient Transfer Learning for NLP,” *ICML*, 2019. PMLR
30. M. Jia et al., “Visual Prompt Tuning,” *ECCV*, 2022. arXiv
31. T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *NeurIPS*, 2023. arXiv
32. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, “Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution,” *ICLR*, 2022. arXiv
33. M. Wortsman et al., “Robust Fine-Tuning of Zero-Shot Models,” *CVPR*, 2022. arXiv
34. C. Schuhmann et al., “LAION-5B: An Open Large-Scale Dataset for Training Next Generation Image-Text Models,” *NeurIPS Datasets and Benchmarks Track*, 2022. arXiv
35. A. Birhane, V. U. Prabhu and E. Kahembwe, “Multimodal Datasets: Misogyny, Pornography, and Malignant Stereotypes,” arXiv:2110.01963, 2021. arXiv
36. P. Mayilvahanan, T. Wiedemer, E. Rusak, M. Bethge and W. Brendel, “Does CLIP’s Generalization Performance Mainly Stem from High Train-Test Similarity?,” *ICLR*, 2024. arXiv
37. Y. Li et al., “Evaluating Object Hallucination in Large Vision-Language Models,” *EMNLP*, 2023. arXiv
38. X. Yue et al., “MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI,” *CVPR*, 2024. arXiv
39. M. Awais et al., “Foundational Models Defining a New Era in Vision: A Survey and Outlook,” arXiv:2307.13721, 2023. arXiv
40. S. Yin et al., “A Survey on Multimodal Large Language Models,” *National Science Review*, 2024. arXiv

41. J. Zhang, J. Huang, S. Jin and S. Lu, “Vision-Language Models for Vision Tasks: A Survey,” *IEEE TPAMI*, vol. 46, no. 8, 2024. arXiv
42. L. Xu et al., “Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment,” arXiv:2312.12148, 2023. arXiv
43. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *CVPR*, 2009, pp. 248–255. doi
44. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019, pp. 4171–4186. doi
45. OpenAI, “GPT-4 Technical Report,” arXiv:2303.08774, 2023. arXiv

ML 10・評估、效率與部署

不洩漏地切分資料、做交叉驗證、為分數加上信賴區間、檢查校準與分布偏移下的穩健性，誠實量測 FLOPs、記憶體與延遲，再把模型匯出、壓縮、監控並做到可重現。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-10-evaluation-deployment/

你將學到

- 如何建立基準模型（baseline）與資料切分，讓結果真正支持你想提出的主張，以及資料洩漏（leakage）如何悄悄破壞這一切。
- *K* 折、群組、依時間順序與巢狀交叉驗證（cross-validation）的運作方式，以及各自適用的時機。
- 如何為測試分數加上不確定性：標準誤、成對 bootstrap 與 McNemar 檢定。
- 校準（calibration）與準確率有何不同、如何量測（Brier 分數、NLL、ECE），以及溫度縮放能修正其中哪一部分。
- 分布偏移（distribution shift）是什麼（共變量、標籤、概念偏移）、穩健性基準如何探測它，以及部署後如何監控。
- 如何正確計算 FLOPs、記憶體與延遲，以及匯出、量化、剪枝、蒸餾和 ONNX Runtime、TensorRT 等推論引擎實際帶來什麼。
- 如何讓實驗可重現並留下文件。

先看全貌

訓練模型只是專案的中段，不是終點。在任何人依賴它之前，你必須回答四個問題：它在真正會遇到的案例上有效嗎？這個數字有多可靠？它知道自己什麼時候可能出錯嗎？它符合要執行的裝置、延遲與預算嗎？上線之後還有第五個問題：它現在還有效嗎？

> **用最簡單的話說**
>
> 一個背下去年考卷的學生，考去年的考卷當然滿分，但這什麼也說明不了。公平的考試要用新題目（乾淨的資料切分）。一次考試可能運氣好，所以你想知道換一天考分數會晃動多少（信賴區間）。好學生遇到可能答錯的題目也會說「我不太確定」（校準）。如果正式考場很吵，你也要在吵雜環境下測試（穩健性）。如果是限時考試，答對但太慢也不行（延遲）。學生開始工作後，還是要有人不時抽查他的成果（監控）。

這一章是 ML 系列的最後一章。它整理 Goodfellow 等人 [1, ch. 11] 的實務方法論與 James 等人 [2] 的統計學習觀點，再補上把訓練好的網路變成可靠服務所需的工程步驟。視覺任務專屬的指標（top-1、mAP、PSNR、FID 等）在DLCV 02 與DLCV 03 中說明；本章聚焦所有任務共通的部分。

先建立有意義的基準模型

白話版。在投入更多算力之前，先看看簡單方法在完全相同的切分上表現如何。基準模型告訴你複雜模型到底多換到了什麼。

影像任務常見的好基準包括：在原始像素或手工特徵上做邏輯迴歸；最近鄰或 SVM 分類器；以及在預訓練模型凍結特徵上接一個小型線性分類頭（見ML 09）。邏輯迴歸學習線性邊界；支援向量機（support vector machine, SVM）最大化帶正則化的間隔；最近鄰直接比較樣本；樹集成則切分特徵空間。傳統圖形識別章節 IP 13 有更完整的推導。

作為參考，二元線性 SVM 最小化

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0,\,1 - y_i(w^\top x_i + b)), \qquad y_i \in \{-1, +1\},$$

其中 w 與 b 是權重與偏差，x_i 是第 i 個特徵向量，$C > 0$ 在間隔寬度與訓練錯誤之間取捨。$\max(0,\, 1 - y_i f(x_i))$ 稱為**鉸鏈損失（hinge loss）**：位於正確一側且間隔至少為 1 的點損失為零，其餘則線性增加。

程式碼：把前處理放進 pipeline 的基準模型

這個例子只使用內建資料。模型選擇用驗證集；選出的模型只在測試集上評估一次。標準化與 PCA 都在各自的訓練 pipeline 內擬合，絕不在整份資料上擬合。

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, balanced_accuracy_score

X, y = load_digits(return_X_y=True)
Xa, Xt, ya, yt = train_test_split(X, y, test_size=.2, stratify=y, random_state=0)
Xtr, Xv, ytr, yv = train_test_split(Xa, ya, test_size=.25, stratify=ya, random_state=1)
models = {      # listed from simplest to most complex
    "linear": make_pipeline(StandardScaler(),
                            LogisticRegression(max_iter=2000)),
    "PCA + RBF SVM": make_pipeline(StandardScaler(), PCA(n_components=30),
                                   SVC(C=10, gamma="scale")),
}
scores = {}
for name, model in models.items():
    model.fit(Xtr, ytr)
    scores[name] = accuracy_score(yv, model.predict(Xv))
best = max(scores, key=scores.get)   # tie -> first listed, i.e. the simpler model
pred = models[best].predict(Xt)
print("validation:", scores)
print("selected:", best)
```

```
print("test accuracy:", accuracy_score(yt, pred))
print("test balanced accuracy:", balanced_accuracy_score(yt, pred))
```

在這個切分上，兩個候選模型的驗證準確率剛好都是 0.9639（360 張中答對 347 張）。「平手時選較簡單的模型」這條規則是事先定好的，因此選出線性模型，它在測試集上的準確率為 0.9639、平衡準確率為 0.9636。這個小實驗藏著兩個教訓。第一，平手規則是評估協定的一部分，應該在看結果之前寫下來。第二，驗證集只有 360 張時，一張影像就值 0.28 個百分點，所以小實驗中許多看似「勝出」的差距其實只是雜訊；後面的信賴區間一節會把這點說得更精確。

資料切分與資料洩漏

白話版。測試集必須和部署時的資料「一樣新」。如果測試影像和訓練影像來自同一位病人、同一段影片、同一個場景或同一段時間，測試分數量到的是記憶，而不是泛化。

依主張決定切分單位

同一位病人、同一段影片、同一批產品或同一台相機的影像彼此相關。把相鄰影格隨機切分，會讓幾乎相同的畫面同時出現在訓練與測試中。先問部署時「新」代表什麼，再依該單位切分：模型會遇到新病人就依病人切，會遇到新影片就依影片切，要裝到新醫院就依院區切，要預測未來就依時間切。



圖 *1* — 依泛化主張選擇切分單位。以群組分開可以避免幾乎重複的情境跨越切分邊界。

分層抽樣（stratification）讓各切分的類別比例相近，但它不能取代群組切分。切分前先去除重複，並把完全或近似重複的樣本放在同一側。若部署母體會隨時間漂移，依時間切分（用過去訓練、用最近一段時間測試）才是誠實的做法。

資料洩漏的各種面貌

資料洩漏是指任何讓「預測當下不該取得的資訊」流進模型或模型選擇過程的途徑。Kapoor 與 Narayanan 發現資料洩漏錯誤影響了 17 個使用 ML 的科學領域、共 329 篇論文，並提出洩漏類型的分類，其中包括缺少訓練／測試分離、在整份資料上做前處理、重複樣本、時間洩漏與不正當特徵 [4]。scikit-learn 文件也列出最常見的程式錯誤 [3]。實用檢查表：

洩漏	例子	修正
在全部資料上前處理	在訓練＋測試上擬合 `StandardScaler`、PCA 或詞彙表	把前處理放進只在訓練折上擬合的 `Pipeline`
相關的單位	同一段影片的影格、同一次掃描的切片、同一張照片的裁切	依群組切分
重複樣本	同一張影像被爬取兩次	切分前先用雜湊或嵌入去重
時間	用下週的資料預測本週	依時間順序切分
目標代理變數	在結果**之後**才記錄的特徵（例如「是否已接受治療」）	稽核每個特徵的記錄時間
重複使用測試集	一邊看測試分數一邊調超參數	決策用驗證集，測試只做一次

最後一列最隱微。每次看了測試分數再修改，測試集就變得更像驗證集一點。公開基準在整個社群的規模上都面臨這個問題。

交叉驗證

白話版。不只做一次訓練／驗證切分，而是輪流：把資料分成 K 份，訓練 K 次，每次留出不同的一份來評估，再取平均。每個樣本都恰好被評估一次，也能看到分數的變動幅度。

K 折交叉驗證與其變形

把索引分成 K 個互斥的折 $F_1, \dots, F_K$。對每個 k，用除了 F_k 以外的所有折訓練出 $\hat{f}^{(-k)}$，並在 F_k 上計算誤差。交叉驗證估計值為

$$\widehat{\mathrm{Err}}_{\mathrm{CV}} = \frac{1}{K}\sum_{k=1}^{K} \frac{1}{|F_k|} \sum_{i \in F_k} \ell\big(y_i, \hat{f}^{(-k)}(x_i)\big),$$

其中 ℓ 是單一樣本的損失（例如 0–1 錯誤），$|F_k|$ 是該折的大小。常見選擇為 $K = 5$ 或 $K = 10$ [2]。各種變形只差在如何組成各折：

- **分層 K 折**讓各折的類別比例相同；類別不平衡時很有用。
- **群組 K 折**把每個群組（病人、影片、院區）只分到一個折，因此沒有群組同時出現在訓練與留出資料中。
- **時間序列切分**永遠用較早的資料訓練、用較晚的資料驗證。
- **留一群組**是每折只放一個群組的群組 K 折；群組很少時有用。

各折分數並不獨立（訓練集互相重疊），所以各折之間的標準差會低估真正的不確定性。把它當成穩定性的粗略指標，而不是正式的信賴區間。

程式碼：隨機 K 折對群組資料做了什麼

我們模擬 40 位病人、每人 25 張影像。同一位病人的影像共享一個「指紋」（病人特有的外觀），而標籤以病人為單位固定。真正的類別訊號很弱。

```
import numpy as np
from sklearn.model_selection import KFold, GroupKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

rng = np.random.default_rng(0)
n_groups, per_group = 40, 25                    # e.g. 40 patients, 25 images each
groups = np.repeat(np.arange(n_groups), per_group)
y = np.repeat(rng.integers(0, 2, n_groups), per_group)   # label fixed per patient
identity = rng.normal(size=(n_groups, 20))[groups]       # patient "fingerprint"
signal = 0.3 * y[:, None] * np.ones((1, 20))             # weak real class signal
X = identity + signal + 0.3 * rng.normal(size=(len(y), 20))

model = RandomForestClassifier(n_estimators=200, random_state=0)
random_cv = cross_val_score(model, X, y, cv=KFold(5, shuffle=True, random_state=0))
group_cv = cross_val_score(model, X, y, cv=GroupKFold(5), groups=groups)
print(f"random K-fold accuracy: {random_cv.mean():.3f} +/- {random_cv.std():.3f}")
print(f"group  K-fold accuracy: {group_cv.mean():.3f} +/- {group_cv.std():.3f}")
```

輸出：

```
random K-fold accuracy: 0.999 +/- 0.002
group  K-fold accuracy: 0.657 +/- 0.158
```

隨機切分回報近乎完美的準確率，因為隨機森林認出了每位病人的指紋，直接抄該病人的標籤。群組 *K* 折符合「對新病人有效」這個主張，回報的是 0.657，而且各折差異很大。兩個數字在算術上都沒錯，但只有一個回答了真正的問題。

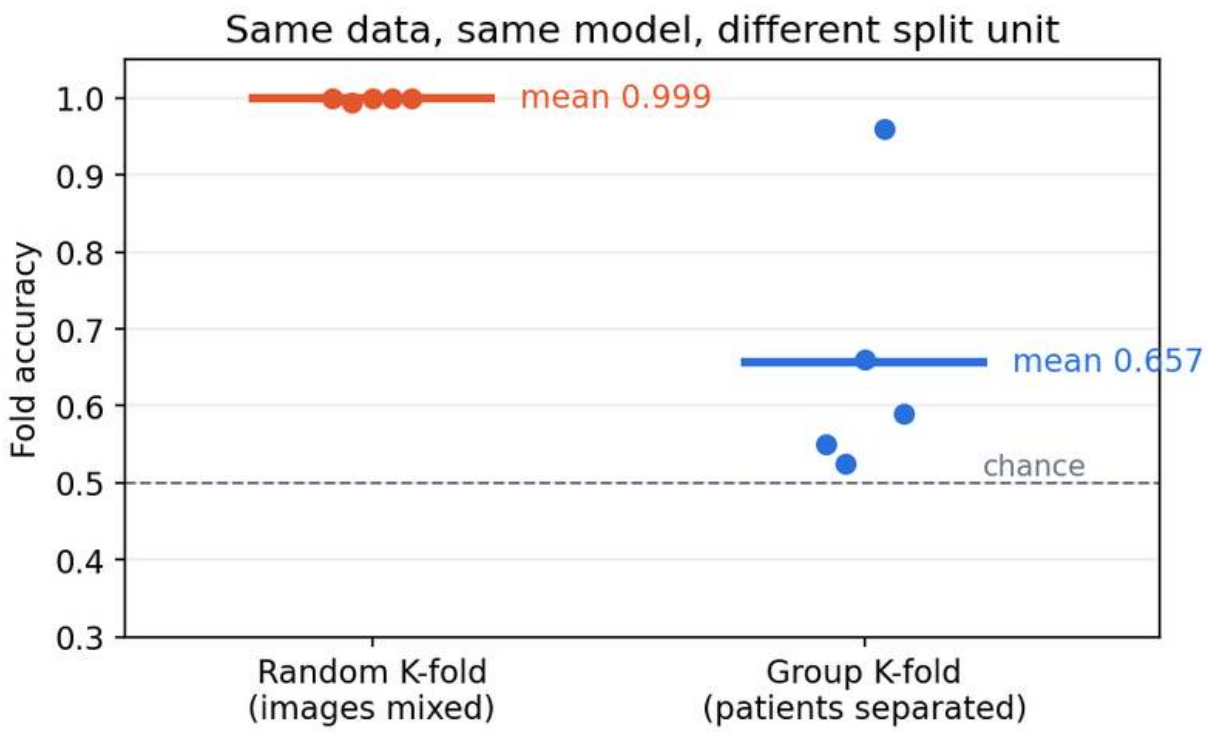


圖 2 — 上方程式碼的輸出，每個點代表一折。造成 *34* 個百分點差距的是切分單位，不是模型。

巢狀交叉驗證

如果同時用交叉驗證來**選**超參數又**回報**表現，回報的數字會偏樂觀：你挑的是剛好在那些折上看起來最好的設定。Varma 與 Simon 用兩類在統計上完全相同的「虛無」資料示範了這點：用交叉驗證調整 SVM 後，在 38% 的模擬資料集上 CV 誤差估計低於 30%，而調好的分類器在獨立測試資料上並不比亂猜好 [5]；Cawley 與 Talbot 則顯示，對模型選擇準則的過擬合可能和不同學習演算法之間的差距一樣大 [6]。

巢狀交叉驗證用兩層迴圈解決這個問題。外層留出一折做評估；在每個外層訓練集內，由內層交叉驗證選擇超參數。外層折只用來為內層選出的設定打分數。在 scikit-learn 中，這只是把 `GridSearchCV` 放進 `cross_val_score`。得到的結果估計的是整個**程序**（包含調參）的表現，也就是你重新訓練時實際會執行的東西。

對深度網路而言，完整的巢狀交叉驗證通常太昂貴。常見的折衷是固定一組訓練／驗證／測試切分，為每個被比較的方法訂明調參預算，並只打開測試集一次。

指標回顧

白話版。先決定一個錯誤的代價，再選數字。單一純量很少能說完整個故事。

任務	有用的量測	常見陷阱
分類	各類別精確率／召回率、macro-F1、平衡準確率、AUROC/AUPRC	準確率隱藏罕見失敗；PR 曲線受盛行率影響
偵測／分割	AP、IoU/Dice、邊界誤差	比較不同 IoU 門檻或評估協定的結果
檢索	Recall@k、在明確相關集合下的 mAP	圖庫中的重複樣本灌高匹配率
影像復原	PSNR/SSIM、感知指標、與觀測的一致性	看似合理的紋理不一定是真實細節
生成	FID/KID、涵蓋率、條件對齊、人工評估	特徵層面的真實感不代表忠於提示
影片／幾何	時間一致性、任務準確率、幾何誤差	單張影格好看可能掩蓋結構不一致

分類指標（混淆矩陣、top-k、ROC/PR 曲線）的定義與陷阱在DLCV 02 中詳細說明，偵測指標（IoU、AP、COCO mAP）則在DLCV 03。對生成模型而言，FID 比較特徵分布的高斯擬合（平均與共變異數）；KID 則使用無偏的核最大平均差異估計，因此在樣本數少時表現較穩定 [7]。兩者都不是人類偏好的通用量度。務必寫明特徵擷取器、參考集合、樣本數、解析度與取樣設定。

有兩個經典事實值得記住。正例只有 1% 時，永遠預測負例的分類器有 99% 準確率而召回率為零，所以對罕見事件而言，單看準確率毫無意義。此外，精確率取決於盛行率：同一個偵測器在事件更罕見的母體中精確率較低，但它的召回率與偽陽性率不變。

統計顯著性與信賴區間

白話版。測試分數是從有限樣本算出來的，換一批樣本就會有些不同。要回報一個範圍而不只是一個點，並在**相同**樣本上比較兩個模型。

準確率的標準誤

若 n 個測試樣本彼此獨立，模型真正的準確率為 p，則答對的個數服從二項分布，觀測準確率 $\hat{p}$ 的標準誤為

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \qquad 95\%\ \text{interval} \approx \hat{p} \pm 1.96\,\mathrm{SE}.$$

這裡 $\hat{p}$ 是答對比例，1.96 是標準常態分布的第 97.5 百分位數。當 $\hat{p} = 0.972$、$n = 540$ 時，$\mathrm{SE} \approx 0.0071$，區間大約是 ± 1.4 個百分點。當 $\hat{p}$ 非常接近 0 或 1、或 n 很小時，這個常態近似很差，應改用 Wilson 或精確二項區間。若測試樣本有群組結構（每位病人多張影像），有效樣本數比較接近群組數，此時必須以群組為單位重抽樣來計算區間。

Bootstrap

Bootstrap 以「從觀測資料中放回抽樣」來估計任意統計量的抽樣分布 [9]。抽出 B 組大小為 n 的重抽樣，在每組上重新計算統計量，並取第 2.5 與第 97.5 百分位數作為近似的 95% 區間（**百分位數區間**）。它適用於沒有簡單公式的指標，例如 macro-F1、AUROC 或 mAP。

比較兩個模型時，要用**成對** bootstrap：每次只抽一次樣本索引，並在同一組重抽樣上評估兩個模型。因為兩個模型通常在許多相同的困難樣本上失敗，配對能消除這部分共同變異，使差值的區間比兩個獨立區間緊得多。

McNemar 檢定

兩個分類器在同一個測試集上評估時，只有它們意見不同的樣本才帶有「誰比較好」的資訊。令 n_{01} 為模型 A 答對而 B 答錯的樣本數，n_{10} 為相反情況。在「兩個模型錯誤率相同」的虛無假設下，每個不一致樣本落在兩邊的機率相等，所以 $n_{10} \sim \text{Binomial}(n_{01}+n_{10}, \frac{1}{2})$。精確 McNemar 檢定就由此計算雙尾二項 p 值。Dietterich 比較了五種比較學習演算法的檢定，發現對只能執行一次的演算法（昂貴模型的典型情況），McNemar 檢定是唯一第一型錯誤可以接受的方法；若能負擔執行十次，他建議使用 5×2 交叉驗證 t 檢定 [8]。

程式碼：成對 bootstrap 與 McNemar

```
import numpy as np
from scipy.stats import binomtest
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
a = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
b = make_pipeline(StandardScaler(), SVC(C=10, gamma="scale")).fit(Xtr, ytr)
ca, cb = a.predict(Xte) == yte, b.predict(Xte) == yte    # per-example correctness

# 1) Percentile bootstrap: resample test examples with replacement.
rng = np.random.default_rng(0)
idx = rng.integers(0, len(yte), size=(10_000, len(yte)))
diff = cb[idx].mean(axis=1) - ca[idx].mean(axis=1)       # paired: same resample
lo, hi = np.percentile(diff, [2.5, 97.5])
print(f"accuracy A={ca.mean():.4f}  B={cb.mean():.4f}  n={len(yte)}")
print(f"B - A = {cb.mean() - ca.mean():+.4f}, 95% bootstrap CI [{lo:+.4f}, {hi:+.4f}]")

# 2) Exact McNemar test: only the discordant examples carry information.
n01 = int(np.sum(ca & ~cb))      # A right, B wrong
n10 = int(np.sum(~ca & cb))      # A wrong, B right
p = binomtest(n10, n01 + n10, 0.5).pvalue
print(f"discordant pairs: A-only={n01}, B-only={n10}, exact McNemar p={p:.4f}")
```

輸出：

```
accuracy A=0.9722  B=0.9852  n=540
```

```
B - A = +0.0130, 95% bootstrap CI [-0.0019, +0.0278]
discordant pairs: A-only=5, B-only=12, exact McNemar p=0.1435
```

SVM 高出 1.3 個百分點，但區間包含零，McNemar 的 p 值是 0.14。540 張中只有 17 張能區分兩個模型，而 12 比 5 在公平硬幣下並不罕見。以這個測試集而言，誠實的結論是：「B 大概至少一樣好；說它更好的證據很薄弱。」

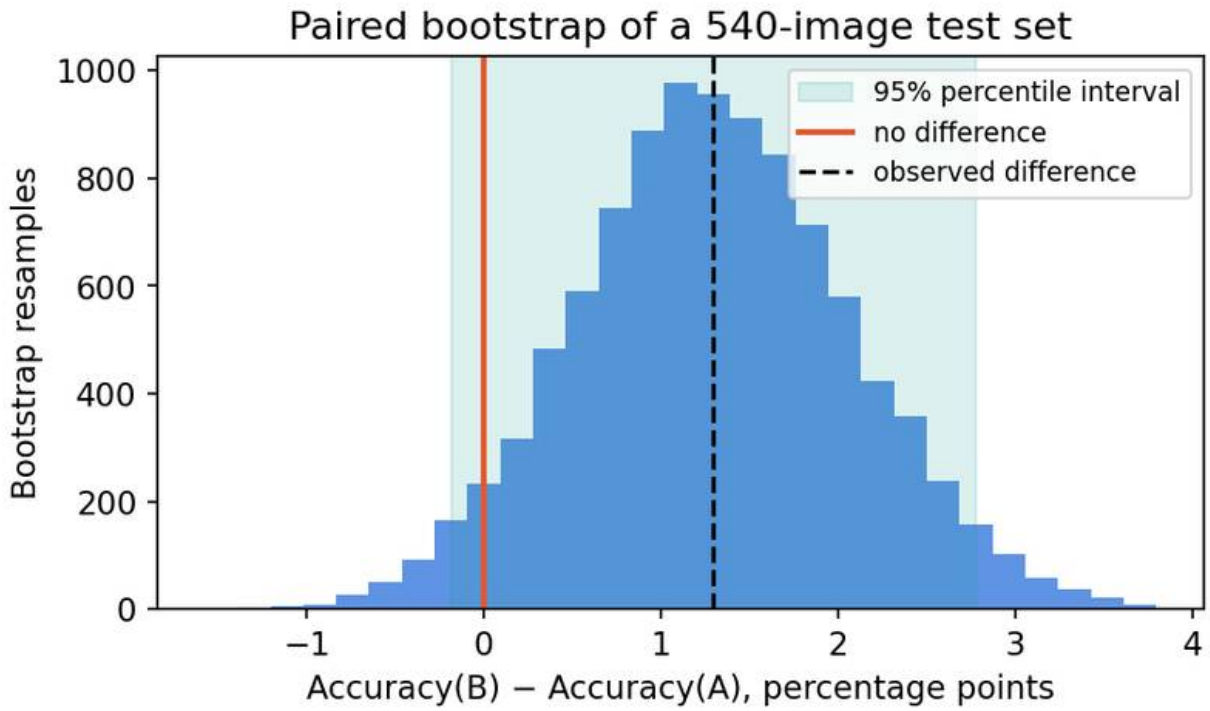


圖 3 — 上方程式碼中 $Accuracy(B) - Accuracy(A)$ 的成對 *bootstrap* 分布。*95%* 區間跨過零。

其他變異來源

測試樣本不是唯一的隨機來源。對神經網路而言，隨機種子會改變初始化、資料順序與資料擴增；資料切分本身也是隨機的；某些 GPU 核心是非確定性的。Bouthillier 等人為整個基準評測過程建模，發現資料取樣、參數初始化與超參數選擇造成的變異會明顯影響結論，並建議在重複實驗中盡可能隨機化這些來源 [10]。實務原則：

- 每個設定用多個種子執行（常見 3–5 個），回報平均與分散程度。
- 用成對檢定，在相同樣本與切分上比較方法。
- 若比較很多模型或很多資料集，總會有一些純屬偶然地「勝出」；要做多重比較校正，或在新資料上確認。
- 回報效果大小與區間。統計顯著的差異在實務上可能無關緊要；實務上很大的差異，在小資料集上也可能尚未被統計證實。

校準：知道自己可能出錯

白話版。模型的信心應該和它答對的頻率相符。在所有以 80% 信心做出的預測中，大約 80% 應該是對的。準確率與校準是兩種不同的性質。

量測機率品質

令 p_{ik} 為樣本 i 屬於類別 k 的預測機率，$y_{ik} \in \{0,1\}$ 為 one-hot 標籤。兩個**合適評分規則**（proper scoring rule，在期望上由真實機率達到最小）為：

$$\mathrm{NLL} = -\frac{1}{n}\sum_{i=1}^{n} \log p_{i,y_i}, \qquad \mathrm{Brier} = \frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} (p_{ik} - y_{ik})^2,$$

其中 y_i 是樣本 i 的真實類別，K 是類別數。NLL 是負對數概似（negative log-likelihood）。有些函式庫會把 Brier 分數再除以 K；請說明你使用哪種慣例。

期望校準誤差（expected calibration error, ECE）依最高類別信心把預測分到 M 個區間 $B_1, \dots, B_M$，並在每個區間比較平均信心與準確率：

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|,$$

其中 $\mathrm{acc}(B_m)$ 是第 m 個區間的答對比例，$\mathrm{conf}(B_m)$ 是該區間的平均信心。**可靠度圖**（reliability diagram）逐區間畫出 acc 對 conf；完美校準的模型落在對角線上。ECE 不是合適評分規則，且取決於區間數與分箱方式，因此應與 NLL 或 Brier 分數及可靠度圖一起回報。

溫度縮放

Guo 等人觀察到，比起數十年前的網路大得多的現代深度網路常常過度自信，而一個純量修正就出奇地有效 [11]。**溫度縮放**（temperature scaling）在 softmax 之前把 logits ℓ 除以 $T > 0$：

$$\hat{p}_k = \frac{\exp(\ell_k / T)}{\sum_j \exp(\ell_j / T)}, \qquad T^\star = \arg\min_{T>0} \ \mathrm{NLL}_{\mathrm{cal}}(T).$$

$T > 1$ 會軟化過度自信的預測；$T < 1$ 則讓信心不足的預測更尖銳。因為把所有 logits 除以同一個正數不會改變誰最大，預測類別（也就是準確率）不變。T 必須在留出的校準集上擬合，絕不能在測試集上擬合。後來 Minderer 等人發現，最新的影像分類器，尤其是 Vision Transformer 這類不使用卷積的模型，屬於校準最好的一群，而且「校準隨分布偏移而衰退」等舊趨勢在它們身上較不明顯 [12]。

程式碼：溫度縮放，以及偏移對它的影響

```
import numpy as np
from scipy.optimize import minimize_scalar
from scipy.special import log_softmax, softmax
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xa, Xte, ya, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=0)
Xtr, Xcal, ytr, ycal = train_test_split(Xa, ya, test_size=0.3, stratify=ya, random_state=0)
net = MLPClassifier(hidden_layer_sizes=(128,), alpha=1e-6, max_iter=300,
                    random_state=0).fit(Xtr, ytr)

def logits(X):                        # log-probabilities = logits up to a constant
    return np.log(np.clip(net.predict_proba(X), 1e-12, 1.0))

def ece(prob, y, bins=10):
    conf, pred = prob.max(1), prob.argmax(1)
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.mean() * abs((pred[m] == y[m]).mean() - conf[m].mean())
    return total

def nll(T, z, y):
    return -log_softmax(z / T, axis=1)[np.arange(len(y)), y].mean()

T = minimize_scalar(nll, bounds=(0.05, 20), method="bounded",
                    args=(logits(Xcal), ycal)).x
print(f"fitted temperature T = {T:.2f}")
noise = np.random.default_rng(0).normal(size=Xte.shape)
for sigma in [0.0, 0.3, 0.6]:         # Gaussian pixel noise = a simple covariate shift
    Xs = np.clip(Xte + sigma * noise, 0, 1)
    z = logits(Xs)
    p_raw, p_ts = softmax(z, axis=1), softmax(z / T, axis=1)
    print(f"noise {sigma:.1f}: acc={(p_raw.argmax(1) == yte).mean():.3f}  "
          f"ECE raw={ece(p_raw, yte):.3f}  ECE scaled={ece(p_ts, yte):.3f}  "
          f"NLL raw={nll(1, z, yte):.3f}  NLL scaled={nll(T, z, yte):.3f}")
```

輸出（scikit-learn 可能印出收斂警告，在這裡無害）：

```
fitted temperature T = 1.19
noise 0.0: acc=0.972  ECE raw=0.007  ECE scaled=0.015  NLL raw=0.108  NLL scaled=0.110
noise 0.3: acc=0.794  ECE raw=0.088  ECE scaled=0.063  NLL raw=0.693  NLL scaled=0.639
noise 0.6: acc=0.435  ECE raw=0.380  ECE scaled=0.347  NLL raw=2.998  NLL scaled=2.598
```

有三點觀察。在乾淨資料上，這個小網路本來就校準得很好，而用 324 張校準影像擬合 T 反而讓 ECE 略微**變差**：在小資料上擬合的修正可能引入雜訊。加入像素雜訊後，模型嚴重過度自信：在 $\sigma = 0.6$ 時它只有 43.5% 答對，信心卻高得多。在乾淨資料上擬合的溫度縮放，在偏移下只幫上一點忙。這與 Ovadia 等人的大型研究一致：他們發現許多不確定性方法的校準會隨偏移增加而惡化 [16]。

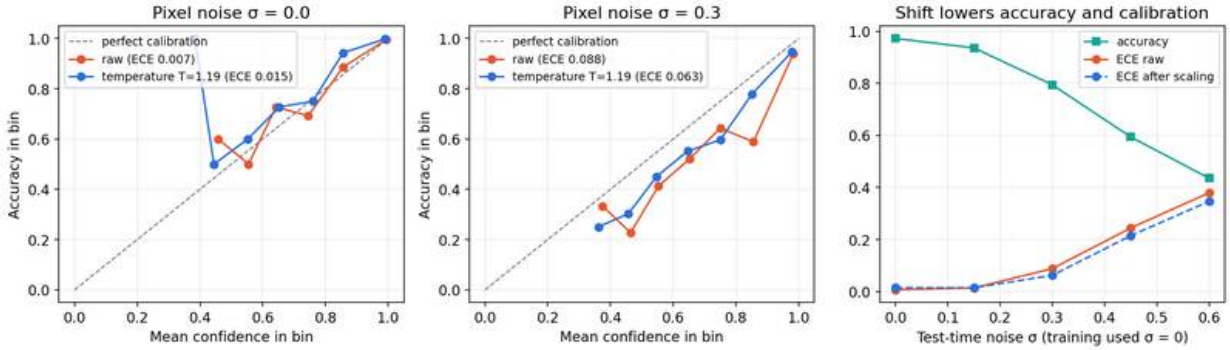


圖 4 — 上述網路的可靠度圖與偏移掃描（使用同一組雜訊樣式，依 σ 縮放）。點落在對角線下方代表過度自信。

穩健性與分布偏移

白話版。部署後的世界很少和訓練集一模一樣。相機會換、光線會變、使用者族群會變。穩健的模型會優雅地退化；脆弱的模型則無聲地失敗。

三種偏移

把聯合分布寫成 $p(x, y) = p(y \mid x)\,p(x) = p(x \mid y)\,p(y)$，其中 x 是輸入、y 是標籤。偏移依訓練時（p_{tr}）與部署時（p_{te}）哪一部分改變來命名：

- **共變量偏移（covariate shift）**：$p_{\text{te}}(x) \neq p_{\text{tr}}(x)$，但 $p(y \mid x)$ 不變。新相機、新光線、新掃描儀。
- **標籤（先驗）偏移（label shift）**：$p_{\text{te}}(y) \neq p_{\text{tr}}(y)$，但 $p(x \mid y)$ 不變。疫情爆發時某種疾病變得更常見。
- **概念偏移（concept shift）**：$p(y \mid x)$ 本身改變。「垃圾郵件」的定義隨時間演變。

在純共變量偏移下，期望測試損失可改寫成加權的訓練損失：$\mathbb{E}_{\text{te}}[\ell] = \mathbb{E}_{\text{tr}}[w(x)\,\ell]$，重要性權重 $w(x) = p_{\text{te}}(x)/p_{\text{tr}}(x)$。對影像而言這些權重很難估計，但這個等式說明了：測試輸入落在訓練資料稀少的區域時，偏移的影響最大。在標籤偏移下，Lipton 等人證明，只要有固定黑箱分類器在留出訓練資料上的混淆矩陣，加上它在未標註測試資料上的預測頻率，就足以估計新的類別先驗並重新加權模型 [17]。

探測偏移的基準

- **ImageNet-C** 對 ImageNet 驗證影像施加 15 種損壞（雜訊、模糊、天氣、數位），並分多個嚴重程度；**ImageNet-P** 則加入擾動序列。兩者量測的是對常見損壞的穩健性，而不是對抗攻擊 [13]。我們的像素雜訊掃描就是一個迷你版本。
- **ImageNetV2** 盡可能依照原始收集流程重新建立 ImageNet 測試集。許多模型的準確率下降了 11–14%，但在原始測試集上的進步轉換成在新測試集上更大的進步，所以這個下降反映的是影像稍微更難，而不是對重複使用的測試集過擬合 [14]。即使是小心的重新收集，本身也是一種偏移。
- **WILDS** 收集了十個具有自然偏移的資料集，包括來自不同醫院的腫瘤組織、新地點的自動相機以及來自新國家的衛星影像，並報告分布內與分布外表現之間有很大落差，而現有的緩解方法都沒能補上 [15]。

實務教訓：一定要加入至少一個依照你預期的部署變化（新院區、新裝置、較晚的時間）建立的**偏移**評估集，並把分布內與偏移後的結果並列回報。

效率：FLOPs、記憶體與延遲

白話版。「快」與「小」各有好幾種意思。運算量較少的模型不一定比較快，參數較少的模型也不一定比較省記憶體。要在目標裝置上量測。

計算運算量

一次**乘加運算**（multiply–accumulate, MAC）是一次乘法加一次加法，所以 1 MAC ≈ 2 個浮點運算（FLOPs）。對有 d_{in} 個輸入、d_{out} 個輸出的全連接層，以及有 C_{in} 個輸入通道、C_{out} 個輸出通道、$k \times k$ 卷積核與 $H' \times W'$ 輸出的卷積層：

$$\text{FLOPs}_{\text{linear}} \approx 2\, d_{\text{in}} d_{\text{out}}, \qquad \text{FLOPs}_{\text{conv}} \approx 2\, C_{\text{in}} k^2 C_{\text{out}} H' W'$$

以上是每個樣本的運算量（偏差與激活函數的成本很小）。各函式庫回報的是 MAC 還是 FLOPs 並不一致，請寫明。

為什麼 FLOPs 不等於速度

Dehghani 等人指出，常見的成本指標（參數量、FLOPs、吞吐量、延遲）可能彼此矛盾，只回報其中一個會讓效率看起來失真 [18]。主要原因：

- **記憶體流量**。若某層必須從記憶體讀取權重，卻只做很少的運算，它受限於記憶體頻寬而非運算。對單一樣本套用線性層時，要讀取 $4 d_{\text{in}} d_{\text{out}}$ 位元組的 fp32 權重，只做 $2 d_{\text{in}} d_{\text{out}}$ 個 FLOPs：每位元組 0.5 個 FLOPs，遠不足以讓現代加速器忙起來。批次化可以提高這個比率（稱為**運算強度**，arithmetic intensity）。
- **平行度**。深度可分離卷積與許多小層的 FLOPs 很少，卻無法充分利用硬體。

- **核心支援**。在目標執行環境上沒有最佳化核心的運算，無論 FLOPs 多少都可能很慢。

記憶體帳

推論時，記憶體要放權重（參數量 × 每個值的位元組數）、目前仍在使用的各層激活值、核心的工作空間，對自迴歸 Transformer 還有會隨上下文長度與批次大小成長的 **KV 快取**。訓練時還要加上梯度與最佳化器狀態：Adam 每個參數多存兩個值，所以使用 Adam 的 fp32 模型在計入激活值之前，就大約需要權重記憶體的 4×。

延遲量測協定

實際延遲取決於硬體、執行環境、批次大小、輸入解析度、數值精度與執行緒設定，而且前幾次呼叫因為一次性的初始化會比較慢。可靠的協定：

1. 固定並回報裝置、執行環境版本、精度、批次大小、輸入尺寸與執行緒數。
2. 先執行暖機迭代並丟棄結果。
3. 計時多次並回報百分位數（p50 與 p90 或 p99），而不只是平均：使用者感受到的是尾端。
4. 在 GPU 上，停止計時前要先同步裝置，因為核心是非同步啟動的。
5. 除了網路本身，也要量測端到端時間（前處理、資料傳輸、解碼）。

柏拉圖前緣（Pareto frontier）包含那些「沒有其他候選在每個面向（準確率、延遲、記憶體）都至少一樣好、且至少在一個面向嚴格更好」的候選模型。呈現前緣能讓取捨一目了然，而不必捏造一個唯一的「最佳模型」。

部署：匯出、壓縮與執行環境

白話版。要上線一個模型，你會把它凍結成可攜的檔案，通常再讓它更小或更省，然後用為目標硬體設計的推論引擎執行。每一步都可能改變輸出，所以每一步都需要重新評估。

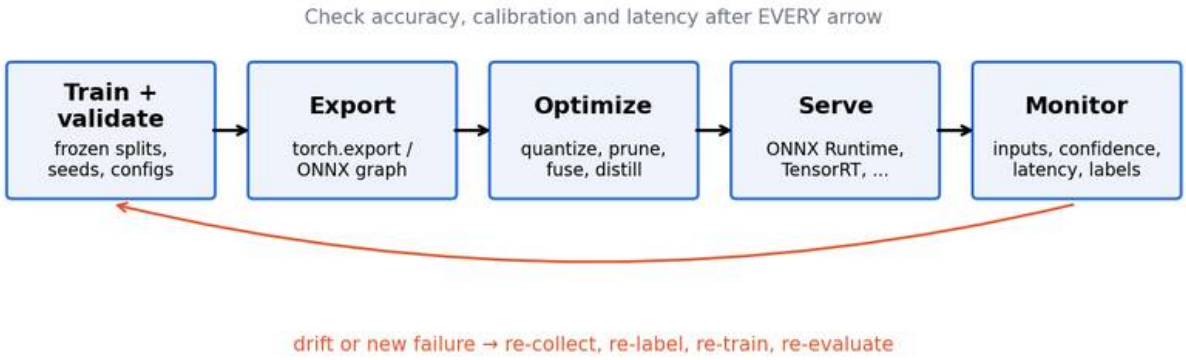


圖 5 — 部署迴圈。每一步之後都應重新檢查準確率、校準與延遲，因為匯出、壓縮與執行環境都會改變數值。

匯出與推論執行環境

訓練程式碼寫在彈性高、由 Python 驅動的框架中。部署則偏好靜態的計算描述：

- `torch.export` 以預先（ahead-of-time）的方式把 PyTorch 模組追蹤成 `ExportedProgram`，也就是一張張量運算圖，可以序列化，並在沒有原始 Python 類別的情況下用不同輸入執行 [19]。
- **ONNX**（Open Neural Network Exchange）是機器學習模型的開放標準檔案格式，讓在一個框架中訓練的模型能被其他工具執行 [20]。
- **ONNX Runtime** 是 ONNX 模型的跨平台推論引擎。它把運算圖切成子圖，交給各個硬體專屬的**執行提供者**（execution provider，例如 CUDA、TensorRT、OpenVINO 或 CoreML）[21]。
- **NVIDIA TensorRT** 是一套 SDK，把模型（來自各框架或 ONNX）編譯成 NVIDIA GPU 上的最佳化引擎，使用層融合與 FP16、BF16、FP8、INT8 等較低精度 [22]。

這些執行環境靠融合運算、為確切的張量形狀挑選快速核心以及降低精度來加速。因此同一個模型在不同執行環境上的輸出可能略有差異。請在一批驗證資料上比較匯出後與原始模型的輸出，並在轉換後重新量測準確率。

量化

量化（quantization）用較少的位元來儲存與計算。在常見的**仿射（非對稱）均勻**量化中，實數值 x 會被映射成整數 q，再映射回來：

$$q = \operatorname{clip}\Big(\operatorname{round}\big(\tfrac{x}{s}\big) + z,\ 0,\ 2^{b}-1\Big), \qquad \hat{x} = s\,(q - z),$$

其中 b 是位元寬度，$s > 0$ 是**尺度**（可表示值之間的間距），z 是整數**零點**，選擇它使實數 0 能被精確表示，$\hat{x}$ 是反量化後的近似值。對包含 0 的範圍 $[x_{\min}, x_{\max}]$，$s = (x_{\max} - x_{\min})/(2^{b}-1)$、$z = \operatorname{round}(-x_{\min}/s)$。**對稱**變體固定 $z = 0$ 並使用有號範圍。Jacob 等人的純整數運算方案說明了如何用 8 位元整數矩陣乘法執行整個網路，以及如何為此進行訓練 [23]。

誤差有兩個來源：範圍內的**捨入**（每個值最多 $s/2$）與範圍外的**截斷**。兩者都取決於尺度的**粒度**：每個張量一個尺度、每個輸出通道一個，或每一小組權重一個。Nagel 等人提供實用指南，涵蓋這些選擇、**訓練後量化**（post-training quantization, PTQ：訓練後用一小份具代表性的資料校準範圍）與**量化感知訓練**（quantization-aware training, QAT：訓練時模擬捨入，讓權重去適應）[24]。Gholami 等人則綜述了更廣的設計空間 [25]。

```
import numpy as np

def quantize(x, bits=8, axis=None):
    """Asymmetric (affine) uniform quantization, per tensor or per channel."""
    qmin, qmax = 0, 2**bits - 1
    lo = x.min(axis=axis, keepdims=True) if axis is not None else x.min()
    hi = x.max(axis=axis, keepdims=True) if axis is not None else x.max()
    lo, hi = np.minimum(lo, 0), np.maximum(hi, 0)      # keep 0 exactly representable
    s = (hi - lo) / (qmax - qmin)                       # scale
    z = np.round(qmin - lo / s)                         # zero point (integer)
```

```
    q = np.clip(np.round(x / s) + z, qmin, qmax)        # stored integers
    return s * (q - z)                                  # dequantized values

rng = np.random.default_rng(0)
# 64 output channels whose weight magnitudes differ by up to 100x
W = rng.normal(size=(64, 256)) * np.logspace(-2, 0, 64)[:, None]
for bits in [8, 4]:
    e_t = np.abs(quantize(W, bits) - W).mean()
    e_c = np.abs(quantize(W, bits, axis=1) - W).mean()
    print(f"{bits}-bit mean |error|: per-tensor {e_t:.5f}  per-channel {e_c:.5f}")
print("zero maps back to:", quantize(np.array([-1.0, 0.0, 3.0]))[1])
```

輸出：

```
8-bit mean |error|: per-tensor 0.00632  per-channel 0.00124
4-bit mean |error|: per-tensor 0.06718  per-channel 0.02112
zero maps back to: 0.0
```

當各通道的範圍差異很大時，共用一個尺度會把大部分格點浪費在最大的通道上，讓小通道被粗糙地捨入。在這裡，逐通道尺度讓 8 位元時的誤差降到約五分之一。在大型語言模型中，少數離群的激活通道會在更大規模上造成同樣的問題，這正是 GPTQ（逐層、補償誤差的權重捨入）與 AWQ（依激活找出並保護重要權重通道）等方法在 3–4 位元純權重量化中處理的問題 [42], [43]。

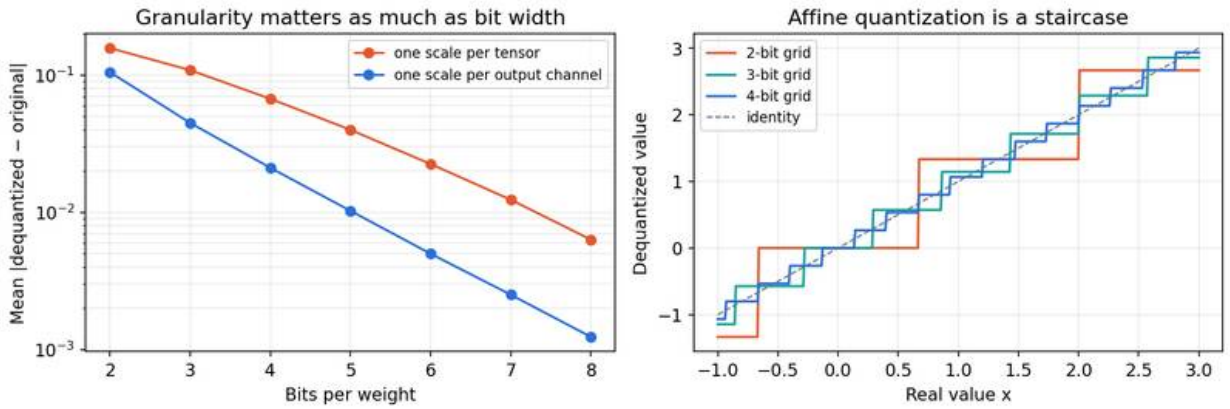


圖 *6* — 每多一個位元，量化誤差大約減半；尺度粒度的選擇則讓整條曲線上下平移。

剪枝

剪枝（pruning）移除貢獻很小的權重、通道、注意力頭或區塊。**非結構化**剪枝把個別權重設為零；它可以移除很大比例的參數，例如 Deep Compression 結合剪枝、以量化實現的權重共享與 Huffman 編碼，大幅縮小網路 [26]。但零星分布的零只有在執行環境具備該稀疏樣式的核心時才會省時間。**結構化**剪枝移除整個通道或注意力頭，產生較小的稠密張量，任何執行環境都能利用。Blalock 等人彙整了 81 篇剪枝論文，發現基準與指標不一致，使人難以判斷哪些方法真的有效，並提出標準化的評測框架 [27]。Hoefler 等人深入綜述了訓練與推論中的稀疏性 [28]。

知識蒸餾

蒸餾（distillation）訓練一個小的**學生**模型去模仿大的**教師**模型。Hinton 等人提出讓學生匹配教師軟化後的輸出分布 [29]。設溫度為 T，教師與學生的 logits 分別為 ℓ^t 與 ℓ^s，軟化機率為 $p^{(T)} = \mathrm{softmax}(\ell/T)$，權重 $\lambda \in [0,1]$：

$$\mathcal{L} = (1-\lambda)\,\mathcal{L}_{\mathrm{CE}}(y,\, p^{s,(1)}) + \lambda\, T^2\, D_{\mathrm{KL}}(p^{t,(T)} \,\|\, p^{s,(T)}).$$

軟化的教師分布帶有「暗知識」：哪些錯誤類別彼此相似。軟項的梯度按 $1/T^2$ 縮放，所以乘上 T^2 能在改變 T 時保持兩項的平衡 [29]。Gou 等人依**傳遞什麼**（輸出、中間特徵、樣本間關係）與**如何傳遞**（離線、線上、自蒸餾）整理了後續研究 [30]。學生會繼承教師的能力，也會繼承它的錯誤與偏差。

程式碼：純權重 int8、延遲與匯出

這個範例把一個小 MLP 的權重以 int8 儲存（每個輸出通道一個尺度），並在 CPU 上量測檔案大小與延遲。只用到 PyTorch 核心功能。

```
import io, time
import numpy as np
import torch
from torch import nn

torch.manual_seed(0)
model = nn.Sequential(nn.Linear(512, 2048), nn.ReLU(),
                      nn.Linear(2048, 2048), nn.ReLU(),
                      nn.Linear(2048, 10)).eval()
x = torch.randn(32, 512)

class Int8WeightLinear(nn.Module):
    """Weight-only int8 storage, symmetric per output channel."""
    def __init__(self, lin):
        super().__init__()
        w = lin.weight.detach()
        scale = w.abs().amax(dim=1, keepdim=True) / 127
        self.register_buffer("q", torch.round(w / scale).to(torch.int8))
        self.register_buffer("scale", scale)
        self.register_buffer("bias", lin.bias.detach().clone())
    def forward(self, x):                       # dequantize, then fp32 matmul
        return nn.functional.linear(x, self.q.float() * self.scale, self.bias)

int8 = nn.Sequential(*[Int8WeightLinear(m) if isinstance(m, nn.Linear) else m
                       for m in model]).eval()

def size_mb(m):
    buf = io.BytesIO()
    torch.save(m.state_dict(), buf)
    return buf.getbuffer().nbytes / 1e6
```

```
def latency_ms(m, x, warmup=10, runs=100):
    with torch.inference_mode():
        for _ in range(warmup):                # first calls pay one-off costs
            m(x)
        times = []
        for _ in range(runs):
            t0 = time.perf_counter()
            m(x)
            times.append((time.perf_counter() - t0) * 1e3)
    return np.percentile(times, [50, 90]).round(2)

with torch.inference_mode():
    gap = (model(x) - int8(x)).abs().max().item()
    agree = (model(x).argmax(1) == int8(x).argmax(1)).float().mean().item()
print(f"fp32 file: {size_mb(model):.1f} MB  p50/p90 latency {latency_ms(model, x)} ms")
print(f"int8 file: {size_mb(int8):.1f} MB  p50/p90 latency {latency_ms(int8, x)} ms")
print(f"max |logit difference| = {gap:.4f}, top-1 agreement = {agree:.3f}")

# Ahead-of-time graph capture: a portable program, independent of this Python class.
exported = torch.export.export(model, (x,))
ops = [n.target for n in exported.graph.nodes if n.op == "call_function"]
print(type(exported).__name__, "->", [str(o).split(".")[1] for o in ops])
```

在 CPU 上（四個執行緒）執行一次的輸出：

```
fp32 file: 21.1 MB  p50/p90 latency [2.74 4.49] ms
int8 file: 5.3 MB  p50/p90 latency [ 9.23 11.06] ms
max |logit difference| = 0.0018, top-1 agreement = 1.000
ExportedProgram -> ['linear', 'relu', 'linear', 'relu', 'linear']
```

你的延遲數字會不同；重點在於趨勢。檔案小了四倍，輸出幾乎沒變。但 int8 版本反而**更慢**，因為它每次呼叫都把權重轉回 fp32，再執行同樣的 fp32 矩陣乘法。只有在執行環境具備該運算的整數（或低精度）核心時，或工作負載受記憶體頻寬限制、讀取較少位元組成為主因時，較低精度才會變快。這正是「壓縮了」與「加速了」之間的落差，也是 TensorRT、ONNX Runtime 以及 PyTorch 自家的 `torchao` 函式庫 [46] 要彌補的地方。這個 MLP 的運算量為每個樣本 $2 \times (512 \cdot 2048 + 2048 \cdot 2048 + 2048 \cdot 10) \approx 10.5$ MFLOPs，兩個版本完全相同。

部署後的監控

白話版。即使程式碼完全沒變，模型也可能變差，因為資料變了。要觀察輸入與輸出，盡可能收集新的標籤，並事先決定什麼情況會觸發調查。

Sculley 等人描述 ML 系統如何累積隱藏的「技術債」：糾纏的特徵、未申報的預測使用者、模型輸出影響未來訓練資料的回饋迴圈，以及依賴會在無預警下改變的外部資料 [33]。Paleyes 等人收集已發表的部署案例研究，發現部署流程的每個階段都會出問題 [39]。有用的訊號，由便宜到最有資訊量：

- **系統健康**：延遲百分位數、錯誤率、記憶體、吞吐量。
- **輸入統計**：影像亮度、解析度、裝置類型、缺失欄位；與訓練時的參考時段比較。
- **輸出統計**：預測類別與信心的分布。這些不需要標籤。
- **延遲到來的標籤**：定期標註一小部分線上資料，是直接估計準確率的唯一方式。

要比較線上時段與參考時段，雙樣本檢定是簡單的起點。對經驗累積分布函數分別為 F_{ref} 與 F_{live} 的一維樣本，**Kolmogorov–Smirnov 統計量**為

$$D = \sup_t |F_{\text{ref}}(t) - F_{\text{live}}(t)|,$$

也就是兩條曲線之間最大的垂直差距。Rabanser 等人比較了多種偏移偵測方法，發現先用預訓練分類器降維（例如取其 softmax 輸出）再做雙樣本檢定，在他們研究的偏移中表現最好 [32]。

```
import numpy as np
from scipy.stats import ks_2samp
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xtr, Xlive, ytr, ylive = train_test_split(X, y, test_size=0.5, random_state=0)
Xtr, Xref, ytr = Xtr[:600], Xtr[600:], ytr[:600]   # reference window from training time
clf = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
ref_conf = clf.predict_proba(Xref).max(1)     # label-free signal: top-class confidence

rng = np.random.default_rng(0)
for name, noise in [("week 1 (clean)", 0.0), ("week 2 (new camera)", 0.25)]:
    batch = rng.choice(len(Xlive), 200, replace=False)
    Xb = np.clip(Xlive[batch] + noise * rng.normal(size=Xlive[batch].shape), 0, 1)
    conf = clf.predict_proba(Xb).max(1)
    stat, p = ks_2samp(ref_conf, conf)
    acc = (clf.predict(Xb) == ylive[batch]).mean()   # usually NOT available live
    print(f"{name}: mean confidence {conf.mean():.3f}, KS={stat:.3f}, p={p:.2g}, "
          f"(hidden) accuracy {acc:.3f}")
```

輸出：

```
week 1 (clean): mean confidence 0.846, KS=0.089, p=0.28, (hidden) accuracy 0.950
week 2 (new camera): mean confidence 0.757, KS=0.277, p=1.3e-08, (hidden) accuracy 0.885
```

信心分布在沒有任何標籤的情況下就標記出第 2 週，而隱藏的準確率證實確實發生了變化。偵測器告訴你資料**有沒有**改變，而不是準確率**是否**下降：無害的偏移也會觸發它，而在很大的時段下，即使微小的變化也會變得「顯著」。用它來觸發調查與標註，而不是自動重新訓練。

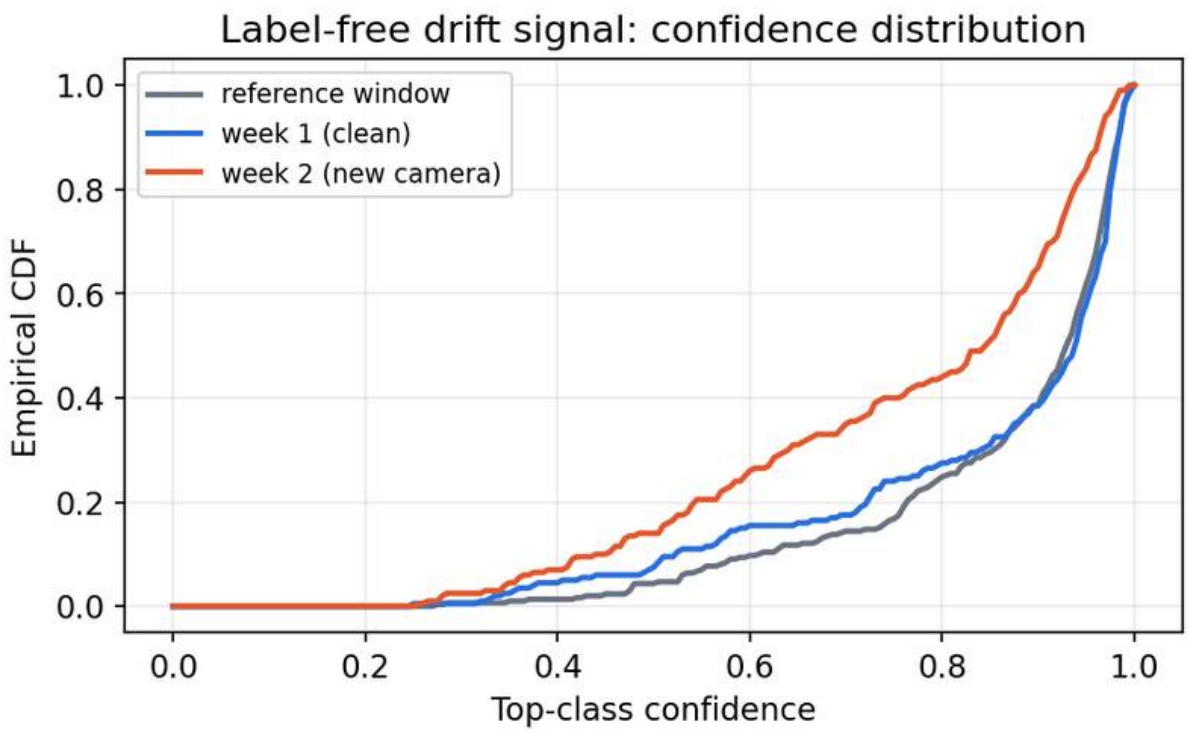


圖 7 — *Kolmogorov–Smirnov* 統計量就是參考曲線與線上時段曲線之間最大的垂直差距。

可重現性與文件

白話版。別人（包括六個月後的你）應該能重跑你的實驗並得到相同的數字，讀者也應該能看出這個模型是做什麼用的、哪些場合不該使用。

讓實驗可重複

為每個函式庫固定並記錄隨機種子，但要知道它的極限：PyTorch 文件明言，不同版本、不同 commit 或不同平台之間無法保證完全可重現的結果，並建議在需要逐位元一致時設定種子、呼叫 `torch.use_deterministic_algorithms(True)` 並關閉 cuDNN benchmark [34]。除了種子，還要記錄：

- 資料集版本、切分 ID（實際的樣本 ID 清單）與去重規則；
- 前處理與資料擴增程式碼、預訓練檢查點及其來源；
- 完整設定（最佳化器、學習率排程、停止規則）、軟體環境與硬體；
- 每一個驗證階段的決策，以及唯一一次的最終測試評估。

Pineau 等人報告的 NeurIPS 2019 可重現性計畫，推出了程式碼提交政策、社群可重現性挑戰，以及正好涵蓋上述項目的檢查表 [37]。

為模型與資料寫文件

模型卡（model cards）是隨訓練好的模型附上的簡短文件，說明預期用途、評估資料、依相關群體與條件細分的表現，以及已知限制 [35]。**資料集說明書**（datasheets for datasets）要求資料集建立者記錄動機、組成、收集過程、前處理、建議用途與維護方式 [36]。兩者都讓本章的評估決策，對依賴它們的人變得透明。

結業實驗

選一個具備適當規模、可合法使用資料集的任務。定義部署母體與切分單位。比較三個候選：一個傳統基準、凍結預訓練特徵加小分類頭，以及一個微調過的網路。對每一個，回報附 bootstrap 區間的測試指標、與最佳基準的成對比較、可靠度圖、一個偏移集上的結果，以及目標裝置上的延遲百分位數。寫一頁模型卡。最終測試集只打開一次；若它揭露了失敗而你修改了模型，就把舊結果視為開發證據，並為修改後的主張取得新的評估。

現代觀點

評估與部署的工程本身已經成為一個研究領域。以下幾篇綜述與回顧勾勒了它的樣貌。

資料洩漏與可重現性。 Kapoor 與 Narayanan 對以 ML 為基礎之科學的回顧，記錄了 17 個領域中 329 篇論文的洩漏問題，並提出「模型資訊表」（model info sheets），要求作者論證其訓練／測試分離為何成立 [4]。結合 Bouthillier 等人的變異研究 [10] 與 NeurIPS 可重現性計畫 [37]，共識是：多數過度樂觀的結果來自評估協定的錯誤，而不是什麼特殊的統計問題。

不確定性與校準。 Gawlikowski 等人綜述深度網路的不確定性估計：他們區分可降低的模型（認知）不確定性與不可降低的資料（偶然）不確定性，把方法分為確定性網路、貝氏網路、集成與測試時資料擴增，並回顧校準方法及其實務限制 [38]。Ovadia 等人則補上實證警告：事後校準在偏移下不夠用，而對多個模型做邊際化的方法（例如集成）表現較好 [16]。

實務部署。 Paleyes 等人綜述已發表的案例研究，把挑戰對應到部署流程的各個階段，並顯示實務工作者在每個階段都會遇到問題 [39]，延伸了 Sculley 等人的技術債觀點 [33]。

模型壓縮。 Cheng 等人提供了 CNN 剪枝、低秩分解、緊湊卷積與蒸餾的經典綜覽 [31]；Gholami 等人 [25] 與 Nagel 等人 [24] 聚焦量化；Hoefler 等人聚焦稀疏性 [28]；Gou 等人聚焦蒸餾 [30]。對大型語言模型，Zhu 等人綜述了針對數十億參數模型的量化、剪枝、蒸餾與低秩方法 [41]。2023–2026 年一個明顯的趨勢是：4 位元純權重量化（GPTQ [42]、AWQ [43]）與 FP8 等低精度格式，已經從論文走進生產工具：TensorRT 列出 FP8、INT8、FP4 與 INT4 等支援精度 [22]，而 PyTorch 的量化工具正整合到 `torchao` 函式庫中 [46]。

全面評估。對基礎模型，HELM 同時在許多情境與多個指標上評估許多模型（準確率、校準、穩健性、公平性、偏差、毒性與效率），並主張單一排行榜數字會掩蓋這些取捨 [40]。同樣的多指標習慣也適用於視覺模型。

規模化。規模定律（scaling laws）擬合模型大小、資料、算力與損失之間的經驗關係。Hoffmann 等人發現在固定算力預算下，語言模型的參數量與訓練 token 數應大致等比例擴增 [44]；Zhai 等人研究 ViT 準確率如何隨模型大小、資料與算力擴展 [45]。一個領域擬合出的常數不應直接搬到另一個領域；可以遷移的教訓是：每個結果都要報告背後的算力與資料。

開放問題。評估資料混入網路規模的預訓練資料（資料污染）很難排除；未知偏移下的校準仍未解決；延遲數字很少能在不同硬體世代之間轉移；生成式與互動式系統的評估（見DLCV 07）仍缺乏公認的協定。

重點整理

- 基準模型與乾淨的切分決定結果是否有意義。依部署時會「新」的單位切分，並把所有前處理放在訓練折之內擬合。
- 交叉驗證在小資料上給出更穩定的估計；樣本相關時用群組或依時間順序的折，在同一份資料上既調參又回報時用巢狀交叉驗證。
- 每個測試分數都需要不確定性。用成對 bootstrap 或 McNemar 檢定，在相同樣本上比較模型。
- 準確率、校準與穩健性是不同的性質。溫度縮放能修正分布內的過度自信，但修不了偏移造成的過度自信。
- FLOPs、參數量、記憶體與延遲可能互相矛盾。要在目標硬體上，以暖機與百分位數量測延遲。
- 量化、剪枝與蒸餾只有在執行環境能利用時才會節省資源；每次匯出或壓縮後都要重新評估準確率。
- 部署後要監控輸入、輸出與少量新標籤，並為模型與資料寫文件，讓別人能重現與判斷。

練習

1. **洩漏稽核。**同事先在全部 1,797 張數字影像上擬合 PCA，再切分訓練與測試並回報測試準確率。說明洩漏在哪裡，以及為什麼 PCA 從未看過標籤卻仍有洩漏。你會如何修正程式碼？

   提示

   主成分方向是用測試影像算出來的，所以表示法被調整得貼近測試分布。把 `PCA` 放進 `Pipeline`，使它只在每個訓練折上擬合。對 PCA 而言這個偏差常常不大，但對使用標籤的監督式前處理（例如依標籤做特徵選擇），偏差可能很大。

2. **罕見的正例。**某資料集只有 1% 是正例影像。永遠預測負例的分類器，準確率、正例召回率與平衡準確率各是多少？你會向醫師回報哪個指標？為什麼？

   提示

   準確率 99%、召回率 0、平衡準確率 0.5（正例召回率 0 與負例召回率 1 的平均）。應回報在臨床選定操作點下的召回率（敏感度）與特異度或精確率，並附上區間，因為真正重要的是漏掉一個正例的代價。

3. **測試集要多大？**你預期準確率約 90%，希望 95% 信賴區間約為 ± 1 個百分點。用常態近似，大約需要多少個獨立測試樣本？如果影像來自 50 位病人，會有什麼不同？

   提示

   解 $1.96\sqrt{0.9 \cdot 0.1/n} = 0.01$，得 $n \approx 3{,}460$。若只有 50 位病人，樣本並不獨立；有效樣本數比較接近 50 而非影像張數，所以區間會寬得多。請用以病人為單位重抽樣的 bootstrap。

4. **手算溫度**。一個三類別模型輸出 logits $(4, 1, 0)$。計算 $T = 1$ 與 $T = 2$ 時最高類別的機率。預測類別會改變嗎？什麼時候擬合出的 T 會小於 1？

   提示

   $T = 1$：$e^4/(e^4 + e^1 + e^0) \approx 0.936$。$T = 2$：$e^2/(e^2 + e^{0.5} + 1) \approx 0.736$。argmax 不變。當模型信心不足時（例如強正則化或標籤平滑之後），擬合出的 $T < 1$。

5. **壓縮的宣稱**。一個剪枝後的模型參數只剩原本的一半，但在你的 GPU 上延遲完全相同。提出三種可能解釋，以及一個能區分它們的實驗。

   提示

   非結構化的零仍被稠密核心拿去相乘；剩下的成本主要來自激活值、記憶體流量或未改變的層；或批次太小，使核心啟動開銷成為主因。逐層剖析時間，並與一個張量真正變小的結構化剪枝模型比較。

6. **設計一個監控方案**。你的瑕疵偵測器在工廠產線上執行；人工檢驗的標籤要兩週後才會回來。設計監控方案：記錄什麼、哪些統計量要與哪個參考比較、什麼門檻會觸發警報，以及警報後要做什麼。

   提示

   每天記錄延遲、影像統計（亮度、模糊程度）、預測瑕疵率與信心直方圖；以 KS 或族群穩定度指標與驗證時的參考時段比較；在持續性變化時才發出警報，而不是單日波動。警報後立即標註一批樣本、計算附區間的準確率，然後才決定要重新校準、微調還是回滾。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 11 “Practical Methodology”. book
2. G. James, D. Witten, T. Hastie, R. Tibshirani and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, Springer, 2023. book
3. scikit-learn developers, “Common pitfalls and recommended practices,” scikit-learn documentation. docs
4. S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023. doi
5. S. Varma and R. Simon, “Bias in error estimation when using cross-validation for model selection,” *BMC Bioinformatics*, vol. 7, art. 91, 2006. doi
6. G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. paper
7. M. Bińkowski, D. J. Sutherland, M. Arbel and A. Gretton, “Demystifying MMD GANs,” ICLR, 2018. arXiv

8. T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Computation*, vol. 10, no. 7, 1998. doi
9. B. Efron, “Bootstrap methods: another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, 1979. doi
10. X. Bouthillier et al., “Accounting for variance in machine learning benchmarks,” MLSys, 2021. arXiv
11. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks,” ICML, 2017. arXiv
12. M. Minderer et al., “Revisiting the calibration of modern neural networks,” NeurIPS, 2021. arXiv
13. D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” ICLR, 2019. arXiv
14. B. Recht, R. Roelofs, L. Schmidt and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?,” ICML, 2019. paper
15. P. W. Koh et al., “WILDS: A benchmark of in-the-wild distribution shifts,” ICML, 2021. paper
16. Y. Ovadia et al., “Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift,” NeurIPS, 2019. arXiv
17. Z. C. Lipton, Y.-X. Wang and A. Smola, “Detecting and correcting for label shift with black box predictors,” ICML, 2018. arXiv
18. M. Dehghani, A. Arnab, L. Beyer, A. Vaswani and Y. Tay, “The efficiency misnomer,” ICLR, 2022. arXiv
19. PyTorch, “torch.export,” PyTorch documentation. docs
20. ONNX, “Open Neural Network Exchange: the open standard for machine learning interoperability.” site
21. ONNX Runtime, “Welcome to ONNX Runtime,” documentation. docs
22. NVIDIA, “NVIDIA TensorRT documentation.” docs
23. B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” CVPR, 2018. doi
24. M. Nagel et al., “A white paper on neural network quantization,” arXiv:2106.08295, 2021. arXiv
25. A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” book chapter in *Low-Power Computer Vision*; arXiv:2103.13630, 2021. arXiv
26. S. Han, H. Mao and W. J. Dally, “Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding,” ICLR, 2016. arXiv
27. D. Blalock, J. J. Gonzalez Ortiz, J. Frankle and J. Guttag, “What is the state of neural network pruning?,” MLSys, 2020. arXiv
28. T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden and A. Peste, “Sparsity in deep learning: pruning and growth for efficient inference and training in neural networks,” *Journal of Machine Learning Research*, vol. 22, 2021. arXiv
29. G. Hinton, O. Vinyals and J. Dean, “Distilling the knowledge in a neural network,” NIPS Deep Learning Workshop, 2014; arXiv:1503.02531. arXiv
30. J. Gou, B. Yu, S. J. Maybank and D. Tao, “Knowledge distillation: a survey,” *International Journal of Computer Vision*, vol. 129, 2021. doi
31. Y. Cheng, D. Wang, P. Zhou and T. Zhang, “Model compression and acceleration for deep

neural networks: the principles, progress, and challenges,” *IEEE Signal Processing Magazine*, vol. 35, no. 1, 2018. doi arXiv
32. S. Rabanser, S. Günnemann and Z. C. Lipton, “Failing loudly: an empirical study of methods for detecting dataset shift,” NeurIPS, 2019. arXiv
33. D. Sculley et al., “Hidden technical debt in machine learning systems,” NeurIPS, 2015. paper
34. PyTorch, “Reproducibility,” PyTorch documentation. docs
35. M. Mitchell et al., “Model cards for model reporting,” ACM FAT*, 2019. arXiv
36. T. Gebru et al., “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, 2021. doi
37. J. Pineau et al., “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, 2021. paper
38. J. Gawlikowski et al., “A survey of uncertainty in deep neural networks,” *Artificial Intelligence Review*, vol. 56, 2023. doi
39. A. Paleyes, R.-G. Urma and N. D. Lawrence, “Challenges in deploying machine learning: a survey of case studies,” *ACM Computing Surveys*, vol. 55, no. 6, 2022. doi
40. P. Liang et al., “Holistic evaluation of language models,” *Transactions on Machine Learning Research*, 2023. arXiv
41. X. Zhu, J. Li, Y. Liu, C. Ma and W. Wang, “A survey on model compression for large language models,” *Transactions of the Association for Computational Linguistics*, vol. 12, 2024. doi
42. E. Frantar, S. Ashkboos, T. Hoefler and D. Alistarh, “GPTQ: accurate post-training quantization for generative pre-trained transformers,” ICLR, 2023. arXiv
43. J. Lin et al., “AWQ: activation-aware weight quantization for LLM compression and acceleration,” MLSys, 2024. arXiv
44. J. Hoffmann et al., “Training compute-optimal large language models,” arXiv:2203.15556, 2022; NeurIPS 2022 version titled “An empirical analysis of compute-optimal large language model training.” arXiv
45. X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer, “Scaling vision transformers,” CVPR, 2022. doi
46. PyTorch, “torchao documentation.” docs

篇 II

電腦視覺深度學習

DLCV 01・深度學習的影像處理與電腦視覺

深度學習如何重新整理影像處理、影像分析與電腦視覺：從傳統圖形識別流程，到 CNN、Vision Transformer、自監督骨幹網路與可提示的基礎模型。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-01-deep-learning-for-image-processing/

你將學到

- 傳統影像工作的三個層次（影像處理、影像分析、電腦視覺），如何對應到深度學習實際使用的兩大類：**低階**（low-level）與**高階**（high-level）視覺任務。
- 為什麼教科書上的圖形識別流程（感測器 → 特徵 → 分類器 → 驗證）會被**端對端學習**（end-to-end learning）取代，以及它又如何以「預訓練骨幹＋輕量 head」的形式悄悄回歸。
- 卷積層為什麼就是一組**學習而來的空間濾波器**，以及卷積核大小、步幅、填補與池化如何決定**感受野**（receptive field）與特徵階層。
- 經典 CNN（LeNet-5、AlexNet、VGG、GoogLeNet、ResNet）背後的設計理由，以及用於密集預測的編碼器–解碼器（FCN、U-Net）。
- **Vision Transformer** 怎麼運作、為什麼需要大規模預訓練，以及 Swin、ConvNeXt 與自監督骨幹網路（DINO、MAE、DINOv2/v3）各自如何回應它。
- 現今任務與代表模型的全貌：分類、偵測、分割（SAM 系列）、復原與增強、生成（擴散模型、潛在擴散、DiT），以及視覺–語言模型（CLIP、LLaVA）。

先看全貌

這篇是深度學習系列的入口。它不會深入任何單一任務，而是先給你一張地圖：影像問題有哪些種類、哪些網路設計能解決它們、這些設計又為什麼長成現在這樣。後續的教學會再分別深入影像分類、物件偵測、影像復原、低光源增強與物件追蹤。

> **用最簡單的話說**
>
> 想像一間照相館裡有兩種員工。**修圖師傅**拿到一張刮傷、模糊或太暗的照片，交回一張比較乾淨的照片：照片進、照片出。**偵探**看著一張照片，告訴你裡面**有什麼**：「一隻貓，坐在沙發上，在左邊。」以前的電腦視覺，是由專家幫這兩位員工準備好一套固定的工具。深度學習則讓他們看過幾百萬個例子之後，自己打造工具。現在甚至出現了一位「超級員工」，看過數十億張圖，你只要用幾個字告訴他要做什麼就好。

為什麼重要：幾乎所有現代相機的處理流程、醫學影像產品、駕駛輔助系統與修圖 App，都是由本篇介紹的元件組成。看懂這張地圖，你就能為新問題挑一個合理的起點、帶著批判的眼光讀論文，也能看出數位影像處理系列裡的哪些傳統想法仍活在網路裡面（其實很多）。

關於本篇

本篇改寫並擴充自作者先前的文章〈深度學習的數位影像處理介紹〉[1]，保留原文的架構與觀點，並補上細節、數學與程式碼。

影像任務的三個層次

白話版。廣義的「影像處理」包含三種工作。有些操作把一張影像變成一張更好的影像；有些操作從影像中取出某個部分或某種性質；有些則試著像人一樣**理解**影像。

精確版。依照 Gonzalez 與 Woods [2] 的分法，可以用「輸入什麼、輸出什麼」把影像工作分成三個層次。

層次	輸入 → 輸出	典型操作	傳統例子
影像處理（狹義；低階）	影像 → 影像	逐像素或鄰域的基本步驟	直方圖等化、濾波、傅立葉轉換（IP 03、IP 04）
影像分析（中階）	影像 → 屬性或被標示出的區域	分割、特徵擷取、描述，運用先驗知識	人臉影像中的膚色區域、X 光影像中的血管區域（IP 10、IP 12）
電腦視覺（高階；理解）	影像 → 意義	辨識、詮釋、判斷	分類、「畫面中有沒有行人？」（IP 13）

這三者的界線並不嚴格，實際系統往往三者都會用到。例如一個血管分割工具，會先去雜訊（低階）、再擷取血管圖（中階），最後可能標記出狹窄處（高階）。這三個層次描述的是**計算的種類**，而不是三個分開的部門。

深度學習如何重畫這張地圖

在深度學習的論文裡，你幾乎看不到「中階」這個詞。這個領域依照**輸出的形式**，只把任務分成兩大類 [1]：

- **低階視覺**：輸出是一張和輸入（大致）同樣大小的影像，目標是逐像素準確的訊號。例如去雜訊、去模糊、超解析度、除霧、低光源增強、影像到影像轉換。
- **高階視覺**：輸出是**語意**描述，可能是一個標籤、一組帶標籤的方框，或是每個像素一個標籤。

原本的「影像分析」被併入兩者之一。分割的輸出是逐像素的圖，**看起來**像影像到影像的任務；但它的數值是需要辨識才能得到的類別標籤，所以被歸為高階視覺。至於輸出一張更乾淨的亮度影像的邊緣或血管增強，則歸為低階視覺。

這樣分的理由很實際。兩類任務在損失函數（像素迴歸 vs. 分類）、架構（全解析度的編碼器–解碼器 vs. 逐步降採樣的骨幹網路）、訓練資料（劣化/乾淨影像對 vs. 人工標註）與評估指標（保真度 vs. 正確率）上都不同。圖 1 畫出了這個重新整理。

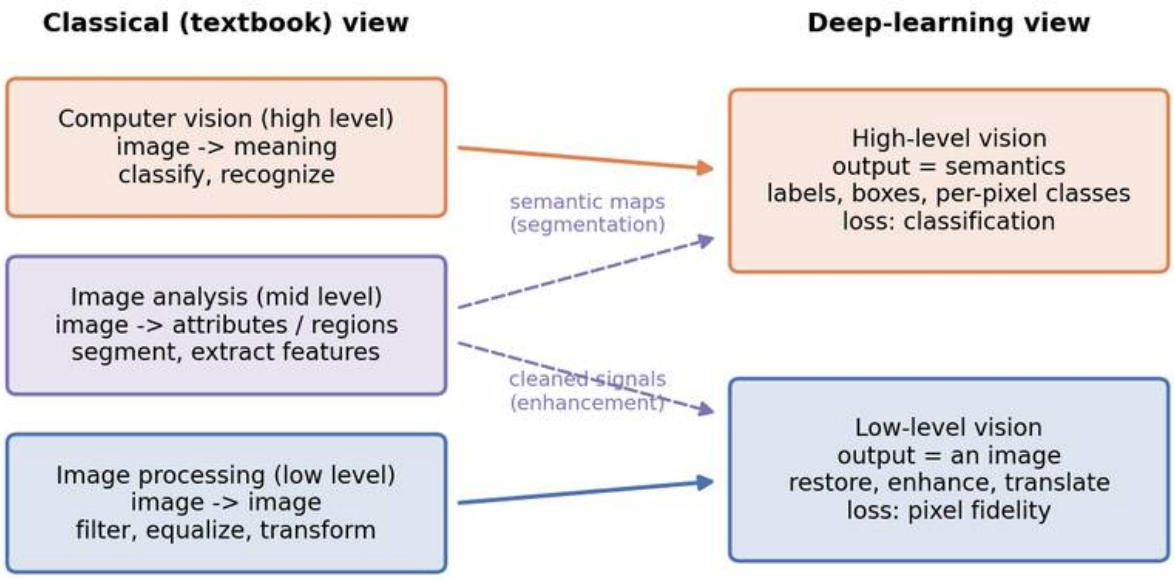


圖 1 — 傳統的三個層次（左）與深度學習使用的兩大類（右）。中階的影像分析依輸出是「清理過的訊號」還是「語意圖」而被分到兩邊。

從圖形識別流程到端對端學習

白話版。在深度學習之前，辨識系統像一條生產線：先有相機，再由人設計的步驟量出有用的性質，最後由分類器做決定。深度學習把中間的步驟融合成一個網路，讓它自己學會要量什麼。

精確版。傳統流程 [3] 有四個階段：

1. **感測器**（sensor）。擷取圖形（相機、掃描器、顯微鏡）。
2. **特徵生成與選擇**（feature generation & selection）。以人工設計的映射 ϕ（動差、紋理統計、邊緣直方圖、關鍵點描述子），從影像 I 計算出向量 $\mathbf{x} = \phi(I) \in \mathbb{R}^d$，再保留最有資訊量的分量。
3. **分類器設計**（classifier）。學習決策規則 $\hat{y} = g(\mathbf{x}; \theta)$，$\theta$ 為參數，例如貝氏分類器、最近鄰規則或支援向量機。
4. **系統驗證**（evaluation）。在保留的資料上量測錯誤率，太高就回到第 2 或第 3 步。

被學習的只有 θ。特徵映射 ϕ 由工程師固定，所以整個系統的上限就是 ϕ 的品質。特徵擷取（IP 12）那一章可以看到，這些映射背後花了多少巧思。

端對端學習讓特徵映射也變成可學習的。深度網路計算

$$\hat{y} = g(\phi(I; \mathbf{w}); \theta),$$

其中 $\phi(\cdot\,; \mathbf{w})$ 是權重為 $\mathbf{w}$ 的多層網路，$g(\cdot\,; \theta)$ 是一個小的 head，$\mathbf{w}$ 與 θ 在標註資料 (I, y) 上以梯

度下降與反向傳播共同最小化損失 $\mathcal{L}(\hat{y}, y)$。輸入就是原始像素；除了架構與損失函數之外，沒有任何東西是人工設計的。

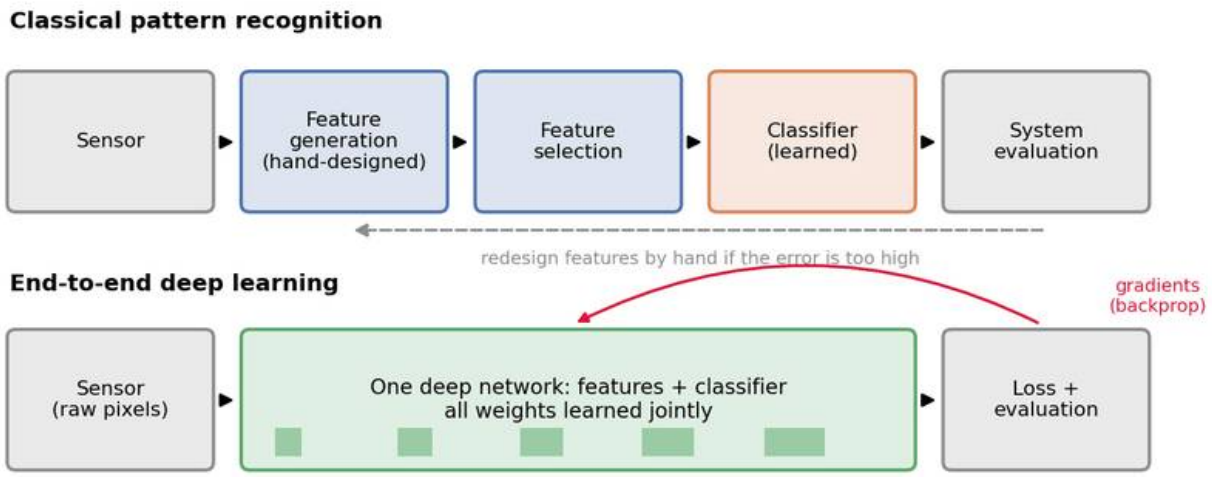


圖 2 — 傳統圖形識別（上）與端對端學習（下）。深度網路裡「特徵」與「分類器」的功能區分仍然存在，但兩者之間沒有清楚的分界。

網路內部的功能區分其實還在：前面的層像特徵擷取器，最後幾層像分類器。但實務上，你通常無法指著某一層說「特徵到這裡結束」[1]。這帶來兩個結果：

- **資料取代了設計**。端對端系統需要大量標註資料；人工設計的 ϕ 原本承載的先驗知識，現在得由網路從資料中學到。
- **分工以更大的規模回歸**。大約從 2021 年起，標準作法是用一個**預訓練骨幹**（在巨大的資料集上訓練一次，常常不需要標註）加上一個在你的小資料集上訓練的**輕量任務 head**。這又是「特徵擷取器＋分類器」，只是特徵擷取器是學來的，而不是設計出來的。我們會在自監督骨幹那一節回到這一點。

卷積神經網路

白話版。卷積層拿一個小視窗在影像上滑動，在每個位置計算加權和，跟 IP 03 的空間濾波器一模一樣。差別在於：CNN 會**學習**濾波器的權重、一次使用很多個濾波器，並把數十層這樣的層與非線性函數交錯疊起來。

卷積層是一組學習而來的濾波器

精確版。設輸入特徵圖為 $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$（通道、高、寬）。一個有 C_{out} 個 $k \times k$ 濾波器的卷積層計算

$$Y_o(x,y) = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i=-r}^{r} \sum_{j=-r}^{r} W_{o,c}(i,j)\, X_c(x+i, y+j), \qquad o = 1, \dots, C_{\text{out}},$$

其中奇數 k 時 $r=(k-1)/2$，$W_{o,c}(i,j)$ 是第 o 個濾波器在輸入通道 c、位移 (i,j) 的權重，b_o 是偏差。嚴格來說這是**相關**（correlation，不翻轉卷積核），如卷積教學所述；由於權重是學出來的，翻不翻轉沒有差別。之後接一個逐元素的非線性函數，例如 $\mathrm{ReLU}(z)=\max(0,z)$。如果沒有它，一疊卷積層會塌縮成一個大的線性濾波器 [1]。

有兩個性質讓這種層特別適合影像：

- **局部連接**（local connectivity）。每個輸出只依賴一個 $k\times k$ 鄰域，因為相鄰像素彼此高度相關，距離遠的像素則關係弱得多。
- **權重共享**（weight sharing）。同一個濾波器用在每一個位置。這讓卷積層具有**平移等變性**（translation equivariance：輸入平移，輸出也跟著同樣平移），也大幅減少參數量。

省下的量非常可觀。從 $224\times224\times3$ 的影像連到 64 個神經元的全連接層需要 $224\cdot224\cdot3\cdot64+64\approx960$ 萬個參數。從 3 個通道到 64 個通道的 3×3 卷積只需要 $3\cdot3\cdot3\cdot64+64=1{,}792$ 個，而且輸出的是 64 張**全解析度**的特徵圖，而不是 64 個數字。

下面的程式把 PyTorch 卷積層的兩個濾波器手動設成 Sobel 卷積核，讓這一層變成邊緣偵測器。訓練則會改由梯度下降來決定這些數字。

```
import torch, torch.nn as nn, torch.nn.functional as F
from skimage import data, img_as_float

x = torch.tensor(img_as_float(data.camera()), dtype=torch.float32)[None, None]  # (1, 1,
 ↪  512, 512)
conv = nn.Conv2d(1, 2, kernel_size=3, padding=1, bias=False)
sobel_x = torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]])
with torch.no_grad():
    conv.weight[0, 0] = sobel_x        # 通道 0：對垂直邊緣有反應
    conv.weight[1, 0] = sobel_x.T      # 通道 1：對水平邊緣有反應
    y = F.relu(conv(x))                # (1, 2, 512, 512)：兩張特徵圖
```

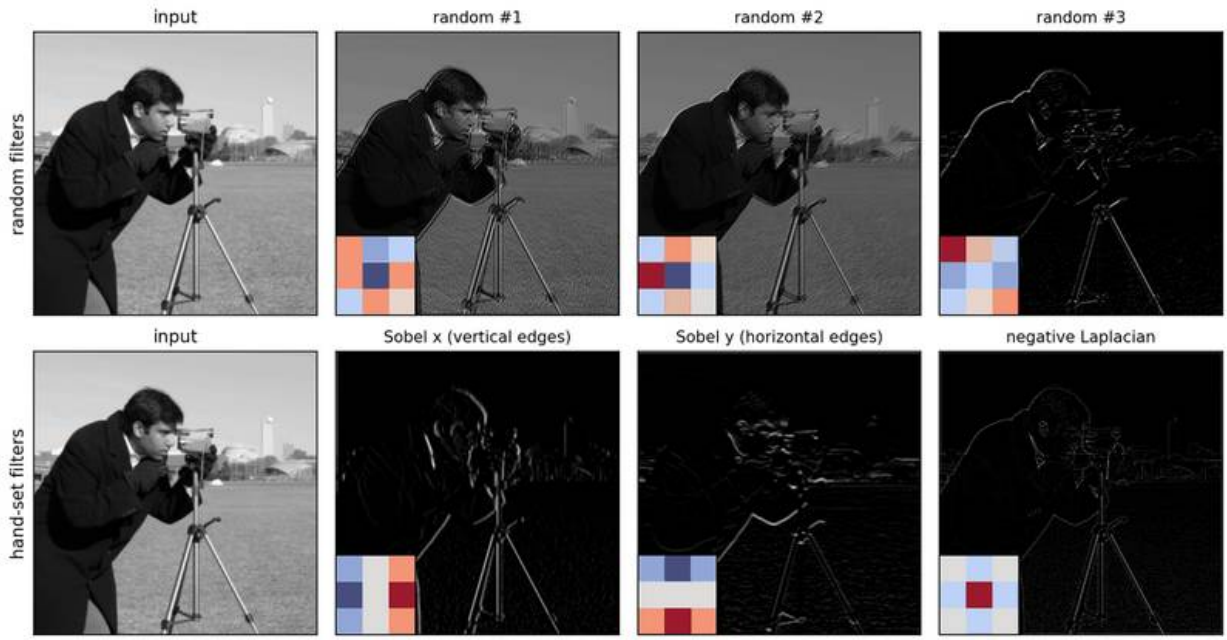


圖 *3* — 攝影師影像經 *3×3* 卷積（並經過 *ReLU*）的特徵圖。上：隨機初始權重，得到平滑與邊緣混雜的結果。下：手動設定的 *Sobel* 與 *Laplacian* 權重。訓練會把隨機濾波器推向有用的濾波器。

訓練好的第一層濾波器長什麼樣子？我們可以用 scikit-learn 內建的 8×8 手寫數字 [36]，在 CPU 上幾秒鐘就訓練完一個極小的 CNN。

```
import torch, torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
X, y = load_digits(return_X_y=True)                    # 1,797 張 8x8 影像
X = torch.tensor(X / 16.0, dtype=torch.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, torch.tensor(y), test_size=0.25, random_state=0)

net = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),     # 8x8 -> 4x4
    nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # 4x4 -> 2x2
    nn.Flatten(), nn.Linear(16 * 2 * 2, 10),
)
opt = torch.optim.Adam(net.parameters(), lr=1e-2)
for epoch in range(30):
    for i in range(0, len(Xtr), 64):
        loss = nn.functional.cross_entropy(net(Xtr[i:i + 64]), ytr[i:i + 64])
        opt.zero_grad(); loss.backward(); opt.step()
print((net(Xte).argmax(1) == yte).float().mean())       # 約 0.97
filters = net[0].weight.detach()[:, 0]                   # (8, 3, 3) 學到的卷積核
```

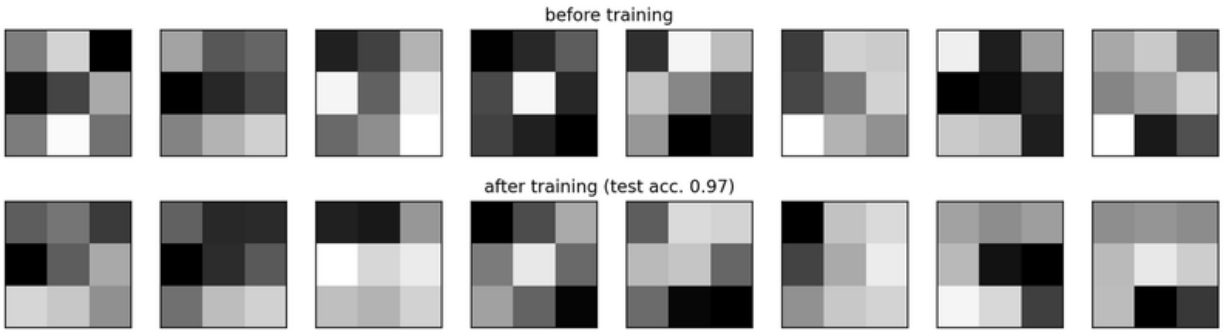


圖 *4* — 小型數字 *CNN* 的八個第一層 *3×3* 濾波器，訓練前（上）與訓練後（下）。其中幾個朝有方向性的明暗轉換移動，也就是人工設計者會挑選的類微分圖樣；沒有人告訴網路要這麼做。（*8×8* 的小輸入與 *3×3* 卷積核讓效果較不明顯；在自然影像上訓練的大型網路會更清楚地呈現這個現象。）

步幅、填補與輸出大小

白話版。填補（padding）在影像外圍加一圈邊框，讓視窗可以停在邊緣像素上。步幅（stride）讓視窗一次跳不只一個像素，因此輸出會變小。

精確版。對寬度為 W 的輸入、卷積核大小 k、每邊填補 p 個像素、步幅 s，輸出寬度為

$$W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1.$$

$k = 3$、$p = 1$、$s = 1$ 時大小不變（“same” 填補）；$s = 2$ 時減半。預設是補零；但在低階網路中，常偏好鏡射或複製邊界的填補（與 IP 03 討論的邊界處理相同），因為補零會在影像邊緣產生暗色瑕疵。

池化

池化（pooling）用一個數字概括每個小視窗，通常取最大值（max pooling）或平均值（average pooling），一般使用 2×2 視窗、步幅 2。它把解析度減半、讓表示對小幅平移較不敏感，並以很低的成本擴大感受野。許多現代網路改用步幅卷積取代池化，讓網路自己學會怎麼降採樣。在分類網路的最後，**全域平均池化**（global average pooling）把 $C \times H \times W$ 的特徵圖變成給分類 head 用的 C 維向量。

感受野

白話版。一個神經元的感受野，是輸入影像中能影響它的那一塊區域。一層 3×3 看得到 3×3 個像素；疊兩層，每個輸出看得到 5×5。降採樣會讓它長得快得多。

精確版。將各層編號為 $\ell = 1, \dots, L$，卷積核大小 k_ℓ、步幅 s_ℓ。令 j_ℓ 為**跳距**（jump），也就是第 ℓ 層兩個相鄰單元在輸入影像上相距多少像素；r_ℓ 為感受野大小；輸入層 $r_0 = 1$、$j_0 = 1$。則

$$r_\ell = r_{\ell-1} + (k_\ell - 1)\, j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\, s_\ell .$$

每一層增加 $k_\ell - 1$ 個輸入單元，而每個輸入單元相當於 $j_{\ell-1}$ 個像素。若處處步幅為 1，L 層 3×3 得到 $r_L = 2L + 1$：線性成長。每一次步幅 2 都讓跳距加倍，後面的層讓感受野長得兩倍快，因此隨深度大致呈指數成長。

```
def receptive_field(layers):
    r, j = 1, 1                       # 感受野與跳距，以輸入像素為單位
    for k, s in layers:               # 每一層的（卷積核大小，步幅）
        r, j = r + (k - 1) * j, j * s
    return r

print(receptive_field([(3, 1)] * 3))                                          # 7
print(receptive_field([(3, 1), (3, 1), (2, 2), (3, 1), (3, 1), (2, 2), (3, 1)])) # 24
```

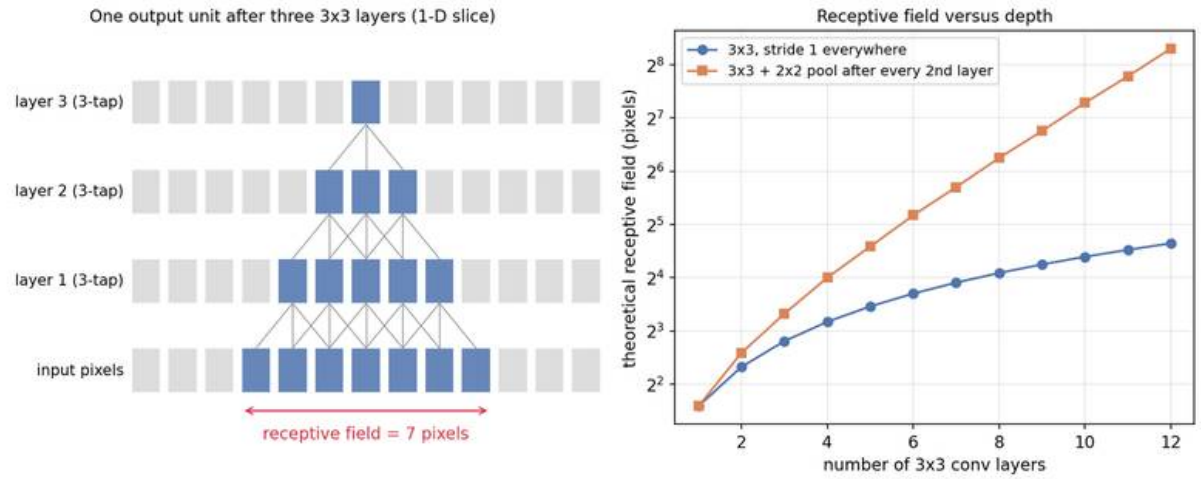


圖 *5* — 左：疊三層 *3* 抽頭的層，讓一個輸出擁有 *7* 個像素的感受野。右：理論感受野對深度的關係，比較處處步幅 *1* 與每兩層之後做一次 *2* 倍降採樣。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

實務上還有兩點要注意。第一，上面的公式給的是**理論**感受野。Luo 等人 [12] 發現，以「每個輸入像素對輸出梯度實際影響多少」來量測的**有效**感受野（effective receptive field）大致呈高斯分佈，只佔理論感受野的一小部分，中心附近的像素佔主導地位。第二，感受野決定網路**能**用到什麼：感受野比雜訊相關長度還小的去雜訊器，無法去除那種雜訊；最後一層只看得到物體一部分的分類器，也無法從上下文辨識它。

特徵階層

由於感受野隨深度變大、解析度變小，CNN 會建立一個**特徵階層**：前面的層對邊緣與色塊有反應（圖 4），中間的層對紋理與部件有反應，後面的層則對大範圍的物體層級圖樣有反應。典型的

骨幹網路分成好幾個**階段**（stage），每個階段結束時做 2 倍降採樣並把通道數加倍，所以特徵從「高解析度、通道少、局部」一路變成「低解析度、通道多、語意」。高階任務讀取後面的階段；密集預測與低階任務還需要前面高解析度的階段，這正是編碼器–解碼器存在的原因（見下文）。

經典 CNN 架構

白話版。 2012 到 2016 年間，一系列一個比一個深的網路贏得了 ImageNet 大規模視覺辨識挑戰賽（ILSVRC）[6]。每一個都帶來一個至今仍在使用的設計想法。

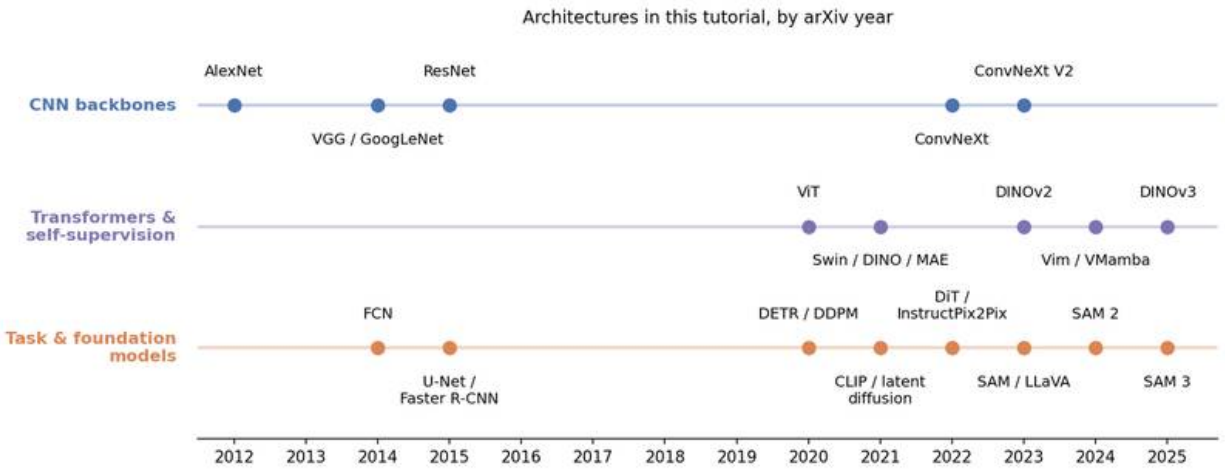


圖 *6* — 本篇提到的架構時間軸（依 *arXiv* 年份）。*LeNet-5（1998）*與 *1980* 年代末更早的卷積網路位於這條軸的左側之外。

LeNet-5，以及 CNN 真正的起點

原文的標題寫著「CNN（1998~）」。1998 年是 LeCun、Bottou、Bengio 與 Haffner 發表 LeNet-5 [5] 的年份，並不是 CNN 的起點：以反向傳播訓練的卷積網路，早在 1989 年就已經用來辨識手寫郵遞區號 [4]。

LeNet-5 是一個以網路形式呈現的標準圖形識別系統 [1]。C1 到 C5 是**特徵擷取器**：卷積層（C1、C3、C5）與子取樣層（S2、S4）交錯，把 32×32 的數字影像變成 120 維的特徵向量；F6 與輸出層

則是**分類器**。包含特徵在內的所有東西，都以分類損失透過梯度下降訓練。端對端的想法其實早已到位，只是還需要再等十五年的資料量與運算力，才成為主流。

AlexNet（2012）

AlexNet [7] 有五個卷積層與三個全連接層（共八個可學習的層，約 6,000 萬個參數）。它的元件個別來看並不新，新的是組合：訓練速度遠快於飽和型 sigmoid 的 ReLU 激活函數、高效率的 GPU

實作、大量的資料擴增，以及全連接層中的 dropout。它以很大的差距贏得 ILSVRC 2012，這也是為什麼 2012 年通常被視為視覺深度學習時代的起點。

VGG（2014）

Simonyan 與 Zisserman [8] 只問了一個問題：如果只用 3×3 卷積，並把網路加深，會怎樣？他們最好的配置有 16 與 19 個權重層。3×3 之所以足夠，答案就在上面的感受野公式：兩層 3×3 看得到 5×5，三層看得到 7×7。輸入、輸出都是 C 個通道時，三層 3×3 需要 $3\cdot 9C^2 = 27C^2$ 個權重，而一層 7×7 需要 $49C^2$ 個，而且前者還多插入了兩個非線性函數。VGG 整齊一致的設計，讓它多年來都是預設的預訓練特徵擷取器。

GoogLeNet / Inception（2014）

22 層深的 GoogLeNet [9] 贏得 ILSVRC 2014。它的 **Inception 模組**平行執行 1×1、3×3、5×5 卷積與一個池化分支，再把輸出串接起來，讓每個模組同時看到好幾種尺度。為了負擔得起，便宜的 1×1 卷積會先減少通道數。1×1 卷積在每個像素上混合通道而不看鄰居，相當於逐像素的全連接層，至今仍是到處可見的基本元件。

ResNet（2015），以及殘差連接為什麼有效

照理說越深越好，但實際上並非如此：He 等人 [10] 觀察到，一個普通的 56 層網路，**訓練誤差**反而比 20 層的高。這不是過擬合（overfitting），而是最佳化失敗：較深的網路理論上可以複製淺網路、再把多出來的層設成恆等映射，但梯度下降找不到那個解。

解法是**殘差區塊**（residual block，圖 7）。與其讓一疊層直接學習映射 $H(\mathbf{x})$，不如讓它學殘差 $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$，再把輸入加回來：

$$\mathbf{y} = \sigma(\mathbf{x} + F(\mathbf{x}; W)),$$

其中 $\mathbf{x}$ 是區塊輸入，F 通常是 conv–BN–ReLU–conv–BN（BN 為批次正規化，batch normalization），權重為 W，σ 為 ReLU。當通道數或解析度改變時，捷徑改用 1×1 卷積取代恆等映射。

為什麼有效：

- **恆等映射變得容易**。如果多出來的深度沒有用處，區塊只需要把 F 推向零，這比讓一疊非線性層模仿恆等映射容易得多。把 F 最後一個 BN 的縮放參數初始化為零，每個區塊一開始就是恆等映射。
- **梯度有一條高速公路**。忽略最後的非線性，$\partial \mathbf{y}/\partial \mathbf{x} = I + \partial F/\partial \mathbf{x}$。恆等項把梯度從損失直接帶到前面的層，不必穿過每一個權重矩陣，所以不會隨深度消失。
- **集成的觀點**。把 L 個區塊展開，會得到許多長短不一的路徑之和，因此一個很深的 ResNet 在某種程度上就像許多較淺網路的集合。

```
class ResidualBlock(nn.Module):
    """ 基本區塊：y = ReLU(x + F(x))，F = conv-BN-ReLU-conv-BN。"""
```

```
    def __init__(self, c):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c), nn.ReLU(),
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c),
        )
    def forward(self, x):
        return torch.relu(x + self.f(x))

blk = ResidualBlock(16)
nn.init.zeros_(blk.f[4].weight)                  # 把最後一個 BN 的縮放設為零，使 F(x) = 0
x = torch.randn(2, 16, 32, 32)
print(torch.allclose(blk(x), torch.relu(x)))     # True：區塊一開始就是恆等映射（再接 ReLU）
```

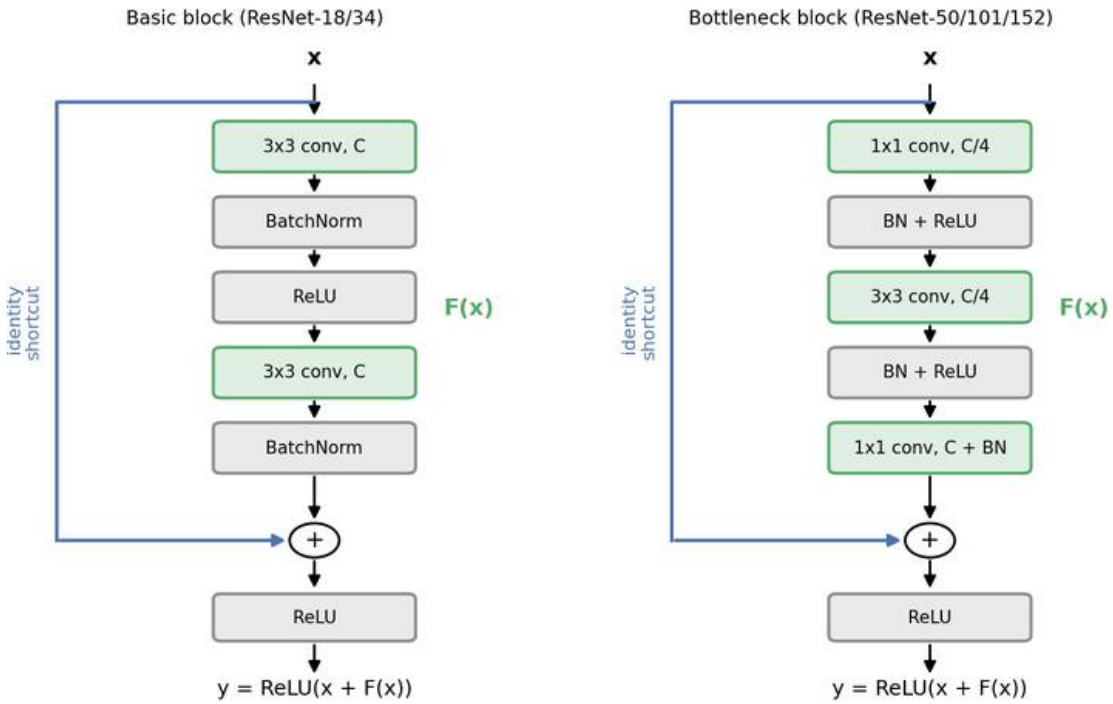


圖 7 — 左：基本殘差區塊。右：*ResNet-50/101/152* 使用的瓶頸區塊，以 *1×1* 卷積在便宜的 *3×3* 卷積前後縮減並恢復通道數。

論文在 ImageNet 上訓練了 **18、34、50、101 與 152 層**的 ResNet，其中較深的三種使用瓶頸區塊。有名的 **1202 層**網路只出現在論文的 CIFAR-10 實驗中 [10]、[11]（32×32 的小影像）；它可以順利訓練，但過擬合，測試結果反而比 110 層的版本差。ResNet 家族贏得 ILSVRC 2015，而殘差連接如今幾乎存在於所有深度網路中，Transformer 也不例外。

用於密集預測的編碼器–解碼器

白話版。分類骨幹為了換取語意而丟掉解析度。分割或復原需要在每個像素上都給出答案，所以要再加上後半段，把解析度找回來。

精確版。 Long 等人提出的**全卷積網路**（fully convolutional network, FCN）[13] 把分類器中的全連接層換成卷積，讓網路對任意大小的輸入輸出一張粗略的類別分數圖，再以學習而來的轉置卷積上採樣，並與前面較精細層的分數融合。**U-Net** [14] 把這個想法做成對稱的形式：收縮的**編碼器**（卷積區塊與降採樣）與擴張的**解碼器**（上採樣與卷積區塊），中間以**跳接**（skip connection）把每個編碼器階段的特徵圖串接到同一解析度的解碼器階段。編碼器提供「是什麼」，跳接提供「在哪裡」。U-Net 原本是為標註很少、需要大量資料擴增的生醫影像設計的；同樣的形狀如今撐起了醫學影像分割、DLCV 04 介紹的幾乎所有復原網路，以及許多擴散模型內部的去雜訊器。

Vision Transformer

白話版。 Transformer [15] 原本是為句子設計的：它把輸入當成一串 token，讓每個 token 都能看到其他所有 token。Vision Transformer（ViT）[16] 把影像切成小方塊（patch），把每個 patch 當成一個「字」，再把這串序列送進一個沒有修改過的 Transformer 編碼器。

切塊、嵌入、注意力

精確版。對影像 $I \in \mathbb{R}^{C\times H\times W}$ 與 patch 大小 P：

1. **切塊。**把 I 切成 $N = HW/P^2$ 個不重疊的 patch，並把每一塊攤平成向量 $\mathbf{p}_i \in \mathbb{R}^{CP^2}$。對 224×224 的 RGB 影像與 $P = 16$，$N = 196$，每個向量有 768 個元素。
2. **嵌入。**把每個 patch 線性映射成 D 維 token，$\mathbf{z}_i = E\,\mathbf{p}_i$，$E \in \mathbb{R}^{D\times CP^2}$。在最前面加上一個可學習的類別 token $\mathbf{z}_{\text{cls}}$，並加上可學習的**位置嵌入**（position embedding）$\mathbf{e}_i$，因為注意力本身不知道 token 來自哪裡：$Z^{(0)} = [\mathbf{z}_{\text{cls}}; \mathbf{z}_1; \dots; \mathbf{z}_N] + [\mathbf{e}_0; \dots; \mathbf{e}_N]$。
3. **編碼。**套用 L 個 Transformer 編碼器區塊，每個區塊包含多頭自注意力（MSA）子層與 MLP 子層，兩者都搭配層正規化（LN）與殘差連接：

$$
\begin{aligned}
Z' &= Z^{(\ell-1)} + \mathrm{MSA}(\mathrm{LN}(Z^{(\ell-1)})),\\
Z^{(\ell)} &= Z' + \mathrm{MLP}(\mathrm{LN}(Z')).
\end{aligned}
$$

4. **Head。**把最後的類別 token（或所有 token 的平均）送進 MLP head，它扮演的就是圖形識別中分類器的角色 [1]。

對 token 矩陣 $Z \in \mathbb{R}^{(N+1)\times D}$，單一注意力頭計算

$$
\mathrm{Attn}(Z) = \mathrm{softmax}\!\left(\frac{QK^{\top}}{\sqrt{d}}\right)V, \qquad Q = ZW_Q,\; K = ZW_K,\; V = ZW_V,
$$

其中 $W_Q, W_K, W_V \in \mathbb{R}^{D\times d}$ 是學習而來的投影，d 是每個頭的維度。softmax 矩陣的第 i 列表示 token i 從其他每個 token 汲取多少資訊。和卷積相比，這裡的「濾波器」是依內容計算出來的，

而且從第一層起就涵蓋整張影像。代價是計算量隨 token 數呈平方成長，$O(N^2 d)$，這也是為什麼使用 patch 而不是像素。

```
img = torch.randn(1, 3, 224, 224)
P, D = 16, 768
patches = img.unfold(2, P, P).unfold(3, P, P)                    # (1, 3, 14, 14, 16, 16)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(1, -1, 3 * P * P)
print(patches.shape)                                              # (1, 196, 768)
embed = nn.Conv2d(3, D, kernel_size=P, stride=P)                  # 切塊＋線性嵌入，一步完成
tokens = embed(img).flatten(2).transpose(1, 2)                    # (1, 196, 768)
```

最後兩行說明了一個實用的等式：patch 嵌入**就是**一個卷積核大小與步幅都等於 P 的卷積。

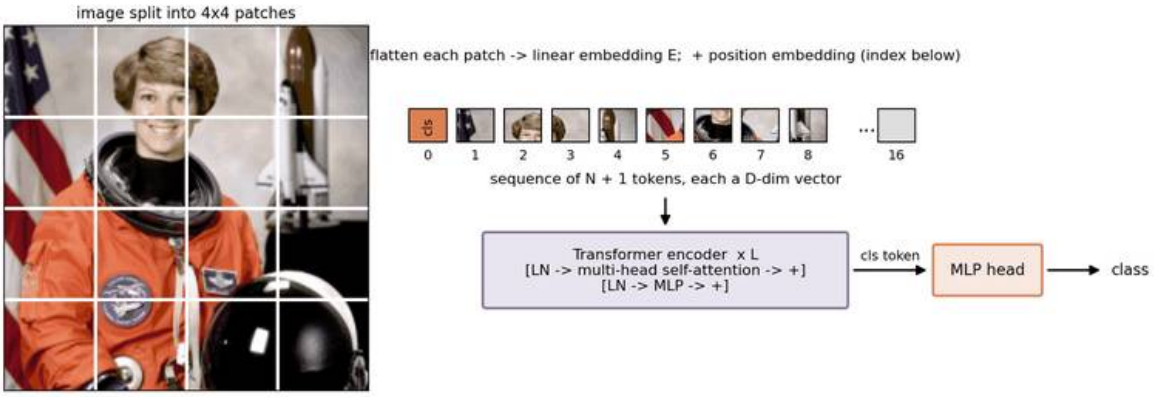


圖 *8* — 一張圖看懂 *ViT*：切塊、線性嵌入、加上位置嵌入與類別 *token*、經過 *Transformer* 編碼器，再由類別 *token* 做分類。（為了清楚起見只畫 *4×4*；*ViT-B/16* 在 *224×224* 輸入上使用 *14×14* 個 *patch*。）

歸納偏置與規模的需求

原文提到 ViT「更難收斂」。更精確的說法 [1]、[16] 與**歸納偏置**（inductive bias）有關，也就是架構在看到資料之前就內建的假設。CNN 假設局部性與權重共享，所以它已經「知道」相鄰像素要一起看，也知道同一個圖樣出現在影像中任何位置都代表同一件事。ViT 幾乎沒有任何假設：除了切 patch 之外，所有空間結構，甚至位置嵌入的二維排列，都必須自己學。

這是一種取捨，而不是缺陷。資料少的時候，CNN 的假設是寶貴的先發優勢，只在 ImageNet-1k 上從頭訓練的 ViT 表現不如同等的 ResNet。資料夠多時，這些假設反而成為限制；在非常大的資料集上預訓練的 ViT，微調後可以追平甚至超越最好的 CNN [16]。實務上的結論是：ViT 都是**預訓練後**才使用，預訓練資料可能是巨大的標註資料集，也越來越常是自監督學習。

ViT 之後的骨幹網路

白話版。 ViT 之後，研究朝三個方向走：讓 Transformer 更適合影像（Swin）、證明現代化之後的 CNN 也能追上（ConvNeXt），以及在沒有標註的情況下預訓練，讓一個骨幹就能服務所有任務（DINO、MAE、DINOv2/v3）。

Swin Transformer：局部性回歸

Swin [17] 只在不重疊的局部視窗（例如 7×7 個 token）內計算注意力，讓計算量對影像大小呈線性。在交替的區塊中，視窗格線會**位移**半個視窗，讓資訊跨越視窗邊界。在階段之間，相鄰的 token 會被合併，使解析度減半。結果是一個階層式、多尺度的特徵金字塔，形狀就像 CNN 骨幹，正好是偵測與分割 head 所需要的。Swin 把 CNN 的歸納偏置，也就是局部性與階層性，放回了 Transformer 裡。

ConvNeXt：CNN 的反擊

Liu 等人 [18] 從 ResNet-50 出發，一步一步採用 Transformer 的訓練方法與設計選擇來將它現代化：「切塊式」的 stem（步幅 4 的 4×4 卷積）、大的 7×7 逐通道卷積（depthwise convolution）、倒置瓶頸的 MLP、較少的激活與正規化層，以及層正規化。得到的純 CNN，**ConvNeXt**，在分類、偵測與分割上的準確度與擴展性都與 Swin 打平。結論是：ViT 的優勢有很大一部分來自訓練方法與宏觀設計，而不是注意力本身。**ConvNeXt V2** [18] 接著加入了為卷積改寫的遮罩自編碼器預訓練（全卷積遮罩自編碼器，FCMAE）以及全域響應正規化（global response normalization）層。如今大型基礎模型多以 ViT 為骨幹，但在低階任務與邊緣裝置上，卷積仍然很常見 [1]。

自監督預訓練：DINO 與 MAE

自監督學習（self-supervised learning）從影像本身產生訓練訊號，因此不需要標註。主流的作法有兩種，而且常常被混為一談。

DINO：自蒸餾（self-distillation）[19]。學生網路 g_s 與教師網路 g_t 架構相同，各自輸出一個 K 維的機率向量。對一張影像做好幾個隨機裁切：兩個大的「全域」視圖與數個小的「局部」視圖。教師只看全域視圖，學生看所有視圖，並被訓練去符合教師的輸出：

$$\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x),\, P_s(x')), \qquad H(a, b) = -a^\top \log b,$$

其中 V 是所有視圖的集合，P_t、P_s 是教師與學生的 softmax 輸出，θ_s 是學生的權重。教師不經由梯度訓練，它的權重是學生權重的指數移動平均，$\theta_t \leftarrow \lambda\theta_t + (1-\lambda)\theta_s$，$\lambda$ 接近 1；教師的輸出還會經過置中（centering）與銳化（sharpening），避免訓練塌縮成常數輸出。讓「小裁切」去符合「整張圖」，會迫使網路從局部辨識出物體。一個令人驚訝的湧現性質是：DINO ViT 的注意力圖在從未看過任何遮罩的情況下，就能分割出主要物體。

MAE：遮罩預測（masked prediction）[20]。遮罩自編碼器（masked autoencoder）隨機遮住大部分的 patch（75 % 效果最好），只用 ViT 編碼看得見的 patch，再讓一個輕量解碼器重建被遮住

的像素：

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{\mathbf{p}}_i - \mathbf{p}_i\|_2^2,$$

其中 $\mathcal{M}$ 是被遮住的 patch 索引集合，$\mathbf{p}_i$ 是 patch i（正規化後）的像素，$\hat{\mathbf{p}}_i$ 是重建結果。這種「遮住再預測」的遊戲，相當於語言模型中遮字預測的影像版本；相對地，DINO 是教師–學生蒸餾法 [1]。MAE 的編碼器只處理四分之一的 token，所以很省計算，微調效果也非常好；DINO 類的特徵則更適合**凍結**使用，直接接一個線性 head。

DINOv2 與 DINOv3：凍結即用的通用特徵

DINOv2 [21] 把 DINO 式的自蒸餾擴展到 10 億參數的 ViT，在一個自動**篩選**（curated）過的大型影像集合上訓練，再蒸餾成較小的模型。它凍結後的特徵，只需要一個輕量 head，就能同時勝任影像層級任務（分類、檢索）與像素層級任務（分割、深度估計）。**DINOv3** [21] 進一步擴大資料與模型規模，並提出 **Gram anchoring**，這是一種正則化方法，能防止密集的 patch 特徵在非常長的訓練過程中退化，讓同一個凍結骨幹提供高品質的密集特徵。

這讓我們回到了圖形識別流程。「特徵擷取器＋分類器」以**預訓練凍結骨幹＋輕量 head** 的形式回來了，只是特徵擷取器現在是在極大規模、沒有標註的情況下學一次就好。

狀態空間骨幹

Vision Mamba（Vim）與 **VMamba** [22] 以選擇性狀態空間模型（selective state-space model）取代注意力，用與序列長度呈線性的成本掃描 token 序列。影像不是序列，所以這些模型會沿不只一條路徑掃描 patch 網格（Vim 是雙向的；VMamba 的二維選擇性掃描會走好幾條路線）。它們對高解析度輸入很有吸引力，但目前仍偏向研究路線，還不是預設的骨幹。

高階視覺任務

白話版。高階任務的差別在於答案要多精確地指出**位置**：整張圖一個標籤、每個物體一個方框，或是每個像素一個標籤。

任務	輸出	典型損失	典型指標	代表模型
分類	一個標籤（或機率向量）	交叉熵	top-1 / top-5 正確率	ResNet、ViT、ConvNeXt [10]、[16]、[18]
物件偵測	一組（方框、類別、分數）	分類＋方框迴歸，或集合匹配	多個 IoU 門檻下的平均精確度（AP）	Faster R-CNN、DETR [23]
語意分割	每個像素一個類別	逐像素交叉熵、Dice（重疊損失，見下文）	mean IoU	FCN、U-Net [13]、[14]
實例分割	每個物體一個遮罩與類別	偵測＋遮罩損失	mask AP	可提示式：SAM 系列 [24]

分類

為整張影像指定一個標籤。網路是骨幹＋全域池化＋線性層，以交叉熵 $\mathcal{L} = -\log \hat{p}_y$ 訓練，其中 $\hat{p}_y$ 是真實類別 y 的預測機率。在自監督學習出現之前，分類也是產生大多數骨幹網路的**預訓練任務**。DLCV 02 會深入介紹。

物件偵測

找出每個物體、它的類別，以及它的位置；位置以軸對齊的方框表示（感興趣區域，ROI，由角點座標或中心與長寬決定）[1]。有兩條設計路線可以說明這個領域 [23]。**Faster R-CNN** 是兩階段式：區域提議網路（region proposal network）從共享的 CNN 特徵中提出候選方框，第二個 head 再對每個候選分類並微調；重複的框由非極大值抑制（non-maximum suppression）移除。**DETR** 把偵測視為**集合預測**：Transformer 解碼器輸出固定數量的預測，再以預測與真實物體之間的二分匹配定義損失，省去了人工設計的錨框與非極大值抑制。方框品質以交併比（intersection over union）衡量：

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

其中 A、B 分別是預測與真實區域。與它密切相關的 Dice 係數 $2|A \cap B| / (|A| + |B|)$，以可微分的軟性形式作為分割損失。平均精確度則在一個或多個 IoU 門檻下概括精確率–召回率曲線。細節請見DLCV 03。

語意分割與實例分割

這裡的感興趣區域是物體確切的像素集合，而不是方框 [1]。

- **語意分割**（semantic segmentation）為每個像素標上類別。兩個重疊的人會變成同一塊「人」的區域。
- **實例分割**（instance segmentation）還要把同一類別的不同物體分開。兩個人會變成「人 1」與「人 2」。

語意分割是上面編碼器–解碼器 [13]、[14] 的本行。實例分割則再加上一個類似偵測的元件，為每個物體提出一個遮罩。

可提示的分割：SAM、SAM 2、SAM 3

分割領域如今有了**可提示的基礎模型** [24]。**SAM**（Segment Anything）接收一張影像與一個提示（點、方框或粗略遮罩），回傳被指定物體的遮罩。笨重的 ViT 影像編碼器每張影像只跑一次，輕量的提示編碼器與遮罩解碼器則每個提示跑一次，所以互動很快。它在 SA-1B 上訓練；SA-1B 包含 1,100 萬張影像上超過 10 億個遮罩，是以「模型在迴圈中」的資料引擎建立的。**SAM 2** 把這個能力擴展到影片，以串流記憶（streaming memory）把物體一幀一幀帶下去，並在新的影片資料集（SA-V）上訓練。**SAM 3** 加入了**可提示的概念分割**（promptable concept segmentation）：給它一個短名詞片語，例如「黃色校車」，或是影像範例，或兩者並用，它會找出、分割並（在影

片中）追蹤**所有**符合的實例，而不只是點擊指向的那一個物體。到了 SAM 3，偵測、實例分割與追蹤之間的界線已經變得很薄；追蹤請見DLCV 06。

低階視覺任務

白話版。低階任務輸入一張影像，回傳同一個場景經過改善或轉換的影像。輸出以逐像素的方式評判，或以看起來有多自然來評判。

一般的形式是一個反問題。劣化的觀測 $\mathbf{y}$ 被建模為

$$\mathbf{y} = \mathcal{D}(\mathbf{x}) + \mathbf{n},$$

其中 $\mathbf{x}$ 是乾淨影像，$\mathcal{D}$ 是劣化運算子（模糊、降採樣、霧、曝光不足），$\mathbf{n}$ 是雜訊，與 IP 05 的模型相同。網路 f_θ 在成對資料上訓練，最小化例如 $\|f_\theta(\mathbf{y}) - \mathbf{x}\|_1$。由於真實的成對資料很少，通常會對乾淨影像套用一個建模好的劣化來**合成**成對資料，而合成劣化與真實劣化之間的落差，正是這個領域的核心難題。

- **影像復原**（image restoration）找回乾淨影像，又可稱為「清晰化」：去雜訊、去模糊、超解析度（小圖變大圖）、除霧（霧濛濛的影像變清楚）、去雨。架構是全解析度的編碼器–解碼器（類 U-Net），或不降採樣的殘差網路，現在也常加入視窗注意力。請見DLCV 04。
- **影像增強**（image enhancement）讓內容更容易看清楚，但沒有唯一的「正確答案」：低光源增強（讓暗影像中的細節變清楚，某種程度上是聰明的調亮；如果是把黑夜變白天，就屬於轉譯了）與水下影像增強。請見DLCV 05。
- **影像對齊**（image alignment，或稱影像配準）估計把一張影像對應到另一張影像的幾何轉換或稠密位移場，是連拍融合、HDR 與醫學影像比對的前置步驟。
- **影像到影像轉換**（image-to-image translation）把影像從一個領域或風格轉到另一個：白天變黑夜、夏天變冬天、草圖變照片。換臉的「深偽」（deepfake）就是同一個想法的（被濫用的）延伸應用。自從擴散模型出現後，轉換與編輯常改由一個以輸入影像為條件的生成模型完成，見下一節。

生成與視覺–語言模型

白話版。有兩類較新的模型不太能放進舊地圖：一類從文字**創造**影像，另一類能**談論**影像。兩者都依賴大規模預訓練，以及把影像和語言連結起來。

擴散模型：從雜訊到影像

去雜訊擴散機率模型（denoising diffusion probabilistic model, DDPM）[27] 定義了一個前向過程，在 T 步內逐漸對影像 $\mathbf{x}_0$ 加入高斯雜訊。寫成封閉形式為

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I),$$

其中 $t \in \{1, \dots, T\}$ 是步數，$\bar{\alpha}_t \in (0,1)$ 是固定的排程，從接近 1 遞減到接近 0。網路 $\boldsymbol{\epsilon}_\theta$ 被訓練來預測加入的雜訊：

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0,t,\boldsymbol{\epsilon}} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2,$$

生成時則反向執行這個過程：從純雜訊開始，一步一步去雜訊。換句話說，生成器就是**反覆套用許多次的學習式去雜訊器**，這讓生成直接和上面的復原問題連在一起。

- **潛在擴散**（latent diffusion）[28] 不在像素上、而是在預訓練自編碼器的壓縮潛在空間中進行擴散，大幅降低成本，使高解析度的文字生成影像變得可行；文字條件透過交叉注意力（cross-attention）加入。它是 Stable Diffusion 的基礎。
- **DiT**（Diffusion Transformer）[28] 把 U-Net 去雜訊器換成作用在潛在 patch 上的 Transformer，並顯示品質會隨模型計算量穩定提升。
- **InstructPix2Pix** [29] 依照文字指令（「把它變成冬天」）編輯給定的影像，讓傳統的影像到影像轉換成為條件生成的一個特例。

視覺–語言模型

視覺問答（visual question answering, VQA），也就是針對一張影像回答自由形式的問題，以前需要串起影像特徵、影像描述（captioning）與文字問答的流程 [1]。兩個進展讓它變成單一模型。

CLIP [25] 以**對比式**（contrastive）目標，在 4 億組影像–文字對上訓練影像編碼器 f 與文字編碼器 g。在一個包含 B 組配對的批次中，令正規化後的嵌入為 $\mathbf{u}_i = f(I_i)$、$\mathbf{v}_j = g(T_j)$，溫度為 τ，影像到文字的損失為

$$\mathcal{L}_{I\to T} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(\mathbf{u}_i^\top \mathbf{v}_i/\tau)}{\sum_{j=1}^{B} \exp(\mathbf{u}_i^\top \mathbf{v}_j/\tau)},$$

總損失則與對稱的文字到影像項平均。配對的組合被拉近，批次中其他所有組合被推開。由於標籤空間變成了「任何一句話」，CLIP 可以做零樣本（zero-shot）分類：把「a photo of a cat」、「a photo of a dog」……等提示嵌入，再挑出與影像最接近的那一個。

LLaVA [26] 透過一個投影層，把 CLIP 視覺編碼器接上大型語言模型（LLM）；投影層把視覺 token 映射到 LLM 的詞嵌入空間，再以關於影像的指令跟隨對話進行微調。目前大多數多模態 LLM 都沿用這種**視覺編碼器＋投影層＋ LLM** 的架構，所以 VQA、影像描述與許多辨識任務，現在都可以透過對同一個模型下提示來完成。

現代觀點

貫穿本篇的趨勢，是從「一個任務一個模型」走向「一個基礎模型＋提示或輕量 head」。以下的綜述論文，是理解這個領域如何走到今天的好地圖。

CNN 架構。 Li 等人 [30] 回顧了 CNN 的歷史、卷積運算本身、經典與進階架構（LeNet 到 ResNet 一脈，以及輕量化與加入注意力的網路）、一維／二維／多維卷積與各種應用。他們也整理了每個元件的改進（層設計、激活與損失函數、正則化、最佳化、快速計算），透過實驗歸納出挑選函數與超參數的經驗法則，最後討論開放問題與有潛力的方向。若想一次看到 CNN 的設計空間被逐個元件攤開，這是最好的單一參考。

Vision Transformer。 Khan 等人 [31]（ACM Computing Surveys）依任務整理 Transformer 的研究：先是基礎（自注意力、大規模預訓練、雙向編碼），接著是辨識（分類、偵測、分割、動作辨識）、生成模型、多模態任務（VQA、視覺推理、視覺定位）、影片、低階視覺（超解析度、增強、上色）與三維點雲。他們的核心對比與本篇一致：Transformer 以極少的歸納偏置建模長距離依賴，因此需要大規模預訓練或架構上的先驗（視窗、階層）才能有好的資料效率。

自監督視覺學習。 Jing 與 Tian [32]（TPAMI）綜述了 2020 年以前那一代的自監督方法，依前置任務（pretext task）分為生成式、上下文式、免費語意標籤式與跨模態式四類，並說明以遷移到下游任務來評估特徵的標準流程。Balestriero 等人 [33] 則以實用的「食譜」形式涵蓋新一代方法，把它們分成四個家族：深度度量學習（對比式）、自蒸餾（DINO 家族）、典型相關分析類方法，以及遮罩影像建模（MAE）。他們強調許多隱藏的「旋鈕」（資料擴增、投影 head、防止塌縮的機制），這也解釋了為什麼自監督結果難以重現。

視覺基礎模型。 Awais 等人 [34]（TPAMI）依模態組合（視覺搭配文字、音訊、深度）、訓練目標（對比式 vs. 生成式）與**提示類型**（文字提示；視覺提示，例如 SAM 的點、框、遮罩；以及異質混合提示）為視覺基礎模型分類。他們列出的開放挑戰包括評估與基準測試、真實世界理解與上下文推理、偏差、對抗強健性與可解釋性。Zhang 等人 [35]（TPAMI）聚焦於用於視覺辨識的視覺–語言模型：架構與預訓練目標、遷移方法（提示微調、特徵轉接器），以及把知識蒸餾到偵測器與分割器。他們主張網路規模的影像–文字資料降低了對群眾標註的依賴，並讓一個模型服務許多任務。

2024–2026 年的最新進展。不引用任何基準數字，大致的局面如下：

- **骨幹網路**。以自監督或影像–文字資料預訓練的大型 ViT 是高階任務的預設選擇，凍結的 DINOv2/DINOv3 特徵 [21] 是密集任務的強基準。現代 CNN（ConvNeXt V2 [18]）仍具競爭力，在延遲、記憶體或高解析度是主要考量時更受青睞。狀態空間骨幹 [22] 是活躍的替代路線。
- **分割與追蹤**已收斂到可提示模型：影片物體用 SAM 2，開放詞彙概念用 SAM 3 [24]。
- **生成**已轉向以 Transformer 為去雜訊器的潛在擴散 [28]，編輯則轉向以指令為條件的擴散 [29]。
- **理解**越來越多交給 LLaVA 架構的多模態 LLM [26]。
- **低階視覺**仍大量依賴在合成劣化上訓練的任務專用編碼器–解碼器，並越來越常結合生成式（擴散）先驗；見 DLCV 04 與 DLCV 05。

開放問題。 (1) 資料與評估：基準測試趨於飽和，也可能與網路規模的預訓練資料重疊，難以區分泛化與記憶 [34]。(2) 真實世界的劣化：在合成雜訊或模糊上訓練的模型，遇到真實相機流程仍會失敗。(3) 保真度與幻覺：生成式先驗產生銳利、但有時是捏造的細節，這在醫學或鑑識用途中無法接受。(4) 效率：注意力對 token 數呈平方成長，基礎模型也很難在裝置端執行。(5) 大規模下的密集特徵：DINOv3 之所以需要 Gram anchoring [21]，正是因為密集特徵品質可能隨訓練規模擴大而退化。(6) 以未篩選網路資料訓練的模型，其偏差、強健性與可解釋性 [34]。

重點整理

- 深度學習把傳統的三個層次縮減為兩類：**低階**（影像 → 影像，以保真度評判）與**高階**（影像 → 語意，以正確率評判）。中階分析則依輸出的形式歸入其中之一。
- **端對端學習**以學習而來的特徵取代人工設計的特徵；而分工以**預訓練骨幹＋輕量 head** 的形式在更大規模下回歸。
- 卷積層是一組具局部連接與權重共享的**學習式空間濾波器**。步幅、填補與池化決定輸出大小；**感受野**依 $r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}$ 成長，而有效感受野比理論值小。
- LeNet-5（1998）不是第一個 CNN；以反向傳播訓練的卷積網路可追溯到 1989 年。ResNet 在 ImageNet 上訓練的是 **18/34/50/101/152** 層；**1202** 層只出現在 CIFAR-10。殘差連接讓恆等映射變得容易，也給梯度一條直達的路徑。
- 帶跳接的編碼器–解碼器（**FCN**、**U-Net**）是密集預測與影像復原的標準形狀。
- **ViT** 的歸納偏置很少，因此需要大規模預訓練。Swin 把局部性加回去；ConvNeXt 則證明現代化的 CNN 可以與之打平。
- **DINO** 是教師–學生自蒸餾；**MAE** 是遮罩預測。DINOv2/v3 提供凍結即用的通用特徵。
- 各種任務越來越收斂到可提示或與語言連結的**基礎模型**：分割用 SAM 1/2/3，生成用潛在擴散與 DiT，視覺–語言用 CLIP 與 LLaVA。

練習

1. **手算感受野**。一個網路依序為：7×7 卷積、步幅 2；3×3 最大池化、步幅 2；接著四個 3×3、步幅 1 的卷積。計算每一層之後的理論感受野與跳距，並用 `receptive_field` 函式驗證。

   提示

   從 $r = 1$、$j = 1$ 開始。第一個卷積之後：$r = 7$、$j = 2$。池化之後：$r = 7 + 2 \cdot 2 = 11$、$j = 4$。之後每一層 3×3 增加 $2 \cdot 4 = 8$，所以 $r = 19, 27, 35, 43$，$j = 4$。

2. **參數預算**。輸入與輸出通道都是 $C = 256$ 時，比較（忽略偏差）一層 5×5 卷積、兩層疊起來的 3×3 卷積，以及一個瓶頸結構（1×1 降到 64 通道、64 通道的 3×3、1×1 回到 256）的權重數。哪些的感受野相同？

   提示

   $25C^2 \approx 164$ 萬；$2 \cdot 9C^2 \approx 118$ 萬；瓶頸 $256 \cdot 64 + 9 \cdot 64^2 + 64 \cdot 256 \approx 7$ 萬。前兩者都看得到 5×5；瓶頸只看得到 3×3，但便宜得多，這就是 ResNet-50 以上使用它的原因。

3. **沒有捷徑時，恆等映射為什麼很難**。在網路內部，區塊的輸入通常來自前一個 ReLU，因此是非負的。證明對這樣的輸入，$F \equiv 0$ 的殘差區塊輸出恰好是 $\mathbf{y} = \mathbf{x}$。接著說明普通區塊（conv–BN–ReLU–conv–BN–ReLU）要輸出 $\mathbf{y} = \mathbf{x}$ 必須學到什麼。這對非常深的普通網路代表什麼？

提示

對 $\mathbf{x} \geq 0$，$\mathrm{ReLU}(\mathbf{x} + 0) = \mathbf{x}$。普通區塊則必須學到恰好抵消兩次卷積與兩次正規化的權重，這是一個精確又脆弱的解，梯度下降很少找得到。殘差區塊只需要 $F = 0$，而權重衰減與零初始化本來就把它往那裡推。因此很深的普通網路很難讓多出來的層「什麼都不做」，這與 ResNet 作者觀察到的退化現象一致。

4. **ViT 的 token 數與計算量**。對 512×512 的輸入，計算 patch 大小為 32、16、8 時的 token 數，以及一層全域注意力的相對成本（$\propto N^2$）。Swin 使用 7×7 視窗的視窗注意力，會如何改變這個成長趨勢？

提示

$N = (512/P)^2$：256、1024、4096 個 token；成本比為 $1 : 16 : 256$。視窗注意力每層成本為 $N \cdot 49$（每個 token 只關注 49 個 token），所以隨 N 線性成長。

5. **學來的與手動設定的濾波器**。修改數字 CNN，讓第一層凍結為四個手動設定的卷積核（兩個 Sobel、一個 Laplacian、一個平均模糊）加上四個隨機卷積核，只訓練其餘部分。與完全學習的網路比較測試正確率；再解凍隨機卷積核，觀察它們會變成什麼樣子。

提示

設定好權重之後執行 `conv.weight.requires_grad_(False)`，並只把其餘參數交給最佳化器。在這個簡單的資料集上兩者正確率相近；重點在於手動設定的微分濾波器本來就是合理的第一層，而解凍後的隨機卷積核往往會朝有方向性、類微分的圖樣移動（可對照圖 4）。

6. **選擇配方**。你有 2,000 張標註過的顯微鏡影像，需要 (a) 每張影像的品質標籤、(b) 逐像素的細胞遮罩、(c) 去雜訊後的影像。根據本篇的地圖，為每一項提出骨幹、head 與訓練策略，並說明哪一項最能受益於凍結的自監督骨幹。

提示

(a) 凍結的 DINOv2/v3，或微調的 ConvNeXt ＋線性 head。(b) U-Net（可搭配預訓練編碼器），或以點提示 SAM 再微調。(c) 全解析度的復原網路，在符合該顯微鏡雜訊特性的合成雜訊/乾淨影像對上訓練。凍結的語意特徵對 (a) 與 (b) 幫助最大；(c) 更依賴逐像素準確的統計特性，而不是語意。

參考文獻

1. Shyandram, “深度學習的數位影像處理介紹,” Medium post, 2024; updated 2026. link
2. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018. publisher page
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008. publisher page

4. Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989. doi:10.1162/neco.1989.1.4.541
5. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi:10.1109/5.726791
6. O. Russakovsky, J. Deng, H. Su, et al., "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015. doi:10.1007/s11263-015-0816-y ·arXiv:1409.0575
7. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *NeurIPS*, 2012. proceedings
8. K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *ICLR*, 2015. arXiv:1409.1556
9. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," *CVPR*, 2015. doi:10.1109/CVPR.2015.7298594 ·arXiv:1409.4842
10. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CVPR*, 2016. doi:10.1109/CVPR.2016.90 ·arXiv:1512.03385
11. A. Krizhevsky, "Learning multiple layers of features from tiny images," technical report, University of Toronto, 2009 (the CIFAR-10/100 datasets). dataset page
12. W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," *NeurIPS*, 2016. arXiv:1701.04128
13. J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," *CVPR*, 2015. arXiv:1411.4038
14. O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," *MICCAI*, 2015. arXiv:1505.04597
15. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *NeurIPS*, 2017. ML anthology
16. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," *ICLR*, 2021. arXiv:2010.11929
17. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical vision Transformer using shifted windows," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00986 ·arXiv:2103.14030
18. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *CVPR*, 2022, arXiv:2201.03545; and S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, "ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders," *CVPR*, 2023, arXiv:2301.00808.
19. M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision Transformers," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00951 ·arXiv:2104.14294
20. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," *CVPR*, 2022. doi:10.1109/CVPR52688.2022.01553 ·arXiv:2111.06377
21. M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024, arXiv:2304.07193; and O.

Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, et al., “DINOv3,” arXiv preprint, 2025, arXiv:2508.10104.

22. L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, “Vision Mamba: Efficient visual representation learning with bidirectional state space model,” *ICML*, 2024, arXiv:2401.09417; and Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, J. Jiao, and Y. Liu, “VMamba: Visual state space model,” *NeurIPS*, 2024, arXiv:2401.10166.
23. S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *NeurIPS*, 2015, arXiv:1506.01497; and N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with Transformers,” *ECCV*, 2020, arXiv:2005.12872.
24. A. Kirillov, E. Mintun, N. Ravi, et al., “Segment anything,” *ICCV*, 2023, arXiv:2304.02643; N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, et al., “SAM 2: Segment anything in images and videos,” *ICLR*, 2025, arXiv:2408.00714; and N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, et al., “SAM 3: Segment anything with concepts,” *ICLR*, 2026, arXiv:2511.16719.
25. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, et al., “Learning transferable visual models from natural language supervision,” *ICML*, 2021. arXiv:2103.00020
26. H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” *NeurIPS*, 2023. arXiv:2304.08485
27. J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *NeurIPS*, 2020. arXiv:2006.11239
28. R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” *CVPR*, 2022, arXiv:2112.10752; and W. Peebles and S. Xie, “Scalable diffusion models with Transformers,” *ICCV*, 2023, arXiv:2212.09748.
29. T. Brooks, A. Holynski, and A. A. Efros, “InstructPix2Pix: Learning to follow image editing instructions,” *CVPR*, 2023. arXiv:2211.09800
30. Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, “A survey of convolutional neural networks: Analysis, applications, and prospects,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, 2022. doi:10.1109/TNNLS.2021.3084827 ·arXiv:2004.02806
31. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, “Transformers in vision: A survey,” *ACM Computing Surveys*, vol. 54, no. 10s, 2022. doi:10.1145/3505244 ·arXiv:2101.01169
32. L. Jing and Y. Tian, “Self-supervised visual feature learning with deep neural networks: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, 2021. doi:10.1109/TPAMI.2020.2992393 ·arXiv:1902.06162
33. R. Balestriero, M. Ibrahim, V. Sobal, et al., “A cookbook of self-supervised learning,” arXiv preprint, 2023. arXiv:2304.12210
34. M. Awais, M. Naseer, S. Khan, R. M. Anwer, H. Cholakkal, M. Shah, M.-H. Yang, and F. S. Khan, “Foundation models defining a new era in vision: A survey and outlook,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, 2025. doi:10.1109/TPAMI.2024.3506283 ·arXiv:2307.13721
35. J. Zhang, J. Huang, S. Jin, and S. Lu, “Vision-language models for vision tasks: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, 2024. doi:10.1109/TPAMI.2024.3369699 ·arXiv:2304.00685
36. scikit-learn developers, “sklearn.datasets.load_digits” (a copy of the test set of the UCI optical recognition of handwritten digits dataset). docs

DLCV 02・深度影像分類

深度網路如何判斷一張影像裡是什麼：損失函數與評估指標、資料集、CNN 與 Transformer 架構背後的設計理由、現代訓練配方、預訓練、穩健性、校準與可解釋性。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-02-image-classification/

你將學到

- 影像分類的精確定義：單標籤、多標籤與細粒度（fine-grained）任務；logits、softmax 與交叉熵（cross-entropy）；top-1 ／ top-5 準確率；混淆矩陣與由它衍生的指標（精確率、召回率、F1、MCC、巨觀與微觀平均、ROC-AUC、平均精確度、mAP）；以及分類器「校準良好」是什麼意思。
- 標準訓練用資料集（MNIST、CIFAR、ImageNet、iNaturalist）各自的用途；哪些評估基準在測什麼（ImageNet-ReaL、ImageNetV2、穩健性測試集、細粒度與遷移測試組、長尾與零樣本評估）；以及如何執行誠實的評估流程。
- 主要骨幹網路（backbone）背後的設計理由：從 AlexNet、VGG、Inception、ResNet，到 MobileNet、EfficientNet、RegNet、ConvNeXt、ViT、DeiT 與 Swin，並附上可以自己驗算的參數量與計算量。
- 現代訓練配方中哪些部分真正重要（資料擴增、Mixup ／ CutMix、標籤平滑、AdamW、暖身＋餘弦排程、隨機深度、EMA），以及為什麼光靠配方就能讓 ResNet-50 提升好幾個百分點。
- 預訓練如何改變了這份工作：線性探測與微調、對比式與遮罩式自監督學習、DINOv2 式的凍結特徵，以及 CLIP 的零樣本（zero-shot）分類。
- 如何誠實地評估：分佈偏移、對抗樣本、捷徑學習、溫度縮放、效率指標與 Grad-CAM。

先看全貌

影像分類對整張影像只問一個問題：這些標籤裡，哪一個最符合？它是最早的深度學習基準任務，到今天仍是大多數骨幹網路誕生的地方。物體偵測、分割或影像復原網路，通常都從一個先以影像分類訓練、或至少以影像分類評估過的骨幹網路出發。

> **用最簡單的話說**
>
> 想像一個小朋友看著圖畫書學認動物。一開始他只看到一團一團的顏色。翻過很多頁之後，他學會先注意耳朵，再注意臉，最後說出「這是貓」。深度分類器也是一層一層這樣做：前面的層注意邊緣和顏色，中間的層注意眼睛、輪子這類零件，最後一層投票選出標籤。訓練就是爸媽在旁邊說了幾千次「不對，那是狗」。這篇教學其餘的部分在講：怎麼做一本更好的圖畫書（資料）、一顆更好的腦袋（架構）、一套更好的教法（訓練配方），以及更好的考試（評估）。因為一個只看過「貓坐在沙發上」的小朋友，可能會以為「沙發」就代表「貓」。

本篇假設你已經知道卷積、CNN 的層與反向傳播。分類的古典面向（貝氏分類器、感知器、用 NumPy 寫的小型 CNN、手推 softmax 與交叉熵）在IP 13。深度學習用於影像的整體歷史

在DLCV 01。這裡我們專注、深入地談分類本身。若想快速複習激活函數與最佳化器，可以參考ML 02・類神經網路。CNN 這一側的發展，Li 等人 [1] 的綜述是很好的單篇入門。

分類問題的定義

白話版。網路看一張影像，對每個類別輸出一個分數。我們把分數轉成機率，與真正的標籤比較，再輕輕推動網路，讓正確類別下一次得到更高的機率。

單標籤、多標籤與細粒度

令 $\mathbf{x} \in \mathbb{R}^{H\times W\times 3}$ 為一張 RGB 影像，$\mathcal{Y} = \{1, \dots, K\}$ 為 K 個類別的集合。參數為 θ 的網路 f_θ 把影像映射到一個 **logits** 向量 $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$，每個類別一個未正規化的分數。

- **單標籤（多類別）**分類假設恰好有一個正確類別 $y \in \mathcal{Y}$。以 softmax 把 logits 轉成機率向量：

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K,$$

其中 p_k 是模型對 $P(y = k \mid \mathbf{x})$ 的估計。訓練時最小化**交叉熵**，也就是正確類別機率的負對數，在一個小批次上取平均：

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B}\sum_{i=1}^{B} \log p_{y_i}(\mathbf{x}_i),$$

其中 B 是批次大小，y_i 是影像 $\mathbf{x}_i$ 的標籤。它對 logits 的梯度（每張影像）是 $\partial \mathcal{L}/\partial \mathbf{z} = \mathbf{p} - \mathbf{e}_y$，$\mathbf{e}_y$ 是正確類別的 one-hot 向量；推導見 DIP 第 13 章。

- **多標籤**分類允許任意個標籤同時成立（一張街景可能同時有「汽車」、「行人」、「腳踏車」）。每個類別各自接一個 sigmoid $\sigma(z_k) = 1/(1+e^{-z_k})$ 與二元交叉熵 $-\sum_k [t_k \log \sigma(z_k) + (1-t_k)\log(1-\sigma(z_k))]$，其中 $t_k \in \{0, 1\}$ 表示類別 k 是否出現。評估指標也要換：以各類別的平均精確度（mean average precision）取代準確率。

- **細粒度**分類要分開外觀非常相近的子類別，例如 200 種鳥或 1,000 種飛機型號。損失函數與單標籤相同，但訊號藏在小地方（鳥喙形狀、翅膀上的條紋），類別通常呈長尾分佈，而且非專家標註者造成的標籤雜訊很常見。iNaturalist [6] 是標準的大型細粒度基準。

評估指標：top-1、top-5 與其他

Top-1 準確率是最高分類別恰好正確的影像比例。**Top-k 準確率**則只要正確類別落在前 k 高分之中就算對。ImageNet 挑戰賽回報 top-5 錯誤率，因為很多影像裡有好幾個物體，而有些類別幾乎無法區分 [2]。但 top-5 會掩蓋真正的錯誤，所以現代論文大多回報 top-1。資料不平衡時，請回報**平衡準確率**（各類別召回率的平均）或逐類別結果，因為分類器只要忽略稀有類別，也能拿到很高的準確率。

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 1.0, 0.1, -1.0],    # 2 images, 4 classes
                       [0.5, 2.5, 2.4, 0.0]])
y = torch.tensor([0, 2])

p = logits.softmax(dim=1)
ce = F.cross_entropy(logits, y)                       # mean of -log p[i, y_i]
ce_ls = F.cross_entropy(logits, y, label_smoothing=0.1)
top1 = (logits.argmax(1) == y).float().mean()
top2 = (logits.topk(2, dim=1).indices == y[:, None]).any(1).float().mean()
print(p.round(decimals=3))
print(f"CE {ce:.3f}  CE+LS {ce_ls:.3f}  top-1 {top1:.2f}  top-2 {top2:.2f}")

# multi-label: one independent sigmoid per class, binary cross-entropy
targets = torch.tensor([[1., 0., 1., 0.], [0., 1., 1., 0.]])
print(f"BCE {F.binary_cross_entropy_with_logits(logits, targets):.3f}")
```

第二張影像在類別 1 和 2 之間幾乎平手。Top-1 算它錯，top-2 算它對。支持與反對 top-5 的理由，都在這一個例子裡。

混淆矩陣與由它衍生的指標

白話版。準確率把所有錯誤壓成一個數字。混淆矩陣（confusion matrix）則把錯誤分開來記：對每一個真實類別，數一數模型分別回答了什麼。幾乎所有其他分類指標，都是這張表裡某些格子的比值，而每一種比值回答的是不同的問題。

二元情況：TP、FP、FN、TN

把某一類當作**陽性**（positive，例如「生病」），其餘當作**陰性**（negative）。每一個預測都會落在四個格子之一：

	預測為陽性	預測為陰性
實際為陽性	真陽性（true positive, TP）	偽陰性（false negative, FN）
實際為陰性	偽陽性（false positive, FP）	真陰性（true negative, TN）

用最簡單的話說

煙霧警報器就是一個分類器。真的失火、它響了，是真陽性。你只是把吐司烤焦、它也響了，是偽陽性：很吵，但沒有人受傷。真的失火、它卻沒響，是偽陰性：這才是危險的那一種。什麼事都沒有、它也安靜，是真陰性。好的警報器不只是「大部分時間都對」（它幾乎每天都安靜，要做到這點太容易了），而是要在失火時響、在烤吐司時安靜。不同的指標，對這兩個目標的看重程度不一樣。

設測試樣本共 $N = \text{TP} + \text{FP} + \text{FN} + \text{TN}$ 個，標準的比值為

$$\text{accuracy} = \frac{\text{TP}+\text{TN}}{N}, \quad \text{precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}, \quad \text{recall (TPR)} = \frac{\text{TP}}{\text{TP}+\text{FN}},$$
$$\text{specificity (TNR)} = \frac{\text{TN}}{\text{TN}+\text{FP}}, \qquad \text{FPR} = \frac{\text{FP}}{\text{FP}+\text{TN}} = 1 - \text{TNR}.$$

- **精確率（precision）**回答「模型說是陽性的時候，有多常是對的？」當誤報代價很高時它最重要：會把正常信件藏起來的垃圾郵件過濾器、自動觸發的產品召回。
- **召回率（recall）**，又稱**敏感度**（sensitivity）或**真陽性率**（true positive rate, TPR），回答「所有真正的陽性中，我們抓到了多少？」當漏抓代價很高時它最重要：癌症篩檢、產線上的瑕疵偵測。
- **特異度**（specificity），即**真陰性率**（true negative rate, TNR），是陰性類別的召回率。**偽陽性率**（false positive rate, FPR）是它的補數，也就是被錯誤標記的陰性樣本比例。

精確率與召回率互相拉扯：降低決策門檻會標記更多樣本，召回率上升，精確率通常下降。把兩者合成一個數字的指標是：

$$F_1 = \frac{2\,\text{P}\,\text{R}}{\text{P}+\text{R}} = \frac{2\,\text{TP}}{2\,\text{TP}+\text{FP}+\text{FN}}, \qquad F_\beta = \frac{(1+\beta^2)\,\text{P}\,\text{R}}{\beta^2\,\text{P}+\text{R}},$$

其中 P 是精確率，R 是召回率，$\beta > 0$ 表示召回率比精確率重要幾倍（F_2 偏重召回率，$F_{0.5}$ 偏重精確率）。F_1 是調和平均，所以只有兩者都高時它才會高。也請注意 F_1 忽略了什麼：真陰性完全沒有出現在式子裡。

另外兩個總結指標用到了全部四格：

$$\text{balanced accuracy} = \frac{\text{TPR}+\text{TNR}}{2}, \qquad \text{MCC} = \frac{\text{TP}\cdot\text{TN} - \text{FP}\cdot\text{FN}}{\sqrt{(\text{TP}+\text{FP})(\text{TP}+\text{FN})(\text{TN}+\text{FP})(\text{TN}+\text{FN})}}.$$

平衡準確率（balanced accuracy）是各類別召回率的平均，所以一個忽略某一類的分類器，不論那一類多稀有，都只能拿到 0.5。**Matthews 相關係數**（Matthews correlation coefficient, MCC）是真實與預測二元標籤之間的 Pearson 相關係數：+1 表示完美，0 表示不比隨機好，−1 表示完全相反。Chicco 與 Jurman [57] 主張在二元評估中應優先使用 MCC，而不是準確率與 F_1，因為只有分類器在四個格子上都表現好時，它才會高。Sokolova 與 Lapalme [58] 系統性地比較了這些指標，以及它們各自如何隨混淆矩陣的變化而反應。

為什麼 99:1 時準確率會誤導。假設 1,000 張掃描影像中有 10 張是生病的。一個永遠回答健康的「模型」可以拿到 99.0% 的準確率。模型 B 標記了 30 張，抓到 10 張中的 8 張；它的準確率較低（97.6%），卻是唯一有用的那一個：

```
import numpy as np
from sklearn.metrics import (accuracy_score, balanced_accuracy_score, confusion_matrix,
                             f1_score, matthews_corrcoef, precision_score, recall_score)

# 1,000 scans, 10 sick (label 1). Model A says "healthy" to everyone.
# Model B flags 30 scans and catches 8 of the 10 sick ones.
y = np.r_[np.ones(10), np.zeros(990)].astype(int)
pred_a = np.zeros(1000, dtype=int)
```

```
pred_b = np.r_[np.ones(8), np.zeros(2), np.ones(22), np.zeros(968)].astype(int)

for name, p in [("all-negative", pred_a), ("model B", pred_b)]:
    tn, fp, fn, tp = confusion_matrix(y, p).ravel()
    print(f"{name:12s} TP={tp} FP={fp} FN={fn} TN={tn}  "
          f"acc={accuracy_score(y, p):.3f} bal-acc={balanced_accuracy_score(y, p):.3f} "
          f"P={precision_score(y, p, zero_division=0):.3f} R={recall_score(y, p):.3f} "
          f"F1={f1_score(y, p, zero_division=0):.3f} MCC={matthews_corrcoef(y, p):.3f}")
# all-negative TP=0 FP=0 FN=10 TN=990  acc=0.990 bal-acc=0.500 P=0.000 R=0.000 F1=0.000
 ↪  MCC=0.000
# model B      TP=8 FP=22 FN=2 TN=968  acc=0.976 bal-acc=0.889 P=0.267 R=0.800 F1=0.400
 ↪  MCC=0.454
```

平衡準確率、F_1 與 MCC 都把模型 B 排在第一；準確率卻把它排在最後。經驗法則：只有在類別大致平衡、所有錯誤代價相同時才用準確率；當某一種錯誤比較嚴重時，回報精確率與召回率（或依實際代價選定 β 的 F_β）；需要一個能反映不平衡的單一總結數字時，用平衡準確率或 MCC。

多類別混淆矩陣

有 K 個類別時，混淆矩陣 $C \in \mathbb{N}^{K \times K}$ 的元素 C_{ij} 是**真實**類別為 i（列）、**預測**類別為 j（行）的測試影像數；這是 scikit-learn [62] 的慣例。對角線是預測正確的數量。把每一列除以該列總和，得到**依列正規化**的矩陣：它的對角線就是各類別的召回率，非對角元素則顯示哪些類別被搞混。圖 1 是一個只用 scikit-learn 8×8 手寫數字中 200 張訓練的邏輯迴歸所得到的結果：大部分錯誤是 3 → 8、4 → 1 與 7 → 9，都是筆畫相似的組合。

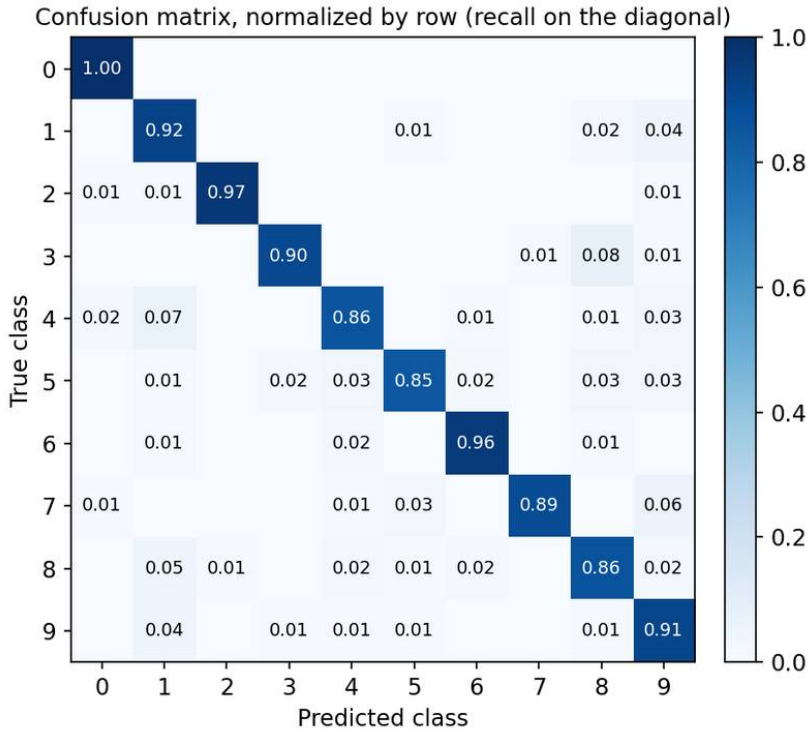


圖 *1* — 以 *200* 張數字訓練、在 *899* 張上測試的邏輯迴歸之依列正規化混淆矩陣（準確率 *91.1%*）。每一列總和為 *1*；對角線是各類別的召回率，較亮的非對角格指出值得修正的混淆。

輪流把每個類別 k 當作「陽性」、其他類別當作「陰性」，就得到逐類別的計數：$\mathrm{TP}_k = C_{kk}$，$\mathrm{FP}_k = \sum_{i \neq k} C_{ik}$（第 k 行的其餘部分），$\mathrm{FN}_k = \sum_{j \neq k} C_{kj}$（第 k 列的其餘部分）。因此逐類別精確率 $\mathrm{P}_k = C_{kk} / \sum_i C_{ik}$，召回率 $\mathrm{R}_k = C_{kk} / \sum_j C_{kj}$。把 K 個逐類別數字合成一個，常見的有三種做法：

- **巨觀平均**（macro average）：對類別直接取平均，$\mathrm{F}_1^{\text{macro}} = \frac{1}{K} \sum_k \mathrm{F}_{1,k}$。每個類別權重相同，所以稀有類別和常見類別一樣重要。
- **微觀平均**（micro average）：先把各類別的 TP_k、FP_k、FN_k 加總，再算一次指標。在單標籤多類別問題中，每一個錯誤都同時是一個 FP（對預測的類別）和一個 FN（對真實的類別），所以微觀精確率 = 微觀召回率 = 微觀 F_1 = 準確率。
- **加權平均**（weighted average）：以各類別的**支持數**（support）n_k（真實樣本數）為權重，對逐類別數值取平均。它很接近微觀平均，幾乎一樣會掩蓋稀有類別的失敗。

計算範例。 100 張測試影像，三個類別的支持數分別為 80（A）、15（B）、5（C）：

真實 ＼ 預測	A	B	C
A（80）	76	3	1
B（15）	4	10	1
C（5）	3	0	2

各行總和為 83、13、4，所以精確率為 $76/83 = 0.916$、$10/13 = 0.769$、$2/4 = 0.500$；召回率為

$76/80 = 0.950$、$10/15 = 0.667$、$2/5 = 0.400$；逐類別 F_1 為 0.933、0.714、0.444。於是

$$\mathrm{F}_1^{\text{macro}} = \tfrac{1}{3}(0.933 + 0.714 + 0.444) = 0.697, \qquad \mathrm{F}_1^{\text{micro}} = \text{accuracy} = \tfrac{76+10+2}{100} = 0.880,$$

而以支持數加權的 F_1 為 $0.80 \cdot 0.933 + 0.15 \cdot 0.714 + 0.05 \cdot 0.444 = 0.875$。微觀與巨觀之間 18 個百分點的差距就是訊號：模型在常見類別上很好，在稀有類別上很差。當每個類別都重要時（罕見疾病、稀有物種）回報巨觀平均；當每一張影像同等重要時，回報微觀平均／準確率。

Cohen's kappa [59] 把觀察到的一致程度，扣掉純靠機率就會有的一致程度：

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_o = \frac{1}{N}\sum_k C_{kk}, \qquad p_e = \frac{1}{N^2}\sum_k \Big(\sum_j C_{kj}\Big)\Big(\sum_i C_{ik}\Big),$$

其中 p_o 是準確率，p_e 是一個與輸入無關、只依觀察到的比例隨機預測各類別的分類器所能得到的準確率。它原本是為兩位人類評分者之間的一致性設計的，在比較模型與標註者時仍很常見。

```
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (average_precision_score, classification_report,
 ↪  cohen_kappa_score,
                             confusion_matrix, precision_recall_fscore_support,
                              ↪  roc_auc_score)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.5, stratify=y, random_state=0)
clf = LogisticRegression(max_iter=2000).fit(X_tr[:200], y_tr[:200])   # only 200 labels
pred, prob = clf.predict(X_te), clf.predict_proba(X_te)

cm = confusion_matrix(y_te, pred)                      # rows = true, columns = predicted
cm_norm = confusion_matrix(y_te, pred, normalize="true")   # each row sums to 1 (Figure 1)
print(classification_report(y_te, pred, digits=3))    # per-class P, R, F1, support +
 ↪  averages
for avg in ["macro", "micro", "weighted"]:
    p, r, f, _ = precision_recall_fscore_support(y_te, pred, average=avg)
    print(f"{avg:8s} P={p:.3f} R={r:.3f} F1={f:.3f}")
print(f"kappa {cohen_kappa_score(y_te, pred):.3f}")
print(f"ROC-AUC (one-vs-rest, macro) {roc_auc_score(y_te, prob, multi_class='ovr'):.4f}")
Y = label_binarize(y_te, classes=range(10))            # one binary column per class
print(f"mean AP over classes {average_precision_score(Y, prob, average='macro'):.4f}")
# macro    P=0.914 R=0.911 F1=0.912
# micro    P=0.911 R=0.911 F1=0.911
# weighted P=0.915 R=0.911 F1=0.912
# kappa 0.901
# ROC-AUC (one-vs-rest, macro) 0.9958
# mean AP over classes 0.9742
```

這裡巨觀與微觀平均幾乎相同，因為手寫數字的各類別是平衡的；換成上面 A ／ B ／ C 的例子就不會如此。

不依賴門檻的評估：ROC 曲線與精確率–召回率曲線

上面每一個指標都需要一條決策規則（argmax，或「分數超過 t 就是陽性」）。會輸出分數的分類器，可以一次在所有門檻上評估：把 t 從高掃到低，每個門檻記下一個點。

- **ROC 曲線**畫的是 TPR 對 FPR。曲線下面積 **ROC-AUC** 等於「隨機取一個陽性樣本，它的分數高於隨機取的一個陰性樣本」的機率；0.5 是隨機，1.0 是完美分開。Fawcett [54] 是標準的入門文獻。
- **精確率–召回率（PR）曲線**畫的是精確率對召回率。它的總結指標**平均精確度**（average precision, AP），是在每次多找回一個陽性樣本的召回率位置上，對精確率取平均：

$$\mathrm{AP} = \sum_{n} (\mathrm{R}_n - \mathrm{R}_{n-1})\, \mathrm{P}_n,$$

其中 P_n 與 R_n 是依分數由高到低排序後、第 n 個門檻的精確率與召回率（這是 scikit-learn [62] 的定義；PASCAL VOC [61] 等基準用過內插的變體，所以要確認論文回報的是哪一種）。

為什麼嚴重不平衡時要看 PR。 FPR 的分母是陰性樣本數。當陰性樣本是陽性的 99 倍時，即使有上千個誤報，FPR 也幾乎不動，所以 ROC 曲線看起來很漂亮，精確率卻很差。精確率的分母是被標記的樣本數，誤報會直接打擊它，而 AP 的隨機基準線等於陽性比例，而不是 0.5。Davis 與 Goadrich [55] 證明：一條曲線在 ROC 空間中占優勢，若且唯若它在 PR 空間中也占優勢；並指出最佳化 ROC-AUC 並不等於最佳化 AP。Saito 與 Rehmsmeier [56] 以模擬與文獻回顧顯示，在不平衡資料上，PR 圖比 ROC 圖更有資訊量。圖 2 把這件事具體化：在陽性只占 1% 的任務上，兩個 ROC-AUC 相同（0.95）的分類器，AP 卻明顯不同（0.45 對 0.36）。

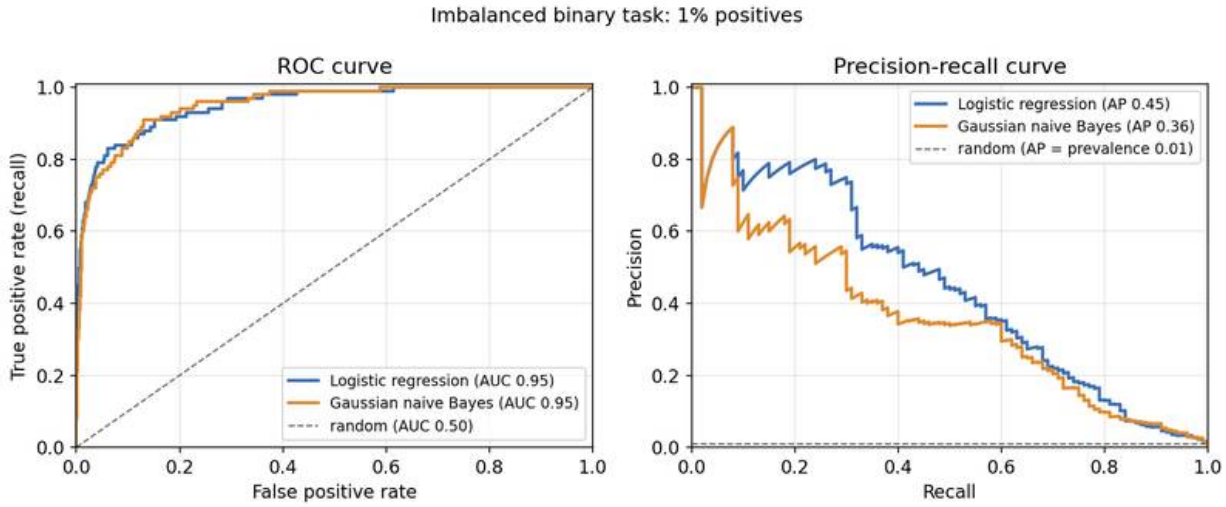


圖 *2* — 在陽性只占 *1%* 的合成任務上（*10,000* 個測試點中有 *100* 個陽性），兩個分類器的 *ROC* 曲線（左）與精確率–召回率曲線（右）。兩者 *ROC-AUC* 都是 *0.95*，但邏輯迴歸在每個召回率下的精確率都高得多（*AP 0.45* 對 *0.36*）。在這個任務上，一個全部判為陰性的分類器可以拿到 *99%* 的準確率。

當類別數 $K > 2$ 時，兩種曲線都以一**對其餘**（one-vs-rest）的方式計算：類別 k 為陽性、其他全

部為陰性，使用分數 p_k。再把 K 個面積做巨觀平均（`roc_auc_score(..., multi_class="ovr")`），或把所有（類別, 影像）配對合併後做微觀平均。

多標籤指標

多標籤分類中，每張影像有一個二元目標向量 $\mathbf{t} \in \{0,1\}^K$，模型則輸出一個分數向量。每個標籤都是自己的一個二元問題，所以自然的排序指標是**逐類別 AP** 及其對類別的平均 **mAP**；它隨 PASCAL VOC 分類任務 [61] 成為標準。需要門檻的指標分成兩大類，Zhang 與 Zhou [60] 有完整回顧：

- **以標籤為基礎**（label-based）的指標：對每個標籤算一個二元指標，再對標籤做巨觀或微觀平均，與多類別的情況完全相同。
- **以樣本為基礎**（example-based）的指標：對每一張影像算一個分數，再對影像取平均。以樣本為基礎的 F_1 為 $\frac{1}{N}\sum_i \frac{2|\hat{Y}_i \cap Y_i|}{|\hat{Y}_i| + |Y_i|}$，其中 Y_i 與 $\hat{Y}_i$ 是影像 i 的真實與預測標籤集合。
- **Hamming loss** 是（影像, 標籤）位元中錯誤的比例，$\frac{1}{NK}\sum_i |\hat{Y}_i \,\triangle\, Y_i|$，其中 $\triangle$ 是對稱差。它很好解讀，但當稀有標籤很多時，什麼都不預測看起來也會很好。

```
import numpy as np
from sklearn.metrics import average_precision_score, f1_score, hamming_loss

# 4 street photos, 3 labels: car, person, bicycle
T = np.array([[1, 1, 0], [1, 0, 0], [0, 1, 1], [1, 1, 1]])            # ground truth
S = np.array([[.9, .6, .2], [.8, .4, .3], [.3, .7, .4], [.6, .2, .7]])  # sigmoid scores
P = (S >= 0.5).astype(int)                                             # threshold 0.5

print("per-class AP:", average_precision_score(T, S, average=None).round(3))
print(f"mAP {average_precision_score(T, S, average='macro'):.3f}")
print(f"example-based F1 {f1_score(T, P, average='samples'):.3f}  "
      f"macro F1 {f1_score(T, P, average='macro'):.3f}  micro F1 {f1_score(T, P,
       ↪  average='micro'):.3f}")
print(f"Hamming loss {hamming_loss(T, P):.3f}")
# per-class AP: [1.    0.917 1.   ]
# mAP 0.972
# example-based F1 0.867  macro F1 0.822  micro F1 0.857
# Hamming loss 0.167
```

校準：機率說的是實話嗎？

準確率只檢查 argmax。**校準**（calibration）檢查信心值是否名副其實：在所有信心為 0.8 的預測中，應該大約有 80% 是對的。正式地說，若對每個信心值 c 都有 $P(\hat{y} = y \mid \hat{p} = c) = c$，分類器就是校準良好的，其中 $\hat{y}$ 是預測類別，$\hat{p} = \max_k p_k$ 是它的機率。常用的總結指標是**期望校準誤差**（expected calibration error, ECE）：把預測依信心分到 M 個區間 B_m，計算

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\, \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|,$$

其中 n 是測試影像數，$\mathrm{acc}(B_m)$ 是第 m 個區間內的準確率，$\mathrm{conf}(B_m)$ 是區間內的平均信心。Guo 等人 [47] 發現，用交叉熵長時間訓練的現代深度網路傾向**過度自信**，而只用一個純量就能修正得相當好。我們會在穩健性一節回來談。

資料集與它們的角色

白話版。基準資料集是一份凍結的考卷。它讓大家能公平比較方法；但當整個領域都在準備同一份考卷，分數衡量的就不只是能力，也包括這份考卷本身。把兩種角色分開會比較清楚：用來訓練模型的資料集，以及只用來評估模型的資料集。

訓練用資料集

資料集	影像數	類別數	解析度	角色
MNIST [3]	訓練 6 萬／測試 1 萬	10 個數字	28×28 灰階	健全性檢查、教學
CIFAR-10 / CIFAR-100 [4]	訓練 5 萬／測試 1 萬	10 ／ 100（20 個超類別）	32×32 RGB	小規模的架構與配方研究
ImageNet-1k（ILSVRC 2012）[2]	約 128 萬訓練／ 5 萬驗證	1,000	不固定，通常以 224×224 訓練	骨幹網路的標準基準
ImageNet-21k [5]	約 1,300–1,400 萬（版本不同）	約 2.1 萬	不固定	更大規模的監督式預訓練
iNaturalist [6]	數十萬，依年份而異	數千個物種	不固定	細粒度、長尾

MNIST 已經被解決，現在只用來抓程式錯誤。CIFAR 小到可以跑大量受控實驗，所以正則化論文特別愛用。ImageNet-1k 在 2012 年挑戰賽之後成為參考標準 [2], [10]：架構通常以「ImageNet-1k 驗證集、224×224 的 top-1」來回報，模型庫裡幾乎每個預訓練骨幹都附有一個 ImageNet 分數。ImageNet-21k 是 ImageNet-1k 所取自的更大階層；Ridnik 等人 [5] 以清理過、考慮語意階層的配方，讓在它上面預訓練變得實際可行。

評估用基準資料集

白話版。訓練資料負責教，評估資料負責考。只考過一份考卷的模型，可能只是學會了那份考卷。所以現代論文會用一整組測試集來評估骨幹網路，每一個都設計來探測一件事：原始分佈、從同一分佈重新抽的樣本、損壞或風格化的影像、不尋常的視角、細粒度類別、很少的標註預算，或稀有類別。

下表列出最常見的評估集。大小以所引用的論文或官方頁面為準。

基準	衡量什麼	大小／類別數	常回報的指標
ImageNet-1k 驗證集 [2]	分佈內準確率；因為 ILSVRC 測試集的標籤保留在評估伺服器上，它成了實質上的測試集	50,000 張，1,000 類	top-1（top-5）

基準	衡量什麼	大小／類別數	常回報的指標
ImageNet-ReaL [8]	同一批驗證影像，重新標註為多標籤的「重新評估」標籤	46,837 張影像、57,553 個標籤（3,163 張沒有任何標籤的影像被捨棄）	ReaL 準確率：top-1 預測只要落在標籤集合內就算對
ImageNetV2 [7]	依原始收集流程重做一次所得到的新樣本上的泛化能力	每個版本 10,000 張（每類 10 張）；三種抽樣版本，通常使用 MatchedFrequency	top-1
ImageNet-C [42]	對合成損壞的穩健性	驗證影像 × 15 種損壞 × 5 個嚴重程度	mean corruption error
ImageNet-R [43]	各種表現形式：藝術、卡通、雕塑、玩具等	30,000 張，200 類	200 類上的 top-1
ImageNet-A [44]	經篩選、專門騙倒 ResNet-50 的自然照片	7,500 張，200 類	200 類上的 top-1
ImageNet-Sketch [63]	轉換到黑白素描領域	50,000 張，1,000 類（每類 50 張）	top-1
ObjectNet [64]	控制偏差的照片，刻意變化旋轉、背景與視角；只有測試集	50,000 張，313 類（其中 113 類與 ImageNet 重疊）	重疊類別上的 top-1
ImageNet-LT [73]	長尾分佈的訓練、平衡的測試	11.58 萬張訓練影像，1,000 類，每類 5 到 1,280 張；平衡的測試集	top-1，常依類別頻率分組回報
iNaturalist [6]	天然長尾、細粒度的物種	見訓練用資料集表	top-1

細粒度與遷移基準。骨幹網路也要看它的特徵能多好地遷移到小型、專門的資料集上，方式是在凍結特徵上做線性探測，或是微調（見下文的遷移學習一節）。

資料集	領域	大小／類別數	常回報的指標
CUB-200-2011 [65]	鳥類物種，附部位與屬性標註	11,788 張，200 類	top-1
Stanford Cars [66]	汽車品牌、型號與年份	196 類；訓練 8,144 ／測試 8,041	top-1
FGVC-Aircraft [67]	三層階層的飛機型號	10,000 張，100 種型號	各類別平均準確率
Oxford-IIIT Pets [68]	貓狗品種	37 類，每類約 200 張	各類別平均準確率
Oxford Flowers-102 [69]	花卉物種	102 類，每類 40 到 258 張；訓練 2,040 ／測試 6,149	各類別平均準確率
Food-101 [70]	菜餚；訓練標籤刻意保留雜訊	101,000 張，101 類（每類訓練 750 ／測試 250）	top-1
Caltech-101 [71]	一般物體	101 類加背景類，每類約 40 到 800 張	各類別平均準確率

Cars 與 Flowers 的訓練／測試大小依 Kornblith 等人 [33] 所列的切分。注意其中好幾個基準回報的是**各類別平均準確率**（mean per-class accuracy，即巨觀平均的召回率），而不是一般的準確率，因為它們的測試集並不平衡；在同一個比較裡混用兩者是常見的錯誤。

有兩個**測試組**把這類資料集綁在一起，用一個數字總結遷移能力：

- Kornblith 等人 [33] 使用了 **12 個資料集**：Food-101、CIFAR-10、CIFAR-100、Birdsnap、SUN397、Stanford Cars、FGVC-Aircraft、PASCAL VOC 2007 分類、Describable Textures、

Oxford-IIIT Pets、Caltech-101 與 Oxford Flowers-102。依資料集不同，回報 top-1 準確率、各類別平均準確率或 11 點 mAP（VOC 2007），並比較線性探測、微調與從頭訓練三種設定。

- **VTAB** [72] 包含 19 個任務，分成三組：7 個自然（一般相機影像）、4 個專門（衛星與醫學影像）、8 個結構（場景幾何，例如計數或距離，大多來自模擬環境）。在 **VTAB-1k** 中，每個任務只提供 1,000 個標註訓練樣本（800 個訓練、200 個驗證），因此它測試的是：一個表徵能否從很少的標籤，適應到與預訓練資料很不一樣的任務。

零樣本評估。 CLIP [40] 這類影像–文字模型在評估時，完全不在目標資料集上訓練：把類別名稱寫成文字提示，再把影像歸到最相似的提示（見下文的 CLIP 一節）。CLIP 論文用這種方式在 30 多個既有資料集上做了評估，回報零樣本 CLIP 在 27 個資料集中的 16 個上勝過以 ResNet-50 特徵做的線性探測，並發現對 ImageNet 集成 80 個提示模板，比單一預設提示多了 3.5 個百分點。這帶來兩點提醒。第一，提示與類別名稱的寫法都是超參數；在測試集上調整它們，和在測試集上調學習率一樣，會洩漏測試資訊。第二，預訓練資料是網路規模、而且大多未公開，所以「零樣本」並不保證測試影像（或近乎重複的影像）從未被看過。

這些數字有多可靠？

有三條研究路線改變了我們解讀 ImageNet 準確率的方式。

- **重現。** Recht 等人 [7] 盡可能照原始收集流程，為 CIFAR-10 和 ImageNet 重建了新的測試集（ImageNetV2）。所有模型都掉分：CIFAR-10 掉 3–15%，ImageNet 掉 11–14%。但排名大致保持，舊測試集上的進步也轉移到新測試集上。他們的解讀是：掉分來自新舊樣本之間難度的細微偏移，而不是整個社群反覆使用測試集而過擬合（overfitting）。
- **標籤。** Beyer 等人 [8] 為 ImageNet 驗證集重新標註、允許一張影像有多個標籤，發現近年的部分「進步」其實是在擬合原始單一標籤標註的特殊習慣。Northcutt 等人 [9] 估計常見測試集中普遍存在標籤錯誤，而且這些錯誤足以改變哪個模型看起來最好。
- **偏差。**每個資料集在視角、背景、拍攝者風格上都有自己的「指紋」。DIP 第 13 章回顧了資料集偏差的研究；實務上的教訓是：只要可能，就用獨立收集的資料來測試。

把基準分數當成關於「這個基準」的陳述

「81.0% top-1」的意思是：在這 50,000 張特定的驗證影像上、用這套前處理、這組標籤所得到的結果。請回報解析度、裁切數量、是否用了額外資料或蒸餾，並在差異小於一個百分點時回報不同隨機種子間的變異。

評估流程

基準分數有多誠實，取決於背後的評估流程。

- **三種切分，三種任務。**在**訓練集**上擬合參數。在**驗證集**上（或在訓練資料內做交叉驗證）選擇超參數、提前停止點、溫度、門檻與提示。**測試集**只在最後碰一次。若基準只公開驗證集（如 ImageNet-1k），就從訓練集中切出一部分用來調整。
- **絕不在測試集上調整。**每一個看過測試結果之後才做的決定（多加一種擴增、多訓練一個 epoch、換一個檢查點），都會讓模型稍微擬合測試集。整個領域的論文累積起來，就是**測試集**

重複使用（test-set reuse），也就是對基準的適應性過擬合。ImageNetV2 [7] 發現在新樣本上排名大致保持，這令人安心；但同一份研究也顯示絕對數字無法直接轉移，而新的測試集是唯一直接的檢驗方法。

- **回報變異**。用多個隨機種子訓練（至少三個，小資料集要更多），回報平均 ± 標準差，或信賴區間。在 10,000 張測試影像上，ImageNetV2 [7] 給出的 95% 信賴區間約為 ±1%；在這樣的測試集上進步 0.3 個百分點，算不上結果。
- **回報完整配方**。解析度、裁切數、額外資料、蒸餾、指標的哪一種變體（top-1 或各類別平均準確率、哪一種 AP 定義），以及標籤集的哪一個版本（原始或 ReaL）。

CNN 架構與其背後的理由

白話版。每一個著名的 CNN 都修正了前一代的某個具體問題：訓練太慢、參數太多、太深就難以最佳化、在手機上太昂貴。知道每個架構解決了什麼問題，比背它們的層表更有用。

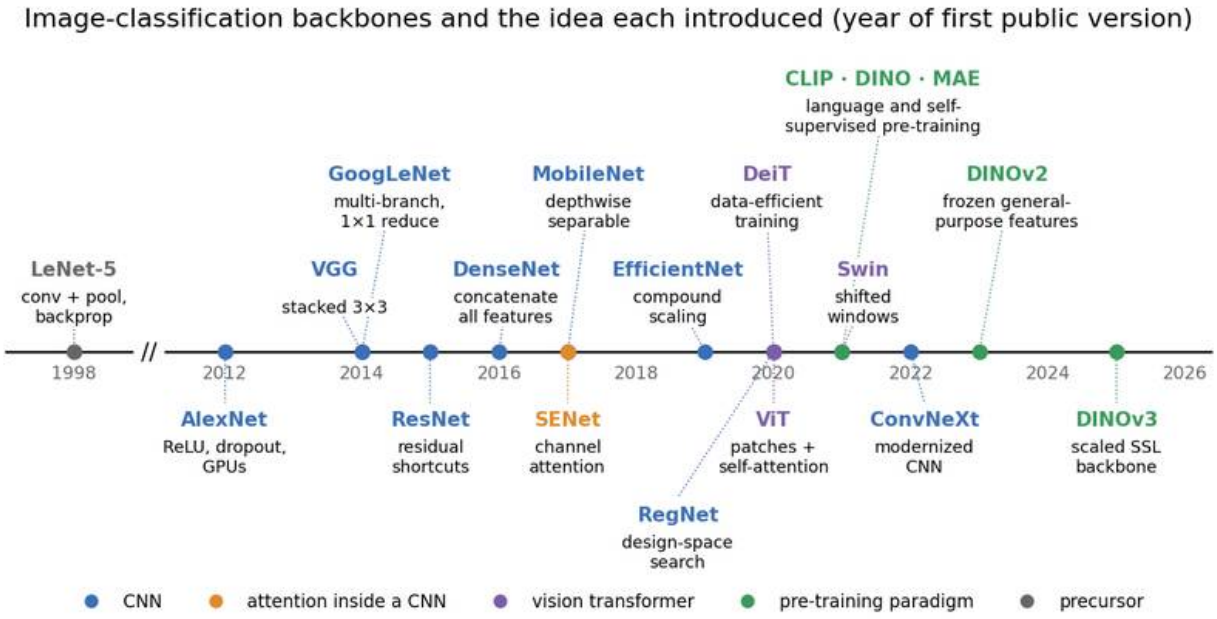


圖 3 — 影像分類骨幹網路與各自引入的主要想法。年份為首次公開版本（通常是 *arXiv*）的年份。顏色區分純 *CNN*、*CNN* 內的注意力機制、視覺 *Transformer*，以及預訓練典範。

AlexNet：深度變得可訓練

2012 年以前，最好的 ImageNet 系統使用手工特徵（SIFT、Fisher 向量）加線性分類器。AlexNet [10] 是一個五層卷積、三層全連接的網路，在 ImageNet 上端到端訓練。讓它能在這個規模上成功的有三個要素。**ReLU** 激活函數 $\max(0, z)$ 對正輸入不會飽和，梯度不會像 tanh 那樣消失，論文回報訓練速度快了好幾倍。**Dropout** 用在大型全連接層上對抗過擬合，那些層占了大部分參數。**GPU** 讓這一切可行：模型被拆在兩張 GPU 上。大量的資料擴增（隨機裁切與翻轉、顏色擾

動）是第四個、常被忘記的要素。設計上的教訓是：只要資料和算力足夠，學出來的特徵勝過設計出來的特徵。

VGG：小卷積核，疊起來

VGG [11] 問了一個更乾淨的問題：如果只用 3×3 卷積，然後一直加深（16–19 個有權重的層），會怎樣？兩層 3×3 疊起來看到的是 5×5 區域，三層是 7×7。對 C 個輸入與輸出通道而言，

$$\underbrace{2\cdot 3^2C^2}_{\text{two }3\times 3} = 18C^2 \;<\; \underbrace{5^2C^2}_{\text{one }5\times 5} = 25C^2,$$

（忽略偏差項）。所以同樣的感受野，疊層的參數更少，而且兩層之間多了一個非線性。VGG 的一致設計讓它多年來都是熱門的特徵擷取器。它的弱點是成本：全連接頭與寬大的前幾層讓它很笨重。

GoogLeNet ／ Inception：寬度、分支與 1×1 瓶頸

GoogLeNet [12] 則改攻成本。每個 **Inception 模組**平行執行 1×1、3×3、5×5 卷積與池化，再把輸出串接起來，讓下一層自己挑選適合的尺度。在很多通道上跑大卷積核很昂貴，所以先用 1×1 **卷積**減少通道數。1×1 卷積就是在每個像素上做一次跨通道的線性映射：它混合通道而不看鄰居，是改變寬度最便宜的方法。GoogLeNet 也用**全域平均池化**取代龐大的全連接頭。後續的 Inception-v3 論文 [13] 把大卷積核分解成小卷積的堆疊，或 $1\times n$ 與 $n\times 1$ 的組合，並提出了**標籤平滑**（label smoothing），我們會在訓練一節再見到它。

ResNet：學習殘差

問題。一直疊加普通的層，最後網路反而變差，連訓練集上都一樣。這種「退化」是最佳化問題，不是過擬合：更深的網路原則上可以複製較淺的網路，再把多出來的層設成恆等映射，但 SGD 就是找不到這個解。

想法。 He 等人 [14] 讓一個區塊學習殘差 $\mathcal{F}$，再把輸入加回去：

$$\mathbf{y} = \mathcal{F}(\mathbf{x};\{W_i\}) + \mathbf{x},$$

其中 $\mathbf{x}$ 是區塊輸入，$\mathcal{F}$ 是權重為 W_i 的兩到三層卷積，$\mathbf{y}$ 是輸出。如果最佳映射接近恆等，區塊只需要把 $\mathcal{F}$ 推向零，這很容易。當形狀改變（通道變多、步幅為 2）時，捷徑改成 1×1 投影。

為什麼好訓練。把 L 個預激活（pre-activation）形式的區塊 $\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l)$ 展開，如後續關於恆等映射的論文 [15] 所分析：

$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i), \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L}\Big(\mathbf{I} + \frac{\partial}{\partial \mathbf{x}_l}\sum_{i=l}^{L-1}\mathcal{F}(\mathbf{x}_i)\Big),$$

其中 $\mathcal{L}$ 是損失函數，$\mathbf{I}$ 是單位矩陣。恆等項把梯度從頂端直接送到任何一個區塊，所以它不會因為許多小 Jacobian 連乘而消失。該論文也發現，讓捷徑保持純粹的恆等（把 BN 與 ReLU 移進殘差分支，即「預激活」）可以更容易地訓練更深的網路。

瓶頸結構。對更深的 ResNet（50 層以上），每個區塊變成 1×1（降維）→ 3×3 → 1×1（升維），如圖 4(c)。在 256 通道時，兩層 3×3 的基本區塊有 $2\cdot9\cdot256^2\approx118$ 萬個權重；降到 64 通道的瓶頸區塊則有 $256\cdot64+9\cdot64^2+64\cdot256\approx7$ 萬個。昂貴的 3×3 跑在很薄的張量上，這正是 50–152 層負擔得起的原因。

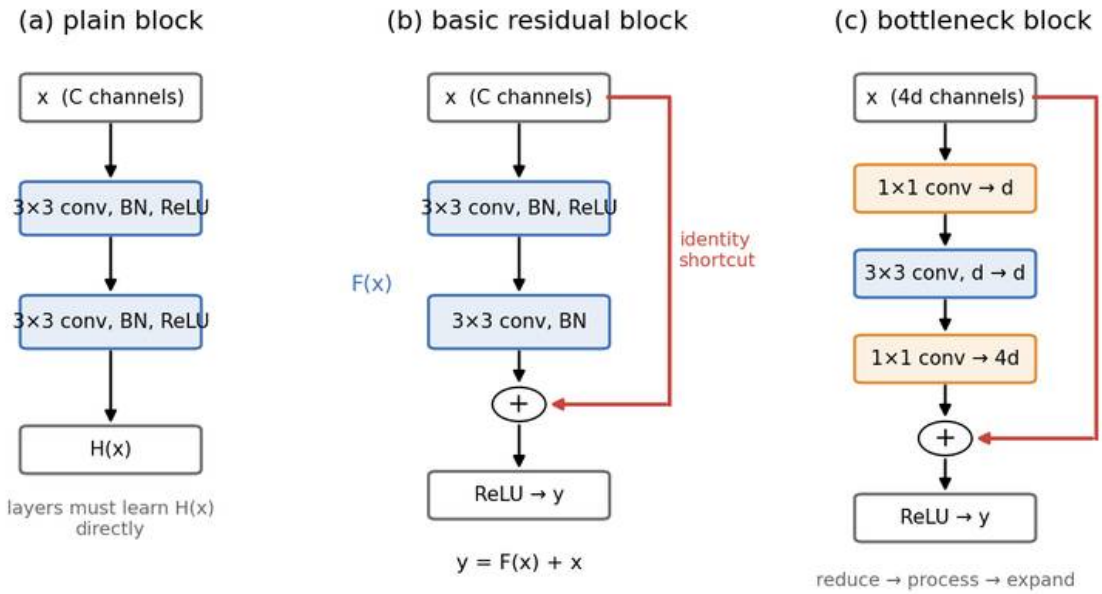


圖 *4* — (*a*) 普通區塊必須學習完整的映射 $H(x)$。(*b*) 殘差區塊學習 $F(x)=H(x)-x$，再透過恆等捷徑把輸入加回去。(*c*) 瓶頸區塊在縮減後的寬度 d 上做 *3×3* 卷積，再升回 *4d*。

本篇實驗所用的基本區塊，以 PyTorch 寫成：

```
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(cout)
        self.conv2 = nn.Conv2d(cout, cout, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)
        self.short = nn.Identity()
        if stride != 1 or cin != cout:   # projection shortcut when shapes change
            self.short = nn.Sequential(nn.Conv2d(cin, cout, 1, stride, bias=False),
             ↪  nn.BatchNorm2d(cout))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.short(x))   # y = F(x) + x
```

DenseNet：把所有東西都再利用

DenseNet [16] 把特徵再利用推得更遠：在一個密集區塊內，第 ℓ 層接收所有前面特徵圖的**串接**，$\mathbf{x}_\ell = H_\ell([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{\ell-1}])$，其中 $[\cdot]$ 表示通道串接，H_ℓ 是 BN–ReLU–卷積。每層只新增少量通道（「成長率」），所以參數效率高，每一層到損失函數都有一條短路徑。代價是記憶體：串接的激活值隨深度增長，使 DenseNet 在實務上比參數量所暗示的還慢。這是之後會一再出現的教訓：參數量、FLOPs 與實際執行時間是三件不同的事。

MobileNet：把卷積拆開

MobileNet [17] 的目標是手機。在 $H \times W$ 的特徵圖上，從 M 個通道到 N 個通道的標準 $K \times K$ 卷積需要 HWK^2MN 次乘加。**深度可分離**（depthwise separable）卷積把它拆成兩步：**逐通道**（depthwise）$K \times K$ 卷積，每個通道各自濾波（HWK^2M）；以及**逐點**（pointwise）1×1 卷積，負責混合通道（$HWMN$）。兩者的成本比為

$$\frac{HWK^2M + HWMN}{HWK^2MN} = \frac{1}{N} + \frac{1}{K^2},$$

所以當 $K = 3$、輸出通道很多時，可分離版本大約便宜 8–9 倍。它押注的是：空間濾波與通道混合不需要一起學。這個賭注押得很成功，深度卷積後來在 EfficientNet 與 ConvNeXt 中再次出現。

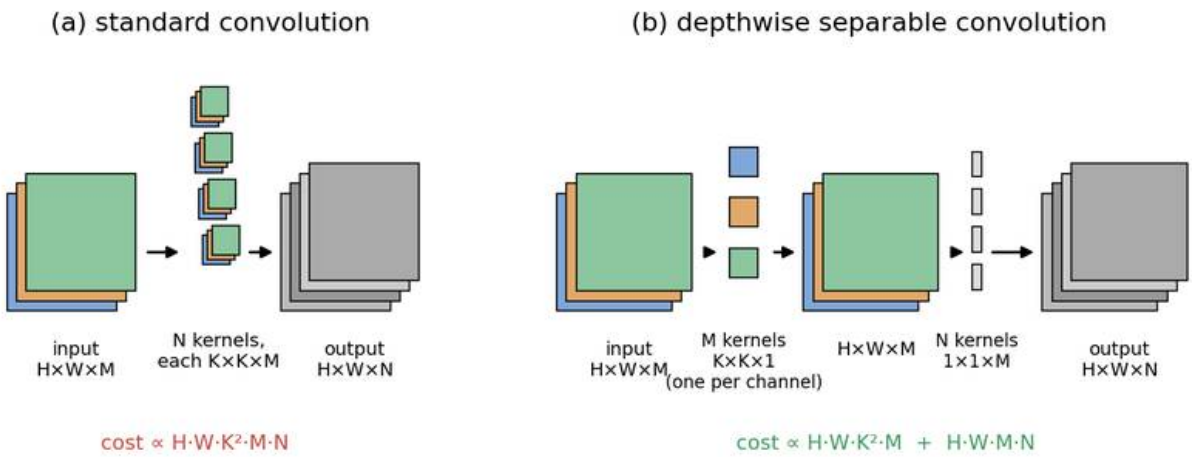


圖 5 — 標準卷積在同一步裡完成濾波與通道混合。深度可分離卷積先對每個通道各自濾波（*depthwise*），再用 *1×1* 卷積核混合通道（*pointwise*）。

```
import torch.nn as nn

def n_params(m):
    return sum(p.numel() for p in m.parameters())

M, N, K = 128, 128, 3
standard = nn.Conv2d(M, N, K, padding=1, bias=False)
```

```
separable = nn.Sequential(nn.Conv2d(M, M, K, padding=1, groups=M, bias=False),   # depthwise
                          nn.Conv2d(M, N, 1, bias=False))                         #
                           ↪  pointwise
print("standard:", n_params(standard), " separable:", n_params(separable),
      f" ratio {n_params(separable) / n_params(standard):.3f} vs 1/N + 1/K^2 = {1/N +
       ↪  1/K**2:.3f}")
# standard: 147456  separable: 17536  ratio 0.119 vs 1/N + 1/K^2 = 0.119
```

EfficientNet：深度、寬度、解析度一起放大

手上有一個好的小網路，要怎麼把它變大？加深、加寬，還是提高輸入解析度？Tan 與 Le [18] 主張這三者互相影響：解析度更高的影像需要更深（更大的感受野）與更寬（更細的圖樣）。**複合縮放**（compound scaling）用一個係數 ϕ 同時放大三者：

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \qquad \text{s.t. } \alpha\beta^2\gamma^2 \approx 2, \;\; \alpha, \beta, \gamma \geq 1,$$

其中 d、w、r 分別是深度、寬度與解析度的倍率。FLOPs 大約與 $d\,w^2 r^2$ 成正比，所以 ϕ 每加 1，成本大約加倍。常數是在基礎網路上做小型網格搜尋得到的（基礎網路本身由架構搜尋找出，使用帶深度卷積與 squeeze-and-excitation 的反向瓶頸區塊）。「平衡地放大」這個想法，比那些特定常數活得更久。

RegNet：設計空間，而不是單一設計

Radosavovic 等人 [19] 研究的是網路的族群，而不是單一贏家。他們從一個廣泛的類 ResNet 網路空間出發，反覆加上限制（例如共用瓶頸比例、各階段寬度遞增），並檢查錯誤率的分佈是否變好。最後留下的空間 RegNet，用一條簡單的量化線性規則描述各階段寬度。這是方法論上的教訓：好的架構遵循簡單的規律，而比較模型的分佈，比拿一個調好的模型去比另一個更可靠。

ConvNeXt：一步一步現代化的 CNN

在視覺 Transformer 拿下排行榜之後，Liu 等人 [20] 問：它們的優勢有多少來自注意力機制，又有多少來自其他一切？他們從一個用 Transformer 風格配方訓練的 ResNet-50 出發，一次只改一件事：仿照 Swin 的階段比例、「patchify」主幹（4×4、步幅 4 的卷積）、用更大的 7×7 深度卷積、反向瓶頸、更少的激活與正規化層、GELU 與 LayerNorm。結果 ConvNeXt 在 ImageNet 與下游任務上，追平了相近成本的 Swin 類 Transformer。教訓是：架構家族的差異，沒有伴隨而來的許多小設計與訓練選擇那麼重要。

注意力機制與 Transformer

白話版。卷積只用固定的權重看一個小鄰域。注意力機制讓每個位置根據內容，自己決定要聽哪些其他位置的話。視覺 Transformer 把這個機制套用到影像小塊上。

Squeeze-and-excitation：CNN 裡的注意力

SE 區塊 [21] 為任何 CNN 加上便宜、依內容而定的**通道**權重。它先用全域平均池化把每個通道「擠壓」成一個數字，經過一個小型兩層 MLP，再用 sigmoid 閘門乘回每個通道：

$$\mathbf{s} = \sigma(W_2\,\delta(W_1\,\bar{\mathbf{u}})), \qquad \tilde{\mathbf{u}}_c = s_c\,\mathbf{u}_c,$$

其中 $\mathbf{u}_c$ 是特徵圖的第 c 個通道，$\bar{\mathbf{u}}$ 是各通道平均值組成的向量，δ 是 ReLU，W_1 以比例 r 降維，W_2 再升回原維度。網路因此能在整體脈絡顯示「動物」時，加強「毛皮」相關的通道。SE 區塊是 EfficientNet 與許多行動裝置網路的一部分。

```
import torch
import torch.nn as nn

class SE(nn.Module):
    """Squeeze-and-excitation: reweight channels using global context."""
    def __init__(self, c, r=16):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(c, c // r), nn.ReLU(), nn.Linear(c // r, c),
         ↪  nn.Sigmoid())

    def forward(self, x):                       # x: (B, C, H, W)
        s = x.mean(dim=(2, 3))                  # squeeze: (B, C)
        w = self.fc(s)[:, :, None, None]        # excitation: (B, C, 1, 1) in (0, 1)
        return x * w

print(SE(64)(torch.randn(2, 64, 8, 8)).shape)
```

ViT：把影像當成一串小塊

視覺 Transformer（Vision Transformer, ViT）[22] 幾乎完全拿掉卷積。影像 $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$ 被切成 $N = HW/P^2$ 個不重疊的 $P\times P$ 小塊（patch）。每個小塊攤平後線性投影成 D 維的 **token**。前面再加上一個可學習的**類別 token**，並加上可學習的**位置嵌入**：

$$\mathbf{z}_0 = [\,\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1\mathbf{E};\ \dots;\ \mathbf{x}_p^N\mathbf{E}\,] + \mathbf{E}_{\text{pos}},$$

其中 $\mathbf{x}_p^i \in \mathbb{R}^{P^2C}$ 是第 i 個攤平的小塊，$\mathbf{E} \in \mathbb{R}^{P^2C\times D}$ 是小塊嵌入矩陣，$\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ 是位置嵌入。接著由 L 層標準 Transformer 編碼器（多頭自注意力與 MLP，各自帶 LayerNorm 與殘差連接）處理這個序列，最後由一個小型分類頭讀取類別 token 的最終狀態，輸出 logits。單一個注意力頭的自注意力為

$$\text{Attn}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\Big)\mathbf{V},$$

其中 $\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q$、$\mathbf{K} = \mathbf{Z}\mathbf{W}_K$、$\mathbf{V} = \mathbf{Z}\mathbf{W}_V$ 是 token 矩陣 $\mathbf{Z}$ 的線性投影，d 是 key 的維度。注意力矩陣大小為 $(N+1)\times(N+1)$：從第一層開始，每個小塊就能注意到所有其他小塊，所以感受野一開始就是全域的。

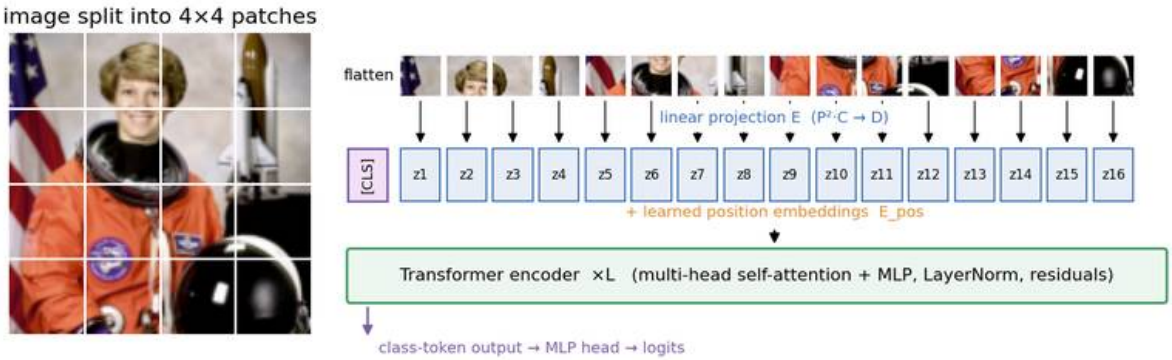


圖 *6* — *ViT* 的輸入流程。所有小塊共用同一個線性投影變成 *token*（等價於核大小 $P\times P$、步幅 P 的卷積）。加上類別 *token* 與可學習的位置嵌入後，用類別 *token* 的最終狀態來分類。

```
import torch
import torch.nn as nn

B, C, H, W, P, D = 2, 3, 32, 32, 8, 64
img = torch.randn(B, C, H, W)

# 1) patchify: (B, C, H, W) -> (B, N, P*P*C) with N = (H/P)*(W/P)
patches = img.unfold(2, P, P).unfold(3, P, P)                  # (B, C, H/P, W/P, P, P)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, -1, C * P * P)
N = patches.shape[1]

# 2) linear patch embedding + class token + learned position embedding
embed = nn.Linear(C * P * P, D)          # same as nn.Conv2d(C, D, kernel_size=P, stride=P)
cls = nn.Parameter(torch.zeros(1, 1, D))
pos = nn.Parameter(torch.randn(1, N + 1, D) * 0.02)
z = torch.cat([cls.expand(B, -1, -1), embed(patches)], dim=1) + pos   # (B, N+1, D)

# 3) one head of scaled dot-product self-attention
Wq, Wk, Wv = (nn.Linear(D, D, bias=False) for _ in range(3))
q, k, v = Wq(z), Wk(z), Wv(z)
A = (q @ k.transpose(1, 2) / D ** 0.5).softmax(dim=-1)         # (B, N+1, N+1), rows sum to 1
out = A @ v
print("tokens:", N + 1, " attention:", tuple(A.shape), " class-token output:", tuple(out[:,
 ↪  0].shape))
```

為什麼 ViT 需要大量資料。CNN 內建了一些假設，稱為**歸納偏差**（inductive bias）：局部性（一個像素主要和鄰居有關）與平移等變性（同一個濾波器用在每個位置）。ViT 幾乎沒有這些；除了切小塊與位置嵌入之外，空間結構都得從資料中學。ViT 論文 [22] 回報：只在 ImageNet-1k 上訓練時，ViT 落後於規模相近的 ResNet；但在大得多的資料集上預訓練（ImageNet-21k，或 3 億張影像的 JFT）之後，就能追平甚至超越。弱先驗在小規模時要付出資料的代價，到了大規模就不再是限制。同一篇論文也嘗試了**混合**（hybrid）模型：把 ResNet 主幹的特徵圖、而不是原始小塊，送進 Transformer；混合模型在計算預算小時較有幫助，模型變大後差距就消失了。

DeiT：只用 ImageNet 訓練 ViT

DeiT [23] 證明「ViT 需要巨量資料」這個結論，有一部分其實是關於訓練配方。用強力的資料擴增（RandAugment、Mixup、CutMix、random erasing）、隨機深度、重複擴增與 AdamW，只在 ImageNet-1k 上訓練的 ViT-B 就達到 83.1% top-1。DeiT 還加入了**蒸餾 token**：第二個可學習的 token，和類別 token 一樣，但它的輸出被訓練去匹配教師網路的預測（他們發現卷積網路當教師效果最好），而類別 token 則以真實標籤訓練。這是知識蒸餾 [51] 的一種 Transformer 專屬形式。測試時把兩個頭的輸出結合起來。

Swin：視窗、位移與階層

全域注意力的成本隨 token 數量呈平方成長，對高解析度輸入與密集預測任務是個問題。Swin [24] 只在不重疊的 $M \times M$ 視窗內計算注意力，並在相鄰層之間把視窗網格位移半個視窗，讓資訊能跨越視窗邊界。各階段之間合併 2×2 相鄰小塊來減少 token 數，形成步幅為 4、8、16、32 的類 CNN 金字塔。對 $h \times w$ 個 token、C 個通道，論文給出

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$

所以在視窗大小 M 固定時，視窗注意力對 token 數 hw 是線性的。Swin 帶回了兩個 CNN 先驗：局部性與階層性，成為偵測與分割也通用的骨幹。視覺 Transformer 的整體版圖可參考 Khan 等人的綜述 [25]。

訓練配方

白話版。同一個網路，訓練方法不同，準確率可以差好幾個百分點。現代訓練配方是一組小技巧，每一個都讓「死背」更難，或讓最佳化更平順。

資料擴增

- **隨機縮放裁切**（random resized crop）隨機取一塊占原圖面積一定比例、長寬比隨機的區域，再縮放到訓練解析度。它教會網路尺度與位置不變性，是 ImageNet 訓練的預設做法。**水平翻轉**只在不改變標籤時使用（大多數照片可以，文字或數字不行）。
- **Mixup** [26] 混合兩張影像與它們的標籤：$\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j$、$\tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1-\lambda)\mathbf{y}_j$，其中 $\lambda \sim \text{Beta}(\alpha, \alpha)$，$\mathbf{y}$ 為 one-hot。它鼓勵模型在訓練樣本之間呈線性行為。
- **CutMix** [27] 把影像 j 的一個隨機矩形貼到影像 i 上，並依貼上的面積比例混合標籤。和 Mixup 不同，每個像素都仍是自然的，網路必須從局部視野辨認物體。
- **RandAugment** [28] 用兩個數字取代學出來的擴增策略：隨機套用 N 個操作（旋轉、剪切、色階分離、對比……），共用同一個強度 M。論文回報這個極小的搜尋空間就能追平或超越先前的自動化策略，而合適的強度取決於模型與資料集大小。

```
import numpy as np
import torch
```

```
import torch.nn.functional as F

def mixup(x, y, num_classes, alpha=0.2):
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0))
    y1 = F.one_hot(y, num_classes).float()
    return lam * x + (1 - lam) * x[idx], lam * y1 + (1 - lam) * y1[idx]

def cutmix(x, y, num_classes, alpha=1.0):
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0))
    H, W = x.shape[-2:]
    h, w = int(H * np.sqrt(1 - lam)), int(W * np.sqrt(1 - lam))
    cy, cx = np.random.randint(H), np.random.randint(W)
    y0, y1_, x0, x1_ = max(cy - h // 2, 0), min(cy + h // 2, H), max(cx - w // 2, 0), min(cx
    ↪  + w // 2, W)
    x = x.clone()
    x[..., y0:y1_, x0:x1_] = x[idx][..., y0:y1_, x0:x1_]
    lam = 1 - (y1_ - y0) * (x1_ - x0) / (H * W)            # actual area kept
    y1 = F.one_hot(y, num_classes).float()
    return x, lam * y1 + (1 - lam) * y1[idx]

def soft_ce(logits, soft_targets):
    return -(soft_targets * logits.log_softmax(1)).sum(1).mean()
```

標籤平滑

以 one-hot 為目標的交叉熵會永遠把正確 logit 往上推；只有在間距無限大時，損失才會到零。**標籤平滑** [13] 把目標換成

$$\mathbf{q} = (1 - \varepsilon)\,\mathbf{e}_y + \frac{\varepsilon}{K}\mathbf{1},$$

其中 ε（常用 0.1）是平滑強度，$\mathbf{e}_y$ 是 one-hot 向量，$\mathbf{1}$ 是全 1 向量。此時最佳解是有限的 logit 間距，對網路有正則化效果，通常也會減少過度自信。在 PyTorch 裡只是一個參數：`F.cross_entropy(logits, y, label_smoothing=0.1)`。

最佳化器、排程與正則化

- **AdamW** [29] 把權重衰減與自適應梯度步驟解耦：不是把 $\lambda\theta$ 加到梯度上（Adam 會再對每個參數重新縮放），而是直接縮小權重，$\theta \leftarrow \theta - \eta(\hat{\mathbf{m}}/(\sqrt{\hat{\mathbf{v}}} + \epsilon) + \lambda\theta)$，其中 η 是學習率，$\hat{\mathbf{m}}, \hat{\mathbf{v}}$ 是 Adam 的動差估計，λ 是衰減係數。它是 Transformer 與現代 CNN 配方的預設選擇。偏差與正規化層的參數通常不做衰減。
- **暖身＋餘弦排程**。訓練最初幾步，正規化統計量與 Adam 的動差估計都還不準，大學習率會不穩定，所以先讓學習率在幾個 epoch 內線性上升，接著沿半個餘弦曲線降到接近零，$\eta_t = \frac{1}{2}\eta_{\max}(1 + \cos(\pi t/T))$，$t$ 是暖身後的步數，T 是剩餘步數；這個想法由 SGDR [30] 推廣開來。

- **隨機深度**（stochastic depth）[31] 在訓練時隨機跳過整個殘差分支（區塊變成恆等映射），跳過機率隨深度增加。它縮短了訓練時的有效深度，效果類似一群較淺網路的集成。
- **權重的 EMA** 維護一個指數移動平均 $\bar{\theta} \leftarrow \mu\bar{\theta} + (1-\mu)\theta$（$\mu$ 接近 1，例如 0.999），評估時使用 $\bar{\theta}$。它把最後幾步 SGD 的雜訊平滑掉。

```
import math
import torch

def warmup_cosine(step, total, warmup):
    if step < warmup:
        return (step + 1) / warmup                      # linear warmup
    p = (step - warmup) / max(1, total - warmup)
    return 0.5 * (1 + math.cos(math.pi * p))            # cosine decay to 0

model = torch.nn.Linear(10, 3)
decay = [p for n, p in model.named_parameters() if p.ndim > 1]      # weights
no_decay = [p for n, p in model.named_parameters() if p.ndim <= 1]  # biases, norm scales
opt = torch.optim.AdamW([{"params": decay, "weight_decay": 0.05},
                         {"params": no_decay, "weight_decay": 0.0}], lr=1e-3)
total = 1000
sched = torch.optim.lr_scheduler.LambdaLR(opt, lambda s: warmup_cosine(s, total, warmup=50))
ema = torch.optim.swa_utils.AveragedModel(model,
 ↪  multi_avg_fn=torch.optim.swa_utils.get_ema_multi_avg_fn(0.999))
for step in range(total):
    loss = model(torch.randn(16, 10)).pow(2).mean()
    opt.zero_grad(); loss.backward(); opt.step(); sched.step(); ema.update_parameters(model)
```

一個小實驗

為了看這些元件一起運作，我們在 scikit-learn 的 8×8 手寫數字上，訓練了一個 7.8 萬參數的 ResNet（「TinyResNet」：一個主幹、三個通道數分別為 16、32、64 的基本區塊、全域平均池化）。影像先放大到 32×32，讓有步幅的階段有空間運作。為了讓過擬合看得見，我們只用 **100 張訓練影像**，另外 500 張用於校準、1,197 張用於測試；使用 AdamW（權重衰減 0.05）、批次大小 32、40 個 epoch，先暖身兩個 epoch 再做餘弦衰減。一組完全不做擴增；另一組使用隨機旋轉（$\pm 15°$）、縮放與平移（類似隨機縮放裁切的仿射變換；不翻轉，因為翻轉後的數字是另一個符號），並搭配標籤平滑 0.1。所有程式碼都在 `scripts/figures/dl_02_image_classification.py`，在 CPU 上大約一分鐘跑完。

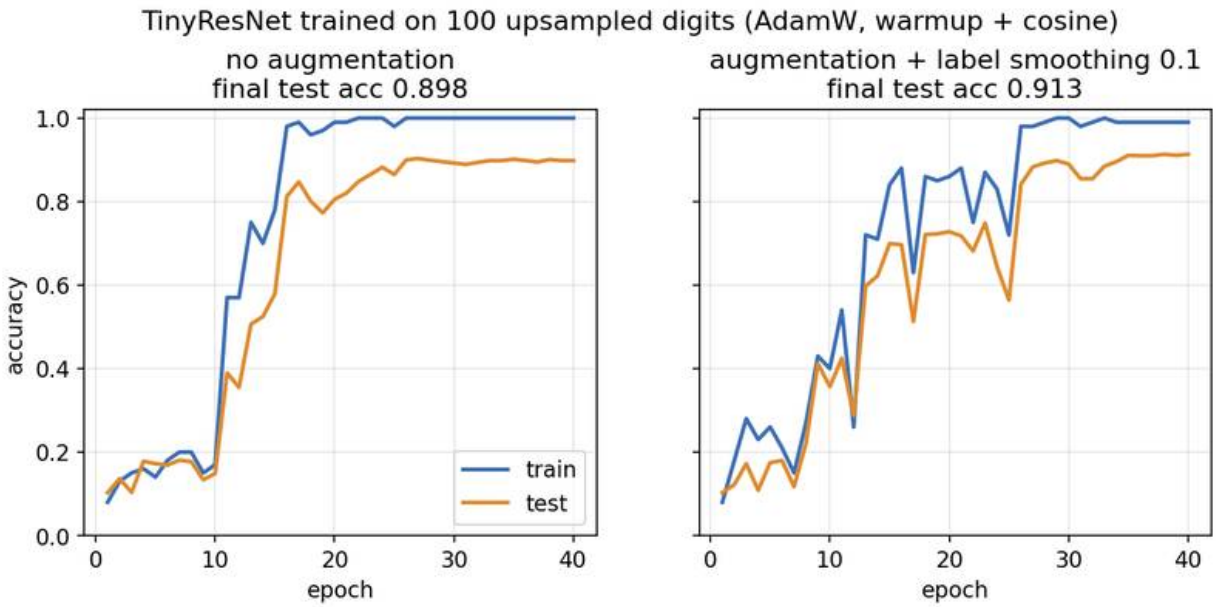


圖 *7* — 以 *100* 張數字影像訓練的學習曲線。沒有擴增（左）時，網路背下了整個訓練集，測試準確率 *89.8%*。加上擴增與標籤平滑（右）後達到 *91.3%*。在這麼簡單的任務上差距不大；在自然影像上，配方的影響大得多。

沒有擴增的模型大約二十個 epoch 後就達到 100% 訓練準確率：只有 100 張影像，背起來很容易。有擴增的模型每個 epoch 看到的都是稍微不同的數字，學得比較慢，泛化稍好（91.3% 對 89.8%）。單一隨機種子上 1.5 個百分點不必過度解讀；重點是機制。

「ResNet strikes back」

配方重要性最驚人的證據，來自 Wightman、Touvron 與 Jégou [32]。他們保持 2015 年的 ResNet-50 架構不變，用現代配方重新訓練：隨機縮放裁切與翻轉、RandAugment、Mixup 與 CutMix、LAMB 最佳化器配餘弦排程、二元交叉熵損失，以及更長的訓練。原汁原味的 ResNet-50 在 224×224 的 ImageNet 上，不用額外資料或蒸餾就達到 **80.4% top-1**，而他們比較的標準 torchvision 權重是 76.1%。這帶來兩個結論。第一，許多「新架構打敗 ResNet-50」的宣稱，其實是拿用新配方訓練的新模型，去比用舊配方訓練的舊模型。第二，比較架構時，要用相同且調好的配方訓練它們，就像 ConvNeXt [20] 與 RegNet [19] 的做法。

遷移學習與微調

白話版。從頭訓練一個骨幹網路需要大量資料。比較好的做法是：從一個已經在大型資料集上訓練好的網路出發，再把它調整到你的任務上。你可以凍結它的特徵、只訓練新的最後一層，也可以溫和地重新訓練所有參數。

令 g_ϕ 為一個預訓練骨幹，把影像映射成特徵向量 $\mathbf{h} = g_\phi(\mathbf{x}) \in \mathbb{R}^D$，$\mathbf{W} \in \mathbb{R}^{K \times D}$ 為新的線性分類頭。

- **線性探測**（linear probe）。凍結 ϕ，只訓練 $\mathbf{W}$（等於在固定特徵上做邏輯迴歸）。它便宜、需要的標籤少，也是衡量表徵好壞的標準方法。
- **完整微調**（full fine-tuning）。同時更新 ϕ 與 $\mathbf{W}$，通常骨幹的學習率比分類頭小 10–100 倍，搭配短暫的暖身，有時還會用逐層學習率衰減（越前面的層學習率越小）。它讓特徵適應新的領域，標註資料夠多時通常勝出。
- 介於兩者之間的，有部分微調（只調最後幾個階段），以及在凍結骨幹上加入小型可訓練模組的參數高效方法。

Kornblith 等人 [33] 量測了 ImageNet 準確率能多好地預測遷移表現。對微調而言，ImageNet 上越好的模型通常遷移得越好，相關性很強。對固定特徵而言，相關性較弱，而且取決於正則化選擇：有些提升 ImageNet 準確率的設定（例如標籤平滑與 dropout），反而產生了在其他任務上更差的凍結特徵。好的分類器與好的特徵擷取器相關，但不是同一件事。

```
import torch
import torch.nn as nn

backbone = nn.Sequential(nn.Conv2d(3, 32, 3, 2, 1), nn.ReLU(), nn.Conv2d(32, 64, 3, 2, 1),
 ↪  nn.ReLU(),
                          nn.AdaptiveAvgPool2d(1), nn.Flatten())          # stand-in for a
                           ↪  pre-trained model
head = nn.Linear(64, 5)

# linear probe: freeze the backbone, train only the head
for p in backbone.parameters():
    p.requires_grad = False
probe_opt = torch.optim.AdamW(head.parameters(), lr=1e-3)

# full fine-tuning: unfreeze, use a smaller learning rate for the backbone than for the head
for p in backbone.parameters():
    p.requires_grad = True
ft_opt = torch.optim.AdamW([{"params": backbone.parameters(), "lr": 1e-5},
                            {"params": head.parameters(), "lr": 1e-3}], weight_decay=0.05)
```

自監督學習與基礎模型分類器

白話版。標籤很貴，影像很便宜。自監督學習自己發明一個答案就藏在影像裡的任務（這兩塊裁切是不是來自同一張照片？被遮住的地方原本是什麼？），透過解這個任務學到特徵，最後才用少量標籤。

對比式學習：SimCLR 與 MoCo

對比式方法把同一張影像的兩個擴增**視圖**的嵌入拉近，把不同影像的視圖推遠。SimCLR [34] 取一個 B 張影像的批次，為每張產生兩個隨機視圖，並對每一對正樣本 (i, j) 最小化 InfoNCE（NT-Xent）損失：

$$\ell_{i,j} = -\log \frac{\exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2B} \mathbb{1}_{[k \neq i]} \exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)},$$

其中 $\mathbf{z}$ 是投影後的嵌入，sim 是餘弦相似度，τ 是溫度，批次中其他 $2B-2$ 個視圖當作負樣本。這其實是一個 $2B-1$ 類的 softmax 分類：「這些之中哪一個是我的搭檔？」SimCLR 發現強力擴增（特別是裁切加上顏色扭曲）、非線性投影頭與大批次都不可或缺。MoCo [35] 則維護一個由緩慢更新的**動量編碼器**（其權重是主編碼器的 EMA）編碼的負樣本**佇列**，避免了對超大批次的需求。

DINO 與 DINOv2：自我蒸餾

DINO [36] 不再使用明確的負樣本。學生網路看幾個裁切（包括小的局部裁切），被訓練去匹配看全域裁切的教師網路的輸出分佈。教師是學生的 EMA；對教師輸出做中心化與銳化，可以防止崩塌成常數。用 ViT 時，DINO 特徵展現出驚人的性質：類別 token 的注意力圖不需任何分割標籤就能把物體分割出來，而在凍結特徵上用簡單的 k-NN 分類器效果就很好。DINOv2 [38] 以自動篩選的資料集與一個大型 ViT（再蒸餾成較小的模型）擴大規模，產生通用的**凍結特徵**，不需微調就能用於分類、分割與深度估計。DINOv3 [39] 繼續擴大規模，並提出「Gram anchoring」，避免密集的小塊特徵在長時間訓練中劣化。

MAE：遮罩影像建模

MAE [37] 借用了遮罩語言模型的想法。它遮掉大比例的小塊（論文中為 75%），只把看得見的小塊送進 ViT 編碼器，再讓一個輕量解碼器重建被遮掉的像素。因為編碼器跳過被遮掉的 token，預訓練很便宜。MAE 特徵的線性可分性不如對比式特徵（線性探測較弱），但**微調**效果非常好。這是一再出現的模式：不同的預訓練目標，適合不同的適應方式。

CLIP：用語言做零樣本分類

CLIP [40] 用數億組影像與說明文字，以對稱的對比損失同時訓練影像編碼器 g 與文字編碼器 h，讓配對的影像與文字有高的餘弦相似度。之後**不需要任何訓練影像**就能建立分類器：為每個類別寫一句提示，例如 `a photo of a dog.`，將它嵌入，然後預測

$$\hat{y} = \arg\max_k \frac{g(\mathbf{x})^\top h(t_k)}{\|g(\mathbf{x})\| \, \|h(t_k)\|},$$

其中 t_k 是類別 k 的提示。文字嵌入扮演線性分類器權重的角色，只是它們是從文字算出來的。論文回報零樣本 CLIP 的準確率追平了一個用 ImageNet 128 萬張標註影像訓練的 ResNet-50，而平均多個提示模板（「a photo of a …」、「a blurry photo of a …」）會有幫助。零樣本準確率高度依賴提示的用字、類別名稱在視覺上是否有意義，以及網路資料對該領域的涵蓋程度。

穩健性與評估陷阱

白話版。一個在測試集上拿 90% 的分類器，碰到不同相機拍的照片、有人刻意加上肉眼看不見的雜訊，或它學到的「簡單線索」（背景、浮水印）不見時，可能會錯得一塌糊塗。而且就算答對了，它的信心也可能不對。

分佈偏移：ImageNet-C、-R 與 -A

Hendrycks 等人為 ImageNet 分類器建立了三個廣泛使用的偏移測試集。**ImageNet-C** [42] 套用 15 種演算法產生的損壞（雜訊、模糊、天氣、數位瑕疵），各有五個嚴重程度，並以相對於基準模型的錯誤率總結（mean corruption error）。**ImageNet-R** [43] 收集了 200 個 ImageNet 類別的各種表現形式（藝術、卡通、雕塑、素描、玩具），測試模型理解的是形狀與概念，還是照片的紋理。**ImageNet-A** [44] 收錄未經修改的自然照片，這些照片經過篩選、專門騙倒 ResNet-50；標準模型在上面表現非常差。這三個測試集揭露不同的弱點，改善其中一個不保證改善其他。大規模預訓練（如 CLIP 與 DINOv2）對這類自然偏移的穩健性提升，往往遠大於在相同資料下更換架構。

對抗樣本：FGSM

Goodfellow 等人 [45] 用網路在高維空間中的**線性**來解釋對抗樣本：對每一個輸入維度做一點

點、與權重方向一致的改變，累加起來就是輸出的巨大變化。**快速梯度符號法**（fast gradient sign method, FGSM）一步就完成擾動：

$$\mathbf{x}_{\text{adv}} = \text{clip}\left(\mathbf{x} + \varepsilon\ \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y))\right),$$

其中 ε 限制每個像素的改變量（ℓ_∞ 範數），$\mathcal{L}$ 是損失函數，clip 把像素值限制在有效範圍內。

```
import torch.nn.functional as F

def fgsm(model, x, y, eps):
    x = x.clone().requires_grad_(True)
    F.cross_entropy(model(x), y).backward()
    return (x + eps * x.grad.sign()).clamp(-1, 1).detach()
```

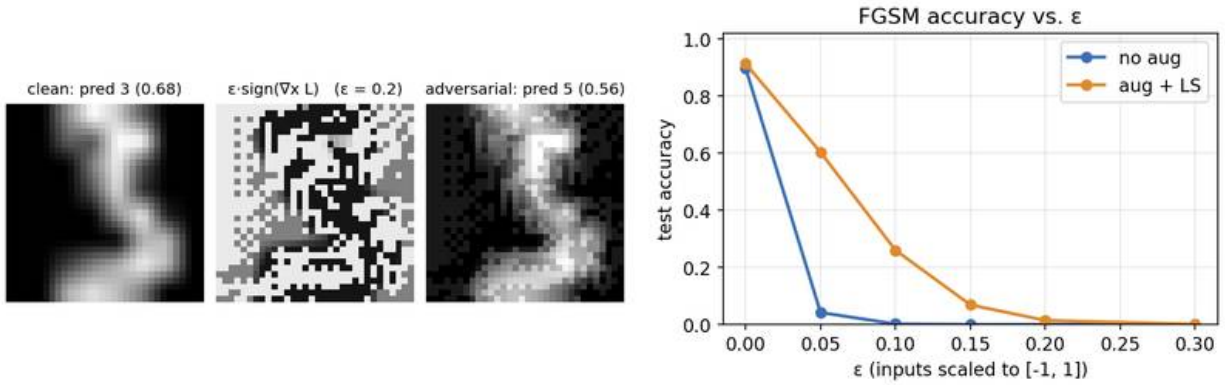


圖 *8* — 在 *TinyResNet* 模型上的 *FGSM*（輸入範圍為 $[-1, 1]$）。$\varepsilon = 0.05$ 時，未擴增模型的測試準確率從 *89.8%* 掉到 *4.2%*，擴增＋標籤平滑模型從 *91.3%* 掉到 *60.3%*；$\varepsilon = 0.2$ 時兩者都接近零。

關於圖 8 有兩點要注意。第一，擴增模型在小 ε 時看起來比較穩健，但已知標籤平滑會讓訓練點附近的損失曲面變平，可能削弱單步梯度攻擊，卻沒有帶來真正的穩健性；正確的檢驗是更強的迭代攻擊。第二，在 $[-1,1]$ 範圍內 $\varepsilon = 0.05$ 對 8×8 的數字來說是看得見的；在自然影像上，肉眼無法察覺的擾動就足以騙過標準的 ImageNet 模型。

捷徑學習

Geirhos 等人 [46] 把許多失敗案例歸在同一個名字下：**捷徑學習**（shortcut learning）。網路會用最容易的線索來最小化訓練損失，而那常常不是我們想要的線索：看背景而不是動物、看醫院的標記而不是病灶、看紋理而不是形狀。捷徑在獨立同分佈的測試集上看起來沒問題，因為測試集裡也有同樣的捷徑；一遇到分佈偏移就會失敗。對策是：刻意在分佈外測試、設計能打破可疑捷徑的測試，並用 Grad-CAM（見下文）這類可解釋性工具檢查模型在看哪裡。

校準與溫度縮放

溫度縮放（temperature scaling）[47] 在 softmax 之前把所有 logits 除以同一個純量 $T > 0$：$\mathbf{p} = \mathrm{softmax}(\mathbf{z}/T)$。$T$ 是在保留的**校準**集（絕不是測試集）上最小化負對數概似而得。$T > 1$ 讓過度自信的預測變柔和；$T < 1$ 讓信心不足的預測變銳利。因為除以 T 不改變 argmax，準確率不變。

```
import torch
import torch.nn.functional as F

def fit_temperature(logits, y, iters=300):
    """Learn one scalar T > 0 on a held-out calibration split."""
    log_t = torch.zeros(1, requires_grad=True)
    opt = torch.optim.Adam([log_t], lr=0.05)
    for _ in range(iters):
        loss = F.cross_entropy(logits / log_t.exp(), y)
```

```
        opt.zero_grad(); loss.backward(); opt.step()
    return log_t.exp().item()

def ece(probs, y, n_bins=10):
    conf, pred = probs.max(1)
    correct = (pred == y).float()
    edges = torch.linspace(0, 1, n_bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.float().mean() * (correct[m].mean() - conf[m].mean()).abs()
    return float(total)

# toy check: an over-confident "model" = true logits multiplied by 3
torch.manual_seed(0)
true_logits = torch.randn(4000, 10)
y = torch.distributions.Categorical(logits=true_logits).sample()
over = 3 * true_logits
T = fit_temperature(over[:2000], y[:2000])
print(f"T = {T:.2f}  ECE {ece(over[2000:].softmax(1), y[2000:]):.3f} -> "
      f"{ece((over[2000:] / T).softmax(1), y[2000:]):.3f}")
# T = 2.90  ECE 0.345 -> 0.018
```

在這個玩具例子裡，logits 被放大了三倍，擬合出的溫度（2.90）找回了這個倍數。而在我們真正的 TinyResNet（未擴增的那個模型）上，校準誤差的方向相反。

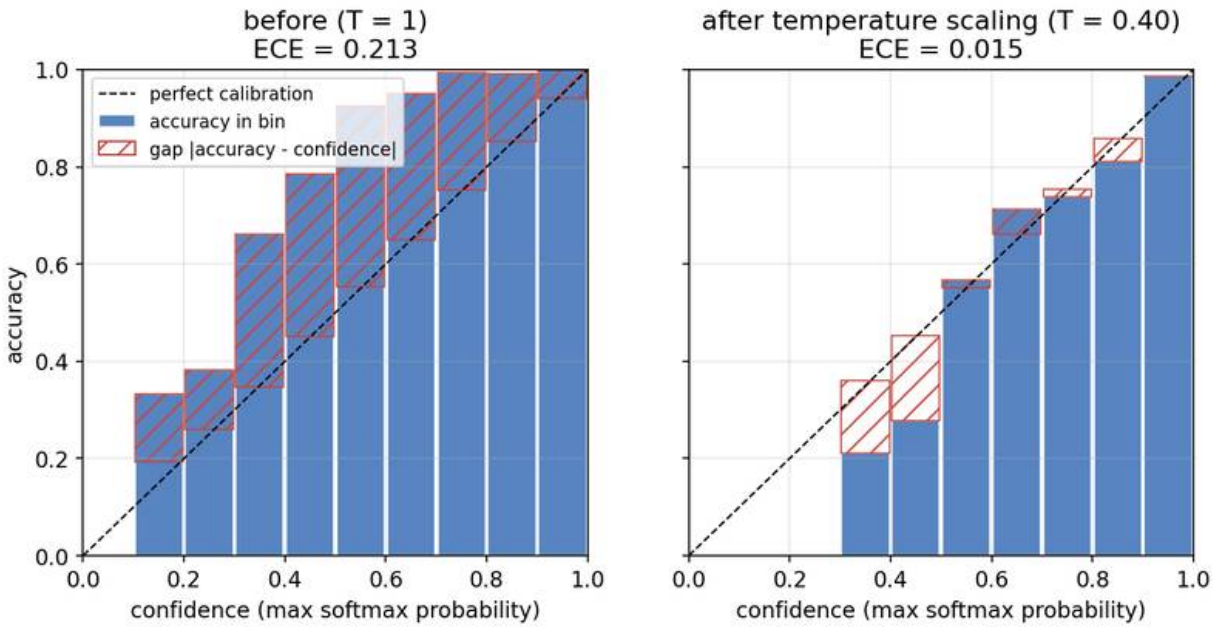


圖 *9* — 未擴增 *TinyResNet* 在 *1,197* 張測試數字上的可靠度圖。縮放前（左）多數區間的準確率高於信心：模型信心不足（*ECE 0.213*）。在另外 *500* 張校準影像上擬合出的溫度 $T = 0.40$，讓 *ECE* 降到 *0.015*，測試 *NLL* 從 *0.515* 降到 *0.298*，而且沒有改變任何一個預測。

為什麼信心不足？我們的小模型訓練時間短、權重衰減又強，使 logits 保持很小。Guo 等人 [47] 觀察到的是大型、長時間訓練的現代網路過度自信，並指出深度、寬度、批次正規化與較小的權重衰減都是成因。校準誤差沒有固定的方向；要量測它。另外也要注意，校準本身在分佈偏移下很脆弱：在分佈內資料上擬合的溫度，不保證在 ImageNet-C 式的偏移上仍然校準良好。

效率

白話版。模型必須放得進裝置、跑得進時間預算。參數量、計算量與實際速度是三種不同的量測，而使用者感受到的只有最後一種。

- **參數量**決定模型在磁碟與記憶體中的大小。一個 float32 參數占 4 個位元組。
- **FLOPs**（或乘加運算 MACs；一次 MAC 通常算作兩個 FLOPs）衡量每張影像的運算量。對核大小 $K \times K$、輸入 C_{in} 通道、輸出 C_{out} 通道、輸出特徵圖為 $H' \times W'$ 的卷積，MACs $= H'W'K^2 C_{\text{in}} C_{\text{out}}$。計算量隨解析度的平方成長。
- **延遲與吞吐量**要在目標硬體上量測。深度卷積的 FLOPs 很少，但運算密度低，所以在 GPU 上常比 FLOPs 所暗示的還慢；DenseNet 的串接則耗費記憶體頻寬。一定要實測。
- **量化**（quantization）把權重與激活值存成 8 位元整數，而不是 32 位元浮點數。Jacob 等人 [50] 描述了只用整數運算的推論，搭配量化感知訓練：在訓練時模擬捨入，讓網路學會容忍它。用少量校準資料做訓練後量化，則是較便宜的替代方案。
- **蒸餾** [51] 訓練一個小的學生網路去匹配大型教師網路的軟預測，就像 DeiT 用蒸餾 token 所做的那樣。

可解釋性：CAM 與 Grad-CAM

白話版。疊在影像上的熱圖，顯示哪些區域把某個類別的分數推高了。它是健全性檢查，不是推理過程的證明。

類別激活映射（class activation mapping, CAM）[48] 適用於以全域平均池化加一層線性層結尾的網路。若 A^k 是最後一個卷積層的第 k 個特徵圖，w_k^c 是它連到類別 c 的權重，則類別分數為 $\sum_k w_k^c \cdot \text{mean}(A^k)$，而 $\sum_k w_k^c A^k$ 這張圖顯示分數來自哪裡。**Grad-CAM** [49] 用空間平均的梯度取代這些權重，把方法推廣到任何可微分的架構：

$$\alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L_{\text{Grad-CAM}}^c = \text{ReLU}\Big(\sum_k \alpha_k^c A^k \Big),$$

其中 y^c 是類別 c 的 logit，A_{ij}^k 是通道 k 在位置 (i, j) 的激活值，Z 是位置總數。ReLU 只保留對該類別有正面影響的區域。粗糙的熱圖（我們的網路中是 8×8）再上採樣到影像大小。

```
import torch.nn.functional as F

def grad_cam(model, x, target=None):
    model.eval()
    feats = model.features(x)                          # (1, K, h, w) last conv stage
    feats.retain_grad()
```

```
    logits = model.fc(feats.mean(dim=(2, 3)))
    c = logits.argmax(1).item() if target is None else target
    logits[0, c].backward()
    alpha = feats.grad.mean(dim=(2, 3), keepdim=True)  # channel importance weights
    cam = F.relu((alpha * feats).sum(1, keepdim=True))
    cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)
    return (cam / (cam.max() + 1e-8))[0, 0].detach(), c
```

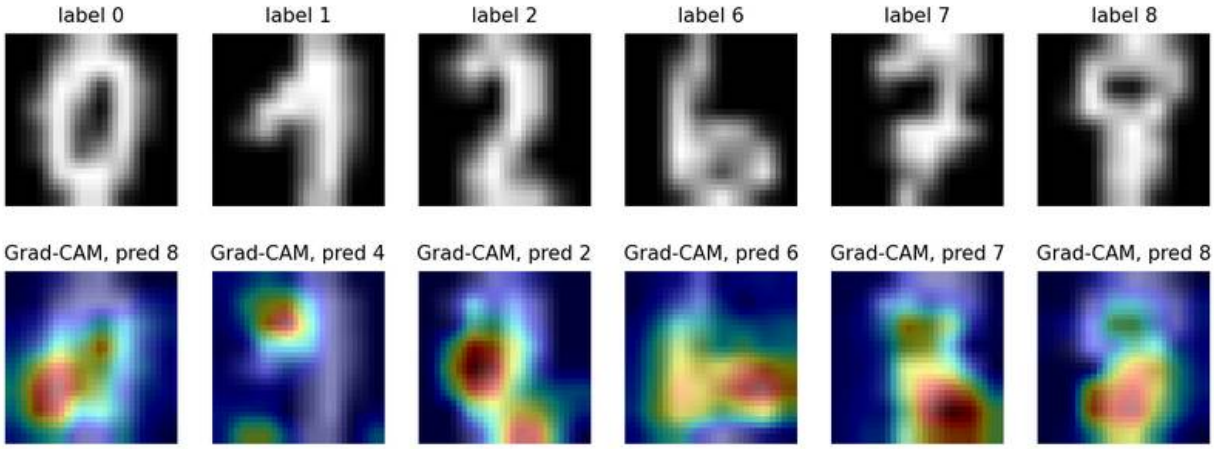


圖 *10* — 以擴增版 *TinyResNet*、對六個數字各自的第一個測試樣本，畫出預測類別的 *Grad-CAM*。其中兩個分錯了：*0* 被判成 *8*，熱區落在圓圈的左下部分；*1* 被判成 *4*，熱區落在它左上方的勾上。熱圖指出了誤導性證據被找到的位置。

解釋方法在模型出錯時最有用，就像圖 10：它們顯示了哪些筆畫看起來像錯誤的類別。但對顯著圖要謹慎。它們很粗糙（受限於最後一層特徵圖的解析度），而且只解釋單一輸入的單一類別分數，不是模型的一般推理方式。

現代觀點

值得一讀的綜述。 Li 等人 [1] 從 CNN 的歷史與基本構成元件談起，介紹經典與進階模型並點出各自的關鍵想法，再以實驗比較激活函數、損失函數與最佳化器，整理出選擇時的經驗法則；最後討論一維、二維與多維卷積的應用，以及一系列開放問題。Khan 等人 [25] 綜述各種任務上的視覺 Transformer。在分類方面，他們把模型分成：單一尺度模型（ViT、DeiT）、多尺度階層模型（如 Swin）、結合卷積的混合模型，以及自監督 Transformer（如 DINO），並強調它們對大規模預訓練的依賴，以及平方複雜度注意力的成本。Gui 等人 [41] 綜述自監督學習，把它分成：以脈絡為基礎的方法（旋轉預測、拼圖等前置任務）、對比式方法（使用負樣本對、像 DINO 的自我蒸餾，或特徵去相關）、生成式方法（像 MAE 的遮罩影像建模），以及結合對比與生成目標的混合方法，並討論與半監督、多模態學習的結合。在基準可靠性方面，ImageNetV2 研究 [7]、Beyer 等人的重新標註研究 [8] 與 Northcutt 等人的標籤錯誤分析 [9] 共同主張：使用多個測試集、多標籤評估，並回報變異。

2023–2026 年的樣貌。五個趨勢定義了當前的實務。

1. **預訓練一次，便宜地適應**。大多數新的分類器都不是從頭訓練。它們從一個以自監督（DINOv2 [38]、DINOv3 [39]）、影像–文字對比（CLIP [40] 及其開源重現）或遮罩建模（MAE [37]）訓練的大型骨幹出發，再以線性探測、k-NN 規則或短暫微調來適應。
2. **零樣本與開放詞彙分類**。影像–文字模型把標籤集合變成文字。類別清單可以在測試時更換、不需重新訓練，模糊了分類與檢索之間的界線。

3. **視覺的縮放定律**。 Zhai 等人 [52] 把 ViT 擴大到數十億參數，刻畫了錯誤率如何隨計算量、資料量與模型大小下降，並觀察到較大的模型在少樣本遷移中更有樣本效率。Cherti 等人 [53] 在公開資料上為開源 CLIP 訓練擬合出可重現的冪律，發現縮放行為取決於訓練資料分佈與下游任務。
4. **架構趨於收斂**。 ConvNeXt [20] 與 Swin [24] 顯示，在相同配方與規模下，卷積與注意力會落在相近的位置。現在的選擇更多取決於硬體、解析度與下游任務（密集預測偏好階層式設計），而不是 top-1 準確率。
5. **評估不再只看 ImageNet-1k top-1**。穩健性測試組（ImageNet-C/R/A [42]–[44] 與 ImageNetV2 [7]）、校準，以及遷移到大量下游資料集，現在都是骨幹論文的標準內容。

開放問題。對自然分佈偏移的穩健性仍落後於乾淨資料上的準確率，而穩健訓練方法帶來的進步往往只針對它所瞄準的那種偏移。沒有反事實測試，捷徑學習 [46] 很難被發現。分佈偏移下的校準仍未解決。長尾與細粒度辨識依然困難，因為稀有類別的樣本最少。基準飽和與標籤雜訊，讓 ImageNet-1k 上的微小準確率差異失去意義。而主流的基礎模型是用網路規模的資料訓練的，其組成、授權與偏差都很難稽核，這直接影響「零樣本」分類器能認出什麼、認不出什麼。

重點整理

- 分類器輸出 logits；單標籤任務用 softmax ＋交叉熵訓練，多標籤任務用獨立 sigmoid ＋二元交叉熵訓練。回報 top-1 準確率，必要時也回報校準（ECE）與逐類別結果。
- 每個里程碑 CNN 都修正了一個瓶頸：ReLU ／ GPU（AlexNet）、疊加的小卷積核（VGG）、1×1 降維與多分支模組（Inception）、為最佳化而設的恆等捷徑（ResNet）、為成本而設的深度可分離卷積（MobileNet）、平衡縮放（EfficientNet）。
- ViT 用卷積的歸納偏差換取彈性；它需要更多資料或更強的配方（DeiT），而視窗加階層（Swin）讓它成為實用的骨幹。ConvNeXt 顯示配方解釋了大部分的差距。
- 訓練配方和架構一樣重要：資料擴增（隨機縮放裁切、Mixup、CutMix、RandAugment）、標籤平滑、AdamW、暖身＋餘弦、隨機深度與 EMA，讓架構不變的 ResNet-50 達到 80.4% top-1。
- 預訓練骨幹是預設的起點。線性探測衡量表徵；微調讓表徵適應任務。CLIP 把類別名稱變成分類器權重，實現零樣本辨識。
- 準確率會掩蓋模型犯的是哪一種錯。請看混淆矩陣；類別不平衡時回報精確率／召回率、平衡準確率或 MCC，並優先使用精確率–召回率曲線與 AP，而不是 ROC-AUC。說清楚平均是巨觀（每個類別同等）還是微觀（每張影像同等；在單標籤任務中等於準確率）。
- 把訓練資料與評估資料分開：用一整組測試集評估骨幹網路（ImageNet 驗證集與 ReaL、ImageNetV2、ImageNet-C/R/A/Sketch、ObjectNet、細粒度與遷移測試組），只在驗證資料上調整，並回報多個隨機種子的平均 $\pm$ 標準差。
- 基準準確率很脆弱：在偏移資料上測試、小心捷徑與對抗輸入、在保留資料上擬合溫度，並檢查 Grad-CAM 熱圖，特別是在出錯的樣本上。

練習

1. **感受野與參數量**。證明三層疊加的 3×3 卷積（步幅 1）感受野為 7×7。在輸入與輸出都是 C 通道、無偏差項的情況下，與單一 7×7 卷積比較參數量。

   提示

   每層 3×3 讓感受野增加 2 個像素：$3\to 5\to 7$。參數量：$3\cdot 9C^2 = 27C^2$ 對 $49C^2$，少了約 45%，還多了兩個非線性。

2. **瓶頸區塊的算術**。一個 ResNet 瓶頸區塊輸入有 $4d$ 個通道，降到 d。以 d 表示 1×1–3×3–1×1 分支的權重數（無偏差），並與寬度 $4d$、兩層 3×3 的基本區塊比較。比值是多少？

   提示

   瓶頸：$4d\cdot d + 9d^2 + d\cdot 4d = 17d^2$。寬度 $4d$ 的基本區塊：$2\cdot 9\cdot 16d^2 = 288d^2$。比值為 $17/288\approx 0.059$，權重少了約 17 倍。

3. **標籤平滑的最佳解**。對一個有 K 個類別、平滑強度 ε 的樣本，假設所有錯誤類別的 logit 相等。證明平滑目標下的交叉熵在「正確類別與每個錯誤類別的 logit 差」為 $\log\left((1-\varepsilon+\varepsilon/K)\,/\,(\varepsilon/K)\right)$ 時最小。代入 $K=1000$、$\varepsilon=0.1$ 計算。

   提示

   交叉熵對 $\mathbf{p}$ 的最小值發生在 $\mathbf{p}=\mathbf{q}$。所以 $p_y = 1-\varepsilon+\varepsilon/K$，其他每個 $p_k=\varepsilon/K$，logit 差就是兩者比值的對數：$\log(0.9001/0.0001)\approx 9.1$。沒有平滑時，最佳間距是無限大。

4. **ViT 的 token 數**。一個使用 16×16 小塊的 ViT 收到 224×224 的影像。有多少 token 進入編碼器？若以 384×384 微調，token 數是多少？注意力矩陣大小成長幾倍？位置嵌入又該怎麼處理？

   提示

   $(224/16)^2 = 196$ 個小塊加一個類別 token：197。在 384 時：$24^2+1 = 577$。注意力矩陣成長 $(577/197)^2\approx 8.6$ 倍。學到的位置嵌入是 14×14 的網格，要以二維內插放大到 24×24；ViT 論文在較高解析度微調時正是這麼做的。

5. **溫度不改變準確率**。證明對任何 $T>0$，$\arg\max_k \mathrm{softmax}(\mathbf{z}/T)_k = \arg\max_k z_k$。接著用 `scripts/figures/dl_02_image_classification.py`，改在測試集上而不是校準集上擬合溫度，比較回報的 ECE。為什麼那個數字過於樂觀？

   提示

   除以正的常數與指數函數都是嚴格遞增的，而 softmax 的分母是共用的。在測試集上擬合 T 等於用測試標籤調整參數，因此回報的 ECE 不再是對新資料表現的誠實估計。

6. **製造一條捷徑。**修改數字實驗，讓訓練集中每個「3」的左上角都有一個亮點，但測試集沒有。訓練 TinyResNet，回報測試集上 3 的準確率，並看幾個訓練用 3 的 Grad-CAM 熱圖。你預期會看到什麼？如果你事先不知道有這條捷徑，要怎麼發現這個問題？

   提示

   網路光看角落的亮點就能分類訓練集中的 3，所以測試集上 3 的準確率會下降、訓練準確率卻仍完美，而 Grad-CAM 往往會亮在角落。不知道捷徑時，可以找熱區落在背景上的熱圖、在獨立收集的資料上評估，並做反事實測試（抹除或移動可疑區域，看預測是否改變）。

7. **巨觀與微觀 F1。**一個三類別模型在 100 張影像上測試。列為真實類別、行為預測：類別 A（90 張）→ 88 A、1 B、1 C；類別 B（5 張）→ 3 A、2 B、0 C；類別 C（5 張）→ 4 A、0 B、1 C。計算逐類別的精確率、召回率與 F_1，再算巨觀 F_1 與微觀 F_1。如果報告的重點是能否偵測稀有的 B 與 C，你會放哪一個數字？為什麼？

   提示

   各行總和為 95、3、2。精確率：$88/95 = 0.926$、$2/3 = 0.667$、$1/2 = 0.500$。召回率：$88/90 = 0.978$、$2/5 = 0.400$、$1/5 = 0.200$。用 $F_1 = 2\mathrm{TP}/(2\mathrm{TP} + \mathrm{FP} + \mathrm{FN})$：$176/185 = 0.951$、$4/8 = 0.500$、$2/7 = 0.286$。巨觀 $F_1 = (0.951 + 0.500 + 0.286)/3 = 0.579$。微觀 F_1 等於準確率 $91/100 = 0.910$。微觀被類別 A 主導；巨觀則揭露 B 與 C 大多被漏掉，所以對稀有類別而言，巨觀（加上逐類別召回率）才是誠實的數字。可以用 `precision_recall_fscore_support(..., average="macro")` 驗算。

參考文獻

1. Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 12, pp. 6999–7019, 2022. doi:10.1109/TNNLS.2021.3084827
2. O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, et al., "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, vol. 115, pp. 211–252, 2015. doi:10.1007/s11263-015-0816-y
3. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi:10.1109/5.726791
4. A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," technical report, University of Toronto, 2009. dataset page
5. T. Ridnik, E. Ben-Baruch, A. Noy, and L. Zelnik-Manor, "ImageNet-21K Pretraining for the Masses," *NeurIPS*, 2021. arXiv
6. G. Van Horn, O. Mac Aodha, Y. Song, Y. Cui, C. Sun, A. Shepard, et al., "The iNaturalist Species Classification and Detection Dataset," *CVPR*, 2018. arXiv
7. B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do ImageNet Classifiers Generalize to ImageNet?," *ICML*, PMLR vol. 97, 2019. paper
8. L. Beyer, O. J. Hénaff, A. Kolesnikov, X. Zhai, and A. van den Oord, "Are We Done with ImageNet?," arXiv:2006.07159, 2020. arXiv
9. C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," *NeurIPS*, 2021. arXiv

10. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems 25* (*NIPS*), 2012. paper
11. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *ICLR*, 2015. arXiv
12. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," *CVPR*, 2015. doi:10.1109/CVPR.2015.7298594
13. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *CVPR*, 2016. doi:10.1109/CVPR.2016.308
14. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. doi:10.1109/CVPR.2016.90
15. K. He, X. Zhang, S. Ren, and J. Sun, "Identity Mappings in Deep Residual Networks," *ECCV*, 2016. arXiv
16. G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," *CVPR*, 2017. arXiv
17. A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017. arXiv
18. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019. arXiv
19. I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, "Designing Network Design Spaces," *CVPR*, 2020. arXiv
20. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *CVPR*, 2022. arXiv
21. J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-Excitation Networks," arXiv:1709.01507, 2017 (journal version in *IEEE TPAMI*). arXiv
22. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
23. H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training Data-Efficient Image Transformers & Distillation through Attention," *ICML*, PMLR vol. 139, 2021. paper
24. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00986
25. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in Vision: A Survey," *ACM Computing Surveys*, 2022. arXiv
26. H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," *ICLR*, 2018. arXiv
27. S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features," *ICCV*, 2019. arXiv
28. E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical Automated Data Augmentation with a Reduced Search Space," *CVPR Workshops*, 2020. doi:10.1109/CVPRW50498.2020.00359
29. I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019. arXiv

30. I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” *ICLR*, 2017. arXiv
31. G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, “Deep Networks with Stochastic Depth,” *ECCV*, 2016. doi:10.1007/978-3-319-46493-0_39
32. R. Wightman, H. Touvron, and H. Jégou, “ResNet Strikes Back: An Improved Training Procedure in timm,” arXiv:2110.00476, 2021. arXiv
33. S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?,” *CVPR*, 2019. arXiv
34. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” *ICML*, 2020. arXiv
35. K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum Contrast for Unsupervised Visual Representation Learning,” *CVPR*, 2020. arXiv
36. M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging Properties in Self-Supervised Vision Transformers,” *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00951
37. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked Autoencoders Are Scalable Vision Learners,” *CVPR*, 2022. doi:10.1109/CVPR52688.2022.01553
38. M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. arXiv
39. O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, et al., “DINOv3,” arXiv:2508.10104, 2025. arXiv
40. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, et al., “Learning Transferable Visual Models From Natural Language Supervision,” *ICML*, PMLR vol. 139, 2021. paper
41. J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao, “A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends,” *IEEE Transactions on Pattern Analysis and Machine Intelligence* (arXiv:2301.05712, 2023). arXiv
42. D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” *ICLR*, 2019. arXiv
43. D. Hendrycks, S. Basart, N. Mu, S. Kadavath, F. Wang, E. Dorundo, et al., “The Many Faces of Robustness: A Critical Analysis of Out-of-Distribution Generalization,” *ICCV*, 2021. arXiv
44. D. Hendrycks, K. Zhao, S. Basart, J. Steinhardt, and D. Song, “Natural Adversarial Examples,” *CVPR*, 2021. arXiv
45. I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and Harnessing Adversarial Examples,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
46. R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, “Shortcut Learning in Deep Neural Networks,” *Nature Machine Intelligence*, 2020. arXiv
47. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” *ICML*, 2017. arXiv
48. B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, “Learning Deep Features for Discriminative Localization,” *CVPR*, 2016. doi:10.1109/CVPR.2016.319
49. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” *ICCV*, 2017. doi:10.1109/ICCV.2017.74

50. B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” *CVPR*, 2018. doi:10.1109/CVPR.2018.00286
51. G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” arXiv: 1503.02531, 2015. arXiv
52. X. Zhai, A. Kolesnikov, N. Houlsby, and L. Beyer, “Scaling Vision Transformers,” *CVPR*, 2022. arXiv
53. M. Cherti, R. Beaumont, R. Wightman, M. Wortsman, G. Ilharco, C. Gordon, et al., “Reproducible Scaling Laws for Contrastive Language-Image Learning,” *CVPR*, 2023. arXiv
54. T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. doi:10.1016/j.patrec.2005.10.010
55. J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” *Proc. 23rd International Conference on Machine Learning* (*ICML*), pp. 233–240, 2006. doi:10.1145/1143844.1143874
56. T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. doi:10.1371/journal.pone.0118432
57. D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, article 6, 2020. doi:10.1186/s12864-019-6413-7
58. M. Sokolova and G. Lapalme, “A Systematic Analysis of Performance Measures for Classification Tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. doi:10.1016/j.ipm.2009.03.002
59. J. Cohen, “A Coefficient of Agreement for Nominal Scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960. doi:10.1177/001316446002000104
60. M.-L. Zhang and Z.-H. Zhou, “A Review on Multi-Label Learning Algorithms,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1819–1837, 2014. doi:10.1109/TKDE.2013.39
61. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010. doi:10.1007/s11263-009-0275-4
62. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. paper
63. H. Wang, S. Ge, E. P. Xing, and Z. C. Lipton, “Learning Robust Global Representations by Penalizing Local Predictive Power,” *NeurIPS*, 2019 (introduces ImageNet-Sketch). arXiv
64. A. Barbu, D. Mayo, J. Alverio, W. Luo, C. Wang, D. Gutfreund, J. Tenenbaum, and B. Katz, “ObjectNet: A Large-Scale Bias-Controlled Dataset for Pushing the Limits of Object Recognition Models,” *NeurIPS*, 2019. dataset page
65. C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, “The Caltech-UCSD Birds-200-2011 Dataset,” Technical Report CNS-TR-2011-001, California Institute of Technology, 2011. dataset page
66. J. Krause, M. Stark, J. Deng, and L. Fei-Fei, “3D Object Representations for Fine-Grained Categorization,” *IEEE International Conference on Computer Vision Workshops* (*ICCVW*), pp. 554–561,

2013 (introduces Stanford Cars). doi:10.1109/ICCVW.2013.77
67. S. Maji, E. Rahtu, J. Kannala, M. Blaschko, and A. Vedaldi, "Fine-Grained Visual Classification of Aircraft," arXiv:1306.5151, 2013. arXiv
68. O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and Dogs," *CVPR*, 2012 (introduces Oxford-IIIT Pets). dataset page
69. M.-E. Nilsback and A. Zisserman, "Automated Flower Classification over a Large Number of Classes," *Indian Conference on Computer Vision, Graphics and Image Processing* (*ICVGIP*), 2008 (introduces Oxford Flowers-102). dataset page
70. L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101 – Mining Discriminative Components with Random Forests," *ECCV*, 2014. dataset page
71. L. Fei-Fei, R. Fergus, and P. Perona, "One-Shot Learning of Object Categories," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2006 (the reference given for Caltech-101). dataset page
72. X. Zhai, J. Puigcerver, A. Kolesnikov, et al., "A Large-scale Study of Representation Learning with the Visual Task Adaptation Benchmark," arXiv:1910.04867, 2019. arXiv
73. Z. Liu, Z. Miao, X. Zhan, J. Wang, B. Gong, and S. X. Yu, "Large-Scale Long-Tailed Recognition in an Open World," *CVPR*, 2019 (introduces ImageNet-LT). arXiv

DLCV 03・物件偵測：從 R-CNN 到 DETR

深度網路如何找出並標記影像中的每一個物件：邊界框迴歸、IoU、錨框、NMS 與特徵金字塔；二階段、一階段、無錨框與 Transformer 偵測器；開放詞彙模型；以及偵測該如何評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-03-object-detection/

你將學到

- 偵測器輸出什麼（邊界框、類別、信心分數），以及偵測和分類、語意分割、實例分割的差別。
- 所有現代偵測器共用的基本元件，包含數學與程式碼：邊界框參數化與迴歸目標、IoU 及 GIoU/DIoU/CIoU 等變體、錨框與標籤指派、非極大值抑制、特徵金字塔，以及焦點損失。
- 四大家族如何運作、又為何被發明：二階段（R-CNN 到 Cascade R-CNN）、一階段（YOLO、SSD、RetinaNet）、無錨框（CornerNet、CenterNet、FCOS），以及用 Transformer 做集合預測（DETR 到 RT-DETR）。
- 開放詞彙與定位（grounding）偵測器（OWL-ViT、Grounding DINO、YOLO-World）以及可提示的基礎模型，如何改變「類別清單」的意義。
- 偵測如何評估（精確率–召回率、AP、VOC 與 COCO 協定、AP50/AP75/APs/m/l）、哪些資料集重要，以及部署時速度受什麼限制。

先看全貌

分類器只回答一個關於整張影像的問題：**這是什麼？**偵測器要回答更難的問題：**什麼東西在哪裡？**它必須找出每一個感興趣的物件，在每個物件外面畫一個緊貼的矩形（**邊界框**，bounding box），寫上名稱，並說明自己有多確定。物件的數量事先不知道：可能是零個、一個，也可能是三百個。

> **用最簡單的話說**
>
> 想像一頁「威利在哪裡？」。**分類器**只會告訴你：「這一頁裡有威利。」**偵測器**則會在威利身上貼一張貼紙，再在每一隻狗、每一顆氣球上各貼一張，並在每張貼紙上寫「這是什麼、有多確定」（「狗，蠻確定的」）。難的地方跟你自己找的時候一樣：遠處的小東西、躲在別的東西後面的東西，還有不要在同一隻狗身上貼兩張貼紙。

偵測是多數「真實世界」視覺系統的入口。自駕車偵測行人與車輛；手機在對焦之前先偵測人臉；工廠偵測瑕疵；而追蹤器（見文末連結的追蹤章節）大多是在**把偵測結果隨時間串起來**。這個領域也產生了許多如今遍布深度學習的想法：多任務損失函數、特徵金字塔、焦點損失，以及以二元匹配做集合預測。

本篇假設你已經熟悉 CNN 與影像分類（DLCV 02），以及IP 12的傳統特徵概念。有兩篇綜述可以框住整個故事：Zou 等人的《Object Detection in 20 Years》[1]，以及 Liu 等人發表於 IJCV 的深度通用物件偵測綜述 [2]。我們會在「現代觀點」回到這兩篇。

偵測問題的定義

白話地說：輸入是一張影像，輸出是一小串（邊界框、類別、分數）三元組。訓練資料則是人工畫好、標上類別名稱的邊界框。

精確地說，給定影像 $I \in \mathbb{R}^{H\times W\times 3}$，偵測器輸出一個集合

$$\hat{Y} = \{(\hat{b}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^{N},$$

其中 $\hat{b}_i \in \mathbb{R}^4$ 是一個邊界框，$\hat{c}_i \in \{1,\dots,K\}$ 是 K 個類別之一，$\hat{s}_i \in [0,1]$ 是信心分數，而 N 隨影像不同而改變。真值（ground truth）是 M 個標註物件構成的集合 $Y = \{(b_j, c_j)\}_{j=1}^{M}$。常見的邊界框格式有兩種：角點 (x_1, y_1, x_2, y_2) 與中心–尺寸 (c_x, c_y, w, h)。邊界框是**與座標軸對齊**的；旋轉框是另一種變體，用於空拍影像與文字偵測。

有三個性質讓偵測比分類困難：

1. **輸出大小不固定**。網路必須決定要輸出**幾個**東西。本篇的每一種架構，本質上都是對這個問題的不同回答。
2. **同時定位與辨識**。模型對物件外觀的變化要具有不變性，對物件的**位置**卻要很敏感；這股張力形塑了每一種偵測頭（head）的設計。
3. **極度不平衡**。一張典型影像只有少數幾個物件，卻有上萬個候選位置與尺寸，幾乎全是背景。

偵測與相鄰任務的比較

圖 1 在同一個合成場景上比較四種經典辨識任務。**分類**每張影像給一個標籤。**偵測**每個物件給一個邊界框與一個標籤。**語意分割**為每個像素標上類別，因此兩顆相碰的球會融成一團。**實例分割**為每個物件給出各自的像素遮罩，也就是「偵測＋每個框一張遮罩」；稍後會看到，Mask R-CNN [16] 是取得它的標準方法。

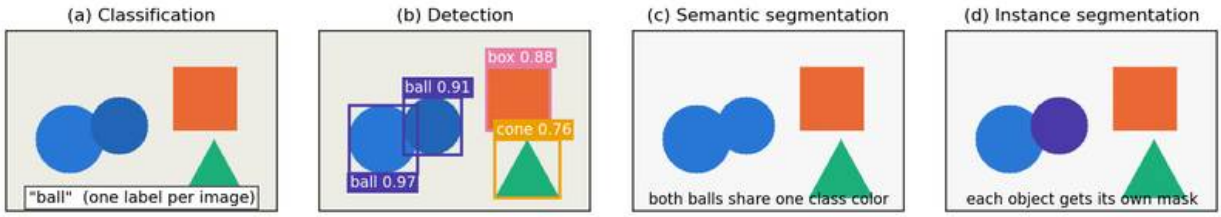


圖 *1* — 同一個場景在四種任務下的樣子：(*a*) 分類、(*b*) 偵測、(*c*) 語意分割、(*d*) 實例分割。

傳統偵測：滑動視窗與手工特徵

白話地說：在深度學習之前，偵測就是「把分類放進迴圈」：在每個位置、每個尺寸切出一個視窗，問分類器裡面有沒有物件，留下回答「有」的視窗。

滑動視窗。在縮放比例為 λ 的影像金字塔上，以步幅 s 滑動固定大小 $w \times h$ 的視窗，視窗數量大約是

$$N_{\text{win}} \approx \sum_{k \ge 0} \frac{(H\lambda^{-k} - h)(W\lambda^{-k} - w)}{s^2},$$

其中 k 是金字塔的層索引。對一張 640×480 的影像、64×128 的視窗、$s = 8$、$\lambda = 1.2$，單一長寬比就有約 10^4 個視窗，每一個都得分類。這就是IP 13的樣板比對（template matching）放大版，也有同樣的弱點：只有一種樣板形狀、要搜尋很多尺度，而且大量運算浪費在背景上。

Viola–Jones（2001）[5] 以三個技巧讓即時人臉偵測成為可能。**Haar-like 特徵**（相鄰矩形區域總和之差）可以用**積分影像**（integral image）$S(x,y) = \sum_{x' \le x, y' \le y} I(x',y')$ 在常數時間內算出，任何矩形總和只需四次查表。**AdaBoost** 從龐大的特徵池中挑出一小組有資訊量的特徵。**注意力串接**（attentional cascade）由一連串越來越複雜的分類器組成，只需少數幾次特徵運算就能拒絕大多數背景視窗。這是對抗不平衡問題的第一個大想法：只在可能有用的地方花運算。

HOG ＋線性 SVM（2005）[6] 用**方向梯度直方圖**（histograms of oriented gradients）取代原始灰階值，它是IP 12中梯度方向描述子的近親。偵測視窗被切成小單元（cell），每個單元以梯度大小加權，統計梯度方向的直方圖；相鄰單元組成彼此重疊的區塊（block）並做對比正規化；串接後的向量由線性 SVM 評分。HOG 對小幅平移與光照變化很穩健，讓行人偵測變得實用。

可變形部件模型（deformable part models, DPM）[7] 加入了結構。一個物件由一個粗略的**根**（root）HOG 濾波器，加上數個解析度較高的**部件**（part）濾波器組成；部件可以相對於根移動，但要付出二次的形變代價：

$$\text{score}(p_0) = F_0 \cdot \phi(p_0) + \sum_{i=1}^{n} \max_{p_i} \Big[F_i \cdot \phi(p_i) - d_i \cdot \psi(p_i - p_0) \Big] + \beta,$$

其中 p_0 是根的位置，p_i 是第 i 個部件的位置，F_i 是學到的濾波器，$\phi(\cdot)$ 是某位置的 HOG 特徵，ψ 是位移特徵 (dx, dy, dx^2, dy^2)，d_i 是學到的形變權重，β 是偏差。部件位置是潛在變數，以**潛在 SVM**（latent SVM）訓練。DPM 及其變體一直稱霸 PASCAL VOC 基準 [3]，直到 2012 年。

這個時代為何結束。傳統偵測器**慢**（要在位置、尺度、長寬比上窮舉掃描，每個視窗都經過手工設計的流程評分），而且**脆弱**（只有一種手工特徵、少數幾個固定樣板，換到新類別、新視角或雜亂場景就表現不佳），也無法在類別之間共享特徵。深度偵測器保留了有用的想法，例如在特徵金字塔中掃描、串接、部件與困難負樣本挖掘（hard-negative mining），並把手工特徵換成學到的特徵。

核心基本元件

接下來的每一種偵測器都由同一組零件組成。我們仔細定義每一個，並附上可執行的程式碼。

邊界框參數化與迴歸目標

白話地說：網路很不擅長直接輸出絕對像素座標，卻很擅長輸出小幅修正。因此我們從一個**參考框**（候選區域、錨框或一個點）出發，預測該怎麼微調它。

給定中心–尺寸形式的參考框 $P = (P_x, P_y, P_w, P_h)$ 與其匹配的真值框 $G = (G_x, G_y, G_w, G_h)$，R-CNN [8] 提出的標準目標為

$$t_x = \frac{G_x - P_x}{P_w}, \qquad t_y = \frac{G_y - P_y}{P_h}, \qquad t_w = \log \frac{G_w}{P_w}, \qquad t_h = \log \frac{G_h}{P_h}.$$

把位移除以參考框尺寸，讓目標具有**尺度不變性**（10 個像素的誤差，對 20 像素的框比對 400 像素的框嚴重得多）。寬高取**對數**，讓縮放變得對稱（放大兩倍與縮小一半分別是 $+\log 2$ 與 $-\log 2$），並保證解碼後的尺寸為正：$\hat{G}_w = P_w \exp(\hat{t}_w)$。實務上，目標常再除以固定的標準差，例如 $(0.1, 0.1, 0.2, 0.2)$，讓四個量的大小相近。

```
import numpy as np

def to_cxcywh(b):
    return np.stack([(b[:, 0] + b[:, 2]) / 2, (b[:, 1] + b[:, 3]) / 2,
                     b[:, 2] - b[:, 0], b[:, 3] - b[:, 1]], axis=1)

def to_xyxy(c):
    return np.stack([c[:, 0] - c[:, 2] / 2, c[:, 1] - c[:, 3] / 2,
                     c[:, 0] + c[:, 2] / 2, c[:, 1] + c[:, 3] / 2], axis=1)

def encode(anchors, gt, std=(0.1, 0.1, 0.2, 0.2)):
    """Regression targets (tx, ty, tw, th) of gt relative to anchors (both xyxy)."""
    P, G = to_cxcywh(anchors), to_cxcywh(gt)
    t = np.stack([(G[:, 0] - P[:, 0]) / P[:, 2],
                  (G[:, 1] - P[:, 1]) / P[:, 3],
                  np.log(G[:, 2] / P[:, 2]),
                  np.log(G[:, 3] / P[:, 3])], axis=1)
    return t / np.array(std)          # normalize so all four have similar scale

def decode(anchors, t, std=(0.1, 0.1, 0.2, 0.2)):
    P = to_cxcywh(anchors)
    t = t * np.array(std)
    c = np.stack([P[:, 0] + t[:, 0] * P[:, 2],
                  P[:, 1] + t[:, 1] * P[:, 3],
                  P[:, 2] * np.exp(t[:, 2]),
                  P[:, 3] * np.exp(t[:, 3])], axis=1)
    return to_xyxy(c)

anchors = np.array([[0, 0, 32, 32], [100, 100, 228, 164]], float)
gt      = np.array([[4, 2, 40, 30], [90, 110, 250, 170]], float)
t = encode(anchors, gt)
print(np.round(t, 3))
print(np.allclose(decode(anchors, t), gt))   # round trip
```

```
[[ 1.875  0.     0.589 -0.668]
 [ 0.469  1.25   1.116 -0.323]]
True
```

這些目標通常以 Fast R-CNN [9] 的 **smooth L1**（Huber）損失函數訓練：

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5\,x^2 & \text{if } |x| < 1, \\ |x| - 0.5 & \text{otherwise,} \end{cases}$$

逐一套用在每個座標差 $x = \hat{t} - t$ 上。誤差小時它像 L2（梯度平滑），誤差大時像 L1（在訓練初期對離群值穩健）。它的弱點是把四個座標當成彼此獨立的數字，而評估指標 IoU 卻把邊界框視為一個整體。這個落差催生了以 IoU 為基礎的損失函數。

IoU 及其變體

白話地說：IoU 問的是，在兩個框各自覆蓋的總面積中，有多少比例是兩者都覆蓋到的？完全吻合時為 1，兩框不相碰時為 0。

對兩個框 A 與 B，

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|},$$

其中 $|\cdot|$ 代表面積。IoU（交並比，intersection over union）具有尺度不變性，是評估的標尺（若一個偵測結果與某個尚未被匹配的真值框的 IoU 超過門檻，就算正確），也用於標籤指派與 NMS。圖 2 顯示這些門檻有多嚴格：把正方形沿兩軸各平移邊長的約 18%，IoU 就掉到 0.5。

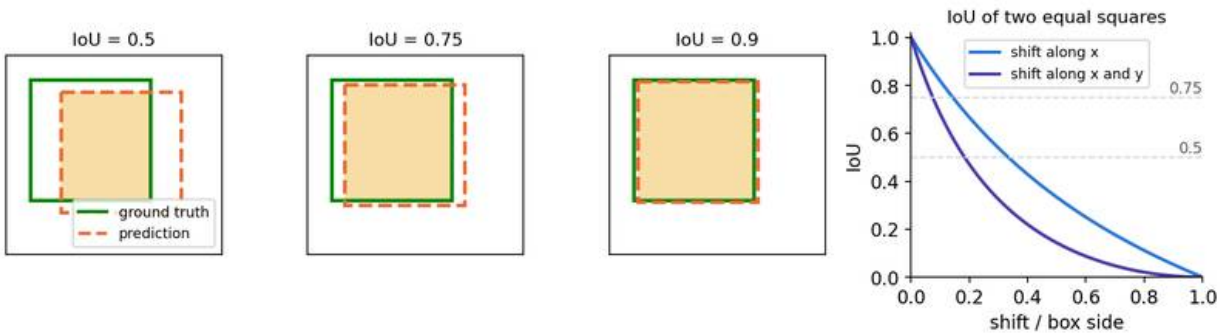


圖 2 — *IoU* 門檻長什麼樣子。左：*IoU* 為 *0.5*、*0.75*、*0.9* 的預測框與真值框。右：兩個相同正方形之一沿 x 軸、或沿 x 與 y 軸平移時的 *IoU*。

拿 1 − IoU 當損失函數有個缺陷：兩框不重疊時，不論相距多遠 IoU 都是 0，梯度也就是 0。常用的修正有三種：

- **GIoU** [11] 加入一個以同時包住兩框的最小框 C 為基礎的懲罰項：$\text{GIoU} = \text{IoU} - \dfrac{|C \setminus (A \cup B)|}{|C|}$，值域為 $(-1, 1]$，兩個不重疊的框離得越遠，它就繼續下降。

- **DIoU** [12] 直接懲罰中心點距離：$\mathrm{DIoU} = \mathrm{IoU} - \dfrac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2}$，其中 ρ 是兩框中心 $\mathbf{b}$ 與 $\mathbf{b}^{gt}$ 的歐氏距離，c 是 C 的對角線長度。它比 GIoU 收斂得更快，因為即使一個框包住另一個框（此時 GIoU 的懲罰項消失），它仍會把中心點拉近。
- **CIoU** [12] 再加入長寬比項：$\mathrm{CIoU} = \mathrm{DIoU} - \alpha v$，其中 $v = \dfrac{4}{\pi^2}\left(\arctan\dfrac{w^{gt}}{h^{gt}} - \arctan\dfrac{w}{h}\right)^2$ 衡量長寬比的差異，$\alpha = \dfrac{v}{(1-\mathrm{IoU})+v}$ 是權衡權重。

對應的損失函數分別是 $1-\mathrm{GIoU}$、$1-\mathrm{DIoU}$ 與 $1-\mathrm{CIoU}$。

```
import numpy as np

def iou_family(a, b, eps=1e-9):
    """a, b: (N, 4) boxes as (x1, y1, x2, y2). Returns IoU, GIoU, DIoU, CIoU."""
    ix1, iy1 = np.maximum(a[:, 0], b[:, 0]), np.maximum(a[:, 1], b[:, 1])
    ix2, iy2 = np.minimum(a[:, 2], b[:, 2]), np.minimum(a[:, 3], b[:, 3])
    inter = np.clip(ix2 - ix1, 0, None) * np.clip(iy2 - iy1, 0, None)
    wa, ha = a[:, 2] - a[:, 0], a[:, 3] - a[:, 1]
    wb, hb = b[:, 2] - b[:, 0], b[:, 3] - b[:, 1]
    union = wa * ha + wb * hb - inter
    iou = inter / (union + eps)
    # smallest enclosing box C
    cx1, cy1 = np.minimum(a[:, 0], b[:, 0]), np.minimum(a[:, 1], b[:, 1])
    cx2, cy2 = np.maximum(a[:, 2], b[:, 2]), np.maximum(a[:, 3], b[:, 3])
    area_c = (cx2 - cx1) * (cy2 - cy1)
    giou = iou - (area_c - union) / (area_c + eps)
    # center distance over enclosing-box diagonal
    rho2 = ((a[:, 0] + a[:, 2]) - (b[:, 0] + b[:, 2])) ** 2 / 4 \
         + ((a[:, 1] + a[:, 3]) - (b[:, 1] + b[:, 3])) ** 2 / 4
    c2 = (cx2 - cx1) ** 2 + (cy2 - cy1) ** 2
    diou = iou - rho2 / (c2 + eps)
    v = 4 / np.pi**2 * (np.arctan(wb / hb) - np.arctan(wa / ha)) ** 2
    alpha = v / (1 - iou + v + eps)
    ciou = diou - alpha * v
    return iou, giou, diou, ciou

gt = np.array([[10, 10, 50, 50]] * 3, float)
pred = np.array([[20, 20, 60, 60],    # overlapping, shifted
                 [60, 10, 100, 50],   # disjoint, nearby
                 [200, 10, 240, 50]], float)  # disjoint, far away
for name, v in zip(["IoU", "GIoU", "DIoU", "CIoU"], iou_family(pred, gt)):
    print(f"{name:5s}", np.round(v, 3))
```

```
IoU   [0.391 0.    0.   ]
GIoU  [ 0.311 -0.111 -0.652]
DIoU  [ 0.351 -0.258 -0.662]
CIoU  [ 0.351 -0.258 -0.662]
```

兩個不重疊的預測框 IoU 都是 0，但 GIoU 與 DIoU 正確地指出比較遠的那個比較差。這裡 CIoU 等於 DIoU，因為所有框都是正方形，沒有長寬比差異。

錨框與標籤指派

白話地說：密集偵測器不自由搜尋，而是在特徵圖的每個格子放一組固定的參考框，也就是**錨框**（anchor）。每個錨框都在問：「這裡有沒有一個大致像我的物件？我該怎麼移動才能貼合它？」

在步幅為 s 的特徵圖上（每個格子涵蓋輸入影像的 $s \times s$ 像素），錨框的中心位於 $((i+\frac{1}{2})s, (j+\frac{1}{2})s)$，並有幾種尺度 σ 與長寬比 $r = h/w$，寬為 $w = \sigma/\sqrt{r}$、高為 $h = \sigma\sqrt{r}$，因此面積維持 σ^2。Faster R-CNN 在每個位置使用 3 種尺度 × 3 種長寬比 = 9 個錨框 [10]（圖 3 左）。

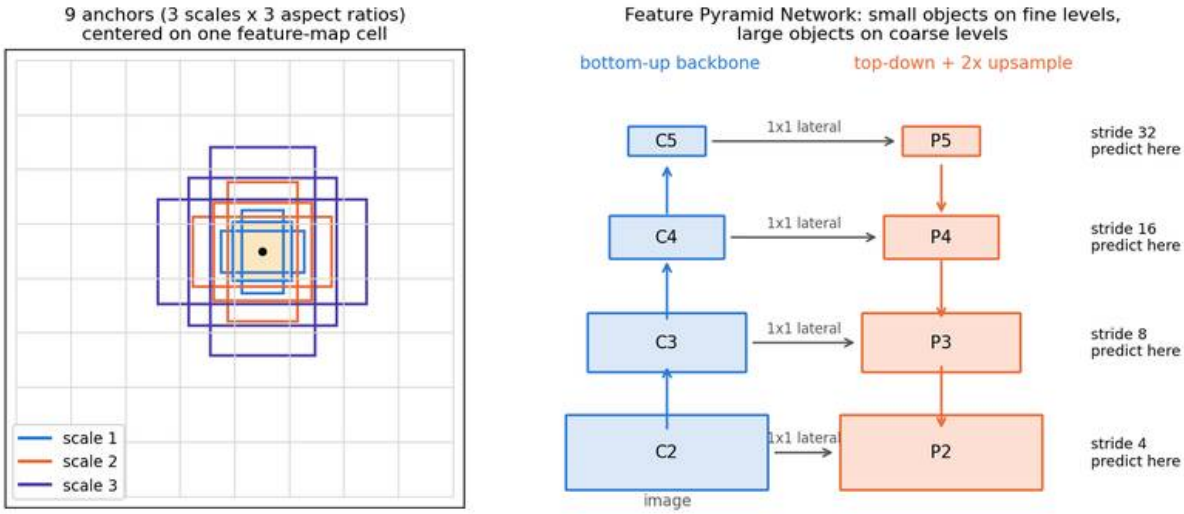


圖 3 — 左：特徵圖某一格上的九個錨框（3 種尺度 × 3 種長寬比）。右：特徵金字塔網路；每一層負責預測對應尺寸範圍的物件。

標籤指派（label assignment）決定哪個錨框要學習哪個物件。經典規則以 IoU 為準：在 Faster R-CNN 的區域提議網路中，若某錨框與某真值框的 IoU 至少 0.7（或它是該真值框最匹配的錨框），就是**正樣本**；若它與所有真值框的 IoU 都低於 0.3，就是**負樣本**；其餘忽略 [10]。RetinaNet 則使用 0.5 與 0.4 [15]。每個正樣本錨框以上述目標朝其匹配的框迴歸。後來發現，指派方式和架構一樣重要：無錨框偵測器（FCOS [29]）改用**位置**與物件尺寸來指派，而 DETR [30] 則以學習出的一對一匹配取代所有固定規則。

錨框也有實際成本：它帶來必須針對每個資料集調整的超參數（尺度、長寬比、門檻），也讓預測數量倍增。YOLO9000 [19] 用 $1 - \text{IoU}$ 當距離，對訓練框做 k-means 分群來決定錨框形狀。

非極大值抑制（NMS）與 Soft-NMS

白話地說：密集偵測器會在同一個物件上反應很多次（相鄰的錨框都看得到它）。**非極大值抑制**（non-maximum suppression, NMS）留下最有信心的框，刪掉與它重疊太多的其他框。

貪婪 NMS（逐類別進行）：依分數排序；反覆取出剩下分數最高的框 M，保留它，並移除所有 $\text{IoU}(M, b_i) > N_t$ 的框 b_i（N_t 通常為 0.5 到 0.7）。它的弱點在擁擠場景：兩個站得很近的真人可能重疊超過 N_t，分數較低的那個就被刪掉，召回率因而下降。

Soft-NMS [13] 把「刪除」改成「降低分數」。以高斯版本為例，

$$s_i \leftarrow s_i \exp\!\left(-\frac{\mathrm{IoU}(M, b_i)^2}{\sigma}\right),$$

其中 s_i 是框 b_i 的分數，σ 控制重疊框被懲罰的速度。重疊很多時分數仍會大幅下降，但真正相鄰的物件會以較低的分數留下來，而不是直接消失。

```
import numpy as np

def iou_one_to_many(box, boxes):
    x1 = np.maximum(box[0], boxes[:, 0]); y1 = np.maximum(box[1], boxes[:, 1])
    x2 = np.minimum(box[2], boxes[:, 2]); y2 = np.minimum(box[3], boxes[:, 3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[..., 2] - b[..., 0]) * (b[..., 3] - b[..., 1])
    return inter / (area(box) + area(boxes) - inter)

def nms(boxes, scores, thr=0.5):
    order = np.argsort(-scores)
    keep = []
    while order.size:
        i = order[0]
        keep.append(i)
        ious = iou_one_to_many(boxes[i], boxes[order[1:]])
        order = order[1:][ious <= thr]          # drop everything that overlaps too much
    return np.array(keep)

def soft_nms(boxes, scores, sigma=0.5, score_thr=0.001):
    scores = scores.copy()
    idx = np.arange(len(scores))
    keep = []
    while idx.size:
        j = np.argmax(scores[idx]); i = idx[j]
        keep.append(i)
        idx = np.delete(idx, j)
        ious = iou_one_to_many(boxes[i], boxes[idx])
        scores[idx] *= np.exp(-ious**2 / sigma)  # decay instead of delete
        idx = idx[scores[idx] > score_thr]
    return np.array(keep), scores

# two people standing close together + duplicates
boxes = np.array([[10, 10, 60, 110], [14, 12, 62, 112], [8, 6, 58, 104],
                  [40, 15, 90, 115], [43, 18, 92, 118]], float)
scores = np.array([0.95, 0.90, 0.80, 0.85, 0.70])
print("greedy NMS keeps:", nms(boxes, scores, 0.5))
keep, s = soft_nms(boxes, scores)
print("Soft-NMS order:", keep, "rescored:", np.round(s[keep], 3))
```

```
greedy NMS keeps: [0 3]
Soft-NMS order: [0 3 2 4 1] rescored: [0.95  0.761 0.183 0.145 0.058]
```

貪婪 NMS 為每個人留下一個框。Soft-NMS 保留所有框，但把重複的框排在兩個真實偵測結果之

後很遠的位置；之後的分數門檻或 AP 計算會把它們當成低信心的猜測。圖 4 在合成場景上呈現同樣的效果。

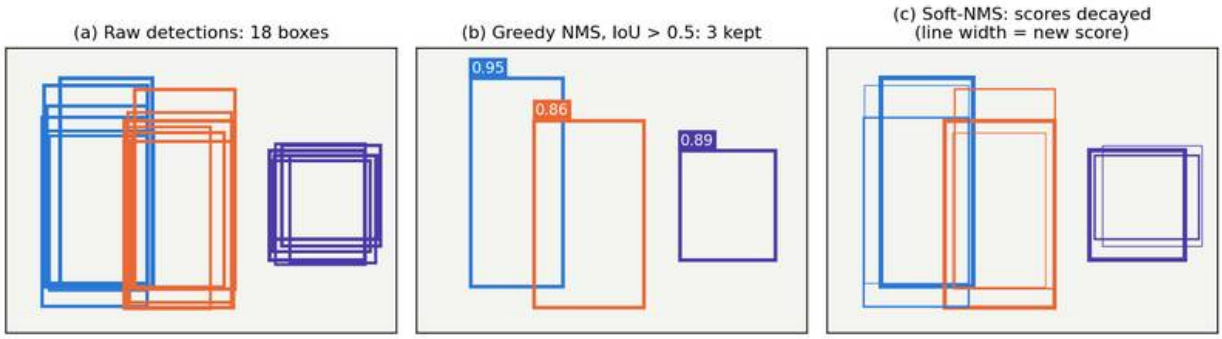


圖 *4* — 在合成偵測結果上做 *NMS*。(*a*) 三個物件周圍的 *18* 個原始框。(*b*) *IoU* 門檻 *0.5* 的貪婪 *NMS* 為每個物件留下一個框。(*c*) *Soft-NMS* 保留所有框，但降低重疊框的分數（線越細，分數越低）。

NMS 還有三個問題：它有一個要手動調整的門檻；它是循序、依資料而定的步驟，不容易在加速器上執行；而且它不屬於訓練的一部分，所以網路從來沒被教過要避免重複。去掉 NMS 是 DETR 與 YOLOv10 [24] 的主要動機之一。

特徵金字塔（FPN）

白話地說：小物件需要高解析度的特徵圖；大物件需要看過大量上下文的特徵。特徵金字塔讓每個尺度都同時擁有兩者。

CNN 骨幹網路本來就會產生步幅為 $4, 8, 16, 32$ 的金字塔 $\{C_2, C_3, C_4, C_5\}$，但淺層的語意較弱。**特徵金字塔網路**（Feature Pyramid Network, FPN）[14] 加上一條由上而下的路徑：從最粗的一層開始，上採樣 2 倍，再加上經 1×1 投影的骨幹橫向連接，

$$P_l = \mathrm{Conv}_{3\times3}\big(\mathrm{Conv}_{1\times1}(C_l) + \mathrm{Up}_{2\times}(P_{l+1})\big),$$

因此每個輸出層 P_l 的通道數相同，語意也都很強。在搭配 FPN 的二階段偵測器中，大小為 $w \times h$ 的感興趣區域從第 $l = \lfloor l_0 + \log_2(\sqrt{wh}/224) \rfloor$ 層擷取特徵（[14] 中 $l_0 = 4$），所以區域大一倍就往上移一層；密集偵測器也使用類似的「尺寸對應層級」規則。後來的頸部（neck）設計（額外的由下而上路徑、加權融合）都在精煉同一個想法；2017 年後幾乎每個偵測器都用上了它（圖 3 右）。

焦點損失與類別不平衡

白話地說：如果 99.9% 的候選都是背景，一般的損失函數會被數百萬個簡單的「這裡沒東西」樣本主導。焦點損失把模型已經答對的樣本「調小音量」。

令 $p \in [0,1]$ 為前景類別的預測機率，$y \in \{0,1\}$ 為標籤。若 $y = 1$，定義 $p_t = p$，否則 $p_t = 1 - p$。**焦點損失**（focal loss）[15] 為

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t),$$

其中 $\gamma \geq 0$ 是聚焦參數，α_t 是類別平衡權重（前景為 α，背景為 $1-\alpha$）。$\gamma = 0$ 時它就是加權交叉熵。以論文預設的 $\gamma = 2$、$\alpha = 0.25$，一個 $p_t = 0.99$ 的簡單負樣本相對於交叉熵被降權 $(0.01)^2 = 10^{-4}$ 倍，而困難樣本幾乎保留全部權重。損失會再除以正樣本錨框的數量。二階段偵測器用別的方法避開這個問題（提議階段先移除大部分背景，再以固定的前景：背景比例取樣），而 SSD [26] 使用**困難負樣本挖掘**（只保留損失最高的負樣本，比例約 3:1）。焦點損失第一次讓**密集**的一階段偵測器和二階段偵測器一樣準確。

二階段偵測器：R-CNN 家族

白話地說：先提出幾千個「可能有東西」的地方，再仔細看每一個，決定它**是什麼**以及邊緣**確切在哪裡**。

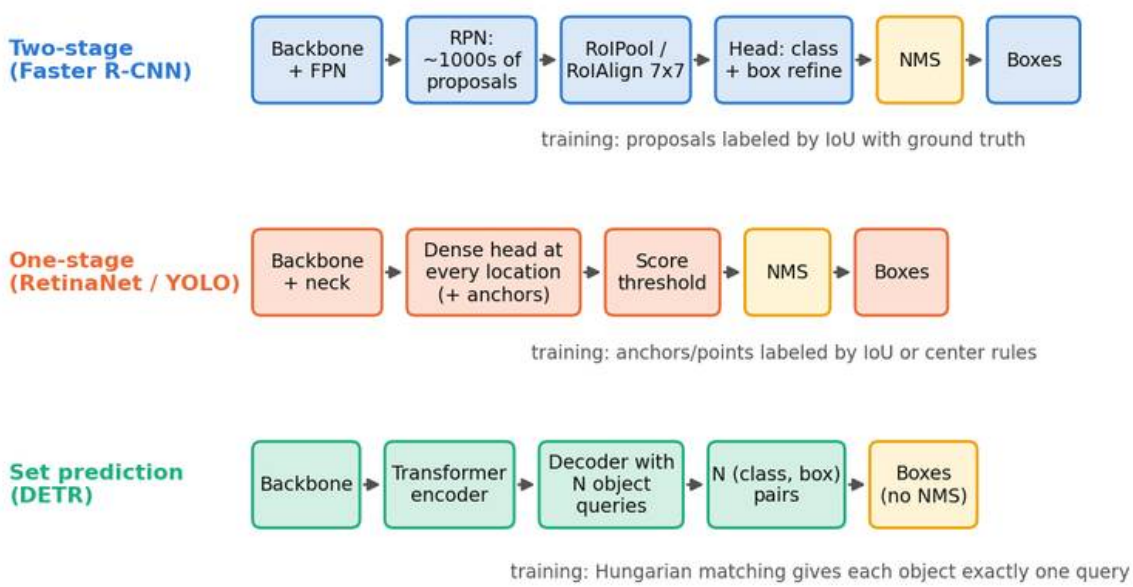


圖 5 — 三種偵測器流程。對二階段與一階段偵測器而言，*NMS*（黃色）是後處理步驟；*DETR* 式的集合預測透過學習一對一指派而把它移除。

R-CNN（2014） [8] 先用一個由下而上的分組演算法，為每張影像產生約 2,000 個與類別無關的候選區域（region proposal），把每個區域變形成固定大小，送進以分類預訓練的 CNN，以每類別一個的線性 SVM 為特徵評分，再用上述目標微調邊界框。它把 PASCAL VOC 2012 的 mean AP 相對於先前最佳結果提升了 30% 以上，並確立了「先在分類上預訓練，再在偵測上微調」這個至今仍是標準的做法。它非常慢，因為 CNN 要對每個候選區域各跑一次。

Fast R-CNN（2015） [9] 對整張影像只跑一次 CNN，再用 **RoI pooling** 從共享的特徵圖上切出每個候選區域：把區域投影到特徵圖上，分成固定的 $H \times W$ 網格（例如 7×7），每格做最大池化，所以不論形狀為何，每個候選區域都得到相同大小的特徵。整個網路以多任務損失訓練：

$$L = L_{\text{cls}}(p, u) + \lambda \, [u \geq 1] \, L_{\text{loc}}(t^u, v),$$

其中 p 是 $K+1$ 類的 softmax，u 是真實類別（背景為 0），$[u \geq 1]$ 只在前景候選區域為 1，t^u 是針對類別 u 預測的框偏移量，v 是目標偏移量，L_{loc} 是 smooth L1 損失。

Faster R-CNN（2015） [10] 讓候選區域本身也由學習產生。一個小型的**區域提議網路**（Region Proposal Network, RPN）在共享特徵圖上滑動，為每個位置的 9 個錨框預測「是否為物件」的分數與框偏移量。最好的候選區域（經 NMS 後）再送進 Fast R-CNN 的偵測頭。由於兩個階段共用骨幹，候選區域幾乎不花成本，整個偵測器也成為一個端到端訓練的網路。

Mask R-CNN（2017） [16] 加上一個小型全卷積分支，在每個 RoI 內為每個類別預測一張二值遮罩，因而得到實例分割。它的關鍵細節是 **RoIAlign**：RoI pooling 會把區域邊界和格子邊緣捨入到特徵網格上，使特徵最多偏移半個步幅。RoIAlign 保留實數座標，並以雙線性內插取樣每一格。這聽起來是小事，但像素級精確的遮罩需要它，高 IoU 下的框準確度也會提升。

Cascade R-CNN（2018） [17] 處理一個微妙的問題。以 IoU ≥ 0.5 為正樣本訓練的偵測頭，學到的是「在 0.5 夠好」的框，在 0.75 卻不夠好。單純提高門檻會讓正樣本太少而過擬合（overfitting）。Cascade R-CNN 串接數個以遞增 IoU 門檻（0.5、0.6、0.7）訓練的偵測頭，每一級都精煉前一級的框，所以每一級拿到的訓練分布品質都更高。它在嚴格的 COCO 指標上帶來大幅提升。

二階段偵測器準確、有彈性（RoI 偵測頭可以預測遮罩、關鍵點或屬性），至今仍是許多實例分割任務的預設選擇，但每個 RoI 一次的偵測頭與 NMS 都要花時間。

一階段偵測器：YOLO、SSD 與 RetinaNet

白話地說：跳過提議步驟。在一次前向傳遞中，對特徵圖的每個位置做一次密集預測，再用 NMS 清理。

YOLO v1（2016） [18] 把影像切成 $S \times S$ 網格（$S = 7$）。每格預測 $B = 2$ 個框（各帶一個信心值）以及一組 C 個類別機率，因此輸出是單一個 $S \times S \times (5B + C)$ 張量（PASCAL VOC 的 20 類對應 $7 \times 7 \times 30$）。物件中心所在的格子負責該物件。它能即時執行，而且因為看得到整張影像，背景誤判比 Fast R-CNN 少；但它定位較不精確，也難以處理小物件與成群的物件，因為每格只能預測固定數量的框與一個類別。

YOLO 家族之後快速演進。下表只列出我們能查證論文的版本；許多廣泛使用的版本是以軟體套件形式維護、沒有論文，本篇不予涵蓋。

版本	年份	主要想法（依論文）
YOLO [18]	2016	單次網格迴歸；即時
YOLO9000 / v2 [19]	2016（arXiv）	以 IoU 距離做 k-means 選出錨框；批次正規化；較高解析度訓練
YOLOv3 [20]	2018（技術報告）	更深的骨幹；在三個尺度預測；各類別獨立的 logistic 分數
YOLOv4 [21]	2020（arXiv）	系統性研究訓練技巧（bag of freebies）與低成本的架構附加元件（bag of specials）
YOLOv7 [22]	2023（CVPR）	可訓練的 bag-of-freebies：重參數化模組、改良的標籤指派、高效的層聚合
YOLOv9 [23]	2024（ECCV）	可程式化梯度資訊（PGI）讓資訊穿過深層而不流失；GELAN 骨幹

版本	年份	主要想法（依論文）
YOLOv10 [24]	2024（NeurIPS）	以一致的雙重指派（一對多與一對一）做免 NMS 訓練；以效率為導向的設計
YOLOv12 [25]	2025（arXiv）	以注意力為核心、同時維持即時速度的設計

可以看出一個反覆出現的主題：「密集網格＋ NMS」的核心想法不變，改變的是標籤指派、頸部、訓練配方，以及最近的注意力機制與免 NMS 偵測頭。

SSD（2016） [26] 在**多個**不同解析度的特徵圖上接上小型卷積預測器，每層都有自己一組多種長寬比的預設框（default box，即錨框），處理尺度的能力遠勝 YOLO v1。它以困難負樣本挖掘與大量資料擴增訓練。

RetinaNet（2017） [15] 結合 FPN 骨幹、密集錨框（每個位置 9 個），以及分類與框迴歸兩個小型子網路，並以焦點損失訓練。論文的分析顯示，一階段偵測器先前落後二階段偵測器的主因是類別不平衡，而不是架構。

無錨框偵測器：用點取代框

白話地說：錨框是對物件長相的猜測。無錨框偵測器拿掉它們，改從**點**預測邊界框：角點、中心點，或物件內部的每一個像素。

CornerNet（2018） [27] 把物件偵測成一對關鍵點：一張左上角熱圖、一張右下角熱圖，以及每個角點的**嵌入向量**（embedding）。同一物件的兩個角點被訓練成嵌入相近，因此以嵌入距離來配對。由於角點常落在物件外面，論文提出**角點池化**（corner pooling），沿著列與行收集角點的證據。

CenterNet，〈Objects as Points〉（2019） [28] 以中心點代表每個物件。網路輸出一張熱圖 $\hat{Y} \in [0,1]^{\frac{W}{R}\times\frac{H}{R}\times K}$（輸出步幅 R，每類一個通道），以及每個位置的尺寸 (w,h) 與次像素偏移。真值中心以高斯函數散佈在熱圖上，熱圖以逐像素的焦點損失訓練。推論時，峰值就是局部最大值，用 3×3 最大池化比較即可找到，因此**不需要以 IoU 為基礎的 NMS**。同樣的設計只要在中心點迴歸更多量，就能延伸到 3D 框與人體姿態。

FCOS（2019） [29] 把偵測當成分割來做。特徵圖上每個落在真值框內的位置都是正樣本，並迴歸到框四邊的距離：

$$l^* = x - x_1, \qquad t^* = y - y_1, \qquad r^* = x_2 - x, \qquad b^* = y_2 - y,$$

其中 (x,y) 是該位置的影像座標，(x_1,y_1,x_2,y_2) 是框。每一層 FPN 只處理最大迴歸距離落在自己範圍內的物件，這解決了框重疊時大部分的歧義。離物件中心遠的位置預測出的框品質差，因此

FCOS 加入**中心度**（centerness）分支：

$$\text{centerness} = \sqrt{\frac{\min(l^*,r^*)}{\max(l^*,r^*)} \cdot \frac{\min(t^*,b^*)}{\max(t^*,b^*)}},$$

它在中心為 1，往邊緣遞減；測試時乘上類別分數，讓偏離中心的框在 NMS 前排得較後面。FCOS 去掉了所有錨框超參數，表現追平甚至超越以錨框為基礎的 RetinaNet。

以 Transformer 做集合預測：DETR 及其後繼者

白話地說：與其預測上千個候選再刪掉重複的，不如直接預測一小**組**答案，並在訓練時讓每個真實物件剛好被一個預測認領。

DETR（2020） [30] 使用 CNN 骨幹、作用在特徵圖上的 Transformer 編碼器，以及一個 Transformer 解碼器；解碼器接收 N 個學習得到的**物件查詢**（object query，論文中為 100 個），平行輸出 N 組（類別、框）預測。沒有錨框、沒有候選區域，也沒有 NMS。由於 N 大於物件數，真值會用特殊的「無物件」類別 $\varnothing$ 補齊到 N 個。

關鍵在於以二**元匹配**（bipartite matching）訓練。令 $y = \{y_j\}$ 為補齊後的真值，$\hat{y} = \{\hat{y}_i\}_{i=1}^{N}$ 為預測。找出使總匹配成本最低的 N 元素排列 $\hat{\sigma}$：

$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_{j=1}^{N} \mathcal{L}_{\text{match}}(y_j, \hat{y}_{\sigma(j)}), \qquad \mathcal{L}_{\text{match}} = -\mathbb{1}_{\{c_j \neq \varnothing\}}\, \hat{p}_{\sigma(j)}(c_j) + \mathbb{1}_{\{c_j \neq \varnothing\}}\, \mathcal{L}_{\text{box}}(b_j, \hat{b}_{\sigma(j)}),$$

其中 $\mathfrak{S}_N$ 是所有排列的集合，c_j 與 b_j 是目標 j 的類別與框，$\hat{p}_{\sigma(j)}(c_j)$ 是被匹配的查詢對類別 c_j 的預測機率，而 $\mathcal{L}_{\text{box}} = \lambda_{L1}\|b_j - \hat{b}_{\sigma(j)}\|_1 + \lambda_{\text{iou}}(1 - \text{GIoU}(b_j, \hat{b}_{\sigma(j)}))$。這個指派問題可以用**匈牙利演算法**（Hungarian algorithm）[31] 在多項式時間內精確求解。接著，訓練損失（**集合損失**，set loss）使用匹配好的配對：

$$\mathcal{L}_{\text{Hungarian}} = \sum_{j=1}^{N} \Big[-\log \hat{p}_{\hat{\sigma}(j)}(c_j) + \mathbb{1}_{\{c_j \neq \varnothing\}}\, \mathcal{L}_{\text{box}}(b_j, \hat{b}_{\hat{\sigma}(j)}) \Big],$$

當 $c_j = \varnothing$ 時，對數機率項的權重降為十分之一，以平衡大量的「無物件」名額。因為每個物件只會匹配到一個查詢，重複的預測會被當成錯誤的「無物件」而受到**懲罰**，解碼器的自注意力因此學會抑制重複，這就是不需要 NMS 的原因。

用 SciPy 寫匹配步驟只需幾行。下面的成本以 DETR 的權重結合三項；`linear_sum_assignment` 可直接處理 $N \times M$ 的長方形矩陣（DETR 的 L1 項是在正規化的 (c_x, c_y, w, h) 上計算，這裡為了簡潔使用角點格式）。

```
import numpy as np
from scipy.optimize import linear_sum_assignment

def giou_matrix(a, b):
    x1 = np.maximum(a[:, None, 0], b[None, :, 0]); y1 = np.maximum(a[:, None, 1], b[None, :,
     ↪  1])
    x2 = np.minimum(a[:, None, 2], b[None, :, 2]); y2 = np.minimum(a[:, None, 3], b[None, :,
     ↪  3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
    union = area(a)[:, None] + area(b)[None, :] - inter
    cx1 = np.minimum(a[:, None, 0], b[None, :, 0]); cy1 = np.minimum(a[:, None, 1], b[None,
     ↪  :, 1])
```

```
    cx2 = np.maximum(a[:, None, 2], b[None, :, 2]); cy2 = np.maximum(a[:, None, 3], b[None,
     ↪  :, 3])
    c = (cx2 - cx1) * (cy2 - cy1)
    return inter / union - (c - union) / c

rng = np.random.default_rng(1)
N, C = 6, 3                                      # 6 queries, 3 real classes (+ "no object")
logits = rng.normal(size=(N, C + 1))
prob = np.exp(logits) / np.exp(logits).sum(1, keepdims=True)
pred_boxes = np.array([[.10, .10, .40, .50], [.55, .20, .90, .60], [.12, .08, .42, .48],
                       [.30, .60, .60, .95], [.00, .00, .20, .20], [.50, .50, .70, .70]])
gt_boxes  = np.array([[.10, .10, .40, .50], [.32, .62, .62, .93], [.56, .22, .88, .62]])
gt_labels = np.array([0, 2, 1])

w_cls, w_l1, w_giou = 1.0, 5.0, 2.0              # weights used by DETR
cost = (-w_cls * prob[:, gt_labels]                          # (N, M) class term
        + w_l1 * np.abs(pred_boxes[:, None] - gt_boxes[None]).sum(-1)
        - w_giou * giou_matrix(pred_boxes, gt_boxes))
rows, cols = linear_sum_assignment(cost)         # works on rectangular N x M matrices
print("query -> gt:", dict(zip(rows.tolist(), cols.tolist())))
print("unmatched queries (trained as 'no object'):", sorted(set(range(N)) -
 ↪  set(rows.tolist())))
```

```
query -> gt: {0: 0, 1: 2, 3: 1}
unmatched queries (trained as 'no object'): [2, 4, 5]
```

查詢 2 幾乎是查詢 0 的複本，卻沒有被匹配，之後會被訓練成「無物件」。這正是取代 NMS 的機制。

代價。 DETR 需要非常長的訓練（數百個 epoch，Faster R-CNN 大約只要十幾個），小物件表現也差，因為在高解析度特徵圖上做全域注意力太昂貴，只能用單一一層粗特徵。後繼者逐一解決了這些問題。

- **Deformable DETR（2021）** [32] 以**可變形注意力**（deformable attention）取代密集注意力：每個查詢只關注參考點附近少數幾個學習得到的取樣點，並橫跨多個特徵層級。這讓多尺度特徵變得負擔得起；依論文所述，它以少 10 倍的訓練 epoch 達到比 DETR 更好的準確度（小物件尤其明顯）。
- **DINO（2022）** [33]（不要和同名的自監督方法混淆）結合了**對比去雜訊訓練**（把加了雜訊的真值框當成額外查詢送入並要求還原，雜訊較大的則必須被拒絕）、**混合查詢選擇**（以編碼器特徵初始化錨點位置），以及 **look-forward-twice** 的框更新。它報告在 COCO 上以標準的 50 層 CNN 骨幹訓練 12 個 epoch 得到 49.4 AP、24 個 epoch 得到 51.3 AP；以大型骨幹並在 Objects365 預訓練後，在 COCO val2017 上達到 63.2 AP。

- **RT-DETR（2024）** [34] 鎖定即時應用。**高效混合編碼器**只在最粗的一層使用注意力，並以卷積融合不同尺度；**IoU 感知的查詢選擇**挑出更好的初始查詢。論文報告其較小與較大版本在 COCO 上分別為 53.1 / 54.3 AP，在 T4 GPU 上達 108 / 74 FPS，在當時比尺寸相當的 YOLO 模型更快也更準，而且不需 NMS。

如今 DETR 路線與 YOLO 路線正在匯流：YOLOv10 借用一對一指派來去掉 NMS，即時 DETR 則借用高效的卷積頸部。

開放詞彙與定位偵測器

白話地說：封閉集合的偵測器只能找出它訓練過的 80 或 365 個類別。開放詞彙偵測器在**測試時以文字**接收類別清單，所以你可以直接要它找「紅色消防栓」或「堆高機」，不必重新訓練。

技巧在於把最後的分類層（一個由 K 個類別權重向量組成的固定矩陣）換成**文字嵌入**。對區域特徵 $\mathbf{z}_i$ 與提示詞（例如 “a photo of a forklift”）的文字嵌入 $\mathbf{e}_k$，類別分數是縮放後的餘弦相似度：

$$s_{ik} = \tau \cdot \frac{\mathbf{z}_i^\top \mathbf{e}_k}{\|\mathbf{z}_i\| \, \|\mathbf{e}_k\|},$$

其中 τ 是學習得到的溫度參數。任何能嵌入的片語都能變成一個類別。視覺端必須學會把區域與語言對齊，這需要遠比任何框標註資料集更大量的影像–文字資料。

- **OWL-ViT（2022）** [35] 從以影像–文字對比預訓練的 Transformer 影像編碼器與文字編碼器出發，移除最後的池化，在每個輸出 token 上接上輕量的框與類別偵測頭；類別偵測頭就是上面的文字嵌入。它支援零樣本（文字查詢）與單樣本（影像範例查詢）偵測，論文也顯示模型越大效果越穩定提升。
- **Grounding DINO（2024）** [36] 把語言深度融入 DINO 偵測器：影像與文字互相做交叉注意力的特徵增強器、語言引導的查詢選擇，以及跨模態解碼器。它在偵測、定位（片語對應區域）與圖說資料上預訓練，報告在 COCO 零樣本遷移（完全不用 COCO 訓練資料）上達 52.5 AP，在一組多樣化真實世界偵測資料集組成的基準上平均 AP 為 26.1。
- **YOLO-World（2024）** [37] 以視覺–語言路徑聚合網路（RepVL-PAN）與區域–文字對比預訓練，把開放詞彙帶進即時偵測器。它的**先提示、後偵測**（prompt-then-detect）模式會事先離線編碼使用者的詞彙，因此推論成本與封閉集合的 YOLO 相當。論文報告在 LVIS 上達 35.4 AP，在 V100 GPU 上 52.0 FPS。

可提示的基礎模型更進一步。SAM 3 [38]（2025）接受一個簡短名詞片語或影像範例作為**概念提示**（concept prompt），為影像與影片中**所有**符合的實例輸出遮罩與身分；作者建立了一個資料引擎，產生含有 400 萬個不重複概念標籤的資料集，並報告在其可提示概念分割基準上的準確度是既有系統的兩倍。在這類系統中，偵測成為一個通用「找出這個概念」模型的其中一種輸出，而不是單獨訓練的網路。

小物件與密集場景

白話地說：極小的物件只有很少的像素，步幅 32 的特徵可能把它壓縮到不到一個格子。人群之所以困難，是因為物件互相遮擋，而 NMS 會把相鄰的物件誤認為重複。

小物件。 COCO 把面積小於 32^2 像素的物件定義為「小」[40]。經過步幅 16 的骨幹網路後，這樣的物件最多只占約 2×2 個特徵格，外觀大多在降採樣中流失。IoU 也變得嚴苛：在 10×10 的框上偏差 2 個像素，IoU 就可能跌破 0.7。常見的對策有高解析度特徵層（步幅 4 的 P_2）、特徵金字塔、推論時把影像切塊、提高輸入解析度、複製貼上與尺度擴增，以及不會讓小框缺少正樣

本的指派規則。Cheng 等人的綜述與基準 [39] 整理了這些策略，並提出兩個大型小物件基準：SODA-D（24,828 張交通影像、278,433 個實例、9 個類別）與 SODA-A（2,513 張高解析度空拍影像、872,069 個實例、9 個類別）。

密集與遮擋場景。在人群中，真實物件彼此大量重疊，固定的 NMS 門檻只能在召回率與重複之間取捨。Soft-NMS [13] 有幫助；集合預測偵測器則透過學習哪些預測是重複的，完全避開門檻。遮擋也讓框的標註變得模糊（要標完整範圍，還是只標看得見的部分？），評估結果因此取決於協定。這些問題在追蹤中會再次出現，因為物件會消失又重新出現。

偵測器的評估

白話地說：把所有偵測結果依信心排序，由上往下走：每個偵測結果要不是命中（它和一個尚未被認領的真實物件夠吻合），就是誤報。記錄精確率與召回率如何變化；這條曲線下的面積就是平均精確率（Average Precision, AP）。

精確率、召回率與 AP

對單一類別與單一 IoU 門檻 θ，把測試集中所有偵測結果依分數由高到低排序。若某偵測結果與同類別、尚未被匹配的真值框 IoU 至少為 θ，就是**真陽性**（TP）；否則是**偽陽性**（FP）。每個真值框只能被匹配一次，重複的偵測就是這樣被懲罰的。取前 k 個偵測結果時，

$$\text{precision}(k) = \frac{\text{TP}(k)}{k}, \qquad \text{recall}(k) = \frac{\text{TP}(k)}{M},$$

其中 M 是該類別真值物件的數量。為了讓曲線單調，精確率以其上包絡線**內插**：$p_{\text{interp}}(r) = \max_{r' \geq r} p(r')$。接著：

- **VOC 11 點 AP**（PASCAL VOC 到 2009 年為止 [3]）：$\text{AP} = \frac{1}{11} \sum_{r \in \{0, 0.1, \ldots, 1\}} p_{\text{interp}}(r)$。
- **VOC 全點 AP**（VOC 2010 起 [41]）：內插曲線下的精確面積，$\text{AP} = \sum_n (r_{n+1} - r_n)\, p_{\text{interp}}(r_{n+1})$，對所有不同的召回值加總。
- **COCO AP** [40]：在 $0, 0.01, \ldots, 1$ 共 101 個召回點上取 p_{interp} 的平均。

圖 6 顯示一條 PR 曲線、它的包絡線，以及 AP 面積。

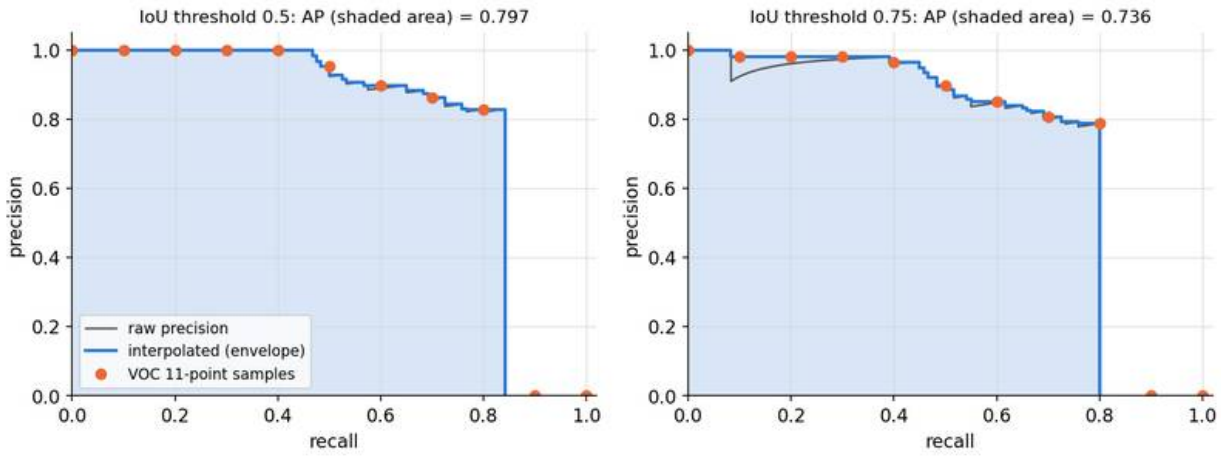


圖 *6* — 合成偵測結果在兩個 *IoU* 門檻下的精確率–召回率。內插包絡線下的陰影面積即全點 *AP*；橘點是 *VOC 11* 點取樣。精確率在召回率終止處降為 *0*，因為有些物件從未被偵測到。

COCO 協定

COCO 的主要指標寫作 **AP** 或 **mAP@[.5:.95]**，對 10 個 IoU 門檻 $\theta \in \{0.50, 0.55, \dots, 0.95\}$ 及全部 80 個類別取平均 [40]：

$$\mathrm{AP}_{\mathrm{COCO}} = \frac{1}{10\,K} \sum_{\theta} \sum_{k=1}^{K} \mathrm{AP}_k(\theta).$$

它獎勵的是精確定位，而不只是找到物件。配套指標有 AP_{50}（VOC 式指標）、AP_{75}（嚴格），以及針對小（面積 $< 32^2$）、中（32^2 到 96^2）、大（$> 96^2$）物件的 AP_S、AP_M、AP_L。每張影像最多只計分 100 個偵測結果。每張影像 1、10、100 個偵測結果下的平均召回率（AR）也會一併報告。注意 COCO 把類別平均後的指標稱為「AP」；論文中的「mAP」通常指同一件事。

下面的程式碼在合成預測上從頭計算所有指標（真值加上抖動、約 15% 漏偵測，再加上一些隨機的誤報）。

```
import numpy as np

def box_iou(a, b):
    """Pairwise IoU between (N,4) and (M,4) xyxy boxes -> (N, M)."""
    x1 = np.maximum(a[:, None, 0], b[None, :, 0]); y1 = np.maximum(a[:, None, 1], b[None, :,
     ↪  1])
    x2 = np.minimum(a[:, None, 2], b[None, :, 2]); y2 = np.minimum(a[:, None, 3], b[None, :,
     ↪  3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
    return inter / (area(a)[:, None] + area(b)[None, :] - inter)

def match(dets, gts, thr):
    """dets: list over images of (boxes, scores); gts: list of gt boxes.
    Returns scores, TP flags (sorted by score), and number of GT."""
```

```
    recs = []
    for img, ((db, ds), gb) in enumerate(zip(dets, gts)):
        for k in range(len(ds)):
            recs.append((ds[k], img, k))
    recs.sort(key=lambda r: -r[0])                        # global ranking by confidence
    used = [np.zeros(len(g), bool) for g in gts]
    tp = np.zeros(len(recs))
    for n, (s, img, k) in enumerate(recs):
        gb = gts[img]
        if len(gb) == 0:
            continue
        ious = box_iou(dets[img][0][k:k + 1], gb)[0]
        ious[used[img]] = -1                              # each GT can be matched once
        j = ious.argmax()
        if ious[j] >= thr:
            tp[n] = 1; used[img][j] = True
    return tp, sum(len(g) for g in gts)

def pr_curve(tp, n_gt):
    ctp, cfp = np.cumsum(tp), np.cumsum(1 - tp)
    return ctp / n_gt, ctp / (ctp + cfp)                  # recall, precision

def ap(rec, prec, mode="all"):
    r = np.concatenate([[0], rec, [1]]); p = np.concatenate([[0], prec, [0]])
    p = np.maximum.accumulate(p[::-1])[::-1]              # monotone envelope
    if mode == "all":                                     # VOC2010+: exact area
        i = np.where(r[1:] != r[:-1])[0]
        return np.sum((r[i + 1] - r[i]) * p[i + 1])
    pts = np.linspace(0, 1, 11 if mode == "voc11" else 101)
    return np.mean([p[r >= t].max() if np.any(r >= t) else 0 for t in pts])

# synthetic data: 50 images, 1-4 objects each; detector = jittered GT + false positives
rng = np.random.default_rng(0)
gts, dets = [], []
for _ in range(50):
    n = rng.integers(1, 5)
    xy = rng.uniform(0, 200, (n, 2)); wh = rng.uniform(20, 80, (n, 2))
    g = np.hstack([xy, xy + wh]); gts.append(g)
    keep = rng.random(n) > 0.15                           # miss ~15% of objects
    jit = rng.normal(0, 0.06, (n, 4)) * np.hstack([wh, wh])
    db = (g + jit)[keep]; ds = rng.uniform(0.4, 1.0, keep.sum())
    m = rng.integers(0, 3)                                # 0-2 false positives
    fxy = rng.uniform(0, 200, (m, 2)); fwh = rng.uniform(20, 80, (m, 2))
    db = np.vstack([db, np.hstack([fxy, fxy + fwh])]); ds = np.concatenate([ds,
     ↪  rng.uniform(0, 0.7, m)])
    dets.append((db, ds))

for thr in (0.5, 0.75):
    tp, n_gt = match(dets, gts, thr)
    rec, prec = pr_curve(tp, n_gt)
    print(f"IoU {thr}: VOC11 {ap(rec, prec, 'voc11'):.3f}  all-point {ap(rec, prec):.3f}"
          f"  COCO-101 {ap(rec, prec, 'coco'):.3f}")
aps = [ap(*pr_curve(*match(dets, gts, t)), "coco") for t in np.arange(0.5, 0.96, 0.05)]
print("AP@[.5:.95] =", round(float(np.mean(aps)), 3))
```

```
IoU 0.5: VOC11 0.776  all-point 0.797  COCO-101 0.797
IoU 0.75: VOC11 0.749  all-point 0.736  COCO-101 0.738
AP@[.5:.95] = 0.532
```

可以看出三個教訓。三種內插規則得到相近但**不相同**的數字，所以不同協定算出的 AP 絕對不能互相比較。AP@[.5:.95] 比 AP50 低得多，因為抖動的框通不過嚴格門檻。而這只是單一類別；真正的 mAP 會對類別取平均，所以稀有類別和常見類別占同樣的份量。

AP 沒告訴你的事

AP 總結的是整個測試集上的排序品質，因此它不會替你選定操作門檻，也不會區分錯誤的**類型**：定位錯誤、與相似類別混淆、重複偵測、背景誤報，在 AP 裡看起來都一樣。診斷模型時，要依錯誤類型與物件大小拆開分析，並且實際去看影像。

資料集與基準

白話地說：偵測的進展由少數幾個公開資料集推動；每一個都改變了「好」的定義。

資料集	類別	規模（依論文）	標註	重要性
PASCAL VOC [3]	20	以今日標準來看很小	框（＋部分遮罩）	2005–2012 年的基準；AP50
MS COCO [4]	標註 91 種；偵測用 80 種	32.8 萬張影像、250 萬個標註實例	框＋實例遮罩	雜亂的日常場景、大量小物件；AP@[.5:.95]
Objects365 [42]	365	大規模、高品質的框	框	強大偵測器常用的預訓練資料集
Open Images V4 [43]	600 個有框類別	920 萬張影像；1,540 萬個框	框、影像標籤、關係	非常大；影像層級有 1.98 萬個概念
LVIS [44]	超過 1000	16.4 萬張影像中約 200 萬張遮罩	實例遮罩	長尾詞彙；稀有類別評估

有兩個趨勢很明顯。類別數持續增加（20、80，到數百乃至上千），隨之而來的是**長尾**問題：在 LVIS 中，許多類別只有少數幾個訓練樣本，忽略頻率的損失函數或取樣器在稀有類別上就會失敗。另外，最大的模型如今都先在 Objects365 或影像–文字資料上預訓練，再於 COCO 上微調，使得「只用 COCO 訓練」與「使用額外資料」成為兩個分開的排行榜。

速度、準確度與部署

白話地說：實務上你是在一條曲線上選一個點：更準通常代表更多運算。真正重要的是在**你的**硬體上、端到端的延遲。

- **FLOPs 不等於延遲。** FLOPs 計算的是算術量，但執行時間還取決於記憶體存取、運算子支援與平行度。深度可分離卷積與注意力在 FLOPs 上可能很便宜，在某些加速器上卻很慢。
- **端到端量測。**回報延遲時要包含前處理、網路**以及**後處理。NMS 的成本隨候選框數量增加（最壞情況與數量的平方成正比）、依資料而定，而且常在 CPU 上執行。RT-DETR 的論文 [34] 正是分析了這一點，這也是 RT-DETR 與 YOLOv10 [24] 等免 NMS 設計對部署很有吸引力的原因之一。

- **輸入解析度**是最強的調整旋鈕：邊長減半，運算量約減為四分之一，但受傷最重的是小物件。
- **量化**（quantization，例如 8 位元整數的權重與活化值）與**重參數化**（訓練時用多分支區塊，之後折疊成單一卷積，如 YOLOv7 [22] 所用）能在小幅損失準確度下降低延遲。框迴歸輸出與分數頭通常對量化最敏感，所以要用真實資料校準，並且檢查 AP75，而不只是 AP50。
- **批次大小 1** 是邊緣裝置的常見情境；以大批次量測的吞吐量數字不能直接套用。

一個你可以自己訓練的小型偵測器

為了親眼看到邊界框迴歸與 IoU 損失端到端地運作，下面的 PyTorch 程式訓練一個小 CNN，在帶雜訊的 32×32 影像中定位一個矩形或橢圓。這是單一物件偵測器：網路透過 sigmoid 直接迴歸 $[0,1]$ 範圍內的 (c_x, c_y, w, h)，以 L1 加上 GIoU 損失訓練，和 DETR 對匹配到的查詢所做的一樣。

```
import time, torch, torch.nn as nn
torch.manual_seed(0)
S = 32
torch.set_num_threads(4)

def make_batch(n):
    """One white rectangle or ellipse per 32x32 noisy image; target = (cx, cy, w, h) in [0,
     ↪  1]."""
    img = 0.1 * torch.randn(n, 1, S, S)
    wh = torch.rand(n, 2) * 0.4 + 0.15
    c = wh / 2 + torch.rand(n, 2) * (1 - wh)
    yy, xx = torch.meshgrid(torch.arange(S), torch.arange(S), indexing="ij")
    xx, yy = (xx[None] + 0.5) / S, (yy[None] + 0.5) / S
    dx = (xx - c[:, 0, None, None]) / (wh[:, 0, None, None] / 2)
    dy = (yy - c[:, 1, None, None]) / (wh[:, 1, None, None] / 2)
    rect = (dx.abs() <= 1) & (dy.abs() <= 1)
    ell = dx**2 + dy**2 <= 1
    shape = torch.where((torch.rand(n) < 0.5)[:, None, None], rect, ell)
    return img + shape[:, None].float(), torch.cat([c, wh], 1)

def xyxy(b):
    return torch.cat([b[:, :2] - b[:, 2:] / 2, b[:, :2] + b[:, 2:] / 2], 1)

def giou(p, g):
    p, g = xyxy(p), xyxy(g)
    lt, rb = torch.max(p[:, :2], g[:, :2]), torch.min(p[:, 2:], g[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(1)
    area = lambda b: (b[:, 2:] - b[:, :2]).prod(1)
    union = area(p) + area(g) - inter
    c = (torch.max(p[:, 2:], g[:, 2:]) - torch.min(p[:, :2], g[:, :2])).prod(1)
```

```
    return inter / union, inter / union - (c - union) / c

net = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 16
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 8
    nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 4
    nn.Flatten(), nn.Linear(32 * 4 * 4, 128), nn.ReLU(),
    nn.Linear(128, 4), nn.Sigmoid())                               # (cx, cy, w, h) in [0, 1]
opt = torch.optim.Adam(net.parameters(), 2e-3)

t0 = time.time()
for step in range(500):
    x, y = make_batch(64)
    pred = net(x)
    iou, g = giou(pred, y)
    loss = (pred - y).abs().sum(1).mean() + (1 - g).mean()        # L1 + GIoU loss
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    x, y = make_batch(1000)
    iou, _ = giou(net(x), y)
print(f"train time {time.time() - t0:.1f}s | test mean IoU {iou.mean():.3f}"
      f" | IoU>=0.5: {(iou >= 0.5).float().mean():.1%} | IoU>=0.75: {(iou >=
      ↪  0.75).float().mean():.1%}")
```

```
train time 21.1s | test mean IoU 0.885 | IoU>=0.5: 100.0% | IoU>=0.75: 97.8%
```

（訓練時間視你的 CPU 而定。）因為恰好只有一個物件，所以沒有指派問題，也不需要 NMS。練習 6 會請你把它擴充到多個物件，而本篇所有困難的問題都會在那裡重新出現。

現代觀點

圖 7 以每篇引用文獻的發表年份，把本篇的方法放在時間軸上。

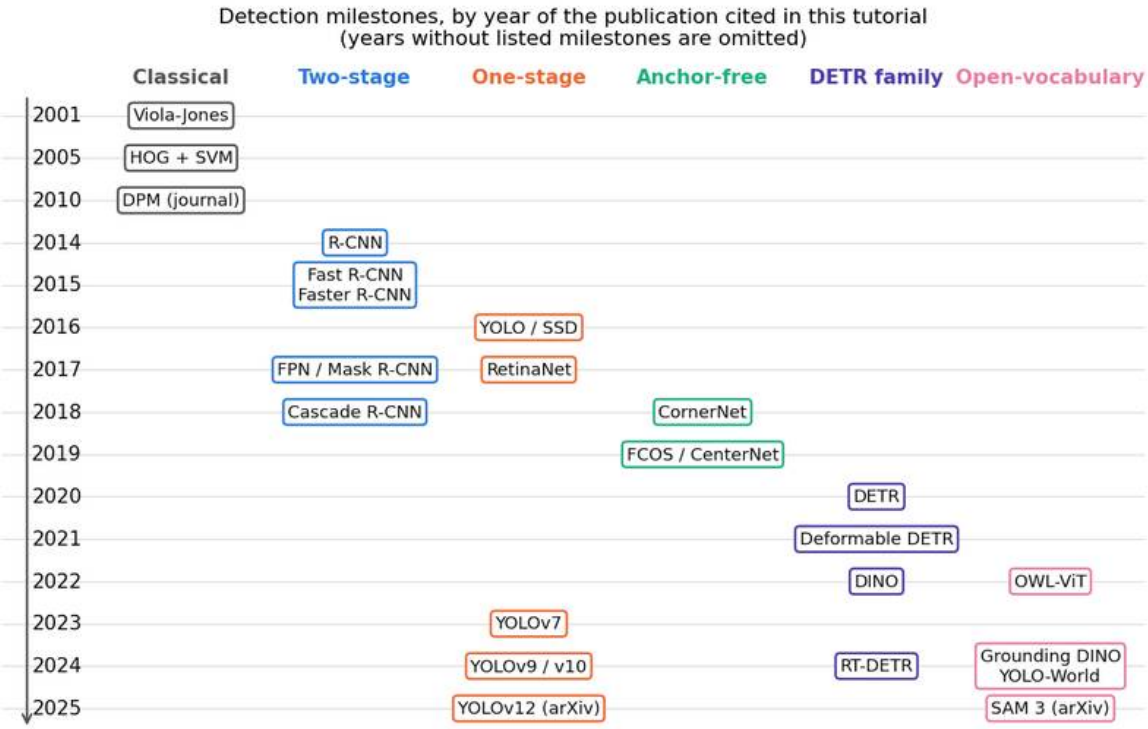


圖 7 — 依本篇引用文獻發表年份排列、按家族分組的里程碑。沒有列出里程碑的年份已省略。

綜述怎麼說

《**Object Detection in 20 Years**》（Zou 等人，*Proceedings of the IEEE*，2023）[1] 是涵蓋最廣的歷史回顧。它涵蓋從傳統時代（Viola–Jones、HOG、DPM）到深度二階段與一階段方法的里程碑偵測器、偵測資料集與指標、基本元件（多尺度偵測、上下文、困難負樣本挖掘、NMS、框迴歸）、加速技術，以及近期最先進的方法。它的主要訊息是：大多數「新」想法都有更早的根源——多尺度掃描變成特徵金字塔，串接變成串接式偵測頭，困難負樣本挖掘變成焦點損失，而 NMS 被重新思考了好幾次。

《**Deep Learning for Generic Object Detection: A Survey**》（Liu 等人，*IJCV*，2020）[2] 深入回顧了 300 多篇研究，並依偵測框架、物件特徵表示、候選區域產生、上下文建模、訓練策略與評估指標加以整理。它是 Transformer 之前 CNN 時代最好的單一參考文獻，並強調進展同樣來自訓練策略、資料與評估，而不只是架構。

《**Object Detection with Transformers: A Review**》（Shehzadi 等人，*Sensors*，2025）[45] 分析了 21 項對 DETR 的近期改進，依骨幹、查詢設計與注意力機制的修改分類，並在共同基準上比較。它的分類法對應到上面描述的修正：用多尺度與稀疏注意力改善小物件與速度，用更好的查詢初始化與去雜訊改善收斂。

《**A Survey on Open-Vocabulary Detection and Segmentation: Past, Present, and Future**》（Zhu 與 Chen，*IEEE TPAMI*，2024）[46] 依照如何利用弱監督，把開放詞彙方法分成六個家族：視覺–語意空間映射、新類別視覺特徵合成、區域感知訓練、偽標籤、知識蒸餾與遷移學習。它涵

蓋 2D、3D 與影片任務，並指出預訓練資料與測試資料在詞彙上的重疊，讓「新類別」的評估變得複雜。

《**Towards Large-Scale Small Object Detection**》（Cheng 等人，*IEEE TPAMI*，2023）[39] 聚焦在最頑強的失敗模式，依類別回顧各種對策，並提出前面介紹的 SODA 基準。它顯示即使是強大的偵測器，小物件的準確度仍遠低於較大的物件。

2023–2026 年的最先進技術

- **即時封閉集合偵測**如今是 YOLO 路線（YOLOv9 [23]、YOLOv10 [24]、YOLOv12 [25]）與即時 DETR（RT-DETR [34]）之間的競賽。兩條路線互相借鏡：YOLO 引入一對一指派與免 NMS 偵測頭，DETR 引入卷積頸部與混合編碼器。在即時預算下使用注意力已經負擔得起。

- **高準確度的封閉集合偵測**由 DETR 式模型主導，例如搭配大型骨幹與大規模預訓練（Objects365 [42] 及更多）的 DINO [33]。COCO 上的進步幅度已經很小，對最大的模型而言這個基準已接近飽和，因此注意力轉向更難的基準：LVIS [44]、開放詞彙與跨領域評估。
- **開放詞彙與定位偵測**（OWL-ViT [35]、Grounding DINO [36]、YOLO-World [37]）正走進實務：在許多應用中，你現在是去**提示**一個偵測器，而不是訓練一個，必要時再微調。
- **可提示的基礎模型**（SAM 3 [38]）以概念提示統一了偵測、分割與追蹤。

尚待解決的問題

1. **定位品質與信心分數**。分類分數與框品質分開訓練，兩者的相關性常常不高，所以「最好」的框可能被一個更有信心、卻更差的框抑制掉。
2. **小、擁擠與被遮擋的物件**仍遠未解決 [39]。
3. **長尾與開放世界辨識**。稀有類別與真正未見過的物件，以及知道模型**何時**不知道。
4. **評估**。COCO AP 隱藏了錯誤類型，在頂端已經飽和；開放詞彙的評估則取決於提示詞，以及預訓練資料裡原本就包含了什麼 [46]。
5. **在真實硬體上的效率**，包括免 NMS 設計、利於量化的偵測頭，以及公平的延遲報告。

重點整理

- 偵測輸出一個大小不定的（框、類別、分數）三元組集合；每種架構都是對「要輸出幾個、誰負責哪個物件？」的不同回答。
- 邊界框以相對於參考框、經尺度正規化的偏移量來迴歸（t_x, t_y 與對數尺度的 t_w, t_h）；以 IoU 為基礎的損失（GIoU、DIoU、CIoU）讓訓練和以 IoU 為基礎的指標一致。
- 錨框與指派規則決定哪個預測學習哪個物件；NMS 事後移除重複，Soft-NMS 緩和它在人群中的失敗。
- FPN 處理尺度；焦點損失處理讓密集偵測器變弱的前景–背景不平衡。
- 二階段（R-CNN、Fast、Faster、Mask、Cascade）精煉候選區域；一階段（YOLO、SSD、RetinaNet）一次密集預測；無錨框（CornerNet、CenterNet、FCOS）從點預測；DETR 以匈牙利匹配預測集合，不需要 NMS。
- 開放詞彙偵測器把類別清單變成文字提示，可提示的基礎模型則融合了偵測、分割與追蹤。

- 用正確的協定量測：VOC 11 點、VOC 全點與 COCO 101 點的 AP 各不相同；COCO AP 對 IoU 0.50–0.95 取平均，APs/m/l 則揭露尺度上的弱點。

練習

1. **手算編碼**。一個錨框中心為 $(100, 100)$、大小為 64×32。一個真值框中心為 $(110, 96)$、大小為 80×40。在不做正規化的情況下計算 (t_x, t_y, t_w, t_h)，再把它們解碼回去。

   提示

   $t_x = 10/64 = 0.156$，$t_y = -4/32 = -0.125$，$t_w = \log(80/64) = 0.223$，$t_h = \log(40/32) = 0.223$。解碼：$c_x = 100 + 0.156 \cdot 64 = 110$，$w = 64e^{0.223} = 80$，$y$ 與 h 同理。

2. **IoU 與平移**。證明兩個相同的 $a \times a$ 正方形沿單一軸平移 d 時，$\text{IoU} = (a - d)/(a + d)$。平移多少時 IoU 為 0.5 與 0.75？再對兩軸等量平移的情況重做一次（圖 2 右）。

   提示

   交集為 $a(a - d)$，聯集為 $2a^2 - a(a - d) = a(a + d)$。IoU 0.5 時 $d = a/3$；IoU 0.75 時 $d = a/7$。斜向平移時 $\text{IoU} = (a - d)^2/(2a^2 - (a - d)^2)$，所以 IoU 0.5 時 $d = a(1 - \sqrt{2/3}) \approx 0.18a$。

3. **貪婪 NMS 何時失敗**。修改 NMS 範例，讓兩個不同的人以 IoU 0.6 重疊，然後分別以門檻 0.5 與 0.7 執行貪婪 NMS，以及 Soft-NMS。哪一種能找回兩個人？高門檻又帶來什麼新問題？

   提示

   門檻 0.5 時，分數較低的那個人被刪掉。門檻 0.7 時兩人都留下，但同一個人 IoU 介於 0.5 到 0.7 的重複框也會留下，增加誤報。Soft-NMS 以降低後的分數保留第二個人（原分數乘上 $e^{-0.36/0.5} \approx 0.49$），在 AP 中仍會計入召回。

4. **焦點損失的數字**。在 $\gamma = 2$、$\alpha = 0.25$ 下，對一個背景錨框計算 $p = 0.01$（簡單）與 $p = 0.6$（困難）時的焦點損失與一般交叉熵。兩者各自差了多少倍？

   提示

   背景時 $p_t = 1 - p$，$\alpha_t = 0.75$。簡單：$\text{CE} = -\log 0.99 = 0.01005$；$\text{FL} = 0.75 \cdot 0.01^2 \cdot 0.01005 \approx 7.5 \times 10^{-7}$，約小了 1.3×10^4 倍。困難：$\text{CE} = -\log 0.4 = 0.916$；$\text{FL} = 0.75 \cdot 0.6^2 \cdot 0.916 \approx 0.247$，只小了約 3.7 倍。

5. **協定之間的差異**。使用 AP 程式碼，（藉由改變亂數種子或抖動量）找出 VOC 11 點 AP **高於**全點 AP 的情況，以及低於的情況。說明為什麼兩者沒有固定的大小關係。

   提示

   11 點規則在固定的召回值取樣包絡線，所以若包絡線剛好在取樣點右側很高、之後馬上下降，就會高估；反之則低估。上面的輸出已經同時呈現兩種情形：IoU 0.5 時 11 點的值較低，0.75 時較高。

6. **從一個物件到多個物件**。把小型 PyTorch 偵測器擴充到最多三個形狀的影像。方案 A：像 YOLO v1 一樣預測 8×8 的（物件分數、框）網格，把每個物件指派給其中心所在的格子。方案 B：預測 5 個（類別、框）名額，並像 DETR 一樣以匈牙利匹配訓練。比較兩者的失敗模式。

提示

方案 A 在兩個中心落在同一格時會失敗，也需要 NMS 去除相鄰格子的重複框。方案 B 不需要 NMS，但訓練較慢，而且訓練初期匹配不穩定（同一個物件可能在不同步驟被指派給不同名額）。對 detach 後的成本矩陣使用 `scipy.optimize.linear_sum_assignment`。

偵測結果是追蹤的原料。下一步——在影片影格之間串連邊界框並維持身分——請見DLCV 06・物件追蹤。

參考文獻

1. Z. Zou, K. Chen, Z. Shi, Y. Guo and J. Ye, "Object detection in 20 years: A survey," *Proceedings of the IEEE*, vol. 111, no. 3, 2023. doi
2. L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu and M. Pietikäinen, "Deep learning for generic object detection: A survey," *International Journal of Computer Vision*, 2020. doi
3. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, "The PASCAL Visual Object Classes (VOC) challenge," *International Journal of Computer Vision*, vol. 88, 2010. doi
4. T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick et al., "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014. doi
5. P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE CVPR*, 2001. doi
6. N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE CVPR*, 2005. doi
7. P. F. Felzenszwalb, R. B. Girshick, D. McAllester and D. Ramanan, "Object detection with discriminatively trained part-based models," *IEEE TPAMI*, 2010. doi
8. R. Girshick, J. Donahue, T. Darrell and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE CVPR*, 2014. arXiv
9. R. Girshick, "Fast R-CNN," in *Proc. IEEE ICCV*, 2015. arXiv
10. S. Ren, K. He, R. Girshick and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE TPAMI*, 2017 (conference version 2015). doi
11. H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *Proc. IEEE/CVF CVPR*, 2019. arXiv
12. Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI*, 2020. arXiv
13. N. Bodla, B. Singh, R. Chellappa and L. S. Davis, "Soft-NMS — Improving object detection with one line of code," in *Proc. IEEE ICCV*, 2017. arXiv
14. T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE CVPR*, 2017. doi
15. T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE ICCV*, 2017. doi

16. K. He, G. Gkioxari, P. Dollár and R. Girshick, “Mask R-CNN,” in *Proc. IEEE ICCV*, 2017. doi
17. Z. Cai and N. Vasconcelos, “Cascade R-CNN: Delving into high quality object detection,” in *Proc. IEEE/CVF CVPR*, 2018. doi
18. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE CVPR*, 2016. doi
19. J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” arXiv:1612.08242, 2016. arXiv
20. J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” tech. report, arXiv:1804.02767, 2018. arXiv
21. A. Bochkovskiy, C.-Y. Wang and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” arXiv:2004.10934, 2020. arXiv
22. C.-Y. Wang, A. Bochkovskiy and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” in *Proc. IEEE/CVF CVPR*, 2023. doi
23. C.-Y. Wang, I-H. Yeh and H.-Y. M. Liao, “YOLOv9: Learning what you want to learn using programmable gradient information,” in *Proc. ECCV*, 2024. doi
24. A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin et al., “YOLOv10: Real-time end-to-end object detection,” in *Proc. NeurIPS*, 2024. arXiv
25. Y. Tian, Q. Ye and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” arXiv:2502.12524, 2025. arXiv
26. W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu and A. C. Berg, “SSD: Single shot multibox detector,” in *Proc. ECCV*, 2016. arXiv
27. H. Law and J. Deng, “CornerNet: Detecting objects as paired keypoints,” in *Proc. ECCV*, 2018. doi
28. X. Zhou, D. Wang and P. Krähenbühl, “Objects as points,” arXiv:1904.07850, 2019. arXiv
29. Z. Tian, C. Shen, H. Chen and T. He, “FCOS: Fully convolutional one-stage object detection,” in *Proc. IEEE/CVF ICCV*, 2019. arXiv
30. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov and S. Zagoruyko, “End-to-end object detection with transformers,” in *Proc. ECCV*, 2020. doi
31. H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, 1955. doi
32. X. Zhu, W. Su, L. Lu, B. Li, X. Wang and J. Dai, “Deformable DETR: Deformable transformers for end-to-end object detection,” in *Proc. ICLR*, 2021. arXiv
33. H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. M. Ni and H.-Y. Shum, “DINO: DETR with improved denoising anchor boxes for end-to-end object detection,” arXiv:2203.03605, 2022. arXiv
34. Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang et al., “DETRs beat YOLOs on real-time object detection,” in *Proc. IEEE/CVF CVPR*, 2024. doi
35. M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn et al., “Simple open-vocabulary object detection with vision transformers,” in *Proc. ECCV*, 2022. arXiv
36. S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang et al., “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *Proc. ECCV*, 2024. doi
37. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang et al., “YOLO-World: Real-time open-vocabulary object detection,” in *Proc. IEEE/CVF CVPR*, 2024. doi
38. N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu et al., “SAM 3: Segment anything with concepts,” arXiv:2511.16719, 2025. arXiv
39. G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng et al., “Towards large-scale small object detection:

Survey and benchmarks," *IEEE TPAMI*, vol. 45, no. 11, 2023. arXiv
40. COCO Consortium, "COCO detection evaluation," official evaluation description (12 metrics, 101-point interpolation). link
41. M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, "The PASCAL Visual Object Classes challenge: A retrospective," *International Journal of Computer Vision*, vol. 111, 2015. doi
42. S. Shao, Z. Li, T. Zhang, C. Peng et al., "Objects365: A large-scale, high-quality dataset for object detection," in *Proc. IEEE/CVF ICCV*, 2019. doi
43. A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin et al., "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," *International Journal of Computer Vision*, 2020. arXiv
44. A. Gupta, P. Dollár and R. Girshick, "LVIS: A dataset for large vocabulary instance segmentation," in *Proc. IEEE/CVF CVPR*, 2019. doi
45. T. Shehzadi, K. A. Hashmi, M. Liwicki, D. Stricker and M. Z. Afzal, "Object detection with transformers: A review," *Sensors*, vol. 25, no. 19, 2025. doi
46. C. Zhu and L. Chen, "A survey on open-vocabulary detection and segmentation: Past, present, and future," *IEEE TPAMI*, 2024. doi

DLCV 04・深度影像復原：低階視覺

同一個退化模型、許多個任務：深度網路如何去除模糊、雜訊、低解析度、霧、雨、雪與暗光，從 SRCNN、DnCNN 一路到 Restormer、all-in-one 模型與擴散先驗。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-04-image-restoration/

你將學到

- 什麼是「低階視覺」(low-level vision)，以及為什麼超解析度、去雜訊、去模糊、除霧、除雨、除雪與低光增強其實是**同一個問題**：反轉一個退化 $y = \mathcal{D}(x) + n$。
- 深度影像復原在實務上怎麼設定：成對資料、**合成退化流程**（synthetic degradation pipeline），以及它與真實世界（盲）退化之間的落差。你會用 PyTorch 實作一條退化流程，並訓練一個小小的殘差去雜訊網路。
- 各任務的里程碑與目前主流家族，從 SRCNN、DnCNN、DehazeNet 與 GoPro 去模糊基準，到 SwinIR、Restormer、NAFNet、MambaIR 與以擴散模型為基礎的超解析度（SUPIR、OSEDiff）。
- **All-in-one** 模型（AirNet、TransWeather、PromptIR、AdaIR、DA-CLIP、InstructIR）如何用一個網路處理未知或混合的退化。
- 該用哪些損失函數與評估指標、為什麼 PSNR 和「看起來真實」會互相矛盾（**感知–失真權衡**，perception–distortion trade-off），以及 NTIRE 挑戰賽如何追蹤這個領域。

先看全貌

低階視覺處理的是像素本身。在系統能辨認人臉、讀出路牌或偵測車輛之前，影像得先清楚到「看得見」。影像復原模型就是流程中負責這一步的部分：拿到一張受損的照片（太小、雜訊多、模糊、起霧、下雨、太暗），輸出對乾淨場景更好的估計。

> **用最簡單的話說**
>
> 想像有人拿給你一張被洗衣機洗過的照片：糊掉了、有斑點、也褪色了。你這輩子看過成千上萬張正常的照片，所以你猜得出它**原本**長什麼樣子：人臉有兩隻眼睛、建築物的邊是直的、天空是平滑的。復原網路做的是同一件事。它看過幾百萬組「洗之前／洗之後」的照片，學會了乾淨照片長什麼樣、洗衣機又會怎麼破壞它，再同時用這兩種知識把損傷還原。

為什麼重要：每支手機相機在你看到照片之前，都已經做過去雜訊、去馬賽克（demosaicking）與超解析度。衛星、顯微鏡、醫學與監視影像在量測之前都要先復原。影音串流服務會把低位元率的畫面放大。自駕車需要偵測器在雨霧中照常運作。從研究的角度看，許多架構想法也是先在復原任務上被驗證的：殘差學習、通道注意力、高效率 Transformer 與擴散先驗，都有知名的復原版本。

關於本篇

本篇改寫並擴充自作者先前的文章〈Low-Level Vision Task-Image Restoration 簡介〉[1]。保留原文的順序與觀點，把 2026 年的更新與更正直接寫進正文，並補上數學、程式碼與綜述文獻。

本篇假設你已熟悉 CNN 與反向傳播（DLCV 01），以及傳統影像復原（IP 05），後者詳細介紹了雜訊模型、Wiener 濾波與退化模型。這裡會直接連結過去，不再重複。

什麼是低階視覺？

白話版。高階視覺問的是「圖裡有什麼？」（分類、偵測）；低階視覺問的是「這張圖逐個像素來看，實際上長什麼樣？」前者關於**理解**，後者關於**感知**。

作者在原文 [1] 中的比喻，來自人與外界互動的方式：我們先**感知**（光打在視網膜上，看見邊緣、顏色與對比），再**理解**（那是一個杯子）。兩者缺一不可。如果感知很差（杯子很模糊），理解也會跟著受影響。低階視覺任務就是為了讓感知重新變好而存在：當我們說一個東西「很模糊」，代表感官對它的判定並不靈敏，於是就有了把它變「清晰」的去模糊任務。

精確版。低階視覺任務把一張影像映射成內容（通常）相同的另一張影像：

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \mathbb{R}^{H'\times W'\times C'},$$

輸出是逐像素的估計，而不是一個標籤。H, W, C 是輸入的高、寬與通道數，H', W', C' 是輸出的（超解析度倍率為 s 時 $H' = sH$，其他情況通常相等）。這帶來三個影響整個領域的特性：

1. **輸出密集且高解析度**。每個像素都重要，所以架構不能大幅降採樣，必須保留細節，這讓高解析度下的運算成本很高。
2. **監督訊號很容易合成**。拿一張乾淨影像，用已知的方式把它弄壞，就得到一組輸入–目標配對。這是相對於辨識任務（需要人工標註）的巨大優勢，也是這個領域最大弱點的來源：合成的損傷終究不是真實的損傷。
3. 「**正確答案**」**並不唯一**。很多張乾淨影像都和同一張受損影像相容。哪一張「最好」，取決於你重視的是忠於真實還是看起來自然，後面的評估章節會詳細討論。

這類任務範圍很廣：去模糊、超解析度、去雜訊、修補（inpainting）、色彩校正、紅外線與可見光融合等等。過去大家常在 Papers with Code 上依任務瀏覽分類與排行榜，但它已停止營運，原網址現在會直接轉址到 Hugging Face 的 Trending Papers 頁面，依任務分類的 SOTA 頁面已無法瀏覽。現在追蹤進展的可靠方式，是每年的 **NTIRE** 挑戰賽報告（New Trends in Image Restoration and Enhancement，CVPR 的一個 workshop），本篇接近結尾時會介紹 [2], [3]。

一個問題、許多面貌：退化模型

白話版。本篇的每一個任務都可以寫成：「相機看到乾淨的場景 x，某種東西破壞了它，我們拿到的是 y。」改變的只有那個「破壞」。只要能把破壞寫下來，就能說出這是哪個任務。

統一的觀點

$$y = \mathcal{D}(x) + n,$$

其中 x 是未知的乾淨影像，$\mathcal{D}$ 是退化運算子（可能是非線性的，也可能未知），n 是雜訊，y 是觀測到的影像。復原就是從 y 估計 x。以下是原文 [1] 與本篇後面用到的具體形式，圖 1 把每一種套用在同一張照片上。

模糊（課本上的模型）。和 IP 05 一樣，

$$g(u,v) = (h * f)(u,v) + n(u,v),$$

其中 f 是清晰影像，h 是點擴散函數（模糊核），$*$ 是二維卷積，n 是加性雜訊，g 是觀測影像，(u, v) 是像素座標。去雜訊是 $h = \delta$（單位脈衝，也就是沒有模糊）的特例。

超解析度。低解析度影像是高解析度影像經過模糊、次取樣再加上雜訊的結果：

$$y = (x * k) \downarrow_s + n,$$

其中 k 是模糊（抗鋸齒）核，$\downarrow_s$ 表示每個方向每 s 個像素取一個，s 是放大倍率（2、3、4.....）。經典的「bicubic」基準設定則把這一切換成 MATLAB 風格的雙三次降採樣，並且不加雜訊。

壓縮。$y = \mathrm{JPEG}_q(x)$，q 是品質因子。JPEG 會量化 8×8 區塊的 DCT 係數（見 IP 08），產生區塊效應與振鈴。它既非加性也非線性，所以通常直接當成黑盒子放進退化流程。

霧（大氣散射模型）。

$$I(p) = J(p)\, t(p) + A\,(1 - t(p)), \qquad t(p) = e^{-\beta d(p)},$$

其中 p 是像素位置，I 是有霧影像，J 是無霧的場景輻射，A 是全域大氣光（霧的顏色），$t \in (0, 1]$ 是透射率，β 是散射係數，d 是場景深度。第一項是被衰減的場景；第二項是被散射進相機的「空氣光」（airlight）。越遠的物體（d 大）t 越小，逐漸融入 A。

雨。雨線（rain streak）常用加性模型：

$$I = B + R,$$

其中 B 是無雨背景，R 是雨層。附著在鏡頭或擋風玻璃上的雨滴（raindrop）行為不同：它會遮住並折射背景，所以常用的模型是 $I = (1 - M) \odot B + R$，其中 M 是雨滴的二值遮罩，$\odot$ 是逐元素乘積，R 是穿過雨滴的光 [39]。

低光（Retinex）。Retinex 理論 [5] 把影像分解成反射率與光照：

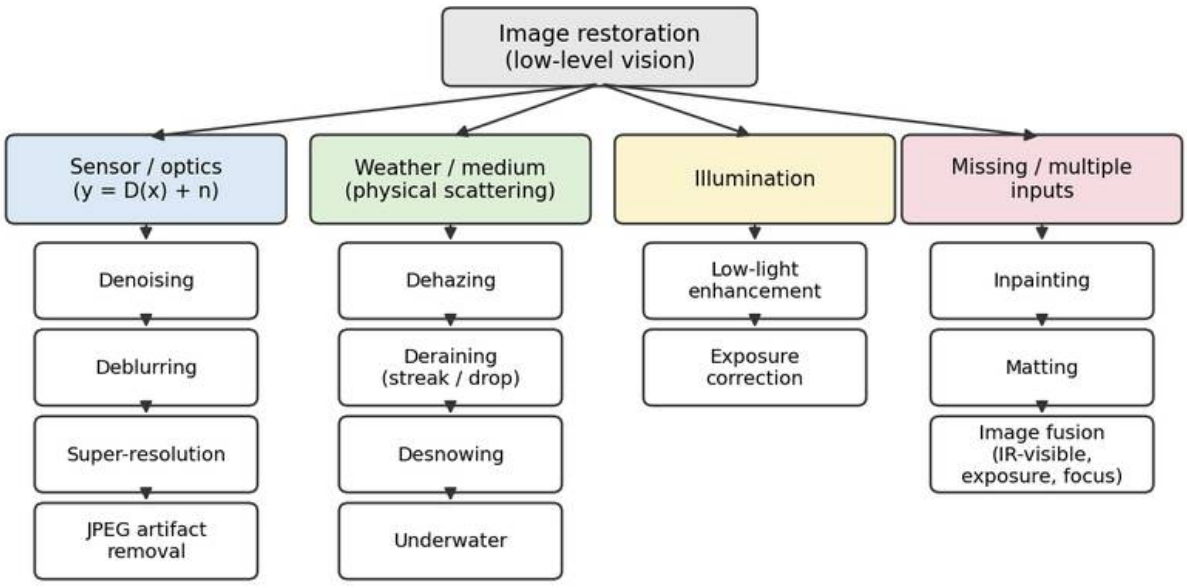
$$S = R \cdot L,$$

其中 S 是觀測影像，R 是反射率（物體表面本身的顏色與紋理，這是我們要的），L 是光照，$\cdot$ 是逐元素乘積。暗照片的 L 很小；增強就是估計 L 並把它提亮，同時原本藏在暗處的雜訊也會被放大。



圖 *1* — 同一張乾淨影像 x 經過本節各種退化模型。每個復原任務都想還原其中一種（或幾種）退化。

這些模型都不完全精確，卻有相同的形狀：乾淨影像進去，運算子加上雜訊出來。這正是原文 [1] 的核心觀點：雖然不是很嚴謹，但透過退化模型來看，它們本質上都可以用同一種方式「理解」，**屬於同一種問題**，同一套工具（而且常常是同一個網路）都適用。圖 2 依照損傷的來源把它們分類。



All-in-one models try to cover several boxes with one network

圖 2 — 依退化來源整理的低階視覺任務分類。*All-in-one* 模型試圖用單一網路涵蓋多個方框。

為什麼困難：病態問題

上面幾乎每一種 $\mathcal{D}$，都有許多不同的 x 會產生相同的 y。降採樣 4 倍丟掉每 16 個像素中的 15 個；模糊把高頻壓到幾乎為零；$t \approx 0$ 的濃霧會抹去遠處的物體。這個反問題是**病態的**（ill-posed），我們需要一個關於乾淨影像長相的**先驗**（prior）。用貝氏（MAP）形式寫：

$$\hat{x} = \arg\min_{x} \ \tfrac{1}{2}\|y - \mathcal{D}(x)\|_2^2 + \lambda\, \Phi(x),$$

第一項（資料保真項）要求 $\hat{x}$ 必須能解釋觀測，Φ 是編碼先驗的正則項（自然的影像 Φ 小），$\lambda > 0$ 平衡兩者。整個領域的歷史，就是一部尋找更好先驗 Φ 的歷史。

深度學習之前：手工先驗的時代

在深度學習興起之前，每個任務都有自己精心推導的先驗，成敗取決於對問題是否有足夠充分的物理或統計知識（prior knowledge）。去雜訊用局部平滑、非局部自相似與稀疏性；去模糊用 Wiener 與限制最小平方濾波，加上重尾的梯度先驗；除霧用暗通道先驗 [25]；超解析度用插值與以範例為基礎的字典；低光增強用直方圖等化與 Retinex 分解。IP 05 涵蓋了這套工具的核心（雜訊模型、平均／中位數／自適應濾波、反濾波、Wiener 與 CLS 濾波），結尾也簡短回顧了走向學習式去雜訊的歷史。作者在 [1] 中的看法是：深度學習逐漸解決並超越這些模型能做到的程度之後，關鍵不再是為每個任務建立強大的理論先驗，而是強大的**學習機制與特徵擷取方法**。退化模型並沒有消失，它從求解器搬進了**資料產生器**。

深度學習的問題設定

白話版。做出大量（受損, 乾淨）的影像對，訓練網路把受損的變成乾淨的。大部分的苦工都在讓損傷看起來夠真實。

以成對資料做監督式訓練

給定乾淨影像集合 $\{x_i\}$ 與一個退化取樣器，建立配對 (y_i, x_i)，其中 $y_i = \mathcal{D}_i(x_i) + n_i$，然後求

$$\theta^\star = \arg\min_{\theta} \ \frac{1}{N}\sum_{i=1}^{N} \ell(f_\theta(y_i),\, x_i),$$

其中 f_θ 是參數為 θ 的復原網路，N 是訓練對數量，ℓ 是損失函數（L1、L2、感知、對抗……見損失函數一節）。訓練時使用隨機裁切的小塊（patch，例如 64×64 到 256×256）並搭配翻轉與旋轉，因為卷積與視窗注意力網路具有平移等變性，而全解析度影像也放不進 GPU 記憶體。

有一個事實可以解釋後面很多現象：用 L2 損失時，最佳網路輸出的是**後驗平均** $f^\star(y) = \mathbb{E}[x \mid y]$。當許多清晰影像都與 y 相容時，它們的平均是模糊的。這就是為什麼以 MSE 訓練的超解析度看起來很平滑，也是後來引入 GAN 與擴散模型的原因。

合成退化流程

配對資料有三種來源。(1) **合成**：用已知模型（雙三次降採樣、高斯雜訊、高斯模糊、JPEG）弄壞乾淨影像。便宜且無限量，但受限於模型有多真實。(2) **實拍**：同一個場景拍兩次，例如短曝光與長曝光，或把連拍平均成乾淨參考（如真實雜訊資料集 SIDD [22]），或把高速攝影的連續影格平均，得到真實的動態模糊（GoPro [44]）。真實但昂貴，且侷限於特定相機。(3) **非成對／自監督**：只用受損影像學習，靠 GAN、循環一致性或雜訊統計。今天大多數強模型都用豐富退化流程產生的合成配對訓練，再同時在合成與實拍資料上評估。

典型的「經典」流程是模糊 → 降採樣 → 雜訊 → JPEG，模仿相機與圖片分享網站依序破壞照片的順序：

```
import numpy as np, cv2, torch, torch.nn.functional as F
from skimage import data, img_as_float32

def gaussian_kernel(sigma, size=21):
    ax = torch.arange(size) - size // 2
    g = torch.exp(-ax**2 / (2 * sigma**2))
    k = torch.outer(g, g)
    return k / k.sum()

def degrade(x, scale=4, sigma_blur=1.6, sigma_noise=0.03, jpeg_q=40, rng=None):
    """x: float tensor (C,H,W) in [0,1]. Returns the low-quality image y = D(x) + n."""
    rng = rng or np.random.default_rng(0)
    C = x.shape[0]
    k = gaussian_kernel(sigma_blur).to(x)[None, None].repeat(C, 1, 1, 1)
    y = F.conv2d(F.pad(x[None], (10,) * 4, mode="reflect"), k, groups=C)   # 1) 模糊  h * x
    y = y[..., ::scale, ::scale]                                           # 2) 降採樣  (.)↓s
    y = y + sigma_noise * torch.from_numpy(rng.standard_normal(y.shape).astype(np.float32))
     ↪  # 3) 雜訊 n
    y = y.clamp(0, 1)[0]
    img = (y.permute(1, 2, 0).numpy() * 255).round().astype(np.uint8)      # 4) JPEG
     ↪  編碼再解碼
    ok, buf = cv2.imencode(".jpg", img[..., ::-1], [cv2.IMWRITE_JPEG_QUALITY, jpeg_q])
    img = cv2.imdecode(buf, cv2.IMREAD_COLOR)[..., ::-1]
    return torch.from_numpy(img.copy()).permute(2, 0, 1).float() / 255

x = torch.from_numpy(img_as_float32(data.astronaut())).permute(2, 0, 1)    # (3, 512, 512)
y = degrade(x)
print(x.shape, "->", y.shape)        # torch.Size([3, 512, 512]) -> torch.Size([3, 128, 128])
```

實際訓練時，每個參數（模糊核的寬度與形狀、雜訊強度與種類、JPEG 品質，甚至順序）都會**對每張影像隨機取樣**，讓網路看到的是一整個退化分布，而不是單一一種。

真實世界與盲退化

只用一種固定 $\mathcal{D}$（例如雙三次 ×4）訓練的模型，遇到真實照片就會失效，因為真實退化是未知的、隨位置變化的，而且是多種效應的混合。這就是**盲**（blind，或稱真實世界）復原問題。大致有兩種策略：

- **把合成分布做得夠寬，寬到能涵蓋真實情況**。 Real-ESRGAN [4] 把經典流程**連續套用兩次**（「高階」退化，每一輪都隨機取樣模糊核、縮放、雜訊與 JPEG），並加入 sinc 濾波器來模擬振鈴與過衝。即使只用合成資料訓練，它在真實低品質影像上的泛化也比雙三次訓練的模型好得多。
- **引入強大的生成先驗**。預訓練的文字生成影像擴散模型已經知道自然影像長什麼樣。DiffBIR [16]、SUPIR [17] 與 OSEDiff [18] 都以受損輸入作為條件來驅動這類模型，讓先驗「補上」合理的細節。這是真實世界超解析度近年的主要趨勢，後面會再討論。

第三種互補的策略是**估計退化本身**（雜訊強度、模糊核、退化種類），再餵給網路，例如 FFDNet 的雜訊強度圖 [21] 與各種 all-in-one 模型。在盲超解析度中，未知的主要是 $y = (x * k)\downarrow_s + n$ 裡的模糊核 k，這條路線發展出自己的一系列方法：

- **以模糊核作為網路的條件**。 SRMD [87] 把退化當作額外輸入交給單一 SR 網路：模糊核先降成低維向量，連同雜訊強度一起展開成與低解析度影像同尺寸的圖，再與影像串接。這樣一個網路就能處理許多種模糊核與雜訊強度，前提是有人提供這些資訊。USRNet [91] 更進一步採用

  深度展開（deep unfolding）：把半二次分裂（half-quadratic splitting）的 MAP 求解器展開成網路，其中資料模組以封閉形式使用給定的模糊核、倍率與雜訊強度，先驗模組則是學習出的去雜訊器，所以一個模型就能處理任意已知的模糊核。
- **估計模糊核，再修正它**。 IKC [88] 觀察到錯誤的模糊核會造成特徵性的瑕疵（太大會過度銳化，太小會過度平滑），並利用這些瑕疵迭代修正初始的模糊核估計，同時由一個以模糊核為條件的 SR 網路（SFTMD）還原影像。DAN [90] 把這個交替過程展開成一個端到端網路：估計器（estimator）同時根據低解析度輸入與目前的 SR 結果預測模糊核，還原器（restorer）用目前的模糊核做超解析度，兩者輪流進行，和上面去模糊一節的傳統迴圈是同一個想法。
- **只從影像本身估計模糊核**。 KernelGAN [89] 在測試時只用那一張測試影像訓練一個小 GAN：生成器學著把影像縮小，讓判別器分不出縮小後影像的區塊與原圖的區塊。由於自然影像的區塊會跨尺度重複出現，學到的縮小運算就是這張影像自己的 SR 模糊核估計，再交給非盲 SR 方法使用。

明確估計模糊核的方法具可解釋性，而且在退化真的是「模糊、降採樣、雜訊」且模糊核大致均勻時很準確。若不是如此（JPEG、相機處理流程、隨位置變化的模糊），它們就很脆弱，而且模糊核的估計誤差會傳到結果裡。這促成了**隱式**的路線：BSRGAN [92] 用一個實用的退化模型訓練，模糊（等向與非等向高斯核）、降採樣、雜訊與 JPEG 等步驟以隨機順序組合；Real-ESRGAN [4] 用它的高階退化流程；兩者在測試時都不估計模糊核。擴散先驗方法 [16]–[18] 也屬於隱式。Liu 等人的綜述 [93] 依照如何建模退化、以及使用哪些資料（外部資料集或單張測試影像）來整理盲 SR 方法，並在合成與真實影像上比較它們。

超解析度（Super-Resolution, SR）

白話版。把小圖變大**而且**變清楚，也就是預測缺少的高解析度像素，而不只是把現有的像素拉伸開來。

問題設定

單張影像超解析度（SISR）在 $y = (x * k)\downarrow_s + n$ 之下，從 $y \in \mathbb{R}^{H\times W\times C}$ 估計 $x \in \mathbb{R}^{sH\times sW\times C}$。傳統做法是**插值法**（interpolation）：最近鄰（nearest neighbor）、雙線性（bilinear）與雙三次（bicubic）。其中雙三次插值以 4×4 鄰域擬合三次多項式，最為知名，至今仍被廣泛使用，既是基準方法，也是產生低解析度測試輸入的標準方式。插值只會重新分配既有資訊，無法重建被 $\downarrow_s$ 丟掉的高頻。

網路怎麼「變大」：三種上採樣策略

輸出比輸入大，所以每個 SR 網路都得決定解析度**在哪裡**提升。

a. 先上採樣（先插值）。 SRCNN [6] 先用雙三次插值把輸入放大到目標尺寸，再接三層卷積（區塊擷取、非線性映射、重建）。簡單直觀，但每一層都在高解析度上運算，速度慢，網路還得清理插值造成的瑕疵。

b. 轉置卷積（反卷積，deconvolution）。 FSRCNN [7] 在低解析度上運算，只在最後用學習得到的轉置卷積放大，速度比 SRCNN 快很多。步長無法整除核大小的轉置卷積會產生不均勻的重疊，容易出現**棋盤狀瑕疵**（checkerboard artifact）[9]，所以現在很少被當作 SR 的最終上採樣器；不過在其他類型的神經網路中（例如分割與生成模型的解碼器）仍然相當常見。

c. 子像素卷積（pixel shuffle）。 ESPCN [8] 在低解析度上計算 Cs^2 個通道，再重新排列成 s 倍大的影像：

$$\mathrm{PS}(T)_{c,\,sh+i,\,sw+j} = T_{cs^2+si+j,\,h,\,w}, \qquad 0 \le i,j < s,$$

其中 T 是有 Cs^2 個通道的低解析度特徵張量，(h,w) 是低解析度位置，(i,j) 是子像素偏移，c 是輸出通道。所有學習都在低解析度進行，重新排列本身沒有參數。這是現代 SR 網路的主流選擇（`torch.nn.PixelShuffle`）。

里程碑

- **SRCNN**（2014/2016）[6]：第一個端到端的 SR CNN，在雙三次放大的輸入上接三層卷積。
- **FSRCNN**（2016）[7] 與 **ESPCN**（2016）[8]：後上採樣，達到即時速度。
- **EDSR**（2017）[10]：拿掉批次正規化（BN 會削弱復原任務所需的數值範圍彈性）的深層殘差網路，用殘差縮放穩定訓練，並有多尺度版本。它贏得 NTIRE 2017 SR 挑戰賽。
- **RCAN**（2018）[11]：非常深的「殘差中的殘差」結構，加上**通道注意力**（channel attention），用全域統計量重新加權特徵通道，讓網路專注於高頻資訊。

- **SRGAN**（2017）[12] 與 **ESRGAN**（2018）[13]：對抗損失與感知（VGG 特徵）損失產生銳利、逼真的紋理，而不是 MSE 的模糊平均。ESRGAN 改良了生成器（不含 BN 的 Residual-in-Residual Dense Block）、使用相對判別器（relativistic discriminator），並在激活前的特徵上計算感知損失。

大約在 2018 年，這些模型的結果在人眼看來已經遠勝插值，領域也開始質疑該怎麼**評判** SR，評判標準本身的界定引起更多疑慮（見感知–失真一節）。

Transformer、真實世界 SR 與擴散模型（目前的主流家族）

- **Transformer**。 SwinIR [14]（2021）以殘差 Swin Transformer 區塊（在位移的局部視窗內做自注意力）處理 SR、去雜訊與 JPEG 去壓縮瑕疵。HAT [15]（CVPR 2023）結合通道注意力、視窗自注意力與重疊交叉注意力模組，讓每個輸出「啟動更多像素」。兩者至今仍是常見的比較基準。
- **狀態空間模型**。 MambaIR [49] 與 MambaIRv2 [50] 以選擇性狀態空間（Mamba）掃描取代注意力，以線性成本取得全域感受野；見骨幹網路一節。
- **真實世界（盲）SR**。前面介紹過的 Real-ESRGAN [4] 與其高階退化流程。
- **擴散先驗**。 DiffBIR [16] 先去除退化，再用預訓練的潛在擴散模型重新生成細節；SUPIR [17] 把這個做法放大，使用大型擴散骨幹、兩千萬張附文字描述的訓練影像，以及負向品質提示（negative-quality prompt）；OSEDiff [18] 把過程蒸餾成**單一步**擴散，直接從低品質影像出發，並以變分分數蒸餾（variational score distillation）維持輸出的真實感。這些方法在真實照片上能產生更有說服力的紋理，但也可能捏造出場景中不存在的細節。

資料集與指標

標準訓練集是 **DIV2K** [19]（為 NTIRE 2017 推出的 2K 解析度影像）。評估使用幾個經典的小型測試集（自然、都市與漫畫影像），在亮度（Y）通道上報告 PSNR 與 SSIM，通常會先裁掉 s 個邊界像素。真實世界 SR 則以感知與無參考指標評估（LPIPS、NIQE 等，見評估指標一節）。

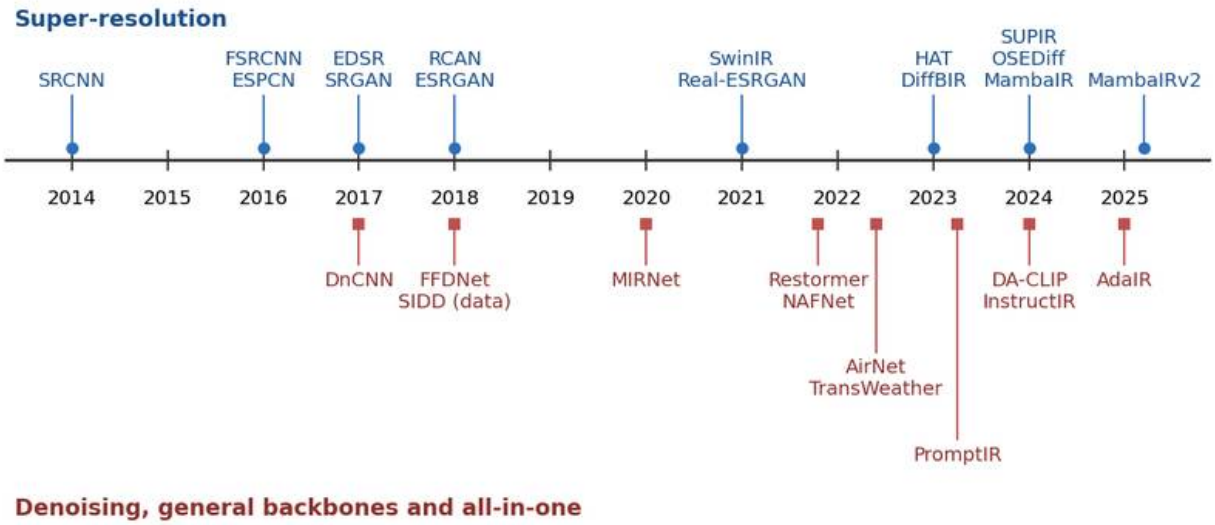


圖 3 — 超解析度（上）以及去雜訊、通用骨幹與 *all-in-one* 復原（下）的里程碑，依發表年份排列（同一年內的位置只是為了方便閱讀）。

去雜訊（Denoising）

白話版。把照片上隨機的顆粒去掉，同時不要把真正的紋理也抹平。雜訊是最單純的退化，所以去雜訊成了大多數深度復原想法最先出現的試驗場。

問題設定

$y = x + n$。合成設定中 $n \sim \mathcal{N}(0, \sigma^2 I)$（加性白高斯雜訊，AWGN，標準差 σ 通常以 0–255 的尺度表示，例如 $\sigma = 25$）。真實相機的雜訊不一樣：它和訊號相關（光子散粒雜訊隨亮度增加），去馬賽克後在空間上具相關性，還會被相機的影像處理流程改變形狀。

里程碑

DnCNN [20] 做了兩個後來成為標準的選擇。第一是**殘差學習**（residual learning）：網路 $\mathcal{R}_\theta$ 不直接預測 x，而是預測雜訊，輸出為

$$\hat{x} = y - \mathcal{R}_\theta(y),$$

其中 $\mathcal{R}_\theta(y) \approx n$。殘差（雜訊）比乾淨影像「更容易學」，使得搭配批次正規化的深層純 CNN 能訓練得很好。單一 DnCNN 也能針對一段範圍的雜訊強度訓練（盲高斯去雜訊），而且同一個網路也能處理 SR 與 JPEG 去區塊。

FFDNet [21] 把**雜訊強度圖** M（與影像同尺寸，每個像素的值為 σ）和降採樣後的輸入一起餵給網路。測試時調整 M，可以在去雜訊與保留細節之間取捨；非均勻的 M 則能處理隨位置變化的雜訊。它的速度快到可以在 CPU 上執行。

真實雜訊。 **SIDD** 資料集 [22] 提供真實的智慧型手機雜訊影像，乾淨參考影像則透過仔細拍攝連拍並加以處理而得。只用 AWGN 訓練的模型在它上面表現很差，這推動了整個領域走向真實的雜訊合成與實拍訓練資料。

通用骨幹。 MIRNet [30]、**Restormer** [23] 與 **NAFNet** [24] 是通用的復原架構，在 SIDD 以及去模糊等任務上都報告了最先進的結果，會在骨幹網路一節介紹。

程式碼：一個小小的殘差去雜訊網路

下面的程式在四張 `skimage` 影像隨機切出的 32×32 小塊上訓練一個五層、DnCNN 風格的網路，每一步都加上新的高斯雜訊，然後在一張從沒看過的影像上測試。在筆電 CPU 上大約一分鐘內就能跑完（我們在負載很重的機器上量到約一分鐘）。

```
import time, numpy as np, torch, torch.nn as nn
from skimage import data, img_as_float32
from skimage.color import rgb2gray

torch.manual_seed(0); torch.set_num_threads(2)
rng = np.random.default_rng(0)
sigma = 25 / 255                                        # 雜訊強度（[0,1] 尺度）
train_imgs = [img_as_float32(im) for im in (data.camera(), data.coins(), data.moon(),
 ↪  data.page())]
test_img = img_as_float32(rgb2gray(data.astronaut()))   # 測試用，不參與訓練

def random_patches(n=32, p=32):
    batch = []
    for _ in range(n):
        im = train_imgs[rng.integers(len(train_imgs))]
        i, j = rng.integers(0, im.shape[0] - p), rng.integers(0, im.shape[1] - p)
        batch.append(im[i:i + p, j:j + p])
    return torch.from_numpy(np.stack(batch))[:, None]   # (n, 1, p, p)

class TinyDnCNN(nn.Module):
    """conv-ReLU, (depth-2) x conv-BN-ReLU, conv。預測的是雜訊，不是影像。"""
    def __init__(self, depth=5, ch=32):
        super().__init__()
        layers = [nn.Conv2d(1, ch, 3, padding=1), nn.ReLU(inplace=True)]
        for _ in range(depth - 2):
            layers += [nn.Conv2d(ch, ch, 3, padding=1), nn.BatchNorm2d(ch),
             ↪  nn.ReLU(inplace=True)]
        layers += [nn.Conv2d(ch, 1, 3, padding=1)]
        self.body = nn.Sequential(*layers)
    def forward(self, y):
        return y - self.body(y)                         # x_hat = y - R(y)：殘差學習

net = TinyDnCNN()
```

```
opt = torch.optim.Adam(net.parameters(), lr=2e-3)
t0 = time.time()
for step in range(300):
    x = random_patches()
    y = x + sigma * torch.randn_like(x)                  # 即時產生合成配對
    loss = nn.functional.l1_loss(net(y), x)
    opt.zero_grad(); loss.backward(); opt.step()
print(f"trained in {time.time() - t0:.0f} s, last L1 = {loss.item():.4f}")

net.eval()
with torch.no_grad():
    x = torch.from_numpy(test_img)[None, None]
    y = x + sigma * torch.randn_like(x)
    x_hat = net(y).clamp(0, 1)
mse = lambda a, b: torch.mean((a - b) ** 2).item()
print(f"PSNR noisy {10*np.log10(1/mse(y.clamp(0,1), x)):.2f} dB -> denoised
 ↪  {10*np.log10(1/mse(x_hat, x)):.2f} dB")
# 例如 PSNR noisy 20.74 dB -> denoised 28.64 dB
```

即使是這個只用四張圖、訓練 300 個小步驟的玩具模型，在沒看過的照片上也能提升約 8 dB，而且勝過手動調整的高斯濾波（圖 4）。拖動滑桿，比較雜訊輸入與小網路的輸出。



雜訊（$\sigma = 25$）



小型殘差 *CNN*

滑桿比較：顆粒明顯的灰階太空人臉部，與小型 CNN 去雜訊後乾淨但略顯平滑的同一區塊。

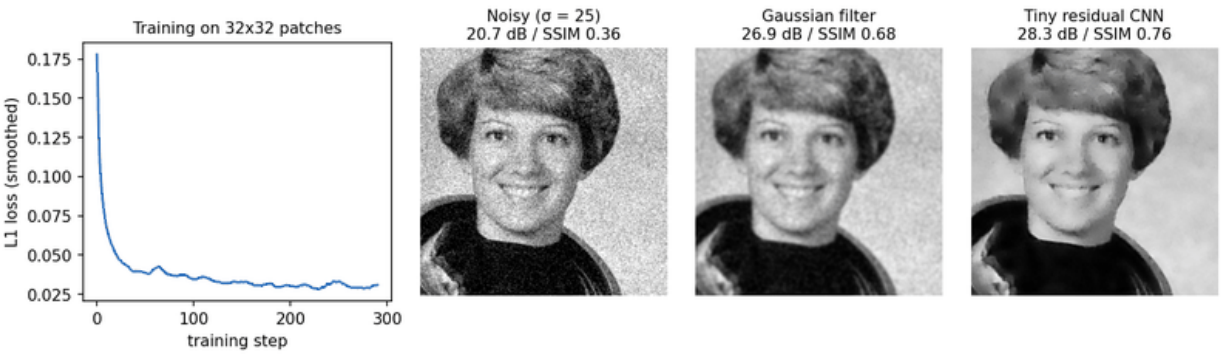


圖 *4* — 小型殘差去雜訊網路。*PSNR* 與 *SSIM* 是在完整的 *512* × *512* 測試影像上計算。*CNN* 去掉的雜訊比高斯濾波多，邊緣也更銳利，但同時把細緻紋理壓平了，這是 *MSE* ／ *L1* 訓練的典型行為。不同次執行、不同機器的數值會略有差異。

除霧（Dehazing）

白話版。去掉霧或霾造成的灰白色面紗，找回對比與顏色，遠處物體尤其明顯。用原文的說法，它是去雜訊與去模糊的一種延伸，只是多了一個物理模型。

傳統做法與暗通道先驗

傳統方法先估計 $I = Jt + A(1-t)$ 中的未知量（透射率圖 t 與大氣光 A），再代回公式求解。最困難的部分，就是想出能把這些未知量確定下來的可靠關係。最知名的是何凱明等人的**暗通道先驗**（dark channel prior）[25]：在無霧的戶外影像中，大多數局部區塊裡至少有一個像素在某個顏色通道上非常暗（陰影、深色或鮮豔的物體）：

$$J^{\text{dark}}(p) = \min_{c\in\{r,g,b\}} \Big(\min_{q\in\Omega(p)} J^c(q) \Big) \approx 0,$$

其中 $\Omega(p)$ 是以 p 為中心的小區塊，J^c 是顏色通道 c。對霧的模型做同樣的雙重取最小值，再利用 $J^{\text{dark}} \approx 0$，就得到透射率估計

$$\tilde{t}(p) = 1 - \omega \min_c \Big(\min_{q\in\Omega(p)} \frac{I^c(q)}{A^c} \Big),$$

其中 $\omega < 1$（論文中取 0.95）刻意保留一點霧，以維持深度感。接著 $\hat{J} = (I-A)/\max(\tilde{t}, t_0) + A$，小小的 t_0 用來避免除以零。這個先驗在大片與大氣光相近的明亮區域（白牆、天空）會失效，而以區塊為單位得到的粗糙 t 也必須再精修，才能貼合邊緣。

深度除霧：兩個流派

原文 [1] 把深度除霧分成兩派，這個分法至今仍很貼切。

1. 物理引導（「套公式」）。學出未知參數，再反轉模型。**DehazeNet** [26] 用 CNN（搭配 Maxout 單元與有界 ReLU）從有霧區塊迴歸出透射率 t。**AOD-Net** [27] 進一步簡化大氣散射模型，把 t 與 A 合併成單一函數 K：

$$J(p) = K(p)\, I(p) - K(p) + b, \qquad K(p) = \frac{\frac{1}{t(p)}\big(I(p) - A\big) + (A - b)}{I(p) - 1},$$

其中 b 是常數偏移。一個輕量 CNN 預測 K，於是整個網路是端到端的，還可以接在偵測器前面一起訓練。

2. 強大的通用架構（「超級模型」）。跳過物理，直接用強大的網路學 $I \mapsto J$。**GridDehazeNet** [28] 使用以注意力為基礎的多尺度網格；**FFA-Net** [29] 堆疊通道注意力與像素注意力區塊，並做自適應特徵融合。這類設計的學習能力很強，換句話說，很有機會也適用於其他退化任務，這正是 MIRNet [30] 等通用復原網路走的路。之後出現了 Transformer 除霧模型：**DehazeFormer** [31] 修改正規化層、激活函數與空間聚合方式，把 Swin Transformer 調整成適合除霧。

資料集

- **RESIDE** [32]：大型基準，包含以深度與散射模型合成的室內外有霧影像，以及用於任務導向評估的真實有霧影像。它的 SOTS 測試集是最常被引用的除霧基準。
- **O-HAZE** [33] 與 **Dense-Haze** [34]：以專業造霧機拍攝的真實有霧／無霧配對，分別是戶外場景與濃霧場景。規模小，但能突顯合成與真實之間的落差。

- **KITTI** [35] **不是**除霧資料集。它是在 Karlsruhe 一帶錄製的自駕車情境視覺基準（立體視覺、光流、里程計、3D 物件偵測、追蹤）。部分除霧研究會在它的影像上合成霧氣（它有光達與立體視覺提供的深度），或用它評估除霧對下游偵測的影響。

除雪（Desnowing）

白話版。去掉飄落的雪。看起來像除雨的親戚，但行為上也很像除霧。

除雪比看起來困難。雪花的大小與透明度差異極大，從不透明的白色團塊到半透明的條紋都有，而大雪還會造成類似霧的**朦朧效應**（veiling effect）。所以除雪模型不只要處理呈現為白點的「雪」，也得處理朦朧的霧氣。**DesnowNet** [36] 推出 **Snow100K** 資料集（在真實影像上合成雪，另附真實雪景影像），以及一個同時估計雪的半透明程度與其殘留的網路。**JSTASR** [37] 明確建模雪的大小與透明度，並加入去除朦朧效應的步驟。現在除雪通常被放進 TransWeather [52] 這類「惡劣天氣」all-in-one 模型中一起研究。

除雨（Deraining）

白話版。把照片裡的雨去掉。「雨」其實指兩種很不一樣的東西：在空中落下的細雨線，以及停在鏡頭或擋風玻璃上的雨滴。

雨線與雨滴

- **雨線**（rain streak）是疊加在場景上、細而亮、有方向性的線條，模型為 $I = B + R$。遠處的雨線累積起來會形成類似霧的面紗。方法的目標是把兩層分開。一個早期具影響力的深度方法同時偵測並去除雨 [38]，並提供了被廣泛使用的輕雨與大雨合成資料集。
- **雨滴**（raindrop）附著在玻璃表面，像一個個小透鏡：遮住並折射背景、常常失焦，而且沒有一致的方向。Qian 等人的注意力 GAN [39] 用注意力圖（雨滴在哪裡？）同時引導生成器與判別器，並推出一個透過玻璃拍攝、有無水滴成對的雨滴資料集。

走向真實場景

大多數除雨資料集是合成的，用合成雨線訓練的模型面對真實的雨常常失效。**NTIRE 2025 Day and Night Raindrop Removal for Dual-Focused Images** 挑戰賽 [40] 反映了研究重心轉向真實資料：它使用真實拍攝的 **Raindrop Clarity** 資料集，涵蓋白天與夜晚，以及對焦在雨滴與對焦在背景兩種情況（訓練 14,139 張、驗證 240 張、測試 731 張）。對焦設定很關鍵：相機對焦在雨滴上時，背景也是模糊的，於是去除雨滴有一部分變成了去模糊。

低光增強（簡介）

白話版。讓暗照片看起來曝光正常、細節可見、色調自然，同時不要把雜訊放大。

訓練時對比暗影像與同一場景正常曝光的參考影像，例如隨 **RetinexNet** [41] 推出、包含低光／正常光配對的 **LOL** 資料集。傳統方法以對比度調整（直方圖等化）與 Retinex 模型為主（較少見的還有反轉大氣散射模型）。深度方法有兩條路線：受 Retinex 啟發、分解 $S = R \cdot L$ 再調整 L 的網路（RetinexNet [41]），以及特別設計的通用網路。**Retinexformer** [42]（ICCV 2023）結合兩者：單階段的 Retinex 公式加上光照引導的注意力，作者稱它是第一個以 Transformer 為基礎的低光增強方法。低光增強本身的內容（零參考方法、曲線估計、雜訊、色彩、影片）多到值得獨立一章：見DLCV 05。

影像融合（簡介）

白話版。把同一場景、各有所長的幾張影像組合成一張，讓結果同時擁有每一張的優點，資訊更豐富多樣。

原文 [1] 提到三種常見設定：**紅外線與可見光**融合（紅外線提供熱目標，可見光相機提供紋理與色彩）、**多重曝光**融合（結合曝光不足與過度照片的細節，得到類似 HDR 的結果），以及**多焦點**融合（由對焦在不同深度的照片合成全景深影像）。與其他任務不同，融合通常沒有真實的「正確融合影像」，所以深度融合依賴保留來源影像梯度與強度的非監督式損失，或使用 GAN。Zhang 等人的綜述 [43] 依這些情境與學習策略整理了深度融合方法。

去模糊（Deblurring）

白話版。從被手震、物體移動或失焦弄糊的影像中，還原出清晰的影像。

問題設定

去模糊就是 $g = h * f + n$ 的直接應用。模糊核 h 已知時稱為**非盲**（non-blind）去模糊（DIP 第 5 章中 Wiener 與 CLS 濾波的設定，以及學習式非盲求解器）；h 未知時稱為**盲**（blind）去模糊。動態場景中的真實動態模糊更麻煩：不同物體的運動不同、深度也不同，所以根本沒有單一的全域 h。

盲去模糊：估計模糊核

白話版。如果你確切知道相機是怎麼晃的，把模糊還原回去就是一個研究得很透徹的濾波問題。盲去模糊的麻煩在於你不知道相機怎麼晃，所以必須同時猜「怎麼晃」和「清晰的照片」。訣竅是輪流猜：先用目前猜的晃動把影像變清晰，再用得到的最清楚的邊緣去改進對晃動的猜測，然後重複。

聯合問題。對空間上均勻的模糊，盲反卷積（blind deconvolution）要解

$$(\hat{f}, \hat{h}) = \arg\min_{f,h} \|h * f - g\|_2^2 + \lambda_f \, \phi(f) + \lambda_h \, \psi(h) \quad \text{subject to} \quad h \geq 0, \;\; \textstyle\sum_{u,v} h(u,v) = 1,$$

其中 g 是模糊影像，f 是未知的清晰影像，h 是未知的模糊核（只在一個小視窗內非零，手震通常是 15×15 到 75×75 像素），ϕ 是影像先驗（通常是稀疏梯度懲罰 $\sum_p |\nabla f(p)|^\alpha$，$0 < \alpha \leq 1$，因為自然影像大多是平坦區域加上少數強邊緣），$\psi$ 是模糊核先驗（例如 $\|h\|_2^2$ 或 $\|h\|_1$，偏好緊湊、稀

疏的運動軌跡），$\lambda_f, \lambda_h > 0$ 是它們的權重。限制條件表示模糊核只是把光重新分配：它非負，而且保持總亮度不變。這也消除了一種不確定性，因為對任何 $c > 0$，$(ch, f/c)$ 和 (h, f) 一樣能解釋 g。另一種不確定性則無法消除：把 f 往一個方向平移、h 往反方向平移，$h * f$ 不變，所以模糊核只能還原到差一個平移的程度。

為什麼這麼病態。未知數（f 和 h）遠多於方程式，很多組解都能完全解釋 g。更糟的是，最明顯的那一組是錯的：$h = \delta$（沒有模糊）且 $f = g$ 完美符合資料。Levin 等人 [74] 指出，在常用的稀疏梯度先驗下，模糊影像的先驗代價往往比清晰影像更低，因為模糊會讓梯度變小，而 ϕ 偏好小梯度。所以對 (f, h) 做天真的聯合 MAP 估計，常會得到「沒有模糊」的解。他們的分析指向一個較適定的目標：只估計模糊核，把影像積分掉，

$$\hat{h} = \arg\max_h \; p(h \mid g) = \arg\max_h \; p(h) \int p(g \mid f, h)\, p(f)\, df,$$

其中 $p(g \mid f, h)$ 是由資料項給出的高斯似然，$p(f)$ 和 $p(h)$ 是 ϕ 與 ψ 背後的先驗，積分則是對所有清晰影像取平均。模糊核的未知數比像素觀測少得多，所以它的後驗遠比整張影像的後驗確定。一旦知道 $\hat{h}$，再用非盲反卷積就能還原 f。

傳統的解法。實務上有效的方法，不是近似上述的邊際化，就是改變先驗，讓清晰的解勝出：

- **變分貝氏（variational Bayes）**。 Fergus 等人 [75] 用重尾的混合高斯模型描述影像梯度，以變分貝氏近似模糊核的後驗，並由粗到細估計模糊核；影像則再用標準的非盲方法還原。
- **邊緣預測與挑選**。 Shan 等人 [76] 以自然影像梯度先驗和仔細設計的雜訊模型輪流估計模糊核與影像，以抑制振鈴。Cho 與 Lee [77] 讓這個交替過程變快：他們用雙邊濾波與衝擊濾波（shock filter）從目前的潛在影像預測清晰邊緣，並以 FFT 在梯度域估計模糊核。Xu 與 Jia [78] 發現比模糊核還細的邊緣會誤導估計，所以第一階段只挑選有資訊量的邊緣，第二階段再精修模糊核的支撐範圍。
- **偏好清晰影像的先驗**。 Krishnan 等人 [79] 把 $\|\nabla f\|_1$ 換成正規化稀疏度 $\|\nabla f\|_1 / \|\nabla f\|_2$，它與尺度無關，而且會隨模糊增加，所以不再偏好模糊的解。Pan 等人 [80] 使用暗通道（與 He 等人除霧先驗 [25] 相同的量）：模糊會把暗像素和周圍的亮像素平均在一起，使暗通道變得不那麼稀疏，因此對暗通道加上 L_0 懲罰就會偏好清晰影像。

標準流程。幾乎所有傳統盲去模糊方法都遵循同一個迴圈。在影像金字塔最粗的一層，從 delta 模糊核開始，交替進行

$$f^{(t+1)} = \arg\min_f \|h^{(t)} * f - g\|_2^2 + \lambda_f\, \phi(f),$$

$$h^{(t+1)} = \arg\min_{h \geq 0,\; \sum h = 1} \|h * \nabla \tilde{f}^{(t+1)} - \nabla g\|_2^2 + \lambda_h \|h\|_2^2,$$

其中 t 是迭代次數，∇ 是影像梯度，$\tilde{f}$ 是經過邊緣預測或挑選後的潛在影像（銳化過，弱梯度設為零）。模糊核這一步在梯度域進行，因為卷積與微分可以交換，$\nabla(h * f) = h * \nabla f$，而模糊的資訊主要藏在梯度裡；搭配二次懲罰時，它在頻域有封閉解，之後再投影回限制條件。迭代幾次後，把模糊核放大到金字塔的下一層（**由粗到細**，coarse-to-fine），因為大模糊在粗尺度上只有幾個像素寬，這樣可以避開不好的局部極小值。最後固定 $\hat{h}$，用**非盲反卷積**產生輸出影像：可以是 IP 05 的 Wiener 或限制最小平方濾波，也可以是超拉普拉斯（hyper-Laplacian）先驗或學習式的非盲求解器。

深度學習與模糊核。神經網路以三種方式進入盲去模糊。

- **先預測模糊核（或濾波器），再反卷積。** Sun 等人 [81] 訓練 CNN，把每個區塊的局部模糊核分類到一組線性運動之一，用馬可夫隨機場（MRF）平滑得到的非均勻運動場，再做反卷積。Schuler 等人 [82] 建構一個深層網路，各階段模仿傳統迴圈（特徵擷取、模糊核估計、影像估計），並在合成模糊上端到端訓練。Chakrabarti [83] 訓練網路為每個區塊預測反卷積濾波器的傅立葉係數，用結果估計一個全域模糊核，最後以標準非盲反卷積收尾。
- **自監督、逐張影像估計。** Deep Image Prior [84] 指出，未經訓練的生成網路，其結構本身就是有用的影像先驗。SelfDeblur [85] 用兩個生成網路：一個編碼器–解碼器負責影像，一個輸出接 softmax 的全連接網路負責模糊核（softmax 自動滿足 $h \geq 0$ 與 $\sum h = 1$），兩者一起擬合單張模糊影像。不需要訓練資料，但每張影像都要跑一次最佳化。
- **不估計模糊核、端到端。** Nah 等人 [44] 的多尺度網路、DeblurGAN [86]（條件式 GAN 加上內容損失），以及通用骨幹 Restormer 與 NAFNet，都直接把模糊影像映射成清晰影像，從頭到尾不產生 h。

哪一種比較好？以模糊核為基礎的方法適合**空間均勻**的模糊（拍攝遠處靜態場景時的手震），提供**可解釋**的中間結果，可以檢查也可以重複使用（例如用來處理同一次曝光的其他影格），而且只要物理模型成立，就能泛化到新的模糊核；但它們較慢，遇到隨位置變化的模糊或邊緣很少的影像就會失效。不估計模糊核的網路能處理物體運動與深度造成的**非均勻**模糊，測試時很快，也是 GoPro 等動態場景基準上目前最好的方法，但它們只能泛化到訓練時看過的模糊分布，也不給出明確的模糊核。Zhang 等人的綜述 [69] 整理了這個轉變。

程式碼：一個交替式盲反卷積的玩具範例。下面的程式用一個彎曲的 15×15 手震模糊核把 256×256 的 `camera` 影像弄糊，然後交替進行封閉形式的影像步驟（帶二次梯度懲罰、類似 CLS 的濾波器）與模糊核步驟（只用經衝擊濾波後最強的 2% 梯度做最小平方，再投影到 $h \geq 0$、$\sum h = 1$）。在 CPU 上約一秒跑完。

```
import numpy as np
from numpy.fft import fft2, ifft2
from scipy.ndimage import convolve, gaussian_filter, laplace
from skimage import data, img_as_float, transform

rng = np.random.default_rng(0)
f_true = transform.resize(img_as_float(data.camera()), (256, 256), anti_aliasing=True)
K = 15                                                  # 模糊核大小 K x K
h_true = np.zeros((K, K))
for t in np.linspace(0, 1, 40):                         # 一條彎曲的「手震」軌跡
    h_true[round(3 + 8 * t), round(2 + 10 * t + 2 * np.sin(3 * t))] += 1
h_true = gaussian_filter(h_true, 0.5); h_true /= h_true.sum()
g = convolve(f_true, h_true, mode="wrap") + 0.005 * rng.standard_normal(f_true.shape)

def otf(h, shape):                                      # 置中的模糊核 -> 轉移函數
    p = np.zeros(shape); p[:h.shape[0], :h.shape[1]] = h
    return fft2(np.roll(p, (-(h.shape[0] // 2), -(h.shape[1] // 2)), (0, 1)))

G = fft2(g)
Dx, Dy = otf(np.array([[0, -1, 1]]), g.shape), otf(np.array([[0], [-1], [1]]), g.shape)

def image_step(h, lam):        # f = argmin ||h*f - g||^2 + lam ||grad f||^2 (封閉解，類似
 ↪  CLS)
    H = otf(h, g.shape)
```

```
    return np.real(ifft2(np.conj(H) * G / (np.abs(H)**2 + lam * (np.abs(Dx)**2 +
    ↪  np.abs(Dy)**2))))

def shock(f, n=5, dt=0.5):      # 邊緣預測：衝擊濾波把柔和的邊緣變成階梯
    f = gaussian_filter(f, 0.7)
    for _ in range(n):
        gy, gx = np.gradient(f)
        f = f - dt * np.sign(laplace(f)) * np.hypot(gx, gy)
    return f

def kernel_step(f, keep=0.02, beta=1e-3):   # 梯度域最小平方，只用強邊緣
    num = den = 0
    for D in (Dx, Dy):
        fx = np.real(ifft2(D * fft2(f)))
        fx[np.abs(fx) < np.quantile(np.abs(fx), 1 - keep)] = 0
        Fx = fft2(fx)
        num, den = num + np.conj(Fx) * D * G, den + np.abs(Fx)**2
    h = np.roll(np.real(ifft2(num / (den + beta))), (K // 2, K // 2), (0, 1))[:K, :K]
    h = np.maximum(h, 0)                            # 投影到 h >= 0 ……
    return h / h.sum()                              # ……以及 sum(h) = 1

def best_shift(fn):             # 盲反卷積只能還原到差一個小平移的 h（與 f）
    return min(fn(dy, dx) for dy in range(-3, 4) for dx in range(-3, 4))
kernel_error = lambda h: best_shift(lambda dy, dx: np.linalg.norm(np.roll(h, (dy, dx), (0,
↪  1)) - h_true)) / np.linalg.norm(h_true)
psnr = lambda f: -best_shift(lambda dy, dx: 10 * np.log10(np.mean((np.clip(np.roll(f, (dy,
↪  dx), (0, 1)), 0, 1) - f_true)[8:-8, 8:-8] ** 2)))

h = np.zeros((K, K)); h[K // 2, K // 2] = 1
h = gaussian_filter(h, 1.0); h /= h.sum()           # 從接近 delta 的模糊核（「沒有模糊」）開始
print(f"kernel error at start: {kernel_error(h):.2f}")
for _ in range(30):                                 # 交替：先更新影像，再更新模糊核
    h = kernel_step(shock(image_step(h, lam=0.02)))
print(f"kernel error after 30 alternations: {kernel_error(h):.2f}")
print(f"PSNR  blurry {psnr(g):.2f} dB | estimated kernel {psnr(image_step(h, 1e-2)):.2f} dB"
      f" | true kernel {psnr(image_step(h_true, 1e-2)):.2f} dB")
# kernel error at start: 1.19
# kernel error after 30 alternations: 0.57
# PSNR  blurry 22.59 dB | estimated kernel 27.11 dB | true kernel 29.67 dB
```

模糊核的相對誤差大約減半，用估計的模糊核做最後的非盲步驟，也補回了模糊輸入與「真實模糊核」結果之間大部分的差距。但要誠實看待這個玩具範例：模糊是合成的、均勻的、循環的（所以 FFT 模型完全成立），雜訊很低，參數（`lam`、`keep`、衝擊濾波的次數）也是針對這張影像調的。程式沒有影像金字塔，所以更大的模糊核會失敗；面對真實照片時，還需要處理邊界、飽和像素，並使用穩健的損失。兩種誤差都是在最佳的小平移之後才量測，原因就是上面說的平移不確定性。

資料與里程碑

Nah 等人 [44]（CVPR 2017）用 **GoPro** 資料集處理這個問題：他們以高速攝影機拍攝清晰的影片，把連續影格平均起來，得到真實的模糊影像，並以中間那一格作為清晰目標。他們的網路是

多尺度的：先還原粗略版本，再逐步在更細的尺度上精修，類似傳統盲去模糊由粗到細的策略，但完全不需要估計模糊核。GoPro 至今仍是最常用的去模糊基準。

今天最常被比較的基準是通用骨幹 **Restormer** [23] 與 **NAFNet** [24]。NAFNet 有一個驚人的發現：常用的非線性激活函數（ReLU、GELU、Sigmoid）其實不是必要的，可以用兩半特徵的逐元素相乘（「SimpleGate」）取代，或直接移除，得到一個「無非線性激活」（nonlinear activation free）的網路，不但更簡單，在 GoPro 與 SIDD 上也更強。Zhang 等人的去模糊綜述 [69] 涵蓋了更廣的範圍（盲與非盲、影片、失焦、損失函數）。

影像修補（Inpainting，簡介）

白話版。把缺失或破損的區域（刮痕、被移除的物體、破洞）補上合理的內容。

修補是最接近影像**生成**的復原任務：在大破洞裡沒有東西可以「還原」，只能創造出和周圍相符的內容。它的退化是一個遮罩，$y = m \odot x$，其中 m 在破洞處為 0、其他地方為 1。前陣子相當知名

的**遮罩自編碼器**（masked autoencoder, MAE）[45] 也可以歸類於此，可視為把修補當作預訓練任務：遮住 75% 的影像區塊，訓練 ViT 把它們重建回來，結果學到了很強的視覺表徵。實務上的修補方面，**LaMa** [46] 使用具備全影像感受野的快速傅立葉卷積，並以大型遮罩訓練，能在高解析度下填補大破洞；如今，以擴散模型為基礎的修補是大範圍、語意層級編輯的預設做法。

去背（Matting，簡介）

白話版。把前景（一個人、他的頭髮）連同柔和、半透明的邊緣一起摳出來，好貼到新背景上。

去背假設每個像素是前景與背景的混合：

$$I = \alpha F + (1 - \alpha) B,$$

其中 $\alpha \in [0, 1]$ 是不透明度（alpha matte），F 是前景顏色，B 是背景顏色。每個像素有七個未知數（α 與兩組 RGB 顏色），卻只有三個觀測值，嚴重不足，所以傳統方法要求使用者提供三**色圖**（trimap：確定前景、確定背景、不確定區域）。它也可以視為分割的一種軟性變形。**Background Matting** [47] 用一張不含主體的背景照片取代三色圖，先在合成影像上訓練網路，再以對抗損失（判斷新合成圖看起來是否自然）在真實影像上訓練。

通用骨幹網路

白話版。在上述所有任務中，總是那幾種網路設計反覆勝出。它們是通用的「影像到影像引擎」，可以拿來訓練上面任何一種退化。

骨幹	核心想法	為什麼適合復原
U-Net [48]	編碼器–解碼器，每個尺度都有跳躍連接	降採樣路徑提供大感受野，跳躍連接保留細節
SwinIR [14]	在全解析度上堆疊位移視窗自注意力的殘差群組	權重隨內容自適應；SR、去雜訊、JPEG 表現強
Restormer [23]	由 Transformer 區塊組成的 U-Net；注意力在**通道**維度計算（轉置注意力），搭配門控前饋網路	成本與像素數呈線性，可擴展到大影像
NAFNet [24]	由簡單卷積區塊組成的 U-Net，使用 LayerNorm、SimpleGate 與簡化通道注意力；沒有非線性激活	每單位 FLOP 的準確度極高；預設的「簡單基準」
MambaIR / MambaIRv2 [49], [50]	選擇性狀態空間（Mamba）掃描，加上局部增強與通道注意力；v2 加入「注意力式」非因果掃描	以線性成本取得全域感受野

反覆出現的設計理由。（1）**全域殘差**：預測 $x - y$（或 x 減去上採樣的 y），讓網路只學缺少的部分。（2）自 EDSR [10] 以來，大多數 SR／復原網路**不用批次正規化**：訓練小塊與完整測試影像的 BN 統計量不同，會損害色彩與數值範圍的保真度。（3）以可控成本取得**大感受野**：多尺度 U-Net、視窗或通道注意力、狀態空間掃描。復原既需要上下文（分辨紋理與雜訊），也需要全解析度的細節。（4）用**注意力或門控**做內容自適應處理：通道注意力（RCAN）、門控（NAFNet、Restormer）。原文的「Extension」清單預言了 ViT、Flow、GAN、Diffusion 等新架構；到了 2026 年，狀態空間模型也加入 CNN 與 Transformer，成為第三種骨幹選項 [49], [50]。

All-in-one 與通用復原

白話版。不要為雨、霧、雜訊各訓練一個模型，而是訓練**一個**模型，讓它自己判斷影像哪裡出了問題，再把它修好。

動機

原文的「Extension」一節把 all-in-one 復原列為 2023 年最熱門的題目之一，大約從 2021 年開始。動機很實際：在實際部署的系統中，我們事先不知道某張影像有哪一種退化（或哪幾種混合），各種天氣也可能隨時出現或同時發生。一整組任務專屬模型需要前面再接一個可靠的分類器，還浪費記憶體；一個能自我調整的模型方便得多。困難在於**任務干擾**：不同退化需要不同、甚至相反的操作（去雜訊是平滑，去模糊是銳化），所以直接做多任務訓練的效果會輸給專用模型。

代表方法

- **AirNet** [51]（CVPR 2022），也就是原文提到的 airnet：以對比學習訓練的退化編碼器，在沒有標籤的情況下擷取影像「**如何**」受損的表徵，再用它引導復原網路。它用單一模型處理未知類型的退化；它的雜訊＋雨＋霧設定後來成為標準的「三任務」基準。
- **TransWeather** [52]（CVPR 2022）：單一 Transformer 編碼器–解碼器處理雨、霧、雪，解碼器中有可學習的**天氣類型查詢**（weather-type query）。

- **PromptIR** [53]（NeurIPS 2023）：Restormer 風格的網路加上輕量的**提示區塊**（prompt block）。可學習的提示元件依照輸入特徵預測出的權重組合，注入解碼器，編碼退化相關的資訊。在去雜訊、除雨、除霧上測試。
- **AdaIR** [54]（ICLR 2025）：觀察到不同退化影響的是**不同的頻帶**，於是從輸入中挖掘低頻與高頻特徵並自適應地調變。涵蓋五種退化：雜訊、霧、雨、動態模糊與低光。
- **DA-CLIP** [55]（ICLR 2024）：以一個訓練好的控制器調整視覺語言模型 CLIP，讓它的影像編碼器能從受損影像同時產生乾淨內容的嵌入與退化的嵌入，再透過交叉注意力引導復原網路。
- **InstructIR** [56]（ECCV 2024）：使用者寫一句自然語言指令（「把雨去掉」、「調亮一點」），文字編碼器把它轉成嵌入，再用這個嵌入調變復原網路。

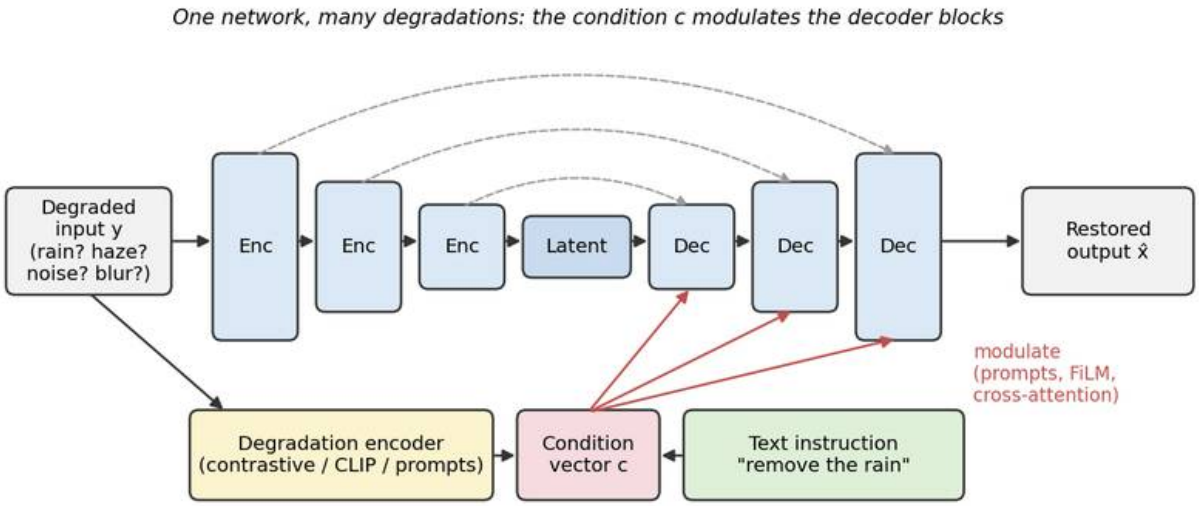


圖 5 — *All-in-one* 復原背後的共同樣板：編碼器–解碼器骨幹，加上一個能感知退化的條件 c。c 可以從影像推論（*AirNet*、*PromptIR*、*AdaIR*、*DA-CLIP*），也可以由文字給定（*InstructIR*），用來調變解碼器。

原文的動機仍是核心；改變的是「**由誰**決定退化類型」。它從固定的任務標籤，轉移到學習出的提示、視覺語言嵌入，最後到人類的指令，背後往往有大型預訓練模型支撐。All-in-one 綜述 [72] 對此有詳細討論。

損失函數

白話版。損失函數決定了什麼叫「好」。像素損失獎勵平均而言接近真實；感知與對抗損失獎勵看起來真實；頻率損失獎勵把細節做對。

像素損失。令 $\hat{x} = f_\theta(y)$，

$$\ell_2 = \frac{1}{P}\sum_p (\hat{x}_p - x_p)^2, \qquad \ell_1 = \frac{1}{P}\sum_p |\hat{x}_p - x_p|, \qquad \ell_\varepsilon = \frac{1}{P}\sum_p \sqrt{(\hat{x}_p - x_p)^2 + \varepsilon^2},$$

其中 P 是像素值的總數，ε 是一個小常數，讓最後一個成為平滑、可微分的 L1 版本。L2 直接最

佳化 PSNR，但對大誤差懲罰很重，而且會收斂到後驗平均。Zhao 等人 [57] 證明在架構固定的情況下，光是換損失函數就能明顯改變結果，而且 L1 與受感知啟發的損失（包括和 SSIM 的組合）常常勝過 L2，甚至在與 L2 相關的指標上也是。L1 是現代大多數 SR 與復原論文的預設選擇。

感知（特徵）損失。在預訓練分類網路 ϕ（通常是 VGG）的特徵空間中比較影像 [58]：

$$\ell_{\text{feat}} = \frac{1}{C_j H_j W_j} \|\phi_j(\hat{x}) - \phi_j(x)\|_2^2,$$

其中 ϕ_j 是第 j 層的特徵圖，有 C_j 個通道、尺寸為 $H_j \times W_j$。它能容忍小的位移，獎勵的是邊緣與紋理吻合，而不是數值完全相同。

對抗損失。判別器 D_ψ 學習分辨復原影像與真實影像，復原網路則學著騙過它，例如 $\ell_{\text{adv}} = -\log D_\psi(\hat{x})$。這會把輸出推向自然影像的流形上（SRGAN [12]、ESRGAN [13]、Real-ESRGAN [4]），代價是可能出現憑空捏造的紋理。實務上以 GAN 為基礎的復原網路會使用加權和，例如 $\ell = \ell_1 + \lambda_p \ell_{\text{feat}} + \lambda_a \ell_{\text{adv}}$，權重為 λ_p, λ_a。

頻率損失。網路傾向先擬合低頻，對細節擬合不足。在傅立葉轉換 $\mathcal{F}$ 上加一項損失，例如對複數值或振幅計算 $\|\mathcal{F}(\hat{x}) - \mathcal{F}(x)\|_1$，就能直接懲罰缺少的高頻。**焦點頻率損失**（focal frequency loss）[59] 更進一步，自適應地加重目前最難重建的頻率成分。頻率項現在在去模糊與 all-in-one 模型中很常見。

評估指標

白話版。「復原結果是否接近真實？」與「它看起來像不像真的照片？」是兩個不同的問題，由不同的指標回答，而且你無法同時把兩者做到最好。

全參考指標

PSNR（峰值訊雜比）：

$$\text{PSNR}(x, \hat{x}) = 10 \log_{10} \frac{\text{MAX}^2}{\text{MSE}(x, \hat{x})},$$

其中 MAX 是像素可能的最大值（1 或 255），MSE 是均方誤差。單位是 dB；提升 1 dB 約等於 MSE 降低 21%。

SSIM（結構相似度）[60] 比較局部平均、變異數與共變異數：

$$\text{SSIM}(a, b) = \frac{(2\mu_a\mu_b + C_1)(2\sigma_{ab} + C_2)}{(\mu_a^2 + \mu_b^2 + C_1)(\sigma_a^2 + \sigma_b^2 + C_2)},$$

在兩張影像的局部視窗 a、b 上計算再取平均，其中 μ 是局部平均，σ^2 是局部變異數，σ_{ab} 是局部共變異數，C_1, C_2 是維持數值穩定的小常數。它和人眼感受到的結構相關性比 PSNR 好，但仍然偏好平滑的結果。

LPIPS [61] 衡量深度網路特徵之間的距離（依人類相似度判斷校準）。越低越好，它會獎勵 PSNR 所懲罰的逼真紋理。

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, img_as_float32
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

x = img_as_float32(data.camera())
y = np.clip(x + np.random.default_rng(0).normal(0, 0.1, x.shape), 0, 1).astype(np.float32)
x_hat = ndi.gaussian_filter(y, 1.0)

def psnr(ref, est, peak=1.0):
    mse = np.mean((ref.astype(np.float64) - est) ** 2)
    return 10 * np.log10(peak ** 2 / mse)

print(f"PSNR  ours {psnr(x, x_hat):.2f} dB | skimage {peak_signal_noise_ratio(x, x_hat,
 ↪  data_range=1):.2f} dB")
print(f"SSIM  {structural_similarity(x, x_hat, data_range=1):.3f}")
# PSNR  ours 27.20 dB | skimage 27.20 dB
# SSIM  0.639
```

一定要寫清楚計算慣例（RGB 或 Y 通道、邊界裁切、數值範圍、是否先四捨五入成 8 位元）：光是這些差異，就足以讓論文中的 PSNR 相差零點幾 dB。

無參考指標

白話版。無參考（no-reference，又稱盲，blind）指標只看一張影像，就猜測人會給它幾分，沒有「標準答案」可以對照。這就像老師沒有參考答案也要批改作文：只能依靠「好作文通常長什麼樣子」的經驗。影像復原需要這類指標，因為我們最想修復的照片（真實的模糊、雜訊、下雨、昏暗照片）根本沒有乾淨的版本。

全參考、部分參考與無參考

影像品質評估（image quality assessment, IQA）方法依照它們對原始影像 x 知道多少來分類：

- **全參考（full-reference, FR）**：完整的 x 可用，指標逐像素或逐特徵比較 $\hat{x}$ 與 x（PSNR、SSIM [60]、LPIPS [61]）。合成基準用的就是這一類。
- **部分參考（reduced-reference, RR）**：只有 x 的少數摘要特徵可用（例如隨壓縮視訊一起傳送的統計量），指標在 $\hat{x}$ 上計算同樣的特徵再比較。
- **無參考（no-reference, NR，盲）**：只有 $\hat{x}$ 可用。指標必須自帶一套「什麼叫好」的模型。

復原需要 NR 指標有三個理由。(1) **真實測試影像沒有真值**：真實世界 SR、去雨、除霧都在沒有人擁有乾淨版本的照片上評估。(2) **參考影像本身可能不完美，或只是眾多合理答案之一**：GAN 或擴散模型的輸出可能看起來非常好，PSNR 卻很低，因為它的紋理與參考影像不同（見下方的感知–失真權衡）。(3) **NR 分數可以在執行時引導系統**，例如在沒有參考影像時，從多個輸出或參數中挑一個。代價是：NR 指標是人類意見的模型，而每個模型都有盲點。

如何驗證 NR 指標：與人類意見的相關性

白話版。只有當一個 NR 指標對影像的排序和人一致時，我們才信任它。所以研究者收集大量影像的大量人類評分，再檢查指標分數與評分有多一致。

平均意見分數。在主觀實驗中，每張影像 i 由多位觀察者評分，平均分就是它的**平均意見分數**（mean opinion score），$\mathrm{MOS}_i = \frac{1}{R}\sum_{r=1}^{R} s_{ir}$，其中 s_{ir} 是評分者 r 給的分數，R 是評分者人數。有些資料集公布的是**差異**分數（DMOS），以失真影像與其參考影像評分之差為基礎；DMOS 通常越低越好。

SRCC（Spearman 等級相關係數）衡量指標是否把影像排成和人相同的順序：

$$\mathrm{SRCC} = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n\,(n^2-1)},$$

其中 n 是測試影像數，d_i 是影像 i 依預測分數的名次與依 MOS 的名次之差（這個簡式假設沒有同名次；有同名次時改算名次的 Pearson 相關）。SRCC 對分數做任何單調轉換都不變，因此它衡量的是**預測的單調性**。

PLCC（Pearson 線性相關係數）衡量分數之間有多線性地一致：

$$\mathrm{PLCC} = \frac{\sum_{i=1}^{n}(q_i-\bar{q})(m_i-\bar{m})}{\sqrt{\sum_{i=1}^{n}(q_i-\bar{q})^2}\,\sqrt{\sum_{i=1}^{n}(m_i-\bar{m})^2}},$$

其中 q_i 是指標給影像 i 的分數，m_i 是它的 MOS，$\bar{q}$、$\bar{m}$ 是兩者的平均。由於各指標的尺度是任意的，常見做法是先用一個擬合出的單調函數（通常是 logistic 函數）轉換 q_i，再在轉換後的分數上

算 PLCC，因此 PLCC 衡量的是校準後的**預測準確度**。兩者都落在 $[-1, 1]$。對 NIQE 這種「越低越好」的指標，與 MOS 的原始相關係數是負的，論文會報告其絕對值。指標通常在資料集的一部分上訓練、在其餘部分測試，而跨資料集的相關性才是對泛化能力更誠實的檢驗。

人類評分資料集。 NR 模型在 IQA 資料集上訓練與測試。這些資料集的差別在於失真從哪裡來，這點對復原非常重要：

資料集	失真來源	內容
LIVE（Release 2）[104]	合成	五種失真類型（JPEG、JPEG2000、高斯模糊、白雜訊、JPEG2000 位元錯誤）的影像，附 DMOS
TID2013 [105]	合成	25 張參考影像 × 24 種失真 × 5 個等級 = 3,000 張失真影像；分數來自五個國家觀察者的 971 次實驗
KADID-10k [106]	合成	81 張原始影像 × 25 種失真 × 5 個等級 = 10,125 張，每張 30 個群眾外包評分
KonIQ-10k [107]	真實（in the wild）	10,073 張照片，約 120 萬個來自 1,459 位群眾外包工作者的評分
SPAQ [108]	真實（智慧型手機）	66 支智慧型手機拍攝的 11,125 張照片，附品質、屬性（亮度、色彩豐富度、對比、雜訊程度、銳利度）與場景標籤，以及 EXIF 資料

資料集	失真來源	內容
PIPAL [109]	復原輸出	影像復原演算法（包含 GAN 方法）的輸出，超過 113 萬個人類判斷，以 Elo 系統轉成分數

合成資料集能精確控制失真，但只有「教科書式」的損壞。真實資料集有真正的相機問題，卻沒有復原輸出。只有 PIPAL 這類資料集，才包含復原論文真正需要評分的那種影像。

第一類：自然場景統計（NSS）模型

白話版。乾淨的照片有一些隱藏的統計習慣。把照片的局部亮度與局部對比去掉之後，剩下的部分會呈現某種典型形狀的鐘形直方圖，而且相鄰的剩餘值彼此只有微弱的關聯。模糊、雜訊與壓縮會以各自特有的方式改變這個形狀。NSS 指標就是量測這個形狀，再問它有多不尋常。

MSCN 係數。對灰階影像 I，**去均值對比正規化**（mean-subtracted contrast-normalized, MSCN）係數為 [94]

$$\hat{I}(i,j) = \frac{I(i,j) - \mu(i,j)}{\sigma(i,j) + C}, \qquad \mu(i,j) = \sum_{k,l} w_{k,l}\, I(i+k, j+l), \qquad \sigma(i,j) = \sqrt{\sum_{k,l} w_{k,l}\, \big(I(i+k,j+l) - \mu(i,j)\big)^2},$$

其中 (i,j) 是像素位置，w 是權重總和為 1 的小型圓對稱高斯視窗，μ 與 σ 是局部加權平均與標準差，C 是一個小常數（8 位元影像取 1），避免在平坦區域除以零。這是一種除法正規化，大致模仿人類早期視覺中的對比增益控制。

廣義高斯擬合。$\hat{I}$ 的直方圖以零均值的廣義高斯分布（generalized Gaussian distribution, GGD）建模：

$$f(t;\, \alpha, \sigma^2) = \frac{\alpha}{2\beta\,\Gamma(1/\alpha)} \exp\!\Big(-(|t|/\beta)^{\alpha}\Big), \qquad \beta = \sigma\sqrt{\frac{\Gamma(1/\alpha)}{\Gamma(3/\alpha)}},$$

其中 t 是一個 MSCN 值，$\alpha > 0$ 是**形狀**參數（$\alpha = 2$ 為高斯，$\alpha = 1$ 為拉普拉斯；α 越小直方圖越尖、尾巴越重，越大則頂部越平），σ^2 是變異數，β 是對應的尺度，Γ 是 gamma 函數。估計 α 的簡單方法是**動差匹配**（moment matching）：比值 $\mathbb{E}[t^2]/(\mathbb{E}|t|)^2 = \Gamma(1/\alpha)\,\Gamma(3/\alpha)/\Gamma(2/\alpha)^2$ 只與 α 有關，所以先從資料算出左邊，再搜尋能對上的 α。

BRISQUE [94]（Blind/Referenceless Image Spatial Quality Evaluator）計算 MSCN 係數的 GGD 形狀與變異數，再加上對相鄰 MSCN 係數乘積（水平、垂直與兩個對角方向）擬合非對稱 GGD 得到的參數，並在兩個影像尺度上重複。一個以合成失真影像的人類評分訓練的支持向量迴歸器（SVR），把這些特徵對應成品質分數。越低越好。它是**需要意見分數的**（opinion-aware）：訓練時需要人類評分。

NIQE [62]（Natural Image Quality Evaluator）則**不需要意見分數**（opinion-unaware）：它從未看過人類評分或失真影像。它在清晰的區塊上計算類似的 NSS 特徵，對一組原始影像的特徵擬合一個均值 ν_1、共變異數 Σ_1 的多變量高斯（multivariate Gaussian, MVG），對測試影像的區塊擬合另一個 MVG (ν_2, Σ_2)，然後報告

$$D(\nu_1, \nu_2, \Sigma_1, \Sigma_2) = \sqrt{(\nu_1 - \nu_2)^{\top} \Big(\frac{\Sigma_1 + \Sigma_2}{2}\Big)^{-1} (\nu_1 - \nu_2)},$$

也就是兩個高斯之間類似 Mahalanobis 距離的量（$^\top$ 為轉置）。越低越好；零代表「統計上和原始影像集一樣」。

PIQE [96]（Perception-based Image Quality Evaluator）也使用 MSCN 係數，但**逐區塊**運作：把影像切成 16×16 的區塊，標記出現區塊效應或雜訊的區塊，再把被標記區塊的失真彙整成一個分數。它不需要用意見分數訓練，還能給出影像哪裡看起來失真的局部圖。在常見的 MATLAB 實作中，分數介於 0 到 100，越低越好。

Ma 等人的分數與感知指數。 Ma 等人 [95] 對超解析影像做了人類評分實驗，並用這些分數學出一個 SR 專用的無參考指標：使用空間域與頻率域的統計特徵，以及兩階段迴歸；他們的分數（「Ma」，越高越好）大致介於 0 到 10。2018 年 PIRM 感知超解析挑戰賽 [63] 把它與 NIQE 合成**感知指數**（Perceptual Index）：

$$\mathrm{PI} = \tfrac{1}{2}\big((10 - \mathrm{Ma}) + \mathrm{NIQE}\big),$$

PI 越低代表感知品質越好。PIRM 在感知–失真平面上依 RMSE 分成三個區域（RMSE 分別不超過 11.5、12.5 與 16），在每個區域內以 PI 排名，因此是在固定的失真預算下比較真實感。許多感知 SR 論文至今仍報告 PI。

程式：實際看看 MSCN 統計量。下面的程式在 `camera` 及其模糊、加雜訊、過度銳化的版本上，計算 MSCN 係數、以動差匹配估計的 GGD 形狀、MSCN 變異數，以及水平相鄰係數的相關（BRISQUE 成對特徵的簡化版）：

```
import numpy as np
from scipy.ndimage import gaussian_filter
from scipy.special import gamma
from skimage import data, img_as_float

def mscn(img, sigma=7/6, C=1.0):
    """Mean-subtracted contrast-normalized coefficients of a grayscale image in [0, 255]."""
    mu = gaussian_filter(img, sigma)
    sd = np.sqrt(np.abs(gaussian_filter(img * img, sigma) - mu * mu))
    return (img - mu) / (sd + C)

def ggd_shape(x):
    """Moment-matching estimate of the generalized Gaussian shape alpha (2 = Gaussian)."""
    alphas = np.arange(0.2, 10.0, 0.001)
    r_theory = gamma(1 / alphas) * gamma(3 / alphas) / gamma(2 / alphas) ** 2
    r_data = np.mean(x ** 2) / np.mean(np.abs(x)) ** 2
    return alphas[np.argmin((r_theory - r_data) ** 2)], np.mean(x ** 2)

rng = np.random.default_rng(0)
clean = img_as_float(data.camera()) * 255
versions = {
    "clean": clean,
    "blur (sigma=2)": gaussian_filter(clean, 2),
    "noise (sigma=20)": np.clip(clean + rng.normal(0, 20, clean.shape), 0, 255),
    "over-sharpened": np.clip(clean + 2.0 * (clean - gaussian_filter(clean, 2)), 0, 255),
}
for name, img in versions.items():
    m = mscn(img)
    alpha, var = ggd_shape(m)
```

```
    rho = np.mean(m[:, :-1] * m[:, 1:]) / var          # correlation of horizontal neighbors
    print(f"{name:17s} GGD shape {alpha:4.2f} | MSCN variance {var:4.2f} | neighbor corr.
     ↪  {rho:+.2f}")
# clean             GGD shape 1.56 | MSCN variance 0.28 | neighbor corr. +0.12
# blur (sigma=2)    GGD shape 0.87 | MSCN variance 0.04 | neighbor corr. +0.89
# noise (sigma=20)  GGD shape 3.12 | MSCN variance 0.68 | neighbor corr. -0.10
# over-sharpened    GGD shape 2.35 | MSCN variance 0.43 | neighbor corr. +0.09
```

每種失真都把特徵推往自己的方向（圖 6）。**模糊**去掉局部對比，大部分 MSCN 值縮向零：變異數很小、直方圖非常尖（α 小）、相鄰值高度相關。**雜訊**加入彼此獨立的能量：變異數變大、直方圖頂部變平（α 大於 2）、相鄰相關接近零（略為負值，因為減去局部平均會讓相鄰值彼此牽連）。

過度銳化則把特徵推往和雜訊相同的方向。這正是 NSS 指標用於復原時的核心弱點：它分不出復原出來的紋理和憑空加上的高頻能量。另外要注意，絕對數值取決於影像內容與 C（乾淨的 `camera` 影像並不是單位高斯），這就是為什麼 BRISQUE 需要訓練迴歸器、NIQE 需要原始參考影像集，而不是使用固定門檻。

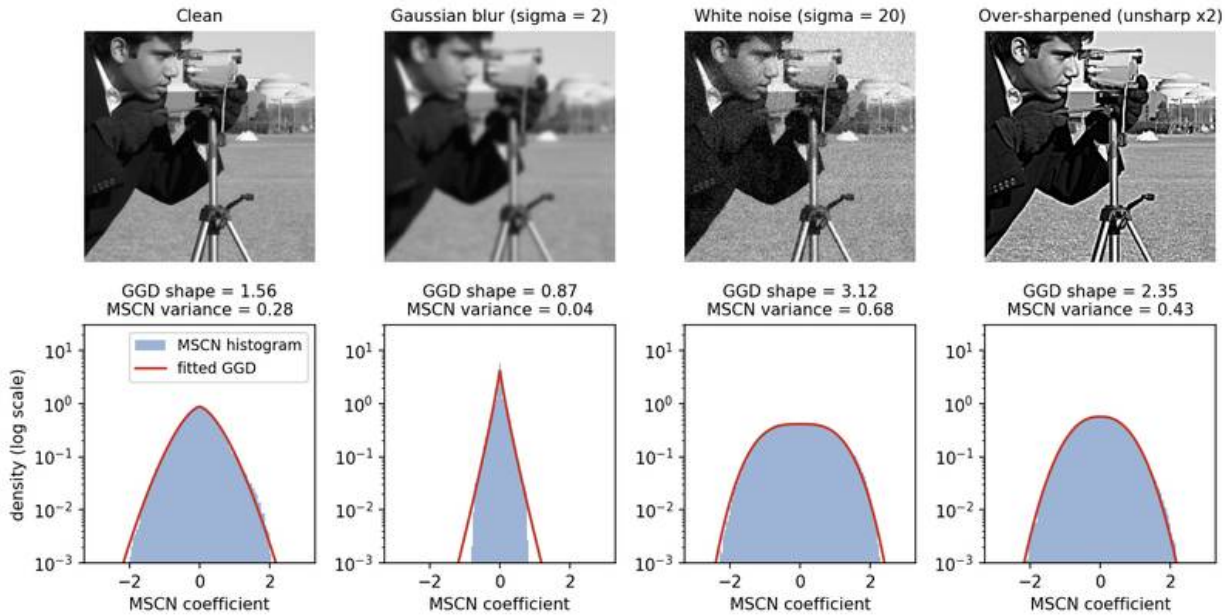


圖 *6 — MSCN 統計量，BRISQUE、NIQE 與 PIQE 的基礎。模糊讓直方圖變窄變尖；雜訊與過度銳化都讓它變寬變平，因此 NSS 指標會把兩者看成相似的變化。*

第二類：學習式深度模型

白話版。不用手工設計的統計量，而是訓練神經網路直接預測人類評分。深度特徵更能掌握內容、語意與複雜的瑕疵，但模型只懂它訓練時看過的影像與意見。

- **NIMA** [97] 用 CNN 預測人類評分的整個分布，而不是單一數字。令 p_k 為預測的分數 k 的機率（1–10 分制），報告的分數是平均值 $\hat{s} = \sum_k k\, p_k$，分布的離散程度則代表人們可能多不一致。

以美學評分訓練的權重判斷的是「這是不是一張好看的照片」，而不是「這是不是忠實的復原」。

- **HyperIQA** [98] 針對真實環境（in the wild）中的失真。它把評估拆成三個階段：骨幹網路先理解內容，接著一個**超網路**（hyper network）依據內容產生品質預測網路的權重（「感知規則」隨畫面內容調整，因為同樣的模糊出現在天空上和臉上，給人的感受不同），最後由產生出來的網路預測分數。

- **MUSIQ** [99] 是多尺度 transformer，以**原始解析度與長寬比**輸入影像，外加縮小的版本，並使用基於雜湊的 2-D 空間與尺度嵌入，因此不會因縮放或裁切而改變正要被評估的瑕疵。
- **MANIQA** [100] 結合 vision transformer 特徵、轉置(通道)注意力區塊與尺度 Swin transformer 區塊，再用雙分支、區塊加權的輸出頭預測分數。它贏得 NTIRE 2022 感知 IQA 挑戰賽的無參考賽道，該賽事資料包含復原與 GAN 演算法的輸出，因此比用相機失真訓練的指標更接近復原的情境。
- **TOPIQ** [101] 採用**由上而下**（top-down）的想法：高階語意特徵決定該看哪裡，再透過跨尺度注意力機制用它們加權低階、對失真敏感的特徵（網路稱為 CFANet，骨幹為 ResNet-50）。同一設計可用於 FR 與 NR 評估，且比基於 transformer 的 IQA 模型輕量。

這些模型都是**需要意見分數的**，輸出「越高越好」的分數，而且都依賴訓練資料集：同一架構在 KonIQ-10k 上訓練與在 PIPAL 上訓練，可能會給復原方法不同的排名。

第三類：視覺–語言與大型多模態模型

白話版。用大量圖文資料訓練出的模型，已經「知道」好照片和壞照片長什麼樣子。你可以直接問它們：把影像和文字提示比較，或乾脆請一個聊天機器人式的模型打分數。

CLIP-IQA [102] 比較影像的 CLIP 嵌入與一對反義提示（例如「Good photo.」與「Bad photo.」）的嵌入。令 $\mathbf{x}$ 為影像嵌入，$\mathbf{t}_1, \mathbf{t}_2$ 為兩個文字嵌入，$s_i = \mathbf{x}^\top \mathbf{t}_i / (\|\mathbf{x}\|\,\|\mathbf{t}_i\|)$ 為它們的餘弦相似度，分數為

$$\text{CLIP-IQA} = \frac{e^{s_1}}{e^{s_1} + e^{s_2}} \in [0, 1],$$

越高越好。基本版本是**零樣本**（zero-shot）的（完全不需要意見分數），換用其他提示對（「Noisy photo.」/「Clean photo.」、「Sharp photo.」/「Blurry photo.」）可以探測單一屬性。提示的措辭會改變分數。

Q-Align [103] 微調一個大型多模態模型，讓它像人類評分者一樣回答：給出**文字評分等級**（bad、poor、fair、good、excellent），而不是數字。測試時，令 X_{l_i} 為模型對等級 l_i 的 logit，分數是只在這五個字上做 softmax 後的期望等級：

$$S = \sum_{i=1}^{5} i \cdot \frac{e^{X_{l_i}}}{\sum_{j=1}^{5} e^{X_{l_j}}},$$

其中 $i = 1, \dots, 5$ 依序代表從 bad 到 excellent 的等級。這類模型跨資料集表現強，但運算量大，也會繼承基礎模型的偏誤。

分布層級的指標

有些指標評的是一組輸出而非單張影像。**FID** [64] 比較復原結果集合與一組真實影像在 Inception 特徵上的分布；它原本是為生成模型設計的，現在用於擴散 SR 這類生成式復原，但它需要大量影像，而且無法告訴你任何單張影像的好壞。原文也提到的 **IS**（Inception Score）衡量類別預測的信心與多樣性，比較適合類別條件式生成而非復原，如今在復原任務中很少使用。

常見陷阱

- **指標可以被刻意迎合**。一個會加顆粒或銳化邊緣的復原模型可以改善 NSS 分數（上面的程式就顯示了這點），卻沒有變得更忠實。任何可微分的指標（以及任何學習式 IQA 網路）一旦被拿來當訓練損失函數或用於模型選擇，都可能被直接最佳化，網路學會的是討好指標而不是討好人。如果一個方法在損失函數中用了 CLIP 或某個 IQA 網路，再用同一個模型評估它就不是獨立的評估。
- **領域不匹配**。大多數需要意見分數的指標是在合成失真（LIVE、TID2013、KADID-10k）或相機照片（KonIQ-10k、SPAQ）上訓練的。復原輸出的瑕疵不一樣：憑空生成的紋理、過度平滑、GAN 特有的圖樣。這種不匹配正是 PIPAL [109] 明確的動機：其作者發現既有的 IQA 方法無法公平評估以 GAN 為基礎的復原。低光源的問題則在於曝光與色彩，而大多數 NSS 特徵會把它們正規化掉；低光源專屬的討論請見DLCV 05。
- **指標彼此不一致**。 NSS、學習式與視覺–語言指標常把同一批方法排出不同的順序，而且都可能與人的判斷不同。分數也取決於實作、權重檔、色彩空間、輸入尺寸以及影像是否被縮放，所以從不同論文抄來的數字往往無法直接比較。
- 「**NR 品質較高**」**不等於**「**較正確**」。 NR 指標無法知道一張清晰的臉或一行文字是否和真實場景相符。憑空生成的車牌也可能拿到滿分。

建議做法

1. 有參考影像時，以 FR 指標（PSNR、SSIM、LPIPS）為主，NR 指標作為補充；並在感知–失真平面上標出你的方法（見下一小節）。
2. 在沒有參考影像的真實影像上，報告**多個來自不同類別的 NR 指標**（例如 NIQE 或 PI、一兩個學習式模型如 MUSIQ、MANIQA 或 TOPIQ，以及一個視覺–語言指標如 CLIP-IQA 或 Q-Align），並註明每個指標是越高還是越低越好。
3. 使用同一套實作並寫明版本與權重檔。開源的 **IQA-PyTorch** 工具箱（`pyiqa`）[110] 以統一介面重新實作了本節大多數的指標（NIQE、BRISQUE、PIQE、Ma 分數與 PI、NIMA、HyperIQA、MUSIQ、MANIQA、TOPIQ、CLIP-IQA、Q-Align，以及 FR 指標）。
4. 不要對著裁判調參：至少保留一個完全沒參與訓練或模型選擇的指標。
5. 重要的主張要有**使用者研究**佐證（平均意見分數或成對比較，盲測且隨機化）；DLCV 05 介紹了 MOS 與 Bradley–Terry 成對比較的流程。指標之間或指標與人不一致時，要明白說出來。

總覽表

指標	類別	需要訓練嗎？	方向	主要對什麼敏感
NIQE [62]	NSS	只需原始影像（不需意見分數）	越低越好	局部統計量的偏離：模糊、雜訊、壓縮
BRISQUE [94]	NSS + 迴歸	需要，合成失真上的人類評分	越低越好	同樣的統計量，對應到人類評分
PIQE [96]	NSS，逐區塊	不需意見分數	越低越好	局部區塊中的區塊效應與雜訊；可輸出失真圖
Ma [95]	手工特徵 + 迴歸	需要，SR 輸出上的人類評分	越高越好	SR 瑕疵、紋理與銳利度
PI [63]	NSS + 學習式	透過其中的 Ma 部分	越低越好	SR 的感知品質；獎勵紋理，包含被放大的雜訊
NIMA [97]	學習式 CNN	需要，評分分布	越高越好	技術品質或美學品質，視權重檔而定
HyperIQA [98]	學習式 CNN + 超網路	需要	越高越好	真實失真，並依內容脈絡判斷
MUSIQ [99]	學習式 transformer	需要	越高越好	原始解析度下的多尺度品質
MANIQA [100]	學習式 transformer	需要	越高越好	復原與 GAN 瑕疵（以這類資料訓練）
TOPIQ [101]	學習式 CNN，由上而下注意力	需要	越高越好	語意重要區域中的失真
CLIP-IQA [102]	視覺–語言	不需要（零樣本版本）	越高越好	整體「好照片」印象；取決於提示
Q-Align [103]	大型多模態模型	需要，評分轉成文字等級	越高越好	類似人類的整體品質判斷
FID [64]	分布層級	使用預訓練 Inception 特徵	越低越好	一組輸出與真實影像之間的分布差距

感知–失真權衡

Blau 與 Michaeli [65] 把這個矛盾形式化。令 $\Delta(x,\hat{x})$ 為任意失真度量（MSE、$1 - $ SSIM……），$d(p_X,p_{\hat{X}})$ 為真實影像分布與復原影像分布之間的散度（感知品質：d 越小，輸出在統計上越無法與真實影像區分）。**感知–失真函數**定義為

$$P(D) = \min_{p_{\hat{X}|Y}} d(p_X, p_{\hat{X}}) \quad \text{subject to} \quad \mathbb{E}[\Delta(X,\hat{X})] \le D,$$

其中最小值取遍所有（可能帶隨機性的）復原方法。他們證明 $P(D)$ 是非遞增的，而且（當散度對第二個引數為凸時）是凸函數：原點附近有一塊無法達到的區域。一個方法不可能同時擁有最小失真與完美的真實感。特別是 MMSE 估計 $\mathbb{E}[x \mid y]$ 的失真最低，但輸出分布模糊、不真實；想看起來真實，就必須接受較高的 MSE。GAN（以及現在的擴散模型）是沿著這條邊界移動的原則性方法。

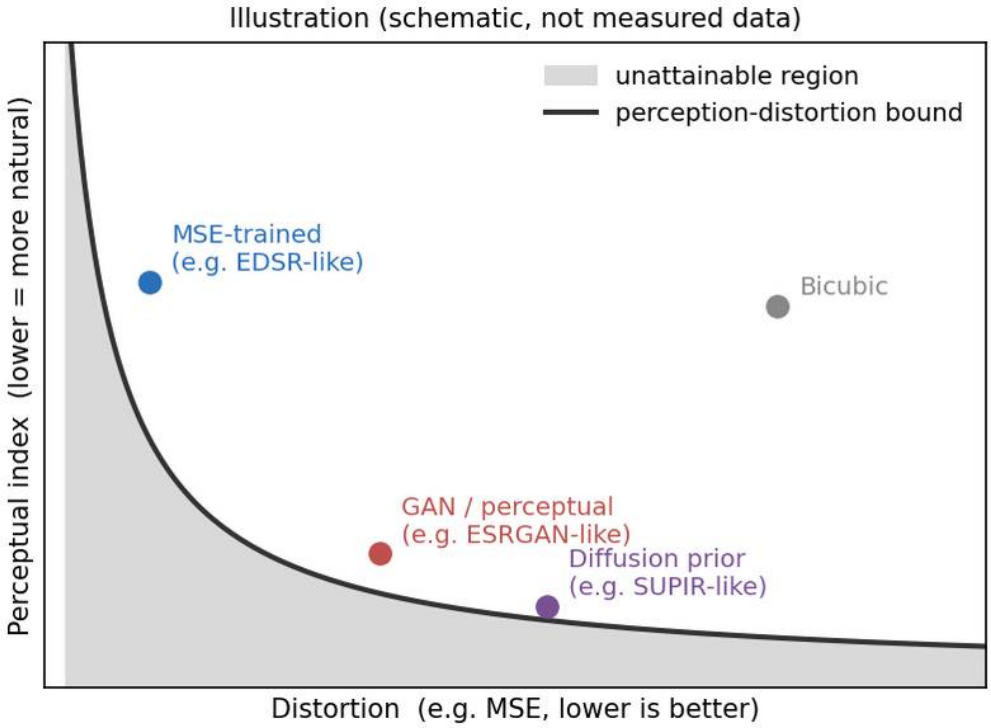


圖 7 — 感知–失真權衡的示意圖 [*65*]（非實測資料）。邊界是凸且遞減的；各方法在保真度（左）與真實感（下）之間取捨。

這正是原文評估指標一節的更新：擴散模型能「生成」看似合理但不一定忠於原圖的細節，保真度（PSNR ／ SSIM）與感知品質之間的落差因此更受重視。**NTIRE 2025** ×4 SR 挑戰賽就分成兩個賽道：以 PSNR 排名的 restoration track，以及以感知分數排名的 perceptual track [3]；2024 年的版本則以 PSNR 排名 [2]。實務建議：同時報告一個失真指標與一個感知指標，並說明你的方法瞄準權衡曲線上的哪個位置。

NTIRE 挑戰賽

白話版。每年 CVPR 的一個 workshop 會在固定資料集上舉辦公開競賽，並發表報告說明勝出的想法：這是了解「現在什麼有用」最簡單的方式。

既然依任務分類的排行榜已經消失，NTIRE 挑戰賽報告就是追蹤復原領域的好管道。每份報告都會說明任務、資料、評估方式，以及名列前茅隊伍的方法。本篇引用的例子：

- **NTIRE 2017 SR** [19]：推出 DIV2K；EDSR [10] 奪冠。
- **NTIRE 2024** ×4 **SR** [2]：在 DIV2K 上做雙三次 ×4，以 PSNR 排名。
- **NTIRE 2025** ×4 **SR** [3]：restoration（PSNR）與 perceptual 兩個賽道分開評比。
- **NTIRE 2025 雨滴去除** [40]：真實的白天與夜晚雨滴影像，並有雙重對焦設定。

近年的報告有兩個共同模式：勝出的作品通常是上述骨幹（HAT、Restormer、NAFNet 或

Mamba 風格，有時再加上擴散先驗）的集成或大型混合模型，並使用大量資料與算力訓練；而挑戰賽也越來越常使用真實拍攝的資料與感知賽道。

現代觀點

原文最後列出了延伸方向（醫學影像、多模態、非監督、新架構、all-in-one、低階結合高階、即時與低運算、水下）。其中大多數現在都是活躍的研究領域。以下依任務整理綜述文獻，再談 2023–2026 年的最新進展與尚未解決的問題。

綜述論文

- **Wang、Chen 與 Hoi，〈Deep Learning for Image Super-resolution: A Survey〉（TPAMI 2021）** [66]。把 SR 分成監督式、非監督式與特定領域 SR；在監督式 SR 中，再依上採樣位置（前、後、漸進、迭代上下採樣）、上採樣運算（插值、轉置卷積、子像素）、網路設計（殘差、遞迴、密集、注意力、多路徑）、損失函數與訓練策略來拆解模型。重點：大部分 SR 的進展都能描述成在這些軸上的選擇，而非監督／真實世界 SR 被點名為主要的開放方向，領域後來也確實往這裡走。
- **Tian 等人，〈Deep Learning on Image Denoising: An Overview〉（Neural Networks 2020）** [67]。把 CNN 去雜訊方法分成加性白雜訊、真實雜訊、盲去雜訊，以及混合了模糊或低解析度的雜訊四類，並在公開基準上比較。重點：瓶頸往往是雜訊模型的真實程度，而不是網路。
- **Elad、Kawar 與 Vaksman，〈Image Denoising: The Deep Learning Revolution and Beyond〉（SIAM J. Imaging Sciences 2023）** [68]。回顧去雜訊從傳統先驗到深度網路的歷史，接著主張去雜訊器如今是其他反問題（plug-and-play、regularization by denoising）以及擴散式影像生成的基本元件，並主張復原應該接受多個合理解（後驗取樣），而不是只給一個答案。重點：前沿已經從「更好的 MMSE 去雜訊器」移到「從後驗分布取樣」。
- **Zhang 等人，〈Deep Image Deblurring: A Survey〉（IJCV 2022）** [69]。涵蓋模糊成因、資料集與指標，以及依架構、損失函數與應用（人臉、文字、立體）分類的 CNN 去模糊方法，包含盲與非盲設定。重點：動態場景去模糊已從估計模糊核，轉向以 GoPro 這類實拍配對訓練的端到端多尺度網路。
- **Gui 等人，〈A Comprehensive Survey and Taxonomy on Single Image Dehazing Based on Deep Learning〉（ACM Computing Surveys 2023）** [70]。把方法分成監督式、半監督式與非監督式，並在各類中依是否及如何使用散射模型細分；也整理資料集、網路模組、損失函數與指標，並實驗比較基準方法。重點：物理引導與無物理的設計並存，合成到真實的落差是主要未解問題。
- **Yang 等人，〈Single Image Deraining: From Model-Based to Data-Driven and Beyond〉（TPAMI 2021）** [71]。回顧雨的外觀模型、以模型為基礎的方法（先驗與最佳化），以及資料驅動的方法（架構、限制條件、損失函數、資料集）。重點：真實的雨模型與真實世界的評估，比在合成資料上繼續微調架構更重要。
- **Jiang 等人，〈A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends〉（TPAMI 2025）** [72]。依架構設計、學習範式與核心創新（退化表徵、提示、混合專家、語言引導）整理 all-in-one 方法，並彙整資料集與評估協定。重點：各論文的任務組合不一致，讓比較變得困難；對未見過與混合退化的泛化仍是開放問題。

- **Li 等人，〈Diffusion Models for Image Restoration and Enhancement: A Comprehensive Survey〉（IJCV 2025）** [73]。依學習範式（監督式 vs. 零樣本使用預訓練先驗）、條件策略、框架設計與建模策略分類以擴散模型為基礎的復原，聚焦於 SR、去模糊、修補與盲／真實世界復原。重點：擴散模型的感知品質最好，但運算成本高，也有保真度與捏造細節的疑慮；蒸餾成少步驟與更好的評估方式是關鍵方向。

2023–2026 年的最新進展

1. **All-in-one 與盲復原成為主流設定**，不再是小眾題目。方法從學習式退化表徵（AirNet）發展到提示（PromptIR）、頻率感知的自適應（AdaIR）與語言引導（DA-CLIP、InstructIR）。
2. **以預訓練擴散模型作為生成先驗**，大幅改變了真實世界 SR 與盲復原（DiffBIR、SUPIR），一步式變體（OSEDiff）則把推論成本從數十、數百步降到一步。
3. **狀態空間模型**（MambaIR、MambaIRv2）加入 CNN 與 Transformer，成為以線性成本取得全域感受野的骨幹選項。
4. **評估方式跟上了生成能力**：挑戰賽把保真度賽道與感知賽道分開 [3]，真實拍攝的資料集（SIDD、O-HAZE、Dense-Haze、Raindrop Clarity）也和合成基準一起成為標準。

尚未解決的問題

- **真實世界的泛化**。合成流程仍然漏掉真實相機的影像處理、隨位置變化的模糊與混合天氣；模型遇到沒見過的組合時表現會下滑。

- **保真度 vs. 捏造**。生成先驗可能憑空創造文字、人臉或紋理。在醫學、鑑識與科學用途中這是不能接受的，而我們還缺少衡量「忠實又清晰」的好指標。
- **評估**。 PSNR ／ SSIM 獎勵模糊；無參考指標可以被刻意迎合；人類評測成本高。對生成式復原而言，指標與人類偏好的一致性仍然很弱。
- **效率**。在手機與相機上做高解析度、即時（以及影片）復原仍是很強的限制，對 Transformer 與擴散模型尤其如此。
- **All-in-one 模型的任務干擾與擴展**，以及如何在不重新訓練的情況下加入新的退化。
- **低階結合高階**。讓復原幫助下游偵測或分割，而不是給人看，是另一種目標（任務導向評估，例如 RESIDE [32]）。

作者的觀點

原文以兩個至今仍成立的觀察作結。第一，由於每個任務都簡單明瞭，現階段 SOTA 的比拼大多靠特殊的架構取勝，也就代表大量的算力與複雜的實驗設計；但這些任務在應用上都相當實用且容易上手，所以在**應用面**與**任務的組合**上還有很多著墨空間（all-in-one 復原正是往這裡發展）。第二，相較於高階任務，這個領域的學習成本不高：入門時很容易想像模型的架構與樣貌（受損的影像進去、更乾淨的影像出來），進而接觸更進一步的內容，是進入深度學習研究的好入口。

重點整理

- 低階視覺關注的是把像素**感知**好；幾乎每個任務都是 $y = \mathcal{D}(x) + n$ 的一個特例，差別只在 $\mathcal{D}$（模糊、降採樣、JPEG、散射、雨層、Retinex 光照、遮罩）。
- 深度學習把退化模型從求解器搬進了**資料產生器**：真實、隨機化的合成流程（以及實拍配對）和網路一樣重要。
- SR 的上採樣方式從雙三次先放大（SRCNN），經過轉置卷積（FSRCNN），演進到今天標準的子像素卷積（ESPCN）。
- 少數幾種通用骨幹主導了這個領域：U-Net 風格的 CNN（NAFNet）、Transformer（SwinIR、Restormer、HAT）與狀態空間模型（MambaIR）；常見的設計習慣是全域殘差、不用 BN、大感受野，以及門控／注意力。
- All-in-one 模型（AirNet、TransWeather、PromptIR、AdaIR、DA-CLIP、InstructIR）透過以學習、提示或文字給定的退化描述作為條件，讓一個網路處理未知的退化。
- L2 ／ L1 損失得到忠實但平滑的結果；感知、對抗與擴散式方法得到真實但較不忠實的結果。**感知–失真權衡**告訴我們在極限上兩者不可兼得，所以兩類指標都要報告。
- 盲去模糊與盲 SR 除了影像，還必須估計**模糊核**。天真的聯合 MAP 會偏好「沒有模糊」的解 [74]；有效的方法會把影像邊際化、使用邊緣預測或偏好清晰影像的先驗、由粗到細交替更新影像與模糊核，最後做非盲反卷積。深度方法則是預測模糊核或以它為條件（SRMD、IKC、DAN、KernelGAN、SelfDeblur），或乾脆完全略過它（以 GoPro 訓練的網路、BSRGAN、Real-ESRGAN）。
- KITTI 是自駕車基準，不是除霧資料集；除霧請用 RESIDE，並搭配 O-HAZE、Dense-Haze 這類真實配對。
- 無參考指標是人類意見的模型，以與平均意見分數的 SRCC ／ PLCC 來驗證。NSS 指標（BRISQUE、NIQE、PIQE、PI）量測 MSCN 統計量的偏離；學習式（NIMA、HyperIQA、MUSIQ、MANIQA、TOPIQ）與視覺–語言（CLIP-IQA、Q-Align）模型則從評分中學習。它們都可能被銳化或雜訊迎合，也都受訓練失真與復原瑕疵之間落差的影響，所以要同時報告多個 NR 指標、FR 指標與使用者研究。

練習

1. **退化的順序**。用 `degrade` 函式，在相同參數下比較 (a) 模糊 → 降採樣 → 雜訊 → JPEG 與 (b) 雜訊 → 模糊 → 降採樣 → JPEG。從視覺上，以及在平坦區塊量測雜訊標準差，說明為什麼順序會影響網路要學的東西。

   提示

   在 (b) 中，模糊與降採樣會過濾雜訊：標準差變小，而且在空間上變得相關（變成平滑的斑塊，而不是逐像素的顆粒）。在 (a) 中，雜訊在低解析度下仍然是白雜訊。只用 (a) 訓練的網路，測試時遇到相關雜訊會去除不足。這也是 Real-ESRGAN 隨機化並重複各個階段的原因之一。

2. **手算 pixel shuffle**。一個特徵張量的形狀為 $(Cs^2, H, W) = (4, 2, 2)$，$s = 2$、$C = 1$，第 k 個通道全部填入常數 k。用 SR 一節的公式寫出 pixel shuffle 的 4×4 輸出，再用 `torch.nn.PixelShuffle(2)` 驗證。

提示

輸出像素 $(2h+i, 2w+j)$ 取自通道 $2i+j$。所以每個 2×2 區塊都是 $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$，$4\times 4$ 的輸出就是把這個區塊重複四次。每個輸出區塊都從同一個低解析度位置上的四個通道各取一個值。

3. **殘差還是直接預測？**把 `TinyDnCNN.forward` 改成回傳 `self.body(y)`（直接預測乾淨影像），以相同的隨機種子與步數訓練兩個版本，比較 PSNR 與損失曲線。再用 $\sigma = 5/255$ 重做一次。殘差學習在什麼情況下幫助最大？

   提示

   在訓練步數很少時，殘差版本通常收斂更快、PSNR 更高，因為它在初始化時就已經大約輸出 y（PSNR 不差），只需要學一個小修正。低雜訊時直接預測的版本最吃力，因為它必須透過好幾層 ReLU 學出一個近乎恆等的映射。

4. **用數字算暗通道**。某個有霧像素 $I = (0.75, 0.78, 0.80)$，其區塊內所有通道 I^c/A^c 的最小值為 0.70，$A = (0.9, 0.9, 0.9)$。取 $\omega = 0.95$、$t_0 = 0.1$，計算這個像素的 $\tilde{t}$ 與還原後的 $\hat{J}$。

   提示

   $\tilde{t} = 1 - 0.95 \times 0.70 = 0.335$。接著 $\hat{J}^c = (I^c - 0.9)/0.335 + 0.9$，約為 $(0.452, 0.542, 0.601)$。輸出比輸入更暗、飽和度更高，這正是去掉霧的面紗後該有的樣子。

5. **感知 vs. 失真**。把小型去雜訊網路的輸出再加上少量新的高斯雜訊（$\sigma = 3/255$），計算加入前後的 PSNR 與 SSIM。雖然兩個指標都下降了，影像看起來卻可能**更**自然（比較不像塑膠）。把這個現象和感知–失真函數 $P(D)$ 連結起來。

   提示

   近似 MMSE 的輸出位於低失真的位置，但離自然影像分布很遠（太平滑）。加入顆粒會讓輸出分布更接近真實照片（d 變小），代價是 MSE 變高：這是沿著（或朝向）凸邊界的移動，而不是越過它。

6. **設計一個 all-in-one 實驗**。你要為雜訊（$\sigma \in [0, 50]$）、雨線與霧建立單一模型。說明訓練資料的混合方式、如何為網路提供條件（測試時沒有標籤）、要和哪些基準比較，以及如何偵測任務干擾。

   提示

   每個任務各自取資料集並平衡取樣；加入一個與網路一起訓練的退化編碼器（如 AirNet 的對比學習，或 PromptIR 的提示）。與三個使用相同骨幹、相同大小的單任務模型比較，也和不加條件的版本比較。任務干擾的跡象是：all-in-one 模型在某個任務上落後單任務模型，在其他任務上卻打平；另外也要在訓練時沒見過的混合退化（雨＋霧）上測試。

7. **動手估計模糊核**。在盲反卷積的程式中，(a) 拿掉 `shock(...)` 並設定 `keep=1.0`（使用全部梯度），比較模糊核誤差與最後的 PSNR；(b) 改從 3×3 的方框模糊核開始，而不是接近 delta 的模糊核；(c) 把 `h_true` 換成 31×31 視窗中一條長 25 像素的直線運動軌跡。用 Levin 等人 [74] 的討論與由粗到細的策略解釋每個結果。

提示

(a) 使用全部梯度時，平滑的潛在影像中大量的弱梯度會主導最小平方擬合，估計不再改善（我們的執行結果中，模糊核誤差不降反升，最後的 PSNR 也停留在模糊輸入的水準）。只挑選少數強而且預測良好的邊緣，模糊核步驟才有資訊量，這正是實務上對付天真聯合 MAP「沒有模糊」偏差的方法。(b) 這個迴圈是非凸的交替最佳化，起點很重要；不同的起點可能收斂到不同的局部極小值。(c) 在全解析度下，從 delta 起點很難找到大的模糊核：我們的執行結果中誤差下降得少很多，也只補回一小部分「真實模糊核」的 PSNR。先在縮小的影像上估計模糊核（那時它只有幾個像素長），再把結果放大，正是影像金字塔的用途。

8. **驗證一個 NR 指標**。五張復原影像的平均意見分數為 MOS $= (4.1, 3.2, 2.5, 1.8, 3.9)$，NIQE 分數為 $(3.4, 4.2, 5.1, 6.3, 3.0)$。(a) 手算 SRCC，並解釋它的正負號。(b) 如果改報告 $\log(\text{NIQE})$，SRCC 會改變嗎？PLCC 呢？(c) 把 MSCN 程式用在一張去雜訊後再過度銳化的影像上，解釋為什麼 NSS 指標可能給它比單純去雜訊的輸出更好的分數。

提示

(a) 由小到大排名時，影像 1–5 的 MOS 名次為 $(5, 3, 2, 1, 4)$，NIQE 名次為 $(2, 3, 4, 5, 1)$。名次差為 $(3, 0, -2, -4, 3)$，所以 $\sum d_i^2 = 38$，$\text{SRCC} = 1 - 6 \cdot 38/(5 \cdot 24) = -0.9$。符號為負，是因為 NIQE 越低代表品質越好；報告時取 $|\text{SRCC}| = 0.9$。只有影像 1 與影像 5 的順序對調。(b) SRCC 只依賴名次，任何像 log 這樣的單調轉換都不會改變它；PLCC 則會改變，這也是為什麼 PLCC 通常在擬合單調對應之後才計算。(c) 去雜訊會留下平滑的影像，其 MSCN 統計量類似輕微模糊（變異數低、直方圖尖）。銳化把高頻能量加回來，把統計量推回接近自然影像的樣子，不論加上的細節是不是真的。

參考文獻

1. Shyandram, “Low-Level Vision Task-Image Restoration 簡介,” Medium post, 2023; updated 2026. link
2. Z. Chen, Z. Wu, E. Zamfir, K. Zhang, Y. Zhang, R. Timofte et al., “NTIRE 2024 Challenge on Image Super-Resolution (×4): Methods and Results,” in *Proc. IEEE/CVF CVPR Workshops*, 2024. arXiv
3. Z. Chen, K. Liu, J. Gong, J. Wang, L. Sun et al., “NTIRE 2025 Challenge on Image Super-Resolution (×4): Methods and Results,” in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
4. X. Wang, L. Xie, C. Dong and Y. Shan, “Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data,” in *Proc. IEEE/CVF ICCV Workshops*, 2021. arXiv
5. E. H. Land and J. J. McCann, “Lightness and Retinex Theory,” *Journal of the Optical Society of America*, vol. 61, no. 1, pp. 1–11, 1971. doi
6. C. Dong, C. C. Loy, K. He and X. Tang, “Image Super-Resolution Using Deep Convolutional Networks,” *IEEE TPAMI*, 2016 (conference version ECCV 2014). arXiv
7. C. Dong, C. C. Loy and X. Tang, “Accelerating the Super-Resolution Convolutional Neural Network,” in *Proc. ECCV*, 2016. arXiv
8. W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert and Z. Wang, “Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network,” in *Proc. IEEE CVPR*, 2016. arXiv

9. A. Odena, V. Dumoulin and C. Olah, “Deconvolution and Checkerboard Artifacts,” *Distill*, 2016. link
10. B. Lim, S. Son, H. Kim, S. Nah and K. M. Lee, “Enhanced Deep Residual Networks for Single Image Super-Resolution,” in *Proc. IEEE CVPR Workshops*, 2017. arXiv
11. Y. Zhang, K. Li, K. Li, L. Wang, B. Zhong and Y. Fu, “Image Super-Resolution Using Very Deep Residual Channel Attention Networks,” in *Proc. ECCV*, 2018. arXiv
12. C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang and W. Shi, “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *Proc. IEEE CVPR*, 2017. arXiv
13. X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, C. C. Loy, Y. Qiao and X. Tang, “ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks,” in *Proc. ECCV Workshops*, 2018. arXiv
14. J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool and R. Timofte, “SwinIR: Image Restoration Using Swin Transformer,” in *Proc. IEEE/CVF ICCV Workshops*, 2021. arXiv
15. X. Chen, X. Wang, J. Zhou, Y. Qiao and C. Dong, “Activating More Pixels in Image Super-Resolution Transformer,” in *Proc. IEEE/CVF CVPR*, 2023. arXiv
16. X. Lin, J. He, Z. Chen, Z. Lyu, B. Dai, F. Yu, W. Ouyang, Y. Qiao and C. Dong, “DiffBIR: Towards Blind Image Restoration with Generative Diffusion Prior,” arXiv:2308.15070, 2023. arXiv
17. F. Yu, J. Gu, Z. Li, J. Hu, X. Kong, X. Wang, J. He, Y. Qiao and C. Dong, “Scaling Up to Excellence: Practicing Model Scaling for Photo-Realistic Image Restoration In the Wild,” in *Proc. IEEE/CVF CVPR*, 2024. arXiv
18. R. Wu, L. Sun, Z. Ma and L. Zhang, “One-Step Effective Diffusion Network for Real-World Image Super-Resolution,” in *Proc. NeurIPS*, 2024. arXiv
19. E. Agustsson and R. Timofte, “NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study,” in *Proc. IEEE CVPR Workshops*, 2017. doi
20. K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, “Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising,” *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017. arXiv
21. K. Zhang, W. Zuo and L. Zhang, “FFDNet: Toward a Fast and Flexible Solution for CNN-Based Image Denoising,” *IEEE Transactions on Image Processing*, 2018. arXiv
22. A. Abdelhamed, S. Lin and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *Proc. IEEE/CVF CVPR*, 2018. doi
23. S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan and M.-H. Yang, “Restormer: Efficient Transformer for High-Resolution Image Restoration,” in *Proc. IEEE/CVF CVPR*, 2022. arXiv
24. L. Chen, X. Chu, X. Zhang and J. Sun, “Simple Baselines for Image Restoration,” in *Proc. ECCV*, 2022. arXiv
25. K. He, J. Sun and X. Tang, “Single Image Haze Removal Using Dark Channel Prior,” in *Proc. IEEE CVPR*, 2009. doi
26. B. Cai, X. Xu, K. Jia, C. Qing and D. Tao, “DehazeNet: An End-to-End System for Single Image Haze Removal,” *IEEE Transactions on Image Processing*, 2016. arXiv
27. B. Li, X. Peng, Z. Wang, J. Xu and D. Feng, “AOD-Net: All-in-One Dehazing Network,” in *Proc. IEEE ICCV*, 2017. doi ·arXiv
28. X. Liu, Y. Ma, Z. Shi and J. Chen, “GridDehazeNet: Attention-Based Multi-Scale Network for Image Dehazing,” in *Proc. IEEE/CVF ICCV*, 2019. arXiv

29. X. Qin, Z. Wang, Y. Bai, X. Xie and H. Jia, “FFA-Net: Feature Fusion Attention Network for Single Image Dehazing,” in *Proc. AAAI*, 2020. arXiv
30. S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, M.-H. Yang and L. Shao, “Learning Enriched Features for Real Image Restoration and Enhancement,” in *Proc. ECCV*, 2020. arXiv
31. Y. Song, Z. He, H. Qian and X. Du, “Vision Transformers for Single Image Dehazing,” *IEEE Transactions on Image Processing*, vol. 32, 2023. arXiv
32. B. Li, W. Ren, D. Fu, D. Tao, D. Feng, W. Zeng and Z. Wang, “Benchmarking Single-Image Dehazing and Beyond,” *IEEE Transactions on Image Processing*, 2019. arXiv
33. C. O. Ancuti, C. Ancuti, R. Timofte and C. De Vleeschouwer, “O-HAZE: A Dehazing Benchmark with Real Hazy and Haze-Free Outdoor Images,” in *Proc. IEEE/CVF CVPR Workshops*, 2018. doi
34. C. O. Ancuti, C. Ancuti, M. Sbert and R. Timofte, “Dense-Haze: A Benchmark for Image Dehazing with Dense-Haze and Haze-Free Images,” in *Proc. IEEE ICIP*, 2019. doi
35. A. Geiger, P. Lenz and R. Urtasun, “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *Proc. IEEE CVPR*, 2012. project page
36. Y.-F. Liu, D.-W. Jaw, S.-C. Huang and J.-N. Hwang, “DesnowNet: Context-Aware Deep Network for Snow Removal,” *IEEE Transactions on Image Processing*, 2018. arXiv
37. W.-T. Chen, H.-Y. Fang, J.-J. Ding, C.-C. Tsai and S.-Y. Kuo, “JSTASR: Joint Size and Transparency-Aware Snow Removal Algorithm Based on Modified Partial Convolution and Veiling Effect Removal,” in *Proc. ECCV*, 2020. doi
38. W. Yang, R. T. Tan, J. Feng, J. Liu, Z. Guo and S. Yan, “Deep Joint Rain Detection and Removal from a Single Image,” in *Proc. IEEE CVPR*, 2017. arXiv
39. R. Qian, R. T. Tan, W. Yang, J. Su and J. Liu, “Attentive Generative Adversarial Network for Raindrop Removal from a Single Image,” in *Proc. IEEE/CVF CVPR*, 2018. arXiv
40. X. Li, Y. Jin, X. Jin, Z. Wu, B. Li, Y. Wang et al., “NTIRE 2025 Challenge on Day and Night Raindrop Removal for Dual-Focused Images: Methods and Results,” in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
41. C. Wei, W. Wang, W. Yang and J. Liu, “Deep Retinex Decomposition for Low-Light Enhancement,” in *Proc. BMVC*, 2018. arXiv
42. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, “Retinexformer: One-stage Retinex-based Transformer for Low-light Image Enhancement,” in *Proc. IEEE/CVF ICCV*, 2023. arXiv
43. H. Zhang, H. Xu, X. Tian, J. Jiang and J. Ma, “Image Fusion Meets Deep Learning: A Survey and Perspective,” *Information Fusion*, vol. 76, pp. 323–336, 2021. doi
44. S. Nah, T. H. Kim and K. M. Lee, “Deep Multi-scale Convolutional Neural Network for Dynamic Scene Deblurring,” in *Proc. IEEE CVPR*, 2017. doi ·arXiv
45. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, “Masked Autoencoders Are Scalable Vision Learners,” in *Proc. IEEE/CVF CVPR*, 2022. arXiv
46. R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park and V. Lempitsky, “Resolution-robust Large Mask Inpainting with Fourier Convolutions,” in *Proc. IEEE/CVF WACV*, 2022. arXiv
47. S. Sengupta, V. Jayaram, B. Curless, S. Seitz and I. Kemelmacher-Shlizerman, “Background Matting: The World is Your Green Screen,” in *Proc. IEEE/CVF CVPR*, 2020. arXiv
48. O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Proc. MICCAI*, 2015. arXiv

49. H. Guo, J. Li, T. Dai, Z. Ouyang, X. Ren and S.-T. Xia, "MambaIR: A Simple Baseline for Image Restoration with State-Space Model," in *Proc. ECCV*, 2024. arXiv
50. H. Guo, Y. Guo, Y. Zha, Y. Zhang, W. Li, T. Dai, S.-T. Xia and Y. Li, "MambaIRv2: Attentive State Space Restoration," in *Proc. IEEE/CVF CVPR*, 2025. arXiv
51. B. Li, X. Liu, P. Hu, Z. Wu, J. Lv and X. Peng, "All-In-One Image Restoration for Unknown Corruption," in *Proc. IEEE/CVF CVPR*, 2022. code & paper
52. J. M. J. Valanarasu, R. Yasarla and V. M. Patel, "TransWeather: Transformer-based Restoration of Images Degraded by Adverse Weather Conditions," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
53. V. Potlapalli, S. W. Zamir, S. Khan and F. S. Khan, "PromptIR: Prompting for All-in-One Image Restoration," in *Proc. NeurIPS*, 2023. arXiv
54. Y. Cui, S. W. Zamir, S. Khan, A. Knoll, M. Shah and F. S. Khan, "AdaIR: Adaptive All-in-One Image Restoration via Frequency Mining and Modulation," in *Proc. ICLR*, 2025. arXiv
55. Z. Luo, F. K. Gustafsson, Z. Zhao, J. Sjölund and T. B. Schön, "Controlling Vision-Language Models for Multi-Task Image Restoration," in *Proc. ICLR*, 2024. arXiv
56. M. V. Conde, G. Geigle and R. Timofte, "InstructIR: High-Quality Image Restoration Following Human Instructions," in *Proc. ECCV*, 2024. arXiv
57. H. Zhao, O. Gallo, I. Frosio and J. Kautz, "Loss Functions for Image Restoration with Neural Networks," *IEEE Transactions on Computational Imaging*, 2017 (arXiv title: "Loss Functions for Neural Networks for Image Processing"). arXiv
58. J. Johnson, A. Alahi and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *Proc. ECCV*, 2016. doi ·arXiv
59. L. Jiang, B. Dai, W. Wu and C. C. Loy, "Focal Frequency Loss for Image Reconstruction and Synthesis," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
60. Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi
61. R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
62. A. Mittal, R. Soundararajan and A. C. Bovik, "Making a 'Completely Blind' Image Quality Analyzer," *IEEE Signal Processing Letters*, 2013. doi
63. Y. Blau, R. Mechrez, R. Timofte, T. Michaeli and L. Zelnik-Manor, "The 2018 PIRM Challenge on Perceptual Image Super-resolution," in *Proc. ECCV Workshops*, 2018. arXiv
64. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *Proc. NeurIPS*, 2017. arXiv
65. Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
66. Z. Wang, J. Chen and S. C. H. Hoi, "Deep Learning for Image Super-Resolution: A Survey," *IEEE TPAMI*, 2021. arXiv
67. C. Tian, L. Fei, W. Zheng, Y. Xu, W. Zuo and C.-W. Lin, "Deep Learning on Image Denoising: An Overview," *Neural Networks*, vol. 131, pp. 251–275, 2020. arXiv
68. M. Elad, B. Kawar and G. Vaksman, "Image Denoising: The Deep Learning Revolution and Beyond — A Survey Paper," *SIAM Journal on Imaging Sciences*, vol. 16, no. 3, pp. 1594–1654, 2023. doi ·arXiv
69. K. Zhang, W. Ren, W. Luo, W.-S. Lai, B. Stenger, M.-H. Yang and H. Li, "Deep Image Deblurring:

A Survey," *International Journal of Computer Vision*, 2022. arXiv
70. J. Gui, X. Cong, Y. Cao, W. Ren, J. Zhang, J. Zhang, J. Cao and D. Tao, "A Comprehensive Survey and Taxonomy on Single Image Dehazing Based on Deep Learning," *ACM Computing Surveys*, 2023. doi ·arXiv
71. W. Yang, R. T. Tan, S. Wang, Y. Fang and J. Liu, "Single Image Deraining: From Model-Based to Data-Driven and Beyond," *IEEE TPAMI*, 2021. doi ·arXiv
72. J. Jiang, Z. Zuo, G. Wu, K. Jiang and X. Liu, "A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends," *IEEE TPAMI*, 2025. arXiv
73. X. Li, Y. Ren, X. Jin, C. Lan, X. Wang, W. Zeng, X. Wang and Z. Chen, "Diffusion Models for Image Restoration and Enhancement: A Comprehensive Survey," *International Journal of Computer Vision*, 2025. arXiv
74. A. Levin, Y. Weiss, F. Durand and W. T. Freeman, "Understanding and Evaluating Blind Deconvolution Algorithms," in *Proc. IEEE CVPR*, 2009, pp. 1964–1971. doi
75. R. Fergus, B. Singh, A. Hertzmann, S. T. Roweis and W. T. Freeman, "Removing Camera Shake from a Single Photograph," *ACM Transactions on Graphics* (Proc. SIGGRAPH), vol. 25, no. 3, pp. 787–794, 2006. doi
76. Q. Shan, J. Jia and A. Agarwala, "High-quality Motion Deblurring from a Single Image," *ACM Transactions on Graphics* (Proc. SIGGRAPH), vol. 27, no. 3, 2008. doi
77. S. Cho and S. Lee, "Fast Motion Deblurring," *ACM Transactions on Graphics* (Proc. SIGGRAPH Asia), vol. 28, no. 5, 2009. doi
78. L. Xu and J. Jia, "Two-Phase Kernel Estimation for Robust Motion Deblurring," in *Proc. ECCV*, 2010, pp. 157–170. doi
79. D. Krishnan, T. Tay and R. Fergus, "Blind Deconvolution Using a Normalized Sparsity Measure," in *Proc. IEEE CVPR*, 2011, pp. 233–240. doi
80. J. Pan, D. Sun, H. Pfister and M.-H. Yang, "Blind Image Deblurring Using Dark Channel Prior," in *Proc. IEEE CVPR*, 2016, pp. 1628–1636. doi
81. J. Sun, W. Cao, Z. Xu and J. Ponce, "Learning a Convolutional Neural Network for Non-uniform Motion Blur Removal," in *Proc. IEEE CVPR*, 2015. arXiv
82. C. J. Schuler, M. Hirsch, S. Harmeling and B. Schölkopf, "Learning to Deblur," *IEEE TPAMI*, vol. 38, no. 7, pp. 1439–1451, 2016. doi ·arXiv
83. A. Chakrabarti, "A Neural Approach to Blind Motion Deblurring," in *Proc. ECCV*, 2016. arXiv
84. D. Ulyanov, A. Vedaldi and V. Lempitsky, "Deep Image Prior," in *Proc. IEEE/CVF CVPR*, 2018, pp. 9446–9454. doi ·arXiv
85. D. Ren, K. Zhang, Q. Wang, Q. Hu and W. Zuo, "Neural Blind Deconvolution Using Deep Priors," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
86. O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin and J. Matas, "DeblurGAN: Blind Motion Deblurring Using Conditional Adversarial Networks," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
87. K. Zhang, W. Zuo and L. Zhang, "Learning a Single Convolutional Super-Resolution Network for Multiple Degradations," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
88. J. Gu, H. Lu, W. Zuo and C. Dong, "Blind Super-Resolution With Iterative Kernel Correction," in *Proc. IEEE/CVF CVPR*, 2019. arXiv
89. S. Bell-Kligler, A. Shocher and M. Irani, "Blind Super-Resolution Kernel Estimation using an Internal-GAN," in *Proc. NeurIPS*, 2019. link ·arXiv
90. Z. Luo, Y. Huang, S. Li, L. Wang and T. Tan, "Unfolding the Alternating Optimization for Blind

Super Resolution," in *Proc. NeurIPS*, 2020. arXiv
91. K. Zhang, L. Van Gool and R. Timofte, "Deep Unfolding Network for Image Super-Resolution," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
92. K. Zhang, J. Liang, L. Van Gool and R. Timofte, "Designing a Practical Degradation Model for Deep Blind Image Super-Resolution," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
93. A. Liu, Y. Liu, J. Gu, Y. Qiao and C. Dong, "Blind Image Super-Resolution: A Survey and Beyond," *IEEE TPAMI*, 2022 (early access). doi ·arXiv
94. A. Mittal, A. K. Moorthy and A. C. Bovik, "No-Reference Image Quality Assessment in the Spatial Domain," *IEEE Transactions on Image Processing*, vol. 21, no. 12, pp. 4695–4708, 2012. doi
95. C. Ma, C.-Y. Yang, X. Yang and M.-H. Yang, "Learning a No-Reference Quality Metric for Single-Image Super-Resolution," *Computer Vision and Image Understanding*, vol. 158, pp. 1–16, 2017. doi ·arXiv
96. N. Venkatanath, D. Praneeth, M. C. Bh, S. S. Channappayya and S. S. Medasani, "Blind Image Quality Evaluation Using Perception Based Features," in *Proc. Twenty First National Conference on Communications (NCC)*, 2015, pp. 1–6. doi ·MATLAB documentation
97. H. Talebi and P. Milanfar, "NIMA: Neural Image Assessment," *IEEE Transactions on Image Processing*, vol. 27, no. 8, pp. 3998–4011, 2018. doi ·arXiv
98. S. Su, Q. Yan, Y. Zhu, C. Zhang, X. Ge, J. Sun and Y. Zhang, "Blindly Assess Image Quality in the Wild Guided by a Self-Adaptive Hyper Network," in *Proc. IEEE/CVF CVPR*, 2020, pp. 3664–3673. doi
99. J. Ke, Q. Wang, Y. Wang, P. Milanfar and F. Yang, "MUSIQ: Multi-scale Image Quality Transformer," in *Proc. IEEE/CVF ICCV*, 2021, pp. 5128–5137. doi ·arXiv
100. S. Yang, T. Wu, S. Shi, S. Lao, Y. Gong, M. Cao, J. Wang and Y. Yang, "MANIQA: Multi-dimension Attention Network for No-Reference Image Quality Assessment," in *Proc. IEEE/CVF CVPR Workshops*, 2022, pp. 1190–1199. doi ·arXiv
101. C. Chen, J. Mo, J. Hou, H. Wu, L. Liao, W. Sun, Q. Yan and W. Lin, "TOPIQ: A Top-Down Approach From Semantics to Distortions for Image Quality Assessment," *IEEE Transactions on Image Processing*, vol. 33, pp. 2404–2418, 2024. doi ·arXiv
102. J. Wang, K. C. K. Chan and C. C. Loy, "Exploring CLIP for Assessing the Look and Feel of Images," in *Proc. AAAI Conference on Artificial Intelligence*, 2023. arXiv
103. H. Wu, Z. Zhang, W. Zhang, C. Chen, L. Liao, C. Li, Y. Gao, A. Wang, E. Zhang, W. Sun, Q. Yan, X. Min, G. Zhai and W. Lin, "Q-Align: Teaching LMMs for Visual Scoring via Discrete Text-Defined Levels," in *Proc. ICML*, PMLR 235, pp. 54015–54029, 2024. link ·arXiv
104. H. R. Sheikh, M. F. Sabir and A. C. Bovik, "A Statistical Evaluation of Recent Full Reference Image Quality Assessment Algorithms," *IEEE Transactions on Image Processing*, vol. 15, no. 11, pp. 3440–3451, 2006 (LIVE Image Quality Assessment Database). doi ·database
105. N. Ponomarenko, L. Jin, O. Ieremeiev, V. Lukin, K. Egiazarian, J. Astola, B. Vozel, K. Chehdi, M. Carli, F. Battisti and C.-C. J. Kuo, "Image Database TID2013: Peculiarities, Results and Perspectives," *Signal Processing: Image Communication*, vol. 30, pp. 57–77, 2015. doi ·database
106. H. Lin, V. Hosu and D. Saupe, "KADID-10k: A Large-scale Artificially Distorted IQA Database," in *Proc. Eleventh International Conference on Quality of Multimedia Experience (QoMEX)*, 2019. doi ·database
107. V. Hosu, H. Lin, T. Sziranyi and D. Saupe, "KonIQ-10k: An Ecologically Valid Database for

Deep Learning of Blind Image Quality Assessment," *IEEE Transactions on Image Processing*, vol. 29, pp. 4041–4056, 2020. arXiv

108. Y. Fang, H. Zhu, Y. Zeng, K. Ma and Z. Wang, "Perceptual Quality Assessment of Smartphone Photography," in *Proc. IEEE/CVF CVPR*, 2020, pp. 3674–3683. doi
109. J. Gu, H. Cai, H. Chen, X. Ye, J. Ren and C. Dong, "PIPAL: A Large-Scale Image Quality Assessment Dataset for Perceptual Image Restoration," in *Proc. ECCV*, 2020. arXiv
110. C. Chen and J. Mo, "IQA-PyTorch: PyTorch Toolbox for Image Quality Assessment," software (pip package `pyiqa`), 2022. GitHub

DLCV 05・低光影像增強

從直方圖等化、Retinex 到曲線估計、Retinex Transformer 與擴散模型：如何讓暗照片變亮，又不放大雜訊、不造成色偏、也不被評估指標騙了。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-05-low-light-enhancement/

你將學到

- 為什麼低光影像增強（low-light image enhancement, LLIE）其實是兩個問題的合體：**色調強化**與**細節（含雜訊）還原**，以及為什麼「直接調亮」行不通。
- 傳統工具箱如何運作：HSV、HSI、YCbCr、Lab 中的亮度通道；直方圖等化與 CLAHE；gamma 曲線；Retinex（SSR、MSR、LIME）；以及「反轉後去霧」的技巧。
- 深度 LLIE 方法如何依**訓練訊號**分類（監督式、展開式、非配對、零參考、生成式），並推導 Retinex-Net 分解損失函數與 Zero-DCE 曲線的數學。
- 如何誠實地評估增強結果：全參考指標、無參考指標（NIQE、BRISQUE、PI、明度順序誤差、學習式評估器）與使用者研究、**GT-mean** 陷阱，以及以任務為導向的評估。
- 常用資料集與挑戰賽（LOL、LOL-v2、LSRW、SDSD、SID、FiveK、ExDark、NTIRE 2024/2025）各自用來做什麼，以及還有哪些問題沒解決。

先看全貌

在暗處拍到的照片，並不只是「把亮度調低的正常照片」。感測器收到的光子非常少，所以影像又暗、又有雜訊，還常常帶著色偏。低光影像增強的目標，是產生「如果當時光線充足，你會拍到的那張照片」：曝光正確、顏色自然、細節乾淨，而且不引入新的假影。

> **用最簡單的話說**
>
> 想像你在幾乎關燈的房間裡畫畫。畫出來又淡又髒，你也搞不清楚哪一筆是哪個顏色。事後開燈、把這張畫「影印得亮一點」是沒用的：髒污也跟著變亮，畫錯的顏色還是錯的。好的低光增強器，就像一位朋友看著你那張淡淡的畫，把它重新畫一遍，畫成白天看起來的樣子：同一張圖、正確的顏色，而且沒有髒污。

為什麼重要：手機每天都在暗處拍照，車輛與機器人必須在夜間看得見，監視系統、科學影像與醫療內視鏡也都在跟同樣的物理限制奮戰。低光也會傷害下游模型：用白天照片訓練的偵測器，到了晚上準確率會下降。因此 LLIE 既是畫質問題，也是電腦視覺的強健性問題。

> **關於本篇**
>
> 本篇由作者先前的 LLIE 文章 [1] 擴寫而成（原文在此），依照原文的大綱，補上數學、程式碼、圖表與文獻綜述。

目的與難點

白話版。我們想把一張暗的影像對應成一張曝光正常的影像。難的地方在於「變亮」和「修復」是兩件不同的工作：變亮改變的是色調（每個區域多亮多暗），修復則是找回被黑暗與雜訊破壞的細節與乾淨的顏色。

精確版。 LLIE 和影像轉換（image-to-image translation）不同：後者會改變影像的屬性（夏天變冬天、照片變畫作），LLIE 則應該完整保留場景與內容，只把光線不足造成的破壞還原。原文 [1] 把這些破壞分成兩個難點：

1. **色調強化**。在全域與局部提高曝光，同時不壓垮亮部、也不把對比壓平。燈具這類亮區必須維持明亮，暗區要被打開，而且區域之間的明暗順序應該保留。
2. **細節還原**。找回埋在雜訊與量化誤差裡的紋理與顏色，同時抑制雜訊而不是放大它。

在深度學習裡，只要網路被訓練成輸出乾淨的影像，細節還原本來就是它在做的事，因此多數方法的設計重點放在色調（要把哪些影像性質或物理模型放進網路）與學習機制（用什麼監督方式與損失函數），而不是另外設計一個專門的細節模組。

為什麼不是單純調亮

線性增益 $y = c\,x$（$c > 1$）會在四個地方失敗，全都可以在圖 1 看到：

- **雜訊**。光子到達是隨機的。光子很少時，訊噪比很低，而增益會把訊號和雜訊一起放大。影像變亮了，卻沒有變乾淨。
- **色偏**。低光環境的光常帶有顏色（鎢絲燈、鈉燈路燈），相機的白平衡與三個色彩通道的雜訊程度也不同。三通道等比例放大會保留色偏；分別放大又可能產生奇怪的顏色。
- **飽和與過曝**。已經接近上限的像素（燈、窗戶、螢幕）會在 1 被截斷。大到足以照亮暗部的全域增益，會毀掉亮部。
- **量化**。一張暗的 8 位元影像可能只用到 20 到 40 個灰階。把它們拉開，就會出現明顯的色階斷層（banding）。

低光拍攝的模型

明確寫出退化模型很有幫助，一方面便於理解，另一方面可用來**合成訓練資料**。請在線性光（數值正比於光子數）上操作，而不是在經過 gamma 編碼的像素上（gamma 請見 IP 03）。常用的是 Foi 等人的 Poisson–Gaussian 模型 [7]：

$$z = \frac{1}{\eta}\,\mathcal{P}(\eta\, k\, x) + n, \qquad n \sim \mathcal{N}(0, \sigma_r^2),$$

其中 $x \in [0,1]$ 是乾淨的線性影像，$k \in (0,1]$ 是到達感測器的光量比例，η 是把強度換算成光子數的尺度（η 越大代表光子越多、相對雜訊越小），$\mathcal{P}(\cdot)$ 是模擬**散粒雜訊**（shot noise）的 Poisson 隨機變數，n 是標準差 σ_r 的高斯**讀出雜訊**（read noise）。之後相機再套上色調曲線並量化。z 的變異數約為 $kx/\eta + \sigma_r^2$，會隨訊號增加，也就是說雜訊是與訊號相關的；這就是為什麼「加高斯雜訊」很難代表真實的低光。

```
import numpy as np
from skimage import data, img_as_float

rng = np.random.default_rng(0)
clean = img_as_float(data.coffee())            # H x W x 3, sRGB-encoded, in [0, 1]

def synth_low_light(x, k=0.05, peak=2000.0, read_std=8e-4, gamma=2.2):
    lin = k * x ** gamma                           # 1) less light, in linear units
    lin = rng.poisson(lin * peak) / peak           # 2) shot noise (Poisson)
    lin = lin + rng.normal(0, read_std, lin.shape) # 3) read noise (Gaussian)
    out = np.clip(lin, 0, 1) ** (1 / gamma)        # 4) re-encode for display
    return np.round(out * 255) / 255               # 5) 8-bit quantization

dark = synth_low_light(clean)
print(clean.mean(), dark.mean())                   # about 0.39 -> 0.10
```

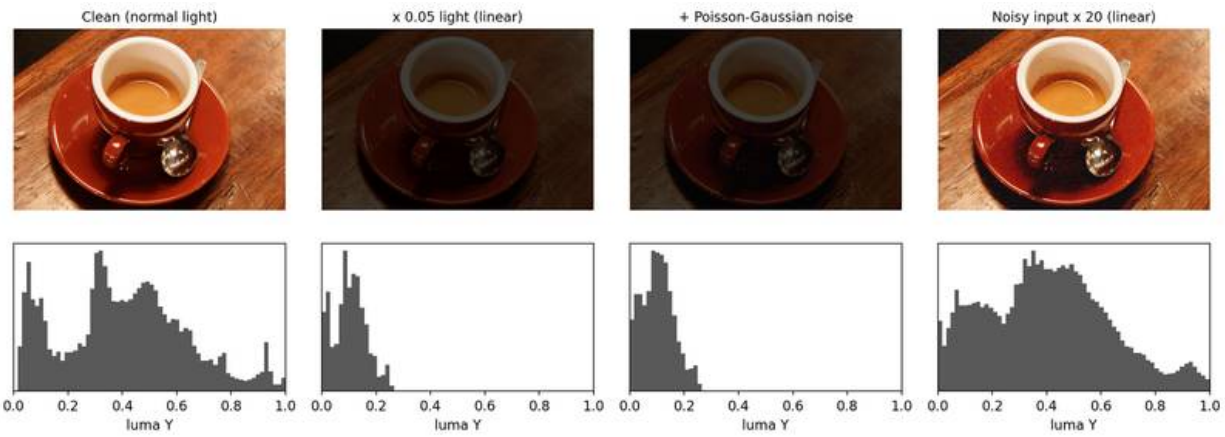


圖 *1* — 合成的低光影像。拿掉 *95%* 的光，直方圖被擠進最下方四分之一。在暗影像裡很難看到雜訊，但用完美的增益把曝光還原後（最右），雜訊就現形了：調亮與去噪是兩個獨立的問題。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

最右邊那張是本篇最重要的一張圖：即使是理想的亮度校正，雜訊依然存在。這就是為什麼 LLIE 是一個復原問題（請見DLCV 04・影像復原），而不只是色調對應問題。

非深度學習的 LLIE 與低光影像的特性

白話版。在深度學習之前，主要有兩大流派：**直方圖**類方法重新分配灰階；**Retinex** 類方法把影像拆成「光照」和「物體」兩部分，再修正光照。第三種想法借自去霧，可以歸進 Retinex 家族；統計方法則延伸了以上各類。

低光影像在色彩空間上的特性

許多色彩空間會把「有多亮」的通道和「是什麼顏色」的通道分開。不同空間稱它為明度（value）、強度（intensity）、亮度（luma）或明亮度（lightness），定義也各不相同。設 $R, G, B \in [0, 1]$（色彩空間本身請見 IP 06）：

$$\begin{aligned} \text{HSV:} \quad & V = \max(R, G, B), \\ \text{HSI:} \quad & I = \tfrac{1}{3}(R + G + B), \\ \text{YCbCr (BT.601):} \quad & Y' = 0.299\,R' + 0.587\,G' + 0.114\,B', \\ \text{CIELAB:} \quad & L^* = 116\,f(Y_{\text{lin}}/Y_n) - 16. \end{aligned}$$

其中 R', G', B' 是經 gamma 編碼的值；Y_{lin} 是線性光的亮度，Y_n 是參考白的亮度；$f(t)$ 在一個小門檻以上為 $t^{1/3}$，以下為線性。幾點說明：

- **V 就是「亮通道」（bright channel）**。它是逐像素的最大值，與去霧中使用的暗通道（dark channel，逐區塊的最小值）[14] 相對。在暗影像中，V 是被黑暗破壞最少的通道，這也是 LIME [17] 用它來初始化光照的原因。

- **亮度權重**。 ITU-R BT.601 [8] 定義亮度為 $Y' = 0.299R' + 0.587G' + 0.114B'$，這個全範圍（full range）形式就是 JPEG/JFIF 使用的版本。8 位元的有限範圍（limited range，又稱 studio range）影片中，BT.601 把黑色放在 16、白色放在 235：$Y = 16 + 219\,Y'$。若 $R', G', B' \in [0, 255]$，就變成 $Y = 16 + 0.257R' + 0.504G' + 0.098B'$，因為 $219/255 \times (0.299, 0.587, 0.114) \approx (0.257, 0.504, 0.098)$。兩個公式描述的是同一個亮度，只差在範圍與偏移量。
- L^* 被設計成感知上均勻，相同的數值差在人眼看來差不多大，因此很適合拿來做對比增強。

實務上的結論是：既然亮度可以和顏色拆開，就可以**只增強亮度通道**並保留色度。圖 2 顯示，選哪個通道影響很大。

```
R, G, B = dark[..., 0], dark[..., 1], dark[..., 2]
V = dark.max(axis=2)                          # HSV value
I = dark.mean(axis=2)                         # HSI intensity
Y = 0.299 * R + 0.587 * G + 0.114 * B         # BT.601 luma, full range (JPEG/JFIF)
Y_limited = 16 + 219 * Y                      # 8-bit limited ("studio") range, 16..235
print(np.allclose(Y_limited, 16 + (65.481 * R + 128.553 * G + 24.966 * B)))  # True
```



圖 2 — 同一條 *gamma* 曲線套用在不同色彩空間。提高 V 或 I 等於三個通道乘上同一個倍數，色相不變但飽和度增強（非常橘）；固定色度只提高 Y 或 L^*，結果則會褪色。兩者都不是「正確答案」：色調與色彩是耦合的。

圖中可以看到兩種效應。以共同倍數縮放 RGB（HSV、HSI）會保留色相與通道比例，於是本來就飽和的物體變得非常飽和；只改 Y' 或 L^*、固定 Cb/Cr 或 a*/b*，則是只加亮度不加顏色，影像看起來就褪色了。色調與色彩的這種耦合，正是後來 HVI-CIDNet [27] 想用學習出來的色彩空間處理的問題。

直方圖等化與 CLAHE

白話版。直方圖等化（histogram equalization, HE）把影像的灰階攤開，讓每個亮度值被使用的機會差不多。暗影像的值擠在 0 附近，等化後就會被拉滿整個範圍。

精確版。對 L 個灰階、正規化直方圖 $p_r(r_k) = n_k/(MN)$ 的影像，HE 以縮放後的累積分布函數（CDF）作為對應：

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j), \qquad k = 0, \dots, L-1,$$

其中 n_k 是灰階 r_k 的像素數，MN 是影像大小。T 的斜率正比於直方圖，所以像素很多的灰階會被拉開。推導請見 IP 03。對彩色影像，常見做法是先轉到有亮度通道的色彩空間，只等化該通道，再轉回來，例如 RGB → Lab → 等化 L^* → RGB。

全域 HE 在低光下有兩個問題：它會過度拉伸大片平坦區域（連同其中的雜訊），而且單一對應無法同時照顧暗角與明亮的燈。**自適應 HE**（adaptive HE）為每個區塊計算對應；**CLAHE**（contrast-limited AHE）在計算 CDF 前先把每個區塊的直方圖在某個上限截斷，藉此限制 T 的斜率，也就限制了雜訊增益，最後在相鄰區塊間內插以避免接縫 [9], [10]。

Gamma 校正

冪次律 $s = r^\gamma$ 在 $\gamma < 1$ 時對暗部的提升大於亮部；它的斜率 $\gamma r^{\gamma-1}$ 在 $r = 0$ 附近很大。它便宜又單調，但屬於全域操作：整張圖只有一個 γ、不會去噪，而且依套用的空間不同，同樣面臨上述「褪色或過飽和」的選擇。

```
from skimage import color, exposure

gamma_out = dark ** 0.4                                    # power-law curve on every
 ↪  channel
lab = color.rgb2lab(dark)
L = lab[..., 0] / 100                                      # L* scaled to [0, 1]
he = lab.copy();    he[..., 0] = 100 * exposure.equalize_hist(L)
clahe = lab.copy(); clahe[..., 0] = 100 * exposure.equalize_adapthist(L, clip_limit=0.01)
he_out, clahe_out = color.lab2rgb(he), color.lab2rgb(clahe)
for name, im in [("gamma", gamma_out), ("HE", he_out), ("CLAHE", clahe_out)]:
    print(f"{name:6s} mean {im.mean():.3f}   std {im.std():.3f}")
```



圖 3 — 傳統增強方法作用在圖 1 的有雜訊輸入上。全部都變亮了，但沒有一個去除雜訊；在輸入最暗的地方，*Retinex* 的輸出出現強烈的彩色雜訊。

Retinex 理論

白話版。 Retinex 認為，眼睛（或相機）接收到的影像是「物體本身的顏色」乘上「照在它上面的光」。如果能估計出光，就能把它除掉，看到物體真正的樣子，再換上一個更好的光。

精確版。 Land 的 Retinex 理論 [11] 把觀察到的影像 S 表示為

$$S(\mathbf{x}) = R(\mathbf{x}) \circ L(\mathbf{x}),$$

其中 $\mathbf{x}$ 是像素位置，$R \in [0,1]$ 是**反射率**（reflectance，物體表面的性質，假設在任何光照下都不變），L 是**照度**（illumination，照在場景上的光，除了陰影邊界之外在空間上平滑變化），$\circ$ 是逐元素相乘。請注意用詞：L 是照度，不是 luminance；luminance 通常指色彩空間裡的亮度通道，例如上面的 Y。

從一個乘積解出兩個未知數是病態問題（ill-posed），因此每個 Retinex 方法都會加上先驗。最經典的是「照度是平滑的」。

單尺度 Retinex（SSR） [12] 用高斯模糊 G_σ 估計 L，並在對數域運算，讓乘法變成加法：

$$\log R_c(\mathbf{x}) = \log S_c(\mathbf{x}) - \log\left(G_\sigma * S_c\right)(\mathbf{x}),$$

對每個色彩通道 c 計算，$*$ 為卷積，σ 為周圍（surround）尺度。σ 小，局部對比強但邊緣出現光暈（halo）；σ 大，色調較好但細節較弱。**多尺度 Retinex（MSR）** [13] 以權重 w_n 平均多個尺度（通常三個）的 SSR 輸出：

$$\log R_c^{\mathrm{MSR}} = \sum_{n=1}^{N} w_n \left[\log S_c - \log(G_{\sigma_n} * S_c)\right].$$

由於 SSR 與 MSR 逐通道運算，顏色容易變灰。**MSRCR**（帶色彩還原的 MSR）[13] 將結果乘上一個色彩還原因子，依據是各通道佔 $R+G+B$ 的比例；其他變體則只對強度通道做 MSR，並保留輸入的色度。

```
from scipy import ndimage as ndi

def ssr(img, sigma=40.0, eps=1e-3):
    log_s = np.log(img + eps)
    log_l = np.log(ndi.gaussian_filter(img, sigma=(sigma, sigma, 0)) + eps)  # L ~ Gaussian
     ↪  blur
    r = log_s - log_l                                          # log R = log S - log L
    lo, hi = np.percentile(r, [1, 99])                         # robust stretch for display
    return np.clip((r - lo) / (hi - lo), 0, 1)

msr = np.mean([ssr(dark, s) for s in (15, 80, 250)], axis=0)   # MSR, equal weights
 ↪  (simplified)
```

做增強時，我們通常不只要 R，因為單獨的 R 看起來平板又不自然。我們要的是把 L 調亮再組合回去：$\hat{S} = R \circ \hat{L}$，例如 $\hat{L} = L^{\gamma}$、$\gamma < 1$。這個「分解、調整 L、重組」的模式，就是大多數 Retinex 深度網路的範本。



圖 *4 — LIME 風格的 Retinex 分解。照度以 max(R, G, B) 初始化後再平滑；反射率為 S / L。把 L 以 gamma 0.35 調亮再乘回去，得到自然的結果；單獨的 R 則顯得平板而過度飽和。*

LIME：把 Retinex 寫成最佳化問題

LIME [17] 只估計照度。它以亮通道 $\hat{T}(\mathbf{x}) = \max_c S_c(\mathbf{x})$ 初始化，再解一個保留結構的平滑問題來精修：讓 T 接近 $\hat{T}$，但除了影像有強結構的地方之外都要平滑。增強後的影像為 S/T^γ；由於除法會放大雜訊，LIME 可以選擇在之後去噪。LIME 論文的影像（常稱 LIME 資料集）成為標準測試集，而這個方法正是傳統配方的好例子：一個物理模型、一個手工設計的先驗、再加上最佳化。

另一個模型：對反轉影像去霧

Retinex 並不是唯一的成像模型。去霧使用的是**大氣散射模型**（atmospheric scattering model）：

$$I(\mathbf{x}) = J(\mathbf{x})\, t(\mathbf{x}) + A\,(1 - t(\mathbf{x})),$$

其中 I 是有霧的觀測影像，J 是無霧的場景輻射，$t \in (0, 1]$ 是透射率（場景光有多少能留下來），A 是全域大氣光。這個模型不能直接套用在暗影像上，因為暗影像看起來並不像有霧的影像。

Dong 等人觀察到，**反轉**後的低光影像 $1 - S$ 確實像有霧：暗區變成明亮、低對比的區域，就像霧一樣。因此他們先反轉影像，套用去霧演算法（精神上與暗通道先驗 [14] 相同），再把結果反轉回來 [15], [16]。以上面的符號表示：

$$I_{\text{haze}} = 1 - S_{\text{Retinex}}, \qquad J_{\text{haze}} = 1 - \hat{S}_{\text{enhanced}}.$$

從「哪些資訊可靠」的角度看：去霧假設暗通道代表無霧的內容；但在低光影像裡，暗通道大多是被破壞的像素，所以重點轉向亮通道，也就是受黑暗影響最少的區域。LIME 論文 [17] 分析了這種反轉做法，並說明它可以改寫成 Retinex 的形式，所以兩個模型本質上是同一個假設，只是 Retinex 的表述較簡單。

統計方法

許多傳統方法在這些模型之上再加入統計量或先驗：保持場景明暗順序的自然度約束（NPE [18]）、分層或二維直方圖模型（DICM 資料集背後的方法 [59]），以及融合多張合成曝光。它們都是手工設計、不需要訓練資料，泛化行為可預期；弱點則是雜訊，通常被忽略或交給另一個去噪器處理。

深度學習 LLIE

白話版。深度網路從資料中學習「暗到亮」的對應。問題不在於要不要用網路，而在於**用什麼訓練訊號**：成對的暗/亮照片、非配對的照片、完全沒有參考影像，或者是一個描述「亮影像長什麼樣子」的生成模型。物理知識（通常是 Retinex）可以放進以上任何一種。

深度學習本身就是一個龐大的最佳化過程，而它的黑盒子特性讓我們能省掉許多手工假設。常見的第一種分法是**以理論為基礎**（通常是 Retinex）與**不以理論為基礎**（學習能力強的通用網路）。實務上兩者常交織在一起，所以不能用二元方式一刀切。圖 5 主要改以訓練訊號分組，參考了綜述論文 [2], [3] 的做法。

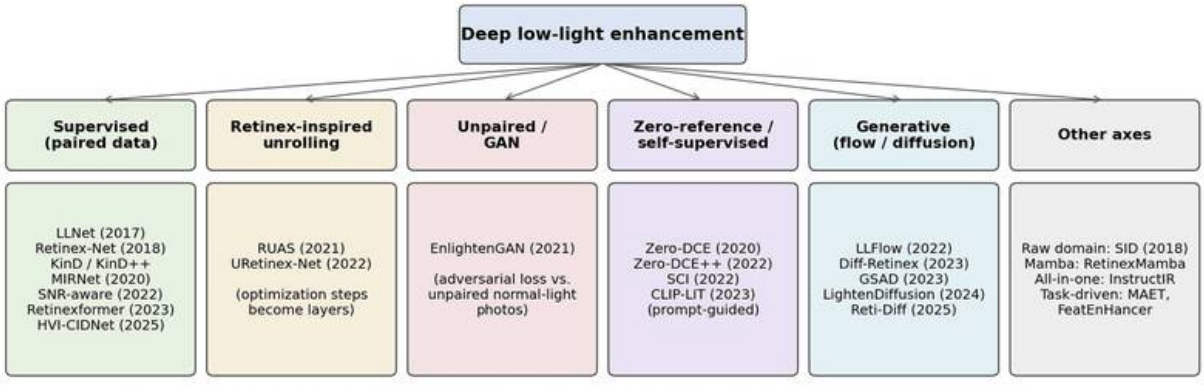


圖 5 — 主要依訓練訊號分類的深度 *LLIE*（*flow* ／擴散那一欄則是依模型家族）。*Retinex* 分解是一種架構上的選擇，出現在好幾欄裡，而不是獨立的一欄。

監督式方法（成對資料）

給定低光與正常光影像對 (x_i, y_i)，訓練 f_θ 最小化重建損失，通常是 $\mathcal{L} = \|f_\theta(x) - y\|_1$ 再加上感知或結構項。成對資料取得成本高（同一靜態場景拍兩種曝光，或用合成的），而且網路會學到參考影像特定的亮度與色彩風格，這一點在後面談評估時很重要。

- **LLNet** [19]（2017）是最早的深度 LLIE 模型之一：一個堆疊稀疏去噪自編碼器（stacked sparse denoising autoencoder），用人工變暗並加雜訊的小區塊訓練，同時學習調亮與去噪。
- **CNN + 亮通道先驗**（Tao 等人 [20]，2017）依序處理兩個難點：先用 CNN 去噪，再以亮通道驅動、類似大氣散射模型的方式增強對比，是把學習與上述反轉想法結合的早期例子。
- **MIRNet** [24]（2020）是一個通用的復原骨幹網路：保留一條全解析度的支線，並行多條低解析度支線、讓它們彼此交換資訊，再用 selective kernel 注意力融合；論文除了增強之外，也在去噪與超解析度上評估。
- **SNR-aware** [25]（2022）估計逐像素的訊噪比圖。訊噪比高的地方，局部卷積就夠了；訊噪比很低的地方，局部鄰域不可靠，模型改用長距離的 Transformer 注意力，從影像其他地方借用資訊。
- **HVI-CIDNet** [27]（2025）處理的是圖 2 的色彩空間問題：sRGB 對顏色非常敏感，容易產生色偏與亮度假影；HSV 解決了亮度問題，卻帶來紅色與黑色的雜訊假影。作者提出 HVI 色彩空間，由極化的色相/飽和度圖與可學習的強度構成，並搭配一個把色彩與強度解耦的網路。

以 Retinex 為基礎的網路

Retinex-Net [21]（2018）把目標從「估計 R」改成「**輸入與輸出各自有自己的 R 和 L**」。流程是：分解低光與正常光影像、把 L 增強成 $\hat{L}$、再與 R 重組。它也提出了 LOL 資料集（見後文）。它的分解網路沒有 R 或 L 的真值，只靠一致性來訓練，損失函數為

$$\mathcal{L}_{\text{decom}} = \mathcal{L}_{\text{recon}} + \lambda_{ir}\,\mathcal{L}_{ir} + \lambda_{is}\,\mathcal{L}_{is},$$

$$\mathcal{L}_{\text{recon}} = \sum_{i\in\{\text{low,normal}\}} \sum_{j\in\{\text{low,normal}\}} \lambda_{ij} \left\| R_i \circ I_j - S_j \right\|_1,$$

$$\mathcal{L}_{ir} = \|R_{\text{low}} - R_{\text{normal}}\|_1, \qquad \mathcal{L}_{is} = \sum_i \left\| \nabla I_i \circ \exp\left(-\lambda_g \nabla R_i\right) \right\|.$$

論文的記號是 $S = R \circ I$，所以這裡的 I 就是照度（我們記號中的 L）。S_j 是輸入影像 j，R_i 與 I_i 是影像 i 分解出的反射率與照度，∇ 是水平加垂直的梯度。各項的意義：

- $\mathcal{L}_{\text{recon}}$：每組分解都必須重建出自己的影像（$\lambda_{ii} = 1$），而且把兩張影像的反射率與照度交叉組合也應該大致成立（$i \neq j$ 時 $\lambda_{ij} = 0.001$）。
- $\mathcal{L}_{ir}$：兩張影像拍的是同一個場景，所以反射率應該相同；網路正是藉此學到「反射率」是什麼。
- $\mathcal{L}_{is}$：照度應該平滑，除了反射率有強邊緣的地方。權重 $\exp(-\lambda_g \nabla R)$ 在物體邊界上放寬平滑懲罰。這是 LIME 結構感知先驗的學習版。

論文使用 $\lambda_{ir} = 0.001$、$\lambda_{is} = 0.1$、$\lambda_g = 10$。增強網路則以 $\|R_{\text{low}} \circ \hat{I} - S_{\text{normal}}\|_1$ 加上同樣的平滑項訓練。

這個模式很快就流行起來，許多方法的名字都帶有「Retinex」：

- **KinD** [22]（2019）把工作拆成三個子網路：分解、反射率還原（因為暗影像的反射率帶有雜訊與色偏，所以放寬了「R 不變」的假設），以及可由使用者控制亮度倍率的照度調整。**KinD++** [23]（2021）進一步改良設計以減少視覺缺陷。
- **Retinexformer** [26]（2023）指出拍攝與調亮都會破壞分解出的成分，於是把模型改寫為帶擾動項的形式：$S = (R + \hat{R}) \circ (L + \hat{L})$，其中 $\hat{R}$ 與 $\hat{L}$ 模擬雜訊、假影與色彩失真。它採**單階段**訓練

  （取代 Retinex-Net 的多階段訓練），使用 L1 損失；它的**照度引導多頭自注意力**（illumination-guided multi-head self-attention）用照度估計來引導注意力，讓曝光程度不同的區域彼此交流。它後來成為常見的比較基準。
- **RetinexMamba** [42]（2024 年預印本）沿用 Retinex Transformer 的架構，把注意力換成 Mamba 狀態空間（state space）模組，其計算量隨像素數線性成長。「Retinex 分解＋新骨幹」多年來一直是很穩定的設計模式。

另一條較少見的路線是以大氣散射模型、而非單純的 Retinex 為基礎。CPGA-Net [28] 結合通道先驗與 gamma 校正，建構一個同時處理全域與局部資訊的輕量網路；CPGA-Net+ [29] 則重新檢視理論上的照度模型，以提升效率。這條研究路線也觀察到：不同功能模組學到的特徵，會對應到先驗的物理角色；這表示受模型引導的網路，並不是完全無法理解的黑盒子。

Retinex 啟發的展開式網路

展開（unrolling，或 unfolding）是把 Retinex 問題的迭代最佳化演算法，每一次迭代變成一個帶有可學習參數的網路階段。網路架構繼承了演算法的結構與可解釋性，而訓練取代了手工調整的先驗。

- **RUAS** [30]（2021）展開照度估計與去噪的最佳化模型，搜尋先驗模組的架構，並以協同式的

無參考（reference-free）策略訓練，所以它同時也屬於非監督那一欄。它非常小也非常快。

- **URetinex-Net** [31]（2022）把 Retinex 分解的最佳化展開成網路，包含初始化、R 與 L 的展開更新，以及照度調整等學習模組。

非配對訓練：EnlightenGAN

EnlightenGAN [32]（2021）使用生成對抗網路（GAN），只需要一組暗影像和另一組不相關的明亮影像。生成器是 U-Net，並以輸入照度推得的自正則化注意力圖引導，產生增強影像。**全域**判別器評估整張影像，**局部**判別器評估隨機小區塊，讓輸出在整體與細節上都像光線充足的照片。由於沒有真值，**自特徵保持**（self-feature-preserving）損失比較輸入與輸出的 VGG 特徵，讓內容維持不變。

零參考方法與曲線估計

白話版。 Zero-DCE 不直接輸出新影像，而是替每個像素輸出一條調亮曲線，套用它，再用不需要任何參考照片的損失來評分：「曝光好不好？顏色平不平衡？曲線平不平滑？局部對比有沒有保留？」

LE 曲線。 Zero-DCE [33]（2020）定義了一條二次的**光增強曲線**（light-enhancement curve）

$$\mathrm{LE}(x;\alpha) = x + \alpha\, x\,(1 - x), \qquad \alpha \in [-1, 1],$$

其中 $x \in [0,1]$ 是像素值。這條曲線有三個好性質：$\mathrm{LE}(0) = 0$ 且 $\mathrm{LE}(1) = 1$，範圍不變、不需要截斷；當 $|\alpha| \le 1$ 時單調，因為 $\mathrm{d}\,\mathrm{LE}/\mathrm{d}x = 1 + \alpha(1 - 2x) \ge 0$，明暗順序得以保留；而且可微分，能用反向傳播訓練。單一條二次曲線不夠靈活，所以要**迭代**套用：

$$\mathrm{LE}_n(\mathbf{x}) = \mathrm{LE}_{n-1}(\mathbf{x}) + \mathcal{A}_n(\mathbf{x})\,\mathrm{LE}_{n-1}(\mathbf{x})(1 - \mathrm{LE}_{n-1}(\mathbf{x})),$$

其中 LE_0 是輸入，$n = 1, \dots, 8$，而 $\mathcal{A}_n$ 是**逐像素、逐通道**的參數圖，而非單一的 α。一個七層的 CNN（DCE-Net，每層 32 個通道、約 79k 個參數、沒有降取樣）以 tanh 輸出 $8 \times 3 = 24$ 張參數圖。每個像素因此都有自己的高階曲線，局部調整就是這樣從看似全域的公式中產生的。

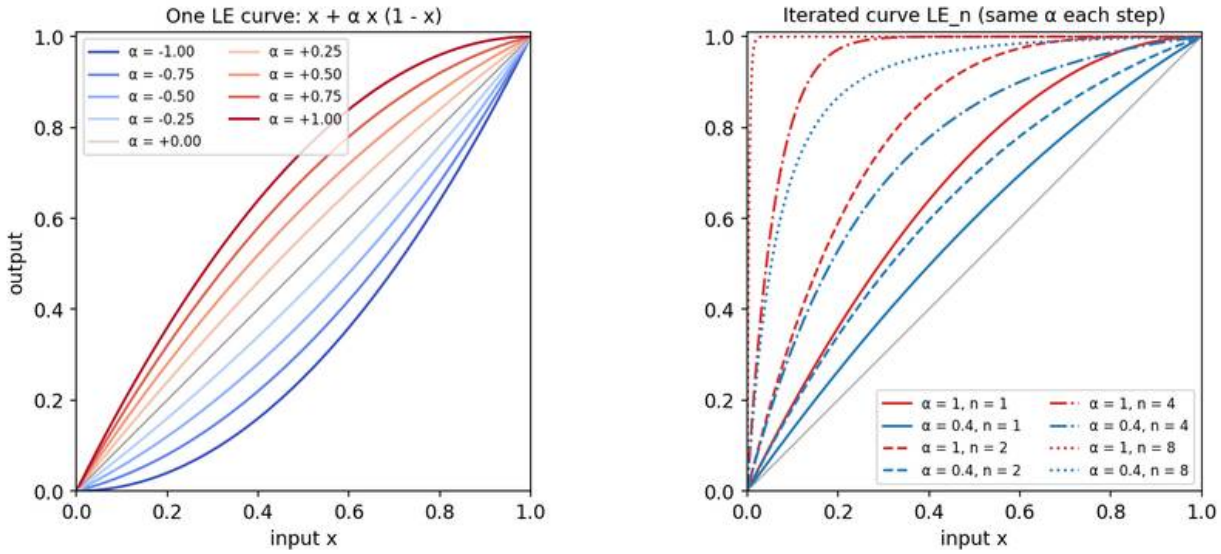


圖 *6 — LE* 曲線。左：α 從 -1 到 1 的單次迭代。右：迭代讓曲線變陡；$\alpha = 1$、$n = 8$ 時，0.02 這個值已經被對應到將近 1，所以暗部裡微小的雜訊值會被極度放大。

```
def le_iterate(x, alphas):
    """Zero-DCE curve: LE_n = LE_{n-1} + a_n * LE_{n-1} * (1 - LE_{n-1})."""
    y = x.copy()
    for a in alphas:              # a can be a scalar or a per-pixel map
        y = y + a * y * (1 - y)
    return y

x = np.array([0.0, 0.02, 0.1, 0.3, 0.6, 1.0])
print(le_iterate(x, [1.0] * 8).round(3))   # [0. 0.994 1. 1. 1. 1.]
print(le_iterate(x, [0.4] * 8).round(3))   # [0. 0.245 0.695 0.93 0.985 1.]
```

第一行輸出顯示了危險之處：對很小的 x，每一步大約乘上 $1 + \alpha$，所以 $\alpha = 1$ 迭代八次的增益接近 $2^8 = 256$。暗部裡的雜訊也會被放大同樣的倍數。

無參考損失。 Zero-DCE 用四個不需要真值的損失函數訓練：

$$\mathcal{L}_{spa} = \frac{1}{K}\sum_{i=1}^{K}\sum_{j\in\Omega(i)}\left(|Y_i - Y_j| - |I_i - I_j|\right)^2,$$

$$\mathcal{L}_{exp} = \frac{1}{M}\sum_{k=1}^{M}|Y_k - E|, \qquad \mathcal{L}_{col} = \sum_{(p,q)\in\varepsilon}(J^p - J^q)^2,$$

$$\mathcal{L}_{tv_A} = \frac{1}{N}\sum_{n=1}^{N}\sum_{c\in\{R,G,B\}}\left(|\nabla_x A_n^c| + |\nabla_y A_n^c|\right)^2.$$

在 $\mathcal{L}_{spa}$（**空間一致性**）中，K 是 4×4 區域的數量，$\Omega(i)$ 是區域 i 的四個鄰居，Y 與 I 分別是增強影像與輸入影像中該區域的平均強度：相鄰區域之間的對比應該保留。在 $\mathcal{L}_{exp}$（**曝光控制**）中，

Y_k 是 M 個不重疊 16×16 區域中第 k 個的平均強度，$E=0.6$ 是目標的良好曝光程度。在 $\mathcal{L}_{col}$（**色彩恆常**）中，J^p 是通道 p 在整張影像上的平均，$\varepsilon=\{(R,G),(R,B),(G,B)\}$：這是灰色世界（gray-world）假設，認為場景的平均顏色是灰色。在 $\mathcal{L}_{tv_A}$（**照度平滑**）中，N 是迭代次數，∇_x,∇_y 是水平與垂直梯度：相鄰像素應該得到相似的曲線。總損失為 $\mathcal{L}_{spa}+\mathcal{L}_{exp}+W_{col}\mathcal{L}_{col}+W_{tv_A}\mathcal{L}_{tv_A}$，論文中 $W_{col}=0.5$、$W_{tv_A}=20$；訓練資料是 SICE [56] 的多重曝光影像，同時提供曝光不足與過曝的輸入。

後續方法改良了這個想法：

- **Zero-DCE++** [34]（2022）改用深度可分離卷積（depthwise separable convolution），只預測 **3** 張曲線參數圖並在所有迭代中重複使用，而且在降取樣後的輸入上估計，再縮放回原尺寸。模型因此縮小到約 10k 個參數，即使在 CPU 上也跑得相當快。
- **SCI** [35]（2022）以權重共享的多階段串接學習照度，並用**自校正**（self-calibrated）模組讓各階段的輸出在訓練時收斂到相同狀態，因此測試時只需要一**個**基本模組。它以非監督損失訓練。
- **CLIP-LIT** [36]（2023）針對逆光影像。它使用視覺-語言模型 CLIP，學習一對文字提示（prompt），一個代表逆光、一個代表光線良好，再迭代地精修提示與增強網路，讓輸出與「光線良好」提示的 CLIP 相似度成為訓練訊號。

用 PyTorch 寫一個零參考小玩具

為了看看這些損失函數如何運作，下面的程式碼在 CPU 上訓練一個迷你的 Zero-DCE 風格網路約半分鐘。它用四張 `skimage` 影像的隨機裁切訓練，每塊都在線性光中以隨機倍率變暗；測試則用訓練時從沒看過的咖啡杯影像。全程不使用任何真值。（損失權重與論文不同，因為各項的正規化方式不同：曝光誤差用平方而非絕對值，梯度用平均而非加總。）

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from skimage import data, img_as_float
from skimage.transform import resize

torch.manual_seed(0); rng = np.random.default_rng(0)
darken = lambda x, k: (k * x ** 2.2) ** (1 / 2.2)          # less light, in linear units
train = [resize(img_as_float(f()), (256, 256), anti_aliasing=True)
         for f in (data.astronaut, data.chelsea, data.rocket, data.immunohistochemistry)]

def batch(n=4, size=64):                                    # random crops, random darkness
    xs = []
    for _ in range(n):
        img = train[rng.integers(len(train))]
        i, j = rng.integers(0, 256 - size, 2)
        xs.append(darken(img[i:i + size, j:j + size], rng.uniform(0.02, 0.3)))
    return torch.tensor(np.stack(xs), dtype=torch.float32).permute(0, 3, 1, 2)

class TinyDCE(nn.Module):
    def __init__(self, n_iter=8, w=16):
```

```
        super().__init__()
        self.n_iter = n_iter
        self.body = nn.Sequential(
            nn.Conv2d(3, w, 3, padding=1), nn.ReLU(), nn.Conv2d(w, w, 3, padding=1),
             ↪  nn.ReLU(),
            nn.Conv2d(w, w, 3, padding=1), nn.ReLU(), nn.Conv2d(w, 3 * n_iter, 3,
             ↪  padding=1), nn.Tanh())
    def forward(self, x):
        alphas = self.body(x).chunk(self.n_iter, dim=1)    # 8 maps of shape B x 3 x H x W
        y = x
        for a in alphas:
            y = y + a * y * (1 - y)                         # the LE curve
        return y, torch.cat(alphas, 1)

def L_exp(y, E=0.6, p=16):   # mean brightness of 16x16 patches should be near E
    return (F.avg_pool2d(y.mean(1, keepdim=True), p) - E).pow(2).mean()
def L_col(y):                # gray-world: channel means should agree
    r, g, b = y.mean((2, 3)).unbind(1)
    return ((r - g) ** 2 + (r - b) ** 2 + (g - b) ** 2).mean()
def L_tv(a):                 # curve parameters should vary smoothly in space
    return (a[..., 1:, :] - a[..., :-1, :]).pow(2).mean() + (a[..., :, 1:] - a[..., :,
     ↪  :-1]).pow(2).mean()
def L_spa(y, x, p=4):        # keep the contrast between neighbouring 4x4 regions
    Y, X = (F.avg_pool2d(t.mean(1, keepdim=True), p) for t in (y, x))
    return sum(((torch.diff(Y, dim=d) - torch.diff(X, dim=d)) ** 2).mean() for d in (2, 3))

net = TinyDCE()
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
for step in range(600):                                    # ~20-40 s on a laptop CPU
    x = batch()
    y, a = net(x)
    loss = L_spa(y, x) + 10 * L_exp(y) + 5 * L_col(y) + 200 * L_tv(a)
    opt.zero_grad(); loss.backward(); opt.step()

test = darken(img_as_float(data.coffee()), 0.05)           # an image never used in training
with torch.no_grad():
    out, _ = net(torch.tensor(test, dtype=torch.float32).permute(2, 0, 1)[None])
print(f"mean brightness: {test.mean():.3f} -> {out.mean():.3f}")   # about 0.10 -> 0.50
```

拖動滑桿，比較暗的輸入與小玩具的輸出：



暗的輸入



零參考小玩具

暗的咖啡杯影像與小網路的輸出：輸出亮得多，木紋清晰可見，顏色比原照片淺、略偏綠且飽和度較低。

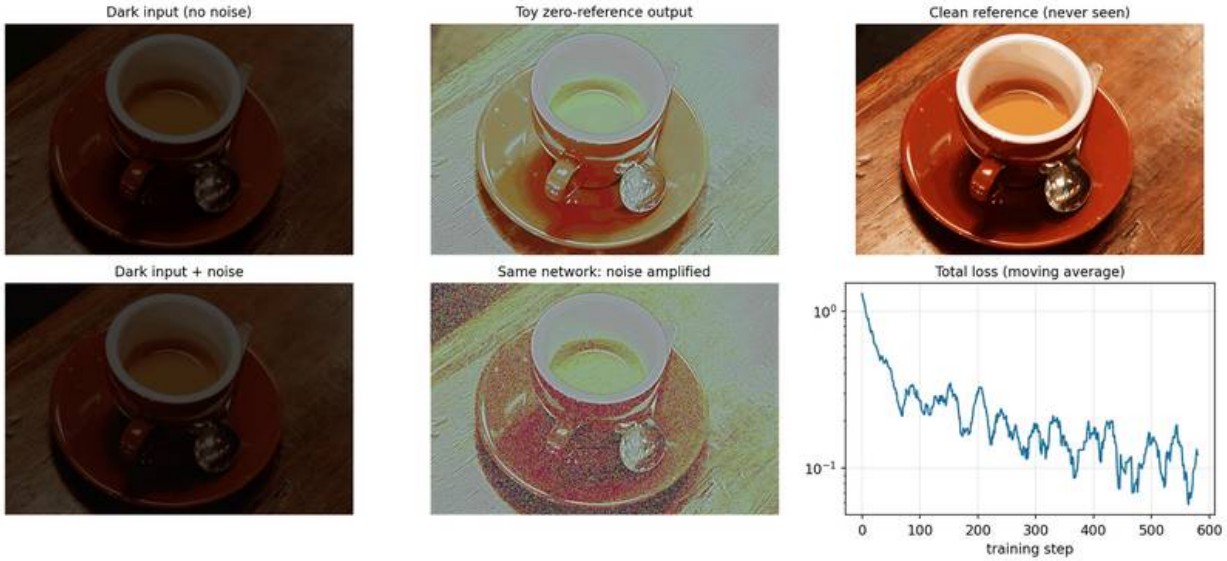


圖 7 — 零參考損失做得到與做不到的事。曝光損失給出好的亮度，但灰色世界的色彩損失把紅褐色場景往灰色拉（上排中間對比上排右邊）；遇到真實雜訊時，逐像素曲線會把雜訊放大（下排中間）。

這個小玩具在一張圖裡同時展示了零參考想法的優點與極限。沒有任何一張參考影像，網路就學到了合理的調亮方式，而且能遷移到沒看過的照片。但每一個損失都是一種先驗，而先驗可能是錯的：場景本來就偏紅或偏褐時，灰色世界假設就失效；固定的曝光目標 E 會把刻意保留的暗區拉平；而損失中沒有任何一項要求去噪。真正的零參考方法用多得多的影像訓練，有些還加入了明確的雜訊處理，但這些失敗模式在已發表的結果中同樣看得到。

正規化流與擴散模型

LLFlow [37]（2022）觀察到：同一張暗影像有許多種可接受的明亮版本，而逐像素損失會把它們平均成一個模糊的折衷。它學習一個條件式正規化流（normalizing flow）：一個可逆網路，在以暗輸入特徵為條件下，把正常光影像對應到高斯分布。這些特徵由呼應傳統方法的先驗引導：輸入的直方圖等化版本、色彩圖（接近 Retinex 的反射率），以及雜訊圖。

2023 年起，**擴散模型**（diffusion model）大量進入 LLIE。它們以暗影像為條件，從隨機雜訊逐步去噪來生成輸出，因此能合成回歸式網路會模糊掉的合理細節：

- **Diff-Retinex** [38]（2023）先用 Retinex Transformer 分解影像，再用條件式擴散模型生成復原後的反射率與照度。
- **GSAD** [39]（2023）加入全域結構感知的正則化，以影像結構引導擴散軌跡上以曲率為基礎的約束，並對困難區域加入不確定性引導項。
- **LightenDiffusion** [40]（2024）在潛在空間中做 Retinex 分解，並以非配對資料訓練。

- **Reti-Diff** [41]（2025）以潛在擴散模型產生精簡的 Retinex 先驗，再用它引導一個復原 Transformer，降低在全解析度上跑擴散的成本。

代價是運算量與保真度：多步去噪讓推論變慢，生成的細節可能看起來對，卻不一定真實。Adhikarla 等人的綜述 [6] 把這個子領域分成六類：內在分解、頻譜與潛在空間、加速、引導式、多模態，以及自主式。

Raw 域的低光

以上方法都作用在處理過的 sRGB 影像上，而相機的處理流程（去馬賽克、白平衡、去噪、色調曲線、壓縮）早已丟掉部分資訊。**Learning to See in the Dark** [44]（2018）改把 **raw** 感測器資料送進 U-Net，用它取代整條處理流程。它的 SID 資料集有 5,094 張短曝光 raw 影像，對應 424 張不同的長曝光參考影像，來自 Sony α7S II（Bayer 感測器）與 Fujifilm X-T2（X-Trans 感測器），曝光比為 ×100、×250 與 ×300。Raw 資料保留光子數與數值之間的線性關係，位元數也更多，因此在 sRGB 中毫無希望的極暗場景也能被救回來。代價是模型綁定在單一感測器上。

All-in-one 復原

低光增強也是通用復原模型中的一項任務。**InstructIR** [43]（2024）接受自然語言指令（「把這張照片調亮」、「去除雜訊」）並據此復原影像，在一個網路中涵蓋去噪、去模糊、去霧與低光增強。

面向下游任務的低光處理

如果目標是偵測而非好看的照片，常見的基準做法是「先增強、再偵測」（請見DLCV 03・物件偵測）。但為人眼調校的增強，不一定是偵測器需要的。**MAET** [45]（2021）以物理方式模擬低光退化，並訓練偵測器同時預測退化參數，讓特徵能感知退化。**FeatEnHancer** [46]（2023）則在網路內部增強階層式特徵，直接用下游任務的損失訓練，而不是增強像素。



圖 *8* — 本篇提到的方法時間軸（依發表年份）。顏色標示以 *Retinex* 為基礎或展開式的設計、非配對或零參考訓練、*flow* 與擴散模型，以及挑戰賽。

評估

白話版。分數有兩類。**全參考**指標把輸出和一張真值照片比較；**無參考**指標只看輸出本身。第三種做法則是問：增強後的影像有沒有幫助下游任務？每一種回答的問題都不同，而且每一種都可能被鑽漏洞。

全參考指標

給定參考影像 y 與輸出 $\hat{y}$（數值在 $[0,1]$），**PSNR** 為

$$\mathrm{PSNR} = 10\log_{10}\frac{1}{\frac{1}{n}\|\hat{y}-y\|_2^2},$$

其中 n 是數值的個數。它是純粹的像素誤差。**SSIM** [47] 比較局部的亮度、對比與結構統計量，與人眼感知的相關性較高。**LPIPS** [48] 衡量兩張影像深度特徵之間的距離，越低越好，對 PSNR 忽略的紋理與模糊很敏感。

無參考指標：為什麼低光評估離不開它

白話版。無參考（no-reference, NR）指標只看一張影像，就猜它的品質好不好，沒有「標準答案」可以比對。低光論文需要這類指標，因為最貼近真實的測試照片根本沒有標準答案；就算有，那也只是眾多可接受亮度中的其中一種。

精確版。有兩個事實把 LLIE 推向無參考評估：

1. **真實測試集是非配對的**。 LIME [17]、MEF [58]、NPE [18]、DICM [59] 與 VV [60] 都是小規模的真實照片集，常有不均勻的光照，沒有真值。LOL [21] 這類成對資料集雖然存在，但它們是在受控條件下拍攝的（靜態場景、兩種曝光），模型可能在上面表現很好卻無法泛化（即 [2] 所報告的泛化落差）。因此幾乎每篇 LLIE 論文都會展示非配對測試集上的結果，而能替這些結果打分數的，只有無參考指標與人的判斷。

2. **「正確」的亮度本身就有歧義**。即使有參考影像，正常光照的照片也只是某位攝影師對曝光與色彩的一種選擇。稍微亮一點或暖一點的輸出可能一樣好看，PSNR 卻會掉好幾 dB（這正是下面 GT-mean 協定出現的原因）。無參考指標至少不會懲罰一個不同但合理的曝光。

代價是：無參考指標本身是一個人類偏好的模型，而每個模型都有盲點。無參考影像品質評估（image quality assessment, IQA）的一般理論，包括 NIQE、BRISQUE、MUSIQ 與 CLIP-IQA 的建構方式，請見DLCV 04・影像復原。以下各小節摘要每一類指標量測的是什麼，並聚焦在它們面對增強後的低光影像時的表現。

經典的自然場景統計指標：NIQE、BRISQUE 與 PI

白話版。自然照片有共同的統計規律：把局部亮度與對比去掉之後，剩下的數值會呈現特定形狀的鐘形分布。模糊、雜訊與壓縮會改變這個形狀。自然場景統計（natural scene statistics, NSS）指標量測的，就是一張影像的統計量離乾淨照片的統計量有多遠。

MSCN 係數。 NIQE 與 BRISQUE 都從灰階影像 I 的**去均值、對比正規化**（mean-subtracted contrast-normalized, MSCN）係數出發 [50]：

$$\hat{I}(i,j) = \frac{I(i,j) - \mu(i,j)}{\sigma(i,j) + C}, \qquad \mu(i,j) = \sum_{k,l} w_{k,l} I(i+k, j+l), \qquad \sigma(i,j) = \sqrt{\sum_{k,l} w_{k,l} (I(i+k,j+l) - \mu(i,j))^2},$$

其中 (i,j) 是像素位置，w 是總和為 1 的小型圓對稱高斯視窗，μ 與 σ 是局部加權平均與標準差，C 是避免在平坦區域除以零的小常數（8 位元影像取 $C = 1$）。對原始高品質影像而言，$\hat{I}$ 的直方圖接近單位高斯；失真會讓它變得更尖（模糊）或尾巴更厚（雜訊、區塊效應）。相鄰 MSCN 係數在四個方向上的乘積，則捕捉剩下的結構。

BRISQUE [50] 對 $\hat{I}$ 擬合廣義高斯分布、對相鄰乘積擬合非對稱廣義高斯分布（兩個尺度），再把擬合得到的形狀與變異數參數，送進一個以人類對合成失真的主觀評分訓練的支援向量迴歸器。越低越好。

NIQE [49] 不需要主觀評分。它挑選清晰的區塊、計算類似的 NSS 特徵，對一組原始高品質影像擬合平均 ν_1、共變異數 Σ_1 的多變量高斯（multivariate Gaussian, MVG），再對測試影像擬合另一個 MVG (ν_2, Σ_2)，回報

$$D(\nu_1, \nu_2, \Sigma_1, \Sigma_2) = \sqrt{(\nu_1 - \nu_2)^\top \left(\frac{\Sigma_1 + \Sigma_2}{2} \right)^{-1} (\nu_1 - \nu_2)},$$

也就是兩個高斯之間類似馬氏距離（Mahalanobis distance）的距離。越低越好：0 代表「統計上與高品質影像集一樣」。

PI。感知指標（Perceptual Index）是為 2018 年 PIRM 超解析度挑戰賽提出的 [63]：

$$\text{PI} = \tfrac{1}{2}((10 - \text{Ma}) + \text{NIQE}),$$

其中 Ma 是 Ma 等人 [64] 學習得到的無參考分數（越高越好，大約介於 0 到 10，以人類對超解析度影像的評分訓練）。PI 越低越好。例如 Zero-DCE [33] 就在非配對測試集上回報 PI，並搭配使用者研究。

它們獎勵什麼，以及在曝光問題上如何誤導。這三個指標都是為結構性失真（模糊、雜訊、壓縮、振鈴）設計的，不是為曝光或色彩：

- MSCN 正規化在定義上就把局部平均與對比除掉了，所以整體亮度的改變幾乎不會影響特徵。增強不足的影像與曝光良好的影像可能拿到相近的分數。
- 灰階特徵忽略色偏與飽和度，而這正是圖 2 裡的錯誤。
- 雜訊與過度銳化會增加高頻能量，對這些統計量來說，一張雜訊被放大、紋理「豐富」的輸出，可能比一張乾淨但略柔的輸出更「自然」。強烈局部對比的 HE 與 CLAHE 輸出就是典型例子。
- 用來定義「高品質」的影像集與訓練用的失真裡，很少有暗景、夜景或逆光場景，所以「自然」其實等於「像白天」。

低光專用的量測：明度順序、色彩豐富度與雜訊

白話版。有些量測直接針對增強時會出錯的地方：方法有沒有把「哪裡比較亮」的關係顛倒？顏色是鮮豔還是褪色？調亮之後露出了多少雜訊？

明度順序誤差（Lightness Order Error, LOE）。NPE 論文 [18] 主張，自然的增強應該保留場景中明度的相對順序：如果燈原本比牆亮，增強後也應該如此，因為這個順序承載了光的方向與深度感。LOE 計算兩個像素的明暗順序被翻轉的次數。令 $L(x) = \max_{c\in\{R,G,B\}} I_c(x)$ 為輸入的明度，$L_e(x)$ 為增強後影像的明度，

$$\mathrm{LOE} = \frac{1}{m}\sum_{x=1}^{m} \mathrm{RD}(x), \qquad \mathrm{RD}(x) = \sum_{y=1}^{m} U(L(x), L(y)) \oplus U(L_e(x), L_e(y)),$$

其中 m 是像素數，$p \geq q$ 時 $U(p,q) = 1$、否則為 0，$\oplus$ 是互斥或（XOR），因此 $\mathrm{RD}(x)$ 就是相對於 x 改變了順序的像素個數。越低越好。實務上有三點要注意：

- 計算量是 $O(m^2)$，所以實作會在降取樣後的影像上計算 LOE。原始數值會隨 m 增加，只有在相同的降取樣設定下才能比較，而許多論文並未說明自己的設定。
- 比較的基準是**輸入**，而不是乾淨影像。任何在最大值通道上單調的色調曲線都會得到 LOE $= 0$，包括什麼都不做。而且暗輸入裡的雜訊早已打亂相鄰像素的順序，所以去除雜訊的方法反而會被懲罰。
- LOE 完全不管結果夠不夠亮。它是防止不自然局部色調映射的護欄，不是品質分數。

色彩豐富度（colorfulness）。Hasler 與 Süsstrunk [65] 以 8 位元 RGB 上的對立色軸，擬合出一個與主觀感受相符的簡單色彩豐富度量：

$$rg = R - G, \qquad yb = \tfrac{1}{2}(R + G) - B, \qquad M = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3\sqrt{\mu_{rg}^2 + \mu_{yb}^2},$$

其中 μ 與 σ 是整張影像上的平均與標準差，權重 0.3 來自作者的擬合。越高代表色彩越豐富。它適合當作描述性統計量，用來偵測褪色（只增強亮度通道，圖 2）或過飽和，但它不知道什麼才是正確的顏色，而色度通道裡的雜訊會灌大 σ_{rg} 與 σ_{yb}。

輸出的雜訊程度。既然調亮會讓雜訊現形（圖 1），回報輸出的雜訊程度就很有幫助。Immerkær 的快速估計法 [66] 用一個能抵消平滑結構、保留雜訊的遮罩對影像濾波：

$$N = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}, \qquad \hat{\sigma} = \sqrt{\frac{\pi}{2}}\,\frac{1}{6\,(W-2)(H-2)}\sum_{i,j} |(I * N)(i,j)|,$$

其中 $W \times H$ 是影像大小，總和只取內部像素。它假設雜訊是白高斯雜訊，所以遇到紋理豐富的影像會高估，遇到空間相關的雜訊（真實相機管線與許多網路產生的正是這種）則會低估。請只用它比較同一張輸入的不同輸出，不要跨場景比較。

下面的程式碼重建圖 1 的暗咖啡杯影像，比較五種「增強」結果與乾淨照片。

```
import numpy as np
from scipy import ndimage as ndi
from skimage import color, data, exposure, img_as_float

rng = np.random.default_rng(0)
clean = img_as_float(data.coffee())
```

```
def synth_low_light(x, k=0.05, peak=2000.0, read_std=8e-4, gamma=2.2):   # 與圖 1 相同
    lin = k * x ** gamma
    lin = rng.poisson(lin * peak) / peak
    lin = lin + rng.normal(0, read_std, lin.shape)
    return np.round(np.clip(lin, 0, 1) ** (1 / gamma) * 255) / 255

def loe(ref, out, step=8):
    """Lightness Order Error on a strided subsample (m = 50 x 75 = 3750 pixels here)."""
    L, Le = (im.max(axis=2)[::step, ::step].ravel() for im in (ref, out))
    U, Ue = L[:, None] >= L[None, :], Le[:, None] >= Le[None, :]
    return (U ^ Ue).sum(axis=1).mean()                  # mean over x of RD(x)

def colorfulness(im):
    """Hasler-Suesstrunk colorfulness, RGB scaled to 0..255."""
    R, G, B = (255 * im[..., c] for c in range(3))
    rg, yb = R - G, 0.5 * (R + G) - B
    return np.hypot(rg.std(), yb.std()) + 0.3 * np.hypot(rg.mean(), yb.mean())

def noise_sigma(im):
    """Immerkaer's noise estimate on BT.601 luma, in 8-bit units."""
    Y = 255 * (0.299 * im[..., 0] + 0.587 * im[..., 1] + 0.114 * im[..., 2])
    N = np.array([[1, -2, 1], [-2, 4, -2], [1, -2, 1]])
    H, W = Y.shape
    return np.sqrt(np.pi / 2) / (6 * (W - 2) * (H - 2)) * np.abs(ndi.convolve(Y, N)[1:-1,
     ↪  1:-1]).sum()

dark = synth_low_light(clean)
lab = color.rgb2lab(dark); L = lab[..., 0] / 100
he = lab.copy(); he[..., 0] = 100 * exposure.equalize_hist(L)
cl = lab.copy(); cl[..., 0] = 100 * exposure.equalize_adapthist(L, clip_limit=0.01)
outputs = {
    "dark input": dark,
    "gain x20 (linear)": np.clip(20 * dark ** 2.2, 0, 1) ** (1 / 2.2),
    "gamma 0.4": dark ** 0.4,
    "HE on L*": np.clip(color.lab2rgb(he), 0, 1),
    "CLAHE on L*": np.clip(color.lab2rgb(cl), 0, 1),
    "clean reference": clean,
}
for name, im in outputs.items():
    print(f"{name:18s} mean {im.mean():.2f}  LOE {loe(dark, im):6.1f}  "
          f"colorfulness {colorfulness(im):5.1f}  noise {noise_sigma(im):5.2f}")
# dark input         mean 0.10  LOE    0.0  colorfulness  23.1  noise  3.32
# gain x20 (linear)  mean 0.39  LOE    0.3  colorfulness  90.0  noise 12.93
# gamma 0.4          mean 0.34  LOE    0.0  colorfulness  61.6  noise 11.19
# HE on L*           mean 0.48  LOE  334.7  colorfulness  28.3  noise 16.17
# CLAHE on L*        mean 0.40  LOE  657.6  colorfulness  27.1  noise 23.06
# clean reference    mean 0.39  LOE  322.0  colorfulness  76.9  noise  4.69
```



圖 *9* — 更亮不代表更好。*HE* 是最亮的輸出，但明度順序被破壞的程度僅次於 *CLAHE*，而且幾乎沒有顏色；有雜訊的線性增益比乾淨照片更「鮮豔」；乾淨照片本身的 *LOE* 也很高，因為 *LOE* 是相對於有雜訊的輸入計算的。

這些數字要一起看，不能各看各的：

- **只看亮度會排錯順序**。 HE 的平均最高（0.48），但 LOE 也很大、雜訊第二高，顏色幾乎都沒了。CLAHE 的局部重新映射則是破壞明度順序最嚴重的。
- **色彩豐富度會被雜訊騙**。線性增益得到 90，高於乾淨照片的 77，部分原因是被放大的色度雜訊拉高了 σ_{rg} 與 σ_{yb}。
- **LOE 懲罰復原**。乾淨照片明明是理想答案，相對於有雜訊的輸入卻得到 LOE 322；什麼都不做反而是 0。低 LOE 是自然色調曲線的必要條件，而不是品質的充分條件。
- **只有雜訊估計把乾淨照片**（4.7）與明亮但多雜訊的輸出（11 到 23）區分開來。沒有任何單一的無參考數字能同時涵蓋「曝光良好、顏色自然、乾淨無雜訊」。

學習式無參考評估器

白話版。不再使用手工設計的統計量，而是訓練一個神經網路去預測人類評分，或直接問大型視覺語言模型一張照片看起來好不好。這類模型更能捕捉人們在意的東西，包括內容與美感，但也會繼承訓練資料的偏差。

主要的類別如下，都可以套用在 LLIE 的輸出上（也有共用的實作，例如 IQA-PyTorch 工具箱

[75]）：

- **NIMA** [67]（2018）用 CNN 預測人類評分在各分數區間上的分布，而不是單一數字。回報的分數是該分布的平均 $\hat{s} = \sum_k k\, p_k$，p_k 是預測為分數 k 的機率。以美感評分訓練的版本量測的是「這是不是一張好看的照片」，和「這是不是忠實的增強」並不相同。
- **MUSIQ** [68]（2021）是多尺度 Transformer，以影像的原始解析度與長寬比輸入，並加入縮小版本，使用以雜湊為基礎的 2D 位置與尺度嵌入。不做縮放對 LLIE 很重要，因為雜訊與細紋理正是縮放會藏起來的東西。越高越好。
- **CLIP-IQA** [69]（2023）利用視覺語言模型 CLIP 與一對反義提示詞「Good photo.」和「Bad photo.」。令 $\mathbf{x}$ 為影像嵌入、$\mathbf{t}_1, \mathbf{t}_2$ 為兩個文字嵌入，$s_i = \mathbf{x}^\top \mathbf{t}_i / (\|\mathbf{x}\|\,\|\mathbf{t}_i\|)$，分數為 $e^{s_1}/(e^{s_1}+e^{s_2})$，介於 $[0,1]$，越高越好。基本版本是零樣本的，不需要主觀評分。換成其他提示詞對（「Bright photo.」/「Dark photo.」、「Noisy photo.」/「Clean photo.」）可以探測單一屬性，對曝光評估很方便，但用字不同分數就會不同。同樣的 CLIP 相似度想法也被 CLIP-LIT [36] 當作訓練訊號，所以 CLIP 類指標不應該是評判以 CLIP 訓練之增強器的唯一裁判。
- **MANIQA** [70]（2022）結合 Vision Transformer、轉置（通道）注意力與 Swin Transformer 區塊，再以雙分支、依區塊加權的方式產生分數。它贏得 NTIRE 2022 感知 IQA 挑戰賽，而該挑戰的影像包含復原演算法與 GAN 演算法的輸出，因此比用相機失真訓練的指標更貼近「演算法輸出」的情境。
- **Q-Align** [71]（2024）微調一個大型多模態模型，讓它以文字評分等級（bad、poor、fair、good、excellent）作答，模仿人類評分者的方式。令 X_{l_i} 為模型對等級 l_i 的 logit，分數為 $S = \sum_{i=1}^{5} i \cdot e^{X_{l_i}} / \sum_{j=1}^{5} e^{X_{l_j}}$，也就是只在這五個詞上做 softmax 後的期望等級。

- **低光專用的評估器**。Kannan 等人 [72]（2023）針對不同 LLIE 方法復原的影像收集人類主觀評分，以此評測現有的無參考 IQA 模型，並提出一個不需主觀評分（opinion-unaware）的模型：以自監督對比學習，學出對低光復原影像失真敏感的特徵。這類資料集才是決定「在增強輸出上該信任哪個指標」的正確測試平台。

低光情境下的注意事項。學習式評估器多半以白天照片訓練。它們傾向給更亮、更銳利、更飽和的影像高分，因而偏好過度增強；一張刻意保留暗調的夜景，即使是最忠實的結果，也可能被扣分。分數取決於權重檔（用哪個訓練集）、輸入尺寸與實作，這些都應該回報。另外，當一個方法在損失函數中使用了某個學習式指標或相同的骨幹網路時，那個指標就不再是獨立的裁判。

使用者研究與成對比較

白話版。好不好看，最終由人決定。兩種標準的問法是：請人在量表上打分數，或請人從兩張中選出比較好的一張。

平均意見分數（mean opinion score, MOS）。N 位評分者各自給一張影像一個固定量表上的分數 s_n（常見為 1 到 5），$\text{MOS} = \frac{1}{N}\sum_{n=1}^{N} s_n$。ITU-R BT.500 [73] 為這類測試訂定標準：觀看條件、評分量表、單刺激與雙刺激以及比較式方法，還有篩除不可靠評分者的程序。例如 Zero-DCE [33] 請 15 位受試者以 1–5 分評分 NPE、LIME、MEF、DICM 與 VV 共 202 張影像的結果，並指示他們留意過曝或曝光不足、色彩偏差，以及不自然的紋理或雜訊。

成對比較。絕對量表在不同評分者之間會漂移；從兩張中選一張比較容易，也比較一致。**Bradley–Terry** 模型 [74] 把大量這樣的選擇轉換成每個方法的分數：每個方法 i 有一個強度

$\pi_i > 0$，且

$$P(i \text{ 勝過 } j) = \frac{\pi_i}{\pi_i + \pi_j},$$

再以最大概似估計從觀察到的勝負擬合各個 π_i。EnlightenGAN [32] 就採用這個協定：9 位受試者在 23 張影像上，比較五種方法輸出的配對，判斷是否有可見雜訊、曝光假影，以及不真實的顏色或紋理，再以 Bradley–Terry 擬合得到排名。

良好的做法：隱藏方法名稱、隨機化順序與左右位置、同時顯示輸入以提供脈絡、使用足夠多的影像與評分者、回報信賴區間或顯著性，並說明顯示器與觀看條件。只有少數評分者與精挑細選影像的研究，只能算展示，不能算證據。

無參考指標的選擇與回報

指標	類型	需要訓練資料？	方向	敏感於	低光情境的注意事項
NIQE [49]	NSS	只需高品質影像（不需主觀評分）	越低越好	灰階結構中的模糊、雜訊、壓縮	幾乎看不到整體曝光與色彩；可能偏好多雜訊、高對比的輸出
BRISQUE [50]	NSS + 迴歸	需要，合成失真的主觀評分	越低越好	與 NIQE 相同的特徵，映射到人類分數	以白天失真訓練；忽略色偏
PI [63], [64]	NSS + 學習式	Ma 分數以超解析度評分訓練	越低越好	紋理與銳利度	為超解析度設計；獎勵細節，包括被放大的雜訊
LOE [18]	任務專用	不需要	越低越好	相對於輸入的明度順序翻轉	取決於降取樣；什麼都不做得 0；懲罰去噪
色彩豐富度 [65]	任務專用（描述性）	不需要（常數已擬合）	視情況而定	對立色的分散程度與偏移	不知道正確顏色；會被色度雜訊灌大
雜訊估計 [66]	任務專用（描述性）	不需要	越低越好	高頻殘差	假設白雜訊；會被紋理與相關雜訊混淆
NIMA [67]	學習式	需要，評分分布	越高越好	美感或技術品質，依權重檔而定	美感模型獎勵好看而非忠實的結果
MUSIQ [68]	學習式	需要，主觀評分	越高越好	原始解析度下的多尺度品質	多以白天照片訓練；依權重檔而定
CLIP-IQA [69]	學習式（視覺語言）	零樣本版本不需要	越高越好	語意上「好照片」的印象；依提示詞而定	提示詞用字有影響；對以 CLIP 訓練的增強器不獨立
MANIQA [70]	學習式	需要，主觀評分	越高越好	復原與 GAN 假影	並非專門針對曝光錯誤訓練
Q-Align [71]	學習式（大型多模態模型）	需要，以文字等級表示的主觀評分	越高越好	整體、類似人類的品質判斷	運算量大；繼承基礎模型的偏差；依版本而定

指標	類型	需要訓練資料？	方向	敏感於	低光情境的注意事項
MOS ／成對比較 [73], [74]	人類	需要人類評分者	MOS 越高／強度越高	人們真正偏好的結果	成本高；小規模研究雜訊大；必須回報協定

實務建議：

1. **在成對資料集上**，以標準協定下的全參考指標（PSNR、SSIM、LPIPS）為主，無參考分數只作為補充。
2. **在非配對資料集上**，回報多個來自不同類別的無參考指標（一個 NSS、一到兩個學習式、LOE），說明實作、權重檔與影像尺寸，並展示大量視覺範例，包括失敗案例。
3. **預期會有分歧**。NSS 與學習式指標對方法的排名經常不同，兩者也都可能與人的判斷不一致；Li 等人的綜述 [2] 指出指標分數與視覺品質常常不符，Liu 與 Fan [5] 則發現畫質與下游任務表現之間還有另一道落差。指標意見不一時，請直接說明，而不是只挑對自己有利的那一個。
4. **不要最佳化裁判**。如果方法的損失函數用了某個指標或相同的骨幹網路（例如 CLIP），就要用別的東西來評估它。
5. **用人來支撐主張**。對於新方法，設計良好的使用者研究或成對比較，是最有力的無參考證據。

GT-mean 陷阱

白話版。在成對基準上，整體亮度的一點點差異就會大幅改變 PSNR。有些論文在評分前，會把每張輸出重新縮放，讓它的平均亮度與真值一致。但實際使用時，你永遠拿不到這個資訊。

精確版。「GT-mean」評估把輸出乘上 $\lambda = \mathbb{E}[y]/\mathbb{E}[f(x)]$，再對 $\lambda f(x)$ 計算指標，$\mathbb{E}[\cdot]$ 為影像平均。由於 LLIE 的「正確」亮度本來就有歧義，這樣做可以大幅提高 PSNR。Retinexformer 的作者在程式碼中提供了這個模式，以便與 LLFlow、KinD 及近期擴散模型等方法比較，但明確表示不建議使用，因為它用真值來修正輸出 [26]。Liao 等人 [51]（2025）研究輸出與參考之間的亮度不匹配，提出用於訓練的 GT-mean 損失函數，並同時回報標準與 GT-mean 指標。讀表格時：先確認使用哪一種設定，絕不混用兩者，並優先選擇在標準協定下表現好的方法。

以任務為導向的評估

由於 LLIE 常是前處理步驟，另一種檢驗方式是在增強後的影像上跑偵測或分割模型（例如在 ExDark [57] 上）並回報準確率。Liu 與 Fan 2025 年的綜述 [5] 發現兩者脫節：監督式方法常產生感知品質高的影像，對視覺任務的提升卻有限；零樣本（zero-shot）方法雖然畫質分數較低，在各種任務上的提升反而更一致。Liu 等人的基準 [4] 也評估了低光下的人臉偵測。完整的評估應該回報保真度、無參考品質、任務表現與執行時間。

基準資料集

白話版。資料集的差別在於：有沒有成對、暗是真實的還是合成的、是 raw 還是 sRGB，以及有沒有偵測用的標註。

資料集	類型	內容與用途
LOL (v1) [21]	成對、真實、sRGB	改變曝光拍攝的 500 組低光/正常光影像（485 組訓練／15 組測試），400×600。最常用的 LLIE 基準；測試集很小。
LOL-v2 [52]	成對，真實＋合成子集	較大的 *Real* 子集（實拍成對）與合成成對資料的 *Synthetic* 子集。兩個子集要分開回報。
LSRW [53]	成對、真實	5,650 組，來自 Nikon D7500（3,170 組）與 Huawei P40 Pro（2,480 組）；5,600 組訓練、50 組測試。測試跨裝置泛化。
SDSD [54]	成對影片	以機電系統對齊、拍攝兩次的動態室內與室外場景；常拆成影格做影像測試。
SID [44]	成對、raw	5,094 張短曝光 raw 影像，搭配 424 張長曝光參考（Sony 與 Fuji）。極低光、raw 到 sRGB。
MIT-Adobe FiveK [55]	成對、修圖	5,000 張 raw 照片，每張由五位專家修圖。用於色調與色彩增強（照片修圖）。
SICE [56]	多重曝光序列	每個場景從曝光不足到過曝的多張拍攝與一張參考；用來訓練曝光校正（例如 Zero-DCE）。
VE-LOL [4]	成對＋偵測	VE-LOL-L 用於增強，VE-LOL-H 附有人臉標註，用於高階任務評估。
ExDark [57]	非配對、有標註	7,363 張低光影像、12 個物件類別、10 種光照條件。以任務為導向的評估（偵測）。
LIME [17]、MEF [58]、NPE [18]、DICM [59]、VV [60]	非配對測試集	小型的真實低光或光照不均照片集，用於視覺比較與無參考指標。

挑戰賽：NTIRE 2024 與 2025

CVPR 的 NTIRE workshop 連續兩年舉辦 LLIE 挑戰賽。**NTIRE 2024** 挑戰賽 [61] 涵蓋超高解析度（4K 以上）、不均勻照明、逆光、極暗與夜景等情境，共有 428 人註冊、22 隊有效提交。**NTIRE 2025** 挑戰賽 [62] 有 762 人註冊、28 隊有效提交。這些報告整理了勝出隊伍的設計，是「在固定且公平的協定下什麼方法有效」的一個好切面。

現代觀點

這個領域在不到十年內從手工先驗走到學習模型，但原文 [1] 提出的核心問題依然相同：如何描述光、如何處理雜訊，以及如何判斷輸出好不好。下面幾篇綜述是最好的入門點。

Li 等人，TPAMI 2022 [2]。引用最多的深度 LLIE 綜述。它依學習策略（監督式、強化學習、非監督式、零樣本與半監督式學習）、網路結構、是否使用 Retinex 模型、資料格式（sRGB 或 raw）、損失函數，以及訓練與測試資料來整理方法。它也提出了 LLIV-Phone：一個由多款手機相機在

各種光照下拍攝的影像與影片資料集，以及一個比較方法的線上平台。主要結論是泛化落差（許多方法在手機資料上表現不佳），以及指標分數與視覺品質經常不一致。

Zheng 等人，"Low-Light Image and Video Enhancement: A Comprehensive Survey and Beyond" [3]。以類似的分類回顧傳統與深度方法，並在資料上更進一步：提出 SICE_Grad 與 SICE_Mix（在單張影像中混合曝光不足與過曝），以及 Night Wenzhou（空拍與街景的夜間影片集），用來測試現有基準沒涵蓋的情況。原文的概觀圖也出自這篇。

Liu 等人，IJCV 2021 [4]。"Benchmarking low-light image enhancement and beyond" 與其說是綜述，不如說是基準：VE-LOL 資料集提供成對資料與人臉標註，以全參考、無參考與語意指標進行比較，並提出一個增強加偵測的聯合模型。它讓以任務為導向的評估成為主流。

Liu 與 Fan，Neurocomputing 2025 [5]。這篇綜述追問哪種增強真正幫得上下游視覺任務，並發現前述畫質指標與任務表現之間的脫節，其中零樣本方法帶來最一致的任務提升。

Adhikarla 等人，2025 [6]。一篇關於擴散模型 LLIE 的綜述，提出六類分類法並做效能分析。它的訊息是：擴散模型帶來真實感與彈性，但推論成本很高，因此加速、引導與潛在空間設計是目前的熱門方向。

NTIRE 報告 [61], [62]。它們不是綜述，但呈現了頂尖隊伍在共同協定、高解析度與困難光照下的實際做法。

目前的最先進技術（2023–2026）。主要有三個方向。第一，**Retinex 引導的骨幹網路**：保留「光與反射率」結構的 Transformer（Retinexformer [26]）與狀態空間模型（RetinexMamba [42]）。第二，**生成模型**：擴散模型（Diff-Retinex [38]、GSAD [39]、LightenDiffusion [40]、Reti-Diff [41]）與 flow（LLFlow [37]）。第三，**更好的表示法與評估協定**：學習出來的色彩空間（HVI-CIDNet [27]）、考慮亮度不匹配的訓練與評估（GT-Mean loss [51]）、語言引導與 all-in-one 復原（CLIP-LIT [36]、InstructIR [43]），以及任務感知的增強（MAET [45]、FeatEnHancer [46]）。

未解決的問題。

- **真實雜訊**。多數 sRGB 方法使用的訓練資料，其雜訊與經過相機處理流程後的真實感測器雜訊不相符。更好的雜訊模型 [7]、raw 域訓練 [44]，或增強與去噪的聯合處理仍是核心課題。
- **影片一致性**。逐影格增強會閃爍。成對的影片資料（SDSD [54]）稀少，時間一致性依然困難，對生成模型尤其如此。
- **評估**。亮度目標本身是病態的、GT-mean 協定、無參考指標對曝光不可靠，以及感知指標與任務指標的落差，都讓比較結果很脆弱。
- **邊緣裝置上的效率**。手機與車輛需要即時、低功耗的模型。曲線方法（Zero-DCE++ [34]、SCI [35]）與輕量的先驗式網路 [28] 證明了極小的模型也能運作，而擴散模型對這類用途仍然太慢。
- **泛化**。在某個資料集或相機上訓練的模型，換到另一個常常失效（[2] 所報告的泛化落差）；混合過曝與曝光不足、逆光，以及有色人工光源仍然困難。

重點整理

- LLIE 是**色調強化加上復原**：完美的亮度校正之後，雜訊、色偏與量化誤差依然存在（圖 1）。
- 亮度可以在獨立通道（V、I、Y'、L^*）中增強，但這個選擇會牽動飽和度。BT.601 亮度為 $0.299R' + 0.587G' + 0.114B'$；$16 + 0.257R' + 0.504G' + 0.098B'$ 是同一個亮度在 8 位元有限範圍

下的寫法。

- Retinex 把影像表示為反射率乘上**照度**，$S = R \circ L$；SSR、MSR、LIME 以平滑先驗估計 L，而反轉影像去霧的做法本質上建立在同一個假設上。
- 深度方法最好依**訓練訊號**分類：監督式、展開式、非配對（EnlightenGAN）、零參考（Zero-DCE、SCI、CLIP-LIT）與生成式（LLFlow、擴散模型）。Retinex 分解橫跨其中各類。
- Zero-DCE 的 LE 曲線 $x + \alpha x(1 - x)$ 保留範圍與順序；它的無參考損失是先驗（曝光、灰色世界、平滑、對比），遇到真實雜訊與色彩鮮豔的場景時可能失效。
- 同時回報全參考、無參考與任務導向的結果，**絕不混用 GT-mean 與標準數字**，並記住 PSNR 較高不代表偵測更好。
- 每個無參考指標都只看到一個面向：NIQE、BRISQUE、PI 看得到結構，卻幾乎看不到曝光與色彩；LOE 看得到明度順序，卻看不到亮度與雜訊；色彩豐富度會被色度雜訊灌大；學習式評估器則繼承了白天照片的偏差。**更亮不代表更好**（圖 9）：請回報多個來自不同類別的無參考指標，再加上設計良好的使用者研究。

練習

1. **單調曲線**。證明 $\mathrm{LE}(x;\alpha) = x + \alpha x(1 - x)$ 在 $[0, 1]$ 上單調不減，若且唯若 $|\alpha| \le 1$。$\alpha = 1.5$ 時會出什麼問題？

   提示

   導數為 $1 + \alpha(1 - 2x)$，對 x 是線性的，所以在 $[0, 1]$ 上的最小值出現在端點：$x = 0$ 時為 $1 + \alpha$，$x = 1$ 時為 $1 - \alpha$。兩者都 ≥ 0 恰好等價於 $|\alpha| \le 1$。$\alpha = 1.5$ 時，$x = 1$ 處的斜率為 -0.5，接近白色的像素會隨輸入變亮而變暗，明暗順序反轉，在亮部造成假影。

2. **迭代曲線的增益**。對很小的輸入 x，證明以固定 α 迭代八次大約會把 x 乘上 $(1 + \alpha)^8$。用這個結果解釋為什麼零參考曲線方法會放大暗部雜訊，並提出一個能減輕問題的損失函數或架構修改。

   提示

   x 很小時 $x(1 - x) \approx x$，所以每一步都是 $x \mapsto (1 + \alpha)x$。$\alpha = 1$ 時就是 $2^8 = 256$，標準差為 σ 的雜訊變成 256σ。可能的修正：加入去噪分支或考慮雜訊的損失、在估計訊噪比低的地方限制 α（類似 SNR-aware 的精神），或懲罰輸出中、平滑後的輸入所沒有的高頻能量。

3. **兩個亮度公式**。一個 8 位元像素為 $(R', G', B') = (200, 100, 50)$。計算它的 BT.601 全範圍亮度與有限範圍亮度，並說明兩者編碼的是同一個量。

   提示

   全範圍：$0.299 \cdot 200 + 0.587 \cdot 100 + 0.114 \cdot 50 = 124.2$。有限範圍：$16 + 0.257 \cdot 200 + 0.504 \cdot 100 + 0.098 \cdot 50 \approx 122.7$。換算回去：$(122.7 - 16) \cdot 255/219 \approx 124.2$。唯一的差別是比例 $219/255$ 與偏移量 16。

4. **沒有真值的 Retinex 損失**。在 Retinex-Net 中，為什麼需要 $\mathcal{L}_{ir} = \|R_{\text{low}} - R_{\text{normal}}\|_1$？描述一個能讓 $\mathcal{L}_{\text{recon}}$ 的自我重建項與 $\mathcal{L}_{is}$ 都變成零、卻毫無用處的分解，並說明 $\mathcal{L}_{ir}$ 如何排除它。

提示

令 $I_i \equiv 1$（完全平滑）且 $R_i = S_i$。每張影像都能被完美重建，平滑損失也為零，但根本沒有做任何分解。由於 $S_{\text{low}} \neq S_{\text{normal}}$，這會讓 $R_{\text{low}} \neq R_{\text{normal}}$，所以 $\mathcal{L}_{ir}$ 很大。要求兩者反射率相等，就迫使兩張影像之間的亮度差異被放進照度裡。

5. **手算 GT-mean**。你的模型輸出平均為 0.40，真值平均為 0.50，除了這個整體倍率之外輸出完美無缺。GT-mean 的縮放因子是多少？如果影像是常數，縮放前後的 PSNR 各是多少？為什麼縮放後的數字無法公平地與正常評估的方法比較？

提示

$\lambda = 0.50/0.40 = 1.25$。縮放前：MSE $= 0.1^2 = 0.01$，PSNR $= 20$ dB。縮放後輸出等於參考，誤差為 0，PSNR 趨於無窮大。實際使用時方法永遠拿不到真值的平均，所以 GT-mean 數字獎勵的是正確的結構，卻忽略了任務中最難的部分：決定亮度。

6. **更換色彩先驗**。在 PyTorch 小玩具中，把灰色世界損失換成保留輸入色度的損失，例如 $\|y/\sum_c y_c - x/\sum_c x_c\|_1$。重新訓練，並把咖啡杯的結果與圖 7 比較。兩種先驗各在什麼情況下較好？

提示

分母要加上一個小 epsilon。保留輸入色度能讓紅褐色的杯子維持紅色，但也會保留光源的色偏（例如鈉燈）。灰色世界能去除色偏，卻會讓真正色彩鮮豔的場景褪色。實際的方法常結合較弱的色彩先驗，與從資料學到的色彩校正。

7. **手算 LOE**。一張三像素的輸入，明度為 $L = (0.1, 0.2, 0.3)$，某個增強器輸出 $L_e = (0.5, 0.4, 0.9)$。計算每個像素的 RD(x) 與 LOE。接著解釋為什麼在圖 9 中，乾淨參考影像的 LOE（322）反而高於 gamma 曲線（0），以及這對「只用 LOE 比較一個會去噪的增強器與單純的色調曲線」意味著什麼。

提示

只有像素對 $(1, 2)$ 的順序改變：輸入中 $0.1 < 0.2$，輸出中卻是 $0.5 > 0.4$。這同時翻轉了 $U(L_1, L_2)$ 與 $U(L_2, L_1)$，所以 RD $= (1, 1, 0)$（像素和自己比較永遠不會翻轉），LOE $= 2/3$。逐通道套用的 gamma 曲線是單調的，會完全保留 $\max(R, G, B)$ 的順序，所以得到 0。乾淨影像沒有暗輸入裡的雜訊，而這些雜訊早已打亂許多相鄰、數值幾乎相等的像素順序，因此乾淨影像在這些像素對上與輸入不一致。換句話說，LOE 獎勵保留輸入的雜訊、懲罰真正的復原；它必須搭配雜訊或品質量測一起看，絕不能單獨使用。

參考文獻

1. Shyandram，〈簡介 Deep Learning Low-Light Image Enhancement (LLIE)〉，Medium 文章，2024；2026 年更新。連結
2. C. Li, C. Guo, L. Han, J. Jiang, M.-M. Cheng, J. Gu and C. C. Loy, "Low-light image and video enhancement using deep learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 12, pp. 9396–9416, 2022. arXiv
3. S. Zheng, Y. Ma, J. Pan, C. Lu and G. Gupta, "Low-light image and video enhancement: A comprehensive survey and beyond," arXiv:2212.10772, 2022. arXiv
4. J. Liu, D. Xu, W. Yang, M. Fan and H. Huang, "Benchmarking low-light image enhancement and beyond," *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1153–1184, 2021. doi project
5. F. Liu and L. Fan, "A review of advancements in low-light image enhancement using deep learning," *Neurocomputing*, vol. 652, art. 131052, 2025. arXiv
6. E. Adhikarla, Y. Liu and B. D. Davison, "Diffusion models for low-light image enhancement: A multi-perspective taxonomy and performance analysis," arXiv:2510.05976, 2025. arXiv
7. A. Foi, M. Trimeche, V. Katkovnik and K. Egiazarian, "Practical Poissonian-Gaussian noise modeling and fitting for single-image raw-data," *IEEE Transactions on Image Processing*, vol. 17, no. 10, pp. 1737–1754, 2008. doi
8. ITU-R, "Recommendation BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios," 2011. ITU
9. S. M. Pizer, E. P. Amburn, J. D. Austin, et al., "Adaptive histogram equalization and its variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987. doi
10. K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, Academic Press, 1994, pp. 474–485. doi
11. E. H. Land, "The retinex theory of color vision," *Scientific American*, vol. 237, no. 6, pp. 108–128, 1977. doi
12. D. J. Jobson, Z. Rahman and G. A. Woodell, "Properties and performance of a center/surround retinex," *IEEE Transactions on Image Processing*, vol. 6, pp. 451–462, 1997. doi
13. D. J. Jobson, Z. Rahman and G. A. Woodell, "A multiscale retinex for bridging the gap between color images and the human observation of scenes," *IEEE Transactions on Image Processing*, vol. 6, no. 7, pp. 965–976, 1997. doi
14. K. He, J. Sun and X. Tang, "Single image haze removal using dark channel prior," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 12, pp. 2341–2353, 2011. doi
15. X. Dong, Y. Pang and J. Wen, "Fast efficient algorithm for enhancement of low lighting video," *ACM SIGGRAPH 2010 Posters*, 2010. SIGGRAPH history doi
16. X. Dong, G. Wang, Y. Pang, W. Li, J. Wen, W. Meng and Y. Lu, "Fast efficient algorithm for enhancement of low lighting video," in *Proc. IEEE International Conference on Multimedia and Expo (ICME)*, 2011. doi
17. X. Guo, Y. Li and H. Ling, "LIME: Low-light image enhancement via illumination map estimation," *IEEE Transactions on Image Processing*, vol. 26, no. 2, pp. 982–993, 2017. doi
18. S. Wang, J. Zheng, H.-M. Hu and B. Li, "Naturalness preserved enhancement algorithm for non-uniform illumination images," *IEEE Transactions on Image Processing*, vol. 22, no. 9, pp. 3538–3548, 2013. doi

19. K. G. Lore, A. Akintayo and S. Sarkar, “LLNet: A deep autoencoder approach to natural low-light image enhancement,” *Pattern Recognition*, vol. 61, pp. 650–662, 2017. doi
20. L. Tao, C. Zhu, J. Song, T. Lu, H. Jia and X. Xie, “Low-light image enhancement using CNN and bright channel prior,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3215–3219. doi
21. C. Wei, W. Wang, W. Yang and J. Liu, “Deep retinex decomposition for low-light enhancement,” in *Proc. British Machine Vision Conference (BMVC)*, 2018. arXiv project
22. Y. Zhang, J. Zhang and X. Guo, “Kindling the darkness: A practical low-light image enhancer,” in *Proc. ACM International Conference on Multimedia (ACM MM)*, 2019, pp. 1632–1640. doi
23. Y. Zhang, X. Guo, J. Ma, W. Liu and J. Zhang, “Beyond brightening low-light images,” *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1013–1037, 2021. doi
24. S. W. Zamir, A. Arora, S. Khan, et al., “Learning enriched features for real image restoration and enhancement,” in *Proc. European Conference on Computer Vision (ECCV)*, 2020, pp. 492–511. arXiv
25. X. Xu, R. Wang, C.-W. Fu and J. Jia, “SNR-aware low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 17693–17703. doi
26. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, “Retinexformer: One-stage Retinex-based transformer for low-light image enhancement,” in *Proc. IEEE/CVF ICCV*, 2023. arXiv code
27. Q. Yan, Y. Feng, C. Zhang, et al., “HVI: A new color space for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2025. arXiv
28. S.-E. Weng, S.-G. Miaou and R. Christanto, “A lightweight low-light image enhancement network via channel prior and gamma correction,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 39, no. 12, art. 2554013, 2025. doi code
29. S.-E. Weng, C.-Y. Hsiao, L.-W. Lu, et al., “Rethinking theoretical illumination for efficient low-light image enhancement” (CPGA-Net+), arXiv:2409.05274, 2024. arXiv
30. R. Liu, L. Ma, J. Zhang, X. Fan and Z. Luo, “Retinex-inspired unrolling with cooperative prior architecture search for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2021, pp. 10556–10565. arXiv
31. W. Wu, J. Weng, P. Zhang, X. Wang, W. Yang and J. Jiang, “URetinex-Net: Retinex-based deep unfolding network for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 5891–5900. doi
32. Y. Jiang, X. Gong, D. Liu, et al., “EnlightenGAN: Deep light enhancement without paired supervision,” *IEEE Transactions on Image Processing*, vol. 30, pp. 2340–2349, 2021. arXiv
33. C. Guo, C. Li, J. Guo, et al., “Zero-reference deep curve estimation for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2020, pp. 1777–1786. arXiv
34. C. Li, C. Guo and C. C. Loy, “Learning to enhance low-light image via zero-reference deep curve estimation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, 2022. arXiv
35. L. Ma, T. Ma, R. Liu, X. Fan and Z. Luo, “Toward fast, flexible, and robust low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 5627–5636. arXiv
36. Z. Liang, C. Li, S. Zhou, R. Feng and C. C. Loy, “Iterative prompt learning for unsupervised backlit image enhancement,” in *Proc. IEEE/CVF ICCV*, 2023. arXiv
37. Y. Wang, R. Wan, W. Yang, H. Li, L.-P. Chau and A. C. Kot, “Low-light image enhancement with

normalizing flow," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 36, no. 3, pp. 2604–2612, 2022. arXiv
38. X. Yi, H. Xu, H. Zhang, L. Tang and J. Ma, "Diff-Retinex: Rethinking low-light image enhancement with a generative diffusion model," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
39. J. Hou, Z. Zhu, J. Hou, H. Liu, H. Zeng and H. Yuan, "Global structure-aware diffusion process for low-light image enhancement," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv
40. H. Jiang, A. Luo, X. Liu, S. Han and S. Liu, "LightenDiffusion: Unsupervised low-light image enhancement with latent-Retinex diffusion models," in *Proc. European Conference on Computer Vision (ECCV)*, 2024. arXiv
41. C. He, C. Fang, Y. Zhang, et al., "Reti-Diff: Illumination degradation image restoration with Retinex-based latent diffusion model," in *Proc. International Conference on Learning Representations (ICLR)*, 2025. proceedings
42. J. Bai, Y. Yin, Q. He, Y. Li and X. Zhang, "RetinexMamba: Retinex-based Mamba for low-light image enhancement," arXiv:2405.03349, 2024. arXiv
43. M. V. Conde, G. Geigle and R. Timofte, "InstructIR: High-quality image restoration following human instructions," in *Proc. European Conference on Computer Vision (ECCV)*, 2024. arXiv
44. C. Chen, Q. Chen, J. Xu and V. Koltun, "Learning to see in the dark," in *Proc. IEEE/CVF CVPR*, 2018, pp. 3291–3300. arXiv
45. Z. Cui, G.-J. Qi, L. Gu, S. You, Z. Zhang and T. Harada, "Multitask AET with orthogonal tangent regularity for dark object detection," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
46. K. A. Hashmi, G. Kallempudi, D. Stricker and M. Z. Afzal, "FeatEnHancer: Enhancing hierarchical features for object detection and beyond under low-light vision," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
47. Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi
48. R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
49. A. Mittal, R. Soundararajan and A. C. Bovik, "Making a 'completely blind' image quality analyzer," *IEEE Signal Processing Letters*, vol. 20, no. 3, pp. 209–212, 2013. doi
50. A. Mittal, A. K. Moorthy and A. C. Bovik, "No-reference image quality assessment in the spatial domain," *IEEE Transactions on Image Processing*, vol. 21, no. 12, pp. 4695–4708, 2012. doi
51. J. Liao, S. Hao, R. Hong and M. Wang, "GT-Mean loss: A simple yet effective solution for brightness mismatch in low-light image enhancement," in *Proc. IEEE/CVF ICCV*, 2025. arXiv
52. W. Yang, W. Wang, H. Huang, S. Wang and J. Liu, "Sparse gradient regularized deep Retinex network for robust low-light image enhancement," *IEEE Transactions on Image Processing*, vol. 30, pp. 2072–2086, 2021. doi
53. J. Hai, Z. Xuan, R. Yang, et al., "R2RNet: Low-light image enhancement via Real-low to Real-normal Network," *Journal of Visual Communication and Image Representation*, vol. 90, art. 103712, 2023. arXiv
54. R. Wang, X. Xu, C.-W. Fu, et al., "Seeing dynamic scene in the dark: A high-quality video dataset with mechatronic alignment," in *Proc. IEEE/CVF ICCV*, 2021, pp. 9680–9689. doi code
55. V. Bychkovsky, S. Paris, E. Chan and F. Durand, "Learning photographic global tonal adjust-

ment with a database of input/output image pairs," in *Proc. IEEE CVPR*, 2011, pp. 97–104. dataset
56. J. Cai, S. Gu and L. Zhang, "Learning a deep single image contrast enhancer from multi-exposure images," *IEEE Transactions on Image Processing*, vol. 27, no. 4, pp. 2049–2062, 2018. doi
57. Y. P. Loh and C. S. Chan, "Getting to know low-light images with the Exclusively Dark dataset," *Computer Vision and Image Understanding*, vol. 178, pp. 30–42, 2019. arXiv
58. K. Ma, K. Zeng and Z. Wang, "Perceptual quality assessment for multi-exposure image fusion," *IEEE Transactions on Image Processing*, vol. 24, no. 11, pp. 3345–3356, 2015. doi
59. C. Lee, C. Lee and C.-S. Kim, "Contrast enhancement based on layered difference representation of 2D histograms," *IEEE Transactions on Image Processing*, vol. 22, no. 12, pp. 5372–5384, 2013. doi
60. V. Vonikakis, "Datasets" (source of the VV test images), personal web page. link
61. X. Liu, Z. Wu, A. Li, et al., "NTIRE 2024 challenge on low light image enhancement: Methods and results," in *Proc. IEEE/CVF CVPR Workshops*, 2024. arXiv
62. X. Liu, Z. Wu, F.-A. Vasluianu, et al., "NTIRE 2025 challenge on low light image enhancement: Methods and results," in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
63. Y. Blau, R. Mechrez, R. Timofte, T. Michaeli and L. Zelnik-Manor, "The 2018 PIRM challenge on perceptual image super-resolution," in *Proc. European Conference on Computer Vision (ECCV) Workshops*, 2018. arXiv
64. C. Ma, C.-Y. Yang, X. Yang and M.-H. Yang, "Learning a no-reference quality metric for single-image super-resolution," *Computer Vision and Image Understanding*, vol. 158, pp. 1–16, 2017. doi arXiv
65. D. Hasler and S. E. Süsstrunk, "Measuring colorfulness in natural images," in *Proc. SPIE 5007, Human Vision and Electronic Imaging VIII*, 2003. doi
66. J. Immerkær, "Fast noise variance estimation," *Computer Vision and Image Understanding*, vol. 64, no. 2, pp. 300–302, 1996. doi
67. H. Talebi and P. Milanfar, "NIMA: Neural image assessment," *IEEE Transactions on Image Processing*, 2018. arXiv
68. J. Ke, Q. Wang, Y. Wang, P. Milanfar and F. Yang, "MUSIQ: Multi-scale image quality transformer," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
69. J. Wang, K. C. K. Chan and C. C. Loy, "Exploring CLIP for assessing the look and feel of images," in *Proc. AAAI Conference on Artificial Intelligence*, 2023. arXiv
70. S. Yang, T. Wu, S. Shi, et al., "MANIQA: Multi-dimension attention network for no-reference image quality assessment," in *Proc. IEEE/CVF CVPR Workshops*, 2022, pp. 1190–1199. doi arXiv
71. H. Wu, Z. Zhang, W. Zhang, C. Chen, et al., "Q-Align: Teaching LMMs for visual scoring via discrete text-defined levels," in *Proc. International Conference on Machine Learning (ICML)*, PMLR 235, 2024. arXiv
72. V. Kannan, S. Malik, N. C. Babu and R. Soundararajan, "Quality assessment of low-light restored images: A subjective study and an unsupervised model," *IEEE Access*, vol. 11, pp. 68216–68230, 2023. doi arXiv
73. ITU-R, "Recommendation BT.500-15: Methodologies for the subjective assessment of the quality of television images," 2023. ITU
74. R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: The method of

paired comparisons,” *Biometrika*, vol. 39, no. 3–4, pp. 324–345, 1952. doi
75. C. Chen and J. Mo, “IQA-PyTorch: PyTorch toolbox for image quality assessment,” software (pip package `pyiqa`). GitHub

DLCV 06・物件追蹤：從單一物件到多物件

電腦視覺如何在影片中跟住物體：重識別、從相關濾波器到 Transformer 與 SAM 2 的單物件追蹤，以及結合卡爾曼濾波器、匈牙利演算法、端到端模型與 MOTA ／ IDF1 ／ HOTA 指標的多物件追蹤。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-06-object-tracking/

你將學到

- 追蹤在「分類 → 偵測」之後的位置，以及每個追蹤器都要做的兩件事：感知跨影格的運動、維持每個物件的身分。
- 一套清楚的分類：重識別（re-identification）與時序追蹤；單物件追蹤（SOT ／ VOT）與多物件追蹤（MOT）；以及多目標跨攝影機追蹤（MTMCT）。
- 單物件追蹤的演進：相關濾波器（MOSSE、KCF）、孿生網路（Siamese；SiamFC、SiamRPN）、單流 Transformer（OSTrack、SeqTrack），以及影片分割模型（SAM 2、SAMURAI、SAM 3）。
- 深入多物件追蹤：「先偵測再追蹤」（tracking-by-detection）流程（偵測器、卡爾曼濾波器、外觀嵌入、匈牙利演算法）、SORT 家族（SORT、DeepSORT、ByteTrack、OC-SORT、BoT-SORT）、聯合偵測與嵌入（JDE、FairMOT）、以 query 為基礎的端到端追蹤器（MOTR、MOTRv2、MOTIP），以及開放詞彙追蹤。
- 怎麼評估追蹤器：Re-ID 的 CMC 與 mAP、SOT 的 success 與 precision、MOT 的 MOTA、IDF1 與 HOTA，以及 HOTA 為什麼會被提出。
- 一個可以直接執行的 NumPy 追蹤器，並在合成場景上實際量測 ID 切換與各項指標。

先看全貌

偵測器看一張圖，告訴你「這裡有三個人」。追蹤器看一段影片，告訴你「這個人和一秒前那個是**同一個**，而那一位剛走到柱子後面，等一下會再出來」。追蹤就是在偵測之上加入時間與記憶。

用最簡單的話說

想像你和朋友一起看足球賽，各自挑一位最喜歡的球員。在一張照片裡找出場上所有球員，是**偵測**。眼睛一直盯著**你的**球員，看她跑、轉身、被別人擋住一下又出現，是**單物件追蹤**。幫**每一位**球員都貼上號碼貼紙，而且就算兩個人撞在一起，貼紙也絕對不會貼錯，是**多物件追蹤**。電視切到另一台攝影機時，你還認得出你的球員，就是**重識別**。

為什麼重要：自駕車要靠追蹤才知道行人是**正走向**馬路，而不只是站在路邊；運動分析要靠追蹤才算得出每位球員跑了多遠；倉儲要靠追蹤來清點堆高機和機器人；影片剪輯要靠追蹤讓遮罩黏在移動的物體上。這些應用不只要知道東西**在哪裡**，還要知道**誰是誰**、隨時間怎麼變化。

關於本篇

本篇由作者先前在 Medium 發表的文章〈物件追蹤 Object Tracking 簡介〉[1] 擴充而成，保留原文架構，並補上技術細節、可執行的追蹤器與最新的文獻回顧。原文請見這裡。

從分類、偵測到追蹤

白話版。分類回答「圖裡**是什麼**」；偵測回答「是什麼、**在哪裡**」；追蹤回答「是什麼、在哪裡、**是哪一個**，而且隨時間持續」。

[1] 的這條路線，正好是電腦視覺任務的一張地圖：

1. **影像分類**把影像 I 對應到一個標籤 y。
2. **物件偵測**把影像對應到一組帶有類別與信心分數的框，$\mathcal{D} = \{(b_j, c_j, s_j)\}_{j=1}^{N}$，其中 b_j 是邊界框，c_j 是類別，$s_j \in [0,1]$ 是分數。偵測模仿的是人類視覺判斷畫面中目標位置的能力，詳細內容請見DLCV 03・物件偵測。
3. **物件追蹤**輸入影格序列 $I_1, \dots, I_T$，輸出**軌跡**：對每個物件 k，給出它在可見影格集合 $\mathcal{T}_k$ 上的一串框（或遮罩）$(b_t^k)_{t \in \mathcal{T}_k}$，而且全部帶著同一個身分 k。

新加入的關鍵元素是身分標籤。偵測是無狀態的：第 t 格與第 $t+1$ 格各自獨立處理。追蹤則是有狀態的：它要把資訊帶到下一格。

追蹤在做什麼？

白話版。追蹤模仿人類視覺對運動物體的感知：我們會預測東西接下來往哪裡去、往那裡看，並在它外觀改變或短暫被擋住時，依然把它當成同一個東西。

更精確地說，每個追蹤器都在解兩個互相耦合的子問題：

- **隨時間定位**（運動感知）：估計每個目標在每一格中的位置，最好能用過去預測未來。這讓追蹤器能撐過模糊、漏偵測與短暫遮擋。
- **維持身分**（資料關聯，data association）：決定第 t 格的哪個觀測屬於第 $t-1$ 格的哪個目標。這能防止兩個交錯而過的人互換標籤。

單物件追蹤因為只有一個身分，主要力氣花在第一個問題；多物件追蹤主要在處理第二個問題；重識別則是**完全沒有**運動資訊時的第二個問題。

[1] 對各類方法列出的困難，都來自這兩件事：外觀變化（姿態、光照、尺度、視角）、被其他物體或場景遮擋、長得很像的干擾物、快速或非線性的運動、攝影機本身的移動，以及目標進出畫面。

追蹤的分類

白話版。追蹤器可以依照**追什麼**（一個或多個物件、一台或多台攝影機）來分，也可以依照**怎麼維持身分**（只靠外觀，或是逐格跟著運動）來分。

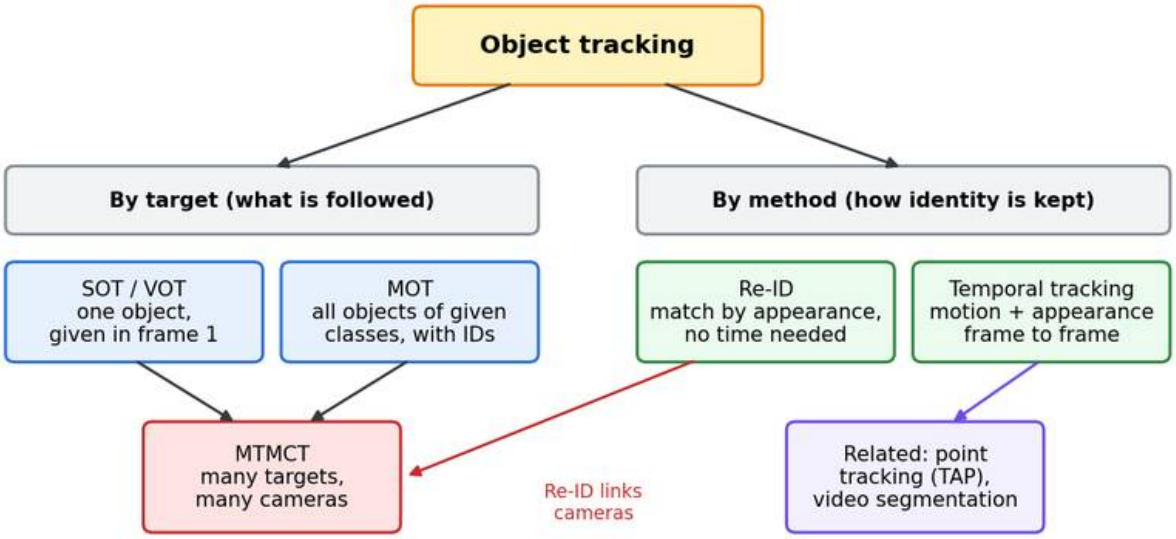


圖 *1* — 追蹤的分類。依目標：單物件（*SOT*，又稱 *VOT*）與多物件（*MOT*）。依方法：只靠外觀的重識別，以及時序追蹤。兩者結合後最困難的情境是多目標跨攝影機追蹤（*MTMCT*）。

依照 [1]：

- **依作法**。重識別（*Re-ID*）只靠外觀（有時加上姿態），在時間相隔很遠、或來自不同攝影機的影像之間比對同一個目標。時序物件追蹤則在同一段影片中逐格跟隨目標，可以利用運動的連續性。
- **依目標**。視覺物件追蹤（*visual object tracking, VOT*）又稱單物件追蹤（*single-object tracking, SOT*），追蹤第一格中以框指定的任意一個物件。多物件追蹤（*multi-object tracking, MOT*）則在**連續影格**中追蹤指定類別的所有物件（例如每一位行人），並為每個物件維持一致的 ID。
- **終極目標**。多目標跨攝影機追蹤（*Multi-Target Multi-Camera Tracking, MTMCT*）在整個攝影機網路中追蹤多個目標，當一個人離開某台攝影機、出現在另一台時，仍維持相同身分。它結合了單一攝影機內的 MOT 與攝影機之間的 Re-ID。

傳統文獻與大多數基準資料集都以**行人**為主要目標 [1]，因為監控與自駕車讓「人」成為最有價值的追蹤類別。新的基準則加入了舞者、運動員、車輛與開放詞彙類別。

重識別（Re-ID）

白話版。 Re-ID 是一個搜尋問題：給你某台攝影機拍到的一個人，請在其他攝影機拍到的照片中找出同一個人，並由最像到最不像排序。

問題定義

Re-ID 被定義成**檢索**（retrieval）問題 [7]。查詢影像（query）q 與由其他攝影機偵測出的行人圖庫（gallery）$\mathcal{G} = \{g_1, \dots, g_M\}$，都經由網路 f_θ 映射成嵌入向量，再依距離排序圖庫，通常使用餘弦距離

$$d(q, g) = 1 - \frac{f_\theta(q)^\top f_\theta(g)}{\|f_\theta(q)\| \, \|f_\theta(g)\|},$$

其中 $f_\theta(\cdot) \in \mathbb{R}^D$ 是參數為 θ 的網路輸出的 D 維嵌入。訓練的目標是讓同一身分的嵌入靠近、不同身分的嵌入遠離。常見做法是把身分分類損失（對 C 個訓練身分做交叉熵）與三**元組損失**（triplet loss）結合：

$$\mathcal{L}_{\text{tri}} = \left[\, d(a, p) - d(a, n) + m \,\right]_+,$$

其中 a 是錨點影像，p 是正樣本（同一身分），n 是負樣本（不同身分），$m > 0$ 是邊界（margin），$[x]_+ = \max(x, 0)$。測試時的身分從未在訓練中出現，所以 Re-ID 是開放集合（open-set）問題：網路必須學到一般性的「是不是同一個人」。

挑戰

[1] 把困難分成**資料因素**與**環境因素**。資料因素：標註身分數量有限、偵測器給出的框有雜訊、身體只拍到一部分、解析度低。環境因素：視角與各攝影機色彩響應不同、光照、遮擋、雜亂背景，以及換了衣服的人。Re-ID 也帶來明顯的**隱私**疑慮，這也是一些監控型資料集被撤下的原因之一（見下文）。原文把城市級監控系統列為主要應用；如今 Re-ID 同樣常見於 MOT 追蹤器裡的外觀模組，以及零售與運動分析。

代表模型

- **強基線（strong baseline）**。 Luo 等人 [14] 證明：一個普通的 ResNet-50，搭配一組訓練「技巧」——學習率暖身（warm-up）、隨機擦除（random erasing）資料增強、標籤平滑（label smoothing）、最後一層 stride 設為 1 以得到更大的特徵圖，以及 **BNNeck**（一層批次正規化，把三元組損失用的特徵與分類用的特徵分開）——就是一個強大且可重現的基線。
- **質心三元組損失（centroid triplet loss, CTL）**。原文提到的模型（ResNet-50，輸入 256×128）[15]，把「樣本對樣本」的三元組換成「樣本對類別質心」的距離，檢索時也是拿查詢影像與各身分的質心比較，而不是與每一張圖庫影像比較。
- **CLIP-ReID**。 Li 等人 [16] 把視覺語言模型 CLIP 用在沒有文字標籤的 Re-ID 上。第一階段凍結編碼器，學習每個身分專屬的文字 token；第二階段再以這些文字特徵作為額外監督，微調影像編碼器。

資料集

資料集	內容	狀態
Market-1501 [12]	1,501 個身分、6 台攝影機、偵測器裁切的框	可取得
MSMT17 [13]	4,101 個身分、15 台室內外攝影機	可取得
DukeMTMC [10]（及 DukeMTMC-reID）	8 台同步的校園攝影機	**2019 年已撤下**

DukeMTMC 與跨攝影機追蹤的 IDF1 指標一起被提出 [10]，其中的 Re-ID 子集也被廣泛使用。由於在校園拍攝行人引發的隱私與倫理疑慮，作者已於 2019 年撤下該資料集，不應再下載或使用；Peng 等人 [11] 追蹤了它在撤下後仍以衍生版本流傳的情形，並主張資料集需要更好的管理（stewardship）。

指標：CMC rank-k 與 mAP

累積匹配特性（cumulative matching characteristic, CMC）在 rank k 的值，是「第一個正確匹配出現在排序後圖庫前 k 名」的查詢比例：

$$\mathrm{CMC}(k) = \frac{1}{|Q|}\sum_{q \in Q} \mathbb{1}\left[\, r_q \le k \,\right],$$

其中 Q 是查詢集合，r_q 是查詢 q 的第一個正確匹配的名次。*Rank-1* 準確率就是 $\mathrm{CMC}(1)$。由於一個查詢在圖庫中通常有好幾個正確匹配，只看 rank-k 會忽略**其他**正確匹配排得好不好。**平均精確度均值**（mean average precision, mAP）補上了這一點：

$$\mathrm{AP}(q) = \frac{1}{G_q}\sum_{i=1}^{G_q} \frac{i}{r_{q,i}}, \qquad \mathrm{mAP} = \frac{1}{|Q|}\sum_{q} \mathrm{AP}(q),$$

其中 G_q 是 q 在圖庫中的正確匹配數，$r_{q,i}$ 是第 i 個正確匹配的名次，所以 $i / r_{q,i}$ 就是找到第 i 個正確匹配那一刻的精確度。

```
import numpy as np

def cmc_map(dist, q_ids, g_ids, max_rank=5):
    """dist: (Q, G) distances between query and gallery embeddings."""
    order = np.argsort(dist, axis=1)                # closest gallery items first
    match = g_ids[order] == q_ids[:, None]          # (Q, G) True where same person
    cmc = (np.cumsum(match, axis=1) > 0).mean(axis=0)[:max_rank]
    aps = []
    for row in match:
        hits = np.flatnonzero(row)                  # ranks (0-based) of true matches
        precision_at_hits = np.arange(1, len(hits) + 1) / (hits + 1)
        aps.append(precision_at_hits.mean())
    return cmc, np.mean(aps)

rng = np.random.default_rng(0)
centres = rng.normal(size=(50, 64))                 # 50 identities, 64-D embeddings
```

```
g_ids = np.repeat(np.arange(50), 4)                 # 4 gallery images per identity
q_ids = np.arange(50)
gallery = centres[g_ids] + 1.6 * rng.normal(size=(200, 64))
query = centres[q_ids] + 1.6 * rng.normal(size=(50, 64))
norm = lambda x: x / np.linalg.norm(x, axis=1, keepdims=True)
dist = 1 - norm(query) @ norm(gallery).T            # cosine distance
cmc, mAP = cmc_map(dist, q_ids, g_ids)
print("rank-1 %.2f  rank-5 %.2f  mAP %.2f" % (cmc[0], cmc[4], mAP))
```

在這個合成圖庫（每個身分 4 張）上，程式輸出 `rank-1 0.76  rank-5 0.92  mAP 0.57`：大多數查詢都能在前面找到一**張**正確影像，但另外三張散落在後面，只有 mAP 看得出來。

單物件追蹤

白話版。你在第一格框住某個東西，追蹤器必須在接下來整段影片中一直把框放在同一個東西上，而且事先不知道它是哪一類物體。

問題定義

給定影格 $I_1, \dots, I_T$ 與初始框 b_1，單物件追蹤器輸出 $\hat{b}_2, \dots, \hat{b}_T$。目標**與類別無關**：可能是人、鳥或咖啡杯。因此 SOT 是一種 *one-shot* 學習：目標唯一的範例就是第一格的影像塊，也就是**模板**（template）z。在每個新影格，追蹤器在上一個位置附近的搜尋區域 x 中尋找目標。[1] 列出的挑戰是：複雜的外觀變化、移動模式，以及光照、遮擋等環境變化。

傳統方法：相關濾波器

在深度學習之前，最快的追蹤器會線上學習一個**相關濾波器**（correlation filter）。MOSSE [17] 在頻率域學習濾波器 H，讓它與訓練影像塊 F_i 做相關後，在目標中心產生尖銳的高斯峰 G_i：

$$\min_{H^*} \sum_i |F_i \odot H^* - G_i|^2 \quad \Rightarrow \quad H^* = \frac{\sum_i G_i \odot F_i^*}{\sum_i F_i \odot F_i^*},$$

其中大寫字母代表二維離散傅立葉轉換，$\odot$ 是逐元素相乘，* 是複數共軛，除法也是逐元素進行。因為相關運算在頻率域變成乘法（見IP 04），訓練與偵測只需要幾次 FFT，在 CPU 上就能達到每秒數百格。KCF [18] 指出，用影像塊的**所有循環位移**來訓練，會得到一個可以被 DFT 對角化的循環矩陣，於是核嶺迴歸（kernel ridge regression）變成逐元素運算，也能使用 HOG 等多通道特徵。它們的弱點是搜尋視窗固定、循環假設造成的邊界效應，以及人工設計的特徵。

孿生網路追蹤器

白話版。離線用大量影片訓練一個網路，只回答一個問題：「模板出現在這張搜尋影像的哪裡？」測試時直接凍結使用。

SiamFC [19] 把**同一個**全卷積網路 φ 同時套用在模板 z 與搜尋區域 x 上，再把兩張特徵圖做互相關：

$$f(z, x) = \varphi(z) \star \varphi(x) + b\,\mathbb{1},$$

其中 $\star$ 是互相關（模板特徵扮演卷積核），b 是可學習的偏差，$\mathbb{1}$ 是全為 1 的圖。輸出是一張分數圖，峰值位置就是新位置。訓練時使用邏輯損失（logistic loss），真實中心附近標為正、其他位置標為負。由於不做線上更新，它速度快、也不容易漂移，但只能預測位置，尺度則靠搜尋一個小型影像金字塔來處理。

SiamRPN [20] 在孿生特徵之後加上**區域提議網路**（region proposal network）：一個分支判斷錨框是目標還是背景，另一個分支迴歸框的偏移量。追蹤因此變成以模板定義類別的「one-shot 偵測」，能輸出長寬比會變化的精確框。

Transformer 單流追蹤器

2022 年後，主流從孿生 CNN 轉向 Transformer。關鍵差別在於模板與搜尋區域**在哪裡**互動（圖 2）。雙流（two-stream）設計中，兩者分別編碼，最後才比較一次。在 OSTrack [21] 這類**單流**（one-stream）設計中，z 與 x 的影像 patch 被串成同一個 token 序列，由同一個 Vision Transformer 處理，因此每一層注意力都同時在擷取特徵、也在建立模板與搜尋區域之間的關係。OSTrack 也會在前幾層丟掉明顯是背景的搜尋 token 以節省計算。SeqTrack [22] 更進一步簡化：由編碼器—解碼器 Transformer 以自迴歸方式，把框的四個座標當成離散 token 序列逐一生成，只用一般的交叉熵訓練——與語言模型的做法相同。

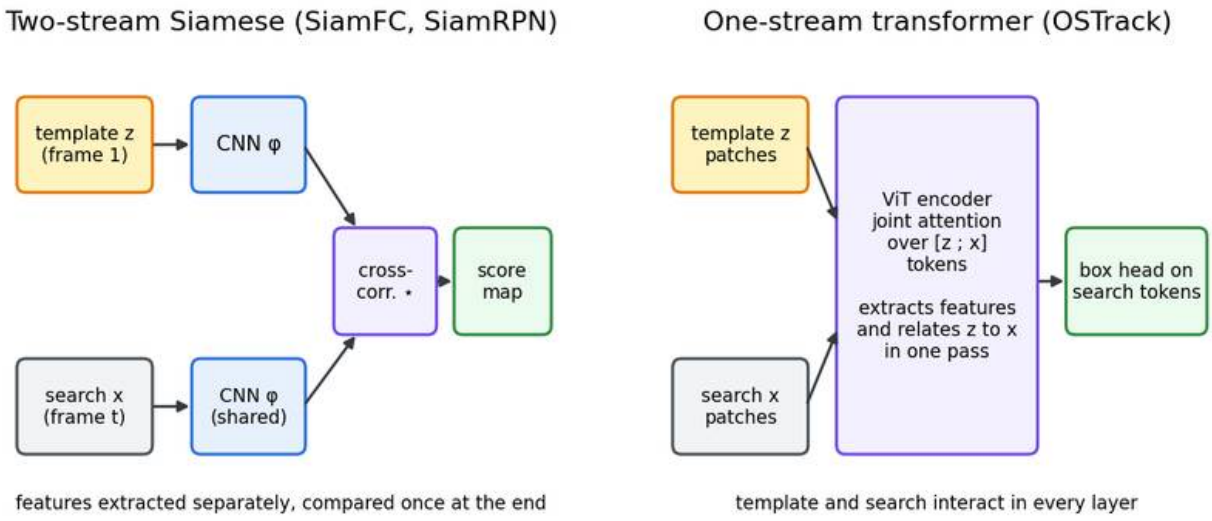


圖 2 — 雙流孿生追蹤器只在最後比較一次模板與搜尋特徵；單流 *Transformer* 追蹤器讓兩者在每一層都互動。

用影片分割模型做追蹤

白話版。新一代「影片中分割一切」的模型，只要你點一下說「就是這個」，就能用像素遮罩在整段影片中跟住它。這**就是**單物件追蹤，只是輸出遮罩而不是框。

SAM 2 [23] 把 Segment Anything 模型延伸到影片，加入**串流記憶**（streaming memory）：過去影格的特徵與預測遮罩，連同被提示的影格，都存在記憶庫中，供目前影格做注意力。只要在一格中用點、框或遮罩提示，就能得到貫穿整段影片的「masklet」，行為就像輸出遮罩的 SOT 追蹤器。SAMURAI [24] 不需重新訓練就把 SAM 2 改造成追蹤器：它加入卡爾曼濾波器運動模型，從 SAM 2 的多個候選遮罩中挑選，並用考慮運動的規則決定哪些過去影格進入記憶，這在有相似干擾物的擁擠場景特別有幫助。SAM 3 [25] 加入**可提示概念分割**（promptable concept segmentation）：給一個名詞片語，例如「黃色校車」，或一張範例影像，它就能在影片中偵測、分割並追蹤**所有**符合的實例，並給每個實例各自的身分。這已經是一種開放詞彙 MOT，SOT 與 MOT 的界線也因此更加模糊。

基準資料集與指標

基準	特色
OTB [26]	早期的標準基準，每段序列都有屬性標記（遮擋、快速運動等）；提出 success plot 與 precision plot
VOT challenge [53]	自 2013 年起每年舉辦的競賽，採用失敗後重新初始化的評估協定；現為 VOTS，同時評估追蹤**與**分割
LaSOT [27]	長時追蹤：1,400 段序列、超過 350 萬格，每一格都由人工標註
GOT-10k [28]	one-shot 協定：訓練與測試的物件類別互不重疊
TrackingNet [29]	大規模、真實場景影片，以獨立的評估伺服器評分

兩個經典指標來自 OTB [26]：

$$\text{Success}(\tau) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}[\,\text{IoU}(\hat{b}_t, b_t) > \tau], \qquad \text{AUC} = \int_0^1 \text{Success}(\tau)\,d\tau,$$

$$\text{Precision}(\delta) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}[\|\hat{c}_t - c_t\|_2 \le \delta],$$

其中 $\hat{b}_t, b_t$ 是預測框與真實框，τ 是 IoU 門檻，$\hat{c}_t, c_t$ 是兩者的中心，δ 是像素距離，慣例取 20。success plot 的 AUC 是 OTB 與 LaSOT 的主要數字。GOT-10k 則報告**平均重疊**（average overlap，即平均 IoU），以及 $\tau = 0.5$ 與 0.75 時的成功率。

```
import numpy as np

def iou_xywh(a, b):
    """IoU between rows of a and b, boxes as [x, y, w, h] (top-left corner)."""
    x1 = np.maximum(a[:, 0], b[:, 0]); y1 = np.maximum(a[:, 1], b[:, 1])
    x2 = np.minimum(a[:, 0] + a[:, 2], b[:, 0] + b[:, 2])
```

```
    y2 = np.minimum(a[:, 1] + a[:, 3], b[:, 1] + b[:, 3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    return inter / (a[:, 2] * a[:, 3] + b[:, 2] * b[:, 3] - inter)

def success_auc(pred, gt, thresholds=np.linspace(0, 1, 21)):
    ious = iou_xywh(pred, gt)
    curve = np.array([(ious > t).mean() for t in thresholds])
    return curve.mean()                                  # area under the success plot

def precision_at(pred, gt, px=20):
    c_pred = pred[:, :2] + pred[:, 2:] / 2; c_gt = gt[:, :2] + gt[:, 2:] / 2
    return (np.linalg.norm(c_pred - c_gt, axis=1) <= px).mean()

rng = np.random.default_rng(0)
gt = np.c_[np.linspace(50, 400, 300), np.full(300, 80.0), np.full(300, 40.0), np.full(300,
 ↪  60.0)]
pred = gt + rng.normal(0, 4, gt.shape)
pred[200:230, :2] += 120                                 # 30 frames lost on a distractor
print("success AUC %.3f   precision@20px %.3f" % (success_auc(pred, gt), precision_at(pred,
 ↪  gt)))
```

輸出為 `success AUC 0.647   precision@20px 0.900`：跟到干擾物上的 30 格讓 precision 掉了 10 個百分點；而其他影格上的小幅抖動對 AUC 的影響大得多，因為 AUC 也會獎勵貼得很緊的框。

多物件追蹤

白話版。在每一格找出所有感興趣的物件，並給每個物件一個號碼，只要它還在畫面中，號碼就不變——即使物件彼此交錯、互相遮擋，或離開後又回來。

MOT 是在**連續影格**中同時追蹤多個目標，並為每個目標維持一致的 ID；單一畫格本身只能做偵測。[1] 列出的挑戰包括：頻繁遮擋、軌跡何時開始與結束、目標之間外觀相似，以及目標之間的相互影響（人群中的人會一起移動、彼此遮擋）。

形式上，給定每一格的偵測 $\mathcal{D}_t$，MOT 要找出一組軌跡 $\mathcal{T} = \{\tau_k\}$，把真實的偵測分組、丟掉誤報，並補上漏掉的偵測。離線（offline）方法對整段影片一起求解，常寫成圖或網路流問題；本節著重的**線上**（online）方法，必須只用第 $1..t$ 格在第 t 格就做出決定，這正是即時應用所需要的。

「先偵測再追蹤」流程

十年來的主流設計是**先偵測再追蹤**（tracking-by-detection）[2]、[4]：每一格都跑偵測器，再把偵測結果跨時間連起來（圖 3）。每一格經過四個步驟：

1. **偵測**。偵測器（Faster R-CNN、YOLO 系列、DETR 系列；見DLCV 03）輸出框與分數。
2. **預測**。運動模型把每條現有軌跡往前推到現在應該在的位置。
3. **描述**。（可選）用 Re-ID 網路把每個偵測框裁切出的影像轉成嵌入向量。

4. **關聯**。在預測的軌跡與偵測之間建立代價矩陣，解指派問題。匹配到的軌跡被更新；沒匹配到的偵測可能開啟新軌跡（**出生**）；太久沒匹配到的軌跡被刪除（**死亡**）。

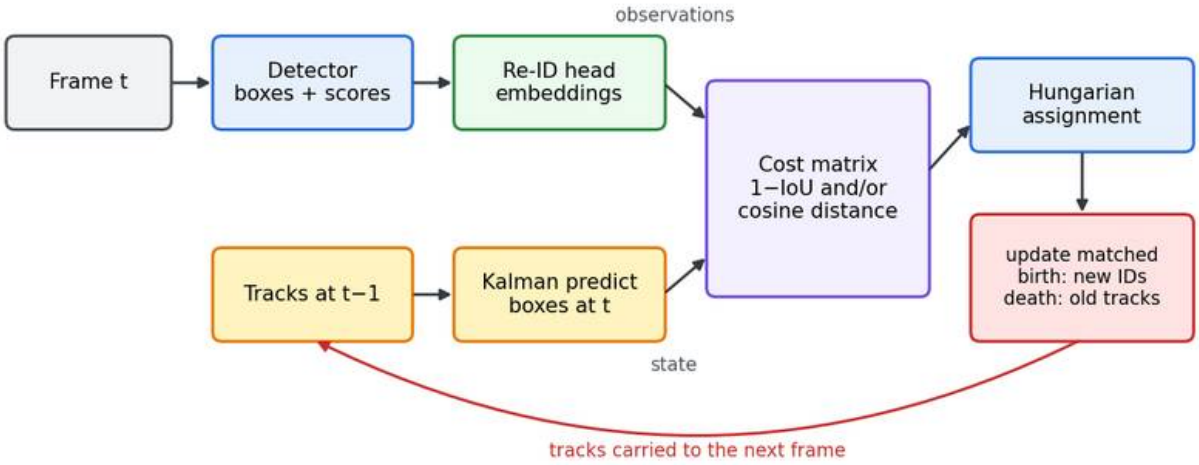


圖 3 — 線上的先偵測再追蹤流程。觀測（框、嵌入）與狀態（預測的軌跡）在代價矩陣中會合，指派結果再更新下一格要用的狀態。

運動模型：卡爾曼濾波器

白話版。先根據物體上一刻的位置與速度猜它現在在哪；再把這個猜測往偵測結果拉一點，而且比較不確定的那一方，就比較不被相信。

幾乎所有先偵測再追蹤的系統，都使用等速模型的**線性卡爾曼濾波器**（Kalman filter）[30]。在我們的程式中，一條軌跡的狀態是

$$\mathbf{x} = [\,c_x,\ c_y,\ w,\ h,\ v_x,\ v_y\,]^\top,$$

其中 (c_x, c_y) 是框的中心，w, h 是寬與高，(v_x, v_y) 是中心的速度（像素／格）。（SORT 使用中心、面積與長寬比，而不是 w, h [32]；BoT-SORT 又改回 w 與 h [36]。）模型分兩步。

預測（時間更新）：

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\,\hat{\mathbf{x}}_{t-1|t-1},$$
$$\mathbf{P}_{t|t-1} = \mathbf{F}\,\mathbf{P}_{t-1|t-1}\,\mathbf{F}^\top + \mathbf{Q},$$

其中 $\mathbf{F}$ 是 6×6 的狀態轉移矩陣（單位矩陣再加上幾個 1，把 v_x, v_y 加到 c_x, c_y 上，因為 $\Delta t = 1$ 格），$\mathbf{P}$ 是狀態共變異數（我們的不確定性），$\mathbf{Q}$ 是過程雜訊共變異數（真實運動可以偏離等速多少）。

更新（量測更新），使用匹配到的偵測 $\mathbf{z}_t = [c_x, c_y, w, h]^\top$：

$$\mathbf{S}_t = \mathbf{H}\,\mathbf{P}_{t|t-1}\,\mathbf{H}^\top + \mathbf{R}, \qquad \mathbf{K}_t = \mathbf{P}_{t|t-1}\,\mathbf{H}^\top\,\mathbf{S}_t^{-1},$$
$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t(\mathbf{z}_t - \mathbf{H}\,\hat{\mathbf{x}}_{t|t-1}), \qquad \mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\,\mathbf{P}_{t|t-1},$$

其中 $\mathbf{H} = [\,\mathbf{I}_4 \;\; \mathbf{0}\,]$ 取出狀態中可觀測的部分，$\mathbf{R}$ 是偵測器的量測雜訊共變異數，$\mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1}$ 是**創新量**（innovation，也就是「意外程度」），$\mathbf{S}_t$ 是它的共變異數，$\mathbf{K}_t$ 是**卡爾曼增益**。若預測非常不確定（$\mathbf{P}$ 很大），更新會幾乎把估計值拉到偵測結果上；若偵測器雜訊很大（$\mathbf{R}$ 很大），就比較相信預測。偵測缺漏時，追蹤器只做預測步驟（稱為「滑行」，coasting），$\mathbf{P}$ 會持續變大（圖 4）。

```
import numpy as np
from scipy.optimize import linear_sum_assignment

# ---------- 1. constant-velocity Kalman filter ----------
class KalmanBox:
    """State x = [cx, cy, w, h, vx, vy]; measurement z = [cx, cy, w, h]."""
    def __init__(self, z, q=1.0, r=4.0):
        self.x = np.r_[z, 0.0, 0.0]
        self.P = np.diag([r, r, r, r, 100.0, 100.0])    # unknown velocity: large variance
        self.F = np.eye(6); self.F[0, 4] = self.F[1, 5] = 1.0   # dt = 1 frame
        self.H = np.eye(4, 6)
        self.Q = q * np.diag([1, 1, 0.1, 0.1, 0.5, 0.5])
        self.R = r * np.eye(4)

    def predict(self):
        self.x = self.F @ self.x
        self.P = self.F @ self.P @ self.F.T + self.Q
        return self.x[:4]

    def update(self, z):
        S = self.H @ self.P @ self.H.T + self.R          # innovation covariance
        K = self.P @ self.H.T @ np.linalg.inv(S)          # Kalman gain
        self.x = self.x + K @ (z - self.H @ self.x)
        self.P = (np.eye(6) - K @ self.H) @ self.P
```

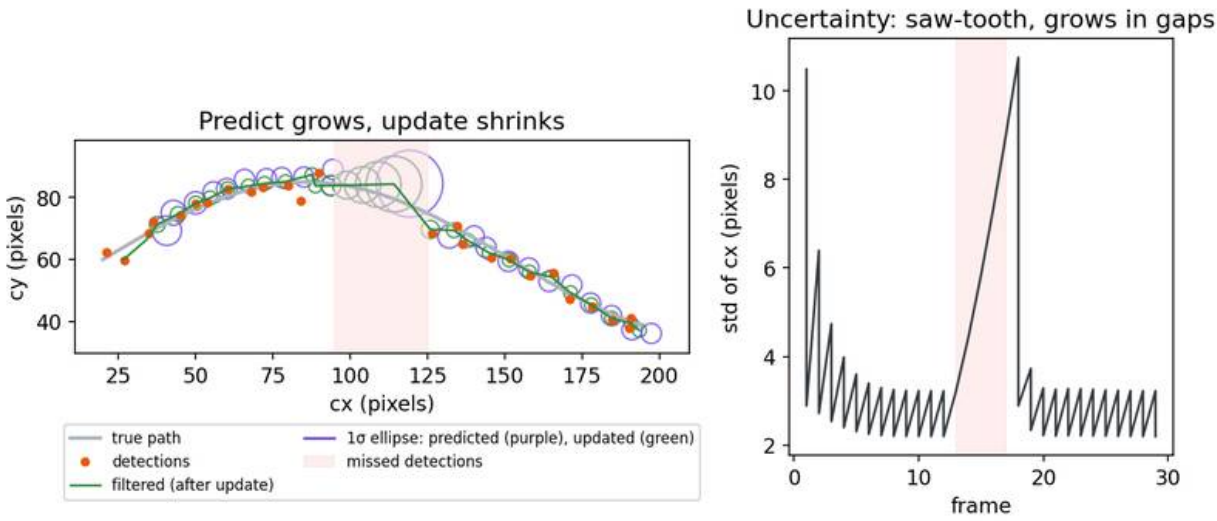


圖 4 — 在合成軌跡上的卡爾曼預測與更新。每次預測都讓不確定性膨脹，每次更新都讓它縮小。連續五格漏偵測時（粉紅色區域），濾波器沿直線滑行，不確定性持續上升——身分錯誤正是在這

種時候發生。

在模型假設下，創新量 $\boldsymbol{\nu}$ 的平方**馬氏距離**（Mahalanobis distance）$d_M^2 = \boldsymbol{\nu}^\top \mathbf{S}^{-1}\boldsymbol{\nu}$ 服從自由度 4 的 χ^2 分布，所以 DeepSORT [33] 用它來**閘控**（gating，直接禁止不合理的匹配）。等速假設也是最大的弱點：舞者、運動員，以及任何由移動攝影機拍攝的畫面，都會違反它。

外觀嵌入

Re-ID 網路（見上一節）把每個偵測裁切影像轉成單位向量 $\mathbf{e}_j$。每條軌跡保有外觀記憶，例如指數移動平均 $\mathbf{m}_k \leftarrow \alpha\,\mathbf{m}_k + (1-\alpha)\,\mathbf{e}_j$ 後再重新正規化，其中 $\alpha \in [0,1)$（我們用 0.9）。外觀代價就是餘弦距離 $1 - \mathbf{m}_k^\top \mathbf{e}_j$。在長時間遮擋後運動預測已經沒用時，正是靠外觀把身分找回來。

關聯：代價矩陣與匈牙利演算法

白話版。為每一組「軌跡 k 配上偵測 j」打一個「有多不合適」的分數，做成一張表，再選出一對一配對中總分最小的那一組。

有 K 條預測軌跡與 N 個偵測時，建立代價矩陣 $\mathbf{C} \in \mathbb{R}^{K\times N}$，例如

$$C_{kj} = \lambda\,(1 - \mathrm{IoU}(\hat{b}_k, b_j)) + (1-\lambda)\,(1 - \mathbf{m}_k^\top \mathbf{e}_j),$$

其中 $\hat{b}_k$ 是軌跡 k 的預測框，b_j 是偵測 j 的框，$\lambda \in [0,1]$ 是運動與外觀的權重（$\lambda = 1$ 就是 SORT 的純 IoU）。接著解**線性指派問題**

$$\min_{\mathbf{X}\in\{0,1\}^{K\times N}} \sum_{k,j} C_{kj} X_{kj} \quad \text{s.t.} \quad \sum_j X_{kj} \le 1,\ \sum_k X_{kj} \le 1,$$

讓每條軌跡最多拿一個偵測，反之亦然。**匈牙利演算法**（Hungarian algorithm）[31] 能在多項式時間內精確求解（$n = \max(K,N)$ 時為 $O(n^3)$）；SciPy 的 `linear_sum_assignment` 實作了一個快速的變形。最後，代價超過門檻的配對會被拒絕，實務上的閘控就是這樣做的。

```
# ---------- 2. IoU and Hungarian association ----------
def iou_matrix(a, b):
    """a: (N,4), b: (M,4) boxes as [cx, cy, w, h] -> (N,M) IoU."""
    a1, a2 = a[:, None, :2] - a[:, None, 2:] / 2, a[:, None, :2] + a[:, None, 2:] / 2
    b1, b2 = b[None, :, :2] - b[None, :, 2:] / 2, b[None, :, :2] + b[None, :, 2:] / 2
    wh = np.clip(np.minimum(a2, b2) - np.maximum(a1, b1), 0, None)
    inter = wh[..., 0] * wh[..., 1]
    area = lambda x: x[..., 2] * x[..., 3]
    return inter / (area(a)[:, None] + area(b)[None, :] - inter + 1e-9)

def associate(cost, max_cost):
    """Hungarian assignment; pairs whose cost exceeds max_cost are rejected."""
    if cost.size == 0:
        return [], list(range(cost.shape[0])), list(range(cost.shape[1]))
    rows, cols = linear_sum_assignment(cost)
    pairs = [(r, c) for r, c in zip(rows, cols) if cost[r, c] <= max_cost]
    mr = {r for r, _ in pairs}; mc = {c for _, c in pairs}
```

```
    return (pairs, [r for r in range(cost.shape[0]) if r not in mr],
            [c for c in range(cost.shape[1]) if c not in mc])
```

在合成場景上的完整追蹤器

為了看清楚每個想法帶來什麼，我們產生一個 80 格、5 位行人的場景，刻意放入幾種經典的失敗情境：

- 物件 0 與 1 **交錯而過**，重疊時位在後方的物件 1 只得到**低信心**偵測；
- 物件 2 與 3 相遇後**反彈回去**（非線性運動，像舞者），重疊時後方那位同樣只有低信心偵測；
- 物件 4 走到**柱子後方** 8 格，這段期間偵測都是低信心；
- 每個偵測都有 5% 機率漏掉，另外偶爾會出現低分的誤報。

每個偵測也帶有一個 16 維外觀向量：所屬物件固定的單位向量加上雜訊。

```
# ---------- 3. synthetic scene ----------
def make_scene(T=80, seed=0):
    rng = np.random.default_rng(seed)
    W, H = 30.0, 70.0
    t = np.arange(T, dtype=float)
    tr = {}
    # pair 1: plain crossing, B is behind A (occluded while they overlap)
    tr[0] = np.c_[80 + 5 * t, np.full(T, 120.0)]
    tr[1] = np.c_[480 - 5 * t, np.full(T, 128.0)]
    # pair 2: they meet and bounce back (non-linear motion, like dancers)
    m = 40
    x2 = np.where(t < m, 120 + 4 * t, 120 + 4 * m - 4 * (t - m))
    x3 = np.where(t < m, 440 - 4 * t, 440 - 4 * m + 4 * (t - m))
    tr[2] = np.c_[x2 + 10, np.full(T, 260.0)]
    tr[3] = np.c_[x3 - 10, np.full(T, 266.0)]
    # walker passing behind a pole (frames 30-37): detections become low-score
    tr[4] = np.c_[60 + 3 * t, 40 + 1.0 * t]
    gt = []                                     # per frame list of (id, box)
    dets = []                                   # per frame list of (box, score, gt_id)
    feats = {k: (lambda v: v / np.linalg.norm(v))(rng.normal(size=16)) for k in tr}
    for f in range(T):
        g, d = [], []
        boxes = {k: np.r_[tr[k][f], W, H] for k in tr}
        for k, b in boxes.items():
            g.append((k, b))
            score = rng.uniform(0.6, 0.95)
            if k == 1 and iou_matrix(b[None], boxes[0][None])[0, 0] > 0.2:
                score = rng.uniform(0.15, 0.45)       # occluded by object 0
            if k == 3 and iou_matrix(b[None], boxes[2][None])[0, 0] > 0.2:
                score = rng.uniform(0.15, 0.45)       # occluded by object 2
            if k == 4 and 30 <= f <= 37:
                score = rng.uniform(0.15, 0.45)       # behind the pole
            if rng.random() < 0.05:
                continue                               # random missed detection
            noisy = b + np.r_[rng.normal(0, 2.0, 2), rng.normal(0, 1.0, 2)]
```

```
            e = feats[k] + 0.15 * rng.normal(size=16)
            d.append((noisy, score, k, e / np.linalg.norm(e)))
        if rng.random() < 0.3:                          # low-score false positive
            fp = np.r_[rng.uniform(50, 550), rng.uniform(50, 320), W, H]
            e = rng.normal(size=16)
            d.append((fp, rng.uniform(0.1, 0.4), -1, e / np.linalg.norm(e)))
        gt.append(g); dets.append(d)
    return gt, dets
```

下面的追蹤器有三個開關。全部關閉時就是 **SORT 風格**：卡爾曼預測、IoU 代價、只對高分偵測（分數 ≥ 0.5）做匈牙利匹配，連續漏 2 格就刪除軌跡。`use_low=True` 加入 **BYTE** 風格的第二輪：沒配到的軌跡再只用 IoU 去配低分偵測。`use_app=True` 在第一輪加入**外觀**代價，並加上第三輪：純靠外觀，把遺失 30 格以內的軌跡重新識別回來。

```
# ---------- 4. SORT-style tracker with optional BYTE and appearance ----------
class Track:
    def __init__(self, tid, z, emb):
        self.id, self.kf, self.emb = tid, KalmanBox(z), emb
        self.lost = 0

def run_tracker(dets, use_low=False, use_app=False, hi=0.5, max_age=2,
                app_age=30, lam=0.5):
    tracks, out, next_id = [], [], 0
    for d in dets:
        for tk in tracks:
            tk.kf.predict()
        boxes = np.array([x[0] for x in d]).reshape(-1, 4)
        scores = np.array([x[1] for x in d])
        embs = np.array([x[3] for x in d]).reshape(-1, 16)
        high = np.where(scores >= hi)[0]
        low = np.where((scores < hi) & (scores >= 0.1))[0]
        active = [tk for tk in tracks if tk.lost <= max_age]
        pred = np.array([tk.kf.x[:4] for tk in active]).reshape(-1, 4)
        # stage 1: high-score detections vs active tracks
        cost = 1 - iou_matrix(pred, boxes[high])
        gate = 0.7
        if use_app and len(active) and len(high):
            tE = np.array([tk.emb for tk in active])
            app = 1 - tE @ embs[high].T            # cosine distance
            cost = np.where(cost > 0.9, 9.0, lam * cost + (1 - lam) * app)
            gate = 0.6
        pairs, ut, ud = associate(cost, gate)
        matched = [(active[r], high[c]) for r, c in pairs]
        rest_tracks = [active[r] for r in ut]
        rest_high = [high[c] for c in ud]
        # stage 2 (BYTE): remaining tracks vs low-score detections, IoU only
        if use_low and rest_tracks and len(low):
            p2 = np.array([tk.kf.x[:4] for tk in rest_tracks])
            pairs2, ut2, _ = associate(1 - iou_matrix(p2, boxes[low]), 0.5)
            matched += [(rest_tracks[r], low[c]) for r, c in pairs2]
            rest_tracks = [rest_tracks[r] for r in ut2]
        # stage 3 (appearance only): long-lost tracks vs leftover high-score dets
```

```
        if use_app and rest_high:
            lost = [tk for tk in tracks if max_age < tk.lost <= app_age]
            if lost:
                app = 1 - np.array([tk.emb for tk in lost]) @ embs[rest_high].T
                pairs3, _, ud3 = associate(app, 0.3)
                matched += [(lost[r], rest_high[c]) for r, c in pairs3]
                rest_high = [rest_high[c] for c in ud3]
        for tk, j in matched:
            tk.kf.update(boxes[j]); tk.lost = 0
            tk.emb = 0.9 * tk.emb + 0.1 * embs[j]; tk.emb /= np.linalg.norm(tk.emb)
        hit = {id(tk) for tk, _ in matched}
        for tk in tracks:
            if id(tk) not in hit:
                tk.lost += 1
        for j in rest_high:                            # birth: unmatched high-score
            tracks.append(Track(next_id, boxes[j], embs[j])); next_id += 1
        tracks = [tk for tk in tracks if tk.lost <= (app_age if use_app else max_age)]
        out.append([(tk.id, tk.kf.x[:4].copy()) for tk in tracks if tk.lost == 0])
    return out
```

指標程式（MOTA、IDF1、HOTA，下文說明）與實驗：

```
# ---------- 5. metrics: MOTA, IDF1, HOTA ----------
def frame_match(g, p, thr):
    if not g or not p:
        return []
    iou = iou_matrix(np.array([b for _, b in g]), np.array([b for _, b in p]))
    r, c = linear_sum_assignment(-iou)
    return [(g[i][0], p[j][0]) for i, j in zip(r, c) if iou[i, j] >= thr]

def mota(gt, hyp, thr=0.5):
    fn = fp = idsw = 0; ngt = 0; last = {}
    for g, p in zip(gt, hyp):
        m = frame_match(g, p, thr)
        ngt += len(g); fn += len(g) - len(m); fp += len(p) - len(m)
        for gi, pi in m:
            if gi in last and last[gi] != pi:
                idsw += 1
            last[gi] = pi
    return 1 - (fn + fp + idsw) / ngt, idsw

def idf1(gt, hyp, thr=0.5):
    gids = sorted({i for g in gt for i, _ in g}); pids = sorted({i for p in hyp for i, _ in
    ↪  p})
    gi = {k: n for n, k in enumerate(gids)}; pi = {k: n for n, k in enumerate(pids)}
    ov = np.zeros((len(gids), len(pids)))
    for g, p in zip(gt, hyp):
        if g and p:
            iou = iou_matrix(np.array([b for _, b in g]), np.array([b for _, b in p]))
            for a in range(len(g)):
                for b in range(len(p)):
                    ov[gi[g[a][0]], pi[p[b][0]]] += iou[a, b] >= thr
```

```
    r, c = linear_sum_assignment(-ov)
    idtp = ov[r, c].sum()
    n_gt = sum(len(g) for g in gt); n_p = sum(len(p) for p in hyp)
    return 2 * idtp / (n_gt + n_p)

def hota(gt, hyp, alphas=np.arange(0.05, 0.96, 0.05)):
    res = []
    for a in alphas:
        pairs = []; n_gt = n_p = 0
        for g, p in zip(gt, hyp):
            pairs += frame_match(g, p, a); n_gt += len(g); n_p += len(p)
        tp = len(pairs)
        if tp == 0:
            res.append((0, 0, 0)); continue
        cnt = {}
        for q in pairs:
            cnt[q] = cnt.get(q, 0) + 1
        gcount = {}; pcount = {}
        for g in gt:
            for i, _ in g: gcount[i] = gcount.get(i, 0) + 1
        for p in hyp:
            for i, _ in p: pcount[i] = pcount.get(i, 0) + 1
        ass = sum(cnt[q] / (gcount[q[0]] + pcount[q[1]] - cnt[q]) for q in pairs) / tp
        det = tp / (n_gt + n_p - tp)
        res.append((np.sqrt(det * ass), det, ass))
    return np.array(res).mean(axis=0)   # HOTA, DetA, AssA averaged over alpha

gt, dets = make_scene(seed=3)
for name, kw in [("IoU only", {}), ("+ low-score pass", dict(use_low=True)),
                 ("+ appearance", dict(use_low=True, use_app=True))]:
    hyp = run_tracker(dets, **kw)
    m, sw = mota(gt, hyp)
    print(f"{name:18s} MOTA {m:.2f}  IDF1 {idf1(gt, hyp):.2f}  "
          f"HOTA {hota(gt, hyp)[0]:.2f}  ID switches {sw}")
```

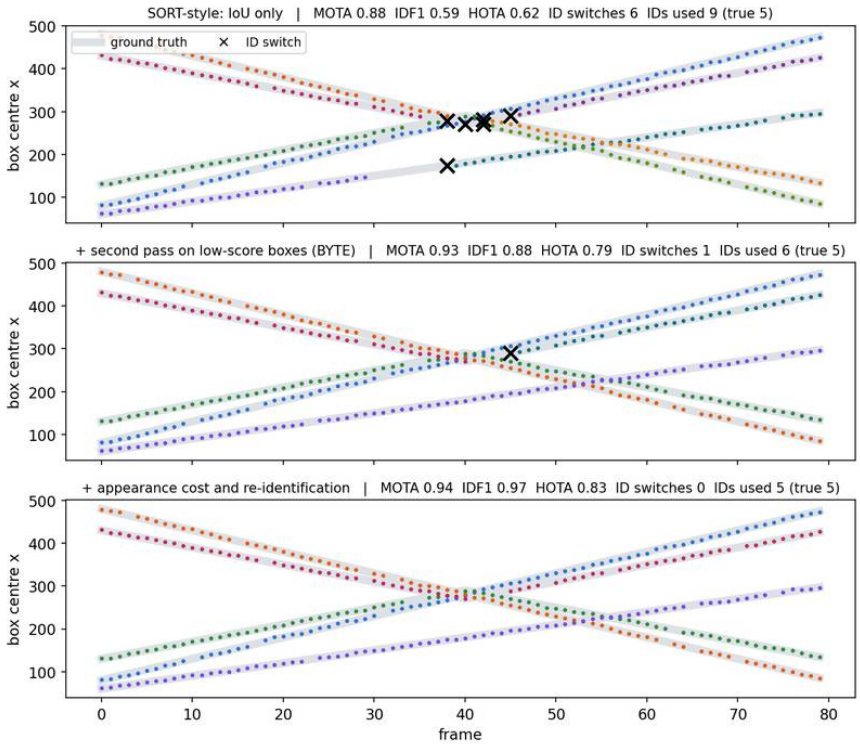


圖 *5* — 同一組合成偵測，用三種方式追蹤。顏色代表預測 *ID*，黑色叉號標示 *ID* 切換。低分關聯消除了遮擋造成的斷裂；外觀則修正了反彈情境，因為等速預測此時指向錯誤的方向。

程式輸出：

追蹤器	MOTA	IDF1	HOTA	ID 切換	使用的 ID 數（真實：5）
SORT 風格，只用 IoU	0.88	0.59	0.62	6	9
+ BYTE 低分偵測第二輪	0.93	0.88	0.79	1	6
+ 外觀與重新識別	0.94	0.97	0.83	0	5

仔細看這張表。只用 IoU 的追蹤器會弄丟被遮擋的物件，因為它們的偵測分數低於門檻，軌跡死掉，再出現時就換了**新 ID**（軌跡斷裂，fragmentation）。低分偵測的第二輪讓這些軌跡活下來，幾乎修好了所有問題，但修不了反彈：卡爾曼濾波器預測兩位舞者會繼續往前走，所以重疊之後其中一位沒被配到，稍後以新 ID 重新出現。只有外觀能解決這個情況。也請注意 MOTA 的變化（0.88 → 0.94）比 IDF1（0.59 → 0.97）小得多：MOTA 主要是在衡量偵測，我們會在指標一節再回來談。

SORT 家族

這些想法正好對應到最常被引用的線上追蹤器：

追蹤器	核心想法
SORT [32]	卡爾曼濾波器 + IoU 代價 + 匈牙利指派；刻意極簡、速度非常快。證明了偵測器的品質主導追蹤品質。
DeepSORT [33]	加入在行人 Re-ID 資料集上訓練的 CNN 外觀描述子、馬氏距離閘控，以及優先處理最近才看到之軌跡的**匹配級聯**（matching cascade）。能撐過更長的遮擋，ID 切換更少。
ByteTrack [34]	關聯**每一個**偵測框：先配高分框，再把剩下的軌跡只用 IoU 去配**低分**框。被遮擋的物件常常分數偏低，因此能被救回而不是被丟掉；而低分的背景框配不到軌跡，就被捨棄。
OC-SORT [35]	「以觀測為中心」：遺失的軌跡被重新找到時，沿著上一次與這一次觀測之間的虛擬軌跡重跑卡爾曼更新（修正滑行期間累積的誤差），並在代價中加入運動方向一致性項。在 DanceTrack 這類非線性運動上特別強。
BoT-SORT [36]	ByteTrack 式的關聯，再加上攝影機運動補償（全域影像對齊）、以 w, h 取代長寬比的卡爾曼狀態，以及融合 IoU 與 Re-ID 的代價。

這個家族的教訓是：在強偵測器之上**精心設計的關聯規則**，在行人基準上非常難被超越。Adžemović 2025 年的綜述 [4] 也得到同樣的結論：在密集、運動大致線性的場景，啟發式方法領先；運動複雜時，學習式關聯表現較好。

聯合偵測與嵌入，以及以偵測器追蹤

對每個裁切框都另外跑一次 Re-ID 網路很昂貴。**JDE** [37] 在單階段偵測器上加一個嵌入頭，一次前向傳遞就同時輸出框與外觀向量。**FairMOT** [38] 指出這種共享對 Re-ID 並「不公平」：以錨框為基礎的偵測器產生的特徵對身分而言很模糊，而且訓練被偵測任務主導。它改用無錨框、CenterNet 風格的偵測器，在高解析度特徵圖上放兩個對等的分支（偵測與 Re-ID）。原文也提到的 **Tracktor** [39]，則在大多數情況下省去顯式關聯：把上一格的框送進偵測器的框迴歸頭，讓每個框被拉到物件的新位置，並保留原本的身分。

以 query 追蹤：端到端 MOT

白話版。不再手寫「預測、比較、配對」，而是給 Transformer 每個物件一個記憶格，讓它自己學會每一格怎麼更新這個記憶格。

DETR 風格的偵測器（見DLCV 03）用 **query** 表示物件。MOTR [40] 把這延伸到影片：每個偵測到的物件的輸出嵌入，變成一個**軌跡 query**（track query）傳到下一格，持續預測同一個物件；新的**偵測 query** 則負責找出新出現的物件。身分由 query 隱式攜帶，推論時沒有卡爾曼濾波器，也沒有匈牙利匹配。訓練時使用考慮軌跡的標籤指派（tracklet-aware label assignment）與在片段上平均的損失。它的弱點是偵測品質：同一個解碼器既要偵測又要關聯，兩個任務互相衝突。MOTRv2 [41] 以另一個預訓練偵測器（YOLOX）的提議框作為錨點來引導 MOTR，緩解了這個衝突。MOTIP [42]（CVPR 2025）把關聯重新定義為 **ID 預測**：給定到目前為止的軌跡，每條軌跡帶有一個從可學習 ID 字典中取出的 ID 標籤，解碼器把每個新偵測分類成既有的某個 ID 或「新物件」。這是 in-context、可端到端訓練的，而且只需要物件層級的特徵。

開放詞彙 MOT

傳統 MOT 只能追蹤訓練集中出現過的類別。**OVTrack** [43]（CVPR 2023）透過從視覺語言模型（CLIP）蒸餾知識來做分類與關聯，並用擴散模型**幻想生成**（hallucinate）的影像對來學習穩健的外觀特徵，藉此追蹤任意類別。SAM 3 的概念提示 [25] 則從分割這一側朝同一個目標前進。

資料集

資料集	領域	考驗什麼
MOT17 [44]	行人，固定與移動攝影機	標準行人基準；以 MOT16 的序列重新發布為 MOT17
MOT20 [45]	非常擁擠的行人場景	嚴重遮擋、小目標
DanceTrack [46]	群舞	外觀一致、運動多變且非線性；無法靠外觀區分人
SportsMOT [47]	籃球、排球、足球（240 段序列）	快速且速度多變的運動，外觀相似但仍可區分
BDD100K MOT [48]	行車影片	從移動車輛上做多類別追蹤（汽車、行人、自行車騎士等）

從 MOT17 轉向 DanceTrack 與 SportsMOT，改變了這個領域的重點：在 MOT17 上，好的偵測器加上 IoU 匹配就能走很遠；但在 DanceTrack 上，運動模型與學習式關聯才是關鍵。

指標：MOTA、IDF1、HOTA

白話版。追蹤器有兩種失敗方式：漏掉物件或憑空多出物件（偵測錯誤），或是把誰是誰搞混（關聯錯誤）。不同指標對這兩者的權重不同。

三個指標都先在每一格、以某個 IoU 門檻把預測與真實值配對。令 GT_t、FN_t、FP_t 與 IDSW_t 分別是第 t 格的真實物件數、漏偵測數、誤報數與身分切換數。所謂身分切換，是指某個真實物件這次配到的預測 ID，與它上一次配到的不同。

MOTA（CLEAR MOT [9]）：

$$\mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t)}{\sum_t \mathrm{GT}_t}.$$

它的範圍是 $-\infty$ 到 1。由於偵測數量通常遠多於切換次數，FN 與 FP 主導了 MOTA：它主要是偵測分數。而且切換只在發生的那一刻被計算一次，所以一個把兩個 ID 換掉後維持 1,000 格的追蹤器，與一個 1 格後就換回來的追蹤器，付出的代價相同。

IDF1 [10] 在真實軌跡與預測軌跡之間做**全域**的一對一匹配（同樣用匈牙利演算法），最大化兩者一致的影格數，然後

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}},$$

其中 IDTP 計算那些配到「與其真實身分對應之預測軌跡」的偵測，IDFP、IDFN 則是其餘的預測與真實偵測。IDF1 衡量追蹤器能維持正確身分多久，所以主要是關聯分數——但因為匹配是全域的，它的行為有時並不直觀，例如偵測變好時 IDF1 反而下降。

HOTA（Higher Order Tracking Accuracy [8]）的提出，正是因為上述兩個指標既不平衡、也無法拆解。對一個定位門檻 α，以每格配對得到真陽性（TP）、FN 與 FP：

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}|}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|}, \qquad \mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}|}\sum_{c \in \mathrm{TP}} \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|},$$

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}, \qquad \mathrm{HOTA} = \frac{1}{19}\sum_{\alpha \in \{0.05, 0.10, \ldots, 0.95\}} \mathrm{HOTA}_\alpha.$$

對一個配對 c（真實 ID g、預測 ID p），$\mathrm{TPA}(c)$ 是整段影片中同樣把 g 配到 p 的所有 TP；$\mathrm{FNA}(c)$ 是 g 被配到其他 ID 或漏掉的偵測；$\mathrm{FPA}(c)$ 是 p 被配到其他物件或沒配到任何物件的偵測。幾何平均代表追蹤器必須在偵測與關聯**兩方面**都好才能得高分；DetA 與 AssA（以及定位準確度 LocA）也可以分開報告，用來診斷問題。對 α 取平均也會獎勵精確的框。（我們的程式用的是簡化版：每格最大化 IoU 的匹配；正式定義使用的匹配分數還會偏好過去已經關聯過的配對。）

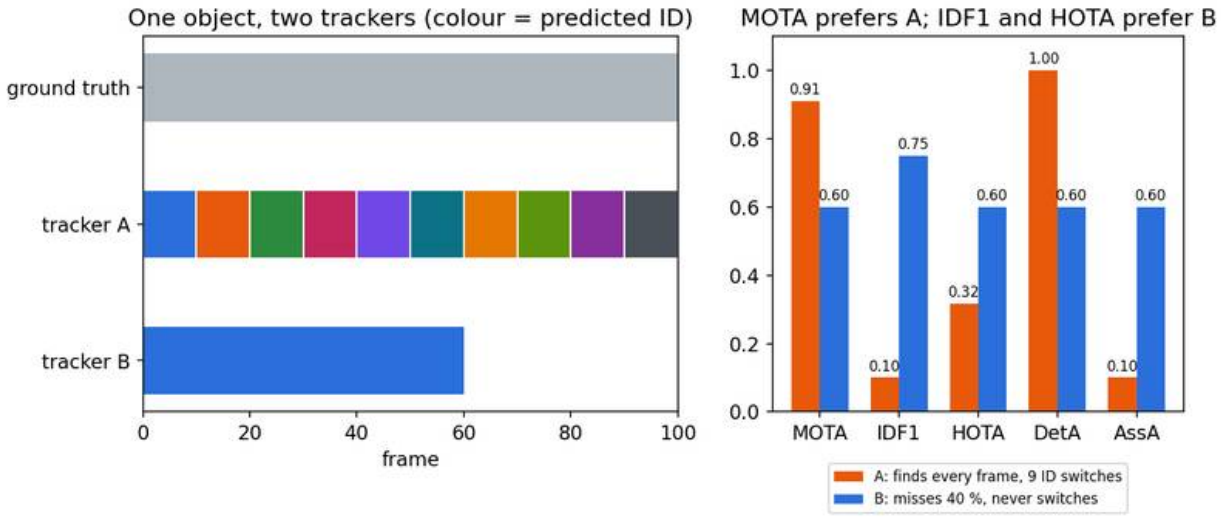


圖 *6 — HOTA* 為什麼被提出。追蹤器 *A* 每格都偵測到，但換了九次身分；追蹤器 *B* 漏掉 *40%* 的影格，但從不換身分。*MOTA* 認為 *A* 遠遠領先；*IDF1* 與 *HOTA* 認為 *B* 較好，而 *DetA* ／ *AssA* 正好說明原因。

圖 6 是用上面的函式算出來的：追蹤器 A 的 MOTA $= 1 - 9/100 = 0.91$，AssA $= 0.10$，所以 HOTA $= \sqrt{1.0 \times 0.10} \approx 0.32$；追蹤器 B 的 MOTA、DetA、AssA 與 HOTA 都是 0.60，IDF1 $= 2 \cdot 60/(100 + 60) = 0.75$。如今 MOT17、MOT20、DanceTrack 與 SportsMOT 上的論文會同時報告 HOTA、IDF1 與 MOTA，而 HOTA 通常是主要的排名指標。

點追蹤：相關的新趨勢

白話版。不追整個物件，而是追單一個**點**——臉頰上的一顆雀斑、箱子的一個角——貫穿整段影片，即使它短暫被擋住也一樣。

追蹤任意點（tracking any point, TAP）的問題是：給定第 t_0 格的查詢點 (x, y)，求它在其他每一格的位置與是否可見。它把光流（兩格之間）推廣到長時間、能處理遮擋的軌跡，也是IP 12傳統關鍵點追蹤的自然延伸。TAPIR [49] 先在每一格獨立比對查詢點的特徵得到粗略軌跡，再用時間卷積加以精修。CoTracker [50] 用 Transformer **聯合**追蹤許多點，讓點與點互相幫忙：被遮擋的點可以從還看得見的鄰近點推斷出來。CoTracker3 [51] 簡化了架構，並改用由既有追蹤器產生偽標籤的真實影片來訓練，而不只靠合成資料。點追蹤器現在被用作物件追蹤器中的運動線索，也用在影片編輯與機器人上。

多目標跨攝影機追蹤（MTMCT）

白話版。很多台攝影機看著同一個地方。先在每台攝影機裡追蹤每個人，再判斷離開 3 號攝影機的那位女士，就是走進 5 號攝影機的同一位。

MTMCT 就是 [1] 所說的終極目標。典型的流程是：先在每台攝影機內做 MOT 得到**短軌跡**（tracklet），用 Re-ID 特徵描述每條短軌跡，再利用外觀、攝影機拓樸（哪些攝影機相連、彼此之間的移動時間），以及在已校正攝影機下的地面 3D 位置，把不同攝影機的短軌跡分群。IDF1 原本就是為這個情境提出的 [10]。**AI City Challenge** 是主要的常態性基準。第九屆（2025）[52] 的 Track 1 是倉儲型環境中的多類別 **3D** 跨攝影機追蹤，類別包括人、人形機器人、自主移動機器人與堆高機，並提供攝影機校正資料與 3D 框標註。從影像上的框走向多視角、已校正的 3D 軌跡，正是 MTMCT 前進的方向。

現代觀點

綜述論文怎麼說

- **Luo 等人〈Multiple object tracking: A literature review〉[3]**（*Artificial Intelligence*，2021；2014 年首次公開）。經典參考文獻。它形式化定義 MOT 問題，依初始化方式（以偵測為基礎或不需偵測）、處理模式（線上或離線）與輸出型態（確定性或機率性）分類方法，並分析外觀模型、運動模型、互動模型、互斥限制與遮擋處理等元件，以及各種推論方法。想不依賴任何網路去理解問題本身，就讀這篇。
- **Ciaparrone 等人〈Deep learning in video multi-object tracking: A survey〉[2]**（*Neurocomputing*，2020）。把深度 MOT 整理成四個階段——偵測、特徵擷取／運動預測、親和度計算與關聯——並回顧深度學習如何進入每個階段。從 MOTChallenge 上的比較得到的主要結論是：大部分進步來自偵測品質與學習式外觀特徵。
- **Adžemović〈Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art〉[4]**（arXiv，2025）。把先偵測再追蹤分成聯合偵測與嵌入、啟發式、以運動為主、親和度學習與離線方法五類，並與端到端追蹤器比較。它指出 2022 年（ByteTrack 與 MOTR）是加速的轉捩點，並總結：啟發式追蹤器在擁擠、運動大致線性的場景領先，

學習式關聯則在複雜運動上勝出。

- **Marvasti-Zadeh 等人〈Deep learning for visual tracking: A comprehensive survey〉[5]**（*IEEE T-ITS*，2022）。針對 SOT。從九個面向（網路架構、網路運用、訓練方式、目標函數、輸出、與相關濾波器的結合、空拍視角、長時追蹤與線上追蹤）分析深度追蹤器，並在 OTB、VOT、LaSOT 與空拍基準上比較。
- **Thangavel（Kugarajeevan）等人〈Transformers in single object tracking: An experimental survey〉[6]**（*IEEE Access*，2023）。把 Transformer 追蹤器分成 CNN-Transformer 混合、雙流兩階段，以及單流單階段全 Transformer 三類，並實驗評估其穩健性與效率；單流設計（OSTrack 類）成為主流模式。
- **Ye 等人〈Deep learning for person re-identification: A survey and outlook〉[7]**（*IEEE TPAMI*，2022）。把封閉世界 Re-ID（特徵學習、度量學習、排序最佳化）與開放世界 Re-ID（異質資料、原始影像、雜訊標籤、非監督與開放集合設定）分開討論，提出 AGW 基線與 mINP 指標，後者衡量找到**所有**正確匹配所需的代價。
- **Luiten 等人〈HOTA〉[8]**（*IJCV*，2020）。不是綜述，但它重新檢視了整個領域的評估方式，以實例與使用者研究說明 MOTA 與 IDF1 的偏差，也是目前排行榜以 HOTA 排名的原因。

2024–2026 的技術現況

- **SOT** 由 OSTrack ／ SeqTrack 一脈的單流 Transformer 追蹤器主導，而分割基礎模型的角色越來越重：加上運動感知記憶的 SAM 2（SAMURAI），不需針對追蹤訓練就有很強的 zero-shot 追蹤能力 [23]、[24]。
- **MOT** 兩個陣營並存。啟發式的先偵測再追蹤（ByteTrack、OC-SORT、BoT-SORT 及其後繼者）仍是實務上的預設選擇：簡單、快速、模組化，大部分工作由強大的偵測器完成。端到端追蹤器（MOTR、MOTRv2、MOTIP）則在運動複雜、外觀模糊的情境領先，例如 DanceTrack 與 SportsMOT [4]、[42]。
- **開放詞彙與可提示追蹤**正進入主流模型：OVTrack 建立了這個任務，SAM 3 則能依文字概念偵測、分割並追蹤帶有身分的實例 [25]、[43]。
- **點追蹤**（TAPIR、CoTracker、CoTracker3）已成熟為通用的運動基本元件 [49]–[51]。
- **MTMCT** 正走向已校正、多類別的 3D 追蹤（AI City Challenge 2025）[52]。

尚待解決的問題

- **長時間遮擋與重新進場**。幾秒之後運動模型就沒用了，只能靠外觀維持身分；但在服裝一致（DanceTrack、運動比賽）或跨攝影機時，外觀並不可靠。
- **非線性運動與攝影機運動**。等速卡爾曼濾波器仍是預設，也是舞蹈、運動與行車資料上的弱點；學習式運動模型還沒有在所有情境都明顯更好。
- **端到端與啟發式關聯之爭**。端到端追蹤器優雅、在複雜運動上勝出，但需要大量影片訓練資料與計算，在擁擠行人場景上仍輸給簡單的啟發式方法。如何設計出兩邊都最好的統一架構，仍是開放問題。
- **開放詞彙與長尾追蹤**。稀有類別，以及能推廣到訓練中從未見過之類別的外觀特徵。
- **評估**。HOTA 解決了很多問題，但基準仍以行人為主，而跨數小時影片與多台攝影機的身分層級評估依然困難。**隱私與資料集倫理**（DukeMTMC 的教訓）也限制了能蒐集哪些資料。

重點整理

- 追蹤 = 偵測 + 時間：每個追蹤器都要同時跟住運動與維持身分，兩者的比重定義了 SOT、MOT 與 Re-ID。
- Re-ID 是以學習嵌入進行的檢索；請同時報告 CMC rank-k 與 mAP；DukeMTMC 已於 2019 年撤下，不應再使用。
- SOT 從相關濾波器（快速、人工特徵）走到孿生網路比對（離線訓練），再到單流 Transformer；SAM 2 類的影片分割如今能以遮罩做 SOT，SAM 3 更能依文字提示擴展到多個實例。
- 線上 MOT 是一個迴圈：偵測 → 卡爾曼預測 → 代價矩陣 → 匈牙利指派 → 更新／出生／死亡。實務上的進步大多來自代價矩陣裡放了什麼，以及哪些偵測被允許參與匹配。
- ByteTrack 的低分偵測第二輪修正遮擋造成的斷裂；外觀修正非線性運動與長時間缺漏；端到端 query 追蹤器則直接學習關聯。
- MOTA 主要衡量偵測，IDF1 主要衡量關聯；HOTA = $\sqrt{\text{DetA} \cdot \text{AssA}}$（再對 IoU 門檻取平均）兼顧並能拆解兩者。
- 目前的前沿是長時間身分維持、非線性運動、開放詞彙類別，以及跨攝影機的 3D 追蹤。

練習

1. **卡爾曼增益的直覺**。對一維等位置模型，先驗變異數為 P、量測雜訊為 R，證明卡爾曼增益為 $K = P/(P+R)$，且後驗變異數為 $PR/(P+R)$。若每次預測都讓 P 增加 Q，連續五格沒有偵測之後 K 會怎麼變？

   提示

   $H = 1$ 時，$S = P + R$，$K = P/S$。後驗變異數為 $(1-K)P = PR/(P+R)$，比 P 與 R 都小。滑行五步後先驗變成 $P + 5Q$，K 往 1 靠近：濾波器會非常相信下一個偵測，這也是為什麼遮擋後一次錯誤的匹配會讓軌跡大幅移動。

2. **手算匈牙利演算法**。軌跡 A、B 與偵測 1、2 的預測 IoU 為 $\begin{bmatrix} 0.6 & 0.5 \\ 0.55 & 0.0 \end{bmatrix}$。貪婪匹配（IoU 最高者優先）的結果是什麼？以 $1 - \text{IoU}$ 為代價的匈牙利演算法結果又是什麼？哪一個的 IoU 總和較大？

   提示

   貪婪法先選 A–1（0.6），剩下 B–2 的 IoU 為 0（被拒絕），總和 0.6。匈牙利演算法選 A–2 與 B–1，總和 $0.5 + 0.55 = 1.05$。貪婪法讓軌跡 B 沒配到，並為偵測 2 開一個新 ID——這正是 ID 切換出現的方式之一。

3. **把追蹤器弄壞**。在合成場景中修改 `make_scene` 裡的反彈，讓物件 2 與 3 直接穿過彼此（速度不反轉）。三個追蹤器中哪一個進步最多？為什麼？

提示

運動變成線性後，等速預測在重疊期間都是正確的。以 `seed=3` 來說，低分偵測追蹤器的 IDF1 從 0.88 提升到約 0.96，也用了正確的 5 個 ID，幾乎追上外觀追蹤器（同樣約 0.96）；只用 IoU 的追蹤器仍會讓被遮擋的物件斷裂。外觀最有幫助的時候，正是運動非線性的時候，這也是 DanceTrack 的設計初衷。

4. **指標計算**。一段影片中只有一個物件，共 200 格。追蹤器每一格都偵測到它，但第 1–100 格用 ID 1，第 101–200 格用 ID 2。假設框完全正確，在單一門檻下計算 MOTA、IDF1、DetA、AssA 與 HOTA。

提示

MOTA $= 1 - 1/200 = 0.995$。最佳全域 ID 匹配涵蓋 100 格：IDF1 $= 2 \cdot 100/(200 + 200) = 0.5$。DetA $= 1$。每個 TP 的 $|\text{TPA}| = 100$、$|\text{FNA}| = 100$、$|\text{FPA}| = 0$，所以 AssA $= 0.5$，HOTA $= \sqrt{0.5} \approx 0.71$。一次切換幾乎不影響 MOTA，卻讓 AssA 減半。

5. **Re-ID 指標**。在 Re-ID 程式中，把雜訊從 1.6 改成 2.2，再改成 1.0。rank-1 與 mAP 怎麼變？哪一個比較敏感？請用 AP 的定義解釋。

提示

雜訊變大時 mAP 下降得比 rank-1 快，因為 rank-1 只需要最容易的那一個正確匹配排在第一，而 AP 會懲罰每一個被往後推的正確匹配。每個查詢有 4 個正確匹配時，最難的那一個通常決定了 AP 損失多少。

6. **設計題**。你要在倉庫中用 12 台已校正的攝影機追蹤堆高機與工作人員。用本篇介紹的元件畫出一個流程，並說明你會報告哪個指標、為什麼。

提示

每台攝影機做偵測，搭配 ByteTrack ／ BoT-SORT 風格的追蹤器；利用校正參數把框投影到地面；跨攝影機關聯先看 3D 位置與時間，再看 Re-ID 外觀（工作人員可能穿著一樣的背心）；兩個類別分開處理。報告 HOTA（平衡，且可拆成 DetA ／ AssA）與 IDF1（跨攝影機的長時身分），與 AI City Challenge 的設定一致。

參考文獻

1. Shyandram，〈物件追蹤 Object Tracking 簡介〉，Medium 文章，2023；2026 年更新。連結
2. G. Ciaparrone, F. Luque Sánchez, S. Tabik, L. Troiano, R. Tagliaferri and F. Herrera, “Deep learning in video multi-object tracking: A survey,” *Neurocomputing*, 2020. arXiv:1907.12740
3. W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu and T.-K. Kim, “Multiple object tracking: A literature review,” *Artificial Intelligence*, vol. 293, 2021 (arXiv 2014). arXiv:1409.7618
4. M. Adžemović, “Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art,” arXiv:2506.13457, 2025. arXiv

5. S. M. Marvasti-Zadeh, L. Cheng, H. Ghanei-Yakhdan and S. Kasaei, “Deep learning for visual tracking: A comprehensive survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, 2022. arXiv:1912.00535
6. J. Thangavel (Kugarajeevan), T. Kokul, A. Ramanan and S. Fernando, “Transformers in single object tracking: An experimental survey,” *IEEE Access*, vol. 11, 2023. arXiv:2302.11867
7. M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao and S. C. H. Hoi, “Deep learning for person re-identification: A survey and outlook,” *IEEE TPAMI*, vol. 44, 2022. arXiv:2001.04193
8. J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé and B. Leibe, “HOTA: A higher order metric for evaluating multi-object tracking,” *International Journal of Computer Vision*, 2020. arXiv:2009.07736
9. K. Bernardin and R. Stiefelhagen, “Evaluating multiple object tracking performance: The CLEAR MOT metrics,” *EURASIP Journal on Image and Video Processing*, 2008. doi
10. E. Ristani, F. Solera, R. S. Zou, R. Cucchiara and C. Tomasi, “Performance measures and a data set for multi-target, multi-camera tracking,” *ECCV Workshop on Benchmarking Multi-Target Tracking*, 2016. arXiv:1609.01775
11. K. Peng, A. Mathur and A. Narayanan, “Mitigating dataset harms requires stewardship: Lessons from 1000 papers,” arXiv:2108.02922, 2021. arXiv
12. L. Zheng, L. Shen, L. Tian, S. Wang, J. Wang and Q. Tian, “Scalable person re-identification: A benchmark,” *ICCV*, 2015. doi
13. L. Wei, S. Zhang, W. Gao and Q. Tian, “Person transfer GAN to bridge domain gap for person re-identification,” *CVPR*, 2018. arXiv:1711.08565
14. H. Luo, Y. Gu, X. Liao, S. Lai and W. Jiang, “Bag of tricks and a strong baseline for deep person re-identification,” *CVPR Workshops*, 2019. arXiv:1903.07071
15. M. Wieczorek, B. Rychalska and J. Dąbrowski, “On the unreasonable effectiveness of centroids in image retrieval,” arXiv:2104.13643, 2021. arXiv
16. S. Li, L. Sun and Q. Li, “CLIP-ReID: Exploiting vision-language model for image re-identification without concrete text labels,” *AAAI*, 2023. arXiv:2211.13977
17. D. S. Bolme, J. R. Beveridge, B. A. Draper and Y. M. Lui, “Visual object tracking using adaptive correlation filters,” *CVPR*, 2010. doi
18. J. F. Henriques, R. Caseiro, P. Martins and J. Batista, “High-speed tracking with kernelized correlation filters,” *IEEE TPAMI*, vol. 37, 2015. arXiv:1404.7584
19. L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi and P. H. S. Torr, “Fully-convolutional Siamese networks for object tracking,” arXiv:1606.09549, 2016 (ECCV 2016 workshops). arXiv
20. B. Li, J. Yan, W. Wu, Z. Zhu and X. Hu, “High performance visual tracking with Siamese region proposal network,” *CVPR*, 2018. doi
21. B. Ye, H. Chang, B. Ma, S. Shan and X. Chen, “Joint feature learning and relation modeling for tracking: A one-stream framework” (OSTrack), *ECCV*, 2022. arXiv:2203.11991
22. X. Chen, H. Peng, D. Wang, H. Lu and H. Hu, “SeqTrack: Sequence to sequence learning for visual object tracking,” *CVPR*, 2023 (the arXiv entry was later extended as SeqTrackv2). arXiv:2304.14394
23. N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma et al., “SAM 2: Segment anything in images and videos,” *ICLR*, 2025. arXiv:2408.00714
24. C.-Y. Yang, H.-W. Huang, W. Chai, Z. Jiang and J.-N. Hwang, “SAMURAI: Adapting segment anything model for zero-shot visual tracking with motion-aware memory,” arXiv:2411.11922,

2024. arXiv
25. N. Carion, L. Gustafson, Y.-T. Hu et al., “SAM 3: Segment anything with concepts,” *ICLR*, 2026. arXiv:2511.16719
26. Y. Wu, J. Lim and M.-H. Yang, “Object tracking benchmark,” *IEEE TPAMI*, vol. 37, 2015. doi
27. H. Fan, L. Lin, F. Yang, P. Chu, G. Deng, S. Yu et al., “LaSOT: A high-quality benchmark for large-scale single object tracking,” *CVPR*, 2019. arXiv:1809.07845
28. L. Huang, X. Zhao and K. Huang, “GOT-10k: A large high-diversity benchmark for generic object tracking in the wild,” *IEEE TPAMI*, vol. 43, 2021. arXiv:1810.11981
29. M. Müller, A. Bibi, S. Giancola, S. Al-Subaihi and B. Ghanem, “TrackingNet: A large-scale dataset and benchmark for object tracking in the wild,” arXiv:1803.10794, 2018 (ECCV 2018). arXiv
30. R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. doi
31. H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955. doi
32. A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, “Simple online and realtime tracking,” *ICIP*, 2016. arXiv:1602.00763
33. N. Wojke, A. Bewley and D. Paulus, “Simple online and realtime tracking with a deep association metric,” *ICIP*, 2017. doi
34. Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” *ECCV*, 2022. arXiv:2110.06864
35. J. Cao, J. Pang, X. Weng, R. Khirodkar and K. Kitani, “Observation-centric SORT: Rethinking SORT for robust multi-object tracking,” *CVPR*, 2023. arXiv:2203.14360
36. N. Aharon, R. Orfaig and B.-Z. Bobrovsky, “BoT-SORT: Robust associations multi-pedestrian tracking,” arXiv:2206.14651, 2022. arXiv
37. Z. Wang, L. Zheng, Y. Liu, Y. Li and S. Wang, “Towards real-time multi-object tracking” (JDE), *ECCV*, 2020. arXiv:1909.12605
38. Y. Zhang, C. Wang, X. Wang, W. Zeng and W. Liu, “FairMOT: On the fairness of detection and re-identification in multiple object tracking,” *IJCV*, 2021. arXiv:2004.01888
39. P. Bergmann, T. Meinhardt and L. Leal-Taixé, “Tracking without bells and whistles” (Tracktor), *ICCV*, 2019. arXiv:1903.05625
40. F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang and Y. Wei, “MOTR: End-to-end multiple-object tracking with transformer,” *ECCV*, 2022. arXiv:2105.03247
41. Y. Zhang, T. Wang and X. Zhang, “MOTRv2: Bootstrapping end-to-end multi-object tracking by pretrained object detectors,” *CVPR*, 2023. arXiv:2211.09791
42. R. Gao, J. Qi and L. Wang, “Multiple object tracking as ID prediction” (MOTIP), *CVPR*, 2025. arXiv:2403.16848
43. S. Li, T. Fischer, L. Ke, H. Ding, M. Danelljan and F. Yu, “OVTrack: Open-vocabulary multiple object tracking,” *CVPR*, 2023. arXiv:2304.08408
44. A. Milan, L. Leal-Taixé, I. Reid, S. Roth and K. Schindler, “MOT16: A benchmark for multi-object tracking,” arXiv:1603.00831, 2016 (MOT16/MOT17). arXiv
45. P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler and L. Leal-Taixé, “MOT20: A benchmark for multi object tracking in crowded scenes,” arXiv:2003.09003, 2020. arXiv

46. P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani and P. Luo, “DanceTrack: Multi-object tracking in uniform appearance and diverse motion,” *CVPR*, 2022. arXiv:2111.14690
47. Y. Cui, C. Zeng, X. Zhao, Y. Yang, G. Wu and L. Wang, “SportsMOT: A large multi-object tracking dataset in multiple sports scenes,” *ICCV*, 2023. arXiv:2304.05170
48. F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan and T. Darrell, “BDD100K: A diverse driving dataset for heterogeneous multitask learning,” *CVPR*, 2020. arXiv:1805.04687
49. C. Doersch, Y. Yang, M. Vecerik, D. Gokay, A. Gupta, Y. Aytar et al., “TAPIR: Tracking any point with per-frame initialization and temporal refinement,” *ICCV*, 2023. arXiv:2306.08637
50. N. Karaev, I. Rocco, B. Graham, N. Neverova, A. Vedaldi and C. Rupprecht, “CoTracker: It is better to track together,” *ECCV*, 2024. arXiv:2307.07635
51. N. Karaev, I. Makarov, J. Wang, N. Neverova, A. Vedaldi and C. Rupprecht, “CoTracker3: Simpler and better point tracking by pseudo-labelling real videos,” arXiv:2410.11831, 2024. arXiv
52. Z. Tang, S. Wang, D. C. Anastasiu et al., “The 9th AI City Challenge,” *ICCV Workshops*, 2025. arXiv:2508.13564
53. VOT Challenge organizers, “The Visual Object Tracking (VOT) challenge series,” website, 2013–present. votchallenge.net

DLCV 07・世界模型：影片、潛在動態與規劃

世界模型是什麼、模型式強化學習如何使用它（World Models、Dreamer 系列、MuZero）、為什麼影片生成器被當成世界模擬器、潛在動態與 JEPA 式預測表示如何運作，以及如何評估它們，並附上完整的 CPU 實驗。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-07-world-models/

你將學到

- 世界模型是什麼，以及觀測、隱藏狀態、信念（belief）與動作的差別。
- 模型式強化學習如何以三種方式使用學到的模型：線上規劃、在想像中學習策略，以及用價值等價模型做樹搜尋；並以 World Models、PlaNet、Dreamer 系列、TD-MPC2 與 MuZero 為例。
- 為什麼影片預測與生成被視為「世界模擬器」、為什麼平方誤差預測器會產生模糊，以及 Genie 這類互動式生成器能證明什麼、不能證明什麼。
- 循環狀態空間模型與 JEPA 式預測表示如何在不重建（或重建）像素的情況下學習潛在動態。
- 如何擬合一個小型動態模型、量測多步誤差、用它規劃，並偵測規劃器何時在利用模型錯誤，全部在 CPU 上完成。
- 如何評估世界模型，以及哪些問題仍然開放。

先看全貌

追蹤問的是物體去了哪裡。世界模型問的是接下來可能發生什麼，尤其是**如果代理人採取某個動作**會發生什麼。機器人靠近杯子時，光有物體標籤不夠：它需要預測夾爪和杯子會怎麼移動、哪些結果不確定，以及哪個動作較可能達成目標。

> **用最簡單的話說**
>
> 推玩具車之前，你會先想像它會跑到哪裡。你可以想像輕推和用力推，挑比較好的那一種，推下去，再看看實際發生了什麼。世界模型就是這種想像力的學習版本。規劃器則是在想像中試推、挑出一個動作的那部分。每推一次就重新看一眼，能讓想像中的小錯誤不會越積越多。

「世界模型」一詞涵蓋好幾個研究傳統。Ding 等人依兩種功能整理文獻：建立能捕捉世界運作方式的內部表示，以及預測未來狀態以模擬並引導決策 [1]。本章採用一個操作型定義：**世界模型是一個學習得到的模型，它表示足夠多的環境資訊，能在指定條件（通常包括代理人自己的動作）下預測相關的變化**。它的用處取決於任務。用來渲染場景的模型、用來學習表示的模型，以及用來控制機器人的模型，可能需要非常不同的資訊。

觀測、狀態、信念與動作

白話版。相機影像不是世界本身，只是世界的一個局部視角。世界模型必須記住目前看不到的東西，例如球的速度，或被遮住的物體。

令 s_t 表示時間 t 的真實環境狀態，o_t 為觀測，a_t 為執行的動作：

$$s_{t+1} \sim p(s_{t+1} \mid s_t, a_t), \qquad o_t \sim p(o_t \mid s_t).$$

第一個分布是**轉移**（動態）模型，第二個是**觀測**模型。狀態可能包含物體位置、速度與接觸條件。影像只呈現一個投影，帶有遮擋與感測器雜訊；Szeliski 的教科書涵蓋這個觀測過程中的幾何與運動 [3]。兩張完全相同的影像可能有不同的未來：同一個位置的球，可能正往左也可能正往右。

因此，學習得到的模型會從歷史建立表示：

$$z_t = E_\theta(o_{0:t}, a_{0:t-1}), \qquad p_\theta(z_{t+1} \mid z_t, a_t),$$

其中 E_θ 是參數為 θ 的編碼器，$o_{0:t}$ 是到時間 t 為止的觀測序列，$p_\theta(z_{t+1} \mid z_t, a_t)$ 是學到的潛在轉移。編碼器不會自動讓 z_t 成為充分的狀態。若它丟掉了速度、被遮住的物體或某個相關的材質性質，轉移模型就無法找回這些資訊。循環記憶或一小段影片上下文會有幫助，但必須在實際的遮擋與延遲下測試。

當狀態只能被部分觀測時，有原則的對象是**信念**：在已知歷史下，對可能狀態的一個機率分布。Bishop 對序列潛在變數模型（隱藏馬可夫模型與線性動態系統）的說明提供了機率背景 [2]。一個緊湊的潛在向量可以近似這個分布中有用的部分；它不必一對一地對應到人類看得懂的物理變數。

預測的分解

對一個候選的動作序列，未來觀測的機率可分解為

$$p_\theta(o_{t+1:t+H} \mid o_{\le t}, a_{0:t+H-1}) = \int p_\theta(z_t \mid o_{\le t}, a_{<t}) \prod_{k=t}^{t+H-1} p_\theta(z_{k+1} \mid z_k, a_k)\, p_\theta(o_{k+1} \mid z_{k+1})\, dz_{t:t+H},$$

其中 H 是預測視野。三個因子分別是**推論**（給定過去，目前的潛在狀態是什麼？）、**轉移**（它在這些動作下如何演變？）與**渲染**（相機會看到什麼？）。如果目標與成本定義在潛在空間中，規劃器可以直接在 z 上運作，省略像素解碼器。本章許多設計選擇，其實就是決定要學哪些因子、怎麼學。

模型式強化學習

白話版。強化學習（reinforcement learning, RL）靠嘗試錯誤學會行動。如果代理人也學會世界如何回應它的動作，它就能在腦中練習，大幅減少真實嘗試的次數。

問題設定

RL 代理人以離散步驟與環境互動。每一步它觀測環境，依**策略** $\pi(a_t \mid s_t)$ 選擇 a_t，得到獎勵 r_t，環境隨之改變。目標是最大化期望**回報**

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k},$$

其中 $\gamma \in [0,1)$ 是折扣因子，讓近期獎勵比遠期獎勵更重要。**價值** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ 是從狀態 s 出發、依循 π 時的期望回報。

無模型（model-free）方法直接從真實轉移學習策略或價值。**模型式**（model-based）方法另外學習轉移與獎勵的模型，並用它減少真實互動。這個想法由來已久：Sutton 的 Dyna 架構（1991）就已經把從真實經驗學習與在學到的模型所模擬的經驗上規劃結合起來，兩者使用同樣的價值更新 [4]。Moerland 等人的綜述沿兩個軸整理這個領域：如何學習動態模型（處理隨機性、不確定性、部分可觀測性與時間抽象），以及規劃與學習如何整合（從哪裡開始規劃、給規劃多少預算、怎麼規劃，以及如何把結果回饋到行動與學習中）[5]。

使用模型的三種方式

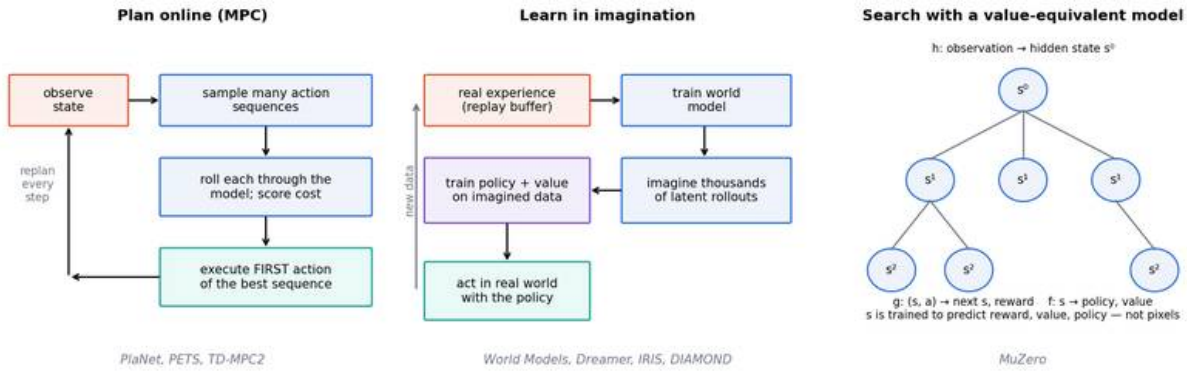


圖 *1* — 使用學到的模型的三種方式。同一個模型可以扮演不只一種角色；差別在於決策是在哪裡算出來的。

1. **線上規劃（模型預測控制）**。每一個真實步驟，都在模型中模擬候選動作序列以尋找好的序列，只執行第一個動作，觀測，再重複。PlaNet [7]、PETS [29] 與 TD-MPC2 [11] 屬於這一類。

2. **在想像中學習**。用模型產生的軌跡訓練一個策略（通常還有價值函數），執行時直接用策略行動，不再搜尋。World Models [6]、Dreamer 系列 [8], [9], [10]、IRIS [12] 與 DIAMOND [13] 屬於這一類。
3. **用價值等價模型搜尋**。學一個只預測規劃所需量（獎勵、價值、策略）、不預測觀測的模型，並在它的潛在空間中做樹搜尋。MuZero [14] 是代表。

三者有一個共同的危險：模型總會在某些地方出錯，而最佳化器非常擅長找到正好是那些地方。我們會在「不確定性與模型被利用」一節回到這點。

為什麼誤差會累積

假設學到的轉移 $\hat{f}$ 的單步誤差最多為 ε，而真實動態 f 是常數為 L 的 Lipschitz 函數（狀態的小變化最多讓下一個狀態變化 L 倍）。從同一個狀態出發、施加相同動作，k 步後預測與真實狀態的差距 e_k 滿足 $e_{k+1} \le L e_k + \varepsilon$，因此

$$e_H \le \varepsilon \sum_{k=0}^{H-1} L^k = \varepsilon \, \frac{L^H - 1}{L - 1} \quad (L \neq 1).$$

當 $L > 1$ 時，這個上界隨視野 H 指數成長；當 $L < 1$ 時，它飽和於 $\varepsilon/(1-L)$。這是最壞情況的上界，但它解釋了兩個常見設計：頻繁重新規劃（MPC）與使用短的模型推演。Janner 等人把第二個想法具體化：他們的 MBPO 演算法從真實狀態分岔出短的模型推演，而不是從頭想像長軌跡，並從理論與實驗兩方面論證這能保留模型資料的好處、同時限制其偏差 [30]。

World Models：壓縮、預測、控制

白話版。先學會用幾個數字概括每張畫面，再學會預測下一個概括，最後在上面訓練一個很小的控制器。控制器甚至可以完全在模型的「夢境」裡訓練。

Ha 與 Schmidhuber 的 *World Models*（以「Recurrent World Models Facilitate Policy Evolution」發表於 NeurIPS 2018，另有互動式線上版本）讓這個想法在深度學習中廣為人知 [6]。他們的代理人有三個部分：

- **V（視覺）**：一個變分自編碼器，把每張 64×64 的畫面壓縮成小的潛在向量 z_t。
- **M（記憶）**：一個帶混合密度輸出的循環網路，在給定 z_t、動作與隱藏狀態 h_t 時，預測 z_{t+1} 的分布。
- **C（控制器）**：一個刻意設計得極小的線性策略 $a_t = W_c[z_t; h_t] + b_c$，以演化策略而非反向傳播訓練。

在賽車任務中，使用 V 與 M 特徵的控制器解決了任務；在 VizDoom 任務中，控制器完全在 M 模擬的環境中訓練，再轉移到真實遊戲 [6]。這個設計把**感知**、**預測**與**決策**分開，並顯示當表示已經帶有相關動態時，控制器可以非常小。它也揭露了核心風險：在不完美夢境中訓練的控制器，可能學會利用夢境的錯誤。

潛在動態：從 PlaNet 到 Dreamer 系列

白話版。不預測未來的影像，而是預測一個緊湊的隱藏狀態，只在訓練時才把影像解碼出來。一個同時包含確定性記憶與隨機部分的隱藏狀態，可以表示代理人記得的東西，也能表示它不確定的東西。

循環狀態空間模型

Hafner 等人的 PlaNet 從影像學習環境動態，並在潛在空間中以快速線上規劃選擇動作；它的關鍵建模想法是同時包含確定性與隨機成分的潛在動態模型，即**循環狀態空間模型**（recurrent state-space model, RSSM）[7]。RSSM 保留一個確定性記憶 h_t 與一個隨機狀態 z_t：

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}), \qquad z_t \sim p_\theta(z_t \mid h_t) \text{（prior）}, \qquad z_t \sim q_\theta(z_t \mid h_t, o_t) \text{（posterior）}.$$

這裡 f_θ 是一個循環單元；**先驗**（prior）在沒看到目前影像的情況下預測 z_t（想像時就用它）；**後驗**（posterior）還會看到影像 o_t（訓練時以及過濾真實觀測時使用）。多個輸出頭讀取模型狀態 (h_t, z_t)，預測影像 $\hat{o}_t$、獎勵 $\hat{r}_t$ 與延續旗標（回合是否繼續）。

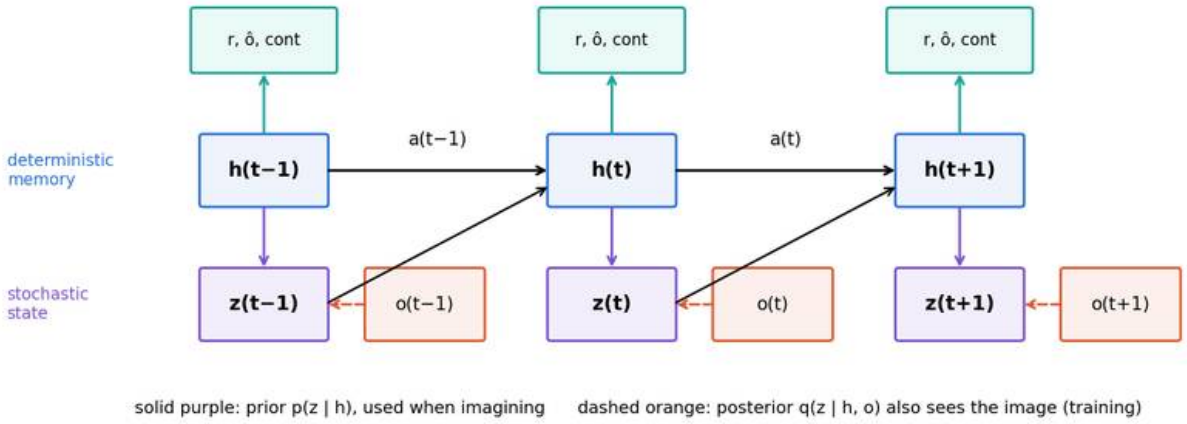


圖 2 — 展開的 *RSSM*。想像時只用先驗路徑（實線）；訓練時 z 還會以真實影像為條件（虛線）。

訓練時最大化觀測序列概似的變分下界。一個簡化的單步損失為

$$\mathcal{L}_t = -\log p_\theta(o_t \mid h_t, z_t) - \log p_\theta(r_t \mid h_t, z_t) + \beta\, D_{\mathrm{KL}}(q_\theta(z_t \mid h_t, o_t) \,\|\, p_\theta(z_t \mid h_t)),$$

其中 $\beta > 0$ 是 KL 項的權重。重建項讓狀態帶有資訊；KL 項則教先驗去預測後驗將會看到的東西，這正是想像所需要的能力。實際實作還會加入損失平衡與其他穩定化技巧，所以請把這個式子當成教學大綱，而不是任何一篇論文的精確損失。

Dreamer：在想像中學習行為

Dreamer（2020）保留 RSSM，但把線上規劃換成一個完全在想像出的潛在軌跡上訓練的 actor–critic。它的關鍵做法是把學到的狀態價值的解析梯度，沿著在緊湊潛在空間中想像出的軌跡反向傳播 [8]。代理人從由真實資料推得的潛在狀態出發，用 actor 想像 H 步，用學到的 critic 評分，並以自舉目標（例如 λ-回報）訓練 critic：

$$V_t^\lambda = \hat{r}_t + \gamma\, \hat{c}_t \big[(1-\lambda)\, v_\psi(s_{t+1}) + \lambda\, V_{t+1}^\lambda \big],$$

其中 $\hat{r}_t$ 與 $\hat{c}_t$ 是預測的獎勵與延續，v_ψ 是 critic，$\lambda \in [0,1]$ 在單步自舉（$\lambda = 0$）與較長的想像回報（$\lambda = 1$）之間取捨。actor 則被更新以提高這些價值。

這條路線持續發展：

- **DreamerV2**（2021）改用**離散**（類別型）潛在變數，成為第一個藉由在獨立訓練的世界模型中學習行為、在 55 款遊戲的 Atari 基準上達到人類水準的代理人 [9]。
- **DreamerV3**（Nature，2025）使用以正規化、平衡與轉換為基礎的穩健化技巧，使單一設定就能在超過 150 個任務上勝過專門方法，並且是第一個在沒有人類資料或課程設計的情況下、從零開始在 Minecraft 中收集到鑽石的演算法 [10]。

另外兩條相關路線說明 RSSM 只是眾多選擇之一。**TD-MPC2** 在一個**隱式**、沒有解碼器的世界模型（不重建影像）的潛在空間中規劃，以單一組超參數在 104 個線上 RL 任務上取得強勁結果，並訓練出一個能執行 80 個任務的 317M 參數代理人 [11]。**IRIS** 以離散自編碼器加上對影像 token 的自迴歸 Transformer 取代 RSSM，只用相當於兩小時的 Atari 遊戲經驗學習 [12]。**DIAMOND** 更進一步走向像素：它讓代理人在**擴散**世界模型中訓練，主張離散潛在變數可能丟掉對控制重要的視覺細節，並展示了這個模型可作為 3D 遊戲的互動式神經遊戲引擎 [13]。

MuZero：只預測規劃需要的東西

白話版。要下好一盤棋，你不需要預測畫面會長什麼樣子。你只需要預測局面有多好、哪些著法看起來有希望，以及會得到哪些獎勵。MuZero 正是學這些，然後進行搜尋。

AlphaZero 式的樹搜尋需要一個遊戲規則的模擬器。MuZero 移除了這個需求：它學一個反覆套用時能預測與規劃最直接相關之量的模型，也就是獎勵、動作選擇策略與價值 [14]。它有三個函數：

$$s^0 = h_\theta(o_1, \dots, o_t), \qquad (r^k, s^k) = g_\theta(s^{k-1}, a^k), \qquad (\mathbf{p}^k, v^k) = f_\theta(s^k),$$

其中 h_θ（表示函數）把觀測歷史編碼成隱藏狀態 s^0，g_θ（動態函數）把隱藏狀態與假設動作 a^k 映射到預測獎勵 r^k 與下一個隱藏狀態，f_θ（預測函數）輸出策略向量 $\mathbf{p}^k$ 與價值 v^k。蒙地卡羅樹搜尋展開 g_θ 來選擇動作。訓練時沿著真實軌跡把模型展開 K 步，並最小化

$$\mathcal{L} = \sum_{k=0}^{K} \Big[\ell^r(u_{t+k}, r^k) + \ell^v(z_{t+k}, v^k) + \ell^p(\boldsymbol{\pi}_{t+k}, \mathbf{p}^k) \Big] + c\|\theta\|^2,$$

其中 u 是觀測到的獎勵，z 是價值目標（自舉回報），$\boldsymbol{\pi}$ 是搜尋產生的造訪次數分布，ℓ^r, ℓ^v, ℓ^p 是各量的損失，c 是 L2 懲罰的權重。

隱藏狀態 s^k 從不被要求重建觀測。它只需要**價值等價**：用它規劃，必須得到與在真實環境中規劃相同的獎勵、價值與策略。以這個設計，MuZero 在 57 款 Atari 遊戲上創下新的最佳成績，並在沒有被告知規則的情況下，於圍棋、西洋棋與將棋上追平 AlphaZero 的超人表現 [14]。對視覺而言的教訓是：世界模型的預測目標應依用途而定；預測每個像素只是一種選擇，不是必要條件。

影片預測與生成：作為世界模擬器

白話版。如果模型能生成逼真的未來影片，也許它學會了世界如何運作。有時候確實如此；但更常見的是，它只學會了影片通常看起來像什麼。判斷是哪一種，才是難處所在。

以動作為條件的影片預測

Finn 等人在一個含 59,000 次機器人推動互動的資料集上，訓練了**以動作為條件的影片預測**模型；由於模型預測的是像素運動而非原始像素，它對沒見過的物體有部分泛化能力，而以不同的未來動作為條件，就產生了對不同未來的「視覺想像」[15]。這是最字面意義上的世界模型：影像進、動作進、影像出。

為什麼平方誤差預測器會模糊

真實的未來是不確定的。假設從同樣的過去出發，球有 $\frac{1}{2}$ 的機率往左、$\frac{1}{2}$ 的機率往右。對平方誤差而言，最佳的單一預測是**條件平均**：

$$\hat{o}^\star = \arg\min_{\hat{o}} \ \mathbb{E}[\|o_{t+1} - \hat{o}\|^2 \mid o_{\le t}] = \mathbb{E}[o_{t+1} \mid o_{\le t}],$$

在這裡就是兩顆半亮的球，一張永遠不會發生的影像。一個從兩種未來中**取樣**的生成模型會產生逼真的影像，但平均而言平方誤差**更高**。

```
import numpy as np

x = np.arange(64)
def frame(center):                      # a 1-D "image" with one bright ball
    return np.exp(-0.5 * ((x - center) / 2.0) ** 2)

# Same past, two equally likely futures: the ball bounces left or right.
futures = np.stack([frame(20), frame(44)])
mean_pred = futures.mean(axis=0)        # what an MSE-trained predictor converges to
sample_pred = futures[0]                # what a good generative sampler might output

def expected_mse(pred):                 # average over the two possible futures
    return np.mean([(pred - f) ** 2 for f in futures])

print(f"peak brightness: true future 1.00, mean prediction {mean_pred.max():.2f}")
print(f"expected MSE of mean prediction:   {expected_mse(mean_pred):.4f}")
print(f"expected MSE of a sharp sample:    {expected_mse(sample_pred):.4f}")
```

輸出：

```
peak brightness: true future 1.00, mean prediction 0.50
expected MSE of mean prediction:   0.0277
expected MSE of a sharp sample:    0.0554
```

清晰的樣本的期望誤差恰好是模糊平均的兩倍。這就是為什麼像素 MSE 會獎勵模糊、為什麼會出現隨機式影片模型，以及為什麼只用一個平方誤差數字評估世界模型，可能選錯模型。

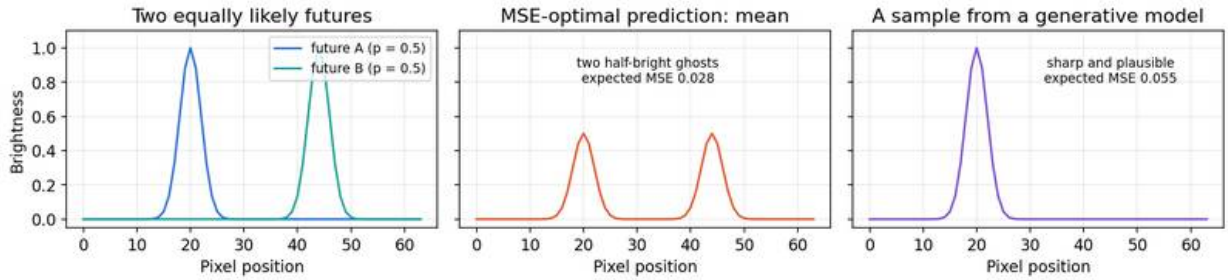


圖 3 — 上方程式碼的輸出。*MSE* 最佳的預測把互不相容的未來平均在一起；取樣結果合理，但 *MSE* 分數較差。

Babaeizadeh 等人的 SV2P 為潛在變數的每個樣本預測一個不同的可能未來，結果比沒有隨機性的同一模型好得多 [16]。現代影片生成器在大得多的規模上使用同樣的原理，採用自迴歸 token 模型或擴散與流模型（見ML 07 與ML 08）。

大型生成式世界模型

- **GAIA-1** 把駕駛世界建模視為對影片、文字與動作離散 token 的下一個 token 預測，能產生逼真的駕駛情境，並可細部控制自車行為與場景特徵 [17]。
- **UniSim** 結合在不同面向上豐富的資料集（影像中的物體、機器人資料中的密集動作、導航資料中的運動），學習真實世界互動的「通用模擬器」；它能模擬高階指令與低階控制，而完全在其中訓練的策略可零樣本部署到真實機器人上 [18]。
- **Genie** 是一個在未標註的網路影片上、不使用動作標籤訓練出的生成式互動環境。它由影片 tokenizer、自迴歸動態模型與一個從畫面變化推斷動作的**潛在動作模型**組成，因此即使訓練時沒有動作標籤，使用者仍能在生成的世界中逐格行動 [19]。這類潛在動作不一定對應到經過校正的實體致動器。
- **Genie 3** 由 Google DeepMind 於 2025 年 8 月發布，能從文字生成可即時導覽的世界，每秒 24 格、720p，並維持數分鐘的一致性。公告列出了限制：動作空間受限、難以模擬多個獨立代理人、真實地點不精確，以及互動只能持續數分鐘而非數小時 [20]。
- **Cosmos** 是 NVIDIA 的開放「世界基礎模型」平台：包含預訓練影片模型、影片 tokenizer、資料整理流程與後訓練範例，目的是微調成機器人與車輛專用的世界模型 [21]。

逼真的影片就代表理解物理嗎？

兩項仔細的研究給出的答案是：不一定。Motamed 等人的 **Physics-IQ** 基準測試流體力學、光學、固體力學、磁學與熱力學；在目前的影片模型（包括 Sora、Runway、Pika、Lumiere、Stable Video Diffusion 與 VideoPoet）中，物理理解嚴重不足，且與視覺真實感無關 [22]。Kang 等人在由古典力學支配的 2D 模擬器上訓練擴散影片模型，發現分布內完美泛化、組合式泛化可觀察到隨規模改善，但分布外則失敗；模型的行為像是在找最相近的訓練樣本，而不是抽象出規則 [23]。

用於機器人控制時，也要問「動作」指的是什麼：力、關節目標、夾爪指令、相機移動，還是推斷出的 token。單位錯誤、時間偏移或控制頻率不同，都會改變條件預測問題，即使影片看起來很有說服力。

預測式表示：JEPA 家族

白話版。預測每一個像素，會把力氣浪費在不重要的細節上，例如樹葉的紋理。不如預測缺失或未來部分的抽象**描述**。如果描述夠好，這個預測就對規劃有用。

聯合嵌入預測架構（joint-embedding predictive architecture, JEPA）用編碼器分別編碼可見的**上下文** x 與被隱藏的**目標** y，並訓練一個預測器把上下文嵌入映射到目標嵌入：

$$\mathcal{L} = \| P_\phi(E_\theta(x),\, m) - \mathrm{sg}[\bar{E}(y)] \|^2,$$

其中 E_θ 是上下文編碼器，$\bar{E}$ 是目標編碼器，P_ϕ 是以目標位置資訊 m（例如在影像或時間中的位置）為條件的預測器，sg（stop-gradient）阻止梯度流入目標分支。沒有任何東西在像素空間中重建。危險在於**崩塌**（collapse）：如果兩個編碼器都輸出常數，損失就是零。實務上用不對稱設計（只有一側有預測器、stop-gradient，以及不以同一梯度訓練的目標編碼器）來避免崩塌；ML 05 更詳細地討論這些自監督機制。

- **I-JEPA** 從一個資訊豐富的上下文區塊，預測同一張影像中數個大型目標區塊的表示，並發現遮罩策略（大而具語意的目標、空間上分散的上下文）是在不使用手工資料擴增的情況下學到語意特徵的關鍵 [24]。
- **V-JEPA** 把特徵預測當成獨立的目標應用到影片上，在 200 萬支影片上訓練，不使用預訓練影像編碼器、文字、負樣本或重建，所得的凍結特徵在動作類與外觀類任務上都表現良好 [25]。
- **V-JEPA 2** 先在超過一百萬小時的網路影片上預訓練不含動作的 JEPA，再用不到 62 小時的未標註機器人影片，後訓練出一個潛在的**以動作為條件**的世界模型 V-JEPA 2-AC；以影像目標進行規劃後，它在兩個實驗室的 Franka 機械手臂上實現零樣本的拾取與放置，且沒有使用來自這些環境的資料、任務專屬訓練或獎勵 [26]。被動影片與動作資料的分工，讓本章所說的觀測／動作區別變得具體。
- **V-JEPA 2.1** 加入可見與被遮罩 token 都參與的密集預測損失、跨中間層的深層自監督，以及適用影像與影片的多模態 tokenizer，目標是空間上有結構、時間上一致的密集特徵 [27]。
- **DINO-WM** 用現成的編碼器展示同樣的想法：它從離線軌跡預測未來的 DINOv2 patch 特徵，並把動作序列最佳化到接近目標影像的特徵來規劃，不重建像素，也不學獎勵模型 [28]。

特徵空間中的目標需要小心。距離 $\|z - z_{\text{goal}}\|^2$ 只會獎勵表示中保留下來的東西。如果表示忽略了夾爪是否閉合，規劃器可能到達一個看起來相似的狀態，卻沒有真的抓住物體。單憑特徵品質更好，並不能證明編碼器能預測任意動作的後果。

各家族之間的關係

家族	預測目標	主要好處	要檢驗的問題
潛在動態（RSSM、TD-MPC2）	緊湊狀態、獎勵、價值	便宜、可重複推演	潛在狀態是否保留任務相關動態？
價值等價（MuZero）	只有獎勵、價值、策略	不在無關細節上花力氣	獎勵改變時還能遷移嗎？
自迴歸影片（GAIA-1、Genie、IRIS）	下一張畫面或 token	明確的序列分解	長推演時誤差會漂移嗎？
擴散／流影片（DIAMOND、Cosmos）	一起生成的畫面或片段	表達力強、能表示不確定的未來	能否在控制延遲內以動作操控結果？

家族	預測目標	主要好處	要檢驗的問題
預測式嵌入（JEPA、DINO-WM）	目標特徵	不需像素重建就能學到結構	動作是否被表示？目標是否有意義？

這些選擇彼此相容。Transformer 可以參數化其中任何一種；潛在世界模型可以透過擴散解碼；影片生成器可以外層用自迴歸、內層用擴散。架構、預測目標、時間分解與訓練目標是各自獨立的設計決策。

從資料學習動態

訓練樣本應保留順序與動作對齊：

$$(o_t,\, a_t,\, o_{t+1},\, r_t,\, \text{done}_{t+1}),$$

其中 done_{t+1} 標記回合結束。記錄每個動作發生在每個相機時間戳之前或之後。依完整回合、環境實例與相關受試者切分；隨機切分相鄰畫面會洩漏幾乎相同的場景，讓轉移預測器看起來比實際更好（資料洩漏的一般討論見ML 10）。

資料收集策略很重要。在緩慢、輕柔互動上訓練的模型，可能在劇烈運動時失敗。當資料中沒有動作時，觀察到的相關性特別麻煩：模型可能學到手靠近時杯子會動，卻完全沒有證據知道其他推法會怎樣。這就是被動影片與介入式資料的差別。

單步損失衡量從真實歷史出發的預測。**多步損失**展開模型自己的預測並懲罰累積誤差。展開會改變訓練難度與梯度路徑；它並不能消除觀測互動範圍之外的分布偏移。

完整 CPU 實驗：擬合、預測、規劃

我們使用二維狀態 $x_t = (\text{position}, \text{velocity})$（位置與速度）、純量動作 u_t，以及一個有阻尼的線性系統：

$$x_{t+1} = Ax_t + Bu_t + \epsilon_t,$$

其中 A 是 2×2 狀態轉移矩陣，B 是動作輸入向量，ϵ_t 是微小的高斯雜訊。這個系統刻意設計成完全可觀測，以便在加入影像編碼器與部分可觀測性之前，先單獨檢視動態與規劃。訓練與測試回合各自獨立產生。我們只在訓練轉移上擬合，並用相同的保留動作同時評估單步預測與開迴路推演。

規劃器使用**交叉熵方法**（cross-entropy method, CEM）：從高斯分布取樣候選動作序列，保留模型下成本最低的「菁英」序列，把高斯的平均與分散重新擬合到它們，然後重複。它只執行一個動作，取得新的真實狀態後重新規劃。這就是模型預測控制。

```
import numpy as np

rng = np.random.default_rng(7)
A = np.array([[1.0, 0.1], [0.0, 0.94]])
B = np.array([0.005, 0.1])
```

```
def collect(n, steps=40):
    x = np.zeros((n, steps + 1, 2))
    x[:, 0] = rng.normal(size=(n, 2)) * [0.8, 0.3]
    u = rng.uniform(-1, 1, size=(n, steps))
    for t in range(steps):
        x[:, t + 1] = (x[:, t] @ A.T + u[:, t, None] * B
                       + rng.normal(scale=0.002, size=(n, 2)))
    return x, u

train, train_u = collect(120)
test, test_u = collect(80)
phi = np.column_stack([train[:, :-1].reshape(-1, 2), train_u.ravel()])
target = train[:, 1:].reshape(-1, 2)
W = np.linalg.lstsq(phi, target, rcond=None)[0]
W_no_action = np.linalg.lstsq(phi[:, :2], target, rcond=None)[0]

def predict(x, u):
    return np.concatenate([x, np.asarray(u)[..., None]], axis=-1) @ W

one_step = predict(test[:, :-1], test_u)
rollout = np.zeros_like(test)
rollout[:, 0] = test[:, 0]
for t in range(test_u.shape[1]):
    rollout[:, t + 1] = predict(rollout[:, t], test_u[:, t])
rmse = lambda a, b: np.sqrt(np.mean((a - b) ** 2))
print("One-step:", rmse(one_step, test[:, 1:]))
print("Without actions:", rmse(test[:, :-1] @ W_no_action, test[:, 1:]))
print("Final-horizon rollout:", rmse(rollout[:, -1], test[:, -1]))

def plan(x, mean, horizon=20, population=256):
    std = np.ones(horizon)
    for _ in range(4):
        candidates = np.clip(rng.normal(mean, std, (population, horizon)), -1, 1)
        imagined = np.broadcast_to(x, (population, 2)).copy()
        costs = np.zeros(population)
        for t in range(horizon):
            imagined = predict(imagined, candidates[:, t])
            costs += (imagined[:, 0] ** 2 + 0.15 * imagined[:, 1] ** 2
                      + 0.02 * candidates[:, t] ** 2)
        costs += 20 * imagined[:, 0] ** 2
        elite = candidates[np.argsort(costs)[:32]]
        mean = elite.mean(axis=0)
        std = elite.std(axis=0) + 0.05
    return mean

real = np.array([1.5, 0.0])
mean = np.zeros(20)
for _ in range(40):
    sequence = plan(real, mean)
    real = A @ real + B * sequence[0]  # real system; never used inside plan()
    mean = np.r_[sequence[1:], 0.0]
print("Controlled final state:", real)
assert rmse(one_step, test[:, 1:]) < 0.01
```

```
assert abs(real[0]) < 0.1
```

輸出：

```
One-step: 0.0019866110218121832
Without actions: 0.04106321931941812
Final-horizon rollout: 0.013772759824274517
Controlled final state: [-0.00453985 -0.0210205 ]
```

單步 RMSE（0.0020）落在雜訊水準；拿掉動作輸入後增加到二十倍（0.0411），可見動作確實重要。即使是這個幾乎完美的線性模型，視野 40 的開迴路推演誤差（0.0138）也約為單步誤差的七倍，是上面推導的誤差累積的溫和例子。MPC 把位置從 1.5 帶到約 −0.0045，而零動作時位置停在 1.5。這些是本腳本的合成結果，不是已發表的比較。

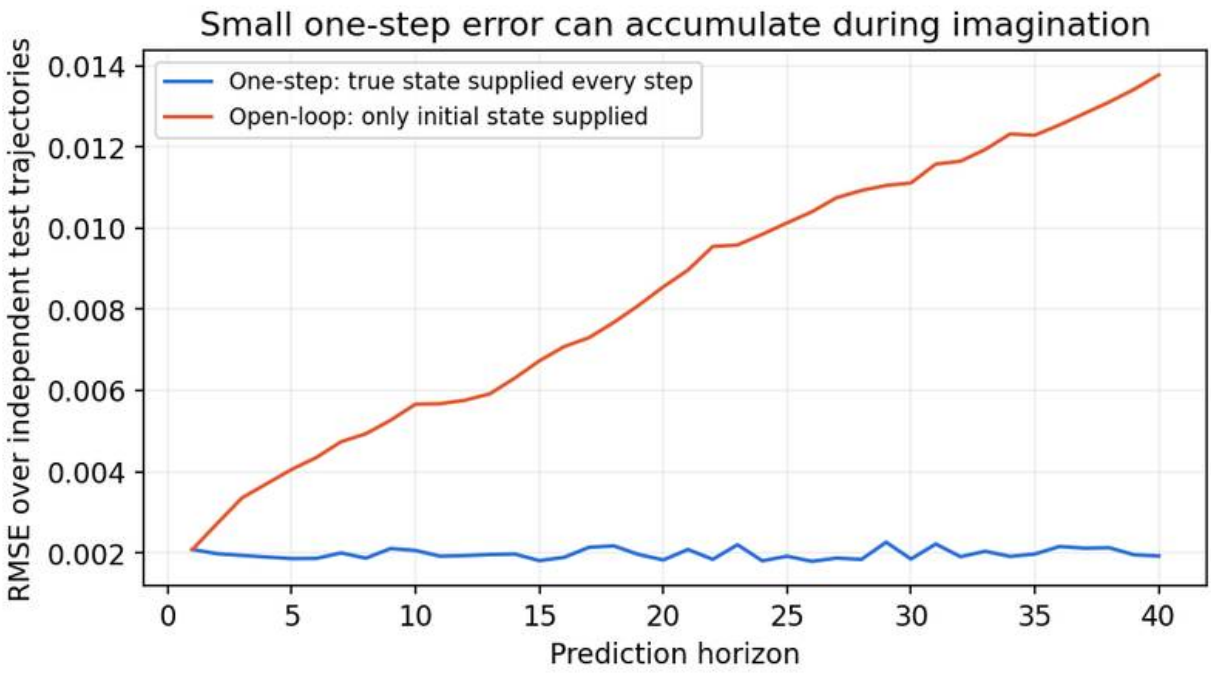


圖 4 — 獨立的合成測試回合。單步評估每一步都拿到真實狀態；開迴路評估只拿到初始狀態與保留的動作。

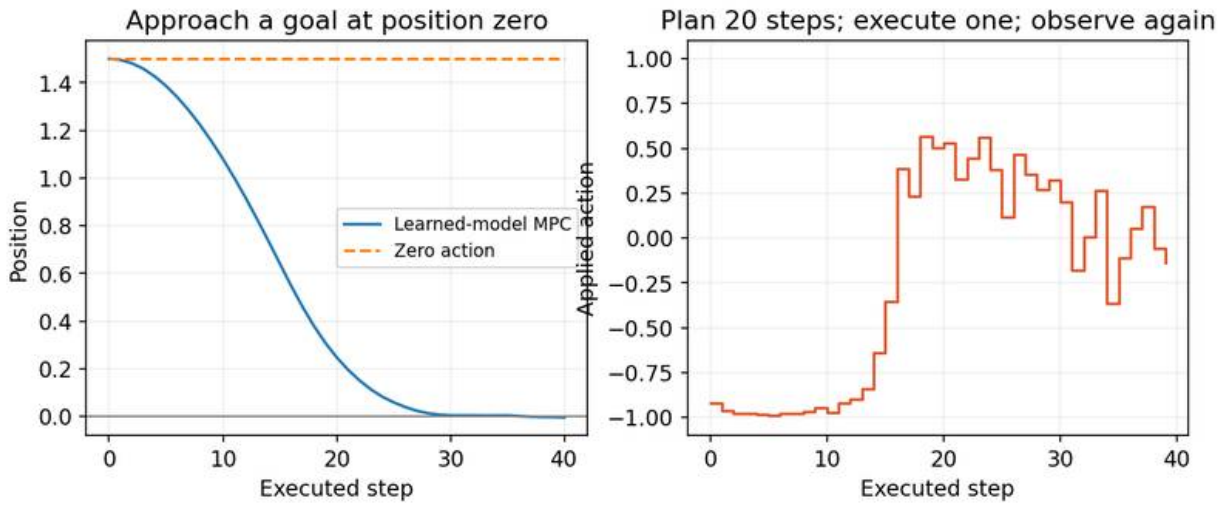


圖 5 — *CEM* 使用學到的轉移模型規劃。在合成環境中只執行第一個動作，再從新的觀測重新規劃。

把 x_t 換成影像嵌入，並不能重現 Dreamer 或 V-JEPA 2-AC：那會多出編碼器學習問題、潛在不確定性，以及需要一個有效的目標或獎勵。

規劃、不確定性與模型被利用

白話版。規劃器會去找模型最喜歡的動作。如果模型在某處是錯的，規劃器會很樂意往那裡跑。知道模型**在哪裡**不確定，才能讓它保持誠實。

有限視野的規劃目標可寫成

$$a^{*}_{t:t+H-1} = \arg\min_{a_{t:t+H-1}} \mathbb{E}_{\theta}\Big[\sum_{k=1}^{H} c(z_{t+k}, a_{t+k-1}) + \lambda_{\mathrm{T}}\, c_{\mathrm{T}}(z_{t+H})\Big],$$

其中 c 是每步成本，c_{T} 是以 λ_{T} 加權的終端成本，期望值針對模型預測的潛在狀態 z。**模型預測控制**執行第一個動作、重新觀測、再重新規劃。回饋可以修正漂移，但無法挽救一個其直接後果就被嚴重誤判的動作。

兩種不確定性

偶然（aleatoric）不確定性是本質上的隨機性（擲硬幣、可能往左也可能往右的球）；資料再多也消除不了。**認知**（epistemic）不確定性來自資料不足（模型從沒見過這種情況）；更多資料可以消除它。對生成模型取樣可以表示偶然不確定性，但樣本多樣不代表已校準。認知不確定性常以**集成**（ensemble）近似：在 bootstrap 重抽樣上或以不同初始化訓練的多個模型，在資料密集處意見一致，在資料稀少處則傾向分歧。Chua 等人的 PETS 把機率式網路（處理偶然雜訊）組成集成（處理認知不確定性），搭配以取樣為基礎的不確定性傳遞與 CEM 規劃，在數個基準上以少得多的樣本追平強大無模型方法的最終表現 [29]。

程式碼：規劃器利用學到的模型

代理人只有動作落在 $[-1, 1]$ 的記錄資料，但規劃器可以選擇 $[-3, 3]$ 之間的任何動作。

```
import numpy as np
from sklearn.neural_network import MLPRegressor

rng = np.random.default_rng(0)
true_reward = lambda a: np.sin(3 * a) * np.exp(-0.3 * a ** 2)   # unknown to the agent

# Logged interactions only cover actions in [-1, 1].
a_data = rng.uniform(-1, 1, 200)
r_data = true_reward(a_data) + 0.05 * rng.normal(size=200)

ensemble = []
for k in range(5):                          # bootstrap + different initialization
    idx = rng.integers(0, 200, 200)
    m = MLPRegressor(hidden_layer_sizes=(64, 64), max_iter=3000, random_state=k)
    ensemble.append(m.fit(a_data[idx, None], r_data[idx]))

grid = np.linspace(-3, 3, 601)
preds = np.stack([m.predict(grid[:, None]) for m in ensemble])
mean, std = preds.mean(0), preds.std(0)

for name, score in [("single model", preds[0]),
                    ("ensemble mean", mean),
                    ("pessimistic: mean - 2 std", mean - 2 * std)]:
    a_star = grid[np.argmax(score)]       # the "planner": pick the best-looking action
    print(f"{name:26s} picks a = {a_star:+.2f}, predicted {score.max():.2f}, "
          f"true {true_reward(a_star):.2f}")
print(f"mean std inside data {std[np.abs(grid) <= 1].mean():.3f}, "
      f"outside {std[np.abs(grid) > 1.5].mean():.3f}")
```

輸出：

```
single model               picks a = -3.00, predicted 1.79, true -0.03
ensemble mean              picks a = -3.00, predicted 1.58, true -0.03
pessimistic: mean - 2 std  picks a = +0.43, predicted 0.87, true 0.91
mean std inside data 0.025, outside 0.321
```

單一模型與集成平均都在資料範圍外外插出一條上升直線，規劃器於是衝到動作空間的邊緣，而那裡真實獎勵約為零。集成在資料外的分歧是資料內的十三倍；**悲觀**分數（平均減兩倍標準差）讓規劃器留在資料範圍內，找到真實獎勵為 0.91 的動作。限制動作、縮短視野、懲罰分歧以及用真實回饋測試，是實務上的對策。不過，集成成員意見一致並不保證正確：五個模型可能共享同樣的偏差。

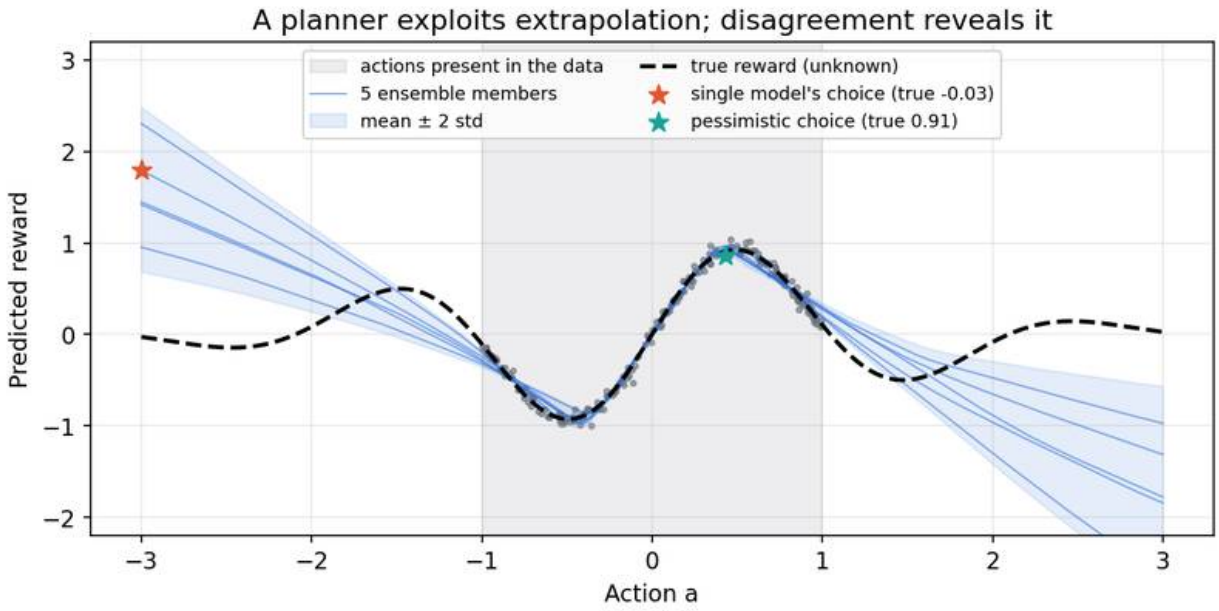


圖 *6* — 上方程式碼的輸出。對學到的模型做最佳化，會把規劃器推向模型最不可靠的地方；集成的分散程度能揭露這些區域。

如何評估世界模型

問題	量測	重要的控制條件
能預測已知動態嗎？	單步與依視野索引的狀態或特徵誤差	獨立回合；固定動作序列
外觀能保持一致嗎？	重建與影片品質指標、身分一致性	比較相機運動與遮擋
動作的作用正確嗎？	相同初始上下文、指定不同動作	真實的以動作為條件的參考軌跡
遵守物理限制嗎？	碰撞、物體恆存、守恆；Physics-IQ 等基準 [22]	任務專屬容許誤差；分布外案例 [23]
對決策有幫助嗎？	成功率、回報、人工介入次數、控制延遲	在真實或可信的環境中測試

像素保真度與決策用處可能不一致，模糊的例子已經說明了這點。平均出來的預測可能平方誤差很低，卻呈現一個不可能的場景；準確的潛在預測器可能省略視覺上無關的紋理。每個指標都要一起報告預測目標、視野、條件與輸出解析度。

比較時，請納入持續性基準（預測「什麼都不變」）、無動作預測器，以及一個適合任務的簡單動態模型。分別評估保留的物體、視角、動作範圍與環境性質。除了想像中的結果，也要報告閉迴路控制結果，並計入編碼器與取樣器的延遲：每個動作要花十秒規劃的規劃器，對以 10 Hz 運作的機器人可能毫無用處。

現代觀點

綜述。 Moerland 等人對模型式 RL 提供了最系統化的整理：動態模型學習（隨機性、不確定性、部分可觀測性、時間抽象）、規劃與學習整合方式的分類，以及隱式（價值等價、MuZero 式）方法 [5]。Ding 等人（ACM Computing Surveys）依功能整理更廣泛的「世界模型」文獻，也就是理解世界與預測未來兩類，並回顧在生成式遊戲、自動駕駛、機器人與社會模擬上的應用 [1]。Zhu 等人從影片生成的角度綜述「通用世界模型」，涵蓋影片生成器、自動駕駛世界模型以及自主代理人中的世界模型，並討論其限制 [31]。Guan 等人則提供了專門針對自動駕駛世界模型的初步綜述 [32]。

2023–2026 年的現況。

- **模型式 RL 趨於成熟**。 DreamerV3 展示了單一設定在超過 150 個任務上都能運作，並於 2025 年發表在 Nature [10]；TD-MPC2 展示了無解碼器的潛在規劃能擴展到 317M 參數的多任務代理人 [11]；IRIS 與 DIAMOND 顯示 Transformer 與擴散世界模型在 Atari 100k 基準上具競爭力 [12], [13]。

- **影片生成器被重新定位為模擬器**。 GAIA-1、UniSim、Genie、Genie 3 與 Cosmos 在帶有動作、文字或推斷出之潛在動作的影片上訓練大型模型 [17], [18], [19], [20], [21]。它們能幫助代理人的最強證據，是 UniSim 把在模擬器中訓練的策略零樣本轉移到真實世界 [18]；最強的警告則是：真實感不代表懂物理 [22], [23]。
- **預測式表示走向控制**。 JEPA 系列從影像（I-JEPA）走到影片（V-JEPA），再到真實機器人上以動作為條件的規劃（V-JEPA 2-AC）與更密集的特徵（V-JEPA 2.1）[24], [25], [26], [27]；DINO-WM 顯示凍結的自監督特徵已能支援零樣本規劃 [28]。

開放問題。

- **長視野一致性**。誤差會累積；生成的世界會漂移；物體身分與畫面外的狀態很難維持數分鐘，更別說數小時 [20]。
- **因果與物理正確性**。在被動影片上訓練的模型學到的是相關性；分布外的物理會失敗 [22], [23]。介入式（動作）資料稀少又昂貴。
- **動作介面**。從影片推斷的潛在動作、機器人關節指令與文字指令是不同的東西；可靠地在它們之間轉換仍未解決。
- **被利用與不確定性**。在想像中訓練的規劃器與策略會利用模型錯誤；大型影片模型大多缺乏校準過的認知不確定性。
- **評估**。目前沒有公認的基準能跨領域把預測品質與後續決策品質連結起來；多數論文只報告影片指標或任務回報其中之一，很少兩者都有。
- **成本**。對大型影片模型取樣可能遠慢於即時，這與閉迴路控制相衝突。

重點整理

- 世界模型預測環境如何變化，通常是在代理人的動作之下；它的預測目標與動作介面決定了它能支援什麼。
- 觀測不等於狀態。部分可觀測性需要記憶或信念估計。

- 模型可以用來線上規劃（MPC）、在想像中訓練策略（World Models、Dreamer），或用價值等價的潛在狀態做搜尋（MuZero）。
- RSSM 結合確定性記憶與隨機潛在變數；KL 項訓練先驗去想像後驗將會看到的東西。
- 平方誤差的影片預測會把可能的未來平均成模糊；隨機與生成式模型以較高的 MSE 換取逼真的樣本，但真實感不等於理解物理。
- JEPA 式模型預測特徵而不是像素；它們需要防止崩塌的設計，以及在特徵空間中有意義的目標。
- 單步誤差會在長視野中累積，最佳化器會利用模型錯誤；集成、悲觀估計、短推演與重新規劃是標準的防禦手段。
- 最有力的證據，是把受控的動作介入與保留資料上的預測，連結到在真實環境中的成功行為。

練習

1. **動作消融**。在 CPU 實驗中拿掉動作欄位。比較單步誤差、視野 40 的誤差與控制行為。為什麼一個不看動作的短期預測器仍可能看起來還不錯？

   提示

   想像的動作不再影響無動作預測器，所以規劃器完全無法比較它們的後果：每個候選都得到相同的預測成本。在一步之內，有阻尼的動態只讓狀態改變一點點，因此類似「保持不變」的預測仍能得到中等分數，這也是為什麼只看單步誤差是很弱的測試。

2. **部分觀測**。隱藏速度，擬合「位置到位置」的動態。然後改用最後兩個位置與前一個動作作為特徵。這段歷史何時有幫助？觀測雜訊在哪裡造成傷害？

   提示

   兩個位置的差可以估計速度（有限差分），所以雙位置模型能恢復大部分狀態。每個位置上的雜訊在除以時間步長時會被放大，因此觀測雜訊越大，好處越小。

3. **規劃視野**。比較視野 5、20 與 40，並把初始平均調整成對應長度。在多個獨立種子上量測最終距離、動作力度與執行時間。

   提示

   保持成本定義不變，並計算所有模型呼叫次數。太短的視野「看不到」提早煞車可以避免衝過頭；太長的視野成本更高，也累積更多模型誤差，不過在這個線性系統中誤差很小。

4. **領域偏移**。以速度阻尼 0.94 收集訓練回合，然後在阻尼為 0.75 的環境中執行控制。比較開迴路執行一個計畫與每一步重新規劃。找出一個訓練資料涵蓋不足的動作範圍。

   提示

   把學到的動態（以 0.94 擬合）與真實執行的動態（0.75）分開；不要在評估轉移上重新擬合。重新規劃使用新的觀測，所以能部分修正有偏差的模型；開迴路執行則做不到。

5. **平均與樣本**。把模糊的例子延伸成三個同樣可能的未來，位置在 10、32 與 54。計算平均預測與單一清晰樣本的期望 MSE。再提出一個會偏好樣本的指標。

   提示

   平均現在有三個三分之一高度的峰；期望 MSE 約為 0.037（平均）與 0.074（樣本），又是兩倍。這個倍數是普遍的：若 X 與 Y 是同一分布的獨立抽樣，則 $\mathbb{E}\|X - Y\|^2 = 2\,\mathbb{E}\|X - \mathbb{E}X\|^2$，所以完美的取樣器的 MSE 永遠是平均的兩倍。分布層級的指標（例如比較生成樣本集合與真實未來集合，就像 FID 類指標對影像所做的）或多個樣本中取最佳的 best-of-N 誤差，會獎勵清晰且合理的預測。

6. **悲觀的取捨**。在集成程式碼中，把「mean − 2 std」的係數 2 換成 0、0.5、1 與 4。畫出每種情況下選到的動作與其真實獎勵。係數太大時會出什麼問題？

   提示

   係數為 0 時規劃器會利用外插；係數增大時它會留在資料範圍內。過度悲觀會讓規劃器避開任何有些微分歧的動作，可能錯過資料範圍內的最佳動作，而在 RL 情境中還會停止探索。

參考文獻

1. J. Ding et al., "Understanding world or predicting future? A comprehensive survey of world models," *ACM Computing Surveys*, vol. 58, no. 3, 2025. doi arXiv
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, ch. 13 "Sequential Data". book
3. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022. book
4. R. S. Sutton, "Dyna, an integrated architecture for learning, planning, and reacting," *ACM SIGART Bulletin*, vol. 2, no. 4, 1991. doi
5. T. M. Moerland, J. Broekens, A. Plaat and C. M. Jonker, "Model-based reinforcement learning: a survey," *Foundations and Trends in Machine Learning*, vol. 16, no. 1, 2023. doi arXiv
6. D. Ha and J. Schmidhuber, "Recurrent world models facilitate policy evolution," NeurIPS, 2018; interactive version "World Models," arXiv:1803.10122. paper interactive
7. D. Hafner et al., "Learning latent dynamics for planning from pixels," ICML, 2019. paper
8. D. Hafner, T. Lillicrap, J. Ba and M. Norouzi, "Dream to control: learning behaviors by latent imagination," ICLR, 2020. arXiv
9. D. Hafner, T. Lillicrap, M. Norouzi and J. Ba, "Mastering Atari with discrete world models," ICLR, 2021. arXiv
10. D. Hafner, J. Pasukonis, J. Ba and T. Lillicrap, "Mastering diverse control tasks through world models," *Nature*, vol. 640, 2025. doi arXiv
11. N. Hansen, H. Su and X. Wang, "TD-MPC2: scalable, robust world models for continuous control," ICLR, 2024. arXiv
12. V. Micheli, E. Alonso and F. Fleuret, "Transformers are sample-efficient world models," ICLR, 2023. arXiv
13. E. Alonso et al., "Diffusion for world modeling: visual details matter in Atari," NeurIPS, 2024. arXiv

14. J. Schrittwieser et al., “Mastering Atari, Go, chess and shogi by planning with a learned model,” *Nature*, vol. 588, 2020. doi arXiv
15. C. Finn, I. Goodfellow and S. Levine, “Unsupervised learning for physical interaction through video prediction,” NIPS, 2016. arXiv
16. M. Babaeizadeh, C. Finn, D. Erhan, R. H. Campbell and S. Levine, “Stochastic variational video prediction,” arXiv:1710.11252, 2017. arXiv
17. A. Hu et al., “GAIA-1: a generative world model for autonomous driving,” technical report, arXiv:2309.17080, 2023. arXiv
18. S. Yang et al., “Learning interactive real-world simulators,” arXiv:2310.06114, 2023. arXiv
19. J. Bruce et al., “Genie: generative interactive environments,” arXiv:2402.15391, 2024. arXiv
20. Google DeepMind, “Genie 3: a new frontier for world models,” blog post, August 5, 2025. blog
21. NVIDIA (N. Agarwal et al.), “Cosmos world foundation model platform for physical AI,” arXiv:2501.03575, 2025. arXiv
22. S. Motamed, L. Culp, K. Swersky, P. Jaini and R. Geirhos, “Do generative video models understand physical principles?,” arXiv:2501.09038, 2025. arXiv
23. B. Kang et al., “How far is video generation from world model: a physical law perspective,” ICML, 2025. arXiv
24. M. Assran et al., “Self-supervised learning from images with a joint-embedding predictive architecture,” CVPR, 2023. doi arXiv
25. A. Bardes et al., “Revisiting feature prediction for learning visual representations from video,” arXiv:2404.08471, 2024. arXiv
26. M. Assran et al., “V-JEPA 2: self-supervised video models enable understanding, prediction and planning,” arXiv:2506.09985, 2025. arXiv
27. L. Mur-Labadia et al., “V-JEPA 2.1: unlocking dense features in video self-supervised learning,” arXiv:2603.14482, 2026. arXiv
28. G. Zhou, H. Pan, Y. LeCun and L. Pinto, “DINO-WM: world models on pre-trained visual features enable zero-shot planning,” arXiv:2411.04983, 2024. arXiv
29. K. Chua, R. Calandra, R. McAllister and S. Levine, “Deep reinforcement learning in a handful of trials using probabilistic dynamics models,” NeurIPS, 2018. arXiv
30. M. Janner, J. Fu, M. Zhang and S. Levine, “When to trust your model: model-based policy optimization,” NeurIPS, 2019. arXiv
31. Z. Zhu et al., “Is Sora a world simulator? A comprehensive survey on general world models and beyond,” arXiv:2405.03520, 2024. arXiv
32. Y. Guan et al., “World models for autonomous driving: an initial survey,” arXiv:2403.02622, 2024. arXiv